408 考纲知识点手册

计算机学科专业基础综合

按 2027 年全国硕士研究生招生考试大纲编排

数据结构 · 计算机组成原理 · 操作系统 · 计算机网络

考查科目数　4

知识点条目　覆盖大纲全部条目

主要素材　语雀「计算机考研 408 笔记」知识库

王道 2027 系列教材与课件

工作区历年真题与强化笔记

本手册为依据 2027 年考试大纲整理的知识点汇编，供复习检索使用。

使用说明

本手册按《2027 年全国硕士研究生招生考试计算机学科专业基础考试大纲》的条目顺序编排，覆盖大纲「考查内容」中数据结构、计算机组成原理、操作系统、计算机网络四科的全部条目。

一、编排方式

全书分四部分，共 24 章。四部分与大纲的四科一一对应，章与该科下的一级条目一一对应，章内小节按大纲的二级、三级条目顺序展开。每一章的开头用引用块照抄本章对应的大纲原文条目，便于对照检查覆盖情况；每一章末尾固定给出「本章小结表」与「易错清单」两张速查内容。

二、阅读约定

本手册用三种提示框把不同性质的内容区分开：

- 核心结论——必须记住的定义、公式、判定条件，考前反复看的就是这些。
- 易错点——统考中反复出现的陷阱，以及容易与相邻概念混淆的地方。
- 真题考法——该知识点在统考中以什么形式出现；能确认年份与题号的，如实标注。无法确认的不标，不编造。

正文中的公式一律用数学排版；算法与代码用等宽框给出；对比性内容尽量做成表格。凡涉及计算的知识点，都给出公式、符号含义、适用范围，并配一组可以自己动手复核的数字算例。

三、素材来源

本手册的素材来自三处：

- 语雀知识库「计算机考研 408 笔记」（月暗）导出的 182 篇笔记 Markdown 全文；
- 工作区中的王道 2027 系列教材、课件、强化笔记与历年真题汇编；
- 工作区 408 计算机考研目录下的各类复习笔记与答题本。

素材在整理过程中做了三件事：一是按大纲重新组织顺序；二是把原笔记中的口语化叙述、错别字与图片占位改写成规范的知识点条目，公式按上下文重新推导书写；三是逐条核对，凡素材之间口径冲突的，以大纲用词与统考真题为准，并在正文中说明差异。

四、使用建议

第一轮复习时，按章通读，重点看「核心结论」与例题，把「易错点」留到做题后再回看；第二轮复习时只看每章的「本章小结表」，遇到想不起来的再翻回正文；考前冲刺时只看「核心结论」与「易错清单」。

说明：本手册为复习资料汇编，用于知识点检索与自查，不替代教材与真题训练。

目录

使用说明 **i**

第一部分　数据结构 **1**

第一章　基本概念 **2**

本章考纲要求 . . . 2

一、 数据结构的基本概念 . . . 2

（一） 数据、数据元素、数据项、数据对象、数据结构 . . . 2

（二） 逻辑结构与存储结构 . . . 3

（三） 抽象数据类型 ADT . . . 5

二、 算法的基本概念 . . . 5

（一） 算法的定义 . . . 5

（二） 算法的五个特性 . . . 6

（三） 好算法的目标 . . . 6

（四） 时间复杂度 . . . 7

本章小结表 . . . 20

易错清单 . . . 21

第二章　线性表 **23**

本章考纲要求 . . . 23

一、 线性表的基本概念 . . . 23

（一） 线性表的定义与逻辑特征 . . . 23

（二） 线性表的基本操作 . . . 24

二、 线性表的实现 . . . 25

（一） 顺序存储 . . . 25

（二） 链式存储 . . . 29

（三） 顺序表与链表的对比 . . . 35

三、 线性表的应用（算法设计题） . . . 36

（一） （○）算法设计题的通用解题框架 . . . 37

（二） 合并两个有序表（两种存储都考） . . . 37

（三） 就地逆置 . . . 38

（四） 快慢指针：找中间结点 . . . 41

（五） 双指针：倒数第 k 个结点（2009 年真题） . . . 41

（六） 判断链表是否有环，并求环的入口 . . . 42

（七） 判断两个链表是否相交（2012 年真题） . . . 43
（八） 删除重复元素 . . . 44
（九） 其他常考算法 . . . 45
（十） 补充：无序表先排序再处理的思路 . . . 47
本章小结表 . . . 47
易错清单 . . . 48
算法设计题复杂度速查表 . . . 49

第三章 栈、队列和数组 **51**
本章考纲要求 . . . 51
一、 栈和队列的基本概念 . . . 51
（一） 栈的定义与特点 . . . 51
（二） 队列的定义与特点 . . . 51
（三） 栈与队列的对比 . . . 51
（四） 上溢与下溢 . . . 52
（五） 共享栈 . . . 52
二、 栈和队列的顺序存储结构 . . . 52
（一） 顺序栈与 top 指针的两种约定 . . . 52
（二） 顺序队列的 front / rear 约定 . . . 54
（三） 循环队列：三种判满方案 . . . 54
（四） 双端队列（受限双端队列） . . . 57
三、 栈和队列的链式存储结构 . . . 57
（一） 链栈 . . . 57
（二） 链队列（带头结点） . . . 58
（三） 链队列（不带头结点） . . . 59
四、 多维数组的存储 . . . 60
（一） 数组的定义与一维地址公式 . . . 60
（二） 二维数组的行优先与列优先 . . . 60
（三） 下标从 0 与从 1 开始的差异 . . . 61
五、 特殊矩阵的压缩存储 . . . 62
（一） 压缩存储的基本思路 . . . 62
（二） 对称矩阵 . . . 62
（三） 三角矩阵 . . . 64
（四） 三对角矩阵（带状矩阵） . . . 65
（五） 稀疏矩阵 . . . 66
六、 栈、队列和数组的应用 . . . 67
（一） 栈在括号匹配中的应用 . . . 67
（二） 栈在表达式求值中的应用 . . . 67

（三） 栈在递归中的应用 . . . 70
（四） 栈在进制转换中的应用 . . . 71
（五） 队列在层次遍历中的应用 . . . 71
（六） 队列在缓冲区与 CPU 调度中的应用 . . . 72
（七） 数组的典型应用：矩阵运算与顺序表 . . . 72
（八） 栈与队列的合法出栈序列 . . . 73
（九） 用栈实现队列、用队列实现栈 . . . 75
本章小结表 . . . 76
易错清单 . . . 77

第四章 树与二叉树 79
本章考纲要求 . . . 79
一、 树的基本概念 . . . 79
（一） 树的定义与术语 . . . 79
（二） 树的性质 . . . 81
二、 二叉树 . . . 83
（一） 二叉树的定义及其主要特性 . . . 83
（二） 二叉树的顺序存储结构和链式存储结构 . . . 85
（三） 二叉树的遍历 . . . 88
（四） 线索二叉树 . . . 93
三、 树、森林 . . . 98
（一） 树的存储结构 . . . 98
（二） 森林与二叉树的转换 . . . 100
（三） 树和森林的遍历 . . . 102
四、 树与二叉树的应用 . . . 103
（一） 哈夫曼树与哈夫曼编码 . . . 103
（二） 并查集 . . . 108
（三） 堆 . . . 112
本章小结表 . . . 115
（四） 表 1 二叉树五个性质一览 . . . 115
（五） 表 2 一般树 / 森林的性质 . . . 115
（六） 表 3 遍历序列的对应与唯一性 . . . 116
（七） 表 4 三种线索二叉树的能力 . . . 116
（八） 表 5 并查集复杂度 . . . 116
（九） 表 6 哈夫曼树 / 堆 / 线索二叉树的关键数量 . . . 117
易错清单 . . . 117

第五章 图 119

本章考纲要求 . . . 119
一、 图的基本概念 . . . 119
（一） 图、有向图与无向图 . . . 119
（二） 完全图、稠密图与稀疏图 . . . 120
（三） 度、入度、出度与握手定理 . . . 120
（四） 路径、回路、简单路径与距离 . . . 121
（五） 连通性：连通、强连通、连通分量 . . . 121
（六） 子图、生成子图与生成树 . . . 122
（七） 图的基本概念小结 . . . 122
二、 图的存储及基本操作 . . . 123
（一） 邻接矩阵 . . . 123
（二） 邻接表 . . . 124
（三） 十字链表（有向图） . . . 125
（四） 邻接多重表（无向图） . . . 126
（五） 四种存储结构的对比 . . . 127
（六） 图的基本操作 . . . 127
三、 图的遍历 . . . 129
（一） 深度优先搜索（DFS） . . . 129
（二） 广度优先搜索（BFS） . . . 130
（三） BFS 的「层次」与 Dijkstra 的异同 . . . 131
（四） 遍历手工模拟 . . . 132
（五） 遍历与图的连通性 . . . 132
（六） 遍历序列的唯一性 . . . 133
（七） DFS 与 BFS 的对比 . . . 133
四、 图的基本应用 . . . 134
（一） 最小（代价）生成树 . . . 134
（二） 最短路径 . . . 138
（三） 拓扑排序 . . . 142
（四） 关键路径 . . . 145
本章小结表 . . . 149
易错清单 . . . 150

第六章 查找 152
一、 查找的基本概念 . . . 152
（一） 查找表、关键字、查找长度 . . . 152
（二） 用判定树统一理解 ASL . . . 153
（三） 一个最小的算例 . . . 153
二、 顺序查找法 . . . 154

（一） 一般线性表的顺序查找 . . . 154
（二） 哨兵（监视哨）的作用 . . . 154
（三） 无序表的 ASL 推导 . . . 154
（四） 有序表顺序查找的 ASL 推导 . . . 155
（五） 概率不等时：把常查的元素放前面 . . . 156
三、 分块查找法 . . . 156
1. 索引顺序表的结构 . . . 156
（一） 查找过程：两步 . . . 156
3. 用折半查找查索引表时的落点 . . . 157
（二） ASL 推导与最优块长 . . . 157
（三） 优缺点与动态版本 . . . 158
四、 折半查找法 . . . 158
（一） 适用条件 . . . 158
（二） 基本实现（左闭右闭） . . . 158
（三） 判定树的构造 . . . 159
（四） 判定树是平衡二叉树 . . . 159
（五） 树高、查找成功与失败的比较次数上限 . . . 159
（六） $ASL_{成功}$ 的推导 . . . 160
（七） $ASL_{失败}$ 的推导 . . . 161
（八） 时间复杂度与优缺点 . . . 162
五、 树形查找 . . . 162
（一） 二叉搜索树（BST） . . . 162
（二） 平衡二叉树（AVL） . . . 166
（三） 红黑树（RBT） . . . 171
六、 B 树及其基本操作、B+ 树的基本概念 . . . 174
（一） m 阶 B 树的定义 . . . 174
（二） B 树的查找 . . . 177
（三） B 树的插入 . . . 178
（四） B 树的删除 . . . 179
（五） B 树的判定 . . . 179
（六） B+ 树的基本概念 . . . 180
（七） B 树与 B+ 树的对比 . . . 180
七、 散列（hash）表 . . . 181
（一） 基本概念 . . . 181
（二） 散列函数的构造方法 . . . 181
（三） 处理冲突的方法一：拉链法 . . . 182
（四） 处理冲突的方法二：开放定址法 . . . 183

（五） 线性探测的堆积（聚集）现象 . . . 184
（六） 平方探测法与表长 $4k+3$ . . . 184
（七） 散列表 ASL 的计算（三个完整算例） . . . 185
（八） 一个经典小题 . . . 188
八、 字符串模式匹配 . . . 188
（一） 朴素算法（BF 算法） . . . 188
（二） KMP 算法的思想 . . . 189
（三） 部分匹配表 PMT 与 next 数组 . . . 189
（四） 手算 next 数组的流程 . . . 190
（五） nextval 数组（优化） . . . 191
（六） 匹配过程与复杂度 . . . 192
九、 查找算法的分析及应用 . . . 193
（一） 各查找方法的横向对比 . . . 193
（二） 选用原则 . . . 193
（三） 三个必须能自己推的结论 . . . 194
本章小结表 . . . 194
易错清单 . . . 195

第七章 排序 **197**
本章考纲要求 . . . 197
一、 排序的基本概念 . . . 197
（一） 什么是排序 . . . 197
（二） 稳定性（stability） . . . 197
（三） 内部排序与外部排序 . . . 198
（四） 比较次数与移动次数 . . . 198
（五） 基于比较的排序的下界 . . . 198
二、 直接插入排序 . . . 199
（一） 算法思想 . . . 199
（二） 手工过程（逐步模拟） . . . 199
（三） 参考代码 . . . 199
（四） 复杂度与特性 . . . 200
三、 折半插入排序 . . . 200
（一） 与直接插入排序的唯一区别 . . . 200
（二） 比较次数与移动次数的分别分析 . . . 201
（三） 折半查找的边界（决定稳定性） . . . 201
四、 起泡排序（bubble sort） . . . 202
（一） 算法思想 . . . 202
（二） 手工过程（逐步模拟） . . . 202

（三） 参考代码（带提前结束优化） . . . 203
（四） 复杂度与特性 . . . 203
五、 简单选择排序 . . . 204
（一） 算法思想 . . . 204
（二） 手工过程（逐步模拟） . . . 204
（三） 参考代码 . . . 204
（四） 比较次数与移动次数 . . . 205
（五） 不稳定的原因与反例 . . . 205
六、 希尔排序（shell sort） . . . 206
（一） 算法思想 . . . 206
（二） 为什么能改进——两条理由 . . . 206
（三） 手工过程（逐步模拟） . . . 206
（四） 参考代码 . . . 207
（五） 复杂度分析 . . . 207
（六） 不稳定的原因与反例 . . . 208
七、 快速排序 . . . 209
（一） 算法思想与分治结构 . . . 209
（二） 一趟划分（Partition）的完整手工过程 . . . 209
（三） 参考代码 . . . 210
（四） 时间复杂度的三种情况 . . . 211
（五） 递归深度与栈空间 . . . 211
（六） 稳定性 . . . 212
（七） 快速排序为什么”快” . . . 212
（八） 不适合的存储结构 . . . 212
八、 堆排序 . . . 213
（一） 堆的定义与存储 . . . 213
（二） 筛选（下滤）操作 . . . 213
（三） 建堆过程（自下而上筛选，$O(n)$） . . . 214
（四） 堆排序的每一趟 . . . 215
（五） 复杂度与特性 . . . 215
（六） 不适用的存储结构与 Top-K 应用 . . . 216
九、 二路归并排序（merge sort） . . . 216
（一） 算法思想 . . . 216
（二） 手工过程（逐步模拟） . . . 217
（三） 参考代码 . . . 217
（四） 归并趟数与复杂度 . . . 218
（五） 链表上的归并排序 . . . 218

十、 基数排序 . . . 219
（一） 算法思想（非比较排序） . . . 219
（二） MSD 与 LSD . . . 219
（三） 手工过程（逐步模拟） . . . 219
（四） 复杂度与稳定性 . . . 220
十一、 外部排序 . . . 221
（一） 为什么需要外部排序 . . . 221
（二） 外部排序的两个阶段 . . . 221
（三） 外部排序的时间开销与优化方向 . . . 222
（四） 败者树（loser tree） . . . 223
（五） 置换-选择排序（生成初始归并段） . . . 223
（六） 最佳归并树与哈夫曼树 . . . 225
（七） 虚段（补空归并段）的数目计算 . . . 226
十二、 排序算法的分析与应用 . . . 227
（一） 完整对比表 . . . 227
（二） 什么情况下选哪个算法 . . . 228
（三） 算法性质反推（考试最常见的形式） . . . 228
本章小结表 . . . 229
易错清单 . . . 230

第二部分 计算机组成原理 **232**

第八章 计算机系统概述 **233**
本章考纲要求 . . . 233
一、 计算机系统的基本组成 . . . 234
（一） 计算机系统的构成 . . . 234
（二） 计算机硬件系统的五大部件 . . . 234
（三） 各部件的工作关系（数据通路初貌） . . . 235
二、 计算机硬件的基本结构 . . . 235
（一） 冯 · 诺依曼结构 . . . 235
（二） 现代计算机以存储器为中心 . . . 236
（三） 计算机的层次结构 . . . 236
（四） 指令集体系结构（ISA）与微架构的界限 . . . 237
三、 计算机软件和硬件的关系 . . . 237
（一） 软件与硬件的逻辑等价性 . . . 237
（二） 固化的概念 . . . 238
四、 计算机系统的工作原理 . . . 238

（一） 「存储程序」工作方式 . . . 238
（二） 高级语言程序与机器语言程序之间的转换 . . . 238
（三） 程序和指令的执行过程 . . . 240
五、 性能评价的两个基本维度 . . . 241
（一） 响应时间 . . . 241
（二） 吞吐量 . . . 242
六、 CPU 时钟周期、主频、CPI 与 CPU 执行时间 . . . 242
（一） 时钟周期与主频 . . . 242
（二） CPI . . . 242
（三） CPU 执行时间的三个等价公式 . . . 243
（四） 用比例法快速解题 . . . 244
（五） 两台机器执行时间之比 . . . 245
七、 MIPS 与 FLOPS 类指标 . . . 245
（一） MIPS . . . 245
（二） GIPS 与指令执行速度 . . . 246
（三） MIPS 的三条重要局限 . . . 247
（四） FLOPS 系列指标 . . . 247
（五） 速率进制 vs 容量进制（必考辨析） . . . 248
八、 各种字长总结 . . . 249
（一） 机器字长 . . . 249
（二） 指令字长 . . . 249
（三） 存储字长与存储容量 . . . 250
（四） 编址单位与地址字长 . . . 250
（五） MAR、MDR 与地址线、数据线 . . . 251
（六） 数据字长 . . . 252
（七） 地址字长 . . . 252
（八） 字长之间的关系梳理 . . . 252
本章小结表 . . . 253
易错清单 . . . 255

第九章 数据的表示和运算 **257**
本章考纲要求 . . . 257
一、 数制与编码 . . . 257
（一） 进位计数制 . . . 257
（二） 进位计数制之间的相互转换 . . . 258
（三） 真值与机器数 . . . 259
（四） 定点数的编码表示 . . . 259
二、 运算方法和运算电路 . . . 263

（一） 基本运算部件 . . . 263
（二） 补码加/减运算器与标志位的生成 . . . 264
（三） 乘/除法运算的基本原理 . . . 267
（四） 乘法电路和除法电路的基本结构 . . . 271
三、 整数的表示和运算 . . . 273
（一） 无符号整数 . . . 273
（二） 带符号整数 . . . 273
（三） 零扩展与符号扩展 . . . 274
（四） 无符号与有符号混合运算的陷阱 . . . 274
（五） 有符号整数加减运算的溢出判断 . . . 275
四、 浮点数的表示和运算 . . . 276
（一） 浮点数的一般格式 . . . 276
（二） 浮点数的规格化 . . . 276
（三） IEEE 754 标准 . . . 276
（四） 浮点数的加减运算 . . . 279
（五） 浮点数加减运算的步骤汇总 . . . 283
本章小结表 . . . 283
（六） 表一：各种编码对照（$n+1$ 位编码，1 位符号位 + n 位数值位） . . . 283
（七） 表二：四个标志位 . . . 283
（八） 表三：乘法与除法运算方法 . . . 284
（九） 表四：IEEE 754 关键数值（单精度） . . . 284
（十） 表五：浮点数加减运算六步 . . . 284
易错清单 . . . 285

第十章 存储器层次结构 **287**
本章考纲要求 . . . 287
一、 存储器的分类 . . . 288
（一） 按存储介质分类 . . . 288
（二） 按存取方式分类 . . . 288
（三） 按信息可更改性与易失性分类 . . . 288
（四） 按作用与存取速度分类 . . . 289
（五） 存储器的性能指标 . . . 289
（六） 存储器的层次化结构 . . . 290
（七） 程序访问的局部性原理 . . . 291
二、 半导体随机存取存储器 . . . 292
（一） 半导体存储芯片的内部结构与译码方式 . . . 292
（二） 基本存储元件 . . . 292
（三） DRAM 的刷新 . . . 293

（四） DRAM 的地址复用 . . . 294
（五） 只读存储器 ROM 与 Flash 存储器 . . . 295
三、 主存储器 . . . 297
（一） DRAM 芯片与内存条 . . . 297
（二） 存储器容量的扩展 . . . 297
（三） 主存与 CPU 之间的连接 . . . 299
（四） 多模块存储器 . . . 301
四、 外部存储器 . . . 303
（一） 磁盘存储器 . . . 303
（二） 固态硬盘 SSD . . . 306
五、 高速缓冲存储器（Cache） . . . 307
（一） Cache 的基本原理 . . . 307
（二） Cache 与主存之间的映射方式 . . . 308
（三） Cache 中主存块的替换算法 . . . 312
（四） Cache 写策略 . . . 313
六、 虚拟存储器 . . . 315
（一） 虚拟存储器的基本概念 . . . 315
（二） 页式虚拟存储器 . . . 316
（三） 段式虚拟存储器 . . . 320
（四） 段页式虚拟存储器 . . . 320
本章小结表 . . . 321
易错清单 . . . 323
七、 本章公式速查 . . . 325

第十一章 指令系统 **327**
本章考纲要求 . . . 327
一、 指令系统的基本概念 . . . 327
（一） 指令与指令系统 . . . 327
（二） 指令集体系结构（ISA） . . . 328
（三） 指令字长与机器字长 . . . 328
（四） 定长指令字与变长指令字 . . . 329
二、 指令格式 . . . 329
（一） 操作码与地址码 . . . 329
（二） 零地址 / 一地址 / 二地址 / 三地址指令 . . . 329
（三） 定长操作码 . . . 330
（四） 扩展操作码（可变长操作码） . . . 330
三、 寻址方式 . . . 332
（一） 指令寻址与数据寻址 . . . 332

（二） 各寻址方式详解 . . . 333
（三） 寻址方式完整对比表 . . . 336
四、 数据的对齐和大/小端存放方式 . . . 337
（一） 大端方式与小端方式 . . . 337
（二） 边界对齐与不对齐存储 . . . 338
（三） 结构体大小的计算规则 . . . 338
五、 CISC 和 RISC 的基本概念 . . . 341
（一） CISC 的基本概念与特点 . . . 341
（二） RISC 的基本概念与特点 . . . 342
（三） CISC 与 RISC 的完整对比 . . . 342
（四） 走向融合 . . . 342
六、 高级语言程序与机器级代码之间的对应 . . . 343
（一） 编译器、汇编器和链接器的基本概念 . . . 343
（二） 选择结构语句的机器级表示 . . . 344
（三） 循环结构语句的机器级表示 . . . 345
（四） 过程（函数）调用对应的机器级表示 . . . 348
（五） MIPS 汇编基础 . . . 351
本章小结表 . . . 355
易错清单 . . . 356

第十二章 中央处理器 **358**
本章考纲要求 . . . 358
一、 CPU 的功能和基本结构 . . . 358
（一） CPU 的功能 . . . 358
（二） 运算器的组成与各部件的作用 . . . 359
（三） 控制器的组成与各部件的作用 . . . 360
（四） 各寄存器的位数与地址、字长的关系 . . . 360
二、 指令执行过程 . . . 361
（一） 指令周期的层级：指令周期、机器周期、时钟周期 . . . 361
（二） 取指周期的流程与数据通路 . . . 362
（三） 间址周期的流程与数据通路 . . . 362
（四） 执行周期的流程与数据通路 . . . 363
（五） 中断周期的流程与数据通路 . . . 363
（六） 各周期微操作的通用分析方法 . . . 364
三、 数据通路的功能和基本结构 . . . 364
（一） 数据通路的定义与组成元件 . . . 364
（二） 单总线结构的数据通路 . . . 365
（三） 多总线结构与专用数据通路 . . . 365

（四） 数据通路的数据传送过程与时钟周期数计算 . . . 365
四、 控制器的功能和工作原理 . . . 366
（一） 时序与控制 . . . 366
（二） 硬布线控制器 . . . 366
（三） 微程序控制器 . . . 367
（四） 微指令的格式与编码方式 . . . 368
（五） 微地址的形成方式 . . . 369
（六） 微指令条数与控制存储器容量的计算 . . . 370
五、 异常和中断机制 . . . 371
（一） 异常和中断的基本概念与区别 . . . 371
（二） 异常和中断的分类 . . . 371
（三） 异常和中断的检测与响应 . . . 372
（四） 多重中断与中断屏蔽 . . . 373
六、 指令流水线 . . . 375
（一） 指令流水线的基本概念 . . . 375
（二） 指令流水线的性能计算 . . . 376
（三） 结构冒险、数据冒险和控制冒险的处理 . . . 377
（四） 超标量和动态流水线的基本概念 . . . 379
七、 多处理器基本概念 . . . 380
（一） Flynn 分类法与向量处理器 . . . 380
（二） 硬件多线程 . . . 381
（三） 多核处理器 . . . 382
（四） 共享内存多处理器（SMP）与 UMA/NUMA、Cache 一致性问题 . . . 382
本章小结表 . . . 383
易错清单 . . . 384

第十三章 总线和输入/输出系统 386
一、 总线和输入/输出系统 . . . 386
二、 总线 . . . 386
（一） 总线的基本概念 . . . 386
（二） 总线的组成及性能指标 . . . 388
（三） 总线事务和定时 . . . 391
三、 I/O 接口（I/O 控制器） . . . 394
（一） I/O 接口的功能和基本结构 . . . 394
（二） I/O 端口及其编址 . . . 395
四、 I/O 方式 . . . 396
（一） 程序查询方式 . . . 396
（二） 程序中断方式 . . . 398

（三） DMA 方式 . . . 404
（四） 三种 I/O 方式的完整对比 . . . 408
本章小结表 . . . 409
易错清单 . . . 410

第三部分 操作系统 **412**

第十四章 操作系统基础 **413**
本章考纲要求 . . . 413
一、 操作系统的基本概念 . . . 413
（一） 操作系统的定义与双重角色 . . . 413
（二） 操作系统的四大功能 . . . 414
（三） 用户接口与程序接口 . . . 414
（四） 操作系统的四大特征 . . . 415
二、 操作系统发展历程 . . . 416
（一） 手工操作阶段 . . . 416
（二） 批处理阶段 . . . 416
（三） 分时操作系统 . . . 417
（四） 实时操作系统 . . . 417
（五） 网络操作系统、分布式系统与微机操作系统 . . . 417
（六） 各类操作系统对比 . . . 418
三、 程序运行环境 . . . 418
（一） CPU 运行模式：内核模式与用户模式 . . . 418
（二） 中断和异常的处理 . . . 420
（三） 系统调用 . . . 422
（四） 程序的链接与装入 . . . 425
（五） 程序运行时的内存映像与地址空间 . . . 427
四、 操作系统结构 . . . 428
（一） 分层结构 . . . 428
（二） 模块化结构 . . . 428
（三） 宏内核 . . . 428
（四） 微内核 . . . 429
（五） 外核 . . . 430
（六） 五种结构对比 . . . 430
五、 操作系统引导 . . . 431
（一） 引导的完整流程 . . . 431
六、 虚拟机 . . . 432

（一） 虚拟机与虚拟机管理程序 . . . 432
（二） 第一类虚拟机管理程序（裸金属架构） . . . 432
（三） 第二类虚拟机管理程序（寄居架构） . . . 433
（四） 两类虚拟机管理程序对比 . . . 433
本章小结表 . . . 433
易错清单 . . . 435

第十五章 进程管理 **437**
本章考纲要求 . . . 437
一、 进程与线程 . . . 438
（一） 进程与线程的基本概念 . . . 438
（二） 进程与线程的状态与转换 . . . 440
（三） 线程的实现 . . . 442
（四） 进程与线程的组织与控制 . . . 443
（五） 进程间通信 . . . 445
二、 CPU 调度与上下文切换 . . . 448
（一） 调度的基本概念 . . . 448
（二） 调度的目标 . . . 448
（三） 调度的实现 . . . 449
（四） CPU 调度算法 . . . 451
（五） 多处理机调度 . . . 456
（六） 上下文及其切换机制 . . . 457
三、 同步与互斥 . . . 458
（一） 同步与互斥的基本概念 . . . 458
（二） 基本的实现方法 . . . 459
（三） 锁 . . . 463
（四） 信号量 . . . 465
（五） 条件变量 . . . 467
（六） 经典同步问题 . . . 468
四、 死锁 . . . 479
（一） 死锁的基本概念 . . . 479
（二） 死锁预防 . . . 480
（三） 死锁避免 . . . 481
（四） 死锁检测和解除 . . . 488
本章小结表 . . . 489
易错清单 . . . 491

第十六章 内存管理 **494**

本章考纲要求 . . . 494
一、 内存管理基础 . . . 494
（一） 内存管理的基本概念 . . . 494
（二） 连续分配管理方式 . . . 497
（三） 页式管理 . . . 501
（四） 段式管理 . . . 505
（五） 段页式管理 . . . 508
二、 虚拟内存管理 . . . 509
（一） 虚拟内存的基本概念 . . . 509
（二） 请求页式管理 . . . 511
（三） 页框分配与回收 . . . 513
（四） 页置换算法 . . . 517
（五） 内存映射文件 . . . 523
（六） 虚拟存储器性能的影响因素及改进方法 . . . 525
本章小结表 . . . 528
（七） 三种非连续分配方式总对比 . . . 528
（八） 页面置换算法总对比 . . . 529
（九） 连续分配三种方式总对比 . . . 529
（十） 有效访问时间速查 . . . 529
易错清单 . . . 530

第十七章 文件管理 **532**
本章考纲要求 . . . 532
一、 文件的基本概念 . . . 532
（一） 数据项、记录与文件 . . . 532
（二） 文件系统及其功能 . . . 533
（三） 文件的属性与分类 . . . 533
文件元数据和索引节点 . . . 533
（四） 文件控制块 FCB . . . 533
索引节点 inode 与目录项的分离 . . . 534
inode 中的多级索引与最大文件大小计算 . . . 535
（五） inode 位图与数据块位图 . . . 536
二、 文件的操作 . . . 537
（一） 建立文件 . . . 537
（二） 删除文件 . . . 537
（三） 打开文件表与文件描述符 . . . 538
（四） 关闭文件 . . . 539
（五） 读文件与写文件 . . . 539

三、 文件的保护 . . . 540
（一） 可控制的访问类型 . . . 540
（二） 访问控制矩阵与访问控制列表 . . . 540
（三） 传统的权限位 rwx . . . 541
（四） 口令保护与加密保护 . . . 541
四、 文件的逻辑结构 . . . 541
（一） 无结构文件与有结构文件 . . . 542
（二） 顺序文件 . . . 542
索引文件 . . . 542
索引顺序文件 . . . 542
（三） 直接文件或散列文件 . . . 543
（四） 各逻辑结构的查找效率对比 . . . 543
五、 文件的物理结构 . . . 544
（一） 连续分配 . . . 544
（二） 链接分配 . . . 545
索引分配 . . . 546
（三） 三种物理结构的对比 . . . 547
六、 目录的基本概念 . . . 548
七、 树形目录 . . . 548
（一） 目录结构演进 . . . 548
（二） 绝对路径、相对路径与当前工作目录 . . . 549
（三） 路径名解析过程 . . . 549
八、 目录的操作 . . . 550
九、 硬链接与软链接 . . . 550
（一） 硬链接（Hard Link） . . . 550
（二） 软链接（符号链接，Symbolic Link） . . . 551
十、 文件系统的全局结构 . . . 552
（一） 文件系统在外存（磁盘）中的结构 . . . 552
（二） 文件系统在内存中的结构 . . . 553
十一、 外存空闲空间管理方法 . . . 553
（一） 空闲表法 . . . 553
（二） 空闲链表法 . . . 554
（三） 位示图法 . . . 554
（四） 成组链接法 . . . 556
十二、 虚拟文件系统 . . . 557
（一） 为什么需要 VFS . . . 557
（二） VFS 的四大对象 . . . 558

（三） VFS 的工作过程（以 write 为例） . . . 558
十三、 文件系统挂载 . . . 559
（一） 挂载的过程 . . . 559
本章小结表 . . . 560
易错清单 . . . 561

第十八章 输入/输出管理 **563**
本章考纲要求 . . . 563
一、 I/O 管理基础 . . . 564
（一） 设备 . . . 564
（二） I/O 控制方式 . . . 566
（三） I/O 软件层次结构 . . . 571
（四） 输入/输出应用程序接口 . . . 573
二、 设备独立软件 . . . 575
（一） 缓冲区管理 . . . 575
（二） 设备分配与回收 . . . 579
（三） 假脱机技术（SPOOLing） . . . 581
（四） 设备驱动程序接口 . . . 583
三、 外存管理 . . . 584
（一） 磁盘 . . . 584
（二） 固态硬盘 . . . 591
本章小结表 . . . 592
易错清单 . . . 594

第四部分 计算机网络 **596**

第十九章 计算机网络概述 **597**
本章考纲要求 . . . 597
一、 计算机网络基本概念 . . . 597
（一） 定义、组成与功能 . . . 597
（二） 分类 . . . 599
（三） 主要性能指标 . . . 602
二、 计算机网络体系结构 . . . 607
（一） 分层结构 . . . 607
（二） 协议、接口、服务等概念 . . . 608
（三） ISO/OSI 参考模型与 TCP/IP 参考模型 . . . 610
本章小结表 . . . 613
易错清单 . . . 615

第二十章 物理层 **617**
本章考纲要求 . . . 617
一、 信道、信号、带宽、码元、波特、速率、信源与信宿 . . . 618
（一） 通信系统的基本模型 . . . 618
（二） 通信的三种方式 . . . 618
（三） 信号：模拟信号与数字信号 . . . 619
（四） 基带信号与带通信号，基带传输与频带传输 . . . 619
（五） 带宽的两种含义 . . . 620
（六） 码元、波特与速率 . . . 620
（七） 速率与传播速度的区别 . . . 621
二、 奈奎斯特定理与香农定理 . . . 622
（一） 为什么码元发得太快就会出错 . . . 622
（二） 奈奎斯特定理（奈氏准则） . . . 622
（三） 香农定理 . . . 623
（四） 两个定理的综合对比 . . . 625
三、 编码与调制 . . . 626
（一） 数字数据编码为数字信号 . . . 626
（二） 数字数据调制为模拟信号 . . . 629
（三） 模拟数据编码为数字信号：PCM . . . 630
（四） 模拟数据调制为模拟信号与频分复用 . . . 631
四、 电路交换、报文交换与分组交换 . . . 631
（一） 三种交换方式的工作过程 . . . 631
（二） 三种交换方式的对比 . . . 632
（三） 分组交换的时延计算 . . . 633
五、 数据报与虚电路 . . . 636
（一） 数据报方式 . . . 636
（二） 虚电路方式 . . . 636
（三） 数据报与虚电路的对比 . . . 637
六、 传输介质 . . . 637
（一） 双绞线 . . . 638
（二） 同轴电缆 . . . 638
（三） 光纤 . . . 639
（四） 无线传输介质 . . . 639
（五） 物理层接口的特性 . . . 640
七、 物理层设备 . . . 640
（一） 中继器 . . . 641
（二） 集线器 . . . 641

本章小结表 . . . 642
易错清单 . . . 644

第二十一章 数据链路层 **646**
本章考纲要求 . . . 646
一、 数据链路层的功能 . . . 647
（一） 链路、数据链路与帧 . . . 647
（二） 数据链路层的功能 . . . 647
（三） 封装成帧、透明传输与差错检测 . . . 647
二、 组帧 . . . 648
（一） 字符计数法 . . . 648
（二） 字符填充法 . . . 648
（三） 零比特填充法 . . . 648
（四） 违规编码法 . . . 649
三、 差错控制 . . . 649
（一） 差错与误码率 . . . 649
（二） 检错编码 . . . 650
（三） 纠错编码：海明码 . . . 653
四、 流量控制与可靠传输机制 . . . 656
（一） 流量控制、可靠传输与滑动窗口机制 . . . 656
（二） 停止-等待协议 . . . 657
（三） 后退 N 帧协议（GBN） . . . 659
（四） 选择重传协议（SR） . . . 660
（五） 三种协议的对比与信道利用率 . . . 661
五、 介质访问控制 . . . 663
（一） 信道划分介质访问控制 . . . 664
（二） 随机访问介质访问控制 . . . 666
（三） 轮询访问：令牌传递协议 . . . 673
六、 局域网 . . . 674
（一） 局域网的基本概念与体系结构 . . . 674
（二） 以太网与 IEEE 802.3 . . . 675
（三） IEEE 802.11 无线局域网 . . . 677
（四） VLAN 基本概念与基本原理 . . . 679
七、 广域网 . . . 681
（一） 广域网的基本概念 . . . 681
（二） PPP 协议 . . . 682
八、 数据链路层设备 . . . 685
（一） 网桥与以太网交换机 . . . 685

（二） 自学习算法与转发表的建立 . . . 685
（三） 转发/过滤/泛洪与冲突域、广播域的划分 . . . 686
（四） 直通式与存储转发式交换 . . . 687
（五） 交换机 vs 集线器 vs 路由器 . . . 688
本章小结表 . . . 688
易错清单 . . . 689

第二十二章 网络层 **692**
本章考纲要求 . . . 692
一、 网络层的功能 . . . 693
（一） 异构网络互连 . . . 693
（二） 路由与转发 . . . 694
（三） SDN 基本概念 . . . 694
（四） 拥塞控制 . . . 695
二、 路由算法 . . . 696
（一） 静态路由与动态路由 . . . 696
（二） 距离-向量路由算法 . . . 696
（三） 链路状态路由算法 . . . 697
（四） 层次路由 . . . 697
三、 IPv4 . . . 698
（一） IPv4 分组 . . . 698
（二） IPv4 地址与 NAT . . . 701
（三） 子网划分、路由聚集、子网掩码与 CIDR . . . 703
（四） ARP 协议、DHCP 协议与 ICMP 协议 . . . 708
四、 IPv6 . . . 711
（一） IPv6 的主要特点 . . . 711
（二） IPv6 地址 . . . 713
（三） IPv4 向 IPv6 的过渡 . . . 713
五、 路由协议 . . . 714
（一） 自治系统与域内、域间路由 . . . 714
（二） RIP 路由协议 . . . 714
（三） OSPF 路由协议 . . . 716
（四） BGP 路由协议 . . . 718
（五） 三种路由协议对比 . . . 720
六、 IP 组播 . . . 720
（一） 组播的概念 . . . 720
（二） IP 组播地址 . . . 721
（三） IGMP 与组播路由 . . . 721

七、 移动 IP 722
（一） 移动 IP 的概念 722
（二） 移动 IP 通信过程 722
八、 网络层设备 723
（一） 冲突域和广播域 723
（二） 路由器的组成和功能 724
（三） 路由表与分组转发 725
本章小结表 727
易错清单 729

第二十三章 传输层 **731**
一、 传输层 731
（一） 传输层提供的服务 731
（二） UDP 协议 736
（三） TCP 协议 741
本章小结表 767
易错清单 769

第二十四章 应用层 **770**
本章考纲要求 770
一、 网络应用模型 770
（一） 客户/服务器模型 770
（二） 对等模型 771
（三） P2P 的三种组织方式 772
二、 DNS 系统 772
（一） 层次域名空间 772
（二） 域名服务器 773
（三） 域名解析过程 774
三、 FTP 777
（一） FTP 协议的工作原理 777
（二） 控制连接与数据连接 779
四、 电子邮件 780
（一） 电子邮件系统的组成结构 780
（二） 电子邮件格式与 MIME 781
（三） SMTP 协议与 POP3 协议 782
五、 WWW 784
（一） WWW 的概念与组成结构 784
（二） HTTP 协议 785

本章小结表 792
易错清单 793

第一部分

数据结构

第一章基本概念

本章考纲要求

第一部分数据结构［考查目标］

1. 掌握数据结构的基本概念、基本原理和基本方法。
2. 掌握数据的逻辑结构、存储结构及基本操作的实现，能够对算法进行基本的时间复杂度与空间复杂度的分析。
3. 能够运用数据结构的基本原理和方法进行问题的分析与求解，具备采用 C 或 C++ 语言设计与实现算法的能力。

一、基本概念（一）数据结构的基本概念（二）算法的基本概念

本章是第一章「基本概念」，对应考纲的两个二级条目。数据结构部分每年第一道选择题几乎都落在「时间复杂度」，是性价比最高的送分点，也是失分集中的重灾区；基本概念则以文字辨析题的形式出现，考查定义边界的精确理解。本章把两部分合并写透：概念部分要求能准确复述定义、能在一段描述中辨认出逻辑结构、存储结构与数据运算三个层面；算法部分要求能对任意一段循环或递归代码独立推出复杂度阶，而不是靠背题。

一、数据结构的基本概念

（一）数据、数据元素、数据项、数据对象、数据结构

这一组术语是层层包含的关系，考试喜欢用「下列选项中，（）是数据的最小单位」「一个数据元素可由若干（）组成」这类题来考边界。

- 数据（**Data**）：信息的载体，是描述客观事物属性的数、字符及所有能输入到计算机中并被计算机程序识别和处理的符号的集合。数据的含义很宽：整数、实数、字符串、图像、声音都是数据。
- 数据元素（**Data Element**）：数据的基本单位，在计算机程序中通常作为一个整体进行考虑和处理。又称元素、记录（**record**）、结点（**node**）、顶点（**vertex**）——这些叫法在不同数据结构中指的是同一层概念。
- 数据项（**Data Item**）：构成数据元素的不可分割的最小单位。又称字段（**field**）、域（**domain**）。
- 数据对象（**Data Object**）：具有相同性质的数据元素的集合，是数据的一个子集。例如整数数据对象是集合 $N = \{0, \pm1, \pm2, \dots\}$。
- 数据结构（**Data Structure**）：相互之间存在一种或多种特定关系的数据元素的集合。

举一个具体例子把五层贯通。一张学生成绩表：

层次	具体内容	说明
数据	整张表格的全部符号	最宽的范围
数据对象	所有「学生成绩记录」的集合	性质相同的元素集合
数据元素	一行记录张三，90，85	作为整体处理的基本单位

层次	具体内容	说明
数据项	姓名、数学、英语	不可再分割的最小单位

数据元素与数据项的边界

「数据的最小单位」是数据项，不是数据元素。「数据的基本单位」是数据元素。这两个词一字之差，考频极高。另外注意：数据项是「不可分割的最小单位」，但数据元素在有些实现里可以只是一个数据项（例如一个整数数组，每个元素就是一个数据项，此时数据元素 = 数据项）。

数据对象与数据的关系

数据对象是数据的子集，不是数据的同义词。数据是所有能被程序处理的符号的集合；数据对象要求「性质相同」。同一个数据集合可以划分出不同的数据对象。

数据结构这一术语本身包含三个层面的内容，缺一不可：

1. 逻辑结构：数据元素之间的抽象关系，与计算机无关。
2. 存储结构（物理结构）：逻辑结构在计算机中的表示（又称映像），是逻辑结构在存储器中的实现。
3. 数据的运算：施加在数据上的操作，包括运算的定义（针对逻辑结构，指出运算的功能）和运算的实现（针对存储结构，指出运算的具体操作步骤）。

数据结构 = 逻辑结构 + 存储结构 + 数据的运算

这是本章最常被直接命中的定义。三个层面互相牵连：逻辑结构决定「能做什么运算」，存储结构决定「运算怎么做、效率如何」。所以同一个逻辑结构在同一种存储结构下实现不同，效率可能差一个数量级——这正是后面各章反复出现的主线。

（二）逻辑结构与存储结构

（1）逻辑结构：线性结构与非线性结构

按数据元素之间关系的抽象形态，分为两大类。

线性结构：数据元素之间存在一对一的关系，除第一个元素外每个元素有且仅有一个直接前驱，除最后一个元素外每个元素有且仅有一个直接后继。典型例子：线性表、栈、队列、串、一维数组。

非线性结构：数据元素之间存在一对多或多对多的关系，一个元素可能有多个直接前驱或直接后继。典型例子：树（一对多）、图（多对多）、集合（元素之间无关系，是另一种独立的逻辑结构）。

如果写得再细一点，逻辑结构可分成四类：集合结构（元素间除「同属一个集合」外无其他关系）、线性结构、树形结构、图形结构（网状结构）。后者三类加上集合结构就是常见的四分法。

逻辑结构与存储结构必须分开判断

「栈和队列的逻辑结构相同」——正确，二者都是线性结构。「线性表的链式存储是非线性结构」——错误，存储结构的选择不改变逻辑结构。逻辑结构是「抽象关系」，存储结构是「内存里的摆放方式」。看到「链式」「顺序」这样的词，它们描述的是存储结构，不能用来判断逻辑结构。

特殊的逻辑结构判定

- 空表、只有一个元素的表，也是线性结构（一对一的定义是「除首尾外每个元素有且仅有一个前驱/后继」，单元素表首尾重合，仍满足）。
- 二叉树是非线性结构，虽然每个结点最多两个后继，但「一对多」已经不属于线性。
- 栈、队列、线性表三者逻辑结构相同（都是线性结构），只是运算定义不同（限制插入删除的位置）。这是 408 常考的文字辨析点。

（2）存储结构：四种基本方式

存储结构要在存储器中表示两件事：数据元素本身的值 + 元素之间的逻辑关系。按关系的表示方式分为四种。

存储方式	关系如何表示	核心特点	典型例子
顺序存储	用物理位置上的相邻（连续单元 + 相对位置）表示逻辑相邻	逻辑相邻 ⇔ 物理相邻；支持随机存取；插入删除需移动元素	顺序表、顺序栈、一维数组实现堆
链式存储	用附加的指针表示逻辑相邻，结点本身可以不相邻	只能顺序存取；插入删除不需移动，改指针即可；有指针域额外开销	单链表、二叉链表、邻接表
索引存储	建立索引表，每项为「关键字 + 地址」	检索速度快；索引表本身占额外空间；插入删除要维护索引表	索引顺序表（分块查找）、B 树索引、数据库索引
散列存储	由关键字通过散列函数直接算出存储地址	检索、插入、删除都近似 $O(1)$；可能产生冲突，需要处理冲突	散列表（Hash Table）

理解这四种的关键在于问一句「关系是怎么被表达的」：顺序靠位置，链式靠指针，索引靠索引表，散列靠函数计算。

（3）四种存储结构对同一组运算的代价差异

同一个逻辑结构换一种存储结构，运算的实现代价会发生质变。下面把「按下标取值、插入、删除、按值查找」四类运算在四种存储结构下的代价摆在一起，可以看出为什么「选存储结构」本质上是在权衡。

存储结构	按下标取值	插入 / 删除	按值查找	额外开销
顺序存储	$O(1)$，地址可算	平均 $O(n)$，需移动元素	$O(n)$；若有序可折半到 $O(\log n)$	无指针开销，但需一整块连续空间
链式存储	$O(n)$，必须从头走	$O(1)$（已知位置）/ $O(n)$（需先定位）	$O(n)$	每个结点多一个指针域
索引存储	取决于数据区组织	$O(n)$（需维护索引表）	先查索引表再到块内查找，平均 $O(\sqrt{n})$ 或更快	索引表本身占空间
散列存储	无法按下标访问	平均 $O(1)$，需处理冲突	平均 $O(1)$，最坏退化为 $O(n)$	需要预留装填余量，冲突处理结构

表中「平均 $O(\sqrt{n})$」对应分块查找：把表分成若干块，块间用索引表（有序），块内顺序查找。设共 n 个元素、分成 b 块、每块 s 个元素（$n = bs$），则平均查找长度为「查索引表 + 块内查找」，当取 $s = \sqrt{n}$ 时整体达到最优 $O(\sqrt{n})$。这个结论在查找一章还会展开，这里只需记住「索引存储的核心是用额外空间换取查找速度」。

顺序存储 ≠ 随机存取，但它们总是同时出现

「能随机存取」的充要条件是存储地址可计算，这与顺序存储天然吻合。链式存储的地址不可计算，只能从头指针顺着走，因此是顺序存取。注意区分两个词：随机存取（random access，可直接访问任意位置，$O(1)$）与顺序存取（sequential access，必须按顺序推进，$O(k)$）。也要与随机存储区分开——随机存储指的是访问时间与存储位置无关的存储器（如 RAM），是硬件概念。

另外：索引存储不是顺序存储，索引表本身通常按顺序存储，但数据区可以离散；散列存储的地址由函数决定，与逻辑关系的顺序无关。

真题考法

基本概念部分常以「下列叙述中，正确的是（）」形式出现，选项通常是：某结构的逻辑结构是否线性、某种存储方式是否支持随机存取、数据元素与数据项谁是基本/最小单位、一个数据元素是否可由多个数据项组成。这类题不需要计算，靠定义精确度得分。

（三）抽象数据类型 ADT

数据类型（**Data Type**）：一个值的集合和定义在这个值集合上的一组操作的总称。例如 C 语言的 `int`，值是某个范围内的整数集合，操作是加减乘除和取模。数据类型分两类：

- 原子类型：值不可再分解，如 `int`、`char`、`double`。
- 结构类型：值由若干成分按某种结构组成，可以分解，如数组、结构体。

抽象数据类型（**Abstract Data Type, ADT**）：一个数学模型以及定义在该模型上的一组操作。它的核心特征是抽象——只关心「能做什么」（逻辑特性），不关心「怎么实现」（存储与实现细节）。

ADT 由三部分构成：

1. 数据对象：该类型的数据是什么样的集合。
2. 数据关系：数据对象中元素之间的关系。
3. 基本操作：施加于数据对象上的一组运算及其语义。

用三元组表示就是 (D, S, P)，其中 D 是数据对象，S 是 D 上的关系集，P 是对 D 的基本操作集。

关键性质：**ADT 与具体实现无关**。同一个 ADT 可以有不同的实现（例如「栈」这个 ADT 可以顺序实现也可以链式实现），只要对外提供相同的操作语义，使用者就不需要改变程序。这样做的意义是信息隐藏和数据封装——使用 ADT 的人只依赖接口，实现者可以自由更换内部结构。

ADT 的判断标准

判断一段描述是不是 ADT，只看它是否只规定了逻辑特性和操作功能，而没有规定用什么存储结构、用什么算法实现。凡出现「用数组」「用指针」「用循环实现」这类字样的，都不是 ADT 的定义，而是实现。

以「栈」为例，它作为 ADT 的定义应当是：数据对象是 $D = \{a_1, a_2, \ldots, a_n\}$（$n \geq 0$ 的有限序列）；数据关系是 $R = \{\langle a_{i-1}, a_i \rangle \mid 2 \leq i \leq n\}$（相邻元素间有序偶关系）；基本操作是 `InitStack(S)`、`Push(S,x)`、`Pop(S,&x)`、`GetTop(S,&x)`、`StackEmpty(S)` 等。定义里完全没有出现「数组」或「链表」——这正是抽象的体现。等到讲栈的顺序存储与链式存储时，才是在为这个 ADT 补上两种不同实现，二者的操作语义完全一致（都满足后进先出），只是时空代价不同。

这个「定义与实现分离」的思想在后续各章反复使用：线性表、栈、队列、串都是 ADT，每一种都有顺序与链式两套实现，考纲也正是按「基本概念 → 顺序存储结构 → 链式存储结构」的顺序展开的。

二、算法的基本概念

（一）算法的定义

算法（**Algorithm**）：对特定问题求解步骤的一种描述，它是指令的有限序列，其中每条指令表示一个或多个操作。

需要把三个词区分清楚：算法（解决问题的方法与步骤）、程序（算法用某种程序设计语言的具体实现）、数据结构（算法操作的对象）。著名论断是「程序 = 数据结构 + 算法」，强调的是算法设计与数据结构选择互相依赖——同一问题换一种数据结构，算法往往完全不同。

算法 ≠ 程序

- 程序可以不满足有穷性。操作系统常驻内存的调度程序、事件驱动的服务器主循环，都是「无限循环」，它们是程序但不是算法。
- 算法必须满足有穷性。
- 算法可以用自然语言、流程图、伪代码描述；程序必须用某种语言可编译运行。

因此「算法一定是程序」「程序一定是算法」两个说法都错。

（二）算法的五个特性

特性	含义	反例（不满足该特性的情形）
有穷性	算法在执行有穷步之后终止，且每一步都在有穷时间内完成	无限循环、死循环
确定性	每条指令有确切含义，无二义性；相同输入只能得到相同输出	「取一个较大的数」「若 x 为正则加 1，否则加 2 或 3」
可行性	算法的每一步都能通过已实现的基本运算执行有穷次而实现	「令 x 等于最小的正整数使……」
输入	有零个或多个输入	无输入的算法合法，如输出常量
输出	有一个或多个输出	无输出的算法无意义，不合法

输入与输出的不对称

输入可以是零个，输出必须至少一个。这是五个特性里唯一「不对称」的一条，考试最爱在这里设选项（例如「算法可以没有输入，也可以没有输出」——后半句错）。另外「有穷性」是算法区别于程序的核心，「确定性」要求「相同输入必得相同输出」，注意这与后面要讲的「随机化算法」不冲突：如果算法内部使用随机数，那么它就不是确定性的，属于另一类算法。

（三）好算法的目标

可从五个维度衡量：

1. 正确性（**Correctness**）：能正确解决求解问题，对合法输入给出满足规格说明的结果。这是最基本、最重要的要求。
2. 可读性（**Readability**）：便于人们阅读、理解和交流，也便于调试修改。
3. 健壮性（**Robustness**）：对非法输入能做出恰当反应或处理，而不会产生莫名其妙的结果。
4. 高效率与低存储量需求：效率指算法执行的时间（时间复杂度），存储量指算法执行过程中所需的最大存储空间（空间复杂度）。这里就顺理成章地引出了本章后半部分的全部内容。

其中，正确性是前提，其余四项常在保证正确性的基础上权衡。时间与空间往往互相冲突：提高时间效率常需增加空间开销（如用散列表换查找时间），节省空间常需牺牲时间（如用多次计算代替查表）。在大多数场景下，时间比空间更宝贵，因此「以空间换时间」是常用策略。

（四）时间复杂度

1.（1）定义：问题规模、语句频度与渐进记号

- 问题规模（**Problem Size**）：算法求解问题输入量的多少，用整数 n 表示。规模的定义要看问题本身：对数组是元素个数，对矩阵是阶数，对字符串是串长，对图是顶点数与边数。
- 语句频度（**Frequency Count**）：该语句在算法中被重复执行的次数。
- 时间复杂度：算法中所有语句的频度之和，记为 $T(n)$，是问题规模 n 的函数。

注意 $T(n)$ 是「真实次数」，我们真正关心的是当 n 趋于无穷时它的增长趋势，于是引入渐进记号。

$$T(n) = O(f(n)) \iff \exists\, c > 0,\ n_0 > 0,\ \forall n \geq n_0 :\ 0 \leq T(n) \leq c\, f(n)$$

含义：当 n 足够大以后，$T(n)$ 被 $f(n)$ 的某个常数倍从上方控制住。所以 O 给出的是上界，我们通常说「算法的时间复杂度是 $O(f(n))$」。

时间复杂度的本质是增长趋势，不是运行时间

「时间复杂度分析」统计的不是算法跑了多少毫秒，而是算法运行时间随数据量变大时的增长趋势。因此：

- 忽略与 n 无关的操作：顺序执行的语句只影响常数项，可以丢掉。
- 省略所有系数：$T(n) = 3n^2 + 5n + 7$ 记为 $O(n^2)$。
- 乘法法则用于嵌套：多层嵌套循环，总频度是各层频度之积（仅当各层相互独立时，后面会看到反例）。

一套可执行的三步流程：① 找出基本操作（循环最内层的语句）→ ② 用 n 表示它的执行次数（解方程或求和）→ ③ 只保留最高阶项并去掉系数。

2.（2）常见复杂度类型与增长速度

按增长速度从低到高排列：

$$O(1) < O(\log n) < O(\sqrt{n}) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n) < O(n!) < O(n^n)$$

助记口诀「常对幂指阶」对应前四档：常数阶、对数阶、幂（多项式）阶、指数阶。当 n 充分大时，增长速度差异极为悬殊：$n = 1000$ 时 $\log_2 n \approx 10$，$n \log_2 n \approx 10^4$，$n^2 = 10^6$，2^n 已经是天文数字。

三个高频陷阱

1. $O(1)$ 不代表只执行 **1** 次，而是「执行次数与 n 无关」。下面这段代码的循环体执行 1000000 次，时间复杂度仍是 $O(1)$。
2. log 的底数写成常数时不影响阶。$\log_2 n$ 与 $\log_{10} n$ 只差一个常数因子 $\log_2 10$，所以统考里 $O(\log_{10} n)$ 与 $O(\log_2 n)$ 视为同一复杂度，统一写 $O(\log n)$。但如果题目问「循环体执行多少次」，则必须写清底数。
3. 对数 **vs** 线性对数 **vs** 线性：判断嵌套循环时，先分别求出两层各自的迭代次数，再决定是相加还是相乘，不能一律相乘。

```
/* 时间复杂度 O(1)：循环次数与 n 无关 */
void algorithm_C(int n) {
    int count = 0;
    for (int i = 0; i < 1000000; i++)   /* 固定 100 万次 */
        count++;
}
```

3.（3）各类循环结构的时间复杂度推导

这是统考第一题的核心，必须掌握「定义法」（解方程）而不是靠记忆答案。

① 线性阶：单层循环，变量每次加 **1**

```
int count = 0;
for (int i = 0; i < n; i++)
    count++;
```

设循环体执行 k 次，则退出时 $i = k$ 首次不小于 n，故 $k = n$，即

$$T(n) = \sum_{i=0}^{n-1} 1 = n = O(n)$$

`while` 写法同理：`while (i < n) { count++; i++; }`，i 从 0 增到 n，执行 n 次。

② 对数阶：变量每次乘 **2**

```
int count = 0;
for (int i = 1; i <= n; i *= 2)
    count++;
```

设循环体执行 k 次后退出，此时 $i = 2^k$ 首次超过 n，即 $2^{k-1} \le n < 2^k$，两边取以 2 为底的对数得

$$k - 1 \le \log_2 n < k \implies k = \lfloor \log_2 n \rfloor + 1$$

$$T(n) = \sum_{r=1}^{\lfloor \log_2 n \rfloor + 1} 1 = O(\log n)$$

关键操作：把「循环变量的变化规律」翻译成等式或不等式，解出 k 与 n 的关系。该方法可推广到一切「每次变化固定倍数」的循环。

③ **2011-01** 真题：起点非 **1** 的对数阶

```
x = 2;
while (x < n / 2)
    x = 2 * x;
```

变量 x 初值为 2，每轮翻倍。设循环体执行 k 次后退出，此时 $x = 2 \cdot 2^k = 2^{k+1}$ 首次不小于 $n/2$，即

$$2^{k+1} \ge \frac{n}{2} \implies k + 1 \ge \log_2 n - 1 \implies k \approx \log_2 n - 2$$

所以循环次数与 $\log_2 n$ 同阶，$T(n) = O(\log n)$，选 A。

2011-01 的两个坑

- 初值 2 和步长 2 合在一起使 $x = 2^{k+1}$ 而不是 2^k，但这只影响常数，不影响阶。

- 上界是 $n/2$ 而不是 n，$\log_2(n/2) = \log_2 n - 1$，同样只差常数。判断阶时这些细节都可以丢，但要能写出等式说明它们确实只差常数。
- 也有把条件误读成「$x < n$」而算出别样的次数，进而错选 B；只要抓住「每轮 $\times 2$」就必然是对数级。

④ **2014-01** 真题：内层与外层无关的嵌套

```
count = 0;
for (k = 1; k <= n; k *= 2)
    for (j = 1; j <= n; j++)
        count++;
```

外层：k 取 $1, 2, 4, \ldots, 2^r$，满足 $2^r \le n < 2^{r+1}$，执行 $r+1 = \lfloor \log_2 n \rfloor + 1$ 次。内层：j 从 1 到 n，每次固定执行 n 次。内层的次数与外层变量 k 完全无关，因此可以直接相乘：

$$T(n) = \sum_{r=0}^{\lfloor \log_2 n \rfloor} \sum_{j=1}^{n} 1 = \sum_{r=0}^{\lfloor \log_2 n \rfloor} n = n(\lfloor \log_2 n \rfloor + 1) = O(n \log n)$$

选 C。

⑤ 方根阶：条件写在循环变量上

```
int count = 0;
for (int i = 0; i * i < n; i++)
    count++;
```

设循环体执行 k 次后退出，$i = k$ 首次满足 $k^2 \ge n$，故 $k = \lceil \sqrt{n} \rceil$，

$$T(n) = O(\sqrt{n})$$

⑥ **2019-01** 真题：条件含平方的 **while**

```
x = 0;
while (n >= (x + 1) * (x + 1))
    x = x + 1;
```

x 从 0 开始每轮只加 1，条件成立等价于 $(x+1)^2 \le n$ 即 $x \le \sqrt{n} - 1$。因此 x 能取到 $0, 1, \ldots, \lfloor \sqrt{n} \rfloor - 1$，循环体执行 $\lfloor \sqrt{n} \rfloor$ 次，

$$T(n) = O(\sqrt{n})$$

选 B。

「条件里出现了平方」不等于 $O(n^2)$

2019-01 的迷惑项就是 $O(n^2)$。判断依据是循环体真正执行了多少次，即循环变量达到退出条件所需的迭代次数，而不是条件表达式的复杂度。条件中的乘法、比较都是常数时间操作。

⑦ **2017-01** 真题：累加类 **while**（需解方程）

```
int i = 0, sum = 0;
while (sum < n)
    sum += ++i;
return i;
```

`sum += ++i` 的含义是先把 i 加 1，再把新的 i 累加到 sum。因此每轮至少执行一次（`++i` 与加法都是常数时间），且 sum 每轮增加的量依次为 $1, 2, 3, \ldots$。

设循环体执行 k 次后 sum 首次达到或超过 n，此时

$$sum = 1 + 2 + 3 + \cdots + k = \frac{k(k+1)}{2} \geq n$$

解一元二次不等式：

$$k^2 + k - 2n \geq 0 \implies k \geq \frac{-1 + \sqrt{1 + 8n}}{2}$$

去掉常数与系数得

$$T(n) = O(\sqrt{n})$$

选 B。

不要把 sum 每轮的增加量当成固定值

本题的最大误区是认为 sum 每轮加 1，于是得出循环 n 次、复杂度 $O(n)$。实际上 sum 每轮增量依次为 $1, 2, 3, \ldots, k$，增长速度越来越快，所以根本不需要 n 轮——大约 $\sqrt{2n}$ 轮就够了。凡是遇到「累加」型循环，先写出累加量的通项，再列方程。

4.（4）嵌套循环的两种情形

嵌套循环是失分重灾区，判定的第一件事是：内层循环的次数是否依赖外层变量？

情形 **A**：内外层无关 → 直接相乘

```
for (int i = 1; i <= n; i++)          /* n 次 */
    for (int j = 1; j <= n; j++)      /* n 次，与 i 无关 */
        count++;
```

$$T(n) = \sum_{i=1}^{n} \sum_{j=1}^{n} 1 = \sum_{i=1}^{n} n = n^2 = O(n^2)$$

若两层都是倍数型：`for(i=1;i<n;i*=2) for(j=1;j<n;j*=2)`，两层各 $\log n$ 次，得 $O(\log^2 n)$。若两层都是方根型：`for(i=0;i*i<n;i++) for(j=0;j*j<n;j++)`，两层各 $\sqrt{n}$ 次，得 $O(\sqrt{n} \cdot \sqrt{n}) = O(n)$。

一般形式：若外层终止条件为 $f(i) \geq n$，内层为 $g(j) \geq n$ 且 f, g 单调，则

$$T(n) = O\big(f^{-1}(n)\big) \cdot O\big(g^{-1}(n)\big)$$

情形 **B**：内层次数依赖外层 → 求和，不能乘

```
int count = 0;
for (int i = 0; i < n; i++)
    for (int j = 0; j < i; j++)
        count++;
```

内层执行 i 次，于是

$$T(n)=\sum_{i=0}^{n-1}\sum_{j=0}^{i-1}1=\sum_{i=0}^{n-1}i=0+1+2+\cdots+(n-1)=\frac{n(n-1)}{2}=O(n^2)$$

虽然结果仍是 $O(n^2)$，但系数是 $1/2$ 而不是 1，说明必须逐轮求和。
若内层是方根型且依赖外层：

```
for (int i = 1; i * i <= n; i++)
    for (int j = 1; j <= i; j++)
        count++;
```

外层 i 从 1 到 $\lfloor\sqrt{n}\rfloor$，内层 j 从 1 到 i：

$$T(n)=\sum_{i=1}^{\lfloor\sqrt{n}\rfloor}\sum_{j=1}^{i}1=\sum_{i=1}^{\lfloor\sqrt{n}\rfloor}i=\frac{\sqrt{n}(\sqrt{n}+1)}{2}=O(n)$$

⑧ **2022-01** 真题：外层倍数型 + 内层上界为外层变量

```
int sum = 0;
for (int i = 1; i < n; i *= 2)
    for (int j = 0; j < i; j++)
        sum++;
```

外层 i 依次取 $1,2,4,\ldots,2^{k-1}$，其中 $2^{k-1}<n\le 2^k$；内层执行次数等于当前的 i。各轮内层次数依次为

$$1,\ 2,\ 4,\ \ldots,\ 2^{k-1}$$

这是一个等比数列，求和：

$$T(n)=\sum_{i\in\{1,2,4,\ldots,2^{k-1}\}}i=1+2+4+\cdots+2^{k-1}=2^k-1$$

由 $2^{k-1}<n\le 2^k$ 得 $2^k<2n$，所以

$$T(n)<2n-1=O(n)$$

选 B。

2022-01 的致命误区：把「外层 $\log n$ × 内层最大 n」相乘

这样会得到 $O(n\log n)$，是本题最强的干扰项 C。错因是：内层并不是每轮都执行 n 次，只有最后几轮才接近 n 次，前面几轮分别只有 $1,2,4,\ldots$ 次。正确做法是逐轮累加，得到等比数列之和与 n 同阶。

判定口诀：内层上界若是外层变量的指数增长结果，总和由最后一项主导，量级是「最大那一项的量级」，即 $O(n)$，而不是「层数 × 最大值」。

可以用求和上限验证：$\sum_{i=1}^{m} 2^{i-1} = 2^m - 1$，其中 $m = \log_2 n$，故和为 $2n-1$。

⑨ 内层倍数型 + 外层线性（出现 $\log(n!)$）

```
int count = 0;
for (int i = 1; i <= n; i++)
    for (int j = 1; j <= i; j *= 2)
        count++;
```

内层 j 从 1 每次乘 2 直到超过 i，执行次数为 $\lfloor \log_2 i \rfloor + 1$，于是

$$T(n) = \sum_{i=1}^{n} \left(\lfloor \log_2 i \rfloor + 1 \right) = \log_2 1 + \log_2 2 + \cdots + \log_2 n + n = \log_2(n!) + n$$

利用斯特林公式或简单放缩可得 $\log_2(n!) = \Theta(n\log n)$，因此

$$T(n) = O(n\log n)$$

需要记住这条常见结论：$\log(n!) = \Theta(n\log n)$。它在归并排序、堆排序、比较类排序下界等分析里反复出现。

⑩ 外层倍数型、内层外层相关的倍数型

```
int count = 0;
for (int i = 1; i <= n; i *= 2)
    for (int j = 1; j <= i; j *= 2)
        count++;
```

外层第 r 轮（$r = 1, 2, \ldots, \lfloor \log_2 n \rfloor + 1$）时 i 约为 2^{r-1}，内层次数约为 $\log_2 i + 1 = r$。故

$$T(n) = \sum_{r=1}^{\lfloor \log_2 n \rfloor + 1} r = \frac{m(m+1)}{2}, \quad m = \lfloor \log_2 n \rfloor + 1$$

$$T(n) = O(\log^2 n)$$

这类循环是「非线性的嵌套非线性变化」，两次都按倍数收敛，所以是 $O(\log^2 n)$。

⑪ 三层线性嵌套：求和公式

```
for (int i = 1; i <= n; i++)
    for (int j = 1; j <= i; j++)
        for (int k = 1; k <= j; k++)
            count++;
```

$$\sum_{i=1}^{n}\sum_{j=1}^{i}\sum_{k=1}^{j}1=\sum_{i=1}^{n}\sum_{j=1}^{i}j=\sum_{i=1}^{n}\frac{i(i+1)}{2}=\frac{1}{2}\left(\frac{n(n+1)(2n+1)}{6}+\frac{n(n+1)}{2}\right)=\frac{n(n+1)(n+2)}{6}=O(n^3)$$

推导中用到两个基本求和公式，建议直接记住并会代数字验证：

$$\sum_{i=1}^{n}i=\frac{n(n+1)}{2},\qquad\sum_{i=1}^{n}i^2=\frac{n(n+1)(2n+1)}{6}$$

再补充一个「方根套方根」的例子：

```
for (int i = 0; i * i < n; i++)
    for (int j = 0; j * j < i; j++)
        count++;
```

内层对每个 i 执行约 $\sqrt{i}$ 次，外层 i 到 $\sqrt{n}$：

$$T(n)=\sum_{i=1}^{\sqrt{n}}\sqrt{i}\approx\int_{0}^{\sqrt{n}}\sqrt{x}\,dx=\frac{2}{3}(\sqrt{n})^{3/2}=\frac{2}{3}n^{3/4}=O(n^{3/4})$$

虽然 $O(n^{3/4})$ 在统考中极少出现，但这个例子很好地说明：只要内外层不独立，就必须老老实实求和。

5.（*5*）递归式的时间复杂度：展开法、递归树与主定理

递归算法的时间复杂度要用递归式（递推式）描述，再用下面四种方法之一求解。统考主要考展开法（也叫迭代法），递归树与主定理属于扩展但能极大提速。

方法一：展开法（迭代展开）

思路：反复把右边的 $T(\cdot)$ 用它自己的定义替换，直到出现边界条件 $T(1)$，然后把层数与每层代价相乘。

以 2012-01 真题（求 n 的阶乘）为例：

```
int fact(int n) {
    if (n <= 1) return 1;
    return n * fact(n - 1);
}
```

递归式为 $T(n)=T(n-1)+O(1)$——注意每层只做一次乘法、一次比较，都是常数时间，所以是 $O(1)$ 而不是 $O(n)$。逐层展开：

$$T(n)=T(n-1)+O(1)=T(n-2)+2O(1)=\cdots=T(1)+n\cdot O(1)=O(n)$$

选 B，辅助空间复杂度为 $O(n)$（递归栈深度为 n）。

同理：

- $T(n)=T(n-1)+n$：展开得 $n+(n-1)+\cdots+1=O(n^2)$。

- $T(n)=T(n/2)+1$（二分查找、对数阶递归）：每次规模减半，共 $\log_2 n+1$ 层，$T(n)=O(\log n)$。
- $T(n)=T(n-1)+T(n-2)$（朴素斐波那契）：呈指数增长，$T(n)=O(2^n)$；加记忆化后为 $O(n)$。

方法二：递归树法

思路：把递归过程画成一棵树。每个结点代表一次子调用的代价，每层所有结点的代价之和是这一层的代价；把各层代价相加即得总代价。只要每层的代价和构成等比数列，求和工作就非常简单。

例：$T(n)=2T(n/2)+n$（二路归并排序的时间递推式）

层次	结点数	每个结点代价	该层代价之和
第 0 层（根）	1	n	n
第 1 层	2	$n/2$	$2\cdot n/2=n$
第 2 层	4	$n/4$	$4\cdot n/4=n$
…	…	…	…
第 j 层	2^j	$n/2^j$	n

规模每次减半，到叶子规模为 1 时停止，共 $\log_2 n+1$ 层。每层代价之和恒为 n，因此

$$T(n)=\underbrace{n+n+\cdots+n}_{\log_2 n+1\ \text{个}}=n(\log_2 n+1)=O(n\log n)$$

例：$T(n)=3T(n/4)+\Theta(n^2)$（主定理的入门例题，答案 $O(n^2)$）

第 j 层有 3^j 个结点，每个结点的分解代价为 $(n/4^j)^2=n^2/16^j$，该层代价之和为

$$3^j\cdot\frac{n^2}{16^j}=n^2\left(\frac{3}{16}\right)^j$$

这是一个公比为 $3/16<1$ 的递减等比数列，故

$$T(n)=n^2\sum_{j\ge 0}\left(\frac{3}{16}\right)^j=n^2\cdot\frac{1}{1-3/16}=\frac{16}{13}n^2=O(n^2)$$

几何级数递减 $\Rightarrow$ 总代价由根结点主导，这正是主定理第三种情况的直观来源。

方法三：代入验证法（猜答案再证明）

先猜出 $T(n)=O(f(n))$，再把归纳假设代回递归式验证 $T(n)\le c\,f(n)$ 对某常数 c 成立。统考一般不要求写完整归纳证明，但用代入法快速验证选项非常有效：把四个选项分别代入递归式看哪个能配平，往往比正推更快。

方法四：主定理（**Master Theorem**）

适用于形如

$$T(n)=a\,T\!\left(\frac{n}{b}\right)+f(n),\qquad a\ge 1,\ b>1,\ f(n)\ \text{渐进正函数}$$

的递归式。注意约束：子问题规模必须等比缩小（n/b 而不是 $n-b$），且各子问题规模相同。方法：设临界指数 $n^{\log_b a}$，把它与 $f(n)$ 比较，分三种情况。

情况	条件	解
情况一	$f(n)=O\big(n^{\log_b a-\varepsilon}\big)$，某 $\varepsilon>0$	$T(n)=\Theta\big(n^{\log_b a}\big)$
情况二	$f(n)=\Theta\big(n^{\log_b a}\big)$	$T(n)=\Theta\big(n^{\log_b a}\log n\big)$
情况三	$f(n)=\Omega\big(n^{\log_b a+\varepsilon}\big)$，某 $\varepsilon>0$，且 $a\,f(n/b)\le c\,f(n)$ 对某 $c<1$ 与足够大的 n 成立	$T(n)=\Theta\big(f(n)\big)$

三种情况的直观含义：情况一是分下去更重要（子问题数量多、叶子层代价最大），情况二是上下一样重（每层代价均衡），情况三是合起来更重要（根结点代价最大）。

情况三里的附加条件 $a\,f(n/b)\le c\,f(n)$ 称为正则性条件，它是技术性要求（保证 f 不会增长得太野），在统考里一般不用自己验证。

再理解一个细节：递归树的叶子数为 $a^{\log_b n}=n^{\log_b a}$，这就是临界指数 $n^{\log_b a}$ 的由来——它正是「分下去」这一侧的总体量级。

主定理用法示例：

- $T(n)=2T(n/2)+n$：$a=2,b=2$，$n^{\log_2 2}=n^1=n$，与 $f(n)=n$ 同阶 $\Rightarrow$ 情况二，$T(n)=\Theta(n\log n)$。
- $T(n)=T(n/2)+1$：$a=1,b=2$，$n^{\log_2 1}=n^0=1$，与 $f(n)=1$ 同阶 $\Rightarrow$ 情况二，$T(n)=\Theta(\log n)$。
- $T(n)=3T(n/4)+n^2$：$a=3,b=4$，$n^{\log_4 3}\approx n^{0.792}$，$f(n)=n^2$ 渐进更大 $\Rightarrow$ 情况三，$T(n)=\Theta(n^2)$（与选项 A 一致）。
- $T(n)=2T(n/2)+n^2$：$a=2,b=2$，$n^{\log_2 2}=n$，$f(n)=n^2$ 渐进更大 $\Rightarrow$ 情况三，$T(n)=\Theta(n^2)$。

完整案例：归并排序为什么是 $O(n\log n)$

以二路归并排序为例走一遍全过程。算法把长度为 n 的数组从中间一分为二，分别递归排序，再把两个有序子数组合并成一个有序数组。合并两个长度约为 $n/2$ 的有序段，需要沿着两个子数组各走一遍，代价是 $\Theta(n)$；子问题有两个，规模各为 $n/2$。因此

$$T(n)=2T\left(\frac{n}{2}\right)+n,\qquad T(1)=\Theta(1)$$

用递归树看：第 0 层合并代价 n，第 1 层两个结点各自合并 $n/2$、合计 n，第 2 层四个结点各自 $n/4$、合计 n……每一层的「分」与「合」代价之和恒为 n，共 $\log_2 n+1$ 层，故

$$T(n)=n(\log_2 n+1)=O(n\log n)$$

这条结论值得单独记住：当每一层的代价之和相同时，总复杂度 = 层数 × 单层代价。这也是「分治法复杂度 = 递归树高度 × 每层代价」这一通式的来源。此时若改用主定理，$a=b=2$，$n^{\log_b a}=n$ 与 $f(n)=n$ 同阶，落入情况二，结果同样是 $O(n\log n)$，两条路径互相印证。

从空间角度看，归并排序需要长度为 n 的辅助数组，故辅助空间 $O(n)$；递归栈深度为 $\log_2 n$，栈帧空间 $O(\log n)$，总空间复杂度取较大者 $O(n)$。这也说明「时间复杂度最优」并不意味着「空间复杂度也最优」——归并排序时间稳定为 $O(n\log n)$，代价是 $O(n)$ 的额外空间；而堆排序时间同为 $O(n\log n)$，额外空间只有 $O(1)$。

主定理的盲区：情况二与情况三之间存在空隙

并非所有形如 $T(n)=aT(n/b)+f(n)$ 的递归式都能套主定理。经典反例是

$$T(n) = 2T\left(\frac{n}{2}\right) + n\log n$$

这里 $n^{\log_2 2} = n$，而 $f(n) = n\log n$ 既不是 $O(n^{1-\varepsilon})$（不满足情况一），也不与 n 同阶（不满足情况二），又不满足 $f(n) = \Omega(n^{1+\varepsilon})$（不满足情况三，因为它只比 n 多一个 log 因子，不是多项式级别的更大）。这个递归式落在情况二与情况三之间的空隙里，主定理失效。

此时改用递归树：每层代价之和都是 $n\log$ 级别的递减量，共 $\log n$ 层，结果为 $T(n) = O(n\log^2 n)$。

结论：主定理是提速工具，不是万能公式。考试若不确定，用递归树或展开法从定义出发最稳。

主定理速用四步

1. 认出形式 $T(n) = aT(n/b) + f(n)$，读出 a、b、$f(n)$。
2. 算出临界指数 $n^{\log_b a}$。
3. 比较 $n^{\log_b a}$ 与 $f(n)$：谁大取谁；同阶则乘一个 $\log n$。
4. 若两者之间只差对数因子（如 n 与 $n\log n$、n 与 $n/\log n$），主定理失效，回退到递归树。

6.（6）均摊分析的一个经典例子：堆化建堆为什么是 $O(n)$

这是「估算不能只看乘起来」的最好例子，也是 2022 年真题（求最小的 10 个数）背后必须理解的分析。

把 n 个元素原地调整成一个堆（从最后一个非叶结点起，逐个向下调整），若粗略估算：需要调整约 $n/2$ 个结点，每个结点最多下沉树高 $O(\log n)$ 层，相乘得到 $O(n\log n)$。但这个估算偏大，因为它忽略了「底层结点远多于顶层结点」这一事实。

设完美二叉树高度为 h（根的高度为 h，叶结点高度为 0）。一个结点下沉的最大层数恰好等于它的高度。第 i 层有 2^{h-i} 个结点、每个结点高度为 i，故总下沉次数为

$$T(h) = \sum_{i=0}^{h} 2^{h-i}\cdot i = 2^0\cdot h + 2^1(h-1) + 2^2(h-2) + \cdots + 2^{h-1}\cdot 1$$

两边乘 2 后与原式错位相减：

$$2T(h) = 2^1 h + 2^2(h-1) + \cdots + 2^h\cdot 1$$

$$2T(h) - T(h) = T(h) = 2^h + 2^{h-1} + \cdots + 2^1 - h = 2^{h+1} - h - 2 = O(2^h)$$

又完美二叉树结点数满足 $n = 2^{h+1} - 1$，即 $2^h = \Theta(n)$，于是

$$T(h) = O(n)$$

所以输入 n 个元素并建堆的时间复杂度是 $O(n)$，而不是 $O(n\log n)$。这个例子告诉我们两条经验：其一，结点数量按层呈指数分布时，低价操作数量巨大，加权求和后由浅层主导；其二，遇到乘积估算与真实求和矛盾时，永远相信求和。

同理，堆的常用操作复杂度为：访问堆顶 $O(1)$、元素入堆 $O(\log n)$、堆顶出堆 $O(\log n)$；用堆求解「求前 k 小」问题的时间复杂度为 $O(n\log k)$、空间复杂度为 $O(k)$（维护一个大小为 k 的大顶堆，遍历数组一次）。

7.（7）空间复杂度

定义：算法运行过程中所需辅助空间随问题规模增长的趋势，同样用渐进记号表示。

算法运行中使用的内存空间分三部分：

1. 输入空间：存储输入数据。
2. 暂存空间：存储运行中的变量、对象、函数上下文等。可再细分为暂存数据（常量、变量、对象）、栈帧空间（每次函数调用在栈顶创建的上下文，函数返回即释放）、指令空间（编译后的程序指令，通常忽略不计）。
3. 输出空间：存储输出数据。

统考口径：空间复杂度一般只统计辅助空间，即「输入空间之外」的那部分，通常记为「暂存数据 + 输出数据 + 栈帧空间」，输入本身不计入。多数教材与真题按「除输入和程序本身之外所需的额外空间」理解，遇到题目明确给了口径时以题面为准。

常见类型从低到高：

$$O(1) < O(\log n) < O(n) < O(n^2) < O(2^n)$$

- $O(1)$：只用了若干常量、变量或固定长度的数组。循环中每次新建的变量在进入下一轮后释放，不累积，仍算 $O(1)$。这类只需 $O(1)$ 辅助空间的算法称为就地算法（**in-place algorithm**）。
- $O(\log n)$：递归深度为对数级，如二分查找递归版、归并排序的栈帧空间。
- $O(n)$：长度为 n 的辅助数组、链表、散列表；或递归深度为 n 的递归（如阶乘递归、单链表递归逆置）。
- $O(n^2)$：二维数组；或「每层递归各开一个长度递减的数组」这类嵌套占用 $n+(n-1)+\cdots+1$ 的情形。
- $O(2^n)$：如建立高度为 n 的满二叉树，结点数 2^n-1。

空间复杂度的两个高频误区

1. 只关注最差空间复杂度。原因是内存是硬性要求，必须保证所有输入下都有足够空间预留。「最差」有两层含义：最差输入分布（如 $n>10$ 才开数组，则最差是 $O(n)$）与运行过程中的峰值内存（某一步同时占用的最大空间）。
2. 递归与循环的栈帧差别：循环里调用 n 次 `func()`，每次都返回并释放栈帧，空间仍是 $O(1)$；而递归 `recur(n-1)` 会同时存在 n 个未返回的调用，占据 $O(n)$ 栈帧空间。看到递归一定要算栈深度。

8.（8）最好、最坏与平均时间复杂度

上一节讨论的都是「执行次数随 n 增长」这一层面的问题，而当算法的执行次数还取决于输入的具体内容（而不只是规模）时，就必须分三种情况讨论。这也是统考主观题的常见小问：「说明你所设计算法的时间复杂度」——如果算法的代价与数据初始状态有关，只回答一个阶是不完整的。

- 最坏时间复杂度：在最坏输入下（执行次数最多）的复杂度，给出运行时间的上界，是最常用的分析指标，保证程序不会更慢。
- 最好时间复杂度：在最好输入下的复杂度，给出下界，参考价值最小。
- 平均时间复杂度：在所有可能输入等概率出现的前提下，算法执行次数的加权平均值，即期望复杂度。

先看一个「逐语句分析」的完整例子，体会如何从代码走到 $T(n)$：

```
void fun(int a[], int n, int s) {
    int i, j, t;
    for (i = s; i < n - 1; i++) {          /* 语句 1: 外层循环 */
        for (j = s; j < n - i - 1; j++) { /* 语句 2: 内层循环 */
            if (a[j] > a[j + 1]) {         /* 语句 3: 比较 */
                t = a[j];                  /* 语句 4: 交换，3 次赋值 */
                a[j] = a[j + 1];
                a[j + 1] = t;
            }
        }
    }
}
```

这是一段从下标 s 开始、对后缀做起泡排序的代码。设参与排序的元素个数为 $m = n - s$，则语句 1 执行 $m - 1$ 次；语句 2 在每轮外层中执行 $m - i - 1$ 次（$i = 1, 2, \ldots, m - 1$），合计

$$\sum_{i=1}^{m-1}(m - i) = \frac{m(m-1)}{2}$$

次；语句 3 与语句 2 同频度；语句 4 只在发生交换时执行。最坏情况（初始为升序，每对都要交换）语句 4 执行 $\frac{m(m-1)}{2}$ 次，最好情况（已降序）为 0 次。于是最坏时间复杂度为

$$T(n) = O\left(\frac{m(m-1)}{2}\right) = O(m^2) = O\big((n-s)^2\big)$$

这段分析给出两点经验。其一，先找准「规模」到底是哪个量：这里真正决定代价的是参与排序的元素个数 $m = n - s$，若 $s = n/2$，参与排序的只剩一半，代价降为 $O(n^2/4)$——阶不变、常数减半，所以不能因为 s 变了就换一个阶。其二，「次数最少」不等于「阶更低」：起泡排序即使输入已经有序，比较次数仍是 $\frac{m(m-1)}{2}$ 次，只是交换为 0 次，最好复杂度依然是 $O(m^2)$——这正是起泡排序必须加「本趟无交换则提前结束」标志位才能把最好情况压到 $O(n)$ 的原因。

由此总结出四条分析惯例：

1. 先写出 $T(n)$ 的表达式（求和式或递推式），再化简出阶。主观题要保留这一步。
2. 最坏情况是默认口径。题目只说「时间复杂度」而未加限定，通常按最坏情况回答；若材料同时给出最好与平均，则三者都要交代。
3. 平均复杂度必须声明概率假设。统考默认「所有可能输入等概率」，例如顺序查找默认每个位置被查到的概率都是 $1/n$。
4. 三者的阶可能相同也可能不同。起泡排序三者都是 $O(n^2)$；快速排序平均 $O(n\log n)$、最坏 $O(n^2)$，差异的关键在枢轴（中轴）的选取与初始序列是否有序。

以顺序查找为例把三种情况算全：在长度为 n 的顺序表中查找给定元素（假定查找成功且等概率）。

- 最好情况：目标在第一个位置，比较 1 次，$O(1)$。
- 最坏情况：目标在最后一个位置或不存在，比较 n 次，$O(n)$。
- 平均情况：比较次数为

$$ASL = \sum_{i=1}^{n} p_i \cdot c_i = \frac{1}{n}(1 + 2 + \cdots + n) = \frac{n+1}{2} = O(n)$$

其中 $p_i = \frac{1}{n}$ 是等概率假设，$c_i = i$ 是命中第 i 个元素所需的比较次数。如果按 2023-01 的思路考察顺序表的其他操作：按位获取第 i 个元素只需一次地址计算，平均 $O(1)$；而插入、删除平均要移动约一半元素，平均 $O(n)$；有序表的折半查找平均 $O(\log n)$。

三者的关系与考法

- 最好 $\le$ 平均 $\le$ 最坏，且一般情况下三者不同阶就是信息（例如快速排序：平均 $O(n\log n)$，最坏 $O(n^2)$，最好 $O(n\log n)$）。
- 统考问「平均时间复杂度」时，如果没说概率分布，默认等概率。
- 如果题目给出的是「有序表」「顺序存储」这两个条件，要立刻想到折半查找 $O(\log n)$ 和随机存取 $O(1)$ 这两个可能答案。

9.（9）常见算法与操作复杂度对照表

这张表建议背下来，它是各章复杂度题的答案来源，也是主观题「说明你所设计算法的时间复杂度」的评分依据。

算法 / 操作	平均时间复杂度	最坏时间复杂度	空间复杂度	备注
顺序查找（无序/有序顺序表）	$O(n)$	$O(n)$	$O(1)$	有序时失败查找可提前终止
折半查找（有序顺序表）	$O(\log n)$	$O(\log n)$	$O(1)$（迭代）/ $O(\log n)$（递归栈）	只适用于顺序存储且有序
分块查找（索引顺序查找）	$O(\sqrt{n})$	$O(\sqrt{n})$	$O(n)$（索引表）	块内顺序查找，块间用索引
直接插入排序	$O(n^2)$	$O(n^2)$	$O(1)$	基本有序时最好可到 $O(n)$
折半插入排序	$O(n^2)$	$O(n^2)$	$O(1)$	比较次数降至 $O(n\log n)$，移动次数不变
起泡排序（bubble sort）	$O(n^2)$	$O(n^2)$	$O(1)$	带标志位优化后最好 $O(n)$
简单选择排序	$O(n^2)$	$O(n^2)$	$O(1)$	比较次数固定 $\frac{n(n-1)}{2}$
希尔排序（shell sort）	约 $O(n^{1.3})$	$O(n^2)$	$O(1)$	复杂度依赖增量序列，不作精确要求
快速排序	$O(n\log n)$	$O(n^2)$	$O(\log n)$（递归栈）	最坏出现在基本有序且取首元素为枢轴
堆排序	$O(n\log n)$	$O(n\log n)$	$O(1)$	建堆 $O(n)$，每次调整 $O(\log n)$
二路归并排序（merge sort）	$O(n\log n)$	$O(n\log n)$	$O(n)$	需要辅助数组
基数排序	$O(d(n+r))$	$O(d(n+r))$	$O(r)$	d 为位数，r 为基数，稳定
顺序表按下标取值	$O(1)$	$O(1)$	$O(1)$	随机存取特性
顺序表插入（平均）	$O(n)$	$O(n)$	$O(1)$	平均移动 $n/2$ 个元素
顺序表删除（平均）	$O(n)$	$O(n)$	$O(1)$	平均移动 $(n-1)/2$ 个元素
单链表按序号查找	$O(n)$	$O(n)$	$O(1)$	顺序存取特性
单链表在已知结点后插入	$O(1)$	$O(1)$	$O(1)$	不需移动元素
栈的入栈 / 出栈	$O(1)$	$O(1)$	$O(1)$	顺序栈与链栈相同
循环队列的入队 / 出队	$O(1)$	$O(1)$	$O(1)$	只改 `front`/`rear`

算法 / 操作	平均时间复杂度	最坏时间复杂度	空间复杂度	备注
二叉树 / 树 / 图的遍历	$O(n)$	$O(n)$	$O(n)$（栈或队列）	图按邻接表为 $O(V+E)$，邻接矩阵为 $O(V^2)$
求前 k 小元素（大顶堆法）	$O(n\log k)$	$O(n\log k)$	$O(k)$	2022 年真题的最优解法
输入 n 个元素并建堆	$O(n)$	$O(n)$	$O(1)$	自底向上堆化，不是 $O(n\log n)$
堆的插入 / 删除堆顶	$O(\log n)$	$O(\log n)$	$O(1)$	向上/向下调整一层为 $O(1)$

历年真题分布（数据结构第一章）

- **2011-01**：`while (x < n/2) x = 2*x;` 求复杂度，答案 $O(\log n)$。
- **2012-01**：阶乘递归 `fact(n)`，答案 $O(n)$（辅助空间 $O(n)$）。
- **2013-01**：两个升序链表（长 m、n）合并为降序链表的最坏时间复杂度，答案 $O(\max(m,n))$；因为每个结点最多被访问移动一次，总代价 $O(m+n)$，而 $m+n\le 2\max(m,n)$，故与 $\max(m,n)$ 同阶，不能选 $O(\min(m,n))$。
- **2014-01**：外层 `k *= 2`、内层 `j <= n`，答案 $O(n\log n)$。
- **2017-01**：`while (sum < n) sum += ++i;`，答案 $O(n^{1/2})$。
- **2019-01**：`while (n >= (x+1)*(x+1)) x++;`，答案 $O(n^{1/2})$。
- **2022-01**：外层 `i *= 2`、内层 `j < i`，答案 $O(n)$。
- **2023-01**：顺序存储有序表中平均时间复杂度为 $O(1)$ 的操作，答案「获取第 i 个元素」。

规律很明显：第一章第一题几乎固定是复杂度计算，且集中在「循环变量非线性变化」与「内外层相关」两类。把上面十个推导例子练熟，这类题就是稳拿分。

本章小结表

考纲条目	核心内容	必须记住的结论
（一）数据结构的基本概念	数据、数据元素（基本单位）、数据项（最小单位）、数据对象（性质相同元素的集合）、数据结构	数据结构 = 逻辑结构 + 存储结构 + 数据的运算
逻辑结构	线性（一对一）与非线性（一对多、多对多）；四分法为集合、线性、树形、图形	栈、队列、线性表逻辑结构相同，都是线性结构
存储结构	顺序（位置表关系）、链式（指针表关系）、索引（索引表）、散列（函数算地址）	顺序存储支持随机存取；链式存储只能顺序存取
抽象数据类型	数学模型 + 定义在其上的一组操作；由数据对象、数据关系、基本操作构成，记为 (D,S,P)	ADT 与实现无关，只看「能做什么」，不看「怎么实现」
（二）算法的基本概念	算法是对特定问题求解步骤的描述，是指令的有限序列	算法 $\neq$ 程序；程序可以不满足有穷性
算法的五个特性	有穷性、确定性、可行性、输入、输出	输入可以是零个，输出必须至少一个
好算法的目标	正确性、可读性、健壮性、高效率与低存储量	正确性最重要；时间与空间常互相冲突
时间复杂度	$T(n)=O(f(n))$ 表示增长趋势的上界	只保留最高阶项、去掉系数；log 底数不影响阶

考纲条目	核心内容	必须记住的结论
循环复杂度推导	定义法：设循环变量满足退出条件，解出迭代次数	独立的嵌套相乘，相关的嵌套求和
递归复杂度	展开法、递归树法、代入法、主定理	$T(n)=2T(n/2)+n=O(n\log n)$；主定理在 $f(n)$ 与 $n^{\log_b a}$ 只差对数因子时失效
空间复杂度	辅助空间随规模的增长趋势；只关注最差	递归要算栈深度：n 层递归为 $O(n)$
最好 / 最坏 / 平均	最坏给上界，平均按等概率加权	顺序查找平均比较次数 $\frac{n+1}{2}=O(n)$

易错清单

1. 数据的最小单位是数据项，基本单位是数据元素。两者互换即为错项。
2. 数据对象是数据的子集，不能说「数据对象就是数据」。
3. 逻辑结构与存储结构不能混判。「链式存储是线性结构/非线性结构」的说法本身跑偏：链式说的是存储方式，线性/非线性说的是逻辑关系。
4. 顺序存储 $\Leftrightarrow$ 随机存取，链式存储 $\Leftrightarrow$ 顺序存取。不要写「链式存储可以随机存取」。
5. 索引存储不是顺序存储，它的数据区可以离散；散列存储的地址由散列函数算出，不是按逻辑顺序排布。
6. 算法与程序不等价：程序可以无限循环，算法必须有穷；算法可以没有输入但不能没有输出。
7. 五特性中没有「高效性」「可读性」。可读性、健壮性属于「好算法的目标」，不属于算法的特性，这是常见的混淆选项。
8. $O(1)$ 不是「执行一次」，而是「次数与 n 无关」；执行一千万次的固定循环仍是 $O(1)$。
9. $\log_2 n$ 与 $\log_{10} n$ 是同一复杂度阶，选项里同时出现 $O(\log_2 n)$ 与 $O(\log_{10} n)$ 时不能据此区分；但问「执行多少次」时必须写底数。
10. 判断循环次数要解方程，不能看条件里有什么运算。2019-01 的条件含平方，答案是 $O(\sqrt{n})$ 而不是 $O(n^2)$。
11. 累加型循环的增量不是常数。`sum += ++i` 的增量是 $1,2,3,\ldots$，方程为 $\frac{k(k+1)}{2}\geq n$，得 $O(\sqrt{n})$。
12. 内外层独立才能相乘。只要内层上界依赖外层变量，就必须逐轮求和，否则会把 2022-01 误判成 $O(n\log n)$。
13. 指数增长的内层，总和由最后一项主导。$1+2+4+\cdots+2^{k-1}=2^k-1$，与 n 同阶，不是「层数 $\times$ 最大值」。
14. $\log(n!)=\Theta(n\log n)$，见到「内层为 $\log i$、外层遍历 i 到 n」的求和要立刻反应过来。
15. 主定理不是万能的。$f(n)$ 与 $n^{\log_b a}$ 只差一个对数因子时（如 $T(n)=2T(n/2)+n\log n$）落在情况二与三的空隙中，必须改用递归树，答案为 $O(n\log^2 n)$。
16. 主定理要求子问题规模等比缩小。像 $T(n)=T(n-1)+O(1)$ 这种线性递减的递归式不满足条件，只能用展开法。
17. 空间复杂度要算递归栈深度。循环调用 n 次函数是 $O(1)$，递归 n 层是 $O(n)$。
18. 建堆是 $O(n)$，不是 $O(n\log n)$；只有「依次插入建堆」才是 $O(n\log n)$。两者要分清。
19. 平均复杂度默认等概率，且注意区分「查找成功」与「查找成功与失败都计入」两种口径——顺序查找失败时的比较次数是 n（或 $n+1$，取决于哨兵设置），会改变平均值。
20. 写主观题时要写规则：题目问「说明时间复杂度」时，必须给出 $T(n)$ 的表达式或求和式再给

出阶，只写一个 $O(n \log n)$ 容易被扣过程分。

第二章线性表

本章考纲要求

二、线性表（一）线性表的基本概念（二）线性表的实现

1. 顺序存储
2. 链式存储

（三）线性表的应用

[考查目标]

1. 掌握数据结构的基本概念、基本原理和基本方法。
2. 掌握数据的逻辑结构、存储结构及基本操作的实现，能够对算法进行基本的时间复杂度与空间复杂度的分析。
3. 能够运用数据结构的基本原理和方法进行问题的分析与求解，具备采用 C 或 C++ 语言设计与实现算法的能力。

本章是数据结构部分的第一道「大题」来源。统考的 41 题（算法设计题，15 分左右）在 2009、2010、2011、2012、2013、2015、2018、2019、2020 这些年份都出在线性表上，选择题也几乎年年涉及。因此本章的写法是「概念讲透 + 代码模板背熟」，而不是只记结论。

一、线性表的基本概念

（一）线性表的定义与逻辑特征

线性表（**Linear List**）是具有相同数据类型的 n（$n \geq 0$）个数据元素的有限序列，通常记作

$$L = (a_1, a_2, \cdots, a_i, \cdots, a_n)$$

其中 a_1 称为表头元素，a_n 称为表尾元素；i 是元素 a_i 的位序（从 1 开始数），n 是表长。当 $n = 0$ 时称为空表。

一句话把握它的逻辑特征：

线性表的四条逻辑特征

1. 有穷性：元素个数 n 是有限的。
2. 同类型：所有元素属于同一数据类型（每个元素占用的存储空间相同，这一点是顺序存储能随机存取的前提）。
3. 有序性：元素之间有顺序关系。除第一个元素外，每个元素有且仅有一个直接前驱；除最后一个元素外，每个元素有且仅有一个直接后继。
4. 一对一：a_1 无前驱，a_n 无后继，其余元素恰好一前一后。

线性表是一种线性结构，它的逻辑结构只有「一个前驱一个后继」这一种形态，与它采用什么存储方式无关。同一个线性表既可以顺序存储（顺序表），也可以链式存储（链表），二者的逻辑结构相同、存储结构不同，因而基本操作的实现方式与效率不同。这句话是本章所有对比题的题眼。

从「按逻辑顺序排列」这一点再往下推，可以得到两个必须记牢的推论。第一，线性表中的每个元素都有一个确定的位置编号，这个编号就是位序，位序是由逻辑关系决定的，不随存储方式改变：同一个线性表无论是用数组还是用链表存，a_3 永远是 a_4 的直接前驱。第二，任意两个元素之间的前后关系由它们在序列中的相对位置决定，因此「插入一个元素」在逻辑上的含义是「使原来的第 i 个元素及其后所有元素的位置编号都加一」，在「删除一个元素」时则是减一。这个逻辑语义，正是顺序存储必须搬移元素、而链式存储只需改指针的根本原因——顺序存储把「位置编号」直接编码成了「物理地址的偏移量」，一旦编号变化，物理位置就必须跟着变；链式存储把「前后关系」编码成了「指针」，编号变化时只要改指针，物理位置完全不用动。

再补充两个常考的相邻概念，避免混淆。

- 线性表与数组：数组是存储结构层面的概念（一段连续的、可随机访问的存储空间），线性表是逻辑结构层面的概念。数组可以存线性表，也可以存矩阵、堆、完全二叉树；线性表可以用数组存，也可以用链表存。因此「数组就是线性表」「线性表就是数组」都不成立。
- 线性表与集合：集合中的元素只要求「互异」，元素之间没有顺序关系；线性表要求元素之间有严格的前后次序。所以线性表不是集合，也不能说「线性表是一种特殊的集合」。
- 线性表与串：串是内容受限的线性表（限定数据元素为字符），本质上是线性表的特例，它的存储和操作都可以沿用本章的方法，只是多了模式匹配这类与「内容」有关的操作。本手册把串的内容放在第三部分展开。

「线性表」不等于「顺序表」

常见错误是把「线性表」直接当成数组。线性表是逻辑结构（一对一的有序序列），顺序表和链表是它的两种存储结构。判断题里出现「线性表只能用顺序存储实现」「线性表的元素必须连续存放」一律是错的。

（二）线性表的基本操作

统考与教材通用的操作名（务必认清函数名与语义，选择题常在「返回位序还是返回值」上设陷阱）：

操作名	语义	返回值约定
`InitList(&L)`	初始化，构造一个空表	成功/失败
`DestroyList(&L)`	销毁表，释放全部空间	无
`ListInsert(&L, i, e)`	在位序 i 处插入元素 e，原第 i 个元素及其后元素依次后移	成功/失败
`ListDelete(&L, i, &e)`	删除位序 i 的元素，用 e 带回它的值	成功/失败
`LocateElem(L, e)`	按值查找，找第一个值为 e 的元素	返回其位序；失败返回 0 或 -1
`GetElem(L, i, &e)`	按位查找，取位序 i 的元素，用 e 带回	成功/失败
`ListLength(L)`	求表长	元素个数
`PrintList(L)`	按前后次序输出所有元素	无
`Empty(L)`	判空	真/假

位序与下标必须分清

位序从 **1** 开始，数组下标从 **0** 开始。位序 i 对应的数组下标是 $i-1$。插入的合法位序范围是 $1 \le i \le n+1$（可以有 $n+1$ 个插入位置，因为可以插到表尾之后）；删除的合法位序范围是 $1 \le i \le n$。这两个范围的

差别是最容易被扣分的细节。

这一部分怎么考

- 选择题：给一段操作语句序列判断其效果（如 2024-01 让判断 `q = p->next; p->next = q->next; q->next = h->next; h->next = q;` 的作用）。
- 选择题：给顺序表 / 单链表判断某操作的平均时间复杂度（如 2023-01 问哪个操作平均时间复杂度是 $O(1)$，答案是「获取第 i 个元素」）。
- 选择题：判断涉及移动元素的操作（如 2026-01 问表头插入、表头删除、表尾插入、表尾删除哪些必然产生元素移动，答案是表头插入与表头删除）。
- 算法设计题：见本章第三节。

在九种操作中，真正需要「设计思路」的只有四类：插入、删除、按位查找、按值查找。其余操作（初始化、销毁、求表长、判空、遍历）都是顺序扫描或一次性赋值，两种存储结构下的写法都很直接。因此本章后续所有分析都围绕这四类操作展开，考纲「（二）线性表的实现」实际上就是在讨论这四类操作在两种存储结构下的不同实现代价。

这里还要提醒一个语义细节：教材中 `ListInsert(&L, i, e)` 里的 i 是位序，而 `LocateElem(L, e)` 返回的也是位序，`GetElem(L, i, &e)` 接收的是位序、带回的是元素值。这三者对「位序 / 值」的取用方向相反，是选择题的常见设错点——把 `GetElem` 说成「返回位序」、把 `LocateElem` 说成「返回值」都是错的。

二、线性表的实现

（一）顺序存储

*1.*定义与两种分配方式

顺序表用一组地址连续的存储单元依次存放线性表的元素，逻辑上相邻的元素物理上也相邻。

静态分配：数组大小在编译时确定，一经定义不能改变。

```
#define MaxSize 100         // 顺序表的最大容量
typedef int ElemType;       // 元素类型

typedef struct {
    ElemType data[MaxSize]; // 静态数组存放元素
    int length;             // 当前表长（元素个数）
} SqList;

void InitList(SqList *L) {
    L->length = 0;          // 空表长度为 0，无须清空数组内容
}
```

动态分配：运行时用 `malloc` 申请空间，空间不够时再申请一块更大的并搬移数据。

```
#define InitSize 100

typedef struct {
    ElemType *data;  // 指向动态分配数组的指针
    int MaxSize;     // 数组的最大容量
    int length;      // 当前表长
} SeqList;
```

```
// 初始化：申请初始容量
void InitList(SeqList *L) {
    L->data = (ElemType *)malloc(sizeof(ElemType) * InitSize);
    L->MaxSize = InitSize;
    L->length = 0;
}

// 扩容：三步骤——申请新空间、搬移旧数据、释放旧空间
void IncreaseSize(SeqList *L, int len) {
    ElemType *p = L->data;
    L->data = (ElemType *)malloc(sizeof(ElemType) * (L->MaxSize + len));
    for (int i = 0; i < L->length; i++) {
        L->data[i] = p[i];          // 逐个搬移，不能整体赋值
    }
    L->MaxSize = L->MaxSize + len;
    free(p);                        // 释放原来那片空间
}
```

静态与动态分配的关键差别

静态分配一旦数组满，无法扩容，只能插入失败；动态分配可以在满时扩容，但扩容要搬移全部 n 个元素，时间代价 $O(n)$，而且必须「先申请、再搬移、最后释放旧空间」，顺序不能反。注意：动态分配虽然可以扩容，但它仍然是顺序存储，仍然支持随机访问。

关于扩容还有一个容易忽略的量化结论。若每次扩容只增加固定长度 `len`（即线性增长），插入 n 个元素的累计搬移代价是 $O(n^2)$；若每次扩容把容量加倍（倍增策略），第 i 次扩容搬移 $1+2+4+\cdots+2^{i-1}\approx 2^i$ 个元素，而此后至少还能再插入 2^i 个元素，把搬移代价均摊到每次插入上是常数，因此 n 次插入的总时间仍是 $O(n)$，单次插入的均摊时间是 $O(1)$。统考不要求推导均摊分析，但选择题里出现「动态分配的顺序表插入元素一定是 $O(n)$」这种说法时可以据此判断：均摊意义下是 $O(1)$，最坏情况（恰逢扩容）才是 $O(n)$。

再强调一个概念区分：「动态分配」和「动态存储结构」不是一回事。动态分配讲的是一块顺序空间如何申请和扩充，它仍然是顺序存储；链式存储才是真正的动态存储结构。把「动态数组」理解成链表是典型错误。

最后一点实现细节：用 `malloc` 申请到的空间里是未初始化的随机数据，但顺序表只靠 `length` 判断哪些位置有效，所以初始化时不必把整个数组清零，只把 `length` 置 0 即可（本手册给出的 `InitList` 就是这么写的）。反之，若题目明确要求「初始化时清空数组内容」，则要循环赋值。

*2.*随机访问特性与地址计算

设第一个元素的存储地址为 $LOC(a_1)$，每个元素占 d 个存储单元，则位序为 i 的元素地址为

$$LOC(a_i) = LOC(a_1) + (i-1)\times d$$

只要给出位序 i，一次乘法加法就能算出地址，访问时间与表长 n 无关，这就是随机访问（随机存取）。因此 `GetElem(L, i)` 的时间复杂度是 $O(1)$。

一句话记住

顺序表 ⇒ 随机访问 ⇒ 按位查找 $O(1)$；链表 ⇒ 顺序访问 ⇒ 按位查找 $O(n)$。

「随机访问」这个特性还带来一个常被忽略的考点：顺序表可以用折半查找。前提是表内元素有序（按关键字递增或递减排列），此时按值查找的时间可以从线性查找的 $O(n)$ 降到 $O(\log_2 n)$。这是链

表做不到的——链表的按位访问是 $O(n)$，无法在 $O(1)$ 时间内取到中间元素，所以折半查找只适用于顺序存储的有序表。这一点经常与「查找」那一章联动出题：题干说「有序表」「折半查找」，那它必然是顺序表。

反过来也要注意：顺序表不一定有序。顺序表只保证「逻辑相邻 = 物理相邻」，不保证值的大小关系。随机访问是顺序存储天然具备的，折半查找则是「顺序存储 + 有序」两个条件同时具备才有的能力，二者不能混为一谈。

*3.*插入与删除：移动次数推导

插入：在位序 i 插入，需要把原来的第 $i, i+1, \cdots, n$ 个元素整体后移一位，共移动 $n-i+1$ 个元素；再把 e 放到位序 i 处，表长加 1。

```
// 在位序 i 处插入元素 e，位序从 1 开始
bool ListInsert(SqList *L, int i, ElemType e) {
    if (i < 1 || i > L->length + 1) return false; // 位序越界
    if (L->length >= MaxSize)        return false; // 表满，不能插入
    for (int j = L->length; j >= i; j--) {         // 从后往前依次后移
        L->data[j] = L->data[j - 1];
    }
    L->data[i - 1] = e;   // 位序 i 对应下标 i-1
    L->length++;
    return true;
}
```

设 P_i 是在第 i 个位置插入的概率（$1 \le i \le n+1$），并假设在每个合法位置插入的概率相等，即 $P_i = \dfrac{1}{n+1}$，则插入一个元素所需移动元素的平均次数为

$$E_{\text{ins}} = \sum_{i=1}^{n+1} P_i (n-i+1) = \frac{1}{n+1} \sum_{i=1}^{n+1} (n-i+1) = \frac{1}{n+1} \cdot \frac{n(n+1)}{2} = \frac{n}{2}$$

所以插入操作的平均时间复杂度为 $O(n)$，其中最好情况（插到表尾，$i = n+1$）移动 0 次，最坏情况（插到表头，$i = 1$）移动 n 次。

删除：删除位序 i 的元素，需要把第 $i+1, i+2, \cdots, n$ 个元素整体前移一位，共移动 $n-i$ 个元素，表长减 1。

```
// 删除位序 i 的元素，并用 e 带回其值；位序从 1 开始
bool ListDelete(SqList *L, int i, ElemType *e) {
    if (i < 1 || i > L->length) return false;   // 位序越界
    *e = L->data[i - 1];                        // 先保存被删元素
    for (int j = i; j < L->length; j++) {       // 从前往后依次前移
        L->data[j - 1] = L->data[j];
    }
    L->length--;
    return true;
}
```

同样假设 $P_i = \dfrac{1}{n}$（删除时只有 n 个合法位置），平均移动次数为

$$E_{\text{del}} = \sum_{i=1}^{n} P_i (n-i) = \frac{1}{n} \sum_{i=1}^{n} (n-i) = \frac{1}{n} \cdot \frac{n(n-1)}{2} = \frac{n-1}{2}$$

所以删除操作的平均时间复杂度也是 $O(n)$，最好情况（删表尾）移动 0 次，最坏情况（删表头）移动 $n-1$ 次。

顺序表三种操作的平均次数（必背）

操作	平均移动 / 比较次数	时间复杂度
按位查找 `GetElem`	0 次（直接算地址）	$O(1)$
插入 `ListInsert`	$\frac{n}{2}$ 次	$O(n)$
删除 `ListDelete`	$\frac{n-1}{2}$ 次	$O(n)$
按值查找 `LocateElem`	$\frac{n+1}{2}$ 次（查找成功）	$O(n)$

代一组数验证：$n=5$ 时，插入平均移动 $5/2=2.5$ 次。手工核一遍：6 个插入位置分别移动 $5,4,3,2,1,0$ 次，和是 15，除以 6 得 2.5，一致。删除平均移动 $(5-1)/2=2$ 次；5 个删除位置分别移动 $4,3,2,1,0$ 次，和是 10，除以 5 得 2，一致。

关于这两个公式，有三个必须澄清的点。

第一，公式里的「平均」是概率意义上的平均。推导时用的假设是「插入到每个合法位置的概率相等」且「删除每个位置的概率相等」。如果题目另外给了概率分布（例如「插到表尾的概率是插到其他位置的 2 倍」），就必须按题目给的 P_i 重新代入公式计算，不能直接套 $\frac{n}{2}$ 和 $\frac{n-1}{2}$。

第二，「移动次数」和「时间复杂度」是两个层次的东西。移动次数是精确的操作计数，时间复杂度是增长速度的量级。插入平均 $\frac{n}{2}$ 次、删除平均 $\frac{n-1}{2}$ 次，都是 $O(n)$。考试可能只问次数（填空或选择），也可能问复杂度，看清题干的问法。

第三，最好情况与最坏情况要记牢。插入：插在表尾之后（位序 $n+1$）移动 0 次，是最好；插在表头（位序 1）移动 n 次，是最坏。删除：删表尾（位序 n）移动 0 次，是最好；删表头（位序 1）移动 $n-1$ 次，是最坏。一种便于自查的记忆法是：「移动元素个数 = 需要被跨过的元素个数」——插入时跨过的是原来第 i 到第 n 个，共 $n-i+1$ 个；删除时跨过的是第 $i+1$ 到第 n 个，共 $n-i$ 个。

第四，循环里的移动方向不能反。插入时循环变量必须从大到小（`for (j = length; j >= i; j--)`），因为元素要向后挪，必须先把最后面的挪走，否则会覆盖还没搬的数据；删除时循环变量必须从小到大（`for (j = i; j < length; j++)`），因为元素要向前挪，必须先把最前面的挪走，否则同样会覆盖。这是写代码时最实际的一个坑。

第五，顺序表的插入删除有「移动区间的选择」技巧。如果题目要求「删除所有值为 x 的元素」，用逐个删除的办法每次都要搬移其后所有元素，最坏会退化到 $O(n^2)$；正确做法是用双指针一趟扫描（见第三节），把保留的元素一次填好，最后统一改表长，时间 $O(n)$。「批量删除要用双指针，不要循环调用单次删除」是统考算法题的重要得分意识。

按值查找的比较次数推导：设要查找的元素在位序 i 的概率为 $P_i=\frac{1}{n}$，找到它需要比较 i 次，则

$$ASL_{\text{成功}}=\sum_{i=1}^{n}P_i\cdot i=\frac{1}{n}\cdot\frac{n(n+1)}{2}=\frac{n+1}{2}$$

若要把查找失败也算进去（线性表的顺序查找，失败时比较 n 次后还要再比一次才能判失败），教材常给出 $ASL_{\text{失败}}=n+1$ 与 $ASL_{\text{总}}=\frac{n+1}{2}+n+1$ 之类的写法。统考考「平均查找长度」时若未说明，按查找成功计算，即 $(n+1)/2$；遇到明确要求算失败的题目，按题目给出的概率模型老老实实列式子。

*4.*顺序表的优缺点

- 优点
  1. 随机访问：按位查找 $O(1)$，这是顺序表最大的优势。
  2. 存储密度高：只存数据本身，不需要额外的指针开销（存储密度 $=1$）。
  3. 存储空间是一整块连续空间，对 CPU 缓存友好，实际运行速度快。
- 缺点
  1. 插入、删除平均要移动约一半的元素，平均时间复杂度 $O(n)$。
  2. 需要预先分配连续空间：静态分配可能溢出，动态分配扩容要整体搬移。
  3. 存储空间利用率受「连续」约束：内存里有很多碎片但拼不出一块够大的连续空间时，仍然申请失败。

关于「存储密度」

存储密度 $=\dfrac{\text{数据本身占用的存储量}}{\text{结点占用的存储总量}}$。顺序表为 1；单链表每个结点除数据域外还要存一个指针域，所以存储密度小于 1（严格小于 1，除非结点结构里数据域是空的）。存储密度高不等于效率高，它只反映「空间换结构信息」的比例。

「随机访问」和「存储密度」是顺序表的两张底牌，而它们都来自「一整块连续空间」这一个事实。正因为空间连续，地址才能用乘法直接算出来（随机访问）；也正因为空间连续且按顺序紧密排列，才没有任何空隙需要额外的结构信息去填充（存储密度为 1）。反过来，空间连续也带来了顺序表最大的软肋——「连续」是一个很硬的约束。插入删除要维持连续，就得搬移；预留空间要维持连续，就得整块申请，一旦内存中最大的空闲连续区小于所需容量，即使总的空闲空间足够大也申请不到。这三利三弊是同一个原因的不同侧面，理解了因果关系就不需要死记。

真题考法

2013 年应用题考过顺序存储结构与链式存储结构的 ASL 对比，2023-01 考过哪类操作平均 $O(1)$，2026-01 考过「哪些操作必然导致元素移动」。这类题的共同点是先判断题目描述的是顺序表还是链表——题干里的「存储空间有足够的空闲空间」「保持元素相对顺序」这类措辞通常指向顺序表。

2026-01 那类题怎么想

题目问「在保持表内元素顺序相对不变的情况下，哪些操作会必然导致移动次数」。分析路径是：先确定讨论对象是顺序表（因为「充足的连续空闲空间」和「元素移动」是顺序表的话题）；再检查四种操作是否需要给新元素腾位置或填补空隙。表头插入要给首位置腾空间，$a_1 \sim a_n$ 全部后移，必然移动；表头删除会让 $a_2 \sim a_n$ 全部前移填补，必然移动；表尾插入只需在 `data[length]` 处赋值，不移动任何已有元素；表尾删除只需把表长减 1，也不移动。答案是「表头插入、表头删除」。关键是分清「元素移动」与「指针修改」——链表做这些操作一个元素都不用挪，但题目限制的是顺序表。

（二）链式存储

*1.*单链表的定义与头结点

单链表用一组任意的存储单元存放元素，每个结点含数据域 `data` 和指针域 `next`，`next` 指向直接后继。逻辑相邻的元素物理上不必相邻，靠指针串起来。

```
typedef struct LNode {
    ElemType data;          // 数据域
    struct LNode *next;     // 指针域，指向直接后继
} LNode, *LinkList;
```

头结点是附加在第一个数据结点之前的、不存放数据的结点，头指针指向它。为什么要引入头结点：

头结点的三个作用（常考）

1. 统一了空表和非空表的处理：空表时 head->next == NULL，非空表时 head->next 指向第一个数据结点，判空变成一条统一的语句。
2. 统一了第一个位置和其他位置的插入删除：对第一个数据结点做前插 / 删除时，其前驱是头结点而不是空，不需要单独写「如果插在表头」的分支。
3. 头指针永远非空（指向头结点），指针操作不会因为「表空导致头指针为 NULL」而崩溃。

```
// 带头结点单链表的初始化
bool InitList(LinkList *L) {
    *L = (LNode *)malloc(sizeof(LNode));  // 创建头结点
    if (*L == NULL) return false;         // 内存分配失败
    (*L)->next = NULL;                    // 空表
    return true;
}

// 判空：带头结点判断头结点的 next 是否为空
bool Empty(LinkList L) { return L->next == NULL; }
```

无头结点时的补救办法

如果题目给的链表不带头结点（如 2026-02 的 head = cu 这类题），可以自己申请一个 dummy 结点指向第一个数据结点，把它当头结点用，之后所有操作就与带头结点的链表完全一致。统考的应用题基本都是带头结点的链表，但选择题里会出现不带头的写法，读题时先确认「头指针指向的是头结点还是第一个数据结点」。

*2.*求表长、按位查找、按值查找

```
// 求表长（不计算头结点）
int ListLength(LinkList L) {
    int len = 0;
    LNode *p = L->next;          // 若有头结点，从头结点的后继开始
    while (p != NULL) { len++; p = p->next; }
    return len;
}

// 按位查找：返回位序为 i 的结点指针；i = 0 时返回头结点
LNode *GetElem(LinkList L, int i) {
    if (i < 0) return NULL;
    LNode *p = L;                // p 从头结点出发（头结点是位序 0）
    int j = 0;
    while (p != NULL && j < i) { p = p->next; j++; }
    return p;                    // i 超长时返回 NULL
}

// 按值查找：返回第一个数据域等于 e 的结点
LNode *LocateElem(LinkList L, ElemType e) {
    LNode *p = L->next;
    while (p != NULL && p->data != e) p = p->next;
    return p;
}
```

单链表不能随机访问：要找第 i 个结点，只能从头指针开始顺链逐个走，比较 / 移动次数与 i 成正

比。设 i 等概率取 $1 \sim n$，则按位查找的平均比较次数为 $\frac{1}{n}\sum_{i=1}^{n} i = \frac{n+1}{2}$，按值查找查找成功时也是 $\frac{n+1}{2}$，二者时间复杂度都是 $O(n)$。

这里要澄清一个常见的表述混淆。平均比较次数相同，不代表两种查找「一样快」——按位查找在顺序表里是 $O(1)$，在链表里是 $O(n)$；只有当两者都在「从头逐个比较」这一情形下（顺序表的按值查找、链表的按位/按值查找）才有相同量级的平均比较次数。做题时先问自己：题目问的是哪种存储结构、哪种查找。

另外，链表的按位查找与按值查找虽然量级相同，代码结构有一个重要差别：按位查找是对「位置计数器」做判断（`j < i`），走到第 i 个就停，不存在找不到的情况（除非 i 越界，此时返回 `NULL`）；按值查找是对「数据域」做判断（`p->data != e`），可能一路走到表尾都找不到，必须用 **`p != NULL`** 作循环条件并在结束后返回 **`NULL`**。写代码时把这两个循环条件写反，就会分别导致「多走 / 少走一个结点」和「解引用空指针」。

还要注意单链表的判空条件与头结点有关。带头结点时判空看 `L->next == NULL`；不带头结点时判空看 `L == NULL`。两者的差别不只是少一个空判断：带头结点时头指针永远指向一个真实的结点，因此「对第一个数据结点做前插或删除」不需要特殊分支；不带头的写法则必须专门处理「表头为空」的情形。这也是统考应用题一律给出带头结点链表的原因——它能让算法描述更简洁、更不容易在边界上出错。

*3.*插入与删除的指针操作

后插（在 p 之后插入 s）：口诀是「先连后，再连前」。

```
s->next = p->next;   // 第一步：新结点先接住 p 原来的后继
p->next = s;         // 第二步：再让 p 指向新结点
```

这两句顺序绝对不能颠倒。如果先写 `p->next = s`，那么原来 p 的后继地址就丢了，`s->next` 再也接不上原链表，链表从 s 处断成两截。

前插（在 p 之前插入）：如果只给 `p` 而没有给它的前驱，正常做法要先找到前驱，$O(n)$。但有一个 $O(1)$ 的技巧——「偷梁换柱」：

```
// 在 p 之前插入值为 x 的结点，O(1)
void InsertBefore(LNode *p, ElemType x) {
    LNode *s = (LNode *)malloc(sizeof(LNode));
    s->data = p->data;   // 先把 p 的数据复制给新结点
    s->next = p->next;   // 新结点接在 p 后面
    p->next = s;
    p->data = x;         // 再把 x 写回 p，逻辑上等于 x 插到了 p 之前
}
```

删除：

- 已知待删结点的位序 i：调用 `GetElem(L, i-1)` 找到前驱 p，再执行

```
q = p->next;            // q 指向待删结点
p->next = q->next;      // 前驱跳过待删结点
free(q);                // 释放空间
```

查找前驱需要 $O(n)$，单看删除动作本身是 $O(1)$。

- 已知待删结点的指针 p：不能直接 `free(p)`，因为无法把 p 的前驱接到 p 的后继上。用同样的「偷梁换柱」：

```
// 删除 p 所指结点，前提是 p 不是尾结点
void DeleteNode(LNode *p) {
    if (p == NULL || p->next == NULL) return;   // 尾结点无法用此法删除
    LNode *q = p->next;
    p->data = q->data;       // 用后继的数据覆盖自己
    p->next = q->next;       // 再把自己变成「后继的后继」
    free(q);                 // 删除的是原来的后继结点
}
```

删除自身的两个致命陷阱

1. 尾结点不能用这个技巧。p 是尾结点时 $p \to next$ 为 `NULL`，`q->data` 会解引用空指针。所以必须先判断 `p->next != NULL`。
2. 指针变量不能改。这个技巧只能改 `p` 所指结点的内容，无法让调用者手里的 `p` 变成别的结点。如果题目说「删除 `p` 所指结点并让 `p` 指向其后继」，必须用前驱法或额外返回新的指针。

2024-01 的典型考法

给定带头结点链表，`p` 指向非首非尾结点，执行 `q = p->next; p->next = q->next; q->next = h->next; h->next = q;`。逐句读：前两句把 q 从 p 后面摘下来（p 跳过 q），第三句让 q 指向原来的第一个数据结点，第四句让头结点指向 q。最终效果是把 q 所指结点移动到链表的头结点之后。这类题只要耐心逐句画图，不要去背结论。

把插入删除的指针操作归纳成一句总原则：「先保存，后修改；先连新，后断旧」。之所以必须这样，是因为链表的一切连接关系都只通过指针保存，而指针赋值是破坏性的——一旦 `p->next` 被改成别的值，原来指向的那个结点就失去了唯一的入口，再也找不回来。所以任何一段链表操作代码，都可以用这条原则检查一遍：

- 后插时先写 `s->next = p->next`（保存旧后继），再写 `p->next = s`（修改）。
- 删除时先用 `q` 记住待删结点，再改 `p->next`，最后 `free(q)`（必须先记住再改，否则 `free` 的是错的空间）。
- 双链表插入时，凡是要读「邻居的原指针」的语句都排在修改它之前。
- 遍历时若要回头用 `p->next`，先用临时变量 `nxt = p->next` 存下来。这个习惯在反转链表、K 个一组翻转、交叉合并里反复用到，是链表题不出错的最重要保障。

同时还要舍得用变量。链表题的代码量不大，但指针关系容易绕晕，多定义两个临时指针（如 **pre**、**nxt**、**tail**、**dummy**）比省变量划算得多。统考判分看的是算法正确性和复杂度，不会因为「多用一个指针变量」扣分。相反，把代码压缩成一行套一行，一旦边界错了就是全错。

关于删除还有一点要补充：被删除的结点必须 **free** 释放。不释放只是「逻辑上删掉了」，链表的实际占用空间不会减少，若题目要求「空间复杂度为 $O(1)$」或者「释放结点空间」，漏掉 `free` 会直接失分。另外，不要 **free** 之后还继续访问该结点（悬空指针），正确顺序总是「先接好前后关系，再 `free`，且 `free` 后不再使用 `q`」。

*4.*头插法与尾插法建立单链表

头插法：每读入一个元素就插到头结点之后。输入顺序与链表中的顺序相反，因此头插法天然可以用来就地逆置链表。

```
// 头插法建立单链表，输入 999 结束
LinkList ListHeadInsert(LinkList *L) {
    *L = (LNode *)malloc(sizeof(LNode));
    (*L)->next = NULL;
    ElemType x;
    scanf("%d", &x);
    while (x != 999) {
        LNode *s = (LNode *)malloc(sizeof(LNode));
        s->data = x;
        s->next = (*L)->next;    // 新结点接住头结点原来的后继
        (*L)->next = s;          // 头结点指向新结点
        scanf("%d", &x);
    }
    return *L;
}
```

尾插法：额外用一个尾指针 `r` 指向表尾，每次把新结点接在 `r` 之后并让 `r` 后移。输入顺序与链表顺序一致。如果不用尾指针、每次都从头找表尾，总时间会退化到 $O(n^2)$。

```
// 尾插法建立单链表，输入 999 结束
LinkList ListTailInsert(LinkList *L) {
    *L = (LNode *)malloc(sizeof(LNode));
    (*L)->next = NULL;
    LNode *r = *L;               // r 始终指向表尾结点
    ElemType x;
    scanf("%d", &x);
    while (x != 999) {
        LNode *s = (LNode *)malloc(sizeof(LNode));
        s->data = x;
        s->next = NULL;
        r->next = s;             // 挂到表尾之后
        r = s;                   // 尾指针后移
        scanf("%d", &x);
    }
    return *L;
}
```

头插法 vs 尾插法的用途

- 头插法：顺序反转，用于链表就地逆置、把升序链表合并成降序链表。
- 尾插法：保持相对顺序，用于合并两个升序链表为一个升序链表、拆分链表后保持原有次序。

两种建立方式的时间复杂度都是 $O(n)$（每次插入本身 $O(1)$，共 n 次）。

*5.*双链表

```
typedef struct DNode {
    ElemType data;
    struct DNode *prior, *next;    // 前驱、后继
} DNode, *DLinkList;
```

在 p 之后插入 s（注意四条语句的顺序，可以这样记：先处理「s 的两根线」，再处理「邻居指向 s 的两根线」）：

```
s->next = p->next;          // 1. s 的后继指向 p 的后继
if (p->next != NULL)
    p->next->prior = s;     // 2. p 的原后继的前驱改为 s
s->prior = p;               // 3. s 的前驱指向 p
p->next = s;                // 4. p 的后继改为 s
```

删除 p 的后继结点 q：

```
q = p->next;
p->next = q->next;
if (q->next != NULL)
    q->next->prior = p;
free(q);
```

双链表语句顺序为什么不能乱

关键点：凡是「读取 **p->next**」的语句，必须写在「修改 **p->next**」的语句之前。上面的插入写法里第 2 句用到了 p->next，第 4 句才改 p->next，顺序是对的。2023-02 就是这样考的：题干先给了 s->next = p->next; p->next = s;，此时 p->next 已经变成 s，原来的后继 r 只能通过 s->next 找到，所以后续必须写 s->prev = s->next->prev; s->next->prev = s;——不能再写 **p->next->prev = s**，那会把 s 的前驱指向 s 自己。

*6.*循环链表

循环单链表：表尾结点的 next 不指向 NULL，而是指回头结点，整条链形成一个环。

```
// 初始化空循环单链表
bool InitList(LinkList *L) {
    *L = (LNode *)malloc(sizeof(LNode));
    if (*L == NULL) return false;
    (*L)->next = *L;          // 头结点的 next 指向自己，代表空表
    return true;
}
// 判空
bool Empty(LinkList L) { return L->next == L; }
// 判断 p 是否为表尾结点（循环单链表特有）
bool isTail(LinkList L, LNode *p) { return p->next == L; }
```

循环双链表：头结点的 prior 指向表尾，表尾的 next 指向头结点。

```
bool InitDList(DLinkList *L) {
    *L = (DNode *)malloc(sizeof(DNode));
    if (*L == NULL) return false;
    (*L)->prior = *L;       // 空循环双链表：两根指针都指自己
    (*L)->next  = *L;
    return true;
}
bool EmptyDList(DLinkList L) { return L->next == L; }  // L->prior == L 等价
```

循环链表带来的能力

1. 从任一结点出发都能遍历整条链表；普通单链表从某结点出发只能走到后继方向。
2. 若设置尾指针 `tail`，则访问表头 `tail->next` 和表尾 `tail` 都是 $O(1)$；普通单链表找表尾是 $O(n)$。
3. 插入删除本身与普通链表相同，但在表头或表尾操作时要额外维护「环」的性质：删表尾后要改尾指针，环上有结点时要保证「最后一个结点的 next 指回头结点」。

2021-01 的经典陷阱

带头结点的非空单循环链表，h 是头指针，p 是尾指针，删除第一个元素。正确语句是

```
q = h->next; h->next = q->next; if (p == q) p = h; free(q);
```

要点有两个：先保存 `q` 再改 `h->next`（否则 q 会指向新的首结点，最终释放错结点）；当链表只有一个数据结点时 `p == q`，删除后表空，必须把尾指针 p 重新指向头结点，否则 p 会指向已释放的空间。这类「边界情形」是循环链表题的固定得分点。

7.静态链表

静态链表借助数组来描述链式存储结构：结点仍有数据域和指针域，但指针域里存放的是数组下标（称为游标），而不是真正的地址。它适用于不支持指针的高级语言。

```
#define MaxSize 50
typedef struct {
    ElemType data;
    int next;          // 下一个元素的数组下标，-1 表示结束
} SLinkList[MaxSize];
```

数组的 0 号分量通常充当头结点（`SLinkList[0].next` 指向第一个数据结点）。插入删除只需修改 `next` 的值，不需要移动元素，这一点和动态链表一致；但静态链表不能随机存取，只能从头结点顺着下标找，而且容量固定、不能动态扩容。

静态链表的两个特征值

- `next == -1`：本结点是链表最后一个结点（表示「指向 NULL」）。
- `next == -2`（或题目约定的其他特殊值）：本结点空闲未使用。

构造静态链表时的通用方法：从链接地址为 **NULL** 的那个结点开始反向追溯，就能还原出完整的逻辑顺序。2016-01 就是先给出一张「地址 / 元素 / 链接地址」表，让你还原链表顺序（c → a → e → b → d）后再做插入。

2016-01 的解题流程

已知表头元素为 c，按 c(1008H) → a(1000H) → e(1010H) → b(1004H) → d(100CH) → NULL 还原出逻辑顺序；把 f 放在 1014H 并插到 a 和 e 之间后变成 c → a → f → e → b → d。于是：

- a 的直接后继变成 f，a 的链接地址为 1014H；
- e 的直接后继仍是 b，e 的链接地址仍为 1004H；
- f 的直接后继是 e，f 的链接地址为 1010H。

按题目要求的 **a**、**e**、**f** 的顺序输出，得 1014H，1004H，1010H。注意题干问的顺序（a, e, f）与链表顺序（a, f, e）不一致，审题时按题干给的顺序作答，否则会误选。

（三）顺序表与链表的对比

比较项	顺序表	链表
存储方式	连续空间，逻辑相邻即物理相邻	任意空间，靠指针建立逻辑关系
存取方式	随机存取，按位查找 $O(1)$	顺序存取，按位查找 $O(n)$
按值查找	$O(n)$，平均比较 $\frac{n+1}{2}$ 次	$O(n)$，平均比较 $\frac{n+1}{2}$ 次
插入 / 删除	平均移动 $\frac{n}{2}$ / $\frac{n-1}{2}$ 个元素，$O(n)$	修改指针，本身 $O(1)$；但找位置要 $O(n)$
空间分配	静态分配需预估容量，可能溢出；动态分配扩容要整体搬移	按需申请，不需要连续空间，不会溢出
存储密度	$=1$（无额外结构开销）	<1（每个结点多一个或两个指针域）
逻辑结构	均为线性表（一对一），逻辑结构相同	同左
适用场合	表长可预估、频繁按位访问、很少插入删除	表长变化大、频繁插入删除、难以预估规模

选择的判断口诀

「常查少改」用顺序表，「常改少查」用链表；要按位随机访问用顺序表，要频繁插删用链表。若既要求高效按位访问又要求高效插删，那是更复杂结构（如树、跳表）的用武之地，不在本章范围。

把这张表再压缩成三句可判断的话，选择题里可以直接用：

1. 问「能不能 $O(1)$ 取到第 i 个元素」：只有顺序表（等价说法：随机存取、随机访问、支持下标直接访问）。
2. 问「插入删除是否要移动元素」：顺序表要，链表不要（链表只改指针）。但要注意，若题目还问「找到位置需要几步」，答案都要加上 $O(n)$ 的定位代价。
3. 问「空间是否可能浪费 / 溢出」：顺序表有预分配问题（静态可能溢出、动态会搬移、连续空间可能申请不到），链表不会溢出，但每个结点都要付指针域的空间代价。

三个常见的对比误区

1. 不能说「链表的插入删除一定是 $O(1)$」。只有「已经拿到待操作结点的指针」时才是 $O(1)$；若题目给的是位序 i，找位置本身要 $O(n)$，整体仍是 $O(n)$。
2. 不能说「链表一定比顺序表省空间」。链表省去了「预留连续大空间」的浪费，但每个结点都要付指针域的税，存储密度更低。
3. 循环链表和双向链表不改变时间复杂度的数量级，它们带来的是「能反向走」「从任一结点出发可遍历全程」「插入删除更方便维护」这类常数级或功能性的好处。

三、线性表的应用（算法设计题）

统考 41 题的固定答题格式是三步，务必按顺序写：

1. 给出算法的基本设计思想（用文字说清楚用了什么技巧、扫描几遍、指针怎么走）；
2. 用 **C** 或 **C++** 描述算法，关键之处给出注释（函数要完整，边界要判断）；
3. 说明时间复杂度（必要时说明空间复杂度）。

答题的常见失分点

- 只写代码不写设计思想，或只写思想不写代码——评分是按三小问分点给的。
- 忘记处理空表、只有一个结点、表长小于 k、结点为尾结点等边界情形。

- 写了 sort 等库函数（408 不允许直接用库函数排序，需要自己写快排 / 归并）。
- 空间复杂度超标：题目要求 $O(1)$ 时用了数组或递归（递归栈也是空间）。

（一）（〇）算法设计题的通用解题框架

线性表的算法题看似花样多，实际上绝大多数都能归入下面五种手法之一。拿到题目先判断它属于哪一类，再套相应模板，比从零推导快得多，也稳得多。

手法	适用特征	典型题目
双指针（同向）	需要「原地筛选 / 删除 / 合并」，保持相对顺序	删重复、删值为 x 的元素、合并有序表
双指针（前后 / 距离固定）	需要「倒数第 k 个」这类定位	2009 倒数第 k 个结点、2012 求共同后缀
快慢指针	需要「中间位置」「是否有环」「环的入口」	2019 重排链表、判断环、找中点
就地逆置	需要「反转次序」或「借助反转改变顺序」	2010 循环左移、链表逆置、升序合并成降序
辅助数组 / 计数数组	题目给了值域范围（如 $data$ 的绝对值不超过 n、$0 \le a_i < n$）	2015 删绝对值重复、2013 主元素、2018 最小正整数

判断的关键在题干的两个信号：一是「空间复杂度」的要求（要求 $O(1)$ 就不能用辅助数组，只能原地改指针或引用计数）；二是题目给出的取值范围或有序性条件（给了值域范围往往暗示可以用数组下标做标记；给了有序性往往暗示可以用双指针一趟扫描）。换句话说，题目里出现的每一个条件几乎都是有用信息，不会白给。

写代码时的两条自保策略：一是先把边界情形在纸上列全（空表、单结点、全是要删的元素、一个元素都不删、表长恰好等于 k 等），写代码时逐个处理；二是写完后拿一组 **3～4** 个元素的小数据手工模拟一遍，确认指针没有走空、没有成环、没有漏结点。这两步花不了两分钟，但能避免绝大多数失分。

（二）合并两个有序表（两种存储都考）

顺序表版本：把两个升序顺序表合并为一个升序顺序表，用三个下标 i, j, k 从前往后比较，谁小谁先写。若要求原地合并且目标数组有足够尾部空间，则改用从后往前的双指针，避免覆盖还没比较的数据。

```
// 将升序顺序表 A、B 合并为升序顺序表 C
void MergeSqList(SqList A, SqList B, SqList *C) {
    int i = 0, j = 0, k = 0;
    while (i < A.length && j < B.length) {
        if (A.data[i] <= B.data[j]) C->data[k++] = A.data[i++];
        else                        C->data[k++] = B.data[j++];
    }
    while (i < A.length) C->data[k++] = A.data[i++];  // A 有剩余
    while (j < B.length) C->data[k++] = B.data[j++];  // B 有剩余
    C->length = k;
}
```

时间复杂度 $O(n+m)$，空间复杂度 $O(1)$（不计结果表本身的空间）。

链表版本：

```
// 将两个带头结点的升序单链表合并为一个升序单链表，用尾插法
LinkList MergeList(LinkList A, LinkList B) {
    LinkList C = (LinkList)malloc(sizeof(LNode));
    C->next = NULL;
    LNode *pa = A->next, *pb = B->next, *r = C;  // r 为新表尾指针
    while (pa != NULL && pb != NULL) {
        if (pa->data <= pb->data) { r->next = pa; r = pa; pa = pa->next; }
        else                      { r->next = pb; r = pb; pb = pb->next; }
    }
    r->next = (pa != NULL) ? pa : pb;    // 剩余部分整段接上，不必逐个插入
    free(A); free(B);                    // 释放两个原头结点
    return C;
}
```

升序合并成降序 = 头插法

若题目要求「两个升序链表合并为一条降序链表」（2013 年选择题的考点），必须改成头插法：每次比较取出较小的结点后，用 `s->next = C->next; C->next = s;` 插到结果表头。因为「当前较小」不等于「全体最大」，不能用尾插法。全部结点处理完后，每个结点最多被访问一次，最坏时间复杂度是 $O(n+m)=O(\max(m,n))$（m,n 同阶时二者等价，这正是 2013-01 选 D 的理由）。

（三）就地逆置

顺序表逆置：首尾双指针交换，走到中间停止，时间 $O(n)$、空间 $O(1)$。

```
void ReverseSqList(SqList *L) {
    ElemType tmp;
    for (int i = 0, j = L->length - 1; i < j; i++, j--) {
        tmp = L->data[i]; L->data[i] = L->data[j]; L->data[j] = tmp;
    }
}
```

链表逆置（迭代法，必须背下来）：把链表看成「已逆置部分」和「未逆置部分」，逐个把未逆置部分的第一个结点接到已逆置部分的最前面。

```
// 带头结点单链表的就地逆置
void ReverseList(LinkList L) {
    LNode *pre = NULL, *cur = L->next;
    while (cur != NULL) {
        LNode *next = cur->next;  // 先保存后继，因为下一步要改 cur->next
        cur->next = pre;          // 反转指针方向
        pre = cur;                // pre 后移
        cur = next;               // cur 后移
    }
    L->next = pre;                // 头结点接上新的第一个结点
}
```

迭代法反转后的一个重要性质

反转某一段结束后：**pre** 指向这一段的最后一个结点，**cur** 指向这一段之后的第一个结点（都是在原链表视角下）。这个性质在「反转部分链表」「重排链表」「K 个一组翻转」里反复使用，务必记住。

为什么迭代法比递归法和「入栈法」都好？把三种做法摆在一起看就清楚了：

方法	时间	空间	说明
迭代法（双指针）	$O(n)$	$O(1)$	原地改指针，最优解，必背
头插法	$O(n)$	$O(1)$	依靠待反转段的前驱结点；若无前驱需先造一个 dummy
递归法	$O(n)$	$O(n)$	递归栈深度为 n，且处理不当容易成环
入栈法	$O(n)$	$O(n)$	借助栈的先进后出实现逆序，空间不达标

因此只要题目要求空间 $O(1)$，答案就只能用迭代法或头插法。递归写法虽然短，但空间复杂度不达标，在「空间复杂度为 $O(1)$」的题目里会被判失分。2019 年重排链表题明确要求 $O(1)$ 空间，就是这一点的直接体现。

用头插法逆置也可以，效果相同，本质就是「把结点依次摘下再头插回头结点之后」。为什么迭代法比递归法和「入栈法」都好？把几种做法摆在一起看就清楚了：

方法	时间	空间	说明
迭代法（双指针）	$O(n)$	$O(1)$	原地改指针，最优解，必背
头插法	$O(n)$	$O(1)$	依靠待反转段的前驱结点；若无前驱需先造一个 dummy
递归法	$O(n)$	$O(n)$	递归栈深度为 n，处理不当容易成环
入栈法	$O(n)$	$O(n)$	借助栈的先进后出实现逆序，空间不达标

只要题目要求空间 $O(1)$，答案就只能用迭代法或头插法。递归写法虽然短，但空间复杂度不达标，在「空间复杂度为 $O(1)$」的题目里会被判失分。2019 年重排链表题明确要求 $O(1)$ 空间，就是这一点的直接体现。

头插法逆置的一个常见用法是把「就地逆置」写成与「头插法建表」同构的代码：

```
// 头插法就地逆置带头结点的单链表
void ReverseByHeadInsert(LinkList L) {
    LNode *p = L->next, *r;
    L->next = NULL;              // 先断开，头结点单独作为新表的头
    while (p != NULL) {
        r = p->next;             // 先保存后继
        p->next = L->next;       // 头插：新结点接住当前的第一个结点
        L->next = p;             // 头结点指向新结点
        p = r;                   // 继续处理原来的后继
    }
}
```

对照着看会发现，「头插法建表」和「头插法逆置」其实是同一段程序——这正是「输入顺序与链表顺序相反」这一特性的直接应用，也是 2013 年「升序链表合并为降序链表」能用头插法解决的原因。它的一个常见用法是把「就地逆置」和「建立链表」合并成一段代码：

```
// 头插法就地逆置带头结点的单链表
void ReverseByHeadInsert(LinkList L) {
    LNode *p = L->next, *r;
    L->next = NULL;                 // 先断开，头结点单独作为新表的头
    while (p != NULL) {
        r = p->next;                // 先保存后继
        p->next = L->next;          // 头插：新结点接住当前的第一个结点
        L->next = p;                // 头结点指向新结点
        p = r;                      // 继续处理原来的后继
    }
}
```

两段代码对照着看，会发现「头插法建表」和「头插法逆置」其实是同一段程序——这正是「输入顺序与链表顺序相反」这一特性的直接应用，也是 2013 年「升序链表合并为降序链表」能用头插法解决的原因。

「数组循环左移 p 位」（**2010** 年真题）的经典做法是三次逆置：先把前 p 个元素逆置，再把后 $n-p$ 个元素逆置，最后把整个数组逆置。

```
// 将 R[0..n-1] 循环左移 p 位，O(n) 时间、O(1) 空间
void LeftShift(int R[], int n, int p) {
    p = p % n;                  // 先取模，避免 p > n 时的无谓搬移
    Reverse(R, 0, p - 1);       // 前 p 个逆置
    Reverse(R, p, n - 1);       // 后 n-p 个逆置
    Reverse(R, 0, n - 1);       // 整体逆置
}
```

自检一下：$R=\langle 1,2,3,4,5\rangle$，$p=2$。前 2 个逆置得 $\langle 2,1,3,4,5\rangle$，后 3 个逆置得 $\langle 2,1,5,4,3\rangle$，整体逆置得 $\langle 3,4,5,1,2\rangle$，正是左移 2 位的结果。

为什么三次逆置能得到循环左移？设原序列为 XY，其中 X 是前 p 个元素、Y 是后 $n-p$ 个元素，目标是 YX。记 X' 表示 X 的逆序，则

$$(X'Y')'=YX$$

即「先分别把两段各自逆置，再把整体逆置」，就能把 XY 换成 YX。这是本题的核心设计思想，答题时写出这一步变换，比只贴代码更容易得满分。同理，循环右移 p 位只需把三段逆置的顺序调整为「整体逆置 $\to$ 前 p 个逆置 $\to$ 后 $n-p$ 个逆置」即可。

循环左移题的常见失分

一是忘记处理 $p\ge n$ 的情形（先取模 `p = p % n`）；二是把三次逆置的边界写错（`Reverse` 通常接收下标区间，注意是左闭右闭还是左闭右开）；三是空间超标——额外开一个长度 n 的辅助数组搬移虽然结果正确，但空间是 $O(n)$，不满足题目「时空两方面都尽可能高效」的要求。三次逆置法时间 $O(n)$（每个元素被访问常数次），空间 $O(1)$，是最优解。

为什么三次逆置能得到循环左移？设原序列为 XY，其中 X 是前 p 个元素、Y 是后 $n-p$ 个元素，目标是 YX。记 X' 表示 X 的逆序，则：

$$(X'Y')'=YX$$

即「先分别把两段各自逆置，再把整体逆置」，就能把 XY 换成 YX。这是本题的核心设计思想，答题时写出这一步变换，比只贴代码更容易得满分。同理，循环右移 p 位只需把三段逆置的顺序调整为「整体逆置 → 前 p 个逆置 → 后 $n-p$ 个逆置」即可。

循环左移题的常见失分

一是忘记处理 $p \geq n$ 的情形（先取模 $p = p \bmod n$）；二是把三次逆置的边界写错（Reverse 函数通常接收下标区间，注意是左闭右闭还是左闭右开）；三是空间超标——如果额外开一个长度 n 的辅助数组搬移，虽然结果正确但空间是 $O(n)$，不满足题目「时空两方面都尽可能高效」的要求。三次逆置法的时间是 $O(n)$（每个元素被访问常数次），空间 $O(1)$，是最优解。

（四）快慢指针：找中间结点

快指针一次走两步，慢指针一次走一步；快指针到表尾时，慢指针正好在中间。

```
// 带头结点单链表求中间结点
// 若结点个数为偶数，本模板返回「下中位」（靠后的那个中间结点）
LNode *MiddleNode(LinkList L) {
    LNode *slow = L, *fast = L;      // 都从头结点出发，便于偶数时返回下中位
    while (fast != NULL && fast->next != NULL) {
        slow = slow->next;
        fast = fast->next->next;
    }
    return slow;
}
```

中间结点的定义要看清题目

- 快慢指针都从头结点出发，n 为偶数时返回的是下中位（第 $n/2+1$ 个数据结点）。
- 快慢指针都从第一个数据结点出发，n 为偶数时返回的是上中位（第 $n/2$ 个数据结点）。

2019 年重排链表题明确要求分界点是下中位数 $\lceil n/2 \rceil$ 个结点，所以要采用第一种起点。题目怎么写，代码就怎么起步，不要凭印象。

（五）双指针：倒数第 k 个结点（**2009** 年真题）

一次遍历就能完成：设前后两个指针 p（前）和 q（后），先让 p 从头结点出发走 k 步，然后 p、q 同时后移，当 p 到达表尾（NULL）时，q 所指就是倒数第 k 个结点。

```
// 查找带头结点单链表中倒数第 k 个结点，找到则输出其数据并返回 1，否则返回 0
int SearchKthFromEnd(LinkList L, int k) {
    LNode *p = L->next, *q = L->next;
    int i = 0;
    while (p != NULL && i < k) { p = p->next; i++; }  // p 先走 k 步
    if (p == NULL && i < k) return 0;                 // 表长不足 k，查找失败
    while (p != NULL) { p = p->next; q = q->next; }   // 同步后移
    printf("%d\n", q->data);
    return 1;
}
```

时间复杂度 $O(n)$（只扫一遍），空间复杂度 $O(1)$。

边界：表长不足 k

第一个 while 结束时若 i < k，说明表长小于 k，必须返回 0。很多答案漏掉这个判断，在 p 已经为 NULL

时还接着做同步后移，逻辑上就错了。另一个写法是让 p 从头结点出发走 k 步、q 从头结点出发，最后 q->next 是倒数第 k 个——两种写法等价，但要保证前后指针的起始位置与步数匹配。

（六）判断链表是否有环，并求环的入口

判环（快慢指针相遇法）：快指针每次走两步、慢指针每次走一步。若链表无环，快指针会先到 NULL 退出；若有环，快指针必然追上慢指针。

```
// 判断带头结点单链表中是否有环
bool HasCycle(LinkList L) {
    LNode *slow = L, *fast = L;
    while (fast != NULL && fast->next != NULL) {
        slow = slow->next;
        fast = fast->next->next;
        if (slow == fast) return true;   // 相遇即有环
    }
    return false;                        // fast 走到 NULL，无环
}
```

为什么快慢指针一定会相遇？两个指针都进环后，每移动一轮，快指针与慢指针的距离就减少 **1**（快走 2、慢走 1）。这相当于操场上的追及问题，只要一直走，快指针一定追上慢指针。

为什么慢指针在环内走不完一圈就被追上？考虑最坏情况：慢指针刚进环时，快指针正好在它前面一个位置，此时两者相距「环长 -1」，因为每轮距离减少 1，所以最多经过「环长 -1」轮就相遇；此时慢指针在环内走了「环长 -1」步，不到一圈。其他情况比这更快。

求环的入口（第一次相遇后再用双指针）：设头结点到环入口的距离为 a，环入口到第一次相遇点的距离为 b，环长为 L。相遇时慢指针走了 $a+b$，快指针走了 $a+b+kL$，而快指针路程是慢指针的两倍，于是

$$2(a+b) = a+b+kL \implies a+b = kL \implies a = kL-b$$

即「从头结点到入口的距离 a」等于「从相遇点继续沿环走到入口的距离 $kL-b$」。所以：相遇后，让一个指针从头结点出发、另一个从相遇点出发，二者同速前进，再次相遇的位置就是环的入口。

```
// 返回环的入口结点；无环返回 NULL
LNode *DetectCycle(LinkList L) {
    LNode *slow = L, *fast = L;
    while (fast != NULL && fast->next != NULL) {
        slow = slow->next;
        fast = fast->next->next;
        if (slow == fast) {              // 第一次相遇
            LNode *p = L;
            while (p != slow) {          // 同速前进，再次相遇即入口
                p = p->next;
                slow = slow->next;
            }
            return slow;
        }
    }
    return NULL;
}
```

求环长：从第一次相遇点出发，沿环走一圈回到该点，走过的步数就是环长，时间 $O(n)$、空间 $O(1)$。

（七）判断两个链表是否相交（2012 年真题）

判定依据（必背）：两个链表如果相交，从交点之后它们共享同一段存储，因此两个链表的尾结点必然相同。注意比较的是结点地址（指针），不是数据域的值！数据域相等只是「值相同」，不代表同一个结点。

相交题的经典陷阱

看到 `p->data == q->data` 就断言相交是错的。两个链表可以有相同的值而完全不共享结点。判断相交一定要比 `p == q`（指针相等）。另外要注意：若链表带环，情况会更复杂，统考题干一般会明确「整个链式结构中不存在环」。

方法一（长度差法）：分别求两个链表长度 len_1, len_2，让较长的那个指针先走 $|len_1 - len_2|$ 步，使两个指针到表尾的距离相等，然后同步后移，第一次指针相同的位置就是交点。

```
// 带头结点，求两个单链表的第一个公共结点
LNode *FindFirstCommon(LinkList A, LinkList B) {
    int lenA = 0, lenB = 0;
    LNode *p = A->next, *q = B->next;
    while (p != NULL) { lenA++; p = p->next; }
    while (q != NULL) { lenB++; q = q->next; }
    p = A->next; q = B->next;
    int d = lenA - lenB;
    while (d > 0) { p = p->next; d--; }     // A 长，p 先走
    while (d < 0) { q = q->next; d++; }     // B 长，q 先走
    while (p != q && p != NULL && q != NULL) { p = p->next; q = q->next; }
    return p;                               // 相交返回交点，不相交返回 NULL
}
```

方法二（路程补偿，代码更短）：两个指针同时同速前进，走到 NULL 时切换到对方的起点继续走。设公共段长度为 c，则两指针总路程都是 $a+c+b+c$ 与 $b+c+a+c$，相等，因此必然在交点相遇；若不相交，则会在 NULL 处相遇。

```
LNode *GetIntersectionNode(LinkList A, LinkList B) {
    LNode *p = A->next, *q = B->next;
    while (p != q) {
        p = (p == NULL) ? B->next : p->next;   // 走到头就换到对方起点
        q = (q == NULL) ? A->next : q->next;
    }
    return p;   // 不相交时 p、q 会同时为 NULL，返回 NULL
}
```

两种方法都是时间 $O(m+n)$、空间 $O(1)$。方法二是 **2012** 年真题的主流解法，但方法一更符合「先说清设计思想」的答题习惯，建议两种都掌握。

2012 年真题的题干是「两个单词共享相同后缀」，本质就是「两个链表从某个结点起共享同一段存储」，因此求「共同后缀的起始位置」与求「第一个公共结点」完全等价。答题时的设计思想可以这样写：先分别求出两个链表的长度，算出长度差 d；让较长链表的指针先走 d 步，使两个指针到表尾的距离相等；然后两个指针同步后移，第一次指向同一结点时即交点。这段文字就是第一小问的完整答案，务必能默写。

长度差法的边界

如果两个链表根本不相交，那么同步后移时两个指针会同时变成 NULL，此时 `p == q` 成立，`while` 循环退出并返回 NULL——这恰好是正确的返回值，所以不需要额外判断。但循环条件必须写成 `p != q &&`

`p != NULL && q != NULL` 或者只写 `p != q`（依赖「同时为 NULL 时相等」这一性质）。若写成 `while (p != NULL && q != NULL && p != q)`，退出后还要再判断一次 `p == q ? p : NULL`，更啰嗦。理解「同时为 **NULL** 也算相遇」这一点，是本题最容易出彩的地方。

（八）删除重复元素

有序顺序表删重复（双指针）：因为有序，重复元素一定相邻。用快指针 `fast` 扫描，慢指针 `slow` 指向「下一个不同元素应写入的位置」。

```
// 从有序顺序表中删除重复元素，返回处理后的表长
int DeleteDuplicates(SqList *L) {
    if (L->length == 0) return 0;
    int slow = 1;                              // data[0] 必然保留
    for (int fast = 1; fast < L->length; fast++) {
        if (L->data[fast] != L->data[fast - 1]) {  // 与前一个不同，是新值
            L->data[slow] = L->data[fast];
            slow++;
        }
    }
    L->length = slow;                          // 不要忘记更新表长
    return slow;
}
```

无序表删除所有值为 x 的元素（双指针）：同样用快慢指针，把不等于 x 的元素往前填，时间 $O(n)$、空间 $O(1)$。如果题目要求「删除值在 s 到 t 之间的所有元素」，只需把判断条件改成 `if (L->data[i] < s || L->data[i] > t)`；若 $s > t$ 或表空则直接返回（判空和参数合法性是常被漏掉的得分点）。

有序表删区间 $[s,t]$（两端定位法）：先找到第一个 $\geq s$ 的元素位置 i 和最后一个 $\leq t$ 的元素位置 j，若 $i \leq j$ 则把 j 之后的所有元素整体前移 $j-i+1$ 位，表长减小 $j-i+1$。因为有序，$[s,t]$ 内的元素必然连续，这个方法比逐个比较更快（仍然是 $O(n)$，但只扫一遍找到边界就整体搬移）。

链表删除重复结点：对有序链表，只需比较相邻结点：

```
// 带头结点的有序单链表删除重复元素，每个值只保留一个
void DeleteSame(LinkList L) {
    LNode *p = L->next, *q;
    while (p != NULL && p->next != NULL) {
        q = p->next;
        if (q->data == p->data) {   // 与后继重复
            p->next = q->next;
            free(q);
        } else {
            p = p->next;
        }
    }
}
```

对无序链表删重复，若不允许额外空间，则用二重循环（$O(n^2)$）；若允许空间，2015 年真题的做法是借助一个标记数组——因为 `|data| ≤ n`，可以开一个长度 $n+1$ 的数组 `flag[]`，扫描时把 `|data|` 作为下标打标记，遇到已标记的结点就删除，这样时间 $O(m)$、空间 $O(n)$，正是题目「时间复杂度尽可能高效」的意图。这类题一定要结合题目给的取值范围来设计，题干里的 `|data| ≤ n` 就是给你的提示。

（九）其他常考算法

删除单链表中值最小的结点（假设存在且唯一）：用一个指针 `minPre` 记住最小值结点的直接前驱，一趟扫描后直接删除：

```
void DeleteMin(LinkList L) {
    if (L->next == NULL) return;
    LNode *minPre = L, *p = L->next;
    while (p->next != NULL) {
        if (p->next->data < minPre->next->data) minPre = p;
        p = p->next;
    }
    LNode *q = minPre->next;
    minPre->next = q->next;
    free(q);
}
```

若要「按递增次序输出各结点的值并释放结点空间」，只需在外面套一层 `while (L->next != NULL)`，每次找出最小值输出并删除。两层扫描，时间 $O(n^2)$、空间 $O(1)$。（若允许 $O(n)$ 辅助空间，可以先存入数组排序，时间 $O(n\log n)$——考试时要看题目对时空的要求择优。）

拆分链表：把一个线性表 $C=(a_1,b_1,a_2,b_2,\cdots)$ 拆成 $A=(a_1,a_2,\cdots)$、$B=(b_n,\cdots,b_2,b_1)$ 两个表。做法是：奇数位置结点用尾插法接到 A（保持相对次序），偶数位置结点用头插法接到 B（实现逆序）。这正是「头插法逆序、尾插法保序」这一特性的直接应用。

重排链表（**2019** 年真题）：把 $L=(a_1,a_2,\cdots,a_n)$ 重排为 $(a_1,a_n,a_2,a_{n-1},\cdots)$，要求空间 $O(1)$，是三个基本功的串联：

1. 用快慢指针找到中间结点（本题取下中位 $\lceil n/2\rceil$），并把链表从中间断开；
2. 把后半段就地逆置；
3. 把前半段与逆置后的后半段交叉合并。

```
// 带头结点单链表重排为先前后尾交替
void ReorderList(LinkList L) {
    if (L->next == NULL) return;
    // 第一步：找下中位结点
    LNode *slow = L, *fast = L;
    while (fast != NULL && fast->next != NULL) {
        slow = slow->next;
        fast = fast->next->next;
    }
    LNode *head2 = slow->next;   // 后半段的第一个结点
    slow->next = NULL;           // 从中间断开
    // 第二步：把后半段就地逆置
    LNode *pre = NULL, *cur = head2;
    while (cur != NULL) {
        LNode *nxt = cur->next;
        cur->next = pre;
        pre = cur;
        cur = nxt;
    }
    head2 = pre;
    // 第三步：交叉合并
    LNode *p1 = L->next, *p2 = head2;
    while (p1 != NULL && p2 != NULL) {
```

```
        LNode *n1 = p1->next, *n2 = p2->next;
        p1->next = p2;
        p2->next = n1;
        p1 = n1;
        p2 = n2;
    }
}
```

重排链表的复杂度

三步各自都是 $O(n)$，总时间复杂度 $O(n)$，空间复杂度 $O(1)$（只用了常数个指针），完全满足 2019 年真题「空间复杂度为 $O(1)$ 且时间上尽可能高效」的要求。暴力解法（把结点存入数组再按下标重建）虽然也是 $O(n)$ 时间，但空间是 $O(n)$，不满足题目要求，会被扣分。

其他常见小题型（知道思路即可）：

- 数组主元素（**2013** 年真题）：摩尔投票法，$O(n)$、$O(1)$。先假设当前元素为候选、计票数为 1，遇见相同加一、不同减一，计票归零则换候选；最后再扫一遍验证候选出现次数是否真的 $> n/2$。验证这一步不能省。
- 未出现的最小正整数（**2018** 年真题）：利用「n 个元素，未出现的最小正整数必然在 $1 \sim n+1$ 之间」这一结论，用原地交换（或辅助标记数组）把值域内的元素放到对应下标处，再从头扫描找第一个不满足 `A[i] == i+1` 的位置；时间 $O(n)$、空间 $O(1)$。
- 两个等长升序序列的中位数（**2011** 年真题）：用类二分的思路，每次同时比较两个序列的中位数，取半区间，逐步把规模缩小一半，时间 $O(\log n)$、空间 $O(1)$。
- 三元组最小距离（**2020** 年真题）：三个有序数组各设一个指针，每次计算当前三元组距离并更新最小值，然后把三个指针中指向最小元素的那个前移一位，循环直到某个数组走到末尾；时间 $O(n_1+n_2+n_3)$。

关于 2011 年与 2020 年这两道题，值得多写几句设计思想，因为它们的「套路感」很强。

2011 年求两等长升序序列的中位数，设计思想可以这样表述：设两个序列为 A、B，长度均为 L。分别取 A 的中位数 a 和 B 的中位数 b（第 $\lceil L/2 \rceil$ 个元素）。若 $a=b$，则 a 就是所求；若 $a<b$，则所求中位数一定不在 A 的前半段（可以舍弃 A 的前 $\lfloor L/2 \rfloor$ 个）和 B 的后半段（舍弃 B 的后 $\lfloor L/2 \rfloor$ 个）；若 $a>b$ 则对称地舍弃 A 的后半段与 B 的前半段。每次把规模缩小一半，且两个序列始终等长，直到两个序列各只剩一个元素，较小者即为所求。因为规模每次减半，时间 $O(\log_2 n)$，只用了常数个变量，空间 $O(1)$。这类「取中位数、比大小、抛掉不可能的一半」的写法，本质与折半查找同源。

2020 年求三元组最小距离，设计思想是：$D=|a-b|+|b-c|+|c-a|$ 这个式子看起来有三个绝对值，但若设 $a \le b \le c$，则 $D=2(c-a)$——距离只由最大者和最小者的差决定。因此要让 D 变小，唯一的办法是把「当前最小的那个元素」换成它所在集合中的下一个更大的元素，从而抬高最小值、缩小极差。于是算法是：三个指针分别指向三个有序数组的第一个元素，循环计算当前 D 并更新答案，然后把三个指针中指向最小值的那个指针向后移一位（若并列最小，移动任意一个即可，但通常约定移动其中一个并继续），直到某个指针越界。每个指针最多走完自己那个数组，总时间 $O(n_1+n_2+n_3)$，空间 $O(1)$。注意题目里的排序条件是「按升序分别存储」，正是它保证了「往后移只会让最小值变大」这一单调性，去掉有序条件这个方法就不成立了。

这两个例子说明同一件事：统考算法题的最优解往往来自对题目给定条件的充分利用（有序、等长、值域范围），而不是来自高级技巧。审题时把每个条件都圈出来，是最有效的解题起点。

（十）补充：无序表先排序再处理的思路

线性表的算法题里有一类「看起来没有下手处」的题目：给一个无序数组，要求找某种满足大小关系的结果。这时一条通用的破题思路是——先用 $O(n\log n)$ 把它变成有序表，再对有序表用双指针等线性手法，总时间仍然是 $O(n\log n)$，通常就是这类题的标准答案。

典型例子是「找出数组中未出现的最小正整数」（2018 年真题）。如果直接扫描，很难判断「哪个正整数没出现」；但一旦想到「n 个元素里未出现的最小正整数一定不超过 $n+1$」，就可以用值域做下标打标记，一趟扫描解决问题。再比如「求两个数组的交集」「找出重复出现的元素」「求数组中两数之差的最小绝对值」，排序后相邻比较或双指针一遍即得。

注意事项有两条。第一，**408** 考试不允许直接调用 `sort` 等库函数，需要自己写出排序。建议把快速排序模板背下来备用：

```
// 快速排序：对 q[l..r] 就地升序排序，平均 O(nlogn)，空间 O(logn)
void QuickSort(int q[], int l, int r) {
    if (l >= r) return;                  // 递归终止：区间内元素不超过 1 个
    int i = l - 1, j = r + 1, x = q[l + r >> 1];  // x 取中间元素作分界点
    while (i < j) {
        do i++; while (q[i] < x);        // i 之前的元素都 <= x
        do j--; while (q[j] > x);        // j 之后的元素都 >= x
        if (i < j) { int t = q[i]; q[i] = q[j]; q[j] = t; }
    }
    QuickSort(q, l, j);
    QuickSort(q, j + 1, r);
}
```

第二，排序会改变元素的相对位置。如果题目要求「保持原有次序」，就不能先排序，只能改用双指针一趟扫描或其他不改变次序的方法。审题时务必看清这一点。

```
// 折半查找：在有序数组 A[low..high] 中查找 x，返回下标，失败返回 -1
int BinarySearch(int A[], int low, int high, int x) {
    while (low <= high) {                    // 左闭右闭区间
        int mid = low + (high - low) / 2;  // 防溢出写法，等价于 (low+high)/2
        if (A[mid] > x)      high = mid - 1;
        else if (A[mid] < x) low = mid + 1;
        else                 return mid;
    }
    return -1;
}
```

折半查找每次把查找区间缩小一半，最多比较 $\lceil\log_2(n+1)\rceil$ 次，时间 $O(\log_2 n)$、空间 $O(1)$。但再次强调：它要求「有序」且「顺序存储」——链表无法在 $O(1)$ 时间内取到中间元素，因此不能在链表上做折半查找。这个结论是「顺序表与链表对比」在查找问题上的延伸，常与本章一起出题。

> **「先排序再处理」的两个前提**
>
> 一是题目没有要求保持原有次序（否则排序就破坏了要求）；二是排序的代价可以接受（若题目要求 $O(n)$ 时间，那排序的 $O(n\log n)$ 就超标了，此时要另想办法，例如用哈希 / 标记数组把时间压到 $O(n)$）。这两点判断错，方向就全错了。

本章小结表

知识点	核心结论
线性表定义	同类型、有限、有序；除首尾外一前一后
位序 vs 下标	位序从 1 开始，下标从 0 开始；插入范围 $1 \sim n+1$，删除范围 $1 \sim n$
顺序表地址	$LOC(a_i) = LOC(a_1) + (i-1) \times d$，随机访问
插入平均移动	$\frac{n}{2}$ 次，时间复杂度 $O(n)$
删除平均移动	$\frac{n-1}{2}$ 次，时间复杂度 $O(n)$
按值查找（顺序）	平均比较 $\frac{n+1}{2}$ 次，$O(n)$
链表按位查找	平均比较 $\frac{n+1}{2}$ 次，$O(n)$，顺序存取
头结点作用	统一空表/非空表、统一首位置插删、头指针恒非空
头插法	顺序反转，用于逆置、升序合并成降序
尾插法	保持相对顺序，需尾指针，$O(n)$
后插口诀	先连后（`s->next=p->next`）再连前（`p->next=s`）
双链表后插	四句顺序：`s->next`、`p->next->prior`、`s->prior`、`p->next`
循环单链表判空	`head->next == head`
循环双链表判空	`head->next == head`（且 `head->prior == head`）
静态链表	数组 + 游标；`next==-1` 表结束；不能随机存取、容量固定
顺序表 vs 链表	顺序表随机存取、密度高；链表插删灵活、无需连续空间
找倒数第 k 个	前后双指针，前指针先走 k 步，一次遍历 $O(n)$
找中间结点	快慢指针，快走 2 慢走 1；起点不同结果不同（上/下中位）
判断环	快慢指针相遇即有环，$O(n)$、$O(1)$
求环入口	相遇后一个从头、一个从相遇点同速走，再次相遇即入口
判断相交	尾结点相同（比地址不比数据）；$O(m+n)$、$O(1)$
就地逆置	迭代法三句：存后继、反转指针、双指针后移
2010 循环左移	三次逆置，$O(n)$、$O(1)$
2019 重排链表	找中点 + 逆置后半 + 交叉合并，$O(n)$、$O(1)$

易错清单

本章最容易被扣分的地方

1. 把「线性表」与「顺序表」混为一谈。线性表是逻辑结构，顺序表和链表是存储结构。判断题说「线性表只能顺序存储」是错的。
2. 位序与下标混用。`L.data[i-1]` 对应位序 i；插入合法范围是 $1 \le i \le n+1$，删除是 $1 \le i \le n$。写代码时越界判断的两个条件是高频扣分点。
3. 插入移动次数记成 $\frac{n-1}{2}$、删除记成 $\frac{n}{2}$。务必用记忆锚点：插入有 $n+1$ 个位置（含表尾后），删除只有 n 个位置，所以插入的分母大、平均移动多（$n/2$），删除的平均移动少（$(n-1)/2$）。
4. 把「链表插入删除是 $O(1)$」绝对化。只有「已拿到结点指针」时才是 $O(1)$；给位序就要 $O(n)$ 找位置。
5. 后插两句顺序颠倒。先写 `p->next = s` 会丢失原来的后继地址，链表断开。
6. 双链表插入时语句顺序错乱。核心规则：读 `p->next` 的语句必须写在改 `p->next` 之前。
7. 用「删自身」技巧时忘了判尾结点。尾结点的 `p->next == NULL`，会解引用空指针；必须 `if (p->next != NULL)`。
8. 循环链表删除时忘记维护环和尾指针。链表只剩一个数据结点时删掉它，尾指针必须重新指向头结点

（2021-01、2016-02 反复考这一点）。

9. 判断相交时比较了 `data` 而不是指针。数据相等不等于共享结点。
10. 中间结点的上中位 / 下中位搞反。快慢指针起点决定返回哪一个，必须按题目要求选起点。
11. 快慢指针循环条件写错。判环、找中点时必须写 `while (fast != NULL && fast->next != NULL)`，只判 `fast` 会在 `fast->next->next` 处解引用空指针。
12. 静态链表误以为可以随机存取。它虽然用数组，但逻辑顺序由游标决定，只能从头顺着游标找，是顺序存取。
13. 算法题漏写边界与复杂度。空表、单结点、表长不足 k、参数 $s > t$ 等情形要显式判断；三小问（设计思想、代码、复杂度）要分别作答，代码里要有注释。
14. 题目要求 $O(1)$ 空间却用了数组或递归。递归调用栈算空间（递归法反转链表是 $O(n)$），辅助数组也是空间；不满足时空要求即使结果正确也会失分。

本章历年统考考点分布（按知识点）

- 顺序表：2010 循环左移（算法题）、2011 两序列中位数（算法题）、2013 主元素（算法题）、2018 未出现的最小正整数（算法题）、2020 三元组最小距离（算法题）、2023-01 顺序表操作的平均时间复杂度（选择）、2026-01 哪些操作必然产生元素移动（选择）。
- 链表基本操作与静态链表：2016-01 静态链表插入后的链接地址（选择）、2016-02 双向循环链表删除结点语句（选择）、2021-01 单循环链表删除第一个元素（选择）、2023-02 非空双向链表插入结点（选择）、2024-01 语句序列的作用（选择）、2026-02 双向链表指针调整（选择）。
- 链表算法题：2009 查找倒数第 k 个结点、2012 求两个链表的共同后缀起始位置（相交）、2015 删除绝对值相等的重复结点、2019 重排链表。
- 综合对比：2013 年应用题考过顺序存储结构与链式存储结构的 ASL 比较。

算法设计题复杂度速查表

把本章所有可能被考的算法及其时空代价集中列在这里，考前扫一眼即可。

算法	时间复杂度	空间复杂度	核心手法
顺序表按位查找	$O(1)$	$O(1)$	地址计算
顺序表按值查找	$O(n)$	$O(1)$	顺序扫描
顺序表插入 / 删除	$O(n)$	$O(1)$	搬移元素
顺序表折半查找（有序）	$O(\log_2 n)$	$O(1)$	区间减半
单链表按位 / 按值查找	$O(n)$	$O(1)$	顺链扫描
单链表求表长	$O(n)$	$O(1)$	顺链扫描
头插法 / 尾插法建表	$O(n)$	$O(1)$（不计结点空间）	头插 / 尾指针
合并两个有序表（顺序或链式）	$O(n+m)$	$O(1)$	双指针归并
升序合并成降序链表	$O(n+m)$	$O(1)$	双指针 + 头插
就地逆置（数组 / 链表）	$O(n)$	$O(1)$	首尾交换 / 双指针反转
数组循环左移 p 位	$O(n)$	$O(1)$	三次逆置
找中间结点（快慢指针）	$O(n)$	$O(1)$	快走 2 慢走 1
找倒数第 k 个结点	$O(n)$	$O(1)$	前后双指针
判断链表是否有环	$O(n)$	$O(1)$	快慢指针相遇
求环的入口 / 环长	$O(n)$	$O(1)$	相遇后同速走 / 绕环一圈

算法	时间复杂度	空间复杂度	核心手法
判断两链表是否相交（长度差法）	$O(m+n)$	$O(1)$	对齐起点后同步走
求第一个公共结点（路程补偿）	$O(m+n)$	$O(1)$	走到头切换到对方起点
有序表删除重复元素（双指针）	$O(n)$	$O(1)$	快慢指针原地填充
无序表删除值为 x 的元素	$O(n)$	$O(1)$	快慢指针原地填充
无序链表删重复（借助标记数组）	$O(n)$	$O(n)$	值域作下标打标记
升序输出并释放链表结点	$O(n^2)$	$O(1)$	每趟找最小值
重排链表（2019）	$O(n)$	$O(1)$	中点 + 逆置 + 交叉合并
数组循环左移（2010）	$O(n)$	$O(1)$	三次逆置
求两等长升序序列中位数（2011）	$O(\log_2 n)$	$O(1)$	类二分减半
求数组主元素（2013）	$O(n)$	$O(1)$	摩尔投票 + 验证
找未出现的最小正整数（2018）	$O(n)$	$O(1)$	值域作下标原地交换
三元组最小距离（2020）	$O(n_1+n_2+n_3)$	$O(1)$	三指针移动最小值

记忆要点

线性表算法题的复杂度几乎是「非 $O(n)$ 即 $O(\log n)$」，空间几乎都是 $O(1)$。出现 $O(n^2)$ 通常意味着暴力解（如「找出最小值并删除」反复执行），只有在题目没有更好要求或明确允许时才用；出现 $O(n)$ 空间通常是因为借助了辅助数组或递归，只有题目不限制空间时才用。考前把表中「$O(1)$ 空间」的那几栏重点记住即可。

第三章栈、队列和数组

本章考纲要求

> 三、栈、队列和数组（一）栈和队列的基本概念（二）栈和队列的顺序存储结构（三）栈和队列的链式存储结构（四）多维数组的存储（五）特殊矩阵的压缩存储（六）栈、队列和数组的应用

本章是数据结构的「计算题富矿」：栈与队列的出入操作序列、循环队列的判空判满、矩阵压缩存储的下标换算，三类题目几乎每年必考，且都能用固定套路在 1 分钟内算出答案。复习时要把每一条公式都自己代入一组数验证一遍，不要只背结论。

一、栈和队列的基本概念

（一）栈的定义与特点

栈（**Stack**）是只允许在一端进行插入和删除操作的线性表。允许插入和删除的一端称为栈顶（**top**），另一端称为栈底（**bottom**）。不含任何元素的栈称为空栈。

栈的插入操作称为入栈（**push**），删除操作称为出栈（**pop**）。由于插入和删除都被限制在栈顶，最后入栈的元素必定最先出栈，因此栈是后进先出（**Last In First Out**，**LIFO**）结构。

需要特别注意：栈是操作受限的线性表，这里的「受限」指「只允许在一端操作」，而不是「两端都能操作」。栈的逻辑结构与普通线性表完全相同，仍然是「一对一」的线性关系，改变的只是允许操作的端点。

易错点

「栈允许在其两端进行操作」是错误说法。栈只在栈顶一端插入和删除；支持两端插入删除的是双端队列（**deque**）。栈底是固定不动的一端，不能直接对栈底元素操作，想访问栈底元素必须先把上面所有元素弹出。

（二）队列的定义与特点

队列（**Queue**）是只允许在一端插入、在另一端删除的线性表。允许插入的一端称为队尾（**rear**），允许删除的一端称为队头（**front**）。不含任何元素的队列称为空队列。

队列的插入操作称为入队（**enqueue**），删除操作称为出队（**dequeue**）。先入队的元素必定先出队，因此队列是先进先出（**First In First Out**，**FIFO**）结构。

（三）栈与队列的对比

比较项	栈 **Stack**	队列 **Queue**
操作端点	只在一端（栈顶）插入和删除	一端（队尾）插入，另一端（队头）删除
存取特性	后进先出 LIFO	先进先出 FIFO
逻辑结构	线性表	线性表
存储实现	顺序栈、链栈	顺序队列（含循环队列）、链队列

比较项	栈 **Stack**	队列 **Queue**
典型应用	括号匹配、表达式求值、递归、进制转换、DFS	层次遍历、缓冲区、CPU 调度、BFS

（四）上溢与下溢

栈和队列在采用顺序存储时容量固定，因此存在两种异常状态：

- 上溢（**overflow**）：栈满（或队满）时仍执行入栈（入队）操作。对栈来说，满了还往里写就是上溢。
- 下溢（**underflow**）：栈空（或队空）时仍执行出栈（出队）操作。空了还往外读就是下溢。

顺序栈的上溢是真上溢，只要数组空间确实不够用就必然发生。而顺序队列的上溢可能是假上溢：由于出队只把 `front` 后移而不移动元素，数组前面的空位无法再利用，此时虽然数组内还有空位，`rear` 却已到达末尾，再入队就报「满」。循环队列正是为解决假上溢而提出的。

链式存储的栈和队列不存在上溢问题（只要内存还够就能申请新结点），但出队或出栈时仍要判空，否则会产生下溢。

易错点

「上溢」「下溢」这两个词只用于顺序存储的讨论语境。链栈、链队列不需要判断「满」，因为结点是动态申请的，所以链式存储不存在上溢。

（五）共享栈

利用栈底位置相对不变的特性，可以让两个顺序栈共享同一个一维数组空间：两个栈的栈底分别设在数组两端，两个栈顶向中间延伸。

设共享数组为 `data[0..MaxSize-1]`，用 `top0` 表示 0 号栈的栈顶，`top1` 表示 1 号栈的栈顶。采用「栈顶指针指向当前栈顶元素」的约定时：

- 初始状态：`top0 = -1`，`top1 = MaxSize`（两个栈都为空）；
- 0 号栈入栈：`data[++top0] = x`；1 号栈入栈：`data[--top1] = x`；
- 判满条件：`top0 + 1 == top1`。

共享栈的核心结论

两个栈的栈顶指针相邻时（`top0 + 1 == top1`），整个数组被占满，此时才会发生上溢。

共享栈的意义是只有在整个存储空间被占满时才上溢，让两个栈的空间相互调节。存取数据的时间复杂度仍为 $O(1)$，对存取效率没有影响。

注意区分：共享栈判满用 `top0 + 1 == top1`（全部空间可用）；循环队列牺牲一个单元判满用 `(rear+1)\%M == front`（只存 **M-1** 个）。两者形式相似，含义完全相反，考场上极易混淆。

二、栈和队列的顺序存储结构

（一）顺序栈与 top 指针的两种约定

顺序栈用一组地址连续的存储单元依次存放栈底到栈顶的元素。`top` 指针的含义有两种约定，两种都要掌握，题目给出哪种就用哪种。

比较项	约定 **A**：**top** 指向栈顶元素	约定 **B**：**top** 指向栈顶元素的下一个位置
初始化	`S.top = -1`	`S.top = 0`
判空	`S.top == -1`	`S.top == 0`
判满	`S.top == MaxSize - 1`	`S.top == MaxSize`
入栈	`S.data[++S.top] = x`（先加后存）	`S.data[S.top++] = x`（先存后加）
出栈	`x = S.data[S.top--]`	`x = S.data[--S.top]`
取栈顶	`S.data[S.top]`	`S.data[S.top - 1]`
栈中元素个数	`S.top + 1`	`S.top`

代入一组数验证约定 B（`MaxSize = 3`）：初始 `top = 0` 为空；依次入栈 A、B、C 后，写入位置为 0、1、2，`top = 3`，此时 `top == MaxSize` 判满，元素个数为 `top = 3`，正确。再出栈一次，`--top` 得 2，栈顶元素是 `data[2] = C`，正确。

易错点

考场上最容易错的不是公式本身，而是入栈时先自增还是先存值。判断方法：看 `top` 的含义。若 `top` 指向栈顶元素，则新元素必须放在 `top+1`，所以先自增再存值；若 `top` 指向空位（栈顶下一位置），则新元素直接放在 `top`，所以先存值再自增。题目里若出现 `top == 0` 表示空栈，说明采用的是约定 B。

顺序栈的完整实现（采用约定 A，`top` 指向栈顶元素）：

```
#define MaxSize 50
typedef struct {
    int data[MaxSize];   // 存放栈中元素
    int top;             // 栈顶指针，指向栈顶元素
} SqStack;

void InitStack(SqStack &S) { S.top = -1; }              // 初始化：空栈

bool StackEmpty(SqStack S) { return S.top == -1; }      // 判空

bool StackFull(SqStack S) { return S.top == MaxSize - 1; }   // 判满

bool Push(SqStack &S, int x) {                          // 入栈
    if (S.top == MaxSize - 1) return false;             // 栈满，上溢
    S.data[++S.top] = x;                                // 先加后存
    return true;
}

bool Pop(SqStack &S, int &x) {                          // 出栈
    if (S.top == -1) return false;                      // 栈空，下溢
    x = S.data[S.top--];                                // 先取后减
    return true;
}

bool GetTop(SqStack S, int &x) {                        // 读栈顶，不修改栈
    if (S.top == -1) return false;
    x = S.data[S.top];
    return true;
}
```

（二）顺序队列的 front / rear 约定

顺序队列用数组 data[0..MaxSize-1] 存放元素，用 front 和 rear 两个指针指示队头与队尾。采用「**front** 指向队头元素、**rear** 指向队尾元素的下一个位置」的约定时：

- 初始化：Q.front = Q.rear = 0;
- 判空：Q.front == Q.rear;
- 入队：Q.data[Q.rear] = x; Q.rear++;（**front** 不动，**rear** 变化）;
- 出队：x = Q.data[Q.front]; Q.front++;（**rear** 不动，**front** 变化）;
- 元素所在区间：[front, rear - 1]。

一句话记住谁动

入队时 **front** 指针不会变，变化的是 **rear** 指针；出队时 **rear** 指针不会变，变化的是 **front** 指针。选择题里经常用这句话直接排除错误选项。

这种朴素顺序队列的问题在于：不断入队出队后 front 和 rear 都单调右移，rear 到达数组末尾就无法再入队，即便数组前面还有大量空位——这就是假上溢。解决办法是把数组在逻辑上首尾相接，即循环队列。

（三）循环队列：三种判满方案

循环队列中所有指针移动都改为 (x + 1) % MaxSize。此时的困难在于：队空时 **front == rear**，队满时 **front** 也可能等于 **rear**，两种情况无法区分，必须额外增加信息。

*1.*方案一：牺牲一个存储单元

规定队列最多存放 MaxSize - 1 个元素，即数组中永远保留一个空位，用它隔开「满」和「空」两种状态。

约定：front 指向队头元素，rear 指向队尾元素的后一个位置。

- 判空：front == rear;
- 判满：(rear + 1) % MaxSize == front;
- 元素个数：(rear - front + MaxSize) % MaxSize。

推导：队满时，若再入队一个元素，新的 rear 会变成 (rear+1)%MaxSize，此时它与 front 重合，队列就再也分不清是空还是满了。因此我们在「下一个 rear 即将等于 front」时就判定为满，把那个位置空出来不用。这就是「牺牲一个单元」的由来。

代入验证 (MaxSize = 5): 初始 front = rear = 0，判空成立。依次入队 10、20、30，则 data[0..2] = {10,20,30}，rear = 3。元素个数 $= (3 - 0 + 5)\%5 = 3$，与实际相符。再入队 40，rear = 4，(4+1)\%5 = 0 == front，判满成立，此时队列中确实是 4 个元素（= MaxSize - 1）。

再验证「绕回」的情形：在上面的状态下出队两次，front = 2；再入队 50、60，data[4]=50 后 rear=0，data[0]=60 后 rear=1。此时元素个数 $= (1 - 2 + 5)\%5 = 4$，队列中实际元素为 data[2],data[3],data[4],data[0] 共 4 个，正确。注意这里 **rear < front**，正是选择题最爱的陷阱情形。

*2.*方案二：增设 *size* 计数器

在结构体中增设一个 `int size`（有的教材叫 `count`、`queSize`）记录当前元素个数，所有 `MaxSize` 个单元都可以使用。

- 入队：`size++`；出队：`size--`；
- 判空：`size == 0`；
- 判满：`size == MaxSize`；
- 两个指针的判据 `front == rear` 此时已经不能用来区分空和满了（两种情况都成立）。

代入验证（`MaxSize = 5`）：连续入队 5 个元素，`size = 5` 判满，此时 `front == rear == 0`（`rear` 绕回一圈），说明仅凭指针无法判断，必须依赖 `size`。

*3.*方案三：增设 *tag* 标记

增设一个 `bool tag`，专门记录最近一次操作是入队还是出队（也有教材记「最近一次删除的是否为插入」）。

- 初始化：`front = rear = 0`，`tag = false`；
- 入队成功后：`tag = true`；
- 出队成功后：`tag = false`；
- 判空：`front == rear && tag == false`；
- 判满：`front == rear && tag == true`。

为什么有效：两个指针相等时，若上一次操作是入队，说明是「刚填满一圈追上 `front`」→ 队满；若上一次操作是出队，说明是「刚把最后一个元素取走」→ 队空。如果入队次数与出队次数恰好相等而互相抵消，则最后一次操作必为出队，`tag` 为 `false`，判空成立，正好符合实际。

三种方案的对比与代码要点：

方案	判空	判满	最大容量	需要额外变量	指针相等时的含义
牺牲一个单元	`front == rear`	`(rear+1)%M == front`	`M - 1`	无	一定是队空
设 size	`size == 0`	`size == M`	`M`	`int size`	不确定，看 `size`
设 tag	`front==rear && !tag`	`front==rear && tag`	`M`	`bool tag`	由 tag 决定

```
// 循环队列：牺牲一个存储单元的写法（rear 指向队尾元素的下一个位置）
#define MaxSize 50
typedef struct {
    int data[MaxSize];
    int front, rear;
} SqQueue;

void InitQueue(SqQueue &Q) { Q.front = Q.rear = 0; }

bool QueueEmpty(SqQueue Q) { return Q.front == Q.rear; }

bool QueueFull(SqQueue Q) { return (Q.rear + 1) % MaxSize == Q.front; }
```

```
bool EnQueue(SqQueue &Q, int x) {
    if ((Q.rear + 1) % MaxSize == Q.front) return false;   // 队满
    Q.data[Q.rear] = x;
    Q.rear = (Q.rear + 1) % MaxSize;                       // rear 循环后移
    return true;
}

bool DeQueue(SqQueue &Q, int &x) {
    if (Q.front == Q.rear) return false;                   // 队空
    x = Q.data[Q.front];
    Q.front = (Q.front + 1) % MaxSize;                     // front 循环后移
    return true;
}

int QueueLength(SqQueue Q) {
    return (Q.rear - Q.front + MaxSize) % MaxSize;
}
```

增设 tag 的入队、出队要点：

```
// 设 tag 的写法：front == rear 时由 tag 区分空满
bool EnQueue(SqQueue &Q, int x) {
    if (Q.front == Q.rear && Q.tag == true) return false;  // 队满
    Q.data[Q.rear] = x;
    Q.rear = (Q.rear + 1) % MaxSize;
    Q.tag = true;                                          // 记「刚做过入队」
    return true;
}

bool DeQueue(SqQueue &Q, int &x) {
    if (Q.front == Q.rear && Q.tag == false) return false; // 队空
    x = Q.data[Q.front];
    Q.front = (Q.front + 1) % MaxSize;
    Q.tag = false;                                         // 记「刚做过出队」
    return true;
}
```

易错点

（1）循环队列的容量不等于数组长度。用「牺牲一个单元」的方案时，MaxSize = 6 的数组最多只能存 5 个元素。

（2）取模的模数永远是数组长度 **MaxSize**，很多错误选项会写成% (MaxSize - 1)、% (M-1)，直接排除。

（3）元素个数公式必须加 **MaxSize** 再取模。写成 (rear - front) % MaxSize 在 rear < front（已绕回）时结果错误。举特例时优先举 rear < front 的例子（如 rear = 1、front = 5），最容易看出哪个含 mod 的式子正确。

（4）要看清题目给的是哪种指针约定。若题目说「front 指向队头元素的前一个位置，rear 指向队尾元素」，则判空是 (rear+1)%M == front、判满是 (rear+2)%M == front，与常见写法完全反过来。做法：先按题目约定画出初始空队列，数出两个指针的初始位置，再判断「空」和「满」时指针相差多少格。

真题考法

循环队列是本章最高频的选择题点。典型问法有三类：

- 给出一维数组 A[0..n-1] 与指针指向约定，问初始时 **front**、**rear** 的值（如 2011 年考「第一个元素要存到 A[0]，求初始指针」，答案为 front = 0，rear = n-1）；

- 给出 end1、end2 的指向与「最多容纳 M-1 个元素」，问判空判满条件（2014 年，答案为「队空 `end1 == end2`；队满 `end1 == (end2+1) mod M`」）；
- 给出队头和队尾指针的当前值，问队列中元素个数或再入队/出队后的状态。

2019 年还考过一道 10 分的应用题：要求队列「入队时允许增加空间、出队后空间可重复使用、整个队列占用空间只增不减、入队出队均为 $O(1)$」，答案是采用不设容量上限的循环单链表，队首指针指向第一个含关键字结点、队尾指针指向队尾元素的下一个空闲结点，判空 `front == rear`、判满 `rear->next == front`，出队时不释放结点只移动 `front`。

（四）双端队列（受限双端队列）

双端队列（**deque**）是允许在两端插入和删除的线性表。统考常考它的三种受限形态：

形态	允许入队	允许出队	别称
输入受限双端队列	仅一端	两端	—
输出受限双端队列	两端	仅一端	—
标准双端队列	两端	两端	—

判断某输出序列是否可由受限双端队列得到，最稳妥的办法是从输出序列的第一项倒推，或用「分段」技巧：

- 一端仅入队、另一端可入可出（输出受限）：最终输出序列必可分成前后两段，前一段相对于入队序列是逆序的（这些元素从「可出端」入队，被后来的元素压在后面），后一段是正序的（从另一端入队）。
- 判断时先尝试找一个分界点，使前面逆序、后面正序；找不到分界点则该序列不可能。

三、栈和队列的链式存储结构

（一）链栈

采用链式存储的栈称为链栈。链栈通常用不带头结点的单链表实现，并规定所有操作都在单链表的表头进行：表头结点即栈顶，表尾结点即栈底。

链栈的优点是便于多个栈共享存储空间、提高效率，且不存在栈满上溢的情况。缺点是每个元素都要额外保存指针域，且在「栈空」时需要特别处理空指针。

操作	带头结点	不带头结点
初始化	申请头结点，`S->next = NULL`	`S = NULL`（或 `top = NULL`）
判空	`S->next == NULL`	`S == NULL`
入栈	头插法：`p->next = S->next; S->next = p;`	头插法：`p->next = top; top = p;`
出栈	摘下 `S->next` 并 `free`	摘下 `top` 并 `free`

```
// 链栈：不带头结点，top 直接指向栈顶结点
typedef struct StackNode {
    int data;
    struct StackNode *next;
} StackNode, *LinkStack;
```

```
void InitStack(LinkStack &S) { S = NULL; }                    // 初始化
bool StackEmpty(LinkStack S) { return S == NULL; }            // 判空

bool Push(LinkStack &S, int x) {                              // 入栈 = 头插法
    StackNode *p = (StackNode *)malloc(sizeof(StackNode));
    if (p == NULL) return false;                              // 内存不足
    p->data = x;
    p->next = S;                                              // 新结点指向原栈顶
    S = p;                                                    // 更新栈顶指针
    return true;
}

bool Pop(LinkStack &S, int &x) {                              // 出栈 = 头删法
    if (S == NULL) return false;                              // 栈空，下溢
    StackNode *p = S;
    x = p->data;
    S = p->next;                                              // 栈顶指针下移
    free(p);
    return true;
}

bool GetTop(LinkStack S, int &x) {
    if (S == NULL) return false;
    x = S->data;
    return true;
}
```

易错点

（1）链栈的入栈是「头插法」，出栈是「头删法」，不要在链尾操作——链尾操作需要 $O(n)$ 时间找到尾结点。

（2）链栈出栈时必须先保存待删结点的地址再移动指针，否则 `free` 后无法再接上链表。

（3）链栈的存储空间是动态申请的，因此不需要判满；但出栈前必须判空，否则会对 NULL 解引用。

（二）链队列（带头结点）

链队列是同时带有队头指针和队尾指针的单链表。队头指针指向队头结点（带头结点时指向头结点），队尾指针指向队尾结点。

带头结点的链队列判空与出队都比较统一：

- 判空：`Q.front == Q.rear`（队头、队尾指针指向同一个头结点）；
- 入队：新结点接在 `Q.rear` 之后，再让 `Q.rear` 指向新结点；
- 出队：摘下 `Q.front->next`，若摘下的结点正是队尾结点，还必须把 `Q.rear` 拉回 `Q.front`，否则 `rear` 会变成野指针。

```
// 链队列：带头结点
typedef struct QNode {
    int data;
    struct QNode *next;
} QNode;

typedef struct {
    QNode *front, *rear;      // 队头指针、队尾指针
} LinkQueue;
```

```
void InitQueue(LinkQueue &Q) {                          // 初始化
    Q.front = Q.rear = (QNode *)malloc(sizeof(QNode));  // 建立头结点
    Q.front->next = NULL;
}

bool QueueEmpty(LinkQueue Q) { return Q.front == Q.rear; }   // 判空

void EnQueue(LinkQueue &Q, int x) {                     // 入队：只动 rear
    QNode *s = (QNode *)malloc(sizeof(QNode));
    s->data = x;
    s->next = NULL;
    Q.rear->next = s;                                   // 新结点接到队尾之后
    Q.rear = s;                                         // 更新队尾指针
}

bool DeQueue(LinkQueue &Q, int &x) {                    // 出队：只动 front
    if (Q.front == Q.rear) return false;                // 队空
    QNode *p = Q.front->next;                           // p 指向真正的队头结点
    x = p->data;
    Q.front->next = p->next;                            // 摘下队头结点
    if (Q.rear == p) Q.rear = Q.front;                  // 若摘的是最后一个结点，rear 归位
    free(p);
    return true;
}
```

（三）链队列（不带头结点）

不带头结点时，队空表现为 `Q.front == NULL && Q.rear == NULL`。出队和入队的边界情况都更多：

- 判空：`Q.front == NULL`（等价于 `Q.rear == NULL`）；
- 入队：新建结点后置 `next = NULL`；若原队列为空，则 **front** 和 **rear** 同时指向新结点，否则接到 `Q.rear` 之后并只更新 `rear`；
- 出队：若出队的是最后一个结点，出队后必须把 **front** 和 **rear** 同时置为 **NULL**，否则 `rear` 悬空。

```
// 链队列：不带头结点
void InitQueue(LinkQueue &Q) { Q.front = Q.rear = NULL; }

bool QueueEmpty(LinkQueue Q) { return Q.front == NULL; }

void EnQueue(LinkQueue &Q, int x) {
    QNode *s = (QNode *)malloc(sizeof(QNode));
    s->data = x;
    s->next = NULL;
    if (Q.front == NULL) {          // 队列原本为空
        Q.front = Q.rear = s;       // 首尾指针同时指向新结点
    } else {
        Q.rear->next = s;           // 否则只更新队尾
        Q.rear = s;
    }
}

bool DeQueue(LinkQueue &Q, int &x) {
    if (Q.front == NULL) return false;   // 队空
```

```
    QNode *p = Q.front;
    x = p->data;
    Q.front = p->next;                    // 队头指针后移
    if (Q.front == NULL) Q.rear = NULL;  // * 出队后为空，rear 也要置空
    free(p);
    return true;
}
```

带头结点与不带头结点的取舍

判空判满与边界处理的差别，本质上就是「有没有那个不存数据的头结点」。带头结点时，`front == rear` 唯一对应队空，出队永远摘 `front->next`，不用讨论「摘到最后一个」这种特殊情况；不带头结点时，队空的表示、第一个元素入队、最后一个元素出队都要单独处理。

统考选择题里最常考的两句话：

- 「最适合用作链式队列的是带有队首指针和队尾指针的非循环单链表」；
- 「只带队首指针的非循环单链表，入队需要遍历到表尾，时间复杂度 $O(n)$，因此不适合」。

另外，带尾指针的循环单链表求队头和队尾都在 $O(1)$ 内（`rear->next` 就是队头结点），也可以用作链队列；带首尾指针的循环单链表虽然可行，但每次修改指针还要维护循环特性，属于画蛇添足。

链队列同样不存在队满上溢，出队前判空是唯一必需的边界检查。与顺序队列相比，链队列的缺点是每个结点都有指针开销，且不能随机访问第 k 个元素。

四、多维数组的存储

（一）数组的定义与一维地址公式

数组是由 n ($n \geq 1$) 个相同类型的数据元素构成的有限序列。每个元素在 n 个线性关系中的序号称为该元素的下标，下标的取值范围称为数组的维界。

一维数组 `a[0..n-1]` 中，每个元素占 L 个存储单元，则：

$$LOC(a_i) = LOC(a_0) + i \times L$$

其中 $LOC(a_i)$ 表示元素 a_i 的存储地址（首地址），$LOC(a_0)$ 是数组起始地址，i 是下标，L 是单个元素占用的字节数。

若下标从 **1** 开始（即 `a[1..n]`），则 a_i 前面有 $i-1$ 个元素，公式变为：

$$LOC(a_i) = LOC(a_1) + (i-1) \times L$$

（二）二维数组的行优先与列优先

二维数组可以看成一维数组的推广。按「先存哪一维」的不同，有两种映射方式：

- 行优先（行主序）：先行后列。先存行号较小的元素，行号相同时先存列号较小的元素。
- 列优先（列主序）：先列后行。先存列号较小的元素，列号相同时先存行号较小的元素。

设二维数组 A 的行下标范围是 $[0, h_1]$、列下标范围是 $[0, h_2]$，每个元素占 L 个单元，则共有 (h_1+1) 行、(h_2+1) 列。

行优先地址公式：

$$LOC(a_{ij}) = LOC(a_{00}) + [\,i \times (h_2 + 1) + j\,] \times L$$

列优先地址公式：

$$LOC(a_{ij}) = LOC(a_{00}) + [\,j \times (h_1 + 1) + i\,] \times L$$

符号说明：i 为行下标，j 为列下标，$h_2 + 1$ 是每行的元素个数（即列数 n），$h_1 + 1$ 是每列的元素个数（即行数 m），L 为每个元素占用的存储单元数。

行优先公式的推导：a_{ij} 之前有完整的 i 行（第 0 行到第 $i-1$ 行），每行 h_2+1 个元素，共 $i \times (h_2+1)$ 个；本行中列号小于 j 的元素还有 j 个。因此 a_{ij} 是数组中的第 $i(h_2+1)+j$ 个元素（从 0 开始编号），偏移量乘以 L 即为字节偏移。

列优先公式同理：a_{ij} 之前有完整的 j 列，每列 $h_1 + 1$ 个元素，共 $j(h_1+1)$ 个；本列中行号小于 i 的元素有 i 个。

代入验证：设 A 为 4×5 的二维数组（$h_1 = 3$，$h_2 = 4$），$L = 2$ 字节，$LOC(a_{00}) = 1000$。

- 行优先求 $LOC(a_{2,3})$：偏移量 $= 2 \times 5 + 3 = 13$，地址 $= 1000 + 13 \times 2 = 1026$。
- 列优先求 $LOC(a_{2,3})$：偏移量 $= 3 \times 4 + 2 = 14$，地址 $= 1000 + 14 \times 2 = 1028$。

两者不同是正确的——同一个元素在两种存储顺序下的位置本来就不一样。再看一个极端情形：行优先下 $LOC(a_{2,0})$ 的偏移量是 10，$LOC(a_{1,4})$ 的偏移量是 9，确实 $a_{1,4}$ 排在 $a_{2,0}$ 前面，符合「先行后列」。

（三）下标从 0 与从 1 开始的差异

统考题目经常混用「C 语言数组下标从 0 开始」和「矩阵元素下标从 1 开始」（如 m_{ij} 满足 $1 \le i \le n$），这是最容易丢分的地方。

设矩阵为 n 行 m 列，行优先，每元素占 L 个单元：

- 下标从 **0** 开始（$i \in [0, n-1]$，$j \in [0, m-1]$）：

$$LOC(a_{ij}) = LOC(a_{00}) + (i \times m + j) \times L$$

- 下标从 **1** 开始（$i \in [1, n]$，$j \in [1, m]$）：

$$LOC(a_{ij}) = LOC(a_{11}) + [\,(i-1) \times m + (j-1)\,] \times L$$

两者的区别仅仅是每个下标都减 **1**，把「下标从 1 起」换成从 0 起的相对编号即可。做题时建议统一换算成「以 0 为基的偏移量」再计算，不要直接背两套公式。

真题考法

2021 年考过一道典型题：二维数组 A 按行优先存储，每个元素占 1 个存储单元，$A[0][0]$ 的地址是 100，$A[3][3]$ 的地址是 220，求 $A[5][5]$ 的地址。

解法：设每行 n 个元素，则 $LOC(a_{33}) = LOC(a_{00}) + (3n+3) \times 1 = 100 + 3n + 3$，令它等于 220 解得 $3n = 117$，即 $n = 39$。于是 $LOC(a_{55}) = 100 + (5 \times 39 + 5) = 100 + 200 = 300$。

这类题的关键是先用已知条件反解出「每行多少个元素」，再套公式。

五、特殊矩阵的压缩存储

（一）压缩存储的基本思路

矩阵的压缩存储：为多个值相同的元素只分配一个存储空间，对零元素不分配空间。

普通二维数组按行优先存放 $n \times n$ 矩阵需要 n^2 个单元；若矩阵中有大量元素相同或为零，就可以只存「有用的部分」，其余部分由映射规则推算出来，从而把空间降到 $O(n)$ 或 $O(n)$ 以下的常数倍。

压缩存储的两步固定套路：

1. 确定存哪些元素、按什么顺序存，数出一维数组的总长度（一般是一个等差数列求和）；
2. 把矩阵下标 (i, j) 映射为一维数组下标 k，即数出 a_{ij} 之前共有多少个元素。

（二）对称矩阵

若 n 阶方阵 A 中任意元素都满足 $a_{ij} = a_{ji}$，则称 A 为对称矩阵。

对称矩阵关于主对角线对称，因此只需存储主对角线 + 下三角区（或主对角线 + 上三角区），共

$$S = 1 + 2 + \cdots + n = \frac{n(n+1)}{2}$$

个元素。一维数组最后一个元素的下标为 $\dfrac{n(n+1)}{2} - 1$（数组下标从 0 开始）。

*1.*存主对角线 + 下三角区，按行优先

元素 a_{ij} 满足 $i \geq j$（在下三角区或主对角线上）时，直接取值：

$$k = \frac{i(i+1)}{2} + j$$

下标从 0 开始。推导：a_{ij} 之前有完整的第 0 行到第 $i-1$ 行，元素个数为 $1+2+\cdots+i = \dfrac{i(i+1)}{2}$；本行中列号小于 j 的元素还有 j 个（列号 $0, 1, \cdots, j-1$）。所以 a_{ij} 是第 $\dfrac{i(i+1)}{2} + j$ 个元素，从 0 开始编号即为该式。

代入验证：$n = 4$，数组下标从 0 开始。求 $a_{2,1}$：$k = \dfrac{2 \times 3}{2} + 1 = 3 + 1 = 4$。逐个手排验证：$a_{00} \to 0$，$a_{10} \to 1$，$a_{11} \to 2$，$a_{20} \to 3$，$a_{21} \to 4$，完全吻合。再验证对角元 $a_{3,3}$：$k = \dfrac{3 \times 4}{2} + 3 = 6 + 3 = 9$，而 $\dfrac{4 \times 5}{2} - 1 = 9$，正好是最后一个元素，吻合。

*2.*存主对角线 + 下三角区，按列优先

列优先时，第 c 列只有 $a_{cc}, a_{c+1,c}, \cdots, a_{n-1,c}$ 共 $n - c$ 个元素（因为是下三角，第 c 列第一个被存储的元素是主对角线上的 a_{cc}）。

a_{ij}（$i \geq j$）所在第 j 列的前面有第 0 列到第 $j-1$ 列，元素个数为

$$\sum_{c=0}^{j-1} (n - c) = jn - \frac{j(j-1)}{2}$$

a_{ij} 在本列中位于 a_{jj} 之后的第 $i - j$ 个位置。因此

$$k = j\,n - \frac{j(j-1)}{2} + (i-j) \quad (i \geq j,\ \text{列优先})$$

代入验证：$n=4$，数组下标从 0 开始。手排第 0 列 $a_{00}, a_{10}, a_{20}, a_{30}$ 占下标 0～3；第 1 列 a_{11}, a_{21}, a_{31} 占下标 4～6；第 2 列 a_{22}, a_{32} 占下标 7、8。

求 a_{21}（$i=2, j=1$）：$k = 1\times 4 - \dfrac{1\times 0}{2} + (2-1) = 4+1 = 5$。手排中 a_{21} 是第 1 列的第 2 个元素（a_{11} 是第 1 个），下标为 $4+1=5$，吻合。

再验证一个 j 较大的情形，a_{32}（$i=3, j=2$）：$k = 2\times 4 - \dfrac{2\times 1}{2} + (3-2) = 8-1+1 = 8$。手排中第 2 列从下标 7 开始（$a_{22}\to 7$，$a_{32}\to 8$），吻合。同时 $k=8$ 正好是数组最后一个下标 $\dfrac{4\times 5}{2} - 1 = 9$ 之前的位置——注意 a_{33} 才是最后一个元素（$k = 3\times 4 - 3 + 0 = 9$），也对得上。

对称矩阵总表（下标从 0 开始）

设 n 阶对称矩阵，只压缩存储主对角线 + 下三角区，一维数组 B 共 $\dfrac{n(n+1)}{2}$ 个单元：

存储顺序	$i \geq j$ 时 k 的取值	$i < j$ 时如何取值
行优先	$k = \frac{i(i+1)}{2} + j$	利用对称性：$a_{ij} = a_{ji}$，代入 $k = \frac{j(j+1)}{2} + i$
列优先	$k = jn - \frac{j(j-1)}{2} + (i-j)$	利用对称性：$a_{ij} = a_{ji}$，代入列优先公式（此时 $j < i$ 视为 $(i', j') = (j, i)$）

记忆要点：行优先的公式 $\dfrac{i(i+1)}{2} + j$ 最好记，也最常考。题目给出「上三角」时不要另背新公式，先用对称性把元素翻到下三角再套公式，一条公式包打天下。

翻下标的验证：$n=4$，按列优先存下三角。求 a_{02}（$i=0<j=2$，在上三角，不属于直接存储区）。用对称性得 $a_{02} = a_{20}$，代入 $i'=2$、$j'=0$：$k = 0\times 4 - \dfrac{0\times(-1)}{2} + (2-0) = 2$。手排第 0 列是 $a_{00}, a_{10}, a_{20}, a_{30}$（下标 0～3），$a_{20}$ 的下标正是 2，吻合。

*3.*存主对角线 + 上三角区

若题目要求存储主对角线 + 上三角区，则按行优先时 a_{ij}（$i \leq j$）之前有完整的第 0 行到第 $i-1$ 行，第 c 行有 $n-c$ 个元素，个数为

$$\sum_{c=0}^{i-1}(n-c) = in - \frac{i(i-1)}{2}$$

再加上本行中列号小于 j 的元素 $j-i$ 个，得

$$k = in - \frac{i(i-1)}{2} + (j-i) \quad (i \leq j)$$

代入验证：$n=4$，按行优先存上三角。手排：第 0 行 $a_{00}, a_{01}, a_{02}, a_{03}$ 占下标 0～3；第 1 行 a_{11}, a_{12}, a_{13} 占下标 4～6；第 2 行 a_{22}, a_{23} 占下标 7、8。求 a_{12}：$k = 1\times 4 - \dfrac{1\times 0}{2} + (2-1) = 4+1 = 5$，手排中 a_{12} 是第 1 行第 1 个，下标 $4+1=5$，吻合。

真题考法

对称矩阵的压缩存储是本章计算题第一高频点。统考的常见包装方式是「给 $n\times n$ 对称矩阵，只存上三角（或下三角）部分，按行（或列）优先存入下标从 0 开始的一维数组 N，求某元素的下标」。三条实战建议：

1. 先判断该元素属于哪一半。若元素在下三角而题目只存上三角，用 $a_{ij}=a_{ji}$ 翻过去（如 2020 年题给 $m_{7,2}$，10×10 对称矩阵只存上三角，$7>2$ 在下三角，翻成 $m_{2,7}$）。
2. 老老实实数「它前面有几个元素」，不要临时推导新公式。例如 2018 年题：12×12 对称矩阵只存上三角、按行优先，求 $m_{6,6}$。第 1 行存 12 个、第 2 行存 11 个、……、第 5 行存 8 个，共 $12+11+10+9+8=50$ 个；$m_{6,6}$ 是第 6 行第一个元素，故下标为 50。
3. 注意题目用的是 **1** 起始下标还是 **0** 起始下标。题目说「m_{ij}，$1\le i\le j\le12$」说明矩阵下标从 1 起，而「存入下标从 0 开始的一维数组」说明数组从 0 起，两者各按各的口径数。

（三）三角矩阵

- 下三角矩阵：主对角线及下三角区以外的元素（即上三角区）全部为同一个常数 c（通常为 0）。
- 上三角矩阵：主对角线及上三角区以外的元素（即下三角区）全部为同一个常数 c。

与对称矩阵的关键区别：三角矩阵不可利用对称性互相转换，下三角矩阵的零区不能通过下标翻转来访问，必须单独存一个常数。

压缩策略（以下三角矩阵为例）：按行优先存主对角线 + 下三角区，并把那个常数存在一维数组的最后一个位置。

一维数组长度为

$$S=\frac{n(n+1)}{2}+1$$

最后一个元素的下标为 $\dfrac{n(n+1)}{2}$。

映射关系（下标从 0 开始）：

$$k=\begin{cases}\dfrac{i(i+1)}{2}+j, & i\ge j\ (\text{下三角区或主对角线})\\[2ex] \dfrac{n(n+1)}{2}, & i<j\ (\text{上三角区，取常数}c)\end{cases}$$

代入验证：$n=3$，下三角矩阵

$$A=\begin{pmatrix}a_{00} & c & c\\ a_{10} & a_{11} & c\\ a_{20} & a_{21} & a_{22}\end{pmatrix}$$

手排（行优先，存下三角 + 常数）：下标 0 存 a_{00}，1 存 a_{10}，2 存 a_{11}，3 存 a_{20}，4 存 a_{21}，5 存 a_{22}，6 存常数 c。数组长度为 $\dfrac{3\times4}{2}+1=7$，最后下标为 6。用公式求 a_{21}：$k=\dfrac{2\times3}{2}+1=3+1=4$，吻合。求 a_{02}（$i<j$）：$k=\dfrac{3\times4}{2}=6$，取得常数 c，吻合。

上三角矩阵按行优先存主对角线 + 上三角区时，a_{ij}（$i\le j$）的映射为

$$k=in-\frac{i(i-1)}{2}+(j-i)$$

其余位置（$i>j$）取一维数组最后一个单元中的常数。

易错点

（1）对称矩阵与三角矩阵的公式看起来一样，但 $i<j$（或 $i>j$）时的处理完全不同：对称矩阵靠 $a_{ij}=a_{ji}$ 翻下标访问，三角矩阵直接返回常数区的那个单元。

（2）三角矩阵的一维数组要多开 **1** 个单元存常数，长度是 $\frac{n(n+1)}{2}+1$ 而不是 $\frac{n(n+1)}{2}$。考试若问「数组至少需要多少个元素」，答 $\frac{n(n+1)}{2}+1$。

（3）用「行优先」还是「列优先」要看清题干，两者的 k 公式不同。若题目没说明，统考默认按行优先。

（四）三对角矩阵（带状矩阵）

三对角矩阵又称带状矩阵：n 阶方阵 A 中，当 $|i-j|>1$ 时 $a_{ij}=0$。也就是说非零元素只集中在以主对角线为中心的 **3** 条对角线上：主对角线、主对角线下方一条、主对角线上方一条。

每行的非零元素个数：第 1 行（下标 $i=1$）只有 2 个，第 2 行到第 $n-1$ 行各有 3 个，第 n 行只有 2 个。总数为

$$2+3(n-2)+2=3n-2$$

一维数组最后一个元素的下标为 $3n-3$（下标从 0 开始）。

按行优先压缩存储的映射公式（下标从 1 开始，即 $1\le i,j\le n$）：

$$k=2i+j-3\quad(|i-j|\le 1)$$

推导：a_{ij} 之前有完整的 $i-1$ 行。其中第 1 行有 2 个元素，第 2 行到第 $i-1$ 行共有 $i-2$ 行、每行 3 个，合计 $2+3(i-2)=3i-4$ 个。a_{ij} 在本行中是第 $j-i+2$ 个（本行元素依次是 $a_{i,i-1},a_{ii},a_{i,i+1}$，对应本行内序号 $1,2,3$；下标 j 对应序号 $j-(i-1)+1=j-i+2$）。所以 a_{ij} 是全体中的第 $(3i-4)+(j-i+2)=2i+j-2$ 个，从 0 开始编号即为 $2i+j-3$。

代入验证一（边界）：$i=1$，取 a_{11}，$k=2+1-3=0$，正是压缩数组的第 1 个元素，正确。取 a_{22}（$i=2,j=2$）：$k=4+2-3=3$。手排：第 1 行 a_{11},a_{12} 占 0、1；第 2 行 a_{21},a_{22},a_{23} 占 2、3、4，a_{22} 下标为 3，正确。

代入验证二（统考真题数据）：100 阶三对角矩阵，元素 $m_{i,j}$（$1\le i,j\le 100$）按行优先压缩存入下标从 **0** 开始的一维数组 N，求 $m_{30,30}$ 的下标。用公式：$k=2\times 30+30-3=87$。

换一种独立方法核算：第 1 行有 2 个元素；第 2 行到第 29 行共 28 行、每行 3 个，共 84 个；合计 $2+84=86$ 个元素排在第 30 行之前，故第 30 行第一个元素 $m_{29,30}$ 的下标为 86。第 30 行按 $m_{29,30},m_{30,30},m_{30,31}$ 的顺序存放，$m_{30,30}$ 再后移一位，下标为 $86+1=87$。两种方法结果一致。

若按列优先存储（下标从 1 开始），由对称的推导可得

$$k=2j+i-3\quad(|i-j|\le 1)$$

即把行优先公式中的 i、j 互换。

易错点

三对角矩阵题的三个坑：

1. 下标基数。公式 $k=2i+j-3$ 是矩阵下标从 **1** 开始的形式。若题目给出的是 C 语言风格的 0 起始下标，需要先按 $+1$ 换算，或直接用「数一数前面有几个元素」的方法避免混淆。
2. 首行和末行只有 **2** 个元素。若直接按「每行 3 个」算，会多算 1 个，答案偏大。
3. 一维数组的下标是 **0** 起还是 **1** 起。题目说「压缩存入下标从 0 开始的一维数组」时，公式结果直接用；若说「一维数组 $N[1..3n-2]$」，则结果要按 1 起始再调一次。

（五）稀疏矩阵

稀疏矩阵：矩阵中非零元素的个数 t 相对于矩阵元素总数 s 非常少，即 $s \gg t$ 的矩阵。稀疏矩阵中零元素的分布通常没有规律，因此不能靠下标映射推算，必须显式记录每个非零元素的位置。

统考要求掌握两种压缩存储结构：

*1.*三元组表（顺序存储）

为每个非零元素存一个三元组 <行下标，列下标，值 >，所有三元组按行优先（或列优先）顺序排成一个顺序表。此外还必须额外保存矩阵的总行数、总列数（通常还有非零元素个数）。

为什么必须额外保存行数和列数：三元组只记录「非零元素出现在哪里」，无法确定矩阵的整体规模。如果最后几行或最后几列全是零，它们不会出现在任何三元组中，只凭三元组无法知道这些全零行、全零列是否存在。

```
#define MaxSize 1000
typedef struct {
    int i, j;        // 行下标、列下标
    int v;           // 元素值
} Triple;

typedef struct {
    Triple data[MaxSize];   // 非零元素的三元组表
    int mu, nu, tu;         // 矩阵的行数、列数、非零元素个数
} TSMatrix;
```

三元组表适合顺序存取，但不适合随机存取：要按行号查找某个元素必须从头扫描。若需频繁按行、按列访问，应改用十字链表。

*2.*十字链表（链式存储）

十字链表的每个非零元素对应一个结点，结点内同时挂在行链表和列链表上，因此可以同时按行、按列遍历非零元素。它是稀疏矩阵的链式压缩存储方式，也是图的「有向图十字链表」的同名不同物结构（图那一章的十字链表用于存弧）。

十字链表的结构要点：

- 每个非零元素结点含 5 个域：行下标 `i`、列下标 `j`、值 `v`、指向同列下一个非零元素的 `down` 指针、指向同行下一个非零元素的 `right` 指针；
- 设立行头指针数组和列头指针数组（或把行头、列头结点也串成一个链表）；
- 整个矩阵用一个总的头结点记录行数、列数、非零元素个数。

三元组表与十字链表的对比：

比较项	三元组表	十字链表
存储方式	顺序存储	链式存储
每个非零元素的空间	3 个域	5 个域（含 2 个指针）
按行访问	顺序扫描，较慢	沿行链 O(行内非零元个数)
按列访问	需转置或重建，很慢	沿列链，与按行同效
插入/删除非零元素	需移动大量元素	修改指针即可

比较项	三元组表	十字链表
需要保存的信息	行数、列数、非零元个数	行数、列数、非零元个数（存在总头结点中）

真题考法

- 2017 年考「适用于压缩存储稀疏矩阵的两种存储结构」，答案是三元组表和十字链表。干扰项「邻接矩阵」（需要为所有行列位置分配空间，本质是完整二维数组，不属于压缩存储）和「二叉链表」（不是矩阵的存储结构）都要能排除。
- 2023 年考「用三元组表存储稀疏矩阵，除三元组外还需要保存什么」，答案是矩阵的总行数和总列数（「包含非零元素的行数/列数」可以通过扫描三元组统计出来，不是重建矩阵所必需的规模信息）。
- 三元组表还可以考转置算法：朴素转置按列扫描，时间复杂度 $O(nu \times tu)$；快速转置先统计每列非零元个数并计算各列起始位置，再一趟扫描直接放到目标位置，时间复杂度 $O(nu + tu)$。

六、栈、队列和数组的应用

（一）栈在括号匹配中的应用

算法原理：从左到右扫描表达式。遇到左括号就入栈（后遇到的左括号要先闭合，因此放在栈顶）；遇到右括号时，若栈空说明右括号多余，匹配失败；否则弹出栈顶左括号，检查类型是否相同，不同则失败。扫描结束后栈必须为空，否则说明左括号多余。

判定的三种失败情形：右括号到来时栈空（右括号多）；弹出的左括号类型不匹配（如 `[` 配 `)`）；扫描结束时栈非空（左括号多）。

```
bool BracketMatch(char str[], int n) {
    SqStack S; InitStack(S);
    if (n % 2 == 1) return false;             // 长度为奇数必不匹配
    for (int i = 0; i < n; i++) {
        if (str[i]=='(' || str[i]=='[' || str[i]=='{') {
            Push(S, str[i]);                   // 左括号入栈
        } else {
            if (StackEmpty(S)) return false;   // 右括号多余
            char top;
            Pop(S, top);                       // 弹出栈顶左括号
            if (str[i]==')' && top!='(') return false;
            if (str[i]==']' && top!='[') return false;
            if (str[i]=='}' && top!='{') return false;
        }
    }
    return StackEmpty(S);                      // 栈空才算匹配
}
```

代入验证：输入 `{[()]}`。扫描 `{`、`[`、`(` 依次入栈；遇 `)` 弹出 `(` 匹配；遇 `]` 弹出 `[` 匹配；遇 `}` 弹出 `{` 匹配；结束栈空，返回 `true`。输入 `([)]`：遇 `)` 时栈顶是 `[`，类型不同，返回 `false`，正确。

（二）栈在表达式求值中的应用

*1.*前缀、中缀、后缀表达式

- 中缀表达式：运算符在两个操作数之间，如 `a+b`。中缀表达式需要括号和优先级规则才能确定运算顺序，计算机直接处理不方便。

- 后缀表达式（逆波兰式）：运算符在两个操作数之后，如 ab+。后缀表达式不需要括号就能唯一确定运算顺序，最适合用栈求值。
- 前缀表达式（波兰式）：运算符在两个操作数之前，如 +ab。

手算转换方法（推荐）：先给中缀表达式按运算顺序加括号，再把每个括号内的运算符移到对应位置的后面（后缀）或前面（前缀），最后去掉所有括号。

例 **1**：中缀 a+b-c*d。

- 确定运算顺序并加括号：((a+b)-(c*d))；
- 后缀：把 + 移到 ab 后得 ab+，* 移到 cd 后得 cd*，- 移到最后得 ab+cd*-；
- 前缀：+ 移到 ab 前得 +ab，* 移到 cd 前得 *cd，- 移到最前得 -+ab*cd。

例 **2**：中缀 a/b+(c*d-e*f)/g。

- 加括号：((a/b)+(((c*d)-(e*f))/g))；
- 后缀：ab/cd*ef*-g/+；
- 前缀：+/ab/-*cd*efg。

例 **3**（统考真题数据）：中缀 x+y*(z-u)/v。

- 加括号：(x+((y*(z-u))/v))；
- 后缀：xyzu-*v/+。

对照 2024 年该题的四个选项，只有 xyzu-*v/+ 与之一致。

*2.*用栈实现后缀表达式的计算

使用一个操作数栈，从左到右扫描后缀表达式：

- 遇到操作数：直接入栈；
- 遇到运算符：连续弹出两个操作数，先出栈的是右操作数，后出栈的是左操作数，执行左 op 右运算，把结果压回栈；
- 扫描结束后，栈中唯一的元素即为表达式的值。

代入验证：后缀 34+5*（对应中缀 (3+4)*5）。

- 遇 3，栈 [3]；
- 遇 4，栈 [3,4]；
- 遇 +，弹出 4（右）、3（左），算 $3+4=7$，栈 [7]；
- 遇 5，栈 [7,5]；
- 遇 *，弹出 5（右）、7（左），算 $7\times 5=35$，栈 [35]。

结果为 35，正确。注意减法、除法必须区分左右操作数：后缀 53- 的值是 $5-3=2$，不是 $3-5=-2$。

*3.*中缀转后缀的算法

初始化一个栈，用于保存暂时还不能确定运算顺序的运算符。从左到右扫描中缀表达式：

- 遇到操作数：直接加入后缀表达式；
- 遇到左括号 (：直接入栈；

- 遇到右括号)：依次弹出栈内运算符并加入后缀表达式，直到弹出 (为止，(不加入后缀表达式；
- 遇到运算符：
  1. 若优先级高于栈顶运算符（或栈空、栈顶是 (），当前运算符直接入栈；
  2. 若优先级小于等于栈顶运算符，则不断弹出优先级大于等于当前运算符的栈顶元素加入后缀表达式，直到栈空或栈顶为 (，再把当前运算符入栈；
- 扫描结束后，把栈中剩余运算符依次弹出并加入后缀表达式。

代入验证：中缀 a/b+(c*d-e*f)/g，扫描到 f 时栈中的内容（栈底到栈顶）。

- 读 a → 输出 a；
- 读 / → 栈空，入栈。栈：/；
- 读 b → 输出 b；
- 读 + → 优先级 1 ≤ 栈顶 /（2），弹出 / 输出；栈空，+ 入栈。栈：+；
- 读 (→ 直接入栈。栈：+，(；
- 读 c → 输出 c；
- 读 * → 栈顶是 (，直接入栈。栈：+，(，*；
- 读 d → 输出 d；
- 读 - → 优先级 1 ≤ 栈顶 *（2），弹出 * 输出；栈顶为 (，停止；- 入栈。栈：+，(，-；
- 读 e → 输出 e；
- 读 * → 优先级 2 > 栈顶 -（1），直接入栈。栈：+，(，-，*；
- 读 f → 输出 f（此时栈底到栈顶依次是 +、(、-、*）。

这个结果与 2014 年真题的答案选项一致。

*4.*用栈直接计算中缀表达式

初始化两个栈：操作数栈 num 和运算符栈 op。

- 遇到操作数：压入 num；
- 遇到 (：压入 op；
- 遇到)：不断执行「求值」（弹出一个运算符、弹出两个操作数、运算、结果压回 num），直到 op 栈顶为 (，然后弹出 (；
- 遇到运算符：当 op 非空且栈顶运算符优先级大于等于当前运算符时，不断求值，再把当前运算符压入 op；
- 全部扫描完后，继续求值直到 op 为空，num 栈顶即为结果。

这里的「中缀转后缀」与「后缀求值」两套逻辑被合并在一次扫描中完成，效率和正确性都更好。

```
// 中缀表达式直接求值：操作数栈 + 运算符栈
void eval(stack<int> &num, stack<char> &op) {
    int a = num.top(); num.pop();      // 先弹出的是右操作数
    int b = num.top(); num.pop();      // 后弹出的是左操作数
    char p = op.top(); op.pop();
    if (p == '+') num.push(b + a);
    else if (p == '-') num.push(b - a);   // * 注意是 b - a
    else if (p == '*') num.push(b * a);
    else num.push(b / a);                 // * 注意是 b / a
}
```

易错点

（1）减法与除法必须区分左右操作数。后缀求值时先出栈的是右操作数，写成 `a - b` 就错了。

（2）中缀转后缀时「不入后缀表达式，遇到）弹出到「并把「丢弃。

（3）中缀转后缀的「弹出」条件是优先级「小于等于」（同级也要弹出，保证左结合），而「直接入栈」的条件是优先级「大于」。选择题常在这里设置干扰。

（4）不要声称「消除递归必须使用栈」。尾递归可以直接用循环改写，迭代法不一定需要显式栈。2017 年真题正是考这个错误说法。

（5）中缀表达式本身不唯一。同一个中缀表达式在运算符优先级相同时（如 `a+b+c`）可以由不同的结合顺序得到不同的后缀形式，但结果值相同。统考默认按「左优先」原则转换。

（6）中缀转后缀时不要漏掉扫描结束后的收尾步骤。全部字符扫描完后，必须把操作符栈中剩余的运算符依次弹出并追加到后缀表达式末尾，否则会丢掉最后几个运算符。选择题常问「扫描到某个字符时栈内还剩哪些运算符」，此时收尾步骤尚未执行。

三种表达式与栈的对应关系

表达式形式	例子（中缀 `a+b-c*d`）	用栈求值的方式
前缀（波兰式）	`-+ab*cd`	从右向左扫描；遇操作数入栈，遇运算符弹两个操作数运算后压回
中缀	`a+b-c*d`	需要两个栈（操作数栈 + 运算符栈）同时处理优先级与括号
后缀（逆波兰式）	`ab+cd*-`	从左向右扫描；遇操作数入栈，遇运算符弹两个操作数运算后压回

前缀求值时先出栈的是左操作数，后缀求值时先出栈的是右操作数——这是因为两者扫描方向相反，弹栈的先后顺序自然也相反。这个对应关系记清了，两类题都不会错。

（三）栈在递归中的应用

函数调用的特点是最后调用的函数最先被执行（**LIFO**）。调用函数时，系统需要用函数调用栈保存三类信息：调用返回地址、实参、局部变量。递归调用时，这个栈称为递归工作栈。

- 每进入一层递归，就把该层的返回地址、实参、局部变量压入栈顶；
- 每退出一层递归，就从栈顶弹出这些信息，恢复到上一层。

递归的优缺点：代码简洁、可读性好；但效率较低，递归层数过多会导致栈溢出（栈空间耗尽），且可能存在大量重复计算（如朴素的斐波那契递归）。

递归与栈的关系（**2015** 年真题考法）：程序 `main()` 调用 `S(1)`，`S(1)` 调用 `S(0)`，则递归最深时，从栈底到栈顶依次是 `main()`、`S(1)`、`S(0)`。这也解释了为什么「用非递归方式重写递归程序时不一定必须使用栈」——尾递归可以改成循环，但函数调用本身必须用栈保存现场。

递归的深度与栈容量的关系：设递归函数每层占用 s 个字节的栈帧，递归深度为 d，则需要的栈空间至少为 $d \times s$ 字节。若系统栈总容量为 S，能支持的最大递归深度约为 $\lfloor S/s \rfloor$。这解释了为什么「太深的递归会栈溢出」——是栈空间不够，而不是算法逻辑错误。

用栈把递归改写成迭代的思路：把递归函数的「参数 + 局部变量 + 返回位置」打包成一个栈元素显式压栈，用循环模拟系统调用栈的行为。二叉树的中序遍历非递归版本（讲义中的 LeetCode 94）就是典型例子：

- 一个结点入栈后有两种处理方式：要么立即出栈（说明该结点没有左孩子），要么让下一个结点

入栈（有左孩子）；

- 一个结点出栈后也有两种处理方式：要么继续出栈（该结点没有右孩子），要么把下一个结点入栈（有右孩子）。

（四）栈在进制转换中的应用

把十进制整数 N 转换为 r 进制，使用除 r 取余法：反复用 N 除以 r，把每次的余数依次入栈，直到商为 0；再依次出栈，得到的序列即为转换结果。

为什么用栈：除 r 取余法得到的余数是从低位到高位产生的，而输出需要从高位到低位，正好是一次逆序，与栈的 LIFO 特性吻合。

代入验证：把十进制的 1348 转换为八进制。

步骤	除法	商	余数（入栈顺序）
1	$1348 \div 8$	168	4
2	$168 \div 8$	21	0
3	$21 \div 8$	2	5
4	$2 \div 8$	0	2

余数入栈顺序为 4、0、5、2，出栈顺序为 2、5、0、4，故 $1348 = (2504)_8$。验算：$2\times8^3+5\times8^2+0\times8+4 = 1024+320+0+4 = 1348$，正确。

时间复杂度：需要做 $\lfloor \log_r N \rfloor + 1$ 次除法，每次入栈和出栈都是 $O(1)$，故总时间复杂度为 $O(\log_r N)$；栈的最大深度也等于这个数量级。

小数部分的转换用乘 r 取整法，取整顺序与输出顺序一致，因此不需要用栈，直接按顺序输出即可。

（五）队列在层次遍历中的应用

树的层次遍历（广度优先遍历）是队列最经典的应用：从根结点开始，根结点入队；每次取出队头结点并访问，然后把它的所有孩子依次入队；重复直到队空。

```
void LevelOrder(BiTree T) {
    SqQueue Q; InitQueue(Q);
    BiTree p;
    if (T != NULL) EnQueue(Q, T);        // 根结点入队
    while (!QueueEmpty(Q)) {
        DeQueue(Q, p);                    // 队头结点出队
        visit(p);                         // 访问该结点
        if (p->lchild != NULL) EnQueue(Q, p->lchild);   // 左孩子入队
        if (p->rchild != NULL) EnQueue(Q, p->rchild);   // 右孩子入队
    }
}
```

为什么层次遍历必须用队列：队列保证「先被发现（入队）的结点先被访问（出队）」，因此访问顺序自然是从上到下、从左到右的层序。若换成栈，就变成了深度优先遍历（DFS）。

同样的逻辑用于图的广度优先搜索（**BFS**）：队列存待访问的顶点，保证按「距起始顶点的距离」由近到远访问，因此 BFS 可以求无权图的最短路径。

（六）队列在缓冲区与 **CPU** 调度中的应用

- 缓冲区管理：为了解决 CPU/主机与慢速外设（打印机、键盘等）速度不匹配的问题，设置数据缓冲区。主机依次写入、外设依次取出，逻辑上就是 FIFO 队列。2009 年真题直接考这一点。
- **CPU** 进程调度：先来先服务（FCFS）调度算法用一个就绪队列保存等待处理机的进程，每次把队首进程投入运行，运行结束或时间片用完再挂到队尾，即轮转（**Round Robin**）调度。多级反馈队列、优先级调度同样以队列作为基本组织形式。
- 其他典型场景：消息传递（异步通信）、线程池任务分发、打印任务排队、电商订单处理等所有需要「先来后到」的场合。

（七）数组的典型应用：矩阵运算与顺序表

数组在统考中除了「压缩存储的下标计算」，还会以矩阵运算和顺序表操作的形式出现。常见的两类：

1. 矩阵的转置。设原矩阵 A 为 $m\times n$，转置矩阵 B 为 $n\times m$，则 $B[j][i]=A[i][j]$。若两个矩阵都用按行优先的一维数组存储，朴素转置需要对 A 做 n 遍扫描（每遍取一列），时间复杂度为 $O(m\times n)$；而如果只按二维数组逐元素赋值，时间复杂度为 $O(m\times n)$、空间复杂度 $O(m\times n)$。对于稀疏矩阵，转置要用到三元组表，见本章第五节的说明。

2. 用数组实现「地址计算」类操作。这类题的核心就是本章第四节的 LOC 公式：给定起始地址、元素大小和行列下标，求目标元素的地址。做题步骤固定为：

- 判断是行优先还是列优先；
- 确定「每行/每列有多少个元素」；
- 数出目标元素前面有多少个元素（这是偏移量）；
- 地址 $=$ 起始地址 $+$ 偏移量 $\times L$。

代入验证：三维数组 $A[0..1][0..2][0..3]$ 按行优先存储，每个元素占 4 字节，起始地址为 0，求 $A[1][1][2]$ 的地址。先按行优先展开为等效二维数组，每行有 $3\times 4=12$ 个元素（即后两维的长度之积）。$A[1][1][2]$ 的前面有完整的第 0 层（12 个元素）、第 1 层中的第 0 行（4 个元素）、第 1 行中的前 2 个元素（2 个），共 $12+4+2=18$ 个元素。地址 $=0+18\times 4=72$。

一般地，n 维数组按行优先存储时，下标 $(i_1,i_2,\ldots,i_n)$ 对应的一维偏移量为

$$\text{offset}=i_1\cdot(d_2d_3\cdots d_n)+i_2\cdot(d_3\cdots d_n)+\cdots+i_{n-1}\cdot d_n+i_n$$

其中 d_k 是第 k 维的长度。这个「多项式求值」形式与行优先的二维公式完全一致，理解来龙去脉后无需死记。

真题考法

数组类的统考题目主要有三种包装：

1. 给起始地址和某个元素的地址，反解出每行元素个数，再求另一个元素的地址（2021 年，见本章第四节）。
2. 问「按行优先存储」「按列优先存储」时某元素的下标或地址（近年多次出现）。
3. 问数组占用的总存储空间：$m\times n$ 的二维数组共需 $m\times n\times L$ 字节；若用压缩存储（如对称矩阵），则只需 $\dfrac{n(n+1)}{2}\times L$ 字节。这类问法的关键是先判断题目问的是压缩前还是压缩后。

（八）栈与队列的合法出栈序列

这是本章综合度最高、也最常出选择题的部分。

*1.*判断一个序列是否为合法出栈序列

模拟法（万能方法）：按入栈序列顺序不断压栈，每压入一个元素后，只要栈顶与目标出栈序列的当前元素相等就弹出并推进目标序列；全部入栈完毕后若栈空且目标序列走完，则该序列合法，否则不合法。

手算技巧：不需要写完整模拟。抓住一条核心性质——

合法出栈序列的判定准则

对任意两个元素 x、y，若它们在入栈序列中 x 在 y 之前，则在出栈序列中：

- 二者顺序可以任意（先出 x 后出 y，或先出 y 后出 x 都合法）；
- 但绝不允许出现这样的模式：y 已经出栈，而比 y 先入栈的 x 尚未出栈、且 x 与 y 之间的元素也仍未出栈——本质上是「出栈序列中不可能出现跨栈的逆序交叉」。

更实用的判定表述：对于出栈序列中的任意两个元素 a、b（a 在出栈序列中排在 b 前面），若在入栈序列中 a 排在 b 后面，则所有在入栈序列中位于 a、b 之间的元素，在出栈序列中也必须排在 b 之前。

代入验证：入栈序列为 `1,2,3`，判断 `3,1,2` 是否合法。3 先出栈，说明 `1,2,3` 全部已入栈，栈内从底到顶为 `1,2`，下一个只能出 `2`，不可能出 `1`。故 `3,1,2` 不合法。

再验证 `2,3,1`：逐一模拟——`1` 入栈，`2` 入栈，弹出 `2`；`3` 入栈，弹出 `3`；弹出 `1`。得到 `2,3,1`，合法。「`3,2,1`」合法（全部入栈后依次弹出）；「`1,2,3`」合法（每入栈一个就立即弹出）。

*2.*卡特兰数与出栈序列的个数

n 个不同元素依次进栈（进栈顺序固定），其可能的出栈序列个数为卡特兰（**Catalan**）数：

$$N=\frac{1}{n+1}C_{2n}^{n}=\frac{(2n)!}{n!\,(n+1)!}$$

符号说明：n 是元素个数（$n\geq 1$），C_{2n}^{n} 是从 $2n$ 个位置中选 n 个的组合数，N 是不同出栈序列的总数。

卡特兰数的推导思路

把「入栈」看作 $+1$、「出栈」看作 -1，任何时刻已入栈的元素个数不少于已出栈的元素个数，即前缀和恒非负（栈不能为空时出栈）。长度 $2n$ 的 ±1 序列中共有 C_{2n}^{n} 个满足「n 个 $+1$、n 个 -1」，其中前缀和非负的序列数为 $\frac{1}{n+1}C_{2n}^{n}$。这正是卡特兰数。

卡特兰数在数据结构中还有一个用途：n 个结点的不同形态二叉树（二叉排序树形态数）也是卡特兰数。因为「先序序列可视为入栈次序、中序序列可视为出栈次序」，二者的对应关系把两个计数问题统一起来。

代入验证一：$n=5$ 时，

$$N=\frac{1}{6}C_{10}^{5}=\frac{1}{6}\times\frac{10\times9\times8\times7\times6}{5\times4\times3\times2\times1}=\frac{30240}{6\times120}=\frac{30240}{720}=42$$

与逐个模拟的结果一致（$n=1,2,3,4,5$ 时 N 分别为 $1,2,5,14,42$）。

代入验证二（先序序列计数）：先序序列为 `a,b,c,d` 的不同二叉树有多少棵？先序序列是入栈次序，

中序序列是出栈序列，$n=4$：

$$N=\frac{1}{5}C_8^4=\frac{1}{5}\times 70=14$$

即共有 14 棵树。

*3.*带限制条件的出栈序列计数

统考常在纯卡特兰数的基础上加条件，此时不能直接套公式，必须按条件分类讨论。常见限制有两类：

限制一：指定某个元素为第一个出栈元素。要让 d 第一个出栈（入栈序列 a,b,c,d,e），必须先把 a,b,c,d 全部压栈，再立即弹出 d。此时栈内从底到顶为 a,b,c，e 尚未入栈。接下来分两类：

- 先弹出 c，则剩余 a,b 已全在栈中、e 未入栈，e 的入栈时机有三种，得到 d,c,b,a,e、d,c,b,e,a、d,c,e,b,a；
- 在弹出 c 之前先把 e 入栈，则 e 必须先出栈，得到 d,e,c,b,a。

共 **4** 种，与 2011 年真题答案一致。

限制二：禁止连续 k 次同类操作。如 2010 年考「不允许连续三次出栈」。判定方法是逐项构造操作序列：若某序列在构造过程中不可避免地要连续弹出 3 次（即栈内积压了 3 个以上元素而没有新的入栈操作可以打断），则该序列不可能。

以入栈序列 a,b,c,d,e,f、「不允许连续三次出栈」为例，逐项验证四个候选序列：

候选序列	构造过程	是否可行
d,c,e,b,f,a	abcd 入栈，弹 d、c（连续 2 次）；e 入栈弹 e；弹 b；f 入栈弹 f；弹 a	可行
c,b,d,a,e,f	abc 入栈，弹 c、b（2 次）；d 入栈弹 d；弹 a；e 入栈弹 e；f 入栈弹 f	可行
b,c,a,e,f,d	ab 入栈弹 b；c 入栈弹 c；弹 a；de 入栈弹 e；f 入栈弹 f；弹 d	可行
a,f,e,d,c,b	a 入栈弹 a；bcdef 全部入栈（此时栈内积压 5 个元素）；弹 f 后，栈顶依次是 e,d,c,b，已无未入栈元素可以打断，只能连续弹 f,e,d 三次	不可行

表中第四项正是 2010 年真题的答案（选项 D）。这类题的通用做法是：找出序列中「连续递减且已无剩余元素可入栈」的那一段，若长度达到 k 就违规。

限制三：入栈序列与出栈序列的对应关系（2022 年考法）。给定完整的 in 和 out，总是可以用模拟法判断 out 是否合法，因此「不能判断」的说法错误。此外：若每入栈一个就立即出栈，out 可以与 in 完全相同；若全部入栈后再依次弹出，out 恰为 in 的倒序。故「in 与 out 一定不同」错误，「可能互为倒序」正确。

限制四：双端队列的输出序列计数。双端队列的两端都能操作，情况比栈复杂，但卡特兰数仍是它的下界参照：输出受限（仅一端出）时可行序列更多。判断时先用「分段逆序」技巧筛选，再用模拟法确认。

易错点

（1）卡特兰数公式只适用于「入栈顺序固定、无额外限制、元素互不相同」的情形。一旦题目加了「不允许连续三次出栈」「以某元素开头」等条件，必须重新分类计数。

（2）卡特兰数计的是「不同出栈序列的个数」，不是「不同操作序列的个数」。若问的是「有多少种不同的入栈/出栈操作序列」，答案会更大。

（3）题目若给的是「栈的入栈序列确定，求可能的出栈序列个数」，注意栈容量可能有限制。若题目还限制了栈的容量，就不能用卡特兰数。

（4）判断合法出栈序列时不要只看首尾元素。要看元素之间的「跨越逆序」关系，或者干脆老老实实按模拟法走一遍。

（5）卡特兰数的递推公式 $N_n = \sum_{k=1}^{n} N_{k-1} N_{n-k}$（$N_0 = 1$）有时也会用到。它对应「最后一个出栈的元素把式子分成左右两部分」的分治思想，与二叉树的形态计数完全一致。

（九）用栈实现队列、用队列实现栈

统考的应用题与选择题都爱考「受限结构的互相模拟」，核心是用两个栈/两个队列抵消掉各自的方向性。

*1.*用两个栈实现队列

设 `A` 为入队栈，`B` 为出队栈：

- 入队：元素直接压入 `A`；
- 出队：若 `B` 非空，直接从 `B` 弹栈顶；若 `B` 为空，则把 `A` 中所有元素依次弹出并压入 `B`，再从 `B` 弹栈顶；
- 查队头：同出队逻辑，但不弹出；
- 元素个数：`A.size() + B.size()`。

为什么正确：`A` 中元素从栈底到栈顶是「早入队 → 晚入队」，全部倒入 `B` 后顺序反转，`B` 的栈顶恰好是最早入队的元素，符合 FIFO。

代入验证：空队列执行 `push(1)`，`push(2)`，`pop`，`push(3)`，`pop`。

操作	`A`（底 → 顶）	`B`（底 → 顶）	输出
`push(1)`	1	—	—
`push(2)`	1, 2	—	—
`pop`	—	—	`B` 空，把 `A` 全部倒入 `B`：`B` = 2, 1（底 → 顶），弹出 1
`push(3)`	3	2	—
`pop`	3	2	`B` 非空，直接弹出 2

输出序列为 1、2，正确。

时间复杂度分析（重要考点）：单个 `pop` 操作最坏为 $O(n)$（需要倒栈），但每个元素至多被压入 `A` 一次、倒入 `B` 一次、从 `B` 弹出一次，因此 n 次操作的总时间复杂度为 $O(n)$，均摊（摊还）时间复杂度为 $O(1)$。

*2.*用两个队列实现栈

设 `Q1` 为主队列，`Q2` 为辅助队列：

- 入栈：元素直接入 `Q1`；

- 出栈：把 Q1 中除队尾元素外的所有元素依次出队并转入 Q2，剩下的那个队尾元素就是栈顶，出队并返回；然后交换 Q1 和 Q2 的角色（或把 Q2 的元素再转回 Q1）；
- 取栈顶：同出栈过程，但不删除最后那个元素。

时间复杂度：单次出栈为 $O(n)$，n 次出栈的总时间复杂度为 $O(n^2)$。用队列实现栈比用栈实现队列代价更高，这是两者的本质差别——栈的 LIFO 特性比队列的 FIFO 更容易「叠加」出来。

真题考法

「用两个栈实现队列」是应用题常客，典型四问是：（1）写出结构定义；（2）说明入队出队过程；（3）画出空队列执行 `push(1)`，`push(2)`，`pop`，`push(3)`，`pop` 的过程；（4）求 n 个元素任意顺序进出后队列为空的总时间复杂度。第（4）问的标准答案是 $O(n)$——每个元素两次入栈、两次出栈，共 $4n$ 次 $O(1)$ 操作。若答成 $O(n^2)$ 会被扣分，因为那是按「单次最坏」而非「总量」估算的。

另一类常考应用题是最小栈：用「数据栈 + 辅助栈」实现 $O(1)$ 取最小值。入栈时，若新元素 $\le$ 辅助栈顶，则同时压入辅助栈；出栈时，若弹出的元素等于辅助栈顶，则辅助栈也弹出。这样辅助栈顶永远保存当前最小值。注意条件是「小于等于」而不是「小于」——用「小于」时，遇到重复的最小值会导致出栈后辅助栈提前弹空。

本章小结表

知识点	核心公式 / 结论	关键条件
栈的特点	后进先出 LIFO，只在栈顶一端操作	逻辑结构仍是线性表
队列的特点	先进先出 FIFO，队尾入、队头出	入队只动 `rear`，出队只动 `front`
顺序栈（top 指向栈顶元素）	判空 `top == -1`；判满 `top == MaxSize-1`；入栈 `data[++top]=x`	先加后存
顺序栈（top 指向栈顶下一位置）	判空 `top == 0`；判满 `top == MaxSize`；入栈 `data[top++]=x`	先存后加；元素个数 = `top`
共享栈	判空 `top0 == -1 && top1 == MaxSize`；判满 `top0 + 1 == top1`	全部 `MaxSize` 个单元可用
循环队列（牺牲一个单元）	判空 `front == rear`；判满 `(rear+1)%M == front`；个数 `(rear-front+M)%M`	容量只有 $M-1$；模数永远是 M
循环队列（设 size）	判空 `size == 0`；判满 `size == M`	容量 M，指针相等时分不清空满
循环队列（设 tag）	判空 `front==rear && !tag`；判满 `front==rear && tag`	入队置 `tag=true`，出队置 `tag=false`
链队列（带头结点）	判空 `front == rear`；出队若摘掉尾结点需 `rear = front`	最适合做链队列
链队列（不带头结点）	判空 `front == NULL`；出队至空需 `front = rear = NULL`	边界情况更多
链栈	入栈头插法、出栈头删法；无上溢	通常在表头操作
二维数组行优先	$LOC(a_{ij}) = LOC(a_{00}) + [\,i \times n + j\,] \times L$	n 为列数
二维数组列优先	$LOC(a_{ij}) = LOC(a_{00}) + [\,j \times m + i\,] \times L$	m 为行数
对称矩阵（下三角，行优先）	$k = \frac{i(i+1)}{2} + j$（$i \ge j$）；$i < j$ 时用 $a_{ij} = a_{ji}$ 翻下标	数组共 $\frac{n(n+1)}{2}$ 个单元，下标从 0

知识点	核心公式 / 结论	关键条件
三角矩阵（下三角，行优先）	$i \geq j$：$k = \frac{i(i+1)}{2} + j$；$i < j$：$k = \frac{n(n+1)}{2}$（取常数）	数组共 $\frac{n(n+1)}{2} + 1$ 个单元
三对角矩阵（行优先）	$k = 2i + j - 3$；数组长度 $3n - 2$	矩阵下标从 1 起，$\|i - j\| \leq 1$；首末行各 2 个
稀疏矩阵	三元组表（顺序，3 域）、十字链表（链式，5 域）	都需额外保存矩阵的总行数、总列数
卡特兰数	$N = \frac{1}{n+1} C_{2n}^{n}$	元素互不相同、入栈顺序固定、无其他限制
中缀转后缀	运算符优先级 $\leq$ 栈顶时弹出；（入栈不输出，）弹到（为止	操作数直接输出
后缀求值	遇运算符弹两个操作数，先出栈的是右操作数	结果压回栈，最后栈顶即答案

易错清单

1. 栈「两端都能操作」是错的。栈只在栈顶一端操作，两端都能操作的是双端队列。
2. 顺序栈的入栈顺序取决于 `top` 的含义。`top == 0` 表示空栈 ⇒ `top` 指向栈顶下一位置 ⇒ 入栈是「先存后加」；`top == -1` 表示空栈 ⇒ `top` 指向栈顶元素 ⇒ 入栈是「先加后存」。
3. 共享栈的判满与循环队列的判满形式相似、含义相反。共享栈 `top0 + 1 == top1` 时是全部空间用满；循环队列 `(rear+1)%M == front` 时是只用了 **M-1** 个。
4. 循环队列的取模模数永远是数组长度 `MaxSize`，不是 `MaxSize-1`，也不是队列容量。
5. 循环队列元素个数必须写成 `(rear - front + MaxSize) % MaxSize`。不加 `MaxSize` 在 `rear < front`（指针已绕回）时出错。
6. 入队只改 `rear`，出队只改 `front`。选择题中用这一点能快速排除一半选项。
7. 链队列出队时若摘掉的是队尾结点，带头结点实现要把 `rear` 拉回 `front`，不带头结点实现要同时把 `front`、`rear` 置空，否则 `rear` 悬空。
8. 带头结点的链队列判空是 `front == rear`；不带头结点是 `front == NULL`。两者不可混用。
9. 链栈、链队列没有上溢问题，但出栈、出队前必须判空，否则下溢。
10. 二维数组的行优先偏移用「列数」乘 i，列优先偏移用「行数」乘 j，两个数不要搞反。
11. 矩阵下标从 **1** 开始时每个下标都要减 **1**。看到「m_{ij}，$1 \leq i, j \leq n$」先换算成从 0 起的相对编号再套公式最保险。
12. 对称矩阵与三角矩阵的公式形式相同，但 `i<j` 时的处理不同：对称矩阵靠 $a_{ij} = a_{ji}$ 翻下标，三角矩阵直接返回常数单元。
13. 三角矩阵的一维数组要多开 **1** 个单元存常数，长度是 $\dfrac{n(n+1)}{2} + 1$。
14. 三对角矩阵的首行、末行各只有 **2** 个元素，数「前面有多少个元素」时不能按「每行 3 个」一刀切。
15. 三元组表必须额外保存矩阵的总行数和总列数。「含非零元素的行数/列数」可以扫描统计出来，不是必需的规模信息。
16. 稀疏矩阵的两种压缩存储是三元组表和十字链表。邻接矩阵本质是完整二维数组，不属于压缩存储；二叉链表与矩阵无关。
17. 后缀表达式中减法和除法要先出栈的是右操作数。写成 `a-b`、`a/b` 会得到相反的结果。

18. 中缀转后缀时（和）都不进入后缀表达式，右括号处理完后要把左括号弹出丢弃。
19. 「消除递归必须使用栈」是错误说法。尾递归可以直接改写成循环；但函数调用本身确实需要用栈保存返回地址、实参、局部变量。
20. 卡特兰数只适用于「入栈顺序固定、元素互不相同、无额外限制」的出栈序列计数。题目一旦加条件（如禁止连续三次出栈、指定首元素），必须分类讨论。
21. 卡特兰数同时是「n 个结点的不同形态二叉树棵数」，两处计数问题通过「先序为入栈、中序为出栈」的对应关系统一。
22. 判断出栈序列合法性时，不要只看首尾元素。必须检查「跨越逆序」关系，或者直接用模拟法逐元素走一遍。

第四章树与二叉树

本章考纲要求

四、树与二叉树（一）树的基本概念（二）二叉树

1. 二叉树的定义及其主要特性
2. 二叉树的顺序存储结构和链式存储结构
3. 二叉树的遍历
4. 线索二叉树的基本概念和构造

（三）树、森林

1. 树的存储结构
2. 森林与二叉树的转换
3. 树和森林的遍历

（四）树与二叉树的应用

1. 哈夫曼（Huffman）树和哈夫曼编码
2. 并查集及其应用
3. 堆及其应用

本章是数据结构部分分值最高、题型最全的一章：选择题几乎年年必考（性质计算、遍历序列、线索二叉树、哈夫曼编码、堆的调整），算法设计题也多次落在这一章（二叉树的递归遍历改造、带权路径长度、表达式树等）。「五个性质」和「遍历序列」是本章的两根主线，其余考点都可以挂在这两根主线上。

一、树的基本概念

（一）树的定义与术语

*1.*树的定义

树（**Tree**）是 n（$n \geq 0$）个结点的有限集合。当 $n = 0$ 时称为空树；当 $n > 0$ 时满足：

1. 有且仅有一个特定的结点称为根（**Root**）；
2. 其余结点可分为 m（$m > 0$）个互不相交的有限集合 $T_1, T_2, \cdots, T_m$，其中每个集合本身又是一棵树，称为根的子树（**Subtree**）。

定义本身是递归的，这决定了树上的算法几乎都用递归实现。

从图论角度看，树是连通无环图，也是 n 个顶点的极小连通图、极大无环图。因此：

树的边数与结点数

对含 n 个结点的树，边数恒为 $n-1$，即 $e = n - 1$。这条结论是后续所有计数性质的出发点。

*2.*结点分类

- 根结点：只有后继没有前驱，非空树中有且仅有一个。
- 叶子结点 / 终端结点：没有后继，度为 **0**。
- 分支结点 / 非终端结点：有后继，度大于 **0**。除根之外的 branch 结点也称内部结点。

易错

「叶子结点」等价于「度为 0 的结点」，但不等于「最后一层的结点」。完全二叉树中最后一层也可能有非叶结点（它还有下一层）；反过来，倒数第二层的结点也可能是叶子。

*3.*基本术语（考点密集）

术语	含义	说明
结点的度	该结点的孩子个数（子树个数、分支个数）	叶子结点的度为 0
树的度	树中所有结点度的最大值	度为 m 的树 $\Leftrightarrow$ 树中至少有一个结点的度为 m
孩子 / 双亲	某结点子树的根 / 对应的上层结点	除根外每个结点恰有一个双亲
兄弟	同一双亲的孩子互称兄弟	双亲在同一层的结点互称堂兄弟
祖先 / 子孙	从根到该结点路径上的所有结点 / 以该结点为根的子树中所有结点	
层次	根为第 1 层，根的孩子为第 2 层，依次类推	
结点的深度	从根到该结点路径上的结点个数，即该结点的层次	自顶向下累加
结点的高度	以该结点为根的子树的高度（该子树中最长路径上的结点数）	自底向上累加
树的高度 / 深度	树中结点的最大层次	根的高度 = 树的高度
有序树 / 无序树	各结点的子树从左到右有 / 无次序	二叉树是有序树
森林	m（$m \geq 0$）棵互不相交的树的集合	树去掉根就变成森林
路径	两结点间所经过的结点序列	树中路径自上向下，兄弟之间无路径
路径长度	路径上边的个数	不是结点个数

深度与高度的区分与联系

- 深度从根往下数，高度从叶子往上数，同一结点两者可以不等。
- 恒有 $height(v) + depth(v) \leq height(T)$，当且仅当 v 位于某条「根到最深叶子」的链上时取等号。
- 若 p 是 v 的双亲，则 $h(p) = \max\{h(\text{所有子结点})\} + 1$，故 $h(p) \geq h(v) + 1$。
- 树的路径长度指从根到每个结点的路径长度之和，其最大值等于「树高 -1」。它与哈夫曼树的带权路径长度 **WPL** 是两个完全不同的概念，不要混用。

真题考法

直接用「深度 / 高度 / 层次」的定义设陷阱：给出某结点的层次，要求最深叶子的层次；或给出一棵树的各度结点数，反求叶子结点数（2010 年统考的经典形式）。此外「路径长度」默认按边数计，若题目按结点数计会明确说明。

（二）树的性质

*1.*结点数与度数的关系

设度为 i 的结点数为 n_i（$i=0,1,\cdots,m$），n 为结点总数。

树的第一性质（最常用）

树中结点总数 = 所有结点的度数之和 + **1**。

$$n=\sum_{i=0}^{m} n_i=\sum_{i=1}^{m} i\cdot n_i+1$$

理由：除根结点外，每个结点恰好有一条边连到它的双亲，故边数 $e=n-1$；另一方面每个度为 i 的结点贡献 i 条边，故 $e=\sum_{i=1}^{m} i\cdot n_i$。两者联立即得。

算叶结点数的标准步骤：

1. 用非叶结点算出总边数 $e=\sum_{i\geq 1} i\cdot n_i$；
2. 由 $n=e+1$ 得总结点数 n；
3. $n_0=n-\sum_{i\geq 1} n_i$。

例：某度为 4 的树中，度为 4、3、2、1 的结点数依次为 20、10、1、10，求叶子结点数。

边数 $e=20\times 4+10\times 3+1\times 2+10\times 1=80+30+2+10=122$，故 $n=122+1=123$。非叶结点共 $20+10+1+10=41$ 个，所以 $n_0=123-41=82$。

*2.*每层最多多少个结点

度为 m 的树第 i 层最多 m^{i-1} 个结点

第 1 层只有根，$1=m^0$；第 2 层最多 m 个；第 3 层最多 m^2 个……由归纳即得第 i 层最多 m^{i-1} 个结点（$i\geq 1$）。m 叉树同理。

*3.*高度为 h 的 m 叉树最多 / 最少多少个结点

容量与下界

- 高度为 h 的 m 叉树最多有 $\dfrac{m^h-1}{m-1}$ 个结点（等比求和 $m^0+m^1+\cdots+m^{h-1}$）。当 $m=2$ 时即 2^h-1。
- 高度为 h 的 m 叉树最少有 h 个结点（每层只挂 1 个，退化成链）。
- 高度为 h、度为 m 的树最少有 $h+m-1$ 个结点（前 $h-1$ 层各 1 个，最后一层挂满 m 个孩子）。

*4.*具有 n 个结点的 m 叉树的最小高度

最小高度公式

$$h_{\min}=\left\lceil \log_m\big(n(m-1)+1\big)\right\rceil$$

推导：要让 n 个结点装进尽量矮的树，就应让每层都放满。高度为 h 时最多容纳 $\dfrac{m^h-1}{m-1}$ 个结点，故必须有

$$n\leq\frac{m^h-1}{m-1}\implies n(m-1)\leq m^h-1\implies m^h\geq n(m-1)+1$$

两边取以 m 为底的对数得 $h \geq \log_m\big(n(m-1)+1\big)$；$h$ 是整数，取下界即得上式。取 $h_0 = \lceil \log_m(n(m-1)+1) \rceil$ 后按层序填满即可达到 h_0，故下界可达。

验证：$m=3$、$n=244$ 时，$n(m-1)+1 = 244\times 2+1 = 489$，$\log_3 489 \approx 5.65$，取整得 $h_{\min}=6$。（也可直接数：高度 5 的满三叉树最多 $\frac{3^5-1}{2}=121<244$；高度 6 的满三叉树最多 $\frac{3^6-1}{2}=364\geq 244$。）

当 $m=2$、$n=1000$ 时，$n(m-1)+1=1001$，$\log_2 1001 \approx 9.97$，$h_{\min}=10$。

*5.*具有 n 个结点的度为 m 的树的最大高度

最大高度为 $n-m+1$

除一个结点的度为 m 外，其余 $n-m$ 个结点的度均为 1，这样树能拉得最长：

$$h_{\max} = (n-m)+1 = n-m+1$$

*6.*度为 m 的树与 m 叉树的区别（高频陷阱）

对比项	度为 m 的树	m 叉树
结点度的上限	不超过 m	不超过 m
是否必须有度为 m 的结点	必须至少有一个	允许所有结点度都 $<m$
能否为空	不能，至少有 $m+1$ 个结点	可以是空树

易错

- 「度为 m 的树」是非空树，因为「度为 m」本身就要求存在一个度为 m 的结点。
- 「m 叉树」可以是空树，所以 n 个结点的 m 叉树高度可以为 0（空树）——但统考一般默认非空，遇到「含 n 个结点」时 $n\geq 1$。
- m 叉树（有序）与 m 叉树形（不考虑左右次序）在计数问题中结果不同。

7.n 个结点能构成多少种不同形态的二叉树 / 树

卡特兰数

- 含 n 个结点的不同形态二叉树数目为第 n 个卡特兰数：

$$b_n = \frac{1}{n+1}C_{2n}^{n} = \frac{(2n)!}{n!(n+1)!}$$

- 含 n 个结点的不同形态树（普通有序树）数目为 $t_n = b_{n-1} = \frac{1}{n}C_{2n-2}^{n-1}$。

  推导（二叉树的递推）：n 个结点的二叉树可看成「1 个根 + i 个结点的左子树 + $n-i-1$ 个结点的右子树」，于是

$$b_0 = 1, \qquad b_n = \sum_{i=0}^{n-1} b_i b_{n-i-1}$$

  代入得 $b_1=1$，$b_2=2$，$b_3=5$，$b_4=14$，$b_5=42$。

  推导（树的递推）：任意一棵普通树经「左孩子右兄弟」表示后必对应一棵根无右子树的二叉树；而 n 个结点的树去掉根后剩下 $n-1$ 个结点，所以 $t_n = b_{n-1}$。

真题考法

「先序序列为 a,b,c,d 的不同二叉树个数」这类题，因为固定先序后每一种树形唯一对应一种填入方式，答案就是结点数对应的卡特兰数（$n=4$ 时为 14）。注意区分问的是二叉树还是普通树，后者要用 $t_n = b_{n-1}$。

二、二叉树

（一）二叉树的定义及其主要特性

*1.*定义

二叉树（**Binary Tree**）是 n（$n \geq 0$）个结点的有限集合。它或为空树，或由一个根结点及两棵互不相交的、分别称为左子树和右子树的二叉树组成。

二叉树 $\neq$ 度为 2 的有序树

- 二叉树可以为空，度为 2 的树不能为空；
- 二叉树的每个结点至多有两棵子树，且左右次序严格区分，即使某结点只有一个孩子，这个孩子也必须明确是左孩子还是右孩子；
- 度为 2 的有序树中，若某结点只有一个孩子，就不存在「左右次序」问题。

这条差别是「先序 + 后序能否唯一确定二叉树」等问题的根源。

二叉树的 5 种基本形态：空树、只有根、根 + 左子树、根 + 右子树、根 + 左右子树。

*2.*几种特殊的二叉树

满二叉树（**Perfect / Full Binary Tree**）：高度为 h 且含 $2^h - 1$ 个结点的二叉树，即每一层都填满。按层序从 1 开始编号后，编号 i 的左右孩子为 $2i$、$2i+1$，双亲为 $\lfloor i/2 \rfloor$。

完全二叉树（**Complete Binary Tree**）：结点编号与同高度满二叉树的编号一一对应（即只允许最下面一层从右往左缺结点）。其特点：

- 只有最下面两层可能出现叶子结点；
- 最多只有一个度为 **1** 的结点，且该结点只有左孩子；
- 按层序从 1 编号后，$i \leq \lfloor n/2 \rfloor$ 的结点是分支结点，$i > \lfloor n/2 \rfloor$ 的结点是叶子结点。

满二叉树与完全二叉树的关系

满二叉树一定是完全二叉树；完全二叉树不一定是满二叉树。二者都适合顺序存储。

二叉排序树 / 二叉搜索树（**BST**）：左子树上所有结点关键字 < 根，右子树上所有结点关键字 > 根，左右子树又各是 BST。中序遍历得到递增有序序列。（详见 D06 查找章。）

平衡二叉树（**AVL**）：任一结点的左、右子树高度之差的绝对值不超过 1。（详见 D06 查找章。）

真二叉树 / 严格二叉树 / 正则二叉树：不存在度为 1 的结点，即结点的度只能是 0 或 2。此时 $n_2 = n_0 - 1$，即内部结点数比叶子结点数少 1。哈夫曼树一定是真二叉树，但不一定是完全二叉树。

*3.*二叉树的五个重要性质（全部推导）

性质 1：第 i 层最多 2^{i-1} 个结点

$i=1$ 时只有根，$1 = 2^0$ 个，成立。设第 i 层最多 2^{i-1} 个，则因每个结点至多两个孩子，第 $i+1$ 层最多 $2 \cdot 2^{i-1} = 2^i$ 个。由数学归纳法得第 i 层最多 2^{i-1} 个结点（$i \geq 1$）。

性质 2：高度为 h 的二叉树最多 2^h-1 个结点

把各层最大值相加（等比数列）：

$$1+2+4+\cdots+2^{h-1}=\frac{2^h-1}{2-1}=2^h-1$$

恰好达到这个上界的二叉树就是满二叉树。反之，高度为 h 的二叉树至少有 h 个结点（每层至少 1 个，退化成单链）。

性质 3：$n_0=n_2+1$

设度为 0、1、2 的结点数分别为 n_0,n_1,n_2，则

- 结点总数：$n=n_0+n_1+n_2$；
- 分支（边）总数：由每个度为 i 的结点引出 i 条分支，得 $e=n_1+2n_2$；
- 又因除根外每个结点恰有一条来自双亲的边，$e=n-1$。

于是 $n_1+2n_2=n_0+n_1+n_2-1$，两边消去 n_1+n_2 得

$$n_0=n_2+1$$

这条性质对任何非空二叉树都成立，是换算 n_0、n_1、n_2 的核心。

性质 4：完全二叉树的编号关系

完全二叉树按层序从 **1** 开始编号为 $1\sim n$，则对编号 i 的结点：

关系	从下标 **1** 开始	从下标 **0** 开始	存在条件
双亲	$\lfloor i/2\rfloor$	$\lfloor (i-1)/2\rfloor$	非根结点
左孩子	$2i$	$2i+1$	$<n$（1 起）/ $\le n-1$（0 起）
右孩子	$2i+1$	$2i+2$	$<n$（1 起）/ $\le n-1$（0 起）
分支结点判据	$i\le\lfloor n/2\rfloor$	$i\le\lfloor n/2\rfloor-1$	——
叶子结点判据	$i>\lfloor n/2\rfloor$	$i>\lfloor n/2\rfloor-1$	——

推导：按层序编号时，第 i 层有 2^{i-1} 个结点，其编号区间为 $[2^{i-1},2^i-1]$。编号 j 的结点之前共有 $j-1$ 个结点，其中叶子之外的第 k 层（$k\le$ 该结点所在层 -1）的结点按「每个分支结点贡献两个孩子」依次编号，逐层推算即得 $j=2i$、$2i+1$、$\lfloor j/2\rfloor=i$ 的对应。$i=\lfloor n/2\rfloor$ 是最后一个有孩子的结点，故 $i>\lfloor n/2\rfloor$ 者必为叶子。

若数组下标从 **0** 开始存储，则编号为 i 的结点：双亲 $\lfloor (i-1)/2\rfloor$，左孩子 $2i+1$，右孩子 $2i+2$。

性质 5：n 个结点的完全二叉树高度为 $\lceil\log_2(n+1)\rceil$

推导：完全二叉树前 $h-1$ 层是满的（共 $2^{h-1}-1$ 个结点），第 h 层至少有 1 个结点，故

$$n_{\min}=(2^{h-1}-1)+1=2^{h-1}$$

当第 h 层也填满时取得最大结点数 $n_{\max}=2^h-1$。因此

$$2^{h-1}\le n\le 2^h-1$$

由左边得 $2^{h-1}\le n<2^h$，取对数 $h-1\le\log_2 n<h$，因 $h-1$ 是整数，故 $h=\lfloor\log_2 n\rfloor+1$。由右边得 $2^{h-1}<n+1\le 2^h$，取对数 $h-1<\log_2(n+1)\le h$，故 $h=\lceil\log_2(n+1)\rceil$。

结论：

$$h = \lfloor \log_2 n \rfloor + 1 = \lceil \log_2 (n+1) \rceil$$

两个式子恒等，考试用哪个都可以。注意这个结论只对完全二叉树成立；一般二叉树只知道 n 时，$h_{\max} = n$（单链），$h_{\min} = \lceil \log_2 (n+1) \rceil$。

验证：$n = 768$ 时，$\lfloor \log_2 768 \rfloor + 1 = \lfloor 9.58 \rfloor + 1 = 9 + 1 = 10$，而 $2^9 - 1 = 511 < 768 \le 1023 = 2^{10} - 1$，一致。

*4.*完全二叉树的 n_0、n_1、n_2 速算

由 n 直接推出 n_0, n_1, n_2

完全二叉树中 $n_1 \in \{0, 1\}$，结合 $n_0 = n_2 + 1$ 得 $n_0 + n_2 = 2n_2 + 1$ 必为奇数：

- 若 $n = 2k$（偶数）：$n_1 = 1$，$n_0 = k$，$n_2 = k - 1$；
- 若 $n = 2k - 1$（奇数）：$n_1 = 0$，$n_0 = k$，$n_2 = k - 1$。

验证：$n = 768 = 2 \times 384$，故 $n_0 = 384$，$n_1 = 1$，$n_2 = 383$，$384 = 383 + 1$ 成立。

*5.*二叉树的空指针域个数

二叉链表有 $n + 1$ 个空指针域

含 n 个结点的二叉链表共有 $2n$ 个指针域；除根结点外，每个结点恰好被一个指针指向，即非空指针数为 $n - 1$。故空指针域个数为

$$2n - (n - 1) = n + 1$$

这条结论是线索二叉树存在的依据：用这 $n + 1$ 个空域存放前驱 / 后继线索。

真题考法

- 由完全二叉树的「第 k 层有 m 个叶结点」求结点总数（2009 年形式）：要让总数最大就把第 k 层剩下的结点都补上两个孩子，算出第 $k + 1$ 层最多结点数再相加；要让总数最小就让叶结点尽量靠右。
- 由 n 直接求 n_0（2011 年形式）：直接用上面的 $n = 2k$ / $n = 2k - 1$ 结论。
- 「非空完全二叉树所有叶结点同层且非叶结点都有 2 个孩子」（2018 年形式）：这实际是一棵满二叉树，于是 $n = 2n_0 - 1$。

（二）二叉树的顺序存储结构和链式存储结构

*1.*顺序存储结构

用一维数组按层序依次存放结点，结点间的父子关系由下标计算隐含给出。

顺序存储的下标约定

从下标 **1** 开始（统考、教材通用）：

- 结点 i 的双亲 $= \lfloor i/2 \rfloor$，左孩子 $= 2i$，右孩子 $= 2i + 1$；
- 若 $i > n$ 或该位置为空，则对应孩子不存在。

从下标 **0** 开始（C 语言数组天然形式）：

- 结点 i 的双亲 $= \lfloor (i - 1)/2 \rfloor$，左孩子 $= 2i + 1$，右孩子 $= 2i + 2$。

必须显式标记空结点才能唯一表示一棵二叉树：因为层序序列不包含空结点信息，同一个层序序列

可能对应多棵不同的二叉树。常用做法是把每个数组元素设计为带 isEmpty 标记的结点：

```
typedef struct TreeNode {
    int data;        // 结点数据元素
    bool isEmpty;    // 该位置是否为空结点
} TreeNode;

// 初始化：全部标记为空
void InitSqBiTree(TreeNode t[], int length) {
    for (int i = 0; i < length; i++)
        t[i].isEmpty = true;
}

// 判断下标为 index 的结点是否为空（下标从 1 开始）
bool isEmpty(TreeNode t[], int length, int index) {
    if (index >= length || index < 1) return true;
    return t[index].isEmpty;
}

// 取下标为 index 的结点的左孩子下标，无则返回 -1
int getLchild(TreeNode t[], int length, int index) {
    int lChild = index * 2;
    if (isEmpty(t, length, lChild)) return -1;
    return lChild;
}

// 取下标为 index 的结点的右孩子下标，无则返回 -1
int getRchild(TreeNode t[], int length, int index) {
    int rChild = index * 2 + 1;
    if (isEmpty(t, length, rChild)) return -1;
    return rChild;
}

// 取下标为 index 的结点的双亲下标，无则返回 -1
int getFather(TreeNode t[], int length, int index) {
    if (index == 1) return -1;              // 根结点无双亲
    int father = index / 2;                 // 整数除法自动向下取整
    if (isEmpty(t, length, father)) return -1;
    return father;
}

// 顺序存储二叉树的先序遍历
void PreOrderSqTree(TreeNode *t, int length, int index) {
    if (isEmpty(t, length, index)) return;
    visitNode(t[index]);
    PreOrderSqTree(t, length, getLchild(t, length, index));
    PreOrderSqTree(t, length, getRchild(t, length, index));
}
```

顺序存储的优缺点

优点：连续内存、缓存友好、不需存指针、支持随机访问（含完全二叉树的 $O(1)$ 找双亲）。

缺点：需要连续空间；插入删除代价高；稀疏时空间浪费严重。

关键结论：顺序存储只适合完全二叉树（含满二叉树）——因为空结点只出现在最底层右侧，可省略不存，n 个结点恰好占 n 个单元。一般二叉树最坏情况下，一棵高度为 h 的单支树只有 h 个结点，却要占据 2^h-1 个单元。

易错：「需要的存储单元数量至少是多少」

题目给定结点数和高度、要求用顺序存储，问最少需要多少单元时：由于树的形态未知，必须按最坏情况考虑，即最深结点恰好落在编号最大的位置上。此时数组长度由高度决定，为 2^h-1（从 1 编号）或 2^h-1 个单元（从 0 编号时需能存到下标 2^h-2，仍为 2^h-1 个单元）。

例：高度为 5、10 个结点的任意二叉树用顺序存储，最少需要 $2^5-1=31$ 个存储单元（2020 年形式）。这里的答案是 **31**，而不是 **10** 或 **15**。

易错：从数组反推二叉树是否合法

若给定数组（含 -1 表示空）问「能否表示一棵二叉树」，判据是：每个非空结点的双亲位置也必须非空。若某结点下标 i 的值非空，而 $\lfloor (i-1)/2 \rfloor$ 或 $\lfloor i/2 \rfloor$ 处为空，就出现了「悬空结点」，不合法。

例（从 0 开始编号）：`R[] = {17,20,35,-1,18,45,-1,-1,19,27}` 中下标 8 的结点 19，其双亲下标 $\lfloor (8-1)/2 \rfloor = 3$，而 `R[3] = -1` 为空，故 19 悬空，该数组不能表示二叉树（2025 年形式）。

*2.*链式存储结构

二叉链表：数据域 + 左指针 + 右指针。

```
typedef struct TreeNode {
    int data;                           // 数据域
    struct TreeNode *lchild, *rchild;   // 左、右孩子指针
} TreeNode, *BiTree;
```

三叉链表：在二叉链表上增加一个指向双亲的指针，便于找双亲。

```
typedef struct TreeNode {
    int data;                           // 数据域
    struct TreeNode *lchild, *rchild;   // 左、右孩子指针
    struct TreeNode *parent;            // 双亲指针
} TreeNode, *BiTree;
```

三种链式结构对比

结构	域	左右指针域中的空域数	找双亲	全部指针域中的空域数
二叉链表	data, lchild, rchild	$n+1$	需从根遍历	$n+1$
三叉链表	data, lchild, rchild, parent	$n+1$	$O(1)$	$2n+1$
线索链表	data, lchild, rchild, ltag, rtag	线索化后为 0	中序线索可 $O(1)$ 找前驱后继	——

三叉链表共 $3n$ 个指针域，非空 $n-1$ 个，空 $3n-(n-1)=2n+1$。注意：考试若只问「`lchild`、`rchild` 中的空指针」，答案仍是 $n+1$（与二叉链表相同），因为多出来的 `parent` 域不参与计数。

真题考法

本考点以「求空指针域个数」「求找双亲的最坏时间」「给出一段找兄弟 / 找堂兄弟的操作问复杂度」等形式出现。没有双亲指针时，找兄弟必须先从根遍历到双亲，这是常见的复杂度设陷阱点。

（三）二叉树的遍历

*1.*遍历的定义与四种方式

遍历：从根出发，按某种次序访问树中所有结点，使每个结点被访问一次且仅一次。

以 N 表示根、L 表示左子树、R 表示右子树，「前 / 中 / 后」指的是根在遍历中的位置：

遍历	次序	口诀
先序（前序）	$N \to L \to R$	根在最前
中序	$L \to N \to R$	根在中间
后序	$L \to R \to N$	根在最后
层次	自上而下、自左而右逐层访问	用队列

前三种属于深度优先遍历（**DFS**），可用递归或栈实现；层次遍历属于广度优先遍历（**BFS**），用队列实现。

四条速记结论

- 先序首个 = 后序末个 = 层次首个 = 根结点；
- 中序序列中，根左边的全部属于左子树，右边全部属于右子树；
- 先序 + 中序、后序 + 中序、层次 + 中序均可唯一确定二叉树；先序 + 后序不行；
- 一棵二叉树的中序序列唯一，但不同的二叉树可能有相同的中序序列（如先序 + 后序的组合）。

*2.*递归遍历（三种）

```
// 先序遍历：根 左 右
void PreOrder(BiTree T) {
    if (T != NULL) {
        visit(T);             // 访问根结点
        PreOrder(T->lchild); // 递归遍历左子树
        PreOrder(T->rchild); // 递归遍历右子树
    }
}

// 中序遍历：左 根 右
void InOrder(BiTree T) {
    if (T != NULL) {
        InOrder(T->lchild);
        visit(T);
        InOrder(T->rchild);
    }
}

// 后序遍历：左 右 根
void PostOrder(BiTree T) {
    if (T != NULL) {
        PostOrder(T->lchild);
        PostOrder(T->rchild);
        visit(T);
    }
}
```

递归遍历的时空复杂度

- 时间：$O(n)$，每个结点恰好访问一次。
- 空间：$O(h)$，即递归工作栈的深度。最好情况（平衡 / 完全二叉树）$h=\lceil \log_2(n+1) \rceil = O(\log_2 n)$；最坏情况（单支链）$h=n=O(n)$。

注意空间复杂度考的是辅助栈而非结点本身，故应为 $O(h)$，只有最坏时才写成 $O(n)$。

*3.*非递归遍历（栈模拟）

核心思想

递归调用的本质是「把局部变量、参数、返回地址压入调用栈」。因此用显式栈完全可以模拟递归：递归隐式维护一个栈，迭代显式维护一个栈，两者等价。

- 先序：访问与处理顺序一致（都是根），代码最简——出栈即访问，然后把右孩子先入栈、左孩子后入栈（保证出栈顺序为「左、右」）。
- 中序：访问顺序（先到最左下）与处理顺序（根在中间）不一致，必须用一个指针 `cur` 负责「访问」、用栈负责「处理」。
- 后序：最难，因为从左子树返回时无法判断右子树是否已访问过，需增加 `pre` 指针记录上一个被访问的结点。

先序遍历（迭代）：

```
void PreOrderIter(BiTree T) {
    if (T == NULL) return;
    BiTree stack[MaxSize];
    int top = -1;
    stack[++top] = T;
    while (top >= 0) {
        BiTree p = stack[top--];   // 出栈即访问
        visit(p);
        if (p->rchild != NULL) stack[++top] = p->rchild; // 右孩子先入栈
        if (p->lchild != NULL) stack[++top] = p->lchild; // 左孩子后入栈
    }
}
```

中序遍历（迭代）：

```
void InOrderIter(BiTree T) {
    BiTree stack[MaxSize];
    int top = -1;
    BiTree cur = T;
    while (cur != NULL || top >= 0) {
        if (cur != NULL) {
            stack[++top] = cur;      // 一路向左，沿途压栈
            cur = cur->lchild;
        } else {
            cur = stack[top--];      // 左子树走完，弹栈处理
            visit(cur);
            cur = cur->rchild;       // 转向右子树
        }
    }
}
```

后序遍历（迭代，用 `pre` 记录历史）：

```
void PostOrderIter(BiTree T) {
    BiTree stack[MaxSize];
    int top = -1;
    BiTree cur = T, pre = NULL;
    while (cur != NULL || top >= 0) {
        while (cur != NULL) {          // 步骤 1：沿左孩子一路入栈
            stack[++top] = cur;
            cur = cur->lchild;
        }
        cur = stack[top--];            // 步骤 2：弹出栈顶，其左子树必已访问完
        // 步骤 3：无右子树，或右子树刚被访问完（pre 指向右孩子），则可访问 cur
        if (cur->rchild == NULL || pre == cur->rchild) {
            visit(cur);
            pre = cur;
            cur = NULL;                // 防止再次进入步骤 1
        } else {
            stack[++top] = cur;        // 右子树未访问，cur 重新入栈
            cur = cur->rchild;         // 先去处理右子树
        }
    }
}
```

后序遍历的取巧写法：先按「根 → 右 → 左」的顺序遍历（即把先序代码中左右孩子入栈次序互换），得到「NRL」序列，再把结果逆序，即得「LRN」。若不允许用逆序函数，可再设一个辅助栈，访问时压入辅助栈，遍历结束后依次出栈。

后序非递归遍历的重要性质

在上述后序迭代过程中，当访问到结点 p 时，栈中恰好是 p 的全部祖先结点，且从栈底到栈顶再加上 p 本身，恰好构成从根到 p 的路径。

这条性质可用于：求根到某结点的路径、求二叉树的最近公共祖先、求根到叶结点的数字之和。

*4.*层次遍历（队列）

```
void LevelOrder(BiTree T) {
    if (T == NULL) return;
    BiTree Q[MaxSize];
    int front = 0, rear = 0;
    Q[rear++] = T;                  // 根结点入队
    while (front != rear) {
        BiTree p = Q[front++];      // 队头出队
        visit(p);
        if (p->lchild != NULL) Q[rear++] = p->lchild; // 左孩子入队
        if (p->rchild != NULL) Q[rear++] = p->rchild; // 右孩子入队
    }
}
```

若要按层分行输出，可在每轮循环开始时记录 `size = rear - front`，只处理这 `size` 个结点。

完全二叉树层次遍历的队列规模

完全二叉树做层次遍历时，辅助队列的最大规模等于叶子结点个数 $= \lceil n/2 \rceil$。也就是说，队列容量只要开到叶子结点数就够用。

队列规模在整个遍历过程中是先增后减、单峰对称的：从 0 开始逐层上升到达峰值，再逐层回落到 0。峰值出现在最宽的一层（即最后一层）的结点全部入队、而它们尚未出队完毕的时刻。

5. 由遍历序列构造二叉树

唯一性总表

序列组合	能否唯一确定	原因
先序 + 中序	能	先序定根，中序分左右
后序 + 中序	能	后序定根，中序分左右
层次 + 中序	能	层次定根，中序分左右
先序 + 后序	不能	单孩子结点无法区分左右
层次 + 先序	不能	同上
层次 + 后序	不能	同上
先序 + 后序（严格二叉树）	能	无单孩子结点，歧义消失
层次 + 先序（完全二叉树）	能	单孩子必为左孩子，形状受限

（1）先序 + 中序

构造步骤（设结点值互不相同）：

1. 先序序列的第一个元素是当前子树（初始为整棵树）的根；
2. 在中序序列中定位该根，根左边的元素构成左子树的中序序列，右边的构成右子树的中序序列；
3. 由左子树结点个数 k，把先序序列切成「根 | 左子树先序（k 个）| 右子树先序」；
4. 对左右子树递归执行上述过程。

关键一步

第 3 步「用左子树的结点数去切分先序序列」是本题型的得分点。很多同学会漏掉这一步，只在中序里切分，导致先序无法继续划分。

例：先序 A B D E C F G，中序 D B E A F C G。

- 先序首元素 A 是根；中序中 A 的位置把序列分为 D B E 与 F C G；
- 左子树 3 个结点，故先序中 A 之后 3 个 B D E 属左子树，其余 C F G 属右子树；
- 递归：左子树先序 B D E、中序 D B E → 根 B，左 D 右 E；右子树先序 C F G、中序 F C G → 根 C，左 F 右 G。

（2）后序 + 中序

与上一情形完全对称，只是根取后序序列的最后一个元素：

1. 后序序列的最后一个元素是根；
2. 在中序中定位根，划分左右子树的中序序列；
3. 由左子树结点个数 k，把后序序列切成「左子树后序（k 个）| 右子树后序 | 根」；
4. 递归。

易错

「后序 + 中序」中，左子树的后序序列在下标上位于前面，右子树紧随其后。如果切分时把右子树放在前面就会全错。建议每次切分后手工验证一次后序（左子树后序 + 右子树后序 + 根是否等于原序列）。

（3）层次 + 中序

1. 层次序列的第一个元素是整棵树的根；
2. 在中序序列中定位该根，根左边是左子树全部结点，右边是右子树全部结点；
3. 从层次序列中筛选出属于左子树的结点（保持原有相对顺序），得到左子树的层次序列；同理得到右子树的层次序列；
4. 对左右子树递归。

为什么必须「重新筛选」

层次序列是整棵树的层次序，左右子树的结点在原序列中是交错出现的，必须各自挑出来组成新的层次序列，才能继续递归。每一步挑出后，新层次序列的第一个元素就是该子树的根。

例：层次 A B C D E F，中序 D B E A F C。

- 根为 A；中序分左右：左 D B E，右 F C；
- 从左子树结点集 $\{D, B, E\}$ 在原层次序列中筛选，得到左子树层次 B D E；右子树层次 C F；
- 左子树：层次首元素 B 为根，中序 D B E 中 B 居中 $\rightarrow$ 左 D 右 E；
- 右子树：层次首元素 C 为根，中序 F C 中 C 末尾 $\rightarrow$ 只有左孩子 F。

(4) 先序 + 后序为什么不能唯一确定

根本原因：单孩子子树无法区分左右

- 先序去掉根后是「左子树先序 + 右子树先序」；
- 后序去掉根后是「左子树后序 + 右子树后序」；
- 但你不知道左子树有几个结点，无法据此切分。

当某个结点同时有左右两个孩子时，才能通过「先序的第二个元素 = 左子树根」「后序的倒数第二个元素 = 右子树根」确定分界。一旦某结点只有一个孩子，该孩子在左在右都不改变先序和后序的结果，歧义就产生了。

最小反例：树 1 中 B 是 A 的左孩子且 C 是 B 的右孩子；树 2 中 B 是 A 的右孩子且 C 是 B 的左孩子。两棵树的先序都是 A B C，后序都是 C B A，但结构不同。

什么时候先序 + 后序能唯一确定

在结点值互不相同、且不附加形状限制时：当且仅当该二叉树是严格二叉树（真二叉树，每个结点度为 **0** 或 **2**），先序 + 后序才能唯一确定；同理，层次 + 先序、层次 + 后序也只有在这种情况下能唯一。

若附加形状约束，条件可以放宽：例如完全二叉树中单孩子只能是左孩子，形状约束本身就消除了歧义，所以「层次 + 先序」对完全二叉树可唯一确定。

典型应用：表达式树是一棵真二叉树，因此给出它的先序（前缀表达式）和后序（后缀表达式）就能唯一还原它。

*6.*遍历序列相等的条件（高频小题）

序列相同 $\Leftrightarrow$ 形状受限

条件	等价的结构要求
先序 = 中序	每个非叶结点只有右子树（不存在左孩子）
后序 = 中序	每个非叶结点只有左子树（不存在右孩子）
先序 = 后序	树中只有一个结点（$n = 1$）
先序 = 层次	每个非叶结点只有右子树，且树是单链

条件	等价的结构要求
中序 = 层次	每个非叶结点只有右子树（单链）

先序 = 中序的推导：先序为 N L R，中序为 L N R，二者唯一的差别是根与左子树的先后。若某非叶结点有左子树，先序先访问该结点再访问左子树，中序先访问左子树再访问该结点，两序列必然不同。故所有非叶结点都不能有左子树，只能有右子树。

注意：「所有结点的度均为 **1**」是不够的——度为 1 还可能包含「只有左孩子」的情况，此时先序与中序不再相同。

*7.*中序序列相邻结点的结构含义

中序前驱 / 后继的结构性质

- 若 v 有左子树，则 v 的中序直接前驱是 v 左子树中最右下的结点，且该结点必无右孩子；
- 若 v 有右子树，则 v 的中序直接后继是 v 右子树中最左下的结点，且该结点必无左孩子。

推导：中序按「左—根—右」进行，访问 v 之前的最后一个结点必在左子树中，且是「最后被访问」的那个，即最右下的结点；若它还有右孩子，则中序还要先访问它的右子树，它就不可能紧邻 v。

这条性质是 **2024** 年真题「中序序列形如⋯**,p,v,q,**⋯，v 有两个孩子」的解答关键：由 v 有左子树知 p 是左子树中最右下结点，故 p 无右孩子；由 v 有右子树知 q 是右子树中最左下结点，故 q 无左孩子。

（四）线索二叉树

*1.*基本概念

在含 n 个结点的二叉链表中，有 $n+1$ 个空指针域。传统二叉链表只能体现父子关系，求某结点的前驱 / 后继必须从根重新遍历。线索二叉树就是把这些空指针利用起来，指向该结点在某种遍历次序下的前驱 / 后继。

规定：

- 若无左子树，令 `lchild` 指向其前驱；
- 若无右子树，令 `rchild` 指向其后继。

为了区分指针指向的是孩子还是线索，需增加两个标志域：

标记	值	含义
`ltag`	0	`lchild` 指向左孩子
`ltag`	1	`lchild` 指向前驱
`rtag`	0	`rchild` 指向右孩子
`rtag`	1	`rchild` 指向后继

```
typedef struct ThreadNode {
    int data;                                // 数据域
    struct ThreadNode *lchild, *rchild;  // 左、右指针
    int ltag, rtag;                          // 左、右线索标志
} ThreadNode, *ThreadTree;
```

三个必背结论

- 加上线索的二叉链表称为线索链表，对应的二叉树称为线索二叉树；
- 线索化的实质是在遍历过程中把空指针改为前驱 / 后继线索，所以线索化的过程就是遍历的过程；
- 线索二叉树类似于一个非循环的双向链表：遍历序列的第一个结点前驱指向 NULL，最后一个结点后继指向 NULL（不带头结点时）。

*2.*线索化的构造过程

核心技巧：设两个指针 pre（刚访问过的结点）与 p（正在访问的结点），则 **pre** 是 **p** 的前驱。遍历到 p 时：

- 若 p->lchild == NULL，把 p->lchild 指向 pre，p->ltag = 1；
- 若 pre->rchild == NULL，把 pre->rchild 指向 p，pre->rtag = 1。

中序线索化：

```
// pre 为全局或引用参数，始终指向刚访问过的结点
void InThread(ThreadNode *p, ThreadNode *&pre) {
    if (p != NULL) {
        InThread(p->lchild, pre);            // 递归线索化左子树
        if (p->lchild == NULL) {             // 左孩子为空，建立前驱线索
            p->lchild = pre;
            p->ltag = 1;
        } else {
            p->ltag = 0;
        }
        if (pre != NULL && pre->rchild == NULL) { // 前驱的右孩子为空，建立后继线索
            pre->rchild = p;
            pre->rtag = 1;
        } else if (pre != NULL) {
            pre->rtag = 0;
        }
        pre = p;                             // 更新 pre
        InThread(p->rchild, pre);            // 递归线索化右子树
    }
}

void CreateInThread(ThreadTree root) {
    ThreadNode *pre = NULL;
    if (root != NULL) {
        InThread(root, pre);
        pre->rchild = NULL;                  // 处理中序最后一个结点
        pre->rtag = 1;
    }
}
```

先序线索化：把「建线索 + 更新 pre」的部分提到递归之前，并且递归左子树前必须判断 **p->ltag == 0**：

```
void PreThread(ThreadNode *p, ThreadNode *&pre) {
    if (p != NULL) {
        if (p->lchild == NULL) { p->lchild = pre; p->ltag = 1; }
        else { p->ltag = 0; }
        if (pre != NULL && pre->rchild == NULL) { pre->rchild = p; pre->rtag = 1; }
        else if (pre != NULL) { pre->rtag = 0; }
        pre = p;
```

```
        if (p->ltag == 0) PreThread(p->lchild, pre);  // 必须判断，否则死循环
        if (p->rtag == 0) PreThread(p->rchild, pre);  // 必须判断，否则死循环
    }
}
```

先序线索化的死循环陷阱

先序线索化时，结点 p 的左指针可能刚被改成指向前驱 `pre`。如果此后不加判断就递归进入 `p->lchild`，程序会顺着线索走回祖先，形成死循环。所以先序线索化递归左右子树前必须先检查 `ltag` / `rtag`。中序线索化因为「建线索」发生在左递归之后，天然没有这个问题。

后序线索化：把「建线索 + 更新 pre」的部分放到左、右递归之后。递归左右子树时不需要判断标记（因为进入时线索尚未建立），但建立线索时要用 `pre`：

```
void PostThread(ThreadNode *p, ThreadNode *&pre) {
    if (p != NULL) {
        PostThread(p->lchild, pre);
        PostThread(p->rchild, pre);
        if (p->lchild == NULL) { p->lchild = pre; p->ltag = 1; }
        else { p->ltag = 0; }
        if (pre != NULL && pre->rchild == NULL) { pre->rchild = p; pre->rtag = 1; }
        else if (pre != NULL) { pre->rtag = 0; }
        pre = p;
    }
}
```

三种线索化只需改动两处：① 线索化改造代码段的位置；② 递归调用左右子树的位置。顺序为：

线索类型	代码顺序
先序	建线索 → 递归左 → 递归右
中序	递归左 → 建线索 → 递归右
后序	递归左 → 递归右 → 建线索

*3.*带头结点的线索二叉树

仿照带头结点的链表，可为线索二叉树增设一个头结点：

- 头结点的 `lchild` 指向二叉树的根结点（`ltag = 0`）；
- 头结点的 `rchild` 指向中序遍历最后一个访问的结点（`rtag = 1`）；
- 中序遍历第一个结点的 `lchild` 指向头结点；
- 中序遍历最后一个结点的 `rchild` 指向头结点。

这样整棵线索二叉树构成一个双向线索链表，可以方便地从任一点出发向两个方向遍历：从头结点的左孩子出发可正向中序遍历，从头结点出发沿 `rchild` 链可逆向遍历。

```
ThreadTree CreateInThreadWithHead(ThreadTree root) {
    ThreadNode *head = (ThreadNode *)malloc(sizeof(ThreadNode));
    head->ltag = 0;                  // 左指针指向根
    head->rtag = 1;                  // 右指针始终为线索
    head->rchild = head;             // 先初始化指向自己
```

```
    if (root != NULL) {
        head->lchild = root;          // 头结点的左指针指向根结点
        ThreadNode *pre = head;
        InThread(root, pre);          // 中序线索化
        pre->rchild = head;           // 最后一个结点的后继指向头结点
        pre->rtag = 1;
        head->rchild = pre;           // 头结点的后继指向最后一个结点
        head->rtag = 1;
    } else {
        head->lchild = head;          // 空树时左指针指向自己
    }
    return head;
}
```

易错

- 先序线索化不能用头结点的 `lchild` 作为 `pre` 的初值后再无条件递归，因为先序的第一个结点是根，前驱应为 `NULL`（或头结点）；
- 后序线索化中，若最后一个结点有右孩子，**`pre->rtag` 不能被设为 1**，否则线索化错误。处理「最后一个结点」时必须先判断其右指针是否为空。

*4.*在线索二叉树上找前驱 / 后继

中序线索二叉树：找后继

- 若 `p->rtag == 1`：后继就是 `p->rchild`；
- 若 `p->rtag == 0`：后继是其右子树中最左下的结点（右子树中第一个被中序访问的结点）。

中序线索二叉树：找前驱

- 若 `p->ltag == 1`：前驱就是 `p->lchild`；
- 若 `p->ltag == 0`：前驱是其左子树中最右下的结点（左子树中最后一个被中序访问的结点）。

```
// 以 p 为根的子树中，第一个被中序遍历访问的结点（最左下）
ThreadNode *FirstNode(ThreadNode *p) {
    if (p == NULL) return NULL;
    while (p->ltag == 0) p = p->lchild;   // 循环到最左下，不一定是叶结点
    return p;
}

// 以 p 为根的子树中，最后一个被中序遍历访问的结点（最右下）
ThreadNode *LastNode(ThreadNode *p) {
    if (p == NULL) return NULL;
    while (p->rtag == 0) p = p->rchild;
    return p;
}

// 中序线索二叉树中求 p 的中序后继
ThreadNode *NextNode(ThreadNode *p) {
    if (p->rtag == 0) return FirstNode(p->rchild); // 右子树最左下
    else return p->rchild;                          // 直接读线索
}

// 中序线索二叉树中求 p 的中序前驱
ThreadNode *PreNode(ThreadNode *p) {
    if (p->ltag == 0) return LastNode(p->lchild);   // 左子树最右下
```

```
    else return p->lchild;                          // 直接读线索
}

// 利用线索做中序遍历（非递归，空间 O(1)）
void InOrderThreadTree(ThreadNode *root) {
    for (ThreadNode *p = FirstNode(root); p != NULL; p = NextNode(p))
        visit(p);
}

// 逆向中序遍历
void RevInOrderThreadTree(ThreadNode *root) {
    for (ThreadNode *p = LastNode(root); p != NULL; p = PreNode(p))
        visit(p);
}
```

中序线索二叉树的最大优势

中序遍历不需要栈，空间复杂度为 $O(1)$（若不计线索化本身的额外标记域）。这是中序线索二叉树被广泛使用的原因。

先序线索二叉树：找后继

- 若 `p->ltag == 0`（有左孩子）：左孩子就是其后继；
- 若 `p->ltag == 1` 且 `p->rtag == 0`（无左孩子有右孩子）：右孩子就是其后继；
- 若 `p->ltag == 1` 且 `p->rtag == 1`（叶结点）：`p->rchild` 即线索，直接给出后继。

先序线索二叉树：找前驱

- 若 `p->ltag == 1`：前驱 = `p->lchild`；
- 若 `p->ltag == 0`：先序遍历的顺序是「根—左—右」，因此 `p` 的左右孩子都只能是 `p` 的后继，不可能是前驱。此时必须从头（根）重新开始遍历才能找到 `p` 的前驱。

先序线索二叉树找前驱的代价

先序线索二叉树不便于找前驱。若确实需要频繁求先序前驱，可把二叉链表改为三叉链表（增加 `parent` 域），此时分四种情况：

情况	p 的先序前驱
p 是根	无前驱
p 是双亲的左孩子	p 的双亲
p 是双亲的右孩子，且双亲无左孩子	p 的双亲
p 是双亲的右孩子，且双亲有左孩子	双亲左子树中最后一个被先序访问的结点（先下左、无左则下右，直至叶子）

后序线索二叉树：找前驱

- 若 `p->ltag == 1`：前驱 = `p->lchild`；
- 若 `p->ltag == 0`（p 必有左孩子）：
  - 若 p 有右孩子：后序前驱 = 右孩子（后序为「左、右、根」，最后访问完右子树再访问根）；
  - 若 p 无右孩子：后序前驱 = 左孩子。

后序线索二叉树：找后继

- 若 `p->rtag == 1`：后继 = `p->rchild`；
- 若 `p->rtag == 0`（p 有右孩子）：后序中左右子树中的结点都只能是 p 的前驱，不可能是后继，除非从根重新遍历，或改用三叉链表。

用三叉链表时，p 的后序后继分四种情况：

情况	p 的后序后继
p 是根	无后继
p 是双亲的右孩子	p 的双亲
p 是双亲的左孩子，且双亲无右孩子	p 的双亲
p 是双亲的左孩子，且双亲有右孩子	双亲右子树中第一个被后序访问的结点（有左走左、无左走右，直至叶子）
附：p 有右孩子（不论位置）	右子树中第一个被后序访问的结点（有左走左、无左走右）

后序线索二叉树遍历仍需栈

后序线索二叉树的遍历仍然需要栈。原因：后序遍历最后访问根结点，若从右孩子返回去访问双亲结点，由于双亲的右孩子指针不一定是空的（它可能指向真实的右孩子而非线索），无法用线索判断「是否已从右子树返回」，因而无法仅靠指针遍历整棵树。

三种线索二叉树的能力对比（必背）

能力	中序线索	先序线索	后序线索
找前驱	✓ 直接可得	× 需从头遍历 / 三叉链表	✓ 直接可得
找后继	✓ 直接可得	✓ 直接可得	× 需从头遍历 / 三叉链表
不用栈完成遍历	✓	✓	× 仍需栈

记忆口诀：「中序两全，先序缺前驱，后序缺后继」。

真题考法

- 给出二叉树的形状与遍历序列，问某结点的左右线索指向谁（2014 年形式）：先把对应的遍历序列完整写出，再看该结点在序列中的前后邻居，同时注意「有真实孩子的一侧不建线索」。
- 后序线索二叉树中叶结点 X 有左兄弟 Y，问 X 的右线索指向谁（2013 年形式）：X 是右孩子，后序遍历完 X 后两个子树都已处理完，下一个是双亲。
- 先序线索二叉树中非根叶结点 X 有左兄弟 Y，问 X 的左线索（先序前驱）指向谁：是以 Y 为根的子树中先序最后访问的结点。
- 后序线索二叉树中，X 是其双亲 p 的左孩子且 p 存在右子树，问 X 的右线索（后序后继）：p 的右子树中后序第一个访问的结点（从 p 的右孩子出发，有左走左、无左走右，直至叶结点），不是「最右下的结点」。

三、树、森林

（一）树的存储结构

树是一对多的逻辑结构，存储时既要保存数据，又要保存父子关系。三种表示法的本质区别只在于「每个结点额外记录谁」。

*1.*双亲表示法

用一维数组顺序存放全部结点，每个结点附一个 parent 域记录双亲在数组中的下标；根结点的 parent 约定为 -1。

```
#define MAX_TREE_SIZE 100

typedef struct {
    ElemType data;                  // 数据域
    int parent;                     // 双亲在数组中的下标
} PTNode;

typedef struct {
    PTNode nodes[MAX_TREE_SIZE];    // 结点数组
    int n;                          // 结点数
} PTree;
```

操作特性：

- 找双亲 $O(1)$，沿 parent 链可一路向上找到根——这正是并查集的基础；
- 找孩子 $O(n)$：需扫描整个数组看谁的双亲是自己。

删除结点的两种策略（被删结点为叶子；非叶需先递归删子树）：

策略	做法	时间复杂度	缺点
打标记	把 parent 改为 -2（区别于根的 -1）	$O(1)$	结点仍占数组空间
物理搬移	把数组尾部元素前移填补，并修正受影响结点的下标	$O(n)$	删除变慢

易错

双亲表示法靠 parent 域显式记录父子关系；二叉树的顺序存储靠下标计算隐含父子关系（结点 i 的左孩子为 $2i$）。一般树的孩子个数不固定，不能直接套用二叉树的顺序存储。

*2.*孩子表示法

把每个结点的所有孩子串成一条单链表，n 个结点就有 n 条孩子链表（叶子对应空链表）；所有链表的头指针集中放在一个数组（表头数组）中，数组每个位置同时保存结点数据。

链表结点中保存的是孩子在表头数组中的下标，而不是孩子数据本身。

```
typedef struct CCNode {             // 孩子链表的结点
    int child;                      // 孩子在表头数组中的下标
    struct CCNode *next;            // 下一个孩子
} CCNode;

typedef struct {
    ElemType data;                  // 数据域
    CCNode *firstChild;             // 孩子链表头指针
} TreeNode;

typedef struct {
    TreeNode nodes[MAX_TREE_SIZE];
```

```
    int n;
} CTree;
```

操作特性：找孩子快（沿链表走一遍），找双亲慢（需遍历所有链表，$O(n)$）。

实用的混合方案

把双亲表示法和孩子表示法拼在一起（结点同时带 parent 域和孩子链表头），找双亲、找孩子都快，代价是多花一点空间。这就是带双亲的孩子链表。

结构对比记忆：孩子表示法与图的邻接表、散列表的拉链法、基数排序的分配收集在示意图和结构体定义上高度相似，可放在一起记忆。

*3.*孩子兄弟表示法（二叉链表 / 二叉树表示法）

每个结点设两个指针：

- 左指针 $\rightarrow$ 第一个孩子（firstChild）；
- 右指针 $\rightarrow$ 下一个兄弟（nextSibling）。

想枚举某结点的全部孩子：先沿左指针找到老大，再沿老大的右指针依次往后找，直到右指针为空。

```
typedef struct CSNode {
    ElemType data;                  // 数据域
    struct CSNode *firstChild;      // 左指针：第一个孩子
    struct CSNode *nextSibling;     // 右指针：下一个兄弟
} CSNode, *CSTree;
```

操作特性：找孩子容易、找双亲难（同孩子表示法）；若需快速回溯双亲，可再加 parent 域构成三叉链表。

孩子兄弟表示法的最大价值

它的存储结构与二叉树的二叉链表在物理形式上完全一致，因此同一棵树可以毫不费力地「解读」成一棵二叉树——这正是下一节「树与二叉树相互转换」的物理基础。

*4.*三种存储结构对比

对比项	双亲表示法	孩子表示法	孩子兄弟表示法
额外记录	双亲下标	孩子链表头	第一个孩子 + 下一个兄弟
找双亲	$O(1)$，极快	需遍历，慢	需遍历，慢
找孩子	需扫描全表，慢	沿链表，快	沿左指针 + 右链，快
存储形式	顺序（数组）	顺序 + 链式	纯链式（二叉链表）
典型用途	并查集等向上回溯场景	需要频繁枚举孩子的场景	树 / 森林与二叉树的转换

（二）森林与二叉树的转换

树、森林与二叉树之间存在一一对应关系：任何一棵树（或森林）都能唯一地转成一棵二叉树，反之亦然。这意味着——凡是在树上不好做的事，都可以转成二叉树来做，做完再翻译回去。

*1.*树 → 二叉树

转换规则：左孩子右兄弟

二叉树中结点 p 的左指针指向原树中 p 的第一个孩子；右指针指向原树中 p 的下一个兄弟。

手工三步走：

1. 断开：对每个结点，只保留它与第一个孩子的连线，其余父子连线全部擦掉；
2. 连线：把同一双亲的兄弟结点从左到右用线串起来；
3. 整形：以树根为轴心顺时针旋转约 45°，兄弟连线就变成「右指针」。

重要结论

根结点没有兄弟，所以由单棵树转成的二叉树，其根结点必定没有右子树。反过来，一棵二叉树若根结点没有右子树，它才可以还原成「一棵树」。

逆过程（二叉树 → 树）：把每个结点的左孩子还原为「第一个孩子」，把沿右指针串起来的一串结点还原为同一双亲的一排兄弟。

*2.*森林 → 二叉树

森林是若干棵树的集合，转换建立在单棵树转换的基础上。推荐用虚构根结点法：

1. 加虚根：虚构一个根结点 r，把森林中每棵树都当作 r 的子树（于是各棵树的根互为兄弟）；
2. 转换：对「虚根 + 森林」整体应用「左孩子右兄弟」规则；
3. 删虚根：把虚根 r 及与它相连的边擦掉，剩下的就是森林对应的二叉树。

等效的手工画法（不画虚根）：先把森林中每棵树各自转成二叉树（它们的根都没有右子树），然后把第 2 棵树的根接在第 1 棵树的根的右指针上、第 3 棵接在第 2 棵的右指针上……依次串联。本质上就是「把各棵树的根视为兄弟」。

逆过程（二叉树 → 森林）：

- 若二叉树根结点无右子树 → 对应一棵树；
- 若二叉树根结点有右子树 → 对应一个森林：把根结点的右指针断开，右子链上每断开一处就分离出一棵新树，对每棵分离出的二叉树分别还原。

两个重要计数结论（真题直接考）

1. 转换前森林中的叶子结点数 = 转换后二叉树中「左孩子指针为空」的结点数。理由：左指针指向第一个孩子，故「有孩子 ⇔ 左指针非空」；叶子 ⇔ 无孩子 ⇔ 左指针为空。右指针反映的是「有没有下一个兄弟」，与是否为叶子无关。
2. 若森林（或树）中有 n 个分支结点，则转换后二叉树中「右孩子指针为空」的结点数恰为 $n+1$。理由：每个分支结点的最右孩子没有下一个兄弟，贡献 1 个空右指针，共 n 个；再加上最后一棵树的根（在森林中）或该树的根（在单棵树中）没有兄弟，贡献 1 个，合计 $n+1$。

例：一棵有 2011 个结点、116 个叶结点的树，转成二叉树后无右孩子的结点个数是多少?

分支结点数 $= 2011 - 116 = 1895$，故无右孩子的结点数 $= 1895 + 1 = 1896$。（2011 年形式）

例：某森林 F 转换为二叉树 T，F 中叶子的个数等于 T 中「左孩子指针为空」的结点个数。（2014 年形式）

（三）树和森林的遍历

*1.*树的遍历

树的孩子个数不固定，无法像二叉树那样把根「夹在两棵子树中间」，所以树没有中根遍历，只有两种（外加层序）：

- 先根遍历：先访问根结点，再从左到右依次先根遍历根的每一棵子树。
- 后根遍历：先从左到右依次后根遍历根的每一棵子树，最后访问根结点。
- 层次遍历：借助队列，自上而下、自左而右逐层访问（原理同二叉树层次遍历）。

例：树 `X(Y(V), Z(U,T), W)`

- 先根遍历序列：`X Y V Z U T W`
- 后根遍历序列：`V Y U T Z W X`
- 层次遍历序列：`X Y Z W V U T`

*2.*森林的遍历

森林的遍历是递归定义的（设森林非空）：

先序遍历森林：

1. 访问第一棵树的根结点；
2. 先序遍历「第一棵树的根的所有子树」组成的森林；
3. 先序遍历「去掉第一棵树后剩下的树」组成的森林。

效果 = 依次对每棵树做先根遍历。

中序遍历森林：

1. 中序遍历「第一棵树的根的所有子树」组成的森林；
2. 访问第一棵树的根结点；
3. 中序遍历「去掉第一棵树后剩下的树」组成的森林。

效果 = 依次对每棵树做后根遍历。

例：森林 F（`A(B(D), C)`、`E(F)`、`G(H)`）

- 先序遍历序列：`A B D C E F G H`
- 中序遍历序列：`D B C A F E H G`

术语口径差异

部分教材把森林的「中序遍历」称作「后序遍历」（因为对每棵树做的是后根遍历）。两种叫法指的是同一个序列。统考以题目采用的口径为准，看到「森林的后序遍历」多数是指对每棵树做后根遍历。

*3.*与二叉树遍历的对应关系（必背）

对应关系总表

树	森林	对应二叉树
先根遍历	先序遍历	先序遍历
后根遍历	中序遍历（部分教材称后序遍历）	中序遍历
层次遍历	——	（无直接对应）

记忆口诀：「先对先，后对中」——树的先根 / 森林的先序 ≡ 二叉树先序；树的后根 / 森林的中序 ≡ 二叉树中序。

用例子验证：

- 树 T_2 先根 `X Y V Z U T W` = 对应二叉树的先序序列 ✓；T_2 后根 `V Y U T Z W X` = 对应二叉树的中序序列 ✓
- 森林 F 先序 `A B D C E F G H` = 对应二叉树先序 ✓；F 中序 `D B C A F E H G` = 对应二叉树中序 ✓

为什么「后根 = 中序」：转换后用左指针连第一个孩子、右指针连后续兄弟。对二叉树做中序遍历时，先沿左指针进入第一个孩子，孩子的右兄弟链在遍历子树的过程中依次处理，所有孩子及其子树处理完成后才访问当前根——这与原树「先遍历各子树、后访问根」的后根遍历完全一致。

最小验证例：原树根 a 有孩子 b、c。原树后根遍历为 `b c a`。转换后二叉树中 a 的左孩子是 b，b 的右孩子是 c，其中序遍历同样为 `b c a`。

真题考法

- 2009：森林转二叉树后，若 u 是 v 的双亲的双亲，问原森林中 u、v 的关系（枚举 L-L、L-R、R-L、R-R 四种路径即可）。
- 2019：树 T 转二叉树 BT 后，与 T 的后根遍历序列相同的是 BT 的中序遍历。
- 2020：已知森林的先根与后根序列，求对应二叉树的后序序列——先用「先根 = 二叉树先序、后根 = 二叉树中序」还原二叉树，再求后序。
- 2021：给出二叉树 T 的先序与中序序列，问 F 中树的棵数——森林中树的棵数 = 二叉树根结点沿右指针链的结点数（即根到最右下的右链长度）。

四、树与二叉树的应用

（一）哈夫曼树与哈夫曼编码

*1.*基本概念

- 结点的权：赋予结点的、代表某种含义的数值（如字符出现频率、判断概率）。
- 路径长度：路径上的分支（边）数目。
- 结点的带权路径长度：从该结点到根的路径长度与该结点权的乘积 $w_i \times l_i$。
- 树的带权路径长度 **WPL**：

WPL 定义

$$WPL = \sum_{i=1}^{n_0} w_i\, l_i$$

其中 n_0 为叶子结点个数，w_i 为第 i 个叶子的权，l_i 为该叶子到根的路径长度。

WPL 只计算叶子结点，不计算内部结点。

*2.*哈夫曼树的定义

哈夫曼树（**Huffman Tree**）又称最优二叉树：在 n_0 个带权叶子结点构成的所有二叉树中，**WPL** 最小的那棵。

要让 WPL 小，就要让权大的叶子离根近（路径短），权小的叶子离根远（路径长）。

*3.*构造算法

构造步骤

1. 初始：构造含 n_0 棵二叉树的森林 $F=\{T_1,\dots,T_{n_0}\}$，每棵树只有一个权为 w_i 的根结点。
2. 选两个最小：从 F 中选取根权值最小的两棵树 T_x、T_y（选两次，可用小顶堆实现）。
3. 合并：新建结点 T_j，权值 $=T_x$ 根权 $+T_y$ 根权，以 T_x 为左子树、T_y 为右子树，把 T_j 加入 F。
4. 重复 2、3，直到 F 中只剩一棵树，即哈夫曼树。

例：权值 $\{7,5,2,4\}$ 构造哈夫曼树。

步骤	森林 F	操作
1	$\{7,5,2,4\}$	初始，4 棵树各一个结点
2	$\{7,5,6\}$	选最小的 2 和 4 合并，新结点权 $2+4=6$
3	$\{7,11\}$	选最小的 5 和 6 合并，新结点权 $5+6=11$
4	$\{18\}$	选 7 和 11 合并，根权 $7+11=18$

$$WPL_{\text{按叶子算}}=7\times1+5\times2+2\times3+4\times3=7+10+6+12=35$$

WPL 的第二种算法（更快）

哈夫曼树的 **WPL** = 所有非叶（内部）结点的权值之和。

$$\text{WPL}=6+11+18=35$$

原理：每次合并产生的新结点权值会被累加进所有子孙叶子的路径长度中；把每个分支结点的权值拆开分配，恰好等于所有叶子的带权路径长度之和。考试中用这个算法能省一半时间。

哈夫曼编码的代码实现（顺序存储，含选两个最小值）：

```
typedef struct HTNode {
    int weight;         // 权值
    char data;          // 待编码字符
    int parent;         // 双亲下标
    int lchild, rchild; // 左右孩子下标
} HTNode, *HuffmanTree;

typedef char **HuffmanCode;

// 在 0..n-1 中选出两个根结点的最小值下标（parent == 0 表示尚未被合并）
void Select(HuffmanTree HT, int n, int &min1, int &min2) {
    int i, min;
    for (i = 0; i < n; i++)          // 第一个最小值
        if (HT[i].parent == 0) { min = i; break; }
    for (i = min + 1; i < n; i++)
```

```
        if (HT[i].parent == 0 && HT[i].weight < HT[min].weight) min = i;
    min1 = min;
    for (i = 0; i < n; i++)          // 第二个最小值（跳过 min1）
        if (HT[i].parent == 0 && i != min1) { min = i; break; }
    for (i = min + 1; i < n; i++)
        if (HT[i].parent == 0 && i != min1 && HT[i].weight < HT[min].weight) min = i;
    min2 = min;
}

// 构造哈夫曼树：n 个叶子，总结点数 m = 2n - 1
void Huffman_Create(HuffmanTree &HT, int *w, char *data, int n) {
    int m = 2 * n - 1;
    HT = (HuffmanTree)calloc(m, sizeof(HTNode));  // calloc 自动清零 parent/lchild/rchild
    for (int i = 0; i < n; i++) {
        HT[i].weight = w[i];
        HT[i].data = data[i];
    }
    for (int i = n; i < m; i++) {
        int min1, min2;
        Select(HT, i, min1, min2);                // 只在 i 之前（已建好的）中选
        HT[i].weight = HT[min1].weight + HT[min2].weight;
        HT[min1].parent = i;
        HT[min2].parent = i;
        HT[i].lchild = min1;                      // 较小的作左孩子
        HT[i].rchild = min2;
    }
}

// 从叶子向上回溯得到编码（得到的是逆序，从后往前填）
void Huffman_Coding(HuffmanTree HT, HuffmanCode &HC, int n) {
    HC = (HuffmanCode)malloc(sizeof(char *) * n);
    char *code = (char *)malloc(sizeof(char) * n);
    code[n - 1] = '\0';
    for (int i = 0; i < n; i++) {
        int j = i, p = HT[j].parent, start = n - 1;
        while (p != 0) {
            code[--start] = (HT[p].lchild == j) ? '0' : '1';
            j = p;
            p = HT[j].parent;
        }
        HC[i] = (char *)malloc(sizeof(char) * (n - start));
        strcpy(HC[i], &code[start]);
    }
    free(code);
}
```

*4.*哈夫曼树的性质

五条性质

1. 结点总数：若哈夫曼树有 n_0 个叶结点，则总结点数为 $2n_0-1$，度为 2 的结点数为 n_0-1。推导：哈夫曼树没有度为 1 的结点，故 $n=n_0+n_2$；又 $n_0=n_2+1$，代入得 $n_2=n_0-1$，$n=2n_0-1$。
2. 严格二叉树：哈夫曼树中结点的度只能是 0 或 2，没有度为 **1** 的结点。
3. 权值单调性：从根往下，任意非叶结点的权值不小于下一层任一结点的权值（越靠近根权值越大）。
4. 两个最小权值必为兄弟：构造过程中权值最小的两个结点一定在第一次合并时成为同一新结点的两个孩子。
5. 形态不唯一，但 **WPL** 唯一：左右子树可以互换、权值相同时合并顺序不同，都会得到不同形态但同为

最优的哈夫曼树；**WPL** 始终相同且最小。高度也可能不同。

哈夫曼树不一定是完全二叉树

哈夫曼树是严格二叉树，但不是完全二叉树。当权值分布很不均匀时，较小权值会被反复合并，树形可能明显向一侧加深，不满足完全二叉树的层序连续性。（2010 年真题的考点）

*5.m 叉哈夫曼树

由 n_0 个初始（叶）结点构造 m 叉哈夫曼树：

补充虚结点的公式

1. 计算 $t=(n_0-1) \bmod (m-1)$。若 $t\neq 0$，需补充 $m-1-t$ 个权为 **0** 的虚叶结点；若 $t=0$ 则不需补充。
2. 每次从所有结点中取出 m 个权值最小的结点，作为新结点的孩子，新结点权值为各孩子权值之和，再加入结点集合。每轮结点数减少 $m-1$ 个。
3. 重复直到只剩一个结点。

为什么要补：保证最后一次合并恰好是 m 个结点，使构造出的树是真正的 m 叉哈夫曼树（根结点恰好有 m 个孩子）。虚结点的权为 0，不影响 **WPL**。

例：权值 $\{2,3,4,5,6,7\}$ 构造三叉哈夫曼树（2013 年统考形式）。

$n_0=6$，$m=3$，$t=(6-1) \bmod 2=1\neq 0$，需补充 $3-1-1=1$ 个权为 0 的虚结点，集合变为 $\{0,2,3,4,5,6,7\}$。

1. 取最小的三个 $0,2,3$ 合并 $\rightarrow 5$，集合 $\{4,5,5,6,7\}$
2. 取最小的三个 $4,5,5$ 合并 $\rightarrow 14$，集合 $\{6,7,14\}$
3. 取 $6,7,14$ 合并 $\rightarrow 27$，根权 27

树的形状：根 27 的三个孩子依次为 6、7、14；14 的三个孩子依次为 4、5、5′；5′ 的三个孩子依次为 0（虚）、2、3。

叶子	权	到根的路径长度
6	6	1
7	7	1
4	4	2
5	5	2
2	2	3
3	3	3
（虚）	0	3

$$WPL=6\times 1+7\times 1+4\times 2+5\times 2+2\times 3+3\times 3+0\times 3=6+7+8+10+6+9=46$$

用「内部结点权值之和」验证：$5+14+27=46$ ✓ 两种算法一致（虚结点权为 0，不产生贡献）。

m 叉哈夫曼树的 WPL 计算

m 叉情况下 $WPL=$ 所有非叶结点的权值之和仍然成立（每次合并产生的权值都是其所有子树中叶子带权路径长度的一份贡献，逐层累加即为 WPL）。这条算法比按叶子逐个数路径长度更可靠。$m=2$ 时两种算法必然一致，可以作为自查手段。

*6.*哈夫曼编码

编码：把数据转换成 0/1 串的过程。

前缀编码：没有任何一个编码是另一个编码的前缀。这样译码时从左到右扫描，每遇到一个完整编码即可立即确定一个字符，无需回溯，实现唯一可译。

前缀编码的判定方法

把给出的编码逐个比较：

- 若存在某个编码是另一个编码的前缀，则不是前缀编码；
- 等长编码一定是前缀编码；
- 「首位互不相同」或「某一位上已经区分开」都可安全排除前缀关系。

例：$\{01, 0000, 0001, 001, 1\}$ 是前缀编码；$\{0, 100, 110, 1110, 1100\}$ 不是，因为 `110` 是 `1100` 的前缀。

哈夫曼编码：对一棵有 n_0 个叶子的哈夫曼树，左分支赋 **0**、右分支赋 **1**（也可以反过来），从根到每个叶子的路径上的赋值序列就构成该叶子对应字符的哈夫曼编码。

哈夫曼编码的四条结论

1. 哈夫曼编码是最优前缀编码，能使文件压缩后的二进制长度最短；
2. 左右分支的 0/1 赋值无强制规定，且权值相同的结点合并顺序可以不同，因此哈夫曼树形态不唯一 $\rightarrow$ 哈夫曼编码不唯一；
3. 但 **WPL** 唯一且最小，即总编码长度唯一；
4. 出现频率高的字符编码短、频率低的编码长——这是哈夫曼编码高效的根源。

例：文件 f 由字符集 $\{U, V, W, X, Y, Z\}$ 组成，出现次数分别为 $\{8, 10, 25, 12, 15, 30\}$。

合并过程：$8+10=18$；$12+15=27$；$18+25=43$；$27+30=57$；$43+57=100$。

字符	出现次数	编码（左 **0** 右 **1**）	编码长度
U	8	000	3
V	10	001	3
W	25	01	2
X	12	100	3
Y	15	101	3
Z	30	11	2

$$WPL = 8\times3 + 10\times3 + 25\times2 + 12\times3 + 15\times3 + 30\times2 = 24+30+50+36+45+60 = 245$$

等价的快速算法：$WPL = 18+27+43+57+100 = 245$ ✓

与定长编码对比：6 个字符用定长编码需 $\lceil \log_2 6 \rceil = 3$ 位，总长 $100\times3 = 300$ 位。

$$\text{压缩率} = \frac{300-245}{300} \approx 18.3\%$$

加权平均长度

加权平均长度 L 表示平均每个符号占用的编码位数，衡量单位符号的编码效率：

$$L=\frac{\sum_{i=1}^{n} f_i l_i}{\sum_{i=1}^{n} f_i}=\frac{WPL}{F}=\sum_{i=1}^{n} p_i l_i$$

其中 f_i 为符号频次，$F=\sum f_i$ 为总频次，$p_i=f_i/F$ 为概率。**WPL** 是总量，L 是平均值，二者通过 $WPL=L\times F$ 互相换算。

例：频次 $\{3,4,5,6,8,10\}$，$F=36$。按哈夫曼算法合并：$3+4=7$；$5+6=11$；$7+8=15$；$10+11=21$；$15+21=36$。

$$WPL=7+11+15+21+36=90,\qquad L=\frac{90}{36}=2.5$$

（2023 年统考形式）

一个重要的常数关系

哈夫曼编码的最长编码长度 +1 = 哈夫曼树的最大高度（按根为第 1 层、高度等于层数的约定）。

真题考法（哈夫曼这一节年年出题）

- 判断题：哈夫曼树是否是完全二叉树（否）、是否有度为 1 的结点（无）、两个最小权值是否必为兄弟（是）、非叶结点权值是否不小于下层（是）——2010 年形式。
- 给出一组编码问哪个是前缀编码——2014 年形式。逐个检查是否有编码是另一个的前缀。
- 给定权值序列求最小 WPL——用「非叶结点权值之和」最快（2021、2023 年形式）。算例：权值 $\{10,12,16,21,30\}$，合并过程为 $10+12=22$、$22+16=38$、$21+30=51$、$38+51=89$，故

$$WPL_{\min}=22+38+51+89=200$$

注意：算出来的 89 是根结点的权值，不是 WPL。看到选项里有 89 不要直接选——那是典型干扰项（2021 年真题的正确选项为 200）。

- 两级 WPL 的对比：「从根到两个叶子的路径权值序列」，要求每个非叶结点权值 = 孩子权值之和（2015 年形式）。
- 给出哈夫曼编码串要求译码——从前向后贪心切分，因为前缀编码的任意前缀都不是完整编码（2017 年形式）。
- 哈夫曼树与定长编码树的对比：定长编码的叶结点全部位于同一层（2022 年形式）。
- 同一哈夫曼树上哪些叶子同层：按合并过程画出树，直接读深度（2026 年形式）。

（二）并查集

*1.*定义与核心思想

并查集（**Disjoint Set Union**，**DSU**）是处理不相交集合的合并与查询问题的数据结构。

操作	功能
`Initial(S)`	把集合 S 中每个元素初始化为独立的单元素集合
`Find(S,x)`	查找 x 所在子集合的根结点
`Union(S,Root1,Root2)`	把 `Root2` 并入 `Root1`（要求两者属于不同集合）

核心思想：用一棵树表示一个集合，根结点作为集合的代表。

- 判断元素属于哪个集合 → 找该元素所在树的根；

- 合并两个集合 → 把一棵树的根挂到另一棵树的根下。

*2.*存储结构：树的双亲表示法

用一个数组：

- 数组的下标代表元素（结点编号）；
- 数组元素的值代表该元素的双亲下标；
- 根结点的值为负数，其绝对值表示该集合的元素个数。

```
#define SIZE 100

typedef struct {
    int data;       // 元素数据值
    int parent;     // 双亲下标（负数表示根，绝对值表示集合大小）
} UFSType;

typedef struct {
    UFSType *elements;  // 双亲指针数组
    int n;              // 元素个数
} UnionFindSet;
```

根结点用负数的好处

根结点既标识了集合（谁是代表），又顺带记录了集合大小，一举两得。按大小合并时直接比较绝对值即可，无需额外数组。

例：数组 `S = [-4, 0, 0, 1, -4, 4, 5, 6]`（下标 0~7）对应：

- 集合一：根 0（`S[0]=-4`），1、2 是 0 的孩子，3 是 1 的孩子 → $\{0,1,2,3\}$
- 集合二：根 4（`S[4]=-4`），5 是 4 的孩子，6 是 5 的孩子，7 是 6 的孩子 → $\{4,5,6,7\}$

数组中只有 2 个负数，故共有 **2** 个集合。

*3.*基本操作

```
// 初始化：每个元素自成一个集合
void Initial(int S[], int n) {
    for (int i = 0; i < n; i++) S[i] = -1;
}

// 查找 x 所在集合的根（朴素版）
int Find(int S[], int x) {
    while (S[x] >= 0) x = S[x];   // 沿双亲链向上，遇到负数即为根
    return x;
}

// 合并：把 Root2 挂到 Root1 下（朴素版）
void Union(int S[], int Root1, int Root2) {
    if (Root1 == Root2) return;
    S[Root1] += S[Root2];         // 累加集合大小
    S[Root2] = Root1;
}
```

朴素实现的致命问题

朴素的 `Union` 不考虑两棵树的大小，反复执行 `Union(i+1, i)` 会退化成一条长度为 n 的链，此时 `Find`

的代价为 $O(n)$。把 n 个元素合并成一个集合的总代价可达 $O(n^2)$。这就是必须优化的原因。

*4.*优化一：路径压缩

在 Find 的过程中，顺便把查找路径上的所有结点直接挂到根下，压缩树高。

```
// 路径压缩 · 递归版
int Find(int S[], int x) {
    if (S[x] >= 0)
        return S[x] = Find(S, S[x]);   // 递归找根，回溯时把 x 挂到根下
    else
        return x;
}

// 路径压缩 · 迭代版（推荐，避免递归深度）
int Find(int S[], int x) {
    int root = x;
    while (S[root] >= 0) root = S[root];   // 第一步：找到根
    while (x != root) {                    // 第二步：把路径上所有结点挂到根下
        int tmp = S[x];
        S[x] = root;
        x = tmp;
    }
    return root;
}
```

路径压缩的性质

- 路径压缩是事后优化：第一次查找仍可能很慢，但之后同一路径上的结点到根的距离变为 1；
- 路径压缩不改变元素所属的集合（根不变），只是改变同一棵集合树内部的父子关系；
- 仅用路径压缩不保证树高有上界（单次最坏仍可能 $O(n)$），但均摊效果好。

*5.*优化二：按秩 / 按大小合并

合并时把较小的树挂到较大的树的根下，避免树高增长过快。

```
// 按大小合并（小树挂大树）
void Union(int S[], int Root1, int Root2) {
    if (Root1 == Root2) return;
    if (S[Root2] < S[Root1]) {      // Root2 的绝对值更大，即集合更大
        S[Root2] += S[Root1];       // 累加集合大小
        S[Root1] = Root2;           // Root1 挂到 Root2 下
    } else {
        S[Root1] += S[Root2];
        S[Root2] = Root1;           // 大小相等时把 Root2 挂到 Root1 下
    }
}
```

按秩合并的高度上界

采用按大小合并后，集合树的深度不超过 $\lfloor \log_2 n \rfloor + 1$。

证明：关键结论是秩（高度上界）为 k 的树至少有 2^k 个结点。归纳：秩首次变为 k 时，必是两棵秩为 $k-1$ 的树合并，结点数 $\geq 2^{k-1} + 2^{k-1} = 2^k$。于是 $n \geq 2^k$，得 $k \leq \lfloor \log_2 n \rfloor$。

*6.*复杂度总表（必背）

实现方式	单次 **Find**	单次 **Union**	n 个元素合并为 **1** 个集合
朴素实现	$O(n)$（最坏树高）	$O(1)$	$O(n^2)$
仅按秩合并	$O(\log_2 n)$（最坏）	$O(1)$	$O(n\log_2 n)$
仅路径压缩	单次最坏 $O(n)$，均摊 $O(\log_2 n)$	$O(1)$	$O(n\log_2 n)$
路径压缩 + 按秩合并	均摊 $O(\alpha(n))$	$O(1)$	$O(n\,\alpha(n))$

关于 $\alpha(n)$

$\alpha(n)$ 是 Ackermann 函数的反函数，增长极慢：$n \le 10^{80}$ 时 $\alpha(n) \le 4$，可视为常数。因此并查集的操作近乎 $O(1)$。

注意 $\alpha(n)$ 是均摊界，不是单次最坏界：单次 `Find` 最坏仍可能是 $O(\log_2 n)$，只是在整条操作序列上摊薄后接近常数。

易错辨析

- 「只要采用路径压缩，任意时刻树高都不超过 $\lfloor \log_2 n \rfloor + 1$」——错。树高上界是按秩合并带来的保证，路径压缩只压低被访问路径的高度。
- 「路径压缩可能改变元素所属的集合」——错。压缩前后元素的根不变。
- 并查集不支持高效地拆分集合（不能把一棵子树从集合树中分离出来）。
- n 个元素合并为一个集合时，若不优化会退化成链，`Find` 最坏 $O(n)$。

*7.*典型应用

（1）图的连通性问题：每个顶点初始为一个集合，每条边对应一次 `Union`，判断两顶点是否连通即比较 `Find(u) == Find(v)`。

（2）Kruskal 最小生成树：用并查集判断新加入的边是否会与已选边构成环。

1. 把所有边按权值从小到大排序；
2. 依次考察每条边 (u, v)：若 `Find(u) != Find(v)`（不连通，不成环），选中该边并 `Union`；否则舍弃；
3. 直到选够 $n-1$ 条边。

注意：图不连通时，Kruskal 得到的是最小生成森林，而不是最小生成树。

例：顶点集 $V = \{0,1,2,3,4,5,6\}$，边按权递增排列为 `e1=(0,1,1) e2=(1,2,2) e3=(3,4,3) e4=(0,2,4) e5=(4,5,5) e6=(2,3,6) e7=(1,5,7) e8=(5,6,8)`

边	结果	原因
`e1=(0,1,1)`	选中	0、1 不连通
`e2=(1,2,2)`	选中	1、2 不连通
`e3=(3,4,3)`	选中	3、4 不连通
`e4=(0,2,4)`	舍弃	0、2 已连通，成环
`e5=(4,5,5)`	选中	4、5 不连通
`e6=(2,3,6)`	选中	合并 $\{0,1,2\}$ 与 $\{3,4,5\}$
`e7=(1,5,7)`	舍弃	1、5 已连通，成环

边	结果	原因
e8=(5,6,8)	选中	顶点 6 接入生成树

最小生成树为 $\{e1, e2, e3, e5, e6, e8\}$，总权值 $1+2+3+5+6+8=25$。处理完 e7 后 $\{0,1,2,3,4,5\}$ 已连通、6 单独成一个连通分量，故有 **2** 个连通分量。

真题考法

- 给出并查集数组，问某个 Find 之后 S[x] 变成什么（路径压缩后，路径上所有结点都直接指向根）；
- 给出操作序列要求写最终数组（按大小合并时要注意「大小相等时第二个根挂到第一个根下」的约定）；
- 在 Kruskal 算法中并查集的作用（判环）。

（三）堆

*1.*定义

堆（**Heap**）是满足「堆序性」的完全二叉树：

- 大根堆（大顶堆）：任一结点的关键字不小于其左右孩子的关键字，即 $a_i \geq \max\{a_{2i}, a_{2i+1}\}$；
- 小根堆（小顶堆）：任一结点的关键字不大于其左右孩子的关键字，即 $a_i \leq \min\{a_{2i}, a_{2i+1}\}$。

堆的三条基本事实

1. 逻辑上是一棵完全二叉树，因此适合顺序存储：下标从 1 开始时，i 的左孩子 $2i$、右孩子 $2i+1$、双亲 $\lfloor i/2 \rfloor$；
2. 堆不是二叉排序树：堆只要求「父不小于（不大于）孩子」，不要求「左子树全小于根、右子树全大于根」，因此中序遍历不有序；
3. 大根堆的次大值一定在根的下一层（即根的左孩子或右孩子之一，或两者）；同理小根堆的次小值也必在根的孩子中。

```
#define MAXSIZE 100

typedef struct {
    int elem[MAXSIZE];   // 下标从 1 开始存储，elem[0] 不用
    int n;               // 元素个数
} Heap;
```

*2.*建堆：*Floyd* 自下而上算法 $O(n)$

关键点：一次性先把整个序列按层序摆成完全二叉树，然后从最后一个非叶结点开始向前逐个「下坠调整」，而不是一个一个插入。这是考试与代码实现的分水岭。

```
// 对以 k 为根的子树进行「下坠」调整（大根堆）
void SiftDown(Heap &H, int k) {
    H.elem[0] = H.elem[k];                          // 暂存待调整的根
    int i = k;
    for (int j = 2 * i; j <= H.n; j = 2 * i) {   // 沿关键字较大的孩子往下
        if (j < H.n && H.elem[j] < H.elem[j + 1]) j++;   // j 指向较大的孩子
        if (H.elem[0] >= H.elem[j]) break;       // 已满足大根堆，结束
        H.elem[i] = H.elem[j];                   // 孩子上移
        i = j;
```

```
    }
    H.elem[i] = H.elem[0];                          // 就位
}

// 自下而上建堆（Floyd 算法）：时间 O(n)
void BuildHeap(Heap &H) {
    for (int i = H.n / 2; i >= 1; i--)   // 从最后一个非叶结点开始
        SiftDown(H, i);
}
```

为什么建堆是 $O(n)$ 而不是 $O(n\log_2 n)$

「下坠」的代价与结点所在高度成正比。自下而上建堆时，绝大多数结点在底层，下坠距离很短：

- 高度为 h 的结点约有 $n/2^{h+1}$ 个，每个下坠至多 h 层；
- 总代价 $\le \sum_{h=0}^{\lfloor \log_2 n \rfloor} \frac{n}{2^{h+1}} \cdot h < 2n = O(n)$。

对比：如果从空堆开始逐个插入建堆，每个元素平均要上升 $O(\log_2 n)$，总代价 $O(n\log_2 n)$。两种建堆方式得到的堆不一定相同。

建堆的常见错误

1. 不要从最后一个结点开始：叶子结点天然满足堆序，从 $n/2$ 开始（最后一个非叶结点）即可；
2. 下坠后要继续探测子树：把根与较大孩子交换后，被交换的孩子所在的子树可能不再满足堆序，必须继续沿该路径往下调整；
3. 不要每交换一次就重新从头扫描：正确的做法是沿着一条根到叶的路径一次下坠到底，代价 $O(\log_2 n)$。

建堆比较次数的量级

自下而上建大根堆的过程中，关键字比较次数最多约 $4n$，时间复杂度 $O(n)$。这是统考中「建堆比较次数」问题的标准答案量级。

*3.*插入元素（向上调整）

步骤：把新元素放到最后（第 $n+1$ 个位置），然后沿双亲方向向上调整，直到满足堆序。

```
// 大根堆插入：先放末尾，再向上调整
bool HeapInsert(Heap &H, int x) {
    if (H.n >= MAXSIZE - 1) return false;
    int i = ++H.n;
    H.elem[i] = x;
    while (i > 1 && H.elem[i] > H.elem[i / 2]) {  // 比双亲大就交换
        int tmp = H.elem[i];
        H.elem[i] = H.elem[i / 2];
        H.elem[i / 2] = tmp;
        i = i / 2;
    }
    return true;
}
```

插入的调整次数

插入时只需与双亲比较，不需要和兄弟比较（因为兄弟一定也满足与双亲的大小关系）。比较次数最多为树高 -1，即 $\lfloor \log_2 n \rfloor$，时间复杂度 $O(\log_2 n)$。

这一点要和「排序时的取数调整」区分开：排序时是从根往下与孩子比较（可能还要先比较两个孩子），每次下坠一层最多 2 次比较。

例：小根堆 8,15,10,21,34,16,12 中插入 3。

第 9 个位置放 3，双亲为 $\lfloor 9/2 \rfloor = 4$ 号元素 21：$3 < 21$，交换；3 到第 4 位，双亲为 2 号元素 15：$3 < 15$，交换；3 到第 2 位，双亲为 1 号元素 8：$3 < 8$，交换。最终根为 3。（2009 年形式）

例：大根堆 25,13,10,12,9 尾部插入 18，比较次数是多少?

18 放第 6 位，双亲 $6/2 = 3$ 号元素 10，比较 $18 > 10$ 交换（第 1 次）；18 到第 3 位，双亲为 1 号元素 25，比较 $18 < 25$ 停止（第 2 次）。共 **2** 次比较。（2011 年形式）

*4.*删除堆顶（向下调整）

步骤：取走堆顶（通常与最后一个元素交换后，令 n 减 1），再把新的根下坠调整。

```
// 大根堆删除堆顶：把末尾元素补到根，再下坠
bool HeapDeleteMax(Heap &H, int &x) {
    if (H.n == 0) return false;
    x = H.elem[1];
    H.elem[1] = H.elem[H.n--];   // 末尾元素补到根，堆规模减 1
    SiftDown(H, 1);
    return true;
}
```

删除的调整次数

从根下坠到叶子，最多走树高 -1 层，每层最多 2 次关键字比较（先比两个孩子、再比孩子与根），时间复杂度 $O(\log_2 n)$。

例：小根堆 8,15,10,21,34,16,12 删除 8 后的比较次数。

用末尾的 12 补到根，有效序列 12,15,10,21,34,16：

1. 比较根的两个孩子 15 与 10，选较小的 10（第 1 次）；
2. 比较 12 与 10，$12 > 10$ 交换（第 2 次）；
3. 12 下移到第 3 位，该位置只有左孩子 16：比较 12 与 16，$12 < 16$ 停止（第 3 次）。

共 **3** 次比较。（2015 年形式）

*5.*堆与堆排序

堆排序（**Heap Sort**）的两阶段：

1. 建堆：对 n 个元素的序列自下而上建成大根堆（升序排序用大根堆），时间 $O(n)$；
2. 反复取数：把堆顶（当前最大）与堆尾交换，堆规模减 1，再对新根下坠调整；重复 $n-1$ 次。每次调整 $O(\log_2 n)$，共 $O(n \log_2 n)$。

堆排序的时空特性

指标	值
建堆时间	$O(n)$
单次调整	$O(\log_2 n)$
总时间	$O(n \log_2 n)$（最好、最坏、平均均如此）
空间	$O(1)$，仅需常数辅助空间

指标	值
稳定性	不稳定（建堆和堆顶、堆尾交换可能使相等元素跨越移动）
适用存储	顺序表，不适用于单链表

每趟都能确定一个元素的最终位置——堆排序和简单选择排序、快速排序一样，属于「每趟至少定位一个元素到最终位置」的排序。

堆的两个实用特性

1. 适合在海量数据中找出前 k 大 / 前 k 小的元素（维护一个大小为 k 的堆）；
2. 堆排序的左右子树调整互不干扰，可以并行执行。

堆排序不稳定的例子

原序列 1 2 2。建大根堆后两个 2 中的一个上到根；取数时先取出的必是根上的 2，剩下两个元素中原本在前的 1 反而先被取出，最终顺序变为 1 2 2（原来的第二个 2 排到了第一个 2 前面），稳定性被破坏。

真题考法

- 「逻辑上是完全二叉树」「可以顺序存储」正确，「可以看成二叉排序树」错误（2020 年形式）；
- 依次插入建堆与自下而上建堆结果不一定相同（2021 年形式特别注意）；
- 插入 / 删除时的关键字比较次数（2011、2015、2024 年形式）——必须逐步模拟；
- 堆排序与外部排序的关系：海量数据（远大于内存）的排序应采用外部排序（多路归并），堆排序虽能取前 k 大但不适合整体外排。

本章小结表

（四）表 1 二叉树五个性质一览

编号	内容	公式	适用范围
1	第 i 层最多结点数	2^{i-1}	任意二叉树
2	高度 h 最多结点数	2^h-1	任意二叉树（取满即满二叉树）
3	叶结点与度 2 结点	$n_0=n_2+1$	任意非空二叉树
4	完全二叉树编号关系	双亲 $\lfloor i/2\rfloor$，左 $2i$，右 $2i+1$	完全二叉树（从 1 编号）
5	完全二叉树高度	$\lceil\log_2(n+1)\rceil=\lfloor\log_2 n\rfloor+1$	完全二叉树

（五）表 2 一般树 / 森林的性质

内容	公式
树的边数	$e=n-1$
结点数与度数	$n=\sum_{i\geq 1} i\cdot n_i+1$
度为 m 的树第 i 层最多	m^{i-1}
高度 h 的 m 叉树最多	$\frac{m^h-1}{m-1}$

内容	公式
高度 h 的 m 叉树最少	h
高度 h、度为 m 的树最少	$h+m-1$
n 个结点的 m 叉树最小高度	$\lceil \log_m(n(m-1)+1) \rceil$
n 个结点的度为 m 的树最大高度	$n-m+1$
n 个结点的二叉树形态数	$b_n = \frac{1}{n+1}C_{2n}^{n}$
n 个结点的树形态数	$t_n = b_{n-1} = \frac{1}{n}C_{2n-2}^{n-1}$

（六）表 3 遍历序列的对应与唯一性

序列组合	唯一性	构造要点
先序 + 中序	唯一	先序首元素定根，用中序切分左右，用左子树结点数切分先序
后序 + 中序	唯一	后序末元素定根，用中序切分左右，用左子树结点数切分后序
层次 + 中序	唯一	层次首元素定根，从中序分左右，从层次序列筛选子树的层次序列
先序 + 后序	不唯一	只对严格二叉树唯一
层次 + 先序 / 层次 + 后序	不唯一	只对严格二叉树唯一；对完全二叉树因形状受限也可唯一

遍历与树 / 森林的对应	对应二叉树的遍历
树的先根遍历 / 森林的先序遍历	先序遍历
树的后根遍历 / 森林的中序遍历	中序遍历
（无对应）	后序遍历

（七）表 4 三种线索二叉树的能力

能力	中序线索	先序线索	后序线索
找前驱	可直接得（`ltag=0` 时取左子树最右下）	需从头遍历 / 三叉链表	可直接得
找后继	可直接得（`rtag=0` 时取右子树最左下）	可直接得	需从头遍历 / 三叉链表
不用栈遍历	可以	可以	仍需栈

（八）表 5 并查集复杂度

实现	单次 **Find**	单次 **Union**	n 个元素合并
朴素	$O(n)$	$O(1)$	$O(n^2)$

实现	单次 **Find**	单次 **Union**	n 个元素合并
仅按秩合并	$O(\log_2 n)$	$O(1)$	$O(n\log_2 n)$
仅路径压缩	均摊 $O(\log_2 n)$	$O(1)$	$O(n\log_2 n)$
路径压缩 + 按秩合并	均摊 $O(\alpha(n))$	$O(1)$	$O(n\,\alpha(n))$

（九）表 **6** 哈夫曼树 / 堆 / 线索二叉树的关键数量

结构	关键数量
二叉链表（n 个结点）	空指针域 $n+1$ 个
哈夫曼树（n_0 个叶子）	总结点 $2n_0-1$，内部结点 n_0-1，度为 1 的结点 0 个
线索二叉树（n 个结点）	线索（`ltag` 或 `rtag` 为 1 的域）共 $n+1$ 条
完全二叉树（层次遍历）	队列最大规模 = 叶子结点数 = $\lceil n/2\rceil$
森林转二叉树（n 个分支结点）	无右孩子的结点 $n+1$ 个；叶子数 = 左孩子为空的结点数

易错清单

1. 「树的高度」与「结点的深度 / 高度」混用。深度从根往下数、高度从下往上数；树高 = 根的高度 = 最深层号。路径长度按边数计，不是结点数。
2. 「度为 m 的树」当成「m 叉树」。前者非空且必须有度为 m 的结点（至少 $m+1$ 个结点）；后者可以为空。
3. 顺序存储二叉树时只按结点数估算空间。若题目强调「任意一棵」二叉树或给了高度，必须按最深结点编号（最坏情况 2^h-1）开空间。
4. 用「层序序列 + 空结点」反推二叉树时忘了校验双亲。任何非空结点的双亲位置都必须非空，否则是悬空结点，该数组非法。
5. 误认为「先序 + 后序」能唯一确定二叉树。只有严格二叉树才可以；一般情况会出现「单孩子结点在左还是在右」的歧义。
6. 误认为「先序序列相同 + 结点数相同」的二叉树个数可以随便猜。答案就是卡特兰数 $\dfrac{1}{n+1}C_{2n}^{n}$；但问普通树的形态数时要换成 $t_n=b_{n-1}$。
7. 「先序 = 中序」误答成「结点的度均为 **1**」。正确条件是所有非叶结点只有右子树（不存在左孩子），必须排除「只有左孩子」的情形。
8. 中序线索二叉树找前驱 / 后继时搞反「最左下」和「最右下」。后继取右子树的最左下，前驱取左子树的最右下。
9. 先序线索化递归时不判断 `ltag` / `rtag`，会顺着刚建立的线索走回祖先，造成死循环。
10. 误认为三种线索二叉树都能不用栈遍历。后序线索二叉树仍需栈，因为无法判断是否已从右子树返回。
11. 误认为三种线索二叉树都能直接找前驱和后继。口诀：中序两全、先序缺前驱、后序缺后继；缺的那一侧必须借助三叉链表（`parent`）或从头遍历。
12. 线索二叉树不带头结点时忘了处理首尾。中序第一个结点的左线索、最后一个结点的右线索应指向 `NULL`；带头结点时则指向头结点，构成双向线索链表。
13. 森林转二叉树后「无右孩子结点数」漏掉 **+1**。n 个分支结点贡献 n 个「最右孩子」，还要加上最后一棵树的根（它没有兄弟），共 $n+1$。

14. 混淆「森林的叶子数」与二叉树中哪种空指针对应。森林叶子数 = 二叉树中左孩子为空的结点数（不是右孩子为空）。
15. 树 / 森林遍历与二叉树遍历的对应关系记混。口诀「先对先，后对中」：先根 / 先序 $\leftrightarrow$ 二叉树先序；后根 / 中序 $\leftrightarrow$ 二叉树中序。
16. 哈夫曼树当成完全二叉树。哈夫曼树是严格二叉树（无度为 1 的结点），但不一定完全。
17. 构造 m 叉哈夫曼树时忘记补虚结点。判据是 $(n_0-1) \bmod (m-1)$ 是否为 0；补 $m-1-t$ 个权为 **0** 的结点。虚结点不影响 WPL。
18. **WPL** 把内部结点也乘路径长度算进去。WPL 只累加叶子；反过来，用「所有非叶结点权值之和」也能算出同一个 WPL（哈夫曼树特有两算法）。
19. 把「编码不唯一」和「**WPL** 不唯一」混为一谈。哈夫曼树形态不唯一 $\rightarrow$ 编码不唯一；但 **WPL**（总编码长度）唯一且最小。
20. 插入堆和删除堆的调整方向搞反。插入向上调（只与双亲比），删除堆顶向下调（要先比两个孩子）。
21. 建堆时从最后一个结点开始，或每交换一次就重新扫描整个数组。正确做法是从 $i=\lfloor n/2 \rfloor$ 开始向前，每个结点沿一条路径一次下坠到底，总代价 $O(n)$。
22. 认为「逐个插入建堆」与「自下而上建堆」结果相同。二者得到的堆不一定相同，且前者时间 $O(n\log_2 n)$、后者 $O(n)$。
23. 并查集按大小合并时比较方向写反。根结点的值是负数，`S[Root1]`、`S[Root2]` 中绝对值更大的是更大的集合，应把小的挂到大的下面。
24. 误认为路径压缩能保证树高 $O(\log_2 n)$。树高上界来自按秩合并；只做路径压缩时单次最坏仍可能是 $O(n)$。
25. 误认为并查集支持高效的集合拆分。并查集只擅长「合并」与「查询」，不支持拆分。
26. 在 **Kruskal** 中把并查集的作用说成「排序」或「求最短路径」。并查集的唯一作用是判断新加入的边是否会与已选边构成环；排序是排序算法的工作。

第五章图

本章考纲要求

五、图（一）图的基本概念（二）图的存储及基本操作

1. 邻接矩阵
2. 邻接表
3. 邻接多重表、十字链表

（三）图的遍历

1. 深度优先搜索
2. 广度优先搜索

（四）图的基本应用

1. 最小（代价）生成树
2. 最短路径
3. 拓扑排序
4. 关键路径

图是数据结构部分分值最高的章节之一，特点是概念判断题多、应用算法题多、算法设计题常考。本章的复习策略是：概念部分把「度数关系」「连通性边数上下界」「存储结构的空间与效率」这三类结论背死；应用部分把 Prim、Kruskal、Dijkstra、Floyd、拓扑排序、关键路径这六个算法的手工执行流程练到能闭眼画表。考纲里的「图的基本应用」四个条目全部要求会手算，不只是会写代码。

一、图的基本概念

（一）图、有向图与无向图

图（**Graph**）由顶点集（Vertex）和边集（Edge）组成，记为 $G=(V,E)$。其中 V 是顶点的有限非空集合，E 是顶点之间关系的集合。$|V|$ 表示顶点个数，称为图的阶；$|E|$ 表示边的条数。要注意图与线性表、树的区别：线性表可以是空表，树可以是空树，但图不能为空图，V 一定是非空集（E 可以为空）。

无向图（undirected graph）：E 是无向边的有限集合。无向边是顶点的无序对，记为 (v,w) 或 (w,v)，二者表示同一条边。

有向图（directed graph，digraph）：E 是有向边的有限集合。有向边又称弧（arc），是顶点的有序对，记为 $<v,w>$，其中 v 称为弧尾（tail，起点），w 称为弧头（head，终点），读作「从 v 到 w 的弧」。显然 $<v,w>\neq<w,v>$。

邻接（adjacent）：两顶点之间存在边时称二者邻接。在有向图中要说清方向——$<v,w>$ 表示 v 邻接到 w，w 邻接自 v。

简单图与多重图：

1. 简单图：不存在重复边，也不存在顶点到自身的边（自环）。

2. 多重图：某两个顶点之间的边数多于一条，或允许顶点通过同一条边与自身关联。

数据结构课程中只讨论简单图，后文若不特别说明，「图」一律指简单图。

带权图（网）：每条边都标有表示某种含义的数值，该数值称为权值（weight）或代价（cost），这样的图称为带权图，也叫网（network）。一条路径上所有边的权值之和称为该路径的带权路径长度。

（二）完全图、稠密图与稀疏图

完全图是边数达到上限的简单图：

- 无向完全图：任意两个顶点之间都存在边，边数为 $\frac{n(n-1)}{2}$。
- 有向完全图：任意两个顶点之间都存在方向相反的两条弧，边数为 $n(n-1)$。

由此可得普通简单图的边数取值范围：n 个顶点的无向图满足 $0 \le e \le \frac{n(n-1)}{2}$；有向图满足 $0 \le e \le n(n-1)$。

边数上下界（必背）

条件	边数
n 个顶点的无向图，e 的范围	$0 \le e \le n(n-1)/2$
n 个顶点的有向图，e 的范围	$0 \le e \le n(n-1)$
n 个顶点的连通无向图，e 最少	$n-1$（即生成树）
n 个顶点的非连通无向图，e 最多	$\frac{(n-1)(n-2)}{2}$，即 C_{n-1}^2
n 个顶点的强连通有向图，e 最少	n（构成一个环）

「非连通图最多 C_{n-1}^2 条边」的含义是：把 $n-1$ 个顶点凑成一个完全图，剩下 1 个顶点孤立，此时边数最多而图仍不连通。

稠密图与稀疏图：边数很少的图称为稀疏图，边数很多的图称为稠密图。二者没有绝对的界限，一般把满足 $|E| < |V|\log|V|$ 的图视为稀疏图。这个区分直接决定了应该用邻接矩阵还是邻接表存储，以及应该用 Prim 还是 Kruskal 求最小生成树。

（三）度、入度、出度与握手定理

无向图的度：顶点 v 的度 $TD(v)$ 是指依附于该顶点的边的条数。

有向图的度：以 v 为终点的弧的数目称为入度 $ID(v)$，以 v 为起点的弧的数目称为出度 $OD(v)$，二者之和为 v 的度：

$$TD(v) = ID(v) + OD(v)$$

握手定理（度的总和）：在具有 n 个顶点、e 条边的无向图中

$$\sum_{i=1}^{n} TD(v_i) = 2e$$

即无向图全部顶点的度数之和等于边数的 **2** 倍，因为每条边给两个端点各贡献 1 度。

在有向图中，每条弧给一个起点贡献出度、给一个终点贡献入度，因此

$$\sum_{i=1}^{n} ID(v_i) = \sum_{i=1}^{n} OD(v_i) = e$$

由握手定理推出的两个高频结论

1. 无向图中度为奇数的顶点个数一定是偶数。因为总度数是偶数，其中偶数度顶点贡献偶数，所以奇数度顶点必须两两配对成偶数个。
2. 无论有向图还是无向图，全部顶点的度数之和都是 $2e$（有向图中「度」按入度加出度理解）。做题时若已知部分顶点的度数，用这条等式可以反推剩余顶点的度数。

（四）路径、回路、简单路径与距离

- 路径（path）：顶点 v_p 到 v_q 之间的一条路径是指顶点序列 $v_p, v_{i1}, v_{i2}, \cdots, v_{im}, v_q$。有向图中的路径必须沿弧的方向走。
- 回路（环，cycle）：第一个顶点和最后一个顶点相同的路径。
- 简单路径（simple path）：路径序列中顶点不重复出现的路径。
- 简单回路：除第一个顶点和最后一个顶点外，其余顶点不重复出现的回路。
- 路径长度：路径上边的数目（不是权值之和）。
- 点到点的距离：从 u 到 v 的最短路径的长度；若不存在路径，距离记为 ∞。

「回路」与「简单路径」的关系

回路不是简单路径。简单路径要求所有顶点互不重复，而回路的首尾顶点必然相同，本身就已经重复了。类似地要区分「简单回路」的定义：只允许首尾顶点重复，中间顶点不允许重复。做题时把「顶点不重复」当成唯一判据就不会错。

（五）连通性：连通、强连通、连通分量

无向图的连通性：

- 连通：无向图中若从顶点 v 到顶点 w 有路径存在，则称 v 和 w 是连通的。
- 连通图：无向图中任意两个顶点都是连通的，则称该图为连通图。
- 连通分量（connected component）：无向图中的极大连通子图。「极大」的含义是：子图必须连通，并且包含尽可能多的顶点和边；只要还能再加进一个原图中的顶点（及关联边）而不破坏连通性，就不算极大。

有向图的连通性：

- 强连通：有向图中若从 v 到 w、从 w 到 v 都有路径，则称 v 与 w 强连通。
- 强连通图：有向图中任何一对顶点都强连通。
- 强连通分量：有向图中的极大强连通子图。

连通分量的三点判别

1. 一个连通图的连通分量只有 1 个，就是它自己；非连通图至少有 2 个连通分量。
2. 极大连通子图强调「极大」，不是「最大」。一个包含 3 个顶点的连通分量和一个包含 5 个顶点的连通分量可以同时存在，二者都是极大的。
3. 孤立顶点（度数为 0 的顶点）本身就是一个连通分量，因为它无法再与任何顶点构成连通子图。

（六）子图、生成子图与生成树

子图：设有图 $G=(V,E)$ 和 $G'=(V',E')$，若 V' 是 V 的子集且 E' 是 E 的子集，则称 G' 是 G 的子图。

生成子图：若子图 G' 满足 $V(G')=V(G)$（顶点集与原图完全相同），则称 G' 是 G 的生成子图。注意「子图」不要求顶点齐全，「生成子图」要求顶点一个不少。

生成树：连通图的生成树是包含图中全部顶点的一个极小连通子图。「极小」的含义是：边尽可能少，但必须保持连通。

生成树的三条必用性质：

1. n 个顶点的连通图，其生成树恰好含有 $n-1$ 条边。
2. 生成树中任意两个顶点之间有且仅有一条通路。
3. 对生成树而言，砍去一条边就变成非连通图，加上一条边就会形成一个回路。

生成森林：非连通图中，各个连通分量的生成树构成该图的生成森林。

推论：n 个顶点的图，如果边数大于 $n-1$，则一定含有回路。反过来，边数恰为 $n-1$ 且连通，则必是一棵树。

为什么生成树恰好是 $n-1$ 条边？这可以从两个方向理解。一方面，要让 n 个顶点全部连通，至少需要 $n-1$ 条边（每加一条边最多把两个尚未连通的「块」连起来，从 n 个孤立的块合并成 1 个块需要 $n-1$ 次合并）；另一方面，一旦边数达到 n 条，由「边数大于 $n-1$ 必有回路」可知此时必然出现回路，就不再是极小连通子图。两条合起来就锁定了 $n-1$ 这个数字。生成树的三条性质——边数恰为 $n-1$、任意两点间有且仅有一条通路、删一边即不连通加一边即出回路——其实是同一件事的三种说法，考试中经常把它们打散成判断题来考。

由「极小」与「极大」这一对概念还可以串起本章最容易混的三个术语：连通分量是极大连通子图（边尽量多、点尽量多，但不破坏连通性）；生成树是极小连通子图（点一个不少，边尽量少）；最小生成树是代价最小的生成树（在生成树这个大集合里，再按权值之和挑最小的）。三者都涉及「子图」，但一个求极大、一个求极小、一个求最优，考纲把它们放在不同条目下，答题时务必对号入座。

树：不存在回路且连通的无向图。因此 n 个顶点的树必定有 $n-1$ 条边。

有向树：一个顶点入度为 0、其余顶点入度均为 1 的有向图。

（七）图的基本概念小结

概念	无向图	有向图
边的记法	(v,w)，无序对	$<v,w>$，有序对，v 弧尾、w 弧头
完全图边数	$n(n-1)/2$	$n(n-1)$
顶点的度	$TD(v)$ 为依附边数	$TD(v)=ID(v)+OD(v)$
度数总和	$\sum TD(v)=2e$	$\sum ID(v)=\sum OD(v)=e$
连通性术语	连通、连通图、连通分量	强连通、强连通图、强连通分量
连通所需最少边数	$n-1$	n
生成树	有（仅连通图有）	无（有向图讲生成树需先转成无向图）

二、图的存储及基本操作

（一）邻接矩阵

邻接矩阵（adjacency matrix）用一个 $n \times n$ 的矩阵表示图。设矩阵为 A、顶点表为 V，则 $A[i][j]=1$ 表示顶点 $V[i]$ 到 $V[j]$ 之间存在边，$A[i][j]=0$ 表示无边。

```
#define MAXV 100           // 顶点数目的最大值

typedef struct {
    int numVertices;           // 图中实际的顶点数
    int numEdges;              // 图中实际的边数
    char vertexList[MAXV];     // 顶点表
    int edges[MAXV][MAXV];     // 邻接矩阵
} MGraph;
```

核心性质（必背）：

1. 主对角线元素没有意义，因为简单图不允许自环。
2. 无向图的邻接矩阵关于主对角线对称，即 $A[i][j]=A[j][i]$。因此可以只存上三角或下三角区，把空间压缩到约 $n(n+1)/2$ 个元素，对应一维数组下标 $k=n\cdot i-\dfrac{i(i-1)}{2}+(j-i)$（行优先、含主对角线）。
3. 无向图中，第 i 个顶点的度 = 第 i 行（或第 i 列）的非零元素个数。
4. 有向图中遵循「行出列入」：第 i 个顶点的出度 = 第 i 行的非零元素个数，入度 = 第 i 列的非零元素个数。

因此在邻接矩阵上求某个顶点的度、出度、入度，都需要扫描一整行或一整列，时间复杂度为 $O(|V|)$。

```
// 计算顶点 V 的出度（行内非零元素个数）
int OutDegree(MGraph graph, int index) {
    int count = 0;
    for (int j = 0; j < graph.numVertices; ++j) {
        if (graph.edges[index][j] != 0) count++;
    }
    return count;
}

// 计算顶点 V 的入度（列内非零元素个数）
int InDegree(MGraph graph, int index) {
    int count = 0;
    for (int i = 0; i < graph.numVertices; ++i) {
        if (graph.edges[i][index] != 0) count++;
    }
    return count;
}
```

带权邻接矩阵（网）：把矩阵元素从 0/1 换成权值。$A[i][j]=k$ 表示从 $V[i]$ 到 $V[j]$ 的边存在且权值为 k；$A[i][j]=\infty$（代码中常写 `INT_MAX` 或约定的大数）表示两顶点之间无边。主对角线元素通常置 **0**（自己到自己的距离为 0），也可以置 ∞，两种约定统考都出现过，解题时按题目给定的矩阵读即可。

```
// 带权图的邻接矩阵初始化（无向网）
void initWeightedMatrix(int n, int edges[][MAXV]) {
    for (int i = 0; i < n; ++i)
        for (int j = 0; j < n; ++j)
            edges[i][j] = (i == j) ? 0 : INT_MAX;   // 对角线 0，其余无穷表示无边
}
```

空间与效率：邻接矩阵的空间复杂度为 $O(|V|^2)$，与边数无关，因此适合存储稠密图。由于可以直接下标寻址，增删改查一条边的时间复杂度都是 $O(1)$。代价是当图很稀疏时会浪费大量空间。

什么时候该用邻接矩阵？判断依据只有一条：边数是否接近 n^2 量级。若图是稠密的（例如 e 与 $n(n-1)/2$ 同阶），邻接矩阵不浪费空间，同时还能享受 $O(1)$ 判边、$O(1)$ 改边、天然对称性、以及「用矩阵幂数路径」这些便利；若图是稀疏的（例如 e 与 n 同阶，像道路网、程序调用图），邻接表只花 $O(n+e)$ 空间，而邻接矩阵仍要老老实实开 n^2 个单元。考试中常把这一点包装成一句叙述让你判断对错，记住「邻接矩阵的空间代价只与顶点数有关，与边数无关」这一句就足以应对。

邻接矩阵的幂与路径数目

设图 G 的邻接矩阵为 A（元素为 $0/1$），则 A^n 的元素 $A^n[i][j]$ 等于由顶点 i 到顶点 j 的长度为 n 的路径的数目。这个结论对无向图和有向图都成立。

用法：题目给出邻接矩阵让你求 $\sum_i A^2[i][i]$（即 2 步回路的数目）、或者判断两点间是否存在长度为 2 的路径时，只需做一次矩阵乘法。注意 $A^2[i][j]=\sum_k A[i][k]\cdot A[k][j]$，本质上就是枚举中转顶点 k，这与 Floyd 算法的思想一脉相承。

（二）邻接表

邻接表（adjacency list）对图中每个顶点 v_i 建立一个单链表，第 i 个单链表的结点表示依附于 v_i 的边（对有向图则是以 v_i 为弧尾的弧），这个单链表称为顶点 v_i 的边表（有向图中称为出边表）。

边表的头指针和顶点数据采用顺序存储，称为顶点表。因此邻接表中存在两种结点：顶点表结点和边表结点，这一点可以与树的孩子表示法对比记忆。

```
#define MAXV 1000                 // 顶点数目的最大值

typedef struct ArcNode {          // 边表结点
    int adjvex;                   // 该弧所指向的顶点在顶点表中的下标
    struct ArcNode *next;         // 指向下一条弧的指针
    // int info;                  // 网中可加边权值
} ArcNode;

typedef struct VNode {            // 顶点表结点
    char data;                    // 顶点信息
    ArcNode *firstarc;            // 指向第一条依附于该顶点的弧
} VNode, AdjList[MAXV];

typedef struct {
    int numVertices, numEdges;    // 图中实际的顶点数和边数
    AdjList vertices;             // 顶点表
} ALGraph;
```

空间复杂度：

- 无向图：每条边 (v,w) 在 v 的表和 w 的表中各存一次，边结点总数为 $2|E|$，空间复杂度 $O(|V|+$

$2|E|$）。

- 有向图：只记录出边，边结点总数为 $|E|$，空间复杂度 $O(|V|+|E|)$。

邻接表只存实际存在的边，因此适合存储稀疏图。代价是查找一条边需要遍历链表，时间效率不如邻接矩阵。

有向图的出度与入度：

- 出度很容易求：找到该顶点，数它所在链表中的结点个数即可，时间复杂度 $O(1)$（定位）+ $O(d)$（数链）。
- 入度很麻烦，因为邻接表只记出边。两种方法：
  1. 遍历整个顶点表的所有链表，统计 `adjvex` 等于目标顶点下标的结点个数，时间复杂度 $O(|V|+|E|)$。
  2. 建立逆邻接表——每个顶点的链表中记录的是以该顶点为弧头的弧。逆邻接表求出度难、求入度容易，正好与邻接表互补。

```
// 邻接表上求某顶点入度：需要遍历所有顶点的链表
int InDegree(ALGraph graph, int index) {
    int count = 0;
    for (int j = 0; j < graph.numVertices; ++j) {
        for (ArcNode *p = graph.vertices[j].firstarc; p; p = p->next) {
            if (p->adjvex == index) count++;
        }
    }
    return count;
}
```

邻接表表示不唯一

用邻接矩阵存储图，其表示方法是唯一的；用邻接表存储图，其表示方法不唯一。因为各条边在单链表中的次序可以任意（头插、尾插、按编号排序都合法）。这一条直接导致后面「邻接表存储时 DFS/BFS 遍历序列不唯一、生成树不唯一」的结论，是选择题的常考陷阱。

（三）十字链表（有向图）

用邻接表存有向图能快速求出度但求入度难，用逆邻接表则相反。十字链表（orthogonal list）是一种同时能快速求入度和出度的有向图存储结构。

核心思想：把所有顶点存到顺序表中，同时为每个顶点配备两个链表——一个链表记录以当前顶点为弧头的弧，另一个链表记录以当前顶点为弧尾的弧。

- 顶点表结点：`data` 存顶点信息；`firstin` 指向以该顶点为弧头的第一条弧；`firstout` 指向以该顶点为弧尾的第一条弧。
- 边表结点：`tailvex` 存弧尾顶点在顶点表中的下标；`headvex` 存弧头顶点的下标；`hlink` 指向弧头相同的下一条弧；`tlink` 指向弧尾相同的下一条弧；`info` 可存权值。

```
#define MAXV 1000
typedef struct ArcNode {          // 边表结点
    int tailvex, headvex;         // 该弧的弧尾、弧头顶点在顶点表中的下标
    struct ArcNode *hlink, *tlink;// 分别指向弧头相同、弧尾相同的下一条弧
    // int info;                  // 网中可加边权值
} ArcNode;
```

```
typedef struct VNode {            // 顶点表头结点
    char data;                    // 顶点信息
    ArcNode *firstin, *firstout;  // 分别指向第一条入弧、第一条出弧
} VNode, XJList[MAXV];

typedef struct {
    int numVertices, numEdges;
    XJList vertices;
} OLGraph;                        // 十字链表
```

手工理解十字链表的走法：以 $< A, B >$ 和 $< A, C >$、$< C, A >$ 和 $< D, A >$ 为例（A, B, C, D 编号为 $0, 1, 2, 3$）：

1. 对 $< A, B >$、$< A, C >$，A 是弧尾，B、C 是弧头。顶点表 A 的 `firstout` 指向以 A 为弧尾的第一条弧（即边结点 B），边结点 B 的 `tlink` 指向弧尾相同的下一条弧（边结点 C），C 的 `tlink` 为空。
2. 对 $< C, A >$、$< D, A >$，A 是弧头。顶点表 A 的 `firstin` 指向以 A 为弧头的第一条弧（边结点 C），C 的 `hlink` 指向弧头相同的下一条弧（边结点 D），D 的 `hlink` 为空。

因此：

- 沿 **tlink** 链遍历，可找到指定顶点的所有出边；统计该链的结点数即为出度。
- 沿 **hlink** 链遍历，可找到指定顶点的所有入边；统计该链的结点数即为入度。

十字链表的空间复杂度为 $O(|V| + |E|)$，因为每条弧只对应一个边表结点。

（四）邻接多重表（无向图）

无向图中 (V_i, V_j) 与 (V_j, V_i) 是同一条边，但在邻接表中每条边都要存两份。如果需要对无向图的边做大量插入、删除操作，邻接表就得处理两次，效率低下。邻接多重表（adjacency multilist）就是为解决这个问题而设计的无向图存储结构。

核心思想：所有顶点存到顺序表中，每个顶点配备一个链表，链表的每个结点代表一条与当前顶点直接关联的边——注意是「一条边一个结点」，而不是「一条边两个结点」。

- 顶点结点：`data` 存顶点数据；`firstedge` 指向依附于该顶点的第一条边。
- 边结点：`i`、`j` 分别存边两端顶点在顶点表中的下标；`ilink` 指向下一条依附于顶点 `i` 的边；`jlink` 指向下一条依附于顶点 `j` 的边；`info` 可存权值。

```
#define MAXV 1000
typedef struct ArcNode {          // 边结点
    int i, j;                     // 边的两个端点在顶点表中的下标
    struct ArcNode *ilink, *jlink;// 分别指向依附于 i、j 的下一条边
    // int info;                  // 网中可加边权值
} ArcNode;

typedef struct VNode {            // 顶点表头结点
    char data;
    ArcNode *firstedge;           // 指向与该顶点相连的第一条边
} VNode, AdjMuList[MAXV];

typedef struct {
```

```
    int numVertices, numEdges;
    AdjMuList vertices;
} AMLGraph;                          // 邻接多重表
```

查找与某顶点关联的所有边（以 A 为例）：

1. 由 A 的 `firstedge` 找到第一条与 A 关联的边结点 (A, B)。
2. 该边结点的 `i` 域存的是 A 的下标，因此沿 `ilink` 找到下一条边结点 (A, D)。
3. (A, D) 的 `i` 域仍是 A 的下标，但其 `ilink` 为空，查找结束。

邻接多重表与邻接表最大的不同：无向图中每条边在邻接表要存两份，在邻接多重表只存一份，因此删除边、删除顶点等操作只需修改指针即可，非常方便。空间复杂度 $O(|V|+|E|)$。

删除顶点时别以为「方便」就等于 $O(1)$

在十字链表和邻接多重表中，一个边结点可能同时被两个顶点的指针链引用。删除顶点时，我们能轻易定位到该顶点关联的边，但不知道该边结点另一端可能存在的另一个前驱指针在哪，仍需遍历，最坏时间复杂度为 $O(|V|+|E|)$。只有在「已知边结点的前驱位置」时才能做到 $O(1)$。

（五）四种存储结构的对比

比较项	邻接矩阵	邻接表	十字链表	邻接多重表
适用对象	稠密图（有向、无向均可）	稀疏图（有向、无向均可）	仅有向图	仅无向图
空间复杂度	$O(\|V\|^2)$	无向：$O(\|V\|+2\|E\|)$；有向：$O(\|V\|+\|E\|)$	$O(\|V\|+\|E\|)$	$O(\|V\|+\|E\|)$
每条边存储份数	无向图存 2 份（对称）	无向图存 2 份	1 份	1 份
求某顶点出度	$O(\|V\|)$	容易，$O(1)+O(d)$	容易，沿 `tlink` 链	无向图不区分出/入度
求某顶点入度	$O(\|V\|)$	难，需遍历整个邻接表	容易，沿 `hlink` 链	无向图不区分出/入度
找相邻边	遍历对应行或列，$O(\|V\|)$	遍历该顶点的边表	很方便	很方便
删除一条边	很方便，$O(1)$	需定位前驱，$O(1)\sim O(\|V\|)$	较方便	方便
删除一个顶点	需处理行与列，$O(\|V\|)$（若要求紧凑移动则 $O(\|V\|^2)$）	无向图不方便	不方便	方便
表示是否唯一	唯一	不唯一	不唯一	不唯一

（六）图的基本操作

考纲「图的存储及基本操作」要求掌握以下接口，并能说出它们在两种主要存储结构下的时间代价：

操作	含义	邻接矩阵	邻接表
`Adjacent(G,x,y)`	判断是否存在边 (x,y) 或弧 $<x,y>$	$O(1)$	$O(1)\sim O(\|V\|)$
`Neighbors(G,x)`	列出与 x 邻接的顶点	$O(\|V\|)$	无向图 $O(1)\sim O(\|V\|)$；有向图出边易、入边难 $O(\|V\|+\|E\|)$
`InsertVertex(G,x)`	插入顶点 x	$O(1)$	$O(1)$
`DeleteVertex(G,x)`	删除顶点 x	$O(\|V\|)$，需将该行该列清零	无向图 $O(\|V\|+\|E\|)$
`AddEdge(G,x,y)`	若无边则插入	$O(1)$	$O(1)$（头插法）
`RemoveEdge(G,x,y)`	若有边则删除	$O(1)$	$O(1)\sim O(\|V\|)$
`FirstNeighbor(G,x)`	求 x 的第一个邻接点	$O(\|V\|)$	$O(1)$
`NextNeighbor(G,x,y)`	求 x 在 y 之后的下一个邻接点	$O(\|V\|)$	$O(1)$

其中 `FirstNeighbor` 与 `NextNeighbor` 是遍历算法的基础原语。它们的语义是：`FirstNeighbor(G, x)` 返回 x 的第一个邻接点编号，不存在则返回 -1；`NextNeighbor(G,x,y)` 返回在 y 之后的下一个 x 的邻接点，不存在则返回 -1。有了这两个函数，图的遍历就可以写成与存储结构无关的代码。

```
// 邻接矩阵版：求顶点 v 的第一个邻接点
int FirstNeighbor(MGraph *G, int v) {
    for (int i = 0; i < G->numVertices; ++i)
        if (G->edges[v][i]) return i;
    return -1;
}

// 邻接矩阵版：求顶点 v 在 w 之后的下一个邻接点
int NextNeighbor(MGraph *G, int v, int w) {
    for (int i = w + 1; i < G->numVertices; ++i)
        if (G->edges[v][i]) return i;
    return -1;
}
```

出题角度

邻接矩阵与邻接表的存储结构是本章考查次数最多的知识点（历年统考中出现 8 次以上）。常见问法：(1) 给出邻接矩阵，求各顶点的度或入度出度（如按「行出列入」直接数）；(2) 给出邻接表，问求某顶点入度需要多少时间、或问「用邻接表存储的无向图，边结点个数是多少」（答 $2e$）；(3) 判断关于存储结构的叙述正误，典型正确项是「邻接矩阵表示唯一、邻接表表示不唯一」，典型错误项是「存储稀疏图用邻接矩阵比邻接表更省空间」。(4) 给出邻接多重表的示意图，直接问某两个顶点的度——此时要注意每条边只算一次，但每个顶点在边结点中出现的次数就是它的度。

三、图的遍历

图的遍历是指从某个顶点出发，沿图中的边依次访问每个顶点，确保每个顶点都只访问一次。为避免重复访问，需要设置一个 visited 数组标记顶点是否已被访问。从某个顶点出发一次遍历只能访问到与它连通的所有顶点，因此对非连通图必须从多个起点出发，调用次数就是连通分量数。

（一）深度优先搜索（DFS）

深度优先搜索（Depth First Search，DFS）是一种「优先走到底、无路可走再回头」的遍历方式：从某顶点出发，访问它的一个邻接顶点，再从这个邻接顶点继续深入，直到走到尽头时回溯，再换一条没走过的分支继续深入，直到所有顶点都被访问。

递归实现：

```
bool visited[MAXV];

void visit(char v) { printf("%c ", v); }

// 从顶点 v 出发对图 G 做深度优先遍历
void DFS(ALGraph G, int v) {
    visit(G.vertices[v].data);       // 访问顶点 v
    visited[v] = true;               // 标记已访问
    for (ArcNode *p = G.vertices[v].firstarc; p; p = p->next) {
        if (!visited[p->adjvex])     // 邻接点未访问过则继续深入
            DFS(G, p->adjvex);
    }
}

// 遍历整张图（处理非连通图）
void DFSTraverse(ALGraph G) {
    for (int i = 0; i < G.numVertices; ++i) visited[i] = false;
    for (int i = 0; i < G.numVertices; ++i)
        if (!visited[i]) DFS(G, i);   // 每调用一次，就找到一个连通分量
}
```

非递归实现（借助显式栈）：递归的本质是系统栈，改成显式栈即可。

```
// 深度优先遍历的非递归实现（显式栈）
void DFS_NonRecursive(ALGraph G, int v) {
    int stack[MAXV], top = -1;
    stack[++top] = v;                // 起点入栈
    while (top >= 0) {
        int cur = stack[top--];      // 弹出一个顶点
        if (visited[cur]) continue;  // 可能被重复压栈，需判重
        visit(G.vertices[cur].data);
        visited[cur] = true;
        // 邻接点全部压栈（压栈顺序决定遍历序列）
        for (ArcNode *p = G.vertices[cur].firstarc; p; p = p->next) {
            if (!visited[p->adjvex])
                stack[++top] = p->adjvex;
        }
    }
}
```

要特别注意：非递归版本中一个顶点可能被多次压栈，因此弹出时必须再次检查 **visited**，否则会

重复访问。

性能分析：遍历时间 = 访问各顶点的时间 + 探索各条边的时间。

存储结构	访问顶点	探索边	总时间复杂度	空间复杂度
邻接矩阵	$O(\|V\|)$	对每个顶点扫描一整行，共 $O(\|V\|^2)$	$O(\|V\|^2)$	$O(\|V\|)$（递归深度/栈）
邻接表	$O(\|V\|)$	每条边被访问 2 次（无向图），共 $O(\|E\|)$	$O(\|V\|+\|E\|)$	$O(\|V\|)$（`visited` 数组 + 递归栈）

深度优先生成树：DFS 过程中经过的边（由「发现新顶点」的那条边构成，即 DFS 树边）和全部顶点组成深度优先生成树。对非连通图做 DFS，得到的是深度优先生成森林。

（二）广度优先搜索（BFS）

广度优先搜索（Breadth First Search，BFS）是一种「由近及远、层层扩张」的遍历方式：从某顶点出发，先访问它的所有邻接顶点（第一层），再依次访问第一层各顶点的所有未访问邻接顶点（第二层），以此类推。BFS 借助队列实现——队列「先进先出」的性质正好对应「按发现的先后次序逐层处理」。

```
void BFS(ALGraph G, int v) {
    int queue[MAXV], front = 0, rear = 0;
    visit(G.vertices[v].data);
    visited[v] = true;
    queue[rear++] = v;                          // 起点入队
    while (front < rear) {
        int cur = queue[front++];               // 队头出队
        for (ArcNode *p = G.vertices[cur].firstarc; p; p = p->next) {
            int w = p->adjvex;
            if (!visited[w]) {                  // 未被访问过则访问并入队
                visit(G.vertices[w].data);
                visited[w] = true;              // 入队时立即标记，防止重复入队
                queue[rear++] = w;
            }
        }
    }
}

void BFSTraverse(ALGraph G) {
    for (int i = 0; i < G.numVertices; ++i) visited[i] = false;
    for (int i = 0; i < G.numVertices; ++i)
        if (!visited[i]) BFS(G, i);             // 每调用一次找到一个连通分量
}
```

BFS 中「标记」的位置

BFS 必须在顶点入队的同时就置 `visited = true`，而不能等到出队时再标记。否则同一个顶点可能被其多个邻接点先后发现并重复入队，导致队列中出现大量冗余元素，甚至死循环。DFS 中则是在访问时立即标记。

性能分析：

- 空间复杂度：两种存储结构都需要一个辅助队列，最坏情况下队列大小为 $O(|V|)$（例如所有顶点都与起点相邻）。
- 时间复杂度：邻接矩阵为 $O(|V|)+O(|V|^2)=O(|V|^2)$；邻接表为 $O(|V|)+O(|E|)=O(|V|+|E|)$。

广度优先生成树：BFS 过程中「发现新顶点」的边与全部顶点构成广度优先生成树。非连通图的 BFS 得到广度优先生成森林。

BFS 求无权图单源最短路径：无权图可以看作所有边权都为 1 的带权图。BFS 按层扩展，第一次访问到某顶点时经过的边数就是它到源点的最短距离。

```
// 从源点 u 出发求无权图单源最短路径
void BFSShortestPath(ALGraph G, int u, int d[], int path[]) {
    for (int i = 0; i < G.numVertices; ++i) {
        d[i] = -1; path[i] = -1; visited[i] = false;   // -1 表示不可达
    }
    int queue[MAXV], front = 0, rear = 0;
    d[u] = 0; visited[u] = true; path[u] = -1;
    queue[rear++] = u;
    while (front < rear) {
        int cur = queue[front++];
        for (ArcNode *p = G.vertices[cur].firstarc; p; p = p->next) {
            int w = p->adjvex;
            if (!visited[w]) {
                visited[w] = true;
                d[w] = d[cur] + 1;        // 层数加一即为最短距离
                path[w] = cur;            // 记录前驱，用于还原路径
                queue[rear++] = w;
            }
        }
    }
}
```

注意目的：`d[]` 存距离，`path[]` 存前驱。求 u 到某顶点的具体路径时，从该顶点沿 `path` 反向回溯到 u 再倒序输出即可。

（三）BFS 的「层次」与 Dijkstra 的异同

BFS 求无权图单源最短路径的做法，本质上是把图按层切开：源点是第 0 层，与源点直接相邻的顶点是第 1 层，再往外是第 2 层……顶点 v 的层号就是它到源点的最短路径长度（以边数计）。Dijkstra 也有「逐轮确定一个顶点」的外壳，而且两者的算法骨架极其相似（都是维护一个候选距离数组、每轮选出一个最小值、确定它、再用它去松弛邻居），这正是考生最容易混淆的地方。二者的差异可以归结为三点。

第一，距离的定义与累加方式不同。BFS 中 `d[v] = d[u] + 1`，每条边的「代价」固定为 1，所以层号可以直接加一；Dijkstra 中 `dist[v] = dist[u] + w(u,v)`，必须把边权累加进去。正因为标量不同，BFS 可以只用一个整数计数器在层与层之间递增，而 Dijkstra 必须对每个顶点单独维护一个 `dist` 值。

第二，能否「一次性确定」的依据不同。BFS 能够一旦发现某顶点就认准它是最短路径，靠的是「所有边权相等」这一前提：先被发现的层号必然不大于后被发现的层号。Dijkstra 之所以能在每轮挑出 `dist` 最小的未确定顶点并宣布它已确定，靠的是「所有边权非负」这一前提：任何绕经其他未确定顶点的路径都不可能更短。一旦出现负权边，Dijkstra 的这个依据就崩掉了（见后文反例），而 BFS 的那一层保证也随之失效，因为一条「更深」的路径可能因为负权而总长更短。

第三，是否保留「层次」这个中间概念。BFS 天然产生一个层次结构，因此除了求距离，它还能直接用来判断二分图（按层交替染色）、求无权图的直径、做多源泛洪等；Dijkstra 不产生层次，它只输出一组距离值和一棵最短路径树。反过来，Dijkstra 能处理任意非负权值，而 BFS 面对「边权不相等」的图只能给出边数意义下的距离，与题目要求的最短路径无关。

一句话概括：**BFS** 是「边权全为 **1** 时的 **Dijkstra**」，而 **Dijkstra** 是 **BFS** 在「边权非负但可能不等」时的推广。

（四）遍历手工模拟

设无向图 G 有 6 个顶点 $1 \sim 6$，边为 1–2、1–3、2–4、3–5、4–5、5–6，采用邻接表存储且每个顶点的边表都按顶点编号升序排列。分别从顶点 1 出发做 DFS 与 BFS：

步骤	**DFS**（栈/递归）	**BFS**（队列）
起始	访问 1	访问 1，队 = [1]
第 1 步	1 的邻接点中最小未访问者为 2，访问 2	1 出队；邻接点 2、3 均未访问，访问 2、3，队 = [2,3]
第 2 步	2 的邻接点 1 已访问，取 4，访问 4	2 出队；邻接点 1 已访问，4 未访问，访问 4，队 = [3,4]
第 3 步	4 的邻接点 2 已访问，取 5，访问 5	3 出队；邻接点 1 已访问，5 未访问，访问 5，队 = [4,5]
第 4 步	5 的邻接点 3 未访问，访问 3	4 出队；邻接点 2、5 均已访问
第 5 步	5 的邻接点 6 未访问，访问 6	5 出队；邻接点 3、4 已访问，6 未访问，访问 6，队 = [6]
第 6 步	6 的邻接点 5 已访问，回溯至 5、4、2、1，均无未访问邻接点，结束	6 出队；邻接点 5 已访问，结束
序列	**1, 2, 4, 5, 3, 6**	**1, 2, 3, 4, 5, 6**

由此得到的两棵生成树也不同：深度优先生成树的树边为 1–2、2–4、4–5、5–3、5–6（呈现一条长链）；广度优先生成树的树边为 1–2、1–3、2–4、3–5、5–6（呈现逐层展开的形态）。两棵树都恰好有 $5 = n - 1$ 条边，也印证了「生成树边数恒为 $n - 1$」。若把邻接表中某两个邻接点的次序对调，上面两个序列都会随之改变——这正是「邻接表表示不唯一 $\Rightarrow$ 遍历序列不唯一」的直观体现。

（五）遍历与图的连通性

对无向图做遍历时：

- 调用 **BFS/DFS** 函数的次数 = 连通分量数。
- 若图连通，只需调用 1 次。

对有向图做遍历时，调用次数需要具体分析，规律是：

- 若从起始顶点出发能到达其余所有顶点，则只需调用 1 次。
- 对强连通图，从任一顶点出发都只需调用 1 次。
- 一般情况下，有向图一次遍历能到达的是「该顶点能到达的所有顶点」集合，与强连通分量并不一一对应。

求无向图连通分量个数的算法（统考算法设计题的常见题型）：

1. 初始化 `visited` 数组全为 `false`，计数器 `count = 0`。
2. 依次检查每个顶点 i：若 `visited[i]` 为 `false`，则以 i 为起点做一次 DFS（或 BFS），并把 `count` 加 1。
3. 搜索过程中跳过已访问顶点，未访问的顶点必然与 i 属于同一连通分量。
4. 遍历结束后 `count` 即为连通分量个数。

```
int count = 0;
void CountComponents(MGraph G) {
    for (int i = 0; i < G.numVertices; ++i) visited[i] = false;
    count = 0;
    for (int i = 0; i < G.numVertices; ++i) {
        if (!visited[i]) {          // 每个未访问顶点开启一个新的连通分量
            DFS_Matrix(G, i);       // 泛洪标记整个连通分量
            count++;
        }
    }
}
```

时间复杂度：无论 DFS 还是 BFS 实现，最坏情况（完全图）下每个顶点都要询问其余 $|V|-1$ 个顶点是否已访问，因此为 $O(|V|^2)$；若用邻接表实现则为 $O(|V|+|E|)$。

同一问题也可以用并查集解决：遍历邻接矩阵的上三角区，对每条边的两端点做 `Union`；遍历结束后统计父数组中值为 -1（即根结点）的元素个数，即为连通分量个数。

（六）遍历序列的唯一性

遍历序列何时唯一（高频结论）

- 邻接矩阵表示唯一 $\Rightarrow$ 图的结构与邻接点次序都固定 $\Rightarrow$ **DFS/BFS** 遍历序列唯一，生成树也唯一（前提是起点固定）。
- 邻接表表示不唯一（边在链表中的次序可任意）$\Rightarrow$ **DFS/BFS** 遍历序列不唯一，生成树也不唯一。

唯一性的充要条件：给定图 G 和起点 v，从 v 出发的 DFS（或 BFS）序列唯一，当且仅当遍历过程中每一步「当前可选的未访问邻接点」都只有一个。换句话说，只要途中某个顶点存在两个及以上未访问的邻接点，选择次序不同就会得到不同的遍历序列。

（七）**DFS** 与 **BFS** 的对比

比较项	深度优先搜索 **DFS**	广度优先搜索 **BFS**
辅助结构	栈（递归的系统栈或显式栈）	队列
扩展方式	一条路走到黑，回溯	由近及远，逐层扩散
邻接矩阵时间复杂度	$O(\|V\|^2)$	$O(\|V\|^2)$
邻接表时间复杂度	$O(\|V\|+\|E\|)$	$O(\|V\|+\|E\|)$
空间复杂度	$O(\|V\|)$	$O(\|V\|)$
生成树名称	深度优先生成树（森林）	广度优先生成树（森林）
典型用途	判断环、求连通分量、逆拓扑序列、割点	求无权图单源最短路径、层次遍历、判二分图

四、图的基本应用

（一）最小（代价）生成树

1.（*1*）最小生成树的概念与唯一性

最小生成树（Minimum-Spanning-Tree，MST）是连通网中各边权值之和最小的那棵生成树。

必背结论：

1. 一个连通网的最小生成树可能有多个，但各边权值之和（代价）总是唯一且最小的。
2. 当图中各边的权值互不相等时，最小生成树唯一。
3. 若无向连通图的边数比顶点数少 1（$e = n - 1$），则它本身就是一棵树，最小生成树就是它自己。
4. 只有连通图才有生成树，非连通图只有生成森林。

MST 唯一的充要条件：设 G 是无向连通带权图，T 是其一棵最小生成树，则 T 是 G 的唯一最小生成树，当且仅当对于不在 T 中的每一条边 e，都有 e 的权重大于 $T + e$ 所产生的圈中其它每条边的权重。

证明思路：往 T 中加入任一条不属于 T 的边 e，必然形成唯一一个圈（因为 T 已经包含全部顶点且连通无环）。若 e 的权小于或等于圈中某条边的权，就可以用 e 替换那条边得到一棵权值和不增大的新生成树，从而 T 不唯一；反之若 e 严格大于圈中所有其它边，任何替换都会使总权值增大，故 T 唯一。

2.（*2*）*Prim* 算法（选点）

算法思想：从某个顶点开始构建生成树，每次把与当前生成树顶点集合距离最近的那个新顶点及其对应边纳入生成树，直到所有顶点都被纳入。

执行步骤：

1. 初始化：设置 `visited[]` 数组标记顶点是否已进入 MST；设置 `dist[]` 数组，`dist[v]` 表示「顶点 v 到当前 MST 顶点集合的最短边长」。任选一个起点 v_0，令 `dist[v0] = 0`、`visited[v0] = true`，其余顶点的 `dist` 初始化为 v_0 到它们的直接边长（无边则为 ∞）。
2. 选点：在所有未加入 **MST**（`visited` 为 **false**）的顶点中挑出 `dist` 值最小的顶点 u。
3. 纳入：令 `visited[u] = true`，把边 $(u,$ 产生 dist[u] 的那个顶点$)$ 加入 MST，累加权值。
4. 更新：对 u 的每个未加入 MST 的邻接顶点 k，执行

$$dist[k] = \min\{dist[k],\ w(u,k)\}$$

注意这里用的是边的权值，不是把路径长度累加。

5. 重复步骤 2 ~ 4 共 $n - 1$ 次，直到所有顶点都加入 MST。

Prim 的 dist 数组含义（与 Dijkstra 的分水岭）

- **Prim**：`dist[k]` = 顶点 k 到 **MST** 顶点集合的最短边权。更新式为 $dist[k] = \min\{dist[k],\ w(u,k)\}$。
- **Dijkstra**：`dist[k]` = 顶点 k 到源点的最短路径长度。更新式为 $dist[k] = \min\{dist[k],\ dist[u] + w(u,k)\}$。

两者的代码结构几乎相同，但一个「不累加、只看单条边」，一个「累加、看整条路径」，这是考试中最容易被混淆的地方。

```
void Prim(MGraph G, int v0) {
```

```
    int dist[MAXV];
    bool visited[MAXV];
    int sum = 0;
    for (int i = 0; i < G.numVertices; ++i) {
        dist[i] = (i == v0) ? 0 :
                  (G.edges[v0][i] != 0 && G.edges[v0][i] < INT_MAX ? G.edges[v0][i] : INT_MAX);
        visited[i] = false;
    }
    visited[v0] = true;
    for (int i = 1; i < G.numVertices; ++i) {     // 共执行 n-1 轮
        int u = -1, min = INT_MAX;
        for (int j = 0; j < G.numVertices; ++j) { // 找 dist 最小的未访问顶点
            if (!visited[j] && dist[j] < min) { min = dist[j]; u = j; }
        }
        if (u == -1) break;                       // 图不连通
        visited[u] = true;
        sum += min;
        for (int k = 0; k < G.numVertices; ++k) { // 用 u 的边更新 dist
            if (!visited[k] && G.edges[u][k] != 0 && G.edges[u][k] < INT_MAX
                && G.edges[u][k] < dist[k])
                dist[k] = G.edges[u][k];
        }
    }
    printf("MST 总权值 = %d\n", sum);
}
```

手工模拟：设有 6 个顶点 A、B、C、D、E、F，无向带权边为 AB=6、AC=1、AD=5、BC=5、BE=3、CD=5、CE=6、CF=4、DF=2、EF=6。从 A 出发运行 Prim：

轮次	当前 **dist**（未确定顶点的候选值）	选中的顶点	加入的边	累计权值
初始	B:6, C:1, D:5, E:∞, F:∞	—	—	0
第 1 轮	选中最小者 C（1）	C	A–C	1
更新	B:min(6,5)=5, D:min(5,5)=5, E:min(∞,6)=6, F:min(∞,4)=4			
第 2 轮	最小者 F（4）	F	C–F	5
更新	B:min(5,∞)=5, D:min(5,2)=2, E:min(6,6)=6			
第 3 轮	最小者 D（2）	D	F–D	7
更新	B:min(5,5)=5, E:min(6,∞)=6			
第 4 轮	最小者 B（5）	B	C–B	12
更新	E:min(6,3)=3			
第 5 轮	最小者 E（3）	E	B–E	15

所得 MST 的边为 {A–C(1), C–F(4), F–D(2), C–B(5), B–E(3)}，总代价 15，共 5 条边，等于 $n-1$。

复杂度：外层循环执行 $n-1$ 次，内层两个并列的非嵌套循环各为 $O(|V|)$，因此总时间复杂度为 $O(|V|^2)$，与边数无关。空间复杂度 $O(|V|)$。**Prim** 适合求解边稠密的图（点少边多）。

3.（*3*）*Kruskal* 算法（选边）

算法思想：每次选择一条权值最小的边，若这条边的两个端点尚未连通则加入 MST，否则跳过（因为加入会形成回路），直到所有顶点连通。Kruskal 是并查集的典型应用。

执行步骤：

1. 把所有边按权值从小到大排序。
2. 初始化并查集，每个顶点各成一个集合。
3. 依次取出排序后的每条边 (u,v)：查询 u、v 的根结点，若不同则合并两集合并把该边加入 MST；若相同则跳过该边。
4. 当加入 MST 的边数达到 $|V|-1$ 时算法结束。
5. 若所有边都处理完仍未达到 $|V|-1$ 条，说明原图不连通，不存在生成树。

```
int parent[MAXV];
void initUF(int n) { for (int i = 0; i < n; ++i) parent[i] = i; }
int find(int x) { return parent[x] == x ? x : (parent[x] = find(parent[x])); }  // 路径压缩
bool unite(int x, int y) {
    int rx = find(x), ry = find(y);
    if (rx == ry) return false;      // 已连通，该边会成环
    parent[ry] = rx;
    return true;
}
int cmpEdge(const void *a, const void *b) {
    int wa = ((Edge *)a)->weight, wb = ((Edge *)b)->weight;
    return (wa > wb) - (wa < wb);    // 避免直接相减溢出
}

void Kruskal(MGraph *G, Edge edgeList[]) {
    int cnt = 0, sum = 0;
    initUF(G->numVertices);
    qsort(edgeList, G->numEdges, sizeof(Edge), cmpEdge);   // 按权值升序
    for (int i = 0; i < G->numEdges && cnt < G->numVertices - 1; ++i) {
        if (unite(edgeList[i].u, edgeList[i].v)) {
            sum += edgeList[i].weight;
            cnt++;
        }
    }
    if (cnt == G->numVertices - 1) printf("MST 总权值 = %d\n", sum);
    else printf(" 图不连通，无生成树\n");
}
```

手工模拟：仍用上一小节的 6 顶点图，按下表依次处理排序后的边：

序号	边	权值	两端是否已连通	处理结果	已选边数
1	A–C	1	否	加入	1
2	D–F	2	否	加入	2
3	B–E	3	否	加入	3
4	C–F	4	否（{A,C} 与 {D,F} 尚未合并）	加入	4
5	A–D	5	是（A–C–F–D 已连通）	跳过	4

序号	边	权值	两端是否已连通	处理结果	已选边数
6	B–C	5	否（{B,E} 与 {A,C,D,F} 尚未合并）	加入	5
—	—	—	已达 $n-1=5$ 条	结束	5

所得 MST 的边为 {A–C(1), D–F(2), B–E(3), C–F(4), B–C(5)}，总代价 15。可以看到 Prim 与 Kruskal 得到了不同的边集但相同的总代价——这正说明该图的 MST 不唯一（因为存在权值相同的边）。

复杂度：边排序需要 $O(|E|\log|E|)$；每条边做一次并查集查询，路径压缩后近似 $O(1)$，至多 $|E|$ 次，因此总时间复杂度为 $O(|E|\log|E|)$（可写为 $O(|E|\log|V|)$）。空间上需要 $O(|V|)$ 的并查集数组与边表。**Kruskal** 适合求解边稀疏而顶点较多的图。

Prim 与 Kruskal 的选择口诀

- **Prim** 选点（P 开头，Point 也是 P 开头），时间复杂度 $O(|V|^2)$，只与顶点数有关，适合边稠密的图。
- **Kruskal** 选边（K 开头，可以联想「快排也是先排序」），时间复杂度 $O(|E|\log|E|)$，只与边数有关，适合边稀疏而顶点较多的图。

4.（*4*）两个算法为什么正确

两个算法都是贪心，它们的正确性依赖同一条性质，可以称为切割性质（cut property）：把图的顶点任意划分成两个非空集合 S 与 $V-S$，横跨这两个集合的边中权值最小的那条边（若权值并列则任取其一）一定属于某棵最小生成树。直觉解释是：设 T 是一棵 MST，若这条最小横跨边 $e=(u,v)$ 不在 T 中，把它加入 T 会形成一个圈，而这个圈必然还有另一条横跨 S 与 $V-S$ 的边 e'（因为从 $u\in S$ 出发绕圈回到 $v\in V-S$ 至少要横跨一次）。由 e 是最小横跨边可知 $w(e)\le w(e')$，于是用 e 换掉 e' 得到的新生成树权值不增大，仍是 MST，故存在一棵 MST 包含 e。

- **Prim** 的做法就是反复应用切割性质：取 S = 当前已加入 MST 的顶点集合，$V-S$ = 尚未加入的顶点集合，此时「横跨 S 与 $V-S$ 的最小边」恰好就是 `dist` 数组中的最小值，所以每轮选 `dist` 最小的顶点是安全的。
- **Kruskal** 的做法则是另一种视角：按权值从小到大逐条考察，当考察到边 $e=(u,v)$ 时，若 u、v 已连通就跳过（加入必成环，且圈中已有边的权值都不大于 $w(e)$，换掉任何一条都不会更优）；若未连通就一定要加入（否则这一时刻 u 所在连通分量与 v 所在连通分量之间的所有横跨边权值都 $\ge w(e)$，而最终必须有一条横跨边被选中，选 e 不会比选别的差）。

理解了切割性质，就能解释两个常考的现象：其一，同一张图用 Prim 和 Kruskal 可能得到不同的 MST 边集，但它们的总代价必然相同（都等于所有被切割性质「锁定」的边权之和的最小值）；其二，只要某条边的权值是全图唯一最小的，它必然出现在每一棵 MST 中；反之若有若干条同样小的边构成环，就不能把它们全部选入，因此「所有权值最小的边一定出现在所有 MST 中」是错误的。

5.（*5*）最小生成树与最短路径的区别

比较项	**Prim**（最小生成树）	**Dijkstra**（单源最短路径）
目标	使生成树所有边的权值之和最小	使源点到每个顶点各自的路径长度最小
`dist` 含义	顶点到已选顶点集合的最短边权	顶点到源点的最短路径长度

比较项	**Prim**（最小生成树）	**Dijkstra**（单源最短路径）
更新方式	$dist[k] = \min(dist[k],\ w(u,k))$	$dist[k] = \min(dist[k],\ dist[u] + w(u,k))$
结果结构	一棵包含全部顶点的树	一棵以源点为根的最短路径树
权值要求	无限制（无向图）	不能有负权边
时间复杂度	$O(\|V\|^2)$	$O(\|V\|^2)$

最关键的区别：MST 保证「整棵树的总权值最小」，但不保证树上源点到某顶点的路径是原图中的最短路径；最短路径树保证「源点到每个顶点的路径最短」，但整棵树的边权之和通常比 MST 大。二者不能互相替代。特别地，即使图的所有边权都为 1，Prim 和 Kruskal 也不能用来求单源最短路径。

出题角度

最小生成树在历年统考中出现 5 次左右，选择题与应用题都会考。典型问法：(1) 判断关于 MST 的叙述——「最小生成树的代价唯一」正确，「所有权值最小的边一定出现在所有 MST 中」错误（等权边成环时不可能全选），「Prim 从不同顶点开始得到的 MST 一定相同」错误；(2) 给出带权图，问「可能是 Kruskal 第 2 次选中但不是 Prim（从某点开始）第 2 次选中的边」，解法是分别按两种规则手算前两步再取差集；(3) 应用题要求写出 Prim 或 Kruskal 逐次选边过程并计算总代价。

（二）最短路径

1.（*1*）无权图的单源最短路径——*BFS*

无权图（或所有边权都相等）的单源最短路径可以直接用 BFS 求。算法过程：

1. 设置队列 `q`、访问标记数组 `visited`、距离数组 `d`、前驱数组 `path`。
2. 源点 s 入队，置 `visited[s] = true`、`d[s] = 0`、`path[s] = -1`。
3. 循环：弹出队头 u，遍历 u 的所有邻接点 v；若 v 未访问，则置 `visited[v] = true`、`d[v] = d[u] + 1`、`path[v] = u`，并把 v 入队。
4. 队列为空时，`d[]` 即为源点到各顶点的最短距离；若 `d[i]` 仍为无穷，说明 i 从源点不可达。

时间复杂度：邻接矩阵 $O(|V|^2)$，邻接表 $O(|V| + |E|)$。

2.（*2*）*Dijkstra* 算法（带权单源最短路，非负权）

BFS 只适用于无权图或各边权值相等的图。当边权不同时，更深的路径完全可能比更浅的路径更短，此时必须使用 Dijkstra 或 Floyd 算法。

Dijkstra 算法需要三个辅助数组：

- `dist[]`：`dist[i]` 表示当前已知的从源点到顶点 i 的最短路径长度。
- `path[]`：`path[i]` 表示源点到 i 的最短路径上 i 的前驱顶点，用于还原路径。
- `final[]`：`final[i] = true` 表示顶点 i 的最短路径已经确定，不会再被更新。

执行步骤：

1. 初始化：`final[]` 全置 `false`；`dist[i] = Edge[s][i]`（源点到 i 的直接边长，无边则 ∞）；`path[i] = (Edge[s][i] 存在) ? s : -1`。令 `final[s] = true`、`dist[s] = 0`、`path[s] = -1`。
2. 选点：在 `final` 为 `false` 的顶点中，找出 `dist` 值最小的顶点 u。

3. 确定：令 `final[u] = true`。此时 `dist[u]` 就是源点到 u 的最短距离。
4. 松弛：检查所有由 u 出发的边 $< u, v >$。若 `final[v] == false` 且

$$dist[u] + w(u, v) < dist[v]$$

则更新 `dist[v] = dist[u] + w(u,v)`，并令 `path[v] = u`。

1. 重复步骤 2 ~ 4 共 $n - 1$ 轮，直到所有顶点的最短路径都确定。

```
void Dijkstra(MGraph G, int v, int dist[], int path[]) {
    bool final[MAXV];
    int n = G.numVertices;
    for (int i = 0; i < n; ++i) {                  // 初始化
        final[i] = false;
        int w = G.edges[v][i];
        bool hasEdge = (w != 0 && w < INT_MAX);
        dist[i] = hasEdge ? w : INT_MAX;
        path[i] = hasEdge ? v : -1;
    }
    final[v] = true; dist[v] = 0; path[v] = -1;
    for (int i = 1; i < n; ++i) {                  // 共 n-1 轮
        int u = -1, min = INT_MAX;
        for (int j = 0; j < n; ++j)                // 选点：找 dist 最小的未确定顶点
            if (!final[j] && dist[j] < min) { min = dist[j]; u = j; }
        if (u == -1) break;                        // 剩余顶点均不可达
        final[u] = true;
        for (int j = 0; j < n; ++j) {              // 松弛：只用「从 u 出发」的边
            if (!final[j] && G.edges[u][j] != 0 && G.edges[u][j] < INT_MAX
                && dist[u] != INT_MAX
                && dist[u] + G.edges[u][j] < dist[j]) {
                dist[j] = dist[u] + G.edges[u][j];
                path[j] = u;
            }
        }
    }
}
```

手工模拟：设 5 个顶点 $v_0 \sim v_4$，有向边 $v_0 \to v_1 = 10$、$v_0 \to v_4 = 5$、$v_4 \to v_1 = 3$、$v_4 \to v_2 = 9$、$v_4 \to v_3 = 2$、$v_1 \to v_2 = 1$、$v_2 \to v_3 = 4$、$v_3 \to v_0 = 7$、$v_3 \to v_2 = 6$。以 v_0 为源点：

轮次	已确定集合	`dist (v0,v1,v2,v3,v4)`	本轮选中	本轮 `path` 变化
初始	{v0}	0, 10, ∞, ∞, 5	—	`path[1]=0, path[4]=0`
第 1 轮	{v0,v4}	0, **8**, **14**, **7**, 5	v4（5）	`path[1]=4, path[2]=4, path[3]=4`
第 2 轮	{v0,v4,v3}	0, 8, **13**, 7, 5	v3（7）	`path[2]=3`（7+6=13<14）
第 3 轮	{v0,v4,v3,v1}	0, 8, **9**, 7, 5	v1（8）	`path[2]=1`（8+1=9<13）
第 4 轮	全部	0, 8, 9, 7, 5	v2（9）	无

最终结果：$v_0 \to v_1 = 8$（$v_0 \to v_4 \to v_1$），$v_0 \to v_2 = 9$（$v_0 \to v_4 \to v_1 \to v_2$），$v_0 \to v_3 = 7$（$v_0 \to v_4 \to v_3$），$v_0 \to v_4 = 5$。

注意第 1 轮后 dist[1] 从 10 被更新为 8、dist[2] 从 ∞ 变为 14、dist[3] 从 ∞ 变为 7——这说明「中转一个顶点」就能让原本不可达的顶点变得可达，让原本较长的路径变短，这也正是每轮都要重新松弛的原因。

复杂度：两层 for 循环，外层执行 $n-1$ 轮，每轮选点 $O(|V|)$，松弛 $O(|V|)$，总时间复杂度 $O(|V|^2)$。空间复杂度 $O(|V|)$（三个一维数组）。若用邻接表加优先队列（堆）优化，可降到 $O(|E|\log|V|)$，但统考按 $O(|V|^2)$ 理解。

Dijkstra 为什么不能处理负权边

Dijkstra 的正确性建立在一条贪心性质上：当前 dist 最小的未确定顶点，其最短路径已经确定，此后不会再被任何路径更新。当所有边权非负时，任何绕路（经过其他未确定顶点）只会让路径更长或持平，因此这条性质成立。

一旦出现负权边，这条性质就失效了。反例：有向图 $1\to2$ 权值为 3，$1\to3$ 权值为 4，$3\to2$ 权值为 -2。

- 初始化：dist[2]=3，dist[3]=4。
- 第 1 轮：dist 最小的未确定顶点是 2（值为 3），令 final[2]=true，认定 1 到 2 的最短距离是 3。
- 第 2 轮：选中顶点 3（值为 4），尝试用 $3\to2$ 松弛，得到 $dist[2]$ 应为 $4+(-2)=2<3$——但 final[2] 已经是 true，按算法规则不会再更新它。

于是 Dijkstra 输出 3，而真实最短路径是 $1\to3\to2$，长度为 2。结论：**Dijkstra** 只适用于所有边权非负的图；求解带负权（但无负权回路）的图必须用 Floyd 或 Bellman-Ford。

3.（3）Floyd 算法（各顶点对之间的最短路径）

算法思想：动态规划。依次允许经过 $v_0, v_1, v_2, \cdots, v_{n-1}$ 中转，每允许增加一个中转顶点，就检查是否能让某些点对之间的距离变短，若可以就更新。设 $dist[i][j]$ 表示当前已知的从 i 到 j 的最短路径长度，则状态转移为

$$dist[i][j]=\min\{dist[i][j],\ dist[i][k]+dist[k][j]\}$$

执行步骤：

1. 初始化：dist[][] 直接取邻接矩阵（注意 $i=j$ 时置 **0**，无边时置 ∞）；path[][] 初始化为 -1（也可初始化为 i，表示从 i 直接到 j）。
2. 三层循环：外层固定中转顶点 k（$0\sim n-1$），中层固定起点 i，内层固定终点 j。对每个 (i,j) 判断 $dist[i][k]+dist[k][j]<dist[i][j]$，若成立则更新距离并记录 path[i][j] = k。
3. 循环结束时（若无负权回路），dist[i][j] 即 i 到 j 的最短路径长度。

```
void Floyd(MGraph G, int dist[MAXV][MAXV], int path[MAXV][MAXV]) {
    int n = G.numVertices;
    for (int i = 0; i < n; ++i) {                       // 初始化
        for (int j = 0; j < n; ++j) {
            int w = G.edges[i][j];
            if (i == j) dist[i][j] = 0;
            else dist[i][j] = (w != 0 && w < INT_MAX) ? w : INT_MAX;
            path[i][j] = (dist[i][j] != INT_MAX && i != j) ? i : -1;
        }
    }
    for (int k = 0; k < n; ++k) {                       // 外层：中转顶点 k（必须在最外层）
        for (int i = 0; i < n; ++i) {                   // 中层：起点 i
            for (int j = 0; j < n; ++j) {               // 内层：终点 j
```

```
                if (dist[i][k] < INT_MAX && dist[k][j] < INT_MAX
                    && dist[i][k] + dist[k][j] < dist[i][j]) {
                    dist[i][j] = dist[i][k] + dist[k][j];
                    path[i][j] = k;                    // 记录中转点
                }
            }
        }
    }
}
```

手工模拟（十字法）：设 4 个顶点 $0,1,2,3$，有向边 $0\to1=1$、$1\to2=3$、$0\to2=5$、$2\to3=1$、$0\to3=10$、$1\to3=8$。∞ 表示不可达。

初始 $dist$（k 尚未允许）：

从\到	**0**	**1**	**2**	**3**
0	0	1	5	10
1	∞	0	3	8
2	∞	∞	0	1
3	∞	∞	∞	0

- $k=0$：没有任何顶点能到达 0，矩阵无变化。
- $k=1$（允许经 1 中转）：只需检查第 1 列非 ∞ 的行与第 1 行非 ∞ 的列。
  - $dist[0][1]+dist[1][2]=1+3=4<5\Rightarrow dist[0][2]=4$，`path[0][2]=1`。
  - $dist[0][1]+dist[1][3]=1+8=9<10\Rightarrow dist[0][3]=9$，`path[0][3]=1`。
- $k=2$（允许经 2 中转）：
  - $dist[0][2]+dist[2][3]=4+1=5<9\Rightarrow dist[0][3]=5$，`path[0][3]=2`。
  - $dist[1][2]+dist[2][3]=3+1=4<8\Rightarrow dist[1][3]=4$，`path[1][3]=2`。
- $k=3$：3 没有出边，无变化。

最终 $dist[0][3]=5$，还原路径：`path[0][3]=2` $\Rightarrow$ 到 3 之前先到 2；`path[0][2]=1` $\Rightarrow$ 到 2 之前先到 1；`path[0][1]=0` $\Rightarrow$ 到 1 之前先到 0。于是路径为 $0\to1\to2\to3$，长度 $1+3+1=5$，与 $dist$ 一致。

Floyd 的三重循环顺序为什么不能换

k 必须放在最外层。因为 k 代表的是「当前允许作为中转的顶点集合」$\{0,1,\cdots,k\}$，这是动态规划的阶段；i,j 是同一阶段内的状态。只有先固定阶段再枚举状态，转移方程 $dist_k[i][j]=\min(dist_{k-1}[i][j],\ dist_{k-1}[i][k]+dist_{k-1}[k][j])$ 才成立。

如果把 k 放到里层（例如写成 i,j,k 三层），就会在同一阶段内反复使用尚未确定的中间结果，导致某些路径组合被漏算。做题时若题目要求「依次允许 $v_0,v_1,\cdots$ 中转」写出每一轮矩阵，本质上就是在验证这个顺序。

复杂度：三重循环，时间复杂度 $O(|V|^3)$；空间复杂度 $O(|V|^2)$（距离矩阵 + 路径矩阵）。

负权支持：Floyd 可以处理带负权边但不能有负权回路的图。原因在于算法会对所有顶点的所有入边做松弛，不受「已确定」限制。若图中存在负权回路，则绕该回路可以无限缩短路径，最短路径不存在（长度趋于 $-\infty$）。

4.（4）三种最短路径算法对比

比较项	**BFS**	**Dijkstra**	**Floyd**
解决问题	无权图单源最短路径	非负权图单源最短路径	各顶点对之间的最短路径
边权要求	无权或各边权相等	所有边权非负	允许负权，不允许负权回路
算法思想	按层扩展	贪心，分已确定/未确定两个集合	动态规划，逐个允许中转顶点
辅助数组	`visited`、`d`、`path`	`dist`、`path`、`final`	`dist[][]`、`path[][]`
时间复杂度	邻接表 $O(\|V\|+\|E\|)$；邻接矩阵 $O(\|V\|^2)$	$O(\|V\|^2)$	$O(\|V\|^3)$
空间复杂度	$O(\|V\|)$	$O(\|V\|)$	$O(\|V\|^2)$
适用场景	无权网络、层次关系	非负权单源问题	需要所有点对距离

出题角度

最短路径历年考查 5 次以上，是选择题与应用题的双料重点。典型问法：(1) 给出图，问「依次得到的各最短路径的目标顶点是」，需按 Dijkstra 逐轮记录被确定顶点的顺序；(2) 给出图与初始 `dist` 数组，问「求出第 2 条最短路径后 `dist` 中的内容」，需精确模拟两轮松弛；(3) 问「所有边权均为 1 的连通图中，一定能求出单源最短路径的算法」，答案是只有 **BFS**，Prim 和 Kruskal 求的是最小生成树，不保证源点到各顶点的路径最短；(4) 判断「Dijkstra 能否处理负权图」，注意题干若强调「无负权回路」，则能处理的是 Floyd 而不是 Dijkstra。

（三）拓扑排序

1.（1）AOV 网与拓扑序列

AOV 网（Activity On Vertex Network）：用顶点表示活动、用有向边表示活动之间的先后制约关系的有向图。例如课程 A 是课程 B 的先修课，就画一条 $A \to B$ 的弧。

拓扑排序（Topological Sorting）：由一个有向无环图的顶点组成的一个线性序列，当且仅当满足以下条件时称为该图的一个拓扑排序——对图中任意一条弧 $< u, v >$，u 在序列中都排在 v 之前。

等价说法：拓扑排序是对有向图的一种遍历，在这种遍历中，对一个顶点 v 的访问只发生在它所有前驱顶点都被访问之后。

必背性质：

1. 只要图为 **DAG**（有向无环图），就至少存在一种拓扑排序结果。每个 AOV 网都有一个或多个拓扑序列。
2. 有向图存在拓扑序列 ⇔ 图中不存在回路 ⇔ 图是 **DAG**。
3. 拓扑排序与图是否连通无关。互不连通的分量互不依赖，它们在拓扑序中的相对顺序是任意的。
4. 若某顶点有多个直接后继，拓扑排序结果通常不唯一。唯一的情形是：每个时刻可选的入度为 0 的顶点只有一个，等价于图中存在一条包含全部顶点的路径（哈密顿路径），此时拓扑序就是这条路径上顶点的顺序。
5. 若一个有向图不能构成拓扑排序，则该图一定有环，即一定含有顶点数目大于 **1** 的强连通分量。注意此时该图不一定是强连通图——有环只是存在强连通分量的充分条件，不是整个图强连通。

怎么判断拓扑序列是否唯一、怎么数拓扑序列的个数？这是选择题的高频操作，方法是模拟人脑版

的 **Kahn** 算法并做分支统计：

1. 在草稿纸上写出当前所有入度为 0 的顶点，这就是当前这一步「可选」的集合。
2. 若某个时刻可选集合中只有 1 个顶点，则这一步没有选择余地，只能选它。
3. 若某个时刻可选集合中有 $k \geq 2$ 个顶点，则这里产生了 k 个分支，需要分别往下推演。
4. 把每条分支路上「所有选择都是被迫的」这一情况记为一个完整的拓扑序列，最后统计叶子数即为拓扑序列总数。

由此可以推出两个直接可用的判据：只要全过程中每一步的可选集合都恰好只有 **1** 个顶点，拓扑序列就唯一；而只要出现过一次可选集合大于 1，序列就不唯一。更本质地看，「每一步可选集合都只有 1 个顶点」等价于「图中存在一条依次经过所有顶点的有向路径」，也就是拓扑序恰好是这条路径上的顶点次序。这也解释了为什么「每个顶点都有唯一的前驱和后继」时拓扑序唯一，而「某个顶点有多个直接后继」时通常不唯一。

2.（*2*）基于入度的拓扑排序（*Kahn* 算法）

算法思想：一个顶点的入度为 **0**，说明它依赖的所有顶点都已被处理，此时就可以输出它。先把所有入度为 0 的顶点入队，然后不断出队、输出，并把出队顶点的所有邻接顶点的入度减 1，若某个邻接顶点入度减到 0 就入队。重复直到队列为空。这是一个顺序的拓扑排序过程。

执行步骤：

1. 统计每个顶点的入度，存入数组 `indegree[]`。
2. 把所有入度为 0 的顶点入队。
3. 循环直到队列为空：
   (a) 队头顶点 u 出队，加入结果序列 `res`，计数 `count++`。
   (b) 遍历 u 的所有邻接顶点 v，令 `indegree[v]--`。
   (c) 若 `indegree[v]` 减到 0，则把 v 入队。
4. 循环结束后判环：若 **`count == n`**，则排序成功，**`res`** 即为一个拓扑序列；否则图中存在环，无拓扑序列（有环的顶点入度永远减不到 0）。

```
bool TopologicalSort(ALGraph G, int res[]) {
    int indegree[MAXV] = {0}, cnt = 0, q[MAXV], front = 0, rear = 0;
    for (int i = 0; i < G.numVertices; ++i)          // 1. 统计入度
        for (ArcNode *p = G.vertices[i].firstarc; p; p = p->next)
            indegree[p->adjvex]++;
    for (int i = 0; i < G.numVertices; ++i)          // 2. 入度为 0 者入队
        if (indegree[i] == 0) q[rear++] = i;
    while (front < rear) {                            // 3. 出队并更新邻接点入度
        int u = q[front++];
        res[cnt++] = u;
        for (ArcNode *p = G.vertices[u].firstarc; p; p = p->next) {
            int v = p->adjvex;
            if (--indegree[v] == 0) q[rear++] = v;
        }
    }
    return cnt == G.numVertices;                      // 4. 判环
}
```

手工模拟：设有向图顶点 A、B、C、D、E、F，弧为 $A \to C$、$A \to D$、$B \to D$、$B \to E$、$C \to F$、$D \to F$。

步骤	各顶点入度（**A,B,C,D,E,F**）	队列内容	输出序列
初始	0, 0, 1, 2, 1, 2	A, B	—
出 A	0, 0, 0, 1, 1, 2	B, C	A
出 B	0, 0, 0, 0, 0, 2	C, D, E	A, B
出 C	0, 0, 0, 0, 0, 1	D, E	A, B, C
出 D	0, 0, 0, 0, 0, 0	E, F	A, B, C, D
出 E	0, 0, 0, 0, 0, 0	F	A, B, C, D, E
出 F	全部为 0	空	A, B, C, D, E, F

`count = 6 = n`，拓扑排序成功，得到一个合法序列 A,B,C,D,E,F。由于 A、B 入队次序可交换、C、D、E 的处理次序也有多种选择，该图的拓扑序列不唯一（例如 B,A,E,C,D,F 同样合法）。

复杂度：统计入度需扫描所有边，$O(|V|+|E|)$；每个顶点至多入队出队一次，$O(|V|)$；每条弧在出队时被扫描一次，$O(|E|)$。因此邻接表存储时总时间复杂度为 $O(|V|+|E|)$；若用邻接矩阵存储，每次找入度为 0 的顶点都要扫描一整行，复杂度升为 $O(|V|^2)$。空间复杂度 $O(|V|)$。

3.（3）基于 DFS 的拓扑排序

DFS 也能实现拓扑排序：在 DFS 中，顶点 u 的所有后继顶点一定会在 u 之前完成（回溯）。因此只要按「完成时间」把顶点压入结果栈（或逆序记录），最后得到的序列就是拓扑序列。若直接按完成次序顺序输出，得到的则是逆拓扑序列。

逆拓扑序列的判别

把 DFS 中「输出顶点信息」的语句移到退出递归之前（即先递归处理所有后继，再输出当前顶点），遍历一个有向无环图，输出结果中每个顶点都排在其所有后继之后，因此得到的是逆拓扑有序序列。反过来，要得到正序拓扑序列，需要把完成序列反转（或用栈）。这个「改一行语句」的考法在统考中出现过。

4.（4）拓扑排序与判环

有向图判环的常用方法：

1. 拓扑排序法（最常用）：执行 Kahn 算法后，若出队顶点数 `count < n`，说明存在环。因为环上的顶点入度永远无法减到 0。
2. **DFS** 三色标记法：DFS 中用一个 `state[]` 数组区分「未搜索 (0) / 搜索中 (1) / 已完成 (2)」。若在搜索过程中遇到了一个 `state` 为 1 的顶点，说明回到了当前递归路径上的顶点，即存在环。搜索完成一个顶点后置 `state = 2`，遇到 2 的顶点直接返回（不会重复检测）。
3. **DFS** 路径栈法：用栈记录当前遍历路径上的顶点，若遇到已在栈中的顶点则有环。

无向图判环的方法：

1. **DFS/BFS** 法：从某顶点 v 出发搜索，若在搜索过程中再次遇到 v，则图有环。对每个顶点都做一次即可找出所有环上的顶点。
2. 剥洋葱法：把度为 1（或 0）的顶点不断入队并删除，同时更新邻接点的度；若最终出队顶点数 `count < n`，说明剩下的是环，图有环。
3. 并查集法：依次处理每条边，若边的两端点已在同一集合中，则加入该边会成环。
4. 边数与连通性法：对 n 个顶点、e 条边的无向图，若 $e > n-1$ 则一定有环；若 $e = n-1$ 且连通则一定是树（无环）。

拓扑排序与邻接矩阵的三角形态

DAG（AOV 网）中各顶点的地位是平等的，顶点编号是人为的。因此可以按照拓扑排序的结果对顶点重新编号，使得新编号下所有的弧都从小编号指向大编号，从而新生成的邻接矩阵是（上）三角矩阵。

反之，若一个图的邻接矩阵是三角矩阵，则该图必然存在拓扑序列，但拓扑序列不一定唯一。注意方向：三角矩阵 ⇒ 有拓扑序列（存在性）；但「有拓扑序列 ⇒ 邻接矩阵是三角矩阵」需要允许重新编号后才成立。

典型考法：给出邻接矩阵「主对角线以下元素均为零」，问拓扑序列的结论。此时按矩阵下标编号，只有小编号指向大编号的弧，沿任何有向路径顶点编号都严格增大，故不可能有环，拓扑序列一定存在；但由于没有具体图的信息，无法判断是否唯一，结论是「存在，可能不唯一」。

DAG 描述表达式：有向无环图可以让相同的子表达式共用同一个顶点，从而避免重复建立结点，这是 DAG 在编译原理中用于表达式优化的经典应用，也是统考出现过的冷门考点。它之所以能用 DAG 而不是树来描述表达式，正是因为「同一个子表达式在表达式中出现多次」这一情形可以被合并在一个顶点上，而树结构无法共享子树。

以表达式 $(x+y)\cdot((x+y)/x)$ 为例，逐步建图：

1. 操作数 x 一个顶点，操作数 y 一个顶点。
2. 子表达式 $(x+y)$ 一个顶点，接收 x、y 两条入弧。它在原表达式中出现了两次，但只需建一个顶点。
3. 子表达式 $((x+y)/x)$ 一个顶点，入弧分别来自 $(x+y)$ 顶点和 x 顶点。
4. 最外层的乘法一个顶点，入弧来自 $(x+y)$ 顶点和 $((x+y)/x)$ 顶点。
5. 输出顶点（也可省去）指向该乘法顶点。

因此最少需要的顶点数为 x、y、加法、除法、乘法共 **5** 个。若改用二叉树表示而不共享子表达式，则需要重复建出第二个加法结点，顶点数会增加到 6 个以上——这正是题目「需要的顶点个数至少是」的考点所在。解题要领是：先把表达式按树展开，再把所有结构相同的子表达式合并成一个顶点，最后数顶点。

出题角度

拓扑排序历年考查 8 次左右，是本章最高频的考点之一。典型问法：(1) 问「可以得到不同拓扑序列的个数」，解法是逐步枚举当前入度为 0 的顶点，做树形分支统计；(2) 给出邻接表存储的有向图，问拓扑排序时间复杂度，答案是 $O(n+e)$；(3) 判断某个序列是否是该图的拓扑序列，逐个检查每条弧的方向约束即可；(4) 问「仅当图中存在一条包含所有顶点的路径时，拓扑序列唯一」；(5) 结合邻接矩阵的三角形态考查拓扑排序与压缩存储。

（四）关键路径

1.（1）AOE 网与关键路径的概念

AOE 网（Activity On Edge Network）：在一个表示工程的带权有向图中，用顶点表示事件、用有向边表示活动、用边上的权值表示活动的持续时间。其中没有入边的顶点称为源点（工程的开始），没有出边的顶点称为汇点（工程的结束）。

AOE 网的两条性质：

1. 只有在某顶点所代表的事件发生后，从该顶点出发的各条有向边所代表的活动才能开始。
2. 只有在进入某顶点的各条有向边所代表的活动都已结束时，该顶点所代表的事件才能发生。另外，有些活动是可以并行进行的。

AOV 网与 **AOE** 网对比：

比较项	**AOV 网**	**AOE 网**
顶点含义	表示活动	表示事件
边含义	无权值，仅表示活动之间的先后顺序	有权值，表示活动的持续时间
典型问题	拓扑排序、判环	关键路径、工程最短工期

路径长度：路径上各个活动的持续时间之和。从源点到汇点具有最大长度的路径称为关键路径，关键路径上的活动称为关键活动。关键路径的长度就是完成整个工程所需的最短时间。

必背性质：

1. 关键路径可能不唯一，但所有关键路径的长度都相同。
2. 增大某个关键活动的持续时间，一定会使关键路径变长，从而延长工期。
3. 缩短某个关键活动的持续时间，不一定能缩短工期——因为可能存在多条关键路径，只有同时缩短所有关键路径上的关键活动才能缩短总工期。
4. 关键活动不能任意缩短：一旦缩短到一定程度，该关键活动可能变成非关键活动，关键路径会转移到其他路径上。

2.（*2*）四个关键量

量	含义	计算公式
$ve(k)$	事件 v_k 的最早发生时间，也是从源点到 v_k 的最长路径长度	$ve(\text{源点})=0$；$ve(k)=\max\{ve(j)+w(j,k)\}$，$j$ 取遍 k 的所有前驱
$vl(k)$	事件 v_k 的最迟发生时间，即在不推迟整个工程的前提下该事件最迟必须发生的时间	$vl(\text{汇点})=ve(\text{汇点})$；$vl(k)=\min\{vl(j)-w(k,j)\}$，$j$ 取遍 k 的所有后继
$e(i)$	活动 a_i 的最早开始时间，即该活动弧的起点所表示事件的最早发生时间	若边 $<v_k,v_j>$ 表示 a_i，则 $e(i)=ve(k)$
$l(i)$	活动 a_i 的最迟开始时间，即该活动弧的终点的最迟发生时间减去该活动持续时间	若边 $<v_k,v_j>$ 表示 a_i，则 $l(i)=vl(j)-w(v_k,v_j)$

时间余量：$d(i)=l(i)-e(i)$，表示在不增加整个工程总时间的情况下，活动 a_i 可以拖延的时间。

$$d(i)=0 \iff l(i)=e(i) \iff a_i \text{ 是关键活动}$$

关键活动组成的路径就是关键路径。

计算 ve 与 vl 的方向不要弄反

- ve 从源点向汇点正推，用 **max**（取最长路径，因为事件必须等所有前驱活动都完成）。
- vl 从汇点向源点倒推，用 **min**（取最紧的约束，因为不能推迟后面任何一个后继）。

一个常见的错误是算 vl 时误用 max，或算 ve 时误用 min。记住口诀：正推求最早取最大，倒推求最迟取最小。

3.（*3*）标准解法五步

1. 正推求所有事件的 $ve()$：从源点开始，按拓扑序依次计算 $ve(k)=\max\{ve(j)+w(j,k)\}$。
2. 倒推求所有事件的 $vl()$：令 $vl(\text{汇点})=ve(\text{汇点})$，按逆拓扑序依次计算 $vl(k)=\min\{vl(j)-w(k,j)\}$。
3. 求所有活动的最早开始时间 $e()$：对表示活动的边 $<v_k,v_j>$，$e=ve(k)$。
4. 求所有活动的最迟开始时间 $l()$：对表示活动的边 $<v_k,v_j>$，$l=vl(j)-w(v_k,v_j)$。
5. 求所有活动的时间余量 $d=l-e$：$d=0$ 的活动就是关键活动，由关键活动连成的源点到汇点的路径就是关键路径。

4.（*4*）完整手工模拟

设有 AOE 网，顶点 $v_1\sim v_6$，v_1 为源点、v_6 为汇点，活动（边）如下：

活动	弧	持续时间	活动	弧	持续时间
a_1	$v_1\to v_2$	3	a_5	$v_3\to v_4$	4
a_2	$v_1\to v_3$	2	a_6	$v_3\to v_6$	3
a_3	$v_2\to v_4$	2	a_7	$v_4\to v_6$	2
a_4	$v_2\to v_5$	3	a_8	$v_5\to v_6$	1

第一步：正推求 ve（从 v_1 向 v_6，取 **max**）

$$ve(1)=0$$

$$ve(2)=ve(1)+3=3$$

$$ve(3)=ve(1)+2=2$$

$$ve(4)=\max\{ve(2)+2,\ ve(3)+4\}=\max\{5,\ 6\}=6$$

$$ve(5)=ve(2)+3=6$$

$$ve(6)=\max\{ve(3)+3,\ ve(4)+2,\ ve(5)+1\}=\max\{5,\ 8,\ 7\}=8$$

第二步：倒推求 vl（从 v_6 向 v_1，取 **min**），$vl(6)=ve(6)=8$

$$vl(6)=8$$

$$vl(5)=vl(6)-1=7$$

$$vl(4)=vl(6)-2=6$$

$$vl(3)=\min\{vl(6)-3,\ vl(4)-4\}=\min\{5,\ 2\}=2$$

$$vl(2)=\min\{vl(5)-3,\ vl(4)-2\}=\min\{4,\ 4\}=4$$

$$vl(1)=\min\{vl(2)-3,\ vl(3)-2\}=\min\{1,\ 0\}=0$$

第三、四、五步：求 e、l、d（$e=ve$(弧尾)，$l=vl$(弧头) − 权值）

活动	弧	权值	$e=ve$(尾)	$l=$ vl(头) − 权值	$d=l-e$	是否关键
a_1	$v_1\to v_2$	3	0	$4-3=1$	1	否
a_2	$v_1\to v_3$	2	0	$2-2=0$	**0**	是
a_3	$v_2\to v_4$	2	3	$6-2=4$	1	否
a_4	$v_2\to v_5$	3	3	$7-3=4$	1	否
a_5	$v_3\to v_4$	4	2	$6-4=2$	**0**	是
a_6	$v_3\to v_6$	3	2	$8-3=5$	3	否
a_7	$v_4\to v_6$	2	6	$8-2=6$	**0**	是
a_8	$v_5\to v_6$	1	6	$8-1=7$	1	否

关键活动为 a_2、a_5、a_7，关键路径为 $v_1\to v_3\to v_4\to v_6$，长度 $2+4+2=8$，与 $ve(6)=8$ 一致。结果的工程解读：整个工程最短需要 8 个时间单位完成。事件 v_3 必须准时在第 2 个时间单位发生（$ve(3)=vl(3)=2$），也就是活动 a_2 一天都不能拖；事件 v_4 必须在第 6 个时间单位发生，由 a_5 保障。反观活动 a_6（$v_3\to v_6$，时长 3），它的最早开始时间是 2、最迟开始时间是 5，余量为 3，意味着只要在第 **5** 个时间单位之前开工即可，提前或推后 3 个单位都不影响总工期。活动 a_8（$v_5\to v_6$，时长 1）余量为 1，a_1、a_3、a_4 的余量也都是 1。这些余量正是工程调度中「松弛时间」的概念。

如果要把工期从 **8** 压缩到 **7**，应当做什么？必须先找出所有关键路径（本例中只有一条：$v_1\to v_3\to v_4\to v_6$），然后缩短这条路径上某个活动的持续时间。但要注意两个约束：其一，如果图中有多条关键路径，必须同时缩短每一条关键路径上的活动，只缩短其中一条路径上的活动不会改变总工期（这也是统考反复考查的结论）；其二，缩短幅度受到余量的限制——若把 a_5 从 4 缩到 3，则关键路径长度变为 $2+3+2=7$，此时路径 $v_1\to v_3\to v_4\to v_6$ 仍然是唯一最长路径；但若继续把 a_5 缩到 2，关键路径长度变为 6，而 $v_1\to v_2\to v_5\to v_6$ 的长度为 $3+3+1=7$，关键路径就转移到了另一条路上，此时 a_5 反而不是关键活动了。这就是「关键活动不能任意缩短，缩到一定程度会变成非关键活动」的具体含义。

5.（5）快速手算关键路径：删边法

考试时间紧张时可以用下面这个技巧：

1. 只求 $ve(k)$，从源点开始正推。
2. 观察每个顶点：如果指向 k 的多条边产生的 ve 值不相等（即 $ve(j)+w(j,k)$ 有大有小），那么产生较小值的那条边一定是非关键边，可以把它从图中删掉；产生最大值的边（可能有两条及以上）保留。

3. 删边后可以继续向源点方向回溯删除，直到遇到一个出度大于 **1** 的交叉点为止。
4. ve 全部求完后，在剩余的边上找出能连通源点和汇点的路径，这些路径就是关键路径。

以本章的模拟为例：$ve(4)=\max\{ve(2)+2,\ ve(3)+4\}$，其中 $ve(2)+2=5<6$，因此边 $v_2\to v_4$（a_3）可删；继续向源点回溯，v_2 的入度只有 1，可继续删 $v_1\to v_2$（a_1），此时遇到 v_1（出度 $2>1$），停止。同理 $ve(6)=\max\{5,8,7\}$，$v_3\to v_6$ 与 $v_5\to v_6$ 可删。剩余边中连接 v_1 与 v_6 的路径只有 $v_1\to v_3\to v_4\to v_6$，即为唯一的关键路径。

出题角度

关键路径历年考查 4 次左右。典型问法：(1) 给出 AOE 网，问「活动 d 的最早开始时间和最迟开始时间分别是」，需要完整算出 ve、vl 再套 e、l 公式；(2) 问「时间余量最大的活动」，余量最大者偏离关键路径最远；(3) 问「加快哪些活动的进度可以缩短工程工期」，答案是必须覆盖所有关键路径的公共活动集合；(4) 判断叙述正误，注意「关键路径是从源点到汇点边数最多的路径」是错的（比的是持续时间之和而不是边数），「缩短任一关键活动一定能缩短工期」也是错的（多条关键路径时未必）。

本章小结表

一、核心公式与结论速查

结论	表达式 / 内容
无向图度数之和	$\sum TD(v_i)=2e$
有向图入度、出度之和	$\sum ID(v_i)=\sum OD(v_i)=e$
无向完全图边数	$n(n-1)/2$
有向完全图边数	$n(n-1)$
连通图最少边数	$n-1$（生成树）
非连通无向图最多边数	$(n-1)(n-2)/2$
强连通有向图最少边数	n（一个环）
生成树边数	$n-1$
最短路径所含边数上限（非负权）	任意连通图两点间最短路径至多 $n-1$ 条边（最短路径必是简单路径）
度数为奇数的顶点个数	必为偶数
邻接矩阵幂	$A^k[i][j]=i$ 到 j 长度为 k 的路径数目

二、六类应用算法总表

算法	算法思想	关键数组 / 结构	时间复杂度	空间复杂度	适用
Prim	选点，贪心	`dist`、`visited`	$O(\|V\|^2)$	$O(\|V\|)$	边稠密图
Kruskal	选边，贪心 + 并查集	边表排序、`parent`	$O(\|E\|\log\|E\|)$	$O(\|V\|)$	边稀疏、点多
Dijkstra	贪心，分已确定/未确定集合	`dist`、`path`、`final`	$O(\|V\|^2)$	$O(\|V\|)$	非负权单源最短路
Floyd	动态规划，逐个允许中转	`dist[][]`、`path[][]`	$O(\|V\|^3)$	$O(\|V\|^2)$	全源最短路，允许负权无负环

算法	算法思想	关键数组 / 结构	时间复杂度	空间复杂度	适用
拓扑排序	入度为 0 者入队，出队后邻接点入度减 1	`indegree`、队列	邻接表 $O(\|V\|+\|E\|)$；邻接矩阵 $O(\|V\|^2)$	$O(\|V\|)$	DAG、判环
关键路径	求最长路径，找 $d=0$ 的活动	`ve`、`vl`、`e`、`l`	$O(\|V\|+\|E\|)$	$O(\|V\|+\|E\|)$	AOE 网工期分析

三、四种存储结构速查

结构	空间	求入度	求出度	删边	删点	适用	唯一性
邻接矩阵	$O(\|V\|^2)$	$O(\|V\|)$	$O(\|V\|)$	$O(1)$	$O(\|V\|)$	稠密图	唯一
邻接表	$O(\|V\|+\|E\|)$	难，$O(\|V\|+\|E\|)$	易	需定位	不方便（无向图）	稀疏图	不唯一
十字链表	$O(\|V\|+\|E\|)$	易（`hlink`）	易（`tlink`）	较方便	不方便	有向图	不唯一
邻接多重表	$O(\|V\|+\|E\|)$	无向图不区分	无向图不区分	方便	方便	无向图	不唯一

易错清单

概念类易错

1. 回路不是简单路径。简单路径要求顶点全部不重复，而回路首尾顶点相同，至少重复了两个位置。要区分「简单路径」与「简单回路」。
2. 图的「距离」是边数，不是权值之和。带权图中两点间的最短路径长度才是权值之和，术语上不要混用。
3. 连通分量是「极大连通子图」，不是「最大连通子图」，也不是「最小连通子图」。极小连通子图指的是生成树。
4. 强连通图最少 n 条边（构成一个环），但 n 条边不一定是强连通图（可能只是几个不相交的环加上别的结构）。
5. 有向图存在拓扑序列 $\Leftrightarrow$ 无环，与是否连通无关。有环时一定含顶点数大于 1 的强连通分量，但整个图未必强连通。
6. 非连通图只有生成森林，没有生成树。生成树只对连通图定义。
7. 稀疏图与稠密图没有绝对界限，一般以 $|E|<|V|\log|V|$ 为界。不要把它当成严格定义来判定。
8. 有向树要求恰好一个顶点入度为 **0**、其余顶点入度均为 **1**，仅有这一条，出度不限制。

存储结构类易错

1. 「存储稀疏图用邻接矩阵比邻接表更省空间」是错的。邻接矩阵恒为 $O(|V|^2)$，与边数无关；邻接表为 $O(|V|+|E|)$。稀疏图中后者更省。
2. 邻接表表示不唯一，邻接矩阵表示唯一。这条决定了遍历序列与生成树是否唯一。
3. 无向图的邻接表边结点数是 $2e$，有向图是 e。无向图每条边存两份；若问「邻接多重表的边结点数」，则是 e（每边一份）。
4. 邻接表求有向图的入度需要遍历整个邻接表，$O(|V|+|E|)$，不是 $O(1)$；求入度快的是逆邻接表和十字链表。

5. 删除顶点在邻接矩阵中是 $O(|V|)$（将该行该列清零）；只有当题目要求把矩阵元素移动成紧凑形式时才需要 $O(|V|^2)$。看清题目问的是哪种。
6. 十字链表用于有向图，邻接多重表用于无向图，不要写反。
7. 邻接矩阵的带权形式中，主对角线元素是 **0** 还是 ∞ 要看题目约定，解题时按给定矩阵读，不要自己假设。

遍历类易错

1. **BFS** 必须在顶点入队时就标记 `visited`，不能等到出队时再标记，否则会重复入队。
2. **DFS** 非递归实现弹出时要再次判 `visited`，因为一个顶点可能被多次压栈。
3. 对无向图，调用 **BFS/DFS** 的次数等于连通分量数；对有向图不能这样直接下结论。
4. 邻接矩阵存储时遍历序列唯一（起点固定时），邻接表存储时不唯一。生成树同理。
5. **BFS** 求单源最短路径只适用于无权图或各边权相等的图，带不同权值时必须用 Dijkstra。
6. 无论 DFS 还是 BFS，空间复杂度都是 $O(|V|)$——BFS 是队列大小，DFS 是递归栈深度。

应用算法类易错

1. **Prim** 选点、**Kruskal** 选边。Prim 的时间复杂度只与顶点数有关，适合稠密图；Kruskal 只与边数有关，适合稀疏图。写反是最常见的失分点。
2. **MST** 唯一性：只有「各边权值互不相等」才是 MST 唯一的充分条件；等权边存在时可能有多个 MST，但代价总是唯一的。
3. 「所有权值最小的边一定出现在所有 **MST** 中」是错的。若若干条最小权边构成环，不可能全部选入。
4. **Prim** 与 **Dijkstra** 的 `dist` 含义完全不同：Prim 是「到已选集合的最短边权」（不累加），Dijkstra 是「到源点的最短路径长」（累加）。代码结构相似但更新式不同。
5. **Dijkstra** 不能处理负权边。原因是贪心性质失效：一个顶点的最短距离一旦被确定就不再更新，而负权边可能让后来确定的顶点提供更短的路径。
6. **Dijkstra** 的三层判断中，松弛只用「从 u 出发」的边，不是「与 u 相连」的边——有向图与无向图的区别就在这里。
7. **Floyd** 的三重循环 k 必须在最外层。k 是 DP 的阶段（允许中转的顶点集合），i, j 是同阶段内的状态。顺序写错结果会出错。
8. **Floyd** 允许负权边但不允许负权回路；存在负权回路时最短路径不存在。
9. 拓扑排序的出队顺序不唯一，但「每个时刻入度为 0 的顶点只有一个」时唯一，此时图中存在一条包含所有顶点的路径。
10. 关键路径比的是路径上活动持续时间之和，不是边的条数。ve 正推取 max，vl 倒推取 min，切勿弄反。
11. 缩短关键活动不一定缩短工期：存在多条关键路径时，只有同时缩短所有关键路径上的活动才行。而延长任一关键活动一定延长工期。
12. 关键活动不能任意缩短，缩短到一定程度它会变成非关键活动，关键路径转移。
13. **AOV** 网顶点表示活动，**AOE** 网顶点表示事件；AOE 网的边才带权值（活动持续时间）。

第六章查找

本章考纲要求

（一）查找的基本概念

（二）顺序查找法

（三）分块查找法

（四）折半查找法

（五）树形查找

1. 二叉搜索树
2. 平衡二叉树
3. 红黑树

（六）B 树及其基本操作、B+ 树的基本概念

（七）散列（hash）表

（八）字符串模式匹配

（九）查找算法的分析及应用

查找这一章在统考中的份量很重，选择题几乎年年必出，而且往往是「两道单选 + 一道大题」的组合。它的特点是知识点碎、公式多、每一种结构都要会手算。真正拉开分差的不是定义，而是三类基本功：**ASL** 的推导与计算、插入/删除后的调整手算、判定类问题的边界。本章按考纲条目顺序展开，所有 ASL 公式都给出推导过程。

一、查找的基本概念

（一）查找表、关键字、查找长度

查找（**search**）：在数据集合中寻找满足某种条件的数据元素的过程。

查找表（**search table**）：用于查找的数据集合，由同一类型的数据元素（或记录）组成。查找表可以是一个数组、一个链表、一棵树、一张散列表，因此「查找表」讲的是逻辑上的数据集合，不是某种特定存储结构。

关键字（**key**）：数据元素中唯一标识该元素的某个数据项的值。基于关键字的查找，结果应当是唯一的。

按对查找表的操作方式，可以把查找表分成两类：

类型	支持的操作	典型结构
静态查找表	只查找，不改变表的内容	顺序表上的顺序查找、折半查找
动态查找表	查找之外还要插入、删除	二叉搜索树、AVL 树、红黑树、B 树、散列表

这个区分很重要：折半查找不能用在动态查找表上，因为有序顺序表插入删除要移动大量元素。

查找长度（**search length**）：一次查找运算中比较关键字的次数。注意它数的是「比较关键字的次数」，不是「访问结点的次数」，也不是「循环次数」。

平均查找长度（**ASL, Average Search Length**）：所有查找过程中关键字比较次数的平均值。

$$ASL = \sum_{i=1}^{n} P_i C_i$$

其中 P_i 是查找第 i 个元素的概率，C_i 是找到第 i 个元素所需的比较次数。等概率时 $P_i = \frac{1}{n}$。

ASL 必须分「成功」和「失败」两种情况讨论，两者用的分母完全不同：

- $ASL_{\text{成功}} = \dfrac{\text{查找成功时比较关键字的总次数}}{\text{表中元素的个数}}$
- $ASL_{\text{失败}} = \dfrac{\text{查找失败时比较关键字的总次数}}{\text{查找失败的情况数（即失败位置的个数）}}$

失败 ASL 的分母最容易搞错

$ASL_{\text{失败}}$ 的分母不是元素个数 n，而是「查找失败可能落在几个位置上」。

- 顺序表的顺序查找：失败位置有 $n+1$ 个（在每个元素前后以及表尾之后）；
- 折半查找判定树：失败结点（外部结点）有 $n+1$ 个；
- 散列表的开放定址法：分母是初次散列可能落在的地址个数，即散列函数值域的大小，通常是 p，不一定是表长 m。

分母选错，整道题就全错。

（二）用判定树统一理解 ASL

所有查找算法都可以画成一棵判定树（**decision tree**），ASL 就是判定树上各结点的平均层数。这让不同算法的分析变得统一：

- 圆结点（内部结点）：查找表里真实存在的元素，对应「查找成功」；成功时的查找长度 = 该结点所在层数。
- 矩形结点（外部结点 / 失败结点）：表示一个查找失败的区间，对应「查找失败」；失败时的查找长度 = 其父结点（内部结点）所在层数，因为到达失败结点之前，比较动作都发生在内部结点上，最后那次比较完才知道「不存在」。

判定树的三条通用规则

1. n 个元素的静态查找表，判定树有 n 个内部结点 + $n+1$ 个外部结点（n 个关键字把整个值域切成 $n+1$ 段）。
2. 成功结点的查找长度 = 自身层数（根为第 1 层）。
3. 失败结点的查找长度 = 其父结点所在层数，外部结点自身不产生比较。

（三）一个最小的算例

设查找表为有序表 $\{7, 12, 20\}$，判定树为：根 12，左孩子 7，右孩子 20。

- $ASL_{\text{成功}} = \dfrac{1+2+2}{3} = \dfrac{5}{3}$。
- 四个失败区间分别在 7 左、7–12 之间、12–20 之间、20 右，父结点层数依次为 2、1……严格来说，左侧失败结点挂在 7 上（层数 2），7–12 之间的失败结点挂在 12 上（层数 1），12–20 之间

挂在 12 上（层数 1），右侧挂在 20 上（层数 2）。故 $ASL_{失败}=\dfrac{2+1+1+2}{4}=\dfrac{3}{2}$。

二、顺序查找法

顺序查找（**sequential search**）：从表的一端开始，逐个把元素的关键字与给定值比较。

（一）一般线性表的顺序查找

```
typedef struct {
    int *elem;      // 动态数组基址
    int tablelen;   // 表长
} SSTable;

// 朴素写法：每次循环要做两次判断
int seq_search(SSTable ST, int target) {
    for (int i = 0; i < ST.tablelen; ++i) {
        if (ST.elem[i] == target) return i;   // 判断是否命中
        // for 的条件里还要判断 i 是否越界
    }
    return -1;   // 查找失败
}
```

朴素写法每一轮要做两次判断：一次判断「当前元素是不是目标」，一次判断「表是否已经走完」。把这两个判断合并成一个，就是哨兵的作用。

（二）哨兵（监视哨）的作用

哨兵：在查找表的一端放置一个与待查关键字相等的元素。这样循环条件里只剩「当前元素是否等于目标」一个判断，越界检查被哨兵自然「兜住」。

```
// 哨兵版：elem 至少分配 tablelen+1 个单元
int seq_search(SSTable ST, int target) {
    ST.elem[ST.tablelen] = target;   // 表尾放置哨兵
    int i = 0;
    while (ST.elem[i] != target) ++i; // 少一次越界判断
    return i;   // 返回 tablelen 说明是哨兵命中，即查找失败
}
```

哨兵的位置不止一种约定

- 放在表尾（下标 n），从下标 0 向表尾扫：找不到时必然停在 n，靠「返回值是否等于 n」判失败。
- 放在表头（下标 **1**），数组 `elem[1..n]` 存数据，从下标 n 向表头扫：找不到时必然停在 0，靠「返回值是否为 0」判失败。这是王道教材的写法。

两种约定都对，关键是自己算 **ASL** 时不要和约定混用。无论哪种，哨兵都只减少判断次数，不改变 **ASL** 的值（哨兵本身也要参与一次比较，而这次比较正好等价于原来那次越界判断）。

（三）无序表的 ASL 推导

设表中有 n 个元素，等概率查找，逐个从前向后比较。

查找成功：找第 1 个元素比 1 次，第 2 个比 2 次，……，第 n 个比 n 次。

$$ASL_{\text{成功}} = \frac{1}{n}\sum_{i=1}^{n} i = \frac{1}{n}\cdot\frac{n(n+1)}{2} = \frac{n+1}{2}$$

查找失败：无序表只有真正走完整张表（比较了全部 n 个元素）才能确定失败。所以每一种失败情况都是 n 次比较：

$$ASL_{\text{失败}} = \frac{1}{n}\cdot\underbrace{(n+n+\cdots+n)}_{n\text{个}} = n$$

无序表顺序查找的必背结论

$$ASL_{\text{成功}} = \frac{n+1}{2}, \qquad ASL_{\text{失败}} = n$$

时间复杂度 $O(n)$；顺序查找既适用于顺序存储，也适用于链式存储（这是它相对折半查找最大的优势）。

（四）有序表顺序查找的 **ASL** 推导

若表已按关键字递增有序，顺序查找可以提前终止：一旦发现 $ST.elem[i] > target$，就能确定表中不存在 $target$，不必再往后找。

```
int seq_search(SSTable ST, int target) {
    for (int i = 0; i < ST.tablelen; ++i) {
        if (ST.elem[i] == target) return i;   // 命中
        if (ST.elem[i] > target)  break;      // 已经不可能再找到
    }
    return -1;
}
```

查找成功的 ASL 不变，仍是 $\frac{n+1}{2}$：成功命中仍发生在第 i 个位置，比较 i 次。

查找失败变得复杂。失败情况共有 $n+1$ 种，画成判定树，失败结点分布在 $n+1$ 个区间里。用 T_i 表示落在第 i 个内部结点左侧或中间的失败情况，其需要比较的次数就是它撞上的那个内部结点的层数：

失败区间	撞上的内部结点	比较次数
第 1 个元素之前	$elem[1]$	1
$elem[1]$ 与 $elem[2]$ 之间	$elem[2]$	2
⋮	⋮	⋮
$elem[n-1]$ 与 $elem[n]$ 之间	$elem[n]$	n
第 n 个元素之后	$elem[n]$	n

前 n 个失败区间的比较次数是 $1, 2, \cdots, n$，最后一个失败区间比较 n 次（要走完整张表才知道没有比它大的）。等概率时：

$$ASL_{\text{失败}} = \frac{(1+2+\cdots+n)+n}{n+1} = \frac{\frac{n(n+1)}{2}+n}{n+1} = \frac{n}{2}+\frac{n}{n+1}$$

有序表顺序查找的两个坑

1. 成功 **ASL** 不变，只有失败 **ASL** 变小：n 对比 $\frac{n}{2}+\frac{n}{n+1}$。$n$ 趋于无穷时两者都趋于 $\frac{n}{2}$，但有限 n 下后者略小——不要记成「有序就都减半」。
2. 题目若问「查找失败最多需要比较多少次」，有序表答案是 n，不是 $n+1$。因为最早撞上「大于目标」的元素时就已经能判定失败，最坏情况是目标比所有元素都大，此时走完 n 个元素、比较 n 次。

（五）概率不等时：把常查的元素放前面

若各元素被查概率 $p_1, p_2, \cdots, p_n$ 不等，$ASL = \sum p_i c_i$ 就是按概率加权。要让它最小，应把概率大的元素放在靠前的位置（查找代价小的位置）。这就是「按频度排序」的优化。

统考考法

顺序查找在统考中多以应用题形式出现，要求写出查找过程、计算 $ASL_{成功}$ 与 $ASL_{失败}$，或者比较不同查找策略的比较次数。选择题则常考「哪种存储结构适用顺序查找」（答案是顺序存储和链式存储都适用）。

三、分块查找法

分块查找（**block search**），又叫索引顺序查找。它把顺序查找和折半查找的优点结合起来：查找效率高于顺序查找，而插入删除又不像有序顺序表那样需要移动大量元素。

1. 索引顺序表的结构

分块查找把长度为 n 的表分成 b 块，每块 s 个元素，并要求：

- 块内无序：块内的记录不要求有序；
- 块间有序：第 $i+1$ 块中所有记录的关键字都大于第 i 块中所有记录的关键字。

再为每一块建立一个索引项，保存这一块的最大关键字和存储区间：

```
typedef struct {
    int maxvalue;       // 本块的最大关键字
    int low, high;      // 本块在表中占据的下标区间
} IndexItem;
IndexItem indexTable[100];
```

因为索引表本身是块间有序的，所以索引表上可以用顺序查找，也可以用折半查找。

（一）查找过程：两步

1. 在索引表中确定待查关键字可能所在的块。索引表有序 ⇒ 可顺序查、也可折半查。
2. 在所确定块的块内顺序查找。块内通常无序 ⇒ 只能顺序查。

索引表命中不等于查找成功

如果块内是无序的，即便在索引表里已经查到了「某个块的最大关键字恰好等于 $target$」，也必须进入该块做块内查找才算真的找到——因为块内无序，那个最大关键字可能并不真实存在于该块中（比如索引表写的是块的上界，而块内元素都小于它）。

算 ASL 时忘记这一点，会漏掉块内的那几次比较。这是分块查找最容易出错的地方。

3. 用折半查找查索引表时的落点

如果索引表用折半查找，要注意终止时的指针位置：

- 若某次比较命中了某个索引项，直接去该索引项所指的块内找即可；
- 若索引表中不存在目标关键字，折半查找终止于 $low > high$，此时 low 指向第一个关键字大于目标的索引项。由于索引项保存的是各块的最大关键字，目标只可能落在 low 所指的那一块里，因此应该去 low 所指的块内继续找；
- 若终止时 low 已经超出索引表范围，说明目标比所有块的最大关键字都大，查找失败。

（二）ASL 推导与最优块长

设表长 n，分成 b 块，每块 s 个元素，于是

$$n = b \cdot s$$

设索引查找的平均长度为 L_I，块内查找的平均长度为 L_S，则

$$ASL = L_I + L_S$$

（1）索引表用顺序查找

索引表有 b 项，顺序查找的平均长度为 $\dfrac{b+1}{2}$；块内有 s 个元素，顺序查找平均长度为 $\dfrac{s+1}{2}$。所以

$$ASL_{\text{成功}} = L_I + L_S = \frac{b+1}{2} + \frac{s+1}{2}$$

把 $b = \dfrac{n}{s}$ 代入：

$$ASL_{\text{成功}} = \frac{1}{2}\left(\frac{n}{s} + 1\right) + \frac{s+1}{2} = \frac{n}{2s} + \frac{s}{2} + 1 = \frac{s^2 + 2s + n}{2s}$$

（2）求最优块长 s

把 s 看成连续变量，对 s 求导：

$$\frac{\mathrm{d}}{\mathrm{d}s}\left(\frac{n}{2s} + \frac{s}{2} + 1\right) = -\frac{n}{2s^2} + \frac{1}{2}$$

令导数为 0，得 $s^2 = n$，即

$$\boxed{s = \sqrt{n}}$$

此时块数 $b = \dfrac{n}{s} = \sqrt{n}$，最小平均查找长度为

$$ASL_{\min} = \frac{n}{2\sqrt{n}} + \frac{\sqrt{n}}{2} + 1 = \frac{\sqrt{n}}{2} + \frac{\sqrt{n}}{2} + 1 = \sqrt{n} + 1$$

分块查找必背

$$ASL_{\text{成功}} = \frac{b+1}{2} + \frac{s+1}{2} = \frac{s^2 + 2s + n}{2s}$$

最优块长 $s = \sqrt{n}$，此时 $ASL_{\min} = \sqrt{n} + 1$。

自验证：$n=10000$，$s=100$，$b=100$，$ASL=\frac{101}{2}+\frac{101}{2}=101=\sqrt{10000}+1$，与公式一致。

（3）索引表用折半查找

索引表有 b 项，折半查找的 ASL 约为 $\lceil\log_2(b+1)\rceil$，块内仍为 $\frac{s+1}{2}$，故

$$ASL_{\text{成功}}=\lceil\log_2(b+1)\rceil+\frac{s+1}{2}$$

此时 ASL 关于 s 单调递减，趋于 $\lceil\log_2 b\rceil+\frac{s}{2}$，因此块长应取得更小——块数多、每块小，让索引表承担更多查找工作。极限情形 $s=1$ 就退化成「索引 + 逐个」的二层结构，接近折半查找的水平。

（4）查找失败的 ASL

分块查找的失败 ASL 计算繁琐（要枚举落进每个块后块内一次不中的情况），统考一般不要求整表求 $ASL_{\text{失败}}$，但可能问某个具体关键字的查找失败比较次数，此时按「索引表比较次数 + 块内比较次数」细致数一遍即可。

（三）优缺点与动态版本

- 优点：查找效率高于顺序查找；插入删除只需在块内移动，不必整体搬移。
- 缺点：需额外存储索引表，且要维护「块间有序」这一前提。
- 动态版本：块内改用链表存储，插入删除进一步简化，适合「动态查找表」。

统考考法

分块查找是考纲内长期未被单独考过的条目，属于「冷门但可能突袭」的考点。一旦出现，几乎必然考 $ASL_{\min}=\sqrt{n}+1$ 这个结论（注意有 $+1$）以及最优块长的推导。选择题也可能问「分块查找为什么块内可以无序」。

四、折半查找法

折半查找（**binary search**），又称二分查找。它每一步都把待查区间缩小一半。

（一）适用条件

折半查找的两个必要条件

1. 存储结构必须是顺序存储（数组）。因为折半查找需要 $O(1)$ 时间跳到区间中点，也就是要随机存取能力。
2. 元素必须按关键字有序（通常要求递增）。

因此：有序链表不适合；无序数组不适合；有序静态链表不适合（逻辑顺序由链接字段决定，不能按下标直接取中点）；无序静态链表更不适合。

只有「有序 + 顺序存储（可随机存取）」才适合。

（二）基本实现（左闭右闭）

```
int binary_search(int nums[], int n, int target) {
    int low = 0, high = n - 1, mid;
    while (low <= high) {
        mid = low + (high - low) / 2;   // 等价于 (low+high)/2，下取整
        if (nums[mid] == target) return mid;      // 命中
        else if (nums[mid] > target) high = mid - 1;  // 前半区间
        else low = mid + 1;                        // 后半区间
    }
```

```
    return -1;   // 失败
}
```

循环终止条件是 $low > high$。终止时 low 恰好指向第一个大于 $target$ 的元素的下标——这个性质在「查找插入位置」类问题里很有用。

（三）判定树的构造

折半查找判定树：把每次比较的 mid 作为结点、把递归的左右半个区间作为左右子树，得到的二叉树。

设 low, h 为当前区间的两端，取 $mid = \left\lfloor \dfrac{low + high}{2} \right\rfloor$（下取整），则：

- 区间元素个数为奇数时，mid 分割后左右两部分元素个数相等；
- 区间元素个数为偶数时，mid 分割后左半部分比右半部分少一个元素。

由此得到判定树最重要的形态特征：

$$\text{对判定树中任一结点：右子树结点数} - \text{左子树结点数} \in \{0, 1\}$$

反过来，若采用上取整 $mid = \left\lceil \dfrac{low + high}{2} \right\rceil$，则对任一结点：

$$\text{左子树结点数} - \text{右子树结点数} \in \{0, 1\}$$

mid 取整方式必须全程一致

上取整和下取整产生的判定树是互为镜像的两棵树，但同一棵树里不能混用。统考常给若干棵二叉树，问「哪一棵可能是折半查找判定树」，判定方法就是：在每个结点上都检查左右子树规模的关系是否符合同一个取整规则。

只要有一处不符（比如某个偶数长度区间把多出来的一半放到了错误的一侧），该树就不可能是判定树。这是 2017 年那道判定树选择题的核心判据。

（四）判定树是平衡二叉树

由上面的规模关系可以推出，判定树中任一结点的左右子树高度差不超过 1，因此：

折半查找判定树一定是平衡二叉树

判定树的形态与关键字的具体取值无关，只与元素个数 n 有关。也就是说，对给定 n，折半查找判定树是唯一的（同一取整约定下）。

这一点与二叉搜索树形成鲜明对照：n 个相同关键字、不同插入顺序会得到不同的二叉搜索树，形态不唯一。

（五）树高、查找成功与失败的比较次数上限

树高：判定树只有最后一层可能不满，前 $h-1$ 层是满的（最多容纳 $2^{h-1}-1$ 个内部结点）。内部结点总数 n 满足

$$2^{h-1} - 1 < n \le 2^h - 1$$

两边加 1 取以 2 为底的对数：

$$2^{h-1} < n+1 \le 2^h \implies h-1 < \log_2(n+1) \le h$$

所以（不含失败结点的层数）

$$\boxed{h = \lceil \log_2(n+1) \rceil}$$

由「成功结点的查找长度 = 自身层数」和「失败结点的查找长度 = 父结点层数」，两种情况的比较次数都不会超过树高：

$$ASL_{\text{成功}} \le h, \qquad ASL_{\text{失败}} \le h$$

折半查找比较次数的上限

- 查找成功最多比较 $h = \lceil \log_2(n+1) \rceil$ 次。
- 查找失败最多也比较 $h = \lceil \log_2(n+1) \rceil$ 次。

自验证：$n = 16$，$\lceil \log_2 17 \rceil = \lceil 4.09 \rceil = 5$。前 4 层最多放 $2^4 - 1 = 15$ 个内部结点，第 16 个必须落在第 5 层，所以某些失败查找要沿路径比较 5 次才进入空区间——答案 5 次。

$n = 600$，$\lceil \log_2 601 \rceil = \lceil 9.23 \rceil = 10$，最多比较 10 次。

统考最爱的两个数字：「长度为 n 的有序顺序表折半查找，最多比较多少次」是高频选择题，标准答案就是 $\lceil \log_2(n+1) \rceil$。注意不要把 n 是 2^k 的特殊情况当成通式：$n = 16$ 时不是 4 次而是 5 次。

（六）$ASL_{\text{成功}}$ 的推导

设判定树高为 h，前 $h-1$ 层是满的，第 h 层剩下 k 个结点：

$$k = n - (2^{h-1} - 1)$$

第 i 层有 2^{i-1} 个结点（$i < h$），第 h 层有 k 个结点。按层数加权求和：

$$ASL_{\text{成功}} = \frac{1}{n}\left[\sum_{i=1}^{h-1} i \cdot 2^{i-1} + h \cdot k\right]$$

利用恒等式 $\sum_{i=1}^{h-1} i \cdot 2^{i-1} = (h-2)2^{h-1} + 1$，得

$$ASL_{\text{成功}} = \frac{1}{n}\Big[(h-2)2^{h-1} + 1 + h \cdot \big(n - 2^{h-1} + 1\big)\Big] = h - 1 + \frac{3 - 2^h + 2^{h-1}}{n}$$

在 n 较大、h 较小时，后面那一项相对 n 可忽略，于是得到统考常用的结论：

$$\boxed{ASL_{\text{成功}} \approx \log_2(n+1) - 1}$$

更严格地说：当 $n = 2^h - 1$（判定树是满二叉树）时，$ASL_{\text{成功}} = h - 1 = \log_2(n+1) - 1$，这是下界；当 n 只是刚超过 $2^{h-1} - 1$ 时，$ASL_{\text{成功}}$ 略大于 $h-1$，但始终满足

$$\log_2(n+1) - 1 \le ASL_{\text{成功}} < h = \lceil \log_2(n+1) \rceil$$

算例：$n = 11$，判定树为满二叉树前 4 层去掉第 4 层的 4 个结点（第 4 层剩 4 个）。各层结点数：第 1 层 1 个，第 2 层 2 个，第 3 层 4 个，第 4 层 4 个。

$$ASL_{\text{成功}}=\frac{1\times1+2\times2+3\times4+4\times4}{11}=\frac{1+4+12+16}{11}=\frac{33}{11}=3$$

用近似公式检验：$\log_2 12-1\approx3.585-1=2.585$，实际 3，落在 $[2.585,\,4)$ 内，符合上面的不等式。

（七）$ASL_{\text{失败}}$ 的推导

失败结点（外部结点）的总数恒为 $n+1$。设判定树高为 h，第 h 层上有

$$k=n-\left(2^{h-1}-1\right)$$

个内部结点。外部结点按父结点层数分两类：

- 父结点在第 h 层的：第 h 层的每个内部结点各带 2 个外部孩子，共 $2k$ 个，查找长度为 h；
- 父结点在第 $h-1$ 层的：剩下 $A=(n+1)-2k$ 个，查找长度为 $h-1$。

于是通用公式为

$$ASL_{\text{失败}}=\frac{(h-1)\cdot A+h\cdot 2k}{n+1},\qquad A=n+1-2k$$

算例 **1**：$n=11$，$h=4$，$k=11-7=4$，$2k=8$，$A=12-8=4$：

$$ASL_{\text{失败}}=\frac{3\times4+4\times8}{12}=\frac{12+32}{12}=\frac{44}{12}=\frac{11}{3}\approx3.67$$

算例 **2**：$n=16$，$h=5$，$k=16-15=1$，$2k=2$，$A=17-2=15$：

$$ASL_{\text{失败}}=\frac{4\times15+5\times2}{17}=\frac{60+10}{17}=\frac{70}{17}\approx4.12$$

算例 **3**：$n=15=2^4-1$（满二叉树），$h=4$，$k=15-7=8$，$2k=16$，$A=16-16=0$，全部外部结点的父层数都是 4：

$$ASL_{\text{失败}}=\frac{4\times0+4\times16}{16}=4=h$$

自验证：A 必为非负整数。若算出 $A<0$，说明 k 或 h 取错了。

做题时的可靠做法

画出判定树逐个数外部结点，比套公式更稳。两条规律：

- 只有最下方两层的内部结点可能「孩子不全」；
- 父层数越浅，外部结点越多。

特殊情形：若 $n=2^h-1$（判定树是满二叉树），则 $k=2^{h-1}$，$A=0$，全部外部结点的父层数都是 h：

$$ASL_{\text{失败}}=h=\lceil\log_2(n+1)\rceil=\log_2(n+1)$$

$ASL_{\text{失败}}$ 与失败查找次数上限

最大失败查找次数 = 树高 $h=\lceil\log_2(n+1)\rceil$。

判定树中最深的外部结点，其父结点恰在第 h 层，所以失败查找最多比较 h 次——与成功查找的上界相同。$n=16$ 时 $h=5$，失败查找最多比较 5 次。

$ASL_{失败}$ 的三个易错点

1. 分母是 $n+1$（外部结点数），不是 n。
2. 外部结点自身不计比较，用的是父结点层数。把外部结点当成第 $h+1$ 层来算，结果会整体偏大。
3. 只有 $n=2^h-1$ 时才有干净的 $ASL_{失败}=h$；一般情况下必须画出判定树逐个数，不能直接套 h。

$n=5$ 的例子：$h=3$，$k=5-3=2$，$2k=4$，$A=6-4=2$：

$$ASL_{失败}=\frac{2\times 2+3\times 4}{6}=\frac{4+12}{6}=\frac{8}{3}\approx 2.67$$

（八）时间复杂度与优缺点

折半查找的时间复杂度为 $O(\log_2 n)$。

优点：时间效率高（$n=2^{20}$ 时顺序查找需约 10^6 次比较，折半查找只需 20 次）；不需要额外辅助空间。

局限：

- 只适用于有序数据。为用折半而先排序是 $O(n\log n)$，得不偿失；频繁插入时保持有序也要 $O(n)$ 的搬移成本。
- 只适用于数组，链表上「跳跃访问」代价太高。
- 小数据量下顺序查找反而更快（折半每轮约 4–6 个基本操作，顺序查找只有 1 次判断）。

统考考法

折半查找是选择题重灾区，常见问法四种：

1. 求最多比较次数——答 $\lceil\log_2(n+1)\rceil$（$n=16$ 答 5，$n=600$ 答 10）。
2. 给一串关键字比较序列，判断「能否是某次折半查找的比较序列」。判据：每一步都在前面比较所确定的上下界之间。如 $500,200,450,180$：比较 500 后知目标 <500；比较 200 后知目标 >200，范围 $(200,500)$；比较 450 后知目标 <450，范围 $(200,450)$——180 已在下界 200 之外，不可能出现。
3. 判断某二叉树能否是折半查找判定树——检查每层左右子树规模是否始终符合同一取整规则。
4. 判断哪些存储结构适合折半查找——只有「有序 + 顺序存储」。

BST 查找路径的判定题用的是完全相同的上下界收缩法，只是一次比较后可能只更新一侧边界。

五、树形查找

（一）二叉搜索树（BST）

二叉搜索树（**Binary Search Tree**），又称二叉排序树、二叉查找树，满足：

1. 对任意结点，左子树中所有结点的值 < 该结点的值 < 右子树中所有结点的值；
2. 左、右子树本身也都是二叉搜索树。

BST 的两条基本性质

- 中序遍历 **BST** 得到的序列是严格递增序列。这条性质是判断一棵树「是不是 BST」的标准方法，也是解「第 k 小元素」「判断合法性」类问题的关键。
- BST 中不允许存在重复关键字（若允许，需在定义中明确放在哪一侧）。

1.（1）查找

从根出发，逐层比较：

- `cur->val < target`：目标在右子树，`cur = cur->right;`
- `cur->val > target`：目标在左子树，`cur = cur->left;`
- 相等：查找成功；
- `cur == NULL`：查找失败。

```
TreeNode *BST_Search(BSTree root, int num) {
    TreeNode *cur = root;
    while (cur != NULL) {
        if (cur->val < num)       cur = cur->right;
        else if (cur->val > num)  cur = cur->left;
        else                      break;      // 命中
    }
    return cur;   // NULL 表示查找失败
}
```

非递归版本空间复杂度 $O(1)$，递归版本空间复杂度 $O(h)$（递归工作栈）。时间复杂度为 $O(h)$，h 为树高：最好 $O(\log n)$，最坏 $O(n)$。

2.（2）插入

插入分两步：

1. 查找插入位置：从根出发，按大小关系向下，直到走到 NULL；
2. 在该位置挂上新结点（新结点一定是叶结点）。

```
void insert(BSTree &root, int num) {
    if (root == NULL) { root = new TreeNode{num, NULL, NULL}; return; }
    TreeNode *cur = root, *pre = NULL;
    while (cur != NULL) {
        if (cur->val == num) return;      // 已存在，不插入
        pre = cur;                        // pre 记录父结点
        cur = (cur->val > num) ? cur->left : cur->right;
    }
    TreeNode *node = new TreeNode{num, NULL, NULL};
    if (pre->val < num) pre->right = node;
    else                pre->left  = node;
}
```

新插入的结点一定是叶结点

这一点在判断题里反复出现。推论：**AVL** 树插入后可能旋转，旋转会把新结点抬升成非叶结点，所以「最后插入的元素一定是叶结点」对 BST 成立、对 AVL 树不成立。

3.（3）构造

从空树开始，把关键字序列逐个插入即可：

```
void Create_BST(BSTree &root, const int str[], int n) {
    root = NULL;
```

```
    for (int i = 0; i < n; ++i) insert(root, str[i]);
}
```

由此可得一条重要区别：

BST 的形态不唯一

对同一组关键字，插入顺序不同会得到不同形态的 **BST**（但中序序列相同）。

例：$\{1,2,3\}$ 按 $2,1,3$ 插入得到平衡的树；按 $1,2,3$ 插入退化成右斜链。因此 BST 的 ASL 与形态强相关，只有折半查找判定树是唯一确定的。

4.（4）删除：三种情况

先在树中找到待删结点，然后按度分三种情况：

情况	待删结点的度	处理方式
情况 **1**	度为 0（叶子）	直接删除，其父结点的相应指针置空
情况 **2**	度为 1（只有左子树或只有右子树）	用其唯一的子结点顶替它的位置（即让父结点直接指向该子结点）
情况 **3**	度为 2（左右子树都有）	用中序前驱（左子树最右下结点）或中序后继（右子树最左下结点）替换其值，再在相应子树中递归删除那个前驱/后继

情况 3 为什么可以用前驱或后继替换？因为 BST 的中序序列是递增序列，待删结点在序列中的位置，恰好可以由它紧邻的前一个元素或后一个元素顶上，替换后整棵树仍满足 BST 性质。前驱一定是左子树的最大值（左子树最右下），后继一定是右子树的最小值（右子树最左下），二者都是叶子或至多带一个子树，删除它们就退化成情况 1 或 2。

```
TreeNode *deleteNode(TreeNode *root, int key) {
    if (root == NULL) return NULL;
    if (root->val < key) { root->right = deleteNode(root->right, key); return root; }
    if (root->val > key) { root->left  = deleteNode(root->left,  key); return root; }
    // 找到待删结点
    if (!root->left && !root->right) { delete root; return NULL; }      // 情况 1
    if (!root->left)  { TreeNode *c = root->right; delete root; return c; }  // 情况 2
    if (!root->right) { TreeNode *c = root->left;  delete root; return c; }  // 情况 2
    // 情况 3：用中序后继替换
    TreeNode *succ = root->right;
    while (succ->left) succ = succ->left;      // 右子树最左下
    root->val = succ->val;                     // 只复制值，不搬动结点
    root->right = deleteNode(root->right, succ->val);
    return root;
}
```

删除再插入：形态是否复原？

设对 BST T_1 删除结点 v 得到 T_2，再把 v 插入 T_2 得到 T_3。

- 若 v 是叶结点：删除不改变其他结点的相对结构；重新插入时按下大小比较仍会走到原来那个空位置，故 T_3 与 T_1 相同。
- 若 v 不是叶结点：删除时用前驱/后继调整了结构；而普通 BST 插入总是把新结点放到空指针位置，所以 v 重新插入后一定会变成叶结点。原来它是非叶结点，故 T_3 与 T_1 一定不同。

对 **AVL** 树，结论变弱：删除叶结点可能引发旋转，重新插入时的旋转未必把树还原，所以 T_3 与 T_1 可能不同（不是「一定相同」也不是「一定不同」）；删除非叶结点后重插也不一定不同（存在能还原的例子）。

还有一种「偷懒」的删除写法：把待删结点的左子树整体挂到右子树最左下结点的左指针上，然后让右子树顶替待删结点的位置。这种做法不改变结点之间的父子关系数量，但多次删除后树会退化成链表，不推荐使用，考试也一般不考。

5.（5）查找效率与 ASL

BST 的查找长度等于「从根走到目标结点经过的结点数」，即目标结点的层数。因此

$$ASL = \frac{1}{n}\sum_{i=1}^{n}(\text{第}i\text{ 个结点的层数})$$

- 最好情况：树尽量平衡，高度 $h = \lfloor \log_2 n \rfloor + 1$，$ASL = O(\log n)$；
- 最坏情况：每个结点只有一个分支，退化成单链表，$h = n$，$ASL = O(n)$。

算例：关键字集合 $\{1, 2, \cdots, 9\}$。若按 $1, 2, \cdots, 9$ 顺序插入，得到右斜链，各结点层数为 $1, 2, \cdots, 9$：

$$ASL_{\text{成功}} = \frac{1 + 2 + \cdots + 9}{9} = \frac{45}{9} = 5$$

若构造得较平衡，$ASL_{\text{成功}}$ 可降到 3 左右——**BST** 的效率完全取决于树形。

6.（6）BST 与折半查找的对比

对比项	折半查找	二叉搜索树
存储要求	有序顺序表	任意（链式也可）
判定树/树形	唯一确定，只与 n 有关	不唯一，与插入顺序有关
平均查找时间	$O(\log n)$	平均 $O(\log n)$，最坏 $O(n)$
插入删除	需移动大量元素，$O(n)$	只改指针，$O(\log n)$（平均）
是否适合动态查找表	不适合	适合

统考考法

BST 是高频选择题来源，典型问法：

- 给一串关键字序列，问「不可能是某棵 **BST** 的一条查找路径」（2011 年）。判据是上下界收缩：沿路径每比较一个关键字就收紧一次区间，后续关键字必须始终落在当前区间内。例如路径 $95, 22, 91, 24, 94, \ldots$：比较 95 后目标 < 95；比较 22 后目标 > 22，区间 $(22, 95)$；比较 91 后目标 < 91，区间 $(22, 91)$；比较 24 后目标 > 24，区间 $(24, 91)$——此时出现 94 已超过上界 91，故该序列不可能是查找路径。
- 给一棵 **BST** 图，问某个子树中所有关键字满足的不等式（2018、2024 年）。只要沿「属于谁的左/右子树」把不等式全部列出来取交集即可。
- 给关键字序列，问「不能生成某棵给定 **BST**」的是哪个（2020 年）。逐个插入模拟即可。
- 删除再插入的形态判断（2013 年）。
- 中序遍历相邻结点的关系判断（2022 年）。

（二）平衡二叉树（AVL）

BST 的效率取决于树形，而一连串有序插入或反复「删右子树最小值」的操作会让树偏向一侧。平衡二叉树就是为解决这个问题而引入的自平衡结构。

平衡二叉树（**AVL 树**）：一棵 BST，且其中任意结点的左、右子树高度之差的绝对值不超过 **1**。AVL 树是最早提出的自平衡二叉查找树。

1.（*1*）结点高度与平衡因子

结点高度：从该结点到最远叶结点的边数（空结点高度为 0，叶结点高度为 1）。

平衡因子（**Balance Factor, BF**）：

$$BF(\text{node}) = h(\text{node} \to \text{left}) - h(\text{node} \to \text{right})$$

AVL 的平衡条件

AVL 树中任一结点的平衡因子 $BF \in \{-1, 0, 1\}$；空结点的平衡因子规定为 0。

$|BF| > 1$ 的结点称为失衡结点。插入或删除后只要出现失衡结点，就必须调整。

为了 $O(1)$ 地取得高度和平衡因子，AVL 的结点要额外维护 `height` 域：

```
typedef struct AVLNode {
    int val;
    int height;
    struct AVLNode *left, *right;
} AVLNode, *AVLTree;

int height(AVLNode *node) { return node == NULL ? 0 : node->height; }

void updateHeight(AVLNode *node) {
    node->height = (height(node->left) > height(node->right)
                    ? height(node->left) : height(node->right)) + 1;
}

int balanceFactor(AVLNode *node) {
    return node == NULL ? 0 : height(node->left) - height(node->right);
}
```

2.（*2*）最小失衡子树与四种旋转

最小失衡子树（最小不平衡子树）：以离插入（或删除）位置最近的那个失衡结点为根的子树。AVL 的调整永远是自底向上找第一个失衡结点，然后只调整这一棵子树。

关键结论

在插入操作中，只要把最小不平衡子树调整平衡，它上面的所有祖先结点都会自动恢复平衡，不必再往上调。

（删除操作不一定：子树高度降低可能继续向上传导失衡，需要反复回溯调整。）

记失衡结点为 `node`，其高度较高的孩子为 `child`。按「插入位置相对 `node` 的方向」分四种情况：

① **LL** 型（左孩子的左子树插入）——右旋

`node` 的 $BF > 1$（左偏），且 `child` 的 $BF \geq 0$。以 `child` 为轴把 `node` 向右旋转：`child` 上升为子树新根，`node` 下降为 `child` 的右孩子；若 `child` 原本有右子树 β（记为 `grandChild`），则把 β 改

挂为 node 的左子树。

结构变化（α, β, γ 表示子树）：

旋转前	旋转后
node 的左孩子是 child；child 的左子树 α、右子树 β；node 的右子树 γ	child 为根；左子树 α，右孩子 node；node 的左子树 β、右子树 γ

② **RR** 型（右孩子的右子树插入）——左旋

node 的 $BF < -1$（右偏），且 child 的 $BF \leq 0$。与 LL 完全镜像：以 child 为轴把 node 向左旋转，child 上升为新根，node 成为 child 的左孩子，child 原来的左子树 β 改挂为 node 的右子树 γ 位置。

旋转前	旋转后
node 的右孩子是 child；node 的左子树 α；child 的左子树 β、右子树 γ	child 为根；左孩子 node，右子树 γ；node 的左子树 α、右子树 β

③ **LR** 型（左孩子的右子树插入）——先左旋再右旋

node 的 $BF > 1$（左偏），但 child 的 $BF < 0$（child 自己右偏），单旋解决不了，要两次旋转：

1. 先对 child 执行左旋，把 child 的右孩子（记为 grandChild，其左子树 β_1、右子树 β_2）抬到 child 的位置；
2. 再对 node 执行右旋，把 grandChild 抬到 node 的位置。

最终 grandChild 成为该子树的新根：左孩子 child（左子树 α、右子树 β_1），右孩子 node（左子树 β_2、右子树 δ）。

步骤	变化
初始	node 的左孩子 child，node 的右子树 δ；child 的左子树 α、右孩子 gc；gc 的左子树 β_1、右子树 β_2
对 child 左旋	gc 顶替 child 的位置，child 成为 gc 的左孩子（child 的右子树变为 β_1）
对 node 右旋	gc 顶替 node 的位置，node 成为 gc 的右孩子（node 的左子树变为 β_2）

LR/RL 的本质

双旋后升上去的是原来的「孙子」，而不是「儿子」。这正是红黑树插入中 LR/RL 型要「儿换爷」染色、而 LL/RR 型要「父换爷」染色的原因。

④ **RL** 型（右孩子的左子树插入）——先右旋再左旋

node 的 $BF < -1$（右偏），但 child 的 $BF > 0$，与 LR 完全镜像：

1. 先对 child 执行右旋：child 的左孩子 gc 上升，child 成为 gc 的右孩子，而 gc 原来的右子树 β_2 改挂为 child 的左子树；
2. 再对 node 执行左旋：gc 上升为子树新根，node 成为 gc 的左孩子，而 gc 原来的左子树 β_1 改挂为 node 的右子树。

最终 grandChild 成为该子树的新根：左孩子 node（左子树 α、右子树 β_2），右孩子 child（左子树 β_1、右子树 δ）。

步骤	变化
初始	node 的左子树 α，右孩子 child；child 的左孩子 gc、右子树 δ；gc 的左子树 β_1、右子树 β_2
对 child 右旋	gc 顶替 child 的位置；child 成为 gc 的右孩子（child 的左子树变为 β_2）
对 node 左旋	gc 顶替 node 的位置；node 成为 gc 的左孩子（node 的右子树变为 β_1）

四种旋转的判定表（背下来）

失衡结点 BF	较高孩子 BF	类型	旋转方法
>1（左偏）	≥ 0	LL	右旋
>1（左偏）	<0	LR	先左旋后右旋
<-1（右偏）	≤ 0	RR	左旋
<-1（右偏）	>0	RL	先右旋后左旋

一句话记忆：「偏向哪边，就先把它正过来」。LL/RR 是「偏心在外侧」，单旋即可；LR/RL 是「先偏外边再偏里边」，要双旋。

旋转的代码只改指针。右旋：

```
AVLNode *rightRotate(AVLNode *node) {
    AVLNode *child = node->left;
    AVLNode *grand = child->right;
    child->right = node;       // child 上升
    node->left   = grand;      // 原 child 的右子树改挂
    updateHeight(node);        // 先更新下层 node
    updateHeight(child);       // 再更新上层 child
    return child;              // 返回新子树根
}
```

左旋把上述代码中所有 left/right 互换即可：

```
AVLNode *leftRotate(AVLNode *node) {
    AVLNode *child = node->right;
    AVLNode *grand = child->left;
    child->left  = node;
    node->right  = grand;
    updateHeight(node);
    updateHeight(child);
    return child;
}
```

旋转的三条注意事项

1. 旋转不改变中序遍历序列——这是 AVL 旋转既能恢复平衡、又保持 BST 性质的根本原因。

2. 必须自底向上找第一个失衡结点（最小不平衡子树）来调整，而不是直接调根；插入情形下调整它后祖先自动恢复。
3. 更新高度必须先 `node` 后 `child`（先下后上），顺序反了高度会算错。

3.（*3*）插入后的调整过程

步骤：

1. 按 BST 规则找到插入位置，插入新结点（新结点一定是叶结点），并回溯更新沿途结点的高度；
2. 自底向上找到第一个失衡结点 `node`；
3. 记 `node` 中高度较高的孩子为 `child`，若 `child` 也有较高的孙子 `grandChild`，则由 `node` → `child` → `grandChild` 的形状判定是 LL / RR / LR / RL；
4. 在最小不平衡子树上执行相应旋转；
5. 因插入只会让子树高度增加，调整完第一个失衡结点后整棵树就平衡了，结束。

完整算例：依次插入 $1,2,3,4,5,6,7$ 到空 AVL 树。

插入后	出现的失衡	类型	调整结果（根）
1,2	无（根 1，右孩子 2，$BF=-1$）	—	1
3	结点 1 的 $BF=-2$，孩子 2 的 $BF=-1$	RR	左旋 1 → 根变 2
4	结点 3 的 $BF=-1$，无失衡	—	2
5	结点 3 的 $BF=-2$，孩子 4 的 $BF=-1$	RR	左旋 3 → 4 顶替
6	结点 2 的 $BF=-2$，孩子 4 的 $BF=-1$	RR	左旋 2 → 4 成为总根
7	结点 5 的 $BF=-2$，孩子 6 的 $BF=-1$	RR	左旋 5 → 6 顶替

最终整棵树是以 4 为根、左右子树各 3 个结点的平衡形态。这个例子说明：连续有序插入不会退化成链，因为每两步就会触发一次旋转。

这是 **2013** 年那道题的原型

「将 $1,2,3,4,5,6,7$ 依次插入初始为空的 AVL 树，求平衡因子为 0 的分支结点个数」，就是照上表模拟到底，再数 $BF=0$ 的非叶结点。

4.（*4*）删除后的调整过程

删除比插入麻烦，因为它不保证一次旋转就结束：

1. 先用 **BST** 的删除方法删掉结点（三种情况同上）；
2. 从删除位置向上回溯，找第一个失衡结点；找不到说明树仍平衡，结束；
3. 记失衡结点为 `node`，取高度最高的儿子为 `child`、高度最高的孙子为 `grandChild`（必须调整长得更高的那一侧才可能恢复平衡），按 LL/RR/LR/RL 执行相应旋转；
4. 若调整后子树高度减 1 导致上层继续失衡，回到第 2 步继续调整，直到整棵树平衡。

删除与插入的两条差别

- 插入：只调一个最小不平衡子树即结束。
- 删除：可能需要多次调整，一直回溯到根。

5.（5）n 个结点的 AVL 树的高度范围

这块是统考计算题的常客，核心是递推。

最大高度（最少结点数）

设 $N(h)$ 表示高度为 h 的 **AVL** 树所含的最少结点数。

要让高度为 h 的 AVL 树结点数最少，根的两棵子树必须高度差取到最大的 **1**，并且两棵子树各自也取最少结点。于是根的一棵子树高度为 $h-1$（取最少结点 $N(h-1)$），另一棵高度为 $h-2$（取最少结点 $N(h-2)$），再加上根本身：

$$\boxed{N(h)=N(h-1)+N(h-2)+1}$$

初值：

$$N(0)=0,\quad N(1)=1,\quad N(2)=2$$

（$h=0$ 是空树，$h=1$ 只有根，$h=2$ 必须左右各一个孩子，共 2 个结点。）

依次递推：

h	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**
$N(h)$	1	2	4	7	12	20	33	54	88	143	232	376

与斐波那契数列的关系：记斐波那契数列 $F_1=1,F_2=1,F_3=2,F_4=3,\ldots$。把 $N(h)+1$ 列出来：$2,3,5,8,13,21,34,\ldots$，正好是 F_{h+2}：

$$N(h)=F_{h+2}-1$$

验证：$h=4$ 时 $F_6=8$，$N(4)=7$，成立；$h=6$ 时 $F_8=21$，$N(6)=20$，成立。

由斐波那契的通项 $F_k\approx\dfrac{\varphi^k}{\sqrt{5}}$（$\varphi=\dfrac{1+\sqrt{5}}{2}\approx1.618$），可推出高度与结点数的关系：

$$h\approx\log_{\varphi}\left((n+1)\sqrt{5}\right)-2$$

换成以 2 为底：

$$h\le1.44\log_2(n+1)$$

AVL 树的高度范围（必背）

- 最少结点数递推：$N(h)=N(h-1)+N(h-2)+1$，$N(0)=0,N(1)=1,N(2)=2$；与斐波那契的关系 $N(h)=F_{h+2}-1$。
- 含 n 个结点的 AVL 树，最坏高度

  $$h\approx1.44\log_2 n$$

  所以查找、插入、删除最坏都是 $O(\log n)$，远好于普通 BST 的 $O(n)$。
- 最大结点数：高度为 h 的 AVL 树最多 2^h-1 个结点（满二叉树，此时所有结点 $BF=0$）。

常见题型一：给定高度求最少结点数

设 $h=6$，则 $N(6)=20$，所以高度为 6 的 AVL 树至少 20 个结点。

常见题型二：给定结点数求最大高度

$n=20$ 时，$N(6)=20$，故最大高度恰好是 6。

常见题型三：所有非叶结点的平衡因子都相同

2012 年那道题：「高度为 6、所有非叶结点的平衡因子均为 1 的 AVL 树，结点总数是多少」。

分析：$BF=1$ 表示每个非叶结点的左子树比右子树高 1。设 $M(h)$ 为满足该条件的树的最少结点数（也是唯一确定的结点数——因为每个结点都必须左高 1，两侧高度被唯一确定：左子树高度 $h-1$、右子树高度 $h-2$）：

$$M(h)=1+M(h-1)+M(h-2)$$

这与 $N(h)$ 的递推完全相同，但初值不同：$BF=1$ 意味着叶结点的 $BF=0$。逐层推：

$$M(1)=1,\quad M(2)=2,\quad M(3)=4,\quad M(4)=7,\quad M(5)=12,\quad M(6)=20$$

所以答案为 **20** 个结点（选项 B）。注意这道题的树形是被 BF 条件唯一钉死的，不是「最少结点」的优化问题。

AVL 三类数字别混

问题	公式/答案
高度 h 的 AVL 树最少结点数	$N(h)=N(h-1)+N(h-2)+1$，$N(6)=20$
高度 h 的 AVL 树最多结点数	2^h-1，$h=6$ 时 63
n 个结点的 AVL 树最大高度	解 $N(h)\le n$，$n=20$ 时 $h=6$
n 个结点的 AVL 树最小高度	$\lceil\log_2(n+1)\rceil$，$n=20$ 时 5

「最少结点」和「最大高度」其实是同一个问题换了个问法——不要分别死记。

6.（6）AVL 的典型应用

- 组织与存储大数据集合，适用于高频查找、低频增删的场景；
- 数据库索引系统；
- 当散列表的某个链表过长时，可以把该链表转换成 **AVL** 树，把链表查找从 $O(n)$ 优化到 $O(\log n)$。

（三）红黑树（RBT）

AVL 树的平衡条件非常严格（$|BF|\le 1$），因此插入删除很容易破坏平衡，必须频繁调整：每次都要算平衡因子、找最小不平衡子树（时间开销大），再做 LL/RR/LR/RL 旋转。

红黑树（**Red-Black Tree, RBT**）放松了平衡条件，换来更少的调整次数，因此在「插入删除频繁」的场景中比 AVL 更实用。

1.（1）五条性质

红黑树首先是一棵二叉搜索树，额外满足：

1. 每个结点或是红色，或是黑色；
2. 根结点是黑色的；
3. 叶结点（外部结点 / **NULL** 结点 / 失败结点）都是黑色的；
4. 不存在两个相邻的红结点（即红结点的父结点和孩子结点都必须是黑的）；
5. 对每个结点，从该结点到任一叶结点的所有简单路径上，所含黑结点的数目相同。

五条性质的口诀

「左根右，根叶黑，不红红，黑路同」

- 左根右：它首先是 BST，满足左 < 根 < 右；
- 根叶黑：根黑，外部叶结点黑；
- 不红红：没有连续两个红结点；
- 黑路同：任一点到任一叶结点的黑结点数相同（这叫黑高一致）。

> 注意红黑树中的「叶结点」指空结点（外部结点），不是带关键字的最下层结点——这一点和 B 树一致，考试极爱在这里设陷阱。

结点结构需要多一个父指针和颜色域：

```
typedef struct RBNode {
    int val;
    struct RBNode *parent;
    struct RBNode *left, *right;
    int color;      // 0 表示黑，1 表示红
} RBNode;
```

*2.（2）*两条重要推论

推论 **1**：从根到叶结点的最长路径不大于最短路径的两倍。

证明思路：由「黑路同」，最长路径与最短路径上的黑结点数目相同，设为 bh。由「不红红」，最长路径上只能「黑、红、黑、红……」交替出现，所以最长路径的结点数至多为 $2bh$；而最短路径可以全是黑结点，结点数为 bh。故最长不超过最短的两倍。

这条性质也是判断一棵树是否为红黑树的核心依据（配合「不红红」和「黑路同」一起检查）。

推论 **2**：含 n 个内部结点的红黑树，高度 $h \le 2\log_2(n+1)$，因此查找的时间复杂度为 $O(\log n)$。

推导：定义结点的黑高（**bh, black height**）为「从该结点出发（不含该结点）到达任一叶结点的简单路径上的黑结点总数」。

若根结点的黑高为 bh，则内部结点数最少的情形是「共 bh 层、全是黑结点」的满二叉树，此时内部结点数至少为 $2^{bh}-1$，即

$$n \ge 2^{bh} - 1$$

另一方面，由「不红红」，总高度 h 最多是黑高的两倍（红黑交替），所以

$$bh \ge \left\lfloor \frac{h}{2} \right\rfloor$$

两式联立：

$$n \ge 2^{\lfloor h/2 \rfloor} - 1 \implies \frac{h}{2} \le \log_2(n+1) \implies \boxed{h \le 2\log_2(n+1)}$$

3.（*3*）与 *AVL* 的对比

对比项	**AVL** 树	红黑树
平衡条件	任一结点左右子树高度差 ≤ 1	最长路径 $\leq$ 最短路径的 2 倍
树高上界	$\approx 1.44\log_2 n$	$\leq 2\log_2(n+1)$
查找效率	略高（树更矮）	略低（树更高，但仍是 $O(\log n)$）
插入/删除的调整	频繁：要算平衡因子、找最小不平衡子树，可能多次旋转	较少：很多时候变色即可，调整可在常数级时间内完成
适用场景	以查为主、很少插入删除	频繁插入、删除，实用性更强

为什么红黑树在插入删除频繁时更优

AVL 的平衡条件严格，插入删除后绝大多数情况都要调整，而且每次调整前都要计算平衡因子、定位最小不平衡子树，定位本身就要 $O(\log n)$ 的代价（虽然可以靠维护 height 域降到 $O(1)$，但旋转次数仍多）。

红黑树的条件宽松，插入删除后很多时候不破坏红黑特性，直接结束；即使需要调整，也常在常数级时间内通过变色完成，旋转次数显著更少。因此在增删频繁的工程场景（如标准库的 `map`/`set`、Linux 内核的调度器与虚拟内存管理）中，红黑树比 AVL 更受欢迎。

4.（*4*）插入操作：变色 + 旋转

插入分三步：

第一步：按 BST 规则查找插入位置（新结点一定是叶结点）。

第二步：给新结点染色——

- 新结点是根：染为黑色；
- 新结点非根：染为红色（为什么染红？因为染红只可能违反「不红红」，而不改变任何路径的黑结点数；染黑必然违反「黑路同」，更难修）。

第三步：若插入后仍满足红黑性质，结束；否则按「新结点的叔叔的颜色」分两类调整：

① 黑叔：旋转 + 染色（交换旋转中涉及的两个结点的颜色）

类型	情形	处理
LL 型	新结点是祖父的左孩子的左孩子	右单旋，然后「父换爷」染色（父与爷互换颜色）
RR 型	新结点是祖父的右孩子的右孩子	左单旋，然后「父换爷」染色
LR 型	新结点是祖父的左孩子的右孩子	先左旋再右旋（左右双旋），然后「儿换爷」染色（儿与爷互换颜色）
RL 型	新结点是祖父的右孩子的左孩子	先右旋再左旋（右左双旋），然后「儿换爷」染色

记忆要点：「黑叔 $\rightarrow$ 旋转 + 染色」。LL/RR 单旋时是「父」和「爷」换色；LR/RL 双旋时最终升上去的是原来的「儿」，所以是「儿」和「爷」换色。

② 红叔：染色 + 变新（不旋转）

做法是「叔父爷染色，爷变新」：

1. 把叔叔、父亲、祖父三个结点的颜色全部取反（红变黑、黑变红）；
2. 把祖父当成「新插入的结点」，回到第二步重新处理（可能继续向上传递，最坏传到根，此时把根染黑即可）。

红黑树插入的两个易错点

1. 红叔时不需要旋转，只染色并向上传递。很多考生一见失衡就条件反射地旋转，这是错的。
2. 根结点被染红后必须立刻染回黑。红叔情形向上传递到根时，若把根染成了红，不满足性质 2，必须强制染黑。此时所有路径的黑结点数 +1，仍然满足「黑路同」。

完整算例：从空红黑树开始，依次插入 $20, 10, 5, 30, 40, 57, \ldots$。

- 插入 20：作为根，染黑。
- 插入 10：非根染红，挂 20 左边，父 20 为黑，满足性质，结束。
- 插入 5：挂 10 左边，父 10 为红——违反「不红红」。叔叔为空（黑）⇒ 黑叔 + **LL** 型：对祖父 20 右旋并「父换爷」染色（10 黑、20 红），根变 10。
- 插入 30：挂 20 右边，父 20 红、叔叔为空 ⇒ 黑叔 + **RR** 型：对祖父 10 左旋并换色，根变 20。
- 插入 40：挂 30 右边，父 30 红、叔叔 10 为红 ⇒ 红叔：叔、父、爷三色全取反，爷爷 20 作为「新结点」向上处理；它此时是根且为红，强制染黑。
- 后续每次插入按同样的两步判定处理。

判断练习：问「某树是不是红黑树」，只需查三条——① 根是否为黑；② 有无连续两个红结点；③ 从根到每个外部叶结点的黑结点数是否相等。三条全过即为红黑树。

5.（5）删除操作用不用掌握?

红黑树删除的考试要求

- 红黑树的删除操作过于复杂，统考不作要求。
- 只需知道三条事实：
  1. 删除操作的时间复杂度是 $O(\log_2 n)$；
  2. 删除结点的处理方式与二叉搜索树的删除相同（三种情况）；
  3. 删除后可能破坏红黑特性，需要调整结点颜色和位置使其重新满足红黑特性。

从 2022 年起「红黑树」进入考纲，考查方式很可能是判断某棵树是否为红黑树（用五条性质逐个排除），或与 AVL 对比优劣。插入调整考的可能性低于性质判定，但也要会做。

六、B 树及其基本操作、B+ 树的基本概念

B 树是本章分值最高、难度最大的考点。理解它的关键是把三个字拆开：「平衡」「多路」「查找」。

- 平衡：B 树中每个结点的任意两棵子树高度之差为 0，因此所有叶结点都在同一层（和 AVL 的「差不超过 1」、红黑树的「不超过 2 倍」都不同，B 树要求绝对平衡）；
- 多路：路数 = 结点分叉数，也叫阶数。m 阶 B 树就是 m 叉树，$m \geq 2$；$m = 2$ 时 B 树退化成二叉树的样子；
- 查找：结点内关键字有序，整体「中序遍历」（用环线法）得到单调递增序列。

一句话理解 B 树

B 树 = 树 + 顺序表：整体是一棵树，每个结点是一个有序顺序表。因此它兼具两者的优点——既有树的对数级查找路径，又有顺序表的局部性与有序性，非常适合做磁盘索引（一次结点访问 = 一次磁盘 I/O）。

（一）m 阶 B 树的定义

一棵 m 阶 B 树，或为空树，或为满足下列特性的 m 叉树：

1. 每个结点至多有 m 棵子树（即最多 $m-1$ 个关键字）；
2. 若根结点不是叶结点，则至少有 **2** 棵子树（即根至少有 1 个关键字）；
3. 除根结点外的所有非叶结点至少有 $\lceil m/2 \rceil$ 棵子树（即至少有 $\lceil m/2 \rceil - 1$ 个关键字）；
4. 所有非叶结点包含信息 $(n, A_0, K_1, A_1, K_2, A_2, \cdots, K_n, A_n)$，其中：
   - K_i 为关键字，满足 $K_1 < K_2 < \cdots < K_n$；
   - A_i 为指向子树根结点的指针；
   - A_{i-1} 所指子树中所有关键字均小于 K_i，A_i 所指子树中所有关键字均大于 K_i；
   - n 为该结点的关键字个数，除根结点外的非空结点满足

$$\left\lceil \frac{m}{2} \right\rceil - 1 \le n \le m - 1$$

根结点满足 $1 \le n \le m-1$；

1. 所有叶结点（空结点）都出现在同一层上，且不带信息（可视为外部结点 / 失败结点，指向它们的指针为空）。

「叶结点」这个词在 B 树里有两种用法

- 经典定义（教材、**2023** 年真题第 **7** 题的语境）：B 树的叶结点是指空结点（外部结点 / 失败结点），与红黑树一致。带关键字的结点叫内部结点。
- 口语化用法：把最下面一层带关键字的结点也叫「叶结点」，上面带关键字的叫「非叶结点」。

两种说法都有人用，必须按上下文判断。一般结论：

- 考试里说「所有叶结点都在同一层」，通常指空结点层；
- 考试里说「含关键字的结点个数」「插入的新关键字最终位于叶结点」，用的是口语化用法。

遇到判断对错的选项时，先确认题目用的是哪一套。

把性质 1–5 用「口语化」重述一遍（这是考试最常用的形式）：

1. 树中每个结点至多有 m 棵子树；
2. 若根结点不是叶结点，则至少有两棵子树；
3. 除根结点外的所有非叶结点至少有 $\lceil m/2 \rceil$ 棵子树，即至少含有 $\lceil m/2 \rceil - 1$ 个关键字；
4. 除根结点外的非空结点满足 $\lceil m/2 \rceil - 1 \le n \le m-1$，根结点满足 $1 \le n \le m-1$；
5. 所有叶结点都出现在同一层上。

*1.*关键性质：*5* 阶 *B* 树（$m = 5$）

项	值
每结点最多子树数	5
每结点最多关键字数	$m - 1 = 4$
非根非叶结点最少子树数	$\lceil 5/2 \rceil = 3$
非根非叶结点最少关键字数	$\lceil 5/2 \rceil - 1 = 2$
根最少关键字数	1

*2.*一棵 m 阶 *B* 树的关键字数量范围

高度为 h 的 m 阶 **B** 树，关键字数最少为 $2\lceil m/2 \rceil^{h-1} - 1$。

推导：设含关键字的结点所在层为第 1 到第 h 层。要让关键字数最少，就让每层结点数最少、每个结点关键字数最少：

- 第 1 层（根）：1 个结点，最少 1 个关键字；
- 第 2 层：根至少 2 棵子树 $\Rightarrow$ 2 个结点，每个最少 $\lceil m/2 \rceil - 1$ 个关键字；
- 第 3 层：每个第 2 层结点至少 $\lceil m/2 \rceil$ 棵子树 $\Rightarrow$ 最少 $2\lceil m/2 \rceil$ 个结点；
- $\vdots$
- 第 h 层：最少 $2\lceil m/2 \rceil^{h-2}$ 个结点。

高度为 h 的 m 阶 **B** 树，关键字数最多为 $m^h - 1$。

推导：让每个结点都装满，即每个结点 $m-1$ 个关键字、m 棵子树：

$$n \le (m-1)\left(1 + m + m^2 + \cdots + m^{h-1}\right) = (m-1) \cdot \frac{m^h - 1}{m-1} = m^h - 1$$

*3.*高度范围的推导（两个方向都要会）

方向一：已知 n 个关键字，求最大高度 h

要让高度最大，就要让各层的分叉尽可能少：根只有 2 个分叉，其余每个非叶结点只有 $\lceil m/2 \rceil$ 个分叉。

含关键字的结点层数分布：第 1 层 1 个，第 2 层 2 个，第 3 层 $2\lceil m/2 \rceil$ 个，……，第 h 层 $2\lceil m/2 \rceil^{h-2}$ 个。

若约定第 h 层的下一层（第 $h+1$ 层）是叶结点（失败结点）层，则第 $h+1$ 层共有

$$2\left\lceil \frac{m}{2} \right\rceil^{h-1}$$

个叶结点。而 n 个关键字恰好把值域切成 $n+1$ 个区间，所以叶结点数必为 $n+1$：

$$n + 1 \ge 2\left\lceil \frac{m}{2} \right\rceil^{h-1}$$

两边取对数：

$$\boxed{h \le \log_{\lceil m/2 \rceil} \frac{n+1}{2} + 1}$$

方向二：已知 n 个关键字，求最小高度 h

要让高度最小，就要让每个结点尽可能满（$m-1$ 个关键字、m 个分叉）：

$$n \le (m-1)\left(1 + m + m^2 + \cdots + m^{h-1}\right) = m^h - 1$$

$$\boxed{h \ge \log_m (n+1)}$$

合并结论：含 n 个关键字的 m 阶 B 树，高度满足

$$\log_m (n+1) \le h \le \log_{\lceil m/2 \rceil} \frac{n+1}{2} + 1$$

用哪个公式要看清题目对「高度」的定义

- 题目说「高度为 2」并配图，一般表示根在第 **1** 层、最下面一层含关键字的结点在第 **2** 层（不含空结点层）。
- 题目说「不包含空结点层」「根为第 1 层」时，用上面的公式。
- 若题目把空结点层也算进高度，所有高度要再 +1，公式也要相应平移。

做 B 树的高度题，第一件事是数清楚题目给的「高度」是否包含空结点层。

算例 **1**：5 阶 B 树有 53 个关键字，高度最高是多少（不含空结点层）？

$h \le \log_3 \dfrac{53+1}{2} + 1 = \log_3 27 + 1 = 4$。答案 **4**。

算例 **2**：高度为 2 的 5 阶 B 树最少含多少关键字？根 1 个关键字 +2 棵子树，每棵子树最少 $\lceil 5/2 \rceil - 1 = 2$ 个关键字，故 $1 + 2 \times 2 = 5$。答案 **5**。

算例 **3**：高度为 5 的 3 阶 B 树最少含多少关键字？3 阶树的非根结点最少 1 个关键字、2 棵子树，各层最少结点数 $1, 2, 4, 8, 16$，合计 31，每结点 1 个关键字。答案 **31**。

算例 **4**：一棵有 20 个关键字的 3 阶 B 树，含关键字的结点个数最多是几个？

3 阶树每结点最多 2 个关键字。要让结点数最多，就让每个结点只放 1 个关键字：此时含关键字的结点构成满二叉树形，$2^5 - 1 = 31 \ge 20 \ge 2^4 - 1 = 15$，$h = 5$ 时 31 个位置足以放 20 个关键字。

关键是不能只放 **10** 个结点（$20 \div 2$）——若某层放满 2 个关键字、另一层只放 1 个，叶结点就不在同一层了，违反 B 树性质。所以正确答案是 **20**（2014 年题的陷阱正是「叶结点必须在同一层」）。

算例 **5**：具有 15 个关键字的 4 阶 B 树，结点个数最多是多少？每个结点放 1 个关键字，构成满二叉形（$2^4 - 1 = 15$），满足同层要求。答案 **15**。

算例 **6**：高度为 3 的 3 阶 B 树，根为第 1 层，第 2 层有 4 个关键字，则结点个数最多是多少？

根最多 3 棵子树 $\Rightarrow$ 第 2 层最多 3 个结点。设这 3 个结点的关键字数为 $n_1, n_2, n_3 \ge 1$，$n_1+n_2+n_3 = 4$。第 3 层结点数 $= \sum(n_i + 1) = 4 + 3 = 7$。总结点数 $= 1 + 3 + 7 = 11$。

答案 **11**。注意第 3 层结点数只由「子树数 = 关键字数 + 1」和第 2 层关键字总数决定，与关键字在各结点间的分布无关。

（二）B 树的查找

B 树的查找与 BST 类似，只是每个结点内部要在有序表中定位。

设查找关键字为 x，从根开始：

- 基本情况：
  - 搜索到的结点为空（到达最下层空结点）$\Rightarrow$ 查找失败，结束；
  - 结点非空且包含 $x \Rightarrow$ 查找成功，结束。
- 迭代情况：结点非空且不含 x，该结点为 $(n, A_0, K_1, A_1, \cdots, K_n, A_n)$：
  - 若 $x < K_1$，进入孩子 A_0；
  - 若 $K_i < x < K_{i+1}$（$i = 1, \ldots, n-1$），进入孩子 A_i；
  - 若 $x > K_n$，进入孩子 A_n。

B 树查找与 B+ 树查找的根本区别

B 树的每个结点里存的都是真实存在的关键字，所以在内部结点命中就立刻成功，不必走到最下层。

B+ 树只有叶结点存记录信息，所以无论成功还是失败，查找都必须走到最下面一层。

查找效率：结点内部关键字有序，可用线性查找（$O(n)$）或折半查找（$O(\log n)$）。

设树高为 h，则查找路径长度至多为 h。若结点内部采用折半查找，查找的代价为

$$O(\log m) \cdot O(\log_m n) = O(\log n)$$

（三）B 树的插入

要领：先查找确定插入位置，一定从终端结点（最下层含关键字的结点）插入；若插入后关键字数超过上限 $m-1$，则把当前结点的中间关键字（第 $\lceil m/2 \rceil$ 个）上移到父结点，当前结点分裂成左右两部分；父结点关键字数 +1 后若也超限，则继续向上分裂。

第一步：查找插入位置并插入

- 若 B 树为空：新建结点作为根，放入关键字；
- 否则：从根开始单程向下查找，一直到最下层含关键字的结点为止，把关键字按序插入该结点。

第二步：调整以维护 **B** 树性质

- 不分裂（仅插入）：被插入结点关键字个数 $\leq m-1$，无需调整，结束。若插入前树为空、新建了根，树高 +1。
- 分裂（插入后分裂，需要迭代）：被插入结点关键字个数达到 m（超上限），需要分裂：
  1. 取该结点的第 $\lceil m/2 \rceil$ 个关键字上移插入父结点；
  2. 第 1 到 $\lceil m/2 \rceil - 1$ 个关键字（及对应的孩子指针）留在左结点；
  3. 第 $\lceil m/2 \rceil + 1$ 到 m 个关键字（及对应的孩子指针）移到新建的右结点；
  4. 把「上移的关键字」和「父结点」分别作为新的「待插入关键字」和「待插入结点」，返回第一步迭代，直到出现「不分裂」情形；
  5. 若分裂一直传导到根，导致根也分裂，则新建一个根，树高 +1。这是 B 树长高的唯一方式。

中间关键字是「下中位数」

上移的是关键字数组的下中位数位置，即第 $\lceil m/2 \rceil$ 个：

- $m=4$（4 阶）：结点最多 3 个关键字，插入后变 4 个，上移第 $\lceil 4/2 \rceil = 2$ 个关键字；左 1 个、右 2 个。
- $m=5$（5 阶）：结点最多 4 个关键字，插入后变 5 个，上移第 $\lceil 5/2 \rceil = 3$ 个关键字；左 2 个、右 2 个。

分裂后的左孩子通常在原结点上修改，右孩子需要新建结点。

算例：往 4 阶 B 树中插入 66。4 阶树每结点最多 3 个关键字，若终端结点原本是 $\{60, 64, 70\}$，插入后变 $\{60, 64, 66, 70\}$ 超限：

1. 上移第 $\lceil 4/2 \rceil = 2$ 个关键字 **64** 到父结点；
2. 左结点保留 $\{60\}$，右结点保留 $\{66, 70\}$；
3. 父结点若因此超限，继续向上分裂。

*1.*插入操作的推论（*2023* 年真题考点）

- 插入操作可能增加树的高度（对：根分裂时新建根，树高 +1）；
- 插入的新关键字最终不一定留在叶结点（对：它可能被同级分裂推到父结点，甚至更高层）。「插入总是发生在最下层」是对的，但「新关键字最终留在最下层」是错的。

（四）B 树的删除

删除是 B 树所有基本操作中最难的，分两步：先删除（可能先做替换），后调整。

第一步：查找并删除

- 若关键字位于终端结点：直接删除；
- 若关键字位于非终端结点：用它的直接前驱或直接后继替代该关键字，从而把对非终端结点的删除转化为对终端结点的删除：
  - 直接前驱 = 该关键字左侧指针所指子树中「最右下」的元素；
  - 直接后继 = 该关键字右侧指针所指子树中「最左下」的元素。

第二步：调整以维护 **B** 树性质

设删除后该结点的关键字数为 n。

- 情况 **1**：直接删除（$n \geq \lceil m/2 \rceil - 1$，仍满足下限）：在该结点内把后面的关键字前移即可，结束。
- 情况 **2**：需要调整（$n < \lceil m/2 \rceil - 1$，下溢）：
  - 情况 **2.1** 「兄弟够借」（旋转）：若右兄弟关键字个数 $\geq \lceil m/2 \rceil$（借走一个仍合法），则做旋转——把父结点中分隔这两个兄弟的关键字下移到被删结点，把右兄弟中的最小关键字上移到父结点。若右兄弟不够借而左兄弟够，则镜像处理（把左兄弟中的最大关键字上移父结点，把父结点中的分隔关键字下移到被删结点）。本质就是「用当前结点的后继（或前驱）来填补空缺」。
  - 情况 **2.2** 「兄弟不够借」（合并）：若左右兄弟都恰含 $\lceil m/2 \rceil - 1$ 个关键字，则把被删结点、其一个兄弟结点、以及父结点中分隔它们的关键字三者合并成一个结点。
    - 合并后父结点关键字数减 1。若父结点是根且关键字数减到 0（原根只有 1 个关键字、2 棵子树），则直接删除根，合并后的新结点成为新的根，树高减 **1**。
    - 若父结点不是根且关键字数减到 $\lceil m/2 \rceil - 2$（下溢），则对父结点重复上述「借 / 合并」过程，继续向上传导，直到满足 B 树要求。

B 树删除的三个易错点

1. 删除非终端结点关键字不是直接删，必须先用前驱或后继替换，再删终端结点里的那个值。
2. 合并是「三方合并」：左（或右）兄弟 + 父结点中的分隔关键字 + 当前结点，三者合成一个结点，不要漏掉父结点中的那个关键字。
3. 根的特殊处理：根的关键字数下限是 1（不是 $\lceil m/2 \rceil - 1$），所以根下溢到 0 时要删根、降低树高，而不是去和兄弟借。

关于「删一个关键字的结果是否唯一」：删除非终端结点的关键字时，用前驱还是用后继替换会造成不同的中间形态。2022 年那道 5 阶 B 树删 260 的题，四个选项分别对应「用后继 + 与右兄弟合并」「用后继 + 与左兄弟合并」「用前驱 + 左兄弟够借」「用前驱 + 右兄弟合并」四种路径。但从最终 **B** 树的合法性与算法确定性角度看，只需按 B 树性质逐一验证选项是否合法即可——把所有可能路径都画出来，看哪个选项构造不出合法的 5 阶 B 树。

（五）B 树的判定

判断一棵树是不是合法的 m 阶 B 树，三步：

1. 查平衡：所有叶结点（含关键字的结点层）必须都在同一层；
2. 查有序：用环线法做「中序遍历」，得到的关键字序列必须单调递增；

3. 查关键字个数：对除根外的每个结点，关键字个数 n 满足 $\lceil m/2 \rceil - 1 \le n \le m-1$；根结点满足 $1 \le n \le m-1$。

2022 年那道题的最好解法就是「判定法」

「在 5 阶 B 树中删除 260 得到 T_1，下列不可能的是 T_1 根结点的关键字序列」。从删除算法入手很繁琐，正确做法是：按每个选项反推出整棵 **B** 树，再检查它是否满足 **5** 阶 **B** 树的性质（$m=5$ 时非根结点关键字数须满足 $2 \le n \le 4$）。选项 D 中含 100 的那个结点只有 1 个关键字，违反 $2 \le n \le 4$，所以不可能。

（六）B+ 树的基本概念

B+ 树的产生源于操作系统的文件索引和数据库索引。可以把它理解成「分块查找的进化——多级分块查找」。

一棵 m 阶 B+ 树满足：

1. 每个分支结点最多有 m 棵子树（孩子结点）；
2. 非叶根结点至少有两棵子树，其他每个分支结点至少有 $\lceil m/2 \rceil$ 棵子树；
3. 结点的子树个数与关键字个数相等（对比 B 树的「子树数 = 关键字数 + 1」）；
4. 所有叶结点包含全部关键字及指向相应记录的指针；叶结点中关键字按大小顺序排列，并且相邻叶结点按大小顺序相互链接（支持顺序查找）；
5. 所有分支结点中仅包含它的各个子结点中关键字的最大值及指向其子结点的指针（只起索引作用，不含记录信息）。

B+ 树的查找：B+ 树有两个头指针，一个指向根结点，一个指向关键字最小的叶结点，因此支持两种查找：

- 从最小关键字开始的顺序查找（沿叶结点链）——这正是 B+ 树支持范围查询的原因；
- 从根结点开始的多路查找。

B+ 树查找必然走到最下层

与 B 树「查找成功可能停在任意一层」不同，**B+** 树中无论查找成功与否，最终一定要走到最下面一层结点，因为只有叶结点包含信息（记录指针）。因此 B+ 树的查找速度稳定，而 B 树不稳定。

（七）B 树与 B+ 树的对比

对比项	m 阶 **B** 树	m 阶 **B+** 树
类比	二叉查找树的进化——m 叉查找树	分块查找的进化——多级分块查找
关键字与分叉	n 个关键字对应 $n+1$ 棵子树	n 个关键字对应 n 棵子树
结点存的信息	所有结点都包含记录的信息（含记录的存储地址）	只有最下层叶结点包含记录信息；非叶结点仅起索引作用，只存「对应子树的最大关键字 + 指针」
根结点关键字数（非叶根）	最少 1 个，最多 $m-1$ 个	最少 **2** 个，最多 m 个（根若同时是叶结点，可以只有 1 个关键字）
非根结点	最少 $\lceil m/2 \rceil - 1$ 个关键字	至少 $\lceil m/2 \rceil$ 棵子树
查找路径	查找成功时可能停在任意一层，速度不稳定	成功或失败都必到最后一层，速度稳定

对比项	m 阶 **B** 树	m 阶 **B+** 树
顺序查找	不支持	支持（叶结点之间有顺序链）
关键字重复	各结点中的关键字互不重复	叶结点包含全部关键字；非叶结点出现过的关键字也会再次出现在叶结点中
应用场景	文件系统、需要随机访问内部结点数据的场景	关系数据库索引、需要范围查询与顺序扫描的场景
相同点	除根外最少 $\lceil m/2 \rceil$ 个分叉（避免结点太「空」）；所有结点的子树一样高（绝对平衡，所有叶结点同层）	同左

B+ 树为什么适合数据库索引

非叶结点不存记录信息，只存最大关键字和指针，故一个磁盘块能装下更多关键字，B+ 树阶更大、树更矮，读磁盘次数更少；叶结点有序链接又天然支持范围查询（`BETWEEN`、`ORDER BY`）与顺序扫描。

B 树的结点要携带记录地址，同样大小的磁盘块装下的关键字少，阶数小、树高更大，且不支持高效范围扫描。这是数据库普遍选 B+ 树的原因。

七、散列（**hash**）表

（一）基本概念

散列表（**Hash Table**，哈希表）：一种根据数据元素的关键字直接计算出它在表中的存储地址的数据结构，是典型的以空间换时间。

散列函数（哈希函数）：$Addr = H(key)$，建立「关键字 $\to$ 存储地址」的映射。

冲突（碰撞）：插入元素时，若根据关键字算出的地址已经被其他元素占用，称为冲突。

同义词：若不同的关键字通过同一个散列函数映射到同一个地址，则称这些关键字互为同义词。

装填因子 α：

$$\alpha = \frac{\text{表中记录数} n}{\text{散列表长度} m}$$

装填因子的地位

散列表的平均查找长度依赖于装填因子 α，而不直接依赖于 n 或 m。

α 越大，表越「满」，冲突可能性越大，ASL 越大。为什么题目问「影响散列表 ASL 的因素」时答案包含装填因子、散列函数、冲突解决策略三项——因为它们分别决定了表的拥挤程度、初始分布的均匀性、以及冲突后的探测代价。

反过来说：不能通过「增大装填因子」来提高查找效率（那是 2011 年那道题的干扰项）。

（二）散列函数的构造方法

设计散列函数要注意四点，四个反面例子很值得记住：

要求	反面例子	问题
定义域必须涵盖所有可能出现的关键字	$H(key) = \sqrt{key} \bmod 13$	不支持负值

要求	反面例子	问题
值域不能超出散列表地址范围	$H(key) = key \bmod 15$（表长 13）	可能映射到非法地址 13、14
尽量减少冲突，地址尽量均匀分布	$H(key) = (key \times 13) \bmod 13$	任何关键字都被映射到 0
散列函数应尽量简单、能快速计算	$H(key) = (key!) \bmod 13$	关键字大时阶乘计算耗时极高

（法一）除留余数法：$H(key) = key \bmod p$

表长为 m 时，取不大于 m 但最接近或等于 m 的质数 p。适用场景最广，只要关键字是整数即可。

为什么对质数取余：取余会被公因子影响。若 p 有多个因子，关键字与 p 的公共因子会导致分布不均；取质数能最大程度均匀分布、减少冲突。

除留余数法算 $ASL_{失败}$ 时分母取 p 还是 m?

取 $\min(m, p) = p$。因为初次散列只可能落在 $0 \sim p-1$ 这些地址上，p 及以后的地址永远不会是「初次探测点」，不属于任何一种失败情况。

例：表长 $m = 11$，$H(key) = key \bmod 7$，则 $ASL_{失败}$ 的分母是 **7**，不是 11。这是 2019 年那道题的得分点。

而 $ASL_{成功}$ 的分母始终是关键字的个数 n。

（法二）直接定址法：$H(key) = key$ 或 $H(key) = a \cdot key + b$。计算最简单且不会冲突；缺点是关键字分布不连续、空位多时浪费空间。适用场景：关键字分布基本连续。

（法三）数字分析法：选取数码分布较为均匀的若干位作为散列地址。关键字是 r 进制数时，某些数位上各种数码出现机会均等，某些数位上只有少数几种数码常出现，应选取分布均匀的那些位。

典型例子：以「手机号码」为关键字把用户信息存入长度 10000 的散列表——手机号前几位（运营商、地区号段）重复度高，应取分布较均匀的后四位。

（法四）平方取中法：取关键字平方值的中间几位作为散列地址。结果的每一位都受原关键字各位影响，故分布较均匀。适用场景：关键字每一位取值都不够均匀。典型例子：以「底盘号 + 发动机号 + 外观号」为车型标识，把 100 种车型存入长度 100 的散列表，取拼接后平方值的中间若干位。

（三）处理冲突的方法一：拉链法

拉链法（又称链接法、链地址法）：把所有同义词存储在同一个链表中。

插入两步：

1. 计算新元素的散列地址；
2. 把新元素插入该地址对应的链表（头插法或尾插法皆可）。

例：散列表长度为 13，$H(key) = key \bmod 13$，用拉链法依次插入

$$\{19, 14, 23, 1, 68, 20, 84, 27, 55, 11, 10, 79\}$$

各地址上的链表为（下表按头插法的最终状态给出，即新元素插在链头）：

地址	0	1	2	3	4	5	6	7	8	9	10	11	12
链表	空	79 → 27 → 1 → 14	空	55 → 68	空	84 → 19	空	20	空	空	23 → 10	11	空

映射关系（key mod 13）：$14, 1, 27, 79$ 都映射到 **1**；$68, 55$ 都映射到 **3**；$19, 84$ 都映射到 **5**；20 映射到 **7**；$23, 10$ 都映射到 **10**；11 映射到 **11**。

若改用尾插法，链中顺序与插入顺序一致，但链长不变。链中元素的先后顺序只影响「第几次比较命中」，不影响链长，因此不影响 $ASL_{失败}$。

下面第 7 节的算例会给出完整的 ASL 计算。

拉链法算查找长度的关键规则

若该地址为空，则查找长度为 **0**——空指针的比较不计入查找长度。

这是拉链法与开放定址法最本质的区别：

- 拉链法：探测到空链表时，那个「空指针」的比较不算一次关键字比较，所以查找长度为 0。
- 开放定址法：探测到空单元时，那个空单元的比较要计入查找长度（因为要先读一次该单元、发现它为空，才能判定失败）。

对比两种方法算 $ASL_{失败}$ 时，这一点必须分清，否则数字会差 1。

拉链法的其他要点：

- 删除：算出地址，顺序查找该链表后按链表删除即可——可直接删除，不需要「删除标记」（相对开放定址法的优势）。
- 优化：插入时保持链表有序可提高效率；链表很长时可转换成 **AVL** 树或红黑树，把查找从 $O(n)$ 降到 $O(\log n)$（Java `HashMap` 即如此）。
- 经验公式：$ASL_{成功} \approx 1 + \dfrac{\alpha}{2}$，$ASL_{失败} \approx \alpha$。统考一般不要求套公式，而是按实际链表逐个数比较次数。

（四）处理冲突的方法二：开放定址法

主要思想：发生冲突时，给新元素另外找一个空闲位置。

为什么叫「开放定址」：一个散列地址，既对同义词开放，也对非同义词开放。

思路是确定一个探测顺序，从初始散列地址出发寻找下一个空闲位置：

$$H_i = \big(H(key) + d_i\big) \bmod m$$

其中 d_i 是增量序列，表示第 i 次冲突时下一个探测地址与初始散列地址的相对偏移量；m 是表长。

按 d_i 的取法分四种：

① 线性探测法：$d_i = 1, 2, 3, \ldots, m-1$

- 插入：算地址后若冲突，顺序查看下一单元（到表尾后回绕表头），直到找到空闲单元或查遍全表。
- 查找：按探测序列比较，命中则成功；探测到空单元则失败。
- 删除：不能简单置空，必须打「已删除」标记。

② 平方探测法（二次探测再散列）：$d_i = 1^2, -1^2, 2^2, -2^2, 3^2, -3^2, \ldots, \pm k^2$

③ 双散列法(再散列法)：$d_i = i \cdot hash_2(key)$，需两个散列函数，$H_i = (H(key) + i \cdot hash_2(key)) \bmod m$。常用套路是令表长 m 为质数、$hash_2(key) = m - (key \bmod m)$，这样 $hash_2(key)$ 与 m 一定互质，保证能探测到所有单元。

④ 伪随机序列法：d_i 取一个伪随机序列

四种探测方法的覆盖率对比

方法	能否保证探测到所有位置
线性探测法	一定可以。只要表中有空闲位置，就一定能插入成功（最坏冲突 $m-1$ 次即可走遍全表）
平方探测法	至少能探测到表中一半的位置；即便表中有空闲位置，也未必能插入成功
双散列法	取决于 $hash_2(key)$ 的设计；若 $hash_2(key)$ 与 m 互质，则能探测所有单元
伪随机序列法	取决于伪随机序列设计是否合理

开放定址法删除元素：算出散列地址，按探测序列找到匹配的关键字后删除。

开放定址法必须用「删除标记」

删除元素不能简单地把该位置置为空，否则会截断它之后的探测路径——之后查找其他元素时，探测到这个空位就会误判为「不存在」，而实际目标还在更后面。

正确做法是设一个「已删除」标志位做逻辑删除。被标记为已删除的空间可以再次使用：插入新元素时若探测到「已删除」位置，可直接占用，既不破坏探测序列连续性，也不浪费空间。

2023 年真题正考这一点：表长 5，$H(k) = (k+4) \bmod 5$，依次插入 2022、12、25 后删除 25，求 $ASL_{失败}$。删除后地址 4 是「删除标记」而不是空位，故从地址 4 出发的失败查找要越过标记再探测地址 **0**，比「真为空」多一次比较。

（五）线性探测的堆积（聚集）现象

堆积（聚集、**clustering**）：线性探测时，冲突后的元素一定被放在某个连续的位置上。数组中连续被占用的区域越长，这块区域发生哈希冲突的可能性就越大，进一步促使聚堆生长，形成恶性循环，最终导致增删查改效率劣化。

堆积现象的特点

- 线性探测法最容易产生堆积，因为它只向相邻的下一格探测；
- 堆积是「同义词 + 非同义词」共同造成的；纯同义词（如拉链法的一条链）不叫堆积；
- 堆积直接影响的是「平均查找长度」，不影响存储效率、散列函数、装填因子；
- 平方探测法、双散列法可以缓解堆积，但双散列法只是缓解，不能完全避免。

（六）平方探测法与表长 $4k+3$

平方探测法的增量序列是

$$d_i = 1^2,\ -1^2,\ 2^2,\ -2^2,\ 3^2,\ -3^2,\ \ldots,\ \pm k^2$$

为什么要求表长是 $4k+3$ 形式的素数

平方探测只能保证探测到表中至少一半的位置。也就是说，即使表里还有空位，也可能插不进去。

若表长 m 是一个可以表示成

$$m = 4k + 3$$

的素数（例如 $m = 7, 11, 19, 23$），则 $m \equiv 3 \pmod 4$。此时：

- 平方序列 $\{i^2 \bmod m\}$ 的取值集合恰好是 $\{0, 1, 2, \ldots, m-1\}$，即覆盖全部 m 个位置；
- 也就是说，只要表中有空位，平方探测一定能找到它。

反之：

- $m \equiv 1 \pmod 4$ 时，平方序列只能覆盖 $\{0, 1, \ldots, m-1\}$ 中的一部分位置（例如 $m = 13$ 时，$\{i^2 \bmod 13\} = \{0, 1, 3, 4, 9, 10, 12\}$，只覆盖 7 个位置）；
- m 不是素数时，情况更复杂，也可能只覆盖一半。

结论：考试中记「表长为 $4k+3$ 的素数时，平方探测可探测到全部位置」即可。

（七）散列表 ASL 的计算（三个完整算例）

*1.*算例 *1*：平方探测法，$ASL_{\text{成功}}$

设关键字 $(9, 1, 23, 14, 55, 20, 84, 27)$，散列函数 $H(key) = key \bmod 7$，表长 10，用平方探测再散列 $H_i = (H(key) + d_i) \bmod 10$，$d_i = 1^2, 2^2, 3^2, \ldots$ 构造散列表并求 $ASL_{\text{成功}}$。

key	**9**	**1**	**23**	**14**	**55**	**20**	**84**	**27**
$H(key)$	2	1	2	0	6	6	0	6
第一次冲突 $+1$			3			7	1	7
第二次冲突 $+4$							4	0
第三次冲突 $+9$								5
实际位置	2	1	3	0	6	7	4	5
成功比较次数	1	1	2	1	1	2	3	4

散列表状态（地址 $0 \sim 9$）：

0	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**
14	1	9	23	84	27	55	20		

$$ASL_{\text{成功}} = \frac{1+1+2+1+1+2+3+4}{8} = \frac{15}{8}$$

*2.*算例 *2*：线性探测法，$ASL_{\text{成功}}$ 与 $ASL_{\text{失败}}$

设表长 $m = 15$，散列函数 $H(key) = key \bmod 13$，关键字集合 $(53, 17, 12, 61, 89, 70, 87, 25, 64, 46)$，用线性探测法处理冲突，求 $ASL_{\text{成功}}$ 与 $ASL_{\text{失败}}$。

key	**53**	**17**	**12**	**61**	**89**	**70**	**87**	**25**	**64**	**46**
$H(key)$	1	4	12	9	11	5	9	12	12	7
第一次冲突 $+1$							10	13	13	
第二次冲突 $+2$									14	
实际位置	1	4	12	9	11	5	10	13	14	7
成功比较次数	1	1	1	1	1	1	2	2	3	1

散列表状态：

0	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**	**13**	**14**
	53			17	70		46		61	87	89	12	25	64

$$ASL_{\text{成功}} = \frac{1+1+1+1+1+1+2+2+3+1}{10} = \frac{14}{10} = \frac{7}{5} = 1.4$$

$ASL_{\text{失败}}$：散列函数值域是 $0 \sim 12$（$p = 13$），所以分母是 **13**，不是 15。对每个可能的初始地址 i，从 i 出发一直线性探测到第一个空位，数比较次数（空位的比较要计入）：

初始地址	探测路径	比较次数
0	0（空）	1
1	1, 2（空）	2
2	2（空）	1
3	3（空）	1
4	4, 5, 6（空）	3
5	5, 6（空）	2
6	6（空）	1
7	7, 8（空）	2
8	8（空）	1
9	9, 10, 11, 12, 13, 14, 0（空）	7
10	10, 11, 12, 13, 14, 0（空）	6
11	11, 12, 13, 14, 0（空）	5
12	12, 13, 14, 0（空）	4

$$ASL_{\text{失败}} = \frac{1+2+1+1+3+2+1+2+1+7+6+5+4}{13} = \frac{36}{13} \approx 2.77$$

注意 $i = 2$、3、8 这些位置的探测次数为什么是 1

因为它们本身就是空位。「探测到空位」这一次访问要算一次比较，所以是 1 而不是 0。如果按「访问到元素才计数」来算就会得到 0，这是最常见的错误。

对比：拉链法在同样情形下的查找长度是 **0**（空指针不计比较）。两套规则不要混。

*3.*算例 *3*：拉链法

表长 13，$H(key) = key \bmod 13$，依次插入 $\{19, 14, 23, 1, 68, 20, 84, 27, 55, 11, 10, 79\}$，用拉链法解决冲突，求 $ASL_{成功}$ 与 $ASL_{失败}$。

各地址的链表（按头插法，新元素插在链头，查找长度从表头算）：

地址	**0**	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**
链	空	$79 \to 27 \to 1 \to 14$	空	$55 \to 68$	空	$84 \to 19$	空	20	空	空	$23 \to 10$	11	空

$ASL_{成功}$：逐个关键字数它在自己链中的位置（即比较次数）：

关键字	**14**	**1**	**23**	**68**	**20**	**84**	**27**	**55**	**11**	**10**	**79**	**19**
比较次数	4	3	2	2	1	2	3	2	1	2	1	2

即比较次数为 1 的有 $20, 11, 79$（3 个），为 2 的有 $23, 68, 84, 55, 10, 19$（6 个），为 3 的有 $1, 27$（2 个），为 4 的有 14（1 个）：

$$ASL_{成功} = \frac{1\times 3 + 2\times 6 + 3\times 2 + 4\times 1}{12} = \frac{3 + 12 + 6 + 4}{12} = \frac{25}{12} \approx 2.08$$

$ASL_{失败}$：分母为 13（散列函数值域 $0 \sim 12$ 共 13 个地址）。对每个地址，沿着链一直比较到 NULL 都查不到，比较次数即该地址的链长：

初始地址	**0**	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**
链长	0	4	0	2	0	2	0	1	0	0	2	1	0

$$ASL_{失败} = \frac{0+4+0+2+0+2+0+1+0+0+2+1+0}{13} = \frac{12}{13} \approx 0.92$$

头插法 / 尾插法：影响的是「谁第几个被比到」，不影响 ASL

同一条链 $\{14, 1, 27, 79\}$：

- 头插法链序 $79 \to 27 \to 1 \to 14$：命中次数依次 $1, 2, 3, 4$（79 最前面）；
- 尾插法链序 $14 \to 1 \to 27 \to 79$：命中次数依次 $1, 2, 3, 4$（14 最前面）。

两种情况下这 4 个关键字的比较次数集合都是 $\{1, 2, 3, 4\}$，总和都是 10，ASL 完全相同——只是「哪个关键字最省事」不同。

做题时仍要按题目给定的插入方式（头插 / 尾插）逐个确定比较次数，否则若题目给的链序与你的假设不一致，个别关键字的次数会数错。

而 $ASL_{失败}$ 只与各地址的链长有关，与链内顺序完全无关。

开放定址法 **vs** 拉链法：$ASL_{失败}$ 的两套规则

	探测到空位	空位比较是否计数
开放定址法	查找失败	计数（要读一次才知道是空的）
拉链法	走到 NULL 指针，查找失败	不计数（空指针不是关键字）

同一次失败查找，开放定址法的比较次数通常比拉链法多 **1**。这是统考反复设置的对比陷阱。

（八）一个经典小题

问：有 k 个关键字互为同义词，用线性探测法把它们全部存入散列表，至少需要多少次探测?

解：第 1 个元素探测 1 次即可存入；第 2 个元素先探测到第 1 个（冲突），再探测 1 次存入，共 2 次；……第 k 个元素共探测 k 次。总计

$$1+2+\cdots+k=\frac{k(k+1)}{2}$$

答案 $\frac{k(k+1)}{2}$ 次。

散列表统考考法汇总

- 提高散列表查找效率的措施（2011 年）：设计冲突少的散列函数、处理冲突时避免堆积。增大装填因子是错的。
- 堆积直接影响什么（2014 年）：平均查找长度。不影响存储效率、散列函数、装填因子。
- 影响 ASL 的因素（2022 年）：装填因子、散列函数、冲突解决策略——三项都影响。
- 给插入序列求 $ASL_{成功}$（2018 年）：线性探测法，逐个数比较次数。
- 给插入序列求 $ASL_{失败}$（2019 年）：注意分母是 $p=7$ 而不是表长 11。
- 插入后删除再求 $ASL_{失败}$（2023 年）：关键是「删除标记」不能当空位，要越过它继续探测。
- 拉链法的 ASL：注意空链的查找长度为 0。
- 装填因子：$\alpha=n/m$，是唯一能描述「表满程度」的量。

八、字符串模式匹配

模式匹配：在主串（目标串）S 中查找模式串 T 第一次出现的位置。

（一）朴素算法（**BF** 算法）

思路：从主串的第 1 个字符起与模式串逐个比较；若某个位置失配，则主串指针回退到本次起始位置的下一位，模式串指针回到开头，重新比较。

```
int NaiveMatch(const char *s, const char *t) {
    int i = 0, j = 0;
    while (s[i] != '\0' && t[j] != '\0') {
        if (s[i] == t[j]) { ++i; ++j; }        // 继续比较后继字符
        else { i = i - j + 1; j = 0; }         // 指针回退，重新开始
    }
    if (t[j] == '\0') return i - j;            // 匹配成功
    return -1;                                 // 匹配失败
}
```

时间复杂度：设主串长 n、模式串长 m。最坏情况下，每个起始位置都要比较 m 次，起始位置有 $n-m+1$ 个，所以

$$O\big((n-m+1)\cdot m\big)=O(nm)$$

最坏情形举例：主串 `aaaaaaaaab`，模式串 `aaab`。前 3 个字符每次都匹配，第 4 个失配，然后回退重来，比较次数接近 nm。

（二）KMP 算法的思想

核心观察：当在模式串的第 j 位失配时，主串指针不回退，模式串指针回退到某个位置。

为什么可以不回退？因为失配前已经匹配的那一段 $T[0..j-1]$ 与主串是对应的。如果要让模式串继续与主串匹配，模式串必须向右滑动尽量少的距离——等价于在 $T[0..j-1]$ 中找最长的、既是真前缀又是真后缀的子串，长度记为 k，则模式串指针只需回到位置 k，主串指针原地不动。

用「甲（主串）乙（模式串）」的比喻：不匹配发生时，前面匹配的那一小段对二者是相同的。用这一小段的头部去匹配它的尾部，最长的那段就是答案。

例：主串 `abbaabbaaba`，模式串 `abbaaba`，在第 7 个字符失配。已经匹配的前缀是 `abbaab`：

- 真前缀：`a`，`ab`，`abb`，`abba`，`abbaa`（不含最后一个字符）
- 真后缀：`b`，`ab`，`aab`，`baab`，`bbaab`（不含第一个字符）

最长的相等真前缀与真后缀是 `ab`，长度 2。所以模式串指针回退到第 3 个字符（下标 2）继续比较，主串不回退。

（三）部分匹配表 PMT 与 next 数组

部分匹配表（**PMT, Partial Match Table**）：对模式串的每个位置 i，记录子串 $T[0..i]$ 的「最长相等真前缀与真后缀」的长度。

高效计算 PMT 的关键洞察

次大匹配必定落在最大匹配之中。

若 $T[0..i]$ 的最大匹配长度是 L，则次大匹配一定是前缀 $T[0..L-1]$ 的最大匹配长度，可以直接查已经算好的表。这就是 KMP 求 `next` 数组的递推来源。

算例：模式串 `abababzabababa`

子串位置	**a**	**b**	**a**	**b**	**a**	**b**	**z**	**a**	**b**	**a**	**b**	**a**	**b**	**a**
PMT	0	0	1	2	3	4	0	1	2	3	4	5	6	5

最后一个位置：前面 `abababz` 的 PMT 是 0，但整体 `abababzababab` 的最大匹配是 6（`ababab`）。末尾再来一个 `a` 时，不能直接 $6+1=7$（因为 `abababz` 的第 7 个字符是 `z`，不等于 `a`）。于是回退到次大匹配 4（`abab`），发现 `abab` 之后的字符是 `a`，与末尾的 `a` 相等，所以 PMT 为 $4+1=5$。

从 **PMT** 到 **next**：为了编程方便，不使用 `PMT` 数组本身，而是把 `PMT` 整体向右偏移一位，并把第 0 位置设为 -1，得到的新数组称为 `next` 数组。

next 数组的两种教材约定（统考必须掌握）

约定 **A**（**0** 基下标，`next[0] = -1`）——多数国内教材与代码实现的写法：

- `next[j]` 表示「$T[0..j-1]$ 的最长相等真前后缀长度」，再整体后移一位、首项置 -1；
- 失配于 j 时执行 `j = next[j]`；
- `next[0] = -1` 只是一个哨兵值，用来表示「模式串第 1 个字符就失配，主串指针后移一位」。

约定 **B**（王道教材，**1** 基下标，`next[1] = 0`）：

- `next[j]` 表示「$T[1..j-1]$ 的最长相等真前后缀长度再加 **1**」；
- 失配于 j 时令 `j = next[j]`；
- `next[1] = 0` 是哨兵，`next[j] = 0` 表示模式串已退到开头，需要主串指针后移。

两者的换算：把 0 基的 next 数组每一项加 **1**，就得到 1 基的 next 数组；反之减 1。

0 基 next	**-1**	**0**	**0**	**1**	**2**	**3**	**1**	**1**	**2**	**3**	**4**	**5**
1 基 next	0	1	1	2	3	4	2	2	3	4	5	6

考试时先看题目给的 `next[1]` 是 **0** 还是 -1，再决定用哪套。

求 **next** 数组的代码（**0** 基约定）：

```
void getnext(const char *s, int next[]) {
    next[0] = -1;
    int i = 0, j = -1;
    while (s[i + 1] != '\0') {
        if (j == -1 || s[i] == s[j]) {
            ++i; ++j;
            next[i] = j;
        } else {
            j = next[j];      // 回退到次大匹配
        }
    }
}
```

KMP 匹配主程序（**0** 基约定）：

```
int KMP(const char *a, const char *b, const int next[]) {
    int i = 0, j = 0;
    while (a[i] != '\0' && b[j] != '\0') {
        if (j == -1 || a[i] == b[j]) { ++i; ++j; }
        else { j = next[j]; }     // 主串指针 i 不回退
    }
    if (b[j] == '\0') return i - j;   // 匹配成功
    return -1;
}
```

（四）手算 **next** 数组的流程

以模式串 ababaaababaa 为例（0 基）：

第 **1** 步：求 **PMT**

对每个位置 i，看子串 $T[0..i]$ 的最长相等真前后缀长度。

char	a	b	a	b	a	a	a	b	a	b	a	a
index	0	1	2	3	4	5	6	7	8	9	10	11
PMT	0	0	1	2	3	1	1	2	3	4	5	6

验证几个：

- $i=4$：子串 ababa，真前缀 aba 与真后缀 aba 相等，长度 3 ✓
- $i=5$：子串 ababaa，真前缀最长匹配是 a（首尾都是 a，ab 不行因为末尾是 aa），长度 1 ✓
- $i=11$：子串 abababaababaa，真前缀 ababaa 与真后缀 ababaa 相等，长度 6 ✓

第 **2** 步：**PMT** 右移一位，首项置 -1，得到 **next**

index	**0**	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**
next	-1	0	0	1	2	3	1	1	2	3	4	5

第 **3** 步：若要 **1** 基约定，每项 **+1**

index	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**
next	0	1	1	2	3	4	2	2	3	4	5	6

next 数组的速算法

不需要每次都画 PMT 表。直接用定义：

$next[j]$（**1** 基约定）= $T[1..j-1]$ 的最长相等真前后缀长度 $+1$。

- $next[1]=0$（哨兵）；
- $next[2]=1$（前 1 个字符没有真前后缀，长度 0，$0+1=1$）；
- 从 $j=3$ 起：看 $T[1..j-1]$ 的最长相等真前后缀长度 k，则 $next[j]=k+1$。

（五）nextval 数组（优化）

问题：如果 $T[j]$ 失配，按 next 回退到 $j'=next[j]$，而恰好 $T[j']=T[j]$，那么这次比较必然还是失配，白比一次。

优化思路：若 $T[j]=T[next[j]]$，则把 nextval[j] 直接设为 nextval[next[j]]（递归跳过所有「回去也必然失配」的位置）。

```
void getnextval(const char *s, const int next[], int nextval[]) {
    nextval[0] = -1;                    // next[0] 为 -1，不能访问 s[-1]
    for (int i = 1; s[i] != '\0'; i++) {
        if (next[i] == -1 || s[i] != s[next[i]])
            nextval[i] = next[i];       // 字符不同，保留原 next
        else
            nextval[i] = nextval[next[i]]; // 字符相同，继续往前跳
    }
}
```

算例：$s=$ ababaaababaa

index	**0**	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**
char	a	b	a	b	a	a	a	b	a	b	a	a
next	-1	0	0	1	2	3	1	1	2	3	4	5
nextval	-1	0	-1	0	-1	3	1	0	-1	0	-1	3

逐项验证：

- $i=1$：`s[next[1]]=s[0]='a'`，`s[1]='b'`，不同 ⇒ `nextval[1] = next[1] = 0` ✓
- $i=2$：`s[next[2]]=s[0]='a'`，`s[2]='a'`，相同 ⇒ `nextval[2] = nextval[0] = -1` ✓
- $i=4$：`s[next[4]]=s[2]='a'`，`s[4]='a'`，相同 ⇒ `nextval[4] = nextval[2] = -1` ✓
- $i=5$：`s[next[5]]=s[3]='b'`，`s[5]='a'`，不同 ⇒ `nextval[5] = 3` ✓

另外几个常用练习：

模式串	**next（0 基）**	**nextval（0 基）**
ababaabab	$-1,0,0,1,2,3,1,2,3$	$-1,0,-1,0,-1,3,0,-1,0$
pqppqpqp	$-1,0,0,1,1,2,3,2$	$-1,0,-1,1,0,-1,3,-1$
abaabcac	$-1,0,0,1,1,2,0,1$	$-1,0,-1,1,0,2,-1,1$

nextval 的三个细节

1. `nextval[0]` 一定等于 `next[0]`（$=-1$ 或 **1** 基约定下的 **0**），因为不存在 `s[-1]` 可比。
2. 判断的是「字符是否相同」，不是「下标是否相同」：`s[i] != s[next[i]]` 时保留原值。
3. `nextval` 只影响比较次数，不影响匹配结果的正确性，也不改变 $O(n+m)$ 的复杂度量级——它减少的是常数因子。

（六）匹配过程与复杂度

匹配过程要点：

- 主串指针 i 只前进不回退；
- 模式串指针 j 失配时跳到 `next[j]`（或 `nextval[j]`），可能多次跳跃，但总跳跃距离不超过模式串长度；
- 匹配成功时返回 $i-j$（模式串在主串中的起始下标）。

为什么 KMP 是 $O(n+m)$

- 求 `next`（或 `nextval`）数组只扫描一遍模式串，$O(m)$；
- 匹配过程中，主串指针 i 单调递增、最多移动 n 次；模式串指针 j 每失配一次就减小，而 j 的增大量总和不超过 i 的增量，所以 j 的减小总量也是 $O(n)$。均摊下来匹配是 $O(n)$。

总计 $O(n+m)$，比朴素算法的 $O(nm)$ 好一个量级。

KMP 统考考法汇总

- 给主串、模式串，问第一次失配后 i 和 j 分别是多少（2015 年）。答：i 不变（仍是失配位置），j 变成该位置对应的 `next` 值。例：$S=$ `abaabaabacacaabaabcc`，$t=$ `abaabc`，第一次失配时 $i=j=5$。失配前已匹配的模式串前缀是 `abaab`，其最长相等真前后缀是 `ab`（长度 2）。所以 $i=5$ 不动，$j=2$。

- 问到匹配成功为止一共比较了多少次（2019 年）。做法：逐段累加「成功比较的次数 + 1 次失配 + 后续成功比较的次数」。模式串 abaabc 若前 5 个匹配、第 6 个失配，则这一段比较 6 次；回退到 $j=2$ 后续匹配若干次。总比较次数需按题目给定的主串逐字符数，不能凭公式猜。
- 问 **nextval**（修正 **next**）下模式串最大滑动距离（2024 年）。失配于 j 时，模式串向右滑动的距离为 $j-nextval[j]$（0 基约定）。对 $S=$ aabaab，0 基 nextval 为 $[-1,0,1,0,1,2]$，各位置滑动距离 $j-nextval[j]$ 依次为 $1,1,1,3,3,3$，最大为 **3**。（若按 1 基约定计算，注意换算。）

九、查找算法的分析及应用

（一）各查找方法的横向对比

查找方法	存储结构要求	是否支持动态（插入删除）	时间复杂度	$ASL_{成功}$	$ASL_{失败}$
顺序查找（无序）	顺序表或链表	插入简单，删除 $O(1)$（链表）	$O(n)$	$\frac{n+1}{2}$	n
顺序查找（有序）	顺序表或链表	同上	$O(n)$	$\frac{n+1}{2}$	$\frac{n}{2}+\frac{n}{n+1}$
分块查找	索引表（顺序）+ 块（顺序或链表）	较方便	$O(\sqrt{n})$（$s=\sqrt{n}$ 时）	$\frac{b+1}{2}+\frac{s+1}{2}$，最优 $\sqrt{n}+1$	需逐个分析
折半查找	有序顺序表	不支持（插入删除 $O(n)$）	$O(\log n)$	$\approx \log_2(n+1)-1$	$\frac{(h-1)A+h\cdot 2k}{n+1}$（逐个数更稳）
二叉搜索树	任意（链式亦可）	支持，$O(\log n)$ 平均、$O(n)$ 最坏	平均 $O(\log n)$，最坏 $O(n)$	与树形相关	与树形相关
AVL 树	任意	支持，但要旋转	$O(\log n)$ 最坏	与树形相关，$O(\log n)$	$O(\log n)$
红黑树	任意	支持，调整次数少	$O(\log n)$ 最坏	$O(\log n)$	$O(\log n)$
B 树 / **B+** 树	任意（面向磁盘）	支持（分裂/合并）	$O(\log_m n)$ 层磁盘访问	与结点内查找方式有关	与结点内查找方式有关
散列表	散列表	支持	期望 $O(1)$	与 α 和冲突处理有关	与 α 和冲突处理有关

（二）选用原则

查找方法的选择

1. 表是静态的（只查不改）+ 顺序存储 + 有序 → 折半查找。效率最高（$O(\log n)$）且不需要额外空间。
2. 表是静态的 + 链式存储 / 无序 → 顺序查找。顺序查找是唯一在链表上也能直接用的方法。
3. 表是动态的（频繁插入删除） → 树形结构。查多改少用 **AVL**；增删频繁用红黑树；数据量大、面向磁盘用 **B/B+** 树。
4. 要求 $O(1)$ 级别的等值查找，且不需要范围查询和有序遍历 → 散列表。散列表的缺点是：不支持范围查询（无序）、最坏情况下会退化（所有关键字同义词）、需要额外空间、装填因子高时 ASL 迅速上升。
5. 既要有序又要快速，且数据量大到内存放不下 → **B+** 树（数据库索引的标准选择）。
6. 数据量小 → 顺序查找反而更快（常数因子小）。
7. 需要范围查询 / 顺序遍历 → B+ 树、有序表；散列表不适合。

（三）三个必须能自己推的结论

1. 折半查找判定树的树高 $h=\lceil\log_2(n+1)\rceil$，它同时是成功与失败查找的比较次数上限。
2. 折半查找的平均查找长度：$n=2^h-1$ 时为 $h-1=\log_2(n+1)-1$（最优）；一般情形下介于 $\log_2(n+1)-1$ 与 $\lceil\log_2(n+1)\rceil$ 之间。
3. 散列表的 ASL 只依赖装填因子 α，不直接依赖 n 或 m——所以「装填因子相同，ASL 就大致相同」是散列表的核心工程性质。

本章小结表

考点	核心结论
查找长度	比较关键字的次数，不是访问结点数
ASL	$ASL=\sum P_iC_i$；成功用 n 做分母，失败用失败位置数做分母
判定树	n 个内部结点 + $n+1$ 个外部结点；失败结点查找长度 = 父结点层数
顺序查找	无序：$ASL_{成功}=\frac{n+1}{2}$，$ASL_{失败}=n$；有序：$ASL_{成功}$ 不变，$ASL_{失败}=\frac{n}{2}+\frac{n}{n+1}$
哨兵	减少一次循环判断，不改变 ASL；可放表头（王道）或表尾
分块查找	$ASL=\frac{b+1}{2}+\frac{s+1}{2}$；最优 $s=\sqrt{n}$，$ASL_{\min}=\sqrt{n}+1$；索引表命中后仍要进块内查（块内无序时）
折半查找适用条件	有序 + 顺序存储；有序链表、无序数组、静态链表都不适合
折半判定树	一定是平衡二叉树；形态唯一，只与 n 有关
mid 取整	下取整 ⇒ 右子树结点数 − 左子树结点数 $\in\{0,1\}$；上取整则相反
折半比较次数上限	$\lceil\log_2(n+1)\rceil$（成功与失败都是）
折半 $ASL_{成功}$	$\log_2(n+1)-1$（满树时取等，一般略大但小于树高）
折半 $ASL_{失败}$	分母 $n+1$；公式 $\frac{(h-1)A+h\cdot 2k}{n+1}$，其中 $k=n-(2^{h-1}-1)$、$A=n+1-2k$；满树（$n=2^h-1$）时 $A=0$，$ASL_{失败}=h$
折半失败比较次数上限	$\lceil\log_2(n+1)\rceil$（与成功查找上界相同）
BST 定义	左 < 根 < 右；中序遍历递增；无重复关键字
BST 插入	新结点一定是叶结点
BST 删除	三种情况：叶子直接删；单子树用子结点顶替；双子树用中序前驱（左子树最右下）或中序后继（右子树最左下）替换
BST 删除再插入	v 是叶结点 ⇒ 形态相同；v 非叶 ⇒ 形态一定不同
BST 与折半	折半判定树唯一；BST 树形随插入顺序变化；BST 支持动态插入删除
AVL 定义	任一结点 $\lvert BF\rvert\le 1$；空结点 $BF=0$
AVL 四种旋转	LL 右旋；RR 左旋；LR 先左后右；RL 先右后左
AVL 插入	调最小不平衡子树即可，祖先自动恢复
AVL 删除	可能多次向上调整
AVL 最少结点	$N(h)=N(h-1)+N(h-2)+1$，$N(0)=0,N(1)=1,N(2)=2$；$N(h)=F_{h+2}-1$
AVL 高度范围	最多 2^h-1 个结点；n 个结点的最大高度 $\approx 1.44\log_2 n$
红黑树五性质	口诀「左根右、根叶黑、不红红、黑路同」；注意「叶结点」指空结点
红黑树结论	最长路径 ≤ 最短路径的 2 倍；$h\le 2\log_2(n+1)$；查找 $O(\log n)$

考点	核心结论
红黑树插入	新结点染红；黑叔 → 旋转 + 染色；红叔 → 染色 + 变新（爷爷变新结点）
红黑树 vs AVL	红黑树平衡条件松、调整少，适合增删频繁；AVL 树更矮，适合以查为主
m 阶 B 树定义	每结点 $\le m$ 棵子树；非根非叶 $\ge \lceil m/2 \rceil$ 棵子树（$\ge \lceil m/2 \rceil - 1$ 个关键字）；根 ≥ 1 个关键字；所有叶结点同层
B 树关键字范围	高度 h 时最少 $2\lceil m/2 \rceil^{h-1} - 1$，最多 $m^h - 1$
B 树高度范围	$\log_m(n+1) \le h \le \log_{\lceil m/2 \rceil} \frac{n+1}{2} + 1$
B 树查找	内部结点命中即成功；失败到达空结点
B 树插入	一定从终端结点插入；超限则上移第 $\lceil m/2 \rceil$ 个关键字并分裂；可能长高（根分裂）
B 树删除	非终端结点先用前驱/后继替换；下溢则借（旋转）或合并（三方合并）；根下溢到 0 则删根降高
B 树判定	三步：叶结点同层、中序递增、关键字个数在 $[\lceil m/2 \rceil - 1, m-1]$ 内（根为 $[1, m-1]$）
B+ 树特点	子树数 = 关键字数；只有叶结点存记录信息；叶结点有序链接；查找必到最后一层
B+ 树应用	关系数据库索引（阶大、树矮、读盘少、支持范围查询）
散列函数	除留余数法 $H(key) = key \bmod p$（p 为不大于 m 的最大质数）；直接定址法；数字分析法；平方取中法
装填因子	$\alpha = n/m$；ASL 只依赖 α，不直接依赖 n 或 m
拉链法空位	空链表查找长度为 **0**，空指针不计比较；可直接删除
开放定址空位	空单元的比较计入查找长度；删除必须留标记
开放定址 $ASL_{失败}$ 分母	散列函数值域大小（通常是 p），不是表长 m
线性探测堆积	同义词 + 非同义词共同造成；直接影响 ASL
平方探测	至少能探测一半位置；表长为 $4k+3$ 的素数时可探测全部位置
双散列	只是缓解堆积，不能完全避免；$hash_2$ 与 m 互质时可探测全部
KMP next（0 基）	`next[0] = -1`；`next[j]` = $T[0..j-1]$ 最长相等真前后缀长度后移一位
KMP next（1 基）	`next[1] = 0`；`next[j]` = $T[1..j-1]$ 最长相等真前后缀长度 +1
nextval	若 $s[i] = s[next[i]]$ 则 `nextval[i]=nextval[next[i]]`，否则等于 `next[i]`
KMP 复杂度	$O(n+m)$；主串指针不回退
朴素匹配复杂度	$O(nm)$

易错清单

查找章 20 个高频陷阱

1. $ASL_{失败}$ 的分母不是 n。顺序查找是 $n+1$；折半查找是 $n+1$ 个外部结点；散列表开放定址是散列函数值域大小（常为 p）。
2. 失败结点的查找长度 = 父结点层数，外部结点自身不产生比较。把它当成第 $h+1$ 层会整体偏大。
3. 折半查找的失败比较次数上限也是 $\lceil \log_2(n+1) \rceil$，不是 $\lfloor \log_2 n \rfloor$。$n = 16$ 时答案是 5 不是 4。
4. $ASL_{成功} = \log_2(n+1) - 1$ 只在 $n = 2^h - 1$ 时取等，一般情况下要略大，且始终小于树高。

5. **mid** 的取整方式必须全程一致。判断某树是否为判定树时，逐层检查左右子树规模关系是否符合同一规则。
6. 折半查找判定树唯一，**BST** 不唯一。别把两者的性质互换。
7. 有序表用顺序查找时，只有 $ASL_{\text{失败}}$ 变小（$\frac{n}{2}+\frac{n}{n+1}$），$ASL_{\text{成功}}$ 仍是 $\frac{n+1}{2}$。
8. 哨兵不改变 ASL，只减少一次循环判断。两种哨兵位置的返回值判失败方式不同（表尾看是否等于 n，表头看是否为 0）。
9. 分块查找：索引表命中不等于查找成功，块内无序时必须进块内再查一次。
10. 分块查找最优块长 $s=\sqrt{n}$，$ASL_{\text{min}}=\sqrt{n}+1$，别漏掉 $+1$。
11. **BST** 新插入的结点一定是叶结点；对 **AVL** 树不成立（旋转会把新结点抬上去）。
12. **BST/AVL** 删除再插入：BST 中 v 是叶结点则形态相同、非叶则一定不同；**AVL** 中只能说「可能不同」，不能用「一定」。
13. **AVL** 插入只需调整最小不平衡子树（祖先自动恢复）；**AVL** 删除可能需要多次调整直到根。
14. **AVL** 的「最少结点数」和「最大高度」是同一个问题的两种问法，不要分别背两套数字。
15. 红黑树的「叶结点」是空结点（外部结点），不是带关键字的最下层结点。性质 3 说的是所有外部结点都是黑的。
16. 红黑树插入遇红叔时只染色不旋转，并把祖父当成新结点继续向上处理；传到根时把根强制染黑。
17. m 阶 **B** 树的关键字数下限是 $\lceil m/2\rceil-1$（非根）和 **1**（根）。给定结点数求「结点数最多」时，别忘了「所有叶结点必须在同一层」这个约束，不能简单用 $n/(m-1)$。
18. **B** 树删除非终端结点关键字必须先替换（前驱或后继），替换后再删终端结点里的值；合并时别漏掉父结点中的分隔关键字（三方合并）。
19. **B+** 树查找无论成功失败都要走到最下一层；B 树可能在内部结点就成功。B+ 树的「子树数 = 关键字数」，B 树是「子树数 = 关键字数 + 1」。
20. 散列表的三组对比：① 开放定址法空位计入比较（拉链法空指针不计）；② 开放定址法删除要留标记（拉链法可直接删）；③ $ASL_{\text{失败}}$ 的分母是 p 不是 m。堆积影响的是 ASL，不是装填因子。

第七章排序

本章考纲要求

> 七、排序（一）排序的基本概念（二）直接插入排序（三）折半插入排序（四）起泡排序（bubble sort）（五）简单选择排序（六）希尔排序（shell sort）（七）快速排序（八）堆排序（九）二路归并排序（merge sort）（十）基数排序（十一）外部排序（十二）排序算法的分析与应用

排序是「算法题 + 选择题」双线考察的章节。选择题集中在前十二个条目的过程模拟（给一趟结果反推算法、给一趟结果反推增量、问比较/移动次数）与算法选择；应用题则考察堆的手工建堆与调整、快排划分、基数排序手工模拟、外部排序的置换-选择与最佳归并树。因此本章的写法是：每个算法都给出可直接照做的分步流程，并用带具体数字的序列把每一趟的状态写出来。

一、排序的基本概念

（一）什么是排序

排序（**sorting**）：重新排列表中的元素，使表中元素按关键字递增（或递减）有序。设含 n 个记录的序列为 $\{R_1, R_2, \ldots, R_n\}$，其对应关键字为 $\{K_1, K_2, \ldots, K_n\}$，则排序即确定一种排列 $p_1, p_2, \ldots, p_n$，使 $K_{p_1} \le K_{p_2} \le \cdots \le K_{p_n}$（升序）。

若待排序列中存在两个及以上关键字相同的记录，排序后它们之间的相对次序可能改变，这就是稳定性问题的来源。

（二）稳定性（**stability**）

稳定性的定义

对序列中任意两个关键字相等的记录 R_i 与 R_j（$K_i = K_j$ 且排序前 $i < j$），若排序后 R_i 仍排在 R_j 之前，则该排序算法稳定；否则不稳定。

- 只有「相等关键字元素在原始序列中的相对次序保持不变」才叫稳定。
- 稳定与否只取决于算法本身，与关键字的具体取值无关。讲稳定性时必须给出存在两个相等关键字的例子，否则无从谈起。

判断一个算法是否稳定，最可靠的方法不是在脑子里试一个例子，而是分析算法的核心操作：

1. 该操作是否会把某个元素跨越另一个与它关键字相等的元素？
2. 比较时用 $<$ 还是 $\le$？若在两元素相等时选择「不动」，通常有利于保持稳定。

稳定性的意义在于多关键字排序。例如先按总分排序、再按语文成绩排序，若第二次排序是稳定的，则语文成绩相同的记录仍保持总分从高到低的次序，等价于一次性完成了「按语文为主关键字、总分为次关键字」的排序。

稳定性易错点

- 折半插入排序是稳定的，容易误判为不稳定。它在折半查找时用的是「小于等于则向右」的策略，使得相等元素被插到已有序的相等元素右边，顺序不变。
- 希尔排序、简单选择排序、快速排序、堆排序不稳定；直接插入、折半插入、起泡、归并、基数排序稳定。

- 「某算法存在稳定版本与不稳定版本」时，统考按稳定理解（例如交换时判 $\geq$ 则不交换）。
- 排序算法的稳定性与「某一趟结果是否稳定」无关，别把「一趟排序后序列没变」当成稳定的证据。

（三）内部排序与外部排序

- 内部排序（**internal sort**）：待排序记录全部存放在内存中，排序全过程在内存完成。前面十种算法都是内部排序。
- 外部排序（**external sort**）：待排序文件很大，内存无法一次容纳全部记录，排序过程中需要多次在内存与外存之间交换数据。外部排序的时间主要由读写外存次数决定，而非比较次数。

内部排序关心「比较次数 + 移动次数」，外部排序关心「磁盘 I/O 次数」，这是两者分析指标的根本区别。

（四）比较次数与移动次数

评价一个内部排序算法，除了渐进时间复杂度，统考经常要求精确计数：

- 比较次数：关键字之间进行大小比较的次数。
- 移动次数：记录整体搬移的次数。一次关键字赋值算一次移动；一次 `swap` 需要 3 次移动（$temp \leftarrow a$；$a \leftarrow b$；$b \leftarrow temp$）。

三种计数模式要分清

1. 与初始状态无关：简单选择排序的比较次数恒为 $\frac{n(n-1)}{2}$；折半插入排序的比较次数只与 n 有关；基数排序的移动次数只与 n 和位数 d 有关。
2. 与初始状态有关：直接插入、起泡、希尔、快速、堆排序的比较/移动次数都随初始序列状态变化。
3. 最好情况：直接插入与优化后的起泡在完全有序时只需 $n-1$ 次比较、0 次移动，达到 $O(n)$。

还有一个常用工具——逆序数。若 $i<j$ 而 $a_i>a_j$，称 (a_i,a_j) 构成一个逆序对，逆序对的总数即逆序数。排序过程就是不断消除逆序对直到逆序数为 0 的过程。

- 只做相邻交换的算法（起泡、直接插入），一次交换/右移最多消除一个逆序，所以平均交换次数为 $O(n^2)$，算法不可能优于 $O(n^2)$。
- 能做长距离移动的算法（希尔、快速、堆、归并），一次操作可能同时消除多个逆序，因此能够突破 $O(n^2)$。

（五）基于比较的排序的下界

起泡、插入、选择、希尔、快速、堆、归并都属于基于比较的排序（comparison sort）：只能通过关键字之间的比较来确定相对次序。这类算法的时间下界是 $\Omega(n\log n)$。

决策树证明

把算法对 n 个元素的所有比较过程画成一棵二叉决策树：每个内部结点是一次比较，两条分支对应两种比较结果；每个叶结点对应一个可能的输出排列。n 个元素共有 $n!$ 种排列，因此叶结点数 l 满足 $n!\leq l$。设树高为 h，二叉树的叶结点数最多 2^h，故

$$n!\leq l\leq 2^h \implies h\geq\lceil\log_2(n!)\rceil=\Omega(n\log n)$$

即最坏情况下至少需要 $\Omega(n\log n)$ 次比较。用斯特林公式 $n!\approx\sqrt{2\pi n}\,(n/e)^n$ 可得 $\log_2(n!)\approx n\log_2 n-1.44n+O(\log n)$。

结论：任何基于比较的排序算法，最坏时间都不能低于 $O(n\log n)$。想达到线性时间，必须放弃比较，改用「按关键字的值直接确定位置」的办法——这就是基数排序。

真题考法

2019 年第 7 题直接考察「选择排序算法时还需考虑哪些因素」，正确答案是数据规模、存储方式、稳定性、初始状态四项都要考虑。这类题要求你对每个算法的「适用条件」有清晰记忆，而不是只记复杂度。

2012 年第 10 题、2023 年第 10 题、2015 年第 9 题、2017 年第 11 题分别考察「每趟至少确定一个元素最终位置的算法有哪些」「移动次数与初始状态无关的算法」「更换为链式存储后效率降低的算法」「不稳定的算法有哪些」，本质都是本章（十二）那张对比表的内容。

二、直接插入排序

（一）算法思想

直接插入排序（**Insertion Sort**）：每次将一个待排序的记录按其关键字大小插入到前面已经排好序的子序列中，直到全部记录插入完成。

整个序列在逻辑上被划为两部分：左侧的有序区（初始只有第 1 个元素）和右侧的无序区。每趟取无序区的第一个元素，在有序区中从右往左找到插入位置，把它插进去。n 个元素共需 $n-1$ 趟。

（二）手工过程（逐步模拟）

以序列 $\{49,\ 38,\ 65,\ 97,\ 76,\ 13,\ 27,\ 49^*\}$ 为例（49^* 表示与前面的 49 关键字相同、用来观察稳定性的第二个记录）：

趟次	待插入元素	插入过程	本趟结束后序列
初始	—	49 视为已排序	`[49] 38 65 97 76 13 27 49*`
1	38	38<49，49 后移，38 放最前	`[38 49] 65 97 76 13 27 49*`
2	65	65≥49，不动	`[38 49 65] 97 76 13 27 49*`
3	97	97≥65，不动	`[38 49 65 97] 76 13 27 49*`
4	76	76<97、76<65？不成立（76>65），97 后移，76 插在 65 后	`[38 49 65 76 97] 13 27 49*`
5	13	13 比 97、76、65、49、38 都小，全体后移，13 放最前	`[13 38 49 65 76 97] 27 49*`
6	27	27<97、76、65、49、38，但 27>13，38 及其后元素后移	`[13 27 38 49 65 76 97] 49*`
7	49*	49\<*97*、*76*、*65*，但 *49*\≥49，故 97、76、65 后移，49* 插在 49 之后	`13 27 38 49 49* 65 76 97`

注意第 7 趟：判断条件是「$a_j > base$ 就继续往前找」，遇到相等的 49 时停止，于是 49^* 落在原 49 的右边，相对次序不变——这就是直接插入排序稳定的原因。

（三）参考代码

```
void InsertSort(int a[], int n) {
    int i, j, base;
    for (i = 1; i < n; ++i) {      // 共 n-1 趟
```

```
        base = a[i];                // 暂存待插入元素，a[i] 的位置成为“空位”
        j = i - 1;
        while (j >= 0 && a[j] > base) {  // 严格大于才后移，保证稳定
            a[j + 1] = a[j];
            --j;
        }
        a[j + 1] = base;            // 插入到正确位置
    }
}
```

（四）复杂度与特性

- 最好情况：序列本身有序，每趟只比较 1 次、不移动，比较次数 $n-1$，移动次数 0，时间 $O(n)$。
- 最坏情况：序列逆序，第 i 趟比较 i 次、移动 $i+1$ 次，总比较次数 $\sum_{i=1}^{n-1} i = \frac{n(n-1)}{2}$，时间 $O(n^2)$。
- 平均情况：$O(n^2)$；记录平均后移 $n/2$ 个位置，约需 $n^2/4$ 次移动。
- 空间：只需一个暂存变量，$O(1)$，原地排序。
- 稳定性：稳定。
- 适用存储：顺序存储和链式存储都适用。链表上无需移动元素，只需从头查找插入位置后改指针（但查找位置仍需 $O(n)$，总时间仍 $O(n^2)$）。

直接插入排序的三句话

1. 时间复杂度：最好 $O(n)$，最坏与平均 $O(n^2)$，空间 $O(1)$。
2. 稳定、原地、自适应（效率强烈依赖初始有序程度）。
3. 特别适合「记录基本有序」或「待排记录数 n 很小」的场合。

直接插入排序易错点

- 第 i 趟结束后，前 $i+1$ 个元素有序，但这部分元素不一定在最终位置上。这与起泡、选择、堆排序「每趟至少确定一个元素的最终位置」不同，是反推算法类型的关键判别点。
- 排序趟数与初始状态无关，恒为 $n-1$ 趟。
- 判断条件必须是 `a[j] > base`（严格大于），写成 `>=` 会使算法变成不稳定。

真题考法

2022 年第 11 题：问「采用直接插入排序而不采用快速排序」的可能原因，正确答案为 I（大部分元素已有序）、II（待排序元素数量很少）、III（要求空间复杂度 $O(1)$）、IV（要求排序稳定）全部正确。

2020 年第 11 题：对大部分元素已有序的数组，直接插入比简单选择效率高的原因是「直接插入的比较次数更少」。注意辅助空间两者都是 $O(1)$，移动次数也不能笼统断言谁更少。

2009 年第 10 题、2012 年第 11 题也从不同角度考过该算法。

三、折半插入排序

（一）与直接插入排序的唯一区别

折半插入排序（**binary insertion sort**）：把直接插入排序中「在有序区从后往前顺序查找插入位置」这一步，换成折半查找（二分查找）。确定位置后，元素的搬移方式与直接插入排序完全相同。

因此两者唯一的差别只在「比较次数」：

比较项	直接插入排序	折半插入排序
排序总趟数	$n-1$	$n-1$（相同）
元素的移动次数	相同（同一初始序列下完全一致）	相同
辅助空间	$O(1)$	$O(1)$（相同）
查找插入位置的比较次数	与初始状态有关；基本有序时极少，最坏 $\frac{n(n-1)}{2}$	只与 n 有关，约 $O(n\log n)$
稳定性	稳定	稳定

（二）比较次数与移动次数的分别分析

比较次数：对第 i 个元素（它前面有 i 个已排序元素），折半查找最多比较 $\lceil \log_2(i+1) \rceil$ 次。全部 $n-1$ 趟加起来

$$\sum_{i=1}^{n-1} \lceil \log_2(i+1) \rceil = O(n\log n)$$

平均约为 $n\log_2 n$ 次。这个次数与待排序序列的初始状态无关，只取决于元素个数 n。

移动次数：插入位置确定后，插入位置之后的元素仍要逐个后移一位，平均每次移动约一半元素，因此总移动次数仍为

$$\sum_{i=1}^{n-1} \frac{i+1}{2} \approx \frac{n^2}{4} = O(n^2)$$

折半插入排序的核心结论

- 比较次数 $O(n\log n)$，移动次数 $O(n^2)$，因此总时间复杂度仍为 $O(n^2)$。
- 比较次数与初始序列状态无关，这是它与直接插入排序最大的区别。
- 只适用于顺序存储（顺序表），不能用于链表。链式存储无法在 $O(1)$ 时间内随机访问中间元素，折半查找无法进行。

（三）折半查找的边界（决定稳定性）

为了保持稳定，折半查找要找的是最后一个不大于 $base$ 的元素的后一个位置。比较时若 `a[mid] <= base`，则插入位置在右半区（`low = mid + 1`），这样相等元素总被插到已有相等元素的右侧。

```
// 在有序区 a[0..i-1] 中找 base 的插入位置，返回应插入的下标
int binarySearch(int a[], int low, int high, int base) {
    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (a[mid] <= base) low = mid + 1;   // 相等时向右，保证稳定
        else high = mid - 1;
    }
    return low;   // low 即为插入位置
}

void BinaryInsertSort(int a[], int n) {
    for (int i = 1; i < n; ++i) {
        int base = a[i];
        int pos = binarySearch(a, 0, i - 1, base);
```

```
        for (int j = i; j > pos; --j)   // 后移，为 base 腾位置
            a[j] = a[j - 1];
        a[pos] = base;
    }
}
```

折半插入排序易错点

- 说「折半插入排序时间比直接插入排序快」是错的。它只减少了比较次数，移动次数没变，而移动比比较更耗时，所以总时间没有实质改善，反而在序列基本有序时比直接插入排序更慢（直接插入此时只需 $O(n)$ 次比较，而折半插入固定要 $O(n\log n)$ 次）。
- 「折半插入排序的比较次数与初始状态无关」是正确的。
- 折半插入排序不能用于链表，这条几乎年年出现在算法选择的判断题里。

真题考法

2012 年第 11 题：对同一待排序序列分别进行折半插入排序和直接插入排序，问两者可能的不同之处。四个选项分别是排序总趟数、元素移动次数、辅助空间数量、元素之间的比较次数。正确答案是 **D**（元素之间的比较次数）。总趟数都是 $n-1$，移动次数相同，辅助空间都是 $O(1)$。

四、起泡排序（bubble sort）

（一）算法思想

起泡排序（**Bubble Sort**）：从前往后依次比较相邻两个元素，若「左 > 右」则交换，一趟扫描结束后，当前无序区中关键字最大的元素被交换到无序区末尾。重复进行，直到某一趟没有发生任何交换（或已完成 $n-1$ 趟）。

逻辑分区与插入排序相反：「无序区在前、有序区在后」，有序区是从后往前逐步扩大的。

（二）手工过程（逐步模拟）

仍以 $\{49,\ 38,\ 65,\ 97,\ 76,\ 13,\ 27,\ 49^*\}$ 为例：

第 **1** 趟（无序区为整个序列，从左到右两两比较）：

- 49 与 38：49>38，交换 → `38 49 65 97 76 13 27 49*`
- 49 与 65：不换 → `38 49 65 97 76 13 27 49*`
- 65 与 97：不换 → `38 49 65 97 76 13 27 49*`
- 97 与 76：交换 → `38 49 65 76 97 13 27 49*`
- 97 与 13：交换 → `38 49 65 76 13 97 27 49*`
- 97 与 27：交换 → `38 49 65 76 13 27 97 49*`
- 97 与 49*：交换 → *'38 49 65 76 13 27 49* 97'*

第 1 趟结束，**97** 到达最终位置（下标 7）。

第 **2** 趟（只扫描前 7 个元素，97 不参与）：49 与 65 不换；65 与 76 不换；76 与 13 交换；76 与 27 交换；76 与 49* 交换。末尾再与 97 比较时不交换。

结果：`38 49 65 13 27 49* 76 97`，**76** 到达最终位置。

第 **3** 趟（只扫描前 6 个元素）：38 与 49 不换；49 与 65 不换；65 与 13 交换 → 49 与 13 交换 → 38 与 13 交换，之后 65 与 27 交换、65 与 49* 交换。

结果：13 38 49 27 49* 65 76 97，**65** 到达最终位置。

第 **4** 趟：13 与 38、38 与 49 不换；49 与 27 交换；49 与 49* 不换（相等不交换）。

结果：13 38 27 49 49* 65 76 97，**49*** 到达最终位置。

第 **5** 趟：38 与 27 交换，其余不再发生交换。

结果：13 27 38 49 49* 65 76 97，**38** 到达最终位置。

第 **6** 趟：从头到尾没有任何交换，flag 保持 false，算法提前结束。剩余 3 个元素 13、27、38 自然有序。

可以看到：每一趟至少把一个元素（当前无序区的最大值）送到它的最终位置，且已排好的有序后缀一定是全局有序的（这与插入排序的「局部有序前缀」截然不同，是判别算法类型的重要特征）。

（三）参考代码（带提前结束优化）

```
void BubbleSort(int a[], int n) {
    for (int i = 0; i < n - 1; ++i) {   // 最多 n-1 趟
        int flag = 0;                    // 本趟是否发生交换
        for (int j = 0; j < n - 1 - i; ++j) {
            if (a[j] > a[j + 1]) {       // 相等不交换，保证稳定
                int t = a[j]; a[j] = a[j + 1]; a[j + 1] = t;
                flag = 1;
            }
        }
        if (!flag) break;                // 本趟无交换，说明已全部有序
    }
}
```

（四）复杂度与特性

- 最好情况：序列完全有序，加 flag 优化后只做 1 趟，比较 $n-1$ 次，交换 0 次，时间 $O(n)$。
- 最坏情况：序列逆序，比较次数 $\sum_{i=1}^{n-1}(n-i)=\frac{n(n-1)}{2}$，交换次数也是 $\frac{n(n-1)}{2}$，每次交换 3 次移动，时间 $O(n^2)$。
- 平均情况：$O(n^2)$。
- 空间：$O(1)$，原地。
- 稳定性：稳定（相等时不交换）。
- 适用存储：顺序存储与链式存储都可用（链表上相邻结点仍可顺序比较交换）。

起泡排序的两个关键性质

1. 每一趟至少确定一个元素的最终位置，且产生的有序区在末尾、是全局有序的。
2. 加 **flag** 优化后，比较次数与初始序列状态有关：完全有序时 $n-1$ 次；逆序时 $\frac{n(n-1)}{2}$ 次。不加优化时，比较次数恒为 $\frac{n(n-1)}{2}$，与初始状态无关。

起泡排序易错点

- 起泡排序的趟数与初始状态有关：有序时 1 趟即可结束，逆序时需要 $n-1$ 趟。这是它与插入、选择排序的重要差别（插入、选择的趟数与初始状态无关）。
- 加不加 flag，直接影响「比较次数是否与初始状态有关」这个结论。答题时先确认题目中的代码是否带优化。
- 起泡排序的交换操作涉及 3 次移动，插入排序的移动只涉及 1 次赋值。因此虽然同为 $O(n^2)$，起泡排序的常数开销通常大于直接插入排序。

- 某些教材把带 `flag` 的优化版本与「冒泡界优化（记录上一趟最后一次交换的位置）」混为一谈。冒泡界优化包含了提前结束优化，但提前结束优化并不带界的收缩，答题时要按题面代码判断。

真题考法

2010 年第 11 题：给出某排序的前三趟结果为 `<2,12,16,5,10,88>` → `<2,12,5,10,16,88>` → `<2,5,10,12,16,88>`，问采用的排序方法。观察可知每趟都把当前最大值推到末尾（88 → 16 → 12），符合起泡排序特征，答案 **A**。

2009 年第 10 题与 2012 年第 10 题都涉及「每趟至少确定一个最终位置」这一判别特征。

五、简单选择排序

（一）算法思想

简单选择排序（**Selection Sort**）：每一趟在待排序元素（无序区）中选出关键字最小（或最大）的元素，与无序区的第一个元素交换，从而把有序区从前往后扩大一个位置。

n 个元素需要 $n-1$ 趟：第 i 趟（i 从 0 计）在区间 $[i,\ n-1]$ 中找最小值下标 `minIdx`，若 `minIdx != i` 则与 `a[i]` 交换。

（二）手工过程（逐步模拟）

以 $\{49,\ 38,\ 65,\ 97,\ 76,\ 13,\ 27,\ 49^*\}$ 为例：

趟次	无序区	选出的最小元素	交换操作	本趟结束后序列
1	`[49 38 65 97 76 13 27 49*]`	13（下标 5）	49 ↔ 13	`[13] 38 65 97 76 49 27 49*`
2	`[38 65 97 76 49 27 49*]`	27（下标 6）	38 ↔ 27	`[13 27] 65 97 76 49 38 49*`
3	`[65 97 76 49 38 49*]`	38（下标 6）	65 ↔ 38	`[13 27 38] 97 76 49 65 49*`
4	`[97 76 49 65 49*]`	49（下标 4）	97 ↔ 49	`[13 27 38 49] 76 49* 65 97`
5	`[76 49* 65 97]`	49*（下标 4）	76 ↔ 49*	`13 27 38 49 49* 65 97`
6	`[65 97]`	65（下标 5）	不交换	`13 27 38 49 49* 65 97`
7	`[97]`	只剩一个，无需处理	—	完成

观察第 4、5 趟：第 4 趟找到的 49 恰好在原 49\ 的左边，交换后保持不变；第 *5* 趟在 *'[76, 49*, 65, 97]' 中选最小值 49\（下标 *4*），与 *76* 交换，*49*\ 落到下标 4，此时 49\ 位于原 *49* 的右边，位置正确。本例中两个 *49* 的相对次序恰好没有被打乱，说明「举一个例子没乱」不能证明算法稳定 *，必须分析操作机制。

（三）参考代码

```
void SelectSort(int a[], int n) {
    for (int i = 0; i < n - 1; ++i) {
        int minIdx = i;
        for (int j = i + 1; j < n; ++j)
            if (a[j] < a[minIdx]) minIdx = j;    // 找无序区最小值下标
```

```
        if (minIdx != i) {                          // 位置不变时不做无谓交换
            int t = a[i]; a[i] = a[minIdx]; a[minIdx] = t;
        }
    }
}
```

（四）比较次数与移动次数

- 比较次数：第 1 趟比较 $n-1$ 次，第 2 趟 $n-2$ 次……最后一次 1 次，总数

$$\sum_{i=1}^{n-1} i = \frac{n(n-1)}{2}$$

与序列的初始状态完全无关，恒为 $\frac{n(n-1)}{2}$。这是所有排序算法中比较次数最”死”的一个。

- 移动次数：每趟最多交换一次，每次交换 3 次移动，因此最多 $3(n-1)$ 次移动（若代码写成「位置相同时不交换」，则更少）。这是所有内部排序算法中移动次数最少的之一（仅次于基数排序这类不靠移动的算法）。
- 空间：$O(1)$。
- 适用存储：顺序存储与链式存储都可用（链表上「找最小值」只需顺序扫描）。

简单选择排序的核心结论

- 比较次数固定为 $\frac{n(n-1)}{2} = O(n^2)$，与初始状态无关。
- 移动次数最多 $n-1$ 次交换，即 $O(n)$ 级别，是「比较得多、交换得少」的典型。
- 时间复杂度最好/最坏/平均都是 $O(n^2)$，非自适应。
- 不稳定。

（五）不稳定的原因与反例

简单选择排序为什么不稳定

不稳定的根源是：每趟把「无序区最小值」与「无序区第一个元素」直接交换，这是一次跨越多个元素的长距离交换。若在「无序区第一个元素」与「最小值所在位置」之间还存在与前者关键字相等（甚至与最小值相等）的元素，这次交换就会把它们之间的相对次序颠倒。

反例（最小规模）：序列 $\{8,\ 5,\ 5^*,\ 1\}$，要求升序。

- 第 1 趟：无序区 `[8，5，5*，1]`，最小值是 1（下标 3），与下标 0 的 8 交换 $\rightarrow \{1,\ 5,\ 5^*,\ 8\}$

排序结果中，原本在前的 5（下标 1）仍在 5*（下标 2）之前，这个例子恰好稳定。换一个例子：

反例：序列 $\{2,\ 2^*,\ 1\}$，要求升序。

- 第 1 趟：无序区 `[2，2*，1]`，最小值是 1（下标 2），与下标 0 的 2 交换 $\rightarrow \{1,\ 2^*,\ 2\}$

排序前 2 在 2^* 之前，排序后 2^* 在 2 之前，相对次序被改变，故简单选择排序不稳定。

另一个常用反例：$\{7,\ 7^*,\ 2\}$ 同理，第 1 趟交换第一个 7 与 2，得到 $\{2,\ 7^*,\ 7\}$。

简单选择排序易错点

- 「比较次数与初始状态无关」是正确的；「移动次数与初始状态无关」是错误的（移动次数与初始状态有关，最多 $n-1$ 次交换，可能一次都不交换）。
- 「每趟至少确定一个元素的最终位置」是正确的（每趟结束，最小值落到下标 i 的最终位置）。

- 排序趟数与初始状态无关，恒为 $n-1$ 趟。
- 判断不稳定时必须给出含两个相等关键字的具体反例，不能只说「因为它做跨越交换」。

真题考法

2020 年第 11 题：对大部分元素已有序的数组，直接插入比简单选择效率更高，原因是前者比较次数更少——因为简单选择排序的比较次数恒为 $\frac{n(n-1)}{2}$，不因数据接近有序而减少。

六、希尔排序（shell sort）

（一）算法思想

希尔排序（**Shell Sort**），又称缩小增量排序（**diminishing increment sort**）：先取一个小于 n 的增量 gap，把序列中所有下标相差 gap 的记录分为同一组，对每组分别进行直接插入排序；然后缩小 gap 重复上述过程；当 $gap=1$ 时，整个序列作为一组再做一次直接插入排序，排序完成。

每一次组内排序用的都是直接插入排序（这是 2015 年真题的直接考点）。

（二）为什么能改进——两条理由

希尔排序能突破 $O(n^2)$ 的两个依据

1. 直接插入排序对「基本有序」的序列效率极高。当 gap 很小时，序列在前几轮已经被整理得大致有序，此时插入排序的效率接近线性 $O(n)$。
2. 直接插入排序每次比较最多让元素移动一位，效率太低。希尔排序在 gap 较大时允许元素一次跨越 gap 个位置移动，一次操作可能消除多个逆序对，从而以低于 n^2 阶的次数把逆序数降到 0。

从逆序数角度看：起泡和直接插入只做相邻交换，一次最多消除一个逆序，所以交换次数为 $O(n^2)$。而希尔排序在 $gap>1$ 时做的是长距离交换，能一次消除多个逆序，因此总操作次数可以低于 $O(n^2)$。

（三）手工过程（逐步模拟）

以序列 $\{49,\ 38,\ 65,\ 97,\ 76,\ 13,\ 27,\ 49^*,\ 55,\ 4\}$ 为例，共 $n=10$，采用 Shell 增量 $gap=5,\ 2,\ 1$。

第 **1** 趟，$gap=5$：下标相差 5 的元素归为一组，共 5 组（下标 0–5、1–6、2–7、3–8、4–9）。

- 组 1：（49，13）→ 13 插到 49 前 → 13 49
- 组 2：（38，27）→ 27 插到 38 前 → 27 38
- 组 3：（65，49*）→ 49\ 插到 *65* 前 → *49*\ 65
- 组 4：（97，55）→ 55 插到 97 前 → 55 97
- 组 5：（76，4）→ 4 插到 76 前 → 4 76

写回原数组：$\{13,\ 27,\ 49^*,\ 55,\ 4,\ 49,\ 38,\ 65,\ 97,\ 76\}$

第 **2** 趟，$gap=2$：下标相差 2 的元素归为一组，共 2 组（偶数组 0,2,4,6,8 和奇数组 1,3,5,7,9）。

- 偶数组：下标 0,2,4,6,8 上的元素为（13，49*，4，38，97），直接插入排序后为（4，13，38，49*，97）
- 奇数组：下标 1,3,5,7,9 上的元素为（27，55，49，65，76），直接插入排序后为（27，49，55，65，76）
- 交错写回：$\{4,\ 27,\ 13,\ 49,\ 38,\ 55,\ 49^*,\ 65,\ 97,\ 76\}$

第 **3** 趟，$gap=1$：全序列做一次直接插入排序（此时序列已基本有序，只需较少比较与移动），得到最终结果

$$\{4,\ 13,\ 27,\ 38,\ 49,\ 49^{*},\ 55,\ 65,\ 76,\ 97\}$$

高频题型：给一趟结果反推增量

方法：若某一趟采用的增量为 gap，则排序后所有下标相差 gap 的元素构成的子序列都必须各自有序。逐个候选 gap 取出这些子序列检查是否有序即可。

例题（**2014** 年第 **10** 题）：某序列第 1 趟希尔排序结果为 9，1，4，13，7，8，20，23，15，则该趟采用的增量可能是多少?

- 检验 $gap=2$：下标 0,2,4,6,8 上的元素为 (9，4，7，20，15)——9>4，不有序，排除；奇数组 (1，13，8，23)——13>8，也不有序。
- 检验 $gap=3$：下标 0,3,6 为 (9，13，20) 有序；下标 1,4,7 为 (1，7，23) 有序；下标 2,5,8 为 (4，8，15) 有序。三组全部有序，故 $gap=3$ 可行。
- 检验 $gap=4$、$gap=5$ 均能找到组内逆序，排除。答案 **B**（**3**）。

例题（**2018** 年第 **10** 题）：初始序列 (8,3,9,11,2,1,4,7,5,10,6)，第 1 趟结果为 (1,3,7,5,2,6,4,9,11,10,8)，第 2 趟结果为 (1,2,6,4,3,7,5,8,11,10,9)。求两趟的增量。

第 1 趟按 $gap=5$ 分组：(8,1,6)→(1,6,8)、(3,4)→(3,4)、(9,7)→(7,9)、(11,5,10)→(5,10,11)、(2,11...) 逐组检查后交错写回恰好得到第 1 趟结果，故 $gap=5$。第 2 趟按 $gap=3$ 检查同样吻合。答案 **D**（**5, 3**）。

（四）参考代码

```
void ShellSort(int a[], int n) {
    for (int gap = n / 2; gap > 0; gap /= 2) {      // 增量序列: n/2, n/4, ..., 1
        for (int s = 0; s < gap; ++s) {             // 当前增量下有 gap 个子序列
            for (int i = s + gap; i < n; i += gap) {// 对每个子序列做直接插入排序
                int base = a[i], j = i - gap;
                while (j >= 0 && a[j] > base) {
                    a[j + gap] = a[j];
                    j -= gap;
                }
                a[j + gap] = base;
            }
        }
    }
}
```

Shell 增量无论 n 多大都从 $\lfloor n/2 \rfloor$ 开始不断除 2，直到 $gap=1$。（Donald Shell 1959 年原论文中 $n=11$ 的示例即取 $gap=5,2,1$。）

（五）复杂度分析

时间复杂度与增量序列的选择直接相关，最优增量序列至今没有明确结论。

增量序列	最坏时间复杂度
Shell 增量 $n/2^k$（$n/2,\ n/4,\ \ldots,\ 1$）	$\Theta(n^2)$；在 n 处于某些特定范围时可达到 $O(n^{1.3})$
Hibbard 增量	优于 Shell 增量

增量序列	最坏时间复杂度
Knuth 增量、Sedgewick 增量	进一步改进，最坏可达 $O(n^{1.5})$ 甚至更好

统考层面需要记住的表述是：希尔排序的时间复杂度约为 $O(n^{1.3})$，与增量序列有关，最坏可达 $O(n^2)$；当输入数组已有序时达到最好时间复杂度 $O(n)$。空间复杂度 $O(1)$。

（六）不稳定的原因与反例

希尔排序为什么不稳定

不稳定根源：当 $gap > 1$ 时，关键字相同的记录可能被划分到不同的子序列中。它们各自在自己的子序列里做插入排序，然后交错写回原数组，相对次序因此被打乱。也就是说，跳步移动让相等元素可以跨越彼此。

反例：序列 $\{0,\ 1,\ 4,\ 3_1,\ 3_2,\ 5,\ 6\}$，取 $gap = 2$。（下标 3 上的 3 记为 3_1，下标 4 上的 3 记为 3_2，3_1 原本在 3_2 之前。）

按下标相差 2 分组：

- 偶数组（下标 0,2,4,6）：`(0,\ 4,\ 3_2,\ 6)` → 直接插入排序后 `(0,\ 3_2,\ 4,\ 6)`
- 奇数组（下标 1,3,5）：`(1,\ 3_1,\ 5)` → 已有序，不变

交错写回原数组，得到

$$\{0,\ 1,\ 3_2,\ 3_1,\ 4,\ 5,\ 6\}$$

排序前 3_1 在 3_2 之前，排序后 3_2 在 3_1 之前，相对次序被改变。故希尔排序不稳定。

希尔排序易错点

- 希尔排序的组内排序是直接插入排序，不是折半插入、快排或归并（2015 年第 11 题）。
- 希尔排序的最后一趟增量一定是 **1**，且 $gap = 1$ 那一趟等价于对整个序列做一次直接插入排序。
- 希尔排序是原地排序（$O(1)$ 空间），这一点与快排、归并不同。
- 希尔排序不能简单地照搬到链表，因为它需要频繁按下标跳转访问下标相差 gap 的元素。
- 「希尔排序每趟至少能确定一个元素的最终位置」是错的。一趟只是让各个增量子序列有序，通常不能保证某个元素已经落在它整个序列的最终位置上（2012 年第 10 题）。
- 希尔排序的复杂度不能写成一个确定值，必须写成「与增量序列有关」。

真题考法

希尔排序是高频考点，2014、2015、2018 年连续考察。三种典型问法：

1. 给一趟结果反推增量（2014-10、2018-10）：检验「下标相差 gap 的子序列是否有序」。
2. 问组内排序用什么算法（2015-11）：答直接插入排序。
3. 与其他算法对比（2012-10、2017-11）：希尔排序不能保证每趟确定一个最终位置；换成链式存储后效率会降低。

七、快速排序

（一）算法思想与分治结构

快速排序（**Quick Sort**）基于分治思想，核心操作是一趟划分（**partition**）：选定一个基准元素（枢轴，**pivot**），通过一次扫描把整个序列分成三部分——左子序列（所有元素 $\leq$ 枢轴）、枢轴、右子序列（所有元素 $\geq$ 枢轴），并且枢轴被放到它的最终位置上。然后对左右两个子序列递归地重复划分，直到子序列长度不超过 1。

（二）一趟划分（**Partition**）的完整手工过程

统考（王道教材）采用挖坑法：取最左端元素为枢轴，暂存到变量 `key`，该位置成为「坑」；先从右往左找比 `key` 小的元素填到左边的坑，再从左往右找比 `key` 大的元素填到右边的坑；如此交替，直到左右指针相遇，最后把 `key` 放入相遇位置。

挖坑法的四条操作规则

设区间为 `[begin, end]`，`key = a[begin]`，`begin` 处为第一个坑。

1. `end` 从右往左移动，跳过所有 $\geq$ `key` 的元素；遇到 `< key` 的元素时，执行 `a[begin] = a[end]`（填左坑），此后 `end` 处成为新坑。
2. `begin` 从左往右移动，跳过所有 $\leq$ `key` 的元素；遇到 `> key` 的元素时，执行 `a[end] = a[begin]`（填右坑），此后 `begin` 处成为新坑。
3. 重复 1、2，直到 `begin == end`。
4. 令 `a[begin] = key`，返回 `begin`，它即枢轴的最终下标。

若在最左端挖坑，必须先移动 `end`（即「先右后左」）。反之若选最右端元素为枢轴，则必须先移动 `begin`。

样例——对序列 $\{25, 32, 12, 45, 8, 60, 17, 5\}$ 做第一趟划分：

初始 `a = [25, 32, 12, 45, 8, 60, 17, 5]`，`key = 25`，`begin = 0`，`end = 7`。下表逐步记录，`_` 表示当前的「坑」。

步骤	操作	序列状态	说明
初始	取出枢轴	`_ 32 12 45 8 60 17 5`	`key = 25`，`begin = 0`，`end = 7`，下标 0 为坑
1	`end`（7）左移找 `< 25`	`_ 32 12 45 8 60 17 5`	`a[7] = 5 < 25`，`end` 停在下标 7
2	填左坑	`5 32 12 45 8 60 17 _`	`a[0] = 5`，坑移到下标 7
3	`begin`（0）右移找 `> 25`	`5 32 12 45 8 60 17 _`	`a[0]=5` 跳过；`a[1] = 32 > 25`，停
4	填右坑	`5 _ 12 45 8 60 17 32`	`a[7] = 32`，坑移到下标 1
5	`end`（6）左移找 `< 25`	`5 _ 12 45 8 60 17 32`	`a[6] = 17 < 25`，停
6	填左坑	`5 17 12 45 8 60 _ 32`	`a[1] = 17`，坑移到下标 6
7	`begin`（1）右移找 `> 25`	`5 17 12 45 8 60 _ 32`	`a[1]=17` 跳过、`a[2]=12` 跳过；`a[3] = 45 > 25`，停
8	填右坑	`5 17 12 _ 8 60 45 32`	`a[6] = 45`，坑移到下标 3

步骤	操作	序列状态	说明
9	end（5）左移找 < 25	5 17 12 _ 8 60 45 32	a[5]=60 跳过；a[4] = 8 < 25，停
10	填左坑	5 17 12 8 _ 60 45 32	a[3] = 8，坑移到下标 4
11	begin（3）右移	5 17 12 8 _ 60 45 32	a[3]=8 跳过，begin 增到 4，与 end 相遇（begin == end == 4）
12	枢轴归位	5 17 12 8 25 60 45 32	a[4] = key = 25

最终结果：{5, 17, 12, 8, 25, 60, 45, 32}。

检验：25 左侧 {5，17，12，8} 全部小于 25；右侧 {60，45，32} 全部大于 25。枢轴 **25** 已到达它的最终位置（下标 **4**）。✓

注意第 7、9 两步：begin 从左往右扫时跳过的是小于等于枢轴的元素（12 被跳过），end 从右往左扫时跳过的是大于等于枢轴的元素（60 被跳过）。这一步的「跳过条件」写反，是手工模拟出错的最常见原因。

继续递归（下述完整过程已逐步核对）：

- 子区间 [0,3] 即 {5,17,12,8}，枢轴 5，划分后为 {5，8，12，17}（5 归位下标 0）
- 再对 {17,12,8}，枢轴 17，划分后为 {8，12，17}（17 归位下标 3）
- 再对 {8,12}，枢轴 8，划分后 {8,12}（8 归位下标 1）
- 子区间 [5,7] 即 {60,45,32}，枢轴 60，划分后为 {32，45，60}（60 归位下标 7）
- 再对 {32,45}，枢轴 32，划分后 {32,45}（32 归位下标 5）

最终结果：{5, 8, 12, 17, 25, 32, 45, 60}，排序正确。

从递归结构可以看到，每次划分都恰好确定一个元素（枢轴）的最终位置，这正是「快速排序每趟至少确定一个元素最终位置」的来源。

（三）参考代码

```
// 挖坑法划分：以 a[low] 为枢轴，返回枢轴最终下标
int Partition(int a[], int low, int high) {
    int key = a[low];                  // 暂存枢轴，low 处成为第一个坑
    while (low < high) {
        while (low < high && a[high] >= key) --high;  // 从右往左找比 key 小的
        a[low] = a[high];                             // 填左坑
        while (low < high && a[low] <= key) ++low;    // 从左往右找比 key 大的
        a[high] = a[low];                             // 填右坑
    }
    a[low] = key;                      // 枢轴归位
    return low;
}

void QuickSort(int a[], int low, int high) {
    if (low < high) {
        int pivotPos = Partition(a, low, high);
        QuickSort(a, low, pivotPos - 1);
        QuickSort(a, pivotPos + 1, high);
    }
}
```

划分顺序不能随意调换

以最左端元素为枢轴时，必须先「从右往左」再「从左往右」。

原因：`Partition` 的最后一步是 `a[low] = key`，要求相遇位置上的元素 $\leq$ `key`。若先从左往右找，在找不到比枢轴更大的元素时会以 `low == high` 跳出，此时相遇位置的元素可能 `> key`，把它填到枢轴位置就会导致划分失败。

反例：`[0, 0, 0, 0, 1]`。若先从左往右找，会得到错误结果 `[1, 0, 0, 0, 0]`。

读者可自行验证：以最右端元素为枢轴时，顺序正好反过来，必须先从左往右。

（四）时间复杂度的三种情况

设序列长度为 n，每次划分的扫描代价为 $O(n)$。

- 最好情况：每次划分都把序列均分为两个长度约 $n/2$ 的子序列。递归深度为 $\lceil \log_2 n \rceil + 1$，每层的总扫描代价为 $O(n)$，因此

$$T(n) = 2T(n/2) + O(n) = O(n \log n)$$

- 最坏情况：序列已经基本有序（正序或逆序）且固定取第一个元素为枢轴。此时每次划分只能分出一个长度为 0 的子序列和一个长度为 $n-1$ 的子序列，分治退化为「每次只减少一个元素」，递归深度达到 $n-1$，

$$T(n) = T(n-1) + O(n) = O(n^2)$$

- 平均情况：枢轴位置的期望接近中点，$T(n) = O(n \log n)$。

快速排序最坏为什么是 $O(n^2)$

关键在于划分的均衡程度取决于「枢轴在序列中的相对大小排名」：

- 枢轴恰是中位数 $\rightarrow$ 均分 $\rightarrow$ 递归树高度 $\log n \rightarrow O(n \log n)$
- 枢轴是最大或最小值 $\rightarrow$ 划分出 0 与 $n-1$ $\rightarrow$ 递归树退化成链 $\rightarrow$ 高度 $n \rightarrow O(n^2)$

而取第一个（或最后一个）元素为枢轴的策略，在序列正序或逆序时必然选出极值，所以「有序」恰恰是这种实现的最坏输入，这时快速排序退化成起泡排序级别的效率。

（五）递归深度与栈空间

快速排序是递归算法，空间开销来自递归工作栈，栈的深度等于递归树的深度：

- 平均 / 最好情况：递归树高 $\log n$，栈空间 $O(\log n)$。
- 最坏情况（正序或逆序输入 + 取首元素为枢轴）：递归深度为 $n-1$，栈空间 $O(n)$。

因此空间复杂度的标准写法是 $O(\log n)$ 到 $O(n)$，平均 $O(\log n)$、最坏 $O(n)$。排序操作本身在原数组上进行，不需要额外数组，所以是原地排序。

基准选取的三种优化：

1. 随机化选取：随机选一个元素作枢轴，与首元素交换后再划分，使最坏情况不容易被人为构造，但伪随机数序列固定的语言里仍有被针对的可能。
2. 三数取中：取首、尾、中点三个元素，把中位数换到最左端作枢轴。这样枢轴「既不太小也不太大」的概率大幅提高。
3. 尾递归优化：划分完成后，先递归较短的那个子区间，将较长的子区间用循环处理。由于较短子序列长度不超过 $\frac{n}{2}$，递归深度被限制在 $\lceil \log_2 n \rceil$ 以内，最坏空间降为 $O(\log n)$。

递归次数相关的两个易混结论

- 递归次数（递归调用的总次数）与初始序列状态有关，与「先处理左分区还是先处理右分区」无关（2010 年第 10 题答案）。分区处理顺序只改变递归调用的先后，不改变需要划分的分区集合。
- 尾递归优化后，「先处理较短的子数组」这一策略确实能降低栈深度，但这是空间优化，不是减少递归次数。答题时要看清题目问的是「递归次数」还是「栈深度」。

（六）稳定性

快速排序为什么不稳定

不稳定根源：划分过程中的交换是远距离的。特别是在挖坑法/左右指针法的最后一次操作把枢轴换到分界线上时，枢轴可能被交换到与它关键字相等的元素右侧；此外左右指针的交换也会让相等的元素跨越彼此。

反例：序列 $\{7,\ 2,\ 4_1,\ 4_2,\ 8,\ 9\}$（$4_1$ 原本在 4_2 之前），以最左端元素 7 为枢轴做一趟划分。

- `end` 从右往左找 `< 7` 的元素：9 跳过、8 跳过，`a[3] = 4_2 < 7`，停。`a[0] = 4_2` $\rightarrow \{4_2,\ 2,\ 4_1,\ 4_2,\ 8,\ 9\}$（坑移到下标 3）
- `begin` 从左往右找 `> 7` 的元素：`a[0]=4_2` 跳过、`a[1]=2` 跳过、`a[2]=4_1` 跳过，`begin` 增到 3，与 `end` 相遇
- `a[3] = key = 7` $\rightarrow \{4_2,\ 2,\ 4_1,\ 7,\ 8,\ 9\}$

排序前 4_1 在 4_2 之前，一趟划分后 4_2 落到下标 **0**、4_1 落到下标 **2**，相对次序颠倒。故快速排序不稳定。

这个例子还有一个附带结论：4_2 跨到了枢轴 7 的左侧——注意它是小于 7 的，所以划分结果本身是正确的，不稳定的原因仅仅在于两个 4 的次序被换掉了。

（七）快速排序为什么”快”

平均时间复杂度与归并、堆排序同为 $O(n\log n)$，但实际运行中快速排序通常更快：

1. 出现最差情况的概率很低（随机输入下枢轴排名极端的情况罕见）。
2. 缓存效率高：划分操作在连续的一段子数组上扫描，符合程序访问的局部性原理；而堆排序需要跳跃式访问元素，缓存命中率低。
3. 常数系数低：比较、赋值、交换的总操作数在三者中最少。

（八）不适合的存储结构

快速排序需要从两端向中间扫描、频繁按下标随机访问与交换元素，因此只适合顺序存储（顺序表）。链式存储无法 $O(1)$ 定位到表尾并向前移动，散列存储不保持线性次序，都不合适（2011 年第 10 题，答案：顺序存储）。

快速排序易错点

- 空间复杂度不能只写 $O(\log n)$ 或只写 $O(n)$，完整表述是「平均 $O(\log n)$，最坏 $O(n)$」，来自递归栈。
- 「初始序列基本有序时快排效率高」是错的，那恰恰是取首元素为枢轴时最坏的情形。正确说法是「初始序列越随机、划分越均匀，快排效率越高」。
- 快排的排序趟数与初始状态有关（递归深度随初始状态变化）。
- 快排的比较次数与初始状态有关。
- 快排不稳定，且不要求空间 $O(1)$——这两条使得「要求空间 $O(1)$ 且要求稳定」时不能选快排（2022 年

第 11 题的 III、IV 正是在说这一点）。

真题考法

快速排序是排序章考频最高的考点，至少有三种典型问法。

考法一：判断某个序列「是否可能是快速排序第 k 趟的结果」。思路：每趟划分至少让一个元素（枢轴）落到最终位置。所以第 k 趟结束后，处于最终位置的元素个数至少为 k。

- 2014 年第 11 题：序列 `2,3,5,4,6,7,9` 等四个选项中，先写出全部元素最终升序 `2,3,4,5,6,7,9`，再逐项对照有几个元素已在最终位置。C 项 `3,2,5,4,7,6,9` 只有 9 在最终位置，不足 2 个，故不可能是第 2 趟结果，答案 **C**。
- 另外常用的一条经验：若第一趟确定的枢轴在最左或最右，第二趟只能再确定 1 个位置；若第一趟枢轴不在两端，第二趟能确定 2 个位置。

考法二：给出一次划分后的结果，反推枢轴是哪个元素。思路：枢轴必须满足「左侧全部 $\le$ 它，右侧全部 $\ge$ 它」，即它已在其最终位置上。

- 2023 年第 11 题：一次划分后为 `68，11，70，23，80，77，48，81，93，88`。逐个检验：11 左侧有 68 >11，不行；70 右侧有 23、48 <70，不行；80 右侧有 77、48，不行；81 左侧的 `68,11,70,23,80,77,48` 全部小于 81，右侧只有 93、88 全部大于 81，**81** 可行，答案 **D**。

考法三：考察划分结果的块间/块内性质。 2024 年第 8 题：第一趟排序把除枢轴外的 $n-1$ 个元素划分为均非空的 P、Q 两块，则正确的是「P 和 Q 块间有序」。因为一次升序划分只保证 $\forall x \in P, \forall y \in Q$ 都有 $x \le y$，但 P、Q 各自内部仍可能无序，两块大小取决于枢轴排名不一定接近，存在重复关键字时块内仍可能有相等元素。

八、堆排序

（一）堆的定义与存储

堆（**Heap**）是满足特定条件的完全二叉树：

- 大顶堆（大根堆，**Max Heap**）：任意结点的值 $\ge$ 其左右孩子的值，堆顶是最大值。
- 小顶堆（小根堆，**Min Heap**）：任意结点的值 $\le$ 其左右孩子的值，堆顶是最小值。

由于堆是完全二叉树，用数组顺序存储最合适。

下标映射（两种约定都要会）

从下标 **1** 开始存储（王道教材惯用）：结点 i 的左孩子为 $2i$，右孩子为 $2i+1$，父结点为 $\lfloor i/2 \rfloor$；$i > \lfloor n/2 \rfloor$ 的结点都是叶子。

从下标 **0** 开始存储（C/C++ 数组惯用）：结点 i 的左孩子为 $2i+1$，右孩子为 $2i+2$，父结点为 $\lfloor (i-1)/2 \rfloor$；$i \ge \lfloor n/2 \rfloor$ 的结点都是叶子。

答题时先看清题目的下标约定，这是最容易丢分的地方。

（二）筛选（下滤）操作

筛选（**sift down** / 下滤）：把某个结点与其两个孩子中较大的那个交换，然后继续对交换后的位置做同样操作，直到该结点不小于两个孩子或成为叶子。

```
// 下标从 1 开始；对以 i 为根的子树做下滤，堆的有效长度为 n
void SiftDown(int a[], int i, int n) {
    a[0] = a[i];                    // a[0] 作暂存单元
    for (int j = 2 * i; j <= n; j = 2 * i) {
```

```
        if (j < n && a[j] < a[j + 1]) ++j;   // 取两个孩子中较大的
        if (a[0] >= a[j]) break;             // 已满足大顶堆性质
        a[i] = a[j];
        i = j;
    }
    a[i] = a[0];
}
```

（三）建堆过程（自下而上筛选，$O(n)$）

建堆：把任意一个完全二叉树调整为堆。方法是从最后一个非叶子结点开始，依次向前对每个结点做一次筛选，直到根结点。

用下标 1 存储时，最后一个非叶子结点是 $\lfloor n/2 \rfloor$。一句话概括：对最后一个非叶子结点到根结点，依次执行下滤（**SiftDown**）。

样例——把 $\{49,\ 38,\ 65,\ 97,\ 76,\ 13,\ 27,\ 49\}$ 建成大顶堆（数组下标 $1-8$，双重循环记 $a_1 = 49, \ldots, a_8 = 49$）：

步骤	对哪个结点筛选	比较与交换	数组状态（下标 **1–8**）
初始	—	未调整	49 38 65 97 76 13 27 49
1	$i=4$（$\lfloor 8/2 \rfloor$）	$97 \geq 49$，无需交换	49 38 65 97 76 13 27 49
2	$i=3$	$65 \geq 13$ 且 $65 \geq 27$，无需交换	49 38 65 97 76 13 27 49
3	$i=2$	孩子 97、76 中较大者 $97 > 38$，交换 38 与 97；换到下标 4 后 $38 \geq 49$？不成立（38<49），再交换 38 与 49	49 97 65 49 76 13 27 38
4	$i=1$	孩子 97、65 中较大者 $97 > 49$，交换；49 换到下标 2 后，孩子 49、76 中较大者 $76 > 49$，再交换	97 76 65 49 49 13 27 38

建堆完成，得到大顶堆 $\{97,\ 76,\ 65,\ 49,\ 49,\ 13,\ 27,\ 38\}$。

建堆为什么是 $O(n)$ 而不是 $O(n \log n)$

- 需要下滤的结点（非叶结点）约有 $n/2$ 个，每个结点下滤最多走 $O(\log n)$ 层。相乘得到 $O(n \log n)$ 是错误估计。
- 正确算法：按层求和，把「每层结点数 × 该层结点到叶子的距离」累加。设完美二叉树高为 h，则

$$T(h) = 2^0 h + 2^1 (h-1) + 2^2 (h-2) + \cdots + 2^{h-1} \cdot 1$$

用错位相减法：$2T(h) - T(h) = -h + 2^1 + 2^2 + \cdots + 2^h = 2^{h+1} - h - 2 = O(2^h)$。

而高度为 h 的完美二叉树结点数 $n = 2^{h+1} - 1$，所以 $T(h) = O(2^h) = O(n)$。

- 直觉：越靠近底层的结点数量越多，但它们需要下滤的距离越短；越靠近根结点的结点下滤距离长，但数量极少。加权求和后总和是 $O(n)$。

两种建堆方式对比（**2021** 年真题考点）

1. 逐个插入建堆：从空堆开始，依次把每个元素插入堆底并向上堆化。每个元素插入代价 $O(\log n)$，总代价 $O(n\log n)$。**2021 年第 11 题考察的就是这种方式，注意它得到的堆与「对整个序列自下而上建堆」的结果不一定相同。**
2. 自下而上筛选建堆：把全部元素先原样放入数组，再从最后一个非叶子结点到根依次下滤。总代价 $O(n)$，效率更高。

答题时务必先确认题目说的是「依次插入建堆」还是「对给定序列建堆」。

（四）堆排序的每一趟

堆排序（**Heap Sort**）（升序排序用大顶堆）：

1. 把待排序序列建成大顶堆，此时堆顶是最大值。
2. 把堆顶元素与当前堆的最后一个元素交换，最大值到达其最终位置，堆的有效长度减 **1**。
3. 对新的堆顶做一次下滤，恢复大顶堆性质。
4. 重复 2、3 共 $n-1$ 轮。

样例——接上建堆结果 $\{97, 76, 65, 49, 49, 13, 27, 38\}$。表中用 | 分隔「当前堆区」与「已经排好的有序区」：

趟次	交换	交换后（堆区	已排好的有序区）	下滤后恢复的堆
1	$97 \leftrightarrow 38$	`38 76 65 49 49 13 27 | 97`	`76 49 65 38 49 13 27 | 97`
2	$76 \leftrightarrow 27$	`27 49 65 38 49 13 | 76 97`	`65 49 27 38 49 13 | 76 97`
3	$65 \leftrightarrow 13$	`13 49 27 38 49 | 65 76 97`	`49 49 27 38 13 | 65 76 97`
4	$49 \leftrightarrow 13$	`13 49 27 38 | 49 65 76 97`	`49 38 27 13 | 49 65 76 97`
5	$49 \leftrightarrow 13$	`13 38 27 | 49 49 65 76 97`	`38 13 27 | 49 49 65 76 97`
6	$38 \leftrightarrow 27$	`27 13 | 38 49 49 65 76 97`	`27 13 | 38 49 49 65 76 97`
7	$27 \leftrightarrow 13$	`13 | 27 38 49 49 65 76 97`	完成

最终结果：$\{13, 27, 38, 49, 49, 65, 76, 97\}$（升序）。

注意这里两个 49 的相对次序已经无法与初始状态对应（初始时 $a_4 = 97$ 之后的 $a_8 = 49$ 与 $a_1 = 49$ 是两个不同记录），这正是不稳定性的表现之一。

（五）复杂度与特性

- 时间复杂度：建堆 $O(n)$；之后 $n-1$ 轮，每轮交换一次并做一次下滤，每次下滤 $O(\log n)$，共 $O(n\log n)$。所以

$$T(n) = O(n) + O(n\log n) = O(n\log n)$$

最好、最坏、平均都是 $O(n\log n)$，与初始状态无关（非自适应）。

- 空间复杂度：$O(1)$，原地排序。这是它相对于快速排序的最大优势之一——最坏时间仍是 $O(n\log n)$，且空间为 $O(1)$。
- 稳定性：不稳定。

（六）不适用的存储结构与 Top-K 应用

- 堆排序依赖完全二叉树的数组下标关系（$2i$、$2i+1$、$\lfloor i/2 \rfloor$）快速定位父子结点，改用链式存储后无法 $O(1)$ 定位，效率降低（2017 年第 11 题）。
- **Top-K** 问题（2022 年真题：从 n 个数中找最小的 10 个数）：
  - 要在 n 个数中找最大的 k 个 $\rightarrow$ 用小顶堆（堆顶是当前候选中最小的，新元素比堆顶大就替换）。
  - 要在 n 个数中找最小的 k 个 $\rightarrow$ 用大顶堆（堆顶是当前候选中最小的，新元素比堆顶小就替换）。
  - 时间复杂度 $O(n \log k)$，空间 $O(k)$。另外还可以用快排的划分思想做到平均 $O(n)$。

堆排序易错点

- 升序排序要用大顶堆，降序排序要用小顶堆。用「小顶堆 + 依次弹出堆顶」也能得到升序，但那样需要额外的辅助数组，不是原地堆排序。
- 建堆的时间复杂度是 $O(n)$，不是 $O(n \log n)$。但逐个插入建堆是 $O(n \log n)$。
- 堆不是二叉排序树。大顶堆只要求「父 $\geq$ 子」，不要求「左子树 < 根 < 右子树」，因此中序遍历堆得不到有序序列（2020 年第 9 题：III 错）。
- 大根堆中「次大值一定在根的下一层」是正确的：根的某个孩子一定是次大值，其它任意更深结点都受其祖先约束而不可能超过它。
- 堆的插入（堆底加入后向上调整）和删除（堆顶与堆底交换、删尾、向下调整）都是 $O(\log n)$。
- 堆排序趟数与初始状态无关，恒为 $n-1$ 趟。

真题考法

堆是 2009、2011、2015、2018、2020、2021、2022、2024 年反复考察的考点，主要三种问法。

1. 给定序列建堆，问建堆后的样子（2018 年第 11 题）：严格按「从最后一个非叶子结点到根」的顺序逐个下滤，每一步都要写出中间状态。
2. 插入/删除后重建堆，问关键字比较次数（2011 年第 11 题：大根堆 `25,13,10,12,9` 插入 18；2015 年第 10 题：小根堆删除堆顶；2024 年第 9 题：连续两次删除）：必须按下标依次比较，逐一计数，注意「比较两个孩子」也算一次比较。
3. 堆的性质判断（2020 年第 9 题）：是否完全二叉树（是）、能否顺序存储（能）、是否是二叉排序树（不是）、次大值位置（在根下一层）。

九、二路归并排序（merge sort）

（一）算法思想

二路归并排序（**Merge Sort**）基于分治思想，分两个阶段：

- 划分阶段：递归地把数组从中点处分成左右两个子数组，直到每个子数组长度为 1（长度为 1 的数组天然有序）。
- 合并阶段：把两个已经有序的相邻子数组合并成一个有序数组，逐层向上，直到整个数组有序。

核心操作 **Merge**（二路归并）：设两个有序表 `L` 与 `R`，用指针 `i`、`j` 分别指向各自的首元素，每次比较 `L[i]` 与 `R[j]`，把较小者取出放入结果表，被取出的那一侧指针后移；当某一侧取空后，把另一侧剩余元素直接复制到结果表。

2022 年第 10 题的考点

「二路归并排序操作的功能」问的是 **Merge** 这一步本身，答案是「将两个有序表合并为一个新的有序表」。

- 「将 M 划分为元素个数大致相等的两部分」描述的是分解过程，不是归并操作。
- 「将 M 分成 n 个部分，每部分仅一个元素」描述的也是分解阶段。
- 「将 M 划分为两部分，一部分的值均小于另一部分」描述的是快速排序的划分结果。

归并的递归顺序与二叉树的后序遍历相同：先递归左子数组，再递归右子数组，最后处理合并。

（二）手工过程（逐步模拟）

以 $\{49, 38, 65, 97, 76, 13, 27, 49^*\}$ 为例（$n = 8$，向上归并时取等号放左侧以保证稳定）：

第 **1** 趟（相邻两个元素两两归并，归并段长度变为 2）：

- `[49]` 与 `[38]` 归并 → `[38 49]`
- `[65]` 与 `[97]` 归并 → `[65 97]`
- `[76]` 与 `[13]` 归并 → `[13 76]`
- `[27]` 与 `[49*]` 归并 → `[27 49*]`

结果：`38 49 | 65 97 | 13 76 | 27 49*`

第 **2** 趟（每两个长度为 2 的归并段合成一个长度为 4 的归并段）：

- `[38 49]` 与 `[65 97]`：38<65 取 38；49<65 取 49；左空，接上 65、97 → `[38 49 65 97]`
- `[13 76]` 与 `[27 49*]`：13<27 取 13；76>27 取 27；76>49\ 取 *49*\；右空，接上 76 → `[13 27 49* 76]`

结果：`38 49 65 97 | 13 27 49* 76`

第 **3** 趟（两个长度为 4 的归并段合成整个序列）：

- 38 与 13 比 → 取 13
- 38 与 27 比 → 取 27
- 38 与 49* 比 → 取 38
- 49 与 49\ 比 → 相等取左侧的 *49**（保证稳定）
- 65 与 49\ 比 → 取 *49*\
- 65 与 76 比 → 取 65
- 97 与 76 比 → 取 76
- 右侧空，接上 97

结果：`13 27 38 49 49* 65 76 97`（排序完成）

共 **3** 趟，恰好等于 $\lceil \log_2 8 \rceil = 3$。

（三）参考代码

```
// 合并 a[left..mid] 与 a[mid+1..right] 为一个有序段
void Merge(int a[], int left, int mid, int right) {
    int *tmp = (int *)malloc(sizeof(int) * (right - left + 1));
    int i = left, j = mid + 1, k = 0;
    while (i <= mid && j <= right)
        if (a[i] <= a[j]) tmp[k++] = a[i++];   // 相等取左侧，保证稳定
        else              tmp[k++] = a[j++];
    while (i <= mid)   tmp[k++] = a[i++];      // 左段剩余
    while (j <= right) tmp[k++] = a[j++];      // 右段剩余
```

```
    for (k = 0, i = left; i <= right; ++i, ++k) a[i] = tmp[k];
    free(tmp);
}

void MergeSort(int a[], int left, int right) {
    if (left >= right) return;                  // 子数组长度 <= 1，终止
    int mid = left + (right - left) / 2;
    MergeSort(a, left, mid);                    // 递归左子数组
    MergeSort(a, mid + 1, right);               // 递归右子数组
    Merge(a, left, mid, right);                 // 合并
}
```

（四）归并趟数与复杂度

二路归并排序的趟数

对 n 个元素做二路归并，从长度为 1 的子序列开始，每趟把归并段长度翻倍，因此

$$\text{趟数} = \lceil \log_2 n \rceil$$

更一般地，对 r 个初始归并段做 k 路平衡归并，趟数 $S = \lceil \log_k r \rceil$。

- 时间复杂度：划分产生高度为 $\lceil \log_2 n \rceil$ 的递归树，每层合并的总操作量为 n（每个元素恰好被搬运两次：一次进 `tmp`、一次回原数组），因此

$$T(n) = 2T(n/2) + O(n) = O(n \log n)$$

最好、最坏、平均都是 $O(n \log n)$，与初始序列状态无关（非自适应）。这也意味着归并排序不因数据接近有序而变快。

- 空间复杂度：归并需要长度 n 的辅助数组，为 $O(n)$；递归栈深度 $O(\log n)$ 是低阶项，通常忽略。所以空间 $O(n)$，是非原地排序。这直接导致它不适合内存紧张的场景（2017 年第 10 题）。
- 稳定性：稳定。合并时遇到相等元素取左侧（`a[i] <= a[j]` 时取 `a[i]`），相对次序不变。

（五）链表上的归并排序

归并排序特别适合链表，且空间可以优化到 $O(1)$：

1. 链表结点的增删只需改指针，合并两个有序链表无需辅助数组。
2. 用迭代划分替代递归划分，可以省去递归栈空间。

代价是：链表上无法 $O(1)$ 随机访问中点，找中点需要快慢指针，但总时间仍是 $O(n \log n)$。

哪些排序适合链表

- 适合：直接插入排序、简单选择排序、起泡排序（顺序扫描即可）、二路归并排序（最优选择）。
- 不适合：折半插入排序（需随机访问）、希尔排序（需按下标跳转）、快速排序（需从两端向中间扫描）、堆排序（需按下标定位父子）。

归并排序易错点

- 归并排序每趟不一定能确定任何一个元素的最终位置。中途几趟只形成越来越长的有序段，段内元素在后续归并时仍可能整体后移（2012 年第 10 题）。
- 归并排序的元素移动次数与初始状态无关（非原地排序，每趟都要把 n 个元素搬进搬出辅助数组），这是很多考生的知识盲区（对比表里「移动次数是否与初始状态有关」这一列，归并填「无关」）。

- 归并排序不稳定的常见误解：只要合并时写成 `if (a[i] < a[j])`（严格小于）就会变不稳定，必须用 `<=` 取左侧。
- 一般教材给出的 `MergeSort` 是自顶向下递归版；考试里也可能出现自底向上迭代版，两者的趟数与结果相同，但迭代版第一趟是相邻两两归并（写结果时要说清是哪一种）。

真题考法

- 2017 年第 10 题：选归并而不选插入的理由，答案是「归并排序的运行效率更高」——归并的代码更长、占用空间更多，所以 I、II 都错。
- 2022 年第 10 题：二路归并操作的功能 = 将两个有序表合并为一个新的有序表。
- 2024 年第 10 题：三个有序序列 (3,5)、(7,9)、(6) 从左至右依次做二路归并。第一次归并 (3,5) 与 (7,9)：3<7 取 3，5<7 取 5，左空接上 7、9，比较 **2** 次；再与 (6) 归并：3<6 取 3，5<6 取 5，6<7 取 6，右空接上 7、9，比较 **3** 次。总比较 **5** 次，答案 **C**。

十、基数排序

（一）算法思想（非比较排序）

基数排序（**Radix Sort**）是一种非比较排序：「基」指数的位。把关键字看成由若干位（十进制即个位、十位、百位……）组成，从最低位开始，逐位对全部元素做一次「分配 + 收集」，直到最高位处理完毕。

- 分配（**distribute** / 装桶）：按当前位的值，把每个元素放进对应的桶（十进制需要 10 个桶，编号 0–9）。
- 收集（**collect**）：按桶的编号从小到大，把各桶中的元素依次连接起来，写回原序列。

每一趟结束后，序列在「已考察的位」范围内是有序的。

（二）MSD 与 LSD

方式	全称	处理顺序	特点
LSD	最低位优先（Least Significant Digit first）	从个位 → 十位 → 百位……逐位排序	实现简单、无需递归、是统考与主流实现。要求每一位的排序必须稳定。
MSD	最高位优先（Most Significant Digit first）	从百位 → 十位 → 个位逐层划分成更小的子序列，最后依次连接	可用于关键字取值范围很大（位数多）的情形，通常需要递归，实现复杂

为什么普遍采用 LSD

后一轮排序会覆盖前一轮排序的结果：若第一轮给出 $a < b$，第二轮却给出 $a > b$，则第二轮的结果取代第一轮。

由于高位的优先级高于低位，所以应该先排低位、再排高位——这样高位的排序结果覆盖低位结果时，正是我们想要的效果。反过来（MSD）必须借助「逐层划分子序列」才能避免低位被覆盖。

（三）手工过程（逐步模拟）

以序列 $\{278, 109, 63, 930, 589, 184, 505, 269, 8, 83\}$ 为例，$d = 3$（最大数是三位数），$r = 10$。

第 **1** 趟，按个位分配（把 8 看作 008、63 看作 063、83 看作 083）：

桶号（个位）	0	1	2	3	4	5	6	7	8	9
装入元素	930	—	—	63, 83	184	505	—	—	278, 8	109, 589, 269

按桶号从小到大收集：930，63，83，184，505，278，8，109，589，269

第 **2** 趟，按十位分配：

桶号（十位）	0	1	2	3	4	5	6	7	8	9
装入元素	505, 8	—	—	930	—	—	63, 269	278	83, 184	589

（8 的十位为 0；63 与 269 的十位都是 6，按第 1 趟收集后的先后顺序，63 在 269 之前——这就是稳定的价值。）

按桶号收集：505，8，109，930，63，269，278，83，184，589

第 **3** 趟，按百位分配：

桶号（百位）	0	1	2	3	4	5	6	7	8	9
装入元素	8, 63, 83	109, 184	269, 278	—	—	505, 589	—	—	—	930

按桶号收集：8，63，83，109，184，269，278，505，589，930

排序完成，结果升序且唯一。

（四）复杂度与稳定性

设最大元素的位数为 d（即排序趟数），待排序数据为 r 进制（r 为桶的个数）。

- 时间复杂度：每趟分配要扫描 n 个元素（$O(n)$）、收集要扫描 r 个桶（$O(r)$），共 d 趟，因此

$$T(n) = O(d(n + r))$$

这是线性级别的复杂度（当 d、r 为常数时即 $O(n)$），因为它不做关键字比较。

- 空间复杂度：需要 r 个桶的辅助空间，若用链队列实现则为 $O(r)$（桶的头尾指针数组）。若同时用计数数组实现分配，则为 $O(n + r)$。
- 稳定性：稳定。前提是每一位用的分配/收集（或该位上的排序）必须是稳定的。
- 移动次数：与关键字的初始排列次序无关（2015 年第 9 题）。

基数排序的稳定性是「正确性的前提」

基数排序要求对每一位的排序必须是稳定的，否则无法得到正确结果。

反例：对 {25, 13, 24} 做 LSD 基数排序。第 1 趟按个位稳定排序得 13，24，25。若第 2 趟按十位排序时不稳定，可能得到 13，25，24——结果错误。

只有当每一位的排序都稳定时，LSD 基数排序才既正确又稳定。反之，若底层排序算法不稳定，采用基数排序后未必正确也未必稳定。

基数排序易错点

- 基数排序不基于比较，所以不受 $\Omega(n \log n)$ 下界约束。
- ” 从最低位开始” 是 **LSD** 的核心。若题目给的序列已经按高位分好组，那是 MSD 的做法。
- 基数排序一趟不保证任何元素到达最终位置（它只是让某一位有序），因此不满足「每趟至少确定一个元素最终位置」。
- 基数排序的适用前提是关键字可以分解为若干位（如整数、字符串、定长编码）。对任意可比类型（如浮点数区间、复杂结构）不能直接用。
- 当关键字的取值范围很大（位数很多）时，趟数 d 增大，效率下降；此时应考虑 MSD 或改用其他算法。
- 基数排序每一趟都要把所有元素” 装桶再倒出”，所以虽然不比较关键字，但元素移动次数是 $n \cdot d$ 级别，常数并不小。

真题考法

- 2013 年第 11 题：对 `110，119，007，911，114，120，122` 做基数排序，问第 2 趟分配收集后的序列。第 **1** 趟按个位：个位 0 的 `110，120`；个位 9 的 `119`；个位 7 的 `007`；个位 1 的 `911`；个位 4 的 `114`；个位 2 的 `122`。收集得 `110,120,911,122,114,007,119`。第 **2** 趟按十位：十位 1 的 `110,911,114,119`；十位 2 的 `120,122`；十位 0 的 `007`。收集得 `007,110,911,114,119,120,122`，答案 **A**。
- 2021 年第 10 题：对 `{93,946,372,9,146,151,301,485,236,327,43,892}` 做 LSD 基数排序，问第 1 趟后 372 前后紧邻的元素。第 1 趟只看个位，个位为 2 的有 `372，892`（按原序），个位为 3 的有 `93，43`，个位为 1 的有 `151，301`。收集后个位 1 的桶最靠前：`151，301，372，892，93，43，...`。所以 372 之前紧邻 301、之后紧邻 892，答案 **C**。

十一、外部排序

（一）为什么需要外部排序

当待排序文件比内存可用容量还大时，无法一次性放入内存排序，必须借助外部存储器，这就是外部排序（**external sort**）。

（二）外部排序的两个阶段

外部排序的完整步骤

阶段一：生成初始归并段（顺段，**run**）

把大文件按内存可用容量分成若干长度为 L 的子文件（L 应小于内存可用容量）。依次把每个子文件读入内存，用适当的内部排序算法排序，把排好序的子文件（称为初始归并段或顺段）写回外存，为下一个子文件腾出内存。

若文件共 N 个记录，内存工作区可容纳 L 个记录，则等分法生成的初始归并段数量为

$$r = \left\lceil \frac{N}{L} \right\rceil$$

阶段二：多趟归并

对这些初始归并段反复做 k 路归并，直到得到整个有序文件。归并一趟后归并段数量变为原来的约 $1/k$。

例：一个含 10000 个记录的文件，内存可用容量为 1000 个记录。

- 阶段一：等分为 10 个临时文件，各含 1000 个记录，依次读入内存排序，得到 **10** 个初始归并段写回外存。
- 阶段二：对这 10 个段做两两归并（**2** 路平衡归并），共进行 4 趟，直到得到完整的有序文件。

「k 路平衡归并」的准确含义

常见误区：认为 k 路平衡归并的归并树必须是严格的 k 叉树（只有度为 0 和度为 k 的结点）。

正确理解：k 路平衡归并指的是

- 最多只能有 k 个段归并为一个；
- 每一趟归并中，若有 m 个归并段参与归并，则经过这一趟处理得到 $\lceil m/k \rceil$ 个新的归并段。

所以「8 个段做 4 路归并，第一趟得到 3 个段」是错误的（不平衡）；正确做法下第一趟应得到 $\lceil 8/4 \rceil = 2$ 个段。

（三）外部排序的时间开销与优化方向

外部排序时间开销的构成

$$\text{外部排序总时间} = \text{读写外存的时间} + \text{内部排序所需时间} + \text{内部归并所需时间}$$

其中读写外存的时间占绝对主导（外存与内存速度差几个数量级）。因此，外部排序的优化目标只有一个：减少读写外存的次数。

归并趟数公式：对 r 个初始归并段做 k 路平衡归并，归并趟数为

$$S = \lceil \log_k r \rceil$$

由该式可以看出，减少 I/O 次数有两条路：

减少外存 I/O 的两条路

路一：增加 k（增大归并路数）

代价 1：k 路归并需要 k 个输入缓冲区，内存开销增大。代价 2：每次从 k 个归并段中选一个最小元素需要 $k-1$ 次关键字比较，内部归并时间增加。→ 用「败者树」可以把每次选最小值的比较次数降到 $\lceil \log_2 k \rceil$，从而抵消这一代价。

路二：减少初始归并段数量 r（增大每个归并段的长度）

等分法下 $r = \lceil N/L \rceil$，而 L 受内存容量限制无法增大。→ 用「置换-选择排序」可以在内存容量不变的前提下生成更少、更长的初始归并段。

例题：某文件经内部排序得到 100 个初始归并段。

1. 要使多路归并三趟完成，k 至少为多少？解 $\lceil \log_k 100 \rceil \le 3$，即 $k^3 \ge 100$，$k \ge 5$（$4^3 = 64 < 100$，$5^3 = 125 \ge 100$）。答案 **5**。
2. 若操作系统要求一个程序同时可用的输入/输出文件总数不超过 13，则按多路归并至少需几趟？又若要 2 趟完成，可取的最低路数是多少？每个归并段需要一个输入文件，输出需要一个输出文件，故 $k+1 \le 13$，$k \le 12$。$\lceil \log_{12} 100 \rceil = 2$ 趟。若要 2 趟完成，$k^2 \ge 100$，$k \ge 10$。答案 **2** 趟，**10** 路。

例题（读写次数计算）：一次 I/O 的物理块大小为 150 个记录，每次可对 750 个记录进行内部排序。对含 150000 个记录的磁盘文件做 **4** 路平衡归并，需要多少次 I/O？

- 文件共占 $150000/150 = 1000$ 个物理块。
- 每次内部排序可处理 $750/150 = 5$ 个物理块，故初始归并段数 $r = 1000/5 = 200$。
- 4 路平衡归并趟数 $S = \lceil \log_4 200 \rceil = 4$。
- 生成初始归并段：读 1000 块、写 1000 块。
- 4 趟归并：每趟读 1000 块、写 1000 块。

- 总 **I/O** 次数 $= 1000 \times 2 + 1000 \times 2 \times 4 = 10000$ 次。

（四）败者树（**loser tree**）

败者树（**loser tree**）是树形选择排序的一种变形，用于 k 路归并中快速选出最小关键字。

败者树的结构与规则

- 败者树是一棵完全二叉树，k 个叶子结点分别对应 k 个归并段当前参与比较的元素；另有「冠军结点」保存最终胜者。
- 双亲结点中存放的是左右孩子比较后的失败者（败者所在归并段的段号），胜者继续向上与其它胜者比较。
- 最顶层再设一个结点（`ls[0]`）存放最终的胜者。
- 由于败者树是完全二叉树，叶子数为 k，则度为 2 的结点数为 $k-1$，不计冠军结点时总结点数为 $2k-1$；计冠军结点则为 $2k$。

5 路归并败者树的工作过程：设 5 个归并段当前的关键字依次为 $b_0 = 9,\ b_1 = 10,\ b_2 = 12,\ b_3 = 15,\ b_4 = 8$。

- 初始构造需比较 $k-1 = 4$ 次，最终胜者是段 4（关键字 8）。
- 输出 8 之后，从段 4 读入下一个关键字，设为 17，此时只需要沿从该叶子到根的路径重新比较：17 与兄弟结点（段 3 的 15）比较，15 胜；15 再与上一层胜者比较……每输出一个记录最多比较 $\lceil \log_2 k \rceil$ 次。

败者树的三个关键结论

1. 败者树的作用：进行 k 路归并时，减少选出最小关键字所需的关键字比较次数，使得增大 k 时代价 2 消失。
2. 构造 k 路归并的初始败者树，需要比较关键字 $k-1$ 次。
3. 基于构造好的败者树，每次从 k 个归并段中找出最小关键字，最多比较 $\lceil \log_2 k \rceil$ 次（最少比较次数取决于新元素来自哪个叶子，即路径长度，可能少于 $\lceil \log_2 k \rceil$）。

处理某个归并段变空的技巧：在每个归并段末尾附加一个关键字为最大值（∞）的记录。当某次选出的冠军是 ∞ 时，说明所有段都已归并完毕。

2024 年第 **11** 题

在外部排序中，利用败者树对初始为升序的归并段进行多路归并，败者树中记录「冠军」的结点保存的是什么？

答：升序归并段多路归并时，每次应输出各段当前首元素中的最小者，比较中的「冠军」是当前最小关键字。但败者树的冠军结点保存的不是关键字本身，而是最小关键字所在的归并段号——因为输出冠军后，需要据此到对应归并段读取下一个记录。答案 **D**（最小关键字所在归并段号）。

（五）置换-选择排序（生成初始归并段）

置换-选择排序（**replacement selection**）用于在内存容量不变的前提下生成更少、更长的初始归并段，从而减小 r。

为什么不能用内部排序：所有内部排序算法的前提是全部记录都在内存中，而内存容量 L 是固定的，无法通过增大 L 来加长归并段。所以要另想办法。

置换-选择排序的操作步骤

设内存工作区可容纳 L 个记录。

1. 从初始文件中输入 L 个记录到内存工作区。
2. 从工作区中选出关键字最小的记录，记为 $MINIMAX$。
3. 把 $MINIMAX$ 记录输出到归并段文件。
4. 若初始文件不为空，从初始文件中输入下一个记录到工作区（补足 L 个）。
5. 从工作区中所有关键字大于当前 $MINIMAX$ 的记录中，选出关键字最小的记录，作为新的 $MINIMAX$。
6. 重复 3–5，直到工作区中选不出新的 $MINIMAX$（即剩余记录的关键字都不大于当前 $MINIMAX$）为止，此时得到一个初始归并段。
7. 重复 2–6（对工作区中剩下的记录重新开始），直到内存工作区为空，得到全部初始归并段。

这个「不断选出不小于旧 $MINIMAX$ 的最小值」的操作，需要借助败者树高效实现。

样例：设内存工作区可容纳 **3** 个记录，文件的关键字序列为

`51, 94, 37, 92, 14, 63, 15, 99, 48, 56, 23, 60, 31, 17, 43, 8, 90, 166, 100`

逐步过程（**#** 表示该归并段结束）：

输出文件 **FO**	工作区 **WA**	输入文件 **FI**（待读）
—	`51, 94, 37`	`92, 14, 63, 15, 99, 48, 56, 23, 60, 31, 17, 43, 8, 90, 166, 100`
37	`51, 94, 92`	`14, 63, 15, 99, 48, 56, 23, 60, 31, 17, 43, 8, 90, 166, 100`
37, 51	`14, 94, 92`	`63, 15, 99, 48, 56, 23, 60, 31, 17, 43, 8, 90, 166, 100`
37, 51, 92	`14, 94, 63`	`15, 99, 48, 56, 23, 60, 31, 17, 43, 8, 90, 166, 100`
37, 51, 92, 94	`14, 15, 63`	`99, 48, 56, 23, 60, 31, 17, 43, 8, 90, 166, 100`
37, 51, 92, 94, #	`14, 15, 63`	`99, 48, 56, 23, 60, 31, 17, 43, 8, 90, 166, 100`
14	`99, 15, 63`	`48, 56, 23, 60, 31, 17, 43, 8, 90, 166, 100`
14, 15	`99, 48, 63`	`56, 23, 60, 31, 17, 43, 8, 90, 166, 100`
14, 15, 48	`99, 56, 63`	`23, 60, 31, 17, 43, 8, 90, 166, 100`
14, 15, 48, 56	`99, 23, 63`	`60, 31, 17, 43, 8, 90, 166, 100`
14, 15, 48, 56, 63	`99, 23, 60`	`31, 17, 43, 8, 90, 166, 100`
14, 15, 48, 56, 63, 99	`31, 23, 60`	`17, 43, 8, 90, 166, 100`
14, 15, 48, 56, 63, 99, #	`31, 23, 60`	`17, 43, 8, 90, 166, 100`
23	`31, 17, 60`	`43, 8, 90, 166, 100`
23, 31	`43, 17, 60`	`8, 90, 166, 100`
23, 31, 43	`8, 17, 60`	`90, 166, 100`
23, 31, 43, 60	`8, 17, 90`	`166, 100`
23, 31, 43, 60, 90	`8, 17, 166`	`100`
23, 31, 43, 60, 90, 166	`8, 17, 100`	—（已读完）
23, 31, 43, 60, 90, 166, #	`8, 17, 100`	—
8	`17, 100`	—

输出文件 **FO**	工作区 **WA**	输入文件 **FI**（待读）
8, 17	`100`	—
8, 17, 100	—（工作区空）	—
8, 17, 100, #	—	—

共得到 **4** 个初始归并段：

- RUN1 = `37，51，92，94`（长度 4）
- RUN2 = `14，15，48，56，63，99`（长度 6）
- RUN3 = `23，31，43，60，90，166`（长度 6）
- RUN4 = `8，17，100`（长度 3）

对比：若用普通内部排序等分，19/3 只能得到 **7** 个长度为 3 的归并段。置换-选择排序只用 **4** 个段（且长度不均），大幅减少了 r。

读写外存次数：每条记录初始时占一个磁盘块，全部 19 条记录要依次读入内存、再逐一写回外存，因此读磁盘 **19** 次、写磁盘 **19** 次。

置换-选择排序的两个极值结论

- 输入文件关键字呈非递减（已有序）时，得到的初始归并段最长，长度为全部记录总数 N。
- 输入文件关键字呈递减（逆序）时，得到的初始归并段最短，最短长度为缓冲区长度 L。
- 若对置换-选择排序的输出文件再做一次置换-选择排序，结果保持原样（因为已经有序）。

2023 年第 42 题

2023 年应用题深入考察了外部排序的置换-选择排序。改编练习（19 个记录、工作区 3 个记录、3 路归并）要求回答：能生成几个初始归并段、各是什么、置换-选择过程读/写磁盘次数。读 **19** 次、写 **19** 次是标准答案的要点。

需注意的还有 2016 年第 11 题（考察外部排序思想，10 TB 数据文件应用归并类外部排序）与 2019 年第 11 题（最佳归并树虚段数）。

（六）最佳归并树与哈夫曼树

经过置换-选择排序后，各初始归并段的长度不等。此时一个自然的问题：按什么顺序归并，才能使读写外存的次数最少？

归并树与 WPL 的对应关系

把归并过程画成一棵树：叶子结点表示初始归并段，权重为该段的记录数；非叶结点表示归并得到的临时文件。

若每次归并都涉及「读 + 写」，则读写外存的总次数恰好等于该归并树带权路径长度 WPL 的 **2** 倍：

$$\text{磁盘 I/O 总次数} = 2 \times WPL = 2 \times \sum_{i=1}^{m} w_i \, l_i$$

其中 w_i 是第 i 个归并段的长度，l_i 是该叶子到根的路径长度（即它参与归并的趟数）。

因此在 k 路归并中，使 **I/O** 次数最少 $\Leftrightarrow$ 使 k 叉归并树的 **WPL** 最短 $\Leftrightarrow$ 构造哈夫曼树。这样得到的归并树称为最佳归并树（**optimal merge tree**）。

哈夫曼树从二叉树推广到 k 叉树同样成立：带权路径长度最短的 k 叉树即 k 叉哈夫曼树。

样例：9 个初始归并段的长度分别为 9，30，12，18，3，17，2，6，24，采用 **3** 路平衡归并。

- 按「任意顺序、凑够 3 个就归并」的普通 3 路归并：总长 $9+30+12+18+3+17+2+6+24=121$，两次归并，读写次数 $=121\times2\times2=484$。
- 按最佳归并树（**3** 叉哈夫曼树）归并：$\mathrm{WPL}=2\times3+3\times3+6\times3+9\times2+12\times2+17\times2+18\times2+24\times2+30\times1=223$，读写次数 $=223\times2=446$。

最佳归并树把 **I/O** 次数从 **484** 降到了 **446**。

（七）虚段（补空归并段）的数目计算

为什么需要虚段

k 叉的最佳归并树一定是一棵严格的 k 叉树（树中只包含度为 k 和度为 0 的结点）。

设度为 k 的结点有 n_k 个，度为 0 的结点（叶子）有 n_0 个，总结点为 n。

- 结点数：$n=n_0+n_k$
- 边数：$k\,n_k=n-1$（树中边数 = 结点数 -1）

联立可得

$$n_0=(k-1)n_k+1 \quad\Longrightarrow\quad n_k=\frac{n_0-1}{k-1}$$

n_k 必须是整数，所以初始归并段数量 m 必须满足 $(m-1)\bmod(k-1)=0$。若不然，就需要补充若干个权值为 **0** 的「虚段」来凑成严格 k 叉树。

虚段数目的计算公式（直接套用）

对 k 路平衡归并，若初始归并段数量为 m：

- 若 $(m-1)\bmod(k-1)=0$：不需要增加虚段。
- 否则，计算 $u=(m-1)\bmod(k-1)$，需要附加

$$\text{虚段个数}=(k-1)-u$$

虚段的权值为 **0**，设置它只是为了方便构造哈夫曼树，构树完成后自动去掉即可（不参与归并、不影响 WPL）。

验算：

- $k=3$，$m=9$：$(9-1)\bmod(3-1)=8\bmod2=0$，不需虚段。（与第 6 节样例一致，可构造真正的 3 叉树。）
- $k=3$，$m=8$：$(8-1)\bmod2=1\neq0$，虚段 $=(3-1)-1=1$ 个。
- $k=12$，$m=120$：$(120-1)\bmod(12-1)=119\bmod11=9\neq0$，虚段 $=(12-1)-9=2$ 个。（2019 年第 11 题，答案 **B**（**2**）。）

2019 年第 11 题与改编应用题的完整解法

题目：外存上有 120 个初始归并段，进行 12 路归并，为实现最佳归并，需要补充的虚段个数是多少?

解答：$k=12$，$m=120$。$(m-1)\bmod(k-1)=119\bmod11=9=u\neq0$，故虚段个数 $=(k-1)-u=11-9=2$。答案 **B**。

改编应用题（19 个记录、工作区 3 个记录，3 路归并）的完整解法：

1. 置换-选择排序得到 **4** 个初始归并段，长度分别为 **4**、**6**、**6**、**3**。
2. 3 路归并：$(4-1)\bmod(3-1)=3\bmod2=1=u\neq0$，虚段个数 $=(3-1)-1=$ **1** 个。

3. 补 1 个权值为 0 的虚段后，5 个叶子的权为 4，6，6，3，0。构造 3 叉哈夫曼树：取最小的 3 个 0，3，4 合成 1 个权为 7 的结点；剩下 6，6，7 合成 1 个权为 19 的根。
4. $WPL=(0+3+4)\times 2+(6+6)\times 1=26$。最佳归并树形态不唯一，但 **WPL** 一定等于 **26**。
5. 读磁盘次数 = **WPL** = **26** 次；写磁盘次数 = **WPL** = **26** 次；磁盘 **I/O** 总次数 $=26\times 2=52$ 次。

逐趟验算：第一趟把 4、6、6 三段归并（或补虚段后的等价组合），带来 $0+3+4=7$ 次读、7 次写；第二趟把得到的 7 与另两段 6、6 归并，带来 $7+6+6=19$ 次读、19 次写。合计读 $7+19=26$ 次、写 $7+19=26$ 次，与 WPL 一致。✓

最佳归并树易错点

- 读写外存次数 = **WPL**（读）与 **WPL**（写），**I/O** 总次数 = $2\times WPL$。不要漏乘 **2**。
- 哈夫曼树只保证 **WPL** 最小，树形不唯一，但 WPL 值唯一。
- 虚段的权值必须取 **0**，且要放在哈夫曼树的最深层（因为权 0 的叶子路径最长也不增加 WPL）。
- 只有当 $(m-1)\bmod(k-1)\neq 0$ 时才需要补虚段；已满足整除条件时补虚段是错误做法。
- 计算 WPL 时不要把非叶结点（临时文件）的权漏掉或重算，只累加「叶子权 × 叶子到根的路径长度」。

十二、排序算法的分析与应用

（一）完整对比表

下表是本章最需要背诵的内容。约定：「比较次数与初始状态有关」指该算法在最好与最坏情况下比较次数不同。

算法	最好时间	平均时间	最坏时间	空间	稳定性	适合链表	适合基本有序	比较次数与初态有关
直接插入排序	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	稳定	适合	非常适合	有关
折半插入排序	$O(n\log n)$（仅比较）	$O(n^2)$	$O(n^2)$	$O(1)$	稳定	不适合	不适合（比较次数反而固定为 $O(n\log n)$）	无关
起泡排序	$O(n)$（带 flag）	$O(n^2)$	$O(n^2)$	$O(1)$	稳定	适合	较适合	带 flag 有关；不带则无关
简单选择排序	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	不稳定	适合	不适合（比较次数固定）	无关
希尔排序	$O(n)$	与增量序列有关，约 $O(n^{1.3})$	$O(n^2)$（Shell 增量）	$O(1)$	不稳定	不适合	适合	有关
快速排序	$O(n\log n)$	$O(n\log n)$	$O(n^2)$	$O(\log n)\sim O(n)$	不稳定	不适合	不适合（有序时性能最差）	有关
堆排序	$O(n\log n)$	$O(n\log n)$	$O(n\log n)$	$O(1)$	不稳定	不适合	无关（不敏感）	有关（堆化次数随状态变化）
二路归并排序	$O(n\log n)$	$O(n\log n)$	$O(n\log n)$	$O(n)$	稳定	非常适合	无关	有关（两两归并次数随状态变化）
基数排序	$O(d(n+r))$	$O(d(n+r))$	$O(d(n+r))$	$O(r)$（或 $O(n+r)$）	稳定	适合	无关	无关（非比较排序）
桶排序	$O(n+k)$	$O(n+k)$	$O(n^2)$	$O(n+k)$	取决于桶内排序	不适合	无关	无关（非比较排序）

对比表里最容易考的七条

1. 稳定的五个：直接插入、折半插入、起泡、归并、基数。
2. 不稳定的四个：希尔、简单选择、快速、堆。（2023 年第 10 题正是问这组。）
3. 最好时间是 $O(n)$ 的三个：直接插入、起泡（带 flag）、希尔（已有序时）。
4. 最好/平均/最坏完全相同的三个：简单选择、堆排序、二路归并（都是 $O(n^2)/O(n\log n)/O(n\log n)$，且与初始状态无关）。
5. 空间不是 $O(1)$ 的三个：快速 $O(\log n)\sim O(n)$、归并 $O(n)$、基数 $O(r)$。
6. 比较次数恒定的两个：简单选择 $\frac{n(n-1)}{2}$、折半插入（只与 n 有关）。
7. 移动次数最少的是简单选择排序（最多 $n-1$ 次交换）。

「每趟至少确定一个元素最终位置」的算法

满足的：简单选择排序、起泡排序、快速排序、堆排序。

不满足的：直接插入排序（前缀有序但不在最终位置）、折半插入排序、希尔排序、二路归并排序、基数排序。

记忆口诀：「选择、起泡、快排、堆排」每趟钉死一个元素（2012 年第 10 题答案：仅 I、III、IV）。

（二）什么情况下选哪个算法

算法选择的判定流程

按下面顺序逐条判断，第一个满足条件的就是答案。

1. 数据在外部存储器、内存放不下 → 多路归并的外部排序（先置换-选择生成初始归并段，再建最佳归并树归并）。
2. 要求算法稳定（多关键字排序、保留原始相对次序）→ 只有直接插入、折半插入、起泡、归并、基数可选；数据量大时选归并或基数。
3. 要求空间复杂度 $O(1)$ → 排除快排（递归栈）、归并（$O(n)$ 辅助数组）、基数（$O(r)$ 桶）。
4. 要求最坏情况下仍是 $O(n\log n)$ → 选堆排序或二路归并；优先堆排序（空间 $O(1)$）。
5. 序列基本有序 → 选直接插入排序或起泡排序（都能接近 $O(n)$）。此时绝不能选快速排序。
6. 序列完全随机、数据量大、只要求平均快、不要求稳定 → 选快速排序（常数系数最小、缓存友好）。
7. 数据元素的关键字可以分解为若干位（整数、定长编码），且取值范围不大 → 选基数排序，可达到线性时间。
8. 要求移动次数最少 → 选简单选择排序或基数排序。
9. 存储结构是链表 → 排除折半插入、希尔、快排、堆排序；优先直接插入或二路归并。
10. 数据量很小（如 $n\le 20$） → 选直接插入排序或简单选择排序，常数开销小。

真题考法

2019 年第 7 题：「选择一个排序算法时，除算法的时空效率，还需要考虑的是?」四个选项为数据规模、数据存储方式、算法稳定性、数据初始状态，四项都需要考虑，答案 D。这道题的四个因素正是上面选择流程的四个维度。

（三）算法性质反推（考试最常见的形式）

给出「某排序方法的第 k 趟结果」，反推用的是哪个算法，按下表特征定位：

观察到的现象	对应算法
前 i 个元素局部有序，后面未处理	直接插入排序
后 i 个元素是全局最大的几个、有序	起泡排序

观察到的现象	对应算法
前 i 个元素是全局最小的几个、有序	简单选择排序
有 i 个元素（或以上）已在各自最终位置上，其余无序	快速排序
序列由若干长度为 2^k 的有序段拼接而成	二路归并排序
序列由若干长度不等的有序段拼接而成，总段数减少	希尔排序（增量为 gap 时）
所有下标相差 gap 的子序列各自有序	希尔排序，增量为 gap
各元素只按某一位稳定分组，整体尚未有序	基数排序

2009 年第 10 题完整解析

序列 11，12，13，7，8，9，23，4，5 是某排序方法第二趟后的结果，问是哪种方法。

- 检查直接插入排序：第 1 趟后前 2 个元素有序，第 2 趟后前 3 个元素有序。本例前三个 11，12，13 恰好有序，后面未处理部分可以任意，符合。
- 排除起泡排序：两趟后末尾应有两个全局最大元素就位，但末尾是 4，5。
- 排除简单选择排序：两趟后开头应有两个全局最小元素就位，但开头是 11，12（全局最小是 4、5）。
- 排除二路归并排序：自底向上两趟后应形成长度约 4 的有序段，但开头 11，12，13，7 并非有序。

答案 **B**（插入排序）。

本章小结表

条目	一句话要点	必记公式 / 数字
（一）基本概念	稳定性 = 相等元素相对次序不变；内部排序比比较/移动，外部排序比 I/O 次数	基于比较的下界 $\lceil \log_2(n!) \rceil = \Omega(n \log n)$
（二）直接插入	无序区首元素插入有序区合适位置	最好 $O(n)$、最坏/平均 $O(n^2)$、空间 $O(1)$、稳定、$n-1$ 趟
（三）折半插入	只把「找位置」换成折半，其余同直接插入	比较 $O(n \log n)$、移动 $O(n^2)$、$O(1)$、稳定、不能用于链表
（四）起泡	每趟把无序区最大值冒到末尾；有序区在尾部且全局有序	最好 $O(n)$（带 flag）、最坏 $O(n^2)$、$\frac{n(n-1)}{2}$ 次比较、稳定
（五）简单选择	每趟选最小值放到无序区首位	比较恒为 $\frac{n(n-1)}{2}$、交换至多 $n-1$ 次、$O(n^2)$、不稳定
（六）希尔	缩小增量、组内直接插入	与增量序列有关（约 $O(n^{1.3})$，最坏 $O(n^2)$）、$O(1)$、不稳定
（七）快速排序	一趟划分把枢轴放到最终位置，再递归左右	平均/最好 $O(n \log n)$、最坏 $O(n^2)$、空间 $O(\log n) \sim O(n)$、不稳定、不适用链表
（八）堆排序	建大顶堆 → 堆顶与堆尾交换 → 下滤，循环 $n-1$ 轮	建堆 $O(n)$、调整 $O(\log n)$、总 $O(n \log n)$、空间 $O(1)$、不稳定、不适用链表
（九）二路归并	分治 + 合并两个有序表	趟数 $\lceil \log_2 n \rceil$、恒为 $O(n \log n)$、空间 $O(n)$、稳定、适合链表
（十）基数排序	LSD：按位分配 + 收集，逐位稳定排序	时间 $O(d(n+r))$、空间 $O(r)$、稳定、不比较关键字、要求关键字可分解

条目	一句话要点	必记公式 / 数字
（十一）外部排序	生成初始归并段 → 多路归并	趟数 $S=\lceil \log_k r \rceil$；败者树构造 $k-1$ 次、每次选最小 $\lceil \log_2 k \rceil$ 次；虚段数 $(k-1)-u$；I/O 总次数 $=2\times WPL$
（十二）分析应用	稳定选插入/归并/基数，$O(1)$ 空间选堆/插入/选择，基本有序选插入	「每趟确定一个最终位置」的是：选择、起泡、快排、堆排

易错清单

逐条核对，每一条都是统考反复设置的陷阱

1. 稳定性判断必须给出含相等关键字的具体反例。「举一个例子没乱」不等于稳定。反例必须包含两个关键字相等的记录，且排序前后它们的相对次序发生颠倒。
2. 折半插入排序是稳定的，且它的比较次数与初始状态无关。不要因为「折半查找」就认为它不稳定。
3. 折半插入排序不能用于链表，希尔排序同理（需按下标跳转）。插入、选择、起泡、归并可用于链表。
4. 直接插入排序的比较次数与初始状态有关，简单选择排序无关。简单选择的比较次数恒为 $\frac{n(n-1)}{2}$。
5. 直接插入与折半插入的移动次数相同，差别只在比较次数。
6. 起泡排序加不加 `flag` 直接改变「比较次数是否与初始状态有关」的结论。不带优化时恒为 $\frac{n(n-1)}{2}$。
7. 排序「趟数」与初始状态的关系：直接插入、折半插入、简单选择、堆排序的趟数与初始状态无关；起泡、希尔、快速、归并的趟数与初始状态有关。
8. 「每趟至少确定一个元素最终位置」只在选择、起泡、快排、堆排成立。希尔、归并、插入、基数都不成立。
9. 快速排序最坏是 $O(n^2)$，而有序序列恰恰是取首元素为枢轴时的最坏输入。「基本有序时选快排」是经典错误。
10. 快速排序的空间复杂度是 $O(\log n)\sim O(n)$，来自递归栈，不是 $O(1)$。最坏情况下递归深度可达 $n-1$。
11. 快速排序的递归次数与初始序列状态有关，与「先处理长分区还是短分区」无关。只有做了尾递归优化后，先处理短分区才能把栈深度限制在 $O(\log n)$。
12. 挖坑法以最左端元素为枢轴时，必须先「从右往左」扫。以最右端元素为枢轴时顺序相反。
13. 建堆的时间复杂度是 $O(n)$，不是 $O(n\log n)$；但逐个插入建堆是 $O(n\log n)$，两种建堆结果可能不同。
14. 升序排序用大顶堆。堆的插入、删除单个元素是 $O(\log n)$。
15. 堆不是二叉排序树，中序遍历不产生有序序列。大根堆中次大值一定在根的下一层。
16. 堆排序改用链式存储后效率降低（失去下标定位能力）。同理希尔排序。
17. 归并排序每趟不保证任何元素到达最终位置。「每趟确定一个最终位置」的算法里没有归并。
18. 归并排序的移动次数与初始状态无关（非原地，每趟都要搬 n 个元素）。合并时必须用 `<=` 取左侧才能保证稳定。
19. 归并排序的空间是 $O(n)$，不是 $O(\log n)$（$O(\log n)$ 只是递归栈）。所以「要求空间 $O(1)$」时不能选归并。
20. 基数排序是稳定排序，但前提是每一位的分配/收集必须稳定。底层不稳定会导致结果既不正确也不稳定。
21. 基数排序一趟不保证任何元素到达最终位置，它只让「已考察的位」有序。
22. **LSD** 是「从最低位开始」，不能反过来。因为高位的排序结果会覆盖低位。
23. 基数排序的时间 $O(d(n+r))$ 中 r 是「基数」（桶的个数），不是记录数。位数 d 很大时效率会下降。
24. 外部排序的优化目标是减少读写外存次数，不是减少关键字比较次数。增加 k 的代价之一是内部归并比较次数增加，靠败者树解决。
25. k 路平衡归并 $\neq$ 严格 k 叉树。判据是「每趟由 m 个段得到 $\lceil m/k \rceil$ 个段」。
26. 败者树的每个双亲结点存的是「败者」的段号，冠军结点存的是「最小关键字所在段号」，不是关键字本身。

27. 构造 k 路败者树比较 $k-1$ 次；之后每次选最小关键字最多比较 $\lceil \log_2 k \rceil$ 次。（不计冠军结点时败者树有 $2k-1$ 个结点。）
28. 置换-选择排序得到的初始归并段长度不均；输入已有序时最长（等于记录总数 N），输入逆序时最短（等于缓冲区长度 L）。
29. 最佳归并树的 **I/O** 总次数 $= 2 \times WPL$，不要漏乘 **2**。WPL 是「叶子权 × 路径长度」之和，不含非叶结点权。
30. 虚段权值必须为 **0**；仅当 $(m-1) \bmod (k-1) \neq 0$ 时才补，个数为 $(k-1)-u$（$u=(m-1) \bmod (k-1)$）。满足整除条件时补虚段是错的。
31. 简单选择排序的移动次数最多 $n-1$ 次交换（即 $3(n-1)$ 次赋值），是移动次数最少的算法之一；但它的比较次数始终是 $\frac{n(n-1)}{2}$，不会因数据接近有序而减少。
32. 外部排序和内部排序的分析指标不同：前者看磁盘 I/O 次数，后者看比较次数与移动次数。

第二部分

计算机组成原理

第八章计算机系统概述

本章考纲要求

第二部分计算机组成原理［考查目标］

1. 掌握单处理器计算机系统中主要部件的工作原理、组成结构以及相互连接方式。
2. 掌握指令集体系结构的基本知识和基本实现方法，对计算机硬件相关问题进行分析，并能够对相关部件进行设计。
3. 理解计算机系统的整机概念，能够综合运用计算机组成的基本原理和基本方法，对高级编程语言（C 语言）程序中的相关问题进行分析，具备软硬件协同分析和设计能力。

一、计算机系统概述（一）计算机系统层次结构

1. 计算机系统的基本组成
2. 计算机硬件的基本结构
3. 计算机软件和硬件的关系
4. 计算机系统的工作原理

” 存储程序” 工作方式，高级语言程序与机器语言程序之间的转换，程序和指令的执行过程。（二）计算机性能指标吞吐量、响应时间；CPU 时钟周期、主频、CPI、CPU 执行时间；MIPS、MFLOPS、GFLOPS、TFLOPS、PFLOPS、EFLOPS、ZFLOPS。

本章是《计算机组成原理》的开篇，也是全书的「总纲」。它只包含两个二级条目，但作用很关键：（一）层次结构负责建立整机概念，告诉你一台机器由什么组成、程序如何在它上面跑起来；（二）性能指标负责建立定量语言，统考中所有关于「快慢」的计算题，最终都要落回 $T_{CPU} = IC \times CPI \times T_c$ 这一个式子上。

从历年统考来看，本章的命题非常有规律：层次结构部分几乎固定出一道文字辨析选择题（冯 · 诺依曼结构、ISA 与微架构的界限、语言与翻译程序）；性能指标部分是计算选择题的必考区，还可能出现在综合应用题第一小问（要求算出主频、CPI、MIPS、执行时间）。字长部分则是近年的新热点，2020、2021 年连续以选择题和大题小问的形式考查。

本章的写法是：先把概念的定义边界说清楚（辨析题的命门），再把每个公式推导一遍并代入数字验证（计算题的命门），最后用小结表和易错清单收口。

从分值分布上看，本章单独成题一般占 2 分选择题 1 ~ 2 道，但它的公式是后面《存储器层次结构》《中央处理器》《指令系统》三章的公共工具：算 Cache 缺失对 CPI 的影响要用 $T_{CPU} = IC \times CPI \times T_c$；算流水线加速比要用「周期数 × 周期长」；算总线带宽要用「一次传输位数 ÷ 传输周期」。所以本章不能只当「背概念」的一章来读，公式必须练到能闭卷推导、闭卷验算。以下所有公式都会给出推导链条和一组带数字的验证，读者应当自己动手把每个验证算一遍。

一、计算机系统的基本组成

（一）计算机系统的构成

计算机系统（**Computer System**）由硬件（**Hardware**）和软件（**Software**）两部分组成，两者缺一不可，只有硬件的机器叫「裸机」，什么都干不了。

- 硬件：有形的物理实体，是计算机系统的物理基础。包括主机箱内的 CPU、主存、主板、总线，以及外部的显示器、键盘、鼠标、硬盘等。
- 软件：由具有各类特殊功能的信息（程序）组成，分为系统软件和应用软件。

分类	定义	典型例子
系统软件	管理整个计算机系统，使系统资源得到合理调度、高效运行	操作系统（OS）、数据库管理系统（DBMS）、语言处理程序（编译/汇编/解释）、系统服务程序、标准库
应用软件	为完成用户特定任务而编写的软件	微信、WPS、Chrome、各类仿真软件、用户自己写的 C 程序

硬件与软件的分工

硬件是物理基础，软件是灵魂。一台计算机的性能既取决于硬件的物理极限（主频、存储带宽），也取决于软件对硬件的利用效率（编译器优化、算法复杂度）。统考反复出现的一道题就是「下列哪项能缩短程序执行时间」，答案往往三项全能：提高主频（改 T_c）、优化数据通路（改 CPI）、编译优化（改 IC）——因为执行时间公式里三个因子各占一个。

（二）计算机硬件系统的五大部件

按冯 · 诺依曼结构，计算机硬件由五大部件组成，这是必须背下来的一条：

1. 运算器（**Arithmetic Unit / ALU** 主体）：对数据进行加工处理，完成算术运算和逻辑运算。
2. 控制器（**Control Unit, CU**）：指挥各部件按指令要求协调工作，是整机的指挥中心。
3. 存储器（**Memory**）：存放程序和数据。分主存（内存）和辅存（外存）。
4. 输入设备（**Input Device**）：把人们熟悉的信息形式（文字、图像）转换为计算机能识别的二进制形式，如键盘、鼠标、扫描仪。
5. 输出设备（**Output Device**）：把计算机内部的二进制结果转换成人能识别的形式，如显示器、打印机。

在这五大部件之上，统考又习惯再加两个组合概念：

组合概念	构成	说明
CPU（中央处理器）	运算器 + 控制器	有时把 CPU 内部的寄存器（通用寄存器、PC、IR、MAR、MDR、PSW）也归入 CPU
主机	CPU + 主存（内存）	注意：不含外存。硬盘是辅存，属于外设，不属于主机

「主机」的范围

「主机 = CPU + 主存」是统考的固定口径。硬盘、光驱、键盘、显示器都不属于主机，它们属于 I/O 设备。见到「主机包括 CPU、主存和硬盘」这种说法，一律判错。另外，「CPU = 运算器 + 控制器」也要连同寄存器一起理解：现代计组把寄存器组看成控制器和运算器共同使用的部件，所以「CPU = 运算器 + 控制器 + 寄存器」也不算错，但严格按考纲原文写「CPU 由运算器和控制器组成」最保险。

（三）各部件的工作关系（数据通路初貌）

把五大部件连起来看，一次程序运行就是一条不断循环的信息流：

- 取指令：PC（程序计数器）给出指令地址 → MAR → 主存 → 取出指令 → MDR → IR（指令寄存器）。
- 分析指令：IR 中的操作码送控制器译码，地址码送地址形成部件。
- 执行指令：控制器发出控制信号，运算器从寄存器或主存取数运算，结果写回寄存器或主存；若为 I/O 指令，则由 I/O 接口驱动外设。
- **PC** 更新：顺序执行时 PC+1（或 + 指令字长对应的字节数），转移时由转移地址改写 PC。

这条链路的具体时序会在《中央处理器》一章展开，这里只要记住一个结论：任何一条指令的执行，都可以拆成「取指—译码—执行」三段，其中取指阶段永远第一个动作。

在这条链路上，有四个寄存器是必须在第一章就记住位置的，后面的所有大题都默认你熟悉它们：

寄存器	全称	作用	谁给它赋值
PC	程序计数器 Program Counter	存放下一条要执行指令的地址	取指后自动增量；转移指令在执行阶段改写
IR	指令寄存器 Instruction Register	存放当前正在执行的那条指令	由 MDR 送来的指令装入
MAR	主存地址寄存器	存放将要访问的主存单元地址	由 PC 或指令的地址码装入
MDR	主存数据寄存器	存放刚从主存读出或将要写入主存的数据	由主存或 CPU 装入

有一个极易混淆的点：**PC** 里装的是「下一条」指令的地址，不是「当前」指令的地址。当前正在执行的指令已经送进 IR 了，PC 在取指阶段结束时就已自增。这正是「顺序执行」与「转移」能够统一起来的原因——顺序执行时 PC 自增即指向下一条；转移指令在执行阶段把目标地址写进 PC，下一条就跳到别处去了。

二、计算机硬件的基本结构

（一）冯 · 诺依曼结构

冯 · 诺依曼结构（**von Neumann Architecture**）由美籍匈牙利数学家冯 · 诺依曼于 1945 年提出，核心思想是把程序当作数据一样存进存储器，让机器自动连续地执行。它的要点有四条：

1. 计算机由运算器、控制器、存储器、输入设备、输出设备五大部件组成。
2. 指令和数据都用二进制数表示，形式上无差别，存放在同一个存储器中（因此存储器既存指令又存数据，称为「指令和数据同等地位」）。
3. 程序执行前，指令和数据需预先存放在存储器中，这就是「存储程序（**Stored-Program**）」原理。
4. 指令按地址顺序访问，由 PC 自动指向下一条指令，程序可自动连续执行。

除此之外，原始的冯 · 诺依曼结构以运算器为中心：输入设备、输出设备、存储器三者之间的数据传送都要经过运算器转运，运算器既是计算部件又是数据中转站，这就形成了性能瓶颈。

冯 · 诺依曼结构的四句话口诀

五大部件、二进制表示、存储程序、按地址顺序执行。加上「早期以运算器为中心」和「指令与数据形式上无差别」两条限定，就覆盖了全部辨析题的选项来源。2009-11、2019-12 都直接考的这里。

冯 · 诺依曼结构的三个高频错项

- 「**CPU** 区分指令和数据的依据是它们的存储单元不同」——错。两者在同一个存储器里，存储形式上无差别。
- 「数据都在指令中直接给出」——错。只有立即寻址的立即数才在指令中直接给出，绝大多数操作数要按地址去存储器或寄存器里取。
- 「**CPU** 区分指令和数据的依据是操作码的译码结果」——错。译码发生在取指之后，而「取出来的是指令还是数据」这件事在取指阶段就已经由指令周期的不同阶段决定了。正确答案是「指令周期的不同阶段」。

关于「CPU 如何区分指令和数据」，正确的理解是按阶段而非按内容：

阶段	从存储器取出的内容	依据
取指周期（Fetch）	一定是指令	PC 提供的地址；此刻取出的内容被当作指令处理
执行周期（Execute）	可能是数据（操作数）	由指令的地址码给出的地址；可能是读操作数，也可能是写结果

（二）现代计算机以存储器为中心

原始的冯 · 诺依曼结构以运算器为中心，I/O 与主存之间的数据传送要绕道运算器，效率低。现代计算机改为以存储器为中心：

- 主存直接与 CPU 通过存储器总线交换信息；
- I/O 设备通过 I/O 接口（I/O 总线）直接与主存交换数据，不再经过运算器；
- 运算器专心负责运算，只与主存和寄存器打交道。

这个改动的直接好处是数据传送不再占用运算器，运算器和主存可以并行工作，整机效率显著提高。所以在「以谁为中心」的选择题里：

- 问「冯 · 诺依曼原始结构以什么为中心」→ 运算器；
- 问「现代计算机以什么为中心」→ 存储器。

（三）计算机的层次结构

计算机系统是一个多层次的层级系统，从最底层的硬件到最顶层的高级语言用户，可以这样分层：

层级	名称	由谁实现	使用者看到的内容
第 0 级	数字逻辑层（硬联逻辑）	门电路	高低电平
第 1 级	微程序机器层	微程序（固化在 ROM 中）	微指令
第 2 级	传统机器层（机器语言层）	硬件直接执行	机器指令
第 3 级	操作系统层	操作系统程序	系统调用、进程、文件
第 4 级	汇编语言层	汇编程序翻译	汇编助记符
第 5 级	高级语言层	编译程序翻译	C、Java 等

层级	名称	由谁实现	使用者看到的内容
第 6 级	应用语言层	应用软件	应用问题本身的描述

请注意一个重要事实：第 **0**～**2** 级是硬件实现的，第 **3**～**6** 级是软件实现的，而第 1 级（微程序机器层）是一个特殊的「软硬交界」层：它由硬件解释执行，但解释的对象是软件写出来的微程序。

（四）指令集体系结构（ISA）与微架构的界限

指令集体系结构（**Instruction Set Architecture, ISA**）是软件和硬件之间的界面，是程序员（或编译器编写者）所能看到的计算机属性，规定「做什么」；微架构（**Microarchitecture**）是 ISA 的具体实现，规定「怎么做」。

属于 **ISA**（程序员可见）	属于微架构（实现细节，程序员不可见）
指令字格式、指令类型、寻址方式	CPU 的时钟周期、主频
数据类型、数据表示（如 IEEE 754）	加法器的进位方式（串行/先行进位）
通用寄存器个数和位数	Cache 的容量、组相联路数、替换算法
程序可访问的寄存器 PC、PSR 等	数据通路的具体结构、流水线级数
是否采用虚拟存储、存储编址方式	微指令编码方式、控制器的具体实现

ISA 辨析题的固定套路

2022-18 的题干给四项，要求选出属于 ISA 的：Ⅰ指令字格式和指令类型（属于）、Ⅱ CPU 的时钟周期（不属于）、Ⅲ通用寄存器个数和位数（属于）、Ⅳ加法器的进位方式（不属于），答案「仅Ⅰ、Ⅲ」。判别口诀：凡是「程序员写程序时必须知道、不知道就写不出正确程序」的，属于 **ISA**；凡是「换了它程序照样正确跑，只是快慢不同」的，属于微架构。时钟周期、进位方式、Cache 容量都只影响快慢，不影响正确性，所以都不属于 ISA。

三、计算机软件和硬件的关系

（一）软件与硬件的逻辑等价性

软件和硬件在逻辑功能上是等价的。这句话的含义是：同一个功能，既可以用硬件实现，也可以用软件实现。

- 用硬件实现：速度快、成本高、不灵活（功能固化后难以更改）。
- 用软件实现：速度慢、成本低、灵活（改代码就能改功能）。

举例：乘法运算可以用硬件乘法器一个周期算完，也可以用程序「移位—相加」循环实现；浮点运算可以用专用的浮点运算部件，也可以用软件模拟；甚至一条复杂指令的功能，既可以用一条硬布线指令实现，也可以编译成若干条简单指令序列实现。

由此推出两条重要延伸：

1. 软硬件之间的界限不是固定不变的，随着器件成本和技术发展，两者可以互相转化。多数功能早期由软件实现，后来移到硬件；也有功能为了灵活性从硬件移回软件。
2. 软件的功能可以用硬件来实现，硬件的功能可以用软件来模拟，所以一台机器只要 ISA 兼容，用不同软硬件划分方式实现的机器在逻辑上是等价的。

（二）固化的概念

固化（**Firmware**）是指把软件写入只读存储器（**ROM**、**PROM**、**EPROM**、**EEPROM**、**Flash**）中，使其成为硬件的一部分，这种固化了的软件称为固件（**Firmware**）。

- 固化之后，这段程序不需要再被装入主存（或只需映射到地址空间），上电即可运行，速度更快、掉电不丢失。
- 但它的本质仍然是软件：它是程序，可以被读出来分析、可以通过重新烧写来修改（对可改写 ROM 而言）。
- 微程序就属于典型的固件：它「用硬件的方式解释执行」，但内容是软件写的微指令序列。

一句话区分

「固化」只是改变了软件的存放介质和调用方式，没有改变它是软件这一事实。因此「固化后的程序就变成了硬件」是错的；「固件是软件和硬件的结合体」是可以接受的说法。

软硬件关系的三个常见错项

- 「硬件实现的性能一定优于软件实现」——错，要看具体功能和实现成本，且这句话的绝对化表述本身就有问题。
- 「软硬件在物理上是等价的」——错，等价的是逻辑功能，不是物理实体。
- 「软件可以脱离硬件独立运行」——错。

四、计算机系统的工作原理

（一）「存储程序」工作方式

存储程序（**Stored Program**）工作方式是冯 · 诺依曼结构的核心，包含两层含义：

1. 事先编制程序：把要解决的问题编成由指令构成的程序。
2. 事先存储程序：把程序（指令序列）和原始数据一起送入主存，然后启动计算机，机器就能自动、连续地逐条取指执行，直到程序结束，中间不需要人工干预。

「自动连续」之所以能实现，靠的是三件事：

- **PC** 自动增量：每取完一条指令，PC 自动指向下一条；
- 指令按地址顺序存放：程序在存储器中连续排列；
- 转移指令可改写 **PC**：实现分支和循环，使「顺序执行」变成「可控制流的执行」。

（二）高级语言程序与机器语言程序之间的转换

这是本章文字题的另一个高频区，要能准确说出每一步由谁完成、产物是什么。

*1.*三种语言与三种翻译程序

语言层次	说明	硬件能否直接执行	翻译程序
机器语言	二进制编码的指令和数据	能，唯一能直接执行的	不需要翻译

语言层次	说明	硬件能否直接执行	翻译程序
汇编语言	用助记符（如 ADD、MOV）代替二进制操作码	不能	汇编程序（**Assembler**）
高级语言	C、C++、Java 等，与机器无关	不能	编译程序或解释程序

两条容易混的边界：

- 汇编语言与机器指令几乎一一对应，一条汇编指令翻译成一条机器指令，所以汇编语言是「机器相关的」，换一种 ISA 就要重写。
- 高级语言与机器无关，同一份 C 源程序经过不同编译器可以生成 x86、ARM、MIPS 等不同 ISA 的目标代码。这也是「可移植性」的来源。
- 硬件描述语言（如 **VHDL**、**Verilog**）不是「程序」，它描述的是电路的结构与行为，用于设计和综合硬件，CPU 不能直接执行它。2015-12 专门用这个选项做干扰。
- 汇编程序（**Assembler**）：把汇编语言源程序翻译成机器语言目标程序。
- 编译程序（**Compiler**）：把高级语言源程序整体翻译成汇编语言或机器语言目标程序，先翻译后执行，生成可独立运行的目标文件。
- 解释程序（**Interpreter**）：逐句读入高级语言源程序，边翻译边执行，不生成独立的目标代码文件，执行效率通常低于编译。

汇编程序 vs 编译程序

「将高级语言源程序转换为机器目标代码文件的程序」→ 编译程序（2016-12）。「汇编程序」的输入是汇编语言源程序，不是高级语言。这两个选项几乎每年都在同一道题里出现，是最经典的干扰项。另外注意「链接程序」只做目标文件的合并与重定位，不做语言翻译。

2.C 语言程序的完整转换过程

现代统考按 GCC 的流程来考（2022-20 就是直接考这四步的顺序），完整过程是：

预处理 → 编译 → 汇编 → 链接

步骤	由谁完成	输入	输出	做了什么事
预处理	预处理器 `cpp`	`.c`	`.i`	展开 `#include` 头文件、替换 `#define` 宏、处理条件编译
编译	编译器 `cc1`	`.i`	`.s`（汇编）	词法/语法/语义分析、优化、生成汇编代码
汇编	汇编器 `as`	`.s`	`.o`（可重定位目标文件）	把汇编指令翻译成机器指令，生成符号表与重定位信息
链接	链接器 `ld`	多个`.o`＋库	可执行文件	符号解析、地址重定位、把库函数合并进来

从语言层次看则是：高级语言源程序 →（编译）→ 汇编语言程序 →（汇编）→ 机器语言目标程序 →（链接）→ 可执行目标程序。只有最后一步产物才能被硬件直接执行，且必须装入主存后才能运行。

语言转换的三类考法

- **2015-12**：硬件能直接执行的语言 → 仅机器语言程序（汇编语言、硬件描述语言都不能）。

- **2016-12**：高级语言源程序 → 机器目标代码文件的程序 → 编译程序。
- **2022-20**：高级语言源程序 → 可执行目标文件的主要过程 → 预处理 → 编译 → 汇编 → 链接。

（三）程序和指令的执行过程

*1.*程序与指令的关系

- 指令（**Instruction**）：计算机执行某种操作的命令，是硬件能理解的最小功能单位。一条指令由操作码（做什么）和地址码（对谁做）两部分组成。
- 程序（**Program**）：完成特定任务的一条指令序列。
- 因此，程序的执行过程 = 指令的逐条执行过程。

*2.*指令执行的三个基本阶段

每条指令的执行都可以划分为：

1. 取指令（**Fetch**）：按 PC 的地址从主存取指令，送入 IR，同时 PC 自动增量。
2. 分析指令（**Decode**）：对 IR 中的操作码译码，判断要做什么操作；地址码送地址形成部件算出操作数地址。
3. 执行指令（**Execute**）：按操作码发出控制信号，完成运算或数据传送，必要时写回结果并更新 PC。

用寄存器级语言写出来，就是教科书上最经典的一段。为了贴近统考习惯，这里用通用的寄存器传送记号（`(X)` 表示 X 的内容，`M(addr)` 表示主存 addr 单元的内容），用汇编语言风格标注：

```
; 取指阶段（所有指令共有）
    (PC) -> MAR
    M(MAR) -> MDR           ; 从主存取指令
    (MDR) -> IR             ; 指令送指令寄存器
    (PC) + 1 -> PC          ; 顺序执行时 PC 增量

; 分析阶段
    OP(IR) -> CU            ; 操作码送控制器译码
    Ad(IR) -> MAR           ; 地址码送 MAR

; 执行阶段（以取数指令 LOAD 为例）
    M(MAR) -> MDR           ; 从主存取操作数
    (MDR) -> ACC            ; 送入累加器
```

这段伪代码的每一步都对应一条微操作，而微操作的执行就是数据在部件之间的一次传送。要注意两点：一是「`(PC) + 1 -> PC`」中的 1 在按字节编址且指令字长多字节时，应当是「指令所占的字节数」，本章按单字长指令简化；二是不同指令的取指和分析阶段完全相同，只有执行阶段不同，这为后面学习「指令周期的划分」和「流水线」埋下伏笔。

一次程序的执行，就是「取指—分析—执行」这个循环不断重复，直到遇到停机指令或程序结束。注意取指阶段是所有指令共有的，这是「CPU 靠指令周期的不同阶段区分指令和数据」这一结论的硬件基础。

三个必须记住的执行顺序结论

1. 取指永远是第一步，任何指令都必须先取出自己。
2. **PC** 的增量发生在取指阶段（取出指令后立即 +1 或 + 指令字节数），所以转移指令在执行阶段改写 PC 才是「有效」的。

3. 操作码译码发生在分析阶段，因此「靠译码结果区分指令和数据」在时序上是说不通的。

*3.*一个完整的执行流程实例

用一个具体的小程序把整条链路走通。设主存按字节编址，指令字长为 2 字节，程序从地址 1000 开始，内容是：

```
; 地址   指令              含义
; 1000   LOAD  R1, [2000]  把主存 2000 单元的数取到 R1
; 1002   ADD   R1, [2004]  R1 加上主存 2004 单元的数
; 1004   STORE [2008], R1  把 R1 写回主存 2008 单元
; 1006   HALT              停机
```

执行的第一轮循环（对应 1000 处的 LOAD）依次发生：

步骤	**PC** 值	发生的事
取指开始	1000	MAR ← 1000，从主存读出 1000、1001 两个字节
取指结束	**1002**	指令装入 IR，**PC** 自动 **+2**
分析	1002	操作码送控制器译码为「取数」；地址码 2000 送 MAR
执行	1002	从主存 2000 读数，经 MDR 送入 R1
下一轮	1002	取第 2 条指令（ADD）

关键观察：在取指结束时，**PC** 已经从 **1000** 变成 **1002**，此时 PC 指向的是下一条指令而不是当前指令。第二轮结束后 PC 变为 1004，第三轮后变为 1006，第四轮 HALT 被取出后程序停止。

如果把 ADD 换成 JMP 1000（无条件转移），则在执行阶段会把 1000 写入 PC，于是程序进入死循环——这就是「转移指令通过改写 PC 改变执行顺序」的直观体现。

程序装入主存后才可执行

「程序存放在磁盘上」和「程序能被 CPU 执行」是两件事。可执行文件必须由操作系统装入主存（或映射到进程地址空间）之后，CPU 才能按 PC 的地址去取指令。这也是「存储程序」与「程序装入」的分工：前者是机器的工作方式，后者是操作系统的职责。

五、性能评价的两个基本维度

（一）响应时间

响应时间（**Response Time**），又称执行时间（**Execution Time**），指从作业提交开始到作业完成所经过的总时间。

它的构成是：

$$T_{总} = T_{CPU} + T_{等待}$$

其中 T_{CPU} 是 CPU 真正花在运行该程序上的时间，$T_{等待}$ 包括磁盘访问、主存访问、I/O 操作、操作系统调度开销等。

性能与响应时间成反比：

$$\text{性能} = \frac{1}{\text{执行时间}}$$

适用场景：个人电脑、银行事务处理、实时系统——关心「这件事多久办完」。考试中的「程序执行时间」「**CPU** 时间」都属于这一维度。

（二）吞吐量

吞吐量（**Throughput**），又称带宽（**Bandwidth**），指单位时间内系统能处理的请求数或任务数。

- 适用场景：数据中心、Web 服务器、多媒体流——关心「单位时间能处理多少事」。
- 注意：吞吐量与响应时间不是同向变化的。例如把 CPU 换成更快的处理器，单个任务响应时间缩短，吞吐量通常上升；但如果在原系统上增加 I/O 缓冲和多道程序度，吞吐量可能显著上升，而单个任务的响应时间反而变长（因为任务之间要竞争资源）。

响应时间与吞吐量的关系

缩短响应时间的措施不一定提高吞吐量，提高吞吐量的措施也可能拉长响应时间。典型例子：用「排队后集中批处理」能提高吞吐量，但单个请求的响应时间变长。两者是不同维度的指标，不能互相替代。

六、CPU 时钟周期、主频、CPI 与 CPU 执行时间

这是本章的核心计算区，所有公式都要能独立推导。

（一）时钟周期与主频

- 时钟周期（**Clock Cycle / Clock Tick**）T_c：CPU 中最基本、最小的时间单位。在一个时钟周期内，CPU 只能完成一个最基本的动作（如一次寄存器传送）。
- 主频（**Clock Rate / Frequency**）f：时钟周期的倒数，表示每秒钟有多少个时钟周期。

$$f = \frac{1}{T_c}, \qquad T_c = \frac{1}{f}$$

单位换算（速率类，按 10^3 进制）：

$$1\ \text{Hz} = 1\ \text{次/秒}, \quad 1\ \text{kHz} = 10^3\ \text{Hz}, \quad 1\ \text{MHz} = 10^6\ \text{Hz}, \quad 1\ \text{GHz} = 10^9\ \text{Hz}, \quad 1\ \text{THz} = 10^{12}\ \text{Hz}$$

数字验证：$f = 1\ \text{GHz} = 10^9\ \text{Hz}$，则

$$T_c = \frac{1}{10^9\ \text{Hz}} = 10^{-9}\ \text{s} = 1\ \text{ns}$$

即 **1 GHz ↔ 1 ns**。同理 $f = 2\ \text{GHz} \Rightarrow T_c = 0.5\ \text{ns}$；$f = 1.5\ \text{GHz} \Rightarrow T_c = \frac{1}{1.5\times10^9} \approx 0.667\ \text{ns}$。

主频与时钟周期的速算

记牢这一组对应关系，考场上能省下大量时间：**1 GHz ↔ 1 ns**，**2 GHz ↔ 0.5 ns**，**4 GHz ↔ 0.25 ns**，**1 MHz ↔ 1 μs**，**1 kHz ↔ 1 ms**。反过来，时钟周期 $T_c = 2\ \text{ns}$ 时，$f = 1/(2\times10^{-9}) = 500\ \text{MHz}$。

（二）CPI

CPI（**Cycles Per Instruction**）：执行一条指令平均所需的时钟周期数。

$$\text{CPI} = \frac{\text{程序执行所需的总时钟周期数}}{\text{程序包含的指令条数}IC}$$

为什么是「平均」？不同类型的指令所需周期数不同：一条寄存器间加法可能只要 1 个周期，一次访存可能要几十个周期。因此 CPI 通常是加权平均值：

$$\text{CPI} = \sum_{i=1}^{n} \left(\text{CPI}_i \times \frac{IC_i}{IC} \right)$$

其中 CPI_i 是第 i 类指令的周期数，$\frac{IC_i}{IC}$ 是第 i 类指令在程序中所占的比例，且 $\sum \frac{IC_i}{IC} = 1$。

IPC（Instructions Per Cycle）是 CPI 的倒数：$IPC = 1/CPI$。现代处理器用流水线和多发射技术，IPC 可以大于 1（例如 4 发射的理想流水线 IPC 最高为 4）。

加权平均 CPI 的两种写法

设各类指令的周期数为 C_i、条数为 IC_i，则

$$\text{CPI} = \frac{\sum_i IC_i\, C_i}{\sum_i IC_i} = \sum_i C_i \cdot \frac{IC_i}{IC}$$

分子是总周期数，分母是总指令数。两种写法等价，做题时哪种数据给全就用哪种。给「比例」用右边，给「条数」用左边。

CPI 的两个常见陷阱

- **CPI** 是平均值，不能把各类指令的 **CPI** 直接相加。必须先按比例加权。
- **CPI** 与主频无关。换一块更高主频的同架构 CPU，CPI 基本不变，只是每个周期变短，执行时间随之变短。所以「提高主频能降低 CPI」是错的。

（三）CPU 执行时间的三个等价公式

从「执行时间 = 总周期数 × 周期长」出发，可以推出统考必备的三个等价形式。

推导过程：

程序执行所需的总时钟周期数 = 指令条数 × 平均 CPI：

$$\text{总时钟周期数} = IC \times CPI$$

而执行时间 = 总时钟周期数 × 单个时钟周期长：

$$T_{CPU} = \text{总时钟周期数} \times T_c = IC \times CPI \times T_c \qquad (1)$$

把 $T_c = 1/f$ 代入，得到只含主频的形式：

$$T_{CPU} = \frac{IC \times CPI}{f} \qquad (2)$$

再把「总时钟周期数」记作 N（即 $N = IC \times CPI$），得到最简洁的形式：

$$T_{CPU} = \frac{N}{f} \qquad (3)$$

CPU 性能公式（必背）

$$T_{CPU} = IC \times CPI \times T_c = \frac{IC \times CPI}{f} = \frac{N}{f}$$

- IC：指令条数（Instruction Count），由指令集和编译器决定；
- CPI：平均每条指令的时钟周期数，由体系结构和指令集决定；
- T_c / f：时钟周期 / 主频，由硬件工艺决定。

> 三个因子分别由「编译器」「体系结构」「工艺」控制，这正是统考爱考的「下列措施能否缩短执行时间」的解题依据。

数字验证（三式互推）：$f = 1$ GHz，程序共执行 $IC = 10000$ 条指令，其中 80% 的指令平均需 1 个周期，20% 的指令平均需 10 个周期。

先算混合 CPI：

$$CPI = 0.8 \times 1 + 0.2 \times 10 = 0.8 + 2.0 = 2.8$$

再算执行时间。用式（1）：$T_c = 1$ ns，

$$T_{CPU} = 10000 \times 2.8 \times 1 \text{ ns} = 28000 \text{ ns} = 28\ \mu\text{s}$$

用式（2）验证：

$$T_{CPU} = \frac{10000 \times 2.8}{1 \times 10^9 \text{ Hz}} = \frac{28000}{10^9} \text{ s} = 2.8 \times 10^{-5} \text{ s} = 28\ \mu\text{s}$$

用式（3）验证：$N = IC \times CPI = 10000 \times 2.8 = 28000$ 个周期，

$$T_{CPU} = \frac{28000}{10^9 \text{ Hz}} = 28\ \mu\text{s}$$

三式结果一致。这就是 **2022-12** 的原题（答案：平均 CPI 为 2.8，执行时间 28 μs）。

（四）用比例法快速解题

当题目只给「变化比例」而不给绝对值时，用比例法最快。由于 $T_{CPU} = IC \times CPI \times T_c$ 是乘积关系，各因子的变化倍数直接相乘：

$$\frac{T_{新}}{T_{旧}} = \frac{IC_{新}}{IC_{旧}} \times \frac{CPI_{新}}{CPI_{旧}} \times \frac{T_{c,新}}{T_{c,旧}}$$

数字验证（**2014-12**）：程序 P 在机器 M 上执行时间 20 s，编译优化后指令数减少到原来的 70%，CPI 增加到原来的 1.2 倍，机器未变（T_c 不变）。

$$\frac{T_{新}}{T_{旧}} = 0.7 \times 1.2 \times 1 = 0.84$$

$$T_{新} = 20 \times 0.84 = 16.8 \text{ s}$$

答案 16.8 s。

（五）两台机器执行时间之比

当两台机器 **ISA** 相同、运行同一个程序时，指令条数 IC 相同，比值式可以大幅简化：

$$\frac{T_{CPU,A}}{T_{CPU,B}} = \frac{IC \times CPI_A / f_A}{IC \times CPI_B / f_B} = \frac{CPI_A}{CPI_B} \times \frac{f_B}{f_A}$$

数字验证（**2017-12**）：M1 主频 1.5 GHz、平均 CPI 为 2；M2 主频 1.2 GHz、平均 CPI 为 1。两者 ISA 相同，运行同一基准程序。

$$\frac{T_{M1}}{T_{M2}} = \frac{2}{1} \times \frac{1.2}{1.5} = 2 \times 0.8 = 1.6$$

答案 1.6。交叉验证：设 $IC = 10^7$，

$$T_{M1} = \frac{10^7 \times 2}{1.5 \times 10^9} \approx 13.33 \text{ ms}, \qquad T_{M2} = \frac{10^7 \times 1}{1.2 \times 10^9} \approx 8.33 \text{ ms}$$

$$\frac{13.33}{8.33} = 1.6 \quad \checkmark$$

比值题的分子分母

「A 比 B」与「B 比 A」互为倒数，看题时一定要圈出「谁比谁」。另外主频高的机器不一定更快：M1 主频 1.5 GHz 比 M2 的 1.2 GHz 高，但 M1 的 CPI 是 2，M2 只有 1，结果 M1 反而慢 1.6 倍。这就是「主频不是性能的唯一决定因素」的标准范例。

CPU 时间的真题分布

2010-12（缩短执行时间的措施）、2012-12（CPU 提速与 I/O 时间）、2013-12（求 MIPS）、2014-12（比例法）、2017-12（两机时间比）、2022-12（平均 CPI 与执行时间）、2023-12（指令执行速度与用户 CPU 时间）。这条线的题目几乎每年一道，且全部围绕同一个公式。

七、MIPS 与 FLOPS 类指标

（一）MIPS

MIPS（Million Instructions Per Second）：每秒执行多少百万条指令。

$$\text{MIPS} = \frac{IC}{T_{CPU} \times 10^6} = \frac{IC}{\dfrac{IC \times CPI}{f} \times 10^6} = \frac{f}{CPI \times 10^6}$$

其中 f 的单位是 Hz。若把 f 直接以 MHz 为单位，则式子简化为：

$$\text{MIPS} = \frac{f(\text{MHz})}{CPI}$$

数字验证（**2013-12**）：主频 1.2 GHz，四类指令比例与 CPI 如下表。

指令类型	所占比例	**CPI**
A	50%	2
B	20%	3

指令类型	所占比例	**CPI**
C	10%	4
D	20%	5

第一步，加权平均 CPI：

$$CPI = 0.5 \times 2 + 0.2 \times 3 + 0.1 \times 4 + 0.2 \times 5 = 1.0 + 0.6 + 0.4 + 1.0 = 3$$

第二步，求 MIPS（把 1.2 GHz 化为 1200 MHz）：

$$\text{MIPS} = \frac{1200}{3} = 400$$

答案 400。

反向验证：$f = 1.2 \times 10^9$ Hz，$CPI = 3$，则每秒执行指令数 $= \frac{1.2\times10^9}{3} = 4 \times 10^8$ 条/秒 $= 400 \times 10^6$ 条/秒 $= 400$ MIPS。✓

再来一组对照验证，把「主频相同、MIPS 不同」这件事坐实：

机器	主频	平均 **CPI**	**MIPS**	每秒指令数
甲	2 GHz	1	$2000/1 = 2000$	2×10^9
乙	2 GHz	4	$2000/4 = 500$	5×10^8
丙	4 GHz	4	$4000/4 = 1000$	1×10^9

三行数据说明了三件事：主频相同而 **CPI** 不同，**MIPS** 差 **4** 倍（甲与乙）；主频翻倍可把 **MIPS** 翻倍（乙与丙）；**MIPS** 与 **CPI** 严格成反比（乙与丙 CPI 相同，MIPS 比 = 主频比）。记住这张表，MIPS 相关的四个选项就都能秒判。

（二）**GIPS** 与指令执行速度

与 MIPS 同类，**GIPS**（**Giga Instructions Per Second**）是每秒执行的十亿条指令数：

$$\text{GIPS} = \frac{f}{CPI \times 10^9} \qquad \text{(f 以 Hz 计)}$$

数字验证（**2023-12**）：主频 1.5 GHz、$CPI = 1.2$、程序指令条数 $IC = 5 \times 10^5$。

指令执行速度：

$$\text{GIPS} = \frac{1.5 \times 10^9}{1.2 \times 10^9} = 1.25$$

用户 CPU 时间：

$$T_{CPU} = \frac{IC \times CPI}{f} = \frac{5 \times 10^5 \times 1.2}{1.5 \times 10^9} = \frac{6 \times 10^5}{1.5 \times 10^9} = 4 \times 10^{-4}\ \text{s} = 0.4\ \text{ms}$$

答案 1.25 GIPS、0.4 ms。

（三）MIPS 的三条重要局限

MIPS 不能作为跨机器的绝对性能指标

1. **MIPS** 与 **CPI** 成反比：MIPS $= f/(CPI \times 10^6)$，CPI 越大 MIPS 越小。所以「MIPS 高」只说明「每秒执行了很多条指令」，不代表「算得快」。
2. **MIPS** 不能跨指令集比较。不同 ISA 的指令功能强弱差异极大：RISC 的一条简单指令可能只做一次加法，而 CISC 的一条复杂指令可能等价于 RISC 的 5 条指令。一台 **MIPS** 为 **100** 的 **CISC** 机器可能比 **MIPS** 为 **400** 的 **RISC** 机器实际更快，因为前者一条指令顶后者多条。
3. **MIPS** 对同一机器上的不同程序也不同。程序里访存指令比例高，CPI 就大，MIPS 就低。因此 MIPS 只能用于同一机器（或同一 **ISA** 系列）上的粗略对比。

MIPS 与 CPI 的关系式

$$\text{MIPS} = \frac{f}{CPI \times 10^6}$$

MIPS 与 **CPI** 成反比，与主频成正比。这是「MIPS 和 CPI 的反比关系」这一考点的全部内容。另外注意：未经特别说明时，MIPS 用的是混合 **CPI**（该程序在该机器上的平均 CPI），而不是某类指令的 CPI。

（四）FLOPS 系列指标

FLOPS（**Floating-point Operations Per Second**）衡量的是浮点运算速度，用于科学计算、超级计算机排名，比 MIPS 更能反映数值计算能力。

指标	全称	含义	数量级
MFLOPS	Million FLOPS	每秒百万（10^6）次浮点运算	10^6
GFLOPS	Giga FLOPS	每秒十亿（10^9）次浮点运算	10^9
TFLOPS	Tera FLOPS	每秒万亿（10^{12}）次浮点运算	10^{12}
PFLOPS	Peta FLOPS	每秒千万亿（10^{15}）次浮点运算	10^{15}
EFLOPS	Exa FLOPS	每秒百亿亿（10^{18}）次浮点运算	10^{18}
ZFLOPS	Zetta FLOPS	每秒十万亿亿（10^{21}）次浮点运算	10^{21}

注意这里全部按 10^3 进制（M→G→T→P→E→Z 每级 $\times 10^3$），这是与存储容量（按 2^{10}）的根本区别。

数字验证（**2021-12**）：神威 · 太湖之光浮点运算速度为 93.0146 PFLOPS，要求每秒完成的浮点操作次数。

$$1\ \text{PFLOPS} = 10^{15}\ \text{次/秒}$$

$$93.0146 \times 10^{15}\ \text{次/秒} \approx 9.3 \times 10^{16}\ \text{次/秒}$$

四个选项换算过来看：A 是 9.3×10^{13}（太小）、B 是 9.3×10^{15}（太小，差 10 倍）、C「9.3 千万亿次」$= 9.3 \times 10^7 \times 10^8$…按中文数量级，「亿亿」$= 10^8 \times 10^8 = 10^{16}$，所以 9.3 亿亿次 $= 9.3 \times 10^{16}$，与计算结果一致。答案 D。

中文数量级对照（考「亿亿次」这种说法时用）

- 万 $=10^4$，亿 $=10^8$，万亿 $=10^{12}$，千万亿 $=10^7\times10^8=10^{15}$（即 P），亿亿 $=10^8\times10^8=10^{16}$，百亿亿 $=10^2\times10^{16}=10^{18}$（即 E）。
- 判断技巧：把选项换算成 10 的幂，与 PFLOPS $=10^{15}$ 的数量级对齐即可，不必纠结中文措辞。

（五）速率进制 **vs** 容量进制（必考辨析）

这是本章最容易被「差 2.4%」坑到的点，必须在读题时第一秒分辨清楚。

场合	进制	换算
存储容量、编址范围、Cache 容量	按 2 的幂	1 KB $=2^{10}$ B $=1024$ B， 1 MB $=2^{20}$ B， 1 GB $=2^{30}$ B， 1 TB $=2^{40}$ B
速率、频率（主频、MIPS、FLOPS、总线带宽、传输速率）	按 10 的幂	1 kHz $=10^3$ Hz， 1 MHz $=10^6$ Hz， 1 GHz $=10^9$ Hz； 1 MB/s $=10^6$ B/s

量化对比：$2^{10}=1024$，而 $10^3=1000$，两者相差

$$\frac{1024-1000}{1000}=2.4\%$$

$2^{20}=1048576$ 与 $10^6=1000000$ 相差约 4.86%。单个单位差别不大，但跨多级相乘就会放大，命题人正是利用这一点设置「差不多但不相等」的干扰项。

主频换算绝不能用 **1024**

主频 1 GHz 就是 10^9 Hz，不是 $1024^3=2^{30}$ Hz。求时钟周期时用 $T_c=1/f$，1 GHz 对应 1 ns 是精确值，不能算成 0.931 ns（那是误按 2^{30} 的结果）。反过来，说「4 GB 内存」时是 4×2^{30} B $=2^{32}$ B，不是 4×10^9 B。

具体差异有多大？把同一串数字用两种进制算一遍，差距就很直观了：

单位	按 10 的幂	按 2 的幂	相差
K / k	$10^3=1000$	$2^{10}=1024$	+2.4%
M	$10^6=1000000$	$2^{20}=1048576$	+4.86%
G	10^9	$2^{30}=1073741824$	+7.37%
T	10^{12}	$2^{40}\approx1.0995\times10^{12}$	+9.95%

差别的规律是：每升一级，误差大约再加 **2.4～2.6** 个百分点。所以「512 GB 的固态硬盘」按厂商进制只有 512×10^9 B，而操作系统按 2^{30} 显示时会变成约 476.8 GB——这不是硬盘缩水，而是进制口径不同。

代入验证一遍：512×10^9 B $\div\,2^{30}=512\times10^9\div1073741824\approx476.84$。可见差距约 6.9%，与上表中 G 级的 7.37% 同量级。这也解释了为什么 8 GB 的 U 盘在电脑上只显示约 7.45 GB。

这个差异在带宽题里最致命。举一个必须分清的对照：

> 某系统以 1 MB/s 的速率把数据写入主存，同时主存中有 1 MB 的数据待读取。问各是多少字节?

- 「1 MB/s」是速率，按 10^3 进制：1 MB/s $= 10^6$ B/s，即每秒写入 1000000 字节。
- 「1 MB 数据」是容量，按 2^{10} 进制：1 MB $= 2^{20}$ B $= 1048576$ 字节。

两者写成同一个数字都是「1 MB」，实际相差 48576 字节（4.86%）。如果题目要求算「传完这 1 **MB** 数据需要多少秒」，正确做法是：

$$t = \frac{2^{20}\ \text{B}}{10^6\ \text{B/s}} = 1.048576\ \text{s}$$

若误把分子也按 10^6 算，就会得到整整齐齐的 1 s——那个「太整齐」的答案恰恰是错项的标志。同理，凡在一条式子里同时出现容量和速率，就要分别用各自进制换算后再相除，绝不能统一按一个进制处理。

判题时的操作准则可以归纳为一句话：

见到单位先问一句「这是容量还是速率」

容量、地址空间、**Cache** 大小、存储器芯片规格 → 用 2^{10} 系列（**1K=1024**，**1M=2^20**）；频率、主频、**MIPS**、**FLOPS**、带宽、传输速率 → 用 10^3 系列（**1k=1000**，**1M=10^6**）。例外说明：现代存储厂商标注硬盘容量时常用 10^3 进制，而统考里的「主存容量」一律按 2^{10} 进制理解；若题目明确写了换算关系，则以题目为准。

八、各种字长总结

字长是「硬件规格与性能的接口」，涉及机器字长、指令字长、存储字长、数据字长、地址字长等多个层次，2020、2021 年连续考查，属于必须彻底理清的一组概念。

（一）机器字长

机器字长（**Machine Word Length**）：**CPU** 在一次整数（定点整数）运算中能够处理的二进制数据的位数。

- 通常等于 **CPU** 中定点运算数据通路的宽度，也就是 **ALU** 的宽度和通用寄存器的位数。
- 32 位机器 / 64 位机器，说的就是机器字长。
- 机器字长直接影响数的表示范围和运算精度：字长越长，表示范围越大、精度越高。
- 机器字长通常取字节的整数倍（8、16、32、64 位）。

机器字长 ⇔ ALU 宽度 ⇔ 通用寄存器位数

机器字长 = **ALU** 宽度 = 通用寄存器位数，这三者在统考口径下是同一个数。但要注意它不等于指令寄存器（IR）的位数（那是指令字长），也不等于浮点寄存器的位数（浮点寄存器通常更宽，以保证精度）。

关于「字」这个概念的一个提醒：「字」的宽度不一定等于机器字长。字是某个系列机中固定的信息宽度计量单位。例如在 80x86 系列中，一个字的宽度是 16 位，所以 32 位是双字、64 位是四字；而在 IBM 303X 系列中，一个字是 32 位，16 位为半字、64 位为双字。统考默认按「字 = 机器字长」理解，除非题目另有说明。

（二）指令字长

指令字长（**Instruction Word Length**）：一条指令中所包含的二进制代码的位数。

- 取决于操作码的长度和操作数地址的个数及各地址的长度。
- 定长指令字机器：所有指令位数相同（目前定长指令字大多是 32 位）。
- 不定长指令字机器：指令有长有短，但每条指令的长度一般都是 **8** 的整数倍（因为存储器按字节编址，便于按字节存取）。

指令字长决定 **IR**（指令寄存器）的位数：若机器中指令字长最大为 32 位，则 IR 至少需要 32 位。

按指令长度与存储字长的关系，可把指令分为：

叫法	指令字长与存储字长的关系	取指需要访存次数
半字长指令	指令字长 = 存储字长 ÷ 2	1 次（取出后有半个单元冗余）
单字长指令	指令字长 = 存储字长	1 次
双字长指令	指令字长 = 存储字长 × 2	2 次

（三）存储字长与存储容量

存储字长（**Storage Word Length**）：一个存储单元所存储的二进制代码的位数。

- 一个存储单元对应一个地址。要读写某个存储单元的内容，必须给出它的地址。
- 主存容量 = 存储单元个数 × 存储字长，例如 1K × 8 位表示 1024 个单元，每个单元 8 位。

（四）编址单位与地址字长

编址单位（**Addressing Unit**）：一个地址所对应的存储单元的宽度。现代计算机绝大多数按字节编址，即编址单位为 8 位，每一个字节有一个地址。

这里有一个必须分清的区别，它决定了后面几章的一大类计算题：

概念	定义	由谁决定	典型值
编址单位	一个地址对应的存储单元宽度	指令集体系结构（ISA）规定	8 位（按字节编址）
存储字长	一个存储单元存放的二进制位数	存储器设计	8/16/32/64 位
存取宽度	一次从主存模块同时读写的位数	存储模块与总线匹配设计	64 位（8 片 ×8 位并行）
传输宽度	总线一次能传送的位数	总线设计（数据线条数）	= MDR 位数

按字节编址时，编址单位 = **8** 位，小于等于存储字长。这就带来两个后果：

1. 一个数据可能占多个存储单元。例如一个 32 位整数占 4 个字节、有 4 个地址；一个 64 位 `double` 占 8 个字节、有 8 个地址。
2. 一次访存取到的信息量取决于数据线宽度，而不是数据本身的长度。这正是一道经典统考题的考点：某机主存按字节编址，由 4 个 64M × 8 位 DRAM 芯片交叉编址构成，存储器总线宽 32 位，主存每次最多读写 32 位。`double` 型变量 x 的地址为 `804001AH`，则读取 x 需要几个存储周期？推导：`double` 占 8 B = 64 位，因此占 8 个连续地址单元，即 `804001AH` ~ `8040021H`。总线一次传 32 位 = 4 B，所以要传 $\lceil 8/4 \rceil = 2$ 次。但地址 `804001AH` 的低两位是 $10_2 = 2$，说明 x 不

是从 4 字节对齐的边界开始的，8 个字节跨在 3 个 4 字节行上（第 1 行只用到 2 个字节，第 2 行 4 个，第 3 行 2 个），因此需要 **3** 个存储周期。这个例子说明的是：编址单位决定「有多少个地址」，存取宽度（数据线宽度）决定「一次能搬多少」，两者不是同一个数。

地址字长 = **MAR** 的位数 = 地址线的根数，它决定了可寻址的空间大小：

$$\text{可寻址单元数} = 2^n \quad (n = \text{MAR 位数})$$

若按字节编址，则

$$\text{可寻址空间} = 2^n \text{ 字节}$$

寻址范围的三个连等式

$$\text{寻址范围} = 2^{\text{MAR 位数}} \text{ 个存储单元} = 2^{\text{地址线条数}} \text{ 个单元}$$

若按字节编址，则每个单元 = 1 B，于是

$$2^{\text{地址线条数}} \text{ B}$$

三个量（**MAR** 位数、地址线根数、地址字长）在统考里永远是同一个数，题目给哪个都能推出寻址范围。反之，给出寻址范围也能反推地址线根数：主存 4 GB 按字节编址，地址线需 $\log_2(4 \times 2^{30}) = 32$ 根。

（五）MAR、MDR 与地址线、数据线

部件	全称	位数由谁决定	说明
MAR	存储器地址寄存器	地址线条数	位数 = 地址线宽度，决定可寻址单元数 2^n
MDR	存储器数据寄存器	数据线条数	位数 = 数据线宽度，也就是一次访存能读写的数据位数

MDR 位数 ≠ 存储字长（这是最常错的等式）

MDR 的位数等于数据总线的宽度，与存储字长不一定相等。当存储字长大于数据线宽度时，一次访存需要分多个存储周期完成，MDR 只装得下一个「传输单位」。例如存储字长 32 位、数据线 16 位时，读一个存储单元要分 2 次传送。反过来，MAR 的位数一定等于地址线位数，这一点没有例外。

数字验证（**2021** 年综合题第一问）：某计算机 M 字长 16 位，按字节编址，CPU 与主存之间的地址线 20 位、数据线 8 位，采用 16 位定长指令字。求 ALU 宽度、可寻址主存空间大小、IR / MAR / MDR 的位数。

- **ALU** 宽度 = 机器字长 = **16** 位（ALU 一次处理一个机器字长）。
- 可寻址主存空间：地址线 20 位 ⇒ 单元数 2^{20}；按字节编址 ⇒ 每个单元 1 B，所以容量 = 2^{20} B = 1 MB。
- **IR** 位数 = 指令字长 = **16** 位（指令字长决定 IR 位数）。
- **MAR** 位数 = 地址线位数 = **20** 位。
- **MDR** 位数 = 数据线位数 = **8** 位（不是 16 位，因为数据线只有 8 位）。

（六）数据字长

数据字长（**Data Word Length**）：存储、传输或处理一个数据时所用的二进制位数。

它和前面几个字长的关系是「具体值与一般值」的关系：

- 数据字长是具体的：说的是某一个数据占多少位，由数据类型决定。C 语言中 `char` 是 1 B（8 位），`short` 是 2 B（16 位），`int` 通常 4 B（32 位），`double` 是 8 B（64 位）。同一个程序里，不同数据的数据字长可以不同。
- 机器字长是一般的：说的是 CPU 一次定点整数运算能处理的位数，同一台机器上是一个固定值。
- 当数据类型宽度不超过机器字长时，一次运算即可处理完；当数据类型宽度大于机器字长时，需要分多次运算（例如 32 位机上做 64 位整数加法，要分低位、高位两次并处理进位）。

浮点数据的存储宽度可以超过机器字长，这正是「浮点寄存器宽度不等于机器字长」的原因（2020-12 的Ⅳ项）。所以数据字长这一概念的价值在于：它提醒你「数据的宽度」和「机器的宽度」是两件事，一个 64 位浮点数在 32 位机上照样能存、能算，只是要拆开来处理。

数据类型	C 语言典型宽度	数据字数（按字节编址占几个单元）
`char`	1 B	1 个字节单元
`short`	2 B	2 个字节单元
`int`	4 B	4 个字节单元
`double`	8 B	8 个字节单元

数据字长与机器字长的关系

数据字长可以小于、等于或大于机器字长。小于或等于时一次处理完；大于时需要拆成多段处理。统考里凡是问「某类型变量占几个存储单元」「读它需要几次访存」，考的都是数据字长与编址单位、存取宽度的配合关系。

（七）地址字长

地址字长（**Address Word Length**）：一个存储地址所占的二进制位数，也就是 MAR 的位数和地址线的根数。

- 它决定可寻址空间：n 位地址字长对应 2^n 个可寻址单元。
- 它与机器字长不一定相等。例如 32 位机器（机器字长 32 位）可以只配 20 位地址线，寻址 1 MB；也可以配 32 位地址线，寻址 4 GB。
- 统考口径：题中说「地址线 20 位」「MAR 20 位」「地址字长 20 位」都是同一件事，直接取 2^{20}。

机器字长 $\neq$ 地址字长

「32 位计算机」说的是机器字长 **32** 位（数据通路宽度），不能由此推断地址线也是 32 位。反过来，题目给了地址线根数就要用它算寻址范围，不要拿机器字长去算。2021 年的那道综合题正是「机器字长 16 位、地址线 20 位」的组合，两者不等，必须各用各的。

（八）字长之间的关系梳理

把这几个字长的约束关系理清，可以覆盖绝大多数选择题：

关系对	约束
存储字长 ↔ 机器字长	机器字长 ≥ 存储字长，通常是存储字长的整数倍（1、2、4 倍）。最常见是 1 倍（相等），这样一次访存即可取到能一次处理的数据
存储字长 ↔ 指令字长	指令字长可以是存储字长的 **0.5** 倍、**1** 倍、**2** 倍等（半字长/单字长/双字长指令）
指令字长 ↔ 机器字长	互不影响，没有任何约束关系。两者只是恰巧各自与存储字长成倍数关系
编址单位 ↔ 存储字长	按字节编址时编址单位为 8 位；存储字长可以等于或大于编址单位
MAR ↔ 地址线	位数相等
MDR ↔ 数据线	位数相等，与存储字长不一定相等
IR ↔ 指令字长	IR 位数 ≥ 最大指令字长

为什么「指令字长与机器字长无关」

机器字长强调「参与 **ALU** 运算的操作数位数」，而指令字长是「指令本身的位数」。有的指令没有参与 ALU 运算的操作数，甚至没有显式操作数（如空操作、停机），所以两者无法互相约束。记住一句话：机器字长看运算，指令字长看编码，两者只是各自与存储字长有倍数关系。

2020-12 的核心判据

题目问「位数一定与机器字长相同的部件」：Ⅰ ALU（是）、Ⅱ指令寄存器（不是，看指令字长）、Ⅲ通用寄存器（是）、Ⅳ浮点寄存器（不是，通常更宽），答案「仅Ⅰ、Ⅲ」。这个判据就是「谁直接参与定点整数运算，谁就等于机器字长」。

字长辨析的三个高频错项

1. 「**MDR** 的位数一定等于存储字长」——错，等于数据线宽度。
2. 「指令字长一定等于机器字长」——错，两者无约束关系；在变长指令系统（如 x86）中更是明显不等。
3. 「机器字长一定等于存储字长」——错，机器字长 ≥ 存储字长，只在最常见情况下相等。

本章小结表

考纲条目	核心内容	必须记住的结论
计算机系统的基本组成	硬件 + 软件；软件分系统软件与应用软件	只有硬件的机器是「裸机」，什么都干不了
硬件五大部件	运算器、控制器、存储器、输入设备、输出设备	**CPU** = 运算器 + 控制器；主机 = **CPU** + 主存（不含外存）
冯 · 诺依曼结构要点	五大部件；指令与数据同为二进制、同等地位、同存一个存储器；存储程序；按地址顺序执行	原始结构以运算器为中心；现代计算机以存储器为中心
CPU 区分指令与数据的依据	取指周期取出的是指令，执行周期取出的是数据	依据是指令周期的不同阶段，不是译码结果、不是存储单元
ISA 与微架构	ISA 规定「做什么」（程序员可见）；微架构规定「怎么做」	指令格式、通用寄存器属 ISA；时钟周期、进位方式、Cache 容量属微架构
软件与硬件的关系	逻辑功能等价；可互相转化；固化	固化只是换了存放介质，固件本质仍是软件

考纲条目	核心内容	必须记住的结论
「存储程序」工作方式	程序与数据预先存入主存，自动连续执行	靠 PC 自动增量 + 按地址顺序存放 + 转移指令改写 PC
语言与翻译程序	机器语言（硬件唯一直接执行）；汇编程序翻译汇编语言；编译程序整体翻译高级语言；解释程序逐句边翻译边执行	高级语言 → 机器目标代码的是编译程序
C 程序转换流程	预处理 → 编译 → 汇编 → 链接	输出依次为`.i` → `.s` → `.o` → 可执行文件
指令执行过程	取指 → 分析（译码）→ 执行	取指永远是第一步；PC 增量发生在取指阶段
响应时间	作业提交到完成的总时间 $T_{总} = T_{CPU} + T_{等待}$；性能 $\propto 1/$ 执行时间	关心「多久办完」；与吞吐量不同向变化
吞吐量	单位时间处理的请求或任务数	关心「单位时间办多少事」；提高吞吐量可能拉长响应时间
时钟周期与主频	$f = 1/T_c$；速率按 10^3 进制	1 GHz ↔ 1 ns，绝不能用 **1024** 换算主频
CPI	$CPI = \frac{总时钟周期数}{IC} = \sum_i CPI_i \times \frac{IC_i}{IC}$	CPI 是加权平均值；与主频无关
CPU 执行时间	$T_{CPU} = IC \times CPI \times T_c = \frac{IC \times CPI}{f} = \frac{N}{f}$	三式等价；IC、CPI、f 分别由编译器、体系结构、工艺决定
比例法	$\frac{T_{新}}{T_{旧}} = \frac{IC_{新}}{IC_{旧}} \times \frac{CPI_{新}}{CPI_{旧}} \times \frac{T_{c,新}}{T_{c,旧}}$	乘积关系，各因子倍数直接相乘
两机时间比	IC 相同时 $\frac{T_A}{T_B} = \frac{CPI_A}{CPI_B} \times \frac{f_B}{f_A}$	主频高不一定更快（2017-12 的 M1 反而慢 1.6 倍）
MIPS	$\text{MIPS} = \frac{f}{CPI \times 10^6} = \frac{f(\text{MHz})}{CPI}$	与 CPI 成反比；不能跨指令集比较
MIPS 局限	指令功能强弱不同；同一机器不同程序 MIPS 不同	只适合同 ISA 系列粗略对比
FLOPS 系列	M/G/T/P/E/Z 每级 $\times 10^3$	按 10 的幂，与存储容量的 2^{10} 进制不同
进制辨析	容量按 2^{10}；速率按 10^3	2^{10} 与 10^3 相差 2.4%，跨级会放大
机器字长	CPU 一次整数运算能处理的位数 = ALU 宽度 = 通用寄存器位数	决定数的表示范围与运算精度；不等于 IR 位数
指令字长	一条指令的位数，由操作码与地址码决定	决定 IR 位数；与机器字长无必然联系
存储字长	一个存储单元的位数	主存容量 = 单元数 × 存储字长
编址单位与地址字长	一个地址对应的单元宽度；地址字长 = MAR 位数	按字节编址时 1 地址 = 1 B；可寻址单元数 $= 2^{\text{地址字长}}$
数据字长	一个数据所占的位数，由数据类型决定（`char` 8 位、`int` 32 位、`double` 64 位）	可小于、等于或大于机器字长；大于时需分段处理
地址字长与机器字长	地址字长 = 地址线根数 = MAR 位数	两者不必相等；32 位机器可以只配 20 位地址线
MAR / MDR	MAR 位数 = 地址线数；MDR 位数 = 数据线数	**MDR** $\neq$ 存储字长（易错点第一名）
字长间关系	机器字长 $\geq$ 存储字长（整数倍）；指令字长是存储字长的 0.5/1/2 倍；指令字长与机器字长无关	记住三句：机器字长看运算，指令字长看编码，MDR 看数据线

易错清单

1. **CPU** 区分指令和数据的依据是「指令周期的不同阶段」，不是「操作码的译码结果」，也不是「所在的存储单元」。2009-11 的正确答案是 C。
2. 「主机 = **CPU** + 主存」，硬盘等外存属于 I/O 设备，不属于主机。见到「主机包括硬盘」直接判错。
3. 「**CPU** = 运算器 + 控制器」，这是考纲原文口径。寄存器可以理解成两者共用，但不要漏掉控制器。
4. 原始冯 · 诺依曼结构以运算器为中心，现代计算机以存储器为中心。两个「中心」不要互换。
5. 「数据都在指令中直接给出」是错的，只有立即寻址才是；数据可以来自寄存器、主存、堆栈等。2019-12 的错项正是这一句。
6. **ISA** 与微架构的分界看「换掉它程序是否仍然正确」：影响正确性的属 ISA（指令格式、寄存器个数与位数），只影响快慢的属微架构（时钟周期、进位方式、Cache 容量）。2022-18 答案「仅 Ⅰ、Ⅲ」。
7. 固化不等于变成硬件。固件写在 ROM 里，本质仍是软件，只是存放介质和调用方式变了。
8. 软硬件等价的是「逻辑功能」，不是物理实体。
9. 高级语言 $\rightarrow$ 机器目标代码文件用编译程序；汇编程序的输入是汇编语言，不是高级语言；解释程序不生成独立目标文件。
10. **C** 程序完整流程的顺序是「预处理 $\rightarrow$ 编译 $\rightarrow$ 汇编 $\rightarrow$ 链接」，2022-20 就是直接考这个顺序，不要把编译与汇编颠倒。
11. 取指阶段是所有指令共有的，且 **PC** 增量发生在取指阶段。因此「靠译码结果区分指令与数据」在时序上说不通。
12. 主频换算一律按 10^3 进制：1 GHz $= 10^9$ Hz，对应 $T_c = 1$ ns 是精确值。不要用 **1024** 去换算频率。
13. T_c 与 f 互为倒数，求执行时间时两式可互换验证；算完用另一种形式代回检查，能避免位数错误（如把 μs 写成 ms）。
14. **CPI** 必须按比例加权，不能把各类指令的 CPI 相加或取算术平均。$0.8 \times 1 + 0.2 \times 10 = 2.8$ 而不是 $(1 + 10)/2 = 5.5$。
15. **CPI** 与主频无关，同架构下提高主频不会降低 CPI，只会缩短每个周期。
16. **MIPS** 与 **CPI** 成反比：MIPS $= f/(CPI \times 10^6)$。CPI 越大 MIPS 越小，两者不可能同增。
17. **MIPS** 不能跨指令集比较。一条 CISC 复杂指令可能顶多条 RISC 指令，所以 MIPS 高的机器不一定更快。
18. **FLOPS** 系列按 10 的幂（M=10^6、G=10^9、T=10^{12}、P=10^{15}、E=10^{18}、Z=10^{21}），与存储容量的 2^{10} 进制完全不同。2^{10} 与 10^3 相差 2.4%，选「差不多」的选项时要留神。
19. 中文数量级要会换算：「亿亿」$= 10^{16}$，「千万亿」$= 10^{15}$（即 P），「百亿亿」$= 10^{18}$（即 E）。2021-12 的四个选项就是靠数量级互相干扰的。
20. **MAR** 位数 = 地址线位数；**MDR** 位数 = 数据线位数。**MDR** 的位数不等于存储字长——这是 2021 年大题反复挖的坑，存储字长 32 位而数据线 16 位时，MDR 只有 16 位，一次访存要传两次。
21. 指令字长决定 **IR** 位数，机器字长决定 **ALU** 宽度和通用寄存器位数，两者毫无约束关系。2020-12 答案「仅 Ⅰ、Ⅲ」的判据就是这一句。
22. 机器字长 $\geq$ 存储字长，通常是 1、2、4 倍关系；指令字长与机器字长无关，只是各自与存储字长成倍数。把「指令字长 = 机器字长」当结论去用，一定会错。

23. 按字节编址时可寻址空间 = $2^{\textbf{MAR}\text{ 位数}}$ 字节。地址线 20 位、按字节编址即 1 MB；同一题目里「字长 16 位」不能拿去做 $2^{20}\times 16$ 的乘法——那是把编址单位和字长搞混了。
24. 执行时间比值题的分子分母要对准。$\frac{T_A}{T_B}=\frac{CPI_A}{CPI_B}\cdot\frac{f_B}{f_A}$，主频在分母上，别写反。
25. 「速度提高 **50%**」不等于「时间减少 **50%**」。速度变为原来的 1.5 倍，时间变为原来的 $1/1.5\approx 0.667$ 倍。2012-12 中 CPU 时间 90 s 变为 $90/1.5=60$ s，加上不变的 I/O 10 s 得 70 s，选 D——若误算成 $90\times 0.5=45$ s 再加 10 s 就会误选 55 s。
26. **PC** 里装的是「下一条」指令的地址，不是当前指令的地址。当前指令已装入 IR，PC 在取指阶段就已自增；转移指令靠执行阶段改写 PC 生效。
27. 机器字长与地址字长不是一回事。「32 位计算机」只说明数据通路宽度是 32 位，地址线可能是 20 位、32 位或更多。算寻址范围必须用地址线根数。
28. 数据字长与机器字长可以不等。64 位 `double` 在 32 位机上照样能存能算，只是要拆成两段处理；不要因为「机器字长 32 位」就断定「数据不可能超过 32 位」。
29. 在一条式子里同时出现容量与速率时，必须分别用两种进制换算。例如「1 MB 数据以 1 MB/s 传输需要 $2^{20}/10^6=1.048576$ s」，而不是 1 s。答案「整整好好是 1」往往就是进制用错的红旗。
30. $\lceil\log_2 C\rceil$ 这类题不能靠「差不多」蒙。容量按 2 的幂还是 10 的幂，结果在很多数据下恰好相同，错误会被掩盖；判分依据是你取数是否按定义，而不是结果碰巧对。

第九章数据的表示和运算

本章考纲要求

第二部分计算机组成原理二、数据的表示和运算（一）数制与编码

1. 进位计数制及其数据之间的相互转换
2. 定点数的编码表示

（二）运算方法和运算电路

1. 基本运算部件：加法器，算术逻辑部件（ALU）。
2. 加/减运算：补码加/减运算器，标志位的生成。
3. 乘/除运算：乘/除法运算的基本原理，乘法电路和除法电路的基本结构。

（三）整数的表示和运算

1. 无符号整数的表示和运算
2. 带符号整数的表示和运算

（四）浮点数的表示和运算

1. 浮点数的表示：IEEE 754 标准。
2. 浮点数的加/减运算

本章是计组计算题最密集的一章。从历年统考看，它的出题方式只有三种：选择题考「边界值 + 编码转换 + 标志位判断」，大题考「**C** 语言程序段的机器级执行结果」，大题考「运算电路与标志位的逻辑表达式」。这三种题型都不要求背结论，要求的是能在纸上把位运算过程一步步推出来。所以本章的写法是：每个知识点给出定义、公式、可执行步骤，再用表格把一次完整的位运算过程摊开，让读者能照着复算一遍。

一、数制与编码

（一）进位计数制

进位计数制用一组固定的数字符号和一个固定的基数（**radix**）来表示数。基数为 r 的 r 进制数中，使用的数字符号是 $0 \sim r-1$；每个数位上的数字所代表的实际值是「该数字 $\times$ 该位的位权」，第 i 位的位权是 r^i。任意一个 r 进制数可以写成按位权展开式：

$$N = \sum_{i=-m}^{n-1} a_i r^i, \qquad a_i \in \{0, 1, \ldots, r-1\}$$

其中整数部分有 n 位，小数部分有 m 位。计算机内部使用二进制，原因是四方面的：制造只有两个稳定状态的物理器件容易（高/低电位、有/无脉冲、正/负极）；二进制的计数与运算规则最简单（加法和乘法都只有四条）；二进制恰好与逻辑命题的「真/假」对应，便于逻辑运算；用逻辑门电路实现算术运算方便。这是统考选择题考过的结论。

进制	基数	数字符号	计算机中的用途
二进制	2	0 1	机器内部的物理表示
八进制	8	0 ~ 7	二进制的一种缩写写法
十六进制	16	0 ~ 9 A ~ F	书写机器数、地址、机器码的通用写法
十进制	10	0 ~ 9	人机交互的输入输出形式

四者之间是「同一个数在不同位权系统下的写法」，本质是同一个数值，不是四种不同的数。

（二）进位计数制之间的相互转换

（1）任意进制 → 十进制：按位权展开求和

直接代入位权公式即可，是唯一不需要技巧的方向。

例：$1101.101B = 1\times 2^3 + 1\times 2^2 + 0\times 2^1 + 1\times 2^0 + 1\times 2^{-1} + 0\times 2^{-2} + 1\times 2^{-3} = 13.625$

例：$2AF.8H = 2\times 16^2 + 10\times 16^1 + 15\times 16^0 + 8\times 16^{-1} = 687.5$

（2）十进制 → r 进制：整数部分除基取余，小数部分乘基取整

这是统考要考的「通用方法」，必须能说出为什么。

整数部分用「除基取余，逆序排列」。设 $N = (a_{n-1}\cdots a_1 a_0)_r$，两边对 r 取余，有 $N \bmod r = a_0$——因为 r 的更高次幂都能被 r 整除，只有最低位的 a_0 留在余数里。于是商为 $(a_{n-1}\cdots a_1)_r$，再对 r 取余就得到 a_1。反复执行直到商为 0，把每次得到的余数从后往前读出来，就是结果。

小数部分用「乘基取整，顺序排列」。设小数 $F = (0.b_{-1}b_{-2}\cdots b_{-m})_r$，两边乘 r，有 $rF = b_{-1} + (0.b_{-2}\cdots b_{-m})_r$——乘基数相当于把小数点右移一位，移出来的整数部分正好是最高小数位 b_{-1}。反复执行直到小数部分为 0 或达到所需精度，把每次得到的整数部分从前往后读出来。

转换方向	循环体	循环终止	读数方向
十进制整数 → r 进制整数	余数 $= N \bmod r$，记下余数，$N = \lfloor N/r \rfloor$	$N = 0$	余数逆序读出
十进制小数 → r 进制小数	$N = N \times r$，记下整数部分，N 取 N 的小数部分	$N = 0$ 或达到所需精度	整数部分顺序读出

例：$43.6875D = 101011.1011B$。整数部分 $43 = 32 + 8 + 2 + 1 = 101011B$（也可以用除 2 取余：$43 \to 21\cdots 1$、$21 \to 10\cdots 1$、$10 \to 5\cdots 0$、$5 \to 2\cdots 1$、$2 \to 1\cdots 0$、$1 \to 0\cdots 1$，逆序得到 $101011B$）；小数部分 $0.6875 \times 2 = 1.375$ 记 1，$0.375 \times 2 = 0.75$ 记 0，$0.75 \times 2 = 1.5$ 记 1，$0.5 \times 2 = 1.0$ 记 1，顺序得到 $.1011B$。

小数部分可能永远取不完

只有形如 $k/2^m$ 的十进制小数才能在有限位内被二进制精确表示。$0.3D$、$0.7D$、$1.2D$ 转二进制都是无限循环小数，必须按题目要求截取到指定位数，这就产生了表示误差。统考 2021 年就考过「下列数值中不能用 IEEE 754 浮点格式精确表示的是」，正确答案是 1.2，而 1.25、2.0、2.5 都能精确表示（$1.25 = 1.01B$、$2.5 = 10.1B$）。判断标准很简单：把十进制小数写成既约分数 p/q，若 q 是 2 的幂就能精确表示。

（3）二进制与八进制、十六进制的快速转换

八进制与十六进制的基数是 2^3 和 2^4，因此可以一位对三（四）位直接映射，不需要经过十进制。

- 二进制 → 八进制：以小数点为界，向两侧每 3 位一组，不足补 0，每组直接写成一位八进制数字。
- 二进制 → 十六进制：以小数点为界，向两侧每 4 位一组，不足补 0，每组直接写成一位十六进制数字。
- 反向转换：每位八（十六）进制数字展开成 3（4）位二进制，去掉两端多余的 0。

例：$1011010.1101B$ → 分组 001 011 010 . 110 100 → $132.64O$；分组 0101 1010 . 1101 → $5A.DH$。

本知识点的考法

统考对「进位计数制」的直接考查不多，通常嵌在其他题目里：给出十六进制机器数要求先转成二进制再判断符号位与溢出；或者判断某个十进制小数能否被 IEEE 754 精确表示。真正的必考项是「定点数的编码表示」，转换速度快慢直接决定后面所有计算题的速度。

（三）真值与机器数

- 真值（**true value**）：带正负号、符合人类书写习惯的数值，如 -3、$+5$、-0.75。真值可以用十进制写，也可以用二进制写（如 $-1011B$）。
- 机器数（**machine number**）：数值在计算机内部的 0/1 编码序列。机器数本身不含符号位以外的任何符号信息，它的正负完全由编码规则决定。

同一个机器数，按不同的编码规则解释会得到不同的真值；同一个真值，用不同的编码规则编码会得到不同的机器数。所以做题时必须先确认「本题用哪种码」或「按什么类型解释」。

机器数不带符号

写机器数时不能带负号，也不能带小数点。例如 $x=-0.75$，其原码要写成 `1.1100000`（8 位机器数，小数点只是约定位置），不能写成 `-0.11`。考试中把机器数写成带符号形式是本考点最低级的失分点。

（四）定点数的编码表示

定点数约定机器数中小数点的位置固定不变：小数点不使用符号表示，只是一个「约定」。计算机中只采用两种约定：

- 定点整数：小数点固定在最低位之后。n 位定点整数的真值是整数。
- 定点小数：小数点固定在符号位之后、最高数值位之前。n 位定点小数的真值绝对值小于 1。

下面统一约定「编码字长为 $n+1$ 位」，即 **1** 位符号位 + n 位数值位。$n+1$ 位编码的模是 2^{n+1}，其可表示的编码状态共 2^{n+1} 个。

*1.*原码

最高位是符号位（0 表示正，1 表示负），其余 n 位是真值的绝对值。

$$[x]_{原}=\begin{cases}x & 0\le x\le 2^n-1\\ 2^n+|x| & -(2^n-1)\le x\le 0\end{cases}$$

- 表示范围：$-(2^n-1)\sim+(2^n-1)$。
- **0** 的表示不唯一：$[+0]_{原}=0\underbrace{00\cdots0}_{n}$，$[-0]_{原}=1\underbrace{00\cdots0}_{n}$。
- 优点：与真值的对应关系直观，乘法时符号位与数值位可以分开处理。
- 缺点：符号位不能直接参与加减运算，加减法规则复杂；有两个零，给判零、比较带来麻烦。

现代计算机不用原码表示整数，但用原码表示浮点数的尾数（IEEE 754 的尾数就是原码形式，只是隐藏了规格化后的最高位 1）。

*2.*反码

正数的反码与原码相同；负数的反码是在原码基础上符号位不变、数值位全部取反。

$$[x]_{反}=\begin{cases}x & 0\le x\le 2^n-1\\ (2^{n+1}-1)+x & -(2^n-1)\le x\le 0\end{cases}$$

- 表示范围：$-(2^n-1)\sim+(2^n-1)$（与原码相同）。
- **0** 的表示不唯一：$[+0]_{反}=0\underbrace{00\cdots0}_{n}$，$[-0]_{反}=1\underbrace{11\cdots1}_{n}$。
- 反码的符号位可以参与运算，但最高位产生的进位必须循环进位（加回到最低位），硬件实现麻烦。它在现代计算机中几乎不用，只作为「原码求补码」的中间过渡形式。

*3.*补码

补码是整个计组的基石。先给定义，再解释为什么这样定义。

定义（从原码出发）：正数的补码与原码相同；负数的补码是符号位不变、数值位按位取反，末位加 **1**（即在反码基础上末位加 1）。

定义（从模运算出发，这才是本质）：$n+1$ 位补码是一个模 2^{n+1} 的模运算系统，其定义为

$$[x]_{补}=x \bmod 2^{n+1}=\begin{cases}x & 0\le x\le 2^n-1\\ 2^{n+1}-|x| & -2^n\le x\le 0\end{cases}$$

注意负数的定义式是 $2^{n+1}-|x|$，写成二进制恰好就是「$|x|$ 的全 $n+1$ 位按位取反后加 1」——这就是补码定义式与「取反加一」口诀的等价性来源。

- 表示范围：$-2^n\sim+(2^n-1)$。比原码、反码多表示一个最小负数 -2^n，因为 $+0$ 与 -0 的编码状态被合并，空出来的一种状态被赋予 -2^n。
- **0** 的表示唯一：$[0]_{补}=\underbrace{00\cdots0}_{n+1}$，全 0。
- 几个必须条件反射的编码：
  - 最小负数 -2^n 的补码是 $1\underbrace{00\cdots0}_{n}$（符号位 1，数值位全 0）。
  - 最大正数 2^n-1 的补码是 $0\underbrace{11\cdots1}_{n}$。
  - -1 的补码是 $\underbrace{11\cdots1}_{n+1}$，全 1。

为什么用补码：三条理由。

1. 加减法统一，符号位与数值位一同参与运算。在模 2^{n+1} 意义下，减法可以改写成加法：$[x-y]_{补}=[x]_{补}+[-y]_{补}\ (\text{mod } 2^{n+1})$。计算机中所有运算都基于加法器实现，这意味着一套加法器就能完成加、减两种运算，不需要为减法单独设计电路，也不需要额外判断「|x| 是否大于 |y|」来决定做加法还是减法。
2. 数 **0** 的表示唯一。判零只需要一个「结果是否全 0」的零标志 ZF，无需区分 $+0$ 和 -0。

3. 比原码多表示一个最小负数 -2^n，表示范围对称性更好，利用率更高。

正因为如此，50 年代以来所有计算机都用补码表示带符号整数，C 语言的 `char`、`short`、`int`、`long` 都是补码。

求 $[-x]_{补}$ 的唯一法则

将 $[x]_{补}$ 连同符号位在内全部按位取反、末位加 **1**，就得到 $[-x]_{补}$。注意「连同符号位」四个字：符号位也要取反。

例：$x=+0.75$，$[x]_{原}=[x]_{补}=0.1100000$（8 位）；$[x]_{补}$ 全部按位取反得 1.0011111，末位加 1 得 1.0100000，即 $[-0.75]_{补}=1.0100000$，真值 $-\left(1-2^{-1}\right)\times 2^{0}\cdots$ 按定点小数位权验证：1.0100000 的位权是 $-1\times 2^{0}+0\times 2^{-1}+1\times 2^{-2}=-1+0.25=-0.75$，正确。

原码与补码的互相转换，有两条不经过反码的速算法：

- 原码 → 补码（负数）：从右往左找到第一个 `1`，把这个 `1` 及其右侧的所有位保持不变，左侧的所有数值位按位取反（符号位保持 `1` 不变）。
- 补码 → 原码（负数）：同样的操作做一遍即可（因为补码的补码就是原码）。
- 补码 → 真值：按位权公式直接算，见下文「位权视角」。

例：$x=-52$ 的 8 位补码。$52=32+16+4=00110100B$，原码为 10110100。从右往左第一个 `1` 在最低位，它左侧的数值位 0110100 全部取反为 1001011……更稳妥的做法是取反加一：10110100 取反 11001011，加 1 得 11001100。验证：11001100 按位权算 $=-128+64+8+4=-52$，正确。

*4.*移码

移码（增码）是在真值上加上一个偏置值（**bias**）后按无符号数编码。$n+1$ 位编码中，偏置值通常取 2^n 或 2^n-1。

$$[x]_{移}=x+2^{n}\quad\text{（以偏置值}2^{n}\text{ 为例）}$$

移码与补码的关系：当偏置值取 2^n 时，$[x]_{移}=x+2^n$，而 $[x]_{补}=x \bmod 2^{n+1}$。两者在二进制上只差符号位是否取反：

$$[x]_{移}=[x]_{补}\text{ 的符号位取反}$$

验证：$x=+3$（$n=4$，5 位编码），$[x]_{补}=00011$，符号位取反得 $10011=19$；而 $x+2^4=3+16=19$，一致。$x=-3$，$[x]_{补}=11101$，符号位取反得 $01101=13$；而 $-3+16=13$，一致。

- 移码只用于表示定点整数，不能表示定点小数。
- 移码的 0 表示唯一：$[0]_{移}=100\cdots0$。
- 移码的核心价值：由于加上了偏置值，移码的全部编码在数轴上是从全 0 单调递增到全 1 的，因此移码可以按无符号数的方式直接比较大小，比较结果与真值的大小关系一致。这就是浮点数用移码表示阶码的原因——浮点数加减运算的对阶步骤要频繁比较两个阶码的大小，用移码比较可以省掉符号位单独判断的逻辑。

*5.*四种编码的横向对比

下表统一按「1 位符号位 + n 位数值位」共 $n+1$ 位编码理解。

编码	表示范围	最大数	最小数	**0** 的编码	主要用途
原码	$-(2^n-1)\sim+(2^n-1)$	$0,11\cdots1$	$1,11\cdots1$	两个：$0,00\cdots0$ 与 $1,00\cdots0$	浮点数尾数、乘除法符号处理
反码	$-(2^n-1)\sim+(2^n-1)$	$0,11\cdots1$	$1,00\cdots0$	两个：$0,00\cdots0$ 与 $1,11\cdots1$	原码与补码互转的中间形式
补码	$-2^n\sim+(2^n-1)$	$0,11\cdots1$	$1,00\cdots0$	唯一：$0,00\cdots0$	计算机中所有带符号整数
移码	$-2^n\sim+(2^n-1)$	$1,11\cdots1$	$0,00\cdots0$	唯一：$1,00\cdots0$	浮点数的阶码
无符号整数	$0\sim2^{n+1}-1$	$1,11\cdots1$	$0,00\cdots0$	唯一：$0,00\cdots0$	地址、编号、无负值的量

原码与反码的「最小数」为何不同

原码的最小负数对应数值位全 1、符号位 1，即 $-(2^n-1)$；而反码的负数越大（越接近 0）数值位越大，所以 $1,00\cdots0$ 才代表 $-(2^n-1)$，$1,11\cdots1$ 代表 -0。这两个编码的最小负数写法正好相反，是出题人最喜欢的陷阱点。

*6.*位权视角：算补码真值的最快方法

对 $n+1$ 位补码 $x_nx_{n-1}\cdots x_0$，最高位的位权是 -2^n，其余各位的位权与无符号数相同：

$$x=\sum_{i=0}^{n-1}x_i2^i+x_n\cdot(-2^n)$$

即「只有最高位（符号位）的位权是负的」。举例：

- $[1011]_{补}=1\times2^0+1\times2^1+0\times2^2+1\times(-2^3)=1+2-8=-5$
- $[0011]_{补}=1+2=3$

用位权视角可以直接推出补码的表示范围：最高位取 1、其余全 0 时，只剩下负权，取得最小值 -2^n；最高位取 0、其余全 1 时，只剩下全部正权，取得最大值 2^n-1。

无符号整数的位权全部为正：$x=\sum_{i=0}^{n}x_i2^i$。唯一差别就是最高位的位权，这是理解「同一串二进制按有符号/无符号解释结果不同」的关键。

2015 年统考真题

「由 3 个 1 和 5 个 0 组成的 8 位二进制补码，能表示的最小整数是（）。」答案是 -125。思路：要最小，符号位必须取 1（负权 $-2^7=-128$）；剩下 2 个 1 应放在位权最小的位置（2^0、2^1），使它们贡献的正值最小，得机器数 10000011，真值 $-128+2+1=-125$。这类题一律用位权法解，比逐个选项代入快得多。

*7.*补码比较大小的技巧

由位权视角可以直接得到两条结论，做选择题时很省时间：

- 同号（同在负半轴或同在正半轴）比较时，补码的真值随机器数（按无符号解释）单调递增。例如同为负数：$F340H<FF30H<FF40H$（真值分别为 -3264、-208、-192），同为非负数：$7010H<7500H<7740H$。

- 异号比较时，直接看符号位：符号位为 0 的一定大于符号位为 1 的，例如 $F300H$（负）$< 7070H$（正）。

另一个常用技巧是快速求「十六进制补码负数」的真值：先求与它互补（相加为全 **0** 且产生最高位进位）的那个正数，再取负号。例：$[x]_{补} = FF3AH$（16 位），因为 $FF3AH + 00C6H = (1)0000H$，而 $00C6H = 198$，所以 $x = -198$。

二、运算方法和运算电路

（一）基本运算部件

*1.*一位全加器

加法器的基本单元是一位全加器（**Full Adder, FA**），它有三个输入（被加数 A_i、加数 B_i、来自低位的进位 C_i）和两个输出（本位和 S_i、向高位的进位 C_{i+1}）。

$$S_i = A_i \oplus B_i \oplus C_i$$

$$C_{i+1} = A_iB_i + (A_i \oplus B_i)C_i$$

第二个式子可以这样理解：当且仅当「A_i 和 B_i 都为 **1**」（自己产生进位）或者「A_i 与 B_i 恰有一个为 **1** 且低位送来进位」（传递进位）时，本位才向高位进位。令 $G_i = A_iB_i$（进位产生函数），$P_i = A_i \oplus B_i$（进位传递函数），则 $C_{i+1} = G_i + P_iC_i$。

把 $n+1$ 个全加器串联，就得到 $n+1$ 位加法器。每位全加器的一位和公式和进位公式必须能默写，2022 年统考大题直接要求写出标志位的逻辑表达式，用的就是这两个式子。

A_i	B_i	C_i	S_i	C_{i+1}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

*2.*串行进位与先行进位

串行进位（行波进位）加法器：把 $n+1$ 个全加器首尾相接，第 i 位的进位 C_i 必须等第 $i-1$ 位算完才能确定。最高位的进位要经过「进位链」逐级传播，最坏情况下的进位延迟与位数成正比，即 $O(n)$ 级门延迟。优点是结构极其简单，每一位只需要一个全加器。

先行进位（并行进位、超前进位）加法器：把进位公式展开，让每一级的进位直接由各位的 G_i、P_i 和最低位进位 C_0 算出，不再逐级等待：

$$
\begin{aligned}
C_1 &= G_0 + P_0C_0 \\
C_2 &= G_1 + P_1G_0 + P_1P_0C_0 \\
C_3 &= G_2 + P_2G_1 + P_2P_1G_0 + P_2P_1P_0C_0 \\
&\vdots
\end{aligned}
$$

这样各位的进位同时产生，延迟从 $O(n)$ 降到接近常数（实际上受门电路扇入限制）。代价是进位逻辑的门电路数量随位数急剧增加，所以实际 CPU 采用分组先行进位：组内先行进位，组间可以串行或再先行。

做题要记住的三句话

① 全加器的一位和是三个输入的异或；② 进位是「产生的进位 + 传递的进位」；③ 先行进位的本质是用增加硬件换掉进位链的串行等待，ALU 的速度瓶颈就在这里。

*3.*算术逻辑部件（*ALU*）

ALU（Arithmetic Logic Unit，算术逻辑单元）的核心部件是带标志加法器，能完成多种算术运算（加、减、乘除的部分步骤、加 1、减 1 等）和逻辑运算（与、或、非、异或等）。

一个典型 ALU 的接口是：两个 n 位操作数输入端 X、Y，一个进位输入端 C_{in}，一个操作控制端 ALU_{op}，输出端 F 以及若干标志位。

- ALU_{op} 的位数决定操作的种类：k 位控制信号最多对应 2^k 种操作。例如 3 位控制端最多支持 8 种操作。
- 多路选择器 **MUX** 用来从多条输入通路或运算结果中选出一条输出。k 位控制信号对应 2^k 路输入。
- **ALU** 的操作数宽度（位数）通常与机器字长一致。

关于 ALU 的三个「重要认识」

① 计算机中所有运算都基于加法器实现；② 加法器不区分操作数是有符号数还是无符号数——它只是把两串 0/1 按位相加；③ 加法器不判定对错，它总是取低 n 位作为结果，同时生成标志信息，由指令去解释结果的正确性。这三句话是理解「同一套电路既能算无符号数又能算有符号数」的总纲。

（二）补码加/减运算器与标志位的生成

*1.*补码加减运算规则

补码加法：两个补码相加，符号位参与运算，两数和的补码 = 两数补码之和；符号位相加产生的进位直接舍去（模 2^{n+1} 运算）。

补码减法：$x - y = x + (-y)$，因此

$$
[x - y]_{\text{补}} = [x]_{\text{补}} + [-y]_{\text{补}} \pmod{2^{n+1}}
$$

而 $[-y]_{\text{补}}$ 只需将 $[y]_{\text{补}}$ 连同符号位按位取反、末位加 1。

这个规则在 $y = -2^n$ 时失效：例如 8 位补码中 -128 的补码是 10000000，按规则求 $[-(-128)]_{\text{补}}$ 得到 10000000，即 -128，这显然不对，因为 8 位补码不能表示 $+128$。但只要结果不溢出，按此规则算出的补码减法结果仍然正确。例：$-7-(-128)$，$[-7]_{\text{补}} = 11111001$，$[-(-128)]_{\text{补}}$ 按规则算为 10000000，相加得 (1)01111001，舍去进位得 $01111001 = 121$，与 $-7+128 = 121$ 一致。这就是模运算系统的「回绕」特性。

*2.*加减运算的统一实现

减法在硬件上不单独实现，而是「把减数取反，加 1，再相加」：

- 加法：MUX 选择 0 那条通路，被加数 Y 原样送入加法器，低位进位 $C_0 = 0$。
- 减法：MUX 选择 1 那条通路，Y 先经过非门按位取反，同时减法控制信号 $Sub = 1$ 作为最低位的进位输入，等效于末位加 1，于是 Y 变成 $[-Y]_{补}$，减法变成加法。

因此无符号整数和带符号整数的加/减运算电路完全一样，这个电路称为整数加减运算部件，它基于带标志加法器实现。二者的区别只在于如何解释结果和看哪个标志位。

*3.*四个标志位

指令执行产生的标志位通常存放在状态寄存器 / 程序状态字寄存器（**PSW**）中。x86 把它们放在 `EFLAGS` 寄存器中，供条件转移指令使用；MIPS、RISC-V 没有专门的状态寄存器，条件分支指令直接根据 ALU 当前的状态标志决定是否转移。不论哪种结构，ALU 都会产生这些标志，区别只是「何时、如何利用」。

设加法器共 $n+1$ 位，最高位（符号位）产生的进位为 C_n（即 C_{out}），次高位（最高数值位）产生的进位为 C_{n-1}，Sub 是加减控制信号（$Sub = 1$ 表示减法，$Sub = 0$ 表示加法），则四个标志位的含义与判定如下。

标志	名称	含义	硬件判定
ZF	零标志	结果全 0 时为 1	结果的各位全部为 0
SF	符号标志	有符号结果的正负	直接取结果的最高位，$SF = F_n$
OF	溢出标志	有符号数加减结果溢出	$OF = C_n \oplus C_{n-1}$
CF	进/借位标志	无符号数加法的进位/减法的借位	$CF = Sub \oplus C_{out}$

逐条解释。

零标志 ZF（**Zero Flag**）：$ZF = 1$ 表示运算结果为 0。不论把操作数看作有符号数还是无符号数，ZF 都有意义，因为「结果是否为 0」与解释方式无关。它主要用于判断相等。

符号标志 SF（**Sign Flag**）：SF 表示有符号数加减运算结果的符号，直接取结果的最高位。对无符号数运算 SF 没有意义——无符号数没有负数，最高位也不代表符号。（有些教材把 SF 叫 `Signal Flag`，统考按「符号标志」理解。）

溢出标志 OF（**Overflow Flag**）：$OF = 1$ 表示有符号数加减运算溢出。硬件上由最高位进位 C_n 与次高位进位 C_{n-1} 求异或得到：

$$OF = C_n \oplus C_{n-1}$$

推导：补码减法已转化为加法，所以溢出只可能发生在两种情形。正数加正数：两个加数符号位都是 0，相加后结果的符号位变成 1（正数相加得负数），说明溢出；此时来自次高位的进位必定为 1，而符号位向外的进位必定为 0，$C_n \oplus C_{n-1} = 0 \oplus 1 = 1$。负数加负数：两个加数符号位都是 1，相加后结果符号位变成 0（负数相加得正数），说明溢出；此时来自次高位的进位必定为 0，而符号位向外的进位必定为 1，$C_n \oplus C_{n-1} = 1 \oplus 0 = 1$。两种情形合起来正好就是 $C_n \oplus C_{n-1}$。

OF 与操作数符号、运算种类的关系：

运算	符号关系	是否可能溢出
加法	两数异号	不会溢出
加法	两数同号	可能溢出（结果变号则溢出）
减法	两数同号	不会溢出
减法	两数异号	可能溢出（结果变号则溢出）

用一句话概括：「同号相加」「异号相减」才可能溢出；溢出发生时，结果的符号一定与两操作数的符号相反。

进/借位标志 CF（**Carry Flag**）：加法时 CF 是进位标志——若最高位产生进位，说明无符号数相加超过了 $2^{n+1}-1$；减法时 CF 是借位标志——若最高位没有产生进位，说明无符号数相减时被减数小于减数，需要向更高位借位。因此

$$CF = Sub \oplus C_{out}$$

- 加法（$Sub=0$）：$CF = C_{out}$，有进位则 $CF=1$。
- 减法（$Sub=1$）：$CF = \overline{C_{out}}$，无进位（即最高位没有产生进位）则 $CF=1$。

如果题目中没有单独的 Sub 控制信号，则用最低位的进位输入 C_{in}（C_0）代替，即 $CF = C_{in} \oplus C_{out}$，因为对加减运算器而言 C_0 就等于 Sub。

CF 与 OF 的区别是本章最核心的辨析

- OF 判断有符号数的溢出，CF 判断无符号数的溢出。两者的判定电路不同、适用对象不同，绝不能互换。
- 对同一个加法器，一次运算会同时产生两个标志：$A+B$ 时 OF 告诉你有符号解释下是否溢出，CF 告诉你无符号解释下是否溢出。至于该看哪个，由数据的类型决定——C 语言中 `int` 看 OF，`unsigned int` 看 CF。
- 加法时「有进位」、减法时「无进位（即需要借位）」都使 $CF=1$，两者形式相反但语义一致：都表示无符号数的结果超出了 $n+1$ 位能表示的范围。
- 无符号数运算产生的 SF 和 OF 并非「无意义」，它们的含义是：把参与运算的机器数和结果都解释为有符号数时，结果的符号和溢出情况。这是同一串 0/1 的另一种解读，不是电路出错。

*4.*标志位的使用：比较大小与条件转移

（**1**）无符号数比较大小：执行 $A-B$，用 ZF、CF 判断。

条件	标志位
$A=B$	$ZF=1$
$A\neq B$	$ZF=0$
$A<B$	$CF=1$（有借位）
$A>B$	$CF=0$ 且 $ZF=0$
$A\leq B$	$CF=1$ 或 $ZF=1$
$A\geq B$	$CF=0$

（**2**）有符号数比较大小：执行 $A-B$，用 ZF、SF、OF 判断。核心思想是用 $SF\oplus OF$ 表示「真实结果的正负」：当没有溢出（$OF=0$）时 SF 就是真实符号；当发生溢出（$OF=1$）时真实符

号与 SF 恰好相反。因此 $SF \oplus OF = 1$ 等价于「真实结果为负」。

条件	标志位
$A = B$	$ZF = 1$
$A \neq B$	$ZF = 0$
$A < B$	$SF \oplus OF = 1$
$A > B$	$SF \oplus OF = 0$ 且 $ZF = 0$
$A \leq B$	$SF \oplus OF = 1$ 或 $ZF = 1$
$A \geq B$	$SF \oplus OF = 0$

2011 年统考真题

「条件转移指令 `bgt`（无符号整数比较大于时转移）的转移条件是（）。」答案 $\overline{CF + ZF} = 1$，即 $CF = 0$ 且 $ZF = 0$。无符号数比较只看 CF 和 ZF，OF 与 SF 都不参与——这正是测试「CF 与 OF 分别管谁」的题。

2024 年统考真题（加法器的输出与标志位）

「若通用寄存器 `rs1`、`rs2` 的内容分别是 $87654321H$ 与 $98765432H$，执行 `add` 指令后 ALU 输出端 F、OF 和 CF 分别是什么？若该 `add` 指令处理的是无符号整数，则应根据哪个标志判断是否溢出？」

计算过程：$87654321H + 98765432H$ 按位相加得 $(1)1FDB9753H$，舍去最高位的进位，$F = 1FDB9753H$。两个加数的最高位都是 1（都解释为负数），而结果的最高位变为 0（正数），同号相加而结果变号，有符号数溢出，$OF = 1$。最高位产生了进位 $C_{out} = 1$，加法时 $CF = C_{out} = 1$，说明无符号解释下也越界，$CF = 1$。若按无符号整数处理，应看 CF 判断是否溢出。

2022 年统考真题（标志位的逻辑表达式）

「设 ALU 输入端 A、B 及输出端 F 的最高位分别为 A_{15}、B_{15}、F_{15}，写出 SF 与 OF 的逻辑表达式。」答案是：

$$SF = F_{15}$$

$$OF_{加} = A_{15}B_{15}\overline{F_{15}} + \overline{A_{15}}\,\overline{B_{15}}F_{15}, \qquad OF_{减} = A_{15}\overline{B_{15}}\,\overline{F_{15}} + \overline{A_{15}}B_{15}F_{15}$$

加法溢出只有两种可能：两个正数（$A_{15} = B_{15} = 0$）相加得负数（$F_{15} = 1$），或两个负数（$A_{15} = B_{15} = 1$）相加得正数（$F_{15} = 0$）。减法溢出同理，只需把 B_{15} 换成 $\overline{B_{15}}$（因为减法已把减数取反）。

MIPS / RISC-V 与 x86 的差别（可考概念题）

x86 把 CF、ZF、SF、OF 等条件标志集中存放在 **EFLAGS** 标志寄存器（即程序状态字寄存器 PSW）中，条件转移指令读取它；而 **MIPS**、**RISC-V** 没有专门的状态寄存器，条件分支指令直接根据 **ALU** 当前产生的状态标志决定是否转移。不论哪种结构，ALU 都会产生这些标志，区别只在于何时、如何利用它们。

（三）乘/除法运算的基本原理

一句话原理

乘法和除法都可以用控制逻辑循环地执行加法和移位操作来实现。乘法 = 累加 + 右移；除法 = 加减 + 左移。理解了这一点，就能理解为什么「没有乘法指令的计算机也能完成乘法运算」——编译器把乘法编译成一段「比较、加法、移位」的循环即可。

*1.*原码一位乘法

符号与数值分开处理：乘积的符号位 = 两个操作数符号位的异或；乘积的数值部分 = 两个绝对值相乘。

绝对值（无符号）相乘的机器算法：被乘数每次与乘数的一位相乘得到一个部分积，部分积逐次右移并与下一个部分积累加。

- 乘数最低位为 1：部分积加上被乘数，然后右移一位。
- 乘数最低位为 0：部分积加 0，然后右移一位。
- 重复 n 次（n 为数值位数），最后乘积的高 n 位在部分积寄存器中、低 n 位在乘数寄存器中。

原理就是手算竖式：$X \times Y = X \times \sum y_i 2^i$，把 y_i 逐位取出决定「加不加 X」，每处理完一位就把累加结果右移（等价于把手算竖式里逐行左移的部分积改成结果右移）。

例：$X = 0.1101$（$13/16$），$Y = 0.1011$（$11/16$），求 $X \times Y$。

步骤	操作	部分积（P）	乘数（Y）
初始	$P_0 = 0$	0.0000	1011
第 1 步	Y 最低位为 1，$P + X$	0.1101	1011
	右移一位	0.0110	1101
第 2 步	Y 最低位为 1，$P + X$	1.0011	1101
	右移一位	0.1001	1110
第 3 步	Y 最低位为 0，$P + 0$	0.1001	1110
	右移一位	0.0100	1111
第 4 步	Y 最低位为 1，$P + X$	1.0001	1111
	右移一位	0.1000	1111

结果 $X \times Y = 0.10001111B = 143/256$，与真值 $13/16 \times 11/16 = 143/256$ 一致。（表中「右移」后乘数寄存器的高位补入部分积移出的最低位，这是把部分积和乘数拼成一个 $2n$ 位寄存器一起右移的写法。）

*2.*补码一位乘法（*Booth* 算法）

补码乘法不能像原码那样「符号与数值分开算」，因为补码的符号位带负权。Booth 算法的核心是重新组织展开式，把乘数按「相邻两位之差」重新编码：

$$[X \cdot Y]_{\text{补}} = [X]_{\text{补}} \times \left(-y_n + \sum_{i=1}^{n} y_{n-i} 2^{-i} \right) = \sum_{i=1}^{n} (y_{n-i+1} - y_{n-i}) [X]_{\text{补}} \cdot 2^{-i}$$

其中 y_{n+1} 是附加位（辅助位）。展开式的每一项 $(y_i - y_{i+1})$ 只可能是 0、$+1$、-1 三种值，因此每一步只需要在「加 X」「减 X」「什么都不加」之间选一个。

迭代规则：每一步考察乘数寄存器的最低位 y_i 与附加位 y_{i+1} 组成的两位，按下表决定操作，然后做算术右移一位。

y_i（乘数最低位）	y_{i+1}（附加位）	对部分积的操作
0	0	加 0（不变）
0	1	加 $[X]_{补}$
1	0	加 $[-X]_{补}$（即减 X）
1	1	加 0（不变）

记忆口诀：两位相同加 0；01 加被乘数；10 减被乘数。

完整流程（标准写法，务必按这个顺序操作）：

1. 附加位 y_{n+1} 初值置 **0**；部分积寄存器 A 初值置 0；乘数寄存器 Q 置乘数。
2. 判断 (y_i, y_{i+1})，按上表对 A 做加法或减法。
3. 把「A 与 Q 拼成的寄存器」整体算术右移一位：A 右移且最高位补 A 原来的符号位，Q 右移且最高位补入 A 移出的最低位，附加位补入 Q 移出的最低位。
4. 重复第 2、3 步共 n 次（n 为乘数的位数），不再做额外的加法。
5. 运算结束后，A（高 n 位）与 Q（低 n 位）拼接就是 $2n$ 位的乘积补码。若题目只保留 n 位结果，取 Q 的内容。

关于「最后一步不移位」的说法

不同教材对最后一次迭代的处理有两种写法：一种是「循环体 = 判断 → 加法 → 算术右移，共 n 次」；另一种是「前 $n-1$ 次移位，第 n 次只加不移位」。这两种写法的寄存器位数约定不同，得到的结果一致。做题时以题目给出的寄存器位数和步骤叙述为准，不要凭记忆套用。统考对 Booth 算法的要求是掌握「看两位 → 加减 → 算术右移」这个迭代结构和进位链的循环控制，不要求手算特定寄存器宽度下的每一位。

例一：$x = 1$，$y = -1$（**4** 位补码）——用最小规模验证规则。

$[x]_{补} = 0001$，$[y]_{补} = 1111$，$[-x]_{补} = 1111$。

步	$y_i y_{i+1}$	操作	A（**4** 位）	Q（**4** 位）	y_{i+1}
初始	—	—	0000	1111	0
1	1 0	减 x：$0000 + 1111$	0000	1111	1
2	1 1	加 0	0000	0111	1
3	1 1	加 0	0000	0011	1
4	1 1	加 0	0000	0001	1

结果 $A \cdot Q = 0000\,0001$，即 $+1$，验证 $1 \times (-1) = -1$ 的补码形式 11111111（低 4 位为 1111，A 位为符号扩展的 0000）。这个例子说明：当乘数为全 **1** 时，**Booth** 算法只在第一步做一次减法，后面全是移位，这正是「连续 1 可以合并操作」的体现。

例二：$x = -3$，$y = +3$（**4** 位补码）——一个可以逐行复算的完整算例。

$[x]_{补} = 1101$，$[-x]_{补} = 0011$，$[y]_{补} = 0011$。取部分积寄存器 A 为 $n+1 = 5$ 位，被乘数 x 按 5 位符号扩展为 11101（检验：$-16 + 8 + 4 + 1 = -3$，正确），$[-x]_{补} = 00011$；乘数 Q 为 4 位，附加位初值 0。

步	y_iy_{i+1}	操作	A（**5** 位）	Q（**4** 位）	附加位
初始	—	—	00000	0011	0
1	1 0	$A+X$：$00000+00011$	00011	0011	0
		算术右移一位	00001	1001	1
2	1 1	加 0	00001	1001	1
		算术右移一位	00000	1100	1
3	0 1	$A-X$：$00000+11101$	11101	1100	1
		算术右移一位	11110	1110	0
4	0 0	加 0	11110	1110	0
		算术右移一位	11111	0111	0

四步移位结束后运算终止。取 A 的低 **4** 位与 Q 拼接，得 1111 0111，按 8 位补码解释：符号位为 1，取反加一得 $00001001=9$，所以结果为 -9，与 $-3\times3=-9$ 一致。

手算 Booth 算法的三条铁律

① 部分积寄存器 A 比乘数多 **1** 位（补码乘法要留出符号扩展位），这样 A 的低 n 位与 Q 拼接正好是 $2n$ 位乘积；② 被乘数 X 要按 A 的位数做符号扩展后再参与加减，否则负数会被当成正数；③ 附加位初值 0，先加减后算术右移，共做 n 次。

Booth 算法的四个高频错误

① 附加位初值不是 **0**；② 判断的是「乘数最低位与附加位」这两位，而不是只看最低位；③ 先做加减再移位，顺序颠倒全盘皆错；④ 移位必须是算术右移（补符号位），用成逻辑右移会得到错误结果。

*3.*原码除法（恢复余数法）与加减交替法

原码除法同样把符号与数值分开：商的符号位 = 两个操作数符号位的异或，数值部分 = 两个绝对值相除。

恢复余数法：先在余数上减去除数，若余数为负说明「商这一位应该是 0」，此时必须把减掉的除数加回来（恢复余数），再左移；若余数为正，商这一位是 1，直接左移。这种方法每步可能多一次加法，故称「恢复余数」。

加减交替法（不恢复余数法）：观察到「恢复余数后左移再减」与「不恢复余数直接左移再加」是等价的，因此可以省掉恢复操作。规则是：

- 若当前余数为正：商上 1，左移一位，减除数。
- 若当前余数为负：商上 0，左移一位，加除数。

最后若余数为负，需要再加一次除数恢复出真正的余数。加法器的操作次数减少，硬件实现更快。

例（不恢复余数法，4 位）：$x=1101$（13），$y=0011$（3）。用 A（4 位余数）与 Q（4 位商）拼接，初始 $A=0000$，$Q=1101$。

步	操作	A	Q	商位
初始	—	0000	1101	—
1	左移一位	0001	1010	—

步	操作	A	Q	商位
	$A-y$：$0001-0011$，余数负	1110	1010	$q_0=0$
2	左移一位	1101	0100	—
	余数负，$A+y$：$1101+0011$	0000	0100	$q_0=1$
3	左移一位	0000	1000	—
	$A-y$：$0000-0011$，余数负	1101	1000	$q_0=0$
4	左移一位	1011	0000	—
	余数负，$A+y$：$1011+0011$	1110	0000	$q_0=0$

最终余数为负，再加一次除数：$1110+0011=0001$。商 $Q=0100=4$，余数 1，验证 $13=3\times4+1$，正确。

*4.*补码除法

补码除法的商和余数都用补码表示，符号位参与运算，规则由「商位与被除数、除数的符号关系」决定。统考要掌握的要点：

- 够减的判断依据是「余数与除数是否同号」：余数与除数同号表示够减，商上 1，下一步减除数；异号表示不够减，商上 0，下一步加除数。这正好与无符号除法「看余数正负」对应，只是判断方式改成了比较符号。
- 补码除法的商符在运算过程中自动形成，不需要像原码那样单独用异或求。
- 商的末位通常采用恒置 **1** 的方法来保证精度（末位商恒定为 1，误差不超过末位权重），这是补码除法的一个经典细节。
- 除法的合法性检查：除数不能为 **0**，且被除数的绝对值必须小于除数的绝对值（否则商溢出，无法用定点小数表示），硬件在除法开始前会做这个检查并可能产生「除零异常」。

（四）乘法电路和除法电路的基本结构

*1.*无符号数乘法电路

以 32 位无符号数乘法为例，逻辑结构包含如下部件：

部件	作用
被乘数寄存器 X	存放被乘数（32 位）
乘积寄存器 P	初值置 0（初始部分积 $P_0=0$）；结束时存放 64 位乘积的高 **32** 位
乘数寄存器 Y	初值置乘数；结束时存放 64 位乘积的低 **32** 位
进位触发器 C	保存加法器产生的进位
计数器 C_n	存放循环次数，初值 32，每循环一次减 1，减到 0 结束

部件	作用
ALU	核心部件，在控制逻辑控制下完成 $P+X$ 的加法

每次循环的三件事：① 控制逻辑根据 Y 的最低位决定是否把 X 加到 P（最低位为 1 则加，为 0 则不加）；② 在写使能控制下把运算结果写回 P，加法进位存入 C；③ 对 C、P、Y 三者同步逻辑右移一位——C 移入 P 的最高位，P 的最低位移入 Y 的最高位，Y 的最低位移出，C 补入 0。

于是 Y 的最低位依次被换成乘数的下一位，控制逻辑每次都从 Y 的最低位读取「当前该处理的乘数位」。循环 32 次后，乘积的高 32 位在 P、低 32 位在 Y。

*2.*有符号数（补码）乘法电路

以 32 位补码一位乘法（Booth）为例，与无符号乘法电路的主要区别：

- 乘数寄存器 Y 后面增加一个附加位（辅助位），初值 $Y_{-1}=0$。
- 控制逻辑每次根据 Y 的最后两位（最低位与附加位）决定执行「加 X」「加 $[-X]_{补}$」还是「加 0」，然后执行算术右移，结果写回寄存器。
- 计数器 C_n 控制循环次数，共执行 32 次。
- 因为要支持「减」，ALU 需要能完成加减运算，且需要一个能产生 $[-X]_{补}$ 的取反加一电路。

2020 年统考真题的两问

① 「基于 ALU、移位器、寄存器以及相应控制逻辑实现乘法指令时，控制逻辑的作用是什么?」答：控制加法和移位操作的次数，并发出控制信号来执行判断、加法、移位等操作。②「若指令系统中没有乘法指令，但有加法、减法和移位等指令，则在计算机上也能实现乘法运算，为什么?」答：乘法可以分解为加法和移位操作的循环，因此可以不断执行比较、加法（减法）、移位等指令来实现乘法的功能。

*3.*乘法运算的溢出判断

n 位乘 n 位的结果有 $2n$ 位。

- 若用 $2n$ 位保存乘积，一定不会溢出（$2n$ 位足以容纳两数之积）。
- 若只用 n 位保存结果（截取低 n 位），则可能溢出，需要判断。

类型	不溢出的机器判断	手算判断
无符号整数乘法	乘积的高 n 位全为 **0**	结果是否超出 $[0,2^n-1]$
带符号整数乘法	乘积的高 $n+1$ 位全为 **0** 或全为 **1**	结果是否超出 $[-2^{n-1},2^{n-1}-1]$

带符号乘法的判断要理解成：高 n 位必须恰好是低 n 位最高位（符号位）的符号扩展。若乘积为正，低 n 位最高位是 0，则要求高 $n+1$ 位全为 0；若乘积为负，低 n 位最高位是 1，则要求高 $n+1$ 位全为 1。这与 2020 年、2024 年统考大题的官方答案一致。

2010 年统考真题

「$r_1=FEH$，$r_2=F2H$，$r_3=90H$，$r_4=F8H$ 均为 8 位补码，若运算结果存放在 8 位寄存器中，会发生溢出的是（）。」先把补码转真值：$r_1=-2$、$r_2=-14$、$r_3=-112$、$r_4=-8$。逐项算：$r_1\times r_2=28$（不溢出）、$r_2\times r_3=1568$（8 位补码范围是 $-128\sim127$，远超出，溢出）、$r_1\times r_4=16$、$r_2\times r_4=112$。答案 B。

*4.*除法电路

除法电路的基本结构：

部件	作用
除数寄存器 Y	存放除数
余数寄存器 R	存放当前余数（部分余数）
商寄存器 Q	初值放被除数，运算过程中逐步写入商
ALU	执行除数 Y 与余数 R 的加减运算
控制逻辑	根据运算结果的符号（或与除数的符号关系）确定这一位商是 1 还是 0，并发出加减、移位控制信号
计数器	控制循环次数

每次循环：① ALU 完成 $R \pm Y$；② 控制逻辑根据结果确定商位并上商（把 Q 空出的最低位置为 1 或 0）；③ 寄存器 R、Q 同步左移一位，R 移出的高位丢弃、Q 空出的低位用来上商。除法电路通过循环执行加减法和左移来实现除法运算——这正是「乘法用右移、除法用左移」的对称性。

三、整数的表示和运算

（一）无符号整数

无符号整数指机器字长的全部二进制位都是数值位，没有符号位，其值就是各数值位的位权之和，相当于数的绝对值。n 位无符号整数的表示范围是 $0 \sim 2^n - 1$。

适用场合：地址运算、编号表示，以及一切「不会出现负值」的量。C 语言中用 `unsigned int`、`unsigned short` 等声明。

- 无符号数加法：n 位二进制直接按位相加，最高位产生的进位就是溢出标志（$CF = 1$）。
- 无符号数减法：$A - B$ 转化为 $A + [-B]_{\text{补}}$，即把 B 的 n 位全部按位取反、末位加 $\mathbf{1}$，再与 A 相加；若最高位没有产生进位，说明发生了借位（$CF = 1$），结果为负（以模 2^n 的形式回绕）。

（二）带符号整数

带符号整数最高位是符号位，其余各位是数值位，统一采用补码表示。n 位带符号整数的表示范围是 $-2^{n-1} \sim 2^{n-1} - 1$。

移位运算是最常用的「快速乘除」。C 语言中 `x<<k`、`x>>k` 分别表示左移、右移 k 位。

移位类型	适用对象	规则	说明
逻辑左移	无符号数	高位移出，低位补 0	移出的最高位为 1 时发生溢出
逻辑右移	无符号数	低位移出，高位补 0	相当于无符号数除以 2^k 向下取整
算术左移	有符号数（补码）	高位移出，低位补 0	与逻辑左移结果相同；移后符号位改变即溢出
算术右移	有符号数（补码）	低位移出，高位补符号位	相当于除以 2^k 向 $-\infty$ 取整

移位的三条关键结论

① 左移 k 位相当于乘以 2^k，右移 k 位相当于除以 2^k。② 左移可能溢出（正数可能变负数，负数可能变

正数），判断方法是「移位后符号位是否改变」；右移不会改变符号性，因此不会溢出。③ 算术移位时原来的符号位也一起移动——硬件做移位时不允许任何一位「保持不动」，所以右移必须动态地把符号位补进空出的高位。这也是「算术右移不能改变符号」的硬件原因。

右移的取整方向

算术右移是向 $-\infty$ 取整（对负数而言不是「向 0 取整」）。例：-7 的 8 位补码是 11111001，算术右移一位得 11111100，真值为 -4，而 $-7/2=-3.5$。若移出的位是 0（能整除），则相当于精确除以 2^k；若移出的是 1，则不能整除，结果向小方向取整。这一条在 C 语言程序题里常常是判断「某表达式能否编译成移位指令」的依据。

（三）零扩展与符号扩展

当把一个短整数赋给更长的整数类型时，需要在机器数的高位补齐若干位，补什么取决于源类型是无符号还是有符号。

扩展名	补位内容	适用场合
零扩展	高位补 0	无符号整数扩展，或把无符号数扩展为更长的无符号数
符号扩展	高位补源机器数的符号位（正数补 0，负数补 1）	带符号整数（补码）扩展

例：`short y = -9`，16 位机器数为 $FFF7H$。扩展为 32 位：

- 符号扩展（作为 `int`）：$FFFFFFF7H$。
- 零扩展（作为 `unsigned int`）：$0000FFF7H$。

定点小数与定点整数的扩展位置不同

定点整数在数值位之前（高位）扩展，定点小数在数值位之后（低位）扩展。原因很直观：定点整数要保住「哪些位是高位」，定点小数要保住「小数的位数」。混用会把数扩展错。

（四）无符号与有符号混合运算的陷阱

这几条几乎每年都会以选择题或程序题的形式出现。

（1）机器数不变、只改变解释方式（同宽度转换）

同宽度的 `short` 与 `unsigned short` 互转、`int` 与 `unsigned int` 互转，机器数一位都不变，变的只是「按哪种类型解释」。例如 `unsigned short usi = 65535; short si = usi;` 得到 `si = -1`，因为 $FFFFH$ 按补码解释是 -1。

（2）窄变量扩展为宽变量

- `short` → `int`：符号扩展。
- `unsigned short` → `int`（或 `unsigned int`）：零扩展（先补齐到目标宽度，目标为 `int` 时再按补码解释）。
- `short` → `unsigned int`：先符号扩展为 `int`，机器数不变，再按无符号解释。

（3）宽变量截断为窄变量

`int` → `short`：直接截取低 **16** 位，高位丢弃，机器数变了，真值可能完全不同。

例：`int i = 32777`，其 32 位机器数为 $00008009H$；`short si = i` 截取低 16 位得 $8009H$，符号

位为 1（负数），真值为 $-(2^{15}-9)=-32759$；`int j = si` 再符号扩展回 $FFFF8009H$，真值仍为 -32759。这就是 2024 年统考选择题的答案。

（4）有符号与无符号一起运算时，编译器默认把有符号数转换成无符号数

```
int a = -1;
unsigned int b = 1;
if (a > b) printf("yes");   /* 会输出 yes！ */
```

因为 a 的机器数 $FFFFFFFFH$ 被当作无符号数解释成 4294967295，远大于 1。这是 C 语言中最经典的坑。统考 2017 年大题中 `for（unsigned i = 0; i <= n - 1; i++）` 在 $n=0$ 时死循环，也是同一个原因：$n-1=0-1$ 在无符号意义下回绕成全 1（$2^{32}-1$），条件永真。

三条必须背下来的话

① 同一机器数按不同类型解释会得到不同的真值，转换时先看「机器数变不变」，再看「解释方式变不变」。② 扩展看源类型，截断丢高位；符号扩展补符号位，零扩展补 0。③ 有符号与无符号混合运算，结果按无符号解释，比较时尤其危险。

（五）有符号整数加减运算的溢出判断

统考反复考，这里给全三种方法。

方法一：根据操作数与结果的符号判断。若送至加法器的两个操作数符号相同，而结果的符号与它们不同，则结果溢出；否则不溢出。等价的说法：同号相加、异号相减才可能溢出。

方法二：根据最高位与次高位的进位判断。记符号位的进位为 C_n、最高数值位的进位为 C_{n-1}，则 $OF=C_n\oplus C_{n-1}$。两者不同说明溢出。

方法三：双符号位（变形补码、模 **4** 补码）判断。用两位符号位 K_1K_2 参加运算：

K_1K_2	含义
00	结果为正，未溢出
11	结果为负，未溢出
01	正溢出（结果本应为正，超出上界）
10	负溢出（结果本应为负，超出下界）

两位符号位不同即溢出，且最高位的符号位代表结果的真正符号。采用双符号位的好处是「溢出后结果仍可挽救」——右规（右移一位、阶码加 1）以后可以得到正确结果，这正是浮点数尾数运算采用双符号位的原因。

无符号整数加减的溢出用 CF 判断：加法时最高位产生进位、减法时最高位产生借位，则无符号数结果越界。

2013 年统考真题（标志位判断的经典）

「已知 8 位机器中整型变量 x、y 的机器数为 $[x]_{补}=11110100$、$[y]_{补}=10110000$，若 $z=2x+y/2$，则 z 的机器数是（）。」解析：$2x$ 为算术左移 1 位，$11110100\to11101000$（$-24\times2=-48$）；$y/2$ 为算术右移 1 位，$10110000\to11011000$（$-80\div2=-40$）；相加 $11101000+11011000=(1)11000000$，舍去进位得 11000000。答案 A。这类题的全部技巧就是「左移/右移 + 补码加法」，用十进制先估一下大小可以自查。

四、浮点数的表示和运算

（一）浮点数的一般格式

浮点数把定点整数与定点小数组合起来表示实数：

$$N = (-1)^S \times M \times 2^E$$

字段	名称	常用编码	作用
S	数符	1 位，0 正 1 负	表示整个浮点数的正负
E	阶码	补码或移码表示的定点整数	位数反映浮点数的表示范围；真值反映小数点的实际位置
M	尾数	原码或补码表示的定点小数	位数反映浮点数的精度；数符表示浮点数的正负

（二）浮点数的规格化

规格化的目的是让尾数的有效数字尽量占满尾数数位，从而在固定位宽下获得最高的精度。
规格化的判定条件：

- 原码表示的尾数：最高数值位必须为 **1**，即形如 $0.1\times\times\cdots$ 或 $1.1\times\times\cdots$。
- 补码表示的尾数：符号位与最高数值位必须相反，即形如 $0.1\times\times\cdots$（正）或 $1.0\times\times\cdots$（负）。

由此可以推出补码尾数的取值范围：最大正值为 $0.11\cdots1$，最小正值为 $0.10\cdots0$；最大负值为 $1.01\cdots1$，最小负值为 $1.00\cdots0$（真值 -1）。
两种规格化操作：

操作	触发条件	做法
左规	尾数最高数值位不是有效位（补码尾数符号位与最高数值位相同，形如 $00.0\times\cdots$ 或 $11.1\times\cdots$）	尾数算术左移一位，阶码减 **1**；可能要做多次
右规	尾数出现溢出（双符号位为 01 或 10）	尾数算术右移一位，阶码加 **1**；只做一次

浮点数的溢出与规格化是两回事：阶码小于最小允许值称为下溢，通常把结果置为机器零（$+0$ 或 -0）；阶码超过最大允许值称为上溢，机器产生异常、停止运算（有的机器置为 $\pm\infty$）。

（三）**IEEE 754** 标准

*1.*格式与偏移量

类型	总位数	符号位	阶码位	尾数位	偏移量（**bias**）
单精度 `float`	32	1	8	23	$2^{8-1}-1=127$

类型	总位数	符号位	阶码位	尾数位	偏移量（**bias**）
双精度 `double`	64	1	11	52	$2^{11-1}-1=1023$
扩展精度（现行标准）	128	1	15	112	$2^{15-1}-1=16383$

阶码采用移码表示，但偏移量取 $2^{n-1}-1$（而不是定点数移码常用的 2^n）。其中 n 是阶码的位数。单精度阶码 8 位、偏移量 127；双精度阶码 11 位、偏移量 1023。

阶码真值的求法：把阶码字段当作无符号数读出，再减去偏移量。

$$E_{\text{真}}=E_{\text{字段}}-127\quad(\text{单精度}),\qquad E_{\text{真}}=E_{\text{字段}}-1023\quad(\text{双精度})$$

例：8 位阶码字段 11000000，按无符号读是 192，所以真指数是 $192-127=65$。

尾数采用原码表示，且隐含一位最高位 **1**（规格化数的隐藏位）。尾数字段只存小数点后的 23（52）位，实际有效尾数是 1.字段。这样用 23 位字段换来 24 位有效精度（双精度是 53 位有效精度）。

*2.*规格化数的真值公式

$$\text{规格化数}=(-1)^S\times 1.f\times 2^{E_{\text{字段}}-\text{bias}},\qquad E_{\text{字段}}\in[1,254]\ (\text{单精度})$$

单精度规格化数的指数范围是 $-126\sim127$，因为阶码字段 0 和 255 被保留作特殊用途，可用的字段值只有 $1\sim254$。

尾数的隐含位为什么是 **1**：规格化数的尾数形如 $1.f$，最高位恒为 1，既然恒为 1 就可以省略不存，读取时自动补上。这就是「隐藏位」的含义，也是「规格化数的尾数取值范围是 $[1,2)$」的原因。

*3.*非规格化数与特殊值

IEEE 754 用阶码字段的全 **0** 和全 **1** 区分出四类取值（零、非规格化数、无穷大、非数），把它们记住是必考内容。

值的类型	符号位	阶码字段（单精度）	尾数字段	真值
正零	0	全 0	全 0	$+0$
负零	1	全 0	全 0	-0
非规格化数	0 或 1	全 0	不全为 **0**	$\pm(0.f)_2\times 2^{-126}$
正无穷	0	全 1（255）	全 0	$+\infty$
负无穷	1	全 1（255）	全 0	$-\infty$
非数	0 或 1	全 1（255）	不全为 **0**	`NaN`

非规格化数的两个特殊约定（必考）：

1. 隐含的最高位变成 **0**，即尾数形式是 $0.f$，不是 $1.f$。
2. 阶码真值固定视为 -126（双精度为 -1022），不是 $0-127=-127$。

这样设计的目的是实现逐渐溢出（**gradual underflow**）：非规格化数的最大值与规格化数的最小值平滑衔接——非规格化数的最大值为 $(0.\underbrace{1\cdots1}_{23}) \times 2^{-126} = (1-2^{-23}) \times 2^{-126}$，规格化数的最小值为 1.0×2^{-126}，两者只差一个 2^{-149} 的间隔。

单精度能表示的最小正非零非规格化数是尾数只有最低位为 1 的情形：$2^{-23} \times 2^{-126} = 2^{-149}$。双精度为 2^{-1074}。

四个「最小值/最大值」必须背下来（单精度）

- 规格化最小正数：2^{-126}（阶码字段 1，尾数全 0）
- 规格化最大正数：$(2-2^{-23}) \times 2^{127} = 2^{128} - 2^{104}$（阶码字段 254，尾数全 1）
- 规格化最小负数：$-(2-2^{-23}) \times 2^{127}$（符号 1、阶码字段 254、尾数全 1）
- 规格化最大负数：-2^{-126}（符号 1、阶码字段 1、尾数全 0）
- 非规格化最小正数：2^{-149}
- 最大正数（规格化）的机器数是 $7F7FFFFFH$，最小规格化正数是 $00800000H$，最大负数是 $80800000H$，最小负数是 $FF7FFFFFH$。

「最大负数」与「最小负数」别搞混

最大负数指「小于 0 的数中绝对值最小的那个」，即 -2^{-126}，它离 0 最近；最小负数指「小于 0 的数中绝对值最大的那个」，即 $-(2-2^{-23}) \times 2^{127}$。中文里「最大/最小」在负数区间最容易读反，读题时先在草稿纸上画一条数轴。

历年真题速览

- **2011** 年：$x = -8.25$，求 `float` 型寄存器内容（答案 `C1040000H`）。
- **2012** 年：`float` 能表示的最大正整数（答案 $2^{128} - 2^{104}$）。
- **2013** 年：`C6400000H` 的真值（答案 -1.5×2^{13}）。
- **2014** 年：两个 `float` 变量 `CC900000H` 与 `B0C00000H` 的大小关系（答案 $x < y$ 且符号相同）。
- **2018** 年：最小的规格化正数（答案 1.0×2^{-126}）。
- **2021** 年：不能用 IEEE 754 精确表示的数值（答案 1.2）。
- **2022** 年：-0.4375 的 IEEE 754 单精度表示（答案 `BEE00000H`）。
- **2023** 年：`80200000H` 的真值——阶码字段全为 0、尾数不全为 0，是非规格化数，尾数按 $0.f$ 还原为 $0.01B = \frac{1}{4}$，真值 $= -(0.01)_2 \times 2^{-126} = -2^{-2} \times 2^{-126} = -2^{-128}$（答案 -2^{-128}）。这道题的陷阱在于隐含位取 **0** 还是取 **1**：若误按规格化数处理（隐含位取 1），会算出 -1.25×2^{-126} 而选错。

*4.*真值 → *IEEE 754* 单精度：完整步骤

以「求 $+8.25$ 的 IEEE 754 单精度表示」为例，把每一步都摊开。

第一步：把十进制真值写成二进制小数。

$8.25 = 8 + 0.25 = 1000.01B$（$0.25 = 0.01B$ 由「乘 2 取整」得到：$0.25 \times 2 = 0.5$ 记 0，$0.5 \times 2 = 1.0$ 记 1）。

第二步：规格化，写成 $1.f \times 2^E$ 的形式。

小数点左移 3 位：$1000.01B = 1.00001B \times 2^3$，所以 $E = 3$，$f = 00001$。

第三步：求阶码字段。

$$E_{\text{字段}} = E + \text{bias} = 3 + 127 = 130 = 10000010B$$

第四步：确定数符与尾数字段。

$+8.25$ 为正数，$S=0$。尾数字段取 f 补齐到 23 位：00001000000000000000000。

第五步：拼接。

0 10000010 00001000000000000000000 → 按 4 位一组写成十六进制：0100 0001 0000 0100 0000 0000 0000 0000 → `41040000H`。

步骤	内容	结果
① 转二进制	$8.25=1000.01B$	1000.01
② 规格化	$1000.01=1.00001\times 2^3$	$E=3$，$f=00001$
③ 求阶码字段	$3+127=130$	10000010
④ 定符号与尾数	正数，尾数补足 23 位	$S=0$，$f=$ 00001000000000000000000
⑤ 拼接	0 10000010 00001000000000000000000	`41040000H`

5.IEEE 754 单精度 → 真值：完整步骤

以「求 `C0A00000H` 的真值」为例。

第一步：展开为二进制，按 **1/8/23** 切分。

`C0A00000H` = 1100 0000 1010 0000 ⋯ → $S=1$，阶码字段 = 10000001，尾数字段 = 01000000000000000000000。

第二步：判断类型。阶码字段既不是全 0 也不是全 1，是规格化数。

第三步：求阶码真值。$10000001B=129$，$E=129-127=2$。

第四步：还原尾数。隐含最高位 1，实际尾数 $=1.0100\cdots 0B=1.01B=1.25$。

第五步：合成真值。$(-1)^1\times 1.25\times 2^2=-5$。

字段	二进制	解读	结果
符号位 S	1	负数	$-$
阶码（8 位）	10000001	$129-127$	$E=2$
尾数（23 位）	0100000⋯	隐含 1 后为 $1.01B$	$M=1.25$
真值	—	$(-1)\times 1.25\times 2^2$	-5

（四）浮点数的加减运算

浮点数加减运算有固定的五步流程，统考大题几乎每年都从这里出，必须能背出每一步的做法和目的。

*1.*操作数检查

检查两个操作数是否为 0。若阶码和尾数均为 0，该浮点数为 0；若浮点数下溢，也被当作机器 0 处理。若加数、被加数或减数为 0，则结果直接是另一个操作数（被减数为 0 时结果是减数的相反数）。两个操作数都不为 0 时，才进入下面的流程。

*2.*对阶

求阶差 $\Delta E=E_1-E_2$。

- $\Delta E = 0$：阶码已相等，不需要对阶。
- $\Delta E \neq 0$：小阶向大阶看齐，把阶码小的那个数的尾数右移 $|\Delta E|$ 位，同时把它的阶码加 $|\Delta E|$，直到两数阶码相等。

为什么必须「小阶向大阶看齐」

如果反过来「大阶向小阶看齐」，需要把尾数左移，尾数高位会被移出，损失的是高位有效数字（严重损失精度）；而小阶向大阶看齐时尾数右移，移出的是低位，损失的只是精度最低的几位。另外，小阶向大阶看齐时阶码只会增大，因此对阶过程本身不会引起阶码上溢。

*3.*尾数加减

对阶后，两个数的阶码相同，按定点小数加减运算的规则对尾数求和或求差（若用补码，符号位参与运算；若用原码，需要先统一成补码）。运算后的结果若已经规格化，可直接进第 5 步。

*4.*结果规格化

右规：尾数相加后出现双符号位 01 或 10（尾数绝对值 ≥ 1，即发生了尾数溢出），把尾数算术右移一位、阶码加 **1**。右规只需要做一次，因为两个规格化尾数相加的结果绝对值不超过 2。

左规：尾数出现 $00.0\times\cdots$ 或 $11.1\times\cdots$（符号位与最高数值位相同，不是规格化形式），把尾数算术左移一位、阶码减 **1**，直到符号位与最高数值位相反为止。左规可能需要做多次。

左规的上限

尾数有 k 位数值位时，左规最多做 k 次；左规过程中阶码不断减 1，可能引起阶码下溢。若阶码减到最小允许值以下，说明结果太小，按机器零（± 0）处理。

*5.*舍入

在对阶右移和规格化右规的过程中，尾数的最低位会被移出。为了减少误差，移出的位通常保留下来参与运算，最后再统一舍入。常见方法：

方法	做法	特点
就近舍入（向偶数舍入）	见下	IEEE 754 默认模式
朝 0 舍入	直接截断多余位	最简单，误差单向累积
朝正无穷舍入	正数只要多余位不全为 0 就进 1；负数直接截尾	上界方向
朝负无穷舍入	负数只要多余位不全为 0 就进 1；正数直接截尾	下界方向
「0」舍「1」入	移出的最高位为 0 则舍，为 1 则末位加 1	类似十进制四舍五入
末位恒置 1	不论移出位是什么，末位恒置 1	误差累积小，损失一位精度

IEEE 754 的「就近舍入（向偶数舍入）」细则：

- 多余位大于最低位的一半（如多余位是 $1001\cdots$）：最低位进 1。
- 多余位小于最低位的一半（如多余位是 $0111\cdots$）：直接截断。
- 多余位恰好等于最低位的一半（多余位是 $1000\cdots$）：看最低位——为 **0** 则截断，为 **1** 则进 **1**，使最低位保持偶数。

*6.*判断溢出

在规格化与舍入过程中，阶码可能加 1 或减 1，因此需要检查阶码是否越界。用 e 表示真实指数，E 表示 IEEE 754 中存储的阶码字段（单精度）：

条件	含义
$E = 1 \sim 254$，即 $-126 \le e \le 127$	规格化数
$e > 127$	指数上溢，结果按 $\pm\infty$ 处理（具体与舍入模式有关）
$e < -126$	不能继续用规格化数表示，转入非规格化数处理
舍入后结果的绝对值小于 2^{-149}	结果变为 ± 0

三条容易被忽略的结论

① 对阶不会引起阶码上溢或下溢（阶码只会向大阶看齐，只增大不减小）。② 右规和尾数舍入都可能引起阶码上溢（阶码要加 1，若原本已是最大值则溢出为 ∞）。③ 左规可能引起阶码下溢；而尾数溢出不等于浮点数上溢——尾数溢出可以通过右规纠正，只有右规后阶码超过上限才是真正的上溢。

2015 年统考真题

「下列有关浮点数加减运算的叙述中，正确的是（）。Ⅰ. 对阶操作不会引起阶码上溢或下溢；Ⅱ. 右规和尾数舍入都可能引起阶码上溢；Ⅲ. 左规时可能引起阶码下溢；Ⅳ. 尾数溢出时结果不一定溢出。」四项全部正确，答案 D。这道题把上面三条结论全考了。

*7.*完整算例一：补码阶码与尾数的浮点加法（统考 *2009* 年真题型）

计算 $X = 2^7 \times \dfrac{29}{32}$ 与 $Y = 2^5 \times \dfrac{5}{8}$ 之和。已知浮点数的阶码 5 位、尾数 7 位，均含 **2** 位符号位（即变形补码：阶码 2 位符号位 + 3 位数值位，尾数 2 位符号位 + 5 位数值位）。

第 **0** 步：写出两个数的初始表示。

- 尾数：$\dfrac{29}{32} = 0.11101B$，写成 7 位尾数 00.11101；$\dfrac{5}{8} = 0.10100B$，写成 00.10100。
- 阶码：$[+7]_{\text{补}} = 00111$，$[+5]_{\text{补}} = 00101$。

① 对阶：阶差 $\Delta E = 7 - 5 = 2$，Y 的阶码小，把 Y 的尾数右移 **2** 位、阶码加 **2**：

$$[Y]_{\text{尾}} = 00.10100 \longrightarrow 00.00101, \qquad [Y]_{\text{阶}} = 00101 + 00010 = 00111\ (= +7)$$

② 尾数相加（变形补码加法，两个符号位都参与运算）：

项	双符号位尾数
$[X]_{\text{尾}}$	00.11101
$[Y]_{\text{尾}}$（对阶后）	00.00101
相加	**01.00010**

③ 规格化：结果的两个符号位是 01，表示尾数正溢出（尾数绝对值 ≥ 1），需要右规——尾数算术右移一位，阶码加 1：

$$[\text{尾}] = 01.00010 \longrightarrow 00.10001, \qquad [\text{阶}] = 00111 + 00001 = 01000\ (= +8)$$

④ 舍入：右规时尾数最低位移出的是 0，无需舍入。

⑤ 判断溢出：结果的阶码为 01000。阶码是变形补码，两个符号位为 01 说明它是正数，真值 $= +8$；而阶码的数值位只有 3 位，其最大正值是 $0111 = +7$，$+8$ 已经超界，因此阶码正溢出，结果无法表示。

最终结论：$X + Y$ 发生溢出。

用十进制核对：$X = 2^7 \times \dfrac{29}{32} = 128 \times \dfrac{29}{32} = 116$，$Y = 2^5 \times \dfrac{5}{8} = 32 \times \dfrac{5}{8} = 20$，$X + Y = 136$。而该格式能表示的最大正数约为 $2^7 \times 1 = 128$，$136 > 128$，确实溢出，与 ①~⑤ 的推导一致。

为什么「双符号位的 01」既可能是尾数溢出、也可能是阶码上溢

尾数出现 01 表示尾数相加后产生了进位，这个进位可以靠「右规」消化掉（尾数右移一位、阶码加 1）。阶码出现 01 却无法消化——阶码已经是变形补码的最高正值 $+8$（数值位只有 3 位），再加就没有位置了。所以尾数溢出可以挽救，阶码上溢才是真正的溢出。

*8.*完整算例二：*IEEE 754* 单精度浮点数的加法

计算 $0.5 + (-0.4375)$。

① 转成 **IEEE 754** 单精度格式：

$0.5 = 2^{-1} = 1.0 \times 2^{-1}$：$S = 0$，$E_{\text{字段}} = -1 + 127 = 126 = 01111110$，$f = 000\cdots0$（23 个 0），得 `3F000000H`。

-0.4375：$0.4375 = \frac{7}{16} = \frac{14}{32} = 0.01110B$，规格化得 1.11×2^{-2}：$S = 1$，$E_{\text{字段}} = -2 + 127 = 125 = 01111101$，尾数字段为 $110\cdots0$，得 `BEE00000H`。

② 对阶： $\Delta E = (-1) - (-2) = 1$，$-0.4375$ 的阶码小，尾数右移 **1** 位：

$$1.11000\cdots0 \to 0.11100\cdots0, \qquad E = -1$$

（为了后续加减，把两个操作数的尾数都写成 24 位定长形式，并把 x 的尾数也表示为 $0.f$ 形式以统一小数点位置。）

操作数	尾数（右对齐，**24** 位）
$x = 0.5$，$E = -1$	0.10000000000000000000000
$y = -0.4375$，对阶后，$E = -1$	−0.01110000000000000000000

③ 尾数相减（因为 y 为负，实际是 x 减去 $|y|$）：

$$0.10000000000000000000000 - 0.01110000000000000000000 = 0.00010000000000000000000$$

④ 规格化：结果尾数最高有效位不在规格化位置，需要左规——尾数左移 4 位、阶码减 4：

$$0.000100\cdots0 \to 1.00000000000000000000000, \qquad E = -1 - 4 = -5$$

⑤ 判断溢出与舍入：$E = -5$ 在 $-126 \sim 127$ 内，且尾数没有多余位需要舍入，结果有效。

⑥ 合成真值：

$$0.5-0.4375=1.0\times 2^{-5}=0.0625=2^{-4}$$

⑦ 写回 **IEEE 754** 格式：$S=0$，$E_{\text{字段}}=-5+127=122=01111010$，$f$ 全 0，得

$$0\ 01111010\ 00000000000000000000000=\texttt{3D800000H}$$

用 0.0625 验证：`0x3D800000` 确实是 0.0625，与 $0.5-0.4375$ 一致。

IEEE 754 加减法与补码浮点数加减法的差别

统考大题主流是补码表示阶码和尾数的浮点数加减运算（例一那种题型）。若用 IEEE 754 原码尾数做加减，尾数相减需要先做「原码减法」的符号处理，比补码尾数更繁琐。IEEE 754 的加减流程必须知道（步骤、对阶方向、隐藏位恢复、规格化后要重新加 127 还原阶码字段），但手算训练以补码浮点数为重点。

（五）浮点数加减运算的步骤汇总

步骤	做什么	关键点
① 操作数检查	判断是否为 0	为 0 则直接取另一操作数
② 对阶	求阶差，小阶向大阶看齐	小阶数的尾数右移，阶码加阶差；对阶不产生阶码上溢
③ 尾数加减	按定点小数规则运算	补码尾数符号位参与运算
④ 规格化	左规或右规	右规：尾数溢出（双符号位 01/10），移 1 位、阶码加 1；左规：符号位与最高数值位相同，逐位左移、阶码逐次减 1
⑤ 舍入	就近舍入等	只在对阶和右规时发生舍入
⑥ 判溢出	检查阶码	上溢为异常/∞，下溢按 0 或非规格化处理

本章小结表

（六）表一：各种编码对照（$n+1$ 位编码，**1** 位符号位 + n 位数值位）

编码	定点整数范围	定点小数范围	零的个数	符号位可否参与运算	用途
无符号	$0\sim 2^{n+1}-1$	—	1	无符号位	地址、编号
原码	$-(2^n-1)\sim 2^n-1$	$-(1-2^{-n})\sim 1-2^{-n}$	2	不能	浮点数尾数
反码	$-(2^n-1)\sim 2^n-1$	$-(1-2^{-n})\sim 1-2^{-n}$	2	可以（需循环进位）	转换中间形式
补码	$-2^n\sim 2^n-1$	$-1\sim 1-2^{-n}$	1	可以	计算机中的带符号整数
移码	$-2^n\sim 2^n-1$	不适用	1	可以	浮点数的阶码

（七）表二：四个标志位

标志	适用数据	判定式	为 **1** 的含义
ZF	所有	结果各位全 0	运算结果为 0
SF	有符号	$SF = F_n$（结果最高位）	有符号结果为负
OF	有符号	$OF = C_n \oplus C_{n-1}$	有符号加减溢出
CF	无符号	$CF = Sub \oplus C_{out}$	加法有进位 / 减法有借位

（八）表三：乘法与除法运算方法

方法	符号处理	迭代操作	结束条件	移位方向
原码一位乘	符号位异或，数值位相乘	依乘数最低位决定加被乘数或加 0	n 次	部分积与乘数右移
补码一位乘（Booth）	符号位自动参与	依乘数最低位与附加位决定加 X、加 $-X$、加 0	n 次	算术右移
原码除法（恢复余数/加减交替）	符号位异或	减除数，余数为负则恢复（或不恢复改为加）	n 次	余数与商左移
补码除法	符号位自动参与	依余数与除数是否同号决定加或减除数	n 次	余数与商左移

（九）表四：**IEEE 754** 关键数值（单精度）

项目	值	机器数（**16** 进制）
规格化最小正数	2^{-126}	`00800000H`
规格化最大正数	$(2-2^{-23}) \times 2^{127} = 2^{128} - 2^{104}$	`7F7FFFFFH`
规格化最大负数	-2^{-126}	`80800000H`
规格化最小负数	$-(2-2^{-23}) \times 2^{127}$	`FF7FFFFFH`
最小正非规格化数	2^{-149}	`00000001H`
最大非规格化数	$(1-2^{-23}) \times 2^{-126}$	`007FFFFFH`
$+0$ / -0	0	`00000000H` / `80000000H`
$+\infty$ / $-\infty$	—	`7F800000H` / `FF800000H`
阶码真值范围	$-126 \sim 127$	阶码字段 $1 \sim 254$

（十）表五：浮点数加减运算六步

步骤	操作	引起溢出的可能
① 检查	判 0、判特殊值	—
② 对阶	小阶向大阶看齐，小阶尾数右移、阶码加阶差	不会
③ 尾数加减	定点小数运算	尾数可能溢出（可右规挽救）

步骤	操作	引起溢出的可能
④ 规格化	左规（阶码减）或右规（阶码加）	左规可能下溢；右规可能上溢
⑤ 舍入	就近舍入等	舍入进位可能引发右规，进而上溢
⑥ 判溢出	检查阶码	上溢为异常，下溢按 0 或非规格化处理

易错清单

数制与编码

1. 机器数不带符号和小数点。写成 `-0.11`、`+1011` 都是错的，机器数是纯 0/1 序列。
2. 除基取余逆序读，乘基取整顺序读。顺序读反是高频错误。小数部分可能永远乘不完，要按题目精度截断。
3. 只有分母是 **2** 的幂的十进制小数才能被二进制精确表示。0.3、0.7、1.2 都是无限循环，据此判断「能否被 IEEE 754 精确表示」。
4. 原码、反码有两个零，补码、移码只有一个零。补码能多表示一个最小负数 -2^n，原码和反码不能。
5. 求 $[-y]_{补}$ 要连同符号位一起取反加一，不能只对数值位操作。
6. 移码与补码的关系是「符号位取反」，且移码只用于表示定点整数。注意 IEEE 754 阶码的偏移量是 $2^{n-1}-1$，不是定点移码常用的 2^n。
7. 位权法是算补码真值最快的方法：只有最高位（符号位）的位权是负的，为 -2^n。
8. 定点整数在高位扩展，定点小数在低位扩展。

加减运算与标志位

1. OF 管有符号，CF 管无符号，两者不能互换。每次加减运算两个标志都会同时产生，该看哪个由数据类型决定。
2. $OF = C_n \oplus C_{n-1}$（最高位进位异或次高位进位），不是 C_n 本身，也不是 $C_n \oplus Sub$。
3. $CF = Sub \oplus C_{out}$：减法时 $CF = \overline{C_{out}}$，即「最高位没有进位」才表示借位。
4. 同号相加、异号相减才可能溢出；溢出时结果符号与被加数符号相反。
5. 双符号位 01 是正溢出，10 是负溢出，最高位符号位代表真正的结果符号。
6. SF 对无符号数无意义，CF 对有符号数无意义，但不会报错——只是判据错。
7. 有符号比较大小要用 $SF \oplus OF$，不能只看 SF；无符号比较大小只看 CF 和 ZF。
8. 左移可能溢出（符号位改变即为溢出），算术右移不会溢出。算术右移是向 $-\infty$ 取整，$-7 >> 1 = -4$ 而不是 -3。

乘法与除法

1. 乘法用右移，除法用左移，两个方向不要记反。
2. **Booth** 算法的附加位初值必须为 **0**，判断的是「乘数最低位与附加位」两位，先加减后移位，移位是算术右移。
3. 无符号乘法不溢出的判据是乘积高 n 位全 **0**；有符号乘法不溢出的判据是高 $n+1$ 位全 **0** 或全 **1**（即高位是低位的符号扩展）。
4. 用 $2n$ 位保存乘积一定不溢出，只有截取低 n 位时才需要判溢出。
5. 除法开始前必须检查除数是否为 **0**、被除数绝对值是否小于除数，否则商溢出；除零会触发异常。

整数表示与运算

1. 同宽度的有符号/无符号互转，机器数不变，只改变解释方式。例如 $FFFFH$ 按 `short` 是 -1，按 `unsigned short` 是 65535。

2. 窄类型 → 宽类型看源类型：有符号做符号扩展，无符号做零扩展。
3. 宽类型 → 窄类型直接截断高位，真值可能完全变化（如 32777 截成 `short` 后是 -32759）。
4. 有符号与无符号混合运算，编译器把有符号数转换成无符号数，比较运算尤其危险（`-1 > 1u` 为真）。
5. `i <= n-1` 当 n 为无符号 **0** 时永真，会造成死循环。

浮点数

1. 规格化数的隐含位是 **1**，非规格化数的隐含位是 **0**，两者的尾数形式分别是 $1.f$ 和 $0.f$。
2. 非规格化数的阶码真值固定为 -126（双精度 -1022），不是 $0-127$。
3. 阶码字段全 **0** 且尾数全 **0** 是 ± 0；阶码全 **1** 且尾数全 **0** 是 $\pm\infty$；阶码全 **1** 且尾数非 **0** 是 `NaN`；阶码全 **0** 且尾数非 **0** 是非规格化数。四类要一一对应，不要混淆。
4. 阶码字段要减 **127**（**1023**）才是真指数，写完真值后要记得再加回来还原字段。
5.「最大负数」绝对值最小、「最小负数」绝对值最大，中文读题必须在数轴上确认。
6. 对阶一定不会引起阶码上溢或下溢；右规和舍入可能上溢，左规可能下溢。
7. 尾数溢出不等于浮点数上溢，尾数溢出可以由右规纠正，只有阶码越界才是真溢出。
8. 对阶一定是小阶向大阶看齐，反过来会造成高位有效数字丢失。
9. 规格化的补码尾数是「符号位与最高数值位相反」，原码尾数是「最高数值位为 1」，两种判据不能混用。

第十章存储器层次结构

本章考纲要求

> 第二部分计算机组成原理三、存储器层次结构（一）存储器的分类（二）层次化存储器的基本结构（三）半导体随机存取存储器
>
> 1. SRAM 存储器
> 2. DRAM 存储器
> 3. Flash 存储器
>
> （四）主存储器
>
> 1. DRAM 芯片和内存条
> 2. 多模块存储器
> 3. 主存和 CPU 之间的连接
>
> （五）外部存储器
>
> 1. 磁盘存储器
> 2. 固态硬盘（SSD）
>
> （六）高速缓冲存储器（Cache）
>
> 1. Cache 的基本原理
> 2. Cache 和主存之间的映射方式
> 3. Cache 中主存块的替换算法
> 4. Cache 写策略
>
> （七）虚拟存储器
>
> 1. 虚拟存储器的基本概念
> 2. 页式虚拟存储器：基本原理，页表，地址转换，TLB（快表）
> 3. 段式虚拟存储器的基本原理
> 4. 段页式虚拟存储器的基本原理

存储器层次结构是计组分值最重、题型最全的一章：选择题每年必考 2～4 道，应用题（第 43 题）几乎每隔一两年就落在这里。它的特点是概念题靠背、计算题靠公式、设计题靠流程，三块都不能瘸腿。

本章的主线只有一条：**CPU** 越来越快，存储器跟不上，于是用「层次 + 局部性」把它们拼成一个整体。所有具体技术都是这条主线的产物——Cache 是为了弥合速度差，虚拟存储器是为了弥合容量差，多模块交叉是为了让主存吞吐率跟上总线，SSD 是为了干掉寻道时间。抓住这条主线，本章的每个公式都能自己推出来，不用死背。

本手册约定：地址一律按字节编址讨论（题目另行说明时以题目为准）；公式中的「行」与「块」分别指 Cache 行（Cache Line）与主存块（Block）。

一、存储器的分类

（一）按存储介质分类

存储元件必须有两个截然不同的物理状态才能表示 0 和 1。按介质分三类：

介质	典型器件	说明
半导体存储器	RAM、ROM、寄存器、Cache、SSD、U 盘	分 MOS 型与双极型两类；速度最快
磁表面存储器	磁盘（HDD）、磁带	靠磁头与磁介质相对运动做电磁转换；非易失；速度慢
光存储器	CD-ROM、DVD-ROM	靠激光在盘面凹坑上读取；非易失

SSD 不是磁盘，CD-ROM 不是 ROM

这是两个一字之差、考频极高的概念：

- 固态硬盘 **SSD** 是半导体存储器，基于闪存 Flash 技术，本质属于电可擦除 ROM（EEPROM）。真正的「磁盘」是机械硬盘 HDD。
- **CD-ROM** 是光存储器（属于辅存），不是计算机术语里那个作为半导体芯片的 ROM。CPU 可以直接从 BIOS ROM 中取指执行，但绝不可能直接执行 CD-ROM 里的内容——辅存必须先调入主存。

（二）按存取方式分类

类型	全称	存取时间特点	典型器件
随机存取 RAM	Random Access Memory	与地址无关，是常数	内存、Cache
顺序存取 SAM	Sequential Access Memory	取决于信息所在位置；以「记录块」为单位编址	磁带
直接存取 DAM	Direct Access Memory	先直接定位到区域，再在区域内顺序查找	磁盘、光盘
相联存取 AM/CAM	Associative / Content Addressed Memory	按内容检索，不需知道地址	快表 TLB、全相联 Cache 的标记比较

三个反直觉结论

1. **ROM** 和 **RAM** 都采用「随机存取」方式。虽然习惯上把 ROM 与 RAM 并列，但 ROM 读取任一单元的时间同样与物理位置无关——都是地址译码后定位。区别只在于 ROM 存储区在程序执行中只能读，不能随意改写。
2. 并非所有半导体存储器都按地址访问。全相联 Cache 的比较器、快表 TLB 是按内容访问（CAM）的，所以「半导体存储器都采用随机存取方式」是错的。
3. 光盘（**CD-ROM**）属于直接存取，不是随机存取，也不是顺序存取。

（三）按信息可更改性与易失性分类

- 可读可写存储器：RAM 芯片。

- 只读存储器：ROM 芯片。名称属于历史遗留，现代 ROM（E^2PROM、Flash）都能重写，只是写入慢、不能随意写。
- 易失性存储器：断电后信息消失。RAM、Cache。
- 非易失性存储器：断电后信息保持。ROM、磁表面存储器、光存储器。

（四）按作用与存取速度分类

层次	实现器件	位置与特点
寄存器存储器	触发器	CPU 内部，字长与机器字长相同；速度与 CPU 匹配，容量极小
高速缓冲存储器 **Cache**	**SRAM**	CPU 与主存之间；现代集成于 CPU 内；容量小、价格高
主存储器 **MM**	主要用 **DRAM**	CPU 可直接访问；可与 Cache、辅存交换数据
辅助存储器	磁盘、SSD	**CPU** 不能直接访问；容量极大、速度慢、位价格低

主存不等于「全是 DRAM」

现代 PC 的主存由 **RAM** 区 + **ROM** 区组成：RAM 区存放运行中的程序和数据，由 DRAM 实现；ROM 区存放 BIOS 等固化程序，由 ROM 芯片实现。所以「系统的主存都用 DRAM 芯片实现」是错误的。某些嵌入式系统的主存甚至全由 ROM 元件构成。

（五）存储器的性能指标

1.存取时间与存取周期

- 存取时间（**Access Time**）T_A：启动一次存储器操作到该操作真正完成所经历的时间。读、写时间可能不同（DRAM 读慢写快，闪存读快写慢）。
  - 读出时间：从收到有效地址和读命令，到数据稳定地送上数据总线。
  - 写入时间：从收到有效地址、数据和写命令，到数据被可靠写入。
- 存取周期（**Memory Cycle Time**）T_C：存储器进行两次独立的、连续的存取操作之间的最小时间间隔。它关注的是连续工作的吞吐能力。

$$T_C = T_A + T_R$$

其中 T_R 为恢复时间（**Recovery Time**）。DRAM 是破坏性读出，读出后必须把数据重写回去并让位线电压复位；地址译码器、读写控制逻辑翻转后也需要时间稳定，这些就是恢复时间的来源。

存取周期 > 存取时间，用哪个要看清题目

- 问「读一次要多久」「单次访问延迟」→ 用存取时间 T_A。
- 问「连续读取的最小时间间隔」「最大带宽」「流水线启动下一体的间隔」→ 必须用存取周期 T_C。

2.存储器带宽

存储器带宽（**Bandwidth**）：单位时间内存储器能传输的连续最大数据量，单位 B/s、MB/s、GB/s。

$$B_m = \frac{W}{T_C}$$

W 为数据位宽（字节数），T_C 为存取周期。

算例：某主存存取周期 50 ns、存取时间 40 ns、数据总线宽 64 位。

- 位宽 $W = 64/8 = 8$ B；连续访问的最小间隔受存取周期约束，取 $T_C = 50$ ns。
- $B_m = 8\ \text{B}/(50 \times 10^{-9}\ \text{s}) = 160 \times 10^6\ \text{B/s} = 160\ \text{MB/s}$。
- 题干中的「存取时间 40 ns」是干扰项。

提高带宽有两条路：缩短 T_C（提高工作频率，如 DDR 代际演进）与增加 W（双通道/四通道内存，位宽翻倍）。若 W 固定，则带宽的极限就是总线的传输能力，与存储器内部能并行准备多少个字无关。

（六）存储器的层次化结构

*1.*为什么需要分层

理想存储器要同时满足容量大、速度快、价格低，但任何单一技术都做不到：SRAM 快而贵且小，磁盘大而便宜却慢，且 CPU 性能提升速度长期快于存储器，形成「性能鸿沟」。分层存储把不同速度、容量、成本的器件组合成金字塔，使整个系统速度接近最高层、容量和单位成本接近最低层。

从 CPU 往外，自上而下：寄存器 → Cache → 主存 → 辅存（磁盘/SSD）→ 远程存储。四条规律：越往下，访问速度越慢、容量越大、位价格越低、被访问频次越低。寄存器、Cache、主存统称内部存储器，CPU 可直接访问；辅存及以下为外部存储器，信息必须先调入主存。

*2.*访问与调度规则

- 子集副本原则：上一层内容是下一层内容的副本。Cache 中的数据在主存中，主存中的数据在辅存中（采用写回策略时，Cache 中的新数据可能暂时还不在主存里）。
- 逐层查找原则：寄存器 → Cache → 主存 → 辅存。Cache 未命中访主存，主存没有访辅存。
- 数据传送只在相邻两层之间进行，单位是定长块。辅存中找到的数据不能直接送 CPU，必须经历：辅存 → 主存 → Cache → 寄存器/CPU。
- 特例：CPU 可以和主存直接交换信息（绕过 Cache）；主存是与 Cache、辅存同时打交道的枢纽。

*3.Cache-*主存层次与主存-辅存层次

对比维度	**Cache-**主存层次	主存-辅存层次
位置与访问权限	最靠近 CPU，CPU 可直接访问两者	CPU 不能直接访问辅存
引入目的	解决速度问题	解决容量问题，形成虚拟存储器
交换单位	主存块 **Block**，通常 8 ~ 128 B	页 **Page**，通常 4 KB ~ 64 KB
缺失处理	硬件自动完成	操作系统（软件）完成
映射方式	直接、全相联、组相联均可	一律全相联
映射记录介质	Cache 行的 Tag 字段	主存中的页表
写策略	直写或回写均可	一律回写
透明性	对所有程序员透明	对应用程序员透明，对系统程序员不透明

两者有四点共性：都基于程序访问的局部性；都要解决地址映射；都要解决替换；缺失时都要把包含目标信息的块从慢速层调入快速层。

缺页代价远大于 Cache 缺失，所以页大、全相联、只回写

主存比 Cache 慢 10 ~ 100 倍，辅存比主存慢约 10000 倍。所以：页面远大于主存块（4 KB 对 32 B）；主存-辅存层次必须用全相联把冲突缺失降到最低；写操作只能回写——每次改主存都同步写磁盘是不可承受的。

（七）程序访问的局部性原理

在较短时间间隔内，程序产生的地址往往集中在存储空间的很小范围内，这叫程序访问的局部性，是分层存储能生效的全部前提。

- 时间局部性（**Temporal Locality**）：某个存储单元在被访问后，短时间内很可能再被访问。来源：循环体、循环变量、高频调用的函数。
- 空间局部性（**Spatial Locality**）：某个存储单元的邻近单元在短时间内很可能被访问。来源：顺序执行的指令、数组的连续元素。

步长 k：访问连续数组时每隔 k 个元素访问一次。步长为 1 即顺序引用模式，是空间局部性最重要的来源；步长越大，空间局部性越差。

三条速判原则

1. 变量重复原则：对同一变量有重复访问 → 时间局部性好。
2. 步长最小原则：步长越小 → 空间局部性越好，步长为 1 最佳。
3. 循环体原则：循环体一般时间和空间局部性都好；循环体越小、迭代次数越多，局部性越好。

算例（一维数组遍历）

```
for (i = 0; i < N; i++)
    sum += v[i];
```

- 标量 `sum`、`i`：每次迭代都被引用 → 时间局部性极好；单变量，讨论空间局部性无意义。
- 数组元素 `v[i]`：按存储顺序逐个读取，步长为 1 → 空间局部性极好；但每个元素只访问一次 → 时间局部性差。

算例（行优先 vs 列优先二维数组）

数组按行优先存放，`M = N = 2048`，每元素 4 B。按行遍历 `a[i][j]`（内层 j 递增）时步长为 1，每取一个元素都能顺带命中同块内后续元素，空间局部性好；按列遍历（内层 i 递增）时相邻访问间隔 $N \times 4 = 8192$ B，远超一个 Cache 块，每次访问几乎都缺失，空间局部性极差。同样一份数据、同样的计算量，仅交换循环次序就可能让 Cache 缺失率相差十倍以上。

局部性的统考考法

【2017-14】给出嵌套循环 `for(i=0..9) { temp=1; for(j=0..i) temp*=a[j]; ... }`，问数组 a 的访问局部性。答案是时间局部性与空间局部性皆有：内层循环顺序访问 `a[0..i]` 步长为 1（空间局部性好），且靠前的元素被外层循环反复重复访问（`a[0]` 访问 10 次、`a[1]` 访问 9 次，时间局部性好）。做题时要把「重复访问 → 时间」「按序访问 → 空间」两条分别对着代码数一遍，不要只凭感觉。

二、半导体随机存取存储器

（一）半导体存储芯片的内部结构与译码方式

一个半导体存储芯片由存储矩阵、译码驱动电路、读写控制电路三部分组成，并对外引出四类信号线：

信号线	方向	作用
地址线	单向（入）	送入地址，经译码器选中单元
数据线	双向	读出数据或写入数据
片选线 CS	单向（入）	选中该芯片；多片组成系统时用于字扩展
读写控制线 WE	单向（入）	下达读/写命令

相关术语：记忆单元（存储元/位元）是能表示 0/1 的物理器件；存储单元是具有相同地址的一组位，即一个编址单位；存储体（存储矩阵/阵列）是所有存储单元的集合。

*1.*单译码结构（线选法）

n 位地址直接送入一个 n–2^n 译码器，每根输出线作为一条字线，被选中的字线激活整行存储单元，其上的所有位构成一个「字」被同时读写。这种芯片称为字片式芯片。

致命缺陷：地址位数多时输出线爆炸。例如 $n = 16$ 时需要 16–65536 译码器，输出 65536 根字线，硅片上布线密集、寄生电容极大、良率骤降，物理上不可实现。所以大容量芯片不用一维单译码。

*2.*双译码结构（重合法）

地址译码器分成 X、Y 两组。以 4096 个单元、12 位地址为例：$A_0 \sim A_5$ 送 X 译码器产生 64 条行选择线，$A_6 \sim A_{11}$ 送 Y 译码器产生 64 条列选择线，行列交叉唯一确定一个单元。若地址为 `000000000001`，则 X 译码器第 2 根输出线 x_1 有效、Y 译码器第 1 根输出线 y_0 有效，选中单元 $(1, 0)$。

被选中的行列交叉点上只有一位，所以这种芯片称为位片式芯片。若存储阵列采用三维结构（多个位平面），则不同位平面在同一行列交叉点上的多位构成一个存储字，被同时读写——这本质上就是位扩展。

$$\text{双译码结构译码输出线总数} = \text{行地址线数} + \text{列地址线数}$$

单译码 vs 双译码

- 单译码（线选）：输出线 $= 2^n$，线多、速度快，只适合小容量。
- 双译码（重合）：输出线 $= 2^{n/2} \times 2$，线少，是大容量芯片的必然选择。
- 记忆锚点：双译码的译码输出线总数 = 行地址线数 + 列地址线数，而不是二者乘积，更不是 2^n。

（二）基本存储元件

存储单元电路必须具备四个条件：两种稳定状态；两状态可由外部信号控制互相转换；能被读出；无外部原因时信息能长期保存。

1.SRAM：六管静态 *MOS* 存储元件

6 个 MOS 管中，4 个构成一对正负反馈门电路，即双稳态触发器；另外 2 个像「守门员」，控制数据进出。只要不断电，互相拉扯的平衡状态就永远保持，不需要任何额外动作维持——这就是「静态」的含义。

- 优点：速度快（直接读触发器状态）；不用刷新；读写简单、控制逻辑少。
- 缺点：面积大（6 个管子）、集成度低、成本高、功耗较大。
- 应用：**Cache（L1/L2/L3）**、DRAM 的行缓冲。

2.DRAM：单管动态 *MOS* 存储单元

只由 **1** 个 **MOS** 管 **+ 1** 个电容 C_S 构成，用电容上有无电荷表示 1/0。写 1 给电容充电，写 0 放电。读出时 C_S 上的电荷被稀释放电，电位下降，所以是破坏性读出，读后必须重写（再生）。电容会漏电，几毫秒内电荷就漏光，所以必须定期刷新。

- 优点：面积极小、集成度高、成本低、静态功耗小。
- 缺点：速度慢（读写要充放电）；必须刷新；断电或不刷新即丢数据。
- 应用：主存（**DDR** 内存条）、显存。

特性	**SRAM**（六管）	**DRAM**（单管）
存储原理	双稳态触发器（电路自锁）	电容充放电（保存电荷）
核心元件	6 个 MOS 管	1 个 MOS 管 + 1 电容
读出后是否需重写	不需要	需要（破坏性读出）
是否需要刷新	不需要	必须定期刷新
读写速度	快（纳秒级）	较慢（需充放电）
集成度	低	高
成本 / 功耗	高 / 较大	低 / 小
典型应用	Cache	主存、显存

SRAM 与 DRAM 的三组混淆点

1. 「静态/动态」指的是维持状态的方式：静态靠电路自锁不需干预，动态靠电荷会衰减、需动态刷新。不是说 SRAM 的数据不变。
2. 两者都是易失性的。只是 DRAM 更「脆弱」——不断电但停止刷新同样会丢数据。
3. **DRAM** 的「读操作」和「刷新操作」不是一回事。刷新本质上是一次没有第 **5** 步（数据输出）的读操作：行地址由芯片内部刷新计数器给出，不发列地址，不向 CPU 输出数据。读操作则必须给行、列地址并把数据送上数据总线。

（三）DRAM 的刷新

*1.*刷新的基本规则

- 按行进行：每次刷新的单位是一行，该行所有列同时完成「破坏性读出 + 内部重新充电恢复」。因此刷新只需要行地址，不需要列地址。
- 占用一个存取周期：刷新一行的耗时等于一次正常读写的时间，记为 t_c。
- 地址由内部计数器生成：刷新行地址不是 CPU 提供的，而是 DRAM 芯片内部（或内存控制器）

的刷新地址计数器自动产生并逐行加 1。芯片内部有一个多路选择器 MUX，正常读写时切到外部行地址缓冲器，刷新周期时切到刷新计数器，屏蔽外部地址。

- 与 **CPU** 争用：刷新期间不能响应 CPU 的读写请求，这就产生了「内存争用」问题。
- 优化方向：为减少刷新总开销，芯片设计趋向减少行数、增加列数（如 128 行 ×2048 列而非对称矩阵），因为刷新开销只与行数有关。

*2.*三种刷新方式

统一算例：DRAM 为 128 行 ×128 列，读写周期 $t_c = 0.5\ \mu$s，最大刷新间隔 2 ms。

2 ms 内需完成全部 128 行刷新；$2\ \text{ms}/0.5\ \mu\text{s} = 4000$ 个时隙，从中选出 128 个时隙，每时隙刷新一行即可。

方式	原理	优点	缺点
集中刷新	前 $4000 - 128 = 3872$ 个周期全部用于读写，最后 128 个周期集中刷新所有行	读写期间不受刷新打扰，存储器速度快	存在长时间死区，期间 CPU 完全无法访存；行数越多死区越长
分散刷新	把存取周期拉长一倍：前半段读写或保持，后半段强制刷新一行，$t_s = 0.5 + 0.5 = 1\ \mu$s	彻底消除集中死区	刷新过于频繁，严重拖慢系统速度（每 1 μs 刷新一行，2 ms 内刷新约 15 遍，远超必要）
异步刷新	把 128 行均匀平摊到 2 ms 内：相邻两行刷新间隔 $= 2\ \text{ms}/128 = 15.625\ \mu$s，每次耗时 0.5 μs	既保证每行在 2 ms 内刷新一次，又把死区打散成 0.5 μs 的微小停顿	需要刷新定时逻辑；现代内存最常用的方式

刷新计算三个必背量

设芯片有 r 行，存取周期 t_c，最大刷新间隔 T_{max}：

- 刷新总次数 $= r$；刷新总耗时 $= r \cdot t_c$。
- 集中刷新死区时间 $= r \cdot t_c$，死区率 $= \dfrac{r \cdot t_c}{T_{\text{max}}}$。
- 分散刷新下存取周期变为 $2t_c$，有效访存速度降为原来的一半。
- 异步刷新下相邻两行刷新间隔 $= \dfrac{T_{\text{max}}}{r}$，每次停顿 t_c。

统一算例：$r = 128$，$t_c = 0.5\ \mu$s，$T_{\text{max}} = 2$ ms。集中刷新死区 $= 128 \times 0.5 = 64\ \mu$s，死区率 $= 64/2000 = 3.2\%$；异步刷新间隔 $= 2000/128 = 15.625\ \mu$s。

刷新与引脚数的统考考法

【2015-17】工作期间需要刷新的是（）。答案 **B. SDRAM**——凡基于电容存储的 DRAM 家族（含 SDRAM、DDR）都需要刷新；SRAM、ROM、Flash 都不需要。【2018-17】2K × 1 位 DRAM，要求地址引脚数最少且刷新开销尽量小，求行数 r、列数 c。答案 **C.（32，64）**。推导分两步：① DRAM 地址复用，地址引脚数 $= \max(\log_2 r, \log_2 c)$，要使它最小须让 r、c 尽量接近，$2\text{K} = 2048 = 2^{11}$，故 $r \cdot c = 2048$，选项 B（64×32）与 C（32×64）的引脚数都是 6，满足「最少」；② 刷新开销只与行数有关，r 越小越好，故选 $r = 32$。

（四）**DRAM** 的地址复用

DRAM 芯片引脚数直接决定封装成本，所以采用行列地址分两次传送：同一组地址引脚先接收行地址（由 $\overline{RAS}$ 行地址选通信号锁存），再接收列地址（由 $\overline{CAS}$ 列地址选通信号锁存）。

$$\text{逻辑地址位数} = \log_2(\text{存储单元数}) = \text{行地址位数} + \text{列地址位数}$$

$$\text{物理地址引脚数} = \max(\text{行地址位数}, \text{列地址位数}) \approx \left\lceil \frac{\log_2 M}{2} \right\rceil$$

其中 M 为芯片的存储单元数（字数）。

算例 **1**：4M × 8 位 DRAM 芯片的地址引脚与数据引脚总数是多少?

- 逻辑地址位数 $= \log_2(4\text{M}) = \log_2(2^{22}) = 22$ 位。
- 复用后物理地址引脚 $= 22/2 = 11$ 根（行地址 11 位 + 列地址 11 位分时复用同一组 11 根引脚）。
- 数据引脚 $= 8$ 根（芯片位宽为 8）。
- 总数 $= 11 + 8 = 19$ 根。（本题即【2014-15】，答案是 19。）

算例 **2**：256K × 8 位 DRAM 芯片。

- 逻辑地址 $= \log_2(2^{18}) = 18$ 位 $\rightarrow$ 物理地址引脚 $18/2 = 9$ 根。
- 数据引脚 8 根。总数 $= 17$ 根。

算例 **3**：8192 × 8192 × 8 位 DRAM 芯片。

- 逻辑地址 $= 13 + 13 = 26$ 位；但行列已明确各 13 位，故物理地址引脚 $= \max(13, 13) = 13$ 根（不是 26 根！）。
- 数据引脚 8 根，总数 $= 21$ 根。芯片内行缓冲容量 $= 8192 \times 8$ 位（激活一整行时导出到读出放大器的数据量）。

DRAM 引脚计算的三个坑

1. 只要题目出现「**DRAM**」，地址引脚就必须除 **2**（或取行列最大值）；SRAM 不复用地址，引脚数就是完整逻辑地址位数。4M × 8 的 SRAM 需要 22 根地址引脚，DRAM 只需 11 根。
2. 问的是「芯片」的引脚数，与系统总容量无关。「256MB 存储器由若干 4M × 8 位 DRAM 芯片构成」这个总容量只用于算芯片片数，不影响单片引脚数。
3. 行列不等时要取最大值。32 × 64 的阵列，行地址 5 位、列地址 6 位，复用后引脚数是 $\max(5, 6) = 6$ 根，而不是 $11/2 = 5.5 \rightarrow 6$ 以外的任何数。

（五）只读存储器 ROM 与 Flash 存储器

1.ROM 的演进

类型	写入方式	擦除方式	特点
MROM（掩膜式）	厂家制造时写入，固定掩膜	不可擦除	内容出厂固定，可靠性高，灵活性最差
PROM（可编程）	用户用编程器写入（熔丝/反向二极管击穿）	不可擦除	只能一次性编程，写错即报废
EPROM（可擦除可编程）	编程器写入，浮置栅 MOS 管	紫外线照射，约 8～20 分钟	可多次修改，但必须整片擦除

类型	写入方式	擦除方式	特点
EEPROM / E^{2}PROM（电可擦除）	加电写入	电擦除	可精确到字节删除/重写，写入时间远长于读取
Flash 闪存	在线擦除和重写（写命令到命令寄存器）	电擦除	以「块 **Block**」为基本单位擦除；读快写慢
SSD 固态硬盘	基于闪存芯片阵列	电擦除	广义属于 ROM，但已无「只读」限制

记忆口诀：名字带 **E** 的都能改写（E 代表 Erasable）；有两个 **E**（**EEPROM**）说明用最方便的电来擦除。擦除粒度：EPROM 整片 > Flash 按块 > EEPROM 按字节。

2.Flash 存储器的特点

Flash 兼有 RAM 和 ROM 的优点：高密度、非易失、功耗低、集成度高、不需后备电源。它沿用 EPROM 的浮栅/热电子注入写入方式，又具备 E^{2}PROM 的可擦除特点，因此又叫快擦型电可擦除重编程 **ROM**。常见应用：BIOS、U 盘、存储卡、SSD。

工作方式有读、编程、擦除、功耗下降四种，编程与擦除通过写命令到命令寄存器来管理。

Flash 最核心的一条：读快写慢，且写前必须先擦

- 读直接感应浮栅 MOS 管的导通/截止，速度与半导体 RAM 相当（纳秒级）。
- 写必须先擦除（按块施加高压使电子隧穿），再编程（注入电子），速度与硬盘相当（微秒～毫秒级）。
- 因此「信息可读可写，并且读写速度一样快」是错误的表述（【2012-16】的答案就是这一句）。
- 擦除单位是块，块内所有页一起被擦；擦干净的块中每页可写一次、读无限次。

3.NOR Flash 与 *NAND Flash*

维度	**NOR Flash**	**NAND Flash**
读出方式	随机存取，可直接按地址取任一字节	按页读出，随机存取粒度粗
读出速度	快，支持片内执行（XIP）	较慢，但顺序/页读吞吐高
写入与擦除速度	慢	快
集成度与位价格	低、贵	高、便宜
耐久性（擦写次数）	约 $10^4 \sim 10^5$ 次	早期约 $10^3 \sim 10^4$ 次，现代 3D NAND 显著提高
典型用途	存放固件代码（BIOS、嵌入式 boot）	U 盘、SSD、存储卡等大容量存储

Flash 的擦写寿命与「先擦后写」

Flash 的每个块可擦除次数有限，反复写同一个块会把它写坏（机械硬盘的扇区不会因为写得多而损坏）。此外，要写某页信息时必须先擦除该页所在的整块；若该块内其他页有数据，得先把它们复制到一个擦除过的新块中，再写入目标页。这就是 SSD 写放大与需要「垃圾回收 + 磨损均衡」的根本原因。

三、主存储器

（一）DRAM 芯片与内存条

现代内存条由多片 DRAM 芯片按低位交叉编址 + 同时启动的方式组装，并配以 64 位（或更宽）的存储器总线。

典型结构算例：用 8 片 16M $\times$ 8 位 DRAM 芯片扩展成一个 128 MB 内存条，按字节编址。每片内部是 $4096 \times 4096 \times 8$ 位阵列，行列地址各 12 位（$2^{12} = 4096$），含 8 个位平面。

- 主存地址共 27 位（128 MB $= 2^{27}$ B），高 24 位为行、列地址，低 3 位用于选择 8 片中的哪一片。
- 读数据时，高 12 位行地址同时送到 8 片芯片，把对应行读入各自的行缓冲；再把列地址同时送到 8 片，各取 8 位，组合成 **64** 位送上总线。
- CPU 再根据主存地址低 3 位，从存储控制器缓冲区中取出自己需要的那一个字节。

这一结构直接解释了数据对齐的必要性：一次访存固定取出第 $0 \sim 7$、$8 \sim 15$、……、$8k \sim 8k+7$ 号单元。一个 `int`（4 B）若从地址 **8** 开始存放（对齐，起始地址是 4 的倍数），只需访问主存 **1** 次；若跨在地址 **6**、**7**、**8**、**9** 上（不对齐），则跨越了两个 8 字节块，需访问 **2** 次。

（二）存储器容量的扩展

当单片芯片的字数或字长不够时，需要在位（字长）和字（地址空间）两个方向扩展。

$$\text{位扩展片数} = \frac{\text{目标数据位宽}}{\text{单片数据位宽}}, \qquad \text{字扩展组数} = \frac{\text{目标地址空间}}{\text{单片地址空间}}$$

$$\text{总片数} = \text{位扩展片数} \times \text{字扩展组数} = \frac{\text{目标总容量}}{\text{单片容量}}$$

以「目标 $M \times N$ 位，单片 $m \times n$ 位」为通用记号：

$$\text{位扩展片数} = \frac{N}{n}, \qquad \text{字扩展组数} = \frac{M}{m}, \qquad \text{总片数} = \frac{N}{n} \times \frac{M}{m} = \frac{M \times N}{m \times n}$$

*1.*位扩展法

适用：芯片数据位宽 < 目标数据位宽（地址空间已够）。目的：加宽字长，地址空间不变。

信号线	连接方式
地址线	全部并联，接系统地址总线
片选 CS	全部并联，同时选中
读写控制线	全部并联
数据线	各芯片单独引出，分别接数据总线的不同位

核心特征：所有芯片同时工作，共同提供一个完整的字。

算例：用 4 片 4K $\times$ 1 位 SRAM 构成 4K $\times$ 4 位存储器。

- 位扩展片数 $= 4/1 = 4$ 片；地址空间不变，仍为 4K，需 12 根地址线 $A_0 \sim A_{11}$。
- 4 片地址线并联、片选并联、WE 并联；第 1 片的数据线接 D_0，第 2 片接 D_1，第 3 片接 D_2，第 4 片接 D_3。

*2.*字扩展法与片选方式

适用：芯片字数 < 目标字数（位宽已够）。目的：扩大地址空间，每单元位数不变。

信号线	连接方式
地址线低位	全部并联，作片内寻址
地址线高位	接译码器（或直接接片选），作片间选择
数据线	全部并联
读写控制线	全部并联
片选 CS	各芯片独立，由高位地址译码后分别控制

核心特征：各芯片分时工作，同一时刻只有一片被选中。

片选方式一：线选法

取字选之外的高位地址线直接接到各芯片的 CS 端。规定某根高位地址线为 0 时选中对应芯片，其余高位必须为 1。

- 优点：不需译码器，电路简单。
- 缺点：地址空间不连续，存在大量空洞；片选线必须满足「每次只有一位有效」。

算例：用 4 片 8K × 8 位 SRAM 组成 32K × 8 位，采用线选法。

- 字扩展 4 片，数据位宽已满足。芯片地址线 13 根（$2^{13} = 8\text{K}$）。
- 若只用 4 根高位线 $A_{13} \sim A_{16}$ 分别接 4 片的 CS，且要求同一时刻只有一根为低，则 CPU 需要 17 根地址线。
- 各片地址范围（地址不连续）：

芯片	高位 $A_{16}A_{15}A_{14}A_{13}$	最低地址	最高地址
第 1 片	111**0**	1110 0000 0000 0000	1110 1111 1111 1111
第 2 片	11**0**1	1101 0000 0000 0000	1101 1111 1111 1111
第 3 片	1**0**11	1011 0000 0000 0000	1011 1111 1111 1111
第 4 片	**0**111	0111 0000 0000 0000	0111 1111 1111 1111

片选方式二：译码片选法（主流）

把高位地址线送入地址译码器，由译码器输出产生片选信号。

- 优点：地址空间连续，充分利用地址资源。
- 缺点：需要译码器，电路略复杂。

算例：用 4 片 8K × 8 位 SRAM 组成 32K × 8 位，采用译码片选法。

- CPU 需 15 根地址线（$2^{15} = 32\text{K}$）、8 根数据线；芯片 13 根地址线、8 根数据线。
- 高位地址线 $= 15 - 13 = 2$ 根（A_{14}、A_{13}），接 **2-4** 译码器。
- $A_0 \sim A_{12}$ 并联到 4 片芯片作片内寻址；译码器输出 0/1/2/3 分别接 4 片的 CS。

芯片	$A_{14}A_{13}$	地址范围（十六进制）
第 1 片	00	0000H ~ 1FFFH
第 2 片	01	2000H ~ 3FFFH
第 3 片	10	4000H ~ 5FFFH
第 4 片	11	6000H ~ 7FFFH

*3.*字位同时扩展法

适用：字数和字长都不够。分两步走：

```
第一步：先按【位扩展】把若干芯片组成一个" 逻辑组"，满足数据位宽要求
第二步：再按【字扩展（译码片选)】把多个" 逻辑组" 连接，满足地址空间要求
```

- 组内：地址线、片选线、控制线并联，数据线按位独立引出（位扩展特征）。
- 组间：片选由高位地址译码后各自控制，同一时刻只有一组工作（字扩展特征）。

算例：用 8 片 8K × 4 位 RAM 构成 32K × 8 位存储器。

- 位扩展片数 $= 8/4 = 2$ 片一组（两片拼成 8K × 8 位）。
- 字扩展组数 $= 32\text{K}/8\text{K} = 4$ 组。
- 总片数 $= 2 \times 4 = 8$ 片 ✓。
- 组内：芯片 ①② 为第 1 组，地址线、CS、WE 并联，① 的数据线接 $D_0 \sim D_3$、② 接 $D_4 \sim D_7$。
- 组间：CPU 高位 A_{14}、A_{13} 接 2-4 译码器，4 个输出分别控制 4 组 CS；$A_0 \sim A_{12}$ 并联到所有芯片。

各组地址分配：第 0 组 0000H ~ 1FFFH，第 1 组 2000H ~ 3FFFH，第 2 组 4000H ~ 5FFFH，第 3 组 6000H ~ 7FFFH。

三种扩展方式对比：

扩展方式	解决的问题	数据线	地址线低位	地址线高位	片选线	时序
位扩展	数据位宽不够	各自独立引出	全部并联	全部并联	全部并联	同时工作
字扩展	地址空间不够	全部并联	全部并联	接译码器	各自独立	分时工作
字位同时	两者都不够	组内并联、按位独立	全部并联	接译码器	组内并联、组间独立	组内同时、组间分时

（三）主存与 CPU 之间的连接

主存与 CPU 之间通过三类总线连接：

- 地址总线 $A_0 \sim A_{k-1}$（单向）：低若干位送片内地址、高若干位送译码器。
- 数据总线 $D_0 \sim D_{w-1}$（双向）：位扩展时按位分组接不同芯片。
- 控制总线（单向）：读/写命令 WE、访存控制信号 MREQ、片选信号 CS。CPU 访问 I/O 与访问主存共用地址线与数据线，靠 MREQ 区分。

*1.*存储器扩展设计题的完整步骤

已知：单片芯片规格 $m \times n$ 位，目标主存规格 $M \times N$ 位（或给定地址范围）。

```
Step 1  算芯片总数与组织方式
        位扩展片数 = N / n          （片/组）
        字扩展组数 = M / m          （组数）
        总片数 = 位扩展片数 * 字扩展组数

Step 2  分配地址线
        CPU 地址线总根数 = log2(M)（或按题目给定地址空间取 log2(地址空间大小)）
        片内地址线根数   = log2(m)       -> 接各芯片地址引脚（全部并联）
        片选译码所需位数 = log2(组数)    -> 接译码器输入

Step 3  分配数据线
        每组内各芯片数据线独立引出，拼满 N 位

Step 4  选片选方式
        组数少、对地址连续性要求低 -> 线选法
        组数多、地址需连续          -> 译码片选法（主流）

Step 5  画连接图并写出各片地址范围
        组内：地址/CS/WE 并联，数据线按位独立
        组间：低位地址并联，高位地址-> 译码器-> 各组 CS
        地址范围 = [组号<<log2(m) , (组号 +1)<<log2(m) - 1]
```

通用模板题

目标 $M \times N$ 位，单片 $m \times n$ 位（M、m 为字数，N、n 为位宽，均取 2 的幂）：

$$\text{总片数} = \frac{N}{n} \times \frac{M}{m}, \quad \text{片内地址线} = \log_2 m, \quad \text{片选译码位数} = \log_2 \frac{M}{m}, \quad \text{CPU 地址线} = \log_2 M$$

算例：32K × 8 位由 8K × 4 位芯片组成。总片数 $= (8/4) \times (32\text{K}/8\text{K}) = 2 \times 4 = 8$ 片；片内地址线 $= \log_2 8\text{K} = 13$ 根；片选译码 $= \log_2 4 = 2$ 位（用 2-4 译码器）；CPU 地址线 $= \log_2 32\text{K} = 15$ 根。

扩展类真题只考几件事

【2009-15】主存 64 KB，ROM 区 4 KB、其余为 RAM 区，用 2K × 8 位 ROM 与 4K × 4 位 RAM 设计。答：**D**（**2**、**30**）。ROM 片数 $= 4\text{KB}/2\text{KB} = 2$；RAM 区 $= 64 - 4 = 60$ KB，4K × 4 位需 2 片一组凑 8 位，$60\text{KB}/4\text{KB} = 15$ 组，共 $15 \times 2 = 30$ 片。【2010-15】若干 2K × 4 位芯片组成 8K × 8 位，地址 0B1FH 所在芯片的最小地址。答：**D**（**0800H**）。位扩展 2 片一组 → 2K × 8；字扩展 4 组。0B1FH = 0000 1011 0001 1111，高 2 位 01 决定组号 1，该组最小地址 = 0100 0000 0000 0000 = 0800H。【2011-15】按字节编址，主存地址空间 64 MB，实际装 4M × 8 位 RAM 共 32 MB，问 MAR 位数。答：**D**（**26** 位）。**MAR** 位数由主存地址空间决定，与实际安装容量无关：$\log_2 64\text{M} = 26$。【2016-16】64 KB 按字节编址，4000H ~ 5FFFH 为 ROM 区，其余 RAM，用 8K × 4 位 SRAM。答：**C**（**14** 片）。ROM 区 = 2000H = 8 KB，RAM 区 = 56 KB；8K × 4 位需 2 片一组，$56/8 = 7$ 组，共 14 片。【2021-15】24 位地址线、32 位数据线、按字编址、字长 32 位，000000H ~ 3FFFFFH 为 RAM，用 512K × 8 位芯片。答：**C**（**32** 片）。先注意「按字编址」：地址范围 400000H = 2^{22} 个字 × 4 B = 16 MB；片数 $= 16\text{MB}/512\text{KB} = 32$。【2023-15】CPU 30 根地址线，按字节编址，须占满全部地址空间，RAM:ROM = 3 : 1，RAM 在低地址、ROM 在高地址。答：**C**（**30000000H～3FFFFFFFH**）。总空间 $2^{30} = 1$ GB，ROM = 256 MB 位于最高段，起始地址 $= 1\text{GB} - 256\text{MB} = 768\text{MB} = 30000000\text{H}$。

扩展题高频失分点

1. 「按字编址」还是「按字节编址」看漏。按字编址时地址单元数 × 每字字节数 = 总容量，与按字节编址的换算完全不同。

2. 把「主存地址空间」当成「实际安装容量」。MAR 位数（以及 CPU 地址线根数）由地址空间决定，装多少内存不影响 MAR。
3. 算完总容量就忘了位扩展。4K × 4 位芯片装 8 位宽的内存，必须先 2 片一组，否则永远算少一半。
4. 地址范围算错。RAM 在低地址段时从 0000H 起；ROM 在高地址段时从「总空间 − ROM 容量」起。把两者顺序搞反直接全错。
5. 译码器位数算成组数。n 个组需要 $\log_2 n$ 根高位地址线，接 n 输出的译码器。

（四）多模块存储器

CPU 速度远快于主存。多模块（多体）存储器是一种空间并行技术：用多个结构完全相同的存储模块并行工作，提高吞吐率，而不改变单个模块的存取速度。

*1.*两种编址方式

对比维度	连续编址（高位交叉 / 顺序编址）	交叉编址（低位交叉）
地址结构	[高位：模块号][低位：模块内地址]	[高位：模块内地址][低位：模块号]
相邻地址	在同一模块内	在不同模块中
体号公式	模块号 $=\lfloor$地址$/m\rfloor$（高位）	体号 = 地址 mod m（低位）
能否并行	不能，无法提高吞吐率	能，可实现流水线并行访问
适用场景	扩充容量（内存条插槽）	突发顺序访问（CPU 取指、DMA、Cache 调块）

高位交叉为什么并行不了：程序具有局部性和连续性，连续执行的指令和数据基本落在同一个存储体里，该体访问频繁、其余体空闲，多体形同虚设。

低位交叉为什么有效：连续地址分散在不同体，各体有独立的读写电路，可以不用等前一个体准备好就开始下一个体，各体并行准备数据。

编址方式与扩展方式不是一回事

- 高位交叉 = 顺序编址，等价于字扩展的思路（分时工作）。
- 低位交叉的「同时启动」方式在连接上就是位扩展（多个芯片同时工作，各自提供总线的一部分位）。名字侧重不同：位扩展强调容量扩展，多模块交叉强调编址方式。

*2.*低位交叉的流水存取时间

设存储周期为 T、总线传输周期为 τ、交叉模块数为 m。访存分三个阶段：T_1 送地址与命令、T_2 存储器准备数据（即存取时间）、T_3 总线传输一次数据（τ）。

教材与真题默认忽略恢复时间，令存储周期 ≈ 存取时间，即经过一个 T 后数据才送上数据线。

轮流启动方式：每个模块一次读写的位数 = 总线数据位数（每次只传一个模块的数据），流水条件是

$$T = m\tau$$

即每隔 τ 启动下一个模块。

$$t_{\text{准备}}(n) = T + (n-1)\tau$$

$$t_{总}(n)=\underbrace{\tau}_{\text{送首地址与命令}}+\underbrace{T+(n-1)\tau}_{\text{存储器准备}n\text{ 个字}}+\underbrace{\tau}_{\text{传送最后一个字}}=T+(n+1)\tau$$

逐步推导（以 $m=4$、$T=4\tau$、连续读 n 个存储字为例）：

1. t_0：CPU 发出首地址与读命令，耗时 τ。
2. $t_1=\tau$：启动体 M_0；此后每间隔 τ 依次启动 M_1、M_2、M_3、M_0……
3. M_0 经过完整存储周期 $T=4\tau$ 后，在 $t_2=\tau+T$ 时刻把第一个数据送上数据线并开始传送。
4. 从此进入流水线稳定状态：每经过一个 τ 就有一个数据准备好送出。第 n 个字在 $\tau+T+(n-1)\tau$ 时刻准备好。
5. 最后加上传送第 n 个字所需的一个 τ。

$$t_{总}(n)=\tau+T+(n-1)\tau+\tau=T+(n+1)\tau$$

平均准备一个字的时间：

$$\bar{t}=\frac{T+(n-1)\tau}{n}\xrightarrow{n\to\infty}\tau$$

即 n 很大时，存储器平均每个存储周期（每 τ）就能向 CPU 提供一个字——稳定流水时一个存储周期内向 **CPU** 提供 m 个数据。

对比高位交叉（连续编址）：连续地址都在同一个体，两次读之间必须间隔 T，故

$$t_{准备}(n)=nT,\qquad \bar{t}=T\quad(\text{与}n\text{ 无关})$$

统一算例：$m=4$ 体低位交叉，τ 为总线传输周期，$T=4\tau$，连续读取 5 个地址连续的存储字 $W_0\sim W_4$。

- 低位交叉：$t_{准备}=T+(5-1)\tau=4\tau+4\tau=8\tau$；总耗时 $=\tau+8\tau+\tau=10\tau=2.5T$。
- 高位交叉：$t_{准备}=5T=20\tau$；总耗时 $=\tau+20\tau+\tau=22\tau=5.5T$。
- 加速比约 2.2；若 n 更大，低位交叉趋近于每 τ 出一个字（加速比趋近 $m=4$）。

低位交叉计算必背四条

1. 流水条件：$T=m\tau$（每隔 τ 启动一个体）。
2. 准备 n 个字的时间：$T+(n-1)\tau$。
3. 含总线开销的总时间：$T+(n+1)\tau$（送首地址 τ + 准备 + 送尾数据 τ）。
4. 稳定流水时：一个存储周期内提供 m 个字，平均每字耗时 τ。

低位交叉的统考考法

【2015-18】4 体交叉编址，主存地址序列 8005,8006,8007,8008,8001,8002,8003,8004,8000，问可能发生访存冲突的地址对。答：**D**（**8000** 和 **8004**）。体号 = 地址 mod 4：8004 体 0、8008 体 0 但相隔 4 步（不冲突）；8000 与 8004 同为体 0 且相邻（相隔 1 步 <4），必然冲突。【2017-13】4 片 64M × 8 位 DRAM 交叉编址构成主存，总线宽 32 位，每次最多读写 32 位，double 型变量 x 地址为 804001AH，读取 x 需要几个存储周期。答：**C**（**3**）。每片 8 位、4 片并行正好 32 位，符合同时启动条件。double 占 8 B，起始地址低 2 位为 10（体号 2），8 个字节跨越 8000 ~ 8007、8008 ~ 800F 等 8 字节边界，需 **3** 次并行读取。【2013-43】32 位机，主存 8 体交叉，每体存储字长 32 位、存储周期 40 ns，总线宽 32 位、时钟 200 MHz（周期 5 ns），突发传送 32 B。一次读突发事务 = 5 ns（送地址）+ 40 ns（准备首个数据）+ 8 × 5 ns（传送 8 次数据）= 85 ns。注意「准备数据」只算第一个数据的时间，后续数据靠流水线补上。

*3.*访存冲突

冲突条件：m 体交叉存储器中，两次连续访问落在同一个体，且时间间隔小于存储周期 T（即访问步数差 $< m$）。

判断步骤：

1. 对每个地址求 mod m，得到体号；
2. 在访问序列中找体号相同的地址对；
3. 检查它们在序列中的间隔步数是否 $< m$。

无冲突的理想条件：相邻 m 个访问地址分别属于 m 个不同的体（体号连续不重复）。

冲突题的两个陷阱

1. 只看体号相同就判冲突是错的，必须同时满足「间隔步数 < m」。8004 与 8008 都是体 0，但序列中相隔 4 步（$= m$），不冲突。
2. 「可能」二字很关键。题目问「可能发生冲突的地址对」，只要存在某种时刻安排让其冲突即可，不需要它在给定序列中真的紧邻。

四、外部存储器

（一）磁盘存储器

*1.*组成与物理结构

磁盘存储器由磁盘控制器、磁盘驱动器、盘片三大部分组成。

- 磁盘控制器：磁盘与主机的 I/O 接口，含控制逻辑、时序电路、「并 → 串」与「串 → 并」转换电路。
- 磁盘驱动器：读写电路、读写转换开关、读写磁头与磁头定位伺服系统。
- 盘片与盘面：每个盘片的两个面各有一个磁头，因此磁头号 = 盘面号。

关键术语：

- 磁道（**Track**）：磁头与盘片相对运动形成的同心圆。磁道从外向里编号，最外圈为 0 号磁道。
- 柱面（**Cylinder**）：多个盘片上相同编号的磁道构成一个柱面，所以磁道号 = 柱面号。
- 扇区（**Sector**）：每个磁道被分成的若干等长区域，磁盘读写的最小单位（传统 512 B，现代常用 4 KB）。

写入顺序：写完一个柱面上所有磁道后才移动到下一个柱面。

*2.*磁盘的物理地址与读写过程

地址结构：[驱动器号][柱面（磁道）号][盘面（磁头）号][扇区号]。

为什么是「柱面号在前、盘面号在后」

若地址结构是（盘面号，柱面号，扇区号），读完盘面 0 的整条磁道后，要继续读相邻磁盘区域就必须启动磁头臂移动到下一个磁道；而（柱面号，盘面号，扇区号）下，同一柱面的不同盘面柱面号相同，只需激活相邻盘面的磁头即可，不必移动磁头臂，省下大量寻道时间。这就是柱面号必须放在盘面号之前的根本原因。

磁盘读写共三步：寻道 → 旋转等待 → 读写。

- 寻道：磁盘控制器把盘地址送到驱动器后，启动磁头定位伺服系统，按磁头号与柱面号把指定磁头移到指定柱面，完成后发出寻道结束信号。
- 旋转等待：盘片旋转，扇区计数器每收到一个扇区标志脉冲加 1，与地址寄存器中的扇区地址比较，一致时输出扇区符合信号。
- 读写：写操作把数据送写入电路生成写电流脉冲；读操作由读出放大电路读出内容送控制器。

磁盘既不能像 RAM 那样按地址随机访问任一单元，也不是完全按顺序存取，而是先定位磁道、再在磁道内顺序查找，因此属于直接存取（**DAM**）。

*3.*性能指标

（**1**）记录密度

- 道密度：沿磁道分布方向上单位长度内的磁道数目。
- 位密度：沿磁道方向上单位长度内存放的二进制信息数目。

低密度存储方式下所有磁道扇区数相同 → 每个磁道位数相同 → 内道位密度比外道高；高密度存储方式下各磁道位密度相同 → 外道扇区数多于内道 → 整盘容量大得多。

（**2**）存储容量

$$\text{非格式化容量（未格式化容量）} = \text{位密度} \times \text{内圈周长} \times \text{总磁道数}$$

$$\text{格式化容量} = \text{每扇区字节数} \times \text{每磁道扇区数} \times \text{磁道数（柱面数} \times \text{盘面数）}$$

$$\text{磁盘总容量} = \text{柱面数} \times \text{磁头数（盘面数）} \times \text{每磁道扇区数} \times \text{每扇区字节数}$$

非格式化容量 > 格式化容量：低级格式化要在每扇区加头部（扇区号、同步信息）、尾部（ECC 纠错码）以及扇区间隔，这些都要占用磁介质空间。

算例：某磁盘 8 个柱面、4 个盘面、每磁道 16 个扇区、每扇区 512 B。

格式化容量 $= 8 \times 4 \times 16 \times 512\ \text{B} = 256\ \text{KB}$。

（**3**）平均存取时间

磁盘出队后要经历寻道、旋转等待、读写数据传输三个过程：

$$\text{响应时间} = \text{排队延迟} + \text{控制器时间} + \text{寻道时间} + \text{旋转等待时间} + \text{数据传输时间}$$

$$\boxed{\text{存取时间} = \text{寻道时间} + \text{旋转（等待）延迟} + \text{传输时间}}$$

- 寻道时间 $T_s = p + m \cdot n$：p 为启动磁头臂的时间，m 为每跨越一个磁道的耗时，n 为需跨越的磁道数。
- 旋转延迟（平均）：

$$T_r = \frac{1}{2} \times \frac{1}{r} = \frac{1}{2r} \quad (r：\text{转/秒}), \qquad T_r = \frac{1}{2} \times \frac{60}{r} = \frac{30}{r}\ \text{s} \quad (r：\text{转/分})$$

定位过程中最多转 1 圈、最少 0 圈，平均需半圈，所以除以 2。

- 传输时间（读 b 字节，每磁道 N 字节，转速 r 转/秒）：

$$T_t = \frac{b}{N} \times \frac{1}{r} = \frac{b}{rN}$$

因为读写一条完整的磁道恰好需要转一圈的时间 $1/r$。等价形式：访问一个扇区的传输时间 $= \dfrac{\text{一圈时间}}{\text{每磁道扇区数}} = \dfrac{\text{扇区容量}}{\text{数据传输率}}$。

旋转延迟只有两种算法，别混

- 转速单位是 **r/min**（如 7200 rpm）：$T_r = \dfrac{60}{2r} = \dfrac{30}{r}$ s。
- 转速单位是 **r/s**：$T_r = \dfrac{1}{2r}$ s。

  题目给「r 秒/转」这种反着写的单位要特别小心——先统一成转/秒或转/分再算。

算例 **A**（【2013-21】）：转速 10000 rpm，平均寻道 6 ms，传输速率 20 MB/s，控制器延时 0.2 ms，读一个 4 KB 扇区。

- 转一圈 $= 60/10000 = 6$ ms $\rightarrow$ 平均旋转延迟 $= 6/2 = 3$ ms。
- 传输时间 $= 4\text{KB}/20\text{MB/s} \approx 0.2$ ms。
- 总时间 $= 6 + 3 + 0.2 + 0.2 = 9.4$ ms。（答案 B）

算例 **B**（【2015-20】）：转速 7200 rpm，平均寻道 8 ms，每磁道 1000 个扇区，访问一个扇区。

- 转一圈 $= 60/7200 = 8.333$ ms $\rightarrow$ 平均旋转延迟 $= 4.167$ ms。
- 一个扇区的传输时间 $= 8.333/1000 \approx 0.0083$ ms（可忽略）。
- 总时间 $= 8 + 4.167 + 0.0083 \approx 12.2$ ms。（答案 B）

（4）数据传输率

$$\text{内部数据传输率} = \frac{\text{转速 (转/分)}}{60} \times \text{内圈周长} \times \text{最内道位密度}$$

因为同一时刻只有一个磁头在读写，所以内部传输率由内圈（周长最小、位密度最高）决定。

磁盘的两个易错断言

1. 「磁盘的最小读写单位是一字节」错误，最小读写单位是扇区。哪怕只读 1 个字节，也要把包含它的整个扇区读出来（【2019-20】答案）。
2. 「格式化容量比非格式化容量小」正确。格式化要写扇区头、尾和间隔，占用空间；非格式化容量是按道密度与位密度算出的全部磁化单元总数。
3. 磁盘是机械设备，读写串行：不可能同一时刻既读又写，也不可能同时读两组数据。

（5）减少延迟的硬件手段

对一个盘面内的扇区采用交替编号，对不同盘面的扇区采用错位命名（均在低级格式化时完成），可以显著减少连续读写多条记录时的磁头等待时间。软件层面则靠磁盘调度算法减少寻道时间。

4.磁盘阵列 RAID

RAID（廉价冗余磁盘阵列）把多个独立物理磁盘组成一个逻辑盘，数据交叉存储、并行访问。基本思想三点：组织成阵列以增加容量；用类似主存多模块交叉的技术让多盘并行工作提高传输率；用冗余技术提高可靠性。

级别	定义	要点	类别
RAID0	无冗余、无校验	条带化交替存放、并行读写，容量与速度都提升，无容错	条带化
RAID1	镜像磁盘阵列	两盘同写互为备份，可靠性高，容量减半	镜像
RAID2	海明码纠错阵列	多个冗余校验盘，冗余开销大，已不再使用	并行访问
RAID3	位交叉奇偶校验	单个冗余盘 + 小条带交叉	并行访问
RAID4	块交叉奇偶校验	大数据块交叉，各盘操作独立	独立访问
RAID5	无独立校验盘的奇偶校验	校验块分布在各个磁盘中，成本不高效率高，广泛使用	独立访问

RAID 的考法

【2013-20】用于提高 **RAID** 可靠性的措施有：Ⅰ磁盘镜像 ✓、Ⅱ条带化 ×（只提速不提供冗余）、Ⅲ奇偶校验 ✓、Ⅳ增加 Cache ×（只提性能，写缓存掉电还可能丢数据）。答案 **B**（仅Ⅰ、Ⅲ）。

（二）固态硬盘 SSD

*1.*结构与读写特性

SSD 基于闪存 Flash 技术，属于电可擦除 ROM（EEPROM）。组成：

- 闪存翻译层：负责把逻辑块号翻译成物理页地址，相当于磁盘控制器。
- 存储介质：闪存颗粒。多个闪存芯片（**Flash Chip**） → 每芯片含多个块（**Block**） → 每块含多个页（**Page**）。

SSD 的四条读写特性

1. 以页为单位读写（页相当于磁盘的扇区）；以块为单位擦除。
2. 写某页前必须先擦除该页所在的整块。若块内其他页有数据，要先把它们复制到另一个已擦除的空块，再写入目标页——这就是 SSD 的「写放大」。
3. 支持随机访问：给定逻辑地址，闪存翻译层可通过电路迅速定位物理地址，没有寻道与旋转延迟。
4. 读快写慢：擦干净的块中每页可写一次、读无限次；读速度与 RAM 相当，写速度与硬盘相当。

*2.*与机械硬盘的对比

维度	机械硬盘 **HDD**	固态硬盘 **SSD**
访问定位方式	移动磁臂 + 旋转盘片（电路控制）	纯电路译码定位
寻道时间 / 旋转延迟	有，且是主要开销	没有
随机访问性能	差	极好
顺序读写性能	中等	高
噪音 / 抗震 / 功耗	有噪音、怕摔、功耗高	安静、耐摔抗震、功耗低
单位容量价格	低	高
寿命特性	扇区不因写入次数多而损坏	块擦除次数有限，反复写同一块会写坏

*3.*磨损均衡

思想：把「擦除」操作平均分布到各个块上，避免某些块被反复擦写而提前失效，从而延长整盘寿命。

- 动态磨损均衡：写入数据时，优先选择累计擦除次数少的新闪存块。
- 静态磨损均衡：SSD 主动监测并自动做数据分配与迁移——让老旧的块承担以读为主的存储任务，让较新的块承担更多的写任务。效果更优，是现代 SSD 的标配。

SSD 不一定比 HDD 快——这句话要看场景

SSD 的随机访问性能碾压 HDD（没有寻道和旋转延迟），但写操作因必须先擦块、可能触发数据搬迁（垃圾回收），写入延迟可能反而高于顺序写磁盘。所以「SSD 读写都比磁盘快」不严谨；正确的说法是「SSD 随机访问性能高、读快，写涉及擦除会慢」。

五、高速缓冲存储器（Cache）

（一）Cache 的基本原理

Cache 是 CPU 与主存之间的小容量高速缓冲层，用来弥补二者速度鸿沟。它由 **SRAM** 构成（无需刷新），L1/L2 集成在 CPU 核内，L3 为多核共享。

核心工作逻辑：程序运行时，硬件基于局部性原理把即将用到的、主存中相邻的一批数据/指令预先成批拷贝到 **Cache**。**Cache** 的内容是主存部分内容的副本（镜像），因此不会增加系统总容量。

术语澄清：主存块（**Block**）$\neq$ 页（**Page**）。主存块是 Cache-主存层次的传输单位（8 ~ 128 B），纯硬件实现；页是虚存-主存层次的单位（4 KB ~ 64 KB），面向操作系统内存管理。

Cache 行除数据区外还有两个核心标识位：

- 有效位 **V**（**Valid Bit**）：标识该行数据是否合法有效。系统上电/复位时所有行 V 清零；装入合法主存块后置 1；淘汰时清 0。**V=0** 时无论 **Tag** 是否匹配都判为缺失。
- 标记位 **Tag**：标识该行存放的是哪个主存块。只有 **Tag** 匹配且 **V=1** 才算命中。采用写回策略时还需 **1** 位脏位。

访问流程：CPU 给出主存地址 $\rightarrow$ 按映射规则定位候选行 $\rightarrow$ 比较 Tag 与 V $\rightarrow$ 命中则从 Cache 取数并按块内地址选出字/字节；缺失则访问主存取块，同时把整块调入 Cache（必要时先替换）。

整个流程——判命中、取数、调块、替换——必须由硬件在一条指令的执行周期内完成，对程序员完全透明。因此 **Cache** 缺失不会引发 **CPU** 陷入内核，不会引起进程切换；而缺页必须由操作系统处理。

*1.*性能指标

- 命中率 h：$h = \dfrac{\text{命中次数}}{\text{总访存次数}}$，$0 \le h \le 1$。
- 缺失率 $= 1 - h$。
- 命中时间 t_c：Cache 命中的访问延迟（即 Cache 自身的访问时间）。
- 主存访问时间 t_m：t_m 通常是 t_c 的几十到上百倍。

平均访问时间：两种模型写法不同，看题目是串行还是并行访问。

串行访问模型（先查 Cache，未命中再访主存）：

$$T_a = h \cdot t_c + (1-h)(t_c + t_m) = t_c + (1-h)\,t_m$$

并行访问模型（同时访问 Cache 与主存，命中则立即取消主存请求）：

$$T_a = h \cdot t_c + (1-h)\,t_m$$

访问效率：

$$e = \frac{t_c}{T_a}$$

两个平均访问时间公式不能乱用

教材常写 $T_a = t_c + (1-h)t_m$（串行：无论命中与否都要花一个 t_c）；也有题目按 $T_a = ht_c + (1-h)t_m$（并行：同时发起，命中就赢）。区别在于缺失时是否还要额外计入 t_c。读完题先确认组织方式，再选公式；如果 $t_c \ll t_m$，两者差别不大，但选项往往就卡在这个差别上。

算例：$t_c = 4$ ns，$t_m = 100$ ns。

- 若 $h = 95\%$，串行模型：$T_a = 4 + 0.05 \times 100 = 9$ ns，$e = 4/9 \approx 44.4\%$。
- 并行模型：$T_a = 0.95 \times 4 + 0.05 \times 100 = 8.8$ ns。

两级 **Cache** 算例：L1 命中率 95%、访问时间 2 ns；L1 缺失时访 L2，L2 命中率 80%、访问时间 10 ns；L2 也缺失则访主存，主存访问时间 100 ns。

$$T_a = 2 + 0.05 \times 10 + 0.05 \times 0.20 \times 100 = 2 + 0.5 + 1 = 3.5 \text{ ns}$$

三级分解检验：L1 命中贡献 $0.95 \times 2 = 1.9$；L2 命中贡献 $0.05 \times 0.8 \times (2+10) = 0.48$；主存贡献 $0.05 \times 0.2 \times (2+10+100) = 1.12$；合计 3.5 ns ✓。

2.Cache 设计要考虑的四类问题

1. 地址映射与变换：主存块按什么规则放到 Cache 行？如何快速定位？（直接映射、全相联、组相联）
2. 块替换：Cache 满时淘汰哪一行？（FIFO、LRU、LFU、随机）
3. 写一致性：写操作如何保证 Cache 与主存一致？（写直达、写回、写分配、非写分配）
4. 地址划分：如何把主存地址切成 `Tag | Index | Offset` 供硬件并行匹配。

（二）Cache 与主存之间的映射方式

三种映射方式在命中率（可选位置越多越高）、查找速度（可选位置越多越慢）、冲突率（可选位置越少越高）之间权衡。

主存块大小的统一约定：设块内地址占 w 位，则块大小 $= 2^w$ 字节。块内地址位数 $w = \log_2(\text{块大小})$，永远是地址的最低位段。

*1.*直接映射

映射规则：

$$\text{Cache 行号} = \text{主存块号 mod Cache 行数}$$

等价说法：把主存按 Cache 大小分区，每个区内的块数与 Cache 行数相同；主存地址分成标记 **Tag**（区地址）+ 行号 **Index**（区内行索引）+ 块内地址 **Offset** 三部分。Index 字段就是上式中的余数。

地址字段位数（设主存地址 n 位，块大小 2^w B，Cache 行数 $C = 2^c$）：

$$w = \log_2(\text{块大小}), \qquad c = \log_2(\text{Cache 行数}), \qquad t = n - c - w$$

$$\text{Cache 行数} = \frac{\text{Cache 数据区容量}}{\text{块大小}}$$

特点：每个主存块只有唯一确定的 1 个可放位置；只需一个共享的比较器；需要行号译码器；硬件成本最低；不需要替换算法（冲突时直接替换）；命中率最低、冲突率最高（Cache 未满也可能冲突）。适用于大容量 Cache。

*2.*全相联映射

映射规则：一个主存块可装入 Cache 的任意一行。

地址只有标记 **Tag** + 块内地址 **Offset** 两个字段（没有行号/组号）：

$$w = \log_2(\text{块大小}), \qquad t = n - w$$

$$t = \log_2(\text{主存块数}) = \log_2\left(\frac{\text{主存容量}}{\text{块大小}}\right)$$

特点：Tag 最长；利用率最高、命中率最高、冲突率最低（只有 Cache 全满才需要替换）；需要每一行一个比较器并行比较，硬件成本最高；替换算法复杂。适用于小容量 Cache，如快表 **TLB**。

全相联的硬件实现要点：主存 Tag 广播到所有行的比较器，比较结果与各行的有效位 V 送入与门（Tag 相等且 V=1 才输出 1），所有与门输出送入或门产生全局 `hit`；命中行打开三态门，数据经 MUX 按块内地址选出字/字节输出；`hit=0` 时阻断 MUX 输出，CPU 转而访问主存。缺失时需从主存取块、按替换算法选牺牲行、写入并更新 Tag 与 V。

*3.*组相联映射

映射规则：Cache 分成若干组，每组 k 行，称 k-路组相联。主存块先用直接映射定位到固定的组，再在组内全相联映射到任意一行：

$$\text{Cache 组号} = \text{主存块号 mod Cache 组数}$$

地址分三段：标记 **Tag** + 组索引 **Index** + 块内地址 **Offset**。

$$\text{组数} = \frac{\text{Cache 总行数}}{\text{路数}k} = \frac{\text{Cache 数据区容量}}{\text{块大小} \times k}$$

$$d = \log_2(\text{组数}), \qquad w = \log_2(\text{块大小}), \qquad t = n - d - w$$

查找流程：用组索引经译码器定位到唯一一组（这一步把查找范围大幅缩小）→ 组内 k 路的 Tag 与有效位并行比较（k 个比较器）→ 命中路的与门打开三态门 → MUX 按块内地址选出数据。

组相联统一了另外两种映射：

- $k=1$（每组 1 行，共 m 组）→ 退化为直接映射（只有 1 个比较器）。
- $m=1$（只有 1 组，共 $k=n$ 行）→ 退化为全相联映射（无需组号译码器）。

组索引位数只由组数决定，与路数无关

这是最容易错的一点。2^d 组就占 d 位，不论每组有几路。增加路数 k 会减少组数，从而减少组索引位数、增加 Tag 位数，但不会改变组索引与路数的直接对应关系。

*4.*三种映射方式对比

对比项	直接映射	全相联映射	k-路组相联
映射规则	行号 = 块号 mod 行数	任意行	组号 = 块号 mod 组数，组内任意
可放位置数	1（唯一确定）	n（全部行）	k（组内）
地址字段	Tag + 行号 + 块内	**Tag** + 块内	Tag + 组号 + 块内
Tag 长度	最短	最长	居中
比较器	**1** 个（共享）	n 个（每行一个）	k 个（每组 k 个）
需要译码器	需要（行号）	不需要	需要（组号）
硬件成本	最低	最高	居中
命中率 / Cache 利用率	低	最高	较高
冲突率	最高（未满也可能冲突）	最低（全满才冲突）	居中
替换算法	不需要	复杂	仅组内，需要
适用	大容量 Cache	小容量（TLB）	最常用（L1/L2/L3）

关联度（路数）与性能的关系（Cache 大小、块大小固定时）：

- 关联度越低 → 命中率越低、判命中开销越小、命中时间越短、Tag 额外空间开销越小。
- 直接映射命中时间最短、额外开销最小；全相联命中时间最长、额外开销最大。

这解释了现代 CPU 为何普遍使用 4 路、8 路、16 路组相联：路数增加时命中率迅速逼近全相联，而硬件成本仍可控；路数为 1 冲突率过高，路数等于总行数则硬件代价过大。

*5.*地址划分与容量计算的解题流程

```
Step 1  由【Cache 数据区容量 / 块大小】求 Cache 总行数
Step 2  由【总行数 / 路数 k】求组数（直接映射时组数 = 行数，全相联时组数 = 1）
Step 3  Offset 位数 w = log2(块大小)
        直接映射: Index 位数 c = log2(行数)
        组相联  : Index 位数 d = log2(组数)
        全相联  : 无 Index
Step 4  Tag 位数 t = 主存地址位数 n - Index 位数 - w
Step 5  每行总位数 = 数据位数 + Tag 位数 + 有效位 (1)
                   + 脏位（写回时要算，直写时不算）
Step 6  Cache 总容量 = 行数 * 每行总位数
```

通用容量公式（设块大小 2^w 字节、Cache 行数 n、主存地址位数由 w 与 Tag 位数 s 决定）：

$$\text{直接映射：}\quad C = n \times \left(1 + t_{\text{tag}} + 8 \times 2^{w}\right)\ \text{位},\quad n = 2^{c},\ t_{\text{tag}} = s - c$$

$$\text{全相联：}\quad C = n \times (1 + s + 8 \times 2^{w})\ \text{位}$$

$$k\ \text{路组相联：}\quad C = kn \times (1 + s - d + 8 \times 2^{w})\ \text{位},\quad \text{组数} = n = 2^{d}$$

（若采用写回策略，括号内还要再加 1 位脏位。）

算例 1（直接映射）：主存地址 32 位，按字节编址，块大小 16 B，Cache 数据区容量 64 KB，直接映射。

- 行数 $= 64\text{KB}/16\text{B} = 2^{12} = 4096$ 行 $\rightarrow c = 12$ 位；$w = \log_2 16 = 4$ 位。
- Tag $= 32 - 12 - 4 = 16$ 位。
- 每行 $= 1$ 位 V + 16 位 Tag + 4 个 32 位字 $= 1 + 16 + 128 = 145$ 位。
- Cache 总容量 $= 4096 \times 145 = 580$ Kbit $= 72.5$ KB；数据占比 $= 64/72.5 \approx 88.3\%$。

算例 2（直接映射 + 写回，即**【2015-15】**）：主存地址 32 位、按字节编址、块大小 4 个字（每字 32 位 = 4 B，故块 16 B）、写回、能存 4K 字数据，求 Cache 总位数。

- 数据容量 $= 4\text{K} \times 4\text{B} = 16$ KB；行数 $= 16\text{KB}/16\text{B} = 1024 \rightarrow c = 10$；$w = 4$。
- Tag $= 32 - 10 - 4 = 18$ 位。
- 每行 $= 1(\text{V}) + 1(\text{脏}) + 18(\text{Tag}) + 128(\text{数据}) = 148$ 位。
- 总容量 $= 1024 \times 148 = 148$ Kbit。（答案 C）

算例 3（直接映射 + 写回，即**【2021-16】**）：主存地址 32 位、按字节编址、Cache 数据区 32 KB、块大小 32 B、直接映射、写回，求每行位数。

- 行数 $= 32\text{KB}/32\text{B} = 1024 \rightarrow c = 10$；$w = 5$；Tag $= 32 - 10 - 5 = 17$。
- 每行 $= 1 + 1 + 17 + 32 \times 8 = 275$ 位。（答案 A）

算例 4（组相联，即**【2022-16】**）：主存地址 32 位、按字节编址、Cache 数据区 32 KB、块大小 64 B、**8** 路组相联，问比较器个数与位数。

- 行数 $= 32\text{KB}/64\text{B} = 512$；组数 $= 512/8 = 64 \rightarrow d = \log_2 64 = 6$；$w = \log_2 64 = 6$。
- Tag $= 32 - 6 - 6 = 20$ 位。
- 访问时用组索引选定一组，组内 **8** 路并行比较 $\rightarrow$ 比较器个数 $= 8$，每个宽度 = Tag 位数 $= 20$ 位。（答案 A）

地址划分题的五个高频错点

1. 把「**Cache** 容量」当成「数据区容量」。题目给的行数/组数由数据区容量算；Tag 字段与总容量才需要加元数据位。
2. 组索引位数按路数算。2^d 组就 d 位，与每组几路无关。
3. 写回忘加脏位、直写多加了脏位。看写策略决定这一位要不要算。
4. 「每行位数」与「总容量」问法不同。前者只乘一次，后者乘行数。
5. 主存块大小单位是「字」时忘记换算成字节。4 个字 × 32 位 = 16 B，$w = 4$ 而不是 $\log_2 4$。

命中与映射的统考考法

【2009-21】访存 1000 次、缺失 50 次，命中率？$h=(1000-50)/1000=95\%$（答案 D）。【2009-14】Cache 16 块、2 路组相联、块大小 32 B、按字节编址，主存 129 号单元所在块装入哪个组号？块号 $=\lfloor 129/32\rfloor=4$，组数 $=16/2=8$，组号 $=4 \bmod 8=4$（答案 C）。也可直接看地址 129 = `100 00001`：低 5 位是块内地址、接着 3 位是组号 $100=4$。【2022-16】同算例 4；【2021-16】同算例 3；【2015-15】同算例 2。这一类型是必拿分项，公式背熟 + 单位换算细致即可。

（三）Cache 中主存块的替换算法

Cache 行数远少于主存块数，多个主存块会映射到同一行/组。新块调入而位置已满时必须淘汰一个。适用性提醒：

- 直接映射不需要任何替换算法——每个块只有唯一位置，调入即覆盖。
- 全相联的替换涉及全部 Cache 行。
- 组相联的替换只在特定组内进行。

统一算例：某 **4** 路组相联 Cache 的第 0 组初始为空，依次访问映射到该组的块号序列

$$1,\ 2,\ 3,\ 4,\ 1,\ 2,\ 5,\ 1,\ 2,\ 3,\ 4,\ 5$$

前 4 次访问把组内 4 个空位填满，状态为 `[1，2，3，4]`；接着访问 1、2 均命中；访问 **5** 时冲突，需按算法淘汰一个。

算法	核心思想	实现机制	本例淘汰谁	优点	缺点
FIFO 先进先出	谁先进入谁先被淘汰（认老不认新）	计数器或循环队列	淘汰 **1**（最早进入）	硬件最简单	性能差，可能产生 **Belady** 异常；不反映局部性
LRU 最近最少使用	淘汰最久未被访问的块	每行一个计数器/时间戳：命中行清零，其余加一；替换时选值最大者	淘汰 **3**（访问历史…3, 4, 1, 2 中最久）	最符合时间局部性，命中率最高，主流采用	硬件开销大，每次访问都要更新计数器
LFU 最不经常使用	淘汰过去被访问次数最少的块	每行一个访问计数器，命中加 1；替换时选值最小者	3 与 4 次数并列（各 1 次），需配合 FIFO 打破平局 → 淘汰 **3**	体现长期高频热点	受「历史包袱」影响：曾经狂热的块赖着不走；成本高
随机替换	随机挑一个淘汰	随机数发生器	任意（如摇到 2 就淘汰对应行）	硬件最简单、速度最快	完全不用局部性，命中率不稳定

LRU 位位数：

$$\text{每块的 LRU 替换位} = \log_2(\text{组内块数})\ \text{bit}$$

2 路组相联需 1 位，4 路需 2 位，8 路需 3 位。

LRU 手工模拟的标准做法（必考题型）：在组内维护一个「最近访问顺序」队列，每次访问（无论命中还是缺失）都执行两步：

1. 命中：把被命中的块移到队列最前端（标记为最新）。
2. 缺失：淘汰队列最末端的块，把新块放到最前端。

初始时若组内有空位，直接填空位，不算替换。判断最终命中次数时，逐项走一遍即可。

替换算法模拟的三个坑

1. 命中的访问也要更新 **LRU** 顺序。很多同学只在缺失时动队列，导致后续淘汰对象选错。
2. **FIFO** 与 **LRU** 在「命中是否影响顺序」上完全不同：FIFO 的进入顺序不受命中影响；LRU 受。
3. **LFU** 的计数器只在命中时加 **1**，缺失调入的新块计数器从 1（或 0）开始，且并列时必须说明用什么规则打破平局。题目没给就按 FIFO 处理。

（四）Cache 写策略

Cache 中存放的是主存的副本。写操作若只改 Cache 不改主存，就会出现不一致：DMA 设备读到旧数据、多核中另一个核读到过期值、程序崩溃后主存状态错误。解决靠写策略，分写命中与写缺失两种情形。

*1.*写命中时的策略

写直通法（**Write-Through**，又称全写法、写穿法）

- 写操作同时更新 **Cache** 与主存，任何时刻两者保持一致。
- 优化：配置写缓冲（**Write Buffer**）队列。CPU 写完 Cache 后把写请求放入写缓冲，由写缓冲异步更新主存，CPU 不必等待主存写完。

维度	说明
一致性	始终一致，主存任何时刻都是最新值
写速度	慢（每次写都要访问慢速主存，靠写缓冲缓解）
实现复杂度	简单，不需要脏位
总线带宽	占用多（每次写都产生主存访问）

写回法（**Write-Back**，又称回写法）

- 写操作只更新 **Cache**，不立即写主存；只有当该行被替换时才写回主存。
- 关键机制：脏位（**Dirty Bit**），每行 1 位。
  - 脏位 = 1：该行被 CPU 修改过，与主存不一致，淘汰前必须先写回主存。
  - 脏位 = 0：该行与主存一致，可直接覆盖，无需写回，节省主存带宽。

维度	说明
一致性	不能保证实时一致，存在「脏数据窗口」
写速度	快（绝大多数写操作只访问 Cache）
实现复杂度	较复杂，需要维护脏位
总线带宽	占用少（只有替换脏块时才写主存）

两种写命中策略对比

对比维度	写直通法	写回法
Cache 与主存一致性	实时一致	存在不一致窗口
每次写是否访问主存	必须访问	通常不访问
是否需要脏位	不需要	需要
适用场景	对一致性要求高（多核共享、DMA 频繁）	对性能要求高（主流 **CPU**）
是否配写缓冲	可以	不需要

*2.*写缺失时的策略

写分配法（**Write-Allocate**）：写缺失时先把目标块从主存调入 **Cache**，再在 Cache 中执行写。设计动机是时间局部性——这个地址被写了，接下来很可能还会被读或再次被写。通常搭配写回法。

非写分配法（**No Write-Allocate**）：写缺失时直接写主存，不把该块调入 Cache。若写完之后短期内不再访问该地址，调块进来反而浪费空间、挤占更有价值的块。通常搭配写直通法。

对比维度	写分配法	非写分配法
写缺失时的处理	调块入 Cache 再写 Cache	直接写主存，不动 Cache
后续读命中率	更高（块已在 Cache 中）	较低
Cache 空间利用	可能引入不常用块	不引入新块，节省空间
通常搭配	写回法	写直通法

*3.*策略组合

实际系统中两种策略成对使用：

组合	逻辑链	定位
写回法 + 写分配法	写回法要把写操作留在 Cache 里 → 写命中直接改 Cache；写缺失就先把块调进来再改	主流 **CPU**，性能优先
写直通法 + 非写分配法	写直通法已保证每次写都到达主存，调块进来的意义很小	一致性优先

写策略的统考考法

【2015-16】指令 `add xaddr, 3`，页式虚存 + TLB，Cache 采用直写，完成该指令至少访问主存几次？答：**B**（**1** 次）。取指时若 TLB 命中且 Cache 命中，访主存 0 次；取操作数同样假设 TLB 与 Cache 都命中，访主存 0 次；但写回结果时直写策略要求同步写主存，故至少 1 次。这正是「写直达法无法避免主存访问」的直接体现。

写策略四条易错

1. 写直通法不需要脏位，写回法必须要有脏位。算 Cache 每行位数时，脏位是否计入完全取决于写策略。
2. 脏位 = **1** 才需要写回；脏位 = 0 的行被淘汰时直接覆盖，不产生主存写。
3. 写分配与写回、非写分配与写直通是「通常」搭配，不是强制。理论上可以交叉，但那样会自相矛盾（写回 + 非写分配意味着写缺失时既不更新 Cache 又要等替换才写回，逻辑失效）。
4. 写缓冲只解决「**CPU** 等待」问题，不改变一致性语义。它让 CPU 不必阻塞，但主存最终一定会被更新。

六、虚拟存储器

（一）虚拟存储器的基本概念

虚拟存储器不是独立的物理硬件，而是一套硬件与操作系统协同实现的内存管理技术。它要解决三个痛点：

1. 突破物理主存容量限制：为程序员提供远超物理主存大小的连续、统一的编程地址空间。
2. 弥合 **CPU** 与外存的效率鸿沟：以主存（DRAM）作为外存的高速缓存，减少对低速外存的直接依赖。
3. 支撑多进程安全并发：为每个进程提供私有、隔离的虚拟地址空间。

核心构件：

- 虚拟地址 **VA**（**Virtual Address**，逻辑地址）：应用程序视角的地址。
- 物理地址 **PA**（**Physical Address**）：硬件实际用来寻址内存单元的地址。
- **MMU**（**Memory Management Unit**，存储管理部件）：集成在 CPU 内，负责地址转换与合法性校验。
- 页表基址寄存器 **PTBR**：记录页表在主存中的首地址。进程切换时只需改 **PTBR**，即可完成页表切换。多级页表下 PTBR 存的是最顶层页表（页目录）的物理基址。

「地址转换由谁完成」必须分清

- **OS**（软件）负责：创建页表、写入映射关系；修改 PTBR；处理缺页异常。
- **MMU**（硬件）负责：每次访存时查 TLB、翻译地址、做权限检查；TLB 缺失时遍历页表（硬件页表漫游器）。

一句话：页表是软件写的，但「查页表」这个动作每次都是硬件做的。所以「地址转换由 MMU 硬件完成」是正确的表述。

两类异常：

- 缺页异常：目标数据未载入主存。MMU 检测到后触发异常，CPU 从用户态陷入内核，由缺页处理程序完成外存 → 主存的页面调入并更新页表，之后重新执行触发缺页的那条指令（不是下一条！）。
- 存储保护异常：地址越界、访问权限不符，由操作系统处理（终止进程或报错）。

虚拟页与物理页：虚拟页 VP（虚页/逻辑页）与物理页 PP（实页/页框、页帧）大小完全相等，故两者页内偏移位数相同。因虚拟存储器容量大于主存，虚拟页号位数 > 物理页号位数。

$$\text{虚拟地址} = \text{虚拟页号 VPN} + \text{页内偏移}, \qquad \text{物理地址} = \text{物理页号 PPN} + \text{页内偏移}$$

MMU 转换过程中「不会」检测的事件

【2024-18】下列事件不是在 MMU 地址转换过程中检测的是（）。答：**B. Cache** 缺失。MMU 负责虚 → 实转换，会检测访问越权、页面缺失、TLB 缺失。**Cache** 缺失是在 **MMU** 完成地址翻译、**CPU** 拿着物理地址访问主存数据时才检测的，二者不在同一环节。

（二）页式虚拟存储器

*1.*基本原理

主存与虚拟地址空间都被划分为大小相等的页，内外存以页为单位交换信息。采用请求分页：只把活跃页面调入主存，非活跃页面留在辅存；访问到不在主存的页面时触发缺页异常。

页式虚拟存储器采用全相联映射（每个虚拟页可放到主存中任意一个空闲物理页框），因为缺页代价极高，必须把冲突缺失降到最低。

Cache-主存与主存**-**辅存的定量对比：Cache 容量 ≪ 主存 ≪ 辅存；Cache 访问速度 ≫ 主存 ≫ 辅存（主存比 Cache 慢 10 ~ 100 倍，辅存比主存慢约 10000 倍）。因此页面（4 KB ~ 64 KB）远大于主存块（16/32/64 B），主存-辅存层次必然用全相联 + 写回。

*2.*页表与页表项

每个进程由操作系统建立独立的页表，记录虚拟页到物理页/磁盘位置的映射。页表每一行是一个页表项 **PTE**：

字段	别名	作用
有效位（装入位）	存在位	=1：虚页已调入主存，存放位置字段即物理页号；=0：未调入，若存放位置为 NULL 表示未分配，否则记录该页在磁盘上的起始地址
脏位（修改位）	—	=1：页面已被修改，替换时需写回辅存；=0：可直接替换
引用位（使用位）	替换控制位	记录页面访问情况，LRU 算法中作为 LRU 位
访问权限位（保护位）	保护位	定义可读可写 / 只读 / 可执行，实现存储保护

*3.*地址转换过程

页表项地址的生成：

$$\text{页表项地址} = \text{PTBR} + \text{虚拟页号} \times \text{页表项大小}$$

完整过程（页面命中）：

1. CPU 送出指令中的虚拟地址，交 MMU 转换；
2. MMU 用 PTBR 与虚拟页号生成页表项地址，访问主存中对应页表项；
3. 主存向 MMU 返回页表项；
4. 有效位 = 1，确认页面在主存，把物理页号 + 页内偏移拼接成物理地址，访问主存对应数据；
5. 主存向 CPU 返回目标数据。

页面缺失（缺页）：

1. 步骤 1 ~ 3 同上；
2. 有效位 = 0，MMU 触发缺页异常，调用操作系统缺页异常处理程序；
3. 主存已满时按替换算法选定待淘汰页面；若其脏位 = 1，需先写回磁盘，否则直接丢弃；
4. 缺页处理程序从磁盘把目标页面调入主存，更新页表项；
5. 返回原进程，重新执行触发缺页的指令，此次按命中流程完成访存。

虚实地址转换算例（【2022-15】）：主存地址 24 位，虚拟地址空间 4 GB，页大小 4 KB，按字节编址。访问虚拟地址 00082840H。

- 页大小 4 KB $= 2^{12}$ → 页内偏移 **12** 位；虚拟地址 32 位（4 GB）→ 虚页号 **20** 位。
- 拆 00082840H：低 12 位 840H 是页内偏移，高 20 位 00082H 是虚页号。
- 00082H $= 8 \times 16 + 2 = 130$。查表：虚页号 130 的存在位 = 1，实页号 = 018H。
- 主存地址 24 位，实页号占高 $24 - 12 = 12$ 位；查得的实页号位数正好，高 4 位补 0，拼接得 018H + 840H = **018840H**。（答案 C）

再算一例（【2013-16】）：主存地址空间 256 MB，虚拟地址空间 4 GB，页大小 4 KB，TLB 全相联 4 项。虚拟地址 03FFF180H。

- 页内偏移 12 位 → 虚页号 = 03FFFH。
- 在 TLB 中找标记为 03FFFH 的表项：有效位 = 1，页框号 = 0153H。
- 物理地址 = 页框号 × 4KB + 偏移 = ‘0153180H‘。（答案 A）

虚实转换题的三步固定套路

1. 先由页大小定页内偏移位数 $= \log_2$(页大小)，其余高位是虚页号；
2. 拆地址、算虚页号、查表（十六进制先转十进制或直接按位看，注意别数错位数）；
3. 物理地址 = 页框号 « 偏移位数 + 原偏移，同时要检查「页框号位数 + 偏移位数」是否等于物理地址位数，不足补 0、多余则本身信息量不够（此时题目通常会设计得刚好）。

缺页的返回点：缺页属于内部异常中的故障（**Fault**），处理完成后必须重新执行原指令，这一点【2019-14】考过——选项「回到发生缺页的指令的下一条指令执行」是错的。

*4.*使用快表 *TLB* 加速地址转换

Cache 虽能缓存部分页表块，但块粒度较大，不能充分利用虚存访问的局部性。于是处理器维护转换旁路缓冲区 **TLB**（**Translation Look-aside Buffer**），专门缓冲经常访问的页表项。

$$\text{TLB 的关键字} = \text{虚拟页号 VPN}$$

TLB 本质是一个容量很小的 Cache：

- 大多采用全相联或组相联方式，且多采用随机替换算法。
- TLB 离 CPU 更近、更快，称快表；主存中的页表称慢表。
- 典型容量 32 ~ 1024 项（1 ~ 2 级）。

全相联 **TLB** 的工作逻辑：虚拟地址分为「虚拟页号 + 页内偏移」。MMU 把虚拟页号与 TLB 内所有表项的 **Tag** 并行比较，若 Tag 匹配且有效位 = 1 则命中，直接取物理页号与页内偏移拼接。

组相联 **TLB** 的工作逻辑（以 2 路为例）：虚拟地址的 VPN 部分划分为「**TLB** 标记 + **TLB** 组索引」。MMU 用组索引定位到对应组（每组 2 项），把 TLB 标记与组内 2 项的标记并行比较，结果与有效位做逻辑与，生成选择信号，选中的物理页号与页内偏移拼接。

TLB 表项的位数计算

$$\text{VPN 位数} = \text{虚拟地址位数} - \log_2(\text{页大小})$$

$$\text{组索引位数} = \log_2\left(\frac{\text{TLB 表项数}}{\text{路数}}\right)$$

$$\text{TLB 标记位数} = \text{VPN 位数} - \text{组索引位数}$$

$$\text{物理页号位数} = \text{主存地址位数} - \log_2(\text{页大小})$$

算例（【2024-17】）：虚拟地址 32 位，主存地址 30 位，页大小 1 KB，TLB 共 32 个表项，**4** 路组相联，求 TLB 标记字段至少几位。

- 页内偏移 $= \log_2 1\text{KB} = 10$ 位 $\rightarrow$ **VPN** $= 32 - 10 = 22$ 位。
- 组数 $= 32/4 = 8 \rightarrow$ 组索引 **3** 位。
- **TLB** 标记 $= 22 - 3 = 19$ 位。（答案 C）

5.TLB、页表、*Cache* 的组合访问

主存数据分两类：存储页表内容的页表块与存储程序数据的数据块。页表块同样会被 Cache 缓存（Cache 硬件不区分内容，只按访问频度缓存）。

三种典型情形：

- 理想情况（不访问主存）：页表项在 Cache 中命中，数据也在 Cache 中命中。只需 2 次 Cache 访问（第一次拿页表项，第二次拿数据）。
- 较糟糕情况（访问主存 **2** 次）：无缺页（页在主存），但页表项在 Cache 中缺失、数据也在 Cache 中缺失。需 2 次主存访问，并把两个块分别调入 Cache。
- 最糟糕情况：在最糟糕情况基础上，访问目标数据触发缺页，需要访问磁盘。

TLB、页、**Cache** 命中/缺失组合的可行性：

场景	**TLB**	页	**Cache**	可行性	说明
最好：不访主存	命中	命中	命中	可能	TLB 命中则页必命中
访主存 1 次	命中	命中	缺失	可能	数据在主存但不在 Cache
访主存 1 次	缺失	命中	命中	可能	TLB 缺失但页仍在主存
访主存 2 次	缺失	命中	缺失	可能	两次主存：查页表 + 取数据
最糟：访磁盘	缺失	缺失	缺失	可能	页不在主存，数据必然不在 Cache
不可能	命中	缺失	缺失	不可能	页缺失说明信息不在主存，TLB 中不可能有有效表项
不可能	命中	缺失	命中	不可能	同上
不可能	缺失	缺失	命中	不可能	页缺失则数据不在主存，必然不在 Cache

一句话记住「不可能」：TLB 命中 ⇒ 页必命中

TLB 是页表的缓存，只有当一个页面已经调入主存（页命中）时，它的有效映射才可能被缓存到 TLB 中。所以「**TLB** 命中而页缺失」永远不可能出现。同理，页缺失 ⇒ **Cache** 必然缺失（数据根本不在主存）。

6.TLB 与 *Cache* 的对比

对比维度	**Cache**	**TLB**
缓存对象	主存中的数据/指令块	内存页表中的页表项
查询 key	物理地址（块号部分）	虚拟页号 **VPN**
输出结果	数据内容	物理页号 **PPN** + 权限
解决的问题	CPU 与主存的速度差	每次地址翻译都要访主存查页表的开销
位置	CPU 芯片内（L1/L2/L3）	MMU 内部
典型容量	32 KB ~ 32 MB（多级）	32 ~ 1024 项（1 ~ 2 级）
映射方式	直接 / 组相联 / 全相联	全相联或组相联
缺失处理	硬件自动完成	硬件或 **OS**（视架构）
组成器件	**SRAM**	**SRAM / CAM**
进程切换	无需刷新（key 是物理地址，全局唯一）	需要刷新（key 是虚拟地址，不同进程同 VA 映射不同 PA）

一句话区分：**Cache** 加速「取数据」，**TLB** 加速「找地址」。

TLB 与 Cache 的统考考法

【2010-17】不可能发生的命中组合？答 **D**（**TLB** 命中、**Cache** 命中、**Page** 未命中）——理由同上。【2020-15】关于 TLB 和 Cache 的错误叙述？答 **D**（都由 **DRAM** 组成）。两者都由 **SRAM/CAM** 组成（追求与 CPU 匹配的速度）；只有主存才由 DRAM 组成。其余 A（命中率都与局部性有关）、B（缺失后都要访主存）、C（缺失处理都可由硬件实现）都正确。【2024-18】见前文「MMU 不检测 Cache 缺失」。

7.多级页表

单级页表要求整张页表在主存中连续存放，当虚拟地址空间很大而实际使用的页面很少时会浪费大量主存。多级页表把页号分级，每级页表自身也存成一个页面，按需分配。

虚页号位数与页表大小：设虚拟地址 n 位、页大小 2^p 字节、页表项 e 字节：

$$\text{虚页号位数} = n - p, \qquad \text{页表项数} = 2^{n-p}, \qquad \text{单级页表大小} = 2^{n-p} \times e \text{ 字节}$$

$$\text{一级页号位数} = p - \log_2 e, \qquad \text{页表级数} = \left\lceil \frac{n-p}{p - \log_2 e} \right\rceil$$

算例：虚拟地址 32 位，页大小 4 KB，页表项 4 B。

- 虚页号 $= 32 - 12 = 20$ 位 $\rightarrow$ 页表项数 $= 2^{20} = 1\text{M}$ 项 $\rightarrow$ 单级页表 $= 1\text{M} \times 4\text{B} = 4$ MB，且必须连续分配 4 MB。
- 采用两级页表：每级页表自身占一个 4 KB 页面 = 1024 个页表项 $\rightarrow$ 每级页号 **10** 位（$10+10=20$ ✓）。
- 一级页表（页目录）只占 4 KB；每个二级页表也只占 4 KB，只为实际用到的虚页号区间分配二级页表。

算例：虚拟地址 48 位、页大小 4 KB、页表项 8 B。$p = 12$，$\log_2 e = 3$，每级页号 $= 12 - 3 = 9$ 位；虚页号 $= 48 - 12 = 36$ 位；级数 $= \lceil 36/9 \rceil = 4$ 级。这与 x86-64 实际使用的 4 级页表一致。

多级页表的两点澄清

1. 多级页表降低的是「页表本身占用的连续主存空间」，不是「一次地址转换的访存次数」。相反，级数越多、TLB 缺失时遍历页表的访存次数越多（4 级页表在 TLB 缺失时最多需 4 次读主存）。实际中靠 TLB 命中 + Cache 缓存页表块把开销压下来。
2. 能装下页面的最小单位就是「页」。页表项数不得超过一页能装下的项数，即每级页号位数 $= p - \log_2 e$，因为一级页表必须正好占一个页面。

（三）段式虚拟存储器

基本原理：按程序的逻辑结构（函数、代码段、数据段、堆、栈等）把虚拟地址空间划分为长度不等的段。每段从 0 开始编址，段内地址连续。

虚拟地址结构：`[段号 S][段内偏移 W]`。

段表：每个进程一张，每个段一个段表项。段表项含：

字段	含义
段长	该段的长度（用于越界检查）
段基址	该段在主存中的起始物理地址
有效位 / 装入位	该段是否已调入主存
访问权限	可读 / 可写 / 可执行
修改位、访问位	供替换使用

地址转换：

$$\text{物理地址} = \text{段表项中的段基址} + \text{段内偏移}$$

转换时必须检查段内偏移是否超过段长，超过则触发越界异常（这是段式存储管理的核心保护机制，页式则不需要，因为页内偏移位数天然不会越界）。

优点：按逻辑单位划分，便于共享（多个进程段表项指向同一段基址）、便于保护（按段设权限）、便于动态链接与动态增长（段长可变）。缺点：段长不等导致主存分配复杂，易产生外部碎片；需要拼接段基址与段内偏移。

（四）段页式虚拟存储器

基本思想：先按逻辑结构分段，再把每段分页。兼取段式「便于共享保护」与页式「无外部碎片、分配简单」的优点。

虚拟地址结构：`[段号 S][段内页号 P][页内偏移 W]`。

查找过程（两次查表）：

1. 用段号查段表，得到该段页表的起始地址与段长（用于越界检查）；
2. 用段内页号查页表，得到物理页号（页框号）；
3. 物理页号与页内偏移拼接成物理地址。

地址转换效率问题：

$$\text{段页式一次访存的主存访问次数} = 3 \text{ 次（查段表} + \text{查页表} + \text{取数据）}$$

所以段页式必须配 TLB。**TLB** 中直接缓存「段号 + 段内页号 → 物理页号」的完整映射，命中时一次即可得到物理页号，避免两次查表。

段页式题目的两条注意

1. 地址字段是三段，不是两段。只写「段号 + 页内偏移」是错的，必须包含段内页号。
2. 越界检查在段这一级做（用段表中的段长），页这一级不需要。

虚拟存储器的统考考法汇总

年份题号	考点	答案要点
2010-17	TLB/页/Cache 组合	不可能：TLB 命中而页缺失
2013-16	TLB 全相联查表	`03FFF180H` → `0153180H`
2015-16	虚存 + 直写访存次数	至少 1 次（写回结果必须写主存）
2019-14	缺页处理	返回原指令重新执行
2020-15	TLB 与 Cache 对比	都由 SRAM/CAM 组成，不是 DRAM
2022-15	页式地址转换	页内偏移 12 位，`00082H`=130 → `018840H`
2024-16	两个层次的对比	主存-外存层次是全相联，不是直接映射
2024-17	TLB 标记位数	VPN 22 位 − 组索引 3 位 = 19 位
2024-18	MMU 检测的事件	Cache 缺失不在 MMU 阶段检测

Cache-主存与主存-辅存四条对照（【2024-16】的核心）

1. 交换单位：Cache-主存用块，主存-辅存用页 ✓
2. 实现者：Cache-主存的替换由硬件实现，主存-辅存的替换由软件（**OS**）实现 ✓
3. 写策略：Cache-主存可回写或直写；主存-辅存必然回写 ✓
4. 映射方式：Cache-主存可用直接/组相联/全相联；主存-辅存通常用全相联——所以说「主存-外存层次通常采用直接映射方式」是错的。

本章小结表

知识点	核心结论
按存取方式分类	RAM 随机、SAM 顺序（磁带）、DAM 直接（磁盘/光盘）、CAM 按内容（TLB、全相联 Cache）
ROM 与 RAM	都采用随机存取方式；ROM 是非易失半导体存储器；CD-ROM 是光存储器，不是 ROM
SSD 归属	半导体存储器，基于 Flash（EEPROM），不是磁盘
存取时间 vs 存取周期	$T_C = T_A + T_R$；连续访问与带宽必须用 T_C
带宽	$B_m = W/T_C$；提高靠缩短 T_C 或加宽 W
层次化四条规律	越往下速度越慢、容量越大、位价格越低、访问频次越低
两个层次的缺失处理	Cache 缺失由硬件处理；缺页由操作系统处理
局部性	时间局部性 = 重复访问；空间局部性 = 邻近访问；步长越小越好
SRAM	6 管双稳态触发器；快、贵、集成度低、不需刷新；用作 **Cache**

知识点	核心结论
DRAM	1 管 1 电容；慢、便宜、集成度高、需刷新；用作主存；破坏性读出需再生
DRAM 刷新	按行、不需列地址、地址由内部计数器产生；集中/分散/异步三种
异步刷新间隔	$T_{\max}$/行数，如 2ms/128 = 15.625 μs
集中刷新死区	= 行数 × t_c；行数越多死区越长
DRAM 地址复用	行列分两次传送；物理地址引脚 = max(行位数, 列位数)；4M × 8 芯片共 19 引脚
Flash	读快写慢；按页写、按块擦；写前必须先擦块；寿命有限
芯片扩展	位扩展片数 = N/n，字扩展组数 = M/m，总片数 = $(N/n)(M/m)$
位扩展连接	地址/CS/WE 全并联，数据线各自独立，各片同时工作
字扩展连接	数据/低位地址/WE 全并联，片选各自独立，各片分时工作
线选法	无译码器、地址空间不连续、有空洞
译码片选法	需译码器、地址空间连续、主流方案
MAR 位数	由主存地址空间决定，与实际安装容量无关
多模块编址	高位交叉（顺序）：相邻地址同体，不能并行；低位交叉：相邻地址异体，可流水
低位交叉体号	体号 = 地址 mod m
低位交叉时间	准备 n 个字：$T + (n-1)\tau$；总时间 $T + (n+1)\tau$；流水条件 $T = m\tau$
低位交叉吞吐	稳定流水时一个存储周期提供 m 个字，平均每字 τ
访存冲突	同体且序列间隔步数 $< m$
磁盘组成	磁盘控制器 + 磁盘驱动器 + 盘片；磁头号 = 盘面号，磁道号 = 柱面号
磁盘地址结构	（驱动器号，柱面号，盘面号，扇区号）；柱面号在前以省寻道
平均存取时间	寻道时间 + 旋转延迟 + 传输时间（+ 控制器延迟）
旋转延迟	$T_r = 1/(2r)$（r 转/秒）$= 30/r$ 秒（r 转/分）
传输时间	$T_t = b/(rN)$，b 为字节数、N 为每磁道字节数、r 转/秒
容量	格式化容量 < 非格式化容量；格式化容量 = 柱面 × 盘面 × 每道扇区 × 每扇区字节
磁盘读写单位	扇区，不是字节
RAID	镜像与奇偶校验提高可靠性；条带化只提性能，不提供冗余
SSD 读写	以页读写、以块擦除；写前必擦；随机访问快、无寻道与旋转延迟
磨损均衡	动态：优先选擦除次数少的块；静态：自动迁移数据，老块承担读任务（更优）
Cache 组成	SRAM；内容为主存副本，不增加系统总容量；对程序员透明
Cache 标识位	有效位 V + 标记 Tag（+ 写回时脏位）
命中率与平均访问时间	串行：$T_a = t_c + (1-h)t_m$；并行：$T_a = ht_c + (1-h)t_m$
访问效率	$e = t_c/T_a$
直接映射	行号 = 块号 mod 行数；Tag 最短；1 个比较器；无需替换算法
全相联	任意行；只有 Tag + 块内地址；n 个比较器；命中率最高
组相联	组号 = 块号 mod 组数，组内任意；k 个比较器；最常用
退化关系	$k = 1$ 退化为直接映射；组数 = 1 退化为全相联
地址划分	$c = \log_2$ 行数（或 $d = \log_2$ 组数），$w = \log_2$ 块大小，$t = n - c - w$

知识点	核心结论
每行位数	Tag + 有效位 1 + 数据（+ 写回时脏位 1）
FIFO	淘汰最早进入者；简单；可能 Belady 异常
LRU	淘汰最久未访问者；命中率最高；LRU 位 $=\log_2$ 组内块数
LFU	淘汰访问次数最少者；有历史包袱；并列需打破平局
写直达	同时写 Cache 与主存；不需脏位；可配写缓冲
写回	只写 Cache，替换时写回；需要脏位；主流
写分配 / 非写分配	写缺失时调块（配写回）/ 不调块直接写主存（配直写）
页式虚存映射	全相联；交换单位是页（4 KB ~ 64 KB）
虚实转换	页表项地址 = PTBR + VPN × 页表项大小；物理地址 = 页框号 + 页内偏移
缺页返回点	重新执行原指令，不是下一条
TLB	VPN 作关键字；全相联或组相联；随机替换；进程切换需刷新
TLB 标记位数	VPN 位数 − 组索引位数
TLB 与 Cache	Cache 用物理地址查、加速取数；TLB 用虚拟地址查、加速找地址
不可能组合	TLB 命中而页缺失；页缺失而 Cache 命中
多级页表	每级页号位数 $= p - \log_2 e$；级数 $= \lceil (n-p)/(p-\log_2 e) \rceil$；按需分配、节省连续主存
段式	段长可变；地址 = 段号 + 段内偏移；需越界检查；便于共享保护，有外部碎片
段页式	段号 + 段内页号 + 页内偏移；查段表、页表两次，需 TLB 加速

易错清单

本章最容易被扣分的地方

1. 以为 **ROM** 采用顺序存取。ROM 与 RAM 都采用随机存取方式，差别只在能否随意改写。
2. 把「半导体存储器都采用随机存取」当成正确。全相联 Cache 的比较器、快表 TLB 是按内容访问（CAM）。
3. 把 **CD-ROM** 归入 **ROM**、把 **SSD** 归入磁盘。CD-ROM 是光存储器；SSD 是半导体 Flash（EEPROM）。计算机术语中的 ROM 特指半导体芯片。
4. 混淆存取时间与存取周期。算带宽、算连续读取最小间隔、算流水线启动间隔，必须用存取周期 T_C。
5. 把 **MAR** 位数与实际安装容量挂钩。MAR 位数由主存地址空间决定，与装了多少内存无关（【2011-15】）。
6. **DRAM** 地址引脚忘了除以 **2**。4M × 8 位 DRAM 只有 11 根地址引脚，不是 22 根；行列不等时取 max。
7. **DRAM** 引脚数被系统总容量干扰。题目问「芯片」引脚数时，总容量只用于算片数。
8. 认为刷新需要列地址。刷新按行进行，只需行地址，且由芯片内部刷新计数器给出，不需要 CPU 提供。
9. 集中刷新的死区与行数关系记反。行数越多（矩阵越「高」），死区越长；所以设计上倾向减少行数、增加列数。
10. 把分散刷新的存取周期当成不变。分散刷新把存取周期拉长一倍（读写 + 刷新各半），有效访存速度减半。
11. 字扩展时忘记位扩展。4K × 4 位芯片装 8 位宽内存必须先 2 片一组，否则片数少一半。
12. **SRAM** 与 **DRAM** 的地址引脚混算。SRAM 不复用地址，引脚数就是完整逻辑地址位数。
13. 低位交叉的体号算成高位。体号 $=$ 地址 mod m；若按地址除以 m 取整，就变成高位交叉了。
14. 低位交叉时间公式漏掉总线开销。准备 n 个字是 $T + (n-1)\tau$，含送首地址与传送末字的总时间是 $T + (n+1)\tau$，两者别混用。

15. 访存冲突只比较体号。必须同时满足「同体」且「序列间隔步数 $< m$」。
16. 磁盘旋转延迟的单位算错。转速给 r/min 时用 $30/r$ 秒；给 r/s 时用 $1/(2r)$ 秒。题目用「r 秒/转」这类反向单位时要格外小心。
17. 把磁盘最小读写单位当成字节。是扇区（传统 512 B）。
18. 认为格式化容量大于非格式化容量。格式化要在每扇区加头、尾与间隔，所以格式化容量更小。
19. **Cache** 映射时把「行数」与「组数」搞混。组索引位数由组数决定，与路数无直接关系：2^d 组就是 d 位。
20. 写回忘加脏位、直写多算了脏位。脏位只在写回策略下存在。
21. 地址位数计算时把 **Cache** 总容量当成数据区容量。行数由数据区容量算；Tag 与元数据位只在算总容量/每行位数时才加。
22. **LRU** 模拟时命中不更新顺序。每次访问（无论命中与否）都要把被访问块移到「最新」位置。
23. 以为直接映射需要替换算法。每个主存块只有唯一位置，调入即覆盖，不需要任何替换算法。
24. 认为 **TLB** 缺失与缺页是同一回事。TLB 缺失只是快表里没有该表项，页可能仍在主存（此时只需查慢表）；缺页是页根本不在主存，必须访磁盘。
25. 写出「**TLB** 命中而页缺失」的组合。这不可能发生——TLB 命中意味着该页已在主存。
26. 以为多级页表降低了一次地址转换的访存次数。恰恰相反，级数越多，TLB 缺失时遍历页表的访存次数越多；多级页表省的是页表本身占用的连续主存空间。
27. 段式虚存忘了越界检查。段长可变，必须检查段内偏移是否超过段长；页式不需要。
28. 段页式地址漏掉「段内页号」。结构是段号 + 段内页号 + 页内偏移三段。
29. 认为缺页处理完成后执行下一条指令。缺页属于故障（**Fault**），处理完必须重新执行原指令。
30. 以为 **Cache** 缺失会陷入内核。Cache 缺失完全由硬件处理，对程序员透明，不引起进程切换；只有缺页才由操作系统处理。
31. 说「**Cache** 增加了系统存储容量」。Cache 只是主存的副本，不增加总容量；增加容量是虚拟存储器（主存-辅存层次）的目标。
32. 主存-辅存层次误以为用直接映射。因缺页代价极高，主存-辅存层次采用全相联，且写操作只回写（【2024-16】）。
33. 认为 **SSD** 的读和写都比磁盘快。SSD 随机访问性能与读性能优异，但写操作要先擦块，还可能触发数据迁移，不一定快。
34. 把「透明」理解成「看得见」。体系结构中透明 = 看不见 = 不需要管。Cache 对程序员透明，是指程序员不用写任何调度 Cache 的代码。

本章历年统考考点分布（按知识点）

- 存储器分类与性能指标：2010-16（RAM/ROM 性质判断）、2011-14（不采用随机存取方式的器件，考 CD-ROM 属直接存取）。
- 半导体存储器：2014-15（4M × 8 DRAM 引脚数 19）、2015-17（需要刷新的是 SDRAM）、2012-16（Flash 读写速度不一样快）、2018-17（2K × 1 DRAM 行列数取 32×64）。
- 主存储器：2009-15（ROM/RAM 芯片数 2、30）、2010-15（`0B1FH` 所在芯片最小地址 `0800H`）、2011-15（MAR 26 位）、2016-16（SRAM 14 片）、2021-15（按字编址，32 片）、2023-15（RAM:ROM=3:1，ROM 地址范围）、2015-18（4 体交叉冲突对）、2017-13（3 个存储周期）、2013-43（8 体交叉 + 突发传送事务时间 85 ns）。
- 外部存储器：2013-20（RAID 可靠性措施）、2013-21（平均存取时间 9.4 ms）、2015-20（12.2 ms）、2019-20（磁盘最小读写单位不是字节）。
- **Cache**：2009-21（命中率 95%）、2009-14（组号 4）、2014-16（指令/数据 Cache 分离的目的）、2015-15（Cache 总容量 148 Kbit）、2016-15（缺失率 12.5%）、2021-16（每行 275 位）、2022-16（比较器 8 个、20 位）、2015-16（直写至少访主存 1 次）。
- 虚拟存储器：2010-17（不可能的命中组合）、2013-16（TLB 查表得 `0153180H`）、2019-14（缺页返回原指令）、2020-15（TLB/Cache 由 SRAM 组成）、2022-15（`018840H`）、2024-16（主存-辅存层次全相联）、2024-17（TLB 标记 19 位）、2024-18（Cache 缺失不在 MMU 检测）。
- 局部性：2017-14（数组 a 的时间与空间局部性皆有）。

七、本章公式速查

公式	表达式	说明
存取周期	$T_C = T_A + T_R$	恢复时间 T_R
存储器带宽	$B_m = W/T_C$	W 为数据位宽（字节）
硬件元数据	元数据比例 $=\frac{\text{每行位数}-\text{数据位数}}{\text{每行位数}}$	衡量 Tag 开销
DRAM 地址引脚	$p = \lceil \log_2 M/2 \rceil$ 或 $\max(\log_2 r, \log_2 c)$	M 为字数
集中刷新死区	$T_{\text{死区}} = r \cdot t_c$	r 为行数
异步刷新间隔	$\Delta = T_{\max}/r$	$T_{\max}$ 通常 2 ms 或 64 ms
芯片总数	$N_{\text{片}} = (N/n)(M/m)$	目标 $M \times N$、单片 $m \times n$
片内地址线	$\log_2 m$	并联到所有芯片
片选译码位数	$\log_2(M/m)$	接译码器
低位交叉准备时间	$T + (n-1)\tau$	流水条件 $T = m\tau$
低位交叉总时间	$T + (n+1)\tau$	含送首地址与传送末字
高位交叉准备时间	nT	与并行无关
磁盘旋转延迟	$1/(2r)$ 秒（r 转/秒）；$30/r$ 秒（r 转/分）	平均半圈
磁盘传输时间	$b/(rN)$	N 为每磁道字节数
平均存取时间	$T_s + T_r + T_t$	寻道 + 旋转 + 传输
格式化容量	柱面 × 盘面 × 每道扇区 × 每扇区字节	—
命中率	$h =$ 命中次数/总访存次数	—
平均访问时间（串行）	$T_a = t_c + (1-h)t_m$	先查 Cache 再查主存
平均访问时间（并行）	$T_a = h\,t_c + (1-h)t_m$	同时发起
访问效率	$e = t_c/T_a$	—
直接映射行号	$=$ 主存块号 mod Cache 行数	—
组相联组号	$=$ 主存块号 mod Cache 组数	—
Cache 行数	= 数据区容量/块大小	—
组数	= 总行数/路数	—
地址字段	$w = \log_2$ 块大小，c 或 $d = \log_2$ 行数/组数，$t = n - c - w$	—
每行位数	$t + 1 +$ 数据位$(+1_{\text{写回}})$	—
LRU 位	$\log_2$(组内块数)	—
页内偏移位数	$\log_2$(页大小)	—
物理地址位数	PPN 位数 + 页内偏移位数	—
页表项地址	PTBR + VPN × 页表项大小	—

公式	表达式	说明
单级页表大小	$2^{n-p} \times e$ 字节	n 虚拟地址位、p 页内偏移位、e 项大小
多级页表每级页号位数	$p - \log_2 e$	每级页表恰占一页
TLB 标记位数	VPN 位数 $-\log_2$(组数)	组数 = 表项数/路数

第十一章指令系统

本章考纲要求

第二部分计算机组成原理［考查目标］

1. 掌握指令集体系结构的基本知识和基本实现方法，对计算机硬件相关问题进行分析，并能够对相关部件进行设计。
2. 理解计算机系统的整机概念，能够综合运用计算机组成的基本原理和基本方法，对高级编程语言（C 语言）程序中的相关问题进行分析，具备软硬件协同分析和设计能力。

四、指令系统（一）指令系统的基本概念（二）指令格式（三）寻址方式（四）数据的对齐和大/小端存放方式（五）CISC 和 RISC 的基本概念（六）高级语言程序与机器级代码之间的对应

1. 编译器、汇编器和链接器的基本概念
2. 选择结构语句的机器级表示
3. 循环结构语句的机器级表示
4. 过程（函数）调用对应的机器级表示

指令系统是「软件与硬件的交界面」：向上，它是编译器生成代码时唯一能使用的「词汇表」；向下，它决定了数据通路与控制器必须实现哪些功能。本章在统考中的分量很重，并且呈现出非常鲜明的两极化特征——一类是纯计算题（扩展操作码能表示多少条指令、相对寻址的转移目标地址、结构体成员地址与填充字节），另一类是纯概念辨析题（CISC 与 RISC 的特征归属、编译器/汇编器/链接器的产物、寄存器的保存责任）。计算题套路固定、可以练到不失分；概念题则要求把「谁负责保存」「谁负责修改」这类归属关系记牢，因为出题人恰恰喜欢把 RISC 的特征安到 CISC 头上。

本章的一条主线必须始终在脑子里：指令做的事，无非是「算出有效地址 → 去某个地方取操作数 → 运算 → 写回」。寻址方式解决的是第二步，栈帧解决的是「过程调用时这些数据放在哪里」，对齐解决的是「放的时候地址有什么约束」。

一、指令系统的基本概念

（一）指令与指令系统

指令（**Instruction**）是计算机执行某种操作的命令，是 CPU 能够直接识别并执行的最小功能单位。一条指令由操作码和若干地址码组成，前者说明「做什么」，后者说明「对谁做」。

指令系统（**Instruction Set**），又称指令集，是一台计算机中所有指令的集合。指令系统是计算机硬件与软件之间的接口规范：硬件必须实现指令系统中每一条指令的功能，而任何软件最终都必须翻译成指令系统中的指令序列才能运行。指令系统一经确定，通常在一系列机型中保持兼容（例如 x86 系列不断扩充新指令但绝不删除老指令），这就是为什么它「只能加不能减」。

（二）指令集体系结构（ISA）

指令集体系结构（**Instruction Set Architecture**，**ISA**）是机器语言程序员或编译程序编写程序时所看到的计算机系统的抽象，它规定了：

- 指令的格式、类型与功能；
- 操作数的寻址方式；
- 可访问的寄存器组与主存空间大小、编址方式；
- 数据类型与格式（整数、浮点数及其长度）；
- 指令执行的顺序控制方式（顺序、转移、中断与异常）。

ISA 是「软硬件之间的契约」。同一个 **ISA** 可以有很多种不同的硬件实现：同样是 x86-64 指令系统，Intel 与 AMD 的微架构完全不同（流水线级数、Cache 大小、执行单元数量都不同），但对程序员而言程序行为一致。反过来，ISA 不同意味着机器语言不兼容。统考里凡是问「计算机的层次结构中，属于软件与硬件界面的是什么」，答案就是 ISA。

ISA 属于硬件与软件的交界面

ISA 在程序员眼中就是「这台机器能做哪些事」的完整描述；在硬件设计者眼中则是「必须实现的功能清单」。它不包括具体的微架构实现（流水线级数、Cache 容量、控制器是微程序还是硬布线）。

（三）指令字长与机器字长

这三个长度必须分清，考试常把它们放进同一条选择题里：

名称	定义	决定谁
机器字长	CPU 一次整数运算能处理的二进制位数，实际上就是定点运算数据通路的位数（ALU、通用寄存器、加法器的宽度）	通用寄存器位数、ALU 宽度
存储字长	一个存储单元（存储字）的二进制位数	主存数据线宽度、MDR 位数
指令字长	一条指令的二进制位数	**IR** 的位数；按字节编址时决定一条指令占几个字节

关键结论：

1. 机器字长一般等于通用寄存器的位数，也等于 **ALU** 的宽度。常说「32 位机器 / 64 位机器」，指的就是机器字长。
2. 指令字长与机器字长之间没有任何约束关系。机器字长强调的是参与 ALU 运算的操作数位数，而有些指令根本没有参与 ALU 运算的操作数（如 `nop`、`jmp`）。所以「指令字长一定等于机器字长」是错的。
3. 指令字长决定 **IR** 的位数：若指令字长最大为 32 位，IR 至少 32 位。
4. 机器字长与存储字长的关系：机器字长通常等于存储字长，也可以是存储字长的整数倍（2 倍、4 倍），即一次能处理多个存储字。
5. 指令字长与存储字长的关系：一条指令可以占半个、一个、两个存储字，分别称为半字长指令、单字长指令、双字长指令。若指令字长是存储字长的 2 倍，取一条指令需要访存 2 次。

「机器字长」不等于「字」

在 80x86 系列中，一个「字」的宽度被约定为 16 位，所以 32 位叫「双字」；而在 IBM 303X 系列中一个

字是 32 位。「字」是计量单位，其宽度由具体机器约定，不必然等于机器字长。

（四）定长指令字与变长指令字

- 定长指令字：所有指令位数相同。优点是译码简单、便于流水线实现（取指阶段一次即可取到完整指令）。RISC 普遍采用定长指令字，典型为 32 位。
- 变长指令字：不同指令长度不同。优点是编码紧凑、能用短指令表示常用操作；缺点是译码复杂、难流水。CISC 普遍采用变长指令字。

二、指令格式

（一）操作码与地址码

一条指令在机器中表示为：

$$\text{指令} = \underbrace{\text{操作码 OP}}_{n\text{ 位}} \;\Big\|\; \underbrace{\text{地址码}A_1\|A_2\|\cdots\|A_k}_{k\text{ 个地址字段}}$$

- 操作码字段（**OP**）：指出指令应执行什么操作，如加、减、传送、转移。操作码的位数 n 决定了该机器最多能有 2^n 条指令（定长操作码时）。
- 地址码字段：给出操作数或操作数地址的信息。地址码的个数决定了指令是几地址指令，也决定了「取操作数需要访存几次」。

若指令字长为 L 位、操作码占 n 位、共有 k 个地址字段，则每个地址字段平均可以分到 $\frac{L-n}{k}$ 位（定长操作码、等长地址字段的情形）。

（二）零地址 / 一地址 / 二地址 / 三地址指令

按地址码个数划分，指令格式有以下几种，它们各自隐含了「操作数从哪来、结果到哪去」的约定：

格式	形式	操作数来源与结果去向	典型指令
零地址	`OP`	无显式地址码，操作数隐含（如在堆栈顶）	`nop`、`halt`、堆栈机的 `add`
一地址	`OP A`	只有一个显式操作数，另一操作数隐含（通常是累加器 ACC）	`inc A`、`neg A`；堆栈机的单操作数指令
二地址	`OP A1, A2`	两个操作数，结果通常写回 A_1（目的操作数）	`add A1, A2`（含义 $A_1 \leftarrow A_1 + A_2$）
三地址	`OP A1, A2, A3`	两个源操作数、一个目的地址，三者互不重叠	`add A3, A1, A2`（$A_3 \leftarrow A_1 + A_2$）

地址码个数与访存次数的关系

设指令字长与存储字长相等、取指令本身需要访存 **1** 次，且操作数均在主存中、结果写回主存，则取操作数并写回的总访存次数为：

- 零地址指令：1 次（只取指令，操作数隐含在寄存器/栈顶）；
- 一地址指令：$1+1=2$ 次（取指令 + 取操作数；结果可能写回累加器，不访存）；
- 二地址指令：取指令 1 次 + 取两个操作数 2 次 + 写回 1 次，最多 4 次；
- 三地址指令：取指令 1 次 + 取两个源操作数 2 次 + 写结果 1 次，共 4 次。

要点是：访存次数取决于「有几个操作数在内存里」以及「是否需要写回内存」，而不是简单地等于地址码个数。若某个地址字段采用寄存器寻址，则该操作数不访存。

二地址与三地址的访存次数为何常常相同

二地址指令把结果写回 A_1，只需 1 次写回；三地址指令把结果写到 A_3，也是 1 次写回。二者都是「读两个操作数 + 写一个结果」。真正影响访存次数的是源操作数的个数与它们的位置，不是地址码的个数本身。

常见出题角度

给出「指令字长 16 位、操作码固定 n 位、有 4 种寻址方式、单地址指令中形式地址多少位」这类条件，问直接寻址的可寻址范围（典型年份：2020 年统考第 16 题）。解题时把每一条指令的位宽像预算一样逐项扣减：总位数 − 操作码位数 − 寻址方式位数 = 形式地址位数，再按「无符号地址」把 2^m 算成范围。

（三）定长操作码

定长操作码：所有指令的操作码长度相同，均为 n 位。此时最多可定义 2^n 条指令。

设计要点：若指令系统共有 m 条指令，则操作码位数必须满足

$$n \geq \lceil \log_2 m \rceil$$

例如指令系统有 48 条指令，则 $\lceil \log_2 48 \rceil = 6$ 位（$2^5 = 32 < 48 \leq 64 = 2^6$）。

定长操作码的优点是译码简单、译码时间短，缺点是当实际指令数远小于 2^n 时编码空间浪费严重。

（四）扩展操作码（可变长操作码）

当指令字长固定、地址码个数又要变化时（零地址、一地址……共存），就使用扩展操作码：操作码长度随地址码个数的减少而变长。核心思想是把短操作码中用不到的编码留作「扩展标志」，供长操作码使用。

*1.*编码方法

以「零地址、一地址、二地址、三地址四种格式共存」为例。设指令字长为 16 位，每个地址字段 4 位，操作码起始长度为 4 位。

- 三地址指令：格式为 `OP(4 位) | A1(4 位) | A2(4 位) | A3(4 位)`。4 位操作码共 16 个编码，若三地址指令用了 m 条（$m < 16$），则剩下 $16 - m$ 个编码保留不用，作为二地址指令操作码的扩展前缀。
- 二地址指令：格式为 `OP(8 位) | A1(4 位) | A2(4 位)`。其操作码由 8 位构成，但高 4 位必须是被保留的「扩展标志」，通常是 `1111`。于是可用编码数为 $16 - m$ 种（对应低 4 位的所有取值）。若二地址指令用了 k 条，则剩余 $(16 - m) - k$ 个编码继续保留。
- 一地址指令：格式为 `OP(12 位) | A1(4 位)`。可用编码数为 $\big[(16 - m) - k\big] \times 16$ 种，用掉 j 条后剩余继续保留。
- 零地址指令：格式为 `OP(16 位)`。可用编码数为 $\big[(16 - m) - k\big] \times 16 - j$ 种（每保留一个 12 位前缀，就能展开出 16 个 16 位编码）。

计数公式（必背）

设 p 为地址字段位数、q 为操作码起始位数（三地址时的操作码位数），逐级向下：

- 三地址最多 2^q 条；

- 二地址最多 $(2^q - m) \times 2^p$ 条，其中 m 为已定义的三地址指令数；
- 一地址最多 $[(2^q - m) \times 2^p - k] \times 2^p$ 条，其中 k 为已定义的二地址指令数；
- 零地址最多 $[(2^q - m) \times 2^p - k] \times 2^p \times 2^p - j$ 条，其中 j 为一地址指令数。

口诀：每减少一个地址字段，操作码就多出 p 位，可用编码数「乘以 2^p」，再减去已用掉的条数。

扩展操作码的三个易错点

1. 保留的是「编码」，不是「位」。是短操作码的未使用编码成为长操作码的前缀，而不是操作码字段本身变短。
2. 每一级都要先「展开」再「扣减」。正确顺序是：先由上一级的剩余编码数乘以 2^p 得到本级的全部可用编码，再减掉本级已定义的指令数。
3. 零地址指令不占地址字段，所以它的可用编码数不需要再乘 2^p，直接就是上一级剩余的前缀数。

*2.*完整算例一：扩展操作码的设计与验证

题目：某机器指令字长 16 位，采用扩展操作码，地址字段宽度为 4 位，要求支持三地址、二地址、一地址、零地址四种格式。现已知三地址指令 6 条、二地址指令 120 条、一地址指令 200 条，问零地址指令最多还能定义多少条?

第一步：确定三地址指令的操作码位数。

三地址指令 6 条，起始操作码只需 $q = 4$ 位即可（$2^4 = 16 \geq 6$），且必须预留扩展编码，因此取 $q = 4$，地址字段各 4 位，校验位数：

$$4 + 4 + 4 + 4 = 16 \text{ 位} \quad \checkmark$$

三地址指令用掉 6 个编码，剩余 $16 - 6 = 10$ 个编码保留作扩展。

第二步：展开二地址指令的可用编码数。

二地址操作码为 8 位（高 4 位是保留标志，低 4 位可自由取值），可用编码数为

$$(16 - 6) \times 2^4 = 10 \times 16 = 160 \text{ 条}$$

二地址指令需要 120 条，$120 \leq 160$，容量够用。剩余 $160 - 120 = 40$ 个编码保留扩展。

第三步：展开一地址指令的可用编码数。

一地址操作码为 12 位，可用编码数为

$$40 \times 2^4 = 640 \text{ 条}$$

一地址指令需要 200 条，$200 \leq 640$，容量够用。剩余 $640 - 200 = 440$ 个编码保留扩展。

第四步：展开零地址指令的可用编码数。

零地址操作码即为完整的 16 位，不再消耗地址字段，可用编码数为

$$440 \times 2^4 = 7040 \text{ 条}$$

结论：零地址指令最多能定义 **7040** 条。

验证（自洽性检验）：把四类指令的编码区间写出来，检查有没有重叠。若规定三地址操作码取 0000~0101（6 条），保留 0110~1111（10 个）；二地址高 4 位取 0110~1111、低 4 位任意，其中前 120 个（即从 0110 0000 开始的 120 个）已用，其余 40 个保留；一地址由这 40 个前缀各展开 16 个编码，前

200 个已用，其余 440 个保留；零地址由这 440 个前缀各展开 16 个编码。逐级区间互不相交，且总数 $16\times16\times16\times16=65536$ 恰好等于 16 位操作码的全部编码空间：$6\times16^3+120\times16^2+200\times16+7040=24576+30720+3200+7040=65536$ ✓

反向验证：什么时候必须加长操作码

若上题改成「三地址指令 13 条、二地址指令 300 条」，取 $q=4$ 时三地址剩余 3 个编码，二地址只能展开 $3\times16=48$ 条，远远不够 300 条，说明操作码起始位数取 **4** 位不够，必须加长到 **5** 位。这就是 2017 年统考第 16 题的考法：先假设操作码位数，逐级验证容量是否够用，不够就加大再验，最后把「操作码位数 + 地址字段总位数」按字节对齐向上取整得到指令字长。

*3.*完整算例二：层级指令条数的直接计算

题目（16 位定长指令字，操作用扩展编码，地址码 6 位，含零地址、一地址、二地址三种格式）：若二地址指令有 12 条、一地址指令有 254 条，则零地址指令最多有多少条？

解答：

- 二地址格式为 `OP(4 位) | A(6 位) | A(6 位)`，共 $4+6+6=16$ 位 ✓。4 位操作码共 16 个编码，用了 12 条，剩余 4 个。
- 一地址格式为 `OP(10 位) | A(6 位)`，可用编码数 $4\times2^6=256$ 条。用了 254 条，剩余 $256-254=2$ 个前缀。
- 零地址格式操作码为 16 位，可用编码数 $2\times2^6=128$ 条。

答案：**128** 条。

验证（用 **16** 位操作码的全部编码空间核对）：二地址指令占 12 条 $\times\,2^{12}$（两个 6 位地址字段）$=49152$ 个编码；一地址指令占 254 条 $\times\,2^6=16256$ 个编码；零地址指令占 128 条 $\times\,2^0=128$ 个编码；剩余未用的编码属于「保留扩展位」，不属于任何已定义指令。三者相加 $49152+16256+128=65536=2^{16}$，恰好把 16 位操作码的编码空间切分完毕，说明结果自洽。

另一种核对方式：零地址指令数 =（一地址剩余前缀数）$\times\,2^{\text{零地址可用的额外位数}}$，即 $(256-254)\times2^6=2\times64=128$ ✓

为什么零地址指令还能「再乘 6 位」

因为零地址指令虽然没有地址字段，但它的操作码可以扩展到占满整个 16 位指令字长。每保留一个 10 位的一地址前缀，其后的 6 位就有 $2^6=64$ 种取值，这 64 种取值都可以是相互不同的零地址指令操作码。漏掉这一步「再乘 2^p」是最常见的失分原因。

三、寻址方式

（一）指令寻址与数据寻址

指令寻址解决的是「下一条要执行的指令在哪里」，由 PC（程序计数器，x86 中称为 EIP）负责。它只有两种基本方式：

- 顺序寻址：$(PC)+$“1”，这里的「1」指一条指令的长度（按字节编址时按字节数增加）。
- 跳跃寻址：由转移指令给出目标地址，修改 PC。

数据寻址（操作数寻址）解决的是「本条指令的操作数在哪里」。指令中的地址字段给出的往往不是操作数的真实地址，而是形式地址（**A** 或 **D**）；CPU 依据寻址方式字段（寻址特征），对形式地址做运算，得到有效地址（**Effective Address**，**EA**）。

$$\text{操作数} = (EA)$$

其中括号表示「取该地址单元的内容」。当需要嵌套取值时（间接寻址），就写成 $((A))$ 这样的多重括号。

形式地址 vs 有效地址

形式地址是指令字里写着的那个数，可能是一个偏移量、一个寄存器编号、一个立即数；有效地址是经过寻址方式换算后，操作数在主存（或寄存器）中的真实位置。整个寻址方式这一节，本质上就是问「EA $= f$(形式地址, 寄存器) 中的 f 是什么」。

（二）各寻址方式详解

*1.*立即寻址

- **EA**：无（不访存取操作数，操作数就在指令里）。
- 形式地址：形式地址字段 A 就是操作数本身。操作数 $= A$。
- 访存次数：取操作数阶段 **0** 次（只需取指令 1 次）。
- 寻址范围：由地址字段的位数 n 决定。无符号数为 $0 \sim 2^n - 1$；带符号补码为 $-2^{n-1} \sim 2^{n-1} - 1$。
- 特点：速度最快；数值范围受限；通常用于给寄存器赋初值、与寄存器内容做常数运算。汇编写法如 `addi $t0,$t1,100`、`MOV EAX,2026H`。

*2.*寄存器寻址

- **EA**：$EA = R_i$（形式地址字段给出寄存器编号，操作数就在该寄存器中）。
- 访存次数：取操作数阶段 **0** 次。
- 寻址范围：由通用寄存器个数决定，n 位寄存器编号可指定 2^n 个寄存器。
- 特点：速度极快、指令字长短（寄存器编号只需 3~5 位）；寄存器资源稀缺，不能放太多数据。汇编写法如 `add $t0,$t1,$t2`。

*3.*直接寻址

- **EA**：$EA = A$。形式地址就是操作数在主存中的真实地址。
- 访存次数：取操作数 **1** 次（取指令 1 次另计）。
- 寻址范围：2^n，其中 n 是形式地址字段位数。例如形式地址 8 位、按字节编址时可寻址 $0 \sim 255$；16 位可寻址 64 KB。
- 特点：简单，不需要加法器；但地址被硬编码在指令中，程序浮动后失效，且寻址空间受形式地址位数限制。

*4.*间接寻址

- **EA**：$EA = (A)$。形式地址 A 是主存中某个单元的地址，该单元里存放的才是操作数的有效地址。操作数 $= ((A))$。
- 访存次数：取操作数需要 **2** 次（第一次取回真实地址，第二次取回操作数）。若是多重间址则次数更多，因此每得到一个地址就需要 1 个「间址标志位」标明本次取回的是地址还是数据。
- 寻址范围：由主存单元的字长决定，而与形式地址字段位数无关——这正是间接寻址最大的价值：用很短的形式地址访问很大的存储空间。
- 特点：便于实现指针、子程序返回参数等；但速度最慢，现代处理器一般不提供一到两条指令就完成的纯数据间接寻址。

*5.*寄存器间接寻址

- **EA**：$EA=(R_i)$。形式地址给出寄存器编号 R_i，该寄存器里存放的是操作数的有效地址（不是操作数本身）。操作数 $=((R_i))$。
- 访存次数：取操作数 **1** 次（地址在寄存器中，取地址不访存）。
- 寻址范围：由寄存器的位数决定（如 32 位寄存器可覆盖 4 GB 地址空间）。
- 特点：兼顾速度与寻址范围，是高级语言中指针的底层实现。汇编写法如 `MOV EAX,[EBX]`、`lw $t0,0($t1)`。

直接 / 间接 / 寄存器间接：看清「谁存的是地址」

- 直接寻址：指令里直接给出地址。
- 间接寻址：指令给出「存放地址的地址」（一次访存换回地址）。
- 寄存器间接寻址：寄存器里存放地址（不需要访存换地址，省一次访存）。

括号的层数就是访存的层数：A 是地址，(A) 取一次，$((A))$ 取两次。

*6.*相对寻址

- **EA**：$EA=(PC)+D$。D 是形式地址字段给出的位移量，用补码表示，计算时须符号扩展到与 PC 同宽。
- 访存次数：若用于取操作数，取操作数 1 次；若用于确定转移目标，不访存。
- 寻址范围：由位移量位数 n 决定，可前后跳转的范围为 $-2^{n-1}\sim 2^{n-1}-1$（相对于 PC 而言，按字节或按字，取决于题目单位）。
- 用途：① 公共子程序的浮动（保证指令与数据的相对位置不变，程序整体搬家后仍能取到同一数据）；② 转移指令的目标地址计算（x86 的 `jmp/jcc`、MIPS 的 `bne` 都是相对寻址）。

相对寻址最大的坑：(PC) 已经自增

当 CPU 完成取指阶段后，PC 已经自动加上了本指令的长度，此时 (PC) 指向的是「下一条指令的地址」，而不是当前指令的地址。所以

$$\text{转移目标地址}=\text{当前指令地址}+\text{当前指令长度}+D$$

写成公式就是 $EA=(PC)+D$，但要记住这里的 (PC) 是已自增的值。题目若写「每取一个字节 PC 自动加 1」「取指后 PC 加 2」一类的话，就是在提示你按哪种方式自增。

相对寻址转移目标计算（经典题型）

题目：某机器主存按字节编址，转移指令采用相对寻址，由 2 个字节组成（第 1 字节为操作码，第 2 字节为相对位移量）。取指时每取 1 个字节 PC 自动加 1。若转移指令所在地址为 2000H，位移量字段为 06H，求转移成功后的目标地址。

步骤：

1. 该指令占 2 字节，取指结束后 PC 已自增：$(PC)=2000\text{H}+2=2002\text{H}$。
2. 位移量 06H 为正数（最高位为 0，无需符号扩展为负），$D=+6$。
3. 目标地址 $=(PC)+D=2002\text{H}+06\text{H}=2008\text{H}$。

答案：**2008H**。

变式一（负位移量）：若指令地址为 2000H、占 2 字节、位移量字段为 `FAH`。`FAH = 1111 1010B`，是补码，符号扩展为 32 位是 `FFFFFFFAH`，其值为 -6。目标地址 $=2002\text{H}+(-6)=2002\text{H}-0006\text{H}=1\text{FFCH}$。

变式二（反求位移量）：若指令地址为 2000H、长 2 字节，要求转移到 2010H，求位移量字段。$D=2010\text{H}-2002\text{H}=000\text{EH}$，写成 8 位补码即 `0EH`。

变式三（**PC** 按指令条数自增）：MIPS 的 bne 采用相对寻址，定长 32 位，按字节编址，则 (PC) 已加 4，且偏移量字段以「指令条数」为单位，需左移 2 位：

$$\text{目标地址} = (PC + 4) + (\text{符号扩展后的 OFFSET} \times 4)$$

*7.*基址寻址

- **EA**：$EA = (B) + A$。B 为基址寄存器，A 为形式地址（在此扮演偏移量的角色）。
- 访存次数：取操作数 1 次。
- 寻址范围：由基址寄存器位数决定（可覆盖整个主存空间），形式地址只用来表示块内偏移。
- 用途：① 程序重定位——由操作系统在装入时为每个程序分配基地址写入基址寄存器（如重定位寄存器、页表基址寄存器），用户程序不可修改；② 扩展有限长度指令的寻址空间——用短地址码访问大空间，只要改变基址寄存器内容，操作数的地址空间就移到另一个区间。
- 寄存器由谁修改：这是容易混淆的地方——用于「多道程序重定位、虚拟存储器」时由操作系统在内核态修改，用户不能改；用于「扩大寻址范围」时由通用寄存器（如 x86 的 EBP、EBX、MIPS 的 $sp）承担，用户程序必须自己修改。

*8.*变址寻址

- **EA**：$EA = (I) + A$。I 为变址寄存器，此处形式地址 A 充当基准地址（如数组首地址，编译时固定），变址寄存器内容充当偏移量/下标。
- 访存次数：取操作数 1 次。
- 寻址范围：由变址寄存器位数 + 形式地址位数共同决定。
- 用途：面向用户，专用于遍历数组、字符串、线性表等成批数据。循环中形式地址（数组首地址）不变，只需让变址寄存器按元素长度递增（如 int 数组每次加 4），即可依次访问每个元素。这是「最适合按下标顺序访问一维数组元素」的寻址方式。

对比项	基址寻址	变址寻址
公式	$EA = (B) + A$	$EA = (I) + A$
寄存器里存什么	基准地址（程序/数据块起点）	偏移量（数组下标）
形式地址 A 是什么	偏移量	基准地址（数组首地址）
谁修改寄存器	操作系统（重定位场景）/ 用户（扩址场景）	用户 / 程序员
执行过程中寄存器内容	通常保持不变	不断变化（随循环递增）
主要应用	程序重定位、短地址访问大空间	数组、线性表、字符串的遍历

基址与变址的「谁在前谁在后」

两者公式形式相同，区别在于谁是基准、谁是变化量。统考按「基址寻址面向系统、变址寻址面向用户」来理解：基址寄存器存基准地址且执行中不变，变址寄存器存偏移量且执行中可变。若题目只说「某寄存器内容 + 形式地址」，就要看题干给的场景来判断。

*9.*堆栈寻址

堆栈按后进先出（**LIFO**）的方式存取数据。寻找存放在堆栈中的操作数地址的方法称为堆栈寻址，操作数总是位于栈顶，因此不需要在指令中给出地址（堆栈寻址本质上属于隐含寻址）。

类型	实现	是否需要堆栈指针	栈顶是否移动	数据是否移动
存储器堆栈（硬堆栈/软堆栈）	在主存中开辟栈区	需要，设置堆栈指针 SP 指向栈顶	移动（SP 随进出栈变化）	不移动
寄存器堆栈（硬堆栈）	用一组寄存器作栈区	不需要	固定不动	移动（数据在寄存器间挪动）

存储器堆栈的操作（以按字节编址、以字节为单位进出栈、栈向低地址增长为例）：

$$\text{入栈：}\ SP \leftarrow SP-1,\quad M[SP] \leftarrow R$$

$$\text{出栈：}\ R \leftarrow M[SP],\quad SP \leftarrow SP+1$$

若以 32 位字为单位进出栈，则上式中的 1 应换成 4。寄存器堆栈速度快但容量有限，存储器堆栈容量大但速度慢；寄存器堆栈必须用专用堆栈指令控制，存储器堆栈则不一定。

（三）寻址方式完整对比表

寻址方式	有效地址 **EA**	取操作数访存次数	操作数所在位置	寻址范围由谁决定	典型用途
立即寻址	无 EA，操作数 $=A$	0	指令中	形式地址字段位数	常数运算、赋初值
寄存器寻址	$EA=R_i$，操作数 $=(R_i)$	0	通用寄存器	寄存器编号位数	寄存器间运算
直接寻址	$EA=A$	1	主存	形式地址字段位数	访问固定地址的变量
间接寻址	$EA=(A)$，操作数 $=((A))$	2（每重间址 +1）	主存	主存单元字长	指针、参数传递
寄存器间接寻址	$EA=(R_i)$，操作数 $=((R_i))$	1	主存	寄存器位数	指针、数组遍历
相对寻址	$EA=(PC)+D$（PC 已自增）	1（取操作数时）	主存 / 指令	位移量位数（补码）	转移目标、程序浮动
基址寻址	$EA=(B)+A$	1	主存	基址寄存器位数	程序重定位、扩址
变址寻址	$EA=(I)+A$	1	主存	变址寄存器位数 + 形式地址位数	数组、循环
堆栈寻址	隐含：$EA=(SP)$	1	主存栈区 / 寄存器组	栈区容量	过程调用、表达式求值

偏移寻址的三种形式

相对寻址、基址寻址、变址寻址在计组中被统称为偏移寻址，它们的数学逻辑完全一致，都是「寄存器

的内容 + 形式地址」。区别在于参与相加的寄存器是谁、由谁控制、用在什么场景。因此「下列不属于偏移寻址的是」这类题的答案通常是间接寻址（间接寻址是「取内容」而不是「相加」）。

变址寻址与地址计算（2018 年统考第 18 题思路）

按字节编址的计算中，`double` 型数组 A 的首地址为 2000H，用变址寻址和循环访问数组，变址寄存器存放的是下标，每次循环取一个元素后自增 1，而实际参与地址计算的是「变址值 × `sizeof(double)`」。若某次取到的元素地址为 2100H，则

$$\text{偏移字节数} = 2100\text{H} - 2000\text{H} = 100\text{H} = 256$$

$$\text{变址寄存器内容} = 256 \div 8 = 32$$

要点：变址寄存器里放的是下标还是字节偏移，必须看题目明确的约定，两者相差一个 sizeof(元素) 因子。

四、数据的对齐和大/小端存放方式

（一）大端方式与小端方式

大端方式（**Big Endian**）：最高有效字节（**MSB**）存放在最低地址，低地址 → 高地址依次存放从高字节到低字节。小端方式（**Little Endian**）：最低有效字节（**LSB**）存放在最低地址。

设 32 位机器数 M = 1234 5678H，存放在首地址 1000H 开始的 4 个字节中：

地址	大端方式	小端方式
1000H	**12H**（MSB）	**78H**（LSB）
1001H	34H	56H
1002H	56H	34H
1003H	**78H**（LSB）	**12H**（MSB）

由此可以得到两条好用的结论：

1. 大端：一个数据的首地址就是它的最高有效字节地址；末地址是最低有效字节（LSB）的地址。
2. 小端：一个数据的首地址就是它的最低有效字节地址；末地址是最高有效字节（MSB）的地址。

求 LSB 地址的题，先算 EA 再判端序

题目若问「该操作数的 LSB（最低有效字节）所在地址」，要先按寻址方式算出首地址（即最低地址），再根据端序判断 LSB 落在首地址还是末地址。

- 大端：MSB 在首地址，LSB 在首地址 +（数据字节数 − **1**）。
- 小端：LSB 在首地址。

以 32 位操作数为例，「首地址 + 3」就是大端时 LSB 的地址。

综合算例（基址寻址 + 大端 + 符号扩展）

题目：某计算机采用大端方式，按字节编址。某指令中操作数的机器数为 `1234FF00H`，采用基址寻址，形式地址（补码表示）为 `FF12H`，基址寄存器内容为 `F0000000H`，求该操作数 **LSB** 所在的地址。

步骤：

1. 求有效地址。形式地址 `FF12H` 是 16 位补码，最高位为 1，为负数，需符号扩展到 32 位：`FFFFFF12H`。

2. 相加：EA = F0000000H + FFFFFF12H = 1 EFFF FF12H。32 位运算丢弃最高位进位，得 EA = EFFF FF12H。这就是该 4 字节操作数的首地址。
3. 判端序。大端方式下 MSB 在最低地址，LSB 在最高地址。
4. 求 **LSB** 地址：首地址 $+3$ = EFFF FF12H + 3 = EFFF FF15H。

答案：**EFFF FF15H**。

要点提醒：① 补码形式地址参与 32 位加法时一定要先符号扩展；② 加法产生的最高位进位要丢弃；③ 大端时 LSB 不在首地址。

大小端与程序正确性：大小端只影响多字节数据的字节排放顺序，不影响单字节数据，也不改变数值本身。同一份程序在大端机和小端机上运行结果可能不同，典型场景是：① 通过指针按字节读取一个多字节整数；② 网络协议的报文字段；③ 直接按字节比较两个数值的存储内容。x86 采用小端，MIPS 传统上采用大端。

看机器指令的立即数字节序判端序

给定机器代码 `C7 45 FC 00 00 00 00` 表示 `int i = 0`，若另一个立即数 -64（32 位补码为 `FFFFFFC0H`）要写成机器代码，则在小端下立即数的 4 个字节为 `C0 FF FF FF`，在大端下为 `FF FF FF C0`。反过来，若题目给出机器代码让你判断端序，就把其中的多字节字段（立即数、偏移量、地址）按两种端序各还原一次，看哪一种结果合理。

（二）边界对齐与不对齐存储

边界对齐要求：一个占 k 字节的数据，其起始地址必须是 k 的整数倍。

- `char`（1 字节）：任意地址都对齐；
- `short`（2 字节）：起始地址必须是 **2** 的倍数；
- `int`、`float`（4 字节）：起始地址必须是 **4** 的倍数；
- `double`、`long long`（8 字节）：起始地址必须是 **8** 的倍数（32 位机器上通常按 4 字节对齐）。

为什么要求对齐？因为主存通常是按字（或按 Cache 行）访问的。若一个 4 字节的 `int` 起始地址为 2（不对齐），它跨越了两个存储字，一次访存取不全，CPU 需要两次访存再拼接，还可能跨越 Cache 行边界造成额外缺失；而在某些 RISC 架构上，不对齐访存会直接触发对齐异常。对齐的代价是浪费少量填充字节，换来的是访存次数的减少与硬件实现简化。

填充字节（**padding**）：为了满足对齐要求，编译器会在成员之间、以及结构体末尾插入无意义的填充字节。

（三）结构体大小的计算规则

设结构体中最大成员的对齐要求为 M（即最大成员的字节数，且不超过机器的最大对齐值）。计算步骤：

1. 从偏移 0 开始，依次为每个成员分配空间；
2. 若当前偏移不是该成员对齐要求的整数倍，就插入填充字节，把偏移抬到最近的整数倍位置；
3. 该成员占用 k 字节后，偏移增加 k；
4. 所有成员分配完毕后，若结构体总大小不是 M 的整数倍，则在末尾补齐到 M 的整数倍（这是为了保证结构体数组中每个元素的地址仍然满足对齐要求）。

$$\text{sizeof(struct)} = \left\lceil \frac{\text{最后一个成员的结束偏移}}{M} \right\rceil \times M$$

*1.*完整算例一：*int / char / short* 结构体

题目：某计算机按字节编址，采用小端方式，int 占 32 位、short 占 16 位、char 占 8 位，数据按边界对齐存储。

```
struct {
    int a;
    char b;
    short c;
} record;
record.a = 273;
```

若 record 的首地址为 0xC008，求地址 0xC008 中的内容，以及 record.c 的地址。

第一步：算各成员偏移与地址。

成员	类型	字节数	对齐要求	起始偏移	实际地址	说明
a	int	4	4	0	0xC008	0 是 4 的倍数，直接放
b	char	1	1	4	0xC00C	紧随 a
填充	—	1	—	5	0xC00D	c 需 2 字节对齐，偏移 5 不是 2 的倍数
c	short	2	2	6	0xC00E	偏移 6 是 2 的倍数 ✓

结构体总大小 = 8 字节，是最大成员对齐要求（4）的整数倍，无需末尾填充。故 record.c 的地址为 **0xC00E**。

第二步：算 record.a 的字节内容。

273 = 0x00000111，即机器数为 00 00 01 11H，4 个字节从高到低是 00H，00H，01H，11H。小端方式下最低有效字节放在最低地址，于是：

地址	内容	对应字节
0xC008	**11H**	LSB
0xC009	01H	
0xC00A	00H	
0xC00B	00H	MSB

答案：地址 **0xC008** 中的内容为 0x11，record.c 的地址为 **0xC00E**。

填充字节的位置与内容

填充字节的内容是不确定的（编译器不保证填 0），考试只考它在哪里、占几字节。另外，填充既可能出现在成员之间（如本例 b 与 c 之间），也可能出现在结构体末尾（当总大小不是最大对齐值的整数倍时），末尾填充最容易被漏掉。

*2.*完整算例二：*short + int* 结构体

题目：按字节编址、小端方式，按边界对齐为下列结构体变量 a 分配空间，a 的首地址为 2020FE00H，成员 x2 的机器数为 12340000H，求其中 34H 所在的存储单元地址。

```
struct record {
    short x1;
    int x2;
} a;
```

第一步：算偏移。

- x1（short，2 字节，对齐要求 2）：偏移 0，地址 2020FE00H ~ 2020FE01H。
- 下一个可用偏移是 2，但 x2（int，4 字节，对齐要求 4）要求偏移是 4 的倍数，因此偏移 2、3 处填充 **2** 个字节。
- x2：偏移 4，地址 2020FE04H ~ 2020FE07H。
- 总大小 8 字节，是 4 的整数倍 ✓。

第二步：算 **x2** 的字节内容。机器数 12340000H 的 4 个字节为 12H，34H，00H，00H（从高到低）。小端方式下低地址放低字节：

地址	内容
2020FE04H	00H（LSB）
2020FE05H	00H
2020FE06H	**34H**
2020FE07H	12H（MSB）

答案：34H 所在的存储单元地址是 2020FE06H。

*3.*完整算例三：含 *double* 的复杂结构体

题目：32 位机、小端、按边界对齐，char 1 字节、short 2 字节、int 4 字节、double 8 字节。

```
struct Node {
    char   tag;      /* 1 字节 */
    int    id;       /* 4 字节 */
    short  flag;     /* 2 字节 */
    double value;    /* 8 字节 */
    char   tail;     /* 1 字节 */
};
```

求各成员的偏移与结构体总大小。

成员	字节数	对齐要求	起始偏移	填充字节数	说明
tag	1	1	0	0	
填充	—	—	1~3	3	使 id 从 4 的倍数开始
id	4	4	4	0	
flag	2	2	8	0	8 是 2 的倍数

成员	字节数	对齐要求	起始偏移	填充字节数	说明
填充	—	—	10~15	6	使 value 从 8 的倍数开始
value	8	8	16	0	16 是 8 的倍数
tail	1	1	24	0	
末尾填充	—	—	25~31	7	总大小须为 8 的倍数

结果：tag 偏移 0、id 偏移 4、flag 偏移 8、value 偏移 16、tail 偏移 24，结构体总大小为 **32** 字节。若改为不按边界对齐（紧凑/打包）存储，则所有成员连续排列：tag 偏移 0、id 偏移 1、flag 偏移 5、value 偏移 7、tail 偏移 15，总大小为 16 字节。

对比结论：对齐存储把结构体从 16 字节撑到 32 字节，浪费了一倍空间，但每个成员都能一次访存取到；打包存储省空间，却可能让 id、value 跨越存储字边界，需要多次访存。这就是「空间换时间」在指令系统与存储系统交界处的体现。

一句话记住结构体大小算法

每个成员从「自己的对齐要求的最小整数倍偏移」开始放，中间不够就补；全部放完后，总大小向上取整到「最大成员的对齐要求」的整数倍。

对齐类真题的三种问法

1. 问某成员地址（如 record.c 的地址）：按上面三步走即可。
2. 问某字节所在地址（如 34H 在哪个单元）：先定位成员，再按端序映射到具体字节。
3. 问某个存储单元里的内容（如 0xC008 中是什么）：先定位该地址属于哪个成员的第几个字节，再按端序还原。

五、CISC 和 RISC 的基本概念

（一）CISC 的基本概念与特点

CISC（**Complex Instruction Set Computer**，复杂指令集计算机）的设计思路是用一条硬件指令完成复杂任务，通过增强指令功能、增加指令条数来缩短「高级语言语义」与「机器指令语义」之间的差距。它产生的原因有两条：一是硬件成本下降而软件成本上升，增加复杂指令可以简化编译器设计；二是向后兼容——同一系列的新机器只能扩充指令、不能删除老指令，导致指令集不断膨胀。

CISC 的核心特征：

- 指令数量多，一般 200 条以上，甚至数百条；
- 指令字长不固定，指令格式和寻址方式种类繁多；
- 访存不受限制：除了专门的取数/存数指令外，运算类指令也可以直接访问主存（如 add eax, [ebp-8]）；
- 控制器以微程序控制为主：指令太复杂，难以用纯硬件逻辑直接实现，控制存储器往往占 CPU 芯片面积 50% 以上；
- 各指令执行时间相差很大，多数复杂指令需要多个时钟周期，流水线实现困难；
- 各指令使用频度相差很大，约 80% 的时间只用到 20% 最简单的指令（**80-20** 规律）；
- 难以用优化编译生成高效目标代码；

- 典型代表：**x86** 系列（Intel Core i5/i7/i9、AMD Ryzen）。

（二）RISC 的基本概念与特点

RISC（**Reduced Instruction Set Computer**，精简指令集计算机）的设计思路基于 80-20 规律：只保留使用频率最高、最简单的指令，复杂功能由简单指令组合实现。

RISC 的核心特征：

- 严格的 **Load/Store** 架构：只有取数（**Load**）和存数（**Store**）指令可以访问主存，所有运算都必须在寄存器之间进行；
- 面向通用寄存器的结构：配置数量庞大的通用寄存器（几十到上百个），以减少访存；
- 指令「长相」高度统一：指令长度固定，指令格式和寻址方式种类很少，译码逻辑大为简化；
- 控制器以硬布线（组合逻辑）控制为主，控制部件通常只占 CPU 芯片面积 10% 左右；
- 深度依赖指令流水线：绝大多数指令在一个时钟周期内完成；
- 极其重视编译优化：把复杂操作推给编译器，靠编译器调度指令顺序以填满流水线；
- 兼容性包袱小但兼容性差：因为精简了指令系统，大多数 RISC 机器无法与老机器保持软件兼容；
- 典型代表：**ARM**、**MIPS**、**RISC-V**。

（三）CISC 与 RISC 的完整对比

对比项目	**CISC**	**RISC**
指令系统	复杂、庞大	简单、精简
指令数量	一般大于 **200** 条	一般小于 **100** 条
指令长度	不固定（变长）	固定（定长，典型 32 位）
寻址方式	种类多（常达十几种）	种类少（通常仅寄存器、立即数、基址/偏移）
访存指令	不加限制，运算指令也能访存	只有 **Load/Store** 可访存
通用寄存器数量	较少	多（几十至上百个）
控制器实现	绝大多数为微程序控制	绝大多数为硬布线（组合逻辑）控制
指令执行时间	相差较大，多数需多个时钟周期	绝大多数在一个时钟周期内完成
流水线难易	难（指令变长、执行时间不一）	易（定长、规整，天然适合流水线）
编译优化	难以用优化编译生成高效目标代码	采用优化编译可生成较为高效的代码
各指令使用频度	相差很大	都比较常用
软件兼容性	大多能向后兼容（高档机含低档机全部指令）	大多数不能与老机器兼容
典型代表	x86（Intel Core、AMD Ryzen）	ARM、MIPS、RISC-V

（四）走向融合

从结构合理性与执行效率看，RISC 的思路更科学，但现代 CISC 处理器早已不是「纯粹」的 CISC：它们在微架构层面把复杂的 CISC 指令打碎成简单的微操作来执行，大量融入了 RISC 的技术成分。因此两者的性能差距正在缩小。判断一台机器是 **CISC** 还是 **RISC**，不能只看品牌，而要看指令字长是否固定、是否严格限制只有 **Load/Store** 访存——这正是 2017 年统考大题判断「计算机 M 是 RISC 还是 CISC」的依据。

CISC/RISC 概念题的高频陷阱

1. 「普遍采用微程序控制器」是 **CISC** 的特征，不是 **RISC** 的。RISC 普遍采用硬布线控制。这是最高频的错误选项。
2. 「通用寄存器数量不多」是 **CISC** 的特征，不是 RISC 的。
3. 「严格限制只有 **LOAD/STORE** 指令能访存」是 **RISC** 的特征，说成 CISC 就是错的。
4. 「大多数 **RISC** 架构能与老机器实现软件兼容」是错的，CISC 才具备这种兼容性。
5. 「指令长度固定」是 **RISC** 的；CISC 指令长度不固定。
6. 注意「**RISC** 大多数指令在一个时钟周期内完成」这句话是正确的。RISC 追求的是单周期完成 + 流水线重叠，而不是「多周期完成」。

真题考法

概念辨析选择题几乎每年一条，形式是「下列关于 RISC/CISC 的叙述中，错误的是」。解题时只需对着上表中的加粗项逐条核对。此外，RISC 的「指令字长固定、寻址方式少、只有 Load/Store 访存」这三条也会在大题里作为判据出现：给一段机器级代码，只要观察到不同指令占用的字节数不同（如 `push` 1 字节、`cmp` 3 字节），或出现了直接对内存操作数做运算的指令，就可以判定为 CISC。

六、高级语言程序与机器级代码之间的对应

（一）编译器、汇编器和链接器的基本概念

从高级语言源程序到可执行目标文件，依次经过预处理、编译、汇编、链接四个阶段：

阶段	输入	输出	主要工作
预处理	`.c` 源程序	`.i` 预处理后的源程序	处理 `#define` 宏展开、`#include` 文件包含、`#ifdef` 条件编译、删除注释
编译	预处理后的源程序	`.s` 汇编语言源程序	词法分析、语法分析、语义分析、中间代码生成、代码优化、生成汇编代码
汇编	`.s` 汇编语言源程序	`.o` 可重定位目标文件（机器语言）	把汇编指令翻译成机器指令，生成符号表与重定位信息
链接	多个`.o` 目标文件 + 库文件	可执行目标文件	符号解析（把每个符号引用与一个符号定义关联）、重定位（把符号地址合并到最终地址空间）、合并段

四个阶段的产物必须严格对应。注意「编译」和「汇编」的分界：编译的产物是汇编语言程序（文本），汇编的产物是机器语言目标文件（二进制）。

「编译」和「汇编」不要混

「预处理 → 编译 → 汇编 → 链接」这个顺序中，把「汇编」和「编译」调换位置是最常见的错误选项（2022 年统考第 20 题就考查了这个顺序）。简记：编译产出汇编代码，汇编产出目标文件。

另外注意：可执行文件与源程序之间不是一一对应的——一个可执行文件可能由多个源文件、多个目标文件和若干库文件链接而成。

链接器与重定位

链接器要解决两个问题：符号解析（`printf` 在哪里定义?）和重定位（每个符号最终被安排在哪个地址?）。这也解释了为什么「编译时地址」和「运行时地址」可能不同，以及为什么直接寻址不利于程序浮动、而相对寻址和基址寻址有利于程序浮动。

（二）选择结构语句的机器级表示

机器级表示中没有「**if**」这条指令。`if-else` 是靠条件测试 + 条件转移来实现的，思路有两种等价写法：

- 正向跳转：条件成立就跳到 Then 块；
- 反向跳转（编译器实际采用的主流写法）：测试条件的反面，条件不成立就跳到 Else 块，否则顺序执行 Then 块。

编译器之所以采用反向跳转，是为了让「条件成立」这条最可能的分支顺序执行，从而提高指令预取与分支预测的命中率。

*1.*条件码与测试指令

|x86 的算术/逻辑运算都会设置程序状态字（EFLAGS）中的条件码：

标志	名称	含义
ZF	零标志	结果为 0 时置 1
SF	符号标志	结果为负时置 1（即结果最高位）
OF	溢出标志	针对有符号数，补码运算溢出时置 1
CF	进位/借位标志	针对无符号数，最高位产生进位或借位时置 1

两条只设置标志位、不改变操作数的指令：

- `cmp`：底层执行减法 $A-B$，只更新标志位，不保存差。用于比较大小、判断相等。
- `test`：底层执行按位与 $A\&B$，只更新标志位。常用于 `test eax,eax` 判断寄存器是否为 0（`test` 两个相同操作数时，ZF 反映该数是否为 0，SF 反映其符号）。

条件转移指令按操作数类型分为三组：

类别	指令	转移条件（逻辑表达式）	适用场景
相等/零	`je` / `jz`	$ZF=1$	相等、结果为零
相等/零	`jne` / `jnz`	$ZF=0$	不相等、结果非零
符号	`js` / `jns`	$SF=1$ / $SF=0$	判断正负
溢出	`jo` / `jno`	$OF=1$ / $OF=0$	溢出检查
无符号数	`ja`（大于）	$CF=0$ 且 $ZF=0$	unsigned 大于
无符号数	`jae`（大于等于）	$CF=0$	unsigned 大于等于
无符号数	`jb`（小于）	$CF=1$	unsigned 小于
无符号数	`jbe`（小于等于）	$CF=1$ 或 $ZF=1$	unsigned 小于等于
有符号数	`jg`（大于）	$SF=OF$ 且 $ZF=0$	int 大于
有符号数	`jge`（大于等于）	$SF=OF$	int 大于等于

类别	指令	转移条件（逻辑表达式）	适用场景
有符号数	`jl`（小于）	$SF \neq OF$	int 小于
有符号数	`jle`（小于等于）	$SF \neq OF$ 或 $ZF = 1$	int 小于等于

有符号用 `jg/jl`，无符号用 `ja/jb`

同一个比较，操作数是有符号还是无符号，编译出的跳转指令完全不同。原因是减法结果相同但「谁大谁小」的解释不同：-1 作为有符号数是负数（小于 0），作为无符号数却大于所有正数。当 `cmp` 的两个操作数中有一个是 `unsigned` 时，必须用 `ja/jb` 系列（只看 CF 和 ZF），不能用 `jg/jl` 系列（要看 SF 和 OF）。

*2.*示例：*`if-else`* 的机器级表示

```
int abs_delta(int x, int y) {
    int result;
    if (x > y)
        result = x - y;
    else
        result = y - x;
    return result;
}
```

假设参数 `x` 在 `[ebp+8]`、`y` 在 `[ebp+12]`：

```
abs_delta:
    push ebp
    mov  ebp, esp
    mov  eax, [ebp+8]       ; eax = x
    mov  edx, [ebp+12]      ; edx = y
    cmp  eax, edx           ; 计算 x - y，设置 ZF/SF/OF/CF
    jle  .L_else            ; x > y 的" 反面" 是 x <= y，成立就跳去 Else
    ; ---- Then 分支 (x > y) ----
    sub  eax, edx           ; result = x - y
    jmp  .L_done            ; Then 执行完必须无条件跳过 Else
.L_else:
    ; ---- Else 分支 (x <= y) ----
    sub  edx, eax           ; y - x
    mov  eax, edx
.L_done:
    pop  ebp
    ret
```

这段代码的三个要点：① `cmp` 只设标志位、不改操作数；② 跳转条件用的是 `x > y` 的反面 `x <= y`（反向跳转）；③ Then 块末尾必须有 `jmp` 跳过 Else 块，否则会「贯穿」执行 Else。

（三）循环结构语句的机器级表示

循环在机器级就是「条件转移（向后跳出）+ 无条件转移（向前跳回）」的组合。编译器先把循环改写成 `goto` 形式，再翻译成测试与跳转。三种循环的差别只在条件判断的位置。

1.do-while 循环（后测试）

```
do
    body_statement
while (test_expr);
```

翻译为 goto：

```
loop:
    body_statement
    t = test_expr;
    if (t) goto loop;
```

特点：循环体在条件判断之前执行，因此循环体至少执行一次。汇编结构最简洁——循环头就是循环体入口。

```
do_while:
    push ebp
    mov  ebp, esp
    mov  eax, 0            ; sum = 0
    mov  ecx, 1            ; i = 1
loop:
    add  eax, ecx          ; sum += i
    add  ecx, 1            ; i++
    cmp  ecx, [ebp+8]      ; 比较 i 与 n
    jle  loop              ; 条件成立，跳回循环体
    pop  ebp
    ret
```

2.while 循环（前测试）

```
while (test_expr)
    body_statement;
```

编译器有两种标准翻译方法：

方法一：**jump to middle**（跳转到中间）——先无条件跳到条件测试处，条件成立才进入循环体：

```
    goto test;
loop:
    body_statement
test:
    t = test_expr;
    if (t) goto loop;
```

方法二：**guarded-do**（带保护的 **do-while**）——先测试一次条件，不成立直接跳过；成立则进入 do-while 结构，循环体末尾再测试：

```
    t = test_expr;
    if (!t) goto done;
```

```
loop:
    body_statement
    t = test_expr;
    if (t) goto loop;
done:
```

特点：条件在循环体之前判断，初始条件为假时循环体一次都不执行。若把 while 翻译成不带前置判断的 do-while，会多执行一次循环体，这是编译错误。

3.for 循环

```
for (init_expr; test_expr; update_expr)
    body_statement;
```

`for` 在语义上完全等价于：

```
init_expr;
while (test_expr) {
    body_statement;
    update_expr;
}
```

翻译为 `goto` 形式：

```
    init_expr;          /* 只执行一次 */
    t = test_expr;
    if (!t) goto done;  /* 初始条件为假则直接跳过 */
loop:
    body_statement
    update_expr;        /* 更新循环变量 */
    t = test_expr;
    if (t) goto loop;
done:
```

特点：`init_expr` 只执行一次；`test_expr` 在每次循环体之前求值；`update_expr` 在每次循环体之后、条件判断之前求值。

*4.*三种循环的对比

循环	条件判断位置	循环体最少执行次数	跳回方式	关键特征
`do-while`	循环体之后	**1** 次	条件成立则向后跳回循环头	结构最简，无前置判断
`while`	循环体之前	**0** 次	需先跳到测试处或先做一次保护性判断	必须有一次前置测试
`for`	循环体之前	**0** 次	与 `while` 相同，多一段初始化	初始化只做一次；更新在体后

循环类题的两个陷阱

1.「至少执行一次」只属于 `do-while`。`while` 和 `for` 初始条件为假时循环体一次都不执行。
2. 条件跳转的方向：循环条件成立时要向后跳回循环头（形成闭环），而跳出循环的条件跳转是向前跳。若题目给的汇编把成立与不成立的条件写反了，就一定是错的。

循环的真题考法

一是给一段 C 循环代码和对应的汇编，问「哪条指令是条件转移」「每轮循环执行几次某条指令」。二是给一个二维数组赋值循环（如 `for(i=0;i<24;i++) for(j=0;j<64;j++) a[i][j]=10;`），问嵌套循环的机器级实现。三是把 `sum += a[i];` 这样的语句翻译成指令序列，考查「变址计算要用移位代替乘法」（`int` 数组左移 2 位、`short` 数组左移 1 位）。

（四）过程（函数）调用对应的机器级表示

过程调用是本章最综合的考点，它把指令系统、寻址方式、栈、寄存器约定全部串起来。

*1.*运行时栈与栈帧

运行时栈（**run-time stack**）是一块从高地址向低地址增长的内存区域，用来存放过程调用时需要保存和恢复的信息。栈指针 ESP（MIPS 中为 `$sp`）始终指向栈顶（当前最低地址）。

每个正在执行的过程在栈上占有一块区域，称为栈帧（**Stack Frame**）。以 x86-32 为例，一个典型栈帧从高地址到低地址依次为：

顺序	内容	由谁压入	说明
1	调用者的实参	调用者（Caller）	C 约定从右至左压栈，第一个参数在最低地址
2	返回地址	`call` 指令自动压入	`call` 的下一条指令的地址
3	旧的 **EBP**	被调用者（Callee）	`push ebp`，保存上一层栈帧的基址
4	被调用者保存的寄存器	Callee	如 `EBX`、`ESI`、`EDI`
5	局部变量	Callee	`sub esp, N` 开辟的空间

EBP 与 **ESP** 的分工：

- **EBP**（基址指针，**Base Pointer**）：指向当前栈帧的底部（高地址端，紧邻返回地址与旧 EBP）。在函数执行期间保持不变，因此可以用固定的偏移量稳定访问数据：
  - `[ebp + 8]`、`[ebp + 12]`、`[ebp + 16]` ……依次是第 1、2、3 个参数（`[ebp+4]` 是返回地址，`[ebp]` 是旧 EBP）；
  - `[ebp - 4]`、`[ebp - 8]` ……是第 1、2 个局部变量。
- **ESP**（栈指针，**Stack Pointer**）：始终指向栈顶（低地址端），随 push/pop 动态变化。

为什么需要 EBP，只用 ESP 行不行

ESP 在函数执行过程中会因为压栈、弹栈而不断变化，若用 ESP 加偏移访问参数，偏移量必须随每次压栈而调整，编译器难以静态确定。而 EBP 在整个函数执行期间不变，参数与局部变量的偏移量都是编译期常量，寻址更高效、更稳定。若编译器能完全静态算出相对 ESP 的偏移（如叶子函数），也可以省略 EBP 以节省一次压栈和一次弹栈。

*2.*调用与返回指令

- `call 目标地址`：隐含两步——① 把返回地址（`call` 的下一条指令的地址）压栈；② 把目标地址送入 EIP，跳转到被调用过程。
- `ret`：从栈顶弹出返回地址送到 EIP，控制权交回调用者。`ret` 本身不清理参数空间，参数由调用者用 `add esp，M` 清理（cdecl 约定）。

*3.*过程调用的完整流程

调用前（**Caller**）：

1. 若 EAX/ECX/EDX 在调用后还需要用，先把它们压栈保存（调用者保存寄存器）；
2. 把参数从右至左依次压栈；
3. 执行 `call Q`。

进入被调用者（**Callee**）：

1. `push ebp` ——保存调用者的 EBP；
2. `mov ebp，esp` ——建立新栈帧，EBP 指向新栈帧底部；
3. `sub esp，N` ——为局部变量开辟 N 字节空间；
4. `push ebx/esi/edi` ——保存需要用到的被调用者保存寄存器。

执行主体：通过 `[ebp+8]`、`[ebp+12]` 访问参数，通过 `[ebp-4]`、`[ebp-8]` 访问局部变量，结果通常放在 EAX。

返回前（**Callee**）：

1. 把返回值放入 EAX；
2. `pop ebx/esi/edi` 恢复被调用者保存寄存器；
3. `leave`（等价于 `mov esp，ebp` + `pop ebp`）撤销栈帧；
4. `ret` 弹出返回地址并跳回。

返回后（**Caller**）：

1. 从 EAX 取返回值；
2. `add esp，M` 清理参数（M 为参数总字节数）；
3. 恢复之前保存的调用者保存寄存器。

*4.*寄存器使用约定

类别	**x86-32** 寄存器	**MIPS** 寄存器	谁负责保存
调用者保存（Caller-saved，易失）	EAX、ECX、EDX	$t0~$t9、$a0~$a3、$v0~$v1	调用者自己压栈，否则被调用者可随意覆盖
被调用者保存（Callee-saved，非易失）	EBX、ESI、EDI、EBP、ESP	$s0~$s7、$sp、$fp	被调用者若使用则必须先保存、返回前恢复

类别	x86-32 寄存器	MIPS 寄存器	谁负责保存
特殊	EIP（不可直接访问）、EFLAGS	$ra（返回地址，由 jal 自动写入）、$zero（恒 0）	—

返回值的存放：x86 放 EAX，MIPS 放 $v0。参数传递：x86 默认全部压栈（fastcall 约定才用寄存器），MIPS 前 4 个参数放 $a0~$a3、多余的压栈。

「调用者保存」到底是谁保存

这个术语描述的不是义务方，而是被保护的对象。「调用者保存寄存器」的意思是：如果调用者希望调用后这些寄存器的值不变，那么调用者必须自己保存它们；被调用者使用它们不需要事先保存。反之，「被调用者保存寄存器」的意思是：被调用者若要用，就必须先保存、返回前恢复。答题时务必写清「谁在什么时机保存/恢复」。

*5.*完整算例：栈帧布局与递归的栈变化

题目：x86-32，按字节编址，int 占 4 字节。

```
int calculate(int x, int y, int z) {
    int diff = x - y;
    return diff * z;
}
int caller(void) {
    return calculate(10, 5, 3);
}
```

（1）栈帧布局与各成员地址

```
caller:
    push ebp
    mov  ebp, esp
    push 3              ; 参数 z
    push 5              ; 参数 y
    push 10             ; 参数 x
    call calculate      ; 自动压入返回地址
    add  esp, 12        ; 清理 3 个参数 (3*4)
    pop  ebp
    ret

calculate:
    push ebp            ; 保存旧 EBP
    mov  ebp, esp       ; 建立新栈帧
    sub  esp, 4         ; 为 diff 开辟 4 字节
    mov  eax, [ebp+8]   ; x
    sub  eax, [ebp+12]  ; x - y
    mov  [ebp-4], eax   ; diff = x - y
    mov  eax, [ebp-4]   ; 取 diff
    imul eax, [ebp+16]  ; diff * z
    mov  esp, ebp       ; 释放局部变量空间 (leave 的前半)
    pop  ebp            ; 恢复旧 EBP
    ret
```

以 `calculate` 执行时的 EBP 为基准，栈中内容为：

地址（相对 **EBP**）	内容	属于
`[ebp+16]`	`z = 3`	参数 3（最后压栈，最高地址）
`[ebp+12]`	`y = 5`	参数 2
`[ebp+8]`	`x = 10`	参数 1（最先入栈的参数，最低地址）
`[ebp+4]`	返回地址	`call` 自动压入
`[ebp]`	旧的 EBP	`push ebp`
`[ebp-4]`	`diff`	局部变量

要点：参数压栈顺序是「从右至左」，所以第一个参数 `x` 的地址最小（`[ebp+8]`），最后一个参数 `z` 的地址最大（`[ebp+16]`）。返回地址在 **`[ebp+4]`**，紧接着就是第一个参数，这正是 `[ebp+8]` 成为第一个参数的原因：`[ebp]` 是旧 EBP、`[ebp+4]` 是返回地址，各占 4 字节。

（**2**）递归的栈变化

以 `int f(int n) { return n < 2 ? 1 : n * f(n-1); }` 为例，`f(3)` 的调用过程：

时刻	动作	栈帧数量	当前栈顶内容
T0	`main` 调用 `f(3)`	1	`f(3)` 的栈帧：参数 3、返回地址、旧 EBP
T1	`f(3)` 调用 `f(2)`（参数 `n-1=2` 已压栈）	2	`f(2)` 的栈帧叠在 `f(3)` 栈帧下方（更低地址）
T2	`f(2)` 调用 `f(1)`	3	`f(1)` 栈帧
T3	`f(1)` 满足 `n<2`，返回 1	$3 \to 2$	弹出 `f(1)` 栈帧，回到 `f(2)` 的 `imul` 之后
T4	`f(2)` 算出 $2 \times 1 = 2$，返回	$2 \to 1$	弹出 `f(2)` 栈帧，回到 `f(3)`
T5	`f(3)` 算出 $3 \times 2 = 6$，返回	$1 \to 0$	弹出 `f(3)` 栈帧，回到 `main`

结论：递归深度为 d 时，同一时刻栈上最多同时存在 d 个栈帧；递归调用次数为 d 次（`f(10)` 需要调用 10 次，`n` 依次为 $10, 9, \cdots, 1$）。每层栈帧的大小由该过程的参数个数、局部变量个数、需要保存的寄存器个数决定，编译期即可确定，因此递归的总栈空间与递归深度成正比。

递归栈类真题的三个问法

1. 问调用次数：`f(10)` 需要调用 `f1` 共 10 次（`n` 从 10 递减到 1），要数清「最后一次 `n=1` 不再递归」。
2. 问哪条指令递归调用：答案就是 `call f1` 这一条；再问「哪几条指令一定会使程序跳转」，则要答 `call`（无条件过程调用）、`jmp`（无条件跳转）、`ret`（无条件返回）——`jle` 这类条件转移不在其列，因为条件不成立时它不跳。
3. 问栈帧偏移与参数访问：参数 `n` 在 `[ebp+8]`；若某变量在 `[ebp-8]`，说明它是第 2 个 4 字节局部变量。

（五）MIPS 汇编基础

MIPS 是最典型的 RISC 机器，也是统考大题（尤其是流水线题）最常出现的指令系统。它的设计把 RISC 的特征体现得淋漓尽致。

*1.*体系结构与寄存器约定

- 指令长度固定 **32** 位，按字节编址，采用大端方式，要求按字边界对齐（字的地址必须是 4 的倍数）。

- **Load/Store** 架构：只有 `lw/sw` 能访存，运算指令的操作数必须来自寄存器。
- 32 个 32 位通用寄存器，编号 0~31，各占 5 位。

寄存器	编号	用途	保存责任
`$zero`	0	恒为 0，写入被丢弃	—
`$v0~$v1`	2~3	过程返回值	调用者保存
`$a0~$a3`	4~7	前 4 个参数	调用者保存
`$t0~$t9`	8~15、24~25	临时变量	调用者保存
`$s0~$s7`	16~23	局部变量/长期数据	被调用者保存（用了必须压栈）
`$k0~$k1`	26~27	内核保留	—
`$gp`	28	全局指针	—
`$sp`	29	栈指针（栈向低地址增长）	被调用者保存
`$fp`	30	帧指针（指向栈帧底部）	被调用者保存
`$ra`	31	返回地址（`jal` 自动写入）	被调用者保存（非叶子过程必须压栈）

*2.*三种指令格式

格式	字段划分（共 **32** 位，从左到右）	用途	寻址方式
R 型	`OP`(6) → `rs`(5) → `rt`(5) → `rd`(5) → `shamt`(5) → `func`(6)	算术、逻辑、移位运算	寄存器寻址
I 型	`OP`(6) → `rs`(5) → `rt`(5) → 立即数 / 偏移量 (16)	立即数运算、`lw/sw`、条件分支	立即数、基址、相对寻址
J 型	`OP`(6) → 目标地址 (26)	`j`、`jal` 无条件跳转	伪直接寻址

R 型指令的 `OP` 恒为 `000000`，具体运算由 6 位的 `func` 字段决定；`shamt` 用于移位指令指定移位位数，非移位指令该字段为 0。

I 型访存指令的有效地址：$EA = (rs) + \text{SignExt}(\text{立即数})$——这就是基址寻址，也是 MIPS 唯一原生的访存寻址方式。

I 型分支指令的目标地址：

$$\text{目标地址} = (PC + 4) + (\text{SignExt}(\text{offset}) \times 4)$$

即 PC 已自增 4，且 16 位偏移量以「指令条数」为单位，需左移 2 位换算成字节地址。

J 型指令的目标地址：把 26 位直接地址左移 **2** 位（末尾补两个 0，因为指令按字对齐）得到 28 位，再与 PC 高 4 位拼接：

$$\text{Target_PC} = PC[31{:}28] \parallel (\text{target} \ll 2)$$

*3.*常用指令

类别	指令	汇编举例	含义
算术	add	add $s1,$s2,$s3	$s1 = $s2 + $s3（带溢出检查）
	sub	sub $s1,$s2,$s3	$s1 = $s2 - $s3
	addi	addi $s1,$s2,100	$s1 = $s2 + 100（立即数符号扩展）
访存	lw	lw $s1,100($s2)	$s1 = M[$s2 + 100]
	sw	sw $s1,100($s2)	M[$s2 + 100] = $s1
逻辑	and / or / nor	and $s1,$s2,$s3	按位与 / 或 / 或非
	andi / ori	andi $s1,$s2,100	立即数零扩展后按位运算
移位	sll / srl	sll $s1,$s2,2	逻辑左移 / 右移（shamt 字段）
条件分支	beq	beq $s1,$s2,L	if ($s1 == $s2) goto L
	bne	bne $s1,$s2,L	if ($s1 != $s2) goto L
	slt / slti	slt $s1,$s2,$s3	if ($s2 < $s3) $s1 = 1; else $s1 = 0
跳转	j	j L	无条件跳转
	jal	jal L	$ra = PC+4; goto L（过程调用）
	jr	jr $ra	goto $ra（过程返回）

MIPS 没有状态码

x86 靠标志位（ZF/SF/OF/CF）判断条件，**MIPS** 没有状态码寄存器。它判断条件只有两条路：① 用 beq/bne 直接比较两个寄存器；② 需要比较大小时，先用 slt/slti 把结果（0 或 1）放进临时寄存器，再用 beq/bne 与该寄存器或 $zero 比较。「先 **slt** 再 **beq**」是 **MIPS** 分支的两步法。

*4.*简单 *C* 语句的 *MIPS* 翻译

（1）if-else 的翻译

```
if (check_val == target_val)
    score = score + bonus;
else
    score = score - penalty;
```

```
    bne  $s0,$s1,label_else   ; 用" 反面" 条件跳过 IF 块
    add  $s2,$s2,$s3          ; score = score + bonus
    j    label_exit
label_else:
    sub  $s2,$s2,$s4          ; score = score - penaltylabel_exit:
```

若条件涉及大小比较（如 if （x > y） max = x; else max = y;），先把 x > y 转成等价形式 y < x：

```
    slt  $t0,$s1,$s0          ; 若 y < x 则 $t0 = 1，即 x > y 成立
    beq  $t0,$zero,label_else ; $t0 == 0 说明 x <= y，跳去 Else
    add  $s2,$s0,$zero        ; max = x
    j    label_exit
```

```
label_else:
    add  $s2,$s1,$zero        ; max = ylabel_exit:
```

（2）循环的翻译（含数组变址）

```
while (i < size) {
    sum = sum + src[i];
    i++;
}
```

MIPS 没有自动变址寻址，lw/sw 的唯一形式是 offset(base) 且 offset 必须是立即数。因此访问 src[i]（int 数组）必须显式算出地址：

$$EA = \text{src 首地址} + i \times 4$$

```
Loop:
    slt  $t0,$s0,$s1          ; i < size ?
    beq  $t0,$zero,Exit       ; 不成立则跳出
    sll  $t1,$s0,2            ; $t1 = i * 4   （左移 2 位代替乘法）
    add  $t1,$t1,$s3          ; $t1 = src + i*4，即 src[i] 的地址
    lw   $t2,0($t1)           ; 取出 src[i]
    add  $s2,$s2,$t2          ; sum += src[i]
    addi $s0,$s0,1            ; i++
    j    Loop                 ; 跳回循环头
Exit:
```

数组变址的移位规律

int（4 字节）数组：下标乘以 4 → sll 左移 **2** 位；short（2 字节）数组：下标乘以 2 → sll 左移 **1** 位；char 数组：下标即字节偏移，不移位。看到 sll rX，rY，2 就应该反应出「这是在算 int 数组元素的地址」。

（3）过程调用的翻译：叶子过程与非叶子过程

叶子过程（内部不再调用其他过程）：不需要栈帧，因为 $ra 不会被覆盖，$t 系列寄存器又属于调用者保存。

```
int leaf_calc(int a, int b) { return (a + b) * 4; }
```

```
leaf_calc:
    add  $t0,$a0,$a1      ; $t0 = a + b
    sll  $v0,$t0,2        ; $v0 = $t0 << 2  （左移代替乘以 4）
    jr   $ra              ; 直接返回
```

非叶子过程（内部还要调用其他过程）：必须建立栈帧并保存 $ra。

```
int non_leaf(int x, int y, int z) {
    int sum = leaf_calc(x, y);
    return sum + z;
}
```

```
non_leaf:
    addi $sp,$sp,-8       ; 开辟 8 字节栈帧
    sw   $ra,4($sp)       ; 必须保存返回地址：后面的 jal 会覆盖 $ra
    sw   $s0,0($sp)       ; 保存被调用者保存寄存器 $s0
    addu $s0,$a2,$zero    ; 把 z 挪到 $s0 暂存（$a2 可能被子过程破坏）
    jal  leaf_calc        ; 调用子过程，返回值在 $v0
    add  $v0,$v0,$s0      ; sum + z
    lw   $s0,0($sp)       ; 恢复 $s0
    lw   $ra,4($sp)       ; 恢复返回地址
    addi $sp,$sp,8        ; 释放栈帧
    jr   $ra              ; 返回
```

非叶子过程必须保存 $ra 的原因

执行 jal 时硬件会自动把下一条指令的地址写入 **$ra**，覆盖原来的值。如果非叶子过程不在开头把 $ra 压栈，本函数原本的返回地址就永久丢失，程序会陷入死循环或跳飞。这是「为什么必须用栈」的核心原因。

本章小结表

考纲条目	核心结论	关键公式 / 数字	常考题型
指令系统的基本概念	ISA 是软硬件界面；机器字长 = ALU/通用寄存器宽度；指令字长与机器字长无约束关系；指令字长决定 IR 位数	机器字长 = 定点数据通路位数	概念选择
指令格式	操作码 + 地址码；定长操作码 $n \geq \lceil \log_2 m \rceil$；扩展操作码逐级「展开再扣减」	每减少一个地址字段，可用编码乘 2^p	扩展操作码计数
零/一/二/三地址指令	访存次数取决于操作数位置与是否写回，不取决于地址码个数	三地址取两源写一结果，最多 4 次访存	位宽分配、寻址范围
寻址方式	偏移寻址 = 寄存器 + 形式地址（相对/基址/变址）；间接寻址不是偏移寻址	相对 $EA = (PC) + D$，**PC** 已自增	转移目标地址计算
各寻址方式访存次数	立即/寄存器 0 次；直接/寄存器间接/相对/基址/变址 1 次；间接 2 次起	括号层数 = 访存层数	选择、计算
大小端	大端：MSB 在低地址；小端：LSB 在低地址	大端 LSB 地址 = 首地址 + 字节数 − 1	字节序列题
边界对齐	每成员从自身对齐要求的整数倍偏移开始；末尾补齐到最大对齐值的整数倍	填充可出现在成员间与结构体末尾	结构体大小/成员地址
CISC 与 RISC	指令数、字长、访存限制、寄存器数、控制器、流水线、编译优化、兼容性	CISC 微程序控制；RISC 硬布线控制	概念辨析选择
编译/汇编/链接	预处理 → 编译 → 汇编 → 链接	编译产出汇编代码；汇编产出目标文件	顺序选择
选择结构	cmp/test 设标志位（不改操作数）+ jcc 反向跳转	有符号 jg/jl（SF/OF）；无符号 ja/jb（CF/ZF）	汇编分析
循环结构	do-while 后测试（至少执行一次）；while/for 前测试	条件成立向后跳回，不成立向前跳出	汇编分析

考纲条目	核心结论	关键公式 / 数字	常考题型
过程调用	调用者保存 vs 被调用者保存；`call` 压返回地址、`ret` 弹出	`[ebp+8]` 为第 1 个参数，`[ebp-4]` 为第 1 个局部变量	栈帧布局、递归
MIPS	32 位定长、Load/Store、无状态码、`$ra` 存返回地址	I 型分支：$(PC+4)+\text{signExt}(off)\times 4$	大题（常与流水线结合）

易错清单

逐条核对，每一条都是统考反复设置的陷阱

1. 扩展操作码的第一级要先「展开」再「扣减」。正确顺序是「上一级剩余编码数 $\times 2^p$ − 本级已用条数」，顺序颠倒会得到完全不同的答案。
2. 零地址指令的可用编码数等于上一级剩余前缀数 $\times 2^p$。漏掉最后这一步乘法是最常见的失分点。
3. 操作码位数取定后必须逐级验证容量。某一级不够用（如二地址需要 300 条但只展开出 48 个编码），说明起始操作码位数取小了，必须加长重算。
4. 相对寻址公式里的 (PC) 是已自增后的值，等于「当前指令地址 + 当前指令长度」，不是当前指令的地址。漏加指令长度会差出整整一条指令的长度。
5. 相对寻址的位移量是补码，为负数时要先符号扩展再做加法，不能当无符号数直接加。
6. 「偏移寻址」只包含相对寻址、基址寻址、变址寻址三种，间接寻址不属于偏移寻址。
7. 基址寻址与变址寻址公式相同但角色相反：基址寄存器存基准地址（执行中不变），变址寄存器存偏移量（执行中递增）；形式地址在基址寻址中是偏移量，在变址寻址中是基准地址。
8. 变址寄存器的内容是下标还是字节偏移，必须看题目约定。两者相差一个 `sizeof(元素)` 因子（`int` 数组差 4 倍）。
9. 间接寻址的寻址范围由主存单元字长决定，与形式地址位数无关；寄存器间接寻址的寻址范围由寄存器位数决定。
10. 大端方式下 **LSB** 不在首地址。求「LSB 所在地址」时，大端是「首地址 + 字节数 − 1」，小端才是「首地址」。
11. 基址寻址中补码形式地址要符号扩展，32 位相加后最高位的进位要丢弃。
12. 填充字节可能出现在结构体末尾。总大小必须是「最大成员对齐要求」的整数倍，这一步最容易被漏掉。
13. 填充字节的内容是不确定的，考试只考位置和字节数，不要假设它是 0。
14. 结构体的对齐值取「最大成员的对齐要求」，不是「所有成员字节数之和」，也不是机器的最大对齐值（取两者中较小者）。
15. 「**RISC** 普遍采用微程序控制器」是错误的。RISC 普遍采用硬布线（组合逻辑）控制，CISC 才以微程序控制为主。
16. 「通用寄存器数量不多」是 **CISC** 的特征，RISC 的通用寄存器数量多。
17. 「严格限制只有 **LOAD/STORE** 指令能访存」是 **RISC** 的特征，CISC 对访存指令不加限制。
18. 「大多数 **RISC** 架构能与老机器软件兼容」是错误的，只有 CISC 具备这种向后兼容性。
19. 「**RISC** 大多数指令在一个时钟周期内完成」是正确的，不要因为「复杂功能要多条指令组合」就误判为错误。
20. 编译与汇编的产物不要颠倒：编译产出汇编语言程序，汇编产出目标文件（机器语言，尚不可执行）；完整顺序是「预处理 → 编译 → 汇编 → 链接」，链接之后才是可执行目标文件。
21. `cmp` 和 `test` 不改变操作数的值，只更新标志位。`cmp` 底层做减法，`test` 底层做按位与。
22. 有符号比较用 `jg/jl`（看 **SF** 与 **OF**），无符号比较用 `ja/jb`（看 **CF** 与 **ZF**）。混用会在负数或大数上出错。
23. `if-else` 的汇编常用「反面条件」跳转：条件不成立才跳到 Else，条件成立时顺序执行 Then 块；Then 块末尾必须有 `jmp` 跳过 Else。

24. 只有 `do-while` 的循环体至少执行一次，`while` 和 `for` 初始条件为假时循环体一次都不执行。
25. 循环条件的条件跳转是「成立则向后跳回」，跳出循环的跳转是向前跳。方向写反即为错误。
26. 参数压栈顺序是「从右至左」，因此第一个参数的地址最小（`[ebp+8]`），最后一个参数的地址最大。
27. `[ebp+4]` 是返回地址，`[ebp]` 是旧的 **EBP**，所以第一个参数在 `[ebp+8]`，第二个在 `[ebp+12]`。这个 8 的来由必须能推出来。
28.「调用者保存」说的是「调用者若想保值就得自己保存」，不是「调用者有义务为被调用者保存」；被调用者可以随意覆盖调用者保存寄存器。
29. 非叶子过程必须在开头把 `$ra` 压栈（MIPS）/ `call` 会自动覆盖返回地址（x86），因为过程中的下一次调用会覆盖返回地址。
30. 递归深度决定栈帧数量：深度为 d 时同一时刻栈上最多有 d 个栈帧，递归总栈空间与深度成正比。`f(10)` 需要调用 10 次，不是 11 次。
31. **MIPS** 没有状态码寄存器，比较大小时必须「先 `slt` 再 `beq/bne`」两步走，不能用 x86 的 `jg/jl` 思路去套。
32. **MIPS** 的 **I** 型分支指令偏移量以「指令条数」为单位，目标地址要算 $(PC+4)+\text{SignExt}(off)\times 4$，漏乘 4 就会算错。
33. **MIPS** 访问数组必须显式计算地址（`sll` 算字节偏移 + `add` 加首地址），没有 x86 那种「基址 + 变址 × 比例」的一条指令完成自动变址。
34. 指令字长与机器字长没有任何约束关系，不要因为「指令字长 16 位」就推断「机器字长也是 16 位」。IR 的位数由指令字长决定，MAR 的位数由地址线位数决定，MDR 的位数由数据线位数决定。

第十二章中央处理器

本章考纲要求

五、中央处理器（CPU）（一）CPU 的功能和基本结构（二）指令执行过程（三）数据通路的功能和基本结构（四）控制器的功能和工作原理（五）异常和中断机制

1. 异常和中断的基本概念
2. 异常和中断的分类
3. 异常和中断的检测与响应

（六）指令流水线

1. 指令流水线的基本概念
2. 指令流水线的基本实现
3. 结构冒险、数据冒险和控制冒险的处理
4. 超标量和动态流水线的基本概念

（七）多处理器基本概念

1. SISD、SIMD、MIMD、向量处理器的基本概念
2. 硬件多线程的基本概念
3. 多核（multi-core）处理器的基本概念
4. 共享内存多处理器（SMP）的基本概念

CPU 是计组分值最重、也最容易出综合大题的一章。本章的考察呈「三条主线」：结构主线（寄存器与数据通路的组成、位数、控制信号）、时序主线（指令周期—机器周期—时钟周期的层级与各周期的微操作序列）、数量主线（数控制存储器的容量、数数据通路需要多少时钟周期、数流水线的总周期与吞吐率）。

只要把「微操作 → 控制信号 → 时钟周期数」这条链条打通，本章的选择题与应用题就都是同一套方法。以下按考纲条目顺序展开。

一、CPU 的功能和基本结构

（一）CPU 的功能

CPU（Central Processing Unit）由运算器和控制器两大部分构成。现代 CPU 还把 Cache、MMU（存储管理部件）、浮点运算部件、异常与中断处理逻辑集成到片内，但按「数据通路 + 控制部件」的两分法理解，任何复杂的 CPU 都不例外。

CPU 的四个基本功能：

- 指令控制：控制指令的执行顺序。程序是指令的有序集合，CPU 必须保证指令按序取、按序执行，由 PC 保证。
- 操作控制：产生每条指令的操作控制信号序列（即微命令序列），送到各执行部件的控制端。

- 时间控制：对各种操作信号进行时间调制，规定每条控制信号的产生时刻与持续时间，即「什么时候送操作数、什么时候发运算控制、什么时候送结果」。
- 数据加工：对数据进行算术运算、逻辑运算与逻辑测试。

一句话记住四条功能

指令控制（顺序）+ 操作控制（命令）+ 时间控制（时序）+ 数据加工（算术逻辑）。

（二）运算器的组成与各部件的作用

运算器的核心任务是「对数据进行加工」，由下列部件组成：

部件	英文/缩写	作用
算术逻辑单元	ALU	完成算术运算、逻辑运算；输出运算结果与标志
通用寄存器组	GPRs / GR	存放操作数、中间结果、地址信息；对汇编程序员可见
暂存寄存器	X、Y、Z 等	暂存 ALU 的输入操作数或输出结果；对程序员透明
累加寄存器	ACC	通用寄存器之一，常作为 ALU 的一个输入端与结果的落点
移位器	—	对操作数或运算结果进行移位（左移、右移、算术/逻辑移位）
程序状态字寄存器	PSW / PSR / FLAGS	保存 OF、SF、ZF、CF 等状态标志与中断、系统状态信息
计数器	—	控制乘、除运算的操作步数

需要特别说明的是暂存寄存器为什么必需。在单总线数据通路中，ALU 是纯组合逻辑部件，做一次运算时必须保证两个输入端同时有效。若第一个操作数从主存读出后直接放在总线上，等第二个操作数再经总线送出时前一个操作数已被总线上的新数据破坏，因此必须先把先到的操作数锁存到暂存寄存器 X（另一端固定接 ALU 输入）。同理，ALU 的输出端不能直接连总线，否则输出会经总线反馈到输入端形成竞争，所以运算结果要先锁存到暂存寄存器 Z（或 Y）。

- 暂存寄存器对应用程序员透明，不参与指令系统，不能用汇编指令直接访问。
- 累加寄存器 **ACC** 是通用寄存器，与「暂存寄存器」性质不同：ACC 程序员可见，暂存寄存器不可见。
- 移位器不是所有运算器都必备，有的机器把移位功能做进 ALU，或让 ALU 输出直接接数据总线/地址总线。做题时必须按题目给的框图判断，不能想当然。

PSW 的作用有两个层次：

1. 保存算术逻辑运算或测试指令所产生的状态信息——溢出标志 **OF**、符号标志 **SF**、零标志 **ZF**、进位/借位标志 **CF**、辅助进位 **AF**、奇偶标志 **PF**。条件转移指令正是根据 PSW 的内容判定是否转移。
2. 保存中断与系统工作状态信息，如中断允许标志 IF、陷阱标志 TF、方向标志 DF、处理器工作状态位（内核态/用户态）。操作系统靠它判断 CPU 当前所处状态。

PSW 的两个高频陷阱

- **PSW** 可能只在某些指令执行后改变。普通传送指令（如 `MOV`）通常不改变标志位，因此「某指令执行后 ZF=1」这类判断要先看该指令是否影响标志。
-「标志位由 ALU 产生」并不等于「ALU 是时序元件」。**ALU** 仍是组合逻辑元件，标志位只是它的组合输出，需要由 PSW 这个时序元件锁存才得以保存。

（三）控制器的组成与各部件的作用

控制器的任务是「根据指令、时序信号和状态条件产生控制信号」。

部件	英文/缩写	作用
程序计数器	PC / IP	存放下一条待执行指令的地址
指令寄存器	IR	存放当前正在执行的指令，其位数 = 指令字长
指令译码器	ID	把 IR 中的操作码翻译成译码信号，同一时刻只有一路有效
存储器地址寄存器	MAR / AR	存放将要访问的主存单元地址，经地址总线送主存
存储器数据寄存器	MDR / DR	存放从主存读出或将要写入主存的数据，是 CPU 与主存间的缓冲
微操作信号发生器	CU	综合译码信息、时序信号与状态条件，产生微命令序列
时序系统	—	产生时钟脉冲、机器周期、节拍电位等定时信号

各部件的工作脉络是：**PC** → **MAR** → 主存 → **MDR** → **IR** → 译码器 → 微操作信号发生器 → 各部件控制端。IR 中的地址码部分还要经地址生成逻辑（按寻址方式）加工，产生的地址按需要送入 PC、MAR 或运算部件。

（四）各寄存器的位数与地址、字长的关系

这是选择题每年都可能出现的一类题，必须区分清楚「哪个寄存器由谁决定」。

PC 与 MAR 由主存地址位数决定。

- **MAR** 的位数等于主存地址总线的位数，即 MAR 位数 $= \log_2$(可寻址单元总数)。它反映的是最多能寻址多少个存储单元。无论取指令还是取操作数，地址都必须先送 MAR。
- **PC** 的位数一般与主存地址总线位数相等，因为 PC 保存的就是一个主存地址。

但如果题目问的是「至少需要多少位」，就要看指令在内存中的存放方式：

地址对齐如何压缩 PC 位数

若按字节编址、指令字长 32 位（4 B）且按字边界对齐存放，则任何一条指令的起始字节地址的低 2 位恒为 00。此时 PC 只需保存「对齐位置」的编号共 2^{30} 个，即 **30** 位；形成真实字节地址时在低位补两个 0 即可。而 **IR** 必须完整保存一条 **32** 位指令，至少需要 **32** 位。

这就是「主存 4 GB、字长 32 位、按字节编址、32 位定长指令字、按字边界对齐」时答案取 **PC = 30** 位、**IR = 32** 位的原因。

MDR/IR 与机器字长、存储字长的关系。

- **IR** 的位数 = 指令字长。定长指令系统中它等于机器字长（或者说指令字长）；变长指令系统中 IR 要能装下最长的指令。
- **MDR** 的位数 = 存储字长，因为 MDR 是 CPU 与主存之间一次数据交换的宽度。常见配置中存储字长与机器字长相等，所以有「MDR 位数 = 存储字长 = 机器字长」的说法；但这一等式并不必然成立。若主存的存储字长与机器字长不等（例如机器字长 64 位、存储字长 32 位），则 MDR 只需 32 位。

位数关系的四个易错点

- **MAR** 只看「可寻址单元个数」，不看编址方式本身。按字节编址的 4 GB 主存，地址 32 位，MAR 就是 32 位；若按字（4 B）编址，则地址位数减 2，MAR 也随之减 2。
- **PC** 与 **MAR** 一般都等于地址总线位数，但在「至少」类问题中 **PC** 可以更短（靠地址对齐省掉低位）。
- **MDR** 由存储字长决定，不由机器字长决定，两者只是常常相等。
- 通用寄存器的位数 = 机器字长（反映一次能处理的数据位数），与地址位数无关。32 位机中 GPR 32 位，但主存可能有 4 GB 即地址 32 位，两者数值相同只是巧合；若主存是 64 GB，则地址 36 位而 GPR 仍是 32 位。

真题考法

- 「汇编语言程序员可见的寄存器」——PC、通用寄存器、基址/变址寄存器、PSW 可见；**MAR**、**MDR**、**IR**、微指令寄存器、暂存寄存器、控制存储器（**CM**）不可见。可见性的判据是「程序能否通过指令语义观察或改变它的状态」，而不是「能否把它直接当操作数写在汇编里」。
- 「PC 和 IR 的位数至少是多少」——按上面对齐法算。
- 「时钟脉冲信号」的性质：时钟周期由机器脉冲源经整形分频产生，其倒数是主频；同步时序电路要求的时钟周期不短于「寄存器之间最长组合逻辑路径传播时间 + 寄存器建立时间」。但「每来一个时钟脉冲就执行一条新指令」是错的：多周期 CPU 中一条指令占多个时钟周期，流水线被阻塞时也可能没有新指令进入。

二、指令执行过程

（一）指令周期的层级：指令周期、机器周期、时钟周期

一条指令从主存取出到执行完毕所需的全部时间称为指令周期（**instruction cycle**）。它是 CPU 执行一条指令的时间，是本章时序分析的最高层级。指令周期可以划分为若干机器周期（**CPU** 周期），每个机器周期又可以划分为若干时钟周期（节拍脉冲 / 振荡周期）。三者是层层包含的关系：

$$\text{指令周期} \supset \text{机器周期} \supset \text{时钟周期}, \qquad T_{\text{指令}} = \sum_i T_{\text{机器}i}, \qquad T_{\text{机器}} = \sum_j T_{\text{时钟}j}$$

层级	定义	能完成的工作	由什么决定
指令周期	取出并执行一条指令的全部时间	一条完整指令	指令功能、寻址方式、是否发生中断
机器周期	从主存读出一个指令字所需的最短时间	一个较复杂的基本操作（如一次存储器读或写）	通常以主存工作周期为基础确定
时钟周期	主频的倒数（$T = 1/f$）	一次微操作	机器脉冲源经整形分频

关于机器周期的宽度，考纲与统考的口径是：机器周期的宽度通常以主存工作周期为基础来确定。原因是主存的读/写周期比 CPU 内部的一次运算操作长得多，若以 CPU 内部操作为准，则存储器根本来不及响应。

不同指令的指令周期长度不同，这是理解取指、间址、执行、中断四周期的关键：

- 空操作指令 NOP：只包含取指周期（但仍会改变 PC 与 IR）。
- 无条件转移指令 JMP：包含取指周期 + 执行周期。
- 采用间接寻址的指令：多出一个间址周期。
- 在指令结束时若检测到中断请求：多出一个中断周期。

因此，一个完整的指令周期可能包含取指周期、间址周期、执行周期、中断周期四个阶段，其中执行周期可以有 0 ~ n 个机器周期。

三条必背结论

1. 任何指令的指令周期都至少包含一个机器周期，也至少包含一个时钟周期，即 $T_{指令} \geq T_{机器} \geq T_{时钟}$。因此「指令周期一定大于或等于一个 CPU 时钟周期」是正确的。
2. 只要不采用 **Cache** 和不采用指令预取，每条指令的指令周期中 **CPU** 至少访问内存一次（取指）。
3. 指令周期的长度由功能决定，不由时钟周期数固定。单周期 CPU 中所有指令都占 1 个时钟周期，但那个时钟周期必须覆盖最慢指令的完整通路，所以周期很长、主频很低。

（二）取指周期的流程与数据通路

取指周期的任务是「按 PC 从主存取出指令送入 IR，并形成下一条指令地址」。用寄存器传送语言（RTL，Register Transfer Language）描述为：

```
(PC)   -> MAR          ; 现行指令地址送地址寄存器
(MAR)  -> MDR  等价于 M(MAR) -> MDR   ; 按地址读出指令字
(MDR)  -> IR           ; 指令送指令寄存器
(PC)+1 -> PC           ; 形成下一条指令地址（"1" 表示一条指令的长度）
```

对应的控制信号序列（以单总线数据通路为例）：

时钟	数据通路（微操作）	控制信号
C1	(PC) → MAR	PCout，MARin
C2	M(MAR) → MDR；(PC)+1 → PC	read，MDRinE，PCinc（+1）
C3	(MDR) → IR	MDRout，IRin

操作控制器在取指周期要发出 `PCout`、`MARin`、`read`、`MDRinE`、`MDRout`、`IRin`、`PCinc` 等信号。注意第三步不能与第二步合并：数据从主存读出并稳定地出现在 MDR 上需要时间，同一时钟周期内不能既完成访存又完成经总线向 IR 的传送。

（三）间址周期的流程与数据通路

间址周期的任务是「取出操作数的有效地址」。它在取指周期结束后，由译码器分析 IR 确定该指令使用了间接寻址才进入。以 `MOV R1,(05H)` 这类源操作数采用间址寻址的指令为例：

```
Ad(IR) -> MAR          ; 把形式地址（05H）送地址寄存器
1      -> R            ; 发读命令
M(MAR) -> MDR          ; 从主存读出有效地址
(MDR)  -> Ad(IR)  或者  (MDR) -> MAR   ; 有效地址送回地址码字段或直接送 MAR
```

时钟	数据通路（微操作）	控制信号
C4	Ad(IR) → MAR	AD(IR)out，MARin
C5	M(MAR) → MDR	read，MDRinE

时钟	数据通路（微操作）	控制信号
C6	(MDR) → MAR	MDRout，MARin

间址周期每增加一次间接寻址，就要多访问一次主存。一次间址需要一个完整的机器周期（因为必须访存一次）。

取指微程序与间址微程序的关系

取指周期的微程序是公共的，对每条机器指令都相同，微程序控制器直接提供；而间址周期与执行周期的微程序是在取指结束后、由指令译码器分析操作码一次性生成的，然后在各自阶段的时钟到来时分别启动。这一点是理解微程序控制器「下地址逻辑」的关键。

（四）执行周期的流程与数据通路

执行周期的任务是「根据 IR 中指令字的操作码和操作数，经 ALU 处理后产生执行结果」。不同指令的执行周期操作不同，没有统一的数据流向，这是它与其他三个周期的根本区别。

仍以「`MOV R1,(05H)`，源操作数间址」为例：间址周期已取得有效地址并放入 MAR，执行周期要把该地址中的数据写入 R1：

时钟	数据通路（微操作）	控制信号
C7	M(MAR) → MDR	read，MDRinE
C8	(MDR) → R1	MDRout，R1in

各类指令在执行周期中的典型微操作（单总线结构、只涉及单次传送的示例）：

- 寄存器之间传送：`(R1) → R2`，一个时钟周期，`R1out`、`R2in`。
- 加法运算并写回：`(R1)→Y`；`(ACC)+(Y)→Z`；`(Z)→ACC`，需 **3** 个时钟周期，因为任何时刻总线上只能有一个数据。
- 存数：`(R1)→MDR`；`(R2)→MAR`；`MDR→M(MAR)`，通常 3 个时钟周期。
- 转移：`Ad(IR) → PC`，一个时钟周期，`IRout`、`PCin`。

（五）中断周期的流程与数据通路

中断周期发生在一条指令执行结束、且检测到中断请求并允许响应时，任务是保存断点并转去中断服务程序。若将断点保存在栈中、SP 指向栈顶、进栈操作为「先修改指针后送入数据」，则：

```
(SP)-1 -> SP          ; 栈指针先减 1（若约定 SP 指向栈顶空单元则改为 (SP)->MAR 后再 (SP)-1->SP）
(SP)   -> MAR         ; 断点要存入的存储单元地址送 MAR
1      -> W           ; 发写命令
(PC)   -> MDR         ; 把断点（当前 PC）送 MDR
(MDR)  -> M(MAR)      ; 断点写入主存
向量地址 -> PC        ; 中断服务程序入口地址送 PC，转向服务程序
```

中断周期的微操作可概括为：关中断、保存断点、引出中断服务程序。这三件事由中断隐指令完成（详见异常与中断机制一节）。

（六）各周期微操作的通用分析方法

综合题的答题步骤可以固定为：

1. 列时序：先确定这条指令要经历哪几个周期（取指？间址？执行？中断？）。
2. 标数据流：每个周期写出所有必须在同一时刻完成的 RTL 微操作。
3. 查冲突：同一时钟周期内检查三条「一」规则——同一寄存器只出现一次、同一总线只使用一次、主存/ALU 只访问一次。
4. 数周期：每个时钟周期完成一次不冲突的数据传送，周期数即为时钟数。

指令执行过程的常见陷阱

- 「空操作指令的指令周期中任何寄存器内容都不改变」是错的。NOP 不完成显式数据运算，但取指仍会改变 PC（自增）和 IR。
- 「每个时钟周期都开始执行一条新指令」是错的（多周期、流水线阻塞都不成立）。
- 外中断在指令执行结束后检测，内中断（异常）在指令执行过程中检测。混用这两句几乎必错。

三、数据通路的功能和基本结构

（一）数据通路的定义与组成元件

数据通路（**datapath**）：指令执行过程中数据所经过的路径，以及路径上涉及的部件。数据通路由操作元件和状态元件两类元件组成，二者必须分清：

类别	别名	特性	典型部件
组合逻辑元件	操作元件	输出只取决于当前输入，不受时钟控制，无记忆	ALU、加法器、移位器、多路选择器 MUX、译码器、扩展器
时序逻辑元件	状态元件	能保存状态，受时钟驱动，有记忆	PC、IR、MAR、MDR、通用寄存器组、暂存寄存器、PSW、流水段寄存器、存储器

判据

「这个部件会不会记住上一次的结果」——会记住的是状态元件，不记住的是操作元件。

- ALU、MUX、扩展器、加法器：操作元件。
- PC、GPRs、暂存寄存器 X/Z、流水段寄存器：状态元件。
- 通用寄存器组的读操作是组合逻辑（给地址就有数据输出），写操作是时序逻辑（要时钟沿触发），因此 GPRs 可以在同一个时钟周期内完成一次「读 + 写」。

数据通路由控制部件产生的控制信号控制：寄存器的写使能、MUX 的选择端、ALU 的功能选择、存储器的读写控制，都来自控制器。

容易混淆的两条

- 生成控制信号的控制部件本身不属于数据通路。所以「指令流水线的数据通路包含生成控制信号的控制部件」是错的。
- 数据通路要包含（或配合）异常事件的检测与响应电路：如运算溢出检测、地址越界/对齐检测、以及异常入口所需的状态保存和 PC 选择通路。所以「数据通路不包含用于异常事件检测及响应的电路」是错的。

（二）单总线结构的数据通路

把 ALU 和所有寄存器都连到一条内部公共总线上，就是单总线结构。

- 由于单总线上某一时刻只能有一个部件输出数据，运算器必须配置暂存寄存器 **X**（暂存 ALU 的一个操作数）与 **Z**（暂存运算结果）；ALU 的另一个操作数直接来自内总线。
- 所有向总线输出的部件都通过三态门控制，防止总线冲突；`PCout`、`MDRout`、`IRout`、`Rout`、`Zout` 等是互斥的。
- 单总线结构不能用来实现单周期处理器：一条稍复杂的指令需要多次总线传送，一个时钟周期内根本做不完。因此单周期 CPU 必须使用专用数据通路。

在单总线结构中，「某一时刻总线上只有一个数据有效」决定了每一组必须前后相继的传送各占一个时钟周期。

（三）多总线结构与专用数据通路

- 多总线结构：CPU 内部有两条或更多条总线。三条总线时，总线 A、B 分别传送两个源操作数，总线 C 传送结果，因此暂存寄存器 **Y**、**Z** 可以取消（三条总线各传不同数据，不会冲突）。配合双口通用寄存器组，`R[R3] ← R[R1] op R[R2]` 只需一个时钟周期。
- 专用数据通路：按指令执行过程中数据与地址的流动方向安排专门连接，不使用共享总线。优点是避免资源竞争、性能高、便于实现流水线；缺点是硬件量大。现代处理器普遍采用专用数据通路。

在专用通路中，若要把一个寄存器的内容传送到另一个寄存器，通常借道 **ALU** 完成（控制 ALU 做「直送」操作）。

真题考法

「单周期处理器为什么不能采用单总线结构」几乎每年都以不同形式出现。答题要点是：单总线一次只能完成一次数据传送，而单周期要求所有指令在一个时钟周期内完成（CPI=1），两者矛盾。单周期的代价是时钟周期必须按最慢指令的关键路径确定，主频低；其好处是控制信号在整条指令执行期间保持不变，不需要复杂的时序控制。

（四）数据通路的数据传送过程与时钟周期数计算

这是主频最高的大题考点。核心方法论只有一句：一个时钟周期完成一次不发生冲突的数据传送；凡是必须在不同时刻发生的传送，就要各占一个时钟周期。

给出两条硬性规则（可直接用于排节拍）：

1. 同一节拍中，同一寄存器只能出现一次（不能既读又写同一寄存器作为源和目的）。
2. 同一节拍中，同一条总线只能使用一次；同一节拍中只访问主存一次、只访问 **ALU** 一次。

完整算例：单总线数据通路执行 `MOV R1,(R2)`（源操作数间址寻址），计算所需时钟周期数。

设按字节编址、指令字长 32 位、按边界对齐，PC 自动加 1（表示加一个指令字长）。执行这条指令的全部微操作序列为：

阶段	节拍	微操作	控制信号	说明
取指	T1	(PC) → MAR	PCout，MARin	送指令地址

阶段	节拍	微操作	控制信号	说明
取指	T2	M(MAR) → MDR；(PC)+1 → PC	read，MDRinE，PCinc	访存取指，同时改 PC
取指	T3	(MDR) → IR	MDRout，IRin	指令入 IR
间址	T4	Ad(IR) → MAR	AD(IR)out，MARin	形式地址送 MAR
间址	T5	M(MAR) → MDR	read，MDRinE	取出有效地址
间址	T6	(MDR) → MAR	MDRout，MARin	有效地址留在 MAR
执行	T7	M(MAR) → MDR	read，MDRinE	按有效地址取操作数
执行	T8	(MDR) → R1	MDRout，R1in	写入 R1

结论：单总线数据通路执行该指令共需 **8** 个时钟周期。其中取指 3 拍、间址 3 拍、执行 2 拍。

对照：若采用三总线专用数据通路，且执行周期中的取数与写回都能并行安排，则取指仍为 3 拍（受主存访问限制），但间址可压缩为 2 拍、执行可压缩为 1 拍，总计约 6 拍。这说明数据通路结构直接决定指令所需的时钟周期数，做题必须按题目框图上「哪些部件之间有直接通路」逐拍判定。

时钟周期数计算的四句口诀

1. 数访存次数：每次主存读/写各占一拍（或题目规定的存储器等待周期数）。
2. 数总线传送次数：单总线上每次「一个源 → 一个目的」占一拍，且要检查是否冲突。
3. 数 **ALU** 运算次数：涉及 ALU 的运算另占一拍（输入已就绪的前提下）。
4. 能并行就并行：多总线结构下同一拍可以同时完成多次传送，能省则省，但受上面两条硬性规则约束。

四、控制器的功能和工作原理

（一）时序与控制

控制器要在正确的时刻发出正确的控制信号，所以必须有时序系统。时序信号的作用是对微操作控制信号进行时间调制，严格规定每条信号的产生时刻与持续时间。基本层次为：时钟脉冲（节拍）→ 机器周期 → 指令周期。

- 同步控制方式：所有操作由统一的时钟节拍控制，实现简单，但机器周期长度必须按最慢操作确定，效率较低。
- 异步控制方式：不给操作规定固定节拍，靠「回答信号」串联，效率高但控制复杂。
- 联合控制方式：多数操作按同步方式，少数慢操作（如访存）用异步回答，是实际机器的常见做法。

（二）硬布线控制器

硬布线控制器（组合逻辑控制器）：用专门的组合逻辑电路直接产生控制信号。它的输入是指令译码信号、时序信号（机器周期/节拍电位）和状态条件，输出是各部件所需的微命令。

设计思路（可按此复述）：

1. 分析微操作命令：对每条机器指令、每个机器周期，列出所有微操作，确定每条微操作对应哪个微命令、在哪个节拍发出。

2. 列出逻辑表达式：把「微命令 = 该微命令在哪些指令、哪个机器周期、什么条件下需要发出」写成布尔表达式。可用与阵列（译码器）—或阵列结构或可编程逻辑阵列 PLA 实现。
3. 时序：用节拍电位（机器周期状态）+ 节拍脉冲共同控制，保证信号按正确的时间顺序出现。

特点：

比较项	硬布线控制器	微程序控制器
控制信号来源	组合逻辑电路直接产生	执行控制存储器中的微指令产生
速度	快（省去了读微指令、译码微指令的时间）	慢（每条微命令要多一次访存 CM 和译码）
规整性	差，是一个多输入/多输出的巨大逻辑网络	规整，逻辑上与「程序设计」类似
修改与扩展	困难（要重新设计硬件逻辑）	容易（只需改写控制存储器中的微程序）
适用场合	指令系统简单规整（典型是 RISC、MIPS）	指令系统复杂的 CISC 处理器
实现方式	组合逻辑（可用有限状态机描述）	存储技术 + 程序设计技术，属「软件时序」

（三）微程序控制器

基本思想：仿照程序设计的方法，把每条机器指令的执行过程分解为若干步，每一步的微命令序列预先编制好并存放起来；执行指令时按顺序取出这些「微命令集合」去控制硬件。这就是「存储程序」思想在控制器设计上的应用。

一组术语必须分清（这是选择题高频区）：

- 微命令（**micro-command**）：控制部件向执行部件发出的控制信号，如 `PCout`、`MARin`、`ADD`、`read`。
- 微操作（**micro-operation**）：执行部件收到微命令后所进行的操作，如 PC 把内容送上总线、ALU 完成加法。微命令与微操作一一对应。
- 相容性 / 互斥性微操作：能在同一个节拍内并行执行的微操作为相容，不能并行为互斥。例如 `read` 与 `write` 互斥，各部件的「输出到总线」控制信号互斥（同一时刻总线上只能有一个数据），而各寄存器的「从总线锁存数据」使能信号可以相容（可同时有多个模块接收）。
- 微指令（**micro-instruction**）：同一节拍内发出的、相容的一组微命令的集合，即「一个状态」的全部控制信号。
- 微程序（**micro-program**）：完成一条机器指令所需的一组微指令序列。每条机器指令对应一段微程序。
- 控制存储器（**Control Storage**，**CS/CM**）：位于 CPU 内部、存放全部微程序的只读存储器。**CS** 按地址访问，不是按内容访问。
- 微地址：微指令在控制存储器中的地址。
- 微指令寄存器（μ**IR**）：在微指令执行期间暂存微指令。
- 微地址寄存器（μ**AR** / μ**PC**）：存放将要执行的微指令的微地址。
- 微周期（**micro-cycle**）：从控制存储器中读取并执行一条微指令所需的全部时间，通常为一个时钟周期。

微命令 / 微指令 / 微程序的三层关系

微命令（**1** 个控制信号）→ 微指令（同拍内相容的微命令集合）→ 微程序（实现一条机器指令的微指令序列）

由此得到两条极重要的等价关系：

- 一条机器指令需要多少个时钟周期，它的微程序就包含多少条微指令（反之亦然）。
- 机器指令与微指令是两个层次的东西。CPU 能直接理解并执行的是机器指令（以及微程序控制器内部的微指令）；伪指令、汇编指令都必须先由汇编器翻译成机器指令。
- 只有相容的微命令才能出现在同一条微指令中，这直接决定了微指令的并行能力。

*1.*控制存储器与主存的比较

项目	主存储器 **MM**	控制存储器 **CS**
存放内容	机器指令与数据	微程序（微指令）
位置	CPU 外部	CPU 内部
存储容量	大	小
存储字长	存储字长（常等于机器字长）	微指令长度
存取速度	慢	快（与 CPU 同周期量级）
实现介质	ROM 与 RAM 均可	一般是 ROM（只读型）
访问方式	按地址访问	按地址访问

CS 与 MM 的常见错误说法

「控制存储器按内容访问」是错的——它和主存一样按地址访问。按内容访问的是相联存储器（如 TLB 的实现方式）。

（四）微指令的格式与编码方式

微指令的基本格式包含三部分：

1. 操作控制字段（微命令字段）：给出本条微指令要发出的所有微命令。
2. 顺序控制字段（下地址字段 / 转移控制字段）：给出下一条微指令的微地址，或指明如何形成下一条微地址。
3. 判别测试字段（P 字段）：表示按什么条件进行分支判断，如按指令操作码无条件分支（P0）、按 PSW 的相等标志条件分支（P1）。

*1.*按操作控制字段的组织方式分类

- 水平型微指令：一次能定义并执行多个可并行的微命令。微指令字较长，微程序短，并行能力强、执行速度快，但微指令字长浪费大、编写困难。
- 垂直型微指令：一条微指令只能完成一个微操作（或 1 ~ 2 个），类似机器指令的格式，要用微操作码 + 操作数地址。微指令字短、编码空间利用率高，但微程序长、并行能力差、执行速度慢。

水平型微指令的并行能力

「水平型微指令并行操作能力强」这一结论的根据是——它把同一条微指令中的操作控制字段分成若干段，同一段内的微命令互斥、不同段之间的微命令相容，因此一次可以同时发出多条相容微命令。判断一条微指令能发出几个微命令，看的就是它的操作控制字段里「能同时置为有效」的位数。

*2.*微指令的编码方式

直接编码（直接控制法）：操作控制字段中每一位对应一个微命令，置 1 发该微命令，置 0 不发。

- 优点：不必译码，控制信号直接产生，速度最快，微命令可任意组合（并行能力最强）。

- 缺点：机器微命令个数很多时微指令字极长，而一条微指令中有效的微命令很少，编码空间利用率低。

字段直接编码（字段直接编码法）：把操作控制字段划分成若干小字段，互斥的微命令编在同一个字段，相容的微命令编在不同字段。每个字段经译码后产生该字段内的一个微命令。

- 关键规定：每个字段必须能够表示「本字段不发出任何微命令」这一状态，因此含 n 个互斥微命令的字段至少需要 $\lceil \log_2(n+1) \rceil$ 位。
- 总位数公式：

$$W_{\text{op}} = \sum_{i=1}^{m} \lceil \log_2 (n_i + 1) \rceil$$

其中 m 为字段（互斥类）个数，n_i 为第 i 个字段内的互斥微命令数。

- 优点：在保持一定并行能力的同时显著缩短微指令字长。
- 缺点：要经过译码，速度比直接编码慢一点。

字段间接编码（字段间接编码法）：一个字段中的微命令含义还要由另一个字段（或另一个字段的某几位）决定，简称「间接编码」。它能进一步缩短微指令字长，但译码电路更复杂、微指令并行能力进一步下降。统考只要求知道它的定义与「进一步缩短字长、并行性更差」这一结论。

完整算例：字段直接编码的位数计算（对照 2012 年统考真题）

某计算机采用微程序控制方式，操作控制字段采用字段直接编码法，共有 33 个微命令，构成 5 个互斥类，分别包含 7、3、12、5、6 个微命令，求操作控制字段至少多少位。

$$\lceil \log_2(7+1) \rceil + \lceil \log_2(3+1) \rceil + \lceil \log_2(12+1) \rceil + \lceil \log_2(5+1) \rceil + \lceil \log_2(6+1) \rceil$$

$$= 3 + 2 + 4 + 3 + 3 = 15 \text{ 位}$$

注意 $\lceil \log_2 13 \rceil = 4$（因为 $2^3 = 8 < 13 \le 16 = 2^4$），$\lceil \log_2 7 \rceil = 3$。答案 **15** 位。若误以为不需要「全 0 不操作」状态，就会算成 $3+2+4+3+3$ 之外的错误结果或漏掉一位；若误用直接编码则得到 33 位。

（五）微地址的形成方式

1. 计数器方式（增量方式）：微地址寄存器带计数功能，微指令按 μPC+1 顺序执行；遇到转移时用转移地址字段修改 μPC。
   - 优点：微指令中不必设置下地址字段，字长较短。
   - 缺点：不能实现多路条件分支，只能二路分支；编制微程序时不灵活。
2. 断定方式（下地址字段法）：微指令中直接给出下一条微指令的地址（下地址字段），由判别测试字段决定是否按该字段转移。
   - 优点：可实现多路分支，微程序编制灵活。
   - 缺点：每条微指令都要带下地址字段，微指令字较长。
3. 结合方式（混合方式）：把以上两者结合，顺序执行时用计数器，需要分支时用下地址字段，兼顾字长与灵活性。

微程序入口地址的形成（结合前面「转移微指令」的概念）：

- 取指微程序是所有指令公共的，其入口地址固定（如 0000）。取指结束后的第一条微指令是转移微指令，它的作用就是「根据机器指令的操作码，跳转到该指令对应微程序的入口地址」。

- 转移微指令通常通过一个专门的小型 ROM（记为 ROM1）实现：以 IR 中的操作码为输入（即索引），输出即该指令微程序的入口地址。
- 同一入口下再分支的情形（如 `lw`/`sw` 首地址相同但后续流程不同）可用第二个 ROM（ROM2）实现，也可用断定法的硬件逻辑实现。例如下地址字段给出 `1010`，当机器指令操作码的某一位为 1 时用异或门把它变成 `1001`：$1010 \oplus 0011 = 1001$，即可用极少的硬件实现二路分支。
- 微程序的最后一条微指令通常取「取下一条机器指令的取指微程序首地址」，从而回到取指，形成循环。

判别测试字段（P 字段）的作用

P 字段本身的位数由「有多少种判别测试条件」决定，因此 P 字段位数 $= \lceil \log_2(\text{测试条件数} + 1) \rceil$。若微程序共有 25 个状态（微指令），则下地址字段至少 $\lceil \log_2 25 \rceil = 5$ 位。

（六）微指令条数与控制存储器容量的计算

这是「数量主线」的核心题型。三条计算步骤：

1. 算微指令条数：

$$N_\mu = N_{\text{公共取指微程序}} + N_{\text{各指令微程序总和}}$$

1. 算微指令字长（位数）：$W_\mu = W_{\text{操作控制字段}} + W_{\text{判别测试字段}} + W_{\text{下地址字段}}$。
2. 算控制存储器容量：$C = N_\mu \times W_\mu$（单位 bit）或 $C = \dfrac{N_\mu \times W_\mu}{8}$（单位 B）；反之，下地址字段位数 $\geq \lceil \log_2 N_\mu \rceil$。

完整算例一（对照 **2014** 年统考真题）：某计算机采用微程序控制器，共有 32 条机器指令，公共取指微程序包含 2 条微指令，各指令对应的微程序平均由 4 条微指令组成，采用断定法（下地址字段法）确定下条微指令地址，求下地址字段至少多少位。

- 各指令微程序微指令数：$32 \times 4 = 128$ 条。
- 控制存储器至少容纳：$2 + 128 = 130$ 条。
- $2^7 = 128 < 130 \leq 256 = 2^8$，故下地址字段至少 $\lceil \log_2 130 \rceil = 8$ 位。

完整算例二（自编，用于练容量）：某机有 60 条机器指令，公共取指微程序 3 条微指令，各指令微程序平均 6 条微指令；微指令字长为操作控制字段 24 位、判别测试字段 2 位、下地址字段（由算例一的方法确定为 9 位）。求控制存储器容量。

- 微指令条数：$3 + 60 \times 6 = 363$ 条。
- 微指令字长：$24 + 2 + 9 = 35$ 位。
- 控制存储器容量：$363 \times 35 = 12705$ 位 $= 12705/8 \approx 1588.1$ B，取整为 **1589 B**（工程上按整数个存储字，即 363 字 $\times$ 35 位）。

微程序计算题的三个陷阱

- 公共取指微程序只算一次，不要按指令条数重复计算。
- 下地址字段位数由「微指令总条数」决定，不是由机器指令条数决定。先算总条数再取对数。
- 单位：题目问「多少位」就不要答字节；问「多少字节」时要注意是否进位取整。

五、异常和中断机制

（一）异常和中断的基本概念与区别

异常（**exception**），又称内中断：CPU 在执行当前指令的过程中，由 CPU 内部检测到的、与当前指令同步的意外事件。

中断（**interrupt**），又称外中断：CPU 外部的设备或部件提出的、与当前指令异步的服务请求。

对比项	内部异常（**exception**）	外部中断（**interrupt**）
来源	CPU 内部，由当前指令或其执行过程引起	CPU 外部设备或部件（定时器、键盘、网卡……）
与当前指令的关系	同步：由当前指令引起	异步：与当前指令无关
检测时机	在指令执行过程中实时检测	在一条指令执行结束后采样中断请求引脚
能否屏蔽	不能用中断屏蔽位屏蔽	分为可屏蔽中断与不可屏蔽中断
典型例子	除数为 0、非法操作码、缺页、越权访问、自陷、溢出	键盘输入、定时器到时、网络数据包到达、打印机缺纸

判断外部中断的唯一标准

问自己一句：「这个事件是不是由当前这条指令同步引起的?」

- 「键盘输入」→ 外部中断；「除数为 0」→ 内部异常。
- 「访存缺页」→ 发生在访存指令的地址转换过程中，属内部故障，不是外部中断。
- 「浮点运算下溢」→ 由当前浮点指令引起，属内部异常。

不要用「事件发生在内存里还是设备外」来判断——缺页发生在内存管理部件里，但它仍是内部异常。

为什么外部中断必须在指令结束后才能检测? 若在指令执行中途响应中断，指令被部分执行，CPU 无法保存一个「精确且可恢复」的中间状态；而异常发生的时刻和引发它的指令是确定的，处理完后可以准确地重新执行或跳过该指令，所以异常可以在指令执行过程中检测并响应。

（二）异常和中断的分类

*1.*内部异常的三类

类型	英文	含义	指令完成情况	返回位置	典型例子
故障	fault	指令执行中发现的、可能被修复的错误	指令尚未成功完成	返回本条指令重新执行	缺页、缺段、访问权限违例、某些对齐错误
自陷	trap	程序主动执行陷阱指令（如 `INT n`、`syscall`）引发	指令已经完成	返回下一条指令	系统调用、断点、单步跟踪、调试
终止	abort	严重且通常不可恢复的硬件错误	发生严重错误，程序状态不可靠	一般不返回原程序	硬件故障、总线错误、电源故障

三类异常的分辨要点

- 自陷（**trap**）属于内部异常，不是外部中断。「自陷是通过陷阱指令预先设定的一类外部中断事件」是错误说法——它与当前指令同步。
- 不能说「内部异常处理后都返回发生异常的指令继续执行」。fault 返回原指令重执行，trap 返回下一条指令，abort 不返回。三者返回位置不同。
- 自陷可在用户态由程序主动执行陷阱指令触发（如 `INT n` 产生软中断），转而进入内核态执行相应处理程序——这是「用户态 → 内核态」的一条正规途径。

*2.*外部中断的两类

- 可屏蔽中断（**maskable interrupt**，`INTR`）：主要来自 I/O 设备。CPU 可通过在中断控制器中设置屏蔽字来屏蔽或不屏蔽它；被屏蔽的中断请求信号不会送到 CPU。一般外设中断都属于可屏蔽中断。
- 不可屏蔽中断（**non-maskable interrupt**，`NMI`）：通常是非常紧急的硬件故障，如电源掉电、内存校验错、硬件看门狗超时、时钟中断、硬件线路故障。它通过专用 `NMI` 请求线提出，不受中断允许标志 **IF** 和中断屏蔽字控制，任何情况下都会被送到 CPU。

不可屏蔽中断（NMI）的四条结论

1. **CPU** 处于关中断状态（**IF=0**）时，仍能响应 **NMI** 请求。
2. **NMI** 的优先级高于所有可屏蔽中断，通常仅低于硬件复位。
3. NMI 同样需要保存断点、同样需要通过中断向量表（或 IDT）寻址服务程序（x86 中向量号固定为 2）。所谓「NMI 没有中断响应周期」是错误说法。
4. 响应 NMI 时 CPU 会自动屏蔽可屏蔽中断（等效于隐式 `CLI`），但不屏蔽 **NMI** 自身，即 NMI 可以嵌套；这种屏蔽持续到执行中断返回指令为止。

统考中若题目未提 NMI，外部中断一律按可屏蔽中断处理；讨论「多重中断」时也只针对可屏蔽中断。

中断优先级的两个层次（务必区分）：

- 响应优先级（中断判优）：由中断查询程序或硬件判优电路决定的优先级，反映「多个中断同时请求时先响应谁」。它由硬件连线或固定逻辑决定，软件不易改动。经验顺序为：不可屏蔽中断 > 内部异常 > 可屏蔽中断；内部异常中，硬件故障 > 软件中断；I/O 设备按速度高低排序，速度高的优先级高（因为速度高的设备数据容易丢失）。
- 处理优先级（中断屏蔽）：由各自的中断屏蔽字动态设定，反映「正在处理某个中断时，允许谁打断它」。可以用中断屏蔽字改变的是处理优先级，不是响应优先级。

（三）异常和中断的检测与响应

可屏蔽中断的响应条件（三条同时满足）：

1. 有中断请求（中断请求信号有效，且该请求未被屏蔽）；
2. CPU 处于开中断状态（IF=1）；
3. 当前指令执行结束，且没有更高优先级的请求正在处理。

「请求有效」不等于「立即响应」

「一旦可屏蔽中断请求信号有效，CPU 将立即响应」是错误的。中断响应是「异步但不立即」的：即使信号有效，若 IF=0、或当前指令尚未执行完、或有更高优先级事件正在处理，CPU 都不会响应。

响应时机与流程：CPU 在每条指令执行结束、取下一条指令之前，采样中断请求引脚（如 `INTR`）。若发现有效且满足响应条件，则进入中断响应过程。中断控制器根据各设备的中断请求和屏蔽情

况，结合中断响应优先级，识别出当前请求的中断类型号，并通过数据总线送给 CPU。

中断隐指令：CPU 在中断响应过程中由硬件自动执行的一系列操作。它不是一条真实的机器指令（在指令系统中找不到它，也无操作码），但功能上等价于一条指令，称为「隐指令」。其内容为：

1. 关中断（将中断允许触发器清 0），防止在保存现场时被新的可屏蔽中断打断；
2. 保存断点——保存 PC（有的机器还同时保存 PSW）到栈或专用寄存器（如 MIPS 的 EPC）；
3. 引出中断服务程序——根据中断类型号从中断向量表取出中断向量（中断服务程序的入口地址）送 PC。

硬软件分工的一句话

硬件负责：关中断、保护断点、引出服务程序（中断隐指令）软件负责：保护现场（保存通用寄存器）、设置新屏蔽字、中断事件处理、恢复现场、开中断、中断返回

中断向量与中断向量表：中断向量是中断服务程序的入口地址（有的体系结构中还包含初始 PSW）；中断向量表是存放所有中断向量的表格，位于内存固定区域，以中断类型号为索引。「中断类型号 → 查表 → 中断向量 → 送 PC」是硬件完成的寻址过程。

断点保存位置：为支持嵌套处理，多数处理器把断点保存在栈中（如 80x86）；若不支持多重中断嵌套，可以把断点保存在专用寄存器中（如 MIPS 用 EPC 存放断点）。后者开销小，因为访问寄存器比访问栈（实际是访问主存）快。

单重中断的中断服务程序执行顺序（因为题目问的是「服务程序内」，硬件已在隐指令阶段完成关中断与保存断点）：

```
保护现场 -> 中断事件处理 -> 恢复现场 -> 开中断 -> 中断返回
```

支持多重中断（中断嵌套）时的完整顺序为：

```
关中断（隐指令）-> 保存断点（隐指令）-> 引出服务程序（隐指令）
-> 保护现场和屏蔽字 -> 设置新屏蔽字 -> 开中断 -> 中断事件处理
-> 关中断 -> 恢复现场和屏蔽字 -> 开中断 -> 中断返回
```

断点与现场不是一回事

- 保护断点：保存 PC（及 PSW），由硬件（中断隐指令）完成。
- 保护现场：保存通用寄存器等用户可见工作寄存器的内容，由中断服务程序（软件/操作系统）完成。

  因此「由操作系统保存的」是通用寄存器的内容，不是 PC；「中断隐指令完成的」不包括「保存通用寄存器」。

（四）多重中断与中断屏蔽

单重中断：CPU 在执行中断服务程序过程中，对新的中断请求一律不予响应。多重中断（中断嵌套）：CPU 暂停现行中断服务程序，转去处理新的更高优先级的中断请求。

CPU 要具备多重中断功能，必须满足两个条件：

1. 在中断服务程序中提前设置开中断指令（在保护现场之后开中断）；
2. 优先级别高的中断源有权中断优先级别低的中断源。

中断屏蔽技术：每个中断源配一个屏蔽触发器，所有屏蔽触发器组合成一个屏蔽字寄存器，其内容称为屏蔽字。CPU 可通过执行相应的 I/O 指令修改屏蔽字的内容，从而动态改变中断处理的先后次序。

屏蔽字的约定与规律：

- 一般约定 **1** 表示屏蔽（禁止），**0** 表示开放（允许申请）。
- 每个中断源对应一个屏蔽字——即处理该中断源的中断服务程序运行期间，屏蔽字寄存器中的内容。
- 屏蔽字中 **1** 越多，该中断的处理优先级越高。
- 每个屏蔽字中至少要有 **1** 个 **1**（至少要能屏蔽自身，防止同一中断源反复打断自己）。
- 屏蔽字中各位的顺序要与中断源一一对应（题目通常会说明第几位对应哪个中断源）。

「改变中断处理次序」的屏蔽字构造方法（可直接照做）：

1. 把中断源按题目要求的处理优先级从高到低排成一队，例如 $A > B > C > D$。
2. 写出第 X 个中断源的屏蔽字时：自身位填 **1**；处理优先级比它低的所有中断源，对应位填 **1**；处理优先级比它高的所有中断源，对应位填 **0**。
3. 换个说法：某中断源 S 的屏蔽字中，凡是「处理优先级不高于 S 」的中断源（含 S 自己）对应的位都是 **1**。

完整算例：按改变后的处理次序写出屏蔽字

题设：某中断系统有 4 个中断源 `1#`、`2#`、`3#`、`4#`，其中断响应优先级为 $1\# > 2\# > 3\# > 4\#$。现要求把中断处理优先级改成 $1\# > 4\# > 3\# > 2\#$。设屏蔽字中一位对应一个中断源，位序自左至右为 1#2#3#4#，**1** 表示屏蔽，**0** 表示开放。求各中断源对应的屏蔽字。

解：按处理优先级 $1\# > 4\# > 3\# > 2\#$，逐个来写。

- `1#` 的处理优先级最高，它不仅屏蔽自身，还必须能屏蔽所有处理优先级比它低的 $4\#, 3\#, 2\#$：$\Rightarrow$ **1111**。
- `4#` 处理优先级第二。比它高的只有 `1#`（对应位为 0），它屏蔽自身及更低的 $3\#, 2\#$（对应位为 1）：$\Rightarrow$ **0111**。
- `3#` 处理优先级第三。比它高的有 `1#`、`4#`（两位为 0），它与更低的 `2#` 为 1：$\Rightarrow$ **0011**。
- `2#` 处理优先级最低。比它高的有 `1#,4#,3#`（三位为 0），只有自身为 1：$\Rightarrow$ **0001**。

验算：每个屏蔽字中 1 的个数依次是 4、3、2、1，与处理优先级从高到低严格对应，符合「1 越多优先级越高」的规律；且每个屏蔽字都至少含一个 1（都不为 0），满足「必须能屏蔽自身」的要求。

接着算「**CPU** 完成各中断服务程序的过程」：设 CPU 执行主程序时，`1#`、`3#`、`4#` 同时发生请求，随后在执行 `3#` 的中断服务程序过程中，`2#` 又提出请求。

- 主程序：三请求同时到达，按响应优先级 $1\# > 3\# > 4\#$ 选 `1#` 先响应。运行 `1#` 时屏蔽字为 1111，其余三者全被屏蔽。
- `1#` 服务程序结束 $\rightarrow$ 恢复旧屏蔽字，剩下 `3#`、`4#` 待响应，按响应优先级选 `3#`。运行 `3#` 时屏蔽字为 0011，`4#`、`2#` 都被屏蔽。
- 此时 `2#` 提出请求但被 0011 屏蔽，不能打断 `3#`。
- `3#` 服务程序结束 $\rightarrow$ 恢复旧屏蔽字，此时 `2#`、`4#` 都已请求且未被屏蔽；按处理优先级 $4\# > 2\#$（`2#` 的屏蔽字 0001 允许 `4#` 打断），标为「`3#`$\rightarrow$ 恢复点」后先响应 `4#`。
- `4#` 运行期间屏蔽字 0111，`2#` 被屏蔽，不能嵌套。
- `4#` 结束 $\rightarrow$ 再按处理优先级响应 `2#`，`2#` 运行期间屏蔽字 0001。
- `2#` 结束 $\rightarrow$ 返回到最早被挂起的 `3#` 服务程序的断点处继续执行剩余部分。
- `3#` 结束 $\rightarrow$ 返回主程序。

最终处理先后次序为：`1#` 全部 $\rightarrow$ `3#` 前半 $\rightarrow$ `4#` 全部 $\rightarrow$ `2#` 全部 $\rightarrow$ `3#` 后半 $\rightarrow$ 主程序。

若把处理优先级改成 1# > 2# > 3# > 4#（与原响应优先级一致），则屏蔽字变为：

中断源	处理优先级 **1#>2#>3#>4#** 时的屏蔽字	处理优先级 **1#>4#>3#>2#** 时的屏蔽字
1#	1111	1111
2#	0111	0001
3#	0011	0011
4#	0001	0111

对照可见：只有 **2#** 与 **4#** 的屏蔽字互换，正是这一次互换把两者的处理次序颠倒过来。这也说明——不改屏蔽字时处理优先级就等于响应优先级；改屏蔽字只能改变处理次序，不能改变响应优先级本身。

多重中断的四条易错点

- 「中断处理期间 **CPU** 一直处于关中断状态」是错的。那是单重中断；多重中断在保护现场之后要开中断。
- 「中断屏蔽字用于确定中断响应的优先级」是错的。屏蔽字改变的是处理优先级（能否被打断），响应优先级由硬件判优电路固定决定。
- 「只有在用户态才能检测和响应中断」是错的。内核态同样可以响应中断，这正是中断嵌套的机制。
- 每个屏蔽字必须至少有一个 **1**（屏蔽自身），写屏蔽字题时若某行为全 0，一定是错的。

六、指令流水线

（一）指令流水线的基本概念

指令流水线（**instruction pipeline**）：把一条指令的执行过程划分为 k 个功能段（流水段），让多条指令在不同功能段中同时工作、重叠执行，从而提高单位时间内完成的指令数。它利用的是时间并行性。

经典五段划分（MIPS 风格）：

流水段	缩写	主要工作	主要功能部件
取指	IF	按 PC 从指令存储器取指令，同时计算 PC+4	指令存储器 IM、PC、加法器、MUX
译码/取数	ID	译码，按 Rs、Rt 从寄存器堆读出操作数	寄存器堆读口、控制器
执行	EX	ALU 运算、计算访存有效地址、分支比较与目标地址	ALU、扩展器、加法器
访存	MEM	读/写数据存储器；分支指令在此更新 PC	数据存储器 DM
写回	WB	把结果写回寄存器堆	寄存器堆写口、MUX

流水寄存器（流水段寄存器）：在相邻流水段之间插入的状态元件，保存上一段的中间结果供下一段使用。它们的作用是把各段的工作隔离开，使得不同指令可以同时在各自的功能段中工作。

- **IF/ID**：保存取出的指令和 **PC+4**（分支指令后面要用 PC+4 计算相对转移地址）。
- **ID/EX**：保存 $R[R_s]$、$R[R_t]$、R_t、R_d、$imm16$、$func$、**PC+4** 以及译码产生的控制信号。指令本身不必再保存（需要的信息已经提取出来）。
- **EX/MEM**：保存 ALU 结果（或有效地址）、Zero 标志、Overflow 标志、要写回的目的寄存器号、要存入 DM 的数据 $R[R_t]$、以及继续往后的控制信号。

- **MEM/WB**：保存从 DM 读出的数据与 ALU 结果、目的寄存器号、以及写回所需的控制信号。

为什么指令系统要在意规整性（有利于流水的指令系统特点）：指令格式规整且长度一致（如 MIPS 32 位定长，PC+4 计算方便）、指令和数据按边界对齐存放、只有 **Load/Store** 指令才能访问存储器、源寄存器位置固定（指令未完成译码即可开始读寄存器）。这四点使各段的行为统一、延迟稳定，便于流水。

流水线设计的两个原则：流水段个数以最复杂指令所用的功能段个数为准；流水段长度以最复杂的操作所花时间（关键路径）为准。这也解释了为什么流水线会「延长单条指令的执行时间」——所有指令都要按最慢指令的节奏走，还要加上流水段寄存器的开销。

二阶段流水线为什么不能成倍提速

① 执行阶段因各指令耗时不同，必须以最慢者为准，故执行阶段很长；② 为统一节拍，取指阶段也要拉长到与最慢执行阶段同长；③ 流水段寄存器的读写增加额外延迟；④ 各种冒险会引入阻塞。所以流水线深度不能无限增加，当段间缓冲开销占比过大时再加深就没有意义。

（二）指令流水线的性能计算

设流水线有 k 段，每段执行时间为 Δt，连续执行 n 条指令。

时钟周期：若忽略流水段寄存器延时，则 $T_{clk} \geq \max(t_1, t_2, \ldots, t_k)$；若段间寄存器延时为 t_r，则 $T_{clk} \geq \max(t_1, \ldots, t_k) + t_r$。

总时间：不留空隙时，第一条指令用 $k\Delta t$，此后每个 Δt 流出 1 条结果，共流出 $n-1$ 条：

$$T = k\Delta t + (n-1)\Delta t = (k+n-1)\Delta t$$

若无阻塞时的理想周期数为 $k+n-1$，而实际额外插入了 s 个阻塞周期，则 $T = (k+n-1+s)\Delta t$。

吞吐率（单位时间完成的指令数）：

$$TP = \frac{n}{T} = \frac{n}{(k+n-1)\Delta t}$$

当 $n \to \infty$ 时，理想最大吞吐率 $TP_{\max} = \dfrac{1}{\Delta t}$，即流水线充满后每周期完成一条指令。

加速比（相对非流水线串行执行；串行执行 n 条指令需 $nk\Delta t$）：

$$S = \frac{nk\Delta t}{(k+n-1)\Delta t} = \frac{nk}{k+n-1}$$

当 $n \to \infty$ 时 $S \to k$，即理想最大加速比等于流水段数 k。

效率（流水线设备的实际利用率。k 个段共 $k \cdot T$ 的时空区，完成 n 条指令占用 nk 个「段 · 周期」）：

$$E = \frac{nk}{k(k+n-1)} = \frac{n}{k+n-1}, \qquad E = \frac{S}{k}$$

完整算例：五段流水线的四项指标

题设：某处理器采用 5 段指令流水线，每段执行时间为 $\Delta t = 2$ ns，段间流水寄存器延时已计入这 2 ns 内。连续执行 100 条指令，执行过程中无任何阻塞。求总时间、吞吐率、加速比、效率。

- 总周期数：$k+n-1 = 5+100-1 = 104$ 个。
- 总时间：$T = 104 \times 2\ \text{ns} = 208\ \text{ns}$。

- 吞吐率：$TP = \dfrac{100}{208\ \text{ns}} \approx 0.4808 \times 10^9$ 条/秒 $\approx 4.81 \times 10^8$ 条/秒。
- 理想最大吞吐率：$\dfrac{1}{2\ \text{ns}} = 5 \times 10^8$ 条/秒（因为 $n = 100$ 有限，实际吞吐率略低于它）。
- 加速比：$S = \dfrac{nk}{k+n-1} = \dfrac{100 \times 5}{104} \approx 4.81$（理想上限为 $k = 5$）。
- 效率：$E = \dfrac{n}{k+n-1} = \dfrac{100}{104} \approx 96.2\%$，验证 $E = S/k = 4.81/5 \approx 96.2\%$。

变式：若每条指令之间都额外插入 1 个阻塞周期（$s = 99$），则周期数变为 $104 + 99 = 203$，$T = 406$ ns，$TP \approx 2.46 \times 10^8$ 条/秒，$S \approx 2.46$。

性能计算的四个陷阱

- 单位：Δt 常用 ns、ps，吞吐率是「条/秒」，别把 $1/\Delta t$（s^{-1}）与「条/秒」混算但忘记换算 10^{-9}。
- 理想最大加速比是 k，但有限 n 条指令时的实际加速比一定小于 k（因为流水线要填充，有 $k-1$ 个周期是「装填开销」）。
- n 很大时才能近似用 $TP_{\max} = 1/\Delta t$ 与 $S = k$。选择题常给「某机主频 1.03 GHz、4 级流水、执行 100 条指令」这类小数据，必须老老实实算 $k + n - 1$。
- 理想 **CPI**：单周期 **= 1**，基本流水线 **= 1**，超流水线 **= 1**，多周期 **> 1**，超标量 **< 1**。别把「超流水线 CPI<1」写对（那是错的，超流水线不改变 CPI，它靠提高主频提速）。

（三）结构冒险、数据冒险和控制冒险的处理

冒险（**hazard**）是指使流水线不能在预定的时钟周期内继续推进（需要阻塞）的情形。

*1.*结构冒险（*structural hazard*）

成因：多条指令在同一时钟周期争用同一硬件资源。典型情形：

- 单存储器时，Load 指令在 MEM 段取数据，同时 IF 段要取指令，两者争用同一个存储器；
- 寄存器堆只有单端口时，ID 段读寄存器与 WB 段写寄存器冲突；
- ALU 未流水化时，多条指令同时要用 ALU。

处理方法：

1. 规整功能段的划分原则：每个功能部件在每条指令中只使用一次，且只能在特定的流水段中使用（例如 ALU 只在 EX 段用、寄存器写口只在 WB 段用）。
2. 增加或分离资源：把寄存器堆的读口和写口独立开来（利用时钟上升沿写、下降沿读，使前半周期写、后半周期读）；把指令存储器 **IM** 与数据存储器 **DM** 分开——这正是指令 **Cache** 与数据 **Cache** 分离的主要目的：减少指令流水线的资源冲突，而不是为了提高命中率。
3. 阻塞（**stall**）：资源不足时让后到的指令等待。代价是流水线出现气泡。

*2.*数据冒险（*data hazard*）

成因：后面指令要用前面指令的结果，但该结果尚未产生。

三类数据相关：

类型	全称	含义	基本按序流水线中是否出现
RAW	写后读（真相关）	前指令写、后指令读同一位置	会，是必须处理的类型

类型	全称	含义	基本按序流水线中是否出现
WAR	读后写（反相关）	前指令读、后指令写同一位置	不会（按序执行不颠倒次序）
WAW	写后写（输出相关）	两条指令写同一位置	不会（按序写回不颠倒次序）

在按序发射、按序执行、按序写回的经典五段流水线中只可能出现 RAW；WAR 与 WAW 只会在乱序执行、多发射的处理器中成为真正的冒险。

处理方法：

1. 转发 / 旁路（**forwarding / bypassing**）：流水段寄存器中已经有了后续指令需要的值，通过增加 MUX 把 EX/MEM 或 MEM/WB 寄存器中的结果直接送回 ALU 输入端，无需等写回。转发是解决冒险的手段，本身不会引起阻塞。
2. 暂停、插入气泡（**stall / bubble**）：硬件检测到相关时冻结 PC 和 IF/ID 寄存器，让后续指令重复一次或插入 nop，直到数据可用。代价是 s 个阻塞周期。
3. 编译调度（软件方法）：由编译器把无关指令调到相关指令之间填充空隙；若无合适指令则插入 `nop`。

Load-use 冒险：转发解决不了的那一类

`load` 指令的数据在 **MEM** 段末才从 DM 读出，而紧随其后的指令在同一周期就要在 **EX** 段使用它，时间上来不及，因此即使有转发也必须至少阻塞 **1** 个周期。这就是典型的「装入—使用数据冒险（load-use data hazard）」。

结论：转发不能解决所有数据冒险，所以「所有数据冒险都可以通过加入转发电路解决」是错误的。与之相对，「所有数据冒险都能通过调整指令顺序和插入 nop 解决」是正确的（前提是保持程序语义）。

相关指令对的快速判定法（用于「下列指令对中不存在数据冒险的是」这类题）：把每条指令写成「写哪些寄存器、读哪些寄存器」，若「前一条写」与「后一条读」有交集，则为 RAW 真相关；若后一条只是写了同一寄存器而并不读它，则不构成 RAW。相邻的两条指令若只共享「被写」的寄存器而不存在「先写后读」关系，就不是数据冒险。此外，两条指令访问同一主存地址但处于基本按序流水线中时不构成冒险（较早的 load 必然先于较晚的 store 到 MEM 段，访存次序不会颠倒）。

*3.*控制冒险（*control hazard*）

成因：遇到会改变指令执行顺序的情形（各类转移指令、调用与返回指令、异常和中断），PC 的更新值来不及在取下一条指令之前准备好，导致流水线中已取出的若干条指令是错的。

延迟损失时间片 C：由于流水线阻塞而带来的延迟执行周期数。它由「分支结果和目标地址在第几段才确定」决定——越早算出并送回 **PC**，C 越小。在经典数据通路中分支在 MEM 段才更新 PC，$C=3$（要丢弃/填充 3 个周期）；若在 ID 段就完成译码并送回转移目标地址，$C=1$；若在 IF 段就能算出目标地址（如 `j` 指令按 PC+4 高 4 位与 26 位立即数拼接），则 $C=0$。

处理方法：

1. 阻塞 / 插入气泡或 **nop**：在分支指令后插入 C 个气泡或 C 条 `nop`。效率极低，一般不单独采用。
2. 分支预测：
   - 静态预测：总是预测「条件不满足（not taken）」，即继续执行顺序指令。若转移概率约 50%，正确率也只有约 50%。
   - 动态预测：根据程序执行的历史动态调整预测，正确率较高，现代处理器几乎都采用。核

心结构是分支历史记录表 **BHT**（记录分支的历史方向）与分支目标缓冲 **BTB**（缓存分支目标地址，使 IF 段就能得到目标地址），每个表项用分支指令地址的低位作索引，因此在 IF 阶段就能取到预测位，从而不必等分支结果算出。

3. 延迟槽 / 分支延迟时间片的调度（静态调度）：由编译程序把分支指令前面与分支无关的指令调到分支指令后面执行，用来填充延迟时间片（称为分支延迟槽，**branch delay slot**），不够时用 nop 填充。
4. 预取两个方向的目标指令、加快并提前形成条件码、提高转移方向的猜准率。

真题考法

- 「不会引起流水线阻塞的是」——数据旁路（转发）。数据相关、条件转移、资源冲突都会引起阻塞。
- 「采用指令 Cache 与数据 Cache 分离的主要目的」——减少指令流水线的资源冲突（结构冒险），不是提高命中率，也不是降低缺失损失。
- 「无转发机制的五段流水线中，存在数据冒险的指令对是」——按上文的「写/读集合」判定，且注意没有转发时，距离 **2** 条以内的 **RAW** 都需要阻塞。
- 「采用转发处理数据冒险、硬件阻塞处理控制冒险时，哪些指令发生阻塞」——关键是识别出 **load-use** 必须阻塞 **1** 拍，以及分支指令之后的指令必须等待分支结果。

（四）超标量和动态流水线的基本概念

指令级并行（**Instruction Level Parallelism**，**ILP**）：单一指令流内部存在的、可同时执行的指令之间的并行性。提高 ILP 有两条基本途径：增加流水线级数（超流水线）和在一个周期内发射多条指令（多发射）。

超流水线（**super-pipelining**）：通过增加流水线级数，把每段的工作量切得更小，从而缩短时钟周期、提高主频。它不改变 **CPI**（理想仍为 **1**），靠主频提速；代价是流水段寄存器开销增大、分支预测失败的代价更大、气泡的负面影响更明显。

超标量（**superscalar**）：处理器内部设置多个独立的功能部件，使得一个时钟周期内可以同时发射并执行多条指令，理想 CPI <1（即 IPC>1）。它常与动态调度技术结合以提高指令执行的并行性。要注意：超标量的本质是「一个周期发射多条指令」，它并不缩短流水线功能段的处理时间（那是加深流水线或提频要做的事）。

多发射的两种实现方式：

比较项	静态多发射（超长指令字 **VLIW**）	动态多发射（超标量）
指令打包由谁决定	编译器在编译时完成（生成超长指令字，「发射包」）	硬件在执行时动态决定
冒险处理	完全或主要由编译器通过代码调度、插入 nop、静态分支预测消除	硬件检测数据冒险并阻塞，结合动态分支预测
可移植性	差：编译结果与具体处理器结构紧密相关，换机器往往要重新编译	强：编译后的代码可在不同超标量处理器上运行
硬件复杂度	简单（把最难的调度交给编译器）	复杂

推测（**speculation**）：由编译器或处理器猜测指令的执行结果，并据此调整指令执行顺序，使指令执行达到最大可能的并行。它依赖于「指令间相关性分析」和「分支方向预测」，一旦推测错误必须有检测与回滚机制，因此推测错误会增加额外开销。

动态调度与乱序执行：动态流水线的重要思想是「在等待解决阻塞时，到后面找指令提前执行」。

其通用模型为：

- 一个指令预取和译码单元：有序发射；
- 多个并列执行的功能单元：乱序执行；
- 一个提交单元：有序提交（保证异常精确、程序状态正确）。

为什么有了编译器调度还需要硬件动态调度：并非所有阻塞都能事先确定（如 **Cache** 缺失就是不可预见的阻塞）；动态分支预测必须依据真实执行情况；动态调度还能把处理器细节屏蔽起来，使同一份编译结果在不同处理器上都可用。

寄存器重命名（**register renaming**）：把指令中引用的体系结构寄存器（逻辑寄存器）映射到一组物理寄存器上，从而消除 **WAR** 与 **WAW** 这类「假相关」，只保留真正的 RAW 相关，让乱序执行能挖掘更多并行性。

推测执行（**speculative execution**）：在分支结果未确定时，按预测方向继续取指并执行，把结果暂时保存在不被提交的位置；预测正确则提交，预测错误则丢弃并回滚。

超标量的三条易错说法

- 「超标量能缩短流水线功能段的处理时间」——错。
- 「超流水线的 CPI 小于 1」——错，超流水线 CPI 仍为 1。
- 「超标量通过一个周期发射多条指令使 CPI 可以小于 1」——对。

七、多处理器基本概念

（一）Flynn 分类法与向量处理器

1966 年 Flynn 提出的分类法按指令流（**instruction stream**）数量和数据流（**data stream**）数量两个维度划分计算机体系结构：

类型	指令流	数据流	核心原理	典型硬件	并行层次
SISD	单	单	顺序处理	传统单核 CPU、微控制器	无显式并行（内部靠 ILP）
SIMD	单	多	数据级并行	GPU、CPU 向量单元（AVX/NEON）、向量处理器	数据级并行
MISD	多	单	冗余或流水线处理	容错系统（如航天飞机飞控计算机）；袁书认为现实中不存在这种计算机	—
MIMD	多	多	线程级 / 任务级并行	多核 CPU、多处理器、集群、超级计算机	线程级及以上

要点：

- **SISD**：仅包含一个处理器和一个存储器，一段时间内只执行一条指令流，每条指令最多处理两个数据（双目运算）或一个数据（单目运算）。现代单核 CPU 在宏观上仍是 SISD，但在微观上通过流水线、超标量、超流水线、**VLIW** 等技术挖掘指令级并行，从而在 SISD 框架内压榨单指令流的性能。
- **SIMD**：一个指令控制部件 + 多个处理单元 + 多个（局部）存储器，所有处于执行状态的处理单元同时执行同一条指令，但各自处理不同的数据。适合循环体向量化、图像/音频处理、图形学渲染、科学计算、神经网络等高度规整的数据级并行任务。

- **MIMD**：一定包含多个计算机或多个处理器，是目前大多数并行处理系统的处理方式。它可以同时支持数据并行（同一程序的多个副本处理不同数据，即 SPMD）和任务并行。

向量处理器（**vector processor**）属于 **SIMD**，专门面向向量型数据的并行计算。它与普通 SIMD 的差别在于：

1. 拥有专门的向量指令集：一条 `VADD` 指令即可完成两个含几十到上百个元素的向量相加，而普通 CPU 必须写循环。
2. 拥有大容量向量寄存器，一个向量寄存器存放一整个向量。
3. 计算单元是高度流水化的向量功能单元，可以每周期送入一组数据并每周期产出一组结果，吞吐率极高。
4. 拥有专门优化的 Load/Store 单元，可在主存与向量寄存器之间流式传输整块连续数据。

向量处理器多采用流水线方式（也有阵列方式，以流水线为主），主存储器容量限定了机器的解题规模，因此需要大容量集中式公共存储器，并常用多端口同时读取的交叉多模块存储器来维持高速运算所需的数据供给。

MISD 在实际中极为罕见，其价值不在加速计算而在提高可靠性与容错：用多个处理器以不同的算法/实现处理同一个关键数据流，再通过表决机制比较输出，从而屏蔽单机的硬件瞬时故障或软件缺陷。

（二）硬件多线程

问题背景：现代处理器经常要等待较慢的存储器。当遇到 Cache 缺失或长流水线暂停时，处理器进入停顿（**stall**），成百上千个时钟周期被浪费。

硬件多线程（**hardware multithreading**）：允许单个物理处理器核心同时管理多个线程的状态，操作系统会把这一个物理核心看成多个「逻辑核心」。当一个线程因停顿时无法继续时，核心快速切换到另一个已就绪的线程，用原本会被浪费的周期执行它的指令。其核心思想是用线程级并行（**TLP**）掩盖延迟和停顿，提高核心资源利用率与整体吞吐量。

硬件上的改造：复制线程状态（为每个线程提供独立的 PC、通用寄存器等）；共享执行资源（ALU、FPU、Cache、流水线等由所有线程共享）。

特性	细粒度多线程 **FGMT**	粗粒度多线程 **CGMT**	同时多线程 **SMT**
线程切换时机	每个时钟周期都切换	只在发生高成本长停顿（如 L2/L3 缺失）时切换	不切换，同一周期执行多个线程的指令
是否清空流水线	不涉及	需要清空流水线，切换开销较大	—
主要优点	能掩盖所有类型的延迟	单线程性能几乎不受影响，实现相对简单	资源利用率与吞吐量最大化
主要缺点	降低单线程执行速度，必须轮流分享执行周期	无法掩盖短延迟（数据依赖、分支预测失败等）	硬件设计最复杂，存在资源竞争与安全问题
典型实现	—	—	Intel 超线程（HT）、AMD SMT

硬件多线程的三条易错点

- 硬件多线程不只用于多核处理器。它是核心内部的技术，单核处理器同样可以采用。所以「硬件多线程技术只可用于多核处理器」是错的。
- **SMT** 下操作系统看到的是多个逻辑核心（如 4 核 8 线程的 CPU，OS 认为有 8 个可用处理器），由操作系统把软件线程调度到这些逻辑核心上，硬件再在微观层面执行。
- 细粒度多线程与粗粒度多线程不能混淆：「每个时钟周期轮流」是细粒度；「只在长停顿时才切换」是粗粒度。

（三）多核处理器

多核（**multi-core**）处理器：把多个处理器核集成在同一个芯片上。每个核有自己独立的控制部件、运算部件、寄存器和私有 Cache（L1，通常还有 L2），多个核共享片上的 L3 Cache、内存控制器等资源，并通过片上互连网络通信。

- 多核处理器中，各核可以执行不同的指令流、处理不同的数据，因此多核处理器属于 **MIMD** 结构。
- 多核的并行主要是线程级并行，需要操作系统的调度与并行编程配合；而流水线、超标量挖掘的是指令级并行，对软件透明。
- 多核相比多处理器系统（多个独立芯片）的优势：通信延迟低、带宽高、共享资源、功耗与体积更优。
- 多核带来的新问题：**Cache** 一致性、共享资源的竞争与同步、以及编程模型上对任务分解与负载均衡的要求。

（四）共享内存多处理器（SMP）与 UMA/NUMA、Cache 一致性问题

共享内存多处理器：多个处理器共享同一物理主存，通过互连网络（总线、交叉开关、网络）连接。

对称多处理器（**SMP**）：将多个相同类型的 CPU 通过总线互连，并以等同地位共享系统所有资源，多个 CPU 对称工作，无主次或从属关系，共享单一物理地址空间。因为每个 CPU 访问内存中任何地址所需时间相同，**SMP** 就是一种 **UMA** 结构。

按「处理器访问存储单元的时间是否一致」划分：

结构	全称	特征	说明
UMA	一致性内存访问	所有处理器访问任何存储单元的时间一致	也就是 SMP。所有 CPU 通过共享总线/交叉开关连到主存。实现简单，但 CPU 个数多时会产生访存冲突、总线成为瓶颈，可扩展性受限
NUMA	非一致性内存访问	处理器对不同存储单元的访问时间可能不一致，取决于位置：访问本地存储器快，访问其他处理器的存储器慢	把物理内存分布到各处理器附近形成「节点」。所有共享存储模块统一编址、形成单一地址空间，非本地内存也用 Load/Store 访问，因此 NUMA 上运行 UMA 程序无需改动，但同主频下性能不如 UMA

NUMA 的两个细分形态：

- **NC-NUMA**（No Cache NUMA）：处理器不带高速缓存。
- **CC-NUMA**（Cache-Coherent NUMA）：处理器带一致性高速缓存，必须解决 Cache 一致性，最常见的是基于目录（**directory**）的 **Cache** 一致性机制——每个存储器中有一个存放目录信息的

存储区，每个主存块对应一个目录项，目录项中有有效位等字段，记录该主存块与各 Cache 行的对应关系。

Cache 一致性问题：在共享内存多处理器中，同一主存块可能同时驻留在多个处理器的私有 Cache 中。若某处理器修改了自己 Cache 中的副本，而其他处理器 Cache 中的旧副本仍然有效并被读取，就会读到过期数据。这就是 **Cache 一致性（cache coherence）**问题。

解决思路：

- 监听 / 总线侦听（**snooping**）协议：各 Cache 通过共享总线监听其他处理器发出的访存事务；某个 Cache 中的副本被修改时，将修改信息广播到所有核的 Cache，若其他核的 Cache 中有该数据则将其置为无效，同时把主存中的数据同步更新。**MESI** 协议就是这类协议的典型代表（每个 Cache 行处于 M/E/S/I 四种状态之一）。
- 基于目录的协议：适用于互连网络不是共享总线、处理器数量较多的情形（如 CC-NUMA），由目录记录每个主存块被哪些 Cache 缓存、是否被修改，从而只向相关节点发送一致性维护消息，避免广播。

多计算机系统（**multicomputer**）与多处理器系统的区别：多计算机的每个节点有自己私有的地址空间，「共享内存」是靠消息传递在软件层模拟的。因为没有共享内存总线瓶颈和 Cache 一致性问题，多计算机系统具有极高的可扩展性，可扩展到成千上万个节点，当代顶级超级计算机几乎都采用这种架构。

真题考法

「下列关于并行处理技术的叙述中，不正确的是」——注意四个判据：多核处理器属于 **MIMD**（对）；向量处理器属于 **SIMD**（对）；硬件多线程只能用于多核（错，单核也可）；**SMP** 中所有处理器共享单一物理地址空间（对）。

本章小结表

主题	核心公式 / 结论	关键判据
寄存器位数	$W_{MAR} = W_{PC} =$ 地址位数；$W_{IR} =$ 指令字长；$W_{MDR} =$ 存储字长；$W_{GPR} =$ 机器字长	「至少」类问题：按边界对齐可省掉地址低位（PC 30 位 vs 地址 32 位）
寄存器可见性	PC、GPR、PSW 可见；MAR、MDR、IR、暂存器、μIR、CM 不可见	能否通过指令语义观察/改变
周期层级	指令周期 ⊃ 机器周期 ⊃ 时钟周期；机器周期以主存工作周期为基准	$T_{\text{指令}} \geq T_{\text{机器}} \geq T_{\text{时钟}}$
取指 RTL	(PC)→MAR；M(MAR)→MDR；(PC)+1→PC；(MDR)→IR	3 个节拍（单总线）
间址 RTL	Ad(IR)→MAR；M(MAR)→MDR；(MDR)→MAR	每次间接寻址增加一次访存
中断周期 RTL	(SP)-1→SP；(SP)→MAR；1→W；(PC)→MDR；(MDR)→M(MAR)；向量地址 →PC	关中断 + 保护断点 + 引出服务程序
操作元件 vs 状态元件	ALU、MUX、加法器、扩展器 = 操作；PC、GPR、暂存器、流水寄存器 = 状态	是否记忆状态
微命令/微指令/微程序	微命令（1 信号）⊂ 微指令（同拍相容命令集）⊂ 微程序（一条机器指令）	微指令条数 = 该指令的时钟周期数

主题	核心公式 / 结论	关键判据
字段直接编码	$W=\sum_{i=1}^{m}\lceil\log_2(n_i+1)\rceil$，必须留一个「全 0 不操作」状态	7,3,12,5,6 → 3+2+4+3+3 = **15** 位
控制存储器容量	$C=N_\mu\times W_\mu$；下地址字段 $\geq\lceil\log_2 N_\mu\rceil$	32 条指令 × 4 条 + 公共 2 条 = 130 条 → 8 位
硬布线 vs 微程序	硬布线：快、难改；微程序：慢、易改、规整	见控制器对比表
异常 vs 中断	异常：同步、指令执行中检测；中断：异步、指令结束后检测	是否由当前指令同步引起
三类异常返回位置	fault 返回原指令重执行；trap 返回下一条；abort 不返回	指令是否已完成
中断隐指令	关中断、保存断点、引出中断服务程序（全由硬件完成）	通用寄存器由软件保存
屏蔽字	1 屏蔽、0 开放；自身位为 1；屏蔽所有处理优先级不高于自己的源；1 越多处理优先级越高	不改屏蔽字则处理优先级 = 响应优先级
流水线时间	$T=(k+n-1+s)\Delta t$	无阻塞时为 $k+n-1$ 个周期
流水线吞吐率	$TP=\frac{n}{T}$，理想 $TP_{\max}=\frac{1}{\Delta t}$	单位统一换算到秒
流水线加速比	$S=\frac{nk}{k+n-1}\to k$	有限 n 时 $S<k$
流水线效率	$E=\frac{n}{k+n-1}=\frac{S}{k}$	—
时钟周期	$T_{clk}\geq\max(t_i)$；有段间寄存器则为 $\max(t_i)+t_r$	2018 年真题：80+20=100 ps
三类冒险	结构（分资源/阻塞）、数据（转发/气泡/调度）、控制（预测/延迟槽/阻塞）	转发不能解决 load-use
Flynn 分类	SISD / SIMD / MISD / MIMD	多核 = MIMD；向量处理器 = SIMD；MISD 多认为不存在
硬件多线程	FGMT 每周期切换；CGMT 长停顿时切换；SMT 同周期多线程	单核也能用硬件多线程
存储模型	UMA = SMP（访问时间一致）；NUMA 本地快远程慢；CC-NUMA 用目录协议	Cache 一致性问题源于私有 Cache 的副本

易错清单

必须逐条自检的易错点

1. **MAR** 由可寻址单元个数决定，不由机器字长决定；MDR 由存储字长决定，不必然等于机器字长。
2. **PC** 在「至少多少位」的题目里可以不等于地址位数（按字边界对齐时省掉低位，4 GB 主存 + 32 位定长指令 → PC 30 位、IR 32 位）。
3. 汇编程序员可见的是 PC、通用寄存器、基址/变址寄存器、PSW；**MAR**、**MDR**、**IR**、微指令寄存器、暂存寄存器、控制存储器不可见。
4. 空操作指令的指令周期中 **PC** 和 **IR** 都会改变，不能说「任何寄存器都不改变」。
5. 机器周期通常以主存工作周期为基准，而不是以 CPU 内部运算时间为基准。
6. **ALU** 是组合逻辑（操作）元件，PSW、GPR、暂存器是时序逻辑（状态）元件；数据通路不包含控制部件，但包含（或配合）异常检测与响应电路。

7. 单周期处理器不能采用单总线结构；单周期的 CPI=1，多周期 CPI>1，基本流水线 CPI=1，超标量 CPI<1，超流水线 CPI=1。
8. 微命令与微操作一一对应；只有相容的微命令能出现在同一条微指令中；一条微指令的并行能力由操作控制字段决定。
9. 控制存储器按地址访问，不是按内容访问；CS 在 CPU 内、用 ROM、存微指令、字长 = 微指令长度。
10. 字段直接编码必须为每个字段留出「全 **0** 不操作」编码，故含 n 个互斥微命令的字段要 $\lceil \log_2(n+1) \rceil$ 位。
11. 下地址字段位数由微指令总条数决定（含公共取指微程序），不是由机器指令条数决定。
12. 公共取指微程序的微指令只计入一次，不要按指令条数重复累加。
13. 异常是同步的、在指令执行过程中检测；外部中断是异步的、在指令执行结束后检测。
14. 自陷（**trap**）是内部异常，不是外部中断；fault 返回原指令重执行，trap 返回下一条，abort 不返回。
15. 中断隐指令 = 关中断 + 保护断点 + 引出中断服务程序，由硬件完成；保存通用寄存器是软件（**OS**）的事。
16. 可屏蔽中断请求有效 $\neq$ 立即响应；必须同时满足「有未屏蔽请求 + 开中断 + 当前指令执行结束」。
17. **NMI** 在关中断状态下也能响应，且优先级高于所有可屏蔽中断；「NMI 没有中断响应周期 / 不需要保存断点」都是错的。
18. 中断屏蔽字改变的是处理优先级，不是响应优先级；每个屏蔽字必须至少含一个 **1**（屏蔽自身）；某中断源的屏蔽字中，所有「处理优先级不高于它」的源对应位均为 1。
19. 多重中断中 **CPU** 不是一直关中断：保护现场前后要按「开中断—处理—关中断」的顺序配合。
20. 转发（旁路）是解决冒险的手段，本身不引起阻塞；但 **load-use** 冒险即使有转发也必须阻塞至少 **1** 拍，所以「所有数据冒险都能靠转发解决」是错的。
21. 指令 **Cache** 与数据 **Cache** 分离的主要目的是减少流水线的结构冒险（资源冲突），不是提高命中率。
22. 基本按序流水线中只会出现 **RAW**；WAR、WAW 只在乱序/多发射中成为真正冒险，靠寄存器重命名消除。
23. 超流水线不改变 **CPI**（仍为 **1**）；超标量的本质是一周期发射多条指令，不缩短功能段处理时间。
24. 动态调度与乱序执行的模型是「有序发射、乱序执行、有序提交」，有序提交是保证异常精确与程序状态正确的关键。
25. 硬件多线程不限于多核，单核也能实现；SMT 让操作系统看到多个逻辑核心。
26. 多核处理器属于 **MIMD**，向量处理器属于 **SIMD**，**MISD** 通常认为现实中不存在。
27. **SMP = UMA**（访问任何地址时间一致）；NUMA 中访问本地存储器快于远程；**CC-NUMA** 用基于目录的协议维护 Cache 一致性。
28. **Cache** 一致性问题来自私有 Cache 中同一主存块的多个副本被分别读写；解决途径是侦听（如 MESI）或基于目录的协议。

第十三章总线和输入/输出系统

一、总线和输入/输出系统

本章考纲要求

（一）总线

1. 总线的基本概念
2. 总线的组成及性能指标
3. 总线事务和定时

（二）I/O 接口（I/O 控制器）

1. I/O 接口的功能和基本结构
2. I/O 端口及其编址

（三）I/O 方式

1. 程序查询方式
2. 程序中断方式：中断的基本概念，中断响应过程，中断处理过程，多重中断和中断屏蔽的概念
3. DMA 方式：DMA 控制器的组成，DMA 传送过程

本章是计组里「概念多、计算题密」的一章：总线带宽、查询开销、中断开销、DMA 传送时间四类计算几乎年年出现，而总线的分类、定时方式、接口寄存器、三种 I/O 方式的对比又几乎年年出选择题。学这一章的正确顺序是先抓住四组对比（串行/并行、同步/异步、统一编址/独立编址、中断/DMA），再把四个公式族练熟。下面按考纲条目顺序展开。

二、总线

（一）总线的基本概念

总线（**bus**）是连接多个部件、供各部件之间交换信息的一组共享传输线。它的两个本质特征是：

- 分时：同一时刻只允许一个部件（主设备）向总线发送信息，其他部件只能等待属于自己的时间段。
- 共享：总线上挂接的多个部件可以在不同时刻分别获得使用权；从设备可以在同一时刻共同接收总线上广播的信息。

总线是为了替代早期「分散连接」而出现的。分散连接下，部件两两之间都要有专门连线，连线数量随部件数平方级增长，且同一时刻每个部件只能与一个部件通信，既难扩展又难维护。改成总线后，所有部件挂在一组公共线上，新增设备只需挂到总线上，可扩展性大幅提升，代价是同一时刻只能有一对主从设备通信，需要总线仲裁来解决争用。

> **总线 = 共享 + 分时**
>
> 共享决定「大家都能用」，分时决定「同一时刻只有一个能发」。这两句话能推出后面所有结论：为什么需要仲裁、为什么需要定时、为什么带宽要按「每次传送多少字节 × 每秒传送多少次」算。

*1.*总线的分类

按数据传送方式分：串行总线与并行总线。

- 并行总线：含多条双向数据线，一次可以同时传送多位。优点是单位时间内传送的数据量大；缺点是各条数据线的电气特性很难完全一致，线长增加后各位可能无法同时到达接收端（时钟偏斜），相邻线之间还会串扰，因此并行总线只适合近距离通信。大多数系统总线属于并行总线。
- 串行总线：只有一条双向数据线或两条单向数据线，逐位按序传送。优点是线少、抗干扰好、传输距离长；缺点是同样时钟频率下单次传送量少。大多数通信总线属于串行总线。

「并行一定比串行快」是错的

并行总线能同时传多位，在时钟频率较低时确实比串行快；但当频率提高到一定程度，并行线之间的串扰和偏斜会让数据无法恢复，反而提不上频率。串行总线线少、串扰小，容易把时钟频率做高，还能用多通道（多条链路）叠加带宽，所以高速串行总线的实际速率完全可以超过并行总线。统考考过这个结论：PCI-Express 就是高速串行总线，PCIe×16 是 16 条串行通路的组合，不是 16 位并行传输。同理，USB 的 D+、D− 是一对差分信号，两者共同表示一个串行数据状态，不是「同时传输 **2** 位数据」。

按功能层次分：片内总线、系统总线、通信总线。

- 片内总线：芯片内部各部件之间的连线，如 CPU 内部寄存器之间、寄存器与 ALU 之间、控制器与执行部件之间的连线。
- 系统总线：计算机系统内各功能部件（CPU、主存、I/O 接口）之间相互连接的总线，是通常默认所指的「总线」。按传送信息内容又细分为三类：
  - 数据总线（**data bus**）：传送数据信息。它是双向总线。这里的数据是广义的，包括 CPU 与主存之间交换的数据或指令、CPU 从 I/O 接口读入的状态字与中断类型号、CPU 写入 I/O 接口的命令字，以及地址/数据线复用时的地址信息。数据总线的条数称为数据总线宽度，它是衡量系统性能的重要指标。注意 **MDR** 的位数与数据总线宽度相等，但与存储字长不一定相等。
  - 地址总线（**address bus**）：给出数据总线上源数据或目的数据在主存单元或 I/O 端口中的地址，是单向总线（由主设备指向从设备）。地址线的位数决定 CPU 可直接寻址的空间大小，n 根地址线最大可寻址 2^n 个单元；地址线位数 = **MAR** 位数。
  - 控制总线（**control bus**）：传送控制命令和时序信号，用于总线控制权协商、设备控制命令与应答等。对其中某一根信号线是单向的（要么是 **CPU** 送往部件，要么是部件送往 **CPU**），但对整个控制总线而言是双向的。常见控制信号包括存储器读/写、I/O 读/写、传输应答、总线请求/许可/忙、中断请求/响应、时钟与复位信号等。
- 通信总线（外部总线）：计算机系统之间、或计算机系统与其他系统（远程通信设备、测试设备）之间传送信息的总线，如 USB。相对而言，系统总线与 I/O 总线也可统称为内部总线。

数据线上「不可能」传什么

统考反复考一类题：问系统总线的数据线上不可能传输下列哪一项。判断依据是看信号的性质属于数据还是属于控制：

- 指令、操作数 → 数据线；
- I/O 接口的命令字、状态字 → 数据线（它们本质是写入/读出寄存器的二进制数据）；
- 中断类型号 → 数据线（中断响应时由中断控制器送上数据总线）；
- 握手（应答）信号、读/写控制信号、总线请求/许可 → 控制总线，不是数据线。

所以第 1 类题的标准答案是「握手信号」。第 2 类题问「I/O 总线的数据线上传输的信息包括」，答案是命令字、状态字、中断类型号三者都包括。

按传输定时方式分：同步总线与异步总线。同步总线用统一时钟定时，异步总线用握手信号定时。这一对概念放在「总线事务和定时」中展开。

单总线结构不等于只有一根线

「单总线结构」是指 CPU、主存、I/O 设备（经 I/O 接口）全部挂在一组系统总线上，而不是说物理上只有一根信号线——这一组总线里仍然包含数据线、地址线、控制线。单总线的优点是结构简单、易扩充；缺点是所有部件共享一组线，同一时刻只能有一个部件向总线发送信息，系统效率受限，多用于小型机或微型机。

（二）总线的组成及性能指标

总线在物理上由一组信号线加总线控制器（仲裁机构）组成；逻辑上则包括三条信息通路（数据、地址、控制）以及总线控制权协商机制。评价一条总线，看下面六个指标。

*1.*总线宽度

总线宽度指数据总线的根数，单位是位（bit），如 8 位、16 位、32 位、64 位。在并行总线中它直接决定一次能并发传送多少位数据，是带宽公式里的第一个因子。计算带宽时通常要先把它换算成字节：$W(\mathrm{B}) = \text{总线宽度}/8$。

*2.*总线时钟频率与总线工作频率

- 总线时钟频率 f_{clk} 是总线时钟周期的倒数。同步总线中传输双方持有完全同步的时钟信号，时钟频率越高传输越快。
- 单时钟传输次数 k 指一个总线时钟周期内传送数据的次数：普通总线取 1；**DDR**（Double Data Rate）在时钟的上升沿和下降沿各传送一次，取 2；QDR 取 4。
- 总线工作频率（也称有效数据传输率）为

$$f_{work} = f_{clk} \times k$$

时钟频率 $\neq$ 工作频率

这是统考反复设的坑：同步总线由时钟信号定时，但时钟频率不一定等于工作频率。双沿传输时工作频率是时钟频率的 2 倍；反过来，如果一个总线周期占多个时钟周期，工作频率又会低于时钟频率。「时钟频率」与「工作频率」必须分开算。

*3.*总线传输周期

总线传输周期（简称总线周期）是指在总线上完成一次总线事务所用的时间，由若干个总线时钟周期组成。完整的总线周期一般包括总线请求、总线仲裁、设备寻址、数据传输、总线释放几个阶段。通常一个总线周期只传送一个总线宽度的数据。

两个推论要记住：

- 总线时钟频率越高，总线周期越短；
- 采用地址/数据线复用会加长总线周期（地址和数据要分时传送），所以复用能省引脚，但不能提高速率，反而可能降低速率。

*4.*总线带宽

总线带宽（**bus bandwidth**）是总线上最大的数据传输速率（单位时间传送的字节数），通常不考虑总线传输周期中总线请求、仲裁、寻址等开销，只按真正传送数据的部分计算。单位是 MB/s 或

GB/s，其中 **1 MB** = 10^6 **B**，**1 GB** = 10^9 **B**（总线带宽沿用通信与存储领域的十进制换算，不用 1024）。

$$BW = \frac{W}{8} \times f_{clk} \times k = \frac{W}{8} \times f_{work}$$

其中 W 为总线宽度（bit），f_{clk} 为时钟频率，k 为单时钟传输次数，f_{work} 为工作频率。

这个公式还有几种等价写法，考试里哪一种方便就用哪一种：

- 按「每次传送的字节数 × 每秒传送次数」写：$BW = D \times f_{work}$，D 为一次传送的字节数。若题目给的是「一个总线周期传送 D 字节，一个总线周期占 m 个时钟周期」，则 $BW = \dfrac{D}{m \cdot T_{clk}} = \dfrac{D \cdot f_{clk}}{m}$。
- 按「总线周期」写：$BW = \dfrac{D}{T_{bus}}$，T_{bus} 为总线传输周期。这就是题目里常见的 $W/(T)$ 形式：$BW = W_{\text{字节}}/T_{\text{总线周期}}$。
- 按「每秒传输次数」写（串行高速总线，如 QPI、PCIe）：BW = GT/s × 每次有效字节数 × 方向数。

四种表述，同一个式子

1. $BW = \dfrac{W}{8} \times f_{clk} \times k$
2. $BW = \dfrac{W}{8} \times f_{work}$
3. $BW = \dfrac{D}{T_{bus}}$（D 为一个总线周期传送的字节数）
4. $BW = \dfrac{D \cdot f_{clk}}{m}$（$m$ 为一个总线周期占用的时钟周期数）

它们互相等价，区别只是题目给了哪几个量。做题第一步永远是：先确定「一次传送多少字节」，再确定「每秒传送多少次」，两者相乘。

5.总线带宽典型算例

算例 1（时钟周期数与总线周期不相等，2009 年统考）

系统总线在一个总线周期中并行传输 4 B，一个总线周期占 **2** 个时钟周期，总线时钟频率 10 MHz，求总线带宽。

一个时钟周期传送 $4\ \text{B}/2 = 2\ \text{B}$，故

$$BW = 2\ \text{B} \times 10\ \text{MHz} = 20\ \text{MB/s}$$

陷阱：直接算成 $4 \times 10 = 40$ MB/s 就忽略了一个总线周期占 2 个时钟周期。

算例 2（双沿传输 + 地址/数据线复用，2014 年统考）

同步总线采用数据线和地址线复用，地址/数据线 32 根，总线时钟频率 66 MHz，每个时钟周期传送两次数据（上升沿、下降沿各一次），求最大数据传输率。

32 根线在数据阶段一次传送 32 bit = 4 B，单时钟传输次数 $k = 2$：

$$BW = 4\ \text{B} \times 66\ \text{MHz} \times 2 = 528\ \text{MB/s}$$

注意：题目问的是峰值传输率，地址/数据线复用只影响事务的时序结构，不需要在结果里再除以 2。

算例 3（多通道存储器总线，2019 年统考）

采用 3 通道存储器总线，内存条为 DDR3-1333（存储器总线工作频率 1333 MHz，总线宽度 64 位），求总带宽。

DDR3-1333 中的 1333 已经是双倍传输后的有效工作频率，不能再乘 2。单通道一次传送 8 B：

$$BW_{单} = 8\ \text{B} \times 1333\ \text{MHz} \approx 10.66\ \text{GB/s}, \qquad BW_{总} = 3 \times 10.66 \approx 32\ \text{GB/s}$$

算例 4（串行全双工总线，2020 年统考）

QPI 是点对点全双工同步串行总线，每方向每次传输 20 位（16 位数据 + 4 位校验位），每个时钟周期传送 2 次，时钟频率 2.4 GHz。求总线带宽。

每秒传输次数 $= 2.4\ \text{GHz} \times 2 = 4.8\ \text{GT/s}$；每次有效数据 16 bit = 2 B；全双工两个方向同时传输：

$$BW = 4.8\ \text{GT/s} \times 2\ \text{B} \times 2 = 19.2\ \text{GB/s}$$

两个易错点：4 位校验位不计入有效数据带宽；全双工的「×2」是方向数，不能漏。

算例 5（非突发事务逐次计算，2023 年统考）

存储器总线宽度 64 位，总线时钟频率 1 GHz，传输一个数据或地址需 1 个时钟周期，不支持突发传送。主存每次准备一个 64 位数据需 6 ns，主存块大小 32 B。求读取一个主存块的时间。

时钟周期 $= 1$ ns。总线宽度 64 位 = 8 B，一个 32 B 主存块需传送 $32/8 = 4$ 次。不支持突发，则每次都要单独走一次完整事务：送地址 1 个周期（1 ns）+ 主存准备 6 ns + 传数据 1 个周期（1 ns）= 8 ns。

$$T = 4 \times 8\ \text{ns} = 32\ \text{ns}$$

总线带宽与「实际数据传输率」不是一回事

总线带宽是总线的固有性能指标，只反映「有数据传送时」的最快速率，不受传送方式影响，也不把地址阶段、仲裁阶段、存储器准备时间算进去。如果题目问的是「处理器一直发起某类事务时总线的实际数据传输率」，那就要用「一批数据量 ÷ 完成这批数据所需的全部时间（含地址、准备、仲裁开销）」来算，两者的答案会明显不同。真题曾在选项中同时给出这两个数来迷惑考生（例如 2024 年那道给足干扰条件的题，答案只需按有数据传送的时钟周期算）。

6.总线复用

总线信号线分为专用与复用两类。专用信号线只传一种信号（系统总线的数据线、地址线、控制线就是专用总线）；复用总线是让一组传输线具有多种用途，分时传送不同类型的信息。常见形式：

- 地址总线与数据总线复用：某时刻传地址，另一时刻传数据。PCI 总线是典型例子。
- 同一功能的线缆复用：DRAM 芯片的行、列地址线复用，在不同时刻分别送行地址和列地址，大幅减少芯片引脚。
- 串行总线可以看作最极端的总线复用：所有信息都通过一根（或一对差分）线传递。

复用的优点是提高总线利用率、减少引脚数目和布线空间、降低成本；缺点是控制逻辑复杂，且分时传送机制会引起总线性能下降（事务时间变长）。

复用能提高速率吗？不能

「采用信号线复用技术可减少信号线数量」正确；「采用地址/数据线复用可提高同步总线数据传输率」错误。能提高数据传输率的是：增加总线宽度、提高总线工作频率、支持突发传送这三项。突发传送之所以有效，是因为它只送一次首地址就把连续多个数据传完，摊薄了地址和仲裁开销，提高了有效（实际）数据传输率。

7.其他性能指标

- 总线负载能力：总线上可连接的设备数量，与总线的驱动能力和电气特性有关。
- 总线复用程度、是否支持突发、是否支持分离事务：都是影响有效带宽的结构性因素。

（三）总线事务和定时

*1.*总线事务与突发传送

总线事务（**bus transaction**）是指在总线上一对主从设备之间完成一次信息交换的过程，即从请求总线到完成总线使用的操作序列。发起事务并能获得总线控制权的设备叫主设备（**master**），被主设备寻址、被动响应的设备叫从设备（**slave**）。

一条总线上可以挂多对主从设备，但同一时刻只有一个活动主设备

主设备既可以发送数据给从设备，也可以接收从设备的数据，是主动的一方；从设备只能被动响应。如果主设备采用广播方式发送数据，则可以有多个从设备同时接收。总线事务的类型由操作性质决定，典型的有存储器读、存储器写、I/O 读、I/O 写、中断响应等。

一个完整的总线周期可分五个阶段：

1. 总线请求：CPU 或 DMA 等主设备需要使用总线时发出传输申请。
2. 总线仲裁：多个主设备同时请求时，由总线仲裁机构决定谁获得下一个传输周期的总线控制权。
3. 设备寻址：获得控制权的主设备把要访问的从设备地址及相关命令送到总线上，启动从设备。
4. 数据传输：主从设备之间进行数据交换，数据单向或双向传送。该阶段通常一次只传送一个字长的数据。
5. 总线释放：主设备不再占用总线，撤除相关信息，释放总线控制权。

「一次总线事务」是常考的定义题

题干若描述「主设备只需给出一个首地址，从设备就能从首地址开始的若干连续单元读出或写入多个数据」，答案是突发（猝发）传输。要能与另外几组概念区分开：并行/串行描述一次通过几根通道传送数据位；同步/异步描述定时方式；它们都不是「事务的组织方式」。

总线突发（猝发）传送方式（成组传送事务）：一次性传送地址上连续的一批数据。主设备只需给出这批数据的首地址，随后就可以在传输阶段连续传送多个地址连续单元的数据；在这批数据传输过程中总线一直被占用，直到整批数据传完才释放。

- 非突发方式：每个传送周期都要先送地址再送数据，主从设备之间通常每次只能传一个字长。
- 突发方式：每个总线周期仍只传一个计算机字长，但不释放总线，直到一组信息全部传完。

突发传送的代价是按地址递增规则访问，因此适用于连续成块的数据（存储器块读、Cache 行填充等）。它的收益是把「地址开销」摊薄到多个数据上，从而提高总线的有效数据传输率。

算例（突发传送的事务时间，2012 年统考）

同步总线时钟频率 100 MHz、宽度 32 位、地址/数据线复用、每传一个地址或数据占 1 个时钟周期，支持突发传送。求一次「主存写」总线事务传输 128 位数据所需的最少时间。

时钟周期 $= 1/100\ \text{MHz} = 10\ \text{ns}$。复用线传首地址需 1 个周期 = 10 ns；32 位宽度下传 128 位需 $128/32 = 4$ 个周期 = 40 ns。

$$T = 10\ \text{ns} + 40\ \text{ns} = 50\ \text{ns}$$

陷阱：漏掉地址周期会得到 40 ns；把复用误当成「要额外多除一次」会得到更离谱的数。题目说「至少」，意味着不计仲裁、存储器等待和额外应答时间。

*2.*总线定时：同步定时与异步定时

主设备获得总线使用权后，每一步操作何时开始、何时结束、信号保持多久，就是总线定时问题。常见有四种：同步方式、异步方式、半同步方式、分离事务方式，前两种是考纲重点。

同步定时方式。进行信息交换的两部件或设备通过统一的时钟进行同步，即都必须使用同一个时钟，在规定时钟节拍内完成规定的总线操作。挂在总线上的所有设备都从时钟线上获得定时信号。例如规定「存储器读」协议：主设备在第 1 个时钟周期发送地址和存储器读命令，从设备总是在第 5 个时钟周期把数据放到总线上作为响应，处理器也在第 5 个周期取数。写操作的传输周期长度同样是固定的。

- 优点：主从设备协调简单，传输协议简单，传输速率高。
- 缺点：总线定时以最慢设备所花时间为标准，因此适合存取时间相差不大的多个功能部件之间通信；存在时钟偏移问题，所以同步总线不能过长，否则会降低传输效率；同步定时中主从设备默认对方正常工作，可靠性不高。
- 适用：总线长度短、设备速度相近、可靠性要求高的场合，如直接与 CPU 相连的系统总线。

异步定时方式。没有统一的时钟限制，只依靠各部件或设备内部的定时操作，以信号握手的方式完成规定的总线操作：主设备发出请求（**Request**）信号，从设备收到后发出回答（**Acknowledge**）信号。由于不需要公共时钟规定请求与应答的时间，总线周期的长度不再相同，且有效规避了同步定时的时钟偏移问题，总线传输距离更长。

- 优点：总线周期灵活可变，主从设备不需强制同步，可以支持速度不同的两个设备挂在同一条总线上。
- 缺点：控制逻辑比同步方式复杂，速度比同步定时慢。

根据请求和回答信号的撤销是否互锁，异步定时分为三种：

方式	请求信号 **REQ** 何时撤销	回答信号 **ACK** 何时撤销	速度	可靠性
不互锁	发出后经过一段时间自动撤销，不等 ACK	收到 REQ 后经过一段时间自动撤销，不等 REQ 撤销	最快	最差
半互锁	必须等收到 **ACK** 之后才能撤销	自动撤销，不受 REQ 约束	中等	中等
全互锁	必须等收到 **ACK** 之后才能撤销	必须等 **REQ** 撤销之后才能撤销	最慢	最高

三种握手方式的记忆口诀

不互锁：谁也不等谁，发完就撤 → 快而不可靠。半互锁：主设备等从设备（请求等应答才撤），从设备不等主设备（应答自动撤）。全互锁：互相等（请求等应答才撤，应答等请求撤了才撤）→ 最慢最可靠。

结论排序：速度不互锁 > 半互锁 > 全互锁；可靠性全互锁 > 半互锁 > 不互锁。

半同步定时方式。保留同步定时的特点：所有地址、命令、数据信号的发出时间都严格参照系统时钟的某个前沿开始，接收方采用系统时钟后沿判断识别；同时又像异步定时那样允许不同速度的设备协同工作。为此增设一条 **Wait** 响应信号线：总线在每个时钟到来时采样 Wait 信号，若无效说明数据未准备好，下个时钟再采样，直到有效才去数据线上取数据。

半同步兼有同步与异步的优点：所有信号仍由公共时钟定时，又允许不同速率设备协同工作，还能有效解决异步传输中的噪声敏感问题。适用于系统工作速度不高、但由许多速度差异较大的设备组成的简单系统。

同步总线的时钟由谁提供

「同步通信方式中，同步时钟信号可由各设备提供」是错误说法。同步总线上的设备依据统一的公共时钟工作，该时钟通常由总线控制器或统一时钟源提供，不能由各设备提供彼此独立的时钟，否则就不叫同步总线了。

同步/异步常考结论汇总

- 异步通信中全互锁协议最慢（正确）。
- 异步通信中非互锁协议可靠性最差（正确）。
- 同步通信中时钟信号可由各设备提供（错误）。
- 半同步通信中握手信号的采样由同步时钟控制（正确）。
- 异步总线由握手信号定时，一次握手过程完成的是一个总线宽度的数据交换，不是一位数据（常说成「完成一位」是错的）。
- 同步总线的时钟频率不一定等于工作频率（正确）。

*3.*分离事务通信方式（拓展）

前面三种定时方式中，从主设备发出地址和读/写命令到数据传输结束，整个传输周期内总线使用权完全由主设备及被选中的从设备占据。但从设备准备数据的阶段，总线其实纯属空闲等待。分离事务通信方式把一个事务分成两个子过程：

1. 主设备获得总线后把事务类型（总线命令）、地址及其他信息（如主设备编号）送到总线上，从设备记录这些信息后，主设备立即释放总线，其他设备便可使用；
2. 从设备准备好数据后重新申请总线，获得使用权后把主设备编号和数据送上总线，主设备据此接收数据。

- 优点：在不传送数据的间隙释放总线，让其他申请者使用，提高总线利用率。
- 缺点：控制相当复杂、开销大，事务响应时间变长。

*4.*总线仲裁

当多个主设备同时请求总线时，必须由总线仲裁机构（总线控制器 / 仲裁器）决定谁获得下一个传输周期的使用权。仲裁方式分为集中式与分布式两大类。

集中式仲裁：仲裁逻辑集中在一处（一般是 CPU 或专门的仲裁器），各主设备通过专门的控制线与它相连。

1. 链式查询方式：所有设备共用一条总线请求（**BR**）线和一条总线忙（**BS**）线，另有一条总线允许（**BG**）线像链条一样从仲裁器依次穿过各设备。仲裁器收到请求后发出 BG 信号，BG 沿链依次传递，第一个收到 **BG** 且自己有请求的设备获得总线控制权，同时截断 BG 不再往下传。
   - 优点：只用三根控制线，结构简单，容易扩充（增减设备只需改链条连接）。
   - 缺点：优先级固定（离仲裁器越近越高），且对故障敏感——链条上某一级损坏，其后的设备都无法获得总线。
2. 计数器定时查询方式：撤销 BG 线，改用一组设备地址线。仲裁器收到请求后启动计数器开始计数，计数值通过地址线送到各设备，计数值与自己设备号相等且有请求的设备获得控制权并停止计数。
   - 优点：优先级可由计数器初值灵活改变（每次从 0 开始 → 固定优先级；每次从中止点继续 → 循环优先级，各设备机会均等）；对故障不如链式敏感。
   - 缺点：需要 $\lceil \log_2 n \rceil$ 根地址线，控制线比链式多。

3. 独立请求方式：每个设备都有一对独立的总线请求线 **BR** 和总线允许线 **BG**，直接连到仲裁器。
   - 优点：响应速度快（仲裁器可并行看到所有请求），优先级灵活（可由程序设定），对单个设备故障不敏感。
   - 缺点：控制线数量多（$2n$ 根），总线控制器复杂，扩展性受引脚限制。

分布式仲裁：不设中央仲裁器，每个主设备都带有仲裁逻辑和自己的仲裁号，各设备把仲裁号送到共享的仲裁线上，每个设备各自判断自己是否胜出（谁的仲裁号大谁赢），胜者获得总线。优点是可靠性高、无单点故障；缺点是逻辑分散、难以实现复杂的优先级策略。

三种集中式仲裁的对比考法

常考「只用三根控制线」→ 链式查询；「优先级固定、对故障敏感」→ 链式查询；「优先级可循环改变」→ 计数器定时查询；「响应最快、控制线最多」→ 独立请求。

三、I/O 接口（I/O 控制器）

（一）I/O 接口的功能和基本结构

外部设备与主机在工作速度、数据格式、时序控制上差异极大，且各自有独立的时钟。因此必须在主机与外设之间设置专门的逻辑部件来解决同步协调、速度匹配和数据格式转换问题，这个部件就是 **I/O 接口（I/O interface）**，也叫设备控制器、**I/O** 模块、适配器（**adapter**）。

I/O 接口不等于 I/O 设备

I/O 设备由执行 **I/O** 操作的机械部分（设备本体）和执行控制 **I/O** 的电子部件（接口/控制器）组成，通常把这两部分分开。所以：

- 磁盘驱动器是设备本体，不是 I/O 接口；磁盘控制器才是接口。
- 打印机适配器、网络控制器、可编程中断控制器都属于 I/O 接口。

1.I/O 接口的功能

1. 数据缓冲：用数据缓冲寄存器匹配 CPU 与外设之间的速度差异，起到「蓄水池」作用。
2. 错误检测（以及必要的纠错）：如奇偶校验、CRC 校验、传输超时检测，把出错信息记入状态寄存器供 CPU 查询。
3. 控制与定时：接收 CPU 的命令，产生设备所需的控制信号与时序，协调主机与外设的异步工作。
4. 数据格式转换：如串行 ↔ 并行转换（串行接口）、字节 ↔ 字装配（不同字长的设备）、数模/模数转换等。
5. 与主机和设备的通信：主机一侧按总线标准通信（地址译码、读写控制、中断请求），设备一侧按设备自身的电气与协议标准通信。
6. 此外还有设备寻址（让 CPU 能选中某个设备及其某个端口）与状态检测功能。

2.I/O 接口的基本结构

I/O 接口内部可分为两部分：连接总线的总线接口（必须按总线标准设计，属标准部分）和连接设备的内部接口（因设备而异，属非标准部分）。不论何种接口，一般都应包含下列部件：

1. 数据缓冲寄存器（**DBR**，数据端口）：输入时存放从设备读来的数据等待 CPU 取走，输出时暂存 CPU 送来的数据等待送往设备。
2. 设备状态寄存器（状态端口）：反馈设备状态，如设备忙、设备就绪、设备错误等。程序查询方

式中 CPU 正是反复读它来判断设备状态。

3. 设备命令寄存器（控制寄存器，控制端口）：接收 CPU 发来的设备控制命令，如设备复位、设备识别、读写控制、启动/停止。简单设备（如普通键盘、字符终端）可能根本没有命令寄存器。
4. 设备存储器（非必需）：供设备自身运算和处理使用，如显卡中的显存。
5. 地址译码器和 **I/O** 控制逻辑：接收地址线和控制线送来的信息，对控制寄存器中的命令字译码，把控制信号送到外设侧；同时把数据缓冲寄存器的数据送到外设或从外设接收数据；还要收集外设状态到状态寄存器。
6. 中断控制逻辑：产生中断请求、接收中断响应、提供中断类型号（向量中断方式下），并参与中断屏蔽判断。

状态寄存器与控制寄存器为何常合二为一

状态寄存器和控制寄存器在传送方向上是相反的（状态字由接口送往 CPU，是读；命令字由 CPU 写入接口，是写），而且 CPU 对它们的访问在时间上一般是错开的，所以很多接口把二者合并成同一个寄存器，通过读、写方向来区分。这就是真题里「状态端口和控制端口可以合用同一个寄存器」这一说法的来源。

*3.*接口三组总线各传什么

与 I/O 接口相连的数据线、地址线、控制线都是双向的（因为 DMA 方式下接口本身也能当主设备），但传送内容不同：

- 数据线：真正的 I/O 数据（经数据缓冲寄存器）、CPU 写入命令寄存器的控制字/命令字、CPU 从状态寄存器读出的状态字。
- 地址线：I/O 端口或设备寄存器的地址（CPU 在访问时给出，由接口内的地址译码器识别并选中寄存器）；DMA 方式下接口作为主设备发出的主存地址。
- 控制线：总线控制命令与时序信号（读/写、时钟）、总线控制权协商信号（DMA 请求）、应答信号、中断响应信号。

「命令字走数据线」与「控制信号走控制线」不是一回事

CPU 向接口控制寄存器写入命令字时，命令字是一组二进制数据值，它在数据总线上传送；而读/写控制信号、握手信号是控制信号，在控制总线上传送。前者是「要做什么」（What），后者是「现在执行读还是写、什么时候有效」（How/When）。2011 与 2012 年都考过这个区分。

（二）I/O 端口及其编址

把接口中 CPU 可访问的寄存器统称为 **I/O** 端口（**I/O port**）：数据缓冲寄存器简称数据端口，状态寄存器简称状态端口，控制寄存器简称控制端口。所有 I/O 端口编号组成的空间称为 **I/O** 地址空间。

访问这些端口的指令称为 **I/O** 指令（如 IA-32 的 `in`、`ins`、`out`、`outs`）。I/O 指令只能在操作系统内核的底层 I/O 软件中使用，因此 **I/O** 指令是特权指令。

I/O 指令传送的是什么

I/O 指令由 CPU 执行，典型功能是「通用寄存器 ↔ **I/O** 端口」之间的数据传送。设备与端口之间的交换由接口电路和设备控制逻辑完成；CPU 的通用寄存器不会绕过接口直接连外设。所以「I/O 指令实现的数据传送发生在通用寄存器和 **I/O** 端口之间」是正确的。

1.I/O 端口的两种编址方式

对比项	统一编址（存储器映射编址）	独立编址（**I/O** 映射方式）
概念	把 I/O 端口当成主存单元一样统一编址	I/O 端口地址与存储器地址无关，各自独立
访问指令	不需要专门的 **I/O** 指令，用普通访存指令（LOAD/STORE）即可读写端口	必须使用专门的输入输出指令（如 x86 的 IN/OUT）
优点	① 端口编址空间大；② 读写控制逻辑简单；③ I/O 访问与主存访问共用一组指令，保护机制可由分段/分页存储管理实现，无需专门保护机制	① 用专用 I/O 指令，程序清晰、便于编程；② **I/O** 端口地址位数少，地址译码速度快；③ **I/O** 端口地址不占用主存地址空间
缺点	① 端口占用了主存地址空间，主存地址空间变小；② 外设寻址时间长（地址位数多、译码慢）	① I/O 指令类型少，一般只能对端口传送，程序设计灵活性差；② 需要 **CPU** 提供存储器读/写与 **I/O** 读/写两组控制信号，增加控制逻辑复杂性
如何区分访问对象	通过地址码区分（按地址范围判断是主存还是端口）	通过不同指令区分（I/O 指令的地址码给出端口号，不同指令产生不同的总线控制信号）
典型处理器	MIPS、ARM 等 RISC 处理器	Intel x86 处理器

统一编址与独立编址的两个反向陷阱

1.「采用独立编址时，I/O 端口地址和主存地址可能相同」——正确。两者用不同地址空间，数值上重叠是允许的，靠指令（及相应的总线控制信号）来区分。
2.「采用统一编址时，CPU 不能用访存指令访问 I/O 端口」——错误。统一编址又称存储器映射 I/O，CPU 正是通过普通访存指令访问 I/O 端口。

统一编址的端口不能被 Cache 缓存

I/O 地址虽然映射到主存空间，但接口中的数据是动态变化的。若把端口内容放进 Cache，CPU 读到的就可能是过时副本，无法了解设备的实时状态变化。所以映射到主存空间的 I/O 区域必须标记为不可缓存。

四、I/O 方式

I/O 方式的发展主线是「尽量减少主机对 **I/O** 控制的干预」。从程序查询（CPU 全程参与）到程序中断（CPU 只在数据就绪时参与），再到 DMA（CPU 只做预处理和后处理），CPU 的参与度逐级降低，数据传输单位从「字节/字」变为「数据块」。

（一）程序查询方式

程序查询方式（程序直接控制方式）：I/O 完全依靠 CPU 执行程序实现。CPU 要与设备交换数据时，先设置接口命令寄存器启动设备；设备准备期间，CPU 反复读取接口中的状态寄存器查询设备是否就绪，据此决定是进行数据传送还是继续等待。反复查询的过程称为轮询（**polling**）。字符设备一次轮询传一个字节或字，块设备一次轮询可传一个数据块。

基本查询流程（以有命令寄存器的复杂设备为例）：

1. 读状态端口，判断设备是否就绪；未就绪则继续查询；
2. 就绪后，向控制端口写入命令与参数（写端口地址、传送长度、传送方向等），启动设备；

3. 再次不断查询状态端口，等待「命令执行完毕/数据就绪」；
4. 就绪后通过数据端口完成一次数据传送；
5. 若是成块传送，修改内存地址与计数器，回到第 3 步循环。

两种查询策略：

- 独占查询（忙等待，**busy-waiting**）：启动设备后就一直查询状态寄存器直到设备就绪，期间 CPU 不能执行其他任务。一次轮询只传一个数据单元。适用于单任务操作系统；对超高速设备也可以接受（设备响应快，等待时间极短）。
- 定时查询：CPU 启动设备的同时启动一个定时器，设备准备期间 CPU 不忙等而是继续工作，定时时间到产生定时中断，在中断服务程序中查询设备状态。缺点是定时时间难以确定：过短则数据未就绪白查，过长则数据已就绪却得不到及时处理。适用于外设不支持中断的多任务系统。

*1.*查询方式下的 *CPU* 时间开销

核心思想：开销 = 每秒查询次数 × 每次查询的 **CPU** 时间。

$$T_{\text{CPU 占空比}} = \frac{n_{\text{查询次数/秒}} \times t_{\text{每次查询}}}{1\ \text{s}}$$

其中每秒查询次数由设备的数据率和每次传送的单位决定：

$$n_{\text{查询次数/秒}} = \frac{\text{设备数据传输率}}{\text{每次传送的字节数}}$$

每次查询耗时 t 可以由「查询程序的时钟周期数 × 时钟周期」得到，也可以由「指令条数 × CPI × 时钟周期」得到。

算例 1（定时查询开销，**2011** 年统考）

处理器主频 50 MHz，采用定时查询方式控制设备 A。查询程序运行一次至少用 **500** 个时钟周期；设备 A 工作期间为不丢数据每秒至少查询 **200** 次。求 CPU 用于设备 A 的 I/O 时间占整个 CPU 时间的百分比至少是多少。

每秒用于查询的时钟周期数 $= 200 \times 500 = 100\,000 = 0.1$ M；CPU 每秒共 50 M 个时钟周期：

$$\frac{0.1\ \text{M}}{50\ \text{M}} = 0.2\%$$

也可以先算一次查询耗时 $500/50\ \text{MHz} = 10\ \mu\text{s}$，每秒查询 200 次共占 2 ms，$2\ \text{ms}/1\ \text{s} = 0.2\%$，结果一致。

算例 2（查询间隔与开销，**2018** 年统考第 **43** 题第 **1** 问）

计算机主频 500 MHz，CPI 为 4，设备 A 的数据传输率为 2 MB/s，接口有 32 位数据缓冲寄存器，每次输入/输出至少执行 10 条指令。

（**1**）最多间隔多长时间查询一次才不丢数据?

设备准备一个 32 位（4 B）数据的时间为

$$t = \frac{4\ \text{B}}{2\ \text{MB/s}} = 2\ \mu\text{s}$$

超过 2 μs 不查，缓冲区数据未取走而新数据又到来，就会覆盖丢失，所以最多每隔 **2** μ**s** 查询一次。

（**2**）**CPU** 用于设备 **A** 输入输出的时间占 **CPU** 总时间的百分比至少是多少?

每次输入/输出需要 $10 \times 4 = 40$ 个时钟周期；每秒查询次数 $= 1\ \text{s}/2\ \mu\text{s} = 5 \times 10^5$ 次：

$$\frac{40\times 5\times 10^5}{500\times 10^6}=\frac{2\times 10^7}{5\times 10^8}=4\%$$

算例 3（查询方式对高速设备不可行的量化，袁春风教材例题的口径）

查询程序每次操作至少 400 个时钟周期，处理器主频 500 MHz。求三种设备下查询开销占比。

- 鼠标：每秒至少查询 30 次 $\rightarrow \dfrac{30\times 400}{500\times 10^6}\approx 0.002\%$，可忽略。
- 软盘：16 位（2 B）为单位，50 kB/s $\rightarrow \dfrac{50\text{ kB/s}}{2\text{ B}}=25\text{ k 次/秒}\rightarrow \dfrac{25\times 10^3\times 400}{500\times 10^6}=2\%$，偏高但尚可忍受。
- 硬盘：16 B 为单位，4 MB/s $\rightarrow \dfrac{4\text{ MB/s}}{16\text{ B}}=250\text{ k 次/秒}\rightarrow \dfrac{250\times 10^3\times 400}{500\times 10^6}=20\%$，CPU 五分之一的时间用于查询，不可取。

结论：设备越快、每次传送单位越小，查询方式的开销越大，因此查询方式不适合高速成块传送。

「程序查询只适合慢速设备」不严谨

中断方式有额外开销（两次上下文切换 + 中断服务程序本身）。当设备准备数据的时间比这些开销还短时，用中断反而比忙等待轮询效率更低（中断成为瓶颈）。因此程序查询方式并不一定只适合慢速设备，一些高速设备（如 NVMe SSD 处理同步 I/O）也采用轮询方式。

（二）程序中断方式

*1.*中断的基本概念

中断（**interrupt**）是指 CPU 在执行程序过程中，出现某些来自 **CPU** 外部的、与当前指令执行无关的紧急事件，CPU 暂停现行程序，转去执行相应的处理程序，处理完后返回断点继续执行。异常（**exception**）则是指令执行过程中在处理器内部发生的、与当前指令同步相关的特殊事件。

中断与异常的判定标准

判断依据是「事件是否与当前指令同步相关」，而不是名称里有没有「存储」「I/O」字样：

- 与当前指令同步 → 异常（内部异常）：整数除以 0、访存缺页、存储保护错、非法指令。
- 与当前指令无关、来自 CPU 外部 → 中断（外部中断）：DMA 传送结束、打印机缺纸、I/O 完成、时钟中断、电源故障。

注意区分：外部中断在当前指令执行结束后检测；异常在当前指令执行过程中检测。

外部中断分为：

- 可屏蔽中断：通过 `INTR` 线请求，受中断允许位（IA-32 中是 `EFLAGS` 的 `IF`）控制，**CPU** 处于关中断状态时不会被响应；优先级最低。
- 不可屏蔽中断（**NMI**）：通过 `NMI` 线请求，用于电源掉电、硬件故障等紧急事件，不受中断允许位和中断屏蔽字控制，关中断状态下也能响应，优先级最高。NMI 的中断向量号通常固定，响应时不需要从外部读中断类型号。

中断源的记录与判优。每个中断源设一个中断请求标记触发器，状态为 1 表示有请求；这些触发器可组成中断请求标记寄存器，可集中在 CPU 中，也可分散在各中断源中。CPU 在每条指令执行的最后时刻自动检测中断请求引脚，发现请求后按优先级决定是否响应、响应哪个。

中断判优（中断响应优先级）依据「该中断源若得不到及时响应，致使机器工作出错的严重程度」确定：

1. 不可屏蔽中断 > 内部异常 > 可屏蔽中断；

2. 内部异常中，硬件故障 > 软件中断；
3. **DMA** 中断请求优先于 **I/O** 设备传送的中断请求；
4. I/O 传送类中断中，高速设备优先于低速设备，输入设备优先于输出设备，实时设备优先于普通设备。

判优可用硬件（链式排队器、中断控制器中的优先级编码器）或软件（按优先级从高到低逐级查询各中断源）实现。软件判优硬件简单、优先级可通过改变查询顺序灵活调整，但速度慢，可能无法实时响应。

中断排队不是「先来先服务」

「中断控制器按所接收中断请求的先后次序进行中断优先级排队」是错误的。中断排队依据预设的优先级（固定优先级或轮询优先级），不是按请求到达的时间先后。

*2.*中断响应过程

CPU 响应中断必须同时满足三个条件：

1. 至少有一个未被屏蔽的中断请求（中断请求触发器置 1，并已送排队器判优）；
2. **CPU** 允许中断且处于开中断状态（中断允许触发器 `EINT` = 1；异常和不可屏蔽中断不受此限制）；
3. 当前指令执行完毕（外部中断，异常不受此限制），且没有更紧急的任务。

三个条件里的常见错误说法

- 「一旦可屏蔽中断请求信号有效，CPU 将立即响应」——错误。请求有效 ≠ 立即响应，还要等当前指令结束、且处于开中断状态、且未被屏蔽。
- 「有中断请求时，CPU 立即暂停当前指令执行」——错误。CPU 不会打断正在执行的指令（这会破坏指令的原子性），只在指令边界响应。
- 「仅在用户态下 CPU 才能检测和响应中断」——错误。内核态下只要开中断且优先级条件满足，同样可以响应（这正是多重中断嵌套的基础）。

CPU 响应中断后进入中断响应周期。该周期内的操作由硬件直接实现，称为中断隐指令。中断隐指令不是指令系统中一条真正的指令，只是「硬件自动完成的一系列操作」的虚拟说法。它完成的操作是：

1. 关中断：置中断允许位为 0，屏蔽所有可屏蔽中断。目的是保证断点和现场在被完整保存之前不被新的中断打断。IA-32 中可通过特权指令 `sti`/`cli` 置位/清位 `IF`。
2. 保存断点：把指令无法直接读取的 `PC` 和 `PSW`（程序状态字寄存器）内容自动保存到栈或特定寄存器中。断点一般是下一条指令的地址；但对异常则不一定——「缺页故障」的断点是发生页故障的当前指令地址（处理完要重新执行该指令），「陷阱」的断点是陷阱指令后面一条指令的地址。断点与异常类型有关。
3. 形成中断服务程序入口地址并送 **PC**（引出中断服务程序 / 中断源识别）：若是特权级切换（用户态 → 内核态），硬件还会自动完成用户栈到内核栈的切换。

中断隐指令到底做了哪些事

统考原题（2012 年）：响应外部中断的过程中，中断隐指令完成的操作，除保护断点外，还包括——正确答案是 Ⅰ 关中断与 Ⅲ 形成中断服务程序入口地址并送 **PC**；Ⅱ「保存通用寄存器的内容」不属于中断隐指令，那是进入中断服务程序后由软件完成的第一件事。

口诀：中断隐指令 = 关中断 + 保护断点（**PC/PSW**）+ 找入口（中断源识别）。

中断源的识别方式有两种：

- 软件识别：CPU 中设原因寄存器（**Cause**）记录异常原因或中断类型标志，操作系统用一个统一的查询程序按优先级顺序查原因寄存器，先查到的先处理。
- 硬件识别（向量中断方式）：处理程序的首地址 PC 和初始 PSW 称为中断向量（**interrupt vector**），所有中断向量集中存放构成中断向量表。每个中断源有唯一中断类型号，中断向量位置与类型号相关（类型 0 存第 0 表项，类型 1 存第 1 表项……）。也由 **CPU** 外部的中断控制器根据各 I/O 设备的中断请求和中断屏蔽情况，结合中断响应优先级，识别当前请求的中断类型号，并通过数据总线送到 CPU。

CPU 获得中断类型号后，据此计算中断向量地址（**VA**），从向量表中取出中断服务程序入口地址送 PC，转去执行中断服务程序。

*3.*中断处理过程

中断服务程序（软件）的执行流程，完整版（支持多重中断）为：

1. 关中断（已由中断隐指令完成）；
2. 保存断点（已由中断隐指令完成，保存 PC 和 PSW）；
3. 保存现场和旧屏蔽字：现场信息指用户可见的工作寄存器（通用寄存器等）的内容，通过压栈指令保存，由软件实现；中断屏蔽字寄存器虽用户不可见，也由操作系统一并保存；
4. 设置新屏蔽字（由 I/O 指令访问中断控制器完成）；
5. 开中断：允许更高级中断请求得到响应，实现中断嵌套；
6. 执行中断服务例程：这是中断请求的真正目的，如程序 I/O 中的数据输入/输出；
7. 关中断：保证恢复现场和屏蔽字时不被中断；
8. 恢复现场和旧屏蔽字；
9. 开中断；
10. 中断返回：最后一条指令通常是中断返回指令 **IRET**，返回到原程序断点继续执行。

对于单级中断（不允许嵌套），去掉第 4、5 步和第 7、9 步中与开/关中断相关的部分，流程简化为：保护现场 → 中断事件处理 → 恢复现场 → 开中断 → 中断返回。

硬件做什么、软件做什么——一条界线

硬件（中断隐指令）：关中断、保存断点（PC、PSW）、中断源识别与引出服务程序、特权级与堆栈切换。
软件（中断服务程序）：保存通用寄存器等现场、设置新屏蔽字、执行服务操作、恢复现场与屏蔽字、中断返回。

记住这条界线，2018、2024 那类「谁负责保存什么」的题就不会错。：

- 2015 年统考：「处理外部中断时，应该由操作系统保存的是」→ 答案是通用寄存器的内容（PC 由硬件保存，TLB 和 Cache 内容不需要在单次中断中专门保存）。
- 2024 年统考：「保存断点和程序状态字在中断响应阶段完成」正确；「保存通用寄存器和设置新中断屏蔽字由软件实现」正确。

`IRET` 指令执行时完成的工作：① 从内核栈弹出 `EIP`、`CS`、`EFLAGS`，恢复断点和程序状态；② 检查当前处理程序的 CPL 是否等于 CS 中最低两位，若相等说明中断前后处于同一特权级，`IRET` 结束；否则继续下一步；③ 从内核栈弹出 `SS` 和 `ESP`，恢复中断前的栈，即由内核态改回用户态并完成堆栈切换。执行完 `IRET`，CPU 自然回到被中断的进程继续执行。

为什么断点要保存到栈里

为支持异常或中断的嵌套处理，大多数处理器把断点保存在栈中（如 80x86）。如果系统不支持多重中断嵌套，可以把断点保存在特定寄存器中而不必送栈（如 MIPS 用 EPC 寄存器专门存放断点），后者开销更小——因为访问栈就是访问存储器，比访问寄存器慢。

*4.*多重中断和中断屏蔽

- 单重中断：CPU 在执行中断服务程序过程中，又出现新的更高优先级的中断请求，但 CPU 不予响应，这就是单重中断。此时 CPU 在中断处理期间一直处于关中断状态。
- 多重中断（中断嵌套）：CPU 暂停现行的中断服务程序，转去处理新的中断请求。此时中断处理期间 CPU 并非始终关中断——在保护好现场并设置新屏蔽字后会重新开中断。

CPU 要具备多重中断功能，必须满足两个条件：

1. 在中断服务程序中提前设置开中断指令；
2. 优先级别高的中断源有权中断优先级别低的中断源。

中断处理优先级与中断屏蔽。这里要区分两个不同概念：

对比项	中断响应优先级（中断判优）	中断处理优先级（中断屏蔽）
由谁决定	由中断查询程序或硬件判优电路决定，反映多个中断同时请求时哪个先被响应	由各自的中断屏蔽字（屏蔽字寄存器内容）动态设定，反映本中断与其他所有中断之间的处理优先关系
可否修改	响应优先级往往是硬件线路已设置好，不便于改动	在多重中断系统中通常用中断屏蔽字动态分配，可在程序中执行 I/O 指令修改屏蔽字内容即可改变处理次序
生效时机	只在当前无正在执行的中断、需要从队列中判优选择时使用	任何时刻立即生效：一出现处理优先级更高的中断就立即响应并抢占
队列含义	队列中存放的是未被响应的中断，它们的处理优先级一定都比正在执行的中断低（否则早就抢占了）	当处理优先级高的中断都执行完毕，中断返回最初状态，再从队列中取响应优先级最高的给予响应

如果不使用中断屏蔽技术，处理优先级与响应优先级相同。

屏蔽字的设置规律：

1. 一般用 **1** 表示屏蔽，**0** 表示正常申请；
2. 每个中断源对应一个屏蔽字（在处理该中断源的中断服务程序时，屏蔽字寄存器中的内容就是该中断源对应的屏蔽字）；
3. 屏蔽字中 **1** 越多，优先级越高；每个屏蔽字中至少有一个 **1**（至少要能屏蔽自身的中断）。

算例（屏蔽字计算，2011 年统考）

某计算机有五级中断 $L_4 \sim L_0$，中断屏蔽字为 $M_4M_3M_2M_1M_0$，$M_i = 1$ 表示对 L_i 级中断进行屏蔽。中断响应优先级从高到低为 $L_0 \to L_1 \to L_2 \to L_3 \to L_4$；要求中断处理优先级从高到低为 $L_4 \to L_0 \to L_2 \to L_1 \to L_3$。求 L_1 的中断处理程序中设置的中断屏蔽字。

解题步骤：

1. 写出要求的处理优先级序列：$L_4 \to L_0 \to L_2 \to L_1 \to L_3$。
2. 对目标中断 L_1：处理优先级比它高的允许打断它 $\to$ 对应位为 **0**；它自己以及处理优先级比它低的要屏蔽 $\to$ 对应位为 **1**。

3. 比 L_1 高的有 L_4、L_0、$L_2 \to M_4=0$、$M_0=0$、$M_2=0$；本级 L_1 与更低的 $L_3 \to M_1=1$、$M_3=1$。
4. 按 $M_4M_3M_2M_1M_0$ 排列：$01010=$ **01010**。

关键：屏蔽字由「处理优先级」决定，不是由「响应优先级」决定。题目给响应优先级只是为了增加干扰或后续提问。

中断屏蔽字 ≠ 响应优先级

「中断屏蔽字用于确定中断响应的优先级」是错误说法。响应优先级由硬件排队或优先级判定电路决定；中断屏蔽字控制的是中断嵌套关系，即处理优先级。这是 2024 年统考的原题考点。

多重中断的其他常见判断

- 「在一条指令执行结束时响应中断」正确；
- 「中断处理期间 CPU 处于关中断状态」在多重中断语境下错误（服务程序中会重新开中断）；在单重中断语境下则正确；
- 「中断请求的产生与当前指令的执行无关」正确（异步性）；
- 「CPU 通过采样中断请求信号检测中断请求」正确；
- 「CPU 处于关中断状态时也能响应 NMI 请求」正确。

5.中断方式与查询方式的对比

对比项	程序查询方式	程序中断方式
CPU 参与方式	CPU 全程主动轮询，设备与 CPU 串行工作	CPU 启动设备后继续执行原程序，设备就绪时主动通知，CPU 与设备并行工作
谁发起数据传送	CPU 轮询发现就绪后主动传送	设备主动发出中断请求，CPU 被动响应
数据传送实现者	CPU 执行查询程序	CPU 执行中断服务程序（软件）
CPU 时间开销	与查询次数成正比，设备越快、传送单位越小开销越大	与中断次数 × 每次中断开销成正比；额外还有两次上下文切换开销
实时性	依赖查询频率，可能漏掉数据或浪费查询	由设备主动触发，实时性好，无需纠结轮询频率
适用场合	单任务系统、超高速设备、操作系统引导阶段	键盘、鼠标、针式打印机等低速字符设备

中断方式为什么不适合高速设备

设外设准备数据的时间为 t_{dev}，中断响应与处理时间为 t_{int}。为避免「前一数据未被取走、后一数据又到来造成覆盖」，必须满足

$$t_{dev} \geq t_{int}$$

即外设准备数据的时间应不小于中断响应和处理时间。「外设准备数据的时间应小于中断处理时间」是错误说法（2022 年统考考点）。设备越快，t_{dev} 越小，而 t_{int} 由几十到上千个时钟周期构成，很快就无法满足，因此高速设备应改用 DMA。

6.中断方式下的 *CPU* 开销与效率计算

$$T_{\text{CPU 占空比}}=\frac{n_{\text{中断次数/秒}} \times C_{\text{每次中断时钟周期数}}}{f_{\text{CPU}}}$$

其中中断次数由设备数据率和每次中断传送的数据量决定：

$$n_{\text{中断次数/秒}} = \frac{\text{设备数据传输率}}{\text{每次中断传送的字节数}}$$

每次中断的时钟周期数 C 通常由「中断响应周期数 + 中断服务程序指令数 × CPI + 其他开销」给出。

算例 1（中断开销，2019 年统考）

设备以中断方式与 CPU 交换数据，CPU 主频 1 GHz，接口中数据缓冲寄存器为 32 位，设备数据传输率 50 kB/s，每次中断开销（含中断响应和中断处理）为 1000 个时钟周期。求 CPU 用于该设备 I/O 的时间占整个 CPU 时间的百分比最多是多少。

每次中断最多传送 32 位 = 4 B：

$$n = \frac{50\ \text{kB/s}}{4\ \text{B}} = 12.5\ \text{k 次/秒}$$

每次中断耗时 $= 1000/1\ \text{GHz} = 1\ \mu\text{s}$，每秒中断开销时间 $= 12.5 \times 10^3 \times 1\ \mu\text{s} = 12.5\ \text{ms}$：

$$\frac{12.5\ \text{ms}}{1\ \text{s}} = 1.25\%$$

算例 2（中断 vs DMA 的开销对比，2010 年统考第 43 题）

CPU 主频 500 MHz，CPI = 5。外设数据传输率 0.5 MB/s，以 **32** 位为传输单位，中断服务程序 18 条指令，中断服务的其他开销相当于 2 条指令的执行时间。

（**1**）中断方式下 **CPU** 用于该外设 **I/O** 的时间占比？

每次中断的时钟周期数 $= 5 \times 18 + 5 \times 2 = 100$；每秒中断次数 $= 0.5\ \text{MB/s} \div 4\ \text{B} = 125\,000$：

$$\frac{100 \times 125\,000}{500 \times 10^6} = \frac{12.5 \times 10^6}{500 \times 10^6} = 2.5\%$$

（**2**）数据传输率提高到 **5 MB/s**，改用 **DMA**，每次 **DMA** 传送块大小 **5000 B**，**DMA** 预处理 + 后处理总开销 **500** 个时钟周期，**CPU** 占比？

每秒 DMA 次数 $= 5\ \text{MB/s} \div 5000\ \text{B} = 1000$：

$$\frac{1000 \times 500}{500 \times 10^6} = \frac{0.5 \times 10^6}{500 \times 10^6} = 0.1\%$$

同样吞吐量下，中断方式的开销是 DMA 的 **25** 倍，这就是高速设备必须用 DMA 的量化理由。

算例 3（中断允许的最大延迟，2014 年统考）

设备中断请求的响应和处理时间为 100 ns，每 400 ns 发出一次中断请求，中断响应所允许的最长延迟时间为 50 ns。求 CPU 用于该设备的 I/O 时间占整个 CPU 时间的百分比至少是多少。

$$\frac{100\ \text{ns}}{400\ \text{ns}} = 25\%$$

陷阱：那 50 ns 是「请求可以等待的上限」，等待期间 CPU 可以执行其他工作，不属于 CPU 用于该设备 I/O 的时间，不能加进分子。这是本题设置 B、C 两个干扰项的原因。

算例 4（异步串行中断，2012 年统考第 43 题）

CPU 主频 50 MHz，CPI = 4。设备 D 以异步串行方式向主机传送 7 位 ASCII 字符，规程中有 1 位奇校验位和 1 位停止位，从 D 接收启动命令到字符送入 I/O 端口需要 0.5 ms。I/O 端口每收到一个字符申请一次中断，中断响应需 10 个时钟周期，中断服务程序共 20 条指令。

（**1**）每传送一个字符在通信线上共需传输多少位？每秒最多可送多少个字符？

1(起始位)+7(数据位)+1(校验位)+1(停止位)= **10** 位。每字符耗时 0.5 ms，故每秒最多 $1000/0.5 = $ **2000** 个字符。

（**2**）读取 **1000** 个字符所需时间，以及 **CPU** 用于该任务的时间（时钟周期数）？

时钟周期 $= 1/50$ MHz $= 20$ ns。设备送字符到端口的耗时 $= 0.5$ ms$/20$ ns $= 2.5 \times 10^4$ 个时钟周期；一次中断响应 10 个周期；中断服务程序前 15 条指令（第 15 条启动 D 工作）$= 4 \times 15 = 60$ 个周期。

单字符传送时间约 $2.5 \times 10^4 + 10 + 60 = 25\,070$ 个时钟周期，1000 个字符约

$$1000 \times 25\,070 = 2.507 \times 10^7 \text{ 个时钟周期}$$

CPU 用于该任务的时间（每次响应 + 全部服务指令）$= 1000 \times (10 + 4 \times 20) = 1000 \times 90 = 9 \times 10^4$ 个时钟周期。

（**3**）中断响应阶段 **CPU** 进行了哪些操作？关中断、保护断点和程序状态、识别中断源。

*7.*中断方式与 *I/O* 端口交换的信息

中断 I/O 方式下，CPU 与打印控制接口的 I/O 端口之间可以交换：打印字符（写数据端口）、设备状态（读状态端口）、控制命令（写控制端口）。不可能交换的是主存地址——数据由 CPU 通过指令搬运，接口不直接访问主存，不关心数据来自内存何处；主存地址是 DMA 控制器必须掌握的信息。

（三）DMA 方式

1.DMA 的基本概念

直接内存访问（**Direct Memory Access**，**DMA**）：在总线上设置 **DMA** 控制器（**DMAC**），由 DMAC 临时接管总线，代替 CPU 控制外部设备与内存之间的批量数据交换。数据无须由 **CPU** 寄存器中转，完全消除了程序查询和程序中断方式中 CPU 进行实际数据传输的开销。

DMA 的工作轮廓是：

1. 传送前 CPU 访问 DMAC 接口，设置主存地址、数据块长度、传输方向等参数；
2. CPU 向设备的 I/O 接口发送命令和参数，启动设备；
3. 传输过程中 CPU 继续执行其他程序；设备需要传送时由 **DMAC** 向 **CPU** 申请总线控制权完成传输；
4. 一个数据块传送结束后，DMAC 通过中断方式请求 CPU 对缓冲区和 DMAC 进行后处理。

DMA 主要用于高速设备的块数据传输（磁盘、显卡、网卡、声卡），也可用于内存之间的大批量数据搬移。

DMA 的三句话概括

1. 请求的是总线控制权，不是 CPU 时间；
2. 实际数据传送由硬件完成，不由 CPU 执行程序完成，因此不涉及现场保护；
3. 传送单位是数据块，CPU 只在块传送开始（预处理）和结束（后处理）时参与。

2.DMA 控制器的组成

DMAC 连接在系统总线上，既可充当从设备（预处理阶段），也可充当主设备（数据传输阶段）。其基本组成：

部件	作用
主存地址寄存器 / 地址计数器 **AR**	存放交换数据的内存地址。预处理阶段由 CPU 设置；每次 DMA 传送后自动修改（递增或递减），指向下一个内存单元
字计数器 **WC**	记录数据块长度。预处理阶段设置；每传送 1 个字或字节自动减 **1**；减到 0 时数据传送完毕，发出 **DMA** 中断请求
数据缓冲寄存器 **DBR**	DMAC 作从设备时接收 CPU 传来的数据，作主设备时暂存传输数据。若不经该寄存器而让数据直接经数据总线从设备到内存，则性能更好
命令寄存器 / 控制寄存器	接收 CPU 的控制命令（启动 DMA、传输方向、工作方式等）
状态寄存器	向 CPU 反馈 DMAC 的状态（传输完成、发生错误及错误类型）
DMA 控制逻辑（含中断机构）	含控制与时序电路、状态标志；负责修改地址计数器和字计数器、指定传输方向、对 DMA 请求与 CPU 响应进行协调和同步；还支持中断机制，一组数据交换完毕时向 CPU 发 DMA 中断请求

握手信号：

- DMAC 需要总线控制权时，通过系统总线中的 **HOLD**（总线请求）信号向 CPU 发出请求；CPU 的总线控制器若允许，回送 **HLDA**（总线响应/授权）信号。
- 支持 DMA 的 I/O 设备通过 **DREQ**（**DMA** 请求）与 **DACK**（**DMA** 应答）两条专用控制线与 DMAC 相连。这两条线通常不属于系统总线，而是 DMAC 与 I/O 接口之间的私有通道。
- 一个 DMAC 支持多个通路，每个通路各有独立的主存地址寄存器和字计数器，可同时为多个设备提供 DMA 服务。

DMAC 的角色在换

- 预处理阶段：CPU 是主设备，DMAC 是从设备（CPU 像访问普通 I/O 端口一样用 `OUT` 等指令写 DMAC 的寄存器）。
- 数据传输阶段：DMAC 是主设备，CPU 变成旁观者。

~~DMAC~~ 连接的对象是「设备控制器」而不是「设备」

物理设备（硬盘、网卡）需要专门的设备控制器处理底层协议、时序与数据缓冲；DMAC 是通用的「搬运工」，无法直接理解成千上万种设备的具体细节。所以 **DMAC** 沟通的对象应该是设备控制器（**I/O** 接口），而不是设备本体。同理，「DMAC 与设备之间」也不是数据传输通路的端点。

3.DMA 传送过程

（**1**）预处理阶段

1. CPU 通过 **I/O** 指令把参数写入 DMAC：主存地址寄存器 **AR**（数据在主存的起始地址）、字计数器 **WC**（要传送的数据总量）、控制/命令寄存器（传输方向：设备 → 内存还是内存 → 设备）。
2. CPU 向 **I/O** 接口（设备控制器）发送启动命令，命令中包含设备地址、磁盘扇区号等信息，写入设备控制器自己的寄存器（如设备地址寄存器 DAR）。
3. CPU 向 DMAC 控制寄存器写入「启动 **DMA**」命令位，然后把发起 **I/O** 请求的进程挂起（阻塞），调度另一个就绪进程投入运行。

预处理结束时：DMAC 已掌握本次任务的所有细节，I/O 设备已开始准备数据，CPU 已完全脱身，**CPU** 与 **I/O** 操作实现并行。

（**2**）数据传送阶段（以周期挪用为例，本质是一个循环，每次循环传一个字）

1. 设备就绪，发出请求：I/O 设备准备好一个数据字放入内部缓冲寄存器后，通过 DREQ 向 DMAC 发出 DMA 请求。
2. **DMAC** 申请总线：DMAC 通过 HOLD（或 BREQ）向 CPU 发出总线请求。
3. **CPU** 授权总线：CPU 在当前机器周期（或总线事务/总线周期）结束后响应，断开自己与总线的连接（总线驱动器进入高阻态），发出 HLDA。
4. **DMAC** 主导数据传输（在一个存储周期内完成）：把 **AR** 中的地址送上地址总线；向控制总线发出相应控制信号（如存储器写 MEMW、I/O 读 IORD）；通过 DACK 回应发出请求的设备；I/O 接口把数据字放上数据总线；主存的存储控制器把数据写入指定单元。
5. 更新状态并归还总线：**AR** 自动加 **1**（或减 **1**），**WC** 自动减 **1**；DMAC 撤销 HOLD，立即把总线控制权归还 **CPU**。CPU 可继续执行若干周期，直到下一次 DREQ 到来——这就是「周期窃取」的精髓。

循环直到 **WC** 减为 **0**，DMAC 通过 INTR 向 CPU 发出中断请求，数据传输阶段结束。

（**3**）后处理阶段

1. DMAC 通过 INTR 发出中断请求（触发原因：正常完成——WC 递减至 0；或异常中止——内存校验错、设备无响应等）。
2. CPU 响应中断、保存现场、转入中断服务程序。
3. CPU 通过 I/O 指令读取 DMAC 的状态寄存器，检查「传输完成」标志、「发生错误」标志及错误类型。
4. 根据状态决策：传输成功则唤醒因 **I/O** 请求而阻塞的进程；传输出错则做错误处理（重传、报错、终止任务）。
5. 中断返回，CPU 继续原程序。

DMA 传送的三段与 CPU 的两次介入

预处理（CPU 设置参数 + 启动设备）→ 数据传送（DMAC 硬件全权搬运，CPU 不参与）→ 后处理（CPU 执行中断服务程序收尾）。

CPU 只在首（预处理）尾（后处理）介入；中间的海量搬运与 CPU 无关。所以不存在由 **CPU** 持续执行的「**DMA** 传送程序」——那是 2023 年统考的错误选项。

4.DMA 与 CPU 共享主存的三种方式

DMAC 和 CPU 都要访问主存，为避免访存冲突，常用三种方式：

方式	基本思想	工作过程	优点	缺点
停止 **CPU** 访存（成组传送）	DMA 优先，一旦开始不传完不罢休	DMAC 申请总线，CPU 在完成当前指令（或总线周期）后释放总线并暂停/等待；DMAC 在连续多个存储周期内把整块数据传完；传完归还总线，CPU 恢复	控制最简单（请求、建立、归还只做一次）；数据传输率最高，适合大块高速传输	传输期间 CPU 无法响应任何中断，实时性差；即使 CPU 当前不需要访存也被强制暂停，**CPU** 时间被浪费
周期挪用（周期窃取）	按需分配，DMA 只在需要时借用一个存储周期	设备准备好一个字，DMAC 就申请一次总线；CPU 在当前机器周期结束后释放；DMAC 占用一个存储周期传完这个字后立即归还	CPU 与 DMA 分时交替使用总线，CPU 不会被长时间阻塞，DMA 响应及时	每次搬一个字都要请求/仲裁/归还，有额外开销（整个挪用过程实际可能花 2 ~ 5 个存储周期）；若 DMA 与 CPU 同时访存，DMA 优先，CPU 被延缓

方式	基本思想	工作过程	优点	缺点
DMA 与 **CPU** 交替访存	把内存工作周期一分为二，各用一半	把每个内存工作周期固定划分为 T_1、T_2 两段，强制规定 **CPU** 只能在 T_1 访存，**DMAC** 只能在 T_2 访存	不需要总线请求（时间预先分配好），CPU 感觉不到 DMA 的存在，可连续工作	硬件代价高（需要复杂的时钟与内存控制逻辑）；资源浪费——某一方空闲时它的时间片就浪费了，实质上使内存总带宽减半；实际应用很少

周期挪用不是一个块传完才还总线

「周期挪用方式下，在整个数据块传送过程中 CPU 不可以访问主存储器」是错误说法。周期挪用只在需要传送数据时窃取相应的存储周期，相邻两次 DMA 传送之间 CPU 仍可正常访存。只有「停止 CPU 访存」（成组/连续传送）方式下，CPU 才会在整个数据块传送期间被禁止访问主存。这正是 2020 年统考 512 B / 32 位缓冲寄存器那道题的正确选项与错误选项的分界。

周期挪用与访存冲突

DMA 操作不会修改 **CPU** 寄存器，所以不存在现场保护问题。若周期挪用期间 CPU 不访问内存，则对 CPU 执行没有性能影响；若恰好也访问内存（形成访存冲突），则 **DMA** 优先访问内存——因为若 DMA 请求得不到及时响应，I/O 传输的数据可能丢失。周期挪用已成为 DMA 传输方式的主要方法。

算例（周期挪用的请求次数，**2012** 年统考第 **43** 题第 **2** 问）

CPU 主频 80 MHz，CPI = 4，平均每条指令访存 1.5 次，主存与 Cache 之间块大小 16 B，Cache 命中率 99%，存储器总线宽度 32 位。

（**1**）**MIPS** 数与每秒 **Cache** 缺失次数?

MIPS $= 80\ \mathrm{M}/4 = 20$。每秒 Cache 缺失次数 $= 20\ \mathrm{M} \times 1.5 \times (1 - 99\%) = 300\,000$ 次/秒。

（**2**）**Cache** 缺失时主存带宽需求?

$16\ \mathrm{B} \times 300\,000/\mathrm{s} = 4.8\ \mathrm{MB/s}$。

（**3**）缺页时磁盘接口每秒发出的 **DMA** 请求次数至少是多少?

缺页率 0.0005%，故每秒缺页异常 $= 300\,000 \times 0.0005\% = 1.5$ 次。页面大小 4 KB = 4096 B，磁盘 I/O 接口的数据缓冲寄存器 32 位 = 4 B，每传 4 B 发一次 DMA 请求：

$$1.5 \times \frac{4096}{4} = 1.5 \times 1024 = 1536\ \text{次/秒}$$

（**4**）**CPU** 与 **DMA** 同时要求使用存储器总线时谁优先？为什么?

DMA 优先级更高，因为若 DMA 请求得不到及时响应，I/O 传输的数据可能丢失。

算例（一个扇区读写中的总线请求次数，**2015** 年统考第 **43** 题第 **3** 问）

磁盘转速 7200 r/min，扇区 512 B，采用周期挪用 **DMA**，磁盘控制器数据缓冲区大小 64 位。

一个扇区读写中 DMA 控制器发出的总线请求次数 $= 512\ \mathrm{B}/64\ \mathrm{b} = 512/8 =$ **64** 次。若 CPU 检测到 DMA 请求时也需要访存，**DMAC** 仍能获得总线使用权，因为一旦磁盘开始读写就必须按时完成数据传送，否则会丢数据。

5.DMA 与中断方式的对比

对比项	程序中断方式	**DMA** 方式
谁请求	I/O 设备请求 CPU 的处理时间（请求 CPU 中断服务）	DMAC 请求总线使用权（总线控制权）

对比项	程序中断方式	**DMA** 方式
谁响应、何时响应	CPU 在一条指令执行结束后检测并响应	CPU 在一个总线事务（机器周期/总线周期）结束后响应并让出总线
传输单位	字节或字（一次中断传一个字）	数据块（一次 DMA 传一整块）
数据传送由谁完成	CPU 执行中断服务程序（软件）完成	**DMAC** 硬件直接控制总线完成，不经 CPU 寄存器
CPU 干预程度	每传一个字节/字都要 CPU 参与，需保护和恢复现场	CPU 只在预处理（设参数、启动）和后处理（结束中断处理）时介入，不改变 **CPU** 现场
优先级	较低	**DMA** 请求优先级高于 **CPU** 访存，也高于普通 **I/O** 传送中断请求
功能范围	不仅能传数据，还能处理各种随机事件（故障、状态变化），灵活性高	仅用于数据传输（也可用于内存间搬移）
适用场合	键盘、鼠标、打印机等低速字符设备	磁盘、SSD、网卡、显卡等高速块设备

DMA 与中断对比题的三个经典错误项

1. 「中断 I/O 方式适用于所有外部设备」——错误。高速设备用中断会造成 CPU 开销过大。
2. 「DMA 方式仅适用于快速外部设备」——片面。DMA 技术上也用于低速设备，只是不划算（初始化开销大）。
3. 「DMA 方式下通过 **CPU** 执行 **DMA** 传送程序进行 I/O 操作」——错误。DMA 传送过程不存在 CPU 持续执行的传送程序。

（四）三种 **I/O** 方式的完整对比

对比项	程序查询方式	程序中断方式	**DMA** 方式
发起者	CPU 主动轮询	设备主动发中断请求	设备/DMAC 向 CPU 申请总线
CPU 与设备关系	串行（CPU 忙等）	并行（CPU 执行原程序）	并行（CPU 执行其他程序）
数据传送实现者	CPU 执行查询程序（软件）	CPU 执行中断服务程序（软件）	DMAC 硬件直接控制总线
传送单位	字节/字（或块，但 CPU 全程参与）	字节/字	数据块
CPU 介入时机	全程	每次数据就绪时	仅预处理与后处理
是否需要现场保护	不需要（无切换）	需要（保存/恢复现场）	不需要（不改变 CPU 现场）
请求的资源	—	CPU 处理时间	总线使用权
响应时机	立即（查询到即处理）	一条指令执行结束后	一个总线事务（机器周期）结束后
CPU 开销公式	$n_{\text{查询}} \times t_{\text{每次查询}}$	$n_{\text{中断}} \times C_{\text{每次中断}}$	$n_{\text{DMA 块}} \times C_{\text{预/后处理}}$
数据是否经 CPU 寄存器	经过	经过	不经过
典型设备	单任务系统、引导阶段、超高速同步设备	键盘、鼠标、针式打印机	磁盘、SSD、网卡、显卡

对比项	程序查询方式	程序中断方式	**DMA** 方式
优点	接口简单，不需额外硬件	消除轮询开销，实时性好，可处理随机事件	CPU 开销极小，吞吐率最高，适合大块高速传输
缺点	CPU 浪费严重，效率低	每次传送都要 CPU 参与，高速时开销过大	需要额外 DMAC 硬件，与 CPU 争用主存

本章小结表

考点	必背结论
总线两特点	共享、分时；同一时刻只有一个主设备能发送信息
系统总线三类	数据总线（双向，宽度 = MDR 位数）、地址总线（单向，位数 = MAR 位数，决定 2^n 寻址空间）、控制总线（单根单向、整体双向）
串行 vs 并行	并行适合近距离（系统总线），串行适合远距离（通信总线）；串行速率可以超过并行
总线带宽公式	$BW = \frac{W}{8} \times f_{clk} \times k = \frac{W}{8} \times f_{work} = \frac{D}{T_{bus}} = \frac{D \cdot f_{clk}}{m}$
提高带宽的手段	增加总线宽度、提高工作频率、支持突发传送（复用不能提速）
时钟频率 vs 工作频率	$f_{work} = f_{clk} \times$ 单时钟传输次数；DDR 取 2，普通取 1
总线复用	优点：省引脚、提高利用率、降成本；缺点：控制复杂、性能下降
突发传送	只送一次首地址，连续传多个数据，期间不释放总线；提高有效数据传输率
总线带宽 vs 实际传输率	带宽是固有指标，不受传送方式影响，不含地址/仲裁/准备开销
同步定时	统一时钟；协调简单、速率高；以最慢设备为标准、有时钟偏移、不能过长、可靠性不高
异步定时	握手信号，总线周期长度可变，可挂不同速度设备；逻辑复杂、速度较慢
三种互锁	速度：不互锁 > 半互锁 > 全互锁；可靠性：全互锁 > 半互锁 > 不互锁
半同步	公共时钟 + **Wait** 信号线；握手信号由时钟边沿采样
分离事务	请求与应答分两段，中间释放总线 → 提高总线利用率
集中式仲裁	链式查询（3 根线，优先级固定，对故障敏感）；计数器定时查询（优先级可循环）；独立请求（最快，线最多，优先级灵活）
I/O 接口非设备	磁盘驱动器是设备；控制器/适配器/网络控制器/中断控制器是接口
接口三类寄存器	数据缓冲寄存器（数据端口）、状态寄存器（状态端口）、控制寄存器（控制端口）；状态与控制方向相反，可合用同一寄存器
数据线上传什么	指令、操作数、命令字、状态字、中断类型号；握手信号走控制线
统一编址	端口与主存同一地址空间，用访存指令访问；占用主存空间、寻址慢；保护机制可复用分页/分段；端口不能进 **Cache**
独立编址	独立地址空间，用专用 **I/O** 指令（x86 IN/OUT）；不占主存空间、译码快；指令类型少、需两组控制信号
I/O 指令	传送发生在通用寄存器 ↔ **I/O** 端口之间；是特权指令
程序查询	CPU 与设备串行工作；开销 $= n_{\text{查询}} \times t_{\text{每次}}$；查询间隔必须 ≤ 设备准备一个数据的时间

考点	必背结论
定时查询	定时器到时查状态；定时过短白查、过长丢数据
中断响应三条件	有未被屏蔽的请求；CPU 开中断（异常和 NMI 不受限）；当前指令执行完毕
中断隐指令	关中断 + 保存断点（**PC/PSW**）+ 找服务程序入口；不清通用寄存器、不保存现场
断点差别	中断断点 = 下一条指令地址；缺页 fault 断点 = 当前指令地址；陷阱断点 = 陷阱指令后一条
服务程序流程	保存现场和旧屏蔽字 → 设置新屏蔽字 → 开中断 → 服务 → 关中断 → 恢复现场和屏蔽字 → 开中断 → 中断返回
单重 vs 多重	单重：处理期间一直关中断；多重：设新屏蔽字后重新开中断，允许嵌套
两种优先级	响应优先级由硬件/查询程序定，解决「同时请求先响应谁」；处理优先级由屏蔽字动态定，决定能否被抢占
屏蔽字规则	1 屏蔽、0 允许；**1** 越多优先级越高；每个屏蔽字至少一个 **1**；由处理优先级决定，不是响应优先级
中断开销	$T = \frac{n_{\text{中断}} \times C_{\text{每次}}}{f_{CPU}}$；须满足 $t_{dev} \geq t_{int}$ 才不丢数据
DMAC 组成	主存地址寄存器 AR、字计数器 WC、数据缓冲寄存器 DBR、（命令/状态寄存器）、DMA 控制逻辑（含中断机构）
DMA 握手线	HOLD（DMAC 请求）、HLDA（CPU 授权）；DREQ/DACK 是 DMAC 与 I/O 接口的专用线，不属系统总线
DMA 三阶段	预处理（CPU 设参数、启动设备）→ 数据传送（DMAC 主设备搬运）→ 后处理（结束中断，CPU 收尾）
AR/WC 变化	每传一次：AR ±1、WC −1；WC = 0 时发 DMA 中断请求
停止 CPU 访存	简单、速率最高；CPU 全程被暂停、无法响应中断
周期挪用	按需借一个存储周期，用完立即归还；CPU 可正常访存（除冲突时 DMA 优先）；挪用过程实际占 2 ~ 5 个存储周期
交替访存	时间片对半分配；无需总线请求；硬件复杂、带宽减半，实际少用
DMA vs 中断	请求总线权 vs 请求 CPU 时间；块 vs 字；总线事务后响应 vs 指令后响应；硬件 vs 软件搬运；不保存现场 vs 保存现场

易错清单

总线和 I/O 章 20 个高频陷阱

1. 并行总线不一定比串行总线快。串行总线线少、串扰小，可把频率做高，实际速率可超过并行；PCIe、USB 都是串行。
2. 地址总线是单向的，控制总线整体是双向的。DMA 控制器也能送主存地址，但地址总线的「单向」是指由主设备指向从设备。
3. **MDR** 位数 = 数据总线宽度 $\neq$ 存储字长；**MAR** 位数 = 地址线位数。
4. 时钟频率 $\neq$ 工作频率。DDR 双沿传输时 $f_{work} = 2f_{clk}$；一个总线周期占多个时钟周期时二者又不同。
5. **DDR3-1333** 的 **1333** 已是有效工作频率，不能再乘 2；QPI 的「每秒传输次数」才是 $f_{clk} \times 2$。
6. 总线带宽不考虑地址、仲裁、存储器准备时间。题目问「实际数据传输率」时要另算，别把两者混用。
7. 地址/数据线复用会加长总线周期，不提高速率；能提高速率的是加宽、提频、突发传送。
8. 突发传送的总线事务要算上首地址那一个周期。2012 年那题漏掉地址周期就会算成 40 ns 而不是 50 ns。
9. 同步定时以最慢设备为标准，所以只适合速度相近的设备；异步定时才能挂不同速度的设备。
10. 异步一次握手完成的是一个总线宽度的数据交换，不是一位。

11. 全互锁最慢但最可靠，不互锁最快但最不可靠；半互锁只锁主设备一侧。
12. 中断隐指令只做三件事：关中断、保存断点（PC/PSW）、引出中断服务程序。保存通用寄存器由软件做（2012、2018 年都考过）。
13. 状态端口与控制端口可以合用同一个寄存器，靠读/写方向区分。
14. 统一编址用访存指令访问端口，不是「不能用访存指令」；独立编址时端口地址与主存地址可以数值相同，靠指令区分。
15. 映射到主存空间的 **I/O** 端口不能被 **Cache** 缓存，否则读到的是过时副本，无法感知设备实时状态。
16. 中断响应优先级 $\neq$ 中断处理优先级。屏蔽字改变的是处理优先级（能否被嵌套抢占），不是响应优先级（同时请求时先响应谁）。2011、2024 年两次考这个区分。
17. 屏蔽字按「处理优先级」写：比本级处理优先级高的位写 0（允许打断），本级和更低的写 1。2011 年 L_1 的答案是 01010。
18. 中断服务程序内的顺序：保存现场 $\rightarrow$ 服务 $\rightarrow$ 恢复现场 $\rightarrow$ 开中断 $\rightarrow$ 中断返回（单级中断）；题干若限定「中断服务程序内」，就不要把硬件负责的关中断、保护断点写进去。
19. 中断方式要求 $t_{dev} \geq t_{int}$，即外设准备数据的时间不小于中断处理时间，否则数据被覆盖丢失。
20. 周期挪用不是整块独占总线。CPU 在相邻两次 DMA 传送之间仍可访存；只有「停止 CPU 访存」方式才在整个块传送期间禁止 CPU 访存。DMA 与 CPU 冲突时 **DMA** 优先，且 DMA 传送不需要保护现场（不修改 CPU 寄存器）。

第三部分

操作系统

第十四章操作系统基础

本章考纲要求

第三部分操作系统［考查目标］

1. 掌握操作系统的基本概念、方法和原理，了解操作系统的结构、功能和服务，理解操作系统所采用的策略、算法和机制。
2. 能够从计算机系统的角度理解并描述应用程序、操作系统内核和计算机硬件协作完成任务的过程。
3. 能够运用操作系统的原理，分析并解决计算机系统中与操作系统相关的问题。

一、操作系统基础（一）操作系统的基本概念（二）操作系统发展历程（三）程序运行环境

1. CPU 运行模式：内核模式，用户模式。
2. 中断和异常的处理
3. 系统调用
4. 程序的链接与装入
5. 程序运行时的内存映像与地址空间

（四）操作系统结构分层，模块化，宏内核，微内核，外核。（五）操作系统引导（六）虚拟机

本章是操作系统部分的开篇，也是整份卷子里「文字辨析题」密度最高的一章：知识点本身不难，但要求精确到词——「内核态只能执行特权指令」错在哪、「中断发生时 CPU 处于什么态」，考的都是对概念边界的把握。

高频点集中在四处：处理机运行模式（用户态 / 内核态）、系统调用、批处理阶段（多道程序设计）、中断和异常；此外操作系统结构（宏内核 / 微内核）、操作系统引导、虚拟机也反复出现。近年还考过「内核态转为用户态的时机」「中断向量表采用的数据结构」「进程虚拟地址空间中各区域的存放内容」等更细的角度。

学习本章的正确姿势是把「用户态 / 内核态」当成一条主线：中断、异常、系统调用、引导过程、虚拟机，本质上都是「谁在什么特权级下、以什么方式获得 CPU 控制权」的不同回答。

一、操作系统的基本概念

（一）操作系统的定义与双重角色

操作系统（**Operating System**，**OS**）是控制和管理整个计算机系统的硬件与软件资源、合理调度任务与分配资源、并为用户和其他软件提供统一接口与运行环境的最基本的系统软件。

从使用层次看，计算机系统自下而上分四层：硬件 → 操作系统 → 应用程序 → 用户。硬件提供基本计算资源，操作系统把资源管起来并向上提供运行环境、充当桥梁，应用程序利用这些资源解决用户问题。这是「人机交互视角」的分层，与计算机组成原理中 CPU 内部结构的分层不是一回事。

操作系统的角色可概括为两重身份、外加一个效果：

1. 作为计算机系统资源的管理者。把用户看作「雇主」、计算机看作「机器」，操作系统就是那位「工人」：协调并控制各个部件工作。
2. 作为用户与计算机硬件系统之间的接口。工人接收雇主命令并执行任务——这对应操作系统提供的用户接口与程序接口；用户使用硬件必须经过操作系统这一必经媒介。
3. 实现对计算机资源的抽象与扩充。只有硬件、没有任何软件支撑的计算机称为裸机；操作系统在裸机之上把底层硬件抽象成更好的逻辑资源，于是呈现给用户的是一台功能更强、更易用的扩充机器（有些教材也叫「虚拟机」，这里指的是操作系统提供的逻辑计算机环境，不是 VMware 那种虚拟化软件造出来的虚拟机）。

（二）操作系统的四大功能

作为资源管理者，操作系统的功能按管理对象分为四类，这四类恰好对应本手册后面四章的内容。

处理机管理：多道程序环境下，处理机的分配和运行都以进程（或线程）为基本单位，所以处理机管理本质上就是对进程的管理——提到「进程管理」时指的也是这块。主要内容包括进程控制、进程同步、进程通信、死锁处理以及处理机调度。

存储器管理：为多道程序提供良好的内存运行环境，提高内存利用率。主要内容包括内存分配与回收、地址映射、内存保护与共享、内存扩充（虚拟存储）。

文件管理：用户可见的持久化信息通常以文件形式组织，负责这部分的部分称为文件系统，功能包括文件存储空间的管理、目录管理、文件读 / 写操作及访问保护。注意操作系统管理的是文件的物理结构、逻辑结构、目录组织，而不是文件的内容——所以「操作系统管理源程序的内容」是错的。

设备管理：处理用户的 I/O 请求，屏蔽设备差异，方便用户使用各类外设并提高设备利用率。主要包括缓冲管理、设备分配、设备驱动处理和虚拟设备。

（三）用户接口与程序接口

操作系统向上提供的接口分两大类，这一组概念经常被拿来设陷阱。

用户接口面向普通终端用户，供其直接向操作系统发出请求：

- 命令接口：用户通过输入命令组织和控制作业的执行。
  - 联机命令接口（交互式命令接口）：适用于分时系统或实时系统，特点是「说一句，做一句」——用户输入一条命令，系统立即解释并执行。
  - 脱机命令接口（批处理命令接口 / 作业控制语言）：适用于批处理系统，特点是「列好清单，再逐项完成」——用户预先提交一组作业控制命令，系统调度到该作业时自动解释执行，无须实时干预。
- 图形用户接口（**GUI**）：通过窗口、图标、菜单、按钮等图形元素操作计算机，底层仍通过系统调用与内核交互。GUI 程序本身不算操作系统的一部分，但它完成的功能都是通过系统调用实现的。

程序接口面向应用程序，本质上就是一组系统调用。它是应用程序在运行时请求操作系统服务、访问系统资源的唯一合法途径。

用户接口与程序接口不能互换

「操作系统提供给应用程序的接口」是系统调用（程序接口），不是库函数、不是中断、不是原语。库函数由编程语言的函数库提供，属于用户程序的一部分，位于系统调用的上层；中断是硬件与操作系统之间的通信手段，对应用程序透明；原语是操作系统内部不可中断的底层程序，不直接暴露给应用层。

（四）操作系统的四大特征

操作系统之所以区别于其他系统软件，在于它具备四个基本特征：并发、共享、虚拟、异步。

并发（**Concurrency**）：两个或多个事件在同一时间间隔内发生。多道程序环境下，内存中同时驻留若干道程序，操作系统通过调度让它们交替执行，宏观上都在运行、微观上轮流占用 CPU。

必须区分并行（**Parallel**）：两个或多个事件在同一时刻真正同时发生。并行需要硬件支持（多流水线或多处理机 / 多核）。单处理机系统中任一时刻只有一个程序占用处理机，多道程序之间只能并发而不能并行；但处理机与设备、处理机与通道、设备与设备之间相互独立，是可以并行的。多核计算机上，多道程序之间既并发也并行。

共享（**Sharing**）：系统中的资源可供内存中多个并发执行的进程共同使用，有两种方式：

- 互斥共享方式：某些资源虽可供多个进程使用，但一个时间段内只允许一个进程访问。典型例子是打印机、磁带机等物理设备，以及软件中的栈、变量、表格等临界资源。
- 同时共享（同时访问）方式：某些资源允许一个时间段内由多个进程「同时」访问，这里的「同时」往往是宏观上的，微观上各进程交替访问（即分时共享）。典型例子是磁盘设备、用可重入代码编写的文件。

判断方法：若若干进程交替使用某设备对结果没有影响，就允许同时访问（磁盘）；若会产生影响，就必须互斥访问（打印机）。

虚拟（**Virtual**）：把一个物理实体变为若干个逻辑上的对应物。物理实体实际存在，逻辑对应物是用户感受到的。三种典型实现：虚拟处理器（多道程序分时使用一个处理器，时分复用）、虚拟存储器（利用覆盖、交换、请求分页 / 分段 / 段页等技术逻辑扩充容量，空分复用）、虚拟设备（SPOOLing 把一台物理 I/O 设备虚拟为多台逻辑设备）。

异步（**Asynchronism**）：多道程序并发执行、争抢有限的系统资源，因此进程的执行不是一贯到底的，而是走走停停、以不可预知的速度向前推进。可以通过同步机制约束执行顺序、限制最终结果。

并发与共享互为存在条件

这是本章第一条必背结论：

1. 资源共享以程序并发为条件——若系统不允许程序并发执行，同一时间段内只有一个程序在跑，自然不存在资源共享问题；
2. 若系统不能对资源共享实施有效管理，则必将影响程序的并发执行，甚至根本无法并发执行。

另外两条从属关系也要记住：没有并发性就谈不上虚拟性（一个时间段内只运行一道程序，虚拟就没有意义）；只有系统拥有并发性，才有可能导致异步性。所以四个特征中，并发和共享是最基本的两个。

真题考法：并发与并行、多道程序系统

- **2009-23**：单处理机系统中可并行的是（处理机与设备、处理机与通道、设备与设备），进程与进程不能并行。
- **2018-23**：多任务操作系统「具有并发和并行的特点」「需要实现对共享资源的保护」正确；「需要运行在多 CPU 平台上」错误——多任务与硬件是否多核无关。
- **2022-23**：多道程序系统「不必支持虚拟存储管理」正确（早期多道批处理直接把所有进程的数据全部装入内存），「进程数越多 CPU 利用率越高」错误——进程过多会因争抢 CPU 而降低利用率。

二、操作系统发展历程

（一）手工操作阶段

早期用户把程序和数据穿孔在纸带或卡片上，装入输入设备后手动控制读入内存并启动运行；程序的装入、执行、结果输出全过程都依赖人工。这一阶段没有操作系统。两个突出缺点：① 用户独占全机，无资源竞争但资源利用率极低；② **CPU** 长时间空闲，等待人工完成 I/O，计算能力没有发挥出来。

（二）批处理阶段

为解决人机矛盾以及 CPU 与 I/O 设备速度不匹配的矛盾，出现了脱机输入 / 输出技术：引入一台外围机，在 CPU 不参与的情况下预先完成 I/O 准备——输入时把纸带 / 卡片上的数据读入磁带供 CPU 后续调入，输出时 CPU 先把结果写入磁带再由外围机输出。由于 I/O 完全脱离主机控制，故称脱机 I/O；由主机直接管理 I/O 的方式则称联机 I/O。脱机 I/O 显著减少了 CPU 空闲时间。

在脱机 I/O 的基础上，为实现作业的自动连续处理，出现了配备监督程序（**Monitor**）的批处理系统，这是操作系统开始出现的标志。

单道批处理系统：作业成批提交，但内存中始终保持只有一道作业。三个特征：

- 自动性：磁带上的作业可自动连续执行，无须人工干预，缓解了一定程度的人机速度矛盾。
- 顺序性：作业按磁带顺序依次装入内存，先装入者先完成。
- 单道性：内存中仅允许一道用户程序运行，待其完成或异常终止后才调入下一道。

缺点是：当正在运行的作业发起 I/O 请求时，高速 CPU 不得不空闲等待低速 I/O 结束，资源利用率依然很低。

多道批处理系统：每次往内存中读入多道程序，作业成批运行。作业先存放在外存的后备队列中，作业调度程序按一定算法选取若干作业调入内存，使其并发执行、共享系统资源。当某道程序因 **I/O** 请求而暂停运行时，借助中断技术 **CPU** 立即转去运行另一道程序。

多道程序设计技术的三个特点（必背）：

1. 多道：计算机内存中同时存放多道相互独立的程序。
2. 宏观上并行：同时进入系统的多道程序都处于运行过程中——它们先后开始运行，但都还没有运行完毕。
3. 微观上串行：内存中的多道程序轮流占有 CPU，交替执行。

主要优点：多道程序并发执行、共享资源，资源利用率大幅提升，CPU 和其他资源更能保持「忙碌」状态，系统吞吐量增大。主要缺点：用户响应时间长，没有人机交互功能——提交作业后只能等待处理完成，中间不能控制自己的作业（无法调试程序、无法在运行过程中输入参数）。

多道程序设计需要解决：① 处理器的分配策略；② 多道程序的内存分配与保护；③ I/O 设备的分配与调度；④ 大量程序与数据的组织、存储方式及其安全性与一致性。此外，引入多道技术后系统失去了封闭性和顺序性（这是异步性的来源），并且由于需要频繁中断，系统开销比单道批处理更大。

多道 ≠ 并行，也不依赖虚拟存储

- 多道批处理系统的「并行」是宏观并行、微观串行。单 CPU 上多道程序之间没有真正的并行，真正的并行要求多处理机 / 多核。
- 多道程序设计技术与硬件是否多核无关，单核也可以是多道批处理系统。

- 多道程序系统不必支持虚拟存储管理。早期多道批处理把需要执行的所有进程的数据一次性全部引入内存。所以「多道 → 必须有虚拟内存」是错的。
- 多道程序设计并不提高单个作业的处理速度，它提高的是资源利用率和系统吞吐量，代价是牺牲了交互性。

（三）分时操作系统

分时技术把处理器时间划分为很短的时间片，轮流分配给各用户程序使用；若某程序在其时间片内未完成计算，则暂停运行、释放处理器给其他程序，待下一轮再继续。由于计算机运行极快、程序轮转迅速，每个用户都获得「独占计算机」的体验。

分时操作系统允许多个用户通过终端同时连接一台主机并与系统交互，彼此互不干扰。它基于多道程序设计，但与多道批处理系统有本质区别：批处理追求高吞吐量和资源利用率，无须人工干预；分时以人机交互为核心目标。四个主要特征：

1. 同时性（多路性）：允许多个终端用户同时使用同一台计算机。
2. 交互性：用户可通过终端以人机对话方式直接控制程序运行并与之交互。
3. 独立性：各用户的操作相互隔离，任一用户都感觉系统为其独占。
4. 及时性：用户请求能在较短时间内获得响应。

主要缺点：不能优先处理一些紧急任务——系统对各个用户 / 作业完全公平，循环服务一个时间片，不区分任务紧急性。

（四）实时操作系统

实时操作系统是为在严格的时间限制内完成关键任务而设计的系统，它摒弃分时的时间片轮转机制，改用基于任务紧迫性或截止时间的调度策略，确保高优先级任务及时获得处理器；主要特点是及时性和可靠性（确定性、高可靠性、强时效性）。

- 硬实时系统：必须保证某个特定动作在规定时间内完成，否则可能引发灾难性后果（飞机的控制系统、导弹制导系统）。
- 软实时系统：允许偶尔违反时间规定，只要不引起永久性损害即可（飞机订票系统、银行管理系统）。

（五）网络操作系统、分布式系统与微机操作系统

网络操作系统在单机操作系统基础上扩展网络功能，支持计算机之间的通信、数据传输与资源共享（如文件、打印机共享）。各计算机保持独立运行，用户需显式指定远程资源的位置才能访问，系统不提供统一的全局视图。

分布式系统由多台计算机组成，节点地位对等、无固定主从关系，资源可被所有用户透明共享，可扩展性与容错能力强，能把一个任务分解为多个子任务分布到不同节点并行执行、协同完成。管理这类系统的操作系统称分布式操作系统，核心特点是分布性、并行性和透明性，其中透明性最关键。

二者区别：网络操作系统仅实现资源共享与通信，用户需主动干预远程操作；分布式操作系统通过协同机制共同完成同一任务，向用户提供单一系统映像。

微机操作系统按用户数与任务并发能力分三类：单用户单任务（早期的 MS-DOS）、单用户多任务（Windows XP 及后续版本、macOS、现代 Linux 桌面版）、多用户多任务（UNIX、Linux 服务器版、Windows Server 系列）。

（六）各类操作系统对比

阶段	核心思想	主要优点	主要缺点	是否支持交互
手工操作	人工装入、人工控制	无资源竞争	用户独占全机，CPU 利用率极低	有人工干预，无系统交互
单道批处理	脱机 I/O + 监督程序自动逐个运行作业	自动性、顺序性，缓解人机速度矛盾	单道性，I/O 时 CPU 空闲，利用率低	无
多道批处理	内存驻留多道程序，I/O 时切换	资源利用率高，吞吐量大	响应时间长，无交互能力，开销大	无
分时系统	时间片轮转，多用户共享一台主机	及时响应，解决人机交互，各用户独立	对紧急任务不能优先处理	有（核心目标）
实时系统	按紧迫性 / 截止时间调度，抢占式	及时性、可靠性强，可优先响应紧急任务	交互流畅性不如分时，开销大	有，但面向特定任务
网络操作系统	互连各机，实现通信与资源共享	扩大资源共享范围	需显式指定远程资源，无全局视图	有
分布式系统	节点对等，协同完成同一任务	分布性、并行性、透明性、可扩展	实现复杂，一致性维护难	有

各操作系统对比的四条排序

资源利用率：多道批处理 > 分时 > 实时；交互性：分时 > 实时 > 批处理；及时性：实时 > 分时 > 批处理；可靠性：实时系统最强。

真题考法：多道批处理系统的特点

- **2016-23**：「批处理系统允许多个用户与计算机直接交互」错误（批处理没有交互能力）；「批处理系统分为单道和多道」「中断技术使多道批处理系统的 I/O 设备可与 CPU 并行工作」正确。
- **2017-28**：多道程序系统的优点是 CPU 利用率高、系统吞吐量大、I/O 设备利用率高；系统开销小是错的——组织作业和切换作业都要额外开销。
- **2022-23**：见前文「并发与共享」处的考法说明。

三、程序运行环境

（一）CPU 运行模式：内核模式与用户模式

*1.*内核与内核态

为确保操作系统正确运行，必须严格区分操作系统代码与用户程序的执行环境。绝大多数计算机系统通过硬件支持实现这一区分：为处理机设置至少两种独立的运行模式——用户态（目态）和内核态（管态、系统态、核心态）。

硬件通过程序状态字寄存器（**PSW**）中的一个模式位标识当前运行模式。不同教材约定可能相反，统考按「**0** 表示内核态，**1** 表示用户态」理解（也有资料写 1 表示内核态，题目若给出约定以题目为准）。系统引导时硬件从内核态开始工作，完成操作系统加载后才切换到用户态执行用户程序。

与两种模式对应，内存空间也分两块：内核空间是操作系统程序和数据占用的内存空间，只有内核态下才可以访问；用户空间是用户代码占用的内存空间，用户态下可以访问。CPU 访问内存时会

进行特权级检查，只有当前特权级高于被访问数据的特权级时才能访问。

操作系统内核是紧靠硬件、常驻内存的核心软件，把与硬件紧密相关的模块（中断处理程序、设备驱动程序）、运行频率较高的模块（时钟管理、进程调度）以及各模块共用的基础操作集中放置，以保护核心模块免受应用程序破坏并提升运行效率。内核功能分两类：

1. 支撑功能：时钟管理（提供系统时间、产生定时中断，用于时间片轮转、实时任务的截止时间控制、批处理作业进度衡量）、中断机制（保存与恢复现场、转移控制权到处理程序）、原语（由若干条指令组成、完成特定功能、具有原子性即不可被中断的操作，如进程切换、进程同步、资源分配，常通过关中断保证原子性）。
2. 系统资源管理功能：进程管理、存储器管理、设备管理，以及支持它们的内核数据结构（PCB、设备控制块、内存分配表、缓冲区、各类链表和消息队列等）及基本操作。

*2.*特权指令与非特权指令

双重模式机制通过指令分类提供保护：

- 特权指令：仅能在内核态执行，可访问全部地址空间（包括用户空间和内核空间），执行关键操作。包括：I/O 指令（输入 / 输出指令）、开 / 关中断指令、置时钟指令、设置中断屏蔽字、修改 PSW（模式位）、存取用于内存保护的寄存器（如页表基址寄存器）、修改中断向量表、停机 / 重启指令，以及由内核态转为用户态的操作本身。
- 非特权指令：可在任意模式下执行，主要用于一般计算和数据处理，其内存访问被限制在用户地址空间，无法直接操作硬件或系统核心资源。例如：算术逻辑运算指令（加法、取非）、数据传送指令、跳转 / 分支指令、压栈指令、设置断点指令、陷入指令（**trap**）。

上述限制由硬件强制实施：若用户程序试图执行特权指令，硬件将产生非法指令异常，操作系统捕获该异常后终止该程序并重新调度。

关于模式的四条高频错误说法

1.「CPU 处于内核态时只能执行特权指令」——错。内核态既能执行特权指令，也能执行非特权指令。
2.「CPU 处于用户态时只能执行非特权指令」——对（2022-27 的答案）。
3.「从用户态转到内核态是靠一条指令修改 PSW 的模式位完成的」——错。修改 PSW 的指令本身是特权指令，用户态根本执行不了。用户态到内核态是在响应中断或异常时由硬件自动完成的。
4.「中断处理程序负责把 CPU 从用户态转为内核态」——错。模式切换发生在中断响应阶段（硬件完成），中断处理程序开始执行时 CPU 已经处于内核态了。

*3.*状态切换的时机与方式

用户态 → 内核态（由硬件在中断 / 异常响应阶段自动完成，共四条途径）：

1. 用户程序请求操作系统服务，执行系统调用（主动）。
2. 发生外中断（如 I/O 完成、时钟信号到来）。
3. 用户程序执行过程中产生异常（如除零、缺页）。
4. 用户程序试图执行特权指令，触发非法指令异常。

内核态 → 用户态：由内核通过特定的返回指令（如 x86 的 `IRET`）完成，该操作在内核态执行。

两条「发生」与「执行」的区分

- 2012-23：问「不可能在用户态发生的事件」，答案是进程切换。系统调用在用户态发生（用户程序发起）、

外部中断和缺页也都可能在 CPU 执行用户程序时发生；只有进程切换必须由内核完成，用户态不可能发生。

- 2024-23：问「中断或异常发生时 CPU 处于内核态」，这是错的——事件发生的那一瞬间 CPU 可能还在用户态，只有响应中断的过程中硬件才切换到内核态。
- 因此做题时务必分清「事件发生时的模式」与「处理程序执行时的模式」，以及「在用户态发生 / 执行」两种不同问法。

内核栈与用户栈：每个进程都有用户栈和内核栈。用户栈运行在用户态，存放用户程序的函数局部变量、参数、返回地址，仅在用户态下被访问。内核栈运行在内核态，当进程通过系统调用、中断或异常进入内核态时，硬件自动切换到内核栈，它存放内核函数的调用链、临时变量及内核执行上下文。

每个进程都需要独立的内核栈：所有进程运行时都可能通过系统调用陷入内核态，若第一个进程陷入内核后主动让出 CPU，第二个进程再陷入内核时若共享内核栈，就会破坏第一个进程的栈数据。

普通函数调用不需要保存 **PSW**，因为全程发生在用户态、不涉及特权级切换；而允许多重中断时 **PSW** 必须保存，因为中断可能在用户态也可能在内核态响应，需要记录被中断时的模式以便正确返回。

真题考法：用户态 / 内核态

- **2011-24**：在用户态执行的是命令解释程序（shell）；进程调度程序、缺页处理程序、时钟中断处理程序都在内核态。
- **2013-28**：会导致用户态切到内核态的是整数除以零（异常）和 **read** 系统调用；`sin()` 函数调用不会（纯计算库函数全程用户态）。
- **2014-25**：不能在用户态执行的是关中断指令；trap 指令、跳转指令、压栈指令都可以在用户态执行。
- **2015-24**：不可能导致用户态转内核态的是 `NOT R0`（寄存器取非，不访存、不触发调用）；`DIV`（可能除零）、`INT n`（软中断）、`MOV`（访存可能缺页或越界）都可能。
- **2021-23**：只能在内核态执行的是 **I/O** 指令；trap、数据传送、设置断点指令都可在用户态。
- **2022-27**：CPU 处于用户态时只能执行非特权指令。
- **2023-26**：会导致 CPU 从内核态转为用户态的是执行系统调用（服务例程执行完毕返回时）；阻塞进程、执行 CPU 调度、唤醒进程完成时仍在内核态。

（二）中断和异常的处理

引入内核态与用户态后，操作系统必须解决两者之间的安全切换问题：系统在内核中设立若干称为「门」的受控入口，而用户程序进入内核态的唯一合法途径就是中断或异常。当 CPU 运行用户程序时一旦发生中断或异常，硬件会自动把处理器切换到内核态并跳转到对应处理程序，这一机制由硬件强制保障。

中断机制对现代操作系统至关重要：当某程序因等待 I/O 等事件暂时无法继续执行时，系统需及时将其挂起、转而调度其他就绪程序。在抢占式多任务系统中，这一调度主要依赖中断（尤其是时钟中断）触发。中断机制是多道程序设计得以实现的前提。

1. 中断与异常的定义

- 中断（**Interrupt**），也称外中断：来自 CPU 执行指令外部的事件，通常用于信息输入 / 输出。例如设备发出的 I/O 结束中断（表示 I/O 操作已完成）、时钟中断（表示一个固定时间片已到，用于计时、进程调度或启动定时任务）。外中断与当前执行指令无关，只发生在指令执行结束后。
- 异常（**Exception**），也称内中断：来自 CPU 执行指令内部的事件，如程序的非法操作码、地址越界、运算溢出、虚存系统的缺页，以及专门的陷入指令（系统调用）等引起的事件。异常与

当前执行指令相关，一旦发生就必须立即处理。

*2.*中断与异常的分类

外中断（硬件中断）分为两类：

- 可屏蔽中断：通过 **INTR** 线发出的中断请求，通过改变屏蔽字可以实现多重中断，使中断处理更灵活（如时钟中断、I/O 中断）。
- 不可屏蔽中断：通过 **NMI** 线发出的中断请求，通常是紧急的硬件故障，如电源掉电。

异常（内中断）分为三类：

- 故障（**Fault**）：由指令执行引起的异常，如非法操作码、缺页故障、除数为 0、运算溢出、存储保护错。故障处理后返回第 **i** 条指令重新执行（例如缺页处理完成后重访原内存地址）。
- 自陷（**Trap**，也称陷入）：一种事先安排的「异常」事件，用于在用户态下调用操作系统内核程序，如条件陷阱指令、系统调用指令。它返回下一条指令（第 i+1 条）。
- 终止（**Abort**）：出现了使 CPU 无法继续执行的硬件故障，如控制器出错、存储器校验错。这类错误一般无法修复，处理程序只能强制退出程序。

从软硬件归属看：故障和自陷属于软件中断（程序性异常），终止和外中断属于硬件中断。

几个容易混淆的归类

- 时钟中断属于外中断（来自 CPU 外部的事件），不是内中断。
- 缺页属于内中断中的故障（**Fault**），不是外中断；缺页处理（调页、页面置换）只在核心态完成。
- 系统调用（访管指令 / **trap** 指令引发）属于内中断中的自陷，也叫访管中断。所以「系统调用属于内中断」是正确的说法。
- 异常不能被屏蔽——包括故障、自陷、终止，以及外中断里的不可屏蔽中断。
- 中断判优时，异常和不可屏蔽中断不受中断屏蔽字影响。

*3.*中断处理流程

以 CPU 正在执行用户程序的第 i 条指令为例，大致过程如下：

1. 检测与判优：CPU 在指令执行过程中发现异常，或在指令执行结束后检测到中断请求信号，并对同时到达的多个请求做优先级裁决。
2. 检查是否可响应：检查 EFLAGS 寄存器的 IF 标志，判断能否响应可屏蔽中断。
3. 查中断向量表：硬件根据中断类型号找到对应的中断向量，其中保存该处理程序的入口地址。
4. 特权级校验与切栈：若当前是用户态，硬件自动改模式位、转入内核态，并把栈指针从用户栈切到内核栈。
5. 保护现场：硬件在中断隐指令阶段自动保存断点（**PC**）和程序状态字（**PSW**）压入内核栈；操作系统软件保存通用寄存器与通用地址寄存器的内容以及中断屏蔽字（每位对应一种中断类型，用于屏蔽同级或低级中断、防止同类型中断嵌套，是按类型屏蔽而非全局关中断）。
6. 执行中断处理程序。
7. 恢复现场并返回：从内核栈弹出软件保存的寄存器与用户栈指针，执行 `IRET` 指令，由硬件自动弹出 EIP（PC）、CS、EFLAGS，恢复用户态执行流。

关于断点：异常在第 i 条指令执行过程中被发现，处理后返回第 i 条指令重新执行（如缺页处理后重访原地址）；中断在指令执行结束后被发现，处理后返回第 $i+1$ 条指令继续执行。若判定为不可恢复的致命错误，则终止该用户程序。

中断向量表：本质是一个数组，索引是中断类型号（连续整数编号），表项是中断服务程序的入口地址，由操作系统初始化。地址关系为：中断向量基址寄存器 + 中断信号偏移量 = 中断向量的地址。用数组实现是因为它需要随机访问且编号连续，能按索引在 $O(1)$ 时间内定位。

真题考法：中断处理

- **2009-22**：能引起外部中断的事件是键盘输入；除数为 0、浮点运算下溢、访存缺页都属于内部异常。
- **2012-24**：中断处理一定会保存而子程序调用不需要保存的是程序状态字寄存器。
- **2015-23**：处理外部中断时应该由操作系统保存的是通用寄存器的内容；PC 和 PSW 由硬件自动保存，TLB 和 Cache 由硬件透明管理。
- **2018-29**：时钟中断服务程序要更新「内核中时钟变量的值」「当前进程占用 CPU 的时间」「当前进程在时间片内的剩余执行时间」三项。
- **2020-27**：由操作系统完成的是「提供中断服务」「初始化中断向量表」「保存中断屏蔽字」；「保存被中断程序的断点」由硬件完成。
- **2022-31**：由操作系统完成的是「保存通用寄存器的内容」和「执行系统调用服务程序」；「保存断点和 PSW」「将 CPU 模式改为内核态」由硬件完成。
- **2023-24**：中断向量表适合采用的数据结构是数组。

*4.*中断处理与子程序调用的区别

比较项	中断处理	子程序调用
程序间关系	中断处理程序与被中断程序相互独立，无固定关联	子程序与主程序属于同一程序，是主从关系
触发方式	通常随机发生	由 CALL 指令显式触发，程序员事先安排
实现基础	需要硬件电路支持才能完成	完全由软件实现
入口地址来源	由中断向量表提供	由 CALL 指令的地址码指定
现场保护	中断隐指令（硬件）自动保存 PC、PSW；OS 保存通用寄存器	CALL 指令把当前 PC 压栈；不保存 PSW
优先级裁决	需对同时到达的多个请求判优	无此操作
执行时的特权级	被调用过程（系统调用）一定在内核态	可能用户态也可能内核态

（三）系统调用

*1.*系统调用的定义与功能

系统调用（**System Call**）是操作系统提供给应用程序（程序员）使用的接口，可视为一种特殊的公共子程序调用，是应用程序主动向操作系统发出的服务请求。

系统中各种共享资源均由操作系统内核统一管理，因此凡是与共享资源有关的操作（存储分配、**I/O** 传输、文件管理等）都必须通过系统调用向内核提出服务请求，由内核代为完成，再把结果返回应用程序——目的是保证系统的稳定性和安全性，防止用户程序进行非法操作。

一个操作系统提供的系统调用有几十条乃至上百条，每个系统调用都有唯一的系统调用号。按功能大致分为六类：

功能类别	具体内容
设备管理	设备的请求或释放、设备启动等
文件管理	文件的读、写、创建及删除等
进程控制	进程的创建、撤销、阻塞及唤醒等
进程通信	进程之间的消息传递或信号传递等
内存管理	内存的分配、回收，获取作业占用内存区的大小和起始地址等
系统控制	时钟设置、系统状态查询等

判断某个操作是否「通过系统调用完成」的诀窍是站在程序员的角度想：这件事是不是我（应用程序）主动请求的?

哪些操作必须通过系统调用，哪些不必

判断诀窍是站在程序员角度想：这件事是不是我（应用程序）主动请求的?

- 需要：创建新进程（`fork`）、打开 / 读写文件（`open`、`read`、`write`）、申请动态内存（`malloc` 空间不够时可能调用 `brk` / `mmap`）、`sleep` 睡眠、设备启动。
- 不需要：页置换（由缺页异常触发，是操作系统内部机制）、进程调度（由时钟中断或阻塞 / 唤醒事件触发）、生成随机整数（一般由用户态库函数 `rand()` 算得）、纯计算类库函数（`strlen`、`sqrt`、`sin`）。
- 陷阱：`printf` 最终会通过 `write` 输出数据，`malloc` 在小块分配时可能完全在用户态完成，所以「库函数一定不含系统调用」也是错的。

2. 系统调用与库函数的区别

比较项	系统调用	库函数
提供者	操作系统内核	编程语言的函数库
运行特权级	内核态（调用者发起时在用户态）	用户态
是否切换特权级	必须切换（用户态 → 内核态 → 用户态）	一般不切换
与共享资源的关系	凡涉及共享资源的操作必须使用	有些封装了系统调用，有些完全不含系统调用
是否可移植	不同操作系统接口不同（Windows 的 Win32 API 与 UNIX/Linux 的 POSIX 标准不同）	依赖于具体语言的运行库

因此：「不同的操作系统为应用程序提供了统一的系统调用接口」是错的——不同操作系统的内核架构不同，系统调用编号、语义和参数形式都可能不同，较统一的只是上层 API 标准。操作系统真正提供给应用程序的接口本质上仍是系统调用，库函数只是一层封装。

系统调用与普通过程调用的四点差异：

1. 运行状态不同：普通过程调用的调用者与被调用者运行在同一特权级；系统调用的调用者在用户态、被调用者在内核态。
2. 状态转换不同：普通过程调用无须特权级切换、可直接跳转；系统调用必须通过陷入指令（**Trap**）触发，先切至内核态再由内核分派。
3. 返回机制不同：系统调用执行完毕后，内核可能根据调度策略决定是否切换进程；普通过程调用总是直接返回原调用点。

4. 嵌套调用限制不同：系统调用的嵌套深度受内核栈空间限制；普通过程调用仅受用户栈大小约束。

3. 系统调用的执行过程

系统调用的四个阶段（必背）

① 传递系统调用参数 → ② 执行陷入指令（**trap**）并切换至内核态 → ③ 定位并执行相应的服务程序 → ④ 恢复现场并返回用户态

对应 2017-24 的答案是 ③→②→④→①。

1. 传递参数：用户程序把系统调用号及参数压入堆栈或放入指定的通用寄存器。
2. 执行陷入指令：该指令在用户态执行，执行后立即引发一个内中断（访管中断），硬件把 CPU 从用户态切到内核态；硬件把 PC、PSW 压入内核栈，内核（软件）保存通用寄存器内容。
3. 定位并执行服务程序：内核按系统调用号查询系统调用表（数组，表项为各服务例程的入口地址），执行服务逻辑。
4. 恢复现场并返回：内核恢复被中断进程的 CPU 现场，把模式切回用户态，控制权交还用户程序，从断点继续执行。

几个必须记住的细节：

- 陷入指令 = **trap** 指令 = 访管指令 = 自陷指令 = 陷阱指令，它是非特权指令，在用户态执行。
- 发出系统调用请求在用户态，对系统调用的响应处理在核心态。
- 系统调用号标识系统调用的类型；中断类型号（中断向量号）标识中断或异常的类型，两者不能混。
- 自陷指令触发异常时保存的断点是下一条指令的地址（若保存的还是该指令的地址，就会陷入死循环）。
- 由硬件完成：保存断点和 **PSW**、把 **CPU** 模式改为内核态；由操作系统完成：保存通用寄存器内容、执行系统调用服务程序。

```
/* 用户态发起系统调用的直观过程（x86 Linux 早期实现示意） */
/* 1. 传递参数：系统调用号放 eax，参数依次放 ebx、ecx、edx */
/* 2. 执行陷入指令（用户态执行），引发访管中断，CPU 转内核态 */
/* 3. 内核查系统调用表，执行对应的服务例程 */
/* 4. 返回用户态，从断点继续执行，返回值在 eax 中 */

int fd = open("data.txt", 0); /* 库函数封装，内部最终是一条 int 0x80 / syscall */
```

真题考法：系统调用

- **2010-23**：操作系统提供给应用程序的接口是系统调用。
- **2017-24**：执行系统调用过程的正确顺序是传递参数 → 陷入指令 → 执行服务程序 → 返回用户态。
- **2019-25**：「执行系统调用服务程序时 CPU 处于内核态」「操作系统通过提供系统调用避免用户程序直接访问外设」「系统调用是内核为应用程序提供服务的接口」正确；「不同操作系统提供统一的系统调用接口」错误。
- **2021-32**：通过系统调用完成的操作是创建新进程；页置换、进程调度不是应用程序发起的，生成随机整数一般无需系统调用。
- **2022-31**：见前文（由硬件完成的 vs 由操作系统完成的）。
- **2024-23**：每个系统调用都有对应的内核服务例程，系统添加新类型设备时需注册相应的中断服务例程，均正确。

（四）程序的链接与装入

*1.*从源程序到可执行程序的三步

把一个用户源程序变成可在内存中执行的程序，通常要经过三步：

1. 编译：由编译程序把用户源代码翻译成若干目标模块。
2. 链接：由链接程序把这些目标模块及其所需的库函数合并，形成一个完整的装入模块（可执行文件）。
3. 装入（加载）：由装入程序把装入模块载入内存，并启动其执行。

*2.*逻辑地址与物理地址

编译后的每个目标模块通常从 0 号单元开始编址，这种地址称为该模块的相对地址（逻辑地址）。链接程序把多个目标模块合并为完整的可执行程序时，依次把各模块的相对地址拼接成一个统一的、从 0 开始编址的逻辑地址空间（虚拟地址空间）。以 32 位系统为例，该逻辑地址空间的范围为 $0 \sim 2^{32}-1$。

进程运行时使用的地址都是逻辑地址：用户程序和程序员只需关注逻辑地址，底层的内存管理机制对其完全透明。不同进程可以拥有相同的逻辑地址，因为它们各自拥有独立的逻辑地址空间，这些地址会被映射到主存的不同位置。

物理地址空间是主存中所有物理存储单元的集合，是地址转换的最终目标——无论执行指令还是访问数据，最终都必须通过物理地址从主存读写。早期系统在程序装入内存时把逻辑地址转换为物理地址，这一过程称地址重定位；现代支持虚拟内存的系统则由内存管理单元（**MMU**）在硬件层面于运行时自动完成：操作系统为每个进程维护一张页表记录逻辑页到物理页框的映射，MMU 依页表动态换算，对用户程序完全透明。

*3.*三种链接方式

静态链接：在程序运行之前，把各目标模块及其所需的库函数链接成一个完整的装入模块，此后不再拆分。链接过程中需完成两项关键操作：

- 地址调整：各目标模块编译后均以 0 为起始地址，链接时需根据其在装入模块中的实际位置统一加上偏移量。例如模块 A 长度为 L、模块 B 紧邻其后存放，则原本从 0 编址的模块 B 在装入模块中从地址 L 处开始，其内部所有地址都要加上 L。
- 外部符号解析：把模块间引用的外部符号（如函数名）替换为确定的地址。若模块 A 中含语句 `CALL B`（B 是模块 B 入口的外部符号），链接程序会把它解析为模块 B 在装入模块中的起始地址 L。

装入时动态链接：编译后生成的一组目标模块在装入内存时采用边装入边链接的方式。装入某个模块时若遇到对外部模块的调用，装入程序立即查找并装入相应的外部目标模块，并替换其中的外部符号。优点是便于修改和更新（只需替换被修改的模块，无须重建整个程序）、便于实现对目标模块的共享。

运行时动态链接：对装入时动态链接的改进。在程序执行过程中，仅当需要调用某个尚未装入内存的目标模块时，操作系统才动态加载并链接该模块；运行中未使用的模块既不会被装入内存，也不会进行链接。优点是既能加快程序装入过程，又能节省内存空间。

链接方式	链接时机	优点	缺点
静态链接	程序运行之前	装入后运行无额外开销	模块修改需重新生成整个装入模块；无法共享目标模块
装入时动态链接	装入内存时边装入边链接	便于修改更新；便于共享目标模块	仍要把用到的模块全部装入，未用到的模块也占空间
运行时动态链接	执行过程中调用到时才链接	加快装入；节省内存，未用模块不装入	实现复杂，运行时才有链接开销

*4.*三种装入方式

绝对装入：仅适用于单道程序环境。内存中始终只有一个程序，其驻留位置在编译前即可确定，编译程序直接生成使用绝对地址的目标代码，装入程序按这些地址把程序和数据载入内存；此时逻辑地址与物理地址完全一致，无须任何修改。程序中的符号地址由编译器或汇编器在编译 / 汇编阶段转换为绝对地址；绝对地址也可由程序员直接指定，但缺乏灵活性。

可重定位装入（静态重定位）：多道程序环境下编译程序无法预知模块在内存中的位置，因此装入模块通常从地址 0 开始编址，程序中的地址都是相对于模块起始位置的逻辑地址。装入时系统根据内存空闲情况为它分配一块连续的内存区域，把整个模块载入，随后一次性把所有逻辑地址改为对应的物理地址。转换在装入时完成且运行期间不再改变，故称静态重定位。缺点是：进程必须在装入前一次性获得所需的全部内存空间，内存不足则无法装入；一旦装入，进程运行期间既不能移动位置，也不能动态申请额外内存。

动态运行时装入（动态重定位）：装入程序把模块载入内存后并不立即转换地址，而是把转换推迟到指令实际执行时进行：系统用重定位寄存器存放该模块在内存中的起始物理地址，程序中保留逻辑地址，物理地址由硬件在运行时计算（逻辑地址 + 重定位寄存器内容）。优点是支持程序在内存中移动，便于操作系统进行紧凑（合并分散的空闲块）、有效减少外部碎片，并允许在运行期间动态调整进程的内存布局。

装入方式	地址转换时机	适用环境	能否在内存中移动	典型配合
绝对装入	编译（汇编）时	单道程序环境	否	无
可重定位装入（静态重定位）	装入时一次性完成	多道程序环境	否	固定分区、单一连续分配
动态运行时装入（动态重定位）	指令执行时由硬件完成	多道程序环境	是	页式、段式等需移动进程的方案

重定位三问

1. 动态重定位依赖硬件支持：必须有重定位寄存器（基址寄存器）和地址变换机构，所以「动态重定位依赖目标程序本身」是错的。
2. 静态重定位不能移动进程：它把地址一次性改死，进程一旦装入就固定，所以「页式存储管理可以采用静态重定位」是错的（页式需要移动和换出换入）。
3. 固定分区可以采用静态重定位：作业装入后位置不变且占连续空间，正适合静态重定位。

（五）程序运行时的内存映像与地址空间

程序被加载运行时，操作系统为它建立一个虚拟地址空间，该空间的逻辑布局称为进程的内存映像，反映运行时代码、数据、堆、栈等部分的组织方式。与磁盘上的静态可执行文件不同，内存映像是动态的：虚拟内存机制下进程的整个地址空间无须全部驻留物理内存，只有在实际访问某一部分时才通过缺页中断把它从磁盘调入。

典型的进程虚拟地址空间从低地址到高地址依次为：

1. 只读代码段：含.init（启动时自动调用的初始化代码）、.text（主程序及自定义函数编译后的机器指令）、.rodata（只读数据，如字符串常量）。代码段只读，防止运行时意外修改指令，同时允许多个进程共享同一份代码以节省内存。
2. 读 / 写数据段：存放全局变量和静态变量。.data 存放已初始化的（如 int x = 5;）；.bss 存放未初始化的（如 int y;）。
3. 堆：运行时动态申请内存的区域，用 malloc() 申请、free() 释放。堆从低地址向高地址增长，大小不固定。
4. 共享库的存储映射区：提供进程依赖的公共函数代码（如 printf()），运行时动态映射到进程地址空间，只读且可被多个进程共享。
5. 用户栈：支持函数调用，存放局部变量、函数参数、返回地址等临时信息，由系统自动管理，遵循后进先出（**LIFO**）。栈从高地址向低地址扩展。

32 位 Linux 的典型布局中，0xFFFFFFFF 附近是操作系统内核区（对用户进程不可见），往下依次是用户栈、共享库映射区、堆、读 / 写数据段、只读代码段，最低处是未使用区。注意代码段和数据段的大小在程序加载时就已确定，而堆和栈可以在运行过程中动态扩展和收缩。

内存映像的三条必背结论

1. 增长方向：堆从低地址向高地址增长，栈从高地址向低地址增长，两者相向而生，中间是共享库映射区。
2. .bss 段存放未初始化的全局 / 静态变量，它在可执行文件中不占实际磁盘空间（只需记录大小），但在内存映像中占位。
3. **PCB** 不属于进程的虚拟地址空间：虽然 PCB 是管理进程的核心数据结构，但它由操作系统内核维护、存放在系统内核区中，对用户进程不可见。

```
/* 各区域存放内容对照示例 */
int g_init = 5;       /* .data : 已初始化的全局变量 */
int g_uninit;         /* .bss  : 未初始化的全局变量 */
const char *s = "hi";/* .rodata: 字符串常量（只读数据段） */

int main(void) {
    static int s_init = 1;  /* .data: 已初始化的静态变量 */
    int local = 3;          /* 栈    : 局部变量 */
    int *p = malloc(4);     /* 堆    : 动态申请，返回的地址交给指针 p */
    free(p);
    return 0;
}
```

真题考法

- 进程虚拟地址空间中各区域的存放内容（含.text、.data、.bss、堆、栈的归属判断）是近年新出现且反复出现的角度，要求看到一段 C 代码能立刻指出每个变量落在哪个区。
- 「代码段和数据段大小在加载时确定、堆栈可动态伸缩」也是常见判断点。

四、操作系统结构

随着操作系统功能不断扩展、代码规模持续增长，合理的结构设计对降低系统复杂度、提升安全性和可靠性至关重要。考纲要求掌握五种：分层、模块化、宏内核、微内核、外核。

（一）分层结构

分层法把操作系统划分为若干层，最底层（层 **0**）为硬件，最高层（层 **N**）为用户接口，每层只能调用紧邻它的下层所提供的功能和服务（单向依赖）。

优点：① 便于调试与验证，系统可自底向上逐层开发与测试——第 1 层完成且验证无误后再开发第 2 层，因为它只依赖已验证的下层；若某层出错，可把错误定位在该层内部，因为它的所有下层都已通过验证。② 易于扩充与维护，在保持层间接口不变的前提下，可以独立修改、替换或新增某一层的实现而不影响其他层。

问题：① 层次划分困难，合理界定各层的功能边界较复杂，且一旦依赖关系固化，系统架构可能缺乏灵活性。② 运行效率较低，执行一个高层功能通常需自上而下穿越多个中间层，每层之间的参数传递与控制转移都会引入额外开销。

分层的两条秒杀结论

看到「分层式架构」就要立刻想到：单向依赖（各层只能调用紧邻下层）与便于调试与验证 / 结构清晰。反过来，「具有非常灵活的依赖关系」必定是错误选项——分层法恰恰是牺牲灵活性换取清晰性。

（二）模块化结构

模块化把操作系统按功能划分为若干具有一定独立性的模块，每个模块实现特定的管理功能，并明确定义与其他模块之间的接口，通过接口通信；还可以进一步把模块细分为子模块，同样规定子模块之间的接口。这种方法称模块-接口法。

模块划分的粒度需要权衡：模块过小虽然降低了单个模块的复杂性，但会导致模块间交互频繁、增加系统整体复杂度；模块过大则模块内部结构复杂、难以维护。

模块的独立性至关重要——模块独立性越高、相互依赖越少，系统结构越清晰。衡量模块独立性的两个标准：

- 内聚性：模块内部各组成部分之间联系的紧密程度。内聚性越高，模块独立性越好。
- 耦合度：模块之间相互依赖和影响的程度。耦合度越低，模块独立性越好。

优点：① 提高了操作系统设计的正确性、可理解性和可维护性；② 增强了系统的可适应性，便于功能扩展；③ 支持并行开发，加速操作系统研制进程。

缺点：① 模块间的接口规定很难满足对接口的实际需求；② 各模块设计者齐头并进，每个决定无法建立在上一已验证的正确决定的基础上，因此无法找到一个可靠的决定顺序（即更难调试和验证）。

（三）宏内核

宏内核（也称单内核、大内核）把系统的主要功能模块集成在一个紧密耦合的整体中，统一运行于内核态，从而为用户程序提供高性能的系统服务。目前主流操作系统如 Linux、Windows、Android、macOS、iOS 都是宏内核或混合内核架构。

优点：性能高。各模块共享地址空间，可直接调用彼此的函数，减少了上下文切换和消息传递的开销，信息共享和相互调用都很方便。

缺点：随着体系结构和应用需求发展，需要操作系统提供的服务日益复杂、代码规模急剧膨胀，导致内核庞大、功能复杂、结构混乱、难以维护；更严重的是，大内核中某个功能模块出错就可能导致整个系统崩溃（例如文件系统运行在内核态，它一出问题系统就直接崩溃），可靠性下降，面临典型的「软件危机」。

需要说明的是，现在的宏内核早已不是当年纯粹的形式，而是广泛吸取微内核架构的优点后揉合而成的混合内核。例如 macOS 和 iOS 采用的 XNU 内核即融合了 Mach 微内核与 BSD 宏内核的特性；谷歌的 Fuchsia 和华为的 HarmonyOS NEXT 则采用了微内核架构。这正说明宏内核与微内核并非对立，而是相互借鉴、融合发展。

（四）微内核

微内核架构把操作系统中最基本的功能保留在内核，其他非核心功能移到用户态执行，以降低内核的设计复杂性。移出的功能组织为若干独立的服务程序（服务器），彼此之间以及与客户进程之间的交互都通过微内核提供的消息传递机制完成。所以微内核结构把操作系统划分为两大部分：微内核 + 多个服务器。

微内核是精心设计的小型内核，仅实现最核心的机制，通常包含：① 与硬件紧密相关的部分；② 基本的资源管理机制；③ 客户与服务器之间的通信支持。它遵循「机制与策略分离」原则——与硬件紧密相关的机制放入微内核，策略交由用户态服务器实现：

微内核中的机制部分	用户态服务器中的策略部分
进程 / 线程的创建、切换、调度队列管理、多处理机同步	进程如何分类、优先级如何确认
低级存储器管理：页表管理、逻辑地址到物理地址的映射等地址变换机制	采用何种页面置换算法、如何进行内存分配与回收
中断和陷入的捕获与分发（识别事件类型后转交服务器）	各类中断 / 异常事件的具体处理逻辑

于是进程管理、存储器管理和 I/O 管理都被一分为二：属于机制的很小一部分在内核，绝大部分在微内核外的各种服务器中实现。大多数服务器都要比微内核大，因此微内核能保持极小规模。

优点（2023 统考真题的四个正确项）：

1. 扩展性和灵活性：新增或修改功能只需调整相应服务器，无须改动内核代码。
2. 可靠性和安全性：只有微内核运行在内核态，其余模块都运行在用户态，一个模块中的错误只会使这个模块崩溃，而不会使整个系统崩溃（文件服务出问题只需强行停止再重启，系统不会崩）；同时内核代码量和特权代码量少，攻击面更小。
3. 可移植性：与 CPU 和 I/O 硬件有关的代码均放在内核中，其他各种服务器与硬件平台无关，因而移植到另一平台所需修改较小。
4. 分布式计算：客户和服务器之间、服务器和服务器之间采用消息传递机制，使微内核系统能很好地支持分布式系统和网络系统。

缺点（主要是性能差）：每次服务请求都需要频繁地在核心态和用户态之间进行切换，操作系统的执行开销偏大；用户态下的各功能模块不能直接相互调用，只能通过内核中的「消息传递」间接通信，性能因此降低。为改善效率，可以把频繁使用的系统服务移回内核，但这又会使微内核的容量明显增大。

尽管宏内核在桌面操作系统中取得了绝对胜利，微内核因其高可靠性和可验证性，在实时、工业控制、航空航天及军事等关键任务领域特别流行。

（五）外核

外核（**exokernel**）的设计思想截然不同：尽可能减少抽象层次，把物理资源直接暴露给用户级系统，自身仅负责资源的安全分配与隔离。

外核是运行于内核态的极小程序，核心职责是把底层硬件资源（磁盘块、内存页、网络包缓冲区等）划分为互不重叠的子集分配给不同的用户级系统（例如一个系统分到磁盘 0 ~ 1023 块、另一个分到 1024 ~ 2047 块），并检查各系统只能访问被授权的资源。应用请求资源时外核只检查是否空闲，空闲就直接交给应用，怎么访问由应用自己决定。外核分离了对硬件的使用与保护，使应用能直接使用未经内核抽象的硬件。其上可在用户空间构建库操作系统（**library OS**）自行实现文件系统、进程调度、虚拟内存管理等传统功能，由于直接操作物理资源、无须经过强制抽象层，避免了不必要的数据拷贝、地址转换与上下文切换开销。

优点：

1. 减少抽象开销（减少了映像层）：传统操作系统强制引入虚拟内存、虚拟文件系统等映射层，每个虚拟机都以为自己的磁盘块号从 0 到最大编号，虚拟机监控程序因而必须维护重映像表；有了外核，这个重映像处理就不需要了，外核只需记录已分配给各虚拟机的资源。
2. 提升灵活性与性能：用户级系统可按应用特性优化资源管理策略（例如数据库直接管理磁盘块），充分发挥硬件潜力。
3. 清晰分离保护与实现：外核仅负责资源保护，功能实现完全置于用户空间，结构简洁且易于验证；它把多道程序（在外核内）与用户操作系统代码（在用户空间内）加以分离，负载不重，因为外核所做的只是保持多个虚拟机彼此不发生冲突。

（六）五种结构对比

结构	核心思想	优点	缺点	代表
分层	分若干层，层 0 是硬件、层 N 是用户接口，各层只能调用紧邻下层（单向依赖）	便于调试验证、结构清晰；易扩充维护	层次划分困难、依赖关系固定不灵活；穿越多层效率低	早期的 THE、Multics
模块化	按功能划分模块，规定接口通信，可再分子模块（模块-接口法）	正确性 / 可理解性 / 可维护性提高；可适应性强；支持并行开发	接口难以设计；各模块决定无先后依赖，难以找到可靠的决定顺序，更难调试验证	多数现代系统采用模块化思想
宏内核	主要功能模块紧密耦合，全部运行在内核态	性能高，模块间可直接调用、信息共享方便	内核庞大复杂、难维护；一个模块出错可能导致整个系统崩溃	Linux、Windows、Android
微内核	只保留最基本功能在内核，其余作为用户态服务器，靠消息传递通信	扩展灵活、可靠安全、可移植、支持分布式	频繁特权级切换与消息传递导致性能差	Minix、Fuchsia、HarmonyOS NEXT
外核	内核只负责资源分配与保护，把物理资源直接交给用户级系统，由库操作系统实现功能	减少抽象与重映像开销、灵活高效、保护与实现分离	用户级系统实现负担重，缺乏统一抽象，应用兼容性差	Exokernel 实验系统

五、操作系统引导

操作系统（如 Windows、Linux 等）以程序形式存储在硬盘等外部存储设备中。操作系统引导是指计算机通过执行一系列引导程序，依次定位启动设备、确定可引导分区、加载其中的操作系统内核，最终完成系统启动的链式过程。

（一）引导的完整流程

1. **CPU** 激活与 **BIOS** 初始化。通电后 CPU 的 `RESET` 引脚产生逻辑值复位 CPU；CPU 唤醒后在 `0xFFFFFFF0` 处执行一条长跳转指令，跳到固化在 ROM 中的启动代码处（即 **BIOS**，基本输入 / 输出系统），把它加载到 RAM 执行。BIOS 中主要存放自诊断程序、**CMOS** 设置程序、系统自动装载程序与主要 I/O 驱动和中断服务；它还在内存最开始处建立临时的中断向量表供引导过程使用。BIOS 针对具体主板设计，与所安装的操作系统无关。
2. 硬件自检（**POST**，**Power-On Self-Test**）与启动设备选择。BIOS 检查硬件能否满足运行的基本条件；有问题则蜂鸣报警、启动中止，无问题则显示 CPU、内存、硬盘等信息。随后 BIOS 依据 **CMOS** 设置（主板上由电池供电、保存启动顺序与系统时间等参数的配置存储器）或 **UEFI** 固件确定启动顺序（**Boot Sequence**）——排在前面的设备优先获得控制权。
3. 加载主引导记录 **MBR**。BIOS 按启动顺序把控制权转交给排在第一位的存储设备，即读取该设备的第一个扇区（最前面的 **512** 字节）并放入内存指定位置（BIOS 的最后一个任务）。若这 512 字节的最后两个字节是 `0x55` 和 `0xAA`（主引导记录签名，标识该磁盘是否为引导盘），该设备就可用于启动；否则控制权转交给启动顺序中的下一个设备。这 512 字节就是主引导记录（**MBR**），作用是告诉计算机到硬盘的哪个位置去找操作系统，由三部分组成：

字节范围	内容
第 1 ~ 446 字节	调用操作系统的机器码（引导代码）
第 447 ~ 510 字节	分区表（Partition Table），共 64 字节，分 4 项、每项 16 字节
第 511 ~ 512 字节	主引导记录签名 `0x55` 和 `0xAA`

每个主分区的 16 个字节中，第 **1** 个字节若为 `0x80` 就表示该主分区是激活分区（活动分区），控制权要转交给它；四个主分区里面只能有一个是激活的。其余字节记录分区的起止物理位置、分区类型、第一个扇区的逻辑地址和扇区总数。扇区总数只有 4 字节、最多表示 2^{32} 个扇区，若每扇区 512 字节则单个分区最大不超过 **2TB**；要使用更大硬盘只能提高每扇区字节数或增加扇区总数（GPT 方案解决的正是这个问题）。

1. 定位活动分区并加载分区引导记录 **PBR**。MBR 中的引导代码扫描分区表找到标记为「活动」的条目，读取其第一个扇区——分区引导记录（**PBR**），有的资料称卷引导记录 **VBR**——加载到内存。PBR 包含该分区特定的引导代码：识别并加载该分区的文件系统驱动（如 FAT32 / NTFS 驱动）、文件系统元数据（簇大小、根目录位置等），以及指向启动管理器文件的指针。
2. 加载启动管理器。按分区引导记录找到并加载活动分区内的启动管理器（`boot loader`），如 Linux 的 **GRUB**、Windows 的 **Bootmgr**，它提供多操作系统或内核版本的选择界面。
3. 加载操作系统内核。启动管理器把用户选定的内核映象加载到内存并跳转执行。内核完全接管系统控制权，重建自己的中断处理机制（替换 BIOS 的实模式中断向量，建立页故障、系统调用、外设中断等机制），不再依赖 **BIOS** 提供的服务，同时启用分页机制、建立内核页表。
4. 内核初始化与用户环境启动。内核初始化内存管理（内核页表、进程地址空间框架）、进程调度（PCB 链表、就绪队列与阻塞队列）和设备驱动子系统，然后启动第一个用户态进程（如 Linux

的 `init`，PID 为 1），由它加载系统各模块直至出现登录界面，启动过程完成。

引导链一句话记忆

CPU 复位 → **ROM** 中的 **BIOS/UEFI** → 硬件自检 **POST** → 按 **CMOS** 启动顺序 → **MBR**（**446** 引导码 + **64** 分区表 + **0x55AA** 签名）→ 活动分区 → **PBR** → 启动管理器（**GRUB/Bootmgr**）→ 加载内核到内存 → 内核接管、重建中断机制、初始化 → 启动 **init**（**PID=1**）

引导过程的易错点

1. **MBR** 只有 **512** 字节，位于磁盘第 **1** 个扇区（**0** 号磁头 **0** 道 **1** 扇区），签名 `0x55` 和 `0xAA` 在最后两个字节。
2. 分区表只有 **64** 字节，只能存 **4** 个主分区项；四个主分区里有且只有一个可以标记为激活。
3. 「计算机开机后操作系统所处的位置」答案是「以上都不对」——先执行 ROM 中的引导程序，把 MBR 中的程序放入内存执行，再由它把操作系统装入内存并移交 CPU 控制权；操作系统本身既不在 ROM 也不在 MBR。
4. 操作系统最终被加载到 **RAM**（不是 BIOS、ROM、EPROM），因为内核需要被 CPU 频繁读写并建立运行时数据结构。
5. 中断向量表的两次建立：BIOS 阶段在内存最低端建立临时中断向量表供引导使用，操作系统初始化阶段重建属于自己的中断向量表，所以「中断向量表需要在操作系统初始化过程中创建」正确；而硬盘分区表在磁盘分区时创建、文件系统根目录和索引节点表在格式化文件系统时创建（2022-24 的干扰项）。
6. 实地址模式：开机后系统首先工作在实地址模式下（Intel 为 80286 及其后处理器提供的 8086 兼容模式，20 位地址空间即 1MB，不支持分页存储管理）；一旦进入保护模式就不再使用 BIOS。BIOS 通过 `INT` 指令调用中断服务程序把内核从磁盘装入内存，例如 `int 0x19` 把启动盘 0 号磁头对应盘面 0 磁道 1 扇区中的引导程序装入内存。

真题考法：操作系统引导

- **2013-29**：计算机开机后操作系统最终被加载到 **RAM**。
- **2022-24**：需要在操作系统初始化过程中创建的是中断向量表。
- **2021**、**2022** 统考均考查过引导的阶段性任务划分（谁把什么加载到哪、哪一步依赖中断机制），要求能把上面的链条按顺序说清。

六、虚拟机

（一）虚拟机与虚拟机管理程序

虚拟机（**Virtual Machine**，**VM**）是指通过虚拟化技术，把一台物理计算机抽象为多台逻辑上独立的计算环境，每台虚拟机都表现为完整硬件系统的精确复制品。虚拟化技术的核心在于隐藏底层物理硬件的细节，向上层提供统一且隔离的执行平台，每台虚拟机都可独立运行一个操作系统。

提供虚拟化功能的程序称虚拟机管理程序，同义术语还有虚拟机监控程序、**Virtual Machine Monitor**（**VMM**）、**Hypervisor**。注意提供虚拟机管理程序并不属于操作系统的任务——第二类虚拟机管理程序本身就是应用程序。两个必须区分的主体：运行在底层硬件上的操作系统称宿主操作系统（**Host OS**），运行在虚拟机中的操作系统称客户操作系统（**Guest OS**）。

（二）第一类虚拟机管理程序（裸金属架构）

第一类虚拟机管理程序直接运行在物理硬件之上，从技术上讲它就像一个操作系统，因为它是系统中唯一处于最高特权级的软件。它具备多道程序功能，向上层提供若干台虚拟机；每台虚拟机都是裸机的精确复制品，因此可以在不同虚拟机上运行任何不同的操作系统。

虚拟机作为用户态的一个进程运行，不允许执行敏感指令；然而虚拟机上的操作系统认为自己运行

在内核态（实际上不是），这种状态称为虚拟内核态。当客户操作系统执行一条仅允许在真实内核态下执行的敏感（特权）指令时，会触发异常并陷入虚拟机管理程序：在支持硬件虚拟化的 CPU（Intel VT-x、AMD-V）上，管理程序能判断指令来自客户操作系统还是用户程序，前者代为完成、后者模拟真实硬件的异常响应；在早期不支持硬件虚拟化的 CPU 上，所有敏感指令被透明重定向到管理程序，由它用软件模拟。

（三）第二类虚拟机管理程序（寄居架构）

第二类虚拟机管理程序以应用程序的形式运行在宿主操作系统（如 Windows 或 Linux）之上，依赖宿主系统进行资源分配与调度，本质上是一个用户态进程；尽管如此，它仍向客户操作系统伪装成完整的硬件平台。**VMware Workstation 是 x86 平台上首个广泛使用的第二类虚拟机管理程序。**

首次启动时，第二类虚拟机管理程序模拟一台新计算机的行为，期望找到的驱动器可以是虚拟设备；用户把客户操作系统安装到虚拟磁盘上（实际上只是宿主操作系统中的一个文件），安装完成后即可正常启动和运行。

虚拟化技术在 Web 主机领域应用广泛：若无虚拟化，服务商只能提供共享托管（用户无法控制软件环境）或独占托管（成本高昂）；借助虚拟化，单台物理服务器可同时运行多个虚拟机，每台虚拟机对用户而言都是一台完整的服务器，支持自定义操作系统与软件，且成本显著降低——这正是当前主流云服务器服务的基础。

（四）两类虚拟机管理程序对比

比较项	第一类（裸金属架构）	第二类（寄居架构）
运行位置	直接运行在硬件之上，唯一处于最高特权级	运行在宿主操作系统之上，本质是用户态进程
资源分配方式	直接控制和分配硬件资源，分配方式类似外核	依赖宿主操作系统分配和调度资源
性能	更好	较差，需要宿主操作系统中介
可支持的虚拟机数量	更多	较少，宿主操作系统自身也消耗资源
运行模式	运行在最高特权级，可执行最高特权指令	部分运行在用户态、部分在内核态，部分特权指令会被 VMM 截获
可迁移性	较差	导出镜像文件即可迁移
典型代表	VMware ESXi、Xen、KVM	VMware Workstation、VirtualBox

真题考法：虚拟机

近年统考考查过虚拟化技术的原理与特点（如两类 VMM 的位置判断、「客户操作系统认为自己处于内核态」这一虚拟内核态的说法是否正确）。答题要点是抓住：第一类 **VMM** 独占最高特权级、第二类 **VMM** 寄居在宿主 **OS** 之上，以及客户操作系统的「内核态」是虚拟的。

本章小结表

考纲条目	核心内容	必须记住的结论
操作系统的概念	控制和管理软硬件资源、合理调度任务与分配资源、为上层提供统一接口与运行环境的最基本系统软件	四层结构：硬件 → 操作系统 → 应用程序 → 用户；裸机 + OS = 扩充机器
双重角色	系统资源的管理者；用户与硬件之间的接口；实现对资源的抽象与扩充	用户接口（命令接口 + GUI）与程序接口（系统调用）不能混淆
四大功能	处理机管理、存储器管理、文件管理、设备管理	处理机管理本质上是进程管理；OS 管文件的组织方式而非内容
四大特征	并发、共享、虚拟、异步	并发与共享互为存在条件；没有并发就谈不上虚拟；并发才能导致异步
并发与并行	并发是同一时间间隔内、宏观同时微观交替；并行是同一时刻真正同时	并行需要多处理机 / 多流水线等硬件支持；单处理机上进程只能并发
手工操作阶段	用户独占全机，人工控制全过程	此阶段没有操作系统；CPU 利用率极低
单道批处理	内存中始终保持一道作业，脱机 I/O + 监督程序	特征为自动性、顺序性、单道性；I/O 时 CPU 空闲
多道批处理	内存驻留多道程序，I/O 时借助中断切换	多道、宏观并行、微观串行；优点利用率高吞吐大，缺点无交互响应长
分时系统	时间片轮转，多用户共用一台主机	同时性、交互性、独立性、及时性；首次实现人机交互
实时系统	按紧迫性 / 截止时间调度，抢占式	及时性、可靠性；分硬实时与软实时
各类系统排序	——	资源利用率：批处理 > 分时 > 实时；交互性：分时 > 实时 > 批处理；及时性：实时 > 分时 > 批处理
CPU 运行模式	内核态（管态）与用户态（目态），由 PSW 的模式位标识	内核态可执行全部指令；用户态只能执行非特权指令
特权指令	I/O、开关中断、置时钟、修改 PSW 与内存保护寄存器、改中断向量表、内核态转用户态	用户程序执行特权指令会触发非法指令异常，由硬件强制实施
状态切换的途径	系统调用、外中断、异常、试图执行特权指令	用户态 → 内核态由硬件在中断响应时自动完成；内核态 → 用户态由 `IRET` 完成
中断与异常	中断是外中断（外设、时钟）；异常是内中断（故障、自陷、终止）	外中断分可屏蔽（INTR）与不可屏蔽（NMI）；异常不可屏蔽
中断处理流程	判优 → 查中断向量表 → 特权级校验与切栈 → 保护现场 → 执行处理程序 → 恢复现场	硬件保存 PC 和 PSW；操作系统保存通用寄存器和中断屏蔽字
中断向量表	索引为中断类型号，表项为处理程序入口地址	本质是数组，由操作系统初始化，支持随机访问
系统调用	应用程序主动请求内核服务的唯一合法接口	与库函数不同：内核提供、必须切换特权级；不同 OS 接口不统一
系统调用过程	传参 → 陷入指令（用户态）→ 内核态执行服务程序 → 返回用户态	陷入指令 = trap = 访管指令，在用户态执行；硬件保存断点与 PSW

考纲条目	核心内容	必须记住的结论
链接方式	静态链接、装入时动态链接、运行时动态链接	越晚链接越省空间、越灵活，但运行时开销与实现复杂度更高
装入方式	绝对装入、可重定位装入（静态重定位）、动态运行时装入（动态重定位）	静态重定位装入后不能移动；动态重定位需要重定位寄存器等硬件支持
内存映像	由低到高：代码段、数据段、堆、共享库映射区、栈	堆向高地址增长、栈向低地址增长；PCB 不在进程虚拟地址空间内
分层结构	层 0 是硬件、层 N 是用户接口，单向依赖	便于调试验证、易扩充维护；划分困难、效率较低
模块化结构	按功能划模块、定接口，模块-接口法	内聚性越高、耦合度越低，模块独立性越好
宏内核	主要功能模块紧密耦合，全部运行在内核态	性能高；内核臃肿难维护，一个模块出错可能全系统崩溃
微内核	只保留最基本机制在内核，其余为用户态服务器，靠消息传递	扩展灵活、可靠安全、可移植、支持分布式；性能差
外核	内核只管资源分配与保护，资源直接交给用户级系统	减少映像层与抽象开销；用库操作系统实现传统功能
操作系统引导	BIOS/UEFI → POST → MBR → PBR → 启动管理器 → 内核 → init	MBR 共 512 字节（446 + 64 + 2），签名 `0x55AA`，只能有 4 个主分区项
虚拟机	把一台物理机虚拟为多台逻辑独立、硬件完整的虚拟机	第一类为裸金属架构（VMM 在最高特权级）；第二类为寄居架构（VMM 是应用）

易错清单

1. 「内核态只能执行特权指令」错（内核态能执行所有指令）；「用户态只能执行非特权指令」对——2022-27 的直接考点。
2. 修改 **PSW** 模式位的指令本身是特权指令，所以用户态 → 内核态不可能靠执行指令完成，一定是中断 / 异常响应时由硬件自动切换。
3. 「中断处理程序负责切换模式」错：中断处理程序开始执行时 CPU 已经在内核态了；「中断或异常发生时 **CPU** 处于内核态」也错：事件发生那一瞬间 CPU 可能仍在用户态。要分清「发生」与「处理」，2012-23 与 2024-23 都考这个区分。
4. 进程切换不可能在用户态发生（2012-23），但系统调用、外部中断、缺页都可能在 CPU 执行用户程序时发生。
5. **trap**、跳转、压栈、数据传送、设置断点指令都是非特权指令，可在用户态执行；只有 **I/O** 指令、开 / 关中断指令这类才必须在内核态（2014-25、2021-23）。
6. `NOT R0` 不会引起模式切换（不访存、不触发调用）；而 `DIV`（可能除零）、`INT n`（软中断）、`MOV`（访存可能缺页或越界）都可能（2015-24）。
7. 中断与异常的归属容易搞反：时钟中断是外中断；缺页、除零、非法操作码、系统调用都是内中断（异常），其中缺页属故障、系统调用属自陷。
8. 异常不能被屏蔽，不可屏蔽中断也不能被屏蔽；中断屏蔽字是按类型屏蔽而非全局关中断。
9. 硬件保存断点（**PC**）和 **PSW**，操作系统保存通用寄存器和中断屏蔽字（2015-23、2020-27、2022-31 反复正反问）。

10. 普通函数调用不保存 **PSW**（全程同特权级）；允许多重中断时才必须保存，因为可能在用户态也可能在内核态响应中断。
11. 中断向量表用数组实现，因为需要按下标随机访问（2023-24）；不要选队列或链表。
12. 「不同的操作系统为应用程序提供统一的系统调用接口」是错的；「操作系统提供给应用程序的接口」指的是系统调用，不是库函数、中断或原语（2010-23、2019-25）。
13. 库函数不等于系统调用：`sin()`、`strlen()`、`rand()` 完全在用户态；`printf` 底层会调用 `write`，`malloc` 在大块分配时可能调用 `brk` / `mmap`。
14. 创建新进程需要系统调用，页置换和进程调度不需要（前者由缺页异常自动触发，后者由时钟中断等隐式触发）——2021-32 的考点。
15. 系统调用顺序必须是「先传参、再陷入、后执行服务程序、最后返回用户态」，不要把返回用户态排到中间（2017-24）。
16. 多道批处理系统不必支持虚拟存储管理（2022-23）；多道程序系统的系统开销并不更小（2017-28 的干扰项）；它也不提高单个作业的运行速度，提升的是资源利用率与吞吐量。
17. 进程数越多 **CPU** 利用率不一定越高，进程过多会因争抢 CPU 反而降低利用率。
18. 静态重定位的进程装入后不能移动、也不能动态申请内存；需要移动进程（页式、紧凑操作）的场景必须用动态重定位。
19. `.bss` 存放未初始化的全局 / 静态变量，已初始化的放 `.data`；代码段只读；堆向高地址增长、栈向低地址增长，方向相反不能记反。
20. **PCB** 不属于进程的虚拟地址空间，它由内核维护、对用户进程不可见。
21. **MBR** 只能描述 **4** 个主分区，四个里只有一个可以是激活分区；签名 `0x55AA` 位于第 511 ~ 512 字节；开机时操作系统既不在 **ROM** 也不在 **MBR** 中，最终落在 RAM。
22. 中断向量表会被建立两次：BIOS 阶段建立临时表供引导使用，操作系统初始化时重建属于自己的中断向量表；分区表、文件系统根目录、索引节点表都不是操作系统初始化阶段创建的（2022-24）。
23. 宏内核性能高但可靠性差，微内核可靠性高但性能差；「微内核具有较好的性能」是错误选项（2023-23 中Ⅰ项为错）。
24. 第二类虚拟机管理程序本身是应用程序，「提供虚拟机管理程序属于操作系统的任务」是错的；客户操作系统的「内核态」是虚拟内核态。

第十五章进程管理

本章考纲要求

（一）进程与线程

1. 进程与线程的基本概念
2. 进程/线程的状态与转换
3. 线程的实现：内核支持的线程，线程库支持的线程
4. 进程与线程的组织与控制
5. 进程间通信：共享内存，消息传递，管道，信号

（二）CPU 调度与上下文切换

1. 调度的基本概念
2. 调度的目标
3. 调度的实现：调度器/调度程序（scheduler），调度的时机与调度方式（抢占式/非抢占式），闲逛进程，内核级线程与用户级线程调度
4. CPU 调度算法
5. 多处理机调度
6. 上下文及其切换机制

（三）同步与互斥

1. 同步与互斥的基本概念
2. 基本的实现方法：软件方法，硬件方法
3. 锁
4. 信号量
5. 条件变量
6. 经典同步问题：生产者-消费者问题，读者-写者问题，哲学家进餐问题等

（四）死锁

1. 死锁的基本概念
2. 死锁预防
3. 死锁避免
4. 死锁检测和解除

本章是操作系统分值与难度最高的一章。历年统考中，本章在选择题中稳定占 3～4 分，在综合应用题中几乎每年出现（PV 操作题 7～9 分、银行家算法或调度算法计算题 6～8 分）。本章是必须拿到满分的大章。

一、进程与线程

（一）进程与线程的基本概念

*1.*进程的定义与进程映像

程序是存放在磁盘上的静态指令和数据集合；要让程序运行，必须把它的指令和数据装入内存，并为它分配 CPU、栈等资源。进程（**Process**）就是程序在一个数据集合上的一次动态执行过程，是系统进行资源分配和调度的独立单位。

进程在某一时刻的全部状态信息称为进程映像（也称进程实体），由三部分构成：

组成部分	内容	特点
PCB（进程控制块）	进程描述信息、控制管理信息、资源清单、处理机上下文	常驻内存，进程存在的唯一标志
程序段	二进制机器码、只读常量（字符串字面量、`const` 全局常量）	静态、只读、可共享
数据段	全局/静态变量（`.data`、`.bss`）、堆（动态分配）、栈（函数调用）	动态、可读可写、进程私有

数据段的细分值得记牢：`.data` 存放已初始化的全局变量和静态变量；`.bss` 存放未初始化（或初始化为 0）的全局变量和静态变量，可执行文件中只记录” 需要多大空间”，装入时由操作系统清零；堆由程序员手动申请释放（`malloc/free`），从低地址向高地址增长，忘记释放即” 内存泄漏”；栈由编译器自动管理，保存局部变量、函数参数、返回地址，从高地址向低地址增长，递归过深会栈溢出。

进程与程序的区别：

- 程序是静态的永久存在，进程是动态的、有生命周期的；
- 进程 = 程序 + 执行状态，即” 进程是程序的一次执行”；
- 同一个程序可以对应多个进程（例如同时打开多个浏览器窗口），它们代码段相同，但数据段、堆、栈各不相同；
- 一个进程也可以执行多个程序。

创建与撤销的本质

创建进程实质上是创建进程实体中的 **PCB**；撤销进程实质上是撤销进程的 **PCB**。

真题考法

2023 年曾考” 进程虚拟空间”、2021 年曾考” 进程组织之进程控制块（PCB 内容）”。选择题常问” 下列哪一项不属于 PCB 的内容” 或” 进程切换时必须更新哪些寄存器”。

*2.*线程的基本概念

引入进程是为了让多道程序并发执行、提高资源利用率和吞吐量；但进程作为资源拥有单位，创建、撤销、切换的系统开销都很大。于是引入线程（**Thread**）——” 轻量级进程”。

线程是程序执行流的最小单位，是处理机独立调度和分派的基本单位；进程是资源分配的基本单位。

线程自己私有的部分（即线程上下文）：

- 线程标识符 TID；
- 程序计数器 PC；
- 一组通用寄存器（线程现场）；
- 栈（每个线程一个独立的栈，保存局部变量、函数调用上下文）。

同一进程内的所有线程共享：代码段、数据段（全局变量、静态变量）、堆、打开的文件描述符、网络套接字、信号处理程序、进程 ID 与父进程 ID、地址空间（页表）。

一句话记忆

线程私有：**TID**、**PC**、寄存器、栈。其余全共享。

因为每个线程有自己的栈，线程的局部变量是线程私有的。如果需要线程间共享数据，值必须放在堆或全局可访问位置。

易错

”同一进程的线程还可以修改别的线程栈里的数据吗？”从地址空间角度看可以（因为共享地址空间），但同一进程的线程属于同一用户，编程时会主动避免这种非法访问。统考按”栈是线程私有的”理解即可。

*3.*进程与线程的比较

比较项	进程	线程
定义	资源分配的基本单位，程序在一个数据集上的一次动态执行	处理机调度的基本单位，进程中的一个执行实体
调度	由操作系统调度	可由用户空间线程库调度，也可由操作系统调度
资源	拥有独立的内存空间和系统资源	基本不拥有资源，共享所属进程的全部资源
地址空间	进程间相互独立（内存保护）	同进程内线程共享地址空间
并发性	进程间可并发	同进程内线程可并发；线程切换开销小，系统并发度更高
系统开销	创建、撤销、切换开销大	创建、撤销、切换开销小
通信	必须借助 IPC 机制	直接读写进程内共享变量即可
独立性	一个进程崩溃不影响其他进程	一个线程崩溃可能导致整个进程崩溃
多处理机	单线程进程只能在一个处理机上运行	多线程进程的不同线程可分配到不同处理机上并行执行
相似之处	PCB、PID、就绪/运行/阻塞三态、父进程创建子进程、不同进程可执行同一程序	TCB、TID、就绪/运行/阻塞三态、父线程创建子线程、不同线程可执行同一程序

线程相比进程减少开销的原因（高频考点）：

1. 线程创建比进程创建快——不必维护内存管理信息、文件管理信息，而是共享它们；
2. 线程终止比进程终止快——释放的资源少得多；
3. 同进程内线程切换比进程切换快——它们共用同一页表，切换时不用切换页表（页表切换在上下文切换开销中占比很大）；
4. 同进程线程间数据传递不经过内核，交互效率更高。

易错

- 页表基址寄存器（页目录基址寄存器）在同一进程的不同线程间切换时不需要修改，因为共用同一个虚拟地址空间；同时原有 TLB 不会失效。切换时要更新的只有程序计数器、通用寄存器组、栈基址寄存器。
- ”操作系统为每个用户级线程建立一个线程控制块”是错误说法（2019 年真题）。内核级线程才有内核维护的 TCB，用户级线程的 TCB 由用户态线程库维护。

（二）进程与线程的状态与转换

*1.*三态模型与五态模型

三种基本状态：

状态	含义
运行态（**Running**）	占有 CPU 正在执行。单处理机下任一时刻至多一个进程处于运行态，多处理机下可以有多个
就绪态（**Ready**）	已获得除处理机以外的一切所需资源，一旦得到处理机便可立即执行
阻塞态（等待态，**Waiting/Blocked**）	正在等待某资源可用（不包括处理机）或等待 I/O 完成，即使处理机空闲也不能运行

五种状态（加上创建态、终止态）：

状态	含义
创建态（**New**）	进程正在被创建，已分配 PCB，正在分配除 CPU 外的其他资源
就绪态	见上
运行态	见上
阻塞态	见上
终止态（**Exit**）	进程运行终止后，操作系统正在回收其占用的资源和 PCB

*2.*状态转换的合法与非法情况

合法转换只有五条：

1. 创建态 → 就绪态：资源分配完毕，PCB 初始化完成，插入就绪队列。
2. 就绪态 → 运行态：被调度程序选中，分配到处理机。
3. 运行态 → 就绪态：时间片用完，或被更高优先级的进程抢占（被动行为，由调度引起）。
4. 运行态 → 阻塞态：进程主动请求某种资源或等待某事件（主动行为，通过系统调用发出）。
5. 运行态 → 终止态：进程正常结束或异常终止。

非法转换（必考）：

- 阻塞态 → 运行态：不允许。阻塞的进程必须先被唤醒变为就绪态，再经调度才能运行。
- 就绪态 → 阻塞态：不允许。进入阻塞态是进程在运行中主动发出的请求，就绪态没有 CPU，无法发出请求。
- 阻塞态 → 就绪态是合法的，且是被动行为——必须由其他进程（如释放 I/O 设备的进程）调用唤醒原语完成。

- 创建态 → 运行态不允许；创建完成后只能先进入就绪态。

判断口诀

”运行”往下走：要么回就绪（被抢），要么去阻塞（自己等），要么结束。”阻塞”只能回到就绪，”就绪”只能去运行。

*3.*引起状态转换的典型事件

事件	转换
时间片用完	运行 → 就绪
被更高优先级进程抢占	运行 → 就绪
主动出让 CPU（`yield`）	运行 → 就绪
请求临界资源 / 申请外设	运行 → 阻塞
启动 I/O 设备、读文件（等待 I/O）	运行 → 阻塞
对值 ≤ 0 的信号量执行 P（wait）操作	运行 → 阻塞
对管程条件变量执行 `wait()`	运行 → 阻塞
申请内存失败 / 创建进程资源不足	运行 → 阻塞
I/O 完成（中断处理）	阻塞 → 就绪
某进程退出临界区并发出唤醒	阻塞 → 就绪
等待的资源被释放、事件到达	阻塞 → 就绪

三个高频陷阱

1. ”键盘输入”不是”运行 → 就绪”。当前进程等待键盘输入，是从运行态转为阻塞态（等待 I/O 事件）。
2. ”缺页异常”不是”运行 → 就绪”。缺页后进程等待页面调入，进入阻塞态。
3. ”执行 **P(wait)** 操作”是”运行 → 阻塞”，只有”主动出让 CPU”才是”运行 → 就绪”（2023 年真题答案即为此）。

真题考法

2015-25（下列哪个事件导致执行态变就绪态）、2018-27（哪些事件导致当前进程阻塞）、2019-24（哪些事件可能唤醒进程）、2022-28（哪些操作导致执行态变阻塞态）、2023-27（哪个操作让线程从执行态变就绪态）。几乎每年一道，属于必得分题。

*4.*挂起态与七状态模型

内存空间有限，操作系统把短期内不执行的进程换出到外存，称为挂起（**Suspend**）；把更可能执行的进程从外存调回内存，称为激活（**Activate**）。加上两个挂起态就得到七状态模型：

- 就绪挂起态：进程在外存，但只要调入内存即可立即运行；
- 阻塞挂起态：进程在外存并且仍在等待某事件。

关键转换：

- 运行 → 就绪挂起：程序被直接挂起；
- 阻塞 → 阻塞挂起：内存紧张时，优先换出阻塞态进程（它占着内存却不能运行）；
- 阻塞挂起 → 就绪挂起：所等待的事件到来（如 I/O 完成）；
- 就绪挂起 → 就绪：内存中没有其他就绪进程，或该进程优先级高于内存中所有就绪进程；

- 就绪 → 就绪挂起：只有在挂起阻塞态进程仍不足以释放足够内存时才这样做。

核心结论

换出/换入（Swapping）由中级调度负责。被挂起的进程即使优先级很高也不能运行，因为它不在内存中。

（三）线程的实现

*1.*内核支持的线程与线程库支持的用户级线程

用户级线程（**ULT**，**User-Level Thread**）：由用户程序自己管理和调度，创建、同步、调度全部由运行在用户空间的线程库（如 POSIX Pthreads、Java 线程库）完成，内核完全不知道其存在。

内核级线程（**KLT**，**Kernel-Level Thread**）：由操作系统内核管理和调度，内核为每个内核级线程建立 TCB，创建、同步、调度通过系统调用完成。

对比表：

对比项	用户级线程（**ULT**）	内核级线程（**KLT**）
管理者	用户空间的线程库，每个进程维护私有的 TCB 列表	操作系统内核，内核为每个线程建立 TCB
内核是否感知	不感知（可用于不支持线程的操作系统）	感知并能控制
切换开销	小，完全在用户空间完成，不需要陷入内核	大，需要通过系统调用，存在用户态/内核态切换
阻塞影响	一个线程阻塞则整个进程被阻塞（内核只看到进程）	一个线程阻塞不影响同进程其他线程继续执行
并行能力	不能利用多处理机，同一个进程的多个线程只能在一个 CPU 上分时复用	可以分配到多个处理器上真正并行执行
CPU 时间分配单位	以进程为单位分配时间片，多线程进程中的每个线程分到的时间更少	以线程为单位分配，多线程进程可获得更多 CPU 时间
调度算法	允许每个进程有自己的线程调度策略，程序员可自定义	由内核统一调度，可利用内核的优先级管理等机制
是否需系统调用	创建、撤销、切换都不需要	创建、撤销、切换都需要

组合方式：内核支持多个内核级线程的建立、调度和管理，同时允许用户程序建立、调度和管理用户级线程；若干个用户级线程通过时分多路复用地映射到若干个内核级线程上。它结合了 KLT 与 ULT 的优点：同一进程的多个线程可同时在多处理机上并行运行，且阻塞一个线程时不需要阻塞整个进程。

*2.*多线程模型

模型	映射关系	优点	缺点
多对一	多个用户级线程映射到一个内核级线程	线程管理开销小、效率高，可在不支持线程的内核上实现	一个线程阻塞则整个进程阻塞；不能利用多处理机

模型	映射关系	优点	缺点
一对一	每个用户级线程对应一个内核级线程	阻塞系统调用不影响其他线程；允许多线程在多处理机上并行	创建一个用户线程就要创建一个内核线程，内核线程数量受限，开销大
多对多	n 个用户级线程映射到 m 个内核级线程（n > m）	兼有前两者的优点，可并行又可控制开销	实现复杂

真题考法

2012-31（进程与线程的基本叙述）、2019-23（线程的描述中错误的——答案是” 操作系统为每个用户级线程建立一个线程控制块”）、2011-25（线程不能共享的是栈指针）。这类题目本质都在考” 哪些共享、哪些私有” 和” 谁负责调度”。

（四）进程与线程的组织与控制

1.PCB 的作用与内容

PCB 是进程存在的唯一标志，进程创建后该结构常驻内存，进程结束时删除。操作系统通过 PCB 管理和控制进程，因此管理控制所需的一切参数一定放在 **PCB** 中。

类别	具体内容
进程描述信息	进程标识符 PID（系统内唯一，PID 是操作系统分配的有限资源，即可分配的 PID 个数固定，对应系统允许同时存在的最大进程数）、用户标识符 UID
进程控制和管理信息	进程状态、进程优先级、代码运行入口地址、程序外存地址、进入内存时间、处理机占用时间、信号量使用情况、CPU 调度信息（调度队列指针等）
资源分配清单	内存/虚存地址、代码段指针、数据段指针、堆栈段指针、打开文件表、I/O 设备信息
处理机相关信息（处理机上下文）	程序计数器 PC、通用寄存器值、地址寄存器值、控制寄存器值、标志寄存器值、状态字、累加器、索引寄存器、其他条件码寄存器，以及基址/界限寄存器的值等内存管理信息

*2.*进程控制块的组织方式

组织形式	做法	特点
线性表	所有 PCB 组织在一张线性表中，表起始地址存放在内存特定位置或专用寄存器中	简单，但查找慢
链接队列	不同状态对应不同的队列：就绪队列、各类阻塞队列	插入删除灵活；就绪队列只有一个，但等待不同事件的阻塞队列有多个
索引表	不同状态对应不同的索引表（就绪索引表、阻塞索引表）	查找快，需要额外索引空间

为什么阻塞队列有多个而就绪队列只有一个

进程处于就绪态的原因只有一个——还没分配到 CPU；而阻塞原因多种多样（等 I/O、等打印机、等信号量、等消息……），因此每一类阻塞原因单独建立一个 **PCB** 等待队列。

*3.*进程控制的原语操作

进程控制的主要功能是对系统中所有进程实施有效管理，包括创建新进程、撤销已有进程、实现状态切换。这些功能都由原语实现。原语是不可被中断的原子操作，执行期间不允许中断，因此原语的执行过程中不能进行进程调度与切换。

创建原语：

1. 为新进程分配唯一的 PID，并申请一个空白 PCB（PCB 有限，申请失败则创建失败）；
2. 为进程分配运行所需资源（内存、文件、I/O 设备、CPU 时间等），资源不足时不是创建失败，而是停留在创建态等待；
3. 初始化 PCB：初始化标志信息、处理机状态信息、处理机控制信息，设置优先级等；
4. 若就绪队列能接纳新进程，将新进程插入就绪队列，等待调度。

引起进程创建的事件：终端用户登录、作业调度、系统提供服务、用户程序的应用请求。

撤销原语：

1. 找到被撤销进程的 PCB；
2. 若进程正在运行，立即剥夺其处理机；
3. 若进程还有子进程，通常应同时撤销所有子进程（统考按此理解；实际 Linux 中父进程结束时子进程由 init 接管成为孤儿进程，Windows 中同一进程组会一起结束）；
4. 归还进程占用的全部资源给父进程或系统；
5. 将 PCB 从所在队列中移出。

阻塞原语（**Block**）：

1. 找到被阻塞进程的 PCB；
2. 若该进程为运行态，则保护现场（保存上下文），将其状态转为阻塞态，停止运行；
3. 把 PCB 插入相应事件的等待队列，将处理机调度给其他就绪进程。

唤醒原语（**Wakeup**）：

1. 在该事件的等待队列中找到相应进程的 PCB；
2. 将其从等待队列中移出，置其状态为就绪态；
3. 把 PCB 插入就绪队列，等待调度。

易错

- 阻塞是进程自身的主动行为，只有运行态的进程（占有 CPU）才可能转为阻塞态；唤醒是被动行为，必须由其他进程调用唤醒原语完成。
- 阻塞原语与唤醒原语必须成对使用，否则进程将永远阻塞。
- 阻止进程调度的情况：中断处理过程中、访问内核临界区时、禁用中断期间、原语执行过程中。注意用户级临界区中允许调度：例如某进程已获得打印机（处于临界区）后主动等待 I/O，仍会阻塞自己并触发调度。

真题考法

2010、2019、2020、2021、2022、2024 年多次考查。典型问法：” 创建进程时必须完成的操作包括哪些”” 撤销进程时不一定执行的是”（2024 年答案为” 终止子进程”）、” 操作系统为新建进程完成的工作按顺序是”。

（五）进程间通信

每个进程的用户空间相互独立、不能直接互访，因此需要借助内核空间（所有进程共享同一份内核空间）来实现通信。

*1.*共享内存

共享内存机制是拿出一块物理内存，映射到多个进程各自的虚拟地址空间中，各进程通过自己的页表把虚拟地址映射到同一物理页框。这样进程 A 写入的内容进程 B 立刻可见，不需要任何数据拷贝，是最快的 **IPC** 方式。

工作过程：

1. 创建共享内存：`shmget`（System V）或 `shm_open`（POSIX），内核在物理内存中分配区域并指定访问权限和大小；
2. 附加到进程地址空间：`shmat` 或 `mmap`，内核修改该进程的页表，把虚拟地址范围映射到共享内存的物理页框；
3. 读写共享内存：进程像访问普通内存一样直接读写，无需系统调用，零拷贝；
4. 同步机制：共享内存本身不提供任何同步，必须配合信号量、互斥锁等使用，否则会竞态；
5. 分离：`shmdt` 或 `munmap`，内核移除该进程页表中的映射，但物理内存仍保留；
6. 销毁：`shmctl` 或 `shm_unlink` 标记删除，所有附加进程都分离且无引用时内核才回收物理内存。

共享内存的三个必须记牢的结论

1. 共享内存位于进程的用户地址空间，不在内核空间；
2. 建立共享内存的操作（修改页表）在内核态；访问共享数据的操作在用户态；
3. 共享的物理页框唯一（f1 = f2 必然相等），但各进程中的逻辑页号可以不同（p1 与 p2 不一定相等）。

分页与分段共享的统一原则

物理地址唯一（必须同源），逻辑地址独立（可以不同）。因此 2023-30 题答案是” 页号 p1、p2 不一定相等，页框号 f1、f2 一定相等”；2019-28 题中” 段 S 在 P1 和 P2 中应该具有相同的段号” 是错误叙述。

补充：进程虚拟地址空间中的内核空间部分映射到相同的物理内存区域（操作系统的代码和全局数据），因此内核空间天然是共享的；但每个进程在内核态执行时使用自己独立的内核栈，内核栈指针保存在该进程的 PCB 中。

*2.*消息传递

直接通信：通信进程之间直接建立链路，一条链路对应一对进程，`send(P1, message)` 发送给进程 P1，`receive(P0, message)` 接收进程 P0 发来的消息。

间接通信：通过操作系统维护的消息队列（或信箱）完成。每个消息队列有唯一标识，只有共享了相同消息队列的进程才能通信。链路属性：只有共享同一消息队列的进程才建立连接；连接可以是单向或双向；一个消息队列可以与多个进程关联；每对进程可以共享多个消息队列。

同步方式：

- 阻塞（同步）模式：发送者在消息被接收者收到前阻塞；接收者在没有消息时阻塞。
- 非阻塞（异步）模式：发送者发出消息后立即继续；接收者没有消息时也立即返回。

消息队列的缺点：不适合大数据传输（Linux 中有 `MSGMAX` 限制单条消息最大长度、`MSGMNB` 限制队

列总长度）；每次读写都要经过用户态与内核态之间的数据拷贝。

*3.*管道

匿名管道：位于内核缓冲区（不在用户空间，也不在磁盘上）的 IPC 机制，主要用于具有亲缘关系的进程（父子、兄弟）之间的单向数据流通信。

工作流程：

1. 创建管道：`pipe(int pipefd[2])`，返回两个文件描述符，`pipefd[0]` 读端、`pipefd[1]` 写端；内核为每个管道分配一个环形缓冲区（循环队列）和两个文件描述符；
2. 进程间共享：父进程 `pipe()` 后 `fork()`，子进程继承父进程的文件描述符表，从而共享同一管道的读写端；
3. 数据读写：写入时数据从用户空间拷贝到内核缓冲区；读取时从内核缓冲区拷贝到用户空间；
4. 生命周期：随进程终止而销毁；所有引用管道的文件描述符关闭后，内核回收缓冲区资源（不会持久化到磁盘）。

pipe 的读写规则与阻塞条件（必背）：

情形	行为
写进程写满的管道	阻塞，直到有空间
读进程读空的管道	阻塞，直到有数据
读进程读一个写端全部关闭的管道	`read` 返回 0（EOF），不阻塞
写进程写一个读端全部关闭的管道	内核向写进程发送 `SIGPIPE` 信号（即”Broken pipe”）
单次写入不超过 `PIPE_BUF`（通常 4096 字节）	保证原子性，多个进程同时写时数据不会交叉混合
管道是否支持定位	不支持 `lseek` 之类的文件定位操作，只能队尾写入、队头读出

其他必须记牢的结论：

- 同一时刻，管道只允许一个读进程执行读操作或一个写进程执行写操作（读写都要互斥进行）；
- 但一个管道允许存在多个读进程或多个写进程（它们都有该管道的文件描述符，只是操作时要抢管道的内核互斥锁）；
- 管道自身通过互斥锁 + 两个等待队列（**`rd_wait`**、**`wr_wait`**）+ 读写指针实现了同步与互斥，用户不需要额外写信号量；
- 管道是半双工的，某一时刻只能单向传输；要实现全双工需创建两个管道；
- 匿名管道只能用于有亲缘关系的进程；命名管道（**FIFO**）存在于文件系统中，可以用于任意进程之间通信。
- 管道以字节流为单位（无格式、无消息边界），不是以消息为单位。

十二个自测判断（全部为” 错误” 的典型说法）

1. ” 管道位于进程的用户空间，可直接在用户态读”——错，管道在内核区，读写要通过系统调用在内核态完成；
2. ” 管道与共享内存这两种 IPC 都自己实现了同步/互斥”——错，共享内存本身没有同步互斥机制；
3. ” 管道是存放在磁盘中的一种特殊文件”——错，匿名管道在内存中；
4. ” 管道支持定位到某个具体位置读写”——错，只能队尾写、队头读；

5. ”所有引用管道的文件描述符都被关闭后，管道内未读完的数据会被持久化到磁盘”——错，内核直接回收缓冲区；
6. ”管道通信是以消息为单位进行写入和读出的”——错，是字节流；
7. ”对管道的写操作不需要互斥”——错，读写都需要互斥。

真题考法

2014-30 曾考管道的特性（答案选”进程对管道进行读操作和写操作都可能被阻塞”）。常考选项包括”一个管道仅支持半双工通信”” 管道位于内存的内核区域”” 一个管道在同一时刻只能有一个读或一个写进程操作”。

*4.*信号

信号（**Signal**）是一种异步的、非阻塞的通信机制，也叫”用户态中断”或”软中断”，用于通知接收进程某个事件已经发生。每个信号有一个唯一编号和对应的处理动作。信号通常只用于简单的事件通知，不适合复杂的数据传输。

信号的来源：

- 硬件来源：硬件异常（浮点除零触发 SIGFPE）、内存访问越界（段错误 SIGSEGV）、非法指令；
- 软件来源：系统调用（kill()、raise()、alarm()、setitimer()）、用户输入（Ctrl+C 触发 SIGINT）、系统事件（子进程退出时父进程收到 SIGCHLD）、定时器到期（SIGALRM）。

常见信号：SIGINT（终端中断）、SIGKILL（强制终止，不可捕获、不可忽略）、SIGTERM（默认终止信号，可捕获）、SIGCHLD（子进程终止）、SIGALRM（定时器）。

传递与响应流程：信号产生 → 信号传递（内核把信号递给目标进程）→ 信号处理 → 执行处理函数 → 信号返回。

进程对信号有三种响应方式：

1. 执行默认操作：多数信号的默认行为是终止进程、生成核心转储文件或暂停进程；
2. 捕捉信号：执行用户自定义的信号处理函数；
3. 忽略信号：不做任何处理。但 SIGKILL 和 SIGSTOP 不能被忽略或捕捉。

核心结论

信号传递不支持排队：短时间内多次传递同一信号，进程只会处理一次。

*5.*四种 *IPC* 机制对比

特性	共享内存	消息传递（消息队列）	（匿名）管道	信号
数据形式	原始字节流，直接内存访问	结构化消息（带类型/优先级）	字节流（无结构）	简单整数信号（几乎无数据）
通信方向	双向	双向	单向（半双工）	单向（发送 → 接收）
同步需求	需要（信号量、互斥锁）	部分需要（阻塞接收）	内核自动同步	无（异步通知）
性能	极高（零拷贝）	中（数据拷贝开销）	中（内核缓冲区拷贝）	极高（无数据传输）
复杂度	高	中	低	低

特性	共享内存	消息传递（消息队列）	（匿名）管道	信号
进程关系	任意进程	任意进程	仅亲缘进程	任意进程（需知道 PID）
持久性	分离后需重新映射	内核维护	随进程终止销毁	瞬时触发
典型应用	高频数据交换（视频处理）	微内核	Shell 管道	事件通知

二、CPU 调度与上下文切换

（一）调度的基本概念

调度分为三个层次：

层次	要做什么	调度发生在	发生频率	对进程状态的影响
高级调度（作业调度）	按某种规则从后备队列中选择合适的作业调入内存，为其创建进程	外存 → 内存（面向作业）	最低	无 → 创建态 → 就绪态
中级调度（内存调度）	按某种规则从挂起队列中选择合适的进程将其数据调回内存	外存 → 内存（面向进程）	中等	挂起态 → 就绪态等
低级调度（进程调度 / **CPU** 调度）	按规则从就绪队列中选择一个进程为其分配处理机	内存 → CPU	最高	就绪态 → 运行态

- 高级调度主要用于多道批处理系统；每个作业只调入一次、调出一次；作业调入时建立 PCB，调出时才撤销 PCB。
- 中级调度即换入/换出（Swapping），目的是提高内存利用率和系统吞吐量。
- 低级调度是最基本的调度，所有操作系统都必须有。

（二）调度的目标

指标	定义	说明
CPU 利用率	$CPU利用率 = \frac{CPU有效工作时间}{总时间}$	希望 CPU 尽可能忙碌
系统吞吐量	系统吞吐量 $= \frac{总共完成的作业数}{总共花费时间}$	短作业能提升吞吐量，长作业会降低吞吐量
周转时间	周转时间 = 作业完成时间 − 作业提交（到达）时间	用户更关心单个作业的周转时间
平均周转时间	平均周转时间 $= \frac{各作业周转时间之和}{作业数}$	操作系统更关心整体表现
带权周转时间	带权周转时间 $= \frac{作业周转时间}{作业实际运行时间}$	必然 ≥ 1，越小越好

指标	定义	说明
平均带权周转时间	平均带权周转时间 $=\frac{\text{各作业带权周转时间之和}}{\text{作业数}}$	综合指标
等待时间	进程处于就绪态的时间之和	CPU 调度算法只影响等待时间，不影响运行时间和 I/O 时间
响应时间	用户提交请求到系统第一次产生响应的时间	交互式系统衡量调度算法好坏的主要标准

等待时间的口径

等待时间只统计进程在就绪队列中等待 **CPU** 的时间之和，不包括在后备队列中等待进入内存的时间（那属于另一指标），也不包括进程运行和 I/O 的时间。

带权周转时间为什么必然 ≥ 1

因为 周转时间 $=$ 等待时间 $+$ 运行时间 $\geq$ 运行时间，故 $\frac{\text{周转时间}}{\text{运行时间}} \geq 1$。两个作业周转时间相同时，实际运行时间更长的那个带权周转时间更小，用户满意度更高。

（三）调度的实现

*1.*调度器 / 调度程序

调度程序（Scheduler）由三部分组成：

组成部分	职责
排队器（**Queuing**）	按进程优先级、到达时间等因素把就绪进程排序并插入相应队列
分派器（**Dispatcher**）	根据调度程序选出的进程，从就绪队列中取出，把处理机分配给它
上下文切换器（**Context Switcher**）	把当前进程的内核上下文存入其 PCB，再从新进程 PCB 取出上下文装入 CPU

调度是一种决策行为，切换是一种执行行为。两者不一定同时发生（例如选出的下一个进程仍是当前进程时就不需要切换）。

*2.*调度的时机

会引起调度的事件：

1. 当前进程正常结束或异常终止（运行 → 终止）；
2. 当前进程主动阻塞（如发起 I/O、P 操作阻塞自己，运行 → 阻塞）；
3. 当前进程时间片用完（运行 → 就绪）；
4. 有更高优先级的进程进入就绪队列（创建新进程、I/O 完成唤醒阻塞进程等），当前进程被抢占（运行 → 就绪）；
5. 中断/异常/系统调用处理结束返回用户空间时，内核检查” 需要重新调度” 标志，若有效则触发调度；

6. 就绪队列为空时刚来的第一个进程直接上处理机，这也算一次 **CPU** 调度。

不能进行调度与切换的情况（必背）：

1. 当前进程处于中断处理过程中——中断处理涉及核心功能和关键资源，逻辑上不属于某一进程，不应被剥夺处理机；
2. 进程正在访问操作系统内核临界区——如调度器访问就绪队列。若此时被切换，可能导致内核数据结构不一致；
3. 禁用中断（屏蔽中断）过程中；
4. 原语执行过程中——加锁、解锁、中断现场保护与恢复等原子操作不可中断，连中断都要屏蔽，更不能进行调度与切换。

内核临界区 vs 用户级临界区

只有内核临界区才禁止调度。用户级临界区中允许调度：例如当前进程已请求到打印机（处于临界区），但它因等待 I/O 主动阻塞，此时完全可以进行调度。2012-30 题的答案就是” 在进程处于临界区时不能进行处理机调度” 这一说法错误（缺少” 内核” 二字）。

内核中常通过 PCB 里的一个标志位（如 Linux 的 `TIF_NEED_RESCHED`）来延迟抢占：

- 设置标志位的时机：时间片耗尽时；有更高优先级进程进入就绪态时（如 `fork()` 出优先级更高的子进程、I/O 完成后唤醒的进程优先级更高）；
- 响应标志位的时机：中断/异常/系统调用处理结束返回用户空间时，内核检查该标志，有效则调用调度程序。

*3.*抢占式与非抢占式

方式	定义	调度时机	特点与适用场景
非抢占式（非剥夺）	进程除非运行结束或自己主动放弃 CPU，否则不会被其他进程抢占	进程运行结束；进程主动阻塞	调度不频繁，系统开销小，吞吐量较大，CPU 利用率一般更高；实时性差，一般用于批处理系统，不适用于分时系统和大部分实时系统
抢占式（剥夺）	进程在执行中可能因时间片用完或有更高优先级进程进入就绪态而被抢占 CPU	主动放弃 CPU；时间片用完；更高优先级进程就绪	调度频繁，实时性强，可防止进程长期独占处理器；等待时间和响应时间更短，适用于分时系统和实时系统

核心结论

若不考虑 `yield` 等主动释放处理机的系统调用，则非抢占式调度下只会出现” 运行 → 终止” 和” 运行 → 阻塞”，不可能出现” 运行 → 就绪”。

*4.*闲逛进程

闲逛进程（**Idle Process**），又称空闲进程或空闲任务，在没有其他进程需要运行时占据 CPU，保证 CPU 始终有任务可执行。

特点：

1. 优先级最低，一旦有其他进程需要 CPU 就立即让出；
2. 持续运行，多数情况下只执行一个空循环，不执行任何实际操作；
3. 只运行在内核态下；

4. 占用资源最小，不需要 CPU 以外的资源，也不会阻塞。

*5.*内核级线程与用户级线程的调度

- 用户级线程的调度由用户态线程库完成：线程切换不需要陷入内核，开销小；但内核以进程为单位分配时间片，一个线程阻塞会导致整个进程阻塞。
- 内核级线程的调度由内核完成：线程切换需要上下文切换、修改内存映像、使高速缓存（Cache）失效，开销大；但可以真正做到多处理机并行，且一个线程阻塞不影响其他线程。
- 在采用一对一或多对多模型的系统中，调度的实际对象是线程，进程只提供虚拟内存、全局变量等资源。

（四）CPU 调度算法

本节的算例统一使用下面这组进程（到达时刻、服务时间、优先级三列，优先级数值越大优先级越高）：

进程	到达时间	服务时间	优先级
P1	0	5	2
P2	1	4	4
P3	2	2	1

*1.*先来先服务（*FCFS*）

算法步骤：按进程到达的先后顺序依次分配 CPU，直到进程执行完毕或被阻塞（例如请求 I/O），非抢占式。既可用于作业调度，也可用于进程调度。

算例（**t=0** 起开始调度）：

执行顺序：P1(0→5) → P2(5→9) → P3(9→11)。

进程	到达	服务	开始	完成	周转时间	带权周转时间
P1	0	5	0	5	5	1.00
P2	1	4	5	9	8	2.00
P3	2	2	9	11	9	4.50

平均周转时间 $= (5+8+9)/3 = 7.33$；平均带权周转时间 $= (1+2+4.5)/3 = 2.50$。

优点：简单、公平、实现容易，不会导致饥饿，对长作业（CPU 繁忙型）有利。缺点：排在长作业后面的短作业等待时间过长，带权周转时间很大，对短作业不利；I/O 繁忙型进程在等待 I/O 期间不能让出 CPU（严格先来后到），使 CPU 和设备利用率都低。

*2.*短作业优先（*SJF*）/ 短进程优先（*SPF*）

算法步骤（非抢占式）：每次调度时，在就绪队列中选择估计运行时间最短的作业/进程，把处理机分配给它，直到它完成或因某事件阻塞才释放处理机。

算例（非抢占式）：

t=0 只有 P1 到达，先运行 P1；P1 在 t=5 结束后，P2、P3 均已到达，选服务时间短的 P3(2) → P2(4)。

执行顺序：P1(0→5) → P3(5→7) → P2(7→11)。

进程	到达	服务	开始	完成	周转时间	带权周转时间
P1	0	5	0	5	5	1.00
P3	2	2	5	7	5	2.50
P2	1	4	7	11	10	2.50

平均周转时间 $=(5+5+10)/3=6.67$；平均带权周转时间 $=(1+2.5+2.5)/3=2.00$。

抢占式版本：最短剩余时间优先（**SRTN**）

算法步骤：每当有新进程进入就绪队列（或发生其他改变就绪队列的事件）就检查一次；比较新进程的剩余执行时间与当前运行进程的剩余执行时间，若新进程更短则立即抢占；若相等则按 FCFS 决定。

算例（抢占式 **SRTN**）：

t=0：只有 P1，P1 运行，剩余 5。t=1：P2 到达（剩余 4）。P1 剩余 4，与 P2 剩余 4 相等，不抢占（按规定相等时按 FCFS，保持 P1）。t=2：P3 到达（剩余 2）。P1 剩余 3 > 2，**P3** 抢占。t=4：P3 完成。就绪队列中 P1 剩余 3、P2 剩余 4，选 P1。t=7：P1 完成。运行 P2（剩余 4），到 t=11 完成。

执行顺序：P1(0→1) → P1(1→2) → P3(2→4) → P1(4→7) → P2(7→11)。

进程	到达	服务	完成	周转时间	带权周转时间
P1	0	5	7	7	1.40
P2	1	4	11	10	2.50
P3	2	2	4	2	1.00

平均周转时间 $=(7+10+2)/3=6.33$；平均带权周转时间 $=(1.4+2.5+1.0)/3=1.63$。**SRTN** 的平均等待时间和平均周转时间优于非抢占式 **SJF**。

408 考试必须记住的细节

1. 题目未特殊说明时，”短作业/进程优先”默认是非抢占式的；
2. 在有前提条件”所有进程可同时运行”或”所有进程几乎同时到达”时，才能说 SJF 的平均等待时间、平均周转时间最少；若无此前提，应说抢占式的 **SRTN** 平均等待时间、平均周转时间最少；
3. 严格来说，该算法应叫”最短下次 **CPU** 执行（**shortest-next-CPU-burst**）”，因为调度取决于进程下一次 CPU 执行的长度，而不是其总长度。

优点：平均等待时间、平均周转时间最短，可提高系统吞吐量。缺点：需要预先知道运行时间（实际不可靠，由用户提供）；对长作业不利，可能造成长作业饥饿；完全不考虑作业的紧迫程度。

*3.*高响应比优先（*HRRN*）

响应比：

$$\text{响应比}=\frac{\text{等待时间}+\text{要求服务时间}}{\text{要求服务时间}}$$

算法步骤：非抢占式。每次都只在当前运行的进程主动放弃处理机（正常/异常完成或主动阻塞）时才需要调度，此时计算就绪队列中所有进程的响应比，选择响应比最高者。

算例：

t=0：只有 P1，直接运行 P1 到 t=5。t=5：就绪队列中有 P2（等待 4，服务 4）与 P3（等待 3，服务 2）。$P2$响应比 $=\frac{4+4}{4}=2.00$；$P3$响应比 $=\frac{3+2}{2}=2.50$。选 P3。t=7：P3 完成。P2 等待 6，响应比 $=\frac{6+4}{4}=2.50$，运行 P2 到 t=11。

执行顺序：P1(0→5) → P3(5→7) → P2(7→11)。平均周转时间 $=(5+5+10)/3=6.67$，与本题的非抢占式 SJF 结果相同（巧合，因为 t=5 时 P3 响应比也最高）。

优点：兼顾等待时间与服务时间——等待时间相同时服务时间短的优先（SJF 的优点）；服务时间相同时等待时间长的优先（FCFS 的优点）；长作业随着等待时间增加响应比不断提高，不会饥饿。
缺点：需要知道服务时间；每次调度都要计算响应比，有一定开销。现实中不可实现，属于”理想型”算法。

*4.*时间片轮转（*RR*，*Round-Robin*）

算法步骤：就绪进程按 FCFS 排队；每次把 CPU 分配给队首进程，让它执行一个时间片（典型 20 ms ~ 50 ms）；时间片用完时由时钟中断通知 CPU，把该进程放到就绪队列队尾，再取队首进程执行；若进程在时间片内完成则直接撤离，并立即调度。

时间片轮转是绝对抢占式算法，进程未在时间片内运行完就会被强行剥夺处理机。计时依靠时钟中断：例如设时间片 10 ms、时钟中断周期 5 ms，则每次中断把该进程的剩余时间减 5 ms，减到 0 时内核就知道时间片用完、需要切换进程。时间片越短，进程切换次数越多，系统开销越大，真正用于运行用户进程的时间越少；影响时间片大小的主要因素是系统响应时间、就绪队列中的进程数目、系统的处理能力。做题时若要精确到”发生了几次调度”，还要把被抢占进程放回队尾这一动作单独计入。

算例（时间片 = **2 ms**）：

时间区间	运行进程	说明
0–2	P1	t=1 P2 到达入队尾；t=2 P3 到达入队尾；P1 剩余 3，入队尾
2–4	P2	P2 剩余 2，入队尾
4–6	P3	P3 剩余 0，完成，完成时间 **6**
6–8	P1	P1 剩余 1，入队尾
8–10	P2	P2 剩余 0，完成，完成时间 **10**
10–11	P1	P1 完成，完成时间 **11**

进程	到达	服务	完成	周转时间	带权周转时间
P1	0	5	11	11	2.20
P2	1	4	10	9	2.25
P3	2	2	6	4	2.00

平均周转时间 $=(11+9+4)/3=8.00$；平均带权周转时间 $=(2.2+2.25+2.0)/3=2.15$。

RR 的两个做题要点

1. 若某一时刻进程 **Pi** 下处理机，同一时刻进程 **Pj** 到达，默认新到达的进程 **Pj** 先进入就绪队列（即排在被剥夺的 Pi 之前）；

2. 时间片大小的影响：时间片足够大（所有进程都能在一个时间片内完成）时，RR 退化为 **FCFS**；时间片太小会导致进程切换过于频繁，CPU 开销增大，真正用于用户进程的时间减少。时间片长短通常由系统响应时间、就绪队列中的进程数目、系统的处理能力决定。

优点：公平，响应快，不会导致饥饿，适用于分时操作系统，可实现人机交互。缺点：频繁的进程切换带来不小开销；不区分任务的紧急程度。

真题考法

2024-30：10 个进程同时就绪，时间片 5 ms，队尾进程 P 只需 25 ms，求 P 的周转时间。P 需要 5 个时间片，每轮 10×5=50 ms，P 在队尾故每轮最后一个执行，共 5 轮，周转时间 = 5×50 = 250 ms（选 C）。

*5.*优先级调度

算法步骤：每个进程/作业都有优先级，调度时选择优先级最高的进程。可用于作业调度、进程调度、I/O 调度。

- 非抢占式优先级调度：只在进程主动释放处理机时调度；执行过程中若有更高优先级进程到来，只需将其插入就绪队列队头即可。
- 抢占式优先级调度：就绪队列发生变化时就检查是否发起抢占，若有更高优先级进程进入则立即暂停当前进程，把处理机分配给它。
- 静态优先级：创建进程时确定，之后一直不变（依据进程类型、资源要求、用户要求）。
- 动态优先级：创建时给定初值，之后动态调整（依据占用 CPU 与就绪等待时间长短）。

算例（非抢占式，数值越大优先级越高）：

t=0：只有 P1，运行 P1。t=5：P1 结束，P2（优先级 4）、P3（优先级 1）就绪，选 P2。t=9：P2 结束，运行 P3 到 t=11。

进程	到达	服务	优先级	完成	周转时间	带权周转时间
P1	0	5	2	5	5	1.00
P2	1	4	4	9	8	2.00
P3	2	2	1	11	9	4.50

优先级设置原则（必背）：

- 系统进程 > 用户进程；
- 交互型（前台）进程 > 非交互型（后台）进程；
- **I/O** 繁忙型进程 > **CPU** 繁忙型（计算型）进程——I/O 设备与 CPU 可并行工作，且 I/O 设备速度远比 CPU 慢，让 I/O 繁忙型进程优先可让设备尽早投入工作，提升资源利用率和吞吐量。

动态优先级的调整时机：某进程在就绪队列中等待时间过长 → 提高其优先级；某进程占用处理机运行时间过长 → 降低其优先级；某进程频繁进行 I/O → 提高其优先级。

降低优先级的合理时机

只有在进程时间片用完时才适合降低其优先级（2010 年真题答案）。刚完成 I/O 进入就绪队列时不应降低（它此前并未占用处理机）；长期处于就绪队列的进程应提高优先级以免饥饿；从就绪态转为运行态时不应降低。

优点：能区分紧急程度、重要程度，适用于实时操作系统，可灵活调整偏好。缺点：若源源不断有

高优先级进程到来，可能导致饥饿。

*6.*多级队列调度

把就绪队列按进程类型分成多个队列（例如前台/交互进程队列与后台/批处理进程队列），每个队列内部可以使用不同的调度算法，队列之间可以有固定的优先级（如只有高优先级队列为空时才调度低优先级队列），也可以按时间比例分配 CPU。

缺点：进程一旦进入某个队列就不能在队列间移动，若队列间采用固定优先级，低优先级队列中的进程仍可能饥饿。多处理机中这种单一调度策略的缺点更突出。

*7.*多级反馈队列（*MLFQ*）

算法步骤：

1. 设置多个就绪队列，优先级从高到低（队列 1 最高）；各队列的时间片从小到大（例如队列 1 时间片 8 ms，队列 2 时间片 16 ms，队列 3 按 FCFS）；
2. 新进程一律进入队列 **1** 队尾，按 FCFS 等待调度；
3. 队列 1 中的进程若在一个时间片内完成则撤离；若未完成则转入队列 **2** 队尾；
4. 仅当第 1 级队列为空时才调度第 2 级队列；仅当第 1、2 级都为空时才调度第 3 级；
5. 若正在执行第 **k** 级队列的进程时，有更高优先级队列（**1**～**k-1** 级）的新进程到达，则被抢占，原进程放回第 **k** 级队列队尾；
6. 处于最低优先级队列的进程按时间片轮转（或 FCFS）反复执行，不再降级；通过在低优先级队列中等待过久的进程提升到高优先级队列（”老化”）来防止饥饿。

算例（**2019-27** 原型）：二级反馈队列，Q1 时间片 10 ms 用 RR，Q2 用短进程优先；Q1 优先，新进程先进 Q1，Q1 中执行完一个时间片未结束则转入 Q2。当前 Q1、Q2 为空，依次创建 P1（需 30 ms）、P2（需 20 ms）后立即开始调度。

- 0–10：P1 在 Q1 运行 10 ms，剩余 20 ms，转入 Q2 队尾；
- 10–20：P2 在 Q1 运行 10 ms，剩余 10 ms，转入 Q2 队尾；
- 此时 Q1 为空，调度 Q2（短进程优先）：P2 剩余 10 ms 先执行（20–30），再执行 P1 剩余 20 ms（30–50）。

P1 在 0–10 运行、10–30 就绪等待（20 ms）、30–50 运行，故 P1 等待时间为 20 ms；P2 在 0–10 就绪等待（10 ms）、10–20 运行，故 P2 等待时间为 10 ms。平均等待时间 = **(20+10)/2 = 15 ms**。

2019 年这道题的踩坑点

算 P1 的等待时间时极易漏掉**”P2** 在 **Q1** 执行的 **10 ms** 期间 **P1** 在等待**”** 这一段。必须把每个时刻谁在运行、谁在就绪逐段列出，再累加就绪时间。

优点：可动态调整进程优先级和时间片大小，兼顾多方面系统目标；能区分对待 CPU 密集型和 I/O 密集型进程（I/O 阻塞后回到原队列可保持较高优先级）；短作业周转时间短，一定程度上避免饥饿。缺点：算法设计复杂，极端情况下仍可能饥饿。

设计 MLFQ 需要考虑的因素（2020 年真题答案选” 全部四项”）：就绪队列的数量、就绪队列的优先级、各就绪队列的调度算法、进程在就绪队列间的迁移条件。

*8.*同一组进程用不同算法的完整对照表

仍用上面那组进程（P1：到达 0、服务 5；P2：到达 1、服务 4；P3：到达 2、服务 2），非抢占式算法中优先级的数值越大优先级越高：

算法	执行顺序	**P1** 周转	**P2** 周转	**P3** 周转	平均周转	平均带权周转	是否可能饥饿
FCFS	P1 → P2 → P3	5	8	9	7.33	2.50	否
SJF（非抢占）	P1 → P3 → P2	5	10	5	6.67	2.00	是
SRTN（抢占）	P1 → P3 → P1 → P2	7	10	2	**6.33**	**1.63**	是
HRRN（非抢占）	P1 → P3 → P2	5	10	5	6.67	2.00	否
RR（时间片 2）	P1 → P2 → P3 → P1 → P2 → P1	11	9	4	8.00	2.15	否
优先级（非抢占）	P1 → P2 → P3	5	8	9	7.33	2.50	是

从对照表能读出的三条规律

1. 平均周转时间最短的是抢占式 **SRTN**，其次是非抢占式 **SJF/HRRN**——只要题目问” 平均等待时间/平均周转时间最少”，先考虑 SRTN（有” 同时到达” 前提时可答 SJF）；
2. **RR** 的平均周转时间通常较大，但它的优势在于响应时间短、公平、不饥饿；
3. **FCFS** 与优先级调度的本算例结果相同只是巧合（恰好优先级顺序与到达顺序一致），不要据此认为两者等价。

（五）多处理机调度

*1.*与单处理机调度的区别

多处理机调度要额外解决两个问题：亲和性与负载均衡。

1. 亲和性（**Affinity**）：让一个进程尽量总在同一个 CPU 上运行，以保持 Cache 命中率，避免频繁到内存取数据。软亲和指尽量让进程在同一 CPU 上运行但不强制；硬亲和可写入 PCB，表示该进程只能由某一个 CPU 运行。
2. 负载均衡：各 CPU 的繁忙程度应大致相等，不能一个 CPU 长期空闲而另一个长期繁忙。

*2.*两种就绪队列组织方式

方式	做法	优点	缺点
公共就绪队列	所有进程共享一个就绪队列	队列需要上锁互斥（防止多个 CPU 同时抢同一个进程），天然实现负载均衡	亲和性差（进程可能被不同 CPU 调度）
私有就绪队列	每个 CPU 拥有自己的就绪队列	天然解决亲和性问题	需要额外的迁移机制实现负载均衡

负载均衡通过迁移实现：推迁移（**push migration**）是系统程序把负载高的 CPU 上的进程推到负载低的 CPU 上；拉迁移（**pull migration**）是空闲 CPU 从超载 CPU 的就绪队列中” 拉” 一些进程过来

运行。推迁移和拉迁移可以并行执行。

（六）上下文及其切换机制

*1.*上下文的组成

上下文（**Context**）是进程/线程在某一时刻的处理器状态，主要包含：

- 程序计数器 PC（下一条要执行的指令地址）；
- 通用寄存器组的值；
- 栈指针 SP、栈基址寄存器的值；
- 程序状态字 PSW、标志寄存器；
- 地址空间相关寄存器（页基址寄存器 / 页目录基址寄存器 PDBR）；
- 其他控制寄存器、条件码寄存器。

上下文的保存位置是 **PCB**，不是栈。（进程的内核栈只保存函数调用信息与中断现场；PCB 中保存的才是切换时恢复的上下文。）

*2.*进程切换的完整过程

1. 挂起当前进程，把它的处理机上下文保存到它的 PCB 中；
2. 把该 PCB 移入相应队列（就绪队列或阻塞队列）；
3. 由调度程序选择另一个进程，更新它的 **PCB**：把状态从就绪改为运行，记录本次开始运行的时间（用于统计 CPU 占用率、响应时间），重置或更新与调度策略相关的字段（如把时间片重置为系统规定的初始值）；
4. 从新进程的 PCB 中取出上下文并装入 CPU 各寄存器；
5. 跳转到新进程的程序计数器所指向的位置继续执行。

Linux 的实现顺序正是如此：`__schedule()` 先 `pick_next_task()` 选出下一个进程，若 `prev != next` 则调用 `context_switch()`：先 `switch_mm_irqs_off()` 切换地址空间（页表），再 `switch_to()` 切换寄存器上下文。

> **为什么”进程切换”必须切换地址空间，而”中断处理”不用**
>
> 中断处理前后仍是同一个进程、同一个地址空间，只是从用户态陷入内核态（用户栈切到内核栈），不涉及地址空间切换，因此中断处理程序的执行不算进程切换。进程切换必须切换页表，所以涉及地址空间切换。

*3.*切换的开销

- 上下文切换是纯开销，切换期间 CPU 不做任何有用工作；
- 页表切换（地址空间切换）在开销中占比很大，还要加上 TLB/Cache 失效的代价；
- 保存和恢复寄存器需要执行多条 `load/store` 指令，比较耗时；
- 优化手段：设置两组寄存器组（一组供内核、一组供用户），切换时只需改指针，无须搬移寄存器内容。

*4.*中断响应、模式切换与进程切换的区别

对比项	中断响应	进程切换
上下文保存/恢复的时机	分开进行，分别位于中断响应和中断返回中	连续进行（两个不同进程之间）
保存的上下文	中断前进程执行的上下文（可能是用户上下文）	一定是内核态下的上下文
是否切换地址空间	不涉及	涉及（切换页表）
特权级	可能发生特权级切换	全程在内核态下进行

模式切换 **vs** 进程切换：

- 模式切换发生在用户态响应中断/异常（用户态 → 核心态）和处理完中断异常后返回用户态（核心态 → 用户态）两种情况；它是同一进程在自身用户态和内核态之间的切换，二者共用地址空间，涉及特权级切换与用户栈/内核栈切换，只需保存和恢复用户态上下文（通用寄存器、PC、PSW、SP 等断点和现场信息）；
- 进程切换发生在内核态，是从一个进程的内核态切换到另一个进程的内核态。PCB 中保存的是进程在内核态被切换时的完整状态；用户态的恢复依赖中断返回机制，不是进程切换的直接结果；
- 因此：任何进程的切换都依靠内核完成；进程的调度和切换均发生在内核态下（注意：用户级线程之间的切换可以由线程库在用户态完成，这是特例）。

典型错误认知

错误：”进程切换是直接在用户态之间切换，PCB 中保存的是用户态上下文。”正确：

1. 切换永远发生在两个进程的内核态之间；
2. PCB 保存的是进程在内核态被切换时的完整状态；
3. 用户态恢复依赖中断返回机制，不是进程切换的直接结果。

真题考法

2024-25：支持页式存储管理的系统中，进程切换时要执行的操作包括”更新程序计数器的值”“更新栈基址寄存器的值”“更新页基地址寄存器的值”——三项都要做。2018 年曾考”同一进程不同线程切换时页基址寄存器是否需要改动”（不需要，因为共用同一虚拟地址空间，原有 TLB 不会失效）。

三、同步与互斥

（一）同步与互斥的基本概念

*1.*临界资源与临界区

临界资源（**Critical Resource**）：一个时间段内只允许一个进程使用的资源。物理设备（摄像头、打印机）是临界资源；共享变量、数据、内存缓冲区同样是临界资源。

临界区（**Critical Section**）：进程中访问临界资源的那段代码。注意一个临界资源的相关临界区是由不同进程各自的临界区共同构成的——各进程对同一临界资源的操作不同，代码段自然也不同。

对临界资源的互斥访问在逻辑上分为四个部分：

部分	作用
进入区	检查能否进入临界区，并设置正在访问临界区的标志
临界区	访问临界资源的代码
退出区	将正在访问临界区的标志清除
剩余区	代码的其余部分

可重入代码

可重入代码是可以被重复调用执行、中断后重新执行也不会出错的代码，可同时被多个进程调用，不属于临界资源；可重入代码不允许被任何进程修改。进程映像中的程序段若被共享，必须是可重入代码。

*2.*同步与互斥

概念	核心思想	定义	等待原因
互斥（**Mutual Exclusion**）	竞争	多个进程因争夺同一个、临时的、不可共享的资源而产生的制约关系（间接制约）	资源正忙
同步（**Synchronization**）	协作	多个进程为共同完成任务，在执行次序上需要遵循某种先后顺序而产生的制约关系（直接制约）	前驱任务未完成，条件不满足

*3.*四条原则

原则	含义
空闲让进	临界区空闲时，应允许一个请求进入临界区的进程立即进入
忙则等待	已有进程进入临界区时，其他试图进入的进程必须等待（即互斥）
有限等待	对请求访问的进程，应保证它能在有限时间内进入临界区（不会饥饿）
让权等待	进程不能进入临界区时，应立即释放处理机（主动阻塞），防止忙等

核心结论

前三条是必须实现的；”让权等待”是可选原则——Peterson 算法、TSL/Swap 指令、自旋锁都不满足让权等待，但它们实现互斥是正确的。2020-32 题的答案就是”仅Ⅰ、Ⅱ、Ⅲ”。

真题考法

2020-32 直接考四条原则；2018-32 考”下列同步机制中可以实现让权等待的是”（答案：信号量方法）；2016-27 考 TSL 实现互斥的性质（答案：等待进入临界区的进程不会主动放弃 CPU）。

（二）基本的实现方法

*1.*软件方法

判断某一软件方法是否满足四条原则的统一方法：

- 判断空闲让进：观察是否存在”表达式 1 与表达式 2 同时成立”的可能；再看没有对方进程时，本进程能否通过循环进入临界区；

- 判断忙则等待：看本进程通过了 while 循环但还没进入临界区时切换到对方，对方能否也通过 while 循环；
- 判断有限等待：本进程卡在进入区时，新来的同类进程能否不断抢先进入；
- 所有软件方法都不满足让权等待：进入区的 `while(表达式)`；循环体为空语句，一直空转，这就是忙等。

（**1**）单标志法

```
// 共享整型变量
int turn = 0;   // turn = i 表示允许进程 Pi 进入临界区

/* 进程 P0 */
while (TRUE) {
    while (turn != 0);   // 若 turn 不等于本进程编号，则循环等待
    临界区;
    turn = 1;            // 访问完毕，把权限让给对方
    剩余区;
}
```

```
/* 进程 P1 */
while (TRUE) {
    while (turn != 1);
    临界区;
    turn = 0;
    剩余区;
}
```

- 满足忙则等待；
- 两个进程必须交替进入临界区。若其中一个进程不再进入临界区，另一个进程因得不到授权也永远进不去，违背**”** 空闲让进**”**；
- 使用 `while` 判断作为进入区，违背**”** 让权等待**”**。

（**2**）双标志先检查

```
bool flag[2];           // flag[i] = TRUE 表示进程 Pi 想进入临界区
flag[0] = flag[1] = FALSE;

/* 进程 P0 */
while (TRUE) {
    while (flag[1] == TRUE);   // 先检查对方是否想进
    flag[0] = TRUE;            // 对方不想进，自己上锁
    临界区;
    flag[0] = FALSE;
    剩余区;
}
```

- 解决了单标志法” 必须交替进入” 的缺点，可以连续使用；
- 缺点：若 P0 通过了 while 检查但尚未执行 `flag[0] = TRUE` 时发生进程切换，P1 也能通过检查并上锁，两个进程可能同时进入临界区，违背**”** 忙则等待**”**。根源在于” 检查” 与” 上锁” 不是一气呵成的，并发程序不能保证 **load** 和 **store** 的原子性。

（**3**）双标志后检查

```
/* 进程 P0 */
while (TRUE) {
    flag[0] = TRUE;             // 先上锁
    while (flag[1] == TRUE);    // 再检查对方
    临界区;
    flag[0] = FALSE;
    剩余区;
}
```

- 解决了先检查法可能同时进入的问题；
- 缺点：若 P0 置 `flag[0] = TRUE` 后发生切换，P1 也置 `flag[1] = TRUE`，则双方都在 while 中无限轮询，谁也进不去，违背" 空闲让进" 和" 有限等待"。

（4）Peterson 算法（正确的软件方案）

```
int turn;             // 表示该让谁进入临界区
bool flag[2];         // 表示进程是否希望进入临界区
flag[0] = flag[1] = FALSE;

/* 进程 P0 */
while (TRUE) {
    flag[0] = TRUE;                       // 1. 表示本进程希望进入
    turn = 1;                             // 2. 礼让 P1
    while (flag[1] == TRUE && turn == 1); // P1 想进且轮到 P1，则等待
    临界区;
    flag[0] = FALSE;                      // 3. 退出临界区
    剩余区;
}
```

```
/* 进程 P1 */
while (TRUE) {
    flag[1] = TRUE;
    turn = 0;
    while (flag[0] == TRUE && turn == 0);
    临界区;
    flag[1] = FALSE;
    剩余区;
}
```

逐条验证四条原则：

- 空闲让进：若 P1 不打算进入临界区，则 `flag[1] = FALSE`，P0 的 while 条件为假，可以立即进入，不会像单标志法那样" 临界区空闲却进不去"；
- 忙则等待：双方都希望进入时由 `turn` 决定，双方互相礼让，后礼让的一方等待，保证同一时刻只有一个进程在临界区；
- 有限等待：P0 进入临界区后，P1 等待；P0 出来时会置 `flag[0] = FALSE`，P1 随即可以进入，不会饥饿；
- 不满足让权等待：进不去时一直执行 while 空循环，直到时间片用完。

软件方法小结：

方法	思想	违背的原则
单标志法	引入共享变量 turn，循环检查 turn 是否为本进程编号	空闲让进（和让权等待）
双标志先检查	引入 flag 数组，先检查对方再上锁	忙则等待
双标志后检查	先设置自身 flag，再检查对方	空闲让进、有限等待
Peterson 算法	引入 turn 与 flag，检查两个条件	让权等待（且只适用于两个进程）

*2.*硬件方法

（**1**）中断屏蔽方法

```
...
关中断;
临界区;
开中断;
...
```

思想：利用” 开/关中断指令” 实现，与原语的实现思想相同——某进程开始访问临界区到结束访问为止都不允许被中断，也就不可能发生进程切换，因此不可能出现两个进程同时访问临界区。由于进程切换通常由中断或系统调用触发，对于单处理机环境，只需在进入临界区时禁止中断，就能保证当前指令流有序执行、不被抢占。非抢占式内核常采用此方法。

优点：简单、高效。缺点：

- 要求所有调用线程都能执行特权操作，因此只适用于操作系统内核进程，不适用于用户进程（开/关中断是特权指令，2021 年真题考点）；
- 只能用于单处理机环境。多处理机下，即使关闭本 CPU 的中断，其他 CPU 上的线程依然可以进入临界区；
- 频繁关中断会影响系统时钟的更新与时间片计时，因此只能在很短的时间内禁用中断。

（**2**）**TestAndSet** 指令（**TS**，也称 **TSL**）

TSL 指令由硬件实现，执行是原子的、不可被中断，即使两条 TSL 指令在不同 CPU 上同时执行，也会按任意次序顺序执行。功能是” 测试并设置”：

```
// 硬件原子指令（伪代码）：返回 lock 的旧值，同时把 lock 置为 TRUE
bool TestAndSet(bool *lock) {
    bool old = *lock;
    *lock = TRUE;
    return old;
}
```

1. 从内存单元读取值；
2. 测试该值是否为 TRUE，返回真或假；
3. 把内存单元的值设置为 TRUE。

```
do {
    ...
```

```
    while (TSL(&lock));   // 上锁失败则一直轮询
    critical section;      // 临界区
    lock = FALSE;          // 退出区解锁
    ...
} while (TRUE);
```

分析：若开始时 lock 为 FALSE，TSL 返回 FALSE 并把 lock 置为 TRUE，while 条件不满足，进程进入临界区；若开始时 lock 为 TRUE，TSL 返回 TRUE，while 条件满足，一直轮询，直到临界区中的进程在退出区解锁或自己时间片耗尽下处理机。

相比软件方法，TSL 把”上锁”和”检查”用硬件变成了一气呵成的原子操作，避免了双标志先检查法的错误。

（3）Swap 指令（EXchange / XCHG）

```
bool key = TRUE;
do {
    key = TRUE;
    do {
        Swap(&key, &lock);   // 原子地交换 key 与 lock
    } while (key == TRUE);
    critical section;
    lock = FALSE;
    ...
} while (TRUE);
```

思想：先用 key 记录之前是否被上锁，然后自己主动上锁（lock = TRUE），再检查 key 是否是 TRUE；若 key 为 FALSE，说明自己是第一个上锁的，可以进入临界区。

逻辑上 Swap 与 TSL 并无太大区别。

两种硬件指令的优缺点：

- 优点：实现简单，容易验证正确性；适用于任意数目的进程，单处理机和多处理机都适用；支持系统中有多个临界区，只需为每个临界区设立一个布尔型变量；
- 缺点：不满足”让权等待”，无法进入临界区的进程会占用 CPU 循环执行 TSL/Swap，导致忙等；从等待进程中随机选择进入临界区，有的进程可能一直选不上，导致饥饿。

TSL 的三个关键结论（2016 年真题）

1. 等待进入临界区的进程不会主动放弃 **CPU**（所以它处于就绪态/运行态，不是阻塞态）；
2. TSL 机制下不存在阻塞态的进程，会一直轮询；
3. while (TSL(&lock)) 应在开中断状态下执行——TSL 本身是一条原子指令，不需要关中断；若整段程序都关中断，则可能很久都不响应中断。

（三）锁

1.互斥锁

用互斥锁保护临界区：进程在进入临界区时 acquire() 获取锁，退出时 release() 释放锁。每个锁有一个 locked 变量表示锁是否可用。

互斥锁必须用硬件机制实现，因为 acquire() 中”检查锁是否可用”和”上锁”之间若发生进程切换，可能违背忙则等待，让多个进程进入临界区。实现方式有两种：获取锁之前关中断（只对单处理机有效）；或者采用原子操作指令（TSL / Swap）。

*2.*自旋锁与忙等待

自旋锁：申请锁失败后一直循环申请，直到成功。

```
// 使用 TSL 指令实现自旋锁
struct Lock { int locked; };   // 0 表示未被占用，1 表示被占用

acquire(Lock *lock) {
    while (TestAndSet(&lock->locked)) {
        // CPU 在此" 忙等待" 或" 自旋"
    }
}

release(Lock *lock) {
    lock->locked = 0;
}
```

自旋锁申请失败时进程处于什么状态

就绪态或运行态。若时间片没用完，则处于运行态一直尝试获取锁；若时间片用完被剥夺处理机，则处于就绪态。注意：单处理器上自旋锁需要抢占式调度器才能使用，否则自旋的线程永远不会放弃处理机。

自旋锁特性总结：

- 具有随机性，不能保证按先来后到顺序使用 CPU 资源；
- 需要忙等，时间片用完才下处理机，违反”让权等待”；
- 等待锁时没有上下文切换（上下文切换开销很大），因此在多处理器系统中若上锁时间很短，等待代价很低；
- 常用于多处理机系统，一个核忙等，其他核照常工作并能快速释放临界区；
- 不适用于单处理机系统（忙等过程中不可能解锁），即使有抢占式调度器性能开销也很大；
- 不提供任何公平性保证，自旋的线程在竞争条件下可能永远自旋，可能导致饿死。

无忙等待锁：申请锁失败后主动阻塞自己，加入该锁的等待队列并让出处理机；其他进程释放锁后从等待队列中唤醒一个进程加入就绪队列。

```
struct Lock {
    int locked;
    waitqueue q;          // 该锁对应的等待队列
};

acquire(Lock *lock) {
    while (TestAndSet(&lock->locked)) {
        block the process into q;   // 主动阻塞
        schedule();                 // 发起调度
    }
}

release(Lock *lock) {
    lock->locked = 0;
    wake a process from q;          // 从等待队列唤醒一个阻塞进程
}
```

无忙等待锁特性：由于有等待队列，可以实现申请锁进程的先来后到，较为公平；未申请到锁时不需要忙等，但要阻塞自己并切换其他进程，存在上下文切换的代价。

忙等待的代价

忙等的本质是”用 CPU 时间换取不发生上下文切换”。因此：

- 临界区很短、多处理器 → 自旋锁划算；
- 临界区很长、单处理器 → 自旋锁极其浪费，应使用阻塞式锁。

（四）信号量

*1.*整型信号量

用一个整数型变量作为信号量，表示系统中某种资源的数量。与普通整型变量的区别是：对信号量的操作只有三种——初始化、**P** 操作、**V** 操作，不能进行值判断。

```
int S = 1;   // 整型信号量，初值表示资源数量

void wait(int S) {          // P 操作，相当于进入区
    while (S <= 0);         // 资源不够就一直轮询（忙等）
    S = S - 1;              // 申请到一个资源
}

void signal(int S) {        // V 操作，相当于退出区
    S = S + 1;              // 释放一个资源
}
```

缺点：`wait` 中用 `while (S <= 0)` 检测资源是否可用，若资源不够则一直轮询，违反”让权等待”。

*2.*记录型信号量

用一个记录型数据结构表示信号量，不存在忙等现象，遵循让权等待。

```
typedef struct {
    int value;          // 剩余资源数
    struct PCB *L;      // 等待队列（PCB 链表，通常用 FIFO）
} semaphore;

void wait(semaphore S) {          // P 操作
    S.value--;                    // 先减 1
    if (S.value < 0) {            // 资源已不够分配
        block(S.L);               // 自我阻塞，由运行态 -> 阻塞态
    }                             // 并插入该信号量的等待队列
}

void signal(semaphore S) {        // V 操作
    S.value++;                    // 先加 1
    if (S.value <= 0) {           // 仍有进程在等待该资源
        wakeup(S.L);              // 唤醒等待队列中的第一个进程
    }                             // 被唤醒进程由阻塞态 -> 就绪态
}
```

物理意义（必背）：

- `S.value > 0`：表示当前还可用的资源数量；
- `S.value = 0`：表示资源刚好用完，没有进程在等待（注意：不是”没有资源”）；
- `S.value < 0`：表示资源已分配完，且有 **|S.value|** 个进程正在等待该资源。

出现负值的原因是 `wait()` 中先递减再判断，互换了递减和测试的顺序；而在整型信号量中信号量的值不能为负。

举例说明 `signal()` 中 `S.value <= 0` 的含义：

设 `sem = 1`：

1. 第一个线程 P 操作后，`sem = 0`（刚好拿走最后一个资源）；
2. 第二个线程 P 操作后，`sem = -1`（资源不够，该线程阻塞在等待队列，等待队列长度 1）；
3. 第三个线程 P 操作后，`sem = -2`（第三个线程也阻塞，等待队列长度 2）；
4. 第一个线程执行 V 操作后，`sem = -1`，因为 `sem <= 0`，说明仍有进程在等待，于是唤醒第二个线程；第二个线程执行完后释放，`sem = 0`，此时仍有第三个线程在等待，再唤醒它；最后第三个线程释放后 `sem = 1`，回到初始状态。

三个必须记住的结论

1. **P(S)**、**V(S)** 都是原语，执行必须一气呵成、不可被打断。对信号量的访问必须是互斥的，因为信号量 S 是多个进程共享的变量，多个进程都可对它读写——这本身就是一个临界区问题。单处理机上可通过关中断实现；多处理机上需要用自旋锁或 `compare_and_swap` 保证原子性。
2. 考试中出现 **P(S)**、**V(S)** 操作，除非特殊说明，否则默认 **S** 为记录型信号量。
3. 信号量是结构体，只能进行 **P/V** 操作，不能进行值判断。写 `if (bowl >= 1)` 是极其错误的。

*3.*用信号量实现互斥

步骤：

1. 分析并发进程的关键活动，划定临界区；
2. 设置互斥信号量 **`mutex`**，初值为 **1**；
3. 在进入区 `P(mutex)`——申请资源；
4. 在退出区 `V(mutex)`——释放资源。

```
semaphore mutex = 1;    // 互斥信号量

P1() {
    ...
    P(mutex);           // 进入区：申请锁
    临界区;
    V(mutex);           // 退出区：释放锁
    ...
}
```

注意：对不同的临界资源要设置不同的互斥信号量；**P**、**V** 操作必须成对出现——缺少 `P(mutex)` 不能保证互斥，缺少 `V(mutex)` 会导致资源永不释放、等待进程永不被唤醒；P、V 必须夹紧使用互斥资源的行为，中间不能有其他冗余代码。

*4.*用信号量实现同步

步骤：

1. 分析什么地方需要实现同步关系，即必须保证” 一前一后” 执行的两个操作；
2. 设置同步信号量 **S**，初值为 **0**；
3. 在” 前操作” 之后执行 **V(S)**；

4. 在”后操作”之前执行 **P(S)**。

```
semaphore S = 0;    // 同步信号量

P1() {
    ...
    语句 x;          // 前操作
    V(S);
    ...
}

P2() {
    ...
    P(S);
    语句 y;          // 后操作，要使用 x 的运行结果
    ...
}
```

同步信号量的初值不一定是 0，也可以是正整数，取决于”所期待的条件是否已经发生”。例如初值为 2 时，前驱进程释放两次后资源数达到 2，后进程才能继续。

*5.*用信号量实现前驱关系

每一对前驱关系就是一个同步问题，因此：

1. 为每一对前驱关系各设置一个同步信号量，初值均为 **0**；
2. 在”前操作”之后对相应的同步信号量执行 **V** 操作；
3. 在”后操作”之前对相应的同步信号量执行 **P** 操作。

例如前驱图 $S_1 \to S_2$、$S_1 \to S_3$、$S_2 \to S_4$、$S_3 \to S_4$，则需 4 个信号量 a, b, c, d（初值全 0），S_1 末尾写 `V(a); V(b);`，S_2 开头写 `P(a);`、末尾写 `V(c);`，S_3 开头写 `P(b);`、末尾写 `V(d);`，S_4 开头写 `P(c); P(d);`。

三句话总结 P/V 模板

互斥：`P(mutex)` …临界区…`V(mutex)`，`mutex` 初值 1，夹紧临界区。同步：前操作之后 `V(S)`，后操作之前 `P(S)`，`S` 初值 0。前驱：每对前驱关系一个信号量，初值 0，前驱末尾 V，后继开头 P。

（五）条件变量

*1.*为什么要引入条件变量

用信号量解决同步问题虽然强大，但同步问题越复杂、条件越多，需要的信号量就越多，必须非常谨慎地处理信号量之间的顺序，否则很容易造成死锁。为此产生了管程（**Monitor**）：管程把共享数据结构和对它的操作封装起来，并保证每次只有一个进程使用管程，天然实现互斥，程序员无需自己实现互斥；管程还提供条件变量，让程序员灵活实现同步。

管程的组成：管程名称、局部于管程的共享数据结构说明、对该数据结构进行操作的一组过程（函数）、对局部于管程的共享数据设置初始值的语句。

管程的特性：

- 把操作函数封装在管程内（类似 Java 的类），外部进程只能通过调用操作函数来获取和释放资源；

- 每次只允许一个进程使用管程，天然实现互斥；
- 管程内的数据只能被管程内的操作函数修改；
- 管程是被进程调用的，进程通过调用管程中定义的操作来访问共享资源；
- 管程是语法范围（语言级别的抽象），由编程语言的语法定义；
- 无法创建和撤销，生命周期与程序一致。

*2.*条件变量的用法

条件变量的本质是" 进程进入管程后因某原因被阻塞，把该原因定义为一个条件变量"。不同进程被阻塞的原因可能不同，所以可以有多个条件变量；每个条件变量保存一个等待队列。

```
// 在管程内部
condition x;                    // 声明一个条件变量

// 资源不够时（注意：先判断资源是否够，再决定是否 wait）
while (资源不够) {
    x.wait();                   // 当前进程阻塞在该条件变量上，并释放管程
}
// 资源可用时
x.signal();                     // 唤醒等待在该条件变量上的一个进程
```

对条件变量使用 `while` 循环而不是 `if`，可以解决假唤醒（**spurious wakeup**）问题。

*3.*条件变量与信号量的区别

对比项	信号量	条件变量
是否有值	有值，代表资源数量	没有值，只实现排队等待和唤醒功能（资源数由管程内的共享数据结构记录）
操作	`wait(S)` / `signal(S)`（P/V）	`x.wait()` / `x.signal()`
释放操作的影响	**V** 操作一定会改变信号量的值	`signal` 操作可能不产生任何影响（当没有进程因该条件变量阻塞时）
是否需要额外互斥	需要自己保证 P/V 原子性	管程天然实现互斥，程序员无需自己处理

核心结论

管程中的 `signal` 操作和信号量的 `V` 操作只是类似，而不是相同——这是高频易错点。

（六）经典同步问题

*1.*生产者-消费者问题

问题描述：一组生产者进程和一组消费者进程共享一个初始为空、大小为 **n** 的缓冲区。生产者每次生产一个产品放入缓冲区，消费者每次从缓冲区取出一个产品。

三条约束：

1. 只有缓冲区没满时，生产者才能放入产品，否则必须等待（同步关系）；
2. 只有缓冲区不空时，消费者才能取出产品，否则必须等待（同步关系）；
3. 缓冲区是临界资源，各进程必须互斥访问（互斥关系）。

信号量设置：

```
semaphore mutex = 1;     // 互斥访问缓冲区
semaphore empty = n;     // 空闲缓冲区的数量（同步）
semaphore full  = 0;     // 已有产品的数量（同步）
```

完整伪代码：

```
semaphore mutex = 1;
semaphore empty = n;
semaphore full  = 0;

producer() {
    while (1) {
        生产一个产品;
        P(empty);              // 1. 申请一个空闲缓冲区（同步 P）
        P(mutex);              // 2. 申请缓冲区互斥锁（互斥 P）
        把产品放入缓冲区;
        V(mutex);              // 3. 释放缓冲区互斥锁
        V(full);               // 4. 产品数加 1，唤醒可能的消费者
    }
}

consumer() {
    while (1) {
        P(full);               // 1. 申请一个产品（同步 P）
        P(mutex);              // 2. 申请缓冲区互斥锁
        从缓冲区取出一个产品;
        V(mutex);              // 3. 释放缓冲区互斥锁
        V(empty);              // 4. 空闲缓冲区加 1，唤醒可能的生产者
        消费产品;
    }
}
```

为什么这样设计（关键分析）：

1. 必须先对同步信号量 **P**，再对互斥信号量 **P**（即 P(empty) 在 P(mutex) 之前）。若颠倒顺序：假设缓冲区已满，生产者先 P(mutex) 拿到锁，再 P(empty) 时发现 empty = 0 而被阻塞；此时消费者想进入临界区消费，却因为 mutex = 0 而被阻塞。双方互相等待对方唤醒自己 → 死锁。
2. **V** 操作的顺序可以交换，因为 V 操作不会引起阻塞（V(mutex) 与 V(full) 谁先谁后都不影响正确性）。
3. ”生产一个产品”和”消费产品”这两个动作最好放在临界区之外。虽然从逻辑上也可以放进临界区，但那样会延长临界区的长度，降低并发度、降低系统效率。
4. 注意唤醒关系：消费者可以唤醒生产者（缓冲区空时），生产者也可以唤醒生产者（缓冲区满时第二个生产者阻塞，第一个生产者完成后将它唤醒）。

单缓冲与多缓冲的区别：当 $n=1$（单缓冲区）时，empty 与 full 的取值只能是 0 或 1，此时 empty 或 full 这两个一般信号量已经天然充当了互斥信号量，可以省去 mutex。但当缓冲区数量 ≥ 2 时，必须用 **mutex** 实现互斥访问。

生产者-消费者问题的四个易错点

1. 顺序不能颠倒：同步 P 必须在互斥 P 之前；
2. **P**、**V** 必须成对：漏掉 V(mutex) 会让缓冲区永不释放；
3. 单缓冲区可省 **mutex**，多缓冲区不可省；
4. 题目中的”产品”/”数据”就是普通资源，不要与后面的信号混淆。

多生产者多消费者（放水果问题）：

爸爸专放苹果，妈妈专放橘子，儿子专吃橘子，女儿专吃苹果，盘子只能放一个水果。

```
semaphore plate  = 1;    // 盘子空位只有 1 个，可充当互斥信号量
semaphore apple  = 0;
semaphore orange = 0;

爸爸 () { while (1) { P(plate); 往盘子里放苹果;   V(apple);  } }
妈妈 () { while (1) { P(plate); 往盘子里放橘子;   V(orange); } }
儿子 () { while (1) { P(orange); 从盘子取出橘子; V(plate); 吃橘子; } }
女儿 () { while (1) { P(apple);  从盘子取出苹果; V(plate); 吃苹果; } }
```

为什么不设 mutex 也能互斥

因为盘子空位只有 1，且 apple、orange、plate 在任何时刻最多只有一个为 **1**，四个进程执行 P 操作时最多只有一个人不会被阻塞，因此天然实现了互斥。缓冲区数量为 **1** 时可能天然互斥，但缓冲区数量为 **2** 时必须用 **mutex**。

共享缓冲区导致的隐含死锁（必考难点）：

两个生产者 A、B 共用一个大小为 N 的缓冲区，消费者需要”一个 A 和一个 B”才能消费。若 A 推进过快把缓冲区全部占满 A，消费者已持有 A 却还需要 B 而 B 放不进去（缓冲区满）→ 死锁。

解决思路：把问题转化为双缓冲区问题，构造两个虚拟缓冲区，各留出配额：

```
semaphore empty    = N;      // 箱子内空位数量
semaphore frame    = 0;      // 箱子内车架数量
semaphore wheel    = 0;      // 箱子内车轮数量
semaphore mutex    = 1;      // 互斥访问箱子
semaphore frameMax = N - 2;  // 最多还能放几个车架（保证至少给车轮留 2 个空位）
semaphore wheelMax = N - 1;  // 最多还能放几个车轮（保证至少给车架留 1 个空位）

工人 1() {   // 生产车架
    while (1) {
        加工一个车架;
        P(frameMax);
        P(empty);
        P(mutex);
        车架放入箱中;
        V(mutex);
        V(frame);
    }
}

工人 2() {   // 生产车轮
    while (1) {
        加工一个车轮;
        P(wheelMax);
        P(empty);
        P(mutex);
        车轮放入箱中;
```

```
        V(mutex);
        V(wheel);
    }
}

工人 3() {   // 组装
    while (1) {
        P(frame); P(mutex); 箱中取出一个车架; V(mutex); V(empty); V(frameMax);
        P(wheel); P(wheel); P(mutex); 箱中取出二个车轮; V(mutex);
        V(empty); V(empty); V(wheelMax); V(wheelMax);
        组装为一台车;
    }
}
```

*2.*读者-写者问题

问题描述：有读者和写者两组并发进程共享一个文件。要求：

- 允许多个读进程同时对文件执行读操作（读者读数据后不改变数据，与消费者不同）；
- 同一时刻只允许一个写者写；
- 任一写者在完成写操作之前不允许其他任何进程工作；
- 写者执行写操作之前，应让已有的读者、写者全部退出。

这是典型的分类互斥（**categorical mutual exclusion**）：同类进程不互斥，异类进程互斥。

通用解题模板（**”** 三段式**”**）：本质是” 连续多个同类进程访问同一个临界资源”——第一个进程开始访问前对资源加互斥锁，后续同类进程不再加锁；最后一个进程访问完后解锁。就像第一个人进房间时开门，后面的人不用开门，最后一个人离开时关门。

（**1**）读者优先

```
semaphore rw    = 1;    // 用于读者与写者、写者与写者之间的互斥
semaphore mutex = 1;    // 用于对 count 变量的互斥访问
int count = 0;          // 正在读的读者数量

writer() {
    while (1) {
        P(rw);              // 写之前加锁
        写文件;
        V(rw);              // 写完之后解锁
    }
}

reader() {
    while (1) {
        P(mutex);           // 各读者互斥地访问 count
        if (count == 0)     // 我是第一个读者
            P(rw);          // 由我负责对文件加锁
        count++;            // 读者数加 1
        V(mutex);
        读文件;
        P(mutex);
        count--;            // 读者数减 1
        if (count == 0)     // 我是最后一个读者
            V(rw);          // 由我负责对文件解锁
        V(mutex);
```

```
    }
}
```

为什么必须给 **count** 再设一个 **mutex**?

假设没有 mutex：进程 1 判断 count == 0 成立（认为自己是第一个读者），此时发生进程切换；进程 2 也判断 count == 0 成立（也认为自己是第一个读者），也去 P(rw) 加锁；结果第一个读者加锁成功、第二个读者加锁失败被永久阻塞。根源在于**"** 判断 **count"** 和**"** 加锁**"** 无法一气呵成。

缺点：只要有一个读者处于活动状态，后来的读者都会被接纳。若读者源源不断到达，写者将始终阻塞，可能饿死。

（**2**）写者优先

```
semaphore rw    = 1;    // 读者与写者、写者与写者互斥
semaphore mutex = 1;    // 互斥访问 rcount
semaphore read  = 1;    // 用来禁止所有读进程（写者到来后封锁后续读者）
semaphore wmutex= 1;    // 互斥访问 wcount
int rcount = 0;         // 读者数量
int wcount = 0;         // 写者数量

reader() {
    while (1) {
        P(read);            // 若有写者在排队/执行，后来的读者阻塞在这里
        P(mutex);
        if (rcount == 0) P(rw);
        rcount++;
        V(mutex);
        V(read);
        读文件;
        P(mutex);
        rcount--;
        if (rcount == 0) V(rw);
        V(mutex);
    }
}

writer() {
    while (1) {
        P(wmutex);
        if (wcount == 0) P(read);   // 第一个写者封锁后续读者
        wcount++;
        V(wmutex);
        P(rw);                       // 等现有读者全部退出
        写文件;
        V(rw);
        P(wmutex);
        wcount--;
        if (wcount == 0) V(read);    // 最后一个写者解除对读者的封锁
        V(wmutex);
    }
}
```

read 的作用分析：开始有若干读者在读，此时来了一个写者，执行 P(read) 之后，后续的读者由于阻塞在 **read** 上都不能再进入读者队列，而写者可以进入写者队列，因此保证了写者优先。同时，第一个写者执行了 P(read) 之后也不能马上开始写，必须等到所有已在读者队列中的读者都读完、最后一个读者通过 V(rw) 唤醒写者。

(3) 读写公平法

```
semaphore rw    = 1;    // 读者与写者、写者与写者互斥
semaphore mutex = 1;    // 互斥访问 count
semaphore queue = 1;    // 让读者和写者排队，实现读写公平
int count = 0;

writer() {
    while (1) {
        P(queue);           // 先排队
        P(rw);
        写文件;
        V(rw);
        V(queue);
    }
}

reader() {
    while (1) {
        P(queue);           // 先排队
        P(mutex);
        if (count == 0) P(rw);
        count++;
        V(mutex);
        V(queue);           // 排队结束，允许下一个进程排队
        读文件;
        P(mutex);
        count--;
        if (count == 0) V(rw);
        V(mutex);
    }
}
```

为什么加一个 **queue** 就实现了公平？对比读者优先方案，读者优先中只要有读者到达就能进入读者队列；而 queue 阻止了读者的这种”特殊权限”——当写者执行了 P(queue) 后，后续到达的读者都阻塞在 queue 上，读者队列逐渐为空（count 减为 0），写者最终能拿到 rw 进入临界区。因此 queue 实现了读写进程优先级平等。

把三种方案放在一起比较：

方案	核心机制	结果
读者优先	第一个读者加 rw 锁，最后一个读者解锁	写者可能饿死
写者优先	第一个写者执行 P(read) 封锁后续读者	读者可能饿死
读写公平	加 queue 信号量让读写统一排队	按到达顺序访问，都不饿死

读者-写者类问题的识别与迁移：该类问题的特点是分类互斥——同类进程不互斥、异类进程互斥。凡是” 同一类可以多人同时进行、不同类必须独占” 的场景，都可套用此模板，关键点是设置一个计数器，第一个同类进程负责上锁，最后一个负责解锁。

*3.*哲学家进餐问题

问题描述：一张圆桌上坐着 5 名哲学家，每两名哲学家之间放一根筷子（共 5 根）。哲学家只做两件事：思考和进餐。饥饿时必须同时拿起左右两根筷子才能进餐，进餐完放下筷子继续思考。

该类题目的特点：只有一类进程，但要占用多种资源才能运行。

最初的错误解法：

```
semaphore chopstick[5] = {1, 1, 1, 1, 1};

philosopher(int i) {
    while (1) {
        P(chopstick[i]);              // 拿左筷子
        P(chopstick[(i+1) % 5]);      // 拿右筷子
        进餐;
        V(chopstick[i]);
        V(chopstick[(i+1) % 5]);
        思考;
    }
}
```

死锁原因：若 5 名哲学家同时拿起各自的左筷子，则所有筷子的信号量都为 0，谁也无法拿起右筷子，5 个进程全部阻塞，形成循环等待 → 死锁。

三种正确解法：

解法一：限制并发进程数（最多允许 **4** 人同时拿筷子）

```
semaphore chopstick[5] = {1, 1, 1, 1, 1};
semaphore limit = 4;      // 最多允许 4 名哲学家同时尝试拿筷子

philosopher(int i) {
    while (1) {
        P(limit);
        P(chopstick[i]);
        P(chopstick[(i+1) % 5]);
        进餐;
        V(chopstick[i]);
        V(chopstick[(i+1) % 5]);
        V(limit);
        思考;
    }
}
```

原理：至少有一名哲学家能同时拿到两根筷子，打破循环等待。破坏了”循环等待”条件。缺点：并发度不高。

解法二：限制申请资源的顺序（奇数号先拿左、偶数号先拿右）

```
philosopher(int i) {
    while (1) {
        if (i % 2 == 1) {                    // 奇数号：先左后右
            P(chopstick[i]);
            P(chopstick[(i+1) % 5]);
        } else {                             // 偶数号：先右后左
            P(chopstick[(i+1) % 5]);
            P(chopstick[i]);
        }
        进餐;
        V(chopstick[i]);
```

```
        V(chopstick[(i+1) % 5]);
        思考;
    }
}
```

原理：相邻两名哲学家拿筷子的顺序相反，不可能形成” 每人持有一根、都在等下一根” 的环。破坏了**”** 循环等待**”** 条件。缺点是不通用，只适用于本题这种对称结构。

解法三：拿筷子前先检测，且拿筷子动作互斥（**AND** 型信号量思想）

```
semaphore chopstick[5] = {1, 1, 1, 1, 1};
semaphore mutex = 1;      // 保证" 拿起两根筷子" 是一个原子操作

philosopher(int i) {
    while (1) {
        P(mutex);                          // 先申请拿筷子的权利
        P(chopstick[i]);                   // 拿左筷子
        P(chopstick[(i+1) % 5]);           // 拿右筷子
        V(mutex);                          // 两根都拿到后才释放权利
        进餐;
        V(chopstick[i]);
        V(chopstick[(i+1) % 5]);
        思考;
    }
}
```

原理：一名哲学家在” 拿两根筷子” 的整个过程中持有 mutex，其他哲学家无法同时尝试拿筷子。因此不存在” 每人拿一根、都在等第二根” 的局面，破坏了**”** 请求并保持**”** 条件（要么一次拿到两根，要么一根都不拿）。

关键辨析：这种方案不会导致饥饿——因为用完餐的哲学家总会放下两根筷子并且不再拿起，所以处于等待中的哲学家一定会等到” 两根筷子都空闲” 的时刻。有人误以为” 旁边的筷子可能被多次拿起”，这是错的，因为 mutex 保证了拿筷子的动作是串行的、完整的。

三种解法小结：

解法	原理	破坏的条件	通用性
限制并发数（n-1 人尝试）	至少一人能拿到两根筷子	循环等待	通用
奇偶不同顺序拿筷子	相邻哲学家拿筷顺序相反，无法成环	循环等待	不通用（对称结构可用）
拿筷子动作整体互斥	要么一次拿到两根，要么一根不拿	请求并保持	通用（推荐）

真题考法

2019 年曾考” 哲学家进餐问题” 的 PV 操作大题，要求写出完整的过程并说明信号量的含义及初值。答题时必须写出信号量定义与初值，并用 P、V 操作（或 wait()、signal()）完整描述每个进程。

*4.*理发师问题

核心思路转换：理发师问题与生产者-消费者问题的思路是一样的——顾客可以视为” 生产者”（生产顾客资源），服务人员可以视为” 消费者”（消费顾客资源）；反过来服务人员叫号也可视为生产了一个” 服务人员资源”。取号和叫号类似于往缓冲区放东西，需要一个互斥锁。

万能模板 **1**（无等待上限，服务人员可休息）

```
int waiting = 0;                // 正在等待的顾客数
semaphore customers = 0;        // 等待服务的顾客数
semaphore barbers   = 0;        // 空闲的服务人员数
semaphore mutex     = 1;        // 对 waiting 的互斥访问

barber() {                      // 服务人员进程（有 N 个）
    while (1) {
        P(customers);           // 有顾客才工作，否则阻塞休息
        P(mutex);
        waiting--;              // 少一名等待的顾客
        V(mutex);
        V(barbers);             // 叫号：生产一个" 空闲服务人员"
        为顾客服务;
    }
}

customer() {                    // 顾客进程
    P(mutex);                   // 取号（互斥）
    if (waiting < N) {
        waiting++;
        V(mutex);
        V(customers);           // 叫醒一名休息的服务人员
        P(barbers);             // 等待被叫号
        接受服务;
    } else {
        V(mutex);               // 没位置了，离店
    }
}
```

万能模板 **2**（有等待上限，服务人员可休息）：在上面的基础上把 `waiting` 作为判断条件，顾客到店时若 `waiting` 已达上限则直接离店（模板 1 中就是这种写法）。

万能模板 **3**（有等待上限，服务人员忙等）：服务人员在没有顾客时也不可以休息，必须不断询问有没有顾客。因此在服务人员进程中加入判断顾客数量的语句——对顾客资源的 **P** 操作就不需要了，因为 P 操作会引发阻塞或休息。

万能模板 **4**（座位已满后顾客仍继续等待，服务人员可休息）：把 `waiting` 变量变为信号量 **`empty`**（表示店里空位数），顾客先 `P(empty)` 阻塞等待空位，再取号等待叫号。

理发师问题的做题关键

1. 只需把模板 **1** 的思路记下来，其余都是从它延伸来的。做题时先默写模板 1，再按题意修补；
2. 服务人员的数量在模板中通常是冗余条件（N 个服务人员对应 N 个进程，或用一个信号量记录空闲服务人员数）；
3. 上述模板都没有满足先来先服务（因为 `V(server)` 后任意一个顾客进程都可能继续执行）。统考一般不要求先来先服务。

*5.*吸烟者问题

供应者每次把两种材料放在桌上，拥有第三种材料的吸烟者才能卷烟抽。设三种材料对应三个信号量 `offer1`、`offer2`、`offer3`，再加一个 `finish` 信号量让供应者知道上一轮已被取走。

```
semaphore offer1 = 0, offer2 = 0, offer3 = 0;
semaphore finish = 0;
int i = 0;                          // 供应者轮流提供的组合
```

```
provider() {
    while (1) {
        if (i == 0)      { 将烟草和纸放到桌上; V(offer1); }
        else if (i == 1) { 将烟草和胶水放到桌上; V(offer2); }
        else             { 将纸和胶水放到桌上; V(offer3); }
        i = (i + 1) % 3;
        P(finish);                // 等吸烟者抽完再放下一组
    }
}

smoker1() {   // 有胶水的人
    while (1) { P(offer1); 卷烟抽; V(finish); }
}

smoker2() {   // 有纸的人
    while (1) { P(offer2); 卷烟抽; V(finish); }
}

smoker3() {   // 有烟草的人
    while (1) { P(offer3); 卷烟抽; V(finish); }
}
```

易错

吸烟者问题也是”单缓冲区”结构（桌上同时只有一组材料），但由于供应者每次只放一组、且吸烟者抽完后才放下一组，不需要额外的 **mutex**。判断是否需要 mutex 的方法是：看是否存在”多个进程可能同时进入缓冲区”的风险。

*6.*独木桥问题

问题描述：桥不允许两车交汇，但允许同方向多辆车依次通过（桥上可以有多个同方向的车）。用 P、V 操作实现交通管理防止堵塞。

识别：这是同类不互斥、异类互斥的分类互斥问题，与读者-写者问题同构（同方向的车相当于”读者”，对向的车相当于”写者”）。

```
semaphore mutex = 1;      // 对两个计数器的互斥访问
semaphore bridge = 1;     // 桥的互斥访问（相当于读者-写者中的 rw）
int countA = 0;           // A 方向在桥上的车数
int countB = 0;           // B 方向在桥上的车数

carA() {
    while (1) {
        P(mutex);
        if (countA == 0) P(bridge);   // 第一辆 A 方向的车负责上锁
        countA++;
        V(mutex);
        过桥;
        P(mutex);
        countA--;
        if (countA == 0) V(bridge);   // 最后一辆 A 方向的车负责解锁
        V(mutex);
    }
}
```

carB() 完全对称。

判断”是否类似读者-写者”的方法

同类进程之间不互斥、异类进程之间互斥 ⇒ 套读者-写者模板。关键是设一个计数器记录同类进程数量，第一个进程上锁、最后一个进程解锁。独木桥、更衣室、录像厅、寿司店等题目都是这个套路。

7.流水线与前驱关系问题

问题描述：一条流水线上有多个工序，后一工序必须等前一工序完成后才能开始；相邻工序之间有一个容量为 1 的缓冲区。

识别：这是前驱关系 + 单缓冲区问题，每个工序既是前一工序的消费者，又是后一工序的生产者。

```
semaphore empty1 = 1, empty2 = 1;   // 各缓冲区空位
semaphore full1  = 0, full2  = 0;   // 各缓冲区产品

工序 1() {
    while (1) {
        加工零件;
        P(empty1);
        把零件放到缓冲区 1;
        V(full1);
    }
}

工序 2() {
    while (1) {
        P(full1);
        从缓冲区 1 取出零件;
        V(empty1);
        加工零件;
        P(empty2);
        把零件放到缓冲区 2;
        V(full2);
    }
}

工序 3() {
    while (1) {
        P(full2);
        从缓冲区 2 取出零件;
        V(empty2);
        加工零件;
    }
}
```

流水线问题的通用套路

对每一段缓冲区设一对信号量：`empty_i`（初值 = 缓冲区容量，容量为 1 时初值 1）和 `full_i`（初值 0）。前一工序放入产品时”先 `P(empty_i)` 再 `V(full_i)`”，后一工序取出产品时”先 `P(full_i)` 再 `V(empty_i)`”。这实际上就是把每个工序都当作一个小型的生产者-消费者。

PV 操作大题的答题规范

1. 必须写出信号量定义并赋初值，并说明每个信号量的含义；
2. 必须写出每个进程的完整循环体，用 P/V（或 `wait/signal`）操作描述；
3. 用中文描述动作（如”把产品放入缓冲区”），不要遗漏；
4. 检查是否存在死锁：连续的 P 操作是否可能造成循环等待；多个进程共享同一缓冲区时是否存在某一方把缓冲区占满的隐患；如果有，加入 `mutex` 让连续 P 操作一气呵成，或转化为双缓冲区。

四、死锁

（一）死锁的基本概念

*1.*定义

死锁（**Deadlock**）：在并发环境下，各进程互相等待对方手里的资源，导致各进程都阻塞、无法向前推进的现象。死锁的定义包含两点：互相循环等待 + 各个进程都被阻塞。

*2.*死锁、饥饿、死循环的区别

对比项	死锁	饥饿	死循环
定义	各进程互相等待对方手里的资源，都阻塞无法推进	由于长期得不到想要的资源，某进程无法向前推进	进程在不满足终止条件时无限循环
原因	循环等待对方手里的资源	优先级倒置、资源分配不公平或错误	编程错误、逻辑错误
例子	银行家算法问题中的死锁状态	低优先级进程长期得不到 CPU	空循环、忘记更新循环条件
进程状态	阻塞态	阻塞态（得不到 I/O 设备）或就绪态（上不了处理机）	运行态
涉及进程数	两个或两个以上	可能只有一个，也可能多于一个	大于等于一个
性质	操作系统（管理者）的问题	操作系统（管理者）的问题	被管理者（程序员）的问题

*3.*产生死锁的原因

1. 对系统资源的竞争：各进程竞争不可剥夺资源，若资源数目满足不了多个进程的需要，就会因争夺资源陷入僵局。可剥夺资源（CPU、主存）不会引发死锁。
2. 进程推进顺序不当：P1 占有 R1 又申请 R2，P2 占有 R2 又申请 R1，互相等待。
3. 信号量使用不当：例如生产者-消费者问题中把实现互斥的 P 操作放在实现同步的 P 操作之前。
4. 由可消耗性资源引起：多个进程通信时 `send/receive` 顺序处理不当，都在等一条永远不会发出的消息。

*4.*四个必要条件

条件	含义
互斥条件	任何时刻只能有一个进程使用一个资源实例。只有对必须互斥使用的资源（临界资源、锁）的争夺才会导致死锁；内存、CPU、扬声器这类可同时共享的资源不会导致死锁
不可剥夺（不可抢占）条件	进程所获得的资源在未使用完之前不能被其他进程强行夺走，只能主动释放
请求并保持（请求与保持）条件	进程已经保持了至少一个资源，但又提出新的资源请求，而该资源被其他进程占有，此时请求进程被阻塞，但对自己已有的资源保持不放
循环等待条件	存在一种进程-资源的循环等待链，链中每一个进程已获得的资源同时被链中下一个进程所请求

四个条件必须”同时满足”才可能死锁，只要破坏其中一个，死锁就不会发生。

循环等待 $\neq$ 死锁

发生死锁时一定有循环等待，但发生循环等待时未必死锁。若同类资源数大于 1，即使存在循环等待，等待链中的某个进程仍可能从链外的进程那里获得资源，从而打破循环。只有当系统中每类资源都只有一个时，循环等待才是死锁的充要条件。

*5.*两个常用结论

设系统有 n 个进程，每个进程最多需要 y 个同类资源：

- 系统必然不会发生死锁的最少资源数为 $n \times (y-1)+1$；
- 反之，若系统有 x 个资源、每个进程最多需要 y 个资源，则可能发生死锁的最小进程数为 $\left\lceil \dfrac{x}{y-1} \right\rceil$。

推导思路：最坏情况是每个进程都只拥有 $y-1$ 个资源并都在申请第 y 个。若资源总数恰好为 $n(y-1)$，资源已用完，必然死锁；再多加一个资源，令其中一个进程获得所需的最大资源，死锁即被解除。

用例题验证：

- 系统有 8 台打印机，K 个进程竞争，每个进程最多需要 3 台。要保证不死锁需 $8 \geq K(3-1)+1 = 2K+1$，即 $K \leq 3.5$，故 $K=4$ 时可能发生死锁，答案是 4。
- 三个进程分别需要 3、4、5 台设备：最坏情况各拿到 2、3、4 台，共 9 台，只要再能额外提供 1 台就能让某个进程完成，故确保不发生死锁的最小设备数 $n=9+1=10$。
- n 个进程每个需要 2 个临界资源：最坏情况各拿到 1 个，共 n 个，再加 1 个即可打破僵局，故至少需要 $n+1$ 个资源。

*6.*死锁处理策略

策略	思想	优缺点
死锁预防（**Prevention**）	设计系统时破坏四个必要条件之一，从根本上排除死锁可能	一定能保证不死锁，但资源利用率低、并发度差
死锁避免（**Avoidance**）	不破坏必要条件，在资源动态分配中防止系统进入不安全状态（银行家算法）	需要预先知道进程运行所需的资源总量，并发性次于检测法
死锁检测与解除（**Detection & Recovery**）	允许死锁发生，定期检测并在发现后解除	并发性最大，但检测有开销，解除可能造成工作损失

按并发性从大到小排序：检测死锁 > 银行家算法 > 资源预分配。

（二）死锁预防

*1.*破坏互斥条件

把互斥资源改造成共享资源。例如用 **SPOOLing** 技术把打印机这种独占设备在逻辑上改造成共享设备；再如通过强大的硬件指令把需要互斥访问的代码构造成不需要锁的数据结构。

缺点：有的资源本身就是非共享的（一个互斥锁不能同时被多个进程共享），有些地方还必须保护

这种互斥性。因此这种方法通常不太可行。

*2.*破坏不可剥夺条件

- 方案一：当进程请求新资源得不到满足时，立即释放它保持的所有资源；
- 方案二：当进程需要的资源被其他进程占有时，由操作系统协助强行剥夺。

缺点：实现复杂、代价大；释放已获得的资源可能造成前一阶段工作失效，使进程前后两次运行的信息不连续；反复申请和释放资源会增加系统开销、降低吞吐量；可能导致进程饥饿。因此只适用于状态容易保存和恢复的资源（如 CPU 寄存器、内存），一般不适用于互斥锁、信号量、打印机、CD 刻录机。

*3.*破坏请求并保持条件

要求" 进程在请求资源时不能持有不可剥夺资源"，有两种协议：

- 协议一（静态分配）：进程在运行前一次申请完它所需要的全部资源，资源未满足就不让它投入运行；
- 协议二（改进版）：允许进程只获得运行初期所需的资源便开始运行，运行过程中逐步释放已用完的资源，然后再请求新的资源。

缺点：需要准确预知需要哪些资源并提前获得；有些资源只在运行初期或快结束时短暂使用，若整个运行期一直保持就会严重降低并发和资源利用率；协议一实现简单但资源浪费严重，且可能导致饥饿；协议二改进了协议一的缺点，能更快完成任务、提高设备利用率、减少饥饿概率。

*4.*破坏循环等待条件

最实用、最经常采用的预防技术：采用顺序资源分配法（资源有序分配算法）。给系统中所有资源编号，规定每个进程必须按编号递增的顺序请求资源，同类资源一次申请完。也就是说，进程一旦申请了资源 R_i，以后的申请就只能申请编号大于 R_i 的资源。

优点：从机制上杜绝了循环等待链。

缺点：

- 不方便增加新设备，可能需要重新分配所有编号；
- 进程实际使用资源的顺序可能与编号递增顺序不一致，导致资源浪费（例如 1 号资源是最后需要的，却被最先持有，使其他只需 1 号资源的进程被长时间阻塞）；
- 必须按规定次序申请资源，用户编程麻烦。

常考对比

死锁预防靠破坏四个必要条件；死锁避免靠银行家算法保证系统始终处于安全状态。二者常被放在一起考（如 2015-26、2019-30、2013-32）。另外，限制用户申请资源的顺序属于死锁预防（破坏循环等待），不属于死锁避免。

（三）死锁避免

*1.*安全状态与安全序列

- 安全序列：指如果系统按这种序列为进程分配资源，则每个进程都能顺利完成。只要能找出一个安全序列，系统就处于安全状态。安全序列可能有多个。
- 不安全状态：如果分配资源后系统中找不出任何一个安全序列，则系统进入不安全状态。

- 关键结论：
  1. 处于安全状态，一定不会发生死锁；
  2. 处于不安全状态，可能发生死锁，但不一定（例如进程提前终止释放了资源，或实际所需资源小于声明的最大需求）；
  3. 发生死锁，一定处于不安全状态；
  4. 若进程提前归还部分资源，系统也有可能从不安全状态重新回到安全状态，但在分配资源以前总要考虑最坏情况；
  5. 单个进程阻塞不是死锁，必须形成阻塞环才是死锁。

死锁避免与死锁检测的区别：

对比项	死锁避免	死锁检测
需要的资源信息	需要进程整个生命周期所需的资源总量（最大需求）	只需要当前时刻的资源请求与分配信息
是否限制资源申请顺序	不会（限制顺序属于死锁预防）	不会
是否会给可能导致死锁的进程分配资源	不会	会（允许死锁发生，之后再解除）
典型算法	银行家算法	资源分配图化简法（死锁定理）

*2.*银行家算法的数据结构

设有 n 个进程、m 类资源：

数据结构	类型	含义
Available	长度为 m 的向量	`Available[j]` 表示第 j 类资源的当前可用数量
Max	$n \times m$ 矩阵	`Max[i][j]` 表示进程 P_i 对第 j 类资源的最大需求
Allocation	$n \times m$ 矩阵	`Allocation[i][j]` 表示已分配给 P_i 的第 j 类资源数量
Need	$n \times m$ 矩阵	`Need[i][j]` 表示 P_i 还需要第 j 类资源的数量

三者关系（必须自洽）：

$$Need[i][j] = Max[i][j] - Allocation[i][j]$$

且满足 $\sum_i Allocation[i][j] + Available[j] =$ 系统中第j类资源总数。

*3.*安全性算法（完整步骤）

1. 令 **Work = Available**，**Finish[i] = FALSE**（$i = 0, 1, \ldots, n-1$）；
2. 寻找满足下列两个条件的进程 P_i：
   - `Finish[i] == FALSE`；
   - `Need[i][j]` $\leq$ `Work[j]`（对所有 j 成立）；若找到，转步骤 3；若找不到，转步骤 4；
3. 该进程可以顺利执行完毕并释放它的全部资源：

$$Work[j] = Work[j] + Allocation[i][j] \quad (\text{对所有} j)$$

`Finish[i] = TRUE`，把 P_i 加入安全序列，返回步骤 **2**；

1. 若所有 `Finish[i] == TRUE`，则系统处于安全状态（步骤 3 中得到的进程顺序就是一个安全序列）；否则系统处于不安全状态。

一句话概括：检查当前剩余可用资源能否满足某个进程的最大需求，可以就把它加入安全序列并回收它持有的全部资源，不断重复，看最终能否让所有进程都加入安全序列。

*4.*资源请求算法

当进程 P_i 发出资源请求向量 $Request_i$ 时：

1. 检查是否超过声明的最大需求：若 `Request_i[j] > Need[i][j]`，说明请求量超过它事先声明的最大需求，报错；
2. 检查系统是否有足够资源：若 `Request_i[j] > Available[j]`，说明资源暂时不够，P_i 必须等待；
3. 试探性分配（修改数据结构）：

$$Available[j] = Available[j] - Request_i[j]$$

$$Allocation[i][j] = Allocation[i][j] + Request_i[j]$$

$$Need[i][j] = Need[i][j] - Request_i[j]$$

1. 执行安全性算法检查此次分配后系统是否安全：
   - 若安全，则正式分配；
   - 若不安全，则撤销本次试探性分配（恢复原先的 Available、Allocation、Need），让 P_i 等待。

*5.*完整算例一：*T0* 时刻的安全性检测

设系统有 5 个进程 P0 ~ P4、3 类资源 A、B、C，各类资源总数分别为 **A = 10**、**B = 5**、**C = 7**。T0 时刻状态如下：

进程	**Allocation (A,B,C)**	**Max (A,B,C)**	**Need (A,B,C)**
P0	0, 1, 0	7, 5, 3	7, 4, 3
P1	2, 0, 0	3, 2, 2	1, 2, 2
P2	3, 0, 2	9, 0, 2	6, 0, 0
P3	2, 1, 1	2, 2, 2	0, 1, 1
P4	0, 0, 2	4, 3, 3	4, 3, 1

第一步：求 **Available**

已分配总量：$\mathrm{A} = 0+2+3+2+0 = 7$；$\mathrm{B} = 1+0+0+1+0 = 2$；$\mathrm{C} = 0+0+2+1+2 = 5$。

$$Available = (10-7,\ 5-2,\ 7-5) = (3,\ 3,\ 2)$$

第二步：逐个查找满足 $Need \le Work$ 的进程

第 1 轮，Work = (3, 3, 2)：

进程	**Need**	是否 $\leq$ **Work**	判定
P0	(7,4,3)	7 > 3	否
P1	(1,2,2)	全部满足	是
P2	(6,0,0)	6 > 3	否
P3	(0,1,1)	全部满足	是（但按编号先取 P1）
P4	(4,3,1)	4 > 3	否

选 **P1**。P1 执行完毕，释放 Allocation(P1) = (2,0,0)：

$$Work = (3,3,2) + (2,0,0) = (5,\ 3,\ 2)$$

安全序列当前为：**P1**

第 2 轮，Work = (5, 3, 2)：

进程	**Need**	是否 $\leq$ **Work**
P0	(7,4,3)	7 > 5，否
P2	(6,0,0)	6 > 5，否
P3	(0,1,1)	是
P4	(4,3,1)	是（按编号先取 P3）

选 **P3**。释放 Allocation(P3) = (2,1,1)：

$$Work = (5,3,2) + (2,1,1) = (7,\ 4,\ 3)$$

安全序列：P1 $\rightarrow$ **P3**

第 3 轮，Work = (7, 4, 3)：

进程	**Need**	是否 $\leq$ **Work**
P0	(7,4,3)	是（按编号先取 P0）
P2	(6,0,0)	是
P4	(4,3,1)	是

选 **P0**（按编号顺序）。释放 Allocation(P0) = (0,1,0)：

$$Work = (7,4,3) + (0,1,0) = (7,\ 5,\ 3)$$

安全序列：P1 $\rightarrow$ P3 $\rightarrow$ **P0**

第 4 轮，Work = (7, 5, 3)：

进程	**Need**	是否 $\leq$ **Work**
P2	(6,0,0)	是
P4	(4,3,1)	是

选 **P2**。释放 Allocation(P2) = (3,0,2)：

$$Work = (7,5,3) + (3,0,2) = (10,\ 5,\ 5)$$

安全序列：P1 → P3 → P0 → **P2**

第 5 轮，Work = (10, 5, 5)：P4 的 Need = (4,3,1) 满足，选 **P4**。

$$Work = (10,5,5) + (0,0,2) = (10,\ 5,\ 7)$$

安全序列：**P1** → **P3** → **P0** → **P2** → **P4**，所有进程 Finish 均为 TRUE，系统处于安全状态。

结果校验

安全序列执行完毕后，Work = (10, 5, 7)，恰好等于系统资源总数 (A=10, B=5, C=7)。这是检查算例是否算错的快捷方法：安全序列跑完后的 **Work** 必须等于资源总数。

安全序列的唯一性

本题中第一步只有 **P1** 能执行（P0、P2、P4 的 Need 都超过 Available），因此所有安全序列都必须以 **P1** 开头。之后有多个分支，因此安全序列不唯一（例如 P1 → P3 → P4 → P2 → P0 也是安全序列）。做选择题时要逐个验证选项中的序列是否真的每一步都可行。

*6.*完整算例二：资源请求的处理

仍用上面的系统，当前 $Available = (3,3,2)$。

情形 **1**：**P0** 请求 **Request = (2, 1, 0)**

1. 检查最大需求：`Request(2,1,0)` $\leq$ `Need(P0)=(7,4,3)`，通过；
2. 检查可用资源：`Request(2,1,0)` $\leq$ `Available(3,3,2)`，通过；
3. 试探性分配：

进程	**Allocation**	**Need**	**Available**
P0	(2, 2, 0)	(5, 3, 3)	(1, 2, 2)
P1	(2, 0, 0)	(1, 2, 2)	
P2	(3, 0, 2)	(6, 0, 0)	
P3	(2, 1, 1)	(0, 1, 1)	
P4	(0, 0, 2)	(4, 3, 1)	

1. 安全性检查，Work = (1, 2, 2)：
   - P1 的 Need = (1,2,2) $\leq$ (1,2,2)，选 P1；Work = (1,2,2)+(2,0,0) = **(3, 2, 2)**；
   - P3 的 Need = (0,1,1) $\leq$ (3,2,2)，选 P3；Work = (3,2,2)+(2,1,1) = **(5, 3, 3)**；
   - P4 的 Need = (4,3,1) $\leq$ (5,3,3)，选 P4；Work = (5,3,3)+(0,0,2) = **(5, 3, 5)**；

- P0 的 Need = (5,3,3) ≤ (5,3,5)，选 P0；Work = (5,3,5)+(2,2,0) = **(7, 5, 5)**；
- P2 的 Need = (6,0,0) ≤ (7,5,5)，选 P2；Work = (7,5,5)+(3,0,2) = **(10, 5, 7)**。得到安全序列 **P1 → P3 → P4 → P0 → P2**，系统安全。因此可以正式把资源分配给 **P0**。

情形 **2**：**P4** 请求 **Request = (3, 3, 0)** ——拒绝

1. `Request(3,3,0)` ≤ `Need(P4) = (4,3,1)`（3 ≤ 4 且 3 ≤ 3），通过；
2. `Request(3,3,0)` ≤ `Available = (3,3,2)`，通过；
3. 试探性分配：`Allocation(P4) = (3, 3, 2)`，`Need(P4) = (1, 0, 1)`，`Available = (0, 0, 2)`；
4. 安全性检查，Work = (0, 0, 2)：逐个检查所有进程的 Need，没有一个能被满足：
   - P0 需 (7,4,3)：A 不足；
   - P1 需 (1,2,2)：A 不足（需 1，只有 0）；
   - P2 需 (6,0,0)：A 不足；
   - P3 需 (0,1,1)：B 不足（需 1，只有 0）；
   - P4 需 (1,0,1)：A 不足。一步都推不动，系统进入不安全状态，因此必须拒绝该请求。

情形 **3**：**P1** 请求 **Request = (0, 2, 0)** ——拒绝（这是最典型的" 看似可行实则不安全" 的例子）

1. `Request(0,2,0)` ≤ `Need(P1)=(1,2,2)`，通过；
2. `Request(0,2,0)` ≤ `Available(3,3,2)`，通过；
3. 试探性分配：

进程	**Allocation**	**Need**	**Available**
P0	(0, 1, 0)	(7, 4, 3)	**(1, 1, 2)**
P1	(2, 2, 0)	**(1, 0, 2)**	
P2	(3, 0, 2)	(6, 0, 0)	
P3	(2, 1, 1)	(0, 1, 1)	
P4	(0, 0, 2)	(4, 3, 1)	

1. 安全性检查，Work = (1, 1, 2)：逐个检查
   - P0 需 (7,4,3)：A 分量 7 > 1，不满足；
   - P1 需 (1,0,2)：(1,0,2) ≤ (1,1,2)，满足。选 P1！Work = (1,1,2) + (2,2,0) = **(3, 3, 2)**；
   - P3 需 (0,1,1) ≤ (3,3,2)，选 P3；Work = (3,3,2)+(2,1,1) = **(5, 4, 3)**；
   - P4 需 (4,3,1) ≤ (5,4,3)，选 P4；Work = (5,4,3)+(0,0,2) = **(5, 4, 5)**；
   - P0 需 (7,4,3)：A 分量 7 > 5，不满足；
   - P2 需 (6,0,0)：A 分量 6 > 5，不满足。死循环，找不到更多可执行进程，系统进入不安全状态。

结论：该请求必须被拒绝，让 P1 等待。这个算例是最经典的考点——请求量本身没有超过 **Need** 也没有超过 **Available**，但分配后系统不安全，所以仍要拒绝。

*7.*经典真题算例

（**1**）**2011-27**（答案：**D**，不存在安全序列）

进程	已分配 **R1 R2 R3**	尚需 **R1 R2 R3**
P1	2 0 0	0 0 1
P2	1 2 0	1 3 2
P3	0 1 1	1 3 1
P4	0 0 1	2 0 0

可用资源 = (0, 2, 1)。P1 的 Need (0,0,1) ≤ (0,2,1)，可执行；释放后 Work = (2, 2, 1)。此后 P2 需 (1,3,2) 的 R2 分量 3 > 2，P3 需 (1,3,1) 的 R2 分量 3 > 2，P4 需 (2,0,0) 可满足但执行后 Work = (2,2,2)，R2 仍只有 2，P2、P3 依然无法执行。最终无法让所有进程加入安全序列，故不存在安全序列。

（**2**）**2018-26**（答案：**A**，系统处于不安全状态）

系统有 4 个同类资源，P1、P2、P3 分别需要 4、3、1 个，已申请到 2、1、0 个。

- 可用资源 = 4 − (2+1+0) = **1**；
- Need：P1 = 4−2 = 2，P2 = 3−1 = 2，P3 = 1−0 = **1**；
- 只有 P3 能满足，P3 执行完毕释放 0 个资源，Work 仍为 **1**；
- 此时 P1、P2 都需 2 个而只有 1 个，谁也无法执行。

结论：不存在安全序列，系统处于不安全状态。

（**3**）**2022-26**（答案：**B**，安全序列个数为 **2**）

进程	已分配 **A B C**	尚需 **A B C**
P0	2 0 1	0 2 1
P1	0 2 0	1 2 3
P2	1 0 1	0 1 3

可用资源 = (1, 3, 2)。P0 的 Need (0,2,1) ≤ (1,3,2) 满足；P1 的 Need 需要 C 资源 3 > 2，不满足；P2 的 Need 需要 C 资源 3 > 2，不满足。第一步只能运行 **P0**。

P0 执行完毕释放 (2,0,1)，Work = (1,3,2) + (2,0,1) = **(3, 3, 3)**。此时 P1 需 (1,2,3) ≤ (3,3,3) 满足，P2 需 (0,1,3) ≤ (3,3,3) 也满足。

- 序列 P0 → P1 → P2；
- 序列 P0 → P2 → P1。

共 **2** 个安全序列。

银行家算法速算技巧

1. 先把 **Available** 算对：Available = 资源总数 − 各进程已分配之和。题目不给总数时，往往给出” 可用资源” 行，直接抄；
2. **Need = Max − Allocation**，逐个算，不要漏；
3. 每选出一个进程就立刻更新 **Work** 并重新从第一个进程开始扫描（题目若规定了扫描顺序，就按规定的顺序取）；
4. 用**”Work** 最终是否等于资源总数**”** 校验；
5. 资源请求题按” 三步检查 + 试探分配 + 安全性算法” 作答，写清” 拒绝” 或” 可以分配” 并给出理由。

（四）死锁检测和解除

*1.*死锁检测的时机

死锁检测算法会按系统真实情况设置合适的时间间隔，每隔一段时间执行一次来检测系统是否存在死锁。若检测过于频繁会浪费系统资源；若间隔太长又没有意义。

检测的触发时机：每隔一段时间，或当 **CPU** 利用率显著降低时。（不能以”死锁发生时”为触发点——只有运行了检测算法之后系统才知道是否死锁；也不能在每次进程申请资源时都检测，开销过大。）

若系统既不采取预防措施也不采取避免措施，就必须提供两个算法：死锁检测算法（确定系统是否已发生死锁）和死锁解除算法（从死锁状态恢复）。

*2.*资源分配图

需要的两种信息：用某种数据结构保存资源的请求和分配信息；提供一种算法利用这些信息检测是否进入死锁状态。

资源分配图的构成：

元素	含义
进程结点（圆圈）	对应一个进程
资源结点（方框）	对应一类资源，方框中的小圆圈表示该类资源有几个实例
请求边（进程结点 → 资源结点）	表示进程想申请几个资源，每条边代表一个
分配边（资源结点 → 进程结点）	表示已经为进程分配了几个资源，每条边代表一个

简化资源分配图的步骤：

1. 在图中找出既不阻塞又不是孤点的进程 P_i——即存在一条与它相连的有向边，且该边对应的资源申请数量小于等于系统中已有的空闲资源数量。若所有连接该进程的边都满足条件，则该进程能继续运行直至完成；
2. 消去它所有的请求边和分配边，使之成为孤立的结点（相当于该进程释放了所占有的全部资源）；
3. 它释放的资源可以唤醒某些因等待这些资源而阻塞的进程，使原来的阻塞进程变为非阻塞进程，于是对这些新解放的进程重复上述操作。

死锁定理：如果某时刻系统的资源分配图是不可完全简化的，那么此时系统发生死锁。

- 若能消去图中所有的边，则称该图是可完全简化的，此时一定没有发生死锁（相当于能找到一个安全序列）；
- 若最终不能消除所有边，则发生了死锁；
- 注意：并不是系统中所有进程都是死锁状态。用死锁检测算法化简资源分配图后，还连着边的那些进程才是死锁进程。

死锁检测不关心”资源总量”

死锁检测只检测系统中是否发生死锁（是否存在循环等待的环路），一旦发现就要采取解除措施，因此不需要知道各进程运行所需的资源总量信息，只需要知道当前时刻的资源请求与分配情况。这正是 2015-26 题中”Ⅱ：S1 需要进程运行所需资源总量信息，而 S2 不需要”正确的理由。

真题考法

2015-26（死锁避免与死锁检测的对比）、2016-25（4 个进程共享 3 个临界资源，问死锁时处于死锁状态的进程数至少是几个——答案 3，因为 p4 只需 R2，无法单独成环，至少需要 p1、p2、p3 三个进程构成环）。

*3.*死锁的解除

方法	做法	特点
资源剥夺法	挂起（暂时放到外存）某些死锁进程，抢占它的资源并分配给其他死锁进程	应防止被挂起的进程长时间得不到资源而饥饿
撤销进程法（终止进程法）	强制撤销部分甚至全部死锁进程，剥夺它们的资源	实现简单，但代价可能很大（有些进程已运行很久、接近结束，被终止则功亏一篑）
进程回退法	让一个或多个死锁进程回退到足以避免死锁的地步	要求系统记录进程的历史信息、设置还原点；过程中进程主动释放资源而非被剥夺

选择”牺牲者”（撤销哪个进程、剥夺谁的资源）时可以考虑的因素：

- 进程的优先级；
- 进程已执行了多长时间；
- 进程还要多久才能完成；
- 进程已经使用了多少资源；
- 进程是交互式还是批处理式。

如何避免解除死锁时造成饥饿？如果一个系统基于”代价”来选择牺牲进程，那么同一进程可能总是被选为牺牲者，结果永远不能完成指定任务而饿死。因此可以在代价的考虑因素中加入回滚次数。

死锁与本节的整体脉络

死锁预防：破坏四个必要条件之一（互斥、不可剥夺、请求并保持、循环等待）。死锁避免：银行家算法，保证系统不进入不安全状态。死锁检测：资源分配图化简，看是否可完全简化（死锁定理）。死锁解除：资源剥夺、撤销进程、进程回退。

本章小结表

知识点	核心结论
进程映像组成	PCB + 程序段 + 数据段；创建进程 = 创建 PCB，撤销进程 = 撤销 PCB
进程 vs 线程	进程是资源分配的基本单位，线程是处理机调度的基本单位
线程私有部分	TID、PC、通用寄存器组、栈；其余全部共享
三态模型	就绪、运行、阻塞（单处理机下运行态至多一个）
五态模型	新建、就绪、运行、阻塞、终止
非法转换	阻塞 → 运行（不允许）；就绪 → 阻塞（不允许）
运行 → 就绪原因	时间片用完、被更高优先级进程抢占、主动 `yield`
运行 → 阻塞原因	请求 I/O、申请临界资源、P 操作阻塞、等管程条件变量

知识点	核心结论
阻塞 → 就绪原因	I/O 完成、资源被释放、事件到达（被动行为）
用户级线程	线程库管理、切换快、一个阻塞则全进程阻塞、不能利用多处理机
内核级线程	内核管理、切换慢、一阻塞不影响其他线程、可多处理机并行
多线程模型	多对一、一对一、多对多（n > m）
PCB 组织	线性表、链接队列、索引表；就绪队列 1 个，阻塞队列多个
进程通信	共享内存（最快、需同步）、消息传递、管道（半双工、亲缘进程）、信号（异步通知）
管道阻塞条件	写满阻塞、读空阻塞；读端全关返回 EOF，写端全关收到 SIGPIPE
三级调度	高级（作业，频率最低）、中级（内存/挂起）、低级（进程，频率最高）
周转时间	完成时间 − 到达时间
带权周转时间	周转时间 / 运行时间（必然 ≥ 1）
等待时间	进程处于就绪态的时间之和
不能调度的情况	中断处理中、内核临界区中、禁用中断时、原语执行中
非抢占式特点	不出现” 运行 → 就绪”；调度不频繁、开销小、实时性差
抢占式特点	出现” 运行 → 就绪”；响应快、适用于分时与实时系统
FCFS	非抢占；公平、不饥饿；对短作业不利、不利于 I/O 繁忙型
SJF（默认非抢占）	平均周转/等待时间最短；需预知运行时间；长作业饥饿
SRTN（抢占式 SJF）	平均周转时间最优；同样长作业饥饿
HRRN	响应比 =（等待时间 + 要求服务时间）/ 要求服务时间；不饥饿
RR	抢占式、公平、响应快、不饥饿；时间片过大退化为 FCFS
优先级调度	抢占/非抢占均可；静态/动态；可能饥饿
MLFQ	队列优先级从高到低、时间片从小到大；I/O 型回到高优先级；设计要考虑 4 项因素
多处理机调度	亲和性（软/硬）、负载均衡（推迁移/拉迁移）；公共队列亲和性差、私有队列需迁移
上下文切换	保存/恢复寄存器与地址空间；PCB 保存的是内核态上下文
模式切换 vs 进程切换	模式切换是同一进程的用户态 ↔ 内核态；进程切换必在内核态、前后不同进程
临界区四原则	空闲让进、忙则等待、有限等待（必做）；让权等待（可选）
单标志法	违背空闲让进（和让权等待）
双标志先检查	违背忙则等待
双标志后检查	违背空闲让进和有限等待
Peterson	满足空闲让进、忙则等待、有限等待；不满足让权等待；只适用于两个进程
中断屏蔽	只适用于内核进程、只适用于单处理机
TSL / Swap	原子操作、适用多处理机；不满足让权等待，可能饥饿
自旋锁	忙等、无上下文切换开销、多处理机短临界区适用、单处理机需抢占式调度器
整型信号量	`while (S <= 0);` ——违反让权等待
记录型信号量	`S.value--` 后判断 `< 0` 则阻塞；`S.value++` 后判断 `<= 0` 则唤醒；遵循让权等待

知识点	核心结论
S.value 含义	>0 表示可用资源数；=0 表示资源刚好用完且无等待者；<0 时其绝对值等于正在等待的进程数
互斥模板	P(mutex) …临界区…V(mutex)，mutex = 1
同步模板	前操作后 V(S)、后操作前 P(S)，S = 0
前驱模板	每对前驱一个信号量（初值 0），前驱末尾 V、后继开头 P
条件变量 vs 信号量	条件变量没有值；signal 可能不产生任何影响，V 一定会改变信号量
生产者-消费者	P(empty) 必须在 P(mutex) 之前，否则死锁；V 操作顺序可交换
读者-写者	分类互斥；第一个读者上锁、最后一个读者解锁；count 需要 mutex 保护
哲学家进餐	三种解法：限并发数、奇偶不同顺序、拿筷整体互斥
死锁四条件	互斥、不可剥夺、请求并保持、循环等待（必须同时满足）
循环等待 vs 死锁	死锁一定有循环等待；循环等待不一定死锁（同类资源 > 1 时可被链外资源打破）
不死锁最少资源数	$x \times (y-1)+1$（x 个进程、每个最多需 y 个同类资源）
可能死锁最小进程数	$\lceil x/(y-1) \rceil$
预防方法	破坏互斥（SPOOLing）、破坏不可剥夺（释放/剥夺）、破坏请求并保持（静态分配）、破坏循环等待（顺序资源分配）
安全状态性质	安全 → 一定不死锁；不安全 → 可能死锁；死锁 → 一定不安全
银行家算法	Need = Max − Allocation；安全性算法反复找 Need ≤ Work 的进程并回收资源
资源请求处理	检查 ≤ Need → 检查 ≤ Available → 试探分配 → 安全性算法 → 安全则分配，否则撤销并等待
死锁检测	资源分配图；死锁定理：图不可完全简化则死锁；还连着边的进程才是死锁进程
死锁解除	资源剥夺、撤销进程、进程回退
并发性排序	检测死锁 > 银行家算法 > 资源预分配

易错清单

1. 进程状态转换的死记法：阻塞态不能直接到运行态，就绪态不能直接到阻塞态。只有运行中的进程才能主动请求阻塞。
2. ”键盘输入”“缺页异常”导致的是”运行 → 阻塞”，不是”运行 → 就绪”；只有”时间片用完”“被抢占”“主动 yield”才是”运行 → 就绪”。
3. **PCB** 是进程存在的唯一标志，进程切换保存的上下文放在 **PCB** 中而不是栈中；每个进程有独立的内核栈。
4. 同一进程的线程切换不需要修改页基址寄存器、不需要切换页表，原有 TLB 不失效；需要更新的是 PC、通用寄存器、栈基址寄存器。
5. ”操作系统为每个用户级线程建立一个线程控制块”是错误的——用户级线程的 TCB 由用户态线程库维护。
6. 用户级线程的一个线程阻塞会导致整个进程阻塞；内核级线程不会。用户级线程无法利用多处理机并行。
7. 管道位于内核缓冲区，不在用户空间也不在磁盘上；读写都需要互斥；不支持 **lseek** 定位；是

字节流而不是消息；所有文件描述符关闭后缓冲区被回收而不持久化。

8. 共享内存本身没有任何同步机制，必须配合信号量；共享物理页框一定相同，但各进程中的逻辑页号可以不同。
9. 等待时间只统计进程处于就绪态的时间，不要把后备队列等待时间和 I/O 时间算进去。
10. 带权周转时间必然 ≥ 1；周转时间相同时，运行时间长的进程带权周转时间更小。
11. 题目未说明时，**”** 短作业优先**”** 默认为非抢占式；只有带” 所有进程同时到达” 前提时才说 SJF 平均周转时间最少，否则应说 SRTN 最少。
12. **RR** 中同一时刻进程下处理机、另一进程到达时，新到达的进程先进入就绪队列。
13. 时间片过大 **RR** 退化为 **FCFS**；时间片过小则切换开销剧增。
14. **2019-27** 多级反馈队列题的坑：算 P1 等待时间时极易漏掉”P2 在 Q1 执行的那 10 ms 期间 P1 在就绪队列等待” 这一段。
15. **2010** 年考过**”** 降低优先级的合理时机**”** 是**”** 进程时间片用完**”**，不是” 完成 I/O 进入就绪队列” 或” 从就绪态转为运行态”。
16. **”** 进程处于临界区时不能进行调度**”** 是错的——只有处于内核临界区时才不能调度，用户级临界区中可以调度。
17. 中断处理过程中、内核临界区中、禁用中断期间、原语执行期间不能进行进程调度与切换。
18. 模式切换不一定是进程切换：中断处理完成返回用户态时，前后可能是同一个进程。进程切换一定发生在内核态、前后一定是不同进程。
19. 四条原则中只有前三条必须实现，让权等待是可选的（Peterson、TSL、Swap、自旋锁都不满足）。
20. 所有软件互斥方法都不满足让权等待，因为进入区是空循环忙等。
21. **TSL/Swap** 机制下不存在阻塞态的进程，等待者一直是就绪态或运行态；`while (TSL(&lock))` 应在开中断状态下执行。
22. 关中断是特权指令，用户进程不能使用；中断屏蔽只能用于单处理机。
23. 自旋锁在单处理机上必须配合抢占式调度器，否则永远自旋无法解锁。
24. 记录型信号量中 `S.value = 0` 表示资源刚好用完、无进程等待，而不是” 没有资源”；`S.value < 0` 时其绝对值是等待进程数。
25. 信号量只能进行 **P/V** 操作，不能做值判断。`if (bowl >= 1)` 是错误写法。
26. 信号量的 **P(S)**、**V(S)** 本身必须原子执行，因为 S 是共享变量；单处理机可关中断，多处理机需要自旋锁或 CAS。
27. 生产者-消费者中 `P(empty)` 必须在 `P(mutex)` 之前，颠倒就会死锁；**V** 操作顺序可以交换。
28. 缓冲区数量为 **1** 时可以省去 `mutex`（一般信号量天然互斥），但数量 ≥ 2 时必须用 `mutex`。
29. 读-写问题中 `count` 必须用 `mutex` 保护，否则多个读者可能都认为自己是第一个读者，造成第二个读者永久阻塞。
30. 读者优先会让写者饿死，写者优先会让读者饿死，加入 `queue` 信号量才是读写公平。
31. 哲学家进餐的死锁根源是循环等待，三种解法分别破坏” 循环等待”（限并发数、奇偶顺序）和” 请求并保持”（拿筷整体互斥）。
32. 循环等待不是死锁的充分条件——同类资源数大于 1 时，等待链外的资源可以打破环。
33. 死锁进程至少 **2** 个；单个进程阻塞不是死锁；发生死锁时一定有循环等待。
34. 死锁和饥饿是操作系统的问题，死循环是程序员的问题；死锁进程处于阻塞态，饥饿进程可能处于就绪态或阻塞态。
35. **”** 银行家算法可以预防死锁**”** 是错的（它属于死锁避免，没有破坏任何必要条件）；**”** 银行家算

法可以判断系统是否处于死锁状态**”** 也是错的（判断死锁要用死锁检测）。

36. 不安全状态不一定发生死锁；但发生死锁时一定处于不安全状态。
37. 银行家算法的请求检查有三步：不超过 Need、不超过 Available、试探分配后系统仍安全；三者缺一不可，任何一步失败都必须拒绝或让进程等待。
38. 死锁检测不需要进程的资源总量信息，只需要当前时刻的请求与分配情况；而死锁避免需要整个生命周期的最大需求。
39. 化简资源分配图后，还连着边的进程才是死锁进程，并不是系统中所有进程都死锁。
40. 死锁检测的合理触发时机是**”** 每隔一段时间或 **CPU** 利用率显著降低时**”**，不是” 死锁发生时”（逻辑悖论）也不是” 每次申请资源时”（开销过大）。

第十六章内存管理

本章考纲要求

三、内存管理（一）内存管理基础

1. 内存管理的基本概念：逻辑地址空间与物理地址空间，地址变换，内存共享，内存保护，内存分配与回收。
2. 连续分配管理方式
3. 页式管理
4. 段式管理
5. 段页式管理

（二）虚拟内存管理

1. 虚拟内存的基本概念
2. 请求页式管理
3. 页框分配与回收
4. 页置换算法
5. 内存映射文件（memory-mapped files）
6. 虚拟存储器性能的影响因素及改进方法

内存管理是操作系统部分分值最重、计算题最密集的一章，在统考中以三种形态出现：年年必有的 **2** 分选择题（地址变换、页表规模、置换算法判定、抖动与工作集）、计算大题（置换算法填表、EAT 计算、页表项与页表占用空间计算）、以及概念辨析题（分页与分段的区别、装入方式与地址变换时机）。特点是「不背公式算不出」，但「背了公式不看条件也一定算错」——地址变换的每一步都要清楚是谁在做、访问了几次内存。本章按考纲顺序把五大块全部写透，并为每个可计算的知识点配完整算例。

一、内存管理基础

（一）内存管理的基本概念

*1.*逻辑地址空间与物理地址空间

程序从源代码到在 CPU 上运行，要经历编译、链接、装入三个步骤，每一步产生的「地址」含义都不同，这是本章第一组必须分清的概念。

- 编译：编译程序把用户源代码编译成若干目标模块。此时每个目标模块都从 0 号单元开始编址，这个 0 号地址只是相对于该模块的。
- 链接：链接程序把这一组目标模块以及所需的库函数链接在一起，形成一个完整的装入模块（可执行文件）。链接完成后，各模块的相对地址被拼接成一个统一的、仍然从 0 号单元开始编址的地址空间，这就是逻辑地址空间（虚拟地址空间）。
- 装入：装入程序把装入模块装入内存运行，此时才真正涉及物理位置。

由此得到两个核心定义：

- 逻辑地址（**Logical Address**）：又称相对地址、虚拟地址。是程序视角看到的地址，由编译链接产生，从 0 开始编址。逻辑地址的集合构成逻辑地址空间。
- 物理地址（**Physical Address**）：又称绝对地址。是内存单元的真实地址，由硬件（内存管理部件 **MMU**）在访存时使用。物理地址的集合构成物理地址空间。

一句话区分两个地址空间

逻辑地址空间是「程序以为自己住在哪里」，物理地址空间是「程序实际住在哪里」。两者可以完全不同：逻辑地址从 0 开始连续编号，而物理地址可以离散、可以从任意位置开始。把逻辑地址翻译成物理地址的过程叫地址重定位，由 **MMU** 完成。

真题考法：逻辑地址在哪一步形成

【2011-30】形成该逻辑地址的阶段是（ ）。A. 编辑 B. 编译 C. 链接 D. 装载答案 **C**。编辑不产生地址；编译后每个目标模块各自从 0 开始编址，还不是完整逻辑地址；链接才把各模块拼成统一从 0 开始编址的完整逻辑地址空间；装载做的是把它送进内存并（或运行时）变换成物理地址。

*2.*地址变换与装入方式

地址变换（地址重定位）就是把逻辑地址转换成物理地址。按照「物理地址在什么时刻生成」，有三种装入方式，而这三种方式正好决定了地址变换的时机——这正是考纲把「装入方式」与「地址变换」并列的用意。

装入方式	物理地址生成时机	是否可移动 / 可申请新空间	硬件支持	典型搭配
绝对装入	编译（汇编）时	不可移动	不需要	单一连续分配
可重定位装入（静态重定位）	装入时一次性生成	装入后不可移动、不能再申请空间	只需一个加法器	固定分区分配
动态运行时装入（动态重定位）	执行时才生成	运行中可移动、可动态申请	必须要有重定位寄存器 + **MMU**	动态分区、分页、分段、段页式

- 绝对装入：编译时已知程序将驻留在哪里，编译程序直接产生绝对地址目标代码，装入时不需修改地址。只能用于单道环境。
- 可重定位装入：编译时起始地址未知，编译器产生可重定位代码；装入时把模块搬到空闲区，并将模块内所有逻辑地址一次性加上该区起始地址。要求装入时一次分配所需的全部内存空间，装入后既不能移动，也不能再申请新空间。
- 动态运行时装入：装入后程序中仍是相对地址，直到真正执行到该条指令时才由重定位寄存器配合 MMU 完成变换。优点是：可分配到不连续的存储区；便于程序段共享；可只装入部分代码即投入运行、运行中动态申请；允许程序在内存中移动（因此进程被多次换出换入到不同位置也能正确运行）。

静态重定位与动态重定位的三个易错点

1. 静态重定位 ≠ 静态链接。前者是装入方式（决定物理地址何时生成），后者是链接方式（决定目标模块何时拼接、是否生成完整逻辑地址），两者是不同阶段的事。
2. 页式管理不适合静态重定位。因为页式下进程的页面离散存放，且运行中页面可能被换出换入到不同页框，物理位置会变化，必须用动态重定位。
3. 单一连续分配与固定分区分配不需要 **MMU** 做地址变换，因为其逻辑地址与物理地址的关系只是简单地「加上一个固定偏移」，甚至可以直接等价；而动态分区、分页、分段都必须有地址变换硬件。

程序的链接同样有三种方式，常在选择题中与装入方式一起出现：静态链接（运行前链接成一个完整装入模块）、装入时动态链接（装入内存时边装入边链接）、运行时动态链接（执行中用到某模块时才链接并调入）。

*3.*内存保护

内存保护的目标是保证各进程在各自的存储空间内运行、互不干扰。考纲点名的两种做法都属于存储区域保护（判断是否「地址越界」）：

- 方法一：上下限寄存器。在 CPU 中设置一对上、下限寄存器存放进程所占内存区间的上、下限地址，每次访存时比较目标地址，超出区间即产生越界异常。
- 方法二：重定位寄存器 + 界地址寄存器，也叫基址寄存器 + 限长寄存器。
  - 重定位寄存器（基址寄存器）：存放进程所占内存的起始物理地址。
  - 界地址寄存器（限长寄存器）：存放进程的最大逻辑地址（即进程长度上限）。
  - 访存顺序：先拿逻辑地址与界地址寄存器比较，不超过才允许，然后与重定位寄存器保存的基址相加得到物理地址。

两个寄存器：一个「比」，一个「加」

界地址寄存器用来比（判断越界），重定位寄存器用来加（完成变换）。比较必须在加法之前完成，否则越界的地址已经被变换成物理地址，保护就失效了。这两个寄存器只能由操作系统内核通过特权指令加载，用户程序不能修改——这正是「页表必须存放在内核区」的同一个道理。

除了「地址越界」，还有一类保护叫访问方式保护（存取控制保护）：判断是否发生了「访问越权」。即使逻辑地址没有越界，若实际访问操作与该页/段的权限不符，也会触发存储保护错。它通过在页表项或段表项中设置访问权限位来实现，通常规定：

- 各程序对本程序所在的存储区可读可写；
- 对共享区或已获授权的其他用户信息可读不可写；
- 对未获授权的信息（如操作系统内核、页表本身）不可访问；
- 数据段可指定为可读可写或只读，程序段（代码段）可指定为只可执行或只读。

真题考法

【2009-26】分区分配内存管理方式的主要保护措施是（ ）。A. 界地址保护 B. 程序代码保护 C. 数据保护 D. 栈保护答案 **A**。分区分配（连续分配）中进程占据一段连续内存，防止越界的核心手段就是界地址（基址 + 限长）保护。【2023-28】涉及「进程对数据段和代码段可以有不同的访问权限」——正确，这正是访问方式保护的体现。

*4.*内存共享

内存共享指允许多个进程访问内存的同一部分。

- 只有只读的区域才可以共享。可写区域一旦共享，一个进程的修改会破坏其他进程的数据，产生不确定性。
- 可重入代码（纯代码，**reentrant code / pure code**）：允许多个进程同时访问但不允许被任何进程修改的代码，执行中不依赖任何会改变的全局状态，因此可以共享。为了让「共享代码 + 私有数据」并存，实际执行时为每个进程配一个局部数据区，把执行中可能改变的部分复制到该数据区，程序只修改自己的私有数据区，绝不去改共享代码。可重入代码不属于临界资源。

为什么分段比分页更容易实现共享

假设共享内容占 N 个页框大小：分页系统要在每个进程的页表中都建立 N 个页表项，分别指向被共享的 N 个页框；分段系统不管共享段多大，都只需要一个段表项指向共享段始址。所以「段的共享非常简单易行」。例如 40 个用户共享 160KB 代码区（页大小 4KB）：分页要每个进程建 40 个页表项，分段只需 1 个段表项。

*5.*内存分配与回收

内存分配与回收是内存管理最基本的功能，操作系统必须解决三个问题：如何记录哪些区域已被划分出去、哪些还空闲；进程结束时如何回收；进程应该放在内存的哪个位置。

分配方式的演化主线是：

$$\text{单一连续分配} \rightarrow \text{固定分区分配} \rightarrow \text{动态分区分配} \rightarrow \text{离散分配（页 / 段 / 段页式）}$$

前三种统称连续分配管理方式（为一个程序分配一段连续的内存空间），最后一种为非连续分配管理方式。连续分配的固有缺陷是：一个 1GB 的进程必须要找到连续的 1GB 空间，即使内存总空闲量足够、但不连续，也无能为力。非连续分配正是为解决这个矛盾而生。

（二）连续分配管理方式

*1.*单一连续分配

把内存划分为系统区和用户区两部分：系统区位于低地址，供操作系统使用；用户区位于高地址，整个用户区只装入一道用户程序，由该程序独占。

- 优点：实现最简单；可以采用覆盖技术扩充内存；因为是单道，不一定需要内存保护（无其他程序可干扰）。
- 缺点：只能用于单道批处理系统；有内部碎片（进程用不满整个用户区）；存储器利用率低。

单一连续分配的碎片类型

单一连续分配只有内部碎片，没有外部碎片。理由是「分配单元」就是整个用户区，用户区之外不存在可分配的空闲块，因此不存在「分配单元之间」的碎片。

*2.*固定分区分配

把整个用户空间预先划分为若干个固定大小的分区，每个分区只装入一道作业，进程与分区一一对应。这是最早出现的、可运行多道程序的内存管理方式。

分区大小的划分有两种做法：

- 分区大小相等：实现简单，但缺乏灵活性。程序太小浪费分区（内部碎片大），程序太大放不进去。
- 分区大小不等：把分区划分为多个较小分区、适量中等分区、少量大分区，以兼顾大小作业。可以按作业的常见大小分布来设计，提高利用率。

操作系统用分区说明表来管理这些分区。表中每个表项对应一个分区，通常按分区大小排列，记录：分区号、分区大小、分区起始地址、状态（已分配 / 未分配）。分配时由操作系统内核检索该表，找到一个能满足程序大小且未分配的分区，分配给该程序并把状态改为已分配；回收时把状态改回未分配。

- 优点：实现简单，无外部碎片；可以采用静态重定位；适合嵌入式微型设备（是代价最小的多

道存储管理方式，不需要特殊数据结构和硬件支持）。

- 缺点：会产生内部碎片，内存利用率低；当用户程序太大时可能所有分区都装不下，只能改用覆盖技术，而覆盖又会降低性能。

*3.*动态分区分配

动态分区分配不预先划分分区，而是在进程装入内存时，根据进程的大小动态地建立分区，分区的大小和数目都是可变的。

内存碎片分为两类，必须分清：

- 内部碎片（**Internal Fragmentation**）：分配单元内部未被使用的内存。产生原因是「分配单元大小 > 进程实际所需」，即「过量分配」。
- 外部碎片（**External Fragmentation**）：分配单元之间未被使用的内存。产生原因是「按需分配」后，相邻已分配区之间留下了小的、无法被任何进程利用的空隙。

三种连续分配的碎片对照

分配方式	内部碎片	外部碎片
单一连续分配	有	无
固定分区分配	有	无
动态分区分配	无	有

「外部碎片是动态分区分配才有的概念」（准确地说，是「按需分配」类方式的通病）。解决外部碎片的方法是紧凑（**Compaction**，拼凑）技术：移动已分配给进程的内存分区，把它们向低地址端挤压合并，从而在高地址端腾出大片连续空闲空间。紧凑的前提是所有程序都支持动态重定位（否则程序移动后内部地址全部失效），代价是移动耗时、开销大。

（1）记录空闲分区的数据结构

数据结构	组织方式	说明
空闲分区表	数组 / 顺序表	每个空闲分区对应一个表项，表项包含分区号、分区大小、分区起始地址等
空闲分区链	双向链表	每个分区的起始部分和末尾部分分别设置前向指针和后向指针，起始部分还可记录分区大小等信息

（2）四种基于顺序搜索的动态分区分配算法

这是考纲明确点名的四个算法，必须能说清楚「空闲分区按什么顺序链接」「从哪里开始找」以及优缺点。

算法	空闲分区链的排列	检索起点	选择规则
首次适应（First Fit）	按地址递增	每次都从链首开始	找到第一个能满足要求的空闲分区

算法	空闲分区链的排列	检索起点	选择规则
邻近适应（Next Fit，循环首次适应）	按地址递增（可排成循环链表）	从上次查找结束的位置开始	找到第一个能满足要求的空闲分区
最佳适应（Best Fit）	按容量递增	从链首开始	找到第一个能满足要求的最小空闲分区
最坏适应（Worst Fit）	按容量递减	从链首开始	找到第一个能满足要求的最大空闲分区

四个算法的优缺点：

- 首次适应：综合性能最好，开销小（回收后一般不需重新排序），能在高地址保留大内存块；缺点是每次从链头查起，易在低地址积累大量小空闲区，且每次分配都要经过它们，增大查找开销。
- 邻近适应：开销小，不必每次从低地址的小分区查起；但低地址与高地址被使用的概率相同，它会把高地址的大分区也逐步切碎，最终无大分区可用，隐含了最佳适应的缺点，通常比首次适应更差。
- 最佳适应：大分区被更多地保留下来，但每次挑最小分区会留下越来越多很小的、难以利用的外部碎片，且开销大（回收后可能需重新排序）。它最容易产生内存碎片。
- 最坏适应：可减少难以利用的小碎片，中等大小分配较多时效果最好；但大分区很快被用完、不利于大进程，且开销大。

最佳适应 $\neq$ 性能最好

「最佳适应」这个名字极具误导性。它只是让本次分配的剩余空间最小（局部最优），但从全局看，它把大分区反复切碎、留下大量小碎片，反而使后续大作业无处可放。所以统考的标准答案是：最容易产生内存碎片的是最佳适应算法；而综合性能最好的是首次适应算法。

（3）同一组作业请求下四种算法的完整分配过程对照

设定场景：内存中初始有 5 个空闲分区，按地址递增依次为

$$100\text{KB},\quad 500\text{KB},\quad 200\text{KB},\quad 300\text{KB},\quad 600\text{KB}$$

依次请求：$A=15\text{KB}$，$B=30\text{KB}$，$C=50\text{KB}$，$D=70\text{KB}$，$E=100\text{KB}$。

各算法的最终结果先列总表：

算法	A=15KB	B=30KB	C=50KB	D=70KB	E=100KB	分配完成后剩余空闲分区（按地址序）
首次适应	100K 区，剩 85K	500K 区，剩 470K	200K 区，剩 150K	300K 区，剩 230K	失败（无 $\geq$100K 的空闲区）	85K、470K、150K、230K、600K
邻近适应	100K 区，剩 85K	500K 区，剩 470K	200K 区，剩 150K	300K 区，剩 230K	失败（同首次适应）	85K、470K、150K、230K、600K
最佳适应	100K 区，剩 85K	200K 区，剩 170K	300K 区，剩 250K	85K 区，恰好用尽	失败（无 $\geq$100K 的空闲区）	170K、250K、500K、600K

算法	**A=15KB**	**B=30KB**	**C=50KB**	**D=70KB**	**E=100KB**	分配完成后剩余空闲分区（按地址序）
最坏适应	600K 区，剩 585K	585K 区，剩 555K	555K 区，剩 505K	505K 区，剩 435K	500K 区，剩 400K	435K、400K、100K、200K、300K

下面给出逐步分配过程，这是考场上真正要写出来的东西。

首次适应（**First Fit**）——空闲区始终按地址递增排列，每次都从链首开始：

步骤	请求	检索过程	选中分区（起始）	分配后该分区剩余
1	A = 15K	从链首起，100K 是第一个 ≥15K 的	100K @ 低地址	85K
2	B = 30K	从链首起，85K 已不够？85K ≥ 30K，就在 **85K** 里分配	85K	55K
3	C = 50K	链首 55K ≥ 50K，仍在最前面的小碎片里分配	55K	5K
4	D = 70K	链首 5K 不够，下一个 500K ≥ 70K	500K	430K
5	E = 100K	链首 5K、430K 中 5K 不够，430K ≥ 100K	430K	330K

可见首次适应确实会在低地址端不断切出越来越小的碎片（本例中切出了 5K 的碎片），这正是它「低地址积累小碎片、增加查找开销」这一缺点的直接体现。

最佳适应（**Best Fit**）——空闲区按容量递增排列，每次选最小的够用分区：

步骤	请求	空闲区容量序	选中的最小够用分区	分配后
1	A = 15K	100, 200, 300, 500, 600	100K	剩 85K
2	B = 30K	85, 200, 300, 500, 600	85K（不是 **200K**）	剩 55K
3	C = 50K	55, 200, 300, 500, 600	55K	剩 5K
4	D = 70K	5, 200, 300, 500, 600	200K	剩 130K
5	E = 100K	5, 130, 300, 500, 600	130K	剩 30K

最佳适应把两个大分区留了下来（500K、600K 未被触碰），确实「保留了大分区」，但代价是造出了 **5K**、**30K** 这种几乎永远用不上的小碎片——这就是它「最容易产生外部碎片」的原因。

最坏适应（**Worst Fit**）——空闲区按容量递减排列，每次选最大的分区：

步骤	请求	当前最大分区	分配后剩余
1	A = 15K	600K	585K
2	B = 30K	585K（切完仍最大）	555K
3	C = 50K	555K	505K
4	D = 70K	505K	435K

步骤	请求	当前最大分区	分配后剩余
5	E = 100K	500K（此时 500K 已大于 435K）	400K

最坏适应的特点是剩余分区大小趋于均匀，本例中剩下的全是三四百 KB 的中等块，确实没有小碎片；但 **600K** 的大块被反复切削、彻底消失，一旦后续来一个 500K 的大作业就无处可放。

邻近适应（**Next Fit**）——排列方式与首次适应相同，但检索从上次结束位置继续：

步骤	请求	指针起点	检索过程	选中分区	剩余
1	A = 15K	链首 100K	100K 够	100K	85K
2	B = 30K	85K 处	85K 够	85K	55K
3	C = 50K	55K 处	55K 够	55K	5K
4	D = 70K	5K 处	5K 不够 → 500K 够	500K	430K
5	E = 100K	430K 处	430K 够	430K	330K

前三次分配与首次适应完全一致（因为指针一直在低地址端），差别要在回收之后才会显现：回收一个低地址分区后，首次适应下次分配会立刻用上它，而邻近适应的指针已经走到高地址，低地址的空闲区会被「跳过」很久，于是低地址碎片越积越多，同时高地址大分区也被逐步切碎——这正是「邻近适应隐含最佳适应缺点」的含义。

真题考法

【2010-28】主存 55MB（初始空闲），最佳适应算法，顺序为：分配 15MB、分配 30MB、释放 15MB、分配 8MB、分配 6MB，此时最大空闲分区大小是（ ）。答案 **B**（**9MB**）。推演：55MB 分 15MB 后剩 40MB；再分 30MB 后剩 10MB；释放 15MB 后有 15MB 与 10MB 两个空闲区；分配 8MB 时最佳适应选 10MB → 剩 2MB；分配 6MB 时只剩 15MB 够用 → 剩 9MB。最大空闲分区 9MB。【2017-25】按字节编址、最佳适应、每次分配回收后都重新排序。当前空闲分区为 20K 处 40KB、500K 处 80KB、1000K 处 100KB、200K 处 200KB。回收起始 60K、大小 140KB 的分区后，空闲分区个数、链首分区的起始地址和大小是（ ）。答案 **B**（**3**、**500K**、**80KB**）。回收的 60K~200K 与紧邻的 20K 处 40KB、200K 处 200KB 上下同时合并，得 380KB；空闲区只剩 3 个。最佳适应按容量递增排，链首是最小的 80KB，起始地址 500K。【2019-32】最容易产生内存碎片的是（ ）。答案 **C**，最佳适应算法。【2024-27】每次回收分区时仅合并大小相等的空闲分区的是（ ）。答案 **A**，伙伴算法（伙伴系统只把大小相等、地址相邻的两个伙伴块合并）。

*4.*真题补充：伙伴系统

四种顺序搜索算法之外，伙伴系统（**Buddy System**）在真题中出现过，只需掌握其判定特征：规定无论已分配还是空闲分区，大小均为 2 的整数次幂；需要大小 s 且 $2^{i-1} < s \le 2^i$ 时把 2^i 的块一分为二，重复划分直到合适为止。回收时只合并「大小相同」且地址相邻的伙伴块（2024-27 的直接考点），合并后大小为原来两倍。它既产生内部碎片，也产生外部碎片。

（三）页式管理

连续分配的根本困境是「要求连续」。非连续分配允许把一个进程分散地装入到许多不相邻的分区中，内存利用率因此大幅提升。基本分页存储管理是最基本的一种。

*1.*页面、页框与页表

- 页框（页帧、物理页面、**Frame**）：把物理地址空间划分为大小相同的固定基本分配单位，编号叫页框号，从 0 开始。

- 页面（页、逻辑页面、**Page**）：把逻辑地址空间也划分为大小相同的单位，编号叫页号，也从 0 开始。
- 页框大小与页面大小必须相等（这样页内偏移才能原样保留）。
- 操作系统以页框为单位为进程分配内存：进程的每个页面分别放入一个页框中，二者一一对应，各页面不必连续存放，也不必遵循先后顺序。
- 页表（**Page Table**）：为记录每个页面被放进哪个页框，操作系统为每个进程建立一张页表：一个进程对应一张页表，进程的每个页面对应一个页表项（**PTE**），页表项记录页号 $\rightarrow$ 页框号的映射（页表项等长且顺序连续存放，故页号可隐含，类比数组下标）；页表项中还通常包含有效位、访问位、修改位、存取权限位等控制信息。

页表寄存器（**PTBR / PTR**）：系统中只设置一个，存放当前运行进程页表在内存的起始地址 F 和页表长度 M。进程未执行时这两个值存放在该进程的 **PCB** 中，被调度时才装入页表寄存器。

页式管理的两个「一定」

1. 页表一定存放在内核区。页表负责地址映射与权限管理，若放在用户区，用户进程就能篡改它，从而非法访问任意内存或使系统崩溃。所以页表由内核维护，受硬件特权级保护。
2. 页式管理的地址是一维的。只要确定了页面大小，逻辑地址结构就确定了：给出一个逻辑地址，硬件可以自动把它切成「页号 + 页内偏移」两部分，不需要程序显式告诉系统偏移量占多少位。

*2.*地址结构与地址变换

逻辑地址结构：分页的逻辑地址被划分为两部分

$$\underbrace{\text{页号}P}_{m\text{ 位}}\ \Big|\ \underbrace{\text{页内偏移量}W}_{k\text{ 位}}$$

- 若用 k 位表示页内偏移量，则该系统一个页面的大小为 2^k 个内存单元（字节）；
- 若用 m 位表示页号，则该系统中一个进程最多允许有 2^m 个页面，即进程的逻辑地址空间最大为 2^{m+k}。

为了便于计算，页面大小一般取 **2** 的整数次幂。此时「切分」就是纯粹的位运算：若页面大小为 2^k B，则逻辑地址末尾 k 位就是页内偏移量，其余高位就是页号。

基本地址变换机构（无快表）的完整过程，设页面大小为 L，页表起始地址为 F，页表长度为 M（表项数），逻辑地址为 A：

1. 进程被调度时，内核把 PCB 中的 F 和 M 装入页表寄存器。
2. 计算页号 $P = A/L$，页内偏移量 $W = A \bmod L$（页面大小为 2 的幂时即取高位 / 低位）。
3. 越界检查：若 $P \geq M$（页号 $\geq$ 页表长度），产生越界中断；否则继续。
4. 查页表：页号 P 对应的页表项地址为

$$\text{PTE 地址} = F + P \times (\text{页表项长度})$$

从该地址取出页表项内容 b，即该页所对应的页框号。（注意：页表里记录的是内存块号，不是内存块的起始地址！b 号页框的起始物理地址 $= b \times L$。）

1. 形成物理地址并访存：

$$E = b \times L + W$$

物理地址计算公式（必背）

$$\boxed{\text{物理地址} = \text{页框号} \times \text{页面大小} + \text{页内偏移量}}$$

等价写法：先由页框号得到页框起始物理地址 $b \times L$，再与页内偏移量 W 拼接（因为页面大小是 2 的幂，b 与 W 在二进制上刚好拼在一起，不会进位冲突）。

无快表时，访问一个逻辑地址需要两次访存：第一次访存查内存中的页表，第二次访存才是访问目标数据。

快表（**TLB**，**Translation Lookaside Buffer**）：由于局部性原理，一段时间内连续查到的往往是同一个页表项。于是在 CPU 中设置一个相联存储器（**associative memory**）实现的、速度远快于内存的高速缓冲器，用来存放当前访问的若干页表项，以加速地址变换。TLB 对应的、内存中的页表被称为慢表。

引入快表后的地址变换过程：

1. CPU 给出逻辑地址，硬件算得页号 P 与页内偏移量 W，把 P 与快表中所有页号同时比较。
2. 命中：直接取出该页对应的页框号，与 W 拼成物理地址然后访存。此时只需一次访存。
3. 未命中：访问内存中的页表，找到页表项得到页框号，再拼成物理地址访存。此时需要两次访存；同时基于局部性原理，应把刚查到的这个页表项存入快表，以便后面再用。若快表已满，需要按一定算法替换掉一个旧页表项。

TLB 的三个细节

1. TLB 中的页号是显式保存的（不像内存页表那样可以隐含），因为 TLB 里存放的是任意若干个页表项，彼此不连续。
2. TLB 位于 **CPU** 内部，由相联存储器实现，具备快速（并行）查找能力。
3. 快表中有的页面一定在内存中；若某页被换出外存，必须同时删除快表中的相应表项，否则会访问到错误的页面。进程切换时，TLB 要刷新（或按 ASID 区分）。

*3.*地址变换算例

算例 **1**（逻辑地址 $\rightarrow$ 物理地址）

某分页系统按字节编址，页面大小为 4KB，逻辑地址为 32 位。已知进程页表起始物理地址为 0020 0000H，页表项长度为 4B。现给出逻辑地址 $A =$ 0000 3020H，求物理地址。

1. 页面大小 $4\text{KB} = 2^{12}\text{B}$，故低 **12** 位为页内偏移量，高 **20** 位为页号。
2. 拆分 $A =$ 0000 3020H：低 12 位 $=$ 020H，即 W；高 20 位 $=$ 3，即 $P = 3$。
3. 页号 3 的页表项物理地址 $= F + P \times 4 =$ 0020 0000H $+ 3 \times 4 =$ 0020 000CH。
4. 设从该地址读出页表项内容为 $b = 10$（即页号 3 映射到 10 号页框）。
5. 物理地址 $E = b \times L + W = 10 \times 4096 + 32 = 40960 + 32 = 40992$；用十六进制拼接更直接：页框号 $10 =$ 00AH 与 $W =$ 020H 拼成 $E =$ 0000 A020H。

校验：$E = 10 \times 2^{12} + 2^5 = 40960 + 32 = 40992$，与 A020H $= 10 \times 4096 + 32$ 一致。

算例 **2**（页表项大小与页表占用空间）

设系统物理内存为 4GB，页面大小为 4KB，问每个页表项至少需要多少字节？整个页表至少占多少字节？

1. 内存块大小 = 页面大小 = $4\text{KB} = 2^{12}\text{B}$。
2. 4GB 内存被划分成 $2^{32}/2^{12} = 2^{20}$ 个内存块。

3. 内存块号范围是 $0 \sim 2^{20}-1$，至少需要 **20 bit** 表示；取整字节数，至少 **3B**（$3\times 8=24$ bit）。
4. 页号隐含不占空间，故一个页表项占 3B。一个进程逻辑地址空间对应 2^{20} 个页面，即 2^{20} 个页表项，页表至少占 $3\times 2^{20}\text{B}=3\text{MB}$。

进一步的细节：若页表项为 3B、页表起始地址为 X，则 M 号页对应的页表项地址为 $X+3M$。一个 4KB 页框可存放 $\lfloor 4096/3\rfloor=1365$ 个页表项，剩余 $4096 \bmod 3=1$B 的页内碎片；因此 1365 号页表项并不在下一个页框的开头，而要额外加 1B，地址为 $X+3\times 1365+1$。

为什么页表项常常做成 4B 而不是 3B

理论上 3B 就够表示 4GB 内存的块号，但 4B 恰好使得一个 **4KB** 页框能整存 **1024** 个页表项（$4096/4=1024$），不产生页内碎片，查表时用「页框号 + 偏移」也更规整。这就是「为了方便页表的查询，常让一个页表项占更多字节，使每个页面恰好装得下整数个页表项」的含义。统考做题时以题目给出的页表项大小为准。

*4.*多级页表

单级页表的缺陷是空间上的：每个进程都有独立的虚拟地址空间，因而都有自己的页表，页表必须覆盖全部虚拟地址空间。32 位环境下虚拟地址空间 4GB、页大小 4KB，需要约 2^{20}（100 万）个页表项；每个页表项 4B，则每个进程的页表就要 **4MB**，100 个进程就是 400MB——纯属浪费，因为大多数进程实际用到的空间远不到 4GB。

多级页表的原理：把长长的单级页表再分页，形成多张页表，同时为这些离散的页表再建立一张索引表（页目录表，又称外层页表 / 顶层页表）。所以两级页表实质上就是「页表的页表」。

32 位、4KB 页、4B 页表项下的经典划分：规定页目录表最多占一个页面，故页目录表可容纳 $4096/4=1024=2^{10}$ 个页表项。于是 32 位逻辑地址结构变为

$$\underbrace{\text{页目录号}}_{10\text{ 位}}\ \big|\ \underbrace{\text{页表索引（二级页号）}}_{10\text{ 位}}\ \big|\ \underbrace{\text{页内偏移量}}_{12\text{ 位}}$$

为什么这样能省空间？关键在于顶级页表覆盖全部虚拟地址空间，而二级页表按需创建：如果某个一级页表项没有被用到，就不需要创建它对应的二级页表。假设只有 20% 的一级页表项被用到，页表占用就是

$$4\text{KB}+20\%\times 4\text{MB}=0.804\text{MB}$$

对比单级页表的 4MB，是巨大的节约。多级页表的代价是地址变换需要多次访存，变换速度变慢。

多级页表的核心结论

1. 多级页表的最大优点是「减少页表所占的连续内存空间」，而不是加快地址变换速度（实际上更慢）、也不是减少页表项所占字节数（反而更多）、更不直接减少缺页中断次数。
2. 不管是几级页表，必须满足各级页表的大小不能超过一个页面。
3. **CPU** 页表基址寄存器中存放的是当前进程「一级页表的起始物理地址」（不是虚拟地址）。
4. 出现「页目录号」字样，就说明一定是多级页表存储管理方式。

多级页表算例 **1**（二级页表求页目录项数）

某计算机采用二级页表分页存储管理，按字节编址，页大小为 2^{10}B，页表项大小为 2B，逻辑地址空间大小为 2^{16} 页。则页目录表中包含的表项个数至少是（　）。

1. 一个页面可存放 $2^{10}/2=2^9=512$ 个页表项。

2. 逻辑地址空间共 2^{16} 页，即需要 2^{16} 个二级页表项。
3. 这些页表项需占 $2^{16}/2^9 = 2^7 = 128$ 个页面，即需要 128 个二级页表。
4. 每个二级页表在页目录表中占一个表项，故页目录表至少 **128** 个表项。答案 B。

多级页表算例 **2**（求页表级数）

某页式系统逻辑地址空间 256TB，页表项大小 8B，页面大小 4KB。该系统应该采用几级页表?

$$256\text{TB} = 2^{48}\text{B}, \quad \text{页数} = 2^{48}/2^{12} = 2^{36}$$

一个页面可存 $2^{12}/2^3 = 2^9 = 512$ 个页表项，故级数

$$\left\lceil \frac{36}{9} \right\rceil = 4 \text{ 级}$$

多级页表算例 **3**（页表总占用）

32 位系统、页大小 4KB、页表项 4B，二级页表。若一个进程的 4GB 虚拟空间中只有 2 个 4MB 的有效区域，且不共享页表，问页表占多少空间? 页目录表恒为 1 页（4KB）；每个 4MB 区域占 $4\text{MB}/4\text{KB} = 1024$ 个页面，恰好由一个二级页表（4KB）覆盖，故合计 $4\text{KB} + 2 \times 4\text{KB} = 12\text{KB}$。若用单级页表，无论实际用了多少都要为全部 2^{20} 个页表项付出 4MB——这就是多级页表的价值。

真题考法

【2010-31】二级页表求页目录项数，答案 128（见上文算例 1）。【2014-32】属于多级页表优点的是：**D.** 减少页表所占的连续内存空间。【2019-31】地址结构「页目录号 10 位 | 页表号 10 位 | 页内偏移 12 位」，虚拟地址 2050 1225H 对应的页目录号、页号分别是（ ）。答案 **A**（**081H**、**101H**）：把十六进制展开成 32 位二进制后按 10/10/12 切分即可。【2021-29】二级页表中 CPU 页表基址寄存器中存放的是：**B.** 当前进程的一级页表的起始物理地址。【2023-30】进程 R 与 S 共享数据 data，页号分别为 p_1, p_2，页框号分别为 f_1, f_2，则（ ）。答案 **C**：p_1 与 p_2 不一定相等（页号相对于各自的虚拟地址空间），f_1 与 f_2 一定相等（同一份物理数据只有唯一的页框）。【2024-25】进程切换时要执行：Ⅰ更新程序计数器 Ⅱ更新栈基址寄存器 Ⅲ更新页基地址寄存器 → **D**（Ⅰ、Ⅱ、Ⅲ）。

*5.*页面大小对性能的影响

页面大小是分页系统的关键设计参数，影响是双向的：页面较小时页内碎片小、内存利用率高，但每个进程需要的页面数更多，页表过长、占用内存多，且缺页率更高（换入换出更频繁）；页面较大时页表短、缺页率较低（局部性原理），但页内碎片增大，一次换入换出的 I/O 数据量也大。因此页面大小必须折中选择（现实中常见 4KB）。记住：页内碎片 $\leq$ 页面大小 **−1**。

（四）段式管理

*1.*段的基本概念与段表

一个进程的地址空间天然由多个逻辑模块组成，如代码段、数据段、栈段、堆段，各有不同属性（可执行、可读写、可增长），分段就是按这种用户自身的逻辑关系来划分。

- 段（**Segment**）：访问方式和存储数据等属性相同的一段逻辑地址空间。若干段组成进程的逻辑地址空间，段的大小不等。
- 基本思想：按用户进程自身的逻辑关系划分为若干段，每段从 0 开始编址；以段为单位分配，段内连续，段间可以不连续。
- 逻辑地址由二元组 $(s, addr)$ 表示，s 是段号，$addr$ 是段内偏移量。

- 段表：每个进程一张，进程的每一段对应一个段表项。由于各段长度不同，段表项必须含段长 C 与段基址 b，此外还常有状态位、存取权限位、外存始址等。段表项等长且连续存放，故段号可隐含。

分段存储管理的五个特点：方便编程（汇编程序天然按逻辑关系分段）；信息共享（段是有内容共性的逻辑单位体，不像页面可能把一个完整逻辑功能切到不同页）；信息保护（可对特定函数、数据方便地设定权限）；动态增长（段长可变，容易应对数据段增长）；动态链接（以独立逻辑单位为链接单元，恰合分段原则）。

引入分段的目的 $\neq$ 实现离散分配

引入分页的目的是实现离散分配、提高内存利用率（系统行为）；引入分段的目的是满足用户需求——方便编程、分段共享、分段保护、动态链接与增长（用户行为）。这是分页与分段对比题的第一层含义，也是最常被混淆的一层。

*2.*段的地址变换

逻辑地址结构：

$$\underbrace{\text{段号}S}_{n\text{ 位}} \mid \underbrace{\text{段内偏移量}W}_{k\text{ 位}}$$

进程未执行时，段表的起始地址 F 和段表长度 M 存放在 PCB 中；当进程被调度时，系统把它们装入段表寄存器。

完整变换步骤：

1. 由逻辑地址取出段号 S 和段内偏移量 W。
2. 第一次越界检查：若 $S \geq M$（段号 $\geq$ 段表长度），产生越界中断；否则继续。
3. 查段表，找到对应段表项（段表项地址 $= F + S\times$ 段表项长度），取出段长 C 与段基址 b。
4. 第二次越界检查：若 $W \geq C$（段内偏移量 $\geq$ 段长），产生越界中断；否则继续。
5. 计算物理地址：

$$E = b + W$$

1. 用 E 访存。

分页与分段越界检查的次数不同

- 分页只做一次检查：$P \geq M$ 则越界。页内偏移量不需要检查，因为页内偏移量的位数由硬件固定，其最大值恰好等于页面大小，不可能越界。
- 分段必须做两次检查：先是 $S \geq M$，再是 $W \geq C$。因为段的长度是变长的，段内偏移量的位数固定不代表该段真的有这么长。

算例：某分段系统中，段表如下：

段号	段长	段基址（起始物理地址）
0	1000	8000
1	500	3000
2	300	2000

段号	段长	段基址（起始物理地址）
3	1500	6000

问：(1) 访问逻辑地址 (1, 200) 得到的物理地址是多少？(2) 访问 (2, 400) 会怎样？

(1) 段号 $1 <$ 段表长度 4，通过；段内偏移 $200 <$ 段长 500，通过；物理地址 $= 3000 + 200 =$ **3200**。

(2) 段号 $2 < 4$，通过；段内偏移 $400 \geq$ 段长 300，越界，产生越界异常（注意不是「段缺失异常」，也不是「越权异常」——那是访问方式保护的事）。

*3.*段的共享与保护

段的保护主要有两种：界地址保护与存取控制保护。

- 界地址保护：通过逻辑地址与段（页）表信息比较来防止越界，即上面说的两次检查。分页方式只检查页号，分段方式要同时检查段号与段内偏移量。
- 存取控制保护：对进程的段（页）设置只读、读/写、不可访问等限制，访问时进行权限检查。若实际访问操作与权限不符，则发生存储保护错（访问越权）。

段的共享：除每个进程自身的段表外，系统还设置一个供相关进程共同使用的共享段表，其表项除记录共享段的物理地址等信息外，还设一个 **count** 字段记录共享该段的进程数。当 P1 与 P2 共享段 S 时，段 **S** 在内存中只有一份；P1、P2 的段表与共享段表中各有一项记录段 S 的物理地址。段 S 的物理地址唯一，但它在 **P1** 与 **P2** 中的逻辑段号不一定相同。某进程释放该段时，系统并不立即调出，只把共享段表中的 **count** 减 **1**；**count** 减至 **0** 时才真正调出该段。

共享段表的两个易错点

1. 「进程 P1 和 P2 共享段 S，则段 S 在 P1 和 P2 中应具有相同的段号」——错误。段号由各进程自己管理，可以不同。
2. 「P1 和 P2 都通过共享段表中同一个段表项」——正确（2019 年真题考的正是汤小丹旧版教材的「共享段表」做法）。新版教材的做法是在每个进程的段表中各设一个段表项、指向被共享的同一个物理段。统考按「共享段表」理解即可，两者不矛盾：共享段只有一份物理内容，这是根本点。

*4.*分段与分页的对比

对比项	分页	分段
信息的单位性质	页是物理单位，是系统行为	段是逻辑单位，是用户行为
划分目的	实现离散分配，提高内存利用率	满足用户需求：方便编程、共享、保护、动态链接与增长
大小	页大小固定，由系统决定	段长不固定，由用户程序的逻辑决定
对用户是否可见	不可见（透明）	可见（不透明），编程时需显式给出段名/段号
地址空间维度	一维	二维
碎片	有少量页内碎片（内部碎片），无外部碎片	会产生外部碎片，无内部碎片
地址变换	查页表，无快表时 2 次访存	查段表，无快表时 2 次访存

对比项	分页	分段
越界检查	1 次（页号 vs 页表长度）	2 次（段号 vs 段表长度；段内偏移 vs 段长）
共享与保护	实现复杂（需 N 个页表项）	实现简单（只需 1 个段表项）

为什么两种地址结构都是「两段」，维度却不同

形式上页式（页号 + 页内偏移）与段式（段号 + 段内偏移）都是两个量组合，像是二维空间。但分页的页号与页内偏移不是两个独立变量：只要给出「数据在逻辑地址空间中的相对位置」这一个数，配合已知的页面大小就能唯一算出页号和页内偏移量，所以地址是一维的。而段大小不等，段号与段内偏移量是两个独立变量，必须显式给出两个值，所以地址是二维的。典型例子：8086 CPU 的程序计数器由 **CS:IP** 两个寄存器给出，这就是「二维」的直接体现。

（五）段页式管理

*1.*基本思想与地址结构

分页与分段各有优缺点：

	优点	缺点
分页管理	空间利用率高，仅有少量页内碎片，不产生外部碎片	不利于实现信息的共享与保护
分段管理	利于实现逻辑模块间信息的共享与保护	段长过大，分配连续空间不方便，产生外部碎片

段页式管理正是二者的结合：

- 基本思想：先将进程按逻辑模块分段，再把各段分页。对内存空间的管理与分页管理相同，划分为若干与页面大小相同的存储块，以存储块为单位分配。
- 一句话概括：采用分段方法分配和管理用户地址空间，采用分页方法管理物理存储空间。它兼有两者的优点，但开销比纯段式或纯页式都大。
- 逻辑地址结构由三部分组成：

$$\underbrace{\text{段号}S}_{n\text{ 位}} \mid \underbrace{\text{页号}P}_{m\text{ 位}} \mid \underbrace{\text{页内偏移量}W}_{k\text{ 位}}$$

- k 位页内偏移量 $\rightarrow$ 页面大小为 2^k 个内存单元；
- m 位页号 $\rightarrow$ 每个段最大有 2^m 个页面；
- n 位段号 $\rightarrow$ 每个进程最多可分为 2^n 个段。

- 段表与页表：段页式管理中，先由段表获得该段对应的页表在内存中的存放位置，再由页表获得每一页对应的页框号。
  - 每个进程只有一张段表；进程有多个段，每个段各有一张页表，所以一个进程对应多张页表。
  - 每个段对应一个段表项，由段号、页表长度、页表存放的块号（页表起始地址）组成；段表项等长，故段号可隐含。
  - 每个页面对应一个页表项，由页号、块号组成；页表项等长，故页号可隐含。

段页式的碎片类型

段页式会产生内部碎片（页内碎片）。因为对内存的管理方式与分页完全相同，分配单位是页框，进程的最后一个页面通常装不满。例如一个段是 5KB，页大小是 4KB，则该段要占 2 个页面，第二个页面里就有 3KB 的页内碎片。段页式不产生外部碎片。

*2.*段页式的地址变换完整流程

进程未执行时，段表的起始地址 F 和段表长度 M 存放在 PCB 中；当进程被调度时，系统把它们装入段表寄存器。

1. 由逻辑地址取出段号 S、页号 P、页内偏移量 W。
2. 越界检查一：若 $S \geq M$（段号 $\geq$ 段表长度），产生越界中断；否则继续。
3. 查段表，找到对应段表项（段表项地址 $= F + S\times$ 段表项长度），取出该段的页表长度与页表存放的块号。
4. 越界检查二：若 $P \geq$ 页表长度，产生越界中断；否则继续。
5. 依据「页表存放的块号 + 页号」查页表，找到对应页表项，取出内存块号（页框号）。
6. 由内存块号与页内偏移量形成物理地址：$E =$ 页框号 $\times$ 页面大小 $+ W$。
7. 用 E 访存。

访存次数：未引入快表时需要三次访存（查段表、查页表、访问目标单元）；引入快表后以段号和页号为关键字查询，命中时仅需一次访存。

段页式也可以实现内存共享：让共享段指向相同的页表，从而实现进程间的段共享。

真题考法（段页式共享）

在段页式存储管理系统中，进程 R 和 S 共享数据段 data，若 x 在 R 和 S 的段表中的段号分别为 p_1, p_2，x 所在页对应的页框号分别为 f_1, f_2，则（　）。答案 **C**：p_1 和 p_2 不一定相等，f_1 和 f_2 一定相等。理由同分页共享：段号是各进程自己地址空间内的编号，物理内容只有一份、页框号唯一。

算例（段页式地址变换）：某系统按字节编址，段表项 4B，页表项 4B，页面大小 4KB。逻辑地址结构为「段号 8 位 | 页号 12 位 | 页内偏移 12 位」。进程的段表起始地址为 `0000 1000H`，段表长度 5。现访问逻辑地址 `0000 0001 0000 0010 0000 0101 0000 1010 B`（即段号 1，页号 2，页内偏移 `00AH`）。

1. 拆分：段号 $S = 1$，页号 $P = 2$，页内偏移 $W =$ `00AH`。
2. 段号 $1 < 5$，通过。
3. 段表项地址 $=$ `1000H` $+ 1 \times 4 =$ `1004H`。设读出：该段页表长度为 16，页表存放块号 **= 5**。
4. 页号 $2 < 16$，通过。
5. 页表起始物理地址 $= 5 \times 4096 = 20480 =$ `5000H`。页表项地址 $=$ `5000H` $+ 2 \times 4 =$ `5008H`。设读出页框号 **= 9**。
6. 物理地址 $E = 9 \times 4096 + 10 = 36864 + 10 = 36874 =$ `900AH`。

二、虚拟内存管理

（一）虚拟内存的基本概念

*1.*传统存储管理方式的弊端与局部性原理

前面讲的所有分配方式（无论连续还是非连续）都有两个共同特征，也就是两个弊端：

- 一次性：作业必须一次性全部装入内存后才能开始运行。后果有两个：作业很大而装不进内存时，作业根本无法运行；大量作业要求运行时内存不够，只能让少数作业先运行，降低了多道程序的并发度。
- 驻留性：作业装入内存后就一直驻留，任何部分都不会被换出，直至作业运行结束。事实上一个时间段内只需访问作业的一小部分数据，这就导致内存中驻留大量暂时用不到的数据，浪费宝贵的内存资源；运行中的进程还会因等待 I/O 而长期阻塞却仍占着内存。

突破这两点的理论依据是局部性原理（**Principle of Locality**）：

- 时间局部性：一条指令或一个数据被访问后，不久之后很可能再次被访问（典型来源是循环）。
- 空间局部性：一个存储单元被访问后，其邻近的存储单元很可能很快被访问（典型来源是数组、顺序执行的指令）。

局部性原理是虚拟存储的基石

局部性越好，虚拟存储系统越能更好地发挥作用。反过来，如果一个程序的访存完全随机、毫无局部性，虚拟存储不但不能提升性能，反而会因为频繁缺页而性能崩溃（抖动）。

*2.*虚拟内存的定义与特征

定义：基于局部性原理，操作系统可以这样控制程序：

- 程序装入时：只把程序中很快会用到的部分先装入内存，暂时用不到的留在外存；
- 程序执行时：如果所访问的信息不在内存，由操作系统负责把它从外存调入内存；
- 发现内存不够时：由操作系统负责把内存中暂时用不到的信息换出外存。

在操作系统的管理下，在用户看来似乎有一个比实际内存大得多的内存，这就是虚拟内存（**Virtual Memory**）。虚拟性是操作系统虚拟性特征的一个体现：物理内存大小并没有变，只是在逻辑上进行了扩充。

三个特征（必背，注意「多次性」是核心）：

- 多次性：作业运行时无需一次全部装入内存，而是允许分批次调入内存。
- 对换性：作业运行时无需一直常驻内存，而是允许在运行时进行换入和换出。
- 虚拟性：从逻辑上扩充了内存的容量。

虚拟内存容量的两个「受限于」

- 最大容量由计算机的地址结构（**CPU** 寻址范围）决定。
- 实际容量 $=\min($内存容量 $+$ 外存容量, CPU 寻址范围$)$。

例：某计算机地址结构为 32 位、按字节编址，内存 512MB，外存 2GB。则虚拟内存最大容量 $=2^{32}\text{B}=4\text{GB}$；实际容量 $=\min(2^{32}\text{B},\ 512\text{MB}+2\text{GB})=2\text{GB}+512\text{MB}$。

易错推论：「虚拟存储容量只受外存容量限制」错，「只受内存容量限制」错，「虚拟地址的大小由主存和硬盘的大小决定」错——虚拟地址空间大小由地址结构（操作系统）决定。

*3.*虚拟内存的实现方式

虚拟内存的实现必须建立在离散分配的内存管理方式基础上（连续分配无法做到「只装入一部分」还让程序正常跑）。主要有三种：

- 请求分页存储管理
- 请求分段存储管理

- 请求段页式存储管理

与传统非连续分配方式相比，区别只在两点功能上：

- 所访问信息不在内存时，由操作系统负责把信息从外存调入内存——请求调页（请求调段）功能；
- 如果内存空间不够，由操作系统负责把内存中暂时用不到的信息换出外存——页面置换（段置换）功能。

无论哪种方式，都需要以下硬件支持：一定容量的内存和外存；页表机制（或段表机制）作为主要数据结构；中断机构（访问部分尚未调入内存时产生中断）；地址变换机构。

真题考法

【2012-25】关于虚拟存储器，正确的是：**B.** 虚拟存储只能基于非连续分配技术。A 错（连续分配无法实现按需调入）；C、D 错（容量受地址结构与外存共同约束）。【2023-28】错误的是：**D.** 虚拟地址的大小由主存和硬盘的大小决定。【2014-28】能加快虚实地址转换的是：**C.** 仅 Ⅰ、Ⅱ（Ⅰ增大 TLB 容量；Ⅱ让页表常驻内存）。Ⅲ增大交换区只增加外存空间，不加快地址变换。

（二）请求页式管理

请求分页系统 = 基本分页系统 + 请求调页（外存 → 内存）+ 页面置换（内外存页面调换）。与基本分页相比，它在三个方面需要额外的硬件支持。

*1.*请求页表的扩充字段

为了实现「请求调页」，操作系统必须知道每个页面是否已调入内存、若未调入则它在外存中的位置；为了实现「页面置换」，必须能判断哪些页面该被换出、哪些页面被修改过（已修改的必须写回外存）。因此请求页表在基本页表项（页号、页框号）基础上扩充如下字段：

字段	含义	作用
状态位（存在位、有效位）	该页是否在内存中。1 表示在内存、页表项有效；0 表示当前在外存，访问该页表项将导致缺页异常	请求调页的依据
访问字段（访问位）	本页在一段时间内被访问的次数，或最近已有多长时间未被访问（简单实现仅用 1 位表示「是否被访问过」）	供页面置换算法选择换出页面时参考
修改位（脏位）	该页在内存期间是否被修改过	决定换出时是否要写回外存；供置换算法参考
外存地址	该页在外存中的地址	缺页异常处理程序据此去外存取页
保护位	该页的允许访问方式（只读、可读写、可执行等）	访问方式保护

「访问字段」的两种表述

汤小丹教材表述为「访问字段：用于记录本页在一段时间内被访问的次数，或记录本页最近已有多长时间未被访问」。而 CLOCK 类算法只需要一位「是否被访问过」。做题时看题目给的是几位：给 1 位就按 0/1 处理；给计数器就按数值大小处理。统考绝大多数题目是 **1** 位。

*2.*缺页中断处理流程

在请求分页系统中，每当所要访问的页面不在内存时，便产生一个缺页中断，请求操作系统把所缺页调入内存。此时缺页的进程会被阻塞，放入阻塞队列，调页完成后再被唤醒，放回就绪队列。

完整处理步骤：

1. 检查访问合法性：由地址变换机构检查逻辑地址是否越界（页号是否超过页表长度）。越界是非法访问，与缺页是两回事，不进入缺页处理。
2. 确认缺页并取外存地址：确认该页不在内存（状态位为 0），读出其外存地址；产生缺页中断，保存 **CPU** 现场，转入缺页中断处理程序。
3. 判定内存是否有空闲页框：有则直接分配一个空闲页框；无则按页面置换算法选出一个淘汰页——修改位为 **0** 的页可直接覆盖，修改位为 **1** 的页必须先把该页写回外存才能腾出页框。
4. 调入所缺之页：启动磁盘 I/O，把所缺页面读入刚分配的页框。
5. 修改页表：填入页框号，把状态位置 **1**，并置访问位、修改位等；同时更新快表（若题目说明缺页处理会更新 TLB）。
6. 唤醒进程：把进程从阻塞队列移入就绪队列（调页期间其他进程可运行）。
7. 重新执行发生缺页的那一条指令。

缺页中断的两个性质

1. 缺页中断属于内中断，是异常中的「故障（**fault**）」，因为它是由当前执行的指令想要访问的目标页面未调入内存而产生的。
2. 一条指令在执行期间可能发生多次缺页中断。若一条指令的多个操作数分属不同页面且都未调入内存，则每一页都要触发一次缺页中断。

> 注意第 7 步：缺页处理完成后是「重新执行该指令」，不是「直接访存」，这一点在做 EAT 计算题时至关重要。

算例（缺页处理中可能执行的操作）：在缺页处理过程中，操作系统可能执行的操作是（　）。Ⅰ. 修改页表　Ⅱ. 磁盘 I/O　Ⅲ. 分配页框答案Ⅰ、Ⅱ、Ⅲ全选。分配页框（第 3 步）、磁盘 I/O（第 4 步）、修改页表（第 5 步）都必然或可能发生。而【2022-29】问「不一定包含的操作」，答案是淘汰内存中的页——只有当内存无空闲页框时才需要淘汰，若还有空闲页框就无需淘汰。【2013-30】问缺页时可能执行的操作：Ⅰ处理越界错（错，越界与缺页是两回事）、Ⅱ置换页、Ⅲ分配内存 → 答案仅Ⅱ、Ⅲ。

*3.*请求分页的地址变换过程

与基本分页相比，请求分页的地址变换需要额外进行：

- 请求调页：查找到页表项时，要判断状态位，若为 0 则触发缺页；
- 页面置换：需要调入页面但没有空闲内存块时执行；
- 修改请求页表中新增的表项；
- 维护快表：快表中有的页面一定在内存中，若某页被换出外存，快表中的相应表项也要删除。

把 **TLB** 命中 / 缺失与页面在内存 / 缺页组合起来，共有四种情况，这是考纲「含缺页与 TLB 缺失的组合情况」所指：

情况	**TLB**	页面在内存	处理过程	访存次数（不含查 **TLB**）
①	命中	在	直接由 TLB 取出页框号 → 访存	**1**
②	缺失	在	查内存页表得页框号 → 把该页表项装入 **TLB** → 访存	**2**

情况	**TLB**	页面在内存	处理过程	访存次数（不含查 **TLB**）
③	缺失	不在	触发缺页中断 → 调页 → 修改页表（并删除/更新 TLB 中相关项）→ 重新执行该指令	缺页处理 + 重新执行的访存

TLB 中会不会有过期的表项

不会。因为「快表中有的页面一定在内存中」这一约束必须被维护：页面被换出外存时，必须同步删除快表中对应的表项。所以永远不会出现「**TLB** 命中但页面已不在内存」的情况，这也是上表只有三种情况的原因。

真题考法

【2011-28】缺页处理过程中操作系统执行的操作可能是：Ⅰ修改页表 Ⅱ磁盘 I/O Ⅲ分配页框 → 答案 **D**（Ⅰ、Ⅱ、Ⅲ）。

（三）页框分配与回收

*1.*驻留集与最少页框数

驻留集（**Resident Set**）：请求分页存储管理中分配给一个进程的页框的集合。

驻留集大小对系统影响很大，是一个两难：

- 驻留集越小：系统能容纳的并发进程越多、CPU 利用率越高；但每个进程的缺页率越高，缺页频繁会导致抖动，系统把大量时间花在缺页处理上，实际推进进程的时间很少。
- 驻留集越大：每个进程缺页少；但多道程序并发度下降，资源（如 CPU）利用率降低。

最小页框数（**Minimum Number of Frames**）：保证进程能够正常执行所需的最少物理页框数量。若分配数量不足，进程会因无法加载必要页面而无法运行（例如一条指令的取指和取数分属两页，只给一页就永远缺页）。

最小页框数由「最复杂的那条指令」决定

最小页框数由最复杂指令执行时的最大页面需求决定。若某条指令需同时访问 k 个页面，则最小页框数至少为 k。

- 单地址指令 + 直接寻址：至少 **2** 个页框（指令页 + 数据页）。例：`LOAD R1, 0x1000`。
- 单级间接寻址：至少 **3** 个页框（指令页 + 间接地址页 + 数据页）。例：`MOV AX, [BX]`。
- 多级间接寻址：每增加一级间接，页框数逐级增加（`MOV AX, [[BX]]` 更复杂）。
- 指令隐含的多页面访问：如 x86 的 `MOV [A], [B]` 若源、目标跨页，需 **2** 个数据页 + **1** 个指令页 = **3** 个页框。
- 跨页指令：如 `CALL 0x2000` 若指令本身跨越页边界（末尾在 `0x1000FFC`、目标在 `0x2000000`），需 **2** 个代码页。

几个典型架构的经验值：简单单地址指令 2 个页框；x86 复杂指令 3 个以上（代码 + 数据 + 堆栈页）；RISC-V 精简指令 2~3 个页框。

硬件不可绕过：指令格式、寻址方式、对齐规则等硬件特性直接约束最小页框数，软件无法通过优化绕过。注意「最小页框数」不等于「有效工作的页框数」：后者通常大于最小值，以减少缺页率、提升性能。

*2.*页框分配算法

在采用固定分配策略时，把系统可供分配的页框分配给各进程，可采用：平均分配算法（平均分，不考虑进程大小差异）、按比例分配算法（按进程大小按比例分配）、考虑优先级的分配算法（高优

先级进程分得更多页框）。

*3.*分配策略与置换策略的组合

- 按驻留集大小是否可变：
  - 固定分配：操作系统为每个进程分配一组固定数目的页框，进程运行期间不再改变。
  - 可变分配：先为每个进程分配一定数目的页框，在进程运行期间可以根据情况适当增加或减少。
- 按置换范围不同：
  - 局部置换：发生缺页时只能选本进程自己的页框进行置换。
  - 全局置换：可以把操作系统保留的空闲页框分配给进程；也可以把别的进程持有的页框置换到外存，再分配给缺页进程。

搭配起来只有三种合法策略：

组合	局部置换	全局置换
固定分配	可以	不可以
可变分配	可以	可以

为什么没有「固定分配 + 全局置换」

全局置换意味着一个进程拥有的页框数必定会改变——因为置换时可能选到了别的进程的页框，那个进程的页框数就减少了；也可能从系统空闲队列拿到新页框，本进程页框数就增加了。这与「固定分配」的定义直接矛盾，所以这种组合不存在。这是 2015 年真题的直接考点。

三种合法策略：固定分配局部置换（分配一定数量内存块且运行期间不变，缺页时只能从本进程页面中换出一页；缺点是难以事先确定驻留集大小，太少频繁缺页、太多降低并发度）；可变分配全局置换（系统保留一个空闲页框队列，缺页时取一块分配；无空闲页框时选一个未锁定的页面换出；优点是灵活，缺点是盲目给进程加页框会降低并发度，且被调出的页可能是任一进程的）；可变分配局部置换（缺页时只在本进程页框中换出，频繁缺页则多分配几个页框、缺页率很低则适当减少，既保证并发能力又不过多调页，但实现复杂、开销较大）。

*4.*页面调入策略

（1）什么时候调入页面

- 预调页策略：根据局部性原理，一次调入若干相邻页面比一次只调入一页更高效；但若调入的一批页面大多没被访问，效率就很低，因此需要以预测为基础，目前预测成功率平均不超过 **50%**。主要用于进程第一次调入内存时，或按工作集表在调度时把工作集内的页全部调入。
- 请求调页策略：进程运行期间发现缺页时才调入。调入的页面一定会被访问到，但一次只能调入一页，**I/O** 开销较大。

实际系统两种策略结合使用：首次装入用预调页批量调入，运行中主要用请求调页。

（2）从何处调入页面

具有对换功能的系统把磁盘空间划分为文件区（存放文件，追求空间利用率，采用离散分配）和对换区（存放被换出的进程数据，追求换入换出速度，采用连续分配），因此对换区的 **I/O** 速度远快于文件区。据此有三种情况：

- 对换区足够：页面的调入调出都在内存与对换区之间进行；运行前先把相关数据从文件区复制到对换区。
- 对换区不够：不会被修改的数据直接从文件区调入，换出时不必写回；可能被修改的部分换出时写回对换区，下次从对换区调入。
- **UNIX** 方式：数据全部放在文件区，未使用过的页面从文件区调入；被使用过的页面换出时写回对换区，下次直接从对换区调入。

（**3**）如何调入页面

访问的页面未在内存（存在位为 0）→ 触发缺页异常，关中断、保存断点、转入处理程序 → 查页表得外存地址 → 内存能容纳则启动磁盘 I/O 调入并修改页表；内存已满则先按置换算法选出淘汰页（修改位为 **1** 必须先写回磁盘）再调入，并置存在位为 **1** → 重新执行该指令。

*5.*抖动（*Thrashing*）与工作集模型

CPU 利用率与并发进程数的关系是相互促进又相互制约的：进程数少时提高并发度可提高 CPU 利用率；但并发进程增加会降低访存的局部性特征，局部性下降 → 缺页率上升 → **CPU** 利用率下降。

抖动的根本原因：同时在系统中运行的进程太多，导致分配给每个进程的页框数太少，不能满足进程正常运行的基本要求。于是进程频繁缺页，排队等待页面换入换出的进程增多，对磁盘的有效访问时间急剧增加，每个进程的大部分时间都用于页面的换入换出，从而处理机利用率急剧下降而趋于 **0**。

抖动的因果链（背成一条链）

进程太多 → 每进程页框太少 → 频繁缺页 → 页面换入换出排队 → 磁盘有效访问时间剧增 → **CPU** 利用率趋于 **0**

工作集（**Working Set**）模型：由于抖动与分配给进程的页框数（驻留集）有关，于是提出工作集概念。

- 定义：在某段时间间隔里，进程实际访问页面的集合。
- 基于局部性原理，可以用最近访问过的页面来确定工作集。工作集 W 由时间 t 和工作集窗口大小 Δ 确定，记作 $W(t,\Delta)$。
- 工作集大小可能小于窗口尺寸（窗口内可能有重复访问的页面）。
- 计算方法：从当前时刻 t 向前取 Δ 次页面访问序列，去掉重复后得到的集合就是工作集。
- 典型应用：操作系统统计进程的工作集大小，据此分配内存块。例如窗口尺寸为 5、监测发现某进程工作集最大为 3，说明它局部性良好，分配三个以上内存块即可。
- 核心结论：驻留集大小不能小于工作集大小，否则将导致频繁换页（抖动）。

工作集的方向：往「过去」看

工作集窗口是向时间前进的反方向（过去）取的窗口，不包含 t 时刻之后的访问。做真题时容易把箭头之后的页面也算进去。正确做法：只取 t 之前最近的 Δ 次访问，再去重。

抖动的预防方法：

1. 页框分配与置换时采取局部置换策略：进程缺页时只能在分配给它的内存空间内置换，不允许从其他进程内存空间获取新物理块。这样即使该进程抖动，也不会波及其他进程。
2. 作业调度时考虑工作集因素：从外存调入作业前，先检查每个进程在内存中的驻留页面是否足够。若都足够才调入新作业并分配页框；若某些进程页面不够，应优先给它们分配页框，暂时不调入新作业。

3. 主动挂起进程：当并发进程数目已影响 CPU 利用率时，主动调出优先级低（或剩余执行时间最多）的进程，将其换出到磁盘，腾出页框给其他进程。

真题考法

【2011-29】当系统发生抖动（thrashing）时，可以采取的有效措施是（ ）。Ⅰ撤销部分进程 Ⅱ增加磁盘交换区的容量 Ⅲ提高用户进程的优先级答案 **A**（仅Ⅰ）。抖动是页框不够，撤销/挂起进程能降低多道程序度、释放页框；增加交换区只是增加外存空间，不增加主存页框；提高优先级不能减少缺页。【2016-29】访问页序列，工作集窗口大小为 6，求 t 时刻的工作集。答案 **A**（$\{6,0,3,2\}$）：t 之前最近 6 次访问为 `6,0,3,2,3,2`，去重得 $\{6,0,3,2\}$。

*6.*页框回收与页面缓冲算法

页框回收：可变分配中「适当减少分配给某进程的页框」就是页框回收。被回收的页框若内容被修改过，必须先写回磁盘才能回收；一般来说回收的页框被放入空闲页框链尾，内容在下次分配前不会被清空——这正是页面缓冲算法的思想来源。

影响页面换入换出的因素：

1. 页面置换算法：最重要的因素，好的算法使缺页率低，从而减少换入换出次数。
2. 已修改页面写回磁盘的频率：若每换出一页就启动一次磁盘写回，开销很大。若建立已修改换出页面链表，把已修改的换出页面先挂在链表上，只有当链表上页面数目达到一定值（如 **64**）时再一起写回磁盘，就能显著减少磁盘 I/O 次数。
3. 磁盘内容读入内存的频率：有了上述链表后，链表上暂时有一批装有数据的页面。若某进程在这批数据写入磁盘前又要访问这些页面，就无须从磁盘调入，可直接从链表取回，只须花很小的开销就使这些页面重新回到该进程的驻留集中。

页面缓冲算法（**Page Buffering Algorithm**）：在原置换算法基础上增设已修改页面链表，保存已修改且要换出的页面，等数目达到一定值时一起换出磁盘。特点是显著降低页面换入换出的频率、大幅减少磁盘 **I/O** 次数；正因开销大幅减小，采用 **FIFO** 这类简单策略即可、不需要特殊硬件支持。

VAX/VMS 的页面缓冲算法采用可变分配局部置换，并在内存中设置两个链表：

- 空闲页面链表：实为空闲物理页框链表，用于分配给频繁缺页的进程。未被修改的页面要换出时，并不真的写到磁盘，而是把其页框挂在空闲链表末尾。注意：挂在空闲链表中的页面里是有数据的，若以后需要这些数据可直接取下，免除了从磁盘读入的操作。
- 修改页面链表：已修改页面要换出时不立即写回磁盘，而是把页框挂在修改页面链表末尾，以降低写回磁盘的频率，进而降低从磁盘读入的频率。

哪些页框可以被回收：不可回收页（空闲页、内核空间使用的页、临时锁定的页）；可交换页（匿名页或共享页，硬盘上无对应区域，需写入交换区 **swap**）；可同步页（如文件映射到内存的页，硬盘上有备份，只需检查是否被修改，改了才写回）。工程上常分文件页（内核缓存的磁盘数据与文件数据，干净页直接释放内存、脏页先写回磁盘再释放）与匿名页（无实际载体如堆栈，不能直接释放，通过 **Swap** 机制写盘后再释放）。二者的回收都基于 **LRU** 算法。

连续页框的回收——伙伴算法：Linux 用伙伴算法按大小分组记录连续空闲页框。回收时先检查是否存在大小相等的伙伴页框块，若有则合并成大小为原来两倍的新空闲页框块，合并后继续检查能否再合并成更大块。

真题考法

【2022-30】下列选项中，不会影响系统缺页率的是（ ）。A. 页面置换算法 B. 工作集的大小 C. 进程的数量 D. 页缓冲队列的长度答案 **D**。页缓冲队列用于减少已修改页写回磁盘的频率，主要影响缺页

处理的时间与 **I/O** 性能，不直接决定页面访问是否缺页；而置换算法、工作集大小、进程数量都会影响内存中驻留页面的集合，从而影响缺页率。

（四）页置换算法

进程运行时，当所要访问的页面不在内存、需要调入，而内存已无空闲空间时，系统必须从内存中选出一页调出到磁盘对换区。选择换出页面的算法称为页面置换算法。

- 页置换算法的好坏直接影响系统性能。不适当的算法可能导致进程抖动：刚被换出的页很快又要被访问，于是又要调入；刚调入又需调出，如此频繁更换页面，以致进程大部分时间都花在页面置换工作上。
- 一个好的页面置换算法应具有较低的页面置换频率。理论上应把以后不再会访问的页面、或在较长时间内不会再访问的页面换出。

页置换的三条基本认识

1. 置换本质上不改变页框号，只改变页号。因为置换是把同一个页框的内容覆盖掉，而不是另找一个页框。
2. 页面的换入换出需要磁盘 **I/O**，开销很大，因此好的算法要追求更少的缺页率。
3. 缺页时不一定发生页面置换！若还有可用的空闲内存块，直接装入即可，不需要置换。
4. 在 408 范围内，页面置换算法通常指局部置换的算法。

*1.*统一算例的引用串

为了便于横向比较，本章所有置换算法都用同一个引用串：

$$\text{引用串} = 7,\ 0,\ 1,\ 2,\ 0,\ 3,\ 0,\ 4,\ 2,\ 3,\ 0,\ 3,\ 2 \qquad (\text{共}13\ \text{次访问})$$

分别为进程分配 **3** 个页框和 **4** 个页框两种情况。下面逐一给出模拟表格。

*2.*最佳置换算法（*OPT*，*Optimal*）

算法思想：每次淘汰的页面将是以后永不使用的、或者是在最长时间内不再被访问的页面。这样可以保证最低的缺页率。

手工模拟方法：逐列扫描引用串，遇到缺页且无空闲页框时，对当前在内存中的每个页面，往后找它下一次出现的位置；下一次出现位置最靠后（或以后不再出现）的那个页面就是被淘汰者。若某个页面以后永不出现，优先淘汰它。

3 个页框的完整模拟：

访问次序	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**	**13**
访问页	7	0	1	2	0	3	0	4	2	3	0	3	2
页框 1	7	7	7	**2**	2	**2**	2	**2**	2	2	**2**	2	2
页框 2		0	0	0	0	0	0	**4**	4	4	**0**	0	0
页框 3			1	1	1	**3**	3	3	3	3	3	3	3
淘汰页	—	—	—	**7**	—	**1**	—	**0**	—	—	**4**	—	—
是否缺页	✓	✓	✓	✓	×	✓	×	✓	×	×	✓	×	×

淘汰理由（对内存中每页往后找下一次出现位置，最远者被淘汰）：

- 第 4 次访问 2：页 7 不再出现、页 0 下次第 5 位、页 1 下次第 7 位 → 淘汰 **7**。
- 第 6 次访问 3：页 0 下次第 7 位、页 1 不再出现、页 2 下次第 9 位 → 淘汰 **1**。
- 第 8 次访问 4：页 2 下次第 9 位、页 0 下次第 11 位、页 3 下次第 10 位 → 淘汰 **0**（最远）。
- 第 11 次访问 0：页 3、页 4 都不再出现，淘汰 **4**（若淘汰 3，第 12 次访问 3 又要缺页）。

缺页次数 = **7**，缺页率 $= 7/13 \approx 53.8\%$。

4 个页框的完整模拟：

访问次序	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**	**13**
访问页	7	0	1	2	0	3	0	4	2	3	0	3	2
页框 1	7	7	7	7	7	**3**	3	**3**	3	3	3	3	3
页框 2		0	0	0	0	0	0	**4**	4	4	4	4	4
页框 3			1	1	1	1	1	1	**2**	2	2	2	2
页框 4				2	2	2	2	2	2	2	2	2	2
淘汰页	—	—	—	—	—	**7**	—	**1**	—	—	—	—	—
是否缺页	✓	✓	✓	✓	×	✓	×	✓	×	×	×	×	×

第 6 次访问 3 时：页 7 不再出现、页 0 下次第 7 位、页 1 下次第 ∞ 位、页 2 下次第 9 位 → 淘汰 **7**。
第 8 次访问 4 时：页 1 不再出现 → 淘汰 **1**。

缺页次数 = **6**，缺页率 $= 6/13 \approx 46.2\%$。

OPT 是不可实现的

OPT 可以保证最低的缺页率，但实际上只有进程执行过程中才知道接下来会访问哪个页面，操作系统无法提前预判页面访问序列。因此最佳置换算法是无法实现的，它的价值在于作为衡量其他算法优劣的理论基准（下界）。

*3.*先进先出算法（*FIFO*）

算法思想：每次淘汰最早进入内存的页面。

实现方法：把调入内存的页面按调入先后排成一个队列（链表也可），需要换出时选择队头。队列最大长度等于系统为进程分配的内存块数。

3 个页框的完整模拟：

访问次序	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**	**13**
访问页	7	0	1	2	0	3	0	4	2	3	0	3	2
页框（队尾 → 队首）	7	7,0	7,0,1	0,1,2	0,1,2	1,2,3	2,3,0	3,0,4	0,4,2	4,2,3	2,3,0	2,3,0	2,3,0
淘汰页	—	—	—	**7**	—	**0**	**1**	**2**	**3**	**0**	**4**	—	—
是否缺页	✓	✓	✓	✓	×	✓	✓	✓	✓	✓	✓	×	×

缺页次数 = **10**，缺页率 $= 10/13 \approx 76.9\%$。这是三个基础算法里最差的。

4 个页框的完整模拟：

访问次序	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**	**13**
访问页	7	0	1	2	0	3	0	4	2	3	0	3	2
页框	7	7,0	7,0,1	7,0,1,2	7,0,1,2	0,1,2,3	0,1,2,3	1,2,3,4	1,2,3,4	1,2,3,4	2,3,4,0	2,3,4,0	2,3,4,0
淘汰页	—	—	—	—	—	**7**	—	**0**	—	—	**1**	—	—
是否缺页	✓	✓	✓	✓	×	✓	×	✓	×	×	✓	×	×

缺页次数 = **7**。

FIFO 的优点是实现简单；缺点是与进程实际运行的规律不相适应——先进入内存的页面也可能是需要经常访问的（例如循环中最常用的数组首元素），故性能差。

4.Belady 异常

Belady 异常的定义

Belady 异常（**Belady's Anomaly**）：对某些页面置换算法，随着分配给进程的页框数（驻留集）增大，缺页次数不减反增的现象。

会发生 **Belady** 异常的算法：**FIFO**、简单 **CLOCK**、改进型 **CLOCK**。不会发生 **Belady** 异常的算法：**LRU**、**OPT**。（这类算法属于「栈算法 / stack algorithm」，具有包含性质 $S_n(t) \subseteq S_{n+1}(t)$，因此不会出现异常。）

Belady 异常的经典实例：引用串

$$1,\ 2,\ 3,\ 4,\ 1,\ 2,\ 5,\ 1,\ 2,\ 3,\ 4,\ 5 \qquad (\text{共12 次访问})$$

分配 **3** 个页框：

访问次序	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**
访问页	1	2	3	4	1	2	5	1	2	3	4	5
页框（新 → 旧）	1	1,2	1,2,3	2,3,4	3,4,1	4,1,2	1,2,5	1,2,5	1,2,5	2,5,3	5,3,4	5,3,4
淘汰页	—	—	—	**1**	**2**	**3**	**4**	—	—	**1**	**2**	—
是否缺页	✓	✓	✓	✓	✓	✓	✓	×	×	✓	✓	×

缺页次数 = **9**。

分配 **4** 个页框：

访问次序	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**
访问页	1	2	3	4	1	2	5	1	2	3	4	5

访问次序	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**
页框（新 → 旧）	1	1,2	1,2,3	1,2,3,4	1,2,3,4	1,2,3,4	2,3,4,5	3,4,5,1	4,5,1,2	5,1,2,3	1,2,3,4	2,3,4,5
淘汰页	—	—	—	—	—	—	**1**	**2**	**3**	**4**	**5**	**1**
是否缺页	✓	✓	✓	✓	×	×	✓	✓	✓	✓	✓	✓

缺页次数 = **10**。

Belady 异常的验证

同一个引用串，**3** 个页框缺页 **9** 次，**4** 个页框缺页 **10** 次。页框多了，缺页反而多了 1 次——这就是 Belady 异常。

原因：FIFO 选择替换「最早进入内存」的页面，完全没有考虑这个页面未来是否可能被经常访问。在上述例子中，第 1 页虽然是最早进入的，但在序列后段还要被反复访问；FIFO 却因为「它最老」把它换出，结果很快又要把它调回来，白白多了一次缺页。当页框数从 3 变为 4 时，页面的「入驻/淘汰节奏」整体错位，恰好把这种盲目性放大成了缺页次数的净增。

真题考法

【2014-30】可能出现 Belady 异常现象的是：Ⅰ LRU　Ⅱ FIFO　Ⅲ OPT。答案 **A**（仅Ⅱ）。

*5.*最近最久未使用置换算法（*LRU*，*Least Recently Used*）

算法思想：每次淘汰最近最久未使用的页面。

实现方法：在每个页面的页表项中用访问字段记录该页面自上次被访问以来所经历的时间 t，淘汰时选 t 值最大的页面。软件上内核维护一个双向页面链表，表头是最近刚使用过的页面，表尾是最近最久没有使用的页面：每次访存把被访问页移到表头，缺页时淘汰表尾页面、新页挂到表头；由于每次访存都要修改链表，故开销很大。硬件上可用移位寄存器记录使用情况，或给页面打时间戳（淘汰时选时间戳最大者）。

手工模拟技巧：需要淘汰页面时逆向检查访问序列——若分配了 n 个内存块，则往前找第 n 个不同的、且在内存中的页号，它就是最近最久未使用的页面。

3 个页框的完整模拟（列内按「最近使用 → 最久未使用」排列）：

访问次序	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**	**13**
访问页	7	0	1	2	0	3	0	4	2	3	0	3	2
最近使用（前）	7	0,7	1,0,7	2,1,0	0,2,1	3,0,2	0,3,2	4,0,3	2,4,0	3,2,4	0,3,2	3,0,2	2,3,0
最久未使用（末）				7	1	2	2	3	0	4	2	2	0
淘汰页	—	—	—	**7**	—	**1**	—	**2**	**0**	**4**	**2**	—	—
是否缺页	✓	✓	✓	✓	×	✓	×	✓	✓	✓	✓	×	×

淘汰理由逐条说明：

- 第 4 次访问 2 缺页，内存中为 $\{7,0,1\}$，按最近使用排序 $1,0,7$，末位 **7** 最久未使用，淘汰 **7**。
- 第 6 次访问 3 缺页，内存状态 $0,2,1$（最近使用序），末位 **1** 最久未使用，淘汰 **1**。
- 第 8 次访问 4 缺页，内存状态 $0,3,2$，末位 **2** 最久未使用，淘汰 **2**。
- 第 9 次访问 2 缺页（刚被淘汰），此时最近使用序为 $4,0,3$，末位 **3** 最久未使用，淘汰 **3**。

缺页次数 = **9**，缺页率 $= 9/13 \approx 69.2\%$。

4 个页框的完整模拟：

访问次序	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**	**13**
访问页	7	0	1	2	0	3	0	4	2	3	0	3	2
最近使用（前）	7	0,7	1,0,7	2,1,0,7	0,2,1,7	3,0,2,1	0,3,2,1	4,0,3,2	2,4,0,3	3,2,4,0	0,3,2,4	3,0,2,4	2,3,0,4
淘汰页	—	—	—	—	—	**7**	—	**1**	**3**	**0**	**4**	—	—
是否缺页	✓	✓	✓	✓	×	✓	×	✓	✓	✓	✓	×	×

缺页次数 = **6**。

LRU 与 OPT 的比较

LRU 用「过去最久未使用」近似「未来最久不使用」（基于时间局部性），因此性能很好，缺页次数单独看与 **OPT** 很接近（本例 3 页框时 LRU 9 次、OPT 7 次；4 页框时两者都是 6 次）。但 **LRU** 实现困难、开销大，实际系统多采用它的近似算法（CLOCK）。

*6.*时钟置换算法（*CLOCK / NRU*）

由于 LRU 实现开销太大，实际系统中大多采用 LRU 的近似算法，**CLOCK** 置换算法就是其中最常用的一种。

简单时钟置换算法（**NRU**，**Not Recently Used**）：

- 算法思想：缺页时从指针处顺序查找未被访问的页面进行置换。它不追求严格的「最近最久未使用」，可理解为淘汰最近较久未访问的页面，实现简单、开销小。
- 实现方法：为每个页面设一个访问位，把内存中的页面链成一个循环队列，指针指向最先调入的页面；页面装入时访问位初始化为 **0**，被访问（读或写）时置 **1**。缺页需要淘汰时，从指针当前位置顺序检查：
  - 访问位为 **0** → 最近没被访问过，选该页换出；
  - 访问位为 **1** → 把它清 **0**，指针移到下一页继续检查。若第一轮扫描中所有页面的访问位都是 1，则已把它们全部清零，第二轮扫描中一定存在访问位为 **0** 的页面。

简单 CLOCK 的扫描轮数

简单 CLOCK 选择一个淘汰页面最多经过两轮扫描。口诀：访问位有 **0** 就选第一个 **0**；全为 **1** 就先全部清 **0**，再选第一个。

3 个页框的完整模拟（记页号（访问位））：

访问次序	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**	**13**
访问页	7	0	1	2	0	3	0	4	2	3	0	3	2
帧 1	7(1)	7(1)	7(1)	**2(1)**	2(1)	2(1)	2(1)	**4(1)**	4(1)	4(1)	**0(1)**	0(1)	0(1)
帧 2		0(1)	0(1)	0(0)	0(1)	0(0)	0(1)	0(0)	2(1)	2(1)	2(0)	2(0)	2(1)
帧 3			1(1)	1(0)	1(0)	**3(1)**	3(1)	3(0)	3(0)	3(1)	3(1)	**3(1)**	3(1)
淘汰页	—	—	—	**7**	—	**1**	—	**2**	**0**	—	**4**	**3**	—
是否缺页	✓	✓	✓	✓	×	✓	×	✓	✓	×	✓	✓	×

关键步骤：

- 第 **4** 次访问 2 缺页：帧 1 的 7 为 1 → 清 0、后移；帧 2 的 0 为 1 → 清 0、后移；帧 3 的 1 为 1 → 清 0、指针回到帧 1（进入第二轮）。帧 1 的 7 已是 0 → 淘汰 **7**，装入 2。共两轮。
- 第 **6** 次访问 3 缺页：第 5 次访问过 0，故 0(1)、1(0)、2(1)。从指针所在的帧 2 起：0 为 1 → 清 0、后移；帧 3 的 1 为 0 → 淘汰 **1**，装入 3。
- 第 **8** 次访问 4 缺页：第 7 次访问过 0（0 变 1），从指针处逐位扫描，淘汰 **2**，装入 4。
- 第 **9**、**11**、**12** 次分别缺页，依次淘汰 **0**、**4**、**3**。

缺页次数 = **9**。**4** 个页框时缺页次数为 **6**（第 6、8 次分别淘汰 7、1，其余命中，与 LRU 的 4 页框结果一致）。

手算 CLOCK 的两条纪律

1. 访问位只在「访问命中」时置 **1**；扫描时访问位为 0 的页面直接淘汰，不置 **1**。常见错误是把「被扫描过的页面」也置 1。
2. 指针位置必须记准：淘汰后指针指向被淘汰页框的下一个页框；不发生缺页的命中时指针不动。每次操作都显式写出指针位置，否则后面全错。

*7.*改进型时钟置换算法

为什么需要改进：简单 CLOCK 只考虑页面是否被访问过。但访问不一定修改，而页面一旦被修改就必须写回外存。因此淘汰时应优先选没有被修改过的页面，从而省下一次磁盘写回。为此再设置一个修改位 M：$M=0$ 表示没被修改过，$M=1$ 表示已被修改过。用二元组 (A, M) 表示「访问位，修改位」，进行至多四轮扫描：

轮次	扫描目标	是否修改标志位
第一轮	找第一个 $(0,0)$ ——最近未被访问、且未被修改	本轮不修改任何标志位
第二轮	若第一轮失败，重新扫描，找第一个 $(0,1)$	本轮把所有扫描过的帧的访问位设为 **0**
第三轮	若第二轮失败，重新扫描，找第一个 $(0,0)$	本轮不修改标志位
第四轮	若第三轮失败，重新扫描，找第一个 $(0,1)$	——

改进型 CLOCK 的三条结论

1. 由于第二轮已把所有帧的访问位清零，第三、四轮扫描一定会有帧被选中，因此至多四轮扫描。
2. 该算法只修改访问位，从不修改修改位。

3. 本质是把页面分成四个优先级，淘汰优先级为 $(0,0)>(0,1)>(1,0)>(1,1)$，即先看是否被访问过，再看是否被修改过。

（**A, M**）四类页面：$(0,0)$ 既不会马上被用、也不用写回，淘汰代价最小，排第 1 位；$(0,1)$ 不会马上被用但要写回，排第 2 位；$(1,0)$ 可能马上要用但不用写回，排第 3 位；$(1,1)$ 既要被用又要写回，排最后。

手工模拟示例：设循环队列 4 帧，指针指向帧 1，各帧状态为 A$(1,0)$、B$(1,1)$、C$(1,0)$、D$(1,1)$。发生缺页后：

- 第一轮：找 $(0,0)$。四帧中没有 $(0,0)$，失败，本轮不改任何标志位。
- 第二轮：重新扫描找 $(0,1)$，同时把扫描过的帧的访问位清零。帧 1 清成 $(0,0)$、后移；帧 2 清成 $(0,1)$，命中，淘汰 **B**。只有帧 1、2 被扫描，帧 3、4 保持不变。共用两轮。

推演所有 $M=0$ 的情形：若 4 帧状态全为 $(1,0)$，第一轮找 $(0,0)$ 失败；第二轮找 $(0,1)$ 时四帧被依次清成 $(0,0)$，无一满足 $(0,1)$，第二轮失败；第三轮重新扫描找 $(0,0)$，帧 1 已是 $(0,0)\rightarrow$ 选中帧 1。共三轮。

真题考法

【2016-26】改进型CLOCK中 A 为访问位、M 为修改位，淘汰次序为(　)。答案 **A.** $(0,0)$, $(0,1)$, $(1,0)$, $(1,1)$。算法展开后的等价优先级正是 $(0,0)>(0,1)>(1,0)>(1,1)$。选项 B（$(0,0),(1,0),(0,1),(1,1)$）是「先看修改位」的错解。

*8.*同一引用串下各算法缺页次数对照

引用串 $7,0,1,2,0,3,0,4,2,3,0,3,2$（13 次访问）：

算法	**3** 个页框时缺页次数	缺页率	**4** 个页框时缺页次数	缺页率	是否可能 **Belady** 异常
OPT	**7**	53.8%	**6**	46.2%	否
FIFO	**10**	76.9%	**7**	53.8%	是
LRU	**9**	69.2%	**6**	46.2%	否
CLOCK（简单）	**9**	69.2%	**6**	46.2%	是
改进型 **CLOCK**	依赖 (A,M) 初值	—	依赖 (A,M) 初值	—	是

结论速记：

- 缺页次数排序（3 页框）：OPT(7) < LRU = CLOCK(9) < FIFO(10)。
- **OPT** 一定最优，但不可实现；**LRU** 性能接近 **OPT**，但开销大；**CLOCK** 是 **LRU** 的廉价近似，性能介于 LRU 与 FIFO 之间；**FIFO** 最简单也最差。
- **4** 个页框下 **LRU** 与 **OPT** 都是 **6** 次，说明 LRU 在页框较多时逼近最优。
- **Belady** 异常只发生在 **FIFO / CLOCK** 类算法上；OPT 与 LRU 由于满足栈性质，绝不会有 Belady 异常。

（五）内存映射文件

内存映射文件（**Memory-Mapped Files**，简称 `mmap`）是一种把磁盘上的文件直接映射到进程虚拟地址空间的机制，使得进程可以像访问内存一样读写文件，从而避免频繁的系统调用（如 `read`/`write`）

和数据拷贝。

内存映射文件与虚拟内存的关键区别

	建立映射的两端
虚拟内存管理	物理内存 ↔ 进程虚拟地址空间
内存映射文件 `mmap()`	磁盘上的文件 ↔ 进程虚拟地址空间

用一句话说：虚拟内存把「物理内存」变成「进程眼里的连续地址空间」；内存映射文件把「磁盘文件」变成「进程眼里的连续地址空间」。

工作原理（完整流程）：

1. 用 `open()` 打开文件得到文件描述符 **fd**，再用 `mmap(addr, length, fd, ...)` 请求映射；内核在进程虚拟地址空间中分配一段连续区域用于映射该文件。
2. 此时并没有立即把文件数据加载到内存（这是理解 mmap 高效的关键）。
3. 按需加载：进程首次访问该区域的某个页面时发现存在位为 **0**，触发缺页中断；内核找到该页对应的文件数据，分配一个物理页框装入并更新页表——此时才真正建立虚拟内存到物理内存的映射。
4. 该页框中的数据即文件在内存中的副本（文件缓存页）。后续读写直接作用于内存中的该页；修改会标记脏位，内核在适当时机（内存不足要换出该页、或关闭文件时）把修改写回磁盘。

多个进程可以并发映射同一文件以实现共享：任一进程的写入其他映射了同一部分的进程都能看到，因为各进程的虚拟内存映射指向物理内存的同一页面，而该页面是磁盘块的副本。系统调用还支持写时复制（**Copy-On-Write**），允许进程既按只读共享，也拥有自己修改的副本。

关闭文件 $\neq$ 解除映射

在多个进程共享某一个磁盘文件时，某个进程关闭文件并不会触发文件同步并且解除映射，因为文件描述符有引用计数，此时只会把引用计数减 1。要解除映射一般需要调用 `munmap(addr, length)`。

内存映射文件的优点：减少数据拷贝（传统 `read/write` 需在内核缓冲区与用户缓冲区之间拷贝，而 `mmap` 直接操作页缓存，实现零拷贝）；方便程序员访问磁盘文件（可像访问内存一样读写，尤其适合频繁随机访问大文件，无需多次 `lseek`）；读入读出全部由内核实现（缺页异常服务程序读入数据、内核负责把脏数据写回，不需用户调用 `read()/write()`）；方便多进程共享同一文件（共享同一份页缓存，也能实现进程间通信）；按需加载、节约物理内存；显著提升 **I/O** 密集型进程的效率。

为什么 **`read/write`** 需要两次拷贝，而 **`mmap`** 可以直接写入用户空间？对 `read/write`，内核先把磁盘块经 DMA 读入内核缓冲区，再把用户需要的指定字长的数据复制到用户 **buf**——必须复制是因为用户没有权限读取一整个磁盘块（整块中可能有敏感数据）；而对缺页处理，与磁盘交换的单位是页面，DMA 预处理时直接把分配给该页的页框物理地址写入 DMA 控制器，由 DMA 直接把页面加载到该页框（用户地址空间），因为读入的整个页面完全属于该进程，不存在读到敏感数据的问题。

页面与磁盘块没有实质关系

操作系统以「页」为单位交换内存与磁盘，而文件系统以「块」为单位管理磁盘存储。若一个页面是磁盘块的 n 倍，一次缺页可能需读多个块；页面与块对齐只是为了方便，二者大小并没有实质关系。

注意区分内存映射 **I/O**（I/O 章节的「统一编址」，把一组内存地址映射到设备寄存器）与内存映射

文件（映射磁盘文件），二者不是一回事。

（六）虚拟存储器性能的影响因素及改进方法

*1.*缺页率及其影响因素

假设一个进程的逻辑空间为 n 页，系统为其分配的内存物理页框数为 m（$m < n$）。在进程运行过程中，访问页面成功（所访问页面在内存中）的次数为 S，访问页面失败（页面不在内存，需要从外存调入）的次数为 F，则总的页面访问次数为

$$A = S + F$$

该进程在其运行过程中的缺页率为

$$f = \frac{F}{A}$$

缺页率（缺页率高即为抖动）是影响虚拟存储器性能的主要因素。它受以下四个因素影响：

1. 页面大小：由局部性原理，页面较大则缺页率较低，页面较小则缺页率较高。页面小时内存碎片少、利用率高，但每个进程要求的页面数多，页表过长、占用大量内存；页面大时页表短，但页内碎片增大。
2. 进程所分配物理页框的数目：页框越多缺页率越低；但超过某个数目后再增加页框对缺页率的改善并不明显，此时只需保证活跃页面在内存中、缺页率维持在很低范围即可。
3. 页面置换算法：算法优劣决定缺页中断次数，缺页率是衡量置换算法的重要指标。
4. 程序固有特性：程序编制的局部化程度越高，缺页程度越低（如按行优先访问行优先存储的二维数组）。

此外，在缺页中断处理时，当由于空间不足而需要置换部分页面到外存时，选择被置换页面还需要考虑置换的代价（例如页面是否被修改过）。没有修改过的页面可以直接放弃，已被修改过的页面则必须先保存，因此两种情况耗时不同。假设被置换页面被修改的概率是 β，被置换页面被修改时的缺页中断处理时间是 t_a，没有被修改时的缺页中断处理时间是 t_b，则缺页中断处理时间为

$$t = \beta \times t_a + (1 - \beta) \times t_b$$

真题考法

【2022-30】不会影响系统缺页率的是：**D.** 页缓冲队列的长度（见前文）。

*2.*有效访问时间（*EAT*）

优先掌握考纲标准公式（不含 **TLB**）。设内存访问时间为 t，缺页处理时间为 $t_{\text{page fault}}$，缺页率为 p，则

$$\boxed{EAT = (1 - p) \times t + p \times t_{\text{page fault}}}$$

每个符号的含义与适用范围：

符号	含义	说明
p	缺页率	$0 \le p \le 1$；一般 p 很小（$10^{-6} \sim 10^{-3}$ 量级）
$1-p$	页面在内存中的概率	此时只需一次访存，时间 $= t$
t	一次内存访问时间	单位通常为 ns
$t_{\text{page fault}}$	缺页处理时间	包括：查页表发现缺失、缺页中断响应与现场保护、选页置换、被修改页写回磁盘、从外存读入所缺页、修改页表、恢复现场并重新执行该指令。主要是磁盘 **I/O** 时间，单位通常为 μs 或 ms

使用前提：题目不考虑快表（**TLB**），即命中时按一次访存计算；且缺页处理时间按题目给出的「缺页中断处理时间 / 缺页服务时间」整体取值。若题目给出的缺页相关时间是磁盘访问时间等分项，需先累加成完整的缺页处理时间。

EAT 公式的两种变形

1. 命中时一次访存（最常见）：

$$EAT = (1-p) \times t + p \times t_{\text{页缺失}}$$

1. 命中时两次访存（无快表的基本分页，每次访问都要先查页表）：

$$EAT = (1-p) \times 2t + p \times t_{\text{页缺失}}$$

1. 有快表时（设查快表时间 λ、命中率 α）：见下文的分情况公式。

做题纪律：先看题目说「访问内存一次」还是「查页表 + 访存」，再决定命中项是 t 还是 $2t$。

算例 **1**（考纲标准形式）：某请求分页系统，内存访问时间 $t = 100$ ns，缺页处理时间 $t_{\text{page fault}} = 8$ ms，缺页率 $p = 10^{-3}$，求有效访问时间 EAT。

统一单位（ns）：$8\ \text{ms} = 8 \times 10^6$ ns。

$$EAT = (1-10^{-3}) \times 100 + 10^{-3} \times 8 \times 10^6 = 0.999 \times 100 + 8000 = 99.9 + 8000 = 8099.9\ \text{ns} \approx 8.10\ \mu\text{s}$$

解读：p 只有千分之一，但 EAT 仍然被抬高到近 **8.1** μ**s**，是正常访存 100 ns 的约 **81** 倍。这说明缺页处理时间比内存访问时间大好几个数量级，因此缺页率对性能的影响是压倒性的。若把 p 降到 10^{-4}：

$$EAT = 0.9999 \times 100 + 10^{-4} \times 8 \times 10^6 = 99.99 + 800 = 899.99\ \text{ns} \approx 0.90\ \mu\text{s}$$

缺页率降低 10 倍，EAT 降低约 9 倍——这就是为什么所有改进手段的着力点都是「降低缺页率」。

算例 **2**（含快表与缺页的完整分情况推导）

设访问内存时间 t，查询 TLB 时间 λ，处理缺页异常时间 ε，TLB 命中率为 α，缺页率为 f。考虑一级页表，且 CPU 处理速度远高于访存速度，因此分析逻辑地址、获取页号与页内偏移量等 CPU 处理时间忽略不计。

情况一：被访问页在内存中，且其对应的页表项在 **TLB** 里（命中）

$$EAT_1 = \lambda + t$$

情况二：被访问页在内存中，但不在 **TLB** 里（**TLB** 缺失但页面存在）

流程：查找 TLB（未命中）→ 访问内存查页表 → 更新 TLB → 用物理地址访存。

$$EAT_2 = \lambda + t + \lambda + t = 2(\lambda + t)$$

（注：查找 TLB 与查页表在实际硬件中可以并行进行，一切以题目要求为准。）

情况三：被访问页不在内存（缺页）

1. 访问 TLB 未命中，用时 λ；
2. 访问页表发现页面缺失，用时 t，触发缺页；
3. 缺页异常处理耗时 ε；
4. 重新执行发生缺页的那条指令，再次访问 TLB 未命中，用时 λ；
5. 页表命中，用时 t；
6. 修改 TLB，用时 λ；
7. 用获得的物理地址访存，用时 t。

$$EAT_3 = 3\lambda + \varepsilon + 3t$$

加权平均：

$$EAT = \alpha\, EAT_1 + (1-\alpha)(1-f)\, EAT_2 + (1-\alpha) f\, EAT_3$$

即

$$EAT = \alpha(\lambda + t) + (1-\alpha)(1-f)\cdot 2(\lambda + t) + (1-\alpha) f \cdot (3\lambda + \varepsilon + 3t)$$

化简得

$$EAT = \big[(2+f) - \alpha(1+f)\big]\lambda + \big[(2-\alpha) + f(1-\alpha)\big]t + f(1-\alpha)\varepsilon$$

验算：取 $\alpha = 1$（TLB 全部命中），则

$$EAT = [(2+f) - (1+f)]\lambda + [(2-1) + 0]t + 0 = \lambda + t = EAT_1$$

公式自洽。再取 $\alpha = 0,\ f = 0$（无 TLB、无缺页）：

$$EAT = (2 + 0 - 0)\lambda + (2-0)t = 2\lambda + 2t = 2(\lambda + t) = EAT_2$$

也自洽。

数值算例：$t = 100$ ns，$\lambda = 10$ ns，$\varepsilon = 8$ ms $= 8 \times 10^6$ ns，$\alpha = 0.9$，$f = 10^{-3}$。

$$EAT = [(2 + 0.001) - 0.9 \times 1.001] \times 10 + [(2 - 0.9) + 0.001 \times 0.1] \times 100 + 0.001 \times 0.1 \times 8 \times 10^6$$

逐项计算：

- TLB 项：$2.001 - 0.9009 = 1.1001$，乘 10 得 11.001 ns；

- 访存项：$1.1 + 0.0001 = 1.1001$，乘 100 得 110.01 ns；
- 缺页项：$0.001 \times 0.1 \times 8 \times 10^6 = 800$ ns。

$$EAT = 11.001 + 110.01 + 800 = 921.011 \text{ ns} \approx 0.92\ \mu\text{s}$$

解读：即使用了 90% 命中率的快表，EAT 仍然主要由缺页项（800 ns）主导。这与算例 1 的结论一致。

做 EAT 题的两个「看题目」

1. 缺页异常处理是只修改页表，还是同时修改页表和 **TLB**？若题目说明同时修改 TLB，则情况三中第 4、6 步的 λ 要相应调整（第 4 步不再需要，或第 6 步并入 ε）。
2. 缺页处理把页面调入内存后，是「重新执行发生缺页的那条指令」，还是「直接拿物理地址去访存」？前者是标准情形（需要再走一遍 TLB 与页表）；若题目说直接访存，则 EAT_3 要减掉第 4、5、6 步的开销。

一切根据题目条件计算，不要死套公式。

*3.*改进方法

虚拟存储器性能的改进围绕降低缺页率与降低缺页处理代价两条主线：

1. 降低缺页率：合理确定驻留集（使驻留集不小于工作集，用可变分配局部置换动态调整）；选用较优的置换算法（优先用 LRU 的近似实现，避免退化为纯 FIFO）；合理选择页面大小（过大加剧页内碎片与单次 I/O 量，过小使页表膨胀、缺页增多）；提高程序的局部性（如按行优先访问行优先存储的数组，减少跨页访问）；控制并发进程数（作业调度时考虑工作集，必要时主动挂起低优先级进程）。
2. 降低缺页处理代价：采用页面缓冲算法（空闲页面链表 + 修改页面链表，批量写回已修改页，被换出页可直接从链表取回）；增大 **TLB** 容量、让页表常驻内存（只加快地址变换，不降低缺页率）；提高磁盘 **I/O** 速度、对换区采用连续分配；预调页策略（首次调入或按工作集批量预取）。
3. 缓解抖动：采用局部置换把抖动限制在进程内部；作业调度时考虑工作集，驻留页面不足则不调入新作业；主动挂起低优先级（或剩余执行时间最多）的进程以腾出页框。

真题考法

【2020-28】影响请求分页系统有效（平均）访存时间的是：Ⅰ缺页率　Ⅱ磁盘读写时间　Ⅲ内存访问时间　Ⅳ执行缺页处理程序的 CPU 时间。答案 **D**（Ⅰ、Ⅱ、Ⅲ、Ⅳ）。因为 $EAT = (1-p)t + p\,t_{\text{page fault}}$ 中，p 对应Ⅰ，t 对应Ⅲ，$t_{\text{page fault}}$ 中包含磁盘读写时间（Ⅱ）与缺页处理程序的 CPU 时间（Ⅳ）。

本章小结表

（七）三种非连续分配方式总对比

对比项	页式管理	段式管理	段页式管理
分配单位	页面（固定大小）	段（可变大小）	先分段、段内再分页
逻辑地址结构	页号 + 页内偏移	段号 + 段内偏移	段号 + 页号 + 页内偏移
地址空间维度	一维	二维	二维
对用户是否可见	不可见	可见	段可见、页不可见

对比项	页式管理	段式管理	段页式管理
数据结构	页表（每进程 1 张）	段表（每进程 1 张）+ 共享段表	段表（每进程 1 张）+ 页表（每段 1 张）
表的表项内容	页号、页框号 + 控制位	段号、段长、段基址 + 控制位	段表项：页表长度、页表块号；页表项：页框号
越界检查次数	1 次（页号 vs 页表长度）	2 次（段号 vs 段表长度；段内偏移 vs 段长）	2 次（段号 vs 段表长度；页号 vs 页表长度）
碎片	页内碎片（内部）	外部碎片	页内碎片（内部）
共享与保护	实现复杂（需 N 个页表项）	实现简单（只需 1 个段表项）	通过共享段指向相同页表实现
无快表访存次数	2 次	2 次	**3** 次
有快表命中访存次数	1 次	1 次	1 次

（八）页面置换算法总对比

算法	淘汰规则	是否可实现	性能	**Belady** 异常	实现开销
OPT	淘汰以后最长时间不再访问的页	不可实现（理论基准）	最优（缺页最少）	否	—
FIFO	淘汰最早进入内存的页	可实现	最差	是	最小
LRU	淘汰最近最久未使用的页	可实现	接近 **OPT**	否	很大
CLOCK（**NRU**）	循环扫描，淘汰第一个访问位为 0 的页（最多两轮）	可实现	介于 LRU 与 FIFO 之间	是	小
改进型 **CLOCK**	按 $(0,0) > (0,1) > (1,0) > (1,1)$ 顺序（最多四轮）	可实现	较好，减少写回开销	是	较小

（九）连续分配三种方式总对比

分配方式	内存划分	多道能力	内部碎片	外部碎片	采用的装入方式	地址变换时机
单一连续分配	系统区 + 一整块用户区	单道	有	无	绝对装入	编译时
固定分区分配	系统区 + 预先划分的若干固定分区	多道（≤ 分区数）	有	无	可重定位装入	装入时
动态分区分配	系统区 + 装入时动态建立的分区	多道	无	有	动态运行时装入	执行时

（十）有效访问时间速查

情形	公式	说明
无 TLB，命中一次访存	$EAT = (1-p)t + p\,t_{\text{pf}}$	考纲标准式
无 TLB，命中需两次访存	$EAT = (1-p)\cdot 2t + p\,t_{\text{pf}}$	命中也要先查页表
有 TLB，命中	$\lambda + t$	只需一次访存
有 TLB，TLB 缺失但页在内存	$2(\lambda + t)$	命中需查页表并更新 TLB
有 TLB，缺页	$3\lambda + \varepsilon + 3t$	含重新执行指令
加权总式	$\big[(2+f)-\alpha(1+f)\big]\lambda + \big[(2-\alpha)+f(1-\alpha)\big]t + f(1-\alpha)\varepsilon$	α 为 TLB 命中率，f 为缺页率

易错清单

- 逻辑地址在「链接」阶段形成（2011-30），物理地址由 **MMU** 在访存时使用。
- 装入方式与地址变换时机配对：绝对装入 → 编译时；可重定位装入 → 装入时；动态运行时装入 → 执行时。只有动态重定位允许程序在内存中移动。
- 静态重定位 ≠ 静态链接；动态重定位 ≠ 动态链接。前者是装入方式，后者是链接方式。
- 内部碎片 **vs** 外部碎片：内部碎片是「分配单元内部」的浪费（过量分配）；外部碎片是「分配单元之间」的浪费（按需分配）。单一连续、固定分区、分页、段页式有内部碎片；动态分区、分段有外部碎片。
- 动态分区四种算法：首次适应综合最好；邻近适应隐含最佳适应的缺点；最佳适应最容易产生外部碎片；最坏适应大分区很快耗尽。别被「最佳」二字误导。
- 回收分区的合并要注意与上块、与下块、与上下块一起合并三种情况，以及合并后分区数目的变化（2017-25 考的是上下同时合并）。
- 分页地址一维、分段地址二维：页号与页内偏移不是独立变量；段号与段内偏移是两个独立变量。
- 分页越界检查 **1** 次，分段 **2** 次。页内偏移量不可能越界；段内偏移量可能越界。
- 页表项存的是「页框号」，不是「页框起始地址」；b 号页框的起始地址为 $b \times L$。
- 页表项常取 **4B** 而非 **3B**，为让一个页面恰好装下整数个页表项。做题以题目给出的页表项大小为准。
- 多级页表优点是「减少页表所占的连续内存空间」，它不加快地址变换、不减少页表项字节数、不直接减少缺页次数。
- **CPU** 页表基址寄存器存的是「一级页表的起始物理地址」（2021-29）；各级页表大小不能超过一个页面。
- 共享段被不同进程引用的段号可以不同（2019-28），但物理内容只有一份。
- 分页共享时页号可以不同、页框号一定相同（2023-30）；段页式共享同理。
- 段页式无快表需 **3** 次访存（查段表 + 查页表 + 访存），快表命中时 1 次。
- 虚拟内存最大容量受「地址结构」限制，实际容量 = min(内存 + 外存, 寻址范围)；绝不能说「由主存和硬盘大小决定」。
- 虚拟内存必须建立在「离散分配」基础上；三特征是多次性、对换性、虚拟性，「多次性」是核

心。

- 缺页中断属于「内中断 / 异常 / 故障」，由当前执行的指令造成；一条指令执行期间可能发生多次缺页中断。
- 缺页处理完成后是「重新执行该指令」，不是「直接访存」——这直接决定 EAT 里要不要重复计算 TLB 与页表访问。
- 缺页 $\neq$ 越界（2013-30）；缺页处理中不一定包含「淘汰内存中的页」（2022-29，还有空闲页框时直接装入）。
- 「固定分配 + 全局置换」不存在（2015-30）：全局置换必然改变进程页框数，与固定分配矛盾。
- 驻留集不能小于工作集，否则抖动；工作集要向「过去」取窗口，不能算入 t 之后的访问（2016-29）。
- **Belady** 异常只发生在 **FIFO**、简单 **CLOCK**、改进型 **CLOCK** 上；**LRU** 与 **OPT** 不会（满足栈性质）。
- **OPT** 不可实现；**FIFO** 最差；**LRU** 性能好但开销大；**CLOCK** 是 **LRU** 的廉价近似。
- 改进型 **CLOCK** 至多四轮，只改访问位、从不改修改位；优先级 $(0,0) > (0,1) > (1,0) > (1,1)$ ——先看访问位，再看修改位（2016-26 的 B 选项即「先看修改位」的错解）。
- 简单 **CLOCK** 至多两轮；手算务必记准指针位置与访问位状态。
- 缺页时不一定发生页面置换；页面置换不改变页框号，只改变哪个页号占用该页框。
- 快表中有的页面一定在内存中，页面换出时必须同步删除快表项，故不会出现「TLB 命中但页面不在内存」。
- `mmap` 映射的是「磁盘文件 $\leftrightarrow$ 进程虚拟地址空间」，与虚拟内存、与「内存映射 I/O」（映射设备寄存器）都不是一回事；且 `mmap` 建立映射时并不立即加载文件数据。
- **EAT** 公式不能用错情形：命中时是「一次访存」还是「查页表 + 访存」；有快表时缺页处理是否更新 TLB、缺页后是重新执行还是直接访存。
- 抖动不能靠「增加交换区容量」或「提高进程优先级」解决，只能靠减少并发进程数 / 增加每进程页框数（2011-29）。

第十七章文件管理

本章考纲要求

四、文件管理（一）文件

1. 文件的基本概念
2. 文件元数据和索引节点（inode）
3. 文件的操作：建立，删除，打开，关闭，读，写。
4. 文件的保护
5. 文件的逻辑结构
6. 文件的物理结构

（二）目录

1. 目录的基本概念
2. 树形目录
3. 目录的操作
4. 硬链接和软链接

（三）文件系统

1. 文件系统的全局结构（layout）：文件系统在外存中的结构，文件系统在内存中的结构。
2. 外存空闲空间管理方法
3. 虚拟文件系统
4. 文件系统挂载（mounting）

本章是操作系统部分「重理解、重计算」的典型一章。文件名、目录项、索引节点、打开文件表这几层数据结构之间的层次关系，以及「逻辑块号 → 物理块号」的转换次数，是统考反复出题的两条主线。学习时建议始终带着一个问题：此刻 **CPU** 手里拿的是文件名、文件描述符，还是物理块号?

一、文件的基本概念

(一）数据项、记录与文件

文件的内部是分层组织的，自底向上依次是:

- 数据项（**data item**）：最低级的数据组织形式。基本数据项是数据中不可再分的最小逻辑单位，如「学号」「姓名」；组合数据项由若干基本数据项组成，如「出生日期」由年、月、日组成。
- 记录（**record**）：一组相关数据项的集合，用于描述一个对象在某方面的属性。例如一条学生记录 = 学号 + 姓名 + 出生日期 + 专业。
- 文件（**file**）：由创建者定义、具有文件名的一组相关信息的集合。文件可分为有结构文件（若干记录组成）与无结构文件（字符流）。

文件的定义与三个视角

从用户角度看，文件是具有符号名（文件名）的一组相关信息的有序集合，它可以存储在磁盘等外存上，并且可以被长期保存。

理解文件要抓住三个层次：

- 用户视角：文件是「按名字存取的信息集合」，通过文件名访问，不必知道数据究竟放在哪个盘块。
- 系统视角：文件是外存空间的分配单位，文件系统为每个文件维护一份元数据（FCB/inode）记录它占用了哪些盘块。
- 进程视角：文件是进程可以打开、读写的对象，打开后以「文件描述符」为句柄操作，与文件名脱钩。

（二）文件系统及其功能

文件系统（**File System**）是操作系统中负责管理和存取文件信息的软件机构，也是磁盘上组织文件的方法。它的核心目标只有一个：实现「按名存取」，同时保证高效、安全、可靠。

- 从系统角度看，文件系统负责：文件存储空间的管理、目录管理、逻辑地址到物理地址的映射（文件读写管理）、文件的共享与保护。
- 从用户角度看，文件系统提供了命令接口（终端里敲 `ls`、`cp`）和程序接口（`open`、`read`、`write` 等系统调用）。

文件系统容量不等于磁盘容量

一个文件系统能容纳的文件数量，受索引节点（**inode**）总数限制（搭建文件系统时往往已经固定，格式化时划分了 inode 区）；而单个文件的最大长度受 inode 中地址项的个数与结构限制。所以：

- 「文件系统最多能存多少个文件」由 **inode** 总数决定，与磁盘容量无关；
- 「单个文件最大有多长」由 **inode** 的索引结构决定，与 inode 总数无关。

这两句话是 2013 年真题的标准解法。用 RAID 磁盘阵列时，一个文件还可以分散在多块磁盘上，所以「文件必须放在一块盘上」也是错的。

（三）文件的属性与分类

常见属性包括：名称、标识符、类型、位置（指向设备和设备上文件的指针）、大小、保护（访问权限）、创建者/所有者 ID、创建与最近修改时间、链接计数等。

按不同角度分类：

分类角度	类别
性质和用途	系统文件、用户文件、库文件
数据形式	源文件（`.c`、`.java`）、目标文件（编译后的二进制）、可执行文件
存取控制	只读文件、读写文件、可执行文件
组织形式	普通文件、目录文件、索引文件、特殊文件（设备文件、命名管道等）

文件元数据和索引节点

（四）文件控制块 FCB

为了能对一个文件进行正确的存取，文件系统必须为文件设置一个描述与控制用的数据结构，称为文件控制块（**File Control Block, FCB**）。文件与 **FCB** 一一对应，FCB 的有序集合称为文件目录，其

中的每一个 FCB 就是一个目录项（**directory entry**）。目录本身也被视为一种文件，称为目录文件。FCB 通常包含三类信息：

类别	典型内容
基本信息	文件名、物理位置、逻辑结构、物理结构、文件类型、文件大小
存取控制信息	文件存取权限（文件主、核准用户、一般用户各自的读/写/执行权限）
使用信息	文件数据块指针、文件建立时间、上次修改时间、当前使用计数

FCB 的作用

FCB 建立了文件名与文件物理位置之间的映射，这正是「按名存取」得以实现的根本原因。用户给出文件名，文件系统查目录找到对应的 FCB，从中取出物理地址，再去访问外存。

因此可以记住两个等式：

$$\text{文件目录大小} = FCB\text{大小} \times \text{目录项个数}$$

$$\text{一个}FCB = \text{一个文件目录项}$$

索引节点 **inode** 与目录项的分离

为什么要把 **FCB** 拆开？检索目录时，其实只需要用到「文件名」这一个字段：只有文件名匹配成功了，才需要读出文件的其他信息。若每个目录项都是完整的 FCB，目录文件会占据大量盘块，平均检索时间随之飙升。

于是有了索引节点（**index node**，简称 **i** 节点、**inode**）方案：把 FCB 中除文件名以外的所有信息抽出来，构成一个独立的数据结构 inode；目录项则「瘦身」为文件名 + **inode** 编号（指针）两项。

一个必背的量化例子

设一个 FCB 占 64B，磁盘块大小 1KB，则每块只能放 $1024/64 = 16$ 个 FCB。若目录共有 640 个目录项，则目录文件占 $640/16 = 40$ 个盘块，按文件名检索平均需要比较 $640/2 = 320$ 个目录项，即平均启动磁盘 **20** 次（每次读入一块）。

改用索引节点后，设「文件名 14B + inode 指针 2B」共 16B，则每块可放 $1024/16 = 64$ 个目录项，检索平均只需读入 $320/64 = 5$ 个盘块。

磁盘 **I/O** 次数从 **20** 次降到 **5** 次，这就是索引节点机制的价值。

引入 inode 后，目录项与 inode 的关系是：

- 目录项只记录「文件名 → inode 编号」这一条逻辑映射，它描述的是文件在文件系统中的位置（逻辑位置）。
- **inode** 记录文件的元数据，其中包括文件在存储设备上的物理位置（直接/间接索引），描述的是文件数据实际存放在哪里。

区分「磁盘索引节点」与「内存索引节点」

磁盘索引节点存放在磁盘的 inode 区，是静态的，主要内容有：文件主标识符、文件类型、文件存取权限、文件物理地址（直接与间接索引项）、文件长度、文件链接计数 **count**、文件存取时间。

内存索引节点是文件被打开时从磁盘复制（调入）到内存的，除上述内容外还要增加：

- **inode** 编号：标识对应外存 inode 的编号；

- 状态位：是否被上锁、是否被修改（修改位置位决定关闭时是否需要写回外存）；
- 访问计数：记录当前有多少个进程打开了该文件；
- 逻辑设备号：标识 inode 所在的逻辑设备（分区）；
- 链接指针：分别指向内存空闲链表和散列队列的指针，用于加速查找。

内存 **inode** 可以被多个进程共享，这正是「多个进程打开同一文件时系统打开文件表只有一个表项」的底层原因。

inode 中的多级索引与最大文件大小计算

UNIX/Linux 混合索引的经典结构是 inode 中设 **13** 个地址项 `iaddr[0] ~ iaddr[12]`：

地址项	类型	作用
`iaddr[0] ~ iaddr[9]`	直接地址（10 项）	直接指向数据块
`iaddr[10]`	一级间接地址（1 项）	指向一个索引块，索引块里放数据块号
`iaddr[11]`	二级间接地址（1 项）	指向索引块，其中每项再指向一个索引块
`iaddr[12]`	三级间接地址（1 项）	三级「套娃」索引

通用解题模板

记：磁盘块（数据块与索引块同大小）大小 B，一个地址项占 p 字节，则

$$\text{一个索引块可容纳的地址项数 } n = \left\lfloor \frac{B}{p} \right\rfloor$$

设第 k 级间接地址项个数为 N_k（直接地址视为 0 级，$k=0$），则第 k 级可覆盖的数据量为

$$S_k = N_k \times n^k \times B \qquad (k = 0, 1, 2, 3, \dots)$$

单个文件最大长度（即可寻址的数据总量）为

$$S_{max} = \sum_{k} N_k \cdot n^k \cdot B$$

注意：索引块本身只存放地址，不计入文件的数据长度。这是「文件最大长度」类题目最容易被扣分的地方。

完整算例（**UNIX** 经典参数）。设磁盘块大小 $B = 4\text{KB}$，每个地址项 $p = 4\text{B}$，直接地址 10 个、一级/二级/三级间接各 1 个。

- 一个索引块能放的地址项数：$n = 4096\text{B}/4\text{B} = 1024$ 个。
- 直接地址：$10 \times 4\text{KB} = 40\text{KB}$。
- 一级间接：$1 \times 1024 \times 4\text{KB} = 4\text{MB}$。
- 二级间接：$1 \times 1024^2 \times 4\text{KB} = 4\text{GB}$。
- 三级间接：$1 \times 1024^3 \times 4\text{KB} = 4\text{TB}$。

$$S_{max} = 4\text{TB} + 4\text{GB} + 4\text{MB} + 40\text{KB}$$

第二组算例（更常考的参数）。设 $B = 1\text{KB}$，$p = 4\text{B}$，直接 10 项，一/二/三级间接各 1 项：

- $n = 1024/4 = 256$；
- 直接：$10 \times 1\text{KB} = 10\text{KB}$；一级间接：$256 \times 1\text{KB} = 256\text{KB}$；二级间接：$256^2 \times 1\text{KB} = 64\text{MB}$；三级间接：$256^3 \times 1\text{KB} = 16\text{GB}$。
- 最大文件长度 $= 16\text{GB} + 64\text{MB} + 256\text{KB} + 10\text{KB}$。

逻辑块所在区间与访问磁盘次数的关系（**inode** 已在内存）：以上参数下，逻辑块号（从 0 开始）与访问磁盘块数的对应关系是

逻辑块号范围	所属区域	需要读的磁盘块数
0 ~ 9	直接地址	1（只读目标数据块）
10 ~ 265	一级间接	2（索引块 + 数据块）
266 ~ 65801	二级间接	3（二级索引块 + 一级索引块 + 数据块）
65802 及以上	三级间接	4（三级 + 二级 + 一级索引块 + 数据块）

其中二级间接起始块号 $= 10 + 256 = 266$，二级间接可覆盖 $256^2 = 65536$ 块，故其末块号 $= 266 + 65536 - 1 = 65801$。

第三个算例（对应真题的偏移量定位思路）。设 $B = 1\text{KB}$、$p = 4\text{B}$、直接 10 项、一/二级间接各 1 项，inode 已在内存。求偏移量 1234 与 307400 处所在磁盘块各需访问几个磁盘块。

- 偏移量 1234：$\lfloor 1234/1024 \rfloor = 1$，即第 1 号逻辑块，落在直接地址区，只需读 **1** 个磁盘块。
- 偏移量 307400：$\lfloor 307400/1024 \rfloor = 300$，直接区覆盖 0~9，一级间接覆盖 $10 \sim 10+256-1 = 265$，300 落在二级间接区，需要读二级索引块、一级索引块、目标数据块共 **3** 个磁盘块。

计算时最容易错的三处

1. 块号从 **0** 还是从 **1** 开始。题目说「第 3 个逻辑块（块号从 1 开始）」时，要先减 1 再判断落在哪个区。统考题目一般会明确说明起始编号，没说明时按题目中已出现的例子推断，不要凭习惯。
2. 索引块不计入文件数据长度。求「单个文件最大长度」只累加数据块；求「占用几个索引块」才去数索引块。
3. 把「地址项个数」当成「索引块个数」。一个一级间接地址项对应一个索引块，但这个索引块里装的是 n 个数据块地址。

（五）inode 位图与数据块位图

磁盘上需要记录两种「哪些编号还能用」的信息，UNIX 风格的文件系统用两张位图分开管理：

- **inode** 位图（**inode bitmap**）：每一位对应 inode 区中的一个 inode，标记它是否已被分配。创建一个文件时先查 inode 位图找一个空闲 inode。
- 数据块位图（**block bitmap**）：每一位对应数据区中的一个磁盘块，标记它是否已被分配。为文件分配数据块时查它。

两张位图都存放在磁盘的固定位置，并在使用时按需把相关块读入内存修改，再在适当时机写回。因为一位只表示一个对象，位图占用空间极小、可常驻内存，是「0 表示空闲、1 表示已分配」的常用约定（注意不同教材符号可能相反，做题以题目给的约定为准）。

inode 编号到磁盘块的换算

设磁盘块大小 B，一个 inode 占 s 字节，inode 区起始块号为 b_0，则每个盘块可容纳的 inode 数

$$m = \left\lfloor \frac{B}{s} \right\rfloor$$

inode 编号为 k（从 0 开始编号）的 inode，位于

$$\text{盘块号} = b_0 + \left\lfloor \frac{k}{m} \right\rfloor, \qquad \text{块内偏移} = k \bmod m$$

算例：$B = 4\text{KB}$，$s = 16\text{B}$，inode 区起始块号 6，则 $m = 4096/16 = 256$。编号 1314 的 inode：$\lfloor 1314/256 \rfloor = 5$，$1314 \bmod 256 = 34$，所以它位于 $6 + 5 = 11$ 号磁盘块，是该盘块中的第 34 个 inode。

追问：需要只把这一个 **inode** 读入内存吗？不行。磁盘 I/O 以「块」为最小单位，无法只读块内某一部分，所以必须把整个 **11** 号块读入内存，再从中取用第 34 个 inode。这也是内存 inode 缓存存在的理由——同一块中其他 inode 后续被访问时可直接命中缓存。

二、文件的操作

文件的基本操作可以分成两类：不涉及打开的「目录级操作」（建立、删除）和基于打开表的「句柄级操作」（打开、关闭、读、写）。

（一）建立文件

`create(path)` 在指定路径下建立一个空文件，可以同时指定文件类型与访问权限。步骤如下：

1. 查询磁盘的 inode 位图（或 inode 表），为该文件分配一个空闲 **inode**，在其中登记文件类型、访问权限、创建时间等信息；
2. 打开 `path` 所指示的父目录，在目录中新增一个目录项，目录项的文件名取自 `path` 的最后一部分（分量名），文件名关联到刚分配的 inode 编号上。

（二）删除文件

`delete(path)` 删除指定路径下的目录项；若该文件已无其他链接，还要回收文件实体。步骤如下：

1. 按路径找到目标文件的目录项，进而找到 inode；
2. 根据 inode 中的访问权限判断「删除」操作是否允许（文件保护）；允许则读取其链接计数：
   - 若链接计数为 1，说明这是最后一条链接，释放该文件占用的所有磁盘块（修改数据块位图），同时释放磁盘 **inode**（修改 inode 位图）；
   - 若链接计数大于 1，则只把它减 1，文件数据保留（其他硬链接仍然有效）；
3. 在文件所在目录中删除对应的目录项。

删除文件不等于删除目录

2013 年真题考过：用户在删除某文件的过程中，操作系统不可能执行的操作是「删除此文件所在的目录」。删除文件只删除该文件自己的目录项，以及在没有剩余链接、没有进程打开它时回收其 inode 和数据块。目录还包含其他文件，绝不能被连带删除。

另外，删除文件也不必去删除该文件的「快捷方式」（符号链接）。符号链接是独立的文件对象，系统并不保证它存在，也不能假定它必须同步删除（2021 年真题考点）。

（三）打开文件表与文件描述符

`open(path, mode)` 打开名为 `path` 的文件，成功时返回一个整数——文件描述符（**file descriptor, fd**）。

open 返回的到底是什么

`open` 返回的是「本进程的文件描述符表」中的表项下标，不是文件内容、不是缓冲区指针、也不是文件大小。

打开的本质是：系统检索到指定文件的目录项后，把该文件的元数据（**inode/FCB**）从外存复制到内存，并在系统打开文件表和进程打开文件表中登记表项，然后把这个表项的索引号作为 fd 返回给用户。

此后的 `read`、`write`、`close` 全部以 fd 为参数，不再使用文件名，也就不再做按名查找。所以 2024 年真题问「哪个系统调用的实现中包含文件按名查找功能」，答案是 `open()`。

两级打开文件表的层次结构：

层次	位置	内容	特点
系统级打开文件表	内核，全系统一张	每个已打开文件的 inode 副本、文件状态、访问权限、打开计数	一个文件只对应一个表项；多个进程打开同一文件时共享它
进程级打开文件表（文件描述符表）	内核，每个进程一张，PCB 中记录其起始地址	fd、读写指针（文件偏移量）、访问模式、指向系统表项的指针	每个进程一张，各自独立

`open` 的完整处理过程（进程 P1 首次以只读方式打开 `/root/dir1/c.mp4`）：

1. 路径查找：根据路径逐级查找目录。若目录缓存未命中，就把目录文件所在盘块读入内存，逐层比较分量名，直到找到目标文件的目录项。这一阶段只用文件名。
2. 定位 **inode**：从目录项取出 inode 编号，去系统打开文件表中检索该 inode 是否已在内存：
   - 未命中：从磁盘把该 inode（所在的整个盘块）载入内核缓冲区；检查访问权限，若越权则打开失败；若不越权，在系统级打开文件表中新增一个表项，打开计数置 1，并记录内存 inode 的位置；
   - 命中：同样检查访问权限，通过后打开计数 **+1**，不重复读盘。
3. 登记进程表：在进程 P1 的进程打开文件表中新增一个表项，记录对应的系统打开文件表项索引、访问模式和初始读写指针。
4. 返回 **fd**：把该进程表项的下标作为 fd 返回给用户进程。

三条几乎每年都考的结论

1. 一个文件在系统打开文件表中只有一个表项（无论多少进程打开它）；但每个进程的打开文件表中都会各有一项，且不同进程的 fd 数值可以不同。
2. 读写指针记录在各进程自己的打开文件表中，所以多个进程读同一个文件时互不干扰、各读各的位置（2017 年真题：f1、f2 是硬链接，两个进程分别打开，读写指针位置不一定相同，但它们共享同一个内存索引节点）。
3. 各进程的访问权限可以不同：进程 P1 以只读打开、进程 P2 以读写打开同一个文件是完全合法的，各自受自己申请的模式约束。

open 不一定产生磁盘 I/O

若文件已被其他进程打开、其 inode 已在内存，则本次 `open` 只需在系统打开文件表中把打开计数 +1，并

新建一个进程表项，不需要读磁盘。因此「open 必然引起文件 I/O」是错的。

（四）关闭文件

`close(fd)` 成功返回 0，失败返回 −1。步骤：

1. 根据 fd 找到进程打开文件表中对应的表项并删除；
2. 把系统打开文件表中该表项的打开计数 **−1**：
   - 若减后不为 0，说明还有其他进程在使用，直接返回；
   - 若减后为 0，删除该系统表项；若该内存 inode 被修改过，则按操作系统策略写回外存（若未修改则直接释放内存 inode）。

关闭不等于删除。关闭只做「把内存中的控制信息同步回外存、释放内存中的打开表项」，它不会删除目录项、不会释放外存 inode、也不会修改磁盘 inode 中的链接计数（2023 年真题：文件 F 仅被进程 P 打开，P 关闭 F 时系统需要完成的是「释放 F 的索引节点所占的内存空间」）。

（五）读文件与写文件

`read(fd, buf, count)` 从文件描述符对应的文件中读数据到用户缓冲区，返回实际读到的字节数。处理过程：

1. 用户程序在用户态调用 `read`，通过陷入（trap）进入内核态；
2. 根据 fd 查进程打开文件表，按访问模式判断是否允许读；
3. 允许则经系统打开文件表找到内存 inode，依据读写指针位置 + **count** 确定要读的逻辑块区间，再由 inode 中的索引信息把逻辑块号映射为物理块号，把这些盘块读入内核缓冲区；
4. 从内核缓冲区把所需那部分数据复制到用户空间 `buf`；
5. 返回实际读取的字节数，并相应推进读写指针。

`write(fd, buf, count)` 的处理过程与之对称：

1. 陷入内核；2. 按 fd 检查是否允许写（只读打开时会失败）；3. 由读写指针和 inode 定位目标盘块的物理位置，先把该盘块读入内核缓冲区，用 `buf` 中的数据覆盖缓冲区中的指定区域，再把该盘块写回磁盘；4. 返回实际写入的字节数。

为什么读/写都要经过内核缓冲区

磁盘是块设备，读写最小单位是块；而内存可以按字节随机访问。用户想改一个字节，也必须先把该字节所在的整块读出来、在内存里改、再整块写回。

另外，中断服务程序是在当时占用 **CPU** 的那个进程的地址空间中执行的，它无法直接把数据写进另一个进程的用户空间，所以必须用「映射到每个进程内核空间」的内核缓冲区作为中转。做题时若问「能否直接把数据从磁盘读到用户空间」，答案是不能（除非使用特殊的直接 I/O 机制）。

本知识点的统考考法

- **2012-28**：`read` 系统调用的参数包含文件描述符而不包含文件名；数据不在内存时进程进入睡眠等待；系统调用导致用户态 → 核心态切换。
- **2014-29**：文件首次被打开的过程中，操作系统做的是「将文件控制块读到内存中」，而不是把文件内容读入内存。
- **2020-23 / 2017-31 / 2023-31**：系统打开文件表只有一项、进程表各自一份、关闭只减计数。
- **2024-29**：只有 `open()` 包含按名查找。

- 大题常以「参数已读入内存，访问第 i 个逻辑块需要几次磁盘 I/O」的形式出现，答法见后文物理结构一节。

三、文件的保护

文件保护解决的是谁能以什么方式访问文件的问题。首先要区分两类机制：口令保护与加密保护是为了防止文件被他人存取或窃取；访问控制是为了控制用户对文件的访问方式。

（一）可控制的访问类型

- 读：从文件中读数据；
- 写：向文件中写数据；
- 执行：把文件装入内存并执行；
- 添加（**append**）：在文件末尾追加新信息；
- 删除：删除文件，释放空间；
- 列表清单：列出文件名和文件属性。

重命名、复制、编辑等高层功能可以由系统程序调用底层系统调用来实现，因此保护只需在底层提供。例如复制文件可以由一系列读请求完成，所以具有读权限的用户同时也就具有了复制、打印权限。

写模式与添加模式的区别

以写模式打开文件时会立即把文件截断为 **0** 字节，原有内容永久丢失，即使之后一个字节都没写；以添加模式打开则保留原有内容，读写指针初始位置在文件末尾，新数据自动追加。

（二）访问控制矩阵与访问控制列表

- 访问控制矩阵（**Access Control Matrix**）：行是用户、列是文件，矩阵元素表示该用户对该文件的访问权限。优点是用户与文件权限的对应关系记录得非常清晰；缺点是占用存储空间太多，且它只起记录权限的作用，无法保证文件系统本身不受损。
- 访问控制列表（**Access Control List, ACL**）：在每个文件和目录的 **FCB**（或 **inode**）中增加一张表，记录各个用户可以对该文件执行哪些操作。这是把访问控制矩阵「按列压缩」得到的稀疏表示。
  - 优点：灵活，可实现复杂的访问方式；
  - 缺点：ACL 的长度无法预计，可能导致复杂的空间管理。

为了控制 ACL 的规模，引入了精简的访问列表：以「组」为单位标记各类用户可以执行哪些操作。典型的三类用户是：

用户类别	含义
拥有者（owner）	创建该文件的用户
组（group）	一组需要共享该文件且具有类似访问需求的用户
其他（other）	系统内的其他用户

当某用户请求访问文件时，系统检查该用户所属类别是否有相应权限。系统需要管理分组信息，并把文件主名、所属组名存放在该文件的 FCB 中。

权限位不能用「每类用户 2 位」表示

2017 年真题考过：4 类用户 × 5 种访问权限，若用二进制位串表示权限，至少需要 $4 \times 5 = 20$ 位。

原因是：一个用户对同一个文件可以同时拥有多种权限（既能读又能写又能执行），所以必须为「每一类用户的每一种权限」各留一个二进制位。把权限压缩成 2 位、认为一种用户只能选一种权限，是典型错误。

（三）传统的权限位 rwx

UNIX 风格文件系统用 9 个二进制位表示权限，分成三组，分别对应文件主（**u**）、同组用户（**g**）、其他用户（**o**）：

位	含义（普通文件）	含义（目录）
r 读	可以读取文件内容	可以列出目录中的文件名
w 写	可以修改文件内容	可以在目录中创建、删除、重命名文件
x 执行	可以把文件装入内存并执行	可以进入该目录（作为路径分量被解析）

用 `ls -l` 看到的 `-rwxr-xr--` 就是从高到低依次描述 u、g、o 三组权限。

访问权限 = 用户访问权限 ∩ 文件属性

文件访问控制由用户访问权限和文件属性共同决定，而不是由用户优先级决定。用户打开文件时申请的权限不能超过文件允许的权限；打开成功后，用户对文件的操作不能超过自己打开时申请的权限。这两条构成了访问控制的两道关卡。

（四）口令保护与加密保护

- 口令（**password**）保护：为文件建立 FCB 的同时附上一个口令，并告知允许共享的其他用户；用户请求访问时须提供口令，系统与 FCB 中存储的口令比对。
  - 优点：保存口令的空间开销小，验证口令的时间开销也很小；
  - 缺点：正确的口令就存放在系统内部，不够安全。
- 加密（**cryptography**）保护：用某个「密码」对文件内容本身进行加密，访问文件时须提供正确的密码才能正确解密。
  - 优点：保密性强，不需要在系统中存储密码；
  - 缺点：编码/译码（加密/解密）需要花费一定时间，且对文件的控制不如系统实现的访问控制灵活。

目录也要保护

对多级目录结构来说，不仅要保护单个文件，还要保护子目录内的文件，即需要目录保护机制。目录操作（搜索、创建、删除目录项）与文件操作不同，因此需要不同的保护机制。

四、文件的逻辑结构

文件的逻辑结构是从用户观点出发看到的文件组织形式，即文件内部这些逻辑记录是如何被组织起来构成逻辑文件的，它独立于文件的物理特性，用户看不见物理存放在哪。

对逻辑结构的基本要求：有利于提高检索速度、方便增删改查、降低存储费用（尽量不要求大片连续存储空间）。

（一）无结构文件与有结构文件

- 无结构文件（流式文件）：文件内部是一系列二进制流或字符流，以字节为单位组织，如`.txt`文件。访问时只能通过读写指针穷举搜索。适用于源程序、可执行文件、库函数等。UNIX 系统中绝大多数文件都是流式文件。
- 有结构文件（记录式文件）：由一组相似的记录组成，每条记录由若干数据项组成，通常有一个数据项作为关键字。按记录长度是否相等分为：
  - 定长记录：检索速度快，第 i 条记录的地址可直接算出，便于处理和修改，广泛用于数据处理；
  - 变长记录：检索只能顺序进行，速度慢，也不便于处理修改，但能节省空间。

顺序文件的随机存取条件

$$\text{第}i\text{ 条记录首地址} = i \times L \quad (i\text{ 从0 起}, L\text{ 为定长记录长度})$$

只有当「定长记录 + 物理上顺序存储」同时成立时，顺序文件才支持随机存取；再保证记录按关键字排序（顺序结构），才支持折半查找等快速检索。变长记录无论怎么存，都无法随机存取；物理上是链式存储时，也无法随机存取。

（二）顺序文件

记录一个接一个顺序排列，记录可定长也可变长，物理上可顺序存储或链式存储。按记录顺序与关键字的关系分为：

- 串结构：记录顺序与关键字无关，通常按存入时间决定顺序。检索某记录必须从头顺序查找，较费时。
- 顺序结构：记录按关键字顺序排列。定长时可折半查找，检索效率显著提高。

适用场合：对单条记录的增删改查困难、性能差，因此适合批量操作（例如磁带这类顺序存储设备上每次读写一大批数据时，顺序文件的存取效率最高）。

索引文件

变长记录文件要找到第 i 条记录必须先顺序读前 $i-1$ 条，为了解决这个矛盾，为主文件的每条记录在索引表中设一个表项，表项内容包括指向该记录的指针（逻辑起始地址）和记录长度。

- 索引表本身是定长记录的顺序文件，所以可以快速定位第 i 个索引项；
- 若索引表中的关键字按顺序排列，还支持按关键字折半查找；
- 每增加/删除一条记录都要修改索引表；
- 可以用不同的数据项建立多张索引表（如学生信息表可分别按「学号」「姓名」建索引），这就是数据库索引的思想。

代价：若每条记录平均只有 8B，而每个索引项占 32B，索引表会比文件内容本身还大数倍，存储空间利用率极低。

索引顺序文件

索引顺序文件是索引文件与顺序文件思想的结合：为文件建立一张索引表，但不是每条记录对应一个索引项，而是一组记录对应一个索引项。每个分组内部就是一个顺序文件（组内不必按关键字排序），索引表与各分组顺序文件（主文件）共同构成索引顺序文件。查找时先在索引表中找组，再

在组内顺序找记录。

索引顺序文件的平均查找次数

设共 n 条记录，平均分成 i 组，每组 n/i 条：

- 纯顺序查找：平均 $\frac{n}{2}$ 次；
- 索引顺序查找：组间平均 $\frac{i}{2}$ 次 + 组内平均 $\frac{n/i}{2}$ 次。

求最优分组，令 $f(i)=\frac{i}{2}+\frac{n}{2i}$ 最小，得 $i=\sqrt{n}$ 时总查找次数约为

$$\frac{\sqrt{n}}{2}+\frac{\sqrt{n}}{2}=\sqrt{n}$$

例如 $n=10^4$ 时，顺序查找平均 5000 次，索引顺序查找平均约 100 次。

多级索引顺序文件：$n=10^6$ 条记录时，先建低级索引表（每 100 条一组，共 10000 个表项），再对这 10000 个定长表项按每 100 个一组建顶级索引表（共 100 个表项）。

建 K 级索引的最优分组与平均查找次数

为 N 条记录的文件建 K 级索引，最优分组是每组 $N^{1/(K+1)}$ 条记录，检索一条记录的平均查找次数为

$$\frac{K+1}{2}\cdot N^{\frac{1}{K+1}}$$

例如 $N=10^6$ 建 2 级索引：最优每组 $10^{6/3}=100$ 条，平均查找次数 $=\frac{100}{2}\times 3=150$ 次。

（三）直接文件或散列文件

给定记录的键值，或通过散列（**Hash**）函数转换后的键值，直接决定记录的物理地址。

- 优点：存取速度很高，不需要顺序查找，也不需要维护庞大的索引表；
- 缺点：会引起冲突（不同关键字的散列函数值相同），需要解决冲突；散列函数设计和空间管理相对复杂。

散列思想同样适合磁盘块的分配与回收：基于 Hash 快速定位空闲物理块，分配效率高；回收时只需标记空闲，后续可快速复用，碎片问题相对不突出。

（四）各逻辑结构的查找效率对比

逻辑结构	记录长度	查找方式	平均查找次数	存储额外开销
顺序文件（串结构）	定长/变长	从头顺序扫描	$n/2$	无
顺序文件（顺序结构）	定长	折半查找	$\log_2 n$	无
索引文件	定长/变长	查索引表	索引表内 $O(1)$~$O(\log n)$，变长索引表内需顺序找	每记录一个索引项，开销大
索引顺序文件	定长/变长	组间索引 + 组内顺序	约 $\sqrt{n}$	每组一个索引项，开销较小
直接/散列文件	任意	散列函数直接定位	接近 $O(1)$	需要处理冲突的空间

五、文件的物理结构

文件的物理结构（存储结构）指系统把文件存放在外存上所形成的存储组织形式，用户看不见。它研究两件事：一是文件的分配方式（对磁盘非空闲块的管理），二是文件存储空间管理（对磁盘空闲块的管理，见后文）。

基础约定：

- 磁盘被划分为一个个磁盘块（物理块），内存与磁盘之间的数据交换以「块」为单位；
- 文件的逻辑地址空间也被划分为一个个文件块，因此文件逻辑地址可表示为「逻辑块号 + 块内地址」；
- Windows 中称「簇」，UNIX 中称「磁盘块」；簇是文件的分配单位，扇区是磁盘上最小的物理单元。

两类「随机存取」要分清

- 逻辑结构支持随机存取：能根据记录号直接算出记录对应的逻辑地址（逻辑块号 + 块内地址）。
- 物理结构支持随机存取：能根据逻辑地址直接算出记录对应的物理地址。

「逻辑上支持随机存取」不等于「物理上支持随机存取」——顺序文件是定长记录且物理顺序存放时两者才同时成立。

（一）连续分配

结构：每个文件在磁盘上占有一组连续的块，逻辑上相邻的块物理上也相邻。

地址转换：用户给出逻辑块号，系统找到该文件 FCB 中的起始块号，则

$$\text{物理块号} = \text{起始块号} + \text{逻辑块号}$$

块内地址保持不变。因此 FCB/inode 的物理地址字段只需保存起始块号 + 总块数。

优缺点：

- 优点：支持顺序访问，也支持随机（直接）访问；访问时寻道数与寻道时间最小，顺序存取速度最快；实现简单。
- 缺点：文件长度不宜动态增加——文件末尾的盘块可能已分配给其他文件，一旦需要扩展就要大量移动盘块；反复增删后会产生外部碎片，存储空间利用率低；记录的插入、删除效率低下。

最大文件长度取决于 FCB 中「总块数」字段的位数和盘块大小：文件最大长度 = 最大块数 × 块大小。

连续分配的大题套路（插入/删除需要几次磁盘 I/O）

- 访问第 **3** 个逻辑块（首块已读入内存、FCB 已在内存）：只需计算物理块号后直接读入，**1** 次。
- 修改某块内容：读入 1 次 + 写回 1 次，共 **2** 次。
- 删除第 **2** 块（file-2）：为减少 I/O，选择把前面较少的块整体后移。若前面只有 file-0、file-1 两块，则需读入 file-1 写入 file-2 位置、读入 file-0 写入 file-1 位置，共 **4** 次 I/O；同时要修改 FCB 的起始块号 **+1**、总块数 **−1**。
- 在第 **4** 条记录前插入一条记录（7 条记录、前后都有空闲空间）：把前 3 条记录依次后移（3 次读 + 3 次写 = 6 次），再写入新记录（1 次），共 **7** 次 I/O；FCB 的起始块号要 −1，总块数 +1。

（二）链接分配

链接分配采用离散分配，可以为文件分配离散的磁盘块，分为隐式链接与显式链接。

*1.*隐式链接

结构：每个盘块（除最后一个）的尾部都有一个指向下一个盘块的指针。FCB 中保存起始块号与结束块号。

地址转换：从起始块号读入 0 号逻辑块，从它的尾部指针知道 1 号逻辑块的物理块号，再读入 1 号块……因此访问第 i 号逻辑块（从 **0** 开始）需要 $i+1$ 次磁盘 **I/O**。

优缺点：

- 优点：消除了外部碎片，提高了磁盘利用率；适合文件长度的动态增长（动态分配块）。
- 缺点：只支持顺序访问，不支持随机访问，查找效率低；指向下一块的指针本身要耗费少量存储空间；可靠性差——链中任一指针损坏，其后的数据全部丢失。

最大文件长度取决于：① 一个盘块中用多少位存放链接指针；② 盘块大小。例如盘块 1KB、指针占 2B，则每块可链 $\lceil 1024/2 \rceil$ 个关系，理论上限受块号编码位数约束。

可以引入「簇」：一个簇 = 若干个连续磁盘块，按簇顺序链接，可减少指针占用、加快查找；代价是增加内部碎片，而且分配空间时必须分配整数个簇（一个簇 4 块时，即使再要 1 块也必须分配整个簇）。

隐式链接的磁盘 I/O 计数（FCB 已在内存）

以 c.mp4 为例，7 个文件块 file-0 ~ file-6：

- 访问第 3 个逻辑块（file-2）：需读入 file-0、file-1 才能拿到 file-2 的块号，再读 file-2，共 **3** 次读盘。
- 修改 file-3 的内容：先读 file-0、file-1、file-2、file-3（4 次），再写回 file-3（1 次），共 **5** 次。
- 删除 file-2：读 file-0、file-1、file-2 拿到 file-3 的地址，修改 file-1 的指针后写回 file-1，共 **4** 次（file-2 的数据块标记为空闲不算磁盘 I/O）。
- 在第 4 条记录前插入：读前 3 块（3 次），把新记录写入空闲块并令其指针指向原第 4 块（1 次写），修改第 3 块的指针并写回（1 次写），共 **5** 次。

*2.*显式链接与文件分配表 *FAT*

结构：把所有「指向下一块」的指针从各盘块尾部抽离出来，集中存放在内存的一张链接表——文件分配表（**File Allocation Table, FAT**）中。FAT 的第 i 个表项存放「物理块 i 的下一个物理块号」。文件的第一块（起始块号）作为首地址写入 **FCB** 的物理地址字段，后续块通过查 FAT 得到。

FAT 的三个关键性质

1. 一个文件系统只设置一张 **FAT**，开机时被读入内存并常驻内存。
2. FAT 的各个表项在物理上连续存储，且每个表项长度相同，因此「物理块号」字段在表项中是隐含的（第 i 项就代表块 i）。
3. FAT 用到的数据结构是静态链表——用下标代替指针的链表。约定 `-1` 表示文件最后一块，`-2` 表示该块空闲。

   因此 **FAT** 也可以用于管理空闲磁盘块（2019 年真题：位图、空闲磁盘块链、FAT 都可以用于管理空闲磁盘块，而索引节点不行，索引节点描述的是已存在文件的元数据）。

地址转换：从 FCB 取起始块号，若目标逻辑块号 $i>0$，就在内存中查 FAT 依次得到第 i 块的物理块号。查 **FAT** 不需要读磁盘，因此访问某个逻辑块只需 **1** 次访盘取出记录。

优缺点：

- 优点：支持顺序访问，也支持随机访问（不必依次访问 0 ~ i 号逻辑块）；块号转换在内存中完成，比隐式链接快很多；不产生外部碎片，文件扩展方便。
- 缺点：FAT 需要占用一定的存储空间，而且必须是连续空间。

FAT 表大小的计算（完整算例）：

- 磁盘/分区共 N 个盘块，则 FAT 需要 N 个表项；
- 表项位数 $w = \lceil \log_2 N \rceil$（因为要能表示 N 个块号，以及 `-1`、`-2` 等状态，实际实现常向上取整到一个字节的整数倍）；
- FAT 占用空间 $= N \times w$（bit）。

算例：某分区 1GB，盘块大小 4KB，则盘块数 $N = 1\text{GB}/4\text{KB} = 2^{30}/2^{12} = 2^{18} = 262144$ 个。表项需 $\lceil \log_2 262144 \rceil = 18$ 位，实际取 3B（或按 24 位）存储，FAT 大小约 $262144 \times 3\text{B} = 768\text{KB}$。

算例（换一组参数）：盘块大小 2KB，FAT 每个表项 2B，则表项能表示的块号范围是 $2^{16} = 65536$ 个块，该文件系统支持的最大文件长度约为 $65536 \times 2\textbf{KB} = 128\textbf{MB}$；若 FAT 表项为 4B，则块号范围 2^{32}，最大文件长度约为 $2^{32} \times 2\text{KB} = 8\text{TB}$。这说明 **FAT** 表项长度直接决定了单个文件的最大长度上限（也可能受分区容量限制，取二者中较小者）。

显式链接（FAT 常驻内存）的磁盘 I/O 计数

同样以 7 个文件块为例，**FAT** 已在内存：

- 访问第 3 个逻辑块：查 FCB 得起始块号，再查内存 FAT 即可定位，**1** 次访盘。
- 修改 file-3：读入 + 写回，**2** 次。
- 删除 file-2：**0** 次——只需修改内存 FAT 中关于该文件的链接关系（把前驱指针指向 file-3），并把 file-2 的表项置为 `-2`（空闲）；由于 FAT 常驻内存，这些修改不需要立即访盘。
- 插入一条记录作为第 4 条：也只需在内存 FAT 中改指针，**0** 次访盘（新数据块本身的写入除外）。

索引分配

实现思想：打开某个文件时，只需把该文件占用的盘块号调入内存即可，完全没必要把整个 FAT 调入内存。于是为每个文件建立一张索引表，第 i 个条目指向文件的第 i 个块，表中记录各逻辑块对应的物理块号。存放索引表的盘块称索引块，存放文件数据的盘块称数据块。

优缺点：

- 优点：支持随机访问（可直接从索引块中读取想要的盘块号）；没有外部碎片问题；文件扩展方便。
- 缺点：由于要为每个文件分配索引块，增加了存储空间开销；索引块太大会浪费空间，太小则无法支持大文件；访问文件可能需要多次磁盘 I/O（要先读索引块）。

按索引层次分为三种方案：

- 单级索引（一级索引）：inode 中放一个索引块地址，指向一个索引块，索引块中依次存放该文件所有数据块号。若 $B = 4\text{KB}$、地址项 4B，一个索引块可放 1024 个块号，对应 $1024 \times 4\text{KB} = 4\text{MB}$ 的文件。
- 多级索引：文件特别大、一个索引块装不下时，用「索引块指向索引块」的方式。一级索引指向的块中，每项再指向一个索引块（二级索引）。$B = 4\text{KB}$、$p = 4\text{B}$ 时，二级索引最多可管理 1024×1024 个数据块，最大文件 $= 1024^2 \times 4\text{KB} = 4\text{GB}$。

  - 优点的另一半：可以支持更大的文件，加快对大型文件的查找速度；
  - 缺点：随着索引级数增加，访问磁盘的次数也在变多。
  - 还有一种链接索引方案：把多个索引块用指针链接起来，每个索引块包含指向下一个索引块的指针。
- 混合索引（**UNIX** 混合索引）：把直接地址、一级间接、二级间接、三级间接混在同一个 **inode** 中，全面照顾各种大小的文件。小文件用直接地址即可直接从 inode 取盘块号；中型文件用一级间接；大型、超大型文件用二级、三级间接。

索引分配的最大文件长度通用公式（再强调一次）

$$S_{max} = \sum_{k} N_k \cdot n^k \cdot B, \qquad n = \left\lfloor \frac{B}{p} \right\rfloor$$

其中 N_k 是第 k 级地址项个数、B 是块大小、p 是地址项大小、$k = 0$ 表示直接地址。

以 2010 年真题为例：inode 有 7 个地址项（4 个直接 + 2 个一级间接 + 1 个二级间接），地址项 4B，索引块与数据块均为 256B。则 $n = 256/4 = 64$，

$$S_{max} = 4 \times 256\text{B} + 2 \times 64 \times 256\text{B} + 1 \times 64^2 \times 256\text{B} = 1\text{KB} + 32\text{KB} + 1024\text{KB} = 1057\text{KB}$$

注意：二级间接里的两个索引层本身占的是索引块，不计入文件数据长度。

（三）三种物理结构的对比

对比项	连续分配	链接分配（隐式）	链接分配（显式/**FAT**）	索引分配
分配单位	一组连续盘块	离散盘块	离散盘块	离散盘块
FCB/inode 需保存	起始块号 + 总块数	起始块号 + 结束块号	起始块号（首块）	索引块地址（多级）
逻辑地址转换	起始 + 逻辑块号	沿链逐块读指针	查内存 FAT	查索引块（可能多级）
随机访问	支持	不支持	支持	支持
顺序访问速度	最快（寻道最少）	慢	快（FAT 在内存）	较快
文件扩展	困难（需大量移动）	容易	容易	容易
外部碎片	有	无	无	无
额外空间开销	无	每块一个指针	一张 FAT（可常驻内存）	每文件至少一个索引块
访问第 i 块需读盘（FCB/inode 已在内存）	1 次	$i+1$ 次（从 0 开始）	1 次	1 + 间接级数次
典型/最佳场景	长度基本不变、要求随机读写速度高	长度常变、只要求顺序访问	长度常变、要求随机读写	长度常变、要求随机读写

访问第 i 个逻辑块需要几次磁盘 I/O ——一份速查表

前提：**FCB / inode** 已在内存（否则还要先读盘装入 inode）。

- 连续分配：**1** 次（直接算出物理块号）。
- 隐式链接：$i+1$ 次（i 从 0 开始计，必须逐块读指针）。

- 显式链接（**FAT** 在内存）：**1** 次。
- 索引分配（单级）：**2** 次（索引块 + 数据块）。
- 索引分配（混合索引）：直接区 **1** 次；一级间接 **2** 次；二级间接 **3** 次；三级间接 **4** 次。

若题目说明「FCB 尚未读入内存」，则在上述次数上再加 1 次读 inode/目录项的 I/O。

物理结构的统考考法

- **2009-28**：适合随机访问且易于文件扩展的是索引结构。连续结构可视为数组——随机访问好但扩展差；链式结构可视为链表——扩展好但随机访问差；索引结构两者兼顾。
- **2013-24**：为支持 CD-ROM 中视频文件的快速随机播放，连续结构性能最好（顺序读取介质、连续块寻道最少）。
- **2010-30 / 2015-29 / 2018-31 / 2022** 大题：混合索引的最大文件长度与逻辑块定位。
- **2016** 大题：FAT 相关计算（表项与最大文件长度、I/O 次数）。
- **2013-26**：与单个文件长度无关的是「索引接点的总数」（只影响文件个数）。

六、目录的基本概念

文件目录是一种数据结构，用于标识系统中的文件及其物理地址，供检索时使用。对目录管理的基本要求是：实现「按名存取」、提高目录检索速度、允许文件重名、支持文件共享。

目录与 FCB/inode 的关系有三种表述要保持一致：

- FCB 的有序集合称为文件目录，一个 **FCB** 就是一个目录项；
- 引入 inode 后，目录项 = 文件名 + **inode** 编号（指针），inode 中存放元数据；目录项描述文件在文件系统中的逻辑位置，inode 描述文件在存储设备上的物理位置；
- 目录本身也是一种文件（目录文件），它也有自己的 inode 和数据块，只不过数据块里存的是目录项而不是普通数据。

因此可以记住：普通文件 = **inode** + 数据块；目录文件 = **inode** + 目录块（目录块中记录了该目录下各文件的名字与 inode 编号）。

常用的目录实现方式：

实现方式	原理	特点
线性列表	用数组或链表存放目录项	实现简单；数组适合静态目录，链表适合频繁增删；检索效率低，需线性扫描
哈希表	用散列函数把文件名映射到索引位置	直接按名访问速度快，适合目录项动态变化；容易浪费空间、冲突处理复杂
键树（Trie）	按文件名字符逐层建树	对大量共享前缀的名字检索极快；实现较复杂，空间需额外优化

七、树形目录

（一）目录结构演进

- 单级目录结构：整个系统只有一张目录表，每个文件占一个目录项。实现了按名存取，但不允许文件重名，查找速度慢，不适用于多用户系统。
- 两级目录结构：分为主文件目录（**MFD**）与用户文件目录（**UFD**）。一个文件系统一张 MFD，一

个 MFD 表项对应一个用户；每个用户一张 UFD，记录该用户所有文件的 FCB。解决了多用户环境下的重名问题，并提高了检索速度，但缺乏灵活性，不能对文件分类。

- 树形（多级）目录结构：最常用。用户可以创建自己的子目录，形成层次结构。
  - 优点：方便对文件分类，层次清晰，可对不同用户赋予不同存取权限，能有效管理和保护文件；
  - 缺点：需要按路径逐级访问中间节点，增加了磁盘访问次数，影响查询速度；不便于实现文件共享（这是引入无环图目录结构与链接的原因）。
- 无环图目录结构：在树形目录基础上增加指向同一节点的有向边，形成有向无环图。可以为不同文件名指向同一文件，甚至指向同一目录。需要为每个共享节点设置共享计数器 **count**，删除时只让计数器减 1，只有减到 **0** 才真正删除节点。
  - 注意：共享文件不同于复制文件——各用户指向的是同一个文件，一个用户修改后其他用户都能看到。

（二）绝对路径、相对路径与当前工作目录

- 文件路径名是由各级目录名与文件名用分隔符（/）连接而成的字符串。
- 绝对路径：从根目录出发的路径，例如 `/照片/2015-08/自拍.jpg`。
- 当前工作目录（当前目录）：由系统为每个进程设置的一个目录，是相对路径的起点。Linux 中用 `.` 表示当前目录，`..` 表示父目录。
- 相对路径：从当前工作目录出发的路径，例如当前目录是 `/照片`时，`./2015-08/自拍.jpg`。

设置当前工作目录的主要作用是加快文件的检索速度（2010-31 真题），它既不节省外存/内存空间，也不直接提高已定位文件的读写传输速度。

（三）路径名解析过程

以绝对路径 `/照片/2015-08/自拍.jpg` 为例：

1. 读入根目录的目录表（目录文件所在盘块）；
2. 在根目录中查找分量名「照片」，得到「照片」目录文件的存放位置（inode 编号），据此读入「照片」目录的目录表——**1** 次读盘；
3. 在「照片」目录中查找「2015-08」，读入该目录的目录表——**1** 次读盘；
4. 在「2015-08」目录中查找「自拍.jpg」，得到该文件的 inode 编号，读出 inode 与数据——总共约 **3** 次读盘。

若把「照片」目录设置为当前工作目录（即把它的目录表留在内存中），用相对路径`./2015-08/自拍.jpg` 访问，就省去了第 1 ~ 2 步的读盘，磁盘 **I/O** 次数减少，访问效率提高。

路径解析中的易错点

1. 每一级中间目录都要读盘（除非命中目录缓存），所以路径越长，开销越大。这正是引入目录项缓存（**dentry cache**）的原因。
2. 从根目录开始的路径一定是绝对路径；但相对路径的写法不一定要带`./`，如 `2015-08/自拍.jpg` 也从当前目录出发。
3. 路径解析的对象是目录项（文件名 → **inode** 编号），不是 **inode** 本身；只有文件名匹配成功后，才需要把 inode 读入内存。

八、目录的操作

统考要求掌握的目录操作：

操作	说明
搜索	根据文件名搜索目录，找到对应目录项
创建文件	在所属目录中增加一个目录项
创建目录	在树形目录结构中创建新的用户文件目录或子目录
删除文件	在目录中删除相应的目录项
删除目录	两种方式：① 不删除非空目录——必须先删除目录中所有文件并递归删除所有子目录；② 可删除非空目录——目录中文件与子目录一并删除
显示目录	列出目录内容（文件名及属性）
修改目录	目录项中保存了某些文件属性，属性变化时须修改相应目录项（如重命名）
移动目录	需要修改相应的路径名
链接目录	通过链接操作让指定文件具有多个父目录，便于共享（树形目录下每个文件/目录只允许有一个父目录，不适合共享）

重名检查与目录项复用

- 创建文件或目录时，必须先检查同一目录内是否已有同名文件/目录，确定不重名后才能建立。不同目录下的文件可以重名，这正是树形目录的优势。
- 删除目录项后，线性列表实现中有三种处理空闲目录项的方法：① 把目录项标记为不再使用（简单，但搜索需跳过）；② 把空闲目录项加入空闲目录项列表（便于复用，需额外结构）；③ 把目录的最后一个目录项复制到该空位并缩短目录长度（目录保持紧凑，但涉及数据移动）。

九、硬链接与软链接

文件共享指系统允许多个用户（进程）共享同一份文件，系统中只需保留该共享文件的一份副本。统考只要求两种实现方式：基于索引节点的共享（硬链接）和基于符号链的共享（软链接）。

（一）硬链接（**Hard Link**）

本质：多个目录项指向同一个索引节点，即多个文件名对应同一份文件数据。UNIX 中用 `ln` 原文件新链接名建立。

- **inode** 与目录项：每个链接都是一个正常的目录项（文件名 + inode 编号），所有目录项的 inode 编号相同，指向同一个 inode；**file** 数据只有一份。
- 链接计数 **count**：inode 中记录该文件被多少个目录项直接指向。创建文件时 count = 1；每建立一个硬链接，count +1；删除一个目录项，count −1；只有 **count** 减到 **0** 时才真正释放文件数据与 **inode**。
- 跨文件系统：不能。inode 编号只在同一个文件系统内唯一，每个文件系统有各自的 inode 表，硬链接无法跨越文件系统。
- 指向性：直接指向 inode，是「同一实体的多个名字」。
- 原文件删除后的行为：删除其中一个链接不影响其他链接对文件的访问；只要 count 仍大于 0，文件数据就仍可正常读写。因此硬链接不会产生悬空指针。
- 访问开销：从目录项直接得到 inode，一次到位，访问速度快。

count 与「删除」的正确理解

在硬链接下，每个拥有链接的用户都可以「删除」文件，但这个删除只是 `count--`；只有当执行 `count--` 之后 **count == 0** 时，系统才真正删除该文件。这就是「硬链接保证原文件被删除后仍能通过其他链接访问数据」的机制。

（二）软链接（符号链接，Symbolic Link）

本质：创建一个 **LINK** 类型的新文件，该文件有自己独立的 **inode** 和数据块，其文件内容就是目标文件的路径名。访问这个符号链时，操作系统截获请求，按其中的路径名去查找目标文件，再对它进行操作。相当于 Windows 的快捷方式。UNIX 中用 `ln -s 原文件新链接名`建立。

- **inode** 与目录项：软链接是独立文件，有独立的 inode 和至少一个数据块（存放路径名），不与目标文件共享 inode。
- 链接计数：建立软链接不改变目标文件的链接计数；软链接自己的计数为 1，删除软链接不影响目标文件。（若再对该软链接建立硬链接，则按普通文件规则各自加 1。）
- 跨文件系统：可以。因为路径名本质上是一串字符，可以指向任意文件系统、甚至网络上的其他机器。
- 指向性：指向路径名，是「指向名字的另一个名字」，属于间接指向。
- 原文件删除后的行为：软链接仍然存在，但已成为无效链接（悬空链接），通过它访问会失败；此时可将符号链删除。若文件主删除原文件的同时有另一用户创建了同名路径的新文件，则 LINK 文件仍然「有效」，但访问到的已不是原来那个文件。
- 访问开销：每次访问都要按路径名逐级查找目录，可能多次读盘，开销大；此外每条符号链本身也要占用一个 inode 和磁盘空间。

硬链接与软链接完整对比

对比项	硬链接	软链接（符号链接）
本质	多个目录项指向同一个 inode	一个独立文件，内容是目标文件的路径名
是否有自己的 inode	没有，与目标文件共用	有，独立 inode 与数据块
目录项内容	文件名 + 目标文件的 inode 编号	文件名 + 自己的 inode 编号（数据块中存路径）
目标文件的链接计数	每建一个硬链接 **+1**	不变
能否跨文件系统	不能	能，还可跨网络
指向性	直接指向 inode（同一实体）	间接，指向路径名
删除原文件后	其他链接照常访问；count 归零才真正删除	变成悬空链接，访问失败
谁能删除「共享文件」	任一拥有硬链接的用户都可使其 count 减 1	只有文件主能删原文件，其他用户只能删自己的 LINK 文件
访问开销	小，直接到 inode	大，需多次读盘逐级解析路径
是否产生悬空指针	不会	会产生（需在访问失败后清理）

链接相关真题

- **2009-31**：F1 计数值为 1，先建软链接 F2、再建硬链接 F3，然后删除 F1。F2 是独立文件，计数恒为 1；F3 与 F1 共享 inode，F1 删除后 count 由 2 减为 1。答案：**F2** 与 **F3** 的引用计数值都是 **1**。
- **2017-31**：f1 的硬链接为 f2，两进程分别打开，得 fd1、fd2。则「f1 和 f2 共享同一个内存索引节点」「fd1 和 fd2 分别指向各自的用户打开文件表中的一项」正确；「读写指针位置保持相同」错误——两个独立的 `open` 建立各自的打开文件表项，读写位置互不影响（只有 `fork`/`dup` 共享同一表项时才联动）。
- **2020** 相关：文件共享、打开文件的相关判断。

十、文件系统的全局结构

（一）文件系统在外存（磁盘）中的结构

磁盘通常被划分为一个或多个分区，每个分区中可以有一个独立的文件系统。整个磁盘与一个分区的典型布局如下：

区域	位置/内容	说明
MBR（主引导记录）	磁盘 0 号扇区	负责引导计算机；其后紧随分区表，记录各分区的起始与结束地址，并标记活动分区
引导块（**boot block**）	每个分区的第一个块	MBR 读取活动分区的第一个扇区即引导块，由其中的程序负责加载并启动该分区中的操作系统。即使该分区没装可启动系统，这一区域也保留（Windows 中称分区引导扇区）
超级块（**super block**）	分区内固定位置	存储文件系统的所有关键信息，系统启动或首次挂载时被读入内存
空闲空间管理信息	紧随超级块	通常以位示图或空闲链表实现，记录未分配的磁盘块
inode 区（**i** 节点表）	空闲空间管理信息之后	每个文件对应一个 inode，完整描述文件的属性与数据块地址
根目录	inode 区之后	整个目录树的起点
数据区（文件和目录）	磁盘剩余空间	存放所有其他文件与目录的实际数据

超级块的典型字段

超级块是文件系统的核心元数据结构，常见字段包括：

- 校验区：区分文件系统类型、检测文件系统合法性；
- **inode** 区起始块号、**inode** 区总块数；
- **bitmap** 起始块号、**bitmap** 总块数（若用两张位图，则分别对应 **inode** 位图与数据块位图）；
- 数据区起始块号、数据区总块数；
- 其他：磁盘块大小、**inode** 大小、根目录的 **inode** 编号、空闲块数目及指针、空闲 inode 数目及指针等。

超级块的内容是在逻辑格式化（对卷的目录区初始化）时写入的。它坏掉，整个文件系统基本就报废了——所以实际系统中超级块常有多份备份。

由超级块参数推导文件系统能力（完整算例）

设磁盘块大小 4KB，一个 inode 占 16B，inode 区起始块号为 6、总块数为 16，bitmap 起始块号为 22、总块数为 16。则：

1. 每个盘块可放的 **inode** 数 $= 4096/16 = 256$ 个；inode 区共 16 块，所以该文件系统最多可容纳 $256 \times 16 = 4096$ 个文件。
2. **inode** 编号为 **1314** 的 **inode** 位置：$\lfloor 1314/256 \rfloor = 5$，$1314 \bmod 256 = 34$，即位于 $6 + 5 = 11$ 号磁盘块，是该块中的第 34 个 inode。

3. 只读一个 **inode** 可以吗：不行，磁盘 I/O 以块为单位，必须把 11 号块整体读入内存，再取其中第 34 个。
4. 数据区最多有多少个磁盘块：由位图决定。若位图共 16 块、每块 4KB，则位图总位数为 $16 \times 4096 \times 8 = 524288$ 位，故数据区最多可管理 524288 个磁盘块。

（二）文件系统在内存中的结构

内存中的结构在挂载时加载、运行期间更新、卸载时释放：

内存结构	内容与作用
内存中的挂载表（**mount table**）	记录每个已挂载文件系统分区的信息：设备标识、挂载点、文件系统类型、挂载选项等
目录项缓存（**dentry cache**）	缓存最近访问过的目录内容（目录名与 inode 号的映射），加快路径解析，减少重复读盘
索引节点缓存（**inode cache**）	缓存最近访问过的 inode，多个进程可共享同一内存 inode
系统级打开文件表	每个当前打开的文件一个 FCB/inode 副本，含文件状态、访问权限、读写指针、打开计数等全局信息
进程级打开文件表	每个进程一张，含文件描述符、读写指针、访问模式，以及指向系统级打开文件表对应表项的指针；其起始地址记录在进程 PCB 中
页面高速缓存（**Page Cache**）	缓存文件的具体数据内容，减少磁盘 I/O；配合延迟写（**write-back**）把多次小写合并

内存结构 ≠ 外存结构

- 外存（磁盘）中有：MBR/引导块、超级块、空闲空间管理信息（位图/空闲链表）、inode 区、根目录、数据区。
- 内存中有：挂载表、目录项缓存、inode 缓存、系统/进程打开文件表、页高速缓存。

考纲条目里的「文件系统在外存中的结构」「文件系统在内存中的结构」，问的就是这两组结构不要混淆。

十一、外存空闲空间管理方法

先明确两个基本概念：

- 卷（**volume**）：文件系统的分区（如 C 卷、D 卷）。它可以是磁盘的一部分，也可以是整个磁盘或 RAID 磁盘阵列。从本质上讲，卷是逻辑磁盘，卷中的扇区在物理设备上不必连续。
- 卷的组成：目录区 + 文件区。目录区存放文件目录信息（FCB）和用于磁盘存储空间管理的信息；文件区存放文件数据。对逻辑卷格式化（初始化）的实质就是对卷内目录区进行初始化——建立空闲空间管理表格、超级块等。

（一）空闲表法

为所有空闲区建立一张空闲表，每个表项记录空闲区的第一个块号和该空闲区的块数，按磁盘块号顺序排列。这属于连续分配的思路。

- 分配：与内存动态分区分配类似，可采用首次适应、最佳适应、最坏适应等算法。系统为新文件分配空间时，顺序检索空闲表各表项，找到第一个大小满足要求的空闲区分配给它，并修改空闲表。

- 回收：同样类似动态分区回收，需分四种情况处理并合并相邻的空闲区：回收区前后都空闲、前后都不空闲、只有前面空闲、只有后面空闲。
- 适用场合：仅当空闲区数量较少时才有效。若存在大量小空闲区，空闲表会变得很大，查询效率很低，因此不适合大型文件系统。适用于建立连续文件。

外存为什么还用连续分配

内存分配很少采用连续分配，但在外存的一些特殊场景中，连续分配具有较高分配速度、能减少磁盘 I/O 频率与寻道时间：

- 磁盘的对换区，为追求换入换出速度采用连续分配；
- 文件系统中，小文件采用连续组织方式分配相邻盘块；
- 多媒体文件（MP3/MP4）为减少寻道时间采用连续组织方式。

（二）空闲链表法

把所有空闲盘区拉成一条空闲链，按链中基本元素的不同分为两种。

空闲盘块链：以单个盘块为单位组成链，每个空闲盘块中存放指向下一个空闲盘块的指针。

- 文件系统中需保存链头、链尾指针；
- 分配：若某文件申请 K 个盘块，从链头开始依次摘下 K 个盘块分配，并修改链头指针；
- 回收：回收的盘块依次挂到链尾，并修改链尾指针（每个回收块都要先读入内存修改指针再写回，磁盘 **I/O** 开销大）；
- 适用：适用于离散分配的物理结构；
- 优点：单个盘块的分配与回收操作极简单；缺点：为一个文件分配多个盘块需多次访问链表，效率低，且管理粒度细导致空闲盘块链通常很长。
- 最难获得连续的空闲磁盘空间。

空闲盘区链：以空闲盘区（一组连续盘块）为单位链接，每个盘区记录本盘区的盘块数和指向下一盘区的指针。

- 分配：若申请 K 个盘块，可采用首次适应等算法从链头开始检索，找到满足要求的空闲盘区分配；也可把不同盘区的盘块同时分配给一个文件，分配后可能要修改链指针与盘区大小；
- 回收：若回收区与某空闲盘区物理相邻则合并，否则作为单独空闲盘区挂到链尾；
- 适用：既适用于离散分配也适用于连续分配，为一个文件分配多个盘块时效率更高；
- 优点：分配回收效率较高，链较短；缺点：需处理盘区的分割与合并，过程较复杂。
- 最容易获得连续的空闲磁盘空间。

（三）位示图法

位示图（**bitmap**）用二进制的一位表示一个盘块的使用状态。统考沿用的约定是：「**0**」表示盘块空闲，「**1**」表示盘块已分配。若磁盘块共有 n 个，则位示图至少要有 n 位。

- 分配：顺序扫描位示图，找到 K 个「0」位（相邻或不相邻皆可），把这些位对应的盘块分配给文件，并把这些位置 **1**。
- 回收：根据回收的盘块号，把位示图中对应的位置 **0** 即可，不需要像链表那样修改指针，因此回收引起的磁盘 I/O 很少。
- 优点：查找高效，能快速找到单个或一组相邻的空闲盘块，便于实现连续分配；结构紧凑，占用空间极小，可常驻内存，避免频繁访问磁盘。占用空间与当前空闲块数量无关（只要磁盘容

量和块大小固定，位图长度就固定）。

- 缺点：位图大小随磁盘容量增大而增大，更适合小型或中等规模系统。

*1.*位示图字号/位号与盘块号的相互换算

设位示图有 m 行、每行 n 位（n 就是「字号」这一行的位数，实际系统里常常就是一个机器字长）。两种编号约定必须分清：

从 1 开始编号（王道教材默认约定）

行号 i、列号（位号）j 均从 **1** 开始，每行 n 位，则：

$$\text{盘块号 } b = n(i-1) + j$$

反向（由盘块号求行列号）：

$$i = (b-1)\ \text{DIV}\ n + 1, \qquad j = (b-1)\ \text{MOD}\ n + 1$$

其中 DIV 是整除，MOD 是取余。

从 0 开始编号

行号 i、位号 j 均从 **0** 开始，每行 n 位，则：

$$\text{盘块号 } b = i \times n + j$$

反向：

$$i = \left\lfloor \frac{b}{n} \right\rfloor, \qquad j = b \bmod n$$

做题时先看题目给的编号起点，再选公式。统考题目一般会明确写「盘块和块内字节均从 0 开始编号」之类的话。

算例 1：由盘块号求位示图位置（从 1 开始）

用 8 个字（字长 32 位）组成的位示图管理内存，共 8 行、32 列，行号与列号均从 1 开始，求块号 100 对应的位置。

$$i = (100-1)\ \text{DIV}\ 32 + 1 = 99\ \text{DIV}\ 32 + 1 = 3 + 1 = 4$$

$$j = (100-1)\ \text{MOD}\ 32 + 1 = 99\ \text{MOD}\ 32 + 1 = 3 + 1 = 4$$

即字号为 **4**、位号为 **4**。

算例 2：位示图所在盘块与块内字节（从 0 开始）

文件系统用位图法表示磁盘空间分配情况，位图存于磁盘的 **32～127** 号块中，每个盘块占 1024 字节，盘块和块内字节均从 **0** 开始编号。要释放的盘块号为 **409612**，求位图中要修改的位所在的盘块号和块内字节序号。

第一步，一个位图块能表示的位数：$1024 \times 8 = 8192$ 位。

第二步，位号即盘块号 409612（0 开始编号时直接对应）：

$$409612 \div 8192 = 50\ \text{余} 12$$

说明该位位于位图的第 **50** 个位图块（从 0 开始）。

第三步，换算为实际盘块号：位图起始于 32 号块，故

$$32 + 50 = 82$$

第四步，块内偏移 12 位，对应字节号：

$$\left\lfloor \frac{12}{8} \right\rfloor = 1$$

所以应修改 **82** 号盘块的第 **1** 个字节（该字节中的第 $12 \bmod 8 = 4$ 位，即从高位起的第 5 位，按位序编号约定略有差异，题目只要求到字节序号）。

算例 3：位示图自身占多大空间

- 容量 10GB 的磁盘分区，簇大小 4KB，用一位标识一个簇是否被分配，则簇数 $= 10 \times 2^{30}/2^{12} = 10 \times 2^{18} = 2621440$ 个，位图大小 $= 2621440/8 = 327680\text{B} = 320\text{KB}$；位图本身也按 4KB 的簇存放，故需 $320\text{KB}/4\text{KB} = \mathbf{80}$ 个簇。
- 页式存储管理中，页大小 4KB、物理内存 16GB，页框数 $= 2^{34}/2^{12} = 2^{22}$，位图大小 $= 2^{22}/8 = 2^{19}\text{B} = \mathbf{512}\text{KB}$。

这两个例子说明：位图大小 = 被管理对象总数 / **8** 字节。

位示图法最常错的两处

1. 位示图中 **0** 和 **1** 的含义。统考约定 0 = 空闲、1 = 已分配；但内存的位示图（页框分配）在某些教材里相反。做题时以题目说明为准。
2. 位图的「位」与「字节」「字」的换算。题目常给「每个盘块 1024 字节」，此时一位图块是 $1024 \times 8 = 8192$ 位，不是 1024 位。

（四）成组链接法

空闲表法和空闲链表法都不适用于大型文件系统——管理结构会随磁盘容量增长而过度膨胀。**UNIX** 系统采用成组链接法，它融合了两者的思想，有效控制了管理结构的规模。

基本思想：把所有空闲盘块划分成若干组（例如每组 100 个盘块）。除最后一组外，每组的第一个盘块用作索引块，记录下一组中所有空闲盘块的块号及其数量；这些索引块依次链接成一条链。第一组的空闲盘块信息（块号列表和数量）在文件系统挂载时被预先加载到内存的一个专用栈结构，称为空闲盘块号栈。

经典例子：空闲区为 201 ~ 7999 号盘块，每组 100 块，则分组为：第一组 201 ~ 300、第二组 301 ~ 400、……、次末组 7801 ~ 7900、最末组 7901 ~ 7999（最末组只有 99 块）。

- 空闲盘块号栈中，超级块里的结构是：空闲盘块数 **N** + 各空闲盘块号；第一组 201 ~ 300 中，**300** 号盘块作为索引块，记录下一组（301 ~ 400）的块号列表。
- 最末组的块号由前一组的最后一个盘块（7900 号）记录，而该盘块中存放的第一个盘块号为「**0**」，用作整条空闲链的结束标志。

成组链接法的分配过程

1. 从空闲盘块号栈顶取出一个盘块号分配给用户，栈指针上移，栈中空闲盘块数减 1；
2. 若栈指针到达栈底（表示当前组已分配完），则需要读取该栈底盘块号所对应的物理盘块内容——其中保存的是下一组的空闲盘块号列表；
3. 先把该内容整体读入空闲盘块号栈以更新它，再把原栈底所指的盘块（其索引作用已完成）分配出去。

以图例说明：系统先依次分配 201 ~ 299 号盘块；当需要分配 300 号盘块时，首先把 300 号盘块的内容（下一组的块号列表）读入空闲盘块号栈，随后把 300 号盘块本身分配给用户。

关键结论：只有当 **0** 号分组（栈中这一组）的空闲盘块分完，才需要进行一次磁盘 **I/O**。

成组链接法的回收过程

1. 系统把释放的盘块号压入空闲盘块号栈顶，栈指针下移，栈中空闲盘块数加 1；
2. 若此时栈已满（达到上限，如 100 项），则需要把当前栈中的全部盘块号写入新回收的这个盘块，使其成为新的索引块；
3. 随后把该盘块号设为新的栈底，并把栈中记录的空闲盘块数重置为 **1**。

关键结论：只有当栈满（**0** 号分组达到上限）时才需要一次磁盘 **I/O**。

两个必背问答：

- 若 0 号分组中仅剩唯一一个索引块，进行一次空闲盘块分配后：该索引块的内容被复制到超级块中，然后该索引块被分配给文件使用。
- 若 0 号分组的盘块数量已达上限，回收一个盘块后：把新回收的块作为新分组的索引块，并把原 **0** 号分组（栈）的信息复制过去。

成组链接法小结

- 描述空闲空间的数据结构（空闲盘块号栈）以及文件系统的关键信息，都存放在磁盘的固定位置；在 UNIX 中这一区域称超级块。
- 挂载时把超级块读入内存，并在运行过程中持续维护内存副本与磁盘副本的一致性。
- 成组链接法通过「分组 + 索引 + 链接」的设计，既避免了长链表的遍历开销，又无须维护完整的空闲表，在可扩展性与性能之间取得了良好平衡。
- 它可以得到连续的空闲磁盘空间，也可得到离散的空闲磁盘空间。

FAT 也算一种空闲空间管理结构

显式链接中的 **FAT** 记录了所有盘块。若某盘块不属于任何文件，就把它的表项标记为 `-2` 表示空闲；分配时遍历查找值为 `-2` 的表项即可，回收时把表项置回 `-2`。所以 2019 年真题中 FAT 被算作可用于管理空闲磁盘块的数据结构。而索引节点不是——它描述的是已存在文件的元数据与数据块地址。

十二、虚拟文件系统

（一）为什么需要 VFS

现代操作系统通常要支持多种文件系统（ext4、NTFS、FAT 等），它们在磁盘组织方式、元数据格式和操作方法上各不相同。若应用程序为每种文件系统分别编写代码，开发困难且系统可维护性差。虚拟文件系统（**Virtual File System, VFS**）位于用户程序与具体文件系统之间，是一个内核软件层、一个抽象层：

- 向上：提供统一的文件访问接口（标准的系统调用 `open`、`read`、`write`、`close` 等），使程序无须关心底层是哪种文件系统、存储在何种设备上；
- 向下：兼容并屏蔽各种不同类型文件系统的差异。

VFS 之所以能衔接各种文件系统，是因为它定义了所有文件系统都支持的、基本的、概念上的接口和数据结构。对于任何特定文件系统，都需要一个映射模块把它自己的特征转换为 VFS 期望的特征。一个新的文件系统要在某操作系统上被使用，就必须满足该操作系统 **VFS** 的规定：必须实现 VFS 规定的「接口函数集」（打开、读写、关闭等操作的函数），且函数的参数格式、返回值规则必须严格符合 VFS 要求。例如 ext4 与 FAT32 的「读文件」实现逻辑完全不同，但输入（文件标识、

缓冲区地址、长度）和输出（实际读取字节数）必须统一。

（二）VFS 的四大对象

VFS 采用面向对象的设计思路，用 C 语言的结构体实现「对象」——每个对象包含数据成员和一组函数指针。四个主要对象是：

对象	代表什么	关键内容	生命周期与存储位置
超级块对象（**superblock**）	一个特定的已挂载文件系统	文件系统类型、大小、所挂载的设备、操作方法（函数）指针、空闲块数目与索引表、空闲 inode 数目与索引表、封锁标记等	内核初始化并注册一个文件系统时在内存分配；**VFS** 超级块只存在于内存中，在文件系统卸载时被自动删除。具体文件系统的超级块（如 ext2 superblock）存放在磁盘上
索引节点对象（**inode / vnode**）	一个特定的文件或目录（物理属性）	文件字节大小、所有者、读/写/执行权限、时间戳、文件类型、数据块位置	具体文件系统的 inode 静态存放于磁盘；使用时调入内存并填写 VFS 的索引节点（**vnode**），因此 vnode 也称动态节点。同一文件系统内 inode 号唯一
目录项对象（**dentry**）	路径中的一个组成部分（目录名或文件名）	文件名、指向父/子目录项的指针、指向对应 vnode 的指针	不对应磁盘上的固定结构，是 VFS 在解析路径时动态生成的内存缓存，组成逻辑目录树
文件对象（**file**）	一个与进程相关的已打开文件	当前读/写位置（文件指针）、访问模式、引用计数，指向对应 vnode 与 dentry 的指针，以及 read/write/seek 等操作接口	进程调用 `open()` 时创建，调用 `close()` 且引用计数归零时销毁。同一文件可同时存在多个文件对象（多个进程各打开一次），但对应的 inode 和 dentry 是唯一的

VFS 四个对象的一句话对照

- 超级块是对一个文件系统的描述；
- 索引节点是对一个文件物理属性的描述；
- 目录项是对一个文件逻辑属性的描述；
- 文件对象是对当前进程打开的文件的描述。

另外注意：普通文件 = **inode** + 数据块；目录文件 = **inode** + 目录块（目录块中记录该目录下各文件的文件名与 inode 编号）。

vnode 不等于 inode

vnode 只存在于主存中，是 VFS 为统一管理各类文件而创建的内存对象；**inode** 既会被调入主存，也在外存中存储（磁盘 inode 是静态的）。两者通过指针关联协同工作：**vnode** 提供统一接口，**inode** 提供实际存储信息。

（三）VFS 的工作过程（以 write 为例）

当进程调用 `write(fd, buf, n)` 时：

1. 内核进入 **VFS** 层，执行 `sys_write()`；
2. 根据文件描述符 fd 定位到对应的文件对象；
3. 通过文件对象获取其关联的 **vnode**；

4. VFS 利用 vnode 中预置的写操作函数指针，调用目标文件系统的具体写方法；
5. 该文件系统把数据写入缓冲区，最终通过设备驱动完成磁盘写入。

因此，对用户程序而言，所有文件操作都通过 VFS 提供的统一接口完成，无须关心底层是 ext4、NTFS 还是其他文件系统。**VFS** 本身并不是一种真正的文件系统——它仅存在于内存之中，不占用磁盘空间，随系统启动而建立，随系统关闭而释放。

VFS 使用的缓存

- 索引节点缓存：每个目录和文件都由一个 vnode 表示，缓存最近访问过的索引节点，加快访问；
- 目录缓存（**dentry cache**）：缓存完整的目录名与其 inode 号之间的映射，加快列出目录与路径解析；
- 页面高速缓存（**Page Cache**）：存放文件的具体数据，需要相同数据时直接从内存读取而不访问磁盘。

十三、文件系统挂载

如同文件在使用之前必须先打开一样，文件系统在被进程访问前也必须先安装，也称挂载（**Mounting**）。

挂载的定义：把一个存储设备（或其分区）上的文件系统「连接」到操作系统目录树中的某个目录上，使其内容可以被访问。用于挂载的那个目录称为挂载点（**mount point**）。

核心要点：

- UNIX/Linux 中所有文件和设备都组织在一个统一的「目录树」中，根目录是 `/`；
- 硬盘、U 盘、网络存储上的数据默认不会自动出现在目录树里；
- 必须通过挂载操作把设备上的文件系统「嫁接」到目录树的某个挂载点（通常是空目录），才能访问其中的文件；
- 未挂载的设备 = 存在但不可见/不可用。

（一）挂载的过程

1. 识别设备：如 `/dev/sdb1`（U 盘第一个分区）、`/dev/nvme0n1p2`（SSD 分区）；
2. 识别文件系统类型：如 ext4、FAT32、NTFS、XFS、Btrfs；
3. 选择挂载点：一个已存在的空目录，如 `/mnt/usb`、`/media/disk`；
4. 执行挂载命令：如 `mount /dev/sdb1 /mnt/usb`；
5. 内核注册：VFS 把该设备的文件系统纳入统一管理，用户即可通过挂载点访问文件。

不同文件系统挂载时内核做的事不同：

文件系统	挂载时内核的动作
FAT	① 读取引导扇区（位于分区开头，偏移 0），获得每扇区字节数、每簇扇区数、保留扇区数、FAT 表个数、根目录项数或根目录起始簇、总扇区数、FAT 表大小等元数据（作用与 UNIX 超级块类似）；② 把第一份 **FAT** 表从磁盘整份读取到内存（因为查 FAT 极其频繁，必须常驻内存）；③ 读取根目录，填充 dentry 缓存；④ 完成挂载
ext4	① 读取超级块；② 读取块组描述符（ext4 把磁盘划分为多个块组，每组有自己的数据块位图、inode 位图和 inode 表）；③ 只读取根目录 `/` 的 **inode**（ext4 不会把所有 inode 都读入内存，其他文件/目录的 inode 在被实际访问到时才按需读入）；④ 完成挂载

挂载完成后的文件访问路径为：

- FAT：路径名 → 目录项 → 获取起始簇号 → 通过内存中的 FAT 表查找后续簇 → 读取数据；
- ext4：路径名 → dentry → inode → 通过 inode 中的块指针（直接/间接/extent）定位数据块 → 读取数据。

挂载点的选择与语义

- 并不是根目录下任何目录都可以作为挂载点。挂载操作会使原有目录中的内容被隐藏，因此根目录以及系统原有目录都不要作为挂载点，否则可能造成系统异常甚至崩溃；挂载点最好是新建的空目录。
- 当一个文件系统被挂载到某个目录上时，在该目录下创建的文件和目录都会被存储在该文件系统中，而不是原来所在的文件系统。
- 挂载后，访问该目录 = 访问设备上的文件系统。
- 挂载后原挂载点目录的内容会被「遮盖」，但不会被删除；卸载（`umount`）后原内容会重新出现。
- 同一个设备可以被挂载到多个不同的挂载点；但同一挂载点在同一时刻只能挂载一个设备。

挂载相关考法

- 2025 年统考考查过虚拟文件系统的原理与特点（VFS 的抽象层作用、四种对象、vnode 与 inode 的区别）。
- 挂载常与「文件系统在内存中的结构」合并出题：挂载时读入什么（超级块、FAT/块组描述符、根目录 inode），什么留在磁盘（其他 inode、数据块）。
- 卸载（umount）时必须把内存中被修改的超级块等元数据写回磁盘（保证一致性），这也是一道常见选择题的选项来源。

本章小结表

知识点	核心结论
数据项/记录/文件	基本数据项 → 组合数据项 → 记录 → 文件，自底向上
FCB	文件与 FCB 一一对应；FCB = 一个目录项；FCB 有序集合 = 文件目录；实现按名存取
inode	FCB 去掉文件名后的部分；目录项 = 文件名 + inode 编号；节省目录检索的磁盘 I/O
磁盘/内存 inode	磁盘 inode 静态；内存 inode 增加编号、状态、访问计数、逻辑设备号、链接指针
inode 最大文件长度	$S_{max} = \sum_k N_k \cdot n^k \cdot B$，$n = \lfloor B/p \rfloor$；索引块不计入数据长度
inode 位图 / 数据块位图	分别标记 inode 与数据块是否已分配；0 空闲、1 已分配
建立文件	分配空闲 inode + 在父目录中新增目录项
删除文件	删除目录项；count = 1 时释放数据块与 inode，否则 count−−；不删除所在目录
打开文件	路径解析 → 载入 inode → 检查权限 → 系统打开文件表（count+1）→ 进程打开文件表 → 返回 fd
关闭文件	删进程表项 → 系统表项打开计数 −1 → 为 0 时删除表项并视情况写回 inode
读/写文件	经内核缓冲区中转；磁盘按块读写；读写指针在进程打开文件表中
文件保护	访问控制矩阵、ACL（可精简为拥有者/组/其他）、口令、加密；权限位 rwx
权限位数计算	用户类别数 × 权限种类数（同类别可同时拥有多种权限）
逻辑结构	顺序文件、索引文件、索引顺序文件（约 $\sqrt{n}$ 次查找）、直接/散列文件
连续分配	起始块号 + 总块数；支持随机访问；外部碎片；扩展困难

知识点	核心结论
隐式链接	指针在盘块尾部；只能顺序访问；第 i 块需 $i+1$ 次读盘
显式链接 FAT	指针集中在 FAT 且常驻内存；支持随机访问；可用 -2 标记空闲块
索引分配	每文件一张索引表；单级/多级/混合（UNIX 13 个地址项）
目录结构	单级（不允许重名）、两级（MFD+UFD）、树形（绝对/相对路径）、无环图（共享计数器）
当前工作目录	作用是加快文件检索速度
硬链接	共用 inode、count+1、不能跨文件系统、原文件删除不影响、访问快
软链接	独立 inode、存路径名、不改 count、可跨文件系统、原文件删除后悬空、访问慢
外存结构	MBR/分区表、引导块、超级块、空闲空间管理信息（位图）、inode 区、根目录、数据区
内存结构	挂载表、目录项缓存、inode 缓存、系统级打开文件表、进程级打开文件表、页高速缓存
空闲表法	记录起始块号与块数；连续分配；适合空闲区少的场合
空闲盘块链	以单块为单位链接；最难获得连续空间
空闲盘区链	以盘区为单位链接；最容易获得连续空间
位示图法	一位一块；0 空闲 1 已分配；1 起编号 $b=n(i-1)+j$；0 起编号 $b=in+j$
成组链接法	UNIX 采用；超级块中的空闲盘块号栈；分组 + 索引 + 链接；只在栈空/栈满时 I/O
VFS	内核抽象层；四对象：超级块、索引节点（vnode）、目录项、文件；vnode 只在内存
挂载	把设备文件系统接到目录树的挂载点；挂载后访问目录即访问该设备；原目录内容被遮盖

易错清单

1. 概念混淆类

- **FCB 与 inode**：FCB 是完整控制块；inode 是 FCB 去掉文件名后的部分。目录项在 FCB 方案中就是 FCB，在 inode 方案中是「文件名 + inode 编号」。
- 目录项与 **inode**：目录项描述文件的逻辑位置（在文件系统中的位置），inode 描述文件的物理位置（在存储设备上的位置）。
- 磁盘 **inode** 与内存 **inode**：内存 inode 多了编号、状态、访问计数、逻辑设备号、链接指针；内存 inode 可被多进程共享。
- **vnode** 与 **inode**：vnode 只在内存中、是 VFS 的统一封装；inode 磁盘上也有。
- 文件系统在外存/内存中的结构：位图、超级块、inode 区在外存；挂载表、各类缓存、打开文件表在内存。

2. 打开文件表类

- 一个文件在系统打开文件表中只有一个表项，无论多少进程打开它；每个进程的打开文件表各有一项。
- 读写指针在进程级打开文件表中（各进程独立）；不同进程的 fd 数值也不相同。
- **open** 不一定读盘：若 inode 已在内存，只需打开计数 +1 并新建进程表项。
- 关闭 $\neq$ 删除：关闭不删除目录项、不释放外存 inode、不修改磁盘 inode 的链接计数。
- 删除文件不删除所在目录，也不必删除该文件的快捷方式/符号链接。
- **open** 返回的是文件描述符（进程打开文件表下标），不是文件内容、不是缓冲区指针。

3. 计算类

- **inode** 最大文件长度：$n = \lfloor B/p \rfloor$，逐级算 $N_k n^k B$ 再求和；索引块不计入文件数据长度；题目可能既有「数据块大小」又有「索引块大小」，一般二者相同，若不同要分别代入。
- 访问第 i 块需要几次读盘：连续 1 次、隐式链接 $i+1$ 次、显式链接 1 次、单级索引 2 次、混合索引按所在区 1/2/3/4 次；前提是 inode 已在内存，否则各加 1。
- 位示图换算：先确认编号从 0 还是从 1 开始；一位图块的位数 = 块字节数 × 8；位图大小 = 被管理对象数 / 8 字节。
- **FAT** 表大小：表项数 = 盘块数 = 容量/块大小；表项位数 = $\lceil \log_2$ 盘块数$\rceil$；FAT 空间 = 表项数 × 表项位数。
- 连续分配的插入/删除 **I/O** 次数：先判断往哪边移动代价小，读 + 写各算一次。
- 权限位数：用户类别数 × 权限种类数，不要压缩成「每类用户 2 位」。
- 索引顺序文件平均查找次数：最优分组 $i = \sqrt{n}$，总次数约 $\sqrt{n}$；多级索引 $(K+1)/2 \cdot N^{1/(K+1)}$。

4. 结构与特性判定类

- 随机访问能力：连续分配支持；隐式链接不支持；显式链接（FAT 常驻内存）与索引分配支持。
- 扩展能力：连续分配差；链接与索引分配好。
- 外部碎片：连续分配有；链接与索引分配没有。
- 最适合随机读写且易扩展：索引结构（2009-28）；**CD-ROM** 视频快速随机播放最优：连续结构（2013-24）。
- 哪个与单个文件长度无关：inode 总数（2013-26，它只决定文件个数）。
- 可用于管理空闲磁盘块的结构：位图、空闲磁盘块链、空闲盘区链、空闲表、FAT、成组链接法；**inode** 不行。
- 占用空间与当前空闲块数量无关的：位图法（2024-26）；空闲表、空闲链表、成组链接法都与当前空闲块数量有关。

5. 目录与链接类

- 当前工作目录的作用是加快文件检索速度，不是省内存/省外存/加快读写传输速度。
- 每一级中间目录都要读盘（除非命中目录缓存），这是路径解析开销的来源。
- 硬链接不能跨文件系统（inode 号只在文件系统内唯一）；软链接可以跨文件系统甚至跨网络。
- 建立硬链接使 **count+1**；建立软链接不改目标文件的 **count**。
- 删除硬链接只是 **count−−**，**count** 归零才真正删除；删除原文件后软链接变悬空。
- 硬链接访问快、软链接访问慢（需重新按路径逐级查找目录）。
- 树形目录不便于文件共享，共享要靠无环图目录结构或链接；无环图需要共享计数器，计数器归零才删除节点。

6. 成组链接法类

- 空闲盘块号栈放在超级块中，挂载时读入内存并常驻，必须保持内存副本与磁盘副本一致。
- 分配时栈空才需要 **I/O**（把栈底盘块中的下一组信息读入栈，再把该盘块分出去）；回收时栈满才需要 **I/O**（把栈中内容写入新回收的盘块，让它成为新索引块）。
- 最末组的结束标志是「**0** 号盘块」，表示链的结束。
- 一个分组的第一块用作索引块，它记录下一组的块号与数量，它本身也是空闲块，最终会被分配出去。

第十八章输入/输出管理

本章考纲要求

第三部分操作系统五、输入/输出（I/O）管理（一）I/O 管理基础

1. 设备：设备的基本概念，设备的分类，I/O 接口，I/O 端口。
2. I/O 控制方式：轮询方式，中断方式，DMA 方式。
3. I/O 软件层次结构：中断处理程序，驱动程序，设备独立软件，用户层 I/O 软件。
4. 输入/输出应用程序接口：字符设备接口，块设备接口，网络设备接口，阻塞/非阻塞 I/O。

（二）设备独立软件

1. 缓冲区管理
2. 设备分配与回收
3. 假脱机技术（SPOOLing）
4. 设备驱动程序接口

（三）外存管理

1. 磁盘：磁盘结构，格式化，分区，磁盘调度方法。
2. 固态硬盘：读写性能特性，磨损均衡。

本章是操作系统部分「考点分散但命题直接」的一章，与计组第六章（总线和输入/输出系统）大量重叠。区别在于：计组关心 **I/O** 硬件与控制时序，操作系统关心 **I/O** 软件的分层、设备的分配与调度、外存的存取时间计算。统考在这章的出题形态很固定，主要有四类：

- 层次与流程排序题：I/O 软件四层的顺序、一次 I/O 请求的处理流程、DMA 传输各阶段的先后顺序。
- 对比判断题：中断与 DMA、单缓冲与双缓冲、阻塞 I/O 与非阻塞 I/O、各种磁盘调度算法的优缺点。
- 时间与距离计算题：单/双缓冲处理 n 块数据的总时间、CPU 用于轮询的开销占比、磁盘访问时间、各调度算法的磁头移动总距离。
- 应用题：以键盘输入或打印输出为例，写出从用户程序发起系统调用到数据落地的完整过程（2019、2023 年都出现过）。

复习策略是把「四层结构 + 三种控制方式 + 五个磁盘调度算法 + 两个缓冲公式」这四根主线背死，其余细节挂在主线上理解。

一、I/O 管理基础

（一）设备

*1.*设备的基本概念

I/O 设备是能够向计算机输入数据或接收计算机输出数据的外部设备。一个完整的 I/O 设备由两部分组成：

- 设备本体：执行具体输入/输出操作的物理装置，如磁盘的盘片与磁头、打印机的机械部分。
- 设备控制器（**I/O** 接口）：连接在系统总线上的专用电路，负责把操作系统发出的抽象命令（如「读取第 x 个扇区」）转换为设备可识别的电信号，并协调主机与设备之间的数据传送。

把这两部分分开的意义在于：操作系统只需要面对统一规格的控制器，不必了解成千上万种设备的具体机械细节。设备本体的差异被控制器封装掉了，这正是整个 I/O 软件能分层的前提。

*2.*设备的分类

设备的分类方法有四种，考纲要求的是前三类中的传输速率、信息交换单位和共享属性。

按传输速率分类：

类别	传输速率	典型设备
低速设备	每秒几字节至数百字节	键盘、鼠标
中速设备	每秒数千字节至数十千字节	激光打印机
高速设备	每秒数百千字节至数百兆字节以上	磁盘、光盘、网卡

按信息交换单位分类（最重要，直接决定 I/O 接口的类型）：

- 块设备（**block device**）：以固定大小的数据块为单位进行信息交换，如磁盘、固态硬盘、U 盘、光盘。基本特征是传输速率较高、可寻址（支持随机访问），可以对任意数据块直接读写。块设备的 I/O 通常采用 **DMA** 方式。
- 字符设备（**character device**）：以字符（字节）为单位进行信息交换，如键盘、鼠标、打印机、串口。基本特征是传输速率较低、不可寻址，只能顺序访问，I/O 通常采用中断驱动方式。大多数字符设备属于独占设备，因此字符设备接口必须提供「打开」和「关闭」操作以获取和释放访问权限。

块设备「隐藏了磁盘的二维结构」

磁盘物理上要用「柱面号—盘面号—扇区号」三维地址定位，但块设备接口把磁盘的所有扇区按数据块排成一个线性序列，上层只需给出「第 n 号逻辑块」即可。这个「二维变一维」的抽象是文件系统能在磁盘上建立的前提，也是块设备接口与字符设备接口最本质的区别。

按共享属性分类（决定设备分配策略）：

- 独占设备：任一时刻只允许一个进程使用。一旦分配给某进程，便由该进程独占，直至其主动释放或终止。典型的独占设备多为低速设备，如打印机、磁带机。独占设备采用静态分配方式。
- 共享设备：允许在一段时间内由多个进程宏观上同时访问，微观上交替使用。磁盘是典型的共享设备。共享设备必须是可寻址、可随机访问的设备，并且采用动态分配方式。
- 虚拟设备：通过 **SPOOLing** 技术把独占设备改造为多个逻辑设备，使多个进程能并发使用。其实质是用高速共享设备（如磁盘）作缓冲区，实现独占设备的逻辑共享。

「宏观同时、微观交替」的准确含义

共享设备是「一段时间内可同时访问」，不是「同一时刻」。物理上一个磁盘的磁头在同一瞬间只能在一个位置，它不可能同一时刻既为进程 A 读文件 X、又为进程 B 读文件 Y。操作系统把多个请求排入队列，用磁盘调度算法快速切换服务，因为切换速度远快于人的感知，所以宏观上表现为「同时共享」。「共享设备可以同一时刻被多个进程访问」是错误说法。

3.I/O 接口的功能与组成

I/O 接口（**I/O interface**），又称设备控制器（**device controller**）或 **I/O** 适配器，是 CPU 与设备之间的桥梁。它接收 CPU 发来的命令、控制设备工作，使 CPU 从繁杂的设备控制事务中解脱出来。

主要功能：

1. 接收和识别命令：接收 CPU 发来的控制命令，对命令字进行译码，产生相应的控制信号。
2. 数据交换：实现 CPU 与控制器之间、控制器与设备之间的数据传送。
3. 标识和报告设备状态：保存设备的状态信息（忙、就绪、错误）供 CPU 查询。
4. 地址识别：识别 CPU 要访问的是哪个设备、哪个寄存器。
5. 数据缓冲：用数据缓冲寄存器弥补 CPU 与设备之间的速度差异。
6. 差错控制：对传送的数据进行校验（如 CRC），发现并报告错误。

基本组成：

部件	作用
数据缓冲寄存器（**DBR**）	缓冲数据，匹配 CPU 与外设的速度差异。输入时暂存设备送来的数据，输出时暂存 CPU 送来的数据
状态寄存器	反馈设备状态（忙、就绪、错误）。程序查询方式中 CPU 靠读它判断设备是否就绪
控制寄存器（命令寄存器）	接收 CPU 发来的控制命令（复位、识别、读写控制等）
设备存储器	非必需部件，用于设备自身的运算处理，如显卡的显存
地址译码器与 **I/O** 控制逻辑	对命令字译码，产生控制信号送往外设；收集外设状态写入状态寄存器

状态寄存器与控制寄存器可以合并

状态寄存器和控制寄存器在传送方向上是相反的（状态寄存器是设备 →CPU，控制寄存器是 CPU→ 设备），而且 **CPU** 对它们的访问在时间上一般是错开的（先写命令，再等状态，不会同时进行），因此有的 I/O 接口把它们合并成一个「控制/状态寄存器」，用同一组地址、靠读写方向区分。

与 **I/O** 接口相连的三类总线都是双向的，它们传输的内容要分清：

- 数据总线：真正的 I/O 数据、CPU 写入控制寄存器的命令字（控制字）、CPU 读出的状态字。三者都经数据总线传输。
- 地址总线：I/O 端口地址（供地址译码器选中寄存器）、DMA 时接口作为主设备发出的主存地址。
- 控制总线：读/写信号、时钟与总线控制权协商信号（DMA 请求 HOLD、授权 HLDA）、应答信号、中断响应信号。

数据线上能传什么

统考考过「在 I/O 总线的数据线上传输的信息包括：I/O 接口中的命令字、状态字、中断类型号」。答案是三者都包括。要特别注意中断类型号也走数据线，但它不是设备控制器发出的，而是可编程中断控制器

在中断请求判优之后生成的。另外要区分「通过数据线写入控制寄存器的命令字」与「通过控制总线发送的控制信号」：前者是一个具体的数据值（回答「做什么」），后者是时序与协调信号（回答「怎么传」）。

4.I/O 端口

I/O 端口（**I/O port**）指设备控制器中的数据缓冲寄存器、状态/控制寄存器等的统称。数据缓冲寄存器简称数据端口，状态/控制寄存器简称状态/控制端口。所有 I/O 端口编号组成的空间称为 **I/O** 地址空间。

对数据端口、控制/状态寄存器的访问通过 **I/O** 指令完成。例如 IA-32 提供 `in`、`ins`、`out`、`outs` 四条专门指令，其中 `in`/`ins` 把控制器中寄存器的内容取到 CPU 通用寄存器，`out`/`outs` 把通用寄存器的内容送到控制器寄存器。这些 **I/O** 指令只能在操作系统内核的底层 **I/O** 软件中使用，因此属于特权指令。

I/O 端口的编址方式有两种：

比较项	统一编址（内存映射 **I/O**）	独立编址（**I/O** 映射方式）
概念	把 I/O 端口当成主存单元一起编址	I/O 端口地址与存储器地址无关
访问方式	不需要专门的 I/O 指令，用统一的 `LOAD`/`STORE` 访存指令即可	需要专门的输入输出指令，如 x86 用 `IN`/`OUT` 访问端口、用 `MOV` 访存
优点	端口编址空间大；读写控制逻辑简单；**I/O** 保护可由分段或分页存储管理实现，无需专门的保护机制	用专用 I/O 指令，程序清晰便于编程；端口地址位数少，地址译码速度快；不占用主存地址空间
缺点	占用主存地址空间，使主存地址空间变小；地址位数多，译码速度慢	I/O 指令类型少，一般只能做传送操作，程序设计灵活性差；需要 CPU 提供存储器读写与设备读写两组控制信号，控制逻辑复杂
如何区分 I/O 与主存	靠不同地址码区分（看地址落在哪个范围）	靠不同指令区分；由于二者独立编址，部分地址空间会重叠，因此用不同指令生成不同的总线控制信号来标识
典型处理器	MIPS、ARM 等 RISC 处理器	Intel x86 处理器

统一编址的 I/O 端口不能被 Cache 缓存

I/O 地址虽然映射到了主存空间，但接口中的数据是动态变化的，如果被 Cache 缓存，CPU 读到的将是过时的副本，无法了解设备的实时状态。因此统一编址的系统必须把 I/O 区间标记为「不可缓存」。这是统一编址方式下一个容易被忽略的陷阱。

（二）**I/O** 控制方式

I/O 控制方式的发展主线只有一条：尽量减少主机对 **I/O** 控制的干预，把主机从繁杂的 **I/O** 控制事务中解脱出来。从程序查询到中断驱动，再到 DMA 与通道，每一次进步都是让 CPU 少管一点、让硬件多管一点。

*1.*轮询方式

轮询方式（程序查询 / 程序直接控制方式）指输入输出完全依靠 CPU 执行程序实现。CPU 要与设备交换数据时，先设置接口的命令寄存器启动设备，然后在设备准备数据的过程中反复读取接口中的状态寄存器查询设备是否就绪，根据查询结果决定是继续等待还是传送数据。

以简单键盘输入为例：按键按下后，键盘控制器把键值放入数据缓冲寄存器 DBR，并把状态寄存器 DSR 的「数据就绪 Ready」位置 1。CPU 需要取键值时，先读 DSR 并判断 Ready 位，为 0 就继续查询，为 1 才读 DBR，读完后 Ready 位自动清 0。

对于复杂设备，流程多一步：CPU 先查询设备状态，就绪后才向 I/O 接口发送命令与参数启动设备，此后设备进入可能较长的设备准备阶段，CPU 则不断查询状态，就绪时才进行实际的数据传输。

轮询有两种策略：

- 独占查询（忙等待）：设备一旦启动，CPU 就一直连续查询，外设与 CPU 完全串行工作，查询期间 CPU 不能执行其他任务。
- 定时查询：CPU 周期性（每隔固定时间）查询一次接口状态，查询之间可以执行其他任务。适用于外设不支持中断的多任务系统。定时查询的难点是定时长度不好确定：太短则数据可能还没准备好、白查一趟；太长则数据可能已就绪而未被及时处理而丢失。

优点：只需软件即可实现，硬件开销少，控制简单。缺点：CPU 被独占，大量时间花在等待与查询上，利用率很低。多见于早期单任务操作系统，现代计算机在操作系统启动引导至多任务系统之前也采用它。

轮询并不一定只适合慢速设备

一般印象里「轮询适合慢速设备、中断适合高速设备」，但这里有反例：对于极高速设备，设备准备数据的时间可能比「两次上下文切换 + 中断服务」的开销还短，此时中断反而成为瓶颈，轮询效率更高。所以程序查询方式在 **NVMe** 这类超高速固态硬盘上也被采用。「轮询只适合慢速设备」是错误说法。

轮询开销占比计算（高频计算题）

这类题只有一个套路：先求「每秒必须查询多少次」，再求「每次查询需要多少个时钟周期」，两者相乘除以 **CPU** 每秒的时钟周期数。

【例】CPU 主频 50MHz，查询程序运行一次至少 500 个时钟周期，设备每秒至少需查询 200 次，求 CPU 用于该设备的 I/O 时间占比。每秒查询需 $200\times500=100000=0.1$ M 个时钟周期，占比 $=0.1\text{M}/50\text{M}=0.2\%$。

【例】主频 500MHz、CPI 为 4，设备 A 数据传输率 2MBps，接口中有一个 32 位数据缓冲寄存器，每次输入输出至少执行 10 条指令。求最多间隔多久查询一次、以及占比。32 位 = 4B，设备准备这 4B 需 $4\text{B}/2\text{MBps}=2\,\mu s$，所以最多每 $2\,\mu s$ 查询一次。CPU 每秒查询 $1s/2\mu s=0.5$M 次，每次至少 $4\times10=40$ 个时钟周期，占比 $=40\times0.5\text{M}/500\text{M}=4\%$。

做题要注意：数据传输率按 **10** 为底（$1\text{M}=10^6$），不要按 2^{20} 算。

*2.*中断方式

中断方式（中断驱动 **I/O**）用中断机制彻底避免 CPU 忙等。完整流程如下：

1. 用户系统调用：用户进程 P1 通过 `read()` 等系统调用请求从设备读数据，陷入内核态。
2. 驱动程序启动设备：设备驱动程序把读命令及相关参数通过 I/O 接口发送给设备。
3. 操作系统进程调度：操作系统把 P1 放入 I/O 等待队列（阻塞态），通过上下文切换调度进程 P2 运行。P2 的执行与设备准备数据并行。
4. 设备发出中断请求：设备准备好数据后主动向 CPU 发中断请求，这是中断方式的核心——不需要 CPU 来问，设备自己会报。

5. **CPU** 响应并执行中断服务程序：CPU 暂停当前进程，保存现场，转入设备中断服务程序，在中断服务程序里完成实际的数据传送，并唤醒 I/O 等待队列中的 P1。
6. 恢复运行：中断服务程序最后一条是中断返回指令，返回断点继续执行；P1 已变为就绪态，等待被调度后完成后续 I/O 操作并返回用户态。

优点：设备主动报告，CPU 与外设并行工作，CPU 利用率显著提高；不需要考虑轮询频率问题，实时性更好。缺点：额外开销是两次进程上下文切换的时间加上中断服务程序本身的开销。当设备准备数据的时间远长于这些开销时，中断方式收益很大；反之则可能不如轮询。此外，每次数据传送都要经过 **CPU** 通用寄存器，且以字（字节）为单位，对块设备进行大数据量传送时效率很低。

谁最先拿到键盘输入的数据

统考考过「本地用户通过键盘登录系统时，首先获得键盘输入信息的程序是」——答案是中断处理程序，不是命令解释程序、系统调用服务程序或用户登录程序。键盘产生数据后先发中断，CPU 响应后执行键盘中断处理程序，由它把数据从控制器数据寄存器取走并放入内核缓冲区。中断服务例程执行结束时，输入数据位于内核缓冲区（2024 年考过这一点），既不是控制器数据寄存器，也不是用户缓冲区。

字符打印的经典过程（2023 年大题原型）

以 `printf("darkmoon")` 为例，可以把整条链路串起来：

1. 用户进程 P 调用库函数，最终执行 `write()` 系统调用，指令序列中有陷阱指令，从用户态陷入内核态。
2. 先把字符串复制到内核缓冲区。原因是中断服务程序执行时 CPU 仍运行在其他进程（如 Q）的环境中，地址空间是 Q 的，如果数据留在 P 的用户空间就取不到了；而内核空间被所有进程共享映射。
3. 驱动程序查询打印机状态寄存器直到就绪，把第一个字符从内核缓冲区经通用寄存器写入打印机控制器的数据寄存器（**I/O** 端口），再向控制寄存器写「启动打印」命令。
4. 执行调度程序，把 P 由运行态改为阻塞态并挂入阻塞队列，切换进程 Q 运行。
5. 打印机取走字符打印完毕后，控制器发出中断请求。CPU 在 Q 执行完一条指令后响应，执行中断隐指令（关中断、保存断点、中断服务程序寻址），转入字符打印中断服务程序。
6. 中断服务程序检查是否打印完毕：未完毕就取下一个字符送数据端口并再次启动打印，然后中断返回继续执行 Q；已完毕就把 P 从阻塞队列取出、状态改为就绪态，放入就绪队列。
7. 当调度器选中 P 后，P 执行系统调用的最后阶段，返回用户态继续执行。

因此：第一个字符是在系统调用服务例程控制下打印的，其余字符都由中断服务程序控制打印。这就是 2023 年第 46 题（键盘输入）与 2019 年大题所考的完整链路。

3.DMA 方式

直接内存访问（**Direct Memory Access**，**DMA**）在总线上设置 **DMA** 控制器（**DMAC**），由 DMAC 临时接管总线，代替 CPU 控制外部设备与内存之间的批量数据交换。数据无须经过 **CPU** 寄存器中转，直接通过系统总线在设备与内存之间交换，完全消除了程序查询和中断方式中 CPU 进行实际数据传送的开销。

DMA 方式主要用于高速设备的块数据传输，磁盘、显卡、网卡、声卡都支持 DMA。它也可以用于内存之间的大批量数据搬移。

DMA 控制器的组成：

部件	作用
主存地址寄存器（地址计数器 **AR**）	存放交换数据的内存地址。预处理阶段由 CPU 设置，每次传送后自动加 1 或减 1

部件	作用
字计数器（**WC**）	记录数据块长度。每传送一个字/字节自动减 1，减到 0 时传送完毕并发出 DMA 中断请求
数据缓冲寄存器（**DBR**）	DMAC 作为从设备时接收 CPU 的数据，作为主设备时暂存传输数据。若采用「周期窃取」模式则不经过它，性能更好
命令寄存器 / 状态寄存器	接收 CPU 控制命令 / 向 CPU 反馈 DMAC 状态
DMA 控制逻辑	控制和时序电路，修改地址计数器与字计数器，指定传输方向，协调 DMA 请求与 CPU 响应，并支持中断机制

DMAC 需要总线控制权时通过 **HOLD** 信号向 CPU 发总线请求，CPU 允许时用 **HLDA** 信号授权。支持 DMA 的设备用 **DREQ**（**DMA** 请求）和 **DACK**（**DMA** 应答）两条专用控制线与 DMAC 相连（注意：这两条线通常不属于系统总线）。一个 DMAC 支持多个通路，每条通路有各自独立的内存地址寄存器和字计数器，可同时为多个设备服务。

DMAC 在总线上的两种角色：预处理阶段 CPU 是主设备、DMAC 是从设备；数据传输阶段 DMAC 是主设备、CPU 反而成了旁观者。现代 PC 中 CPU、DMAC、I/O 接口、内存都通过系统总线相连，不存在所谓的「设备和内存之间的专用数据通路」。

DMA 的三种内存争用处理方式：

方式	工作方式	优点	缺点
停止 **CPU** 访问内存	DMAC 一旦获得总线就连续占用多个存储周期，直到整个数据块传完才归还	控制最简单（总线请求/建立/归还各只需一次）；数据传输率最高	DMA 期间 CPU 无法响应任何中断，实时性差；CPU 即使不需要访存也被强制暂停，浪费 CPU 时间
DMA 与 **CPU** 交替访问内存	把每个存储周期固定分为两半，CPU 只用其中一半，DMAC 只用另一半	无需总线请求，CPU 感觉不到 DMA 存在	硬件代价高；双方各自的空闲时间片都被浪费，内存带宽实际减半
周期挪用（周期窃取）	设备每准备好一个字，DMAC 就申请并占用一个存储周期完成传送，随后立即归还总线	按需分配，对 CPU 影响最小；已成为 **DMA** 的主流方法	总线请求、仲裁、归还本身有额外开销，一次挪用实际需 2 ~ 5 个存储周期

周期挪用的三个易错点

1. **CPU** 让出总线的时机是一个存储（总线）周期结束之后，不是「立即暂停当前指令」。一条指令可能包含多个总线周期，CPU 必须做完当前这个总线周期才能让出总线。这与中断请求在一条指令执行结束后响应是两回事。
2. 周期挪用期间 **CPU** 并不是完全不能访问主存。DMA 只在需要传数据时「偷走」一个周期，两次窃取之间的间隙 CPU 完全可以正常访存。「整个数据块传送过程中 CPU 不可以访问主存储器」是错误说法——那是「停止 CPU 访问内存」方式的特点。
3. **DMA** 操作不修改 **CPU** 寄存器，因此不存在现场保护和恢复问题。如果周期挪用发生时 CPU 不需要访存，则对 CPU 执行没有任何性能影响；如果恰好形成访存冲突，则 **DMA** 优先访问内存。

DMA 传输的三个阶段：

阶段	具体操作	结束时状态
预处理阶段（CPU 是主设备，DMAC 是从设备）	① 用 I/O 指令向 DMAC 写入主存起始地址（地址寄存器 AR）、传输长度（字计数器 WC）、传输方向（控制/命令寄存器标志位）；② 向 I/O 接口发启动命令，写入磁盘扇区号、设备地址等；③ 向 DMAC 控制寄存器写入「启动 DMA」命令位	请求进程被阻塞，调度其他进程运行
数据传输阶段（DMAC 是主设备，CPU 袖手旁观）	周期挪用循环：设备备好一个字 → 经 DREQ 通知 DMAC → DMAC 经 HOLD 申请总线 → CPU 在当前机器周期/总线事务结束后用 HLDA 授权并断开总线连接（驱动器进高阻态）→ DMAC 送主存地址到地址总线、发内存写 `MEMW` 与 I/O 读 `IORD` 控制信号、用 DACK 应答设备，设备把数据字送到数据总线，内存控制器写入 → AR 加 1、WC 减 1，撤销 HOLD 立即归还总线	WC 减到 0
后处理阶段（中断服务程序）	DMAC 经 `INTR` 发「DMA 完成」中断 → CPU 响应中断、保存现场，进中断服务程序 → 读 DMAC 状态寄存器判断是否成功 → 成功则把等待进程由阻塞态改为就绪态并放入就绪队列；失败则执行错误处理程序 → 恢复现场、中断返回	进程就绪，等待被调度

DMA 传输顺序记忆

② 初始化 **DMA** 控制器并启动磁盘 → ③ 从磁盘传输一块数据到内存缓冲区 → ①**DMA** 控制器发出中断请求 → ④ 执行「**DMA** 结束」中断服务程序（2017 年原题的四个选项就是这个顺序）。口诀：先配置、再搬运、后中断、最后收尾。

*4.*中断方式与 *DMA* 方式的区别

对比维度	中断方式	**DMA** 方式
核心思想	以 **CPU** 为核心，CPU 亲自处理每一次数据传送请求	以存储器为核心，把数据传送「外包」给专业硬件
数据传送主体	软件（CPU 执行中断服务程序）	硬件（DMA 控制器）
数据是否经过 CPU	每个字/字节都经过 CPU 通用寄存器	不经过 **CPU** 寄存器，设备与内存直接交换
CPU 介入程度	高，每个数据单位传送都要执行一次中断服务程序	低，只在预处理和后处理阶段介入
请求的对象	**CPU** 处理时间（请求中断 CPU）	总线控制权（请求系统总线的临时使用权）
响应时机	一条指令执行结束后	一个总线（机器）周期结束后，响应更快
数据传送单位	字（字节）	数据块
对 CPU 现场的影响	完全中止当前程序，需完整保护现场、恢复现场	不改变 **CPU** 现场，无需保护和恢复
中断次数	每个数据单位一次，频繁	整个数据块仅一次（DMA 完成中断）
CPU 与外设并行度	低（并行性只体现在设备准备数据期间）	高
适用设备	中、低速设备的少量数据传送，如键盘、鼠标	高速设备的批量数据传送，如磁盘、网卡

对比维度	中断方式	**DMA** 方式
功能范围	广泛，除数据传送外还能处理各种随机事件（时钟中断、电源掉电等）	功能单一，只用于成块数据的快速输入输出
优先级	低于 DMA	优先级高，DMA 与 CPU 同时访存时 DMA 优先

关于两者区别的两个常见错误说法

1.「中断方式适用于所有外部设备，DMA 方式仅适用于快速外部设备」——太绝对。中断方式对速率极高的设备效率很低，并不适用；DMA 理论上也能用于低速设备，只是浪费资源。
2.「中断服务程序不直接控制 I/O 硬件」——错误。设备驱动程序和中断服务程序共同直接控制硬件：驱动程序负责发起操作（向命令/数据/地址寄存器写入、启动设备），中断服务程序负责响应完成（从状态寄存器读取结果、向硬件发送确认信号清除中断）。

三种控制方式的综合使用

在实际系统中，这三种方式常常配合使用。以从磁盘读数据为例：

1. 进程 P1 用程序查询方式设置传送参数，初始化 DMAC 的相关寄存器；
2. CPU 向磁盘控制器发「寻道」命令，随后切换到其他进程 P2 执行；寻道完成后磁盘发出「寻道结束」中断；
3. CPU 响应中断，在中断服务程序中发「查找扇区」命令，返回 P2；扇区找到后磁盘发出「查找结束」中断；
4. CPU 响应中断，在中断服务程序中启动 **DMA** 传送；
5. DMAC 完成数据块传送后发出「DMA 结束」中断，CPU 在中断服务程序中做数据校验等后处理。

结论：在用 **DMA** 方式进行硬盘等高速设备的数据传送时，仍然要用到程序查询和中断方式。

（三）I/O 软件层次结构

为使复杂的 I/O 软件具有清晰的结构、良好的可移植性和易适应性，I/O 软件采用层次式结构：每层都利用下层提供的服务完成某些子功能，并屏蔽实现细节，向高层提供服务。整个 I/O 软件可视为四层结构。

*1.*用户层 *I/O* 软件

职责：实现与用户交互的接口。用户直接调用用户层的、与 I/O 操作有关的库函数来操作设备。

关键点：

- 大部分 I/O 软件在操作系统内部，但仍有一小部分在用户层，包括与用户程序链接在一起的库函数。
- 用户层软件必须通过一组系统调用来获取操作系统服务。库函数本身运行在用户态，它不直接操作硬件，而是通过系统调用陷入内核态，把请求交给下一层的设备独立性软件。
- 这一层的目标是隐藏所有底层 **I/O** 操作的复杂性，为用户提供简单、方便、统一的接口。

显式 **I/O** 与隐式 **I/O**：通过库函数和系统调用进行的 I/O 是显式 **I/O**（最常见）。此外还有由虚拟内存系统自动触发的隐式 **I/O**：访问一个不在物理内存的虚拟地址会触发缺页异常、自动发起一次磁盘 I/O；用 `mmap` 把文件映射到虚拟地址空间后，首次访问或修改该地址也会触发磁盘 I/O。

*2.*设备独立性软件

设备独立性（设备无关性）指应用程序独立于具体使用的物理设备。为实现这一点，引入了逻辑设备和物理设备两个概念：应用程序用逻辑设备名请求某类设备，系统执行时再把逻辑设备名映射为物理设备名。

使用逻辑设备名有两大好处：① 增加设备分配的灵活性；② 易于实现 **I/O** 重定向（用于 I/O 操作的设备可以更换，而不必修改应用程序）。

主要功能可分为两方面：

- 执行所有设备的公有操作：设备的分配与回收；将逻辑设备名映射为物理设备名；设备保护（禁止用户直接访问设备）；缓冲管理；差错控制；提供独立于设备、大小统一的逻辑块，屏蔽设备之间信息交换单位大小和传输速率的差异；设备命名（把物理名映射为 `/dev/sda1` 这类符号名）。
- 向用户层（或文件层）提供统一接口：无论何种设备，向用户提供的接口相同，如统一使用 `read`/`write`。

差错控制的两类错误：暂时性错误（如网络丢包）通常通过重试纠正，连续失败才视为持久性错误，主要由驱动程序处理；持久性错误（如磁盘划痕）由操作系统记入坏块表，后续 I/O 自动绕过。

I/O 软件四层的排列顺序（**2011**、**2012**）

自上而下是：用户层 **I/O** 软件 → 设备独立性软件 → 设备驱动程序 → 中断处理程序。中断处理程序最接近硬件，必须在最底层；驱动程序必须在设备独立性软件之下。一次磁盘 I/O 请求的处理流程是：用户程序 → 系统调用处理程序 → 设备驱动程序 → 中断处理程序。

另外，**SPOOLing** 技术的归属有争议：在经典教材（如汤子瀛《计算机操作系统》）中它被划入用户层 I/O 软件，而 **408** 大纲把它划入设备独立性软件。理解上：SPOOLing 的入口和守护进程确实工作在用户态，但它的核心功能（实现设备共享）属于设备独立性软件的职责。统考遇到这类有争议的问题通常会回避，按大纲理解即可。

*3.*设备驱动程序

职责：与硬件直接相关，接收上层软件发来的抽象 **I/O** 请求（如 `read`、`write`），转换为对设备的具体要求后发送给设备控制器，控制 I/O 设备工作；同时把设备控制器发来的信号传送给上层软件，从而为 I/O 内核子系统隐藏设备控制器之间的差异。

具体功能：

1. 抽象请求到具体操作的转换：把逻辑盘块号翻译成硬件能识别的柱面号、磁头号、扇区号等参数。
2. 合法性与权限校验：检查用户请求的操作是否被允许（如只读打开的设备不能写）。
3. 设备状态检查：读控制器状态寄存器，确认设备处于可接收命令的就绪状态。
4. 参数设置与传送：向控制器的命令、数据、方式等寄存器写入本次任务所需参数。
5. 发出 **I/O** 命令并启动设备：设备空闲则立即启动；设备忙则把请求进程的 PCB 挂到该设备的等待队列上。
6. 响应中断：及时响应设备控制器发出的中断请求，并按中断类型调用相应的中断处理程序。

特点：

- 硬件相关性：与特定硬件的物理特性紧密耦合，每类设备配置一个驱动程序，通常由设备制造厂商提供而非 OS 设计者设计。因此在系统中增加一个新设备时需要安装厂商提供的新驱动程

序。

- 与 **I/O** 控制方式紧密相关：程序查询方式的驱动需要循环查询状态；中断方式的驱动需要启动设备后阻塞进程、由中断唤醒；DMA 方式的驱动需要配置 DMA 地址、计数和方向。「驱动程序与 **I/O** 控制方式无关」是错误说法（2022 年考过）。
- 一部分必须用汇编语言书写，很多基本部分已固化在 ROM 中。
- 可重入性：必须在一次调用完成前能被另一个任务（如中断）再次安全调用。典型场景是：驱动代码正在响应 `write` 系统调用执行时，被该设备完成上一次操作产生的硬件中断打断，CPU 转去执行该驱动的另一部分代码（中断服务程序）。如果两部分代码共享全局或静态变量而无保护，就会数据错乱。可重入函数通过不使用这类共享数据或加特殊保护来避免。

设备处理方式（不重要，了解即可）：为每类设备设一个进程；为整个系统设一个 I/O 进程（或分设输入进程和输出进程）；不设专门进程，只为各类设备设驱动程序供用户或系统进程调用（目前用得较多）。

*4.*中断处理程序

职责：保存被中断进程的 CPU 环境，转入相应的中断处理程序进行处理，处理完毕再恢复被中断进程的现场并返回。

主要任务：进程上下文切换、对中断信号源进行测试、读取设备状态、修改进程状态等。

位置：由于中断处理与硬件紧密相关，对用户而言应尽量加以屏蔽，因此放在操作系统的底层，系统的其余部分尽可能少地与它发生联系。

整个中断过程包括中断响应和中断处理两个阶段：中断响应完全由硬件完成，中断处理则由 **CPU** 执行中断服务程序完成。虽然不同中断源对应的服务程序不同，但所有中断服务程序的结构相同，包括准备阶段、处理阶段和恢复阶段。I/O 中断的典型过程是：设备控制器准备好数据后通过中断控制器向 CPU 发中断请求 → 保护被中断进程的 CPU 上下文 → 转入相应的设备中断处理函数 → 进行中断处理 → 恢复被中断进程的上下文。

以一次 **read** 命令总结四层各自的分工

1. 用户要读某设备内容，通过操作系统提供的 `read` 命令接口——经过用户层。
2. 操作系统给用户的接口一般是统一的通用接口，用户发出的 `read` 先经设备独立层解析（确定逻辑块号、检查高速缓存、找到对应驱动程序），然后交给下层。
3. 不同类型的设备对 `read` 的行为不同，需要针对具体设备把 `read` 解析成不同指令——经过设备驱动层。
4. 命令解析完毕后需要中断正在运行的进程、转而执行实际的数据传送，这就需要中断处理程序。

（四）输入/输出应用程序接口

不同类型的设备读写方式差异很大，为便于应用程序访问，操作系统把大量差异隐藏起来，并把设备分成几种常规类型，相应地把 I/O 核心子系统与高层之间的接口也分为若干类。

*1.*字符设备接口

字符设备以字符为单位存取和传输，如键盘、打印机、鼠标。特点是传输速率较低、不可寻址，I/O 通常采用中断驱动方式。具体特征：

- 流式/顺序访问（不可寻址）：数据只能从当前位置顺序读写，没有「块」也没有地址。由于不可寻址，通常为字符设备建立一个字符缓冲区，用户程序通过 `get()` 从缓冲区取字符、通过 `put()` 把字符输出到缓冲区。

- 需要打开和关闭操作：大多数字符设备属于独占设备，接口中必须提供打开和关闭操作来获取和释放访问权限，以实现互斥共享。
- 控制命令丰富：通常支持特殊的 `ioctl()` 系统调用进行设备特定设置（如设置串口波特率）。

*2.*块设备接口

块设备以数据块为单位存取和传输，如磁盘、固态硬盘、U 盘、SD 卡、CD。特点是传输速率较高、可寻址，I/O 常采用 **DMA** 方式。具体特征：

- 可寻址/随机访问：可以直接命令设备「读第 500 号数据块」，隐藏了磁盘的二维结构——把磁盘所有扇区按数据块顺序划分，二维结构变为线性序列，文件系统因此可以在其上构建。
- 固定大小的数据单元：无论读写，基本单位都是一个「块」。即使只想读 1 个字节，操作系统也可能从硬件读整个块再提取所需字节。
- 缓冲机制：操作系统大量使用内存（Buffer Cache）缓存块设备数据。请求的块若已在缓存中就直接从内存返回，极大提高性能；写操作也可能先写缓存，稍后统一刷到设备。
- 将抽象命令映射为低层操作：用户使用的 `open()`、`read()`/`write()`、`lseek()` 无法被设备直接识别，块设备接口把它们映射为设备能识别的具体操作。

*3.*网络设备接口

网络设备接口是专门用于与其他计算机交换数据的接口，不直接与读写相关，而是围绕数据包的收发和连接的管理设计，典型设备是以太网卡。关键特征：

- 不使用常规文件 **API**：应用程序通常不用 `open` 再 `read`/`write` 的方式操作网卡，而是使用一套独立的套接字（**Socket**）**API**。
- 数据包导向：数据以网络数据包形式封装传输，每个包包含目标地址、源地址等信息。
- 连接管理：需要处理建立连接（TCP）、无连接通信（UDP）、监听端口、超时处理等网络概念。
- 异步通信：网络不可靠且有延迟，发送一个数据包后不知道它何时到达、对方何时回复。

主要系统调用：`socket()` 创建通信端点、`bind()` 把地址绑定到套接字、`send()`/`recv()` 收发数据包、`close()` 关闭连接。

*4.*阻塞 *I/O* 与非阻塞 *I/O*

以一次典型的 `read` 操作为例，I/O 包含两个独立的阶段：① 等待数据准备（内核等待数据到达：等键盘输入、等磁盘寻道读出、等网络包到达）；② 数据从内核复制到用户空间。阻塞与非阻塞的核心区别在于它们如何处理第一个阶段。

比较项	阻塞 **I/O**	非阻塞 **I/O**
数据未就绪时的行为	系统调用不立即返回，进程由运行态转为等待（阻塞）态，CPU 去执行其他进程	系统调用立即返回一个错误码，进程不进入等待态
后续处理	数据准备好后内核唤醒该进程，系统调用继续把数据从内核复制到用户空间后返回	进程需反复主动调用该系统调用来检查数据是否就绪，这个过程称为轮询
数据就绪后	由内核主动唤醒并完成复制	仍需进程自己发起调用完成复制，然后才能处理数据
优点	代码逻辑线性、顺序，直观易懂；`read()` 返回后数据一定有效	并发潜力高，一个线程可同时照看多个 I/O 通道，线程利用率高

比较项	阻塞 **I/O**	非阻塞 **I/O**
缺点	并发能力差，一个线程任一时刻只能处理一个 I/O 请求，等待期间整个线程被挂起	编程模型复杂（要写轮询循环）；轮询本身消耗 CPU（忙等待）

为什么非阻塞 I/O 的并发度更高

核心在于线程资源的释放。阻塞模型下一个线程与一个 I/O 通道「绑定等死」，线程生命周期被漫长的 I/O 等待占据；非阻塞模型下线程发起 I/O 调用后立即被「释放」，可以转而为其他成千上万个 I/O 通道服务或执行其他计算。就像一个服务员可以同时照看很多张桌子，而不是站在一张桌子旁死等客人吃完。

阻塞与非阻塞、同步与异步的层次

阻塞/非阻塞描述的是「等待数据准备」这一阶段进程的行为；同步/异步描述的是「数据复制」阶段由谁完成。阻塞 I/O 和非阻塞 I/O 在数据复制阶段都是同步的——都必须由进程自己发起调用、由内核把数据复制到用户空间后才算完成。真正的异步 **I/O** 是发起请求后立即返回，数据准备和数据复制全部由内核完成，内核在整个操作完成后才通知进程，进程在此期间不需要任何轮询或等待。

二、设备独立软件

（一）缓冲区管理

*1.*为什么需要缓冲

缓冲（**buffer**）是一段临时存储区，可以由专门的硬件寄存器组成，但硬件成本高、容量小，一般只用于速度要求极高的场合（如 TLB、设备控制器中的数据缓冲寄存器），更多利用内存作为缓冲区。缓冲区管理的主要功能是组织好这些缓冲区并提供获得和释放的手段。

引入缓冲区的目的有四个：

1. 缓和 **CPU** 与 **I/O** 设备间速度不匹配的矛盾。凡在数据到达速率与离去速率不同的地方都可设置缓冲区。CPU 运算速率远高于 I/O 设备，若在打印机或控制器中设缓冲区，CPU 可快速暂存输出数据后立即去做别的计算，打印机慢慢从缓冲区取数据打印，CPU 效率大幅提高。
2. 减少对 **CPU** 的中断频率，放宽对 **CPU** 中断响应时间的限制。例如速率为 9.6kb/s 的数据通信，若只用一位缓冲接收，则每 100μs 就要中断 CPU 一次，且 CPU 必须在 100μs 内响应，否则数据被冲掉。若设置 8 位缓冲寄存器，中断频率降为原来的 1/8；再设一个 8 位寄存器，响应时间可从 100μs 放宽到 800μs。
3. 解决数据粒度（数据单元大小）不匹配的问题。生产者数据粒度小于消费者时，生产者可连续生产几个单元，凑够消费者要求的单元大小后消费者再取走；反之消费者也可分几次取走一个大数据单元。
4. 提高 **CPU** 与 **I/O** 设备之间的并行性，提高系统吞吐量和设备利用率。

*2.*缓冲与缓存的区别

两者都是一段临时存储区，都能协调高速设备与低速设备的速度差异，但工作原理不同：缓冲是一个「中转站」，缓存是数据源的副本。

特性	缓冲（**Buffer**）	缓存（**Cache**）
核心目的	缓和速度/粒度不匹配	加速对数据的重复访问
工作模式	数据的「中转站」	数据源的「高速副本」「快捷方式」

特性	缓冲（**Buffer**）	缓存（**Cache**）
数据性质	中转数据，可能是唯一的存在（源方发出后不再保留）	一定是源数据的子集（副本）
数据流向	通常是单向的，被消费后即失效	可被反复读取
性能指标	关注吞吐量	关注命中率

磁盘高速缓存就是这个意义上的缓存：操作系统在内存的内核空间开辟高速缓存，把最近从块设备读出或写入的数据保存在 RAM 中。它逻辑上属于磁盘，物理上驻留在内存。读取时先查缓存，命中则直接从内存返回，未命中才发起真正的磁盘 I/O；写入有写通（直写）（同时写缓存和磁盘，安全但性能提升有限）和回写（只写缓存并标记为脏，之后择机刷回磁盘，性能极高但有数据丢失风险，是现代 OS 普遍采用的策略）两种策略。

缓冲区在用户空间还是内核空间

用户进程提出 I/O 请求时指定的缓冲区在用户空间（如 `fread(buf, size, num, fp)` 中的 `buf`）。陷入内核后，内核通常会在内核空间再开辟一个或两个缓冲区，底层 I/O 软件直接使用内核空间的缓冲区存放 I/O 数据。为什么不直接用用户空间的缓冲区？因为外设进行 I/O 期间用户进程被挂起，用户空间的缓冲区所在页面可能被替换出去，那样就无法获得缓冲区中的 I/O 数据；此外还便于解决不同块设备读写单位不一致、便于共享等问题。

*3.*单缓冲

假设用户进程请求块设备读入若干块数据，采用单缓冲时操作系统在主存（内核空间）中为其分配一个缓冲区（若无特别说明，一个缓冲区的大小就是一个块）。

设：从磁盘把一块数据输入到缓冲区的时间为 T，操作系统把缓冲区中的数据传送到用户进程工作区的时间为 M，CPU 对这一块数据处理的时间为 C。

取初始状态为工作区是满的、缓冲区是空的（若无明确说明，通常认为缓冲区与工作区大小相等）：

- 若 $T > C$：工作区数据处理完用时 C 时缓冲区还没充满，需再等到 T 时刻缓冲区才满，然后缓冲区向工作区传送数据，工作区满时缓冲区同时为空，用时 M。到达下一个相同状态，整个过程用时 $T + M$。
- 若 $T < C$：缓冲区早已充满并等待，工作区处理完才能传送，用时 $C + M$。

单缓冲处理每块数据的时间

$$\text{单缓冲每块时间} = \max(T,\ C) + M$$

即「输入」与「处理」可以并行，但「传送」必须串在两者之后。因为只有一个缓冲区，设备向缓冲区写入数据与 CPU 处理工作区数据可以重叠，但缓冲区向工作区传送时必须独占缓冲区。

处理 n 块数据的总时间为：

$$(n-1)\left[\max(T, C) + M\right] + (C + T + M)$$

即前 $n-1$ 块按流水节奏并行，最后一块需要完整的串行收尾（输入、传送、处理各走一遍）。

*4.*双缓冲

由于缓冲区是共享资源，生产者与消费者使用缓冲区时必须互斥。如果消费者尚未取走缓冲区中的数据，即使生产者又生产出新数据也无法送入缓冲区，生产者只能等待。设置两个缓冲区便可解决

这个问题：设备输入时先把数据送入缓冲区 1，装满后转向缓冲区 2 写入；此时操作系统可从已满的缓冲区 1 取数据送用户工作区并由 CPU 处理。

取初始状态为工作区空、其中一个缓冲区满、另一个缓冲区空：

- 若 $T > C + M$：设备输入所需时间大于传送与处理的总时间，设备可连续工作。当缓冲区 2 的数据处理完毕（用时 $C + M$）时缓冲区 1 尚未装满，系统需等待其装满才能把下一块从缓冲区 1 传送到工作区。处理每块数据所需时间为 T。
- 若 $T < C + M$：设备输入较快，CPU 无须等待设备。缓冲区 1 很快装满，但系统仍需等缓冲区 2 的数据完全处理完毕才能切换并传送缓冲区 1 的内容。处理每块数据所需时间为 $C + M$。

双缓冲处理每块数据的时间

$$\text{双缓冲每块时间} = \max(T,\ C + M)$$

即「处理」与「传送」串行（两者都发生在工作区侧，且传送依赖处理的完成），但「传送 + 处理」整体与「输入」并行。

处理 n 块数据的总时间为：

$$(n-1)\,[\max(T,\ C + M)] + (C + T + M)$$

双缓冲在通信中的应用：两台机器通信时每台需配置发送缓冲区和接收缓冲区。若要实现全双工通信（双方同时收发数据），每台机器必须分别设置发送缓冲区和接收缓冲区；若每台只配一个缓冲区，则无法同时进行发送与接收。

*5.*循环缓冲

若输入与输出速度相差较大，双缓冲难以充分发挥作用，此时采用多于两个缓冲区的方案。循环缓冲（环形缓冲）由若干个大小相等的缓冲区构成，每个缓冲区有一个链接指针指向下一个，最后一个的指针指向第一个，构成一个循环队列。系统维护两个指针：`in` 指向下一个待写入的空缓冲区，`out` 指向下一个待读取的满缓冲区，输入进程和计算进程分别推进 `in` 和 `out`。

*6.*缓冲池

单缓冲、双缓冲、循环缓冲都是为特定生产者和消费者进程专门设置的专用缓冲。系统较大时会有许多这样的循环缓冲，既消耗大量内存，利用率又不高。为提高利用率，引入既可用于输入又可用于输出的公用缓冲池。

缓冲池与缓冲区的区别：缓冲区仅仅是一组内存块的链表，而缓冲池是包含了一个管理的数据结构及一组操作函数的管理机制。

缓冲池中具有相同类型的缓冲区链接成一个队列，按使用状况形成三个队列：空白缓冲队列（空缓冲区链成，用于取空缓冲区写入）、输入队列（装满输入数据的缓冲区链成，供用户程序读取）、输出队列（装满输出数据的缓冲区链成，等待被输出设备读取）。

另外按缓冲区在实际运算中扮演的功能不同，设置四种工作缓冲区（`h` 代表 host/用户进程，`s` 代表 system/设备，`in`/`out` 代表方向）：

工作缓冲区	用途
`hin`（收容输入）	缓冲池与 I/O 设备之间，收容输入的数据
`sin`（提取输入）	缓冲池与用户进程空间之间，提取输入的数据

工作缓冲区	用途
hout（收容输出）	缓冲池与用户进程空间之间，收容输出的数据
sout（提取输出）	缓冲池与 I/O 设备之间，提取输出的数据

四种工作方式（规律：取出操作一律从队头取，挂入队列一律挂到队尾）：

工作方式	从哪个队列队首取	装满/取空后挂到哪个队列队尾
收容输入（输入进程）	空缓冲队列	输入队列
提取输入（计算进程）	输入队列	空缓冲队列
收容输出（计算进程）	空缓冲队列	输出队列
提取输出（输出进程）	输出队列	空缓冲队列

缓冲方式的分类归属

单缓冲、双缓冲、循环缓冲属于生产者消费者进程的专用缓冲；缓冲池属于进程共享缓冲。这与内存管理中「请求分页的可变分配全局置换使用系统空页队列」的思想类似，都是把资源集中管理以提高利用率。

单/双缓冲时间计算的两个真题

【2011-31】某文件占 10 个磁盘块，逐个读入主存缓冲区并送用户区分析。一个缓冲区与一个磁盘块大小相同，把一块读入缓冲区的时间 $T = 100\ \mu s$，缓冲区数据传送到用户区的时间 $M = 50\ \mu s$，CPU 分析一块的时间 $C = 50\ \mu s$。求单缓冲和双缓冲下读入并分析完该文件的时间。

- 单缓冲：每块 $\max(100, 50) + 50 = 150\ \mu s$，总时间 $= (10-1) \times 150 + (50+100+50) = 1350 + 200 = 1550\ \mu s$。
- 双缓冲：每块 $\max(100,\ 50 + 50) = \max(100, 100) = 100\ \mu s$，总时间 $= (10 - 1) \times 100 + (50 + 100 + 50) = 900 + 200 = 1100\ \mu s$。

答案选 **B（1550μs、1100μs）**。

【2013-27】系统缓冲区和用户工作区均采用单缓冲，从外设读入一块到系统缓冲区 $T = 100$，从系统缓冲区读入用户工作区 $M = 5$，对用户工作区中一块的分析时间 $C = 90$。求读入并分析 2 个数据块的最短时间。

代入 n 块公式：$(2 - 1) \times [\max(100, 90) + 5] + (90 + 100 + 5) = 1 \times 105 + 195 = 300$。答案选 **C（300）**。

算这类题最容易踩的坑

1. 不要用「$n\times$ 每块时间」。n 块的总时间必须用 $(n - 1)$ 倍的流水时间加上最后一块的完整串行时间 $(C + T + M)$，因为第一块必须先走完「输入 $\rightarrow$ 传送 $\rightarrow$ 处理」才能开始流水。若题目只问「处理每块的平均时间」才用 $\max(\cdot) + M$ 或 $\max(\cdot)$。
2. 单缓冲的 M 在 max 之外，双缓冲的 M 在 max 之内。这是两者公式唯一的差别，也是判断题目该用哪个公式的关键。
3. 注意题目给的是「系统缓冲区 $\rightarrow$ 用户工作区」还是反过来。M 的方向不影响公式形式，但要确认 T 是设备 $\rightarrow$ 缓冲区的时间。
4. 若题目强调「初始状态缓冲区为空、工作区为空」，需按题目给的初始状态重新推一遍，公式给出的是稳定流水后的每块时间。

（二）设备分配与回收

1.设备分配的任务与总原则

设备分配的任务是按照一定策略，为提出 I/O 请求的进程分配合适的设备，并分配相应的控制器和通道，确保 CPU 与 I/O 设备之间能正常通信。

总原则：充分发挥设备的使用效率，尽可能让设备忙碌，又要避免因不合理的分配方法造成进程死锁。

> 通道是一种独立于 CPU、专用于 I/O 控制的处理器，可以控制 I/O 设备与内存直接进行数据交换。通道的启动需要 CPU 执行相应指令，操作结束时向 CPU 发中断信号。数据交换过程中 CPU 无需干预，提高了 CPU 与 I/O 的并行程度。

2.设备分配应考虑的因素

（**1**）设备的固有属性（按共享属性分三类）：

- 独占设备：一个时段只能分配给一个进程（如打印机），利用率低。
- 共享设备：一段时间内可同时分配给多个进程使用（如磁盘），各进程宏观上同时共享、微观上交替使用，因此要注意对访问先后次序进行合理调度，一般采用动态分配方式。
- 虚拟设备：采用 SPOOLing 技术把独占设备改造成虚拟的共享设备，可同时分配给多个进程使用。

（**2**）设备分配算法：通常只采用先来先服务（**FCFS**）或优先级高者优先。FCFS 优先分配给队首进程；最高优先级优先算法把高优先级进程排在队列前方，优先级相同时按 FCFS 排队。

（**3**）设备分配中的安全性：

方式	做法	优点	缺点
安全分配方式	为进程分配一个设备后就将进程阻塞，本次 I/O 完成后才唤醒。一个时段内每个进程只能使用一个设备	破坏了「请求和保持」条件，不会死锁	对一个进程来说 CPU 与 I/O 设备只能串行工作
不安全分配方式	进程发出 I/O 请求后系统分配设备，进程可继续执行并发新的 I/O 请求，只有某个请求得不到满足时才阻塞。一个进程可同时使用多个设备	进程的计算任务与 I/O 任务可并行处理，进程推进迅速	可能具备「请求并保持」条件，可能发生死锁（需配合死锁避免、检测与解除）

（**4**）设备分配方式：

- 静态分配：进程运行前一次性分配其所需的全部设备、控制器和通道，运行结束后归还。破坏了「请求和保持」条件，不会发生死锁，主要用于独占设备。缺点是进程可能长期不用已分配的设备而其他进程又用不上，设备利用率低。
- 动态分配：运行过程中按需申请。利用率高，但有死锁可能。

静态分配与安全分配方式的关系

两者都通过「不让进程同时持有并等待」来破坏死锁条件，但层次不同：静态分配说的是分配时机（一次性分配全部资源），安全分配方式说的是分配后进程的状态（立刻阻塞）。另外注意：静态分配「可以提高设备利用率」是错误说法，恰恰相反，它降低利用率。

3.设备分配管理中的数据结构

设备分配的四个数据结构体现了「通道 → 控制器 → 设备」的从属关系。一个设备控制器可供多个同类设备共享使用，一个通道可供多个设备控制器共享使用。

数据结构	数量	主要字段
设备控制表 **DCT**（Device Control Table）	系统为每个设备配置一张	设备类型、设备标识符（物理设备名，系统内唯一）、设备状态（忙/闲）、指向控制器表的指针、重复执行次数或时间、设备队列的队首指针
控制器控制表 **COCT**（Controller Control Table）	每个设备控制器一张	控制器标识符、控制器状态（忙/闲）、与控制器连接的通道表指针、控制器队列的队首/队尾指针
通道控制表 **CHCT**（Channel Control Table）	每个通道一张	通道标识符、通道状态（忙/闲）、与通道连接的控制器表首址、通道队列的队首/队尾指针
系统设备表 **SDT**（System Device Table）	整个系统只维护一张	每个表目对应一个设备，含设备类型、设备标识符、指向 **DCT** 的指针、驱动程序入口

四张表的记忆要点

SDT 全系统只有一张，是「总目录」，每个表目指向一个 DCT；**DCT** 每个设备一张，其「指向控制器表的指针」连接到 COCT；**COCT** 每个控制器一张，其「与控制器连接的通道表指针」连接到 CHCT；**CHCT** 每个通道一张。请求流程与这套链式结构一致：**SDT** → **DCT** → **COCT** → **CHCT**。

*4.*设备分配的步骤

以独占设备为例：

1. 分配设备：根据 I/O 请求中的物理设备名查找 SDT，找出设备的 DCT，根据 DCT 中的设备状态字段判断状态。若忙，把进程 PCB 挂入设备等待队列；若空闲，把设备分配给该进程。
2. 分配控制器：设备分配后，根据 DCT 找到对应的 COCT，查询控制器状态。若忙则挂入控制器等待队列，空闲则分配给该进程。
3. 分配通道：控制器分配后，根据 COCT 找到对应的 CHCT，查询通道状态。若忙则挂入通道等待队列，空闲则分配给该进程。

只有设备、控制器、通道三者都分配成功，这次设备分配才算成功，之后便可启动 I/O 设备进行数据传送。

用物理设备名分配的三大缺点：① 用户编程时必须使用物理设备名，底层细节对用户不透明；② 换一个物理设备程序就无法运行（即使有同类型设备也不行）；③ 若请求的物理设备正忙，即使系统中还有同类型设备，进程也必须阻塞等待。

改进方法是引入设备独立性：建立逻辑设备名到物理设备名的映射，用户编程时只提供逻辑设备名。改进后用逻辑设备名分配时，操作系统根据逻辑设备名在 SDT 中查找一个用户指定类型且空闲的设备分配给进程，只有所有该类型的设备都忙时才需要阻塞该进程。

逻辑设备表 **LUT**（Logical Unit Table）：每个表项为〈逻辑设备名，物理设备名，设备驱动程序入口地址〉。

- 进程首次请求设备分配时，系统为其分配一台物理设备，同时在 LUT 中新增一个表项；
- 以后该进程再次通过相同的逻辑设备名请求设备时，直接查表即可。

LUT 的两种建立方式：

- 整个系统只有一张 **LUT**：各用户所用的逻辑设备名不允许重复，适用于单用户操作系统。
- 每个用户一张 **LUT**（存入用户进程的 PCB 中）：不同用户的逻辑设备名可重复，适用于多用户操作系统。可类比文件系统中单级目录（不允许重名）与多级目录（不同级可重名）的区别。

▌设备分配相关的三道真题

【2009-32】程序员利用系统调用打开 I/O 设备时，通常使用的设备标识是——逻辑设备。用户程序通过系统调用请求设备时使用与具体硬件无关的逻辑设备名，物理设备名、主设备号、从设备号是内核或驱动程序识别具体硬件时使用的信息，不直接暴露给普通应用程序。

【2020-30】关于具备设备独立性的系统，错误叙述是「更换物理设备后必须修改访问该设备的应用程序」。设备独立性的目标正是让应用程序用逻辑设备名或文件名访问设备，更换设备时只需修改映射或驱动配置，应用程序通常不需要改变。

【2023-32】设备分配需要考虑的因素包括：Ⅰ设备类型、Ⅱ访问权限、Ⅲ占用状态、Ⅳ逻辑设备与物理设备的映射关系，答案是 D（四项全部）。设备类型决定分配规则，权限决定进程能否使用，占用状态决定能否立即分配，映射关系决定从哪个逻辑请求找到哪个实际设备。

补充：在 UNIX/Linux 中，设备名唯一确定一个特殊文件的 **i** 节点，i 节点中含主设备号（确定设备类型，据此定位对应的设备驱动程序）和次设备号（作为参数传给驱动程序，确定进行 I/O 操作的具体设备位置）。这就是「一切皆文件」设计的实现基础。

（三）假脱机技术（**SPOOLing**）

*1.*为什么需要 *SPOOLing*

以打印机为例，未引入 SPOOLing 时 CPU 需要亲自处理每一次数据搬运：从内存读数据 → 写入寄存器 → 从寄存器写到 I/O 端口。这带来四个弊端：

1. **CPU** 资源被独占，效率极低。打印机很慢，CPU 每发一小块数据就可能要停下来查询控制器状态，整个打印过程中 CPU 大部分时间花在等待和搬运上。
2. 用户进程长时间阻塞。发起打印请求的进程在打印任务彻底完成前无法被唤醒，打印几十页文档可能被「冻结」几分钟。
3. 计算与 **I/O** 无法并行。对同一进程而言两者严格串行。
4. 系统整体吞吐量低。慢速 I/O 长时间占用 CPU，其他需要 CPU 的进程也被迫等待。

2.SPOOLing 系统的组成

SPOOLing（**Simultaneous Peripheral Operations On-Line**，外部设备联机并行操作），又称假脱机技术，其思想源于早期的脱机 **I/O** 方式：依赖专用外围控制机，在主机不参与的情况下完成 I/O——输入时外围机把低速输入设备（如纸带机）的数据预先读入高速磁盘；输出时 CPU 先把结果高速写入磁盘，再由外围机送往低速输出设备。由于整个 I/O 过程脱离主机控制，因此称为脱机。

随着多道程序设计技术的发展，操作系统不再需要物理外围机，而通过专门的输入进程与输出进程在主机直接控制下用软件模拟外围机的功能。这种「以磁盘缓冲、用软件模拟脱机 I/O」的机制正是 SPOOLing 的本质：「脱机」承袭自早期脱机 **I/O** 的思想，「假」则体现在用多道程序和磁盘空间替代了物理外围机。

SPOOLing 系统的四个组成部分：

组成部分	位置	作用
输入井和输出井	磁盘上的两个专用存储区域	输入井模拟脱机输入的磁盘，暂存输入设备的数据；输出井模拟脱机输出的磁盘，暂存用户程序的输出数据。每个进程的数据以井文件形式保存，所有文件按顺序链接成输入队列或输出队列
输入缓冲区和输出缓冲区	内存中的两个缓冲区	输入缓冲区暂存由输入设备送来的数据，随后传送到输入井；输出缓冲区暂存从输出井读出的数据，随后传送到输出设备
输入进程和输出进程	内存中的系统进程	输入进程模拟脱机输入的外围控制机，把输入设备的数据经输入缓冲区送到输入井（CPU 需要输入数据时直接从输入井读）；输出进程模拟脱机输出的外围控制机，把用户程序的输出数据写入输出井，并在输出设备空闲时经输出缓冲区送到输出设备

组成部分	位置	作用
井管理程序	内核	控制作业与磁盘井之间的信息交换

SPOOLing 的前提条件

- 需要大容量、高速度的外存（通常为磁盘）作为输入井和输出井；
- 需要多道程序技术的支持；
- SPOOLing 是用软件的方式、建立在通道技术和多道程序技术基础之上的技术。

「SPOOLing 由用户作业控制设备与输入/输出井之间的数据传送」是错误说法（2016 年考过）——设备与井之间的数据传送由系统的输入/输出进程（假脱机服务程序）控制，对用户作业是透明的。

*3.*如何把独占设备改造为共享设备（打印机经典例子）

打印机是典型的独占设备。通过 SPOOLing 技术可把它改造为可供多个用户共享的虚拟设备。

当多个用户进程发出打印请求时，**SPOOLing** 系统接受请求但并不真正立即把物理打印机分配给它们，而是由输出进程为每个进程完成两项操作：

1. 在磁盘输出井中为其分配一个空闲盘块，把待打印数据写入其中；
2. 申请一张用户请求打印表，记录用户的打印要求，把该表挂到假脱机打印队列上。

完成这两步后，用户进程即认为打印任务已完成，可继续执行后续代码。实际上真正的打印尚未开始，但这个过程对用户完全透明。真正的打印由后台的假脱机打印进程负责：当打印机空闲时，该进程检查假脱机打印队列，若队列非空则取出队首请求表，根据表中信息把对应的输出数据从输出井读入内存缓冲区，再送到物理打印机输出；一个任务完成后继续处理下一个；队列为空则进入等待状态，直至有新的打印请求把它唤醒。

结果：由于用户进程只需把数据写入磁盘即可返回，而物理打印由后台进程按队列顺序依次执行，多个用户便可并发提交打印请求。系统为每个请求在输出井中分配独立存储区域，使各进程都以为自己独占了一台打印机。一台物理打印机被虚拟为多台逻辑设备，实现了高效且透明的共享。

4.SPOOLing 的三大特点与优点

SPOOLing 系统的三大特点

1. 提高了 **I/O** 速度：把对低速 I/O 设备的操作转化为对磁盘输出井中数据的存取，如同脱机输入输出一样，有效缓和了 CPU 与低速 I/O 设备之间速度不匹配的矛盾。
2. 将独占设备改造为共享设备：在假脱机打印系统中，实际上并未把物理设备分配给任何进程。
3. 实现了虚拟设备功能：每个进程都认为自己独占了一台设备。

SPOOLing 是一种以空间换时间的技术：牺牲磁盘存储空间（输入井、输出井），换取显著的时间节省。

SPOOLing 引入后的具体优点：

- 提高了用户进程的响应速度。数据一旦拷贝到输出缓冲区并由井管理程序接管，用户进程的任务就算完成，可立即返回。对用户进程而言，它感觉不是在跟慢速打印机打交道，而是跟高速磁盘打交道，只需把数据快速「甩」到输出井。这种把慢速设备「伪装」成高速设备的技术称为设备虚拟化。
- 缓和 **CPU** 与慢速 **I/O** 设备的速度矛盾，极大提高 **CPU** 利用率。CPU 只参与两次高速传送：从用户空间拷贝到内核缓冲区（内存 → 内存，极快）、井管理程序把数据写入磁盘输出井（内存 → 磁盘，很快）。真正的「磁盘井 → 打印机」由独立的输出进程处理，可通过 DMA 进一步减少对 CPU 的打扰。
- 实现计算与 **I/O** 操作的真正并行。CPU 全力执行进程 B 的计算任务，输出进程把磁盘井中进程 A 的打印数据送往打印机，输入进程从读卡器为进程 C 准备输入数据存入输入井——输入、计算、输出宏观上同时进行。

- 提高系统吞吐量。综合以上优点，单位时间内完成的总任务量得到质的飞跃。

关于 SPOOLing 的三个易错判断

1. **SPOOLing** 不能缩短物理设备的传输时间。它并不改变打印机本身的速度，只是让 CPU 不必等待这个慢速过程。认为「SPOOLing 缩短了数据在 I/O 设备与内存之间的物理传输时间」是错误的。
2. 用户进程的大部分等待时间不发生在对磁盘井的读写上。恰恰相反，对磁盘井的读写非常快、等待时间极短，这才是 SPOOLing 的优势所在。
3. 输出进程不是由用户作业创建的，它是一个系统级的后台服务进程（守护进程），由操作系统启动时创建和管理，通常常驻内存而非按需创建，数据来源是输出井而不是任何用户作业的地址空间。

5.SPOOLing 与脱机 *I/O* 的区别与联系

比较项	脱机 **I/O**	**SPOOLing**（假脱机）
是否需要专用外围机	需要物理的外围控制机	不需要，用输入/输出进程模拟外围机的功能
由谁控制	脱离主机控制，由外围机独立完成	在主机直接控制下由系统进程完成
依赖的技术	外围机硬件 + 磁带/磁盘	多道程序设计技术 + 通道技术 + 高速外存
共同点	都用高速外存做缓冲，都实现低速 I/O 与 CPU 计算的并行，都能把一个独占设备虚拟为多个逻辑设备	同左

一句话总结联系与区别：**SPOOLing** 是脱机 **I/O** 思想的「软件化」——用多道程序和磁盘空间替代了物理外围机，从而在主机直接控制下取得与脱机 **I/O** 同样的并行效果。

（四）设备驱动程序接口

*1.*为什么需要统一接口

外设种类繁多、控制接口不一致，不同外设的设备驱动程序千差万别。如果计算机系统中每出现一种新外设，都要为添加新驱动程序而修改操作系统，会给操作系统开发者和系统用户带来很大麻烦。

因此操作系统为所有外设的设备驱动程序规定了统一的接口：新设备驱动程序只要按统一接口规范编制，就可以在不修改操作系统的情况下在系统中添加新驱动程序并使用新外设进行 I/O。

*2.*统一接口的实现方式

对每种设备类型（如磁盘），操作系统定义一组驱动程序必须支持的函数（磁盘至少包含读、写、格式化等）。驱动程序中通常包含一张表格，这张表格具有针对这些函数指向驱动程序自身实现的指针（函数指针表）。装载驱动程序时操作系统记录该表的地址，需要调用某个函数时通过这张表发出间接调用。这个函数指针表定义了驱动程序与操作系统其余部分之间的接口，给定类型的所有设备都必须服从这一要求。

工作流程举例：应用程序需要从某块 SSD 读数据，内核上层软件发出通用的 `read()` 请求，内核根据目标设备查找「该 SSD 驱动的函数指针表」，看到标准 `read()` 接口对应的是 `samsung_ssd_read()` 的地址，于是间接调用该函数完成实际的硬件操作。

统一接口的价值示例：一台采用统一编址（内存映射）的打印机驱动可能用 `memcpy` 把整页数据拷到大块内存缓冲区，用直接读内存地址的方式获取状态；而一台采用独立编址的打印机驱动则用 `outb` 循环逐字节写数据端口，先发命令再从端口读回状态字节。但两者向操作系统提供的接口函数签名完全相同。上层模块调用时完全不知道也不需要关心数据是 `memcpy` 过去的还是一个字节一个字节发出去的。这就

是设备驱动程序接口的强大之处：把上层软件的调用意图与下层硬件的复杂实现彻底解耦。

*3.*设备名到驱动程序的映射

因为采用了统一的设备驱动程序接口，内核中与设备无关的 I/O 软件包含了所有外设统一公共接口中的处理部分。为简化处理，内核高层 **I/O** 软件把所有外设都抽象成一个文件，设备名和文件名在形式上没有任何差别，称为设备文件名。内核中与设备无关的 I/O 软件必须把不同的设备名和文件名映射到对应的设备驱动程序。

在 UNIX/Linux 中，一个设备名唯一确定一个特殊文件的 **i** 节点：i 节点中包含主设备号，主设备号用于定位相应的设备驱动程序；i 节点还包括次设备号，作为参数传给设备驱动程序，用来确定进行 I/O 操作的具体设备位置。除了普通文件和目录文件，还有一类特殊文件包括设备文件、链接文件等，设备特殊文件又分块设备特殊文件和字符设备特殊文件。

关于设备驱动程序的一道辨析题

「对于设备驱动程序的描述，错误的是：不同型号的同类设备提供给相同上层系统的接口可能不同。」

解析：同一个操作系统对同类型的所有设备提出了统一的接口规范要求，因此不同型号的同类设备为同一个系统提供的接口是一样的；而不同系统提出的接口规范可能不一样，所以「同一型号的同类设备提供给不同上层系统的接口可能不同」是正确的。此外，设备驱动程序可以通过 I/O 端口访问设备（正确）；用户对设备的访问依靠驱动程序提供的接口，设备自身的中断处理也需要驱动程序接口支持（正确）。

另一道题:「操作系统为所有块设备定义统一的设备驱动程序接口，最主要的目标是——向上层软件（如文件系统）隐藏不同硬件设备的物理实现差异」。接口的目的不是提高 CPU 访问寄存器的效率，也与编程语言无关，更不只是简化中断流程。

三、外存管理

（一）磁盘

*1.*磁盘结构

磁盘存储器由记录介质（盘片）、磁盘驱动器、磁盘控制器三大部分组成。

- 磁盘控制器：磁盘存储器与主机的 **I/O** 接口，包括控制逻辑、时序电路、「并 → 串」转换和「串 → 并」转换电路。
- 磁盘驱动器：包括读写电路、读/写转换开关、读/写磁头与磁头定位伺服系统。
- 盘片：涂有磁性介质的物理盘片，通过设置磁场方向写入 0 和 1。

五个基本术语：

术语	定义
盘面（**surface**）	盘片的一个面。每个盘片的两个面各有一个磁头，因此盘面号就是磁头号
磁道（**track**）	磁头与盘片相对运动形成的圆。磁头位于不同半径上得到不同磁道。磁道从外向里编址，最外面的为 **0** 号磁道
柱面（**cylinder**）	所有盘面上相同编号的磁道构成的圆柱面。因此磁道号就是柱面号
扇区（**sector**）	每个磁道被分成的若干圆弧，是磁盘可寻址的最小单位，以扇区为单位进行读写
磁头（**head**）	每个盘面对应一个磁头，同一柱面上所有磁头「共同进退」

关键性质：

- 读写磁盘时总是写完一个柱面上所有的磁道后，再移到下一个柱面。这个性质正是「柱面号在地址高位」的物理原因。
- 一个磁道存储的信息量相同，所以内磁道的位密度比外磁道大。
- 硬盘属于机械式部件，读写操作是串行的，不可能在同一时刻既读又写，也不可能在同一时刻读两组数据或写两组数据。

磁盘的分类（了解）：按磁头相对盘片径向是否固定分为固定头磁盘（每个磁道一个磁头）和活动头磁盘（磁头臂可伸缩定位磁道）；按盘片是否可更换分为固定盘磁盘和可换盘磁盘。

*2.*格式化与分区

磁盘从出厂到可用，要经过「低级格式化 → 分区 → 高级格式化 → 安装操作系统」四步。

第一步：低级格式化（物理格式化）

在磁盘能存储数据之前进行，主要工作：

1. 把磁盘划分为扇区，以便磁盘控制器能读写数据。每个扇区的数据结构包括头部、数据区域和尾部，头部和尾部包含扇区号和纠错代码（**ECC**）。控制器写入数据时 ECC 根据数据区域所有字节计算并更新；读取时重新计算并与存储值比较，不一致则表明数据损坏。ECC 能纠正少量错误并报告可恢复的软错误，读写时由控制器自动处理。
2. 检测并标记坏道，避免存储数据时使用这些损坏区域。
3. 初始化逻辑块号到无损磁盘扇区的映射，使制造商能够测试磁盘。

第二步：分区

把磁盘划分为多个独立部分，每个分区可被独立管理和使用。每个分区由一个或多个柱面组成，每个分区的起始扇区和大小都记录在磁盘主引导记录的分区表中。分区类型包括：主分区（每个磁盘最多四个，其中一个可标记为活动分区用于引导操作系统）、扩展分区（为克服主分区数量限制，可包含多个逻辑分区）、逻辑分区（在扩展分区内部创建）。操作系统可把每个分区当作一个单独磁盘，例如一个分区存操作系统可执行代码、另一个存用户数据。分区对于实现多个 **OS** 并存是必需的。

第三步：高级格式化（逻辑格式化）

在分区内创建文件系统，包括：

- 创建文件系统：定义数据在磁盘上的组织方式（FAT32、NTFS、ext4 等）。
- 设置文件系统结构：设置超级块（**superblock**）、**inode** 表、目录结构等。
- 初始化目录和文件表：设置根目录和必要的文件表，以及对保存空闲磁盘块信息的数据结构（如位示图、空闲表）进行初始化。

为提高效率，大多数操作系统把多个相邻扇区组合成更大的块，称为簇（**cluster**）。一簇只能存放一个文件的内容，文件占用空间只能是簇的整数倍，文件大小小于一簇（甚至 0 字节）也要占用一簇。磁盘 **I/O** 按块（扇区）完成，而文件系统 **I/O** 按簇完成，这有效确保了 I/O 有更多的顺序访问和更少的随机访问。

第四步：安装操作系统（在逻辑格式化之后执行）。

▌格式化各步考什么

【2017-29】磁盘逻辑格式化程序所做的工作是：建立文件系统的根目录（Ⅱ）、对保存空闲磁盘块信息的数据结构进行初始化（Ⅳ）。

- 「对磁盘进行分区（Ⅰ）」属于分区操作；
- 「确定磁盘扇区校验码所占位数（Ⅲ）」属于低级格式化（物理格式化）的工作。

答案选 B（仅Ⅱ、Ⅳ）。

【2017-26】某文件系统的簇和磁盘扇区大小分别为 1KB 和 512B，文件大小 1026B，系统分配给该文件的磁盘空间是：需要簇数 $\lceil 1026/1024 \rceil = 2$，分配空间 $2 \times 1024\text{B} = 2048\text{B}$。按簇分配、不足一簇也占一簇，答案选 D。

坏块的处理：简单磁盘在格式化时扫描并标记坏块，文件系统不再分配它们；正常操作时才变坏的块需人工运行特殊程序搜索锁定，其中数据通常丢失。更复杂的磁盘由控制器维护坏块列表，低级格式化时预留一些备用块（操作系统看不到），用备用块逻辑地替代坏块，称为扇区备用（**sector sparing**）或扇区转寄（**sector forwarding**）。总之对坏块的处理实质上就是用某种机制使系统不去使用坏块。

*3.*磁盘地址结构

磁盘的物理地址结构为：

驱动器号	柱面（磁道）号	盘面（磁头）号	扇区号
一台电脑可能有多个硬盘	移动磁头臂（寻道）	激活某个磁头	通过旋转把特定扇区转到磁头下方

实际计算题中通常只考虑后三段，即（柱面号，盘面号，扇区号），三段都从 0 开始编号。把簇号转换成磁盘物理地址的过程由磁盘驱动程序完成。

为什么柱面号在前

假设磁盘有 8 个柱面、4 个盘面、8 个扇区，则需要各 3、2、3 个二进制位。

- 若地址结构是（盘面号，柱面号，扇区号）：连续读取 $(00,000,000) \sim (00,001,111)$ 时，同一柱面不同盘面的部分可以不移动磁头换盘面磁头读完，但要读到下一个柱面 $(00,001,000)$ 起就必须启动磁头臂把磁头移到下一个磁道。
- 若地址结构是（柱面号，盘面号，扇区号）：连续读取 $(000,00,000) \sim (000,01,111)$ 时，柱面号相同、只是盘面号不同，不需要移动磁头臂，只需激活相邻盘面的磁头即可，节省大量时间。

结论：把柱面号放在高位，使逻辑上相邻的块在物理上尽量落在同一柱面内，从而把「寻道」这个最耗时的操作推迟到万不得已时才做。这是「柱面号在地址高位」的根本原因。

*4.*磁盘的性能指标与访问时间

（**1**）记录密度：沿磁道分布方向上单位长度内的磁道数目叫道密度；沿磁道方向上单位长度内存放的二进制信息数目叫位密度。

- 低密度存储方式：所有磁道上的扇区数相同，因此内道的位密度比外道高。
- 高密度存储方式：每个磁道上的位密度相同，因此外道的扇区数比内道多，整个磁盘容量比低密度盘高得多。

（**2**）存储容量：未格式化容量按道密度和位密度计算，包括头空、ID 域、CRC 码等信息，是可用磁化单元的总数；格式化后的实际容量只包含数据区，一般比未格式化容量小 10%～20%。

（**3**）数据传输速率：完成磁头定位和旋转等待后单位时间内读出或写入的二进制信息量。由于磁盘同一时刻只有一个磁头读写，内部数据传输速率等于单位时间内磁头划过的磁道弧长乘以位密度：

$$\text{内部数据传输速率} = \frac{\text{每分钟转速}}{60} \times \text{内圆周长} \times \text{最内道位密度}$$

若转速为 r（转/秒）、每条磁道容量为 N 字节，则 $D = rN$。

（**4**）平均存取时间：

$$\text{响应时间} = \text{排队延迟} + \text{控制器时间} + \text{寻道时间} + \text{旋转等待时间} + \text{数据传输时间}$$

后三项之和称为存取时间：

$$\boxed{\text{存取时间} = \text{寻道时间} + \text{旋转等待时间} + \text{数据传输时间}}$$

时间项	公式	说明
寻道时间 T_s	$T_s = s + m \cdot n$	s 为启动磁头臂的时间（约 2ms），m 为磁头每跨越一条磁道的耗时（约 0.2ms），n 为需要跨越的磁道数
旋转延迟（延迟时间）T_r	$T_r = \frac{1}{2} \cdot \frac{1}{r} = \frac{1}{2r}$	r 为转速（转/秒）。定位过程中最多旋转 1 圈、最少不用旋转，所以平均需要旋转半圈，故除以 2。若转速为 r 转/分，则旋转一圈的时间为 $60/r$ 秒
传输时间（读数据）T_t	$T_t = \frac{b}{N} \cdot \frac{1}{r} = \frac{b}{rN}$	转速 r（转/秒），本次读/写字节数为 b，每条磁道字节数为 N。b 字节需 b/N 条磁道，读写一条磁道刚好是转一圈的时间 $1/r$
传输时间（访问一个扇区）	$\frac{\text{旋转一圈的时间}}{\text{一条磁道的扇区总数}} = \frac{\text{一个扇区的容量}}{\text{数据传输率}}$	用于求一个扇区扫过磁头的时间

转速单位的陷阱

转速可以用 r 转/秒或 r 转/分表示，有些题目会给出「r 秒/转」或「r 分/转」，此时公式需要相应取倒数。看到「7200rpm」要立刻意识到这是转/分，旋转一圈的时间是 $60/7200$ 秒 $= 8.33$ ms，平均旋转延迟为 4.17 ms。单位判断错误是这类题失分的第一大原因。

另外注意：磁盘的传输速率中 **K**、**M**、**G**、**T** 一般以 **10** 为底（$1\text{K} = 10^3$、$1\text{M} = 10^6$、$1\text{G} = 10^9$、$1\text{T} = 10^{12}$），不是以 2 为底。

磁盘访问时间完整算例（**2022-44** 大题原型）

某磁盘驱动器有 4 个双面盘片，每个盘面 20000 个磁道，每个磁道 500 个扇区，每个扇区 512 字节，转速 7200rpm，平均寻道时间 5ms。

（**1**）地址信息的三个字段名称及各占多少位

三个字段是柱面号、磁头号（盘面号）、扇区号（CHS）。

- 柱面号：磁道数 20000，需 $\lceil \log_2 20000 \rceil = 15$ 位；
- 磁头号：4 个双面盘片共 $4 \times 2 = 8$ 个盘面（8 个磁头），需 $\lceil \log_2 8 \rceil = 3$ 位；
- 扇区号：每道 500 个扇区，需 $\lceil \log_2 500 \rceil = 9$ 位。

（**2**）一个扇区的平均访问时间

转速 7200rpm $= 120$ 转/秒，旋转一圈 $1/120$ s $= 8.33$ ms。

- 平均旋转延迟 $= \dfrac{1}{2} \times 8.33$ ms $= 4.17$ ms；
- 传输时间（一个扇区）$= \dfrac{8.33 \text{ ms}}{500} = 0.017$ ms（也等于 512B/ 数据传输率）；
- 平均寻道时间 $= 5$ ms。

$$T_a = 5 + 4.17 + 0.017 \approx 9.19 \text{ ms}$$

（**3**）周期挪用 **DMA** 下总线请求次数

磁盘控制器数据缓冲区 64 位 $= 8$ 字节，一个扇区 512 字节需 $512/8 = 64$ 次传送，故一个扇区读写过程中 **DMA** 控制器向 **CPU** 发送 **64** 次总线请求。若 CPU 检测到 DMA 总线请求时也需要访问主存，**DMA** 控制器仍可获得总线使用权，因为 **DMA** 的总线请求优先级高于 **CPU**，以防止高速设备的数据来不及传送而被覆盖丢失。

（**4**）磁盘容量

$$\text{容量} = \text{盘面数} \times \text{每面磁道数} \times \text{每道扇区数} \times \text{每扇区字节数} = 8 \times 20000 \times 500 \times 512 \text{ B} \approx 40.96 \text{ GB}$$

5.磁盘调度方法

磁盘调度的主要目标是减少平均寻道时间：寻道时间在存取时间中占主导且直接受调度算法影响；旋转延迟与传输时间都与转速成反比，属于硬件参数，OS 层面难以优化。

下面统一使用一组经典请求序列贯穿演示所有算法：磁头初始位于 **100** 号磁道，请求队列（按到达顺序）为

$$55,\ 58,\ 39,\ 18,\ 90,\ 160,\ 150,\ 38,\ 184$$

若算法需要方向，约定磁头先沿磁道号增大的方向移动；磁盘磁道号为 $0 \sim 199$。

（1）先来先服务（FCFS）

按进程请求访问磁盘的先后顺序进行调度，是最简单的调度算法。

访问顺序：$100 \to 55 \to 58 \to 39 \to 18 \to 90 \to 160 \to 150 \to 38 \to 184$

移动距离：$45 + 3 + 19 + 21 + 72 + 70 + 10 + 112 + 146 = 498$ 个磁道，平均寻道长度 $= 498/9 \approx 55.3$。

- 优点：公平、简单。请求较少且访问的磁道比较集中时，性能尚可。
- 缺点：大量进程竞争磁盘、请求磁道很分散时，磁头频繁长距离移动，寻道时间长。

（2）最短寻道时间优先（SSTF）

每次优先处理与当前磁头距离最近的磁道。可以保证每次寻道时间最短，但不能保证总的寻道时间最短（本质是贪心）。

访问顺序：$100 \to 90 \to 58 \to 55 \to 39 \to 38 \to 18 \to 150 \to 160 \to 184$

移动距离：$10 + 32 + 3 + 16 + 1 + 20 + 132 + 10 + 24 = 248$ 个磁道，平均 $= 248/9 \approx 27.5$。

> 另一种等价的移动距离算法：从 100 先一路减到 18 再跳到 184，即 $(100 - 18) + (184 - 18) = 82 + 166 = 248$。

- 优点：性能优于 FCFS，平均寻道时间短。
- 缺点：可能产生「饥饿」现象——磁头在一个小区域内来回移动，较远的磁道（如 184 号）长期得不到服务；每次最短不一定全局最短。

（3）扫描算法（SCAN，电梯调度算法）

为避免 SSTF 的饥饿问题，规定磁头只有在到达最外侧磁道后才转向内侧移动，到达最内侧磁道后才转向外侧移动，在 SSTF 基础上引入了磁头移动方向的约束。因为行为像电梯，所以也叫电梯调度算法。

访问顺序（先向磁道号增大方向，抵达 199 后折返）：$100 \to 150 \to 160 \to 184 \to 199 \to 90 \to 58 \to 55 \to 39 \to 38 \to 18$

移动距离：$(199 - 100) + (199 - 18) = 99 + 181 = 280$ 个磁道（若按「移动到端点」计）。若按王道教材把端点也算作经过的磁道（即含 200 号），则为 $(50 + 10 + 24 + 16) + (110 + 32 + 3 + 16 + 1 + 20) = 282$ 个磁道，平均 $282/9 \approx 31.3$。

- 优点：寻道性能较好，可避免「饥饿」现象。
- 缺点：① 只有到达最边上的磁道才能改变方向，而处理完 184 号请求后其实已不需要继续外移；② **SCAN** 算法对各个位置磁道的响应频率不平均，不利于远离磁头当前方向的那一端——刚处理过 90 号的位置下次要等磁头走很长一段距离，而 184 号附近刚被服务过很快又会轮到。

（4）循环扫描算法（C-SCAN）

针对 SCAN 响应频率不平均的问题：规定磁头只朝某个特定方向移动时才处理磁道访问请求，返回时直接快速移动到起始端而不处理任何请求。

访问顺序（向磁道号增大方向服务，到 199 后快速回到 0，再向增大方向服务）：$100 \to 150 \to 160 \to 184 \to 199 \to 0 \to 18 \to 38 \to 39 \to 55 \to 58 \to 90$

移动距离（按王道含端点计法）：$50+10+24+16+200+18+20+1+16+3+32=390$ 个磁道，平均 $390/9\approx 43.3$。

- 优点：比起 SCAN，对各个位置磁道的响应频率很平均，消除了对两端磁道请求的不公平。
- 缺点：① 仍要求到达最边上磁道才改变方向；② 磁头返回时其实只需返回到 18 号磁道即可，不必回到最边缘；③ 平均寻道时间比 SCAN 更长。

（5）LOOK 与 C-LOOK

SCAN 和 C-SCAN 都要求磁头严格从磁盘一端移动到另一端。实际上可以改进：磁头只需移动到当前方向上最远的请求位置即可返回，无须抵达物理端点。改进后的算法分别称为 **LOOK** 和 **C-LOOK**（因为朝某方向移动前会「查看」该方向是否还有待处理请求）。

LOOK：访问顺序 $100\to 150\to 160\to 184\to 90\to 58\to 55\to 39\to 38\to 18$，移动距离 $(184-100)+(184-18)=84+166=250$ 个磁道，平均 $250/9\approx 27.5$。

C-LOOK：访问顺序 $100\to 150\to 160\to 184\to 18\to 38\to 39\to 55\to 58\to 90$，移动距离 $(184-100)+(184-18)+(90-18)=84+166+72=322$ 个磁道，平均 $322/9\approx 35.8$。

各算法的移动距离对照表（同一请求序列）

磁头初始 100，请求序列 $55, 58, 39, 18, 90, 160, 150, 38, 184$（9 个请求），磁道范围 $0\sim 199$，初始方向为磁道号增大：

算法	访问顺序	移动总距离	平均寻道长度
FCFS	55, 58, 39, 18, 90, 160, 150, 38, 184	**498**	55.3
SSTF	90, 58, 55, 39, 38, 18, 150, 160, 184	**248**	27.5
SCAN（到端点）	150, 160, 184, 199, 90, 58, 55, 39, 38, 18	**280**（含端点写法 282）	31.3
LOOK	150, 160, 184, 90, 58, 55, 39, 38, 18	**250**	27.5
C-SCAN（到端点）	150, 160, 184, 199, 0, 18, 38, 39, 55, 58, 90	**390**	43.3
C-LOOK	150, 160, 184, 18, 38, 39, 55, 58, 90	**322**	35.8

读表要点：① **SSTF** 和 **LOOK** 在这组数据上移动距离相同（**248 vs 250**），说明「贪心最近」和「带方向的最近」在请求分布均匀时效果接近；② **C-SCAN** 因强制回到端点而最差，C-LOOK 是它的改进；③ 若题目没有特别说明，**SCAN** 就按 **LOOK** 理解、**C-SCAN** 就按 **C-LOOK** 理解（见下条提示）。

做磁盘调度计算题的四个关键约定

1. 若题目中无特别说明，**SCAN** 就是 **LOOK**，**C-SCAN** 就是 **C-LOOK**。反过来，**SCAN** 与 **C-SCAN** 必须到达端点，**LOOK** 与 **C-LOOK** 可以跳过端点。判断依据是题干的措辞：「扫描算法」「电梯算法」通常按 LOOK 处理；明确说「移动到最外侧/最内侧」「到达端点」才按 SCAN 处理。
2. 必须先确定磁头的初始移动方向。方向不同，访问序列和移动距离都不同。若题目只说「从外侧向内侧移动」，由于磁道号由外向内递增，即向磁道号增大的方向移动。
3. 注意「移动距离」是否包含回头到端点的部分。C-SCAN 必须计入从一端快速回到另一端的距离（如 199→0 距离 199）。
4. 注意请求序列中可能包含与当前磁头位置相同的磁道（如 2021 年题中当前在 184 且请求序列首项就是 184）。此时访问该磁道移动距离为 0，但必须计入「已完成的请求」，且它会被算法安排在某个位置处理（SSTF 会立刻处理它）。

▌磁盘调度真题四个算例

【2009-29】磁头在 105 道、向磁道号增大方向移动，请求序列 $35, 45, 12, 68, 110, 180, 170, 195$，采用 **SCAN**（电梯）算法得到的访问序列是：

小于 105 的请求 $12, 35, 45, 68$；大于 105 的请求 $110, 170, 180, 195$。SCAN 先沿当前方向依次处理：$110 \to 170 \to 180 \to 195$（到端点后折返），再按递减顺序访问剩余：$68 \to 45 \to 35 \to 12$。

答案 **A**：**110, 170, 180, 195, 68, 45, 35, 12**。

【2015-32】硬盘 200 个磁道（$0 \sim 199$），请求序列 $130, 42, 180, 15, 199$，磁头在 58 号磁道从外侧向内侧移动（即磁道号增大方向），按 **SCAN** 处理完后磁头移过的磁道数：

先处理当前方向上的请求：$58 \to 130 \to 180 \to 199$，距离 $199 - 58 = 141$；到达 199 后折返，处理剩余请求 $42 \to 15$，距离 $199 - 15 = 184$。总距离 $= 141 + 184 = 325$。

答案 **C**（**325**）。

【2018-30】磁臂黏着（系统总是访问磁盘的某个磁道而不响应对其他磁道的访问请求）不会出现在哪种算法中？

答案 **A**：**FCFS**。FCFS 严格按请求到达顺序服务，不会因为某个磁道离当前磁头近而反复优先服务它，已进入队列的请求只需等待有限个更早到达的请求。**SSTF** 尤其容易因不断优先处理近距离请求而使远距离请求长期等待；SCAN 与 C-SCAN 在特定热点请求下也可能黏着（这正是 NStepSCAN 和 FSCAN 要解决的问题）。

【2024-32】磁盘 400 个磁道（$0 \sim 399$），采用 **C-SCAN** 调度，完成对 200 号磁道的请求后磁头向磁道号减小的方向移动，还有 7 个请求：$300, 120, 110, 0, 160, 210, 399$。求磁头移动总距离。

C-SCAN 只沿一个方向扫描，到端点后直接回到另一端再沿原方向继续扫描。

第一步递减扫描：先处理小于 200 的请求，按递减访问 $200 \to 160 \to 120 \to 110 \to 0$，距离 $200 - 0 = 200$。第二步循环回卷：到达 0 后磁头直接移动到 399，距离 $399 - 0 = 399$，399 号本身有请求，到达后立即处理。第三步继续递减扫描：$399 \to 300 \to 210$，距离 $399 - 210 = 189$。

完整访问顺序：$160 \to 120 \to 110 \to 0 \to 399 \to 300 \to 210$，总距离 $200 + 399 + 189 = 788$。

答案 **C**（**788**）。

*6.*减少旋转延迟的优化：交替编号与错位编号

磁盘是连续自转设备，磁头读/写一个物理块后需要经过短暂的处理时间才能开始读/写下一块。在这个处理时间里，磁盘仍在继续旋转，如果逻辑上相邻的两个扇区在物理上也相邻，那么等 CPU 处理完当前块时，下一个扇区已经转过去了，就必须再等磁盘转一整圈才能读到它，造成严重的延迟。

解决办法是在低级格式化时做两种编号优化：

1. 交替编号（对一个盘面内的扇区）：把逻辑上连续的扇区在物理上隔开存放，即物理编号按「1, 3, 5, ⋯, 2, 4, 6, ⋯」这类方式排布。这样在读完一个扇区、经过处理时间后，下一个逻辑扇区刚好转到磁头下方，避免多转一圈。
2. 错位编号（对不同盘面的相同位置）：同柱面不同盘面的扇区错位编号，连续读/写相邻两个盘面的逻辑记录时也能减少磁头延迟时间。

一句话记忆：交替编号解决「同一盘面内相邻扇区」的延迟，错位编号解决「同一柱面内不同盘面」的延迟。两者都在低级格式化时完成。

*7.*提高磁盘 *I/O* 速度的其他方法

方法	做法
磁盘高速缓存	在内存中开辟内核空间缓存最近读写的盘块。逻辑上属于磁盘，物理上属于内存
调整磁盘请求顺序	用磁盘调度算法减少寻道时间

方法	做法
提前读（预读）	顺序访问文件时，读当前块的同时把下一个盘块也读入缓冲区
延迟写	缓冲区数据本应立即写回磁盘，但考虑到可能不久后再被访问，把它挂在空闲缓冲区队列末尾。随着空闲缓冲区使用逐渐前移，到达队首并被申请时才真正写回磁盘。只要它仍在队列中，任何访问该数据的进程都可直接读出而不必访问磁盘
优化物理块分布	用位示图时找一片相邻空闲盘块；用空闲盘块链表时把同一磁道上的若干盘块组成一簇，以簇为单位分配。这样访问这几个盘块时不必移动或较少移动磁头
虚拟盘（**RAM** 盘）	用内存空间仿真磁盘，对用户透明。主要问题是内存易失，掉电或故障数据会丢失，因此常存放临时文件。与磁盘高速缓存的区别是虚拟盘的内容完全由用户控制，而磁盘高速缓存的内容由 **OS** 控制
磁盘阵列 **RAID**	用多个独立物理磁盘组成逻辑盘，数据分割交叉存储、并行访问

「不能改善磁盘 **I/O** 性能」的选项

【2012-32】下列不能改善磁盘设备 I/O 性能的是「在一个磁盘上设置多个分区」。

设置多个分区的主要目的是逻辑隔离、安装多个文件系统或便于管理，不能直接减少寻道、旋转延迟或传输次数。而重排 I/O 请求次序（磁盘调度）、预读与滞后写（延迟写）、优化文件物理块分布都能直接改善性能。

【2015-28】在系统内存中设置磁盘缓冲区的主要目的是「减少磁盘 **I/O** 次数」。缓冲区不能直接缩短平均寻道时间（那是磁盘调度的事），也不是可靠性或设备独立性的主要实现机制。

（二）固态硬盘

*1.*固态硬盘的结构

固态硬盘（**Solid State Disk**，**SSD**）基于闪存（**Flash Memory**）技术，属于电可擦除 **ROM**（**EEPROM**）。其存储介质与 U 盘类似，但容量更大、存取性能更好。

组成：

- 闪存翻译层（**Flash Translation Layer**）：负责翻译逻辑块号、找到对应的页（**Page**），相当于磁盘控制器。
- 存储介质（闪存颗粒）：多个闪存芯片 → 每个芯片包含多个块（**block**） → 每个块包含多个页（**page**）。

层次关系：闪存芯片（chip）→ 块（block）→ 页（page）。读写的单位是页，擦除的单位是块，块大于页。

*2.*读写性能特性

固态硬盘的五条读写特性

1. 以页为单位读/写（相当于磁盘的「扇区」）。注意：需要写某页信息时，必须先对该页所在的块进行擦除操作。
2. 以块为单位擦除。擦干净的块，其中的每一页都可以写一次、读无限次。
3. 支持随机访问。系统给定一个逻辑地址，闪存翻译层可以通过电路迅速定位到对应的物理地址，没有寻道时间和旋转延迟。
4. 读快、写慢。要写的页如果已有数据，则不能直接写入，必须先把块内其他页全部复制到一个新的（已擦除的）块中，再写入新的页。这就是写放大的根源：一次看似很小的写操作，实际可能触发整个块的搬迁与重写。
5. 容易磨损。SSD 的一个「块」被擦除次数过多（反复写同一个块）可能会坏掉，而机械硬盘的扇区不会因为写的次数太多而坏掉。

与机械硬盘的对比：

比较项	机械硬盘（**HDD**）	固态硬盘（**SSD**）
访问方式	移动磁臂 + 旋转盘片定位，有寻道时间和旋转延迟	用电路控制访问位置，随机访问性能高
随机读性能	差（受机械延迟限制）	远优于机械硬盘
写性能	与读性能接近	写比读慢（需先擦除，有写放大）
噪音与抗震	有噪音、怕震动	安静无噪音、耐摔抗震
能耗与价格	能耗高、单位容量便宜	能耗低、单位容量更贵
寿命机制	扇区不会因写次数多而损坏	块被擦除次数过多会损坏

三组容易混淆的单位与操作

1. 闪存的「擦除」单位是块（**block**），「读/写」单位是页（**page**）。题目常问「写一页需要先擦除什么」，答案是该页所在的整块，不是该页。
2. 闪存翻译层不是设备控制器——它功能上相当于磁盘的磁盘控制器（负责逻辑地址到物理地址的翻译），但它是 SSD 内部的一个软件/固件层。
3. **SSD**「读快写慢」不是说它比机械硬盘慢。SSD 的写速度仍然远快于机械硬盘，只是相对于它自己的读速度而言较慢。

*3.*磨损均衡

为什么需要磨损均衡：SSD 的块有擦除次数上限，如果反复写同一部分块，这些块会先损坏，而其余块还远未达到寿命，整盘就提前报废。磨损均衡（**Wear Leveling**）的思想就是把「擦除」平均分布到各个块上，以提升整体使用寿命。

动态磨损均衡与静态磨损均衡

- 动态磨损均衡：写入数据时，优先选择累计擦除次数少的空闲（新）闪存块。实现简单，但只对「有写入发生的块」起作用——那些长期只被读、从不被写的数据所占据的块，其擦除次数一直很低，无法参与均衡。
- 静态磨损均衡：SSD 控制器监测并自动进行数据分配与迁移，让老旧的闪存块承担以读为主的存储任务，让较新的闪存块承担更多的写任务。即使没有新数据写入，控制器也会定期扫描并搬移冷数据，主动制造写入机会来均衡磨损。静态磨损均衡比动态磨损均衡更优秀、更彻底，因为它是唯一能解决「冷数据长期占据低磨损块」问题的策略。

实际效果：得益于磨损均衡算法，SSD 的实际使用寿命显著提升。例如一块 256GB 的 SSD，若其闪存块的擦除次数上限为 3000 次左右，通过磨损均衡可以使整个盘的总写入量接近「块数 × 单块擦除上限」，而不是由最先损坏的那一小部分块决定。

SSD 的常考判断

统考考过「下列关于固态硬盘的说法中，错误的是」。常见错误选项有：

- 「固态硬盘的写速度比机械硬盘慢、性能甚至弱于常规硬盘」——错误。SSD 的写速度确实比它自己的读速度慢，但仍远快于机械硬盘。
- 「固态硬盘不需要磨损均衡」——错误。恰恰相反，磨损均衡是 SSD 必不可少的机制。
- 「固态硬盘的擦除单位是页」——错误。读/写单位是页，擦除单位是块。

本章小结表

考点	核心结论
设备分类（按信息交换单位）	块设备（磁盘，可寻址，DMA）；字符设备（键盘打印机，不可寻址，中断驱动）
设备分类（按共享属性）	独占（打印机，静态分配）；共享（磁盘，动态分配，可寻址可随机访问）；虚拟（SPOOLing 改造）
I/O 接口组成	数据缓冲寄存器、状态寄存器、控制寄存器、设备存储器、地址译码与 I/O 控制逻辑
统一编址 vs 独立编址	统一：访存指令访问、占主存空间、译码慢、端口不能 **Cache**；独立：专用 I/O 指令、不占主存空间、译码快、程序灵活性差
轮询方式	CPU 忙等、设备与 CPU 串行、不用中断；分独占查询与定时查询
中断方式	设备主动请求、CPU 与外设并行、每个字/字节中断一次、需保护恢复现场
DMA 方式	请求总线控制权、块传送、不经过 CPU 寄存器、不改变 CPU 现场、整块只中断一次
中断 vs DMA 响应时机	中断：一条指令执行结束后；DMA：一个总线周期结束后
I/O 软件四层（自上而下）	用户层 I/O 软件 → 设备独立性软件 → 设备驱动程序 → 中断处理程序
设备独立性软件职责	公有操作（分配回收、逻辑 → 物理映射、保护、缓冲、差错控制、统一逻辑块）+ 统一接口
字符/块/网络设备接口	字符：get/put、需 open/close、不可寻址；块：read/write/seek、线性块序列；网络：socket API、数据包导向
阻塞 vs 非阻塞 I/O	区别在等待数据准备阶段：阻塞则进程挂起等待；非阻塞则立即返回错误码并轮询
缓冲的四个目的	缓和速度矛盾、减少中断频率、解决粒度不匹配、提高并行性
单缓冲每块时间	$\max(T, C) + M$；n 块总时间 $(n-1)[\max(T, C) + M] + (C + T + M)$
双缓冲每块时间	$\max(T, C + M)$；n 块总时间 $(n-1)\max(T, C + M) + (C + T + M)$
循环缓冲	多个等大缓冲区连成循环队列，维护 `in`/`out` 指针
缓冲池	三个队列（空白、输入、输出）+ 四种工作缓冲区（`hin`/`sin`/`hout`/`sout`）；规律：队头取、队尾挂
缓冲 vs 缓存	缓冲是「中转站」（数据可能是唯一的）；缓存是数据源副本（求命中率）
DCT/COCT/CHCT/SDT	SDT 全系统一张；DCT 每设备一张；COCT 每控制器一张；CHCT 每通道一张。流程 SDT→DCT→COCT→CHCT
安全 vs 不安全分配	安全：分配后立即阻塞，破坏「请求和保持」，不死锁但串行；不安全：可继续请求，可并行但可能死锁
静态 vs 动态分配	静态：运行前全部分配，不死锁，利用率低，用于独占设备；动态：按需申请，利用率高，可能死锁
LUT	⟨ 逻辑设备名，物理设备名，驱动程序入口地址 ⟩；首次分配建表，再次请求直接查表
SPOOLing 组成	输入井/输出井（磁盘）、输入缓冲区/输出缓冲区（内存）、输入进程/输出进程、井管理程序
SPOOLing 三大特点	提高 I/O 速度；把独占设备改造为共享设备；实现虚拟设备（以空间换时间）
SPOOLing 条件	大容量高速度外存 + 多道程序技术 + 通道技术（软件实现）
脱机 vs 假脱机	脱机需物理外围机、脱离主机控制；假脱机用系统进程模拟外围机、在主机控制下完成

考点	核心结论
设备驱动程序统一接口	用函数指针表定义驱动与 OS 其余部分的接口，新增设备无需修改 OS
磁盘地址结构	（柱面号，盘面号，扇区号）或（驱动器号，柱面号，盘面号，扇区号）；柱面号在前，因为同柱面不同盘面换磁头不需寻道
磁盘管理四步	低级格式化（物理格式化）→ 分区 → 高级格式化（逻辑格式化）→ 安装操作系统
存取时间公式	$T_a = T_s + \frac{1}{2r} + \frac{b}{rN}$，其中寻道时间 $T_s = s + mn$ 占主导
FCFS / SSTF	FCFS 公平但寻道长；SSTF 贪心最短，可能饥饿（磁臂黏着）
SCAN / LOOK	SCAN 到端点才折返，方向约束避免饥饿，对两端不公平；LOOK 只到最远请求处折返
C-SCAN / C-LOOK	单向服务、回程不服务，响应频率均匀；C-LOOK 只需回到最远的请求处
交替编号 / 错位编号	同盘面相邻扇区交替编号；同柱面不同盘面错位编号；均在低级格式化时完成
SSD 读写单位	读/写以页为单位，擦除以块为单位；写前需擦除整块
SSD 写放大	写有数据的页要先把它所在块的其他页全部复制到新块再写入
磨损均衡	动态：写时优先选擦除次数少的块；静态：控制器主动迁移，让老块承担读、新块承担写（更优秀）

易错清单

本章 30 个最容易失分的点

1. 「共享设备可以同一时刻被多个进程访问」是错的。共享设备是「一段时间内」可同时访问，微观上交替使用，且必须是可寻址、可随机访问的设备。块设备接口把磁盘的二维结构隐藏成线性块序列，这是文件系统能建立在磁盘上的前提。
2. 统一编址的 **I/O** 端口不能被 **Cache** 缓存，因为接口中的数据是动态变化的，缓存会导致 CPU 读到过时状态。
3. 统一编址不需要专门的 **I/O** 指令，用 `LOAD/STORE` 即可；独立编址需要 `IN/OUT` 这类专用指令，且 I/O 指令是特权指令。
4. 状态寄存器和控制寄存器可以合二为一，因为传送方向相反且 CPU 访问时间错开。
5. 中断类型号在数据线上传输，但它不是设备控制器发出的，而是可编程中断控制器判优后生成的。
6. 中断响应发生在一条指令执行结束后；**DMA** 响应发生在一个总线周期结束后，DMA 响应更快。中断方式请求的是 CPU 处理时间，DMA 方式请求的是总线控制权。
7. **DMA** 数据传送不经过 **CPU** 寄存器，不改变 **CPU** 现场，所以无需保护和恢复现场。
8. 周期挪用方式下，**CPU** 在整个数据块传送过程中并非不能访问主存，只在 DMA 占用那一个周期时让出总线。「CPU 不可以访问主存储器」是「停止 CPU 访问内存」方式的特点。
9. 周期挪用一次实际需 **2～5** 个存储周期，因为总线请求、仲裁、归还本身有开销。
10. **DMA** 与 **CPU** 同时访存时 **DMA** 优先，这是为了防止高速设备数据丢失。
11. 「中断方式适用于所有外部设备」是错的；极高速设备用中断反而效率更低，轮询（如 NVMe SSD 的同步 I/O）反而更优。
12. **DMA** 方式的数据通路位于「设备接口和主存之间」，不是 CPU 和主存之间，也不是设备接口和 DMA 控制器之间（2024 年考过）。
13. **I/O** 软件四层自上而下是：用户层 **I/O** 软件 → 设备独立性软件 → 设备驱动程序 → 中断处理程序。驱动程序必须在设备独立性软件之下，中断处理程序最接近硬件。一次磁盘 I/O 请求的处理流程是：用户程序 → 系统调用处理程序 → 设备驱动程序 → 中断处理程序。

14. 计算柱面号、磁头号、扇区号（**CHS**）的是设备驱动程序，不是用户程序、系统调用程序或中断处理程序（2013 年考过）。
15.「驱动程序与 **I/O** 控制方式无关」是错的。程序查询、中断、DMA 三种控制方式直接决定驱动程序的编码方式。
16. 程序员的系统调用传的是逻辑设备名，物理设备名、主设备号/次设备号是内核与驱动程序使用的（2009、2020 年考过）。主设备号确定设备类型并据此定位驱动程序，次设备号被驱动程序用来确定具体设备。
17. 单缓冲每块时间 $\max(T, C) + M$，M 在 max 之外；双缓冲 $\max(T, C + M)$，M 在 max 之内。这是两者唯一差别，也是判断用哪个公式的关键。
18. n 块数据的总时间不能简单用 $n\times$ 每块时间，必须用 $(n-1)$ 倍流水时间加最后一块的完整串行时间 $(C + T + M)$。
19. 单缓冲、双缓冲、循环缓冲是专用缓冲；缓冲池是共享缓冲。
20. 缓冲池四个工作缓冲区 `hin`**(**收容输入**)**、`sin`**(**提取输入**)**、`hout`**(**收容输出**)**、`sout`**(**提取输出**)**，规律是「取出从队头、挂入到队尾」。
21. **SDT** 全系统只有一张；**DCT**、**COCT**、**CHCT** 分别是每设备、每控制器、每通道一张。分配请求流程是 SDT→DCT→COCT→CHCT，且只有设备、控制器、通道三者都分配成功，设备分配才算成功。
22. 静态分配降低设备利用率，不是提高。它的优点是破坏「请求和保持」从而避免死锁。
23. **SPOOLing** 不能缩短物理设备的传输时间，也不改变设备本身的速度，它只让 CPU 不必等待。
24.「**SPOOLing** 由用户作业控制设备与输入/输出井之间的数据传送」是错的，由系统的输入/输出进程控制，对用户透明。
25. **SCAN/C-SCAN** 必须到端点，**LOOK/C-LOOK** 可跳过端点；若题目无特别说明，**SCAN** 即 **LOOK**、**C-SCAN** 即 **C-LOOK**。
26. 转速单位要看清：7200rpm 是「转/分」，旋转一圈是 $60/7200\text{s} = 8.33\text{ms}$，平均旋转延迟为 4.17ms。若题目给「秒/转」则公式要取倒数。
27. 磁盘传输速率中 **K/M/G/T** 以 **10** 为底（$1\text{M} = 10^6$），不要按 2^{20} 算。
28.「柱面号在前」的原因是同柱面不同盘面换磁头不需要寻道，这是把最耗时的寻道操作尽量推迟。
29. 逻辑格式化做的是「建根目录 + 初始化空闲块管理结构」；分区、确定校验码位数分别属于分区和低级格式化（2017 年考过）。
30. **SSD** 的读/写单位是页，擦除单位是块；写有数据的页必须先擦除它所在的整块。「SSD 的性能弱于机械硬盘」「SSD 擦除单位是页」都是错的。

第四部分

计算机网络

第十九章计算机网络概述

本章考纲要求

第四部分计算机网络［考查目标］

1. 掌握计算机网络的基本概念、基本原理和基本方法。
2. 掌握典型计算机网络的结构、协议、应用以及典型网络设备的工作原理。
3. 能够运用计算机网络的基本概念、基本原理和基本方法进行网络系统的分析、设计和应用。

一、计算机网络概述（一）计算机网络基本概念

1. 计算机网络的定义、组成与功能
2. 计算机网络的分类
3. 计算机网络的主要性能指标

（二）计算机网络体系结构

1. 计算机网络分层结构
2. 计算机网络协议、接口、服务等概念
3. ISO/OSI 参考模型和 TCP/IP 参考模型

本章是计算机网络的入门章，也是整个 25 分的基本框架。统考对本章的考查方式非常固定：每年网络部分的第一道选择题（题号 **33** 起）几乎都落在本章，且集中在两个点上——「体系结构各层的功能与归属」和「性能指标的计算」。前者靠背结论和牢记层次顺序，后者靠把公式代进去算对。本章按考纲顺序把这两块写透，并把历年出现过的原题考法逐条标出。

需要注意本章的一个特点：概念题的选项往往只差一两个字，例如「协议的内部实现细节」「面向连接与可靠的关系」「网络体系结构与协议的区别」，判错就不是算错，而是理解边界不清。所以本章对每个术语都给出「是什么、不是什么」。

一、计算机网络基本概念

（一）定义、组成与功能

*1.*计算机网络的定义

计算机网络（Computer Network）的严格定义是：将地理位置不同的、具有独立功能的多台计算机及其外部设备，通过通信线路和通信设备连接起来，在网络操作系统、网络管理软件及网络通信协议的管理和协调下，实现资源共享和信息传递的计算机系统。

这个定义读起来长，但考试真正要抠的是三个限定词：

- 自治（**autonomous**）：每台计算机都有自己的软硬件系统，能独立运行，相互之间没有主从关系，一台计算机不能强制控制另一台。这是把「计算机网络」与「多终端分时系统」区分开的关键——分时系统里的终端没有自治能力，它只是主机的输入输出设备。

- 互连：计算机之间通过通信链路（双绞线、光纤、无线电波等）和通信设备（集线器、交换机、路由器）连接，能进行数据交换。
- 集合：至少两台，且是「计算机的集合」，而不是「计算机与终端的集合」。

最简洁的记法是：计算机网络 = 一些互相连接的、自治的计算机的集合。

计算机网络与分布式系统的区别

分布式系统（Distributed System）在用户看来像一台统一的计算机，用户提交任务时不需要指出由哪台机器完成；而计算机网络中，用户必须显式地指定使用哪一台计算机（例如要写明访问哪台服务器的地址）。分布式系统是建立在计算机网络之上的软件层，是网络的更高形态。两者不是并列的「另外一种东西」。

*2.*两个容易混淆的名词：*internet* 与 *Internet*

- **internet**（互连网）：通用名词，泛指由多个计算机网络互连而成的计算机网络。这些网络之间可以使用任意通信协议，不一定是 **TCP/IP**。
- **Internet**（互联网 / 因特网）：专用名词，特指当前全球最大的、开放的、由众多网络和路由器互连而成的特定计算机网络，它必须使用 **TCP/IP** 协议族。

统考的写法是：出现在「泛指」语境里用小写 internet，出现在「全球互联网」语境里用大写 Internet。

*3.*计算机网络的组成

考纲要求从三个不同角度说出组成，这三个角度不是互相替代的，而是三种切分方式，必须能分别复述。

（1）从物理组成看：硬件、软件、协议

组成部分	具体内容	地位
硬件	主机（端系统）、通信设备（集线器、交换机、路由器）、传输介质（双绞线、光纤、无线电波）	提供物理基础
软件	网络操作系统、协议软件、网络管理软件、应用软件	提供可用的功能
协议	通信双方必须共同遵守的规则、标准或约定	核心，缺了协议硬件软件无法互通

（2）从工作方式看：边缘部分与核心部分

这是理解因特网结构最重要的一刀。

- 边缘部分：由所有连接在因特网上的主机（端系统）组成。边缘部分由用户直接使用，用来通信（传送数据、音频、视频）和资源共享。边缘部分中两台主机之间的通信方式有两种：
  - 客户 / 服务器方式（**C/S**）：客户是服务请求方，服务器是服务提供方。客户程序必须知道服务器程序的地址，服务器程序不需要知道客户程序的地址，但需要一直运行并等待请求。
  - 对等连接方式（**P2P**）：不区分服务请求方与提供方，每一台主机既是客户又是服务器，可以平等地、直接地通信。
- 核心部分：由大量网络和连接这些网络的路由器组成。核心部分向边缘部分提供连通性和交换服务。核心部分中最关键的设备是路由器，它负责转发分组、选择路径。

边缘与核心的一句话结论

边缘部分是资源与需求的所在地，核心部分是转发与交换的枢纽。核心部分不产生信息，只为边缘部分

提供连通性和交换。判断某设备属于哪一部分，看它是「用户直接使用的主机」还是「负责转发的中间设备」。

（3）从功能看：通信子网与资源子网

- 通信子网：负责数据通信（数据的传输、交换、通信控制与差错控制），由通信控制处理机（如路由器）、通信线路和其他通信设备组成。它对应 OSI 参考模型的下三层（物理层、数据链路层、网络层）。
- 资源子网：负责数据处理（面向应用的数据处理与资源共享），由主机、终端、联网外设、软件与信息资源组成。它对应 OSI 参考模型的上四层（传输层、会话层、表示层、应用层）。

通信子网对应下三层、资源子网对应上四层

这是一个高频的「送分与送命只差一步」的对应关系：下三层 = 通信子网 = 管传输；上四层 = 资源子网 = 管处理。注意切分点落在传输层——传输层属于资源子网，因为它是端到端、面向进程的，已经介入端系统的处理。若题目把传输层划进通信子网，那是错的。

*4.*计算机网络的功能

考纲要求的四大功能，每一条都要能举出例子：

1. 数据通信：计算机网络最基本、最重要的功能，实现计算机与计算机之间各种信息的传递，例如收发电子邮件、文件传输、即时通信。
2. 资源共享：包括硬件资源（打印机、存储设备、处理器）、软件资源（应用程序、工具软件）和数据资源（数据库、文件、资料）的共享。资源共享是计算机网络最有吸引力的功能。
3. 分布式处理：当某个任务非常庞大时，可以把它拆成若干部分，交给网络中多台计算机分工协作完成，从而提高处理能力，例如大规模科学计算、分布式数据库。负载均衡也可归入这一条。
4. 提高可靠性：网络中的计算机可以互为备份。当一台计算机或一条链路出现故障时，任务可以转移到其他计算机或路径上继续运行，从而提升整个系统的可靠性。可替代性（可用性）的另一面就是容错。

有的教材把「负载均衡」单列为第五条功能，统考以考纲的四条为准：数据通信、资源共享、分布式处理、提高可靠性。

本章真题考法（一）

- **2024-33**：给出分组交换网络拓扑与各段链路带宽，问 H1 到 H2 的最大吞吐量。答案是两端接入链路的带宽（本题为 10 Mb/s），中间并行路径无论聚合带宽多大都不能突破端系统的接入瓶颈。陷阱：只看中间链路，忽略源端与目的端的公共瓶颈。
- **2010-33**：问「不属于网络体系结构所描述的内容」——答案是协议的内部实现细节。体系结构只描述层次划分、每层功能、各层使用的协议、层间接口，不描述具体实现。

（二）分类

考纲要求掌握四种分类角度，每一种都要能说出「分几类、依据是什么、各类的特征」。

*1.*按分布范围分类

类型	英文与缩写	覆盖范围	典型例子	关键技术特征
个域网	Personal Area Network, PAN	约 10 m 内	蓝牙、ZigBee、无线耳机	围绕个人设备，速率低、功耗低
局域网	Local Area Network, LAN	10 m ~ 1 km，一个房间、一栋楼、一个校园	以太网、WiFi、令牌环	传输速率高、时延小、误码率低，通常为广播式共享介质
城域网	Metropolitan Area Network, MAN	5 ~ 50 km，一个城市	城市有线电视网、城域以太网	介于 LAN 与 WAN 之间，早期用 DQDB
广域网	Wide Area Network, WAN	几十 ~ 几千 km，跨城市、国家、洲	因特网骨干、公共交换电话网	传输速率相对低、时延大，采用点对点链路与交换技术

局域网与广域网的区分依据不是「范围大小」

统考反复强调：**LAN** 与 **WAN** 的本质区别在于所使用的技术，而不是单纯的地理范围。局域网使用广播式共享介质的传输技术（早期总线型以太网、令牌环），广域网使用点对点链路加交换结点的传输技术。用一个校园网跨越两个城市，它仍是局域网技术。若选项只讲「范围大小」，那是干扰项。

补充一个与「分布范围」并列的分类角度：按传输技术可分为广播式网络与点对点网络。局域网基本采用广播式通信（一个结点发送，其余结点都能收到），广域网基本采用点对点通信。

*2.*按使用者分类

- 公用网（**Public Network**）：由电信运营商等建设、经营，只要按规定交纳费用，任何单位或个人都可以使用，例如公用电话网、公用数据网。特点是面向全社会、使用门槛低、但资源被共享。
- 专用网（**Private Network**）：由某个部门或单位为满足本单位业务需要而建设、使用，不向本单位以外的人开放，例如军队、铁路、银行、电力系统的内部网络。特点是安全性高、可控性强、但建设与维护成本高。

两者的关系：专用网租用公用网的传输线路是很常见的做法——线路是公用网的，逻辑网络是专用网（如 VPN 虚拟专用网）。

*3.*按交换技术分类

这是本章最重要的一组对比，几乎每年都会以计算或对比题的方式出现。

（1）电路交换（Circuit Switching）

通信全过程分三个阶段：建立连接 → 数据传输 → 释放连接。

- 建立连接阶段：在通信双方之间建立一条独占的物理通路，沿途结点为该连接分配通信资源（时隙、频段等）。这一阶段费时较长。
- 数据传输阶段：数据透明地沿这条通路传输，中间结点不对数据做处理，因此不具备差错控制能力。
- 释放连接阶段：归还占用的通信资源。

优点	缺点
通信时延小（通路一旦建立，传输几乎无额外时延）	建立连接时间长
数据按序传输，不会失序	线路利用率低（空闲时资源也被独占）
无冲突，实时性强	灵活性差（一方断开则连接中断，中途难以改路）
控制简单	不具备差错控制能力

电路交换适用于数据量很大且传送时间远大于呼叫（建立连接）时间的场合，典型例子是传统电话网。

（2）报文交换（Message Switching）

采用存储转发技术：整个报文先被传送到相邻结点，全部接收并存储完整后，再查找转发表把它转发到下一个结点，如此重复直到目的结点。

- 报文（Message）是用户数据加上源地址、目的地址等信息构成的完整数据块。
- 因为必须收完整个报文才能转发，无法流水线处理，所以转发时延很长；相应地，交换结点必须能缓存整个报文，缓存空间开销大；一旦出错，整个报文都要重传，出错重传开销大。

优点	缺点
无需建立连接	转发时延高
动态分配线路	缓存开销大
可靠性高	出错重传开销大
线路利用率高	不适用于实时通信

（3）分组交换（Packet Switching）

同样采用存储转发，但把较长的报文划分成若干较小的、长度固定的数据段，每段前面加上首部（分组头，含目的地址、源地址等控制信息），构成一个分组（Packet）。分组交换机根据分组中的目的地址查找自己的转发表进行转发。它解决了报文交换中「报文过长导致转发时延长、缓存开销大」的问题。

- 优点：无需建立连接；线路利用率高（不同应用的分组可以交替使用链路，实现公平共享）；可采用流水线方式加速传输；分组短小，出错概率和出错重传开销都更小。
- 缺点：仍存在转发时延；每个分组都要携带首部，分组头开销比报文交换更大；分组可能出现失序、丢失、重复。

分组交换进一步分为两种方式，都由网络层提供：

对比维度	数据报服务	虚电路服务
连接建立	不需要	必须先建立
目的地址	每个分组都携带完整目的地址	仅建立连接阶段使用，之后每个分组只用短虚电路号
路由选择	每个分组独立选路与转发	同一条虚电路的分组按同一路由转发
分组顺序	不保证有序到达	保证有序到达

对比维度	数据报服务	虚电路服务
可靠性	网络尽力转发，不承诺交付，可靠性由上层协议（如 TCP）保证	可靠性由网络自身保证
对故障的适应性	可重新选路，适应能力强	经过故障结点的全部虚电路均不能工作
差错与流量控制	由用户主机上层协议负责	可由分组交换网负责，也可由主机负责

虚电路不是电路交换

虚电路是分组交换的一种，是逻辑连接；电路交换是真正的物理线路交换。关键区别：电路交换预分配带宽，一条物理线路在通信期间被一条连接独占；虚电路采用多路复用技术，多条虚电路共享同一段物理线路的带宽，分组仍然要存储转发。另外，虚电路服务是 **OSI** 参考模型特有的，TCP/IP 模型的网络层只提供无连接的数据报服务。

*4.*按拓扑结构分类

拓扑	结构描述	优点	缺点	典型场景
星形	所有结点连到中心结点（集线器 / 交换机）	结构简单、控制简单、故障易诊断与隔离、易于扩充	中心结点是瓶颈与单点故障，可靠性差，线缆用量大	以交换机为中心的现代局域网
总线	所有结点共享一条公共总线	结构简单、成本低、扩充方便、可靠性较高（无中心）	冲突严重、介质访问控制复杂、总线故障全网瘫痪、故障难隔离	早期以太网（10BASE-5/2）
环形	结点首尾相连成闭合环，数据沿单一方向逐站传递	时延确定、无冲突、控制简单	单点故障可能使全网中断（需双环或旁路）、扩充困难	令牌环网、FDDI
网状	结点之间有多条链路相连（全连接或部分连接）	可靠性高、容错能力强、易于扩展、可绕开故障	结构复杂、成本高、路由算法复杂	广域网骨干

补充：树形拓扑（星形的层次化扩展）在教材中也常单列，其特点是根结点故障会导致整棵树分裂。

（三）主要性能指标

这一节是本章的计算核心，七个指标必须做到「听到名字就能写出定义式和单位」。

*1.*速率与带宽

速率（**Rate**）又称数据率（**Data Rate**）、比特率（**Bit Rate**），指连接到网络上的结点在单位时间内往信道上传送的数据量，单位是 **bit/s**（**b/s**、**bps**），也可写作 kbit/s、Mbit/s、Gbit/s。

带宽（**Bandwidth**）在计算机网络中有两种含义，必须分清：

1. 本来含义（模拟信号语境）：某个信号具有的频带宽度，即该信号包含的最高频率与最低频率之差，单位是赫兹（**Hz**）。例如某信道允许 300 Hz ~ 3 kHz 的信号通过，则带宽为 2700 Hz。
2. 计算机网络中的含义（数字信道语境）：某信道所能传送的最高数据率，单位是 **bit/s**。例如「这条链路的带宽是 100 Mb/s」。

带宽两种含义的识别方法

看单位。单位是 **Hz** 就是频带宽度（物理层信道特性，用于奈氏准则、香农定理）；单位是 **bit/s** 就是最高

数据率（用于吞吐量、时延带宽积、发送时延的计算）。题目给出带宽的数值时，先确认单位再代公式。

速率与存储容量换算进制的差别

速率（数据率、带宽）中的 $k=10^3$、$M=10^6$、$G=10^9$，是十进制；而存储容量（如文件大小、内存容量）中的 $1\text{KB}=2^{10}\text{B}$，是二进制。统考题目常在题干里明确写出「$1\text{M}=10^6$」或「$1\text{M}=2^{20}$」，以题干给定为准；若题干未给，网络题的速率一律按十进制，数据量按题目给出的口径。这是网络计算题最常见的失分点之一。

*2.*吞吐量

吞吐量（**Throughput**）指单位时间内通过某个网络（或信道、接口）的实际数据量，单位也是 bit/s。

- 吞吐量受网络带宽或链路带宽的限制：吞吐量的最大值不超过所经路径上最小的一段链路带宽。
- 因此求「从 A 到 B 的最大吞吐量」时的标准做法是木桶效应：把端到端路径上每一段的带宽列出来，取最小值。

用 2024-33 的结构说明：若路径为「H1 ——(10 Mb/s)——路由器——(三条并行路径：1000 Mb/s、1 Mb/s、100 Mb/s)——路由器——(10 Mb/s)——H2」，则

$$\text{最大吞吐量}=\min(10,\ 1000+1+100,\ 10)=10\ \text{Mb/s}$$

中间网络的聚合能力再大，也受两端 10 Mb/s 接入链路的夹持。默认不要自行脑补「分流、负载均衡、多路径」，除非题目明确写出。

*3.*时延

时延（**Delay / Latency**）指数据从网络（或链路）的一端传送到另一端所耗费的时间，由四部分组成：

$$\text{总时延}=\text{发送时延}+\text{传播时延}+\text{处理时延}+\text{排队时延}$$

（**1**）发送时延（传输时延）

$$\text{发送时延}=\frac{\text{数据长度（bit）}}{\text{发送速率（bit/s）}}$$

指主机或路由器把数据帧 / 分组的全部比特推送到链路上所需的时间。

- 数据长度越大，发送时延越大；发送速率越高，发送时延越小。
- 发送时延与链路的物理长度无关。
-「发送速率」由网卡与链路决定，不同链路的发送速率可能不同，此时必须分别计算。

（**2**）传播时延

$$\text{传播时延}=\frac{\text{信道长度（m）}}{\text{电磁波在信道上的传播速率（m/s）}}$$

指一个比特从链路的一端传播到另一端所需的时间。传播时延只与信道长度和介质中的传播速率有关，与数据量、与链路带宽都无关。

常见传播速率（记数量级即可）：自由空间 3.0×10^8 m/s；铜缆约 2.3×10^8 m/s；光纤约 2.0×10^8 m/s。

（3）处理时延

数据在交换结点为存储转发而进行的必要处理所花的时间，例如分析分组首部、从分组中提取数据、进行差错检验、查找转发表确定转发端口。

（4）排队时延

分组进入路由器后，先在输入队列中排队等待处理；路由器确定转发端口后，还要在输出队列中排队等待转发。队列长度取决于网络负载，因此排队时延是四者中最不确定的一项。

四个时延的核心对比

时延	决定因素	与带宽的关系	与链路长度的关系
发送时延	数据长度、发送速率	反比	无关
传播时延	信道长度、传播速率	无关	正比
处理时延	结点处理能力	弱相关	无关
排队时延	网络负载	强相关	无关

一句话：发送时延看「数据多长、发得多快」，传播时延看「路有多长、波跑多快」。

提高带宽不能减少传播时延

这是极易混的一对：「在高速链路上，瓶颈往往是传播时延而不是发送时延」。把链路带宽从 10 Mb/s 提升到 10 Gb/s，能把发送时延压到极小，但传播时延一点也不会变。卫星链路的单向传播时延约 250 ms，无论带宽多大都消不掉。所以「提高带宽就能降低总时延」是错的。

多分组经过多段链路的总时延公式

若报文被分成 m 个等长分组，端到端经过 n 个路由器（即 $n+1$ 段链路），各段链路发送速率相等、每段链路传播时延相等且忽略处理与排队时延，则：

$$T = (m+n) \times t_{\text{发送}} + (n+1) \times d_{\text{传播}}$$

理由：第一个分组要经历 $(n+1)$ 次发送与 $(n+1)$ 次传播才能到终点，其后每个分组因流水线而只额外增加一次发送时延。

完整算例一（点对点，四时延中只算两项）

主机 A 通过一条长 1000 km 的光纤链路向主机 B 连续发送 10 个长度为 1500 B 的分组，链路带宽 100 Mb/s，光纤中信号传播速率为 2×10^8 m/s，中间不经过转发设备，忽略处理与排队时延。求从 A 开始发送第一个比特到 B 收到最后一个比特所需的时间。

1. 单个分组的发送时延：$t = \dfrac{1500 \times 8}{100 \times 10^6} = \dfrac{12000}{10^8} = 120\ \mu s$。
2. 10 个分组连续发送的发送时延：$10 \times 120\ \mu s = 1200\ \mu s = 1.2$ ms。
3. 传播时延：$d = \dfrac{10^6\ \text{m}}{2 \times 10^8\ \text{m/s}} = 5 \times 10^{-3}\ \text{s} = 5$ ms。
4. 总时延 $= 1.2 + 5 = 6.2$ ms。

第 4 步要注意：发送与传播是流水进行的，第 1 个分组的比特已经在链路上跑了，所以总时间不是 $10 \times (0.12 + 5)$，而是「把 10 个分组全部推上链路的时间」加上「最后一个比特走完全程的传播时延」。

完整算例二（含路由器的多分组时延）

主机 H1 与 H2 通过一台路由器互连，两段链路带宽均为 100 Mb/s，每段链路的时延带宽积均为 1000 bit。H1 向 H2 发送一个大小为 1 MB（1M $= 10^6$）的文件，分组长度为 1000 B，忽略分组拆装时间、处理时延与排队时延。求从 H1 开始发送到 H2 收到全部数据所需的最少时间。

1. 分组总个数：$m = \dfrac{10^6\ \text{B}}{1000\ \text{B}} = 1000$。
2. 单个分组的发送时延：$t = \dfrac{1000 \times 8}{100 \times 10^6} = 80\ \mu s$。
3. 由时延带宽积求单向传播时延：$d = \dfrac{1000\ \text{bit}}{100 \times 10^6\ \text{bit/s}} = 10\ \mu s$。
4. 路由器数 $n = 1$，代入公式：$T = (m+n)t + (n+1)d = (1000+1) \times 80 + 2 \times 10 = 80080 + 20 = 80100\ \mu s = 80.10$ ms。

这道题的三个易错点：一是分组个数必须用「文件大小 ÷ 每个分组长度」，不是「÷ 每个分组的数据载荷」；二是路由器数 $n = 1$ 时公式里是 $m + n = 1001$ 次发送，不是 1000 次；三是时延带宽积要先换算成传播时延再代入。

*4.*时延带宽积

时延带宽积（**Bandwidth-Delay Product**）定义为：

$$\text{时延带宽积} = \text{传播时延} \times \text{带宽}$$

它的物理意义是：发送端发送的第一个比特即将到达终点时，发送端已经发出了多少个比特。因此时延带宽积又称「以比特为单位的链路长度」，也叫链路的「管道容量」。

- 时延带宽积的单位是比特（**bit**），不是秒，也不是 bit/s。
- 一条链路的时延带宽积表示「这条链路上同时能容纳多少个比特在途中」。

算例：链路带宽 10 Mb/s，单向传播时延 20 ms，则

$$\text{时延带宽积} = 10 \times 10^6\ \text{bit/s} \times 20 \times 10^{-3}\ \text{s} = 2 \times 10^5\ \text{bit} = 200\ \text{kb}$$

也就是说，这条链路上任意时刻最多有 200 kb 的数据在飞行途中。若发送 1000 B（8000 bit）的分组，链路上可同时容纳 $2 \times 10^5 / 8000 = 25$ 个分组。

时延带宽积揭示的结论

如果采用停止等待协议（每发一个分组就等确认），链路最多只能容纳 1 个分组，链路的利用率极低——有效数据率仅为 $\dfrac{\text{分组长度}}{\text{RTT}}$，与该链路的时延带宽积相比小得可以忽略。这正是「管道容量」这个概念的用途：它告诉你需要多少「在途数据」才能把链路喂满，也是后面滑动窗口、TCP 发送窗口大小的理论依据。

*5.*往返时间 *RTT*

往返时间（**Round-Trip Time, RTT**）指从发送端发送数据分组开始，到发送端收到接收端发回的相应确认分组为止，总共经历的时间。

- RTT 包含：数据分组的发送时延 + 往返的传播时延 + 中间结点的处理与排队时延 + 确认分组的发送时延。很多题目为了简化会说明「忽略确认分组的发送时延与传播时延之外的其他时延」。
- RTT 不等于单向时延的两倍（严格说，它比两倍的单向传播时延还要多出确认分组的发送时延与处理时延），但在多数题目中可以近似看作两倍的单向传播时延。

- 在 RTT 主导的场合（如 TCP 慢启动、停止等待协议），有效数据率 $\approx \dfrac{\text{数据长度}}{\text{RTT}}$。

算例：RTT = 20 ms，分组长度 1000 B，链路带宽 10 Mb/s，采用停止等待协议。忽略发送时延，则

$$\text{有效数据率} = \frac{1000 \times 8\ \text{bit}}{20 \times 10^{-3}\ \text{s}} = 400\ \text{kb/s}$$

只有带宽 10 Mb/s 的 4%，与前面时延带宽积给出的「链路可容纳 25 个分组」完全吻合——每 25 个分组的位置上只发了 1 个。

*6.*利用率

- 信道利用率：某个信道有百分之多少的时间是有数据通过的。

$$U_{\text{信道}} = \frac{\text{有数据通过的时间}}{\text{有数据通过的时间} + \text{无数据通过的时间}}$$

- 网络利用率：全网络中所有链路的信道利用率的加权平均值。

利用率越高，看上去链路越「划算」，但：

$$D = \frac{D_0}{1 - U}$$

其中 D_0 表示网络空闲时的时延，U 为网络利用率，D 为当前网络时延。

算例：

网络利用率 U	时延倍数 $\frac{D}{D_0}$	含义
0	1	网络空闲，时延为 D_0
0.5	2	时延翻倍
0.8	5	时延是空闲时的 5 倍
0.95	20	时延是空闲时的 20 倍
$\to 1$	$\to \infty$	时延趋于无穷，网络接近拥塞崩溃

若 $D_0 = 10$ ms，$U = 0.6$，则 $D = \dfrac{10}{1 - 0.6} = 25$ ms。

利用率不是越大越好

由 $D = \dfrac{D_0}{1 - U}$ 可见，当 U 接近 1 时时延急剧上升。因此网络利用率并非越高越好，适度保留冗余才能保证时延可控。题目若说「利用率越大越好」或「提高利用率一定能提高网络性能」，都是错的。另外要分清 D_0 是「网络空闲时的时延」，不是「最小可能时延」，也不是「发送时延」。

本章真题考法（二）：性能指标计算题

- **2024-33**：多路径分组交换网络的最大吞吐量，用木桶效应取最小带宽（含两端接入链路）。
- 时延带宽积类：给出「两段链路带宽均为 100 Mb/s、时延带宽积均为 1000 bit、文件 1 MB、分组 1000 B」，求总时间（答案为 80.10 ms 量级）。
- 存储转发分组交换的时延题：所有链路传输速率 100 Mb/s、分组 1000 B（其中首部 20 B）、文件 980000 B，忽略分组拆装时间与传播时延。分组数 $= 980000/980 = 1000$，发送时延 $= 1000 \times 8/10^8 = 80\ \mu s$，总时延 $= (1000 + 1) \times 80\ \mu s = 80.08$ ms（答案对应 80.08 ms）。

- 报文交换与分组交换对比题：甲经一个路由器（存储转发）与乙互连，两段链路速率均为 10 Mb/s，报文 8 Mb（$1\text{M} = 10^6$），分组 10 kb。报文交换需 $800 + 800 = 1600$ ms；分组交换需 $(800 + 1) \times 1$ ms $= 801$ ms，即 1600 ms 与 801 ms。结论：分组交换在长报文传输上显著优于报文交换，且路由器越少、分组越短，优势越明显（但分组越短首部开销越大）。

二、计算机网络体系结构

（一）分层结构

*1.*为什么需要分层

计算机网络要解决的问题极为复杂：不同厂商的设备要互通、不同介质的链路要互联、上层应用不关心底层怎么传数据。分层的思路是把庞大而复杂的问题分解成若干个较小的、容易处理的局部问题，每一层只解决其中一部分。

分层带来的好处：

- 各层之间相互独立：上一层只需要知道下一层通过接口提供的服务，不需要知道下一层如何实现。
- 灵活性好：某一层的内部实现发生变化（例如把铜缆换成光纤），只要接口不变，其余各层都不受影响。
- 结构上可分割：各层都可以采用最合适的技术来实现。
- 易于实现和维护：整个系统被分解为若干相对独立的子系统。
- 能促进标准化：每一层的功能及其提供的服务都有了精确的说明。

分层的原则：

- 每层都实现一种相对独立的功能。
- 层间的接口自然清晰，信息量尽可能少。
- 各层功能的选择应有助于制定国际标准。

*2.*体系结构、实体与对等层

计算机网络体系结构（**Network Architecture**）是计算机网络各层及其协议的集合，也就是这个计算机网络及其部件所应完成的功能的精确定义。

体系结构不包括实现细节

体系结构是抽象的，只描述「做什么」，不描述「怎么做」。它回答的问题是：网络划分为哪些层次、每一层完成什么功能、各层使用什么协议、相邻层之间通过什么接口和服务协作。至于某个协议在某台设备上用什么数据结构、什么算法、什么代码实现，不属于体系结构的描述内容。

相关术语：

- 实体（**Entity**）：第 n 层中能够收发信息、执行协议的活动元素，可以是硬件（如网卡）也可以是软件进程（如浏览器进程）。网卡内部含有固件与控制器，既能收发比特流（物理层）又能封装数据帧（数据链路层），是一个典型的硬件实体。
- 对等实体（**Peer Entity**）：通信双方同一层次中的实体。例如 A 的网络层实体与 B 的网络层实体就是对等实体。只有对等实体之间才谈「协议」。
- 对等层通信：对等层之间的通信是「逻辑上」的，数据并不是从第 n 层直接飞到对方的第 n 层，而是逐层向下传递到物理层，经传输介质到达对方后，再逐层向上传递。这个「看起来像同层

直接对话」的效果，是靠封装与解封装实现的。

分层体系结构的三个基本构成

网络体系结构 = 层次划分 + 各层协议 + 层间接口（服务）的集合。

单向依赖关系贯穿始终：第 n 层只能使用第 $n-1$ 层提供的服务，并向第 $n+1$ 层提供服务；同一层内协议控制对等实体的通信。同一结点内相邻层之间是「服务」关系，不同结点同一层之间是「协议」关系。

*3.*协议三要素

协议（**Protocol**）是为进行网络中的数据交换而建立的规则、标准或约定，即控制对等实体之间通信的规则的集合。协议由三个要素组成：

要素	含义	通俗理解
语法（Syntax）	规定数据与控制信息的结构或格式	「话怎么说」——数据长什么样、字段怎么排
语义（Semantics）	规定需要发出何种控制信息、完成何种动作以及做出何种响应	「话是什么意思」——收到后要做什么
同步（Timing / 时序）	规定事件实现顺序的详细说明，即执行各种操作的条件与时序关系	「什么时候说」——先后次序、速度匹配

2020-33：协议三要素的图形识别

题目给出两条竖直时间轴与多条消息箭头，表示发送方与接收方之间消息发送与响应的先后关系。图中没有给出报文格式，也没有解释字段含义，因此只描述了时序（同步），答案是「仅 III」。

判图口诀：有字段排列 → 语法；有「收到后做什么」的说明 → 语义；只有箭头先后、没有格式与含义 → 时序。

在 TCP/IP 语境下，「同步」也常被写作「时序」。三者缺一不可——只规定格式而不规定什么人什么时候做出什么响应，通信就无法完成。

（二）协议、接口、服务等概念

*1.*协议、接口与服务

这三层术语的关系是本章最容易出辨析题的地方，必须从「方向」上彻底分清。

- 协议（**Protocol**）：控制对等实体之间通信的规则的集合。协议是「水平」的——它约束的是不同结点上同一层次的实体。
- 服务（**Service**）：下层（第 n 层）向上层（第 $n+1$ 层）提供的一组操作（功能调用）。服务是「垂直」的——它只在同一结点内的相邻层之间发生。
- 接口（**Interface**）/ 服务访问点（**SAP**）：同一系统中相邻两层实体交换信息的逻辑接口。上层要使用下层提供的服务，必须通过与下层交换一些命令，这些命令称为服务原语（**Service Primitive**）。

关于接口与 SAP 的关系，统考的口径是：

1. 接口是大框架，描述两层之间的交接处与交互入口；
2. **SAP** 是接口的具体接入点，用于区分不同的服务类型（考试时认为「接口 ≈ SAP」）；
3. 服务原语是在 **SAP** 上传输的控制命令。

常见的 SAP 举例：数据链路层帧的「类型」字段、IP 数据报的「协议」字段、TCP 报文段或 UDP 用户数据报的「端口号」字段、应用层的「用户界面」。

服务与协议的区别（必背）

1. 协议是水平的，是控制对等实体之间通信的规则；服务是垂直的，是下层向上层通过层间接口提供的。
2. 本层协议的实现要依靠下层提供的服务；本层通过协议向上一层提供服务。
3. 协议对上层是透明的：上层实体只看得见下层提供的服务，看不见（也不需要知道）该服务是用什么协议实现的。
4. 并非在一个层内完成的全部功能都称为服务，只有那些能被上一层所看见的功能才称为服务。
5. 协议的实现保证了能够向上一层提供服务；但「服务」不等于「协议」——同一服务可以由不同的协议实现。

*2.*面向连接服务与无连接服务

按通信前是否需要建立连接，下层向上层提供的服务分为两类：

对比项	面向连接服务	无连接服务
连接建立	必须先建立连接	不需要
三个阶段	连接建立、数据传输、连接释放	只有数据传输
资源占用	通信期间一直占用	随用随取，用完即走
数据顺序	一般保证按序到达	不保证顺序
典型协议	TCP、虚电路	UDP、IP 数据报
开销	建立与释放连接的开销	每个分组都要带完整目的地址，但无连接管理开销

「面向连接」不等于「可靠」

这是一个反复考的陷阱。面向连接只是一种连接管理方式，可靠性必须由确认、重传、序号等机制另行提供。

- TCP 是面向连接的，同时也是可靠的。
- 但 X.25 的虚电路是面向连接的，其可靠性由网络层保证；而帧中继的虚电路面向连接却是不可靠的。
- 反过来，无连接也不等于不可靠——例如 802.11 无线局域网的某些机制在无连接的基础上也提供了确认重传。

结论：「面向连接」与「可靠」是两个独立的属性，不能互相推导。同理，「无连接」与「不可靠」也不是同义词，只是在 TCP/IP 体系中恰好是对应关系。

*3.*服务数据单元 *SDU* 与协议数据单元 *PDU*

- 协议数据单元（**PDU, Protocol Data Unit**）：对等层次之间传送的数据单元。第 n 层的 PDU 由两部分构成：协议控制信息 **PCI**（首部）+ 服务数据单元 **SDU**。

$$n\text{-PDU} = n\text{-PCI} + n\text{-SDU}$$

- 服务数据单元（**SDU, Service Data Unit**）：同一系统内层与层之间交换的数据单元，也就是上一层交给本层、要求本层代为传送的数据。

两者的关系是「同一份数据在不同层次的不同称呼」。当上层把数据整体交给下层、下层不做拆分也不做合并时：

$$n\text{-SDU} = (n+1)\text{-PDU}$$

即：第 n 层的服务数据单元，就是第 $n+1$ 层的协议数据单元。

各层 PDU 的名称（必背）：

OSI 层次	**PDU** 名称（统考叫法）	说明
应用层	报文（Message）/ 数据	应用进程之间交换的数据
表示层	报文 / 数据	只做表示转换，不改变 PDU 名称
会话层	报文 / 数据	只做会话管理
传输层	报文段（Segment）；UDP 称用户数据报	TCP 的 PDU 叫报文段
网络层	分组 / 数据报（Packet / Datagram），常称 IP 数据报	加 IP 首部（至少 20 B）
数据链路层	帧（Frame）	加帧头（目的 MAC、源 MAC、类型）与帧尾（FCS）
物理层	比特（Bit）	比特流，不参与封装

PDU 与 SDU 的两个易错点

1. **PDU** 在水平方向（对等层之间）使用，**SDU** 在垂直方向（相邻层之间）使用。把 SDU 说成「对等层之间传送的数据单元」是错的。
2. 传输层的 **PDU** 叫「报文段」，不叫「报文」。「报文（Message）」是应用层的 PDU。网络层的 PDU 叫「分组 / 数据报」，教材上也写作「IP 数据报」或简称「IP 分组」，但不要写成「数据包」之外又另造名字。

本章真题考法（三）：协议、接口、服务

- **2020-33**：协议三要素的图形识别，答案为「仅 III（时序）」。
- **2010-33**：不属于体系结构描述内容的是「协议的内部实现细节」。
- 常考判断：「服务是水平的、协议是垂直的」是错的（反了）；「协议是水平的、服务是垂直的」是对的。「**IP** 数据报的协议字段是一个 **SAP**」是对的；「**SAP** 就是协议」是错的。

（三）ISO/OSI 参考模型与 TCP/IP 参考模型

1.ISO/OSI 参考模型

OSI（Open Systems Interconnection Reference Model，开放系统互连参考模型）由国际标准化组织 **ISO** 提出，是一个法律上的国际标准。它的设计思路是先有模型、后有协议：先定义好七层的功能框架，再逐层制定协议。结果是模型本身严谨完备，但过于复杂，实现困难，市场化失败。

七层名称（自下而上）：物理层、数据链路层、网络层、传输层、会话层、表示层、应用层。

记忆口诀：物、链、网、输、会、示、用。

各层的主要功能：

- 物理层（**Physical Layer**）：在传输介质上透明地传送比特流，为数据链路层提供一个传输比特的物理连接。它规定接口的机械特性、电气特性、功能特性、过程特性（如引脚数目、电压范围、信号含义、时序）。物理层不参与数据的封装，它看到的只是比特流。PDU 为比特。该层设备：中继器、集线器。

- 数据链路层（**Data Link Layer**）：在相邻结点之间无差错地传送以「帧」为单位的数据。主要功能：成帧（帧定界与同步）、差错控制、流量控制、介质访问控制。设备：网卡、网桥、交换机。相邻结点间的流量控制属于数据链路层。
- 网络层（**Network Layer**）：负责把分组从源结点转发到目的结点。主要功能：路由选择、分组转发、拥塞控制、网际互连（实现异构网络互联）。设备：路由器、三层交换机。OSI 的网络层既支持无连接方式，也支持面向连接方式（虚电路）。
- 传输层（**Transport Layer**）：提供端到端（进程到进程）的数据传输服务，是自下而上第一个提供端到端服务的层次。主要功能：复用与分用、差错控制、端到端流量控制、可靠传输、拥塞控制、连接建立与释放。OSI 的传输层只支持面向连接的方式。
- 会话层（**Session Layer**）：管理进程间的会话（为表示层实体或用户进程建立连接，并在连接上有序地传输数据）。主要功能：会话管理、同步服务（在数据流中插入检查点，实现「文件断点续传」）。
- 表示层（**Presentation Layer**）：解决不同主机上信息表示方式不一致的问题。主要功能：数据格式转换、字符编码转换、加密与解密、压缩与解压。
- 应用层（**Application Layer**）：体系结构中的最高层，直接为用户的应用进程提供服务，包含大量面向具体应用的协议（HTTP、FTP、SMTP、DNS 等）。

网络层与传输层的「端到端」区别

- 网络层常被称为「主机到主机」的通信：它负责把分组从源主机送到目的主机，中间要经过路由器，每台路由器都要参与网络层处理。
- 传输层是真正的「进程到进程」的端到端通信：只有两端的端系统才需要运行传输层，中间路由器不处理传输层数据。2009-33 问「自下而上第一个提供端到端服务的层次」，答案就是传输层。
- 另外，相邻结点间的流量控制是数据链路层的功能（**2022-33**），端到端的流量控制是传输层的功能，两者不能混。

2.TCP/IP 参考模型

TCP/IP 参考模型的设计思路与 OSI 相反：先有协议、后有模型，是事实上的国际标准。它共四层：

- 网络接口层（链路层）：对应 OSI 的物理层 + 数据链路层，负责与硬件打交道，把 IP 数据报封装成帧并通过物理网络发送。
- 网际层（**IP** 层 / 网络层）：核心是 **IP** 协议。它只提供无连接、不可靠的数据报服务（尽力而为的交付），负责逻辑寻址、路由选择与 IP 数据报转发；不提供流量控制与可靠传输。
- 传输层：提供端到端的进程间通信，同时支持面向连接的 **TCP** 和无连接的 **UDP**。
- 应用层：对应 OSI 的会话层 + 表示层 + 应用层，把三层功能合并。

五层教学模型：408 教材普遍采用一种折中的五层结构，即物理层、数据链路层、网络层、传输层、应用层——下面四层与 OSI 的下四层一一对应，最上面一层把 OSI 的会话层、表示层、应用层合并。五层模型既保留了 OSI 中「有物理介质、有链路、有网络、有端到端」的清晰划分，又避免了 OSI 上三层划分过细的问题。

*3.*两个模型的对比

对比维度	**OSI** 参考模型	**TCP/IP** 参考模型
层数	7 层	4 层（教学用 5 层）
设计顺序	先有模型，后有协议	先有协议，后有模型

对比维度	**OSI** 参考模型	**TCP/IP** 参考模型
标准性质	法律上的国际标准	事实上的国际标准
网络层提供的服务	面向连接（虚电路）+ 无连接	只有无连接（IP 数据报）
传输层提供的服务	只有面向连接	面向连接（TCP）+ 无连接（UDP）
可靠性由谁保证	由网络层保证（传输层一定可靠）	由传输层保证（网络层不可靠）
上三层划分	会话层、表示层、应用层三层分立	合并为一层应用层
网络互连能力	未充分定义	核心能力，靠 IP 实现异构网络互联
适用场景	理论完备、实现复杂、市场失败	简单实用、实现广泛、市场成功

两个模型的三条「反着记」的结论

1. **OSI** 的网络层有连接也有无连接，传输层只有面向连接；**TCP/IP** 的网络层只有无连接，传输层两种都有。（方向正好相反）
2. **OSI** 认为可靠性由网络层负责；**TCP/IP** 认为网络层不可靠，可靠性交给传输层。
3. **OSI** 是「法律上的」国际标准；**TCP/IP** 是「事实上的」国际标准。

*4.*数据在各层的封装与解封装

封装（**Encapsulation**）：发送方从应用层往下逐层加首部（数据链路层还要加帧尾）的过程。

$$\text{应用数据} \rightarrow \text{报文} \xrightarrow{+\text{TCP 首部}} \text{报文段} \xrightarrow{+\text{IP 首部}} \text{数据报/分组} \xrightarrow{+\text{帧头帧尾}} \text{帧} \xrightarrow{\text{转为信号}} \text{比特流}$$

逐层看：

1. 应用层：应用进程产生数据（Data），称为报文（Message）。
2. 传输层：加上 TCP 首部（源端口、目的端口、序号、确认号等），构成报文段（**Segment**）。
3. 网络层：加上 IP 首部（源 IP、目的 IP、协议、TTL 等，至少 20 B），构成 **IP** 数据报 / 分组。
4. 数据链路层：加上帧头（目的 MAC、源 MAC、类型）与帧尾（FCS 帧校验序列），构成帧（**Frame**）。
5. 物理层：把帧转换为比特流，在传输介质上以信号形式发送。物理层是「管道的存在」，数据链路层是「管道里的交通规则」。

解封装（**Decapsulation**）：接收方从物理层往上逐层剥去本层的首部（与帧尾），把剩下的数据交给上一层，直到应用层还原出原始数据。

2017-33：封装效率计算

OSI 参考模型的应用层欲发送 400 B 数据（无拆分），除物理层和应用层之外，其他各层在封装 PDU 时均引入 20 B 额外开销，求应用层数据传输效率。

会引入 20 B 开销的层有 5 个：表示层、会话层、传输层、网络层、数据链路层。总开销 $= 5 \times 20 = 100$ B，链路上总数据量 $= 400 + 100 = 500$ B。

$$\text{效率} = \frac{400}{500} \times 100\% = 80\%$$

两个陷阱：一是物理层与应用层不计开销，所以是 5 层不是 7 层；二是题目明确「无拆分」，因此同一层的首部开销不能重复计算（若报文被分成多个分组，每个分组都要带一次网络层与数据链路层首部）。

*5.*各层典型协议与设备归属

这张表要能默写。层 →PDU→ 协议 → 设备的对应关系，是本章选择题的主要出题方式。

层次	**PDU** 名称	典型协议	典型设备	最高工作在此层的设备
应用层	报文（Message）	HTTP、HTTPS、FTP、SMTP、POP3、DNS、DHCP、Telnet、SNMP	应用网关	网关
表示层	报文	数据格式与加密压缩规范（OSI 专有）	—	—
会话层	报文	会话管理协议（OSI 专有）	—	—
传输层	报文段（Segment）/ 用户数据报	TCP、UDP	—	—
网络层	分组 / 数据报（Packet）	IP、ICMP、IGMP、ARP、OSPF、RIP、BGP	路由器、三层交换机	路由器（第 3 层）
数据链路层	帧（Frame）	以太网 802.3、PPP、HDLC、802.11、VLAN	网卡、网桥、二层交换机	交换机（第 2 层）
物理层	比特（Bit）	RS-232、RJ-45 相关规范	中继器、集线器	集线器（第 1 层）

ARP 的层次归属

ARP（地址解析协议）的归属在不同教材中有分歧：有的把它放在网络层，有的放在网络接口层（链路层）。统考一般按网络层理解（因为 ARP 报文直接封装在以太网帧中，但它服务于 IP 地址到 MAC 地址的解析，属于网络层的配套协议）。遇到单选题时，若选项把 ARP 与 IP、ICMP、IGMP 并列放在网络层，按网络层选；若题干强调「封装在帧里」，则以题干给出的口径为准。

2016-33：设备的最高功能层

R1（路由器）、Switch（交换机）、Hub（集线器）实现的最高功能层分别是 **3**、**2**、**1**。

- 路由器工作在网络层（第 3 层），能连接不同网络；
- 交换机工作在数据链路层（第 2 层），识别 MAC 地址与帧；
- 集线器只是多端口中继器，工作在物理层（第 1 层），不识别帧、不识别地址。

变体考法：问「Hub、Switch、Router 依次工作在 OSI 的第几层」，答案相同；问「中继器/集线器属于哪一层」，答物理层。

本章小结表

考纲条目	核心内容	必须记住的结论
计算机网络的定义	地理位置不同、具有独立功能的多台计算机通过通信线路与设备连接，在协议管理下实现资源共享与信息传递	三个限定词：自治、互连、计算机集合
internet 与 Internet	前者泛指互连网（可用任意协议），后者特指全球互联网（必用 TCP/IP）	大小写不同，含义不同
组成（物理）	硬件、软件、协议	协议是核心

考纲条目	核心内容	必须记住的结论
组成（工作方式）	边缘部分（主机，C/S 与 P2P）、核心部分（网络与路由器）	核心部分只提供连通性与交换
组成（功能）	通信子网（下三层，管传输）、资源子网（上四层，管处理）	切分点在传输层：传输层属资源子网
功能	数据通信、资源共享、分布式处理、提高可靠性	考纲四条，负载均衡可归入分布式处理
分类（分布范围）	PAN / LAN / MAN / WAN	LAN 与 WAN 区别在技术，不在范围
分类（使用者）	公用网 / 专用网	专用网可租用公用网线路（VPN）
分类（交换技术）	电路交换 / 报文交换 / 分组交换	分组交换可流水线；分组交换分数据报与虚电路
分类（拓扑）	星形 / 总线 / 环形 / 网状	星形中心是瓶颈；总线冲突重；环形扩充难；网状可靠
速率	数据率、比特率，单位 bit/s	$k = 10^3$, $M = 10^6$（十进制）
带宽	模拟信号：频带宽度（Hz）；计网：最高数据率（bit/s）	看单位判断是哪种含义
吞吐量	单位时间通过某网络/信道/接口的实际数据量	最大值 $\leq$ 路径上最小带宽（木桶效应）
发送时延	数据长度（bit）÷ 发送速率（bit/s）	与链路长度无关
传播时延	信道长度（m）÷ 电磁波传播速率（m/s）	与带宽、数据量无关，提高带宽消不掉
处理时延	分析首部、差错检验、查转发表	由结点处理能力决定
排队时延	输入队列排队 + 输出队列排队	由网络负载决定，最不确定
时延带宽积	传播时延 × 带宽，单位 **bit**	又称「以比特为单位的链路长度」「管道容量」
RTT	从发出数据分组到收到确认分组的总时间	有效数据率 ≈ 数据长度 / RTT
利用率	信道利用率、网络利用率	$D = \frac{D_0}{1-U}$，利用率不是越大越好
分层结构	各层独立、灵活、易实现、促标准化	体系结构 = 层次 + 协议 + 接口，不含实现细节
协议三要素	语法、语义、同步（时序）	语法管格式、语义管动作、同步管次序
协议 / 接口 / 服务	协议对等（水平），服务下层给上层（垂直），SAP 是接口接入点	协议对上层透明；同一服务可由不同协议实现
面向连接 / 无连接	三阶段 vs 无连接；TCP/虚电路 vs UDP/IP	面向连接 $\neq$ 可靠
SDU 与 PDU	n-PDU $= n$-PCI $+ n$-SDU； n-SDU $= (n+1)$-PDU	PDU 水平用，SDU 垂直用
OSI 七层	物、链、网、输、会、示、用	传输层是自下而上第一个端到端的层次
TCP/IP 四层	网络接口层、网际层、传输层、应用层	网际层只有无连接、不可靠的数据报服务
五层教学模型	物理、数据链路、网络、传输、应用	下四层对应 OSI，上三层合并成应用层

考纲条目	核心内容	必须记住的结论
封装与解封装	报文 → 报文段 → 分组 → 帧 → 比特	每层加自己的首部（链路层还加帧尾）
层次归属	集线器 1 层、交换机 2 层、路由器 3 层	2016-33 答案 3、2、1

易错清单

1. 「数据通信」是计算机网络最基本的功能，「资源共享」是最有吸引力的功能。不要把「提高可靠性」说成最基本的功能。
2. 分时系统中的终端不是自治的，因此「主机 + 若干终端」不构成计算机网络。
3. 通信子网对应 **OSI** 下三层，资源子网对应上四层，切分点在传输层。把传输层划入通信子网是错的。
4. **LAN** 与 **WAN** 的本质区别是技术（广播式 **vs** 点对点、共享介质 **vs** 交换），不是地理范围。只讲范围的选项是干扰项。
5. 报文交换不能流水线，必须收完整个报文才能转发；分组交换可以流水线，这是分组交换时延更小的根本原因。
6. 电路交换不具备差错控制能力（数据透明传输，中间结点不处理），不要把它说成「可靠传输」。
7. 虚电路不是电路交换。电路交换预分配带宽、独占物理线路；虚电路是多路复用、逻辑连接、共享带宽，仍需存储转发。虚电路服务是 **OSI** 特有的，**TCP/IP** 网络层没有。
8. 速率中的 $k = 1000$、$M = 10^6$ 是十进制，存储容量中的 $1\textbf{KB} = 2^{10}\textbf{B}$ 是二进制。网络计算题以题干给定的换算为准。
9. 带宽有 **Hz** 和 **bit/s** 两种含义，单位是 Hz 时是频带宽度，单位是 bit/s 时是最高数据率，代公式前先确认。
10. 发送时延与链路长度无关，传播时延与带宽无关。「把带宽提高一倍，传播时延减半」是错的。
11. 总时延不是「每个分组的时延 × 分组数」。分组是流水发送的，m 个分组、n 个路由器时总时延为 $(m+n)t_{\text{发送}} + (n+1)d_{\text{传播}}$。
12. 求最大吞吐量必须用木桶效应，且不能忽略源端和目的端的接入链路——中间并行路径的聚合带宽再大也不能突破两端接入瓶颈（2024-33 的陷阱）。
13. 不要自行脑补「分流、负载均衡、多路径」，除非题目明确写出相关条件。绕远路只会增加转发时延，不会缩短总时间。
14. 时延带宽积的单位是比特（**bit**），不是秒、不是 bit/s。它表示的是「链路上同时能容纳的比特数」。
15. $\dfrac{D_0}{1-U}$ 中的 D_0 是网络空闲时的时延，不是发送时延，也不是「最小网络时延」的任意口径；利用率越大时延增长越快，利用率并非越大越好。
16. 「服务是水平的、协议是垂直的」是错的，正确说法是协议水平、服务垂直。
17. 协议对上层是透明的：上层只看得见服务，看不见协议。把这句话反过来（「服务对协议透明」）就是错的。
18. 协议的三要素是语法、语义、同步（时序），不包括「分层」「接口」「服务」；而「协议的内部实现细节」不属于网络体系结构描述的内容（2010-33）。
19. **PDU** 是对等层之间传送的数据单元，**SDU** 是同一系统相邻层之间交换的数据单元，两者的适用方向不能互换。

20. 传输层的 **PDU** 叫「报文段」，应用层的 **PDU** 叫「报文」，两者不能混用；网络层 PDU 叫「分组 / 数据报」。
21. 面向连接不等于可靠，无连接也不等于不可靠。连接管理方式与可靠性是两个独立属性，不要用前者推后者。
22. 传输层是自下而上第一个提供端到端服务的层次（2009-33）；网络层是「主机到主机」，传输层才是「进程到进程」。
23. 相邻结点间的流量控制属于数据链路层（2022-33），端到端的流量控制属于传输层；物理层只传比特。
24. **TCP/IP** 的网际层只提供无连接、不可靠的数据报服务（2011-33）；需要可靠传输时由上层 TCP 负责，不要写成「网络层提供可靠传输」。
25. **OSI** 是法律上的国际标准，**TCP/IP** 是事实上的国际标准；且 OSI「先有模型后有协议」，TCP/IP 相反，两者不要颠倒。
26. 封装效率计算中，物理层与应用层不计开销（2017-33 的 OSI 七层里只有 5 层加 20 B）；题干说「无拆分」时，同一层首部不能重复计算。
27. 设备的最高工作层次：集线器 **1**、交换机 **2**、路由器 **3**（2016-33）。集线器是多端口中继器，不识别帧，更不识别地址。
28. **ARP** 的层次归属有教材差异，统考一般按网络层理解；遇到与 IP、ICMP、IGMP 并列的选项时按网络层选，并以题干口径为准。

第二十章物理层

本章考纲要求

第四部分计算机网络二、物理层（一）通信基础

1. 信道、信号、带宽、码元、波特、速率、信源与信宿等基本概念
2. 奈奎斯特定理与香农定理
3. 编码与调制
4. 电路交换、报文交换与分组交换
5. 数据报与虚电路

（二）传输介质

1. 双绞线、同轴电缆、光纤与无线传输介质
2. 物理层接口的特性

（三）物理层设备

1. 中继器
2. 集线器

物理层处于五层模型的最底层。它要解决的问题只有一句话：怎样在连接各种计算机的传输介质上，透明地传输比特流。

理解这一章必须抓住两个「不关心」：

- 物理层不关心传输介质具体是什么（双绞线、同轴电缆、光纤、无线电波都可以）。考纲把「传输介质」列在物理层之下，是因为介质是物理层的服务对象，但物理层规范本身描述的是接口，不是介质。
- 物理层不关心比特流代表什么含义。物理层对数据链路层「透明」——数据链路层看不见物理层究竟用什么方法把这串比特搬过去。

因此本章的命题重心非常集中，可以分成四块：

1. 定量计算（本章最大的得分区）：奈氏准则与香农定理、波特率与比特率的换算、PCM 数据率、分组交换时延。这四类每年必考其中一到两类，且都是「套公式就能算对、套错就必错」的题。
2. 编码波形的判读：给出波形判断编码类型，或者给出编码类型画出/读出比特串。曼彻斯特与差分曼彻斯特是绝对主力。
3. 文字辨析：三种交换方式的对比、数据报与虚电路的区别、物理层接口四种特性、中继器与集线器的性质。
4. 概念边界：带宽的两种含义、速率与传播速度的区别、模拟信号与数字信号的区分。

下面按考纲条目顺序逐个展开。凡是带公式的考点，本文都会给出推导、符号含义、单位、以及一个带数字的完整算例，考生可以自己算一遍核对。

一、信道、信号、带宽、码元、波特、速率、信源与信宿

（一）通信系统的基本模型

一个最简单的通信系统由五部分组成，源点和终点是设备，发送器和接收器是变换环节，信道是传输媒介：

环节	英文	作用	典型设备
源点（信源）	Source	产生要传输的数据，输出数字比特流	主机、麦克风
发送器	Transmitter	把数据编码/调制成适合信道传输的信号	调制器
信道	Channel	信号传输的媒介（一个方向一个信道）	双绞线、光纤、自由空间
接收器	Receiver	把收到的信号还原成数字比特流	解调器
终点（信宿）	Destination	接收并使用还原后的比特流	主机、扬声器

传输方向是单向的：源点 → 发送器 → 信道 → 接收器 → 终点。

- 源点（**Source**）：也叫信源，产生要传输的数据，输出的是数字比特流。例如一台主机、一个麦克风。
- 发送器（**Transmitter**）：把源点产生的比特流编码或调制成适合在信道上传输的信号。典型发送器就是调制器。
- 信道（**Channel**）：信号的传输媒介，可以看成一条线路的逻辑部件。一个方向上的传输构成一个信道，因此同一条物理传输介质上可以建立多条信道。
- 接收器（**Receiver**）：把从信道收到的信号还原成目的设备能处理的数字比特流。典型接收器就是解调器。
- 终点（**Destination**）：也叫信宿，从接收器取得比特流并进行处理。

信源与信宿的定义要点

信源 = 源点 = 产生并发送数据的一方；信宿 = 终点 = 接收并使用数据的一方。二者都是端设备（主机），不是中间设备。题目里问「发送器 / 调制器属于哪一部分」时，答案是发送器，不是信源——这是常见偷换。

（二）通信的三种方式

按信号传播方向与时间的关系，信道可分为三类：

通信方式	别称	信号方向	需要的信道数	典型例子
单工通信	单向通信	只能 A → B，不能反向	1 个	无线电广播、电视广播
半双工通信	双向交替通信	双方都能收发，但同一时刻只能一方发送	2 个	对讲机
全双工通信	双向同时通信	双方可以同时发送和接收	2 个	电话、以太网链路

「需要几个信道」与「需要几条介质」不是一回事

- 半双工：需要 **2** 个信道，但可以共用同一条物理介质（分时使用）。
- 全双工：需要 **2** 个信道，一般需要两条物理介质（或者用不同频段/波长在同一条光纤上实现）。

很多资料把「半双工只有一条介质」简化成「只需要一个信道」，这是错的：只要允许双向通信，方向就是两个，就是两个信道。统考按「半双工 2 信道共用 1 介质、全双工 2 信道 2 介质」理解。

（三）信号：模拟信号与数字信号

信号（**Signal**）是数据的电磁波或光脉冲表现形式。

- 模拟信号（**Analog Signal**）：信号的参量（幅度、频率、相位）连续取值，取值有无穷多个。例如电话线里的话音电流、传统的调幅广播。
- 数字信号（**Digital Signal**）：信号的参量只取有限个离散值，通常用两种电平表示二进制的 0 和 1。例如计算机网卡输出的方波。

对应的两类数据：

- 模拟数据：连续变化的物理量，如话音、温度、图像。
- 数字数据：离散取值的量，如计算机里的比特流。

数据与信号可以交叉组合，得到考纲要求的四种转换，这是本章后续所有编码/调制内容的总框架：

数据 → 信号	转换设备	采用的技术	典型场景
数字数据 → 数字信号	数字发送器（数字放大器）	编码：NRZ、NRZI、RZ、曼彻斯特、差分曼彻斯特、4B/5B	主机与网卡、局域网内部
数字数据 → 模拟信号	调制器（Modem）	调制：ASK、FSK、PSK、QAM	主机到调制解调器
模拟数据 → 数字信号	编码器（PCM 编解码器）	采样、量化、编码（PCM）	音频数字化、话音编码
模拟数据 → 模拟信号	放大器、调制器	频分复用（FDM）、放大调制	电话机到本地端局

编码与调制的一句话区分

编码（**Encoding**）= 把数据变换成数字信号；调制（**Modulation**）= 把数据变换成模拟信号。判题时先看题干给出的目标信号是数字还是模拟，再决定用「编码」还是「调制」的词。

（四）基带信号与带通信号，基带传输与频带传输

- 基带信号（**Baseband Signal**）：来自信源的、没有经过调制的原始电信号，其频谱从零频（直流）附近开始，占据从 0 到某个截止频率的一段低通频带。计算机内部、网卡输出的方波就是基带数字信号。
- 带通信号（**Bandpass Signal**）：把基带信号调制到某一载波频率附近后形成的信号，其频谱集中在一个以载波频率为中心的有限频带内，不包含直流分量。
- 基带传输：把数字数据编码成数字信号后直接在信道上传输，不经过载波调制。例如 10Base5 同轴电缆、以太网双绞线上的曼彻斯特码。
- 频带传输（又称宽带传输）：把数字数据调制到载波上变成模拟信号再传输。例如 ADSL、有线电视网络。

名称里的字面线索

以太网标准名「10BaseT / 100BaseT」里，`Base` 就是 **Baseband**（基带），`T` 是 **Twisted pair**（双绞线），`100` 是速率 100 Mb/s。而 `100BaseFX` 的 `F` 是光纤。看到 `Base` 就能断定用的是基带传输，这是判断「某种以太网用什么介质、什么传输方式」的快速入口。

（五）带宽的两种含义

带宽（**Bandwidth**）在不同语境下含义不同，这是概念题的高频陷阱：

语境	含义	单位	说明
信号 / 信道（模拟口径）	该信号或信道能通过的频率范围，即最高频率与最低频率之差	Hz	例如话音信道 300 ～ 3400 Hz，带宽 3100 Hz；奈氏准则、香农定理里的 W 就是这个
网络（数字口径）	单位时间内从网络某一点到另一点所能通过的最高数据率	b/s（bit/s）	例如「该链路带宽为 100 Mb/s」

Hz 与 b/s 不能混用

在奈氏准则与香农定理的公式里，W 的单位必须是 **Hz**，答案 C 的单位是 b/s。见到题干写「带宽 3 kHz」，代进 W 的是 3×10^3 Hz；见到「带宽 100 Mb/s」，那就不是 W，不能往定理里代，它描述的是链路的发送能力。

（六）码元、波特与速率

码元（**Symbol**，又称信号码元）：一个固定时长的信号波形，用来表示一个 N 进制数。这个固定时长叫码元宽度，码元是数字通信中承载信息的基本单位。

一个码元可以取多少种不同的状态，取决于编码/调制方式用了多少种可区分的波形。设码元的离散状态数为 N：

- 若 $N=2$，一个码元就是「一个二进制位」，码元与比特一一对应；
- 若 $N=4$，一个码元可以表示 4 种情况，恰好对应 2 个二进制位；
- 一般地，N 必须是 2 的整数次幂（或取最接近的较大值），一个码元携带的比特数为 $\log_2 N$。

码元与比特的关系（必背）

$$k=\log_2 N \quad \text{(bit/码元)}$$

其中 N 是一个码元可取的离散状态数（电平种类数、相位种类数、频率种类数，或它们的组合数）。

状态数 N 必须取 **2** 的幂。如果题目给出「3 种幅值」这种非 2 的幂，实际一个码元只能可靠区分 $2^{\lfloor\log_2 3\rfloor}=2$ 种状态，即携带 1 bit——统考一般不会这样出，但知道了不吃亏。

波特（**Baud**）与波特率：

- 码元传输速率（又称调制速率、波特率、符号率）：单位时间内数字通信系统传输的码元个数（脉冲个数或信号变化次数），单位是波特（**Baud**），记作 Baud 或 B。
- 信息传输速率（又称比特率、数据传输速率、速率）：单位时间内数字通信系统传输的比特数，单位是比特/秒（**b/s**）。

两者由码元携带的比特数 k 联系起来：

$$R_b=R_B\times k=R_B\times\log_2 N$$

其中 R_b 是比特率（b/s），R_B 是波特率（Baud），$k = \log_2 N$ 是每个码元携带的比特数。

反之：

$$R_B = \frac{R_b}{\log_2 N}$$

三条换算结论，考场直接用

1. $R_b = R_B \log_2 N$；$N = 2$ 时才相等（$R_b = R_B$）。
2. 波特率是每秒码元数，比特率是每秒比特数，两者只在每码元恰好 **1 bit** 时数值相等。
3. 曼彻斯特、差分曼彻斯特编码中，每个比特需要用 **2** 个码元（两个半位）表示，因此 $R_B = 2R_b$，即波特率是比特率的 **2** 倍。

算例 **1**（比特率换算）：某通信链路的数据传输速率为 2400 b/s，采用 4 相位调制，求波特率。

4 相位表示码元有 $N = 4$ 种状态，每个码元携带 $\log_2 4 = 2$ bit，因此

$$R_B = \frac{2400}{2} = 1200 \text{ Baud}$$

算例 **2**（反向验证）：某信道的波特率为 2400 Baud，每个码元携带 3 bit，求信息传输速率。

$$R_b = 2400 \times 3 = 7200 \text{ b/s}$$

算例 **3**（状态数）：一个码元有 16 种离散状态，求一个码元可表示的比特数。

$$\log_2 16 = 4 \text{ bit}$$

（七）速率与传播速度的区别

这是 2014-35 直接考过的辨析：「下列因素中，不会影响信道数据传输速率的是——信噪比、频率带宽、调制速度、信号传播速度」，答案是信号传播速度。

- 传输速率（数据传输速率）：每秒能发送多少比特，由带宽、信噪比、调制所能区分的状态数、发送速率共同决定。它只关乎「发送端多快能把比特推到线路上」。
- 传播速度（信号传播速度）：电磁波或光在介质中每秒前进多少米，决定传播时延：

$$T_p = \frac{\text{信道长度}}{\text{信号传播速度}}$$

它只影响「比特从一端跑到另一端要多久」，不改变信道每秒能发出多少比特。

传播时延 ≠ 发送时延

- 发送时延 $T_t = \dfrac{\text{分组长度 (b)}}{\text{发送速率 (b/s)}}$，取决于分组大小与链路速率。
- 传播时延 $T_p = \dfrac{\text{信道长度 (m)}}{\text{传播速率 (m/s)}}$，取决于距离与介质，与带宽无关。

  光纤中光速约为 2×10^8 m/s，铜缆中约为 2.3×10^8 m/s。「提高带宽能降低传播时延」是错的，带宽再大也不能让信号跑得更快。

本章真题分布（据 **2009—2024** 年统考选择题）

- 奈氏/香农与调制状态数：2009-34、2014-35、2016-34、2017-34、2022-34、2023-34。
- 波特率与比特率换算：2011-34。
- 编码波形判读：2013-34（曼彻斯特）、2015-34（NRZ 与曼彻斯特）、2021-34（差分曼彻斯特）。
- 分组交换时延与吞吐量：2010-34、2013-35、2023-33、2024-33。
- 数字调制方式：2024-34（FSK）。
- 数据报与虚电路：2020-34。
- 传输介质：2019-34。物理层接口特性：2012-34、2018-34。

二、奈奎斯特定理与香农定理

这两个定理回答同一个问题：一条信道的极限数据传输速率是多少？区别在于有没有考虑噪声。这是本章最核心、分值最稳定的考点。

（一）为什么码元发得太快就会出错

信道中存在各种干扰（噪声）。如果发送端单位时间内发送的码元个数太多，即码元宽度太窄，那么每个码元在接收端造成的波形会「拖尾」并侵入相邻码元的时段，接收端就分不清相邻码元的界限，这种现象叫码间串扰（**Inter-Symbol Interference**，**ISI**）。

于是有两条互不相同的上限：

- 码元传输速率有上限——发得太快就会码间串扰。这条上限由奈氏准则给出，与噪声无关，只与带宽有关。
- 信息传输速率有上限——即使码元速率没超限，噪声也会让接收端分辨不清码元处在哪个状态。这条上限由香农定理给出。

> **奈氏准则限的是「码元速率」，不是「比特率」**
>
> 奈奎斯特定理没有直接限制数据传输速率的上限；它限制的是码元传输速率的上限为 $2W$ **Baud**。要想提高比特率，只能让每个码元携带更多比特（增大 N），而 N 又受噪声限制——这时就要用香农定理。这句话是 2017-34 的解题核心。

（二）奈奎斯特定理（奈氏准则）

内容：在理想低通信道（无噪声、带宽有限）中，为了避免码间串扰，极限码元传输速率为

$$R_{B,\max} = 2W \quad \text{(Baud)}$$

最大数据传输速率为

$$C_{\max} = 2W \log_2 N \quad \text{(b/s)}$$

符号含义与单位：

符号	含义	单位	说明
W	信道的带宽（频率范围宽度）	Hz	必须是频率口径的带宽
N	一个码元的离散状态数（电平/相位/频率/振幅组合的种类数）	无单位	由调制方式决定，$N = 2^k$
$\log_2 N$	每个码元携带的比特数	bit/码元	$N = 2$ 时为 1

符号	含义	单位	说明
$R_{B,\max}$	极限码元传输速率	Baud	等于 $2W$
$C_{\max}$	极限（最大）数据传输速率	b/s	奈氏条件下能达到的上限

使用前提（三个条件缺一不可）：① 理想低通信道；② 无噪声；③ W 用 **Hz** 计。

奈氏准则的三个常见坑

1. W 是 **Hz**，不是 **b/s**。把「链路速率 100 Mb/s」当成 W 代进去，必然全错。
2. N 是「状态数的乘积」而不是「相加」。「采用 4 个相位，每个相位有 4 种振幅」的 QAM，$N = 4 \times 4 = 16$，不是 $4 + 4 = 8$。2009-34 的易错点正是这里。
3. N 不是码元数，是状态数。不要把「每秒钟发多少个码元」与「一个码元有几种状态」搞混。

算例 1（**2009-34** 原题）：在无噪声情况下，某链路带宽 3 kHz，采用 4 个相位、每个相位 4 种振幅的 QAM 调制，求最大数据传输速率。

状态数 $N = 4 \times 4 = 16$，每个码元携带 $\log_2 16 = 4$ bit：

$$C_{\max} = 2 \times 3 \times 10^3 \times 4 = 24 \times 10^3 \text{ b/s} = 24 \text{ kb/s}$$

算例 2（**2022-34** 原题）：带宽 200 kHz 的无噪声信道，采用 4 个幅值的 ASK 调制，求最大数据传输速率。

$N = 4$，$\log_2 4 = 2$ bit：

$$C_{\max} = 2 \times 200 \times 10^3 \times 2 = 800 \times 10^3 \text{ b/s} = 800 \text{ kb/s}$$

算例 3（反求状态数，**2023-34** 原题）：某无噪声理想信道带宽 4 MHz，采用 QAM 调制，最大数据传输率是 48 Mb/s，求采用的 QAM 方案。

由 $C_{\max} = 2W \log_2 N$ 得

$$48 \times 10^6 = 2 \times 4 \times 10^6 \times \log_2 N \implies \log_2 N = 6 \implies N = 2^6 = 64$$

即 **QAM-64**，每个码元携带 6 bit。

算例 4（反求码元速率）：带宽 4 kHz 的无噪声信道，采用 8 电平 PAM 调制，求极限码元速率与数据率。

$$R_{B,\max} = 2W = 2 \times 4 \times 10^3 = 8000 \text{ Baud}, \qquad R_b = 8000 \times \log_2 8 = 8000 \times 3 = 24 \text{ kb/s}$$

（三）香农定理

内容：在带宽受限且有高斯白噪声干扰的信道中，信道的极限数据传输速率（信道容量）为

$$C = W \log_2\left(1 + \frac{S}{N}\right) \quad \text{(b/s)}$$

符号含义与单位：

符号	含义	单位	说明
W	信道带宽	Hz	频率口径
S	信道内所传信号的平均功率	W	—
N	信道内部的高斯噪声功率	W	注意这个 N 与奈氏准则里的状态数 N 是两个完全不同的量
S/N	信噪比（signal-to-noise ratio）	无单位（倍数）	必须换算成纯比值再代入
C	信道的极限数据传输速率（信道容量）	b/s	有噪声时的上限

信噪比的两种表示与换算（必考）：

- 比值形式：S/N，无单位，例如 $S/N = 1000$。
- 分贝形式：单位为 dB，定义为

$$\text{信噪比 (dB)} = 10\log_{10}\left(\frac{S}{N}\right)$$

反过来，由 dB 求倍数：

$$\frac{S}{N} = 10^{\frac{\text{dB}}{10}}$$

常用换算建议直接背下来：

dB	**10**	**20**	**30**	**40**
S/N	10	100	1000	10000

香农定理的四条要点

1. 公式里的 N 是噪声功率，与奈氏准则的状态数 N 同名不同义，写题时最好改记成 S/N 整体。
2. C 的单位是 b/s，且 C 与码元状态数 N 无关——香农定理不关心你用什么调制方式，它给出的是物理上限。
3. W 越大、S/N 越大，C 越大；但 C 随 S/N 是对数增长，靠加大功率提升速率收益极低。
4. 香农定理给出的上限不能突破。任何实际的编码与调制方案，其数据传输速率都只能在 C 以下（含）实现。

两个定理的适用条件绝不能互换

- 无噪声、理想低通信道 → 用奈氏：$C_{\max} = 2W\log_2 N$。题干里出现「无噪声」「理想信道」就锁定它。
- 有噪声信道 → 用香农：$C = W\log_2(1 + S/N)$。题干里给出「信噪比」「噪声」，或者问「极限速率」同时给了信噪比，就锁定它。
- 同时给出噪声和调制方式怎么办？两个都算，取小者作为实际可达到的最大可靠速率。这是「综合题」的标准套路。
- 一句话记牢：奈氏管码元速率上限（不考虑噪声），香农管信息速率上限（考虑噪声）。

算例 **5**（**2016-34** 原题）：某链路带宽 8 kHz，信噪比 30 dB，实际数据传输速率约为理论最大值的 50%，求实际速率。

先把 dB 换成倍数：$S/N = 10^{30/10} = 1000$。代入香农定理：

$$C = 8 \times 10^3 \times \log_2(1 + 1000) = 8 \times 10^3 \times \log_2 1001 \approx 8 \times 10^3 \times 9.97 \approx 79.7 \text{ kb/s}$$

取 50%：

$$79.7 \times 0.5 \approx 39.85 \text{ kb/s} \approx 40 \text{ kb/s}$$

算例 **6**（**2017-34** 原题）：若信道在无噪声情况下的极限数据传输速率不小于信噪比为 30 dB 条件下的极限数据传输速率，求信号状态数至少是多少。

香农上限（带宽记为 W）：

$$C_S = W \log_2(1 + 1000) = W \log_2 1001 \approx 9.97W$$

要求奈氏上限不小于它：

$$2W \log_2 N \ \geq \ 9.97W \implies \log_2 N \ \geq \ 4.985 \implies N \ \geq \ 2^{4.985} \approx 31.7$$

可选状态数中最小的是 **32**。

算例 **7**（反求信噪比）：某信道频率范围为 2 kHz 到 10 kHz，要求按香农定理算出的极限速率不低于 40 kb/s，求信噪比至少是多少。

带宽 $W = 10 - 2 = 8$ kHz（注意：给频率范围要先相减得带宽）：

$$40 \times 10^3 = 8 \times 10^3 \times \log_2(1 + S/N) \implies \log_2(1 + S/N) = 5 \implies 1 + S/N = 2^5 = 32$$

$$\frac{S}{N} = 31$$

算例 **8**（反求带宽）：信噪比 20 dB，要求信道容量达到 20 kb/s，求所需带宽至少是多少。

$S/N = 10^{20/10} = 100$，$\log_2(1 + 100) \approx 6.66$：

$$20 \times 10^3 = W \times 6.66 \implies W \approx 3003 \text{ Hz} \approx 3 \text{ kHz}$$

算例 **9**（两定理同时出现，取小者）：某信道带宽 4 kHz，所受高斯噪声使信噪比为 15，若采用 16 电平的调制方式，求该信道实际能达到的最大数据传输速率。

- 奈氏上限：$C_{\max} = 2 \times 4000 \times \log_2 16 = 8000 \times 4 = 32$ kb/s
- 香农上限：$C = 4000 \times \log_2(1 + 15) = 4000 \times 4 = 16$ kb/s

实际最大可靠速率受更紧的那个约束决定，取两者较小值：

$$R_{\max} = \min(32, 16) = 16 \text{ kb/s}$$

（四）两个定理的综合对比

对比项	奈奎斯特定理（奈氏准则）	香农定理
考虑噪声	不考虑（理想低通信道，无噪声）	考虑高斯白噪声
限制对象	码元传输速率上限 $2W$ Baud	信息传输速率上限
公式	$C_{\max} = 2W \log_2 N$	$C = W \log_2(1 + S/N)$
影响因素	带宽 W、码元状态数 N	带宽 W、信噪比 S/N
与调制方式的关系	有关（N 由调制方式决定）	无关（不给 N 留位置）
与信噪比的关系	无关	直接相关，对数增长
核心用途	求「无噪声时能达到多快」、反求 N	求「有噪声时的物理上限」、反求 W 或 S/N
典型真题	2009-34、2017-34、2022-34、2023-34	2016-34、2017-34

计算题的四种提问方式

1. 给 W 和调制方式，求最大速率 → 奈氏。注意 N 的乘法组合。
2. 给 W 和 **dB**，求最大速率 → 香农。先 $10^{\mathrm{dB}/10}$ 换算。
3. 给速率和 W（或给信噪比），求 N 或带宽 → 反向套公式，注意取满足不等式的最小 2 的幂。
4. 给「实际速率是理论值的 x%」 → 先算理论值再乘比例，别忘了这一步（2016-34 就考了 50%）。

三、编码与调制

（一）数字数据编码为数字信号

编码（**Encoding**）把数字数据变换成数字信号，得到的通常是基带信号。这一类编码共有六种要掌握，核心是记住每种编码的跳变规则、自同步能力、以及波特率与数据率的关系。

*1.*归零编码（*RZ*）

- 规则：用高电平表示 **1**、低电平表示 **0**（或相反），并且每个码元的中间时刻信号都回归到零电平。
- 接收方只要在信号归零之后的时刻采样即可判断电平，因此自带同步机制，称为自同步信号。
- 代价：每个码元都有一段「归零」时间不携带信息，大部分带宽被浪费用于传输归零，编码效率低。
- 波特率与数据率：一个码元携带 1 bit，但需在 1 个码元时隙内完成「有电平 + 归零」两个阶段，因此波特率是数据率的 **2** 倍（带宽效率约 0.5 bit/Hz 量级）。

*2.*不归零编码（*NRZ*）

- 规则：整个码元时间内的电平保持不变，用高低电平直接表示 1 和 0（例如高电平 =1、低电平 =0），中间不归零。
- 优点：一个时钟周期全部用来传数据，编码效率最高，波特率等于数据率（每码元 1 bit）。
- 缺点：没有自同步能力。如果连续出现多个相同的比特（例如一长串 0），电平长期不变，接收方无法判断每个比特的边界在哪里，收发双方必须另接一根时钟线。
- 统考波形题里，NRZ 的特征是「连续相同比特时电平保持不动」。

*3.*反向不归零编码（*NRZI*）

- 规则：用电平的跳变表示 **0**，用电平保持不变表示 **1**（统考与多数教材口径；也有教材定义相反，以题目给出的约定为准）。整个码元时间内不归零。
- 特点：跳变本身是一种通知机制，它把时钟信息嵌进了数据里。因此 NRZI 既能传输时钟信号实现自同步，又尽量不损失系统带宽，是 RZ 与 NRZ 优点的折中。
- 应用：USB 2.0 就采用 NRZI 编码。
- 波特率与数据率：每码元 1 bit，波特率 = 数据率。

*4.*曼彻斯特编码（*Manchester*）

- 规则：每个码元的中间必定发生一次电平跳变，这次跳变既充当时钟信号（用于接收方同步），又表示数据。
- 数据含义由跳变方向决定。两种约定并存，统考按题图约定判断：

约定	位中间由低 → 高	位中间由高 → 低
IEEE 802.3 常用约定	表示 0	表示 1
相反约定（部分教材）	表示 1	表示 0

- 关键的波形描述：每个比特周期被中点跳变劈成两个相等的半位，前半位与后半位电平必定相反。因此曼彻斯特波形中绝不可能出现连续两个相同电平跨越一个完整比特周期的情况。
- 优点：自同步（跳变频繁，时钟容易提取）；抗干扰能力强，因为数据由跳变方向而非绝对电平决定，直流分量小。
- 代价：每个比特需要 **2** 个码元，波特率是数据率的 **2** 倍，编码效率只有 **0.5 bit/Hz**，所需带宽是 **NRZ** 的 **2** 倍。
- 应用：传统 10BaseT 以太网（双绞线以太网）就采用曼彻斯特编码。

*5.*差分曼彻斯特编码（*Differential Manchester*）

- 规则：每个码元的中间仍然必定发生一次电平跳变，但这个中间跳变只表示时钟，不表示数据。
- 数据由每个码元开始处是否发生跳变表示：

位起始处	表示的数据
有跳变	0
无跳变（电平延续上一位的后半位）	1

- 波形描述与判读步骤（这是解题的固定动作）：
  1. 先用题目给的比特周期虚线（或码元宽度）把波形切成若干段；
  2. 观察每一段的中点：必然有一次跳变（用来对齐时钟）；
  3. 观察每一段的起始边界：与上一段末尾电平比较，变了就是 **0**，没变就是 **1**；
  4. 逐段编号即得比特串。
- 优点：既有曼彻斯特的全部自同步与抗干扰优点，又因为数据由「是否跳变」的相对关系决定，抗干扰能力比曼彻斯特更强（极性反转不影响判读）。
- 波特率与数据率：与曼彻斯特相同，波特率 = **2** × 数据率。

- 应用：IEEE 802.5 令牌环网。

曼彻斯特 vs 差分曼彻斯特，只看一个位置

- 曼彻斯特：看位中间的跳变方向（上跳还是下跳）。
- 差分曼彻斯特：看位开始处是否跳变（跳 = 0，不跳 = 1）。

两者位中间都必变。所以看到「位中间必变」不能立刻断定是曼彻斯特——还要看它是否复用位起始处的跳变来表示数据。2015-34 的判断依据就是：编码 1 全周期电平不动 → NRZ；编码 2 位中间必变且用方向表示数据 → 曼彻斯特。

6.4B/5B 编码

- 规则：把每 **4 bit** 数据映射为 **5 bit** 的码组再发送。5 bit 共 32 种组合，从中挑选 16 种「跳变足够多」的码组用于表示 16 种 4 bit 数据，其余码组作控制码或禁用（保证不会出现长串连续相同电平）。
- 目的：保证信号中有足够的电平跳变，便于接收方提取时钟（解决 NRZ 类编码的同步问题），同时把编码开销控制在可接受范围。
- 代价：每 4 bit 数据要发 5 bit，额外开销 **25%**，即

$$R_B = R_b \times \frac{5}{4} = 1.25 R_b$$

- 应用：100BaseFX 快速以太网。100 Mb/s 的数据率对应 $1.25 \times 10^8 = 125$ MBaud 的线路波特率。

*7.*六种编码对比汇总

编码	位中间是否必变	数据如何表示	自同步	每比特码元数	波特率：数据率	主要缺点
归零 RZ	是（归零）	高电平 =1，低电平 =0	有	2	2 : 1	浪费带宽，效率低
不归零 NRZ	否	高电平 =1，低电平 =0	无	1	1 : 1	需另设时钟线
反向不归零 NRZI	否	跳变 **=0**，不变 **=1**	有	1	1 : 1	仍可能出现长串不变
曼彻斯特	是	位中间跳变方向	有	2	2 : 1	带宽开销加倍
差分曼彻斯特	是	位起始处是否跳变（跳 =0，不跳 =1）	有（更强）	2	2 : 1	带宽开销加倍
4B/5B	设计保证有	查表映射	有	1.25	5 : 4	25% 额外开销

编码类真题的三问

统考在这一块的出题形式固定为两种：

1. 给波形判编码类型：看位中间动不动 → 动则排除 NRZ/NRZI；看数据是由「绝对电平」还是「跳变方向/有无跳变」决定 → 区分曼彻斯特、差分曼彻斯特、NRZI。2015-34 就是这个套路。
2. 给编码类型读比特串：2013-34（10BaseT 曼彻斯特，答案 `00110110`）、2021-34（差分曼彻斯特，答案 `10111001`）都是此类。解题时务必先按虚线切分比特周期，再做判断。

（二）数字数据调制为模拟信号

调制（**Modulation**）把数字数据变换成模拟信号，使用载波（**Carrier**）的三个参量之一或多个来携带信息：幅度、频率、相位。对应的三种基本调制方式如下。

幅移键控（**Amplitude Shift Keying**，**ASK**）：改变载波的幅度表示 0 和 1（有载波为 1、无载波为 0，或两个不同幅度）。实现简单，但抗干扰能力差，受幅度噪声影响大。

频移键控（**Frequency Shift Keying**，**FSK**）：改变载波的频率表示 0 和 1，需要两个不同频率的载波。抗干扰能力强于 ASK，但占用带宽较大。

相移键控（**Phase Shift Keying**，**PSK**）：改变载波的相位表示数据。分为：

- 绝对相移（**PSK**）：用相位的绝对值表示数据；
- 相对相移（**DPSK**，差分相移键控）：用相邻码元的相位差表示数据，不需要绝对相位参考，抗干扰与实现上更实用。

正交幅度调制（**Quadrature Amplitude Modulation**，**QAM**）：同时改变载波的幅度和相位，是 ASK 与 PSK 的组合。若有 m 种相位、n 种振幅，理论上可组合出 $m \times n$ 个状态点（星座点）。

各种调制方式的状态数与比特率：

调制方式	改变的载波参量	状态数 N	每码元比特数	备注
2ASK	幅度	2	1	—
4ASK	幅度	4	2	4 个幅值
2FSK	频率	2	1	需 2 个频率载波（2024-34 考点）
4FSK	频率	4	2	4 个频率
2PSK / BPSK	相位	2	1	二相
4PSK / QPSK	相位	4	2	四相
8PSK	相位	8	3	八相
QAM-16	幅度 + 相位	16	4	4×4
QAM-32	幅度 + 相位	32	5	星座图非矩形排列
QAM-64	幅度 + 相位	64	6	8×8，2023-34 答案
QAM-128	幅度 + 相位	128	7	—

QAM 状态数的两种算法要分清

- 题干说「m 个相位，每个相位 n 种振幅」$\rightarrow N = m \times n$（乘法组合）。
- 题干直接说「QAM-64」$\rightarrow N = 64$。
- 不要把 **QAM-32** 强行写成 4×8 之外的东西；只要题目给了「方案名」，N 就是名字里的数字；只有题目给了相位数与振幅数时才去乘。

算例 **1**（调制方式与数据率）：某信道采用 QAM-16 调制，波特率为 2400 Baud，求数据率。

$N = 16$，每码元 $\log_2 16 = 4$ bit：$R_b = 2400 \times 4 = 9600$ b/s。

算例 **2**（由速率推调制方案）：带宽 4 MHz 的无噪声信道要达到 48 Mb/s，求最小 N。

$$\log_2 N = \frac{48 \times 10^6}{2 \times 4 \times 10^6} = 6 \implies N = 64$$

算例 3（比较）：同样 3000 Baud 的波特率下，2ASK、QPSK、QAM-64 的数据率分别是 3000 b/s、6000 b/s、18000 b/s。波特率相同，比特率取决于每码元比特数——这就是提高数据率的两条路径之一（另一条是提高波特率，但受奈氏上限约束）。

（三）模拟数据编码为数字信号：PCM

把模拟数据（话音、音频、图像）变换成数字信号，最典型的方法是脉冲编码调制（**Pulse Code Modulation**，**PCM**），分三步：

第一步：采样（**Sampling**）。按固定的时间间隔 T_s 取出模拟信号的瞬时值，把连续时间信号变成离散时间信号。采样频率 $f_s = 1/T_s$。

采样定理（又称奈奎斯特定理，注意与信道容量的奈氏准则同名但内容不同）：

$$f_s \geq 2f_{\max}$$

其中 $f_{\max}$ 是原始模拟信号中最高频率分量的频率。$\boxed{2f_{\max}}$ 称为奈奎斯特采样频率。

第二步：量化（**Quantization**）。把采样得到的瞬时幅度值四舍五入到有限个预设的离散电平上。若量化级数为 Q（即分成 Q 个等级），则每个样本需要

$$n = \log_2 Q \quad \text{(bit)}$$

位二进制编码表示。量化会引入量化误差，量化级数越多误差越小、所需位数越多。

第三步：编码（**Encoding**）。把每个量化后的电平值用 n 位二进制码表示，得到数字比特流。

PCM 的数据率：

$$R_b = f_s \times n = f_s \times \log_2 Q \quad \text{(b/s)}$$

$$\text{所需带宽（无噪声时按奈氏反推）} \quad W \geq \frac{R_b}{2\log_2 N}$$

其中 f_s 是采样频率（Hz），n 是每个样本的二进制位数。

PCM 的四条易错

1. 采样定理说的是 $f_s \geq 2f_{\max}$，比的是「最高频率分量」，不是信号的频率范围或带宽。若给出频率范围 $0 \sim 4$ kHz，则 $f_{\max} = 4$ kHz，$f_s \geq 8$ kHz。
2. 采样频率与信道带宽是两回事。千万级采样率（如 CD 的 44.1 kHz）不需要同量级带宽的「信道容量」，PCM 数据率才是要传输的比特率。
3. 「采样一次 = 得到一个样本 = 得到一个码元」。若每个样本编成 n 位，则这 n 位合起来才是一个码元所承载的信息，此时码元状态数 $N = 2^n$。
4. 量化级数 Q 必须是 **2** 的幂。若题目给「8 级」「256 级」，n 分别是 3 和 8。

算例 1（**PCM** 数据率）：话音信号最高频率 4 kHz，采样频率取 8 kHz，量化级数 256，求 PCM 数据率。

$n = \log_2 256 = 8$ bit：

$$R_b = 8000 \times 8 = 64 \text{ kb/s}$$

这就是传统数字电话的 64 kb/s 标准速率（DS0）。

算例 **2**（经典综合题）：有一个无噪声的 **8 kHz** 信道，每个信号包含 8 级量化，每秒采样 24k 次，求可以获得的最大传输速率。

- 直接按「每秒采样次数 × 每样本位数」计算：$n = \log_2 8 = 3$ bit，$R_b = 24 \times 10^3 \times 3 = 72$ kb/s。
- 但还要用奈氏准则检查信道是否支持：$C_{\max} = 2 \times 8 \times 10^3 \times \log_2 8 = 48$ kb/s。

两者取小，实际最大传输速率为 **48 kb/s**。这道题的设计意图正是考查「PCM 数据率可能超过信道容量，必须取小」这一判断。

算例 **3**（反求量化级数）：某信道带宽 6 kHz，无噪声，采用 PCM 传输一个最高频率 3 kHz 的模拟信号，要求数据率尽可能高，求最大量化级数。

- 采样定理要求 $f_s \geq 2 \times 3000 = 6000$ Hz，取 $f_s = 6$ kHz 时恰好满足。
- 奈氏上限：$C_{\max} = 2 \times 6000 \times \log_2 N = 12000 \log_2 N$。
- 而 PCM 数据率 $R_b = 6000 \times \log_2 N$，恒小于 $C_{\max}$，所以带宽不构成约束；量化级数由题目给定的位数上限决定。这说明当 $f_s = 2W$ 时，**PCM** 数据率天然在信道容量之内，这一点在综合题中常被用来排除错误的约束条件。

（四）模拟数据调制为模拟信号与频分复用

模拟数据本身已是模拟信号，若要在适合长距离传输的频段上传送，需要把它的频谱搬移到载波附近，这叫模拟数据调制为模拟信号，常用手段仍是调幅（AM）、调频（FM）、调相（PM），对应设备是放大器与调制器。典型场景是电话机到本地端局之间的传输。

频分复用（**Frequency Division Multiplexing**，**FDM**）：把信道的总频带划分成若干互不重叠的子频带，每个子频带构成一个子信道，供一路信号使用。所有用户同时占用不同的频率资源。

- 每个子信道之间需要留保护频带（**guard band**），防止相邻子信道互相干扰。
- 特点：各路信号在时间上重叠、在频率上分离；所有用户同时在用。
- 应用：有线电视、传统模拟载波电话系统。

复用技术的一句话区分（为后续章节铺垫）

- 频分复用 **FDM**：按频率分，大家同时用；用于模拟信号，如载波电话、有线电视。
- 时分复用 **TDM**：按时间分，大家轮流用；用于数字信号。
- 波分复用 **WDM**：本质就是光的频分复用，在一根光纤中同时传多个不同波长的光载波。
- 码分复用 **CDM**：用正交的码片序列区分不同用户，大家同时同频使用。

四、电路交换、报文交换与分组交换

（一）三种交换方式的工作过程

*1.*电路交换（*Circuit Switching*）

电话网是典型代表。通信过程分为三个明确的阶段：

1. 建立连接：主叫方向网络请求，网络沿途为这次通话分配一条独占的物理通路（包括链路与交换机时隙/带宽资源）。这一阶段有呼叫建立时延。
2. 传输数据：数据沿已经建好的通路透明地从源传到目的，中间结点不对数据做任何处理（不存储、不检查、不转发决策）。整条通路被这次通信独占。

3. 释放连接：通信结束后归还所占资源。

特点：

- 传输期间时延小且稳定（通路已建立，没有每跳的存储转发开销），适合实时性要求高的业务，如话音。
- 资源利用率低：独占通路，空闲时别人也用不了。
- 不具备差错检验能力：中间结点不处理数据，可靠传输要靠上层或端设备负责。
- 建立连接本身有额外时延；若连接建立失败，通信无法进行。

*2.*报文交换（*Message Switching*）

报文（**Message**）是用户数据加上源地址、目的地址等信息构成的完整数据块。

工作过程：发送方把整个报文发给相邻交换结点；交换结点必须把整个报文完整接收下来（存储），再查转发表找出下一跳，然后把整个报文转发出去（转发）。

特点：

- 采用存储转发（**Store-and-Forward**）机制：结点要收完整个报文才能开始转发，因此无法流水线化。
- 需要的缓存空间很大（要能装下整个报文），转发时延很长（转发时延 = 排队 + 处理 + 传输时延）。
- 线路利用率高于电路交换（多个报文可以分时共享链路），但远低于分组交换。
- 报文长度不受限，长报文容易长时间占用链路。

*3.*分组交换（*Packet Switching*）

分组（**Packet**）是把报文划分成若干等长的数据段，每段前面加上首部（分组头）构成的数据单元。首部中含目的地址等控制信息。

工作过程：发送方构造分组并发送；交换结点缓存收到的分组，按分组中的目的地址查转发表，决定从哪个端口转发出去；接收方接收分组并还原成原报文。

特点：

- 仍然采用存储转发，但以「分组」为单位：结点收完一个短分组就可以开始转发，下一个分组可以紧跟着到达并转发，从而实现流水线（**pipelining**）传输，把端到端时延压缩到「最后一段链路的收尾加上第一段的启动」。
- 缓存需求小，链路利用率最高，多路分组可以交替复用同一条链路，公平共享。
- 代价是引入了新的问题：分组可能失序、丢失；无端到端预留资源，各分组独立路由、动态竞争链路资源，负载过大时结点队列无限堆积会造成网络拥塞。
- 分组过小会降低有效载荷比例（首部开销占比上升），分组过大则流水线收益下降；实际系统按最大传送单元 **MTU** 权衡。

三种交换「存储转发」的区别（一句话）

电路交换：不存储转发，通路独占。报文交换：存储转发，单位是整个报文，不可流水线。分组交换：存储转发，单位是短分组，可以流水线。后者正是分组交换能把时延压到 $(p+k-1)T_t$ 的原因。

（二）三种交换方式的对比

对比维度	电路交换	报文交换	分组交换
是否需要建立连接	需要（先建后传，再释放）	不需要	不需要（数据报方式）；虚电路方式需要
资源占用	独占通路，全程占用	不独占，逐跳存储转发	不独占，逐跳存储转发，统计复用
中间结点是否处理数据	不处理，透明传输	处理（收完整个报文再转发）	处理（按分组首部查表转发）
存储转发单位	无	整个报文	分组
能否流水线传输	不涉及	不能	能
结点缓存需求	小	很大	小
传输时延	小且稳定（通路已建立）	最大（收完才转发）	介于两者之间，随负载波动（排队时延）
建立时延	有（呼叫建立时间）	无	无
线路利用率	最低	中等	最高
差错控制	不具差错检验能力	由结点和端系统共同处理	数据报不保证，由上层（如 TCP）保证；虚电路可由网络保证
灵活性	差（通路固定，故障即中断）	较好	好（分组可独立选路）
适用场景	话音等实时、长时、连续业务	早期电报、传真等短报文业务	数据通信的主流，互联网的基础
典型例子	传统电话网 PSTN	早期电报网	因特网、以太网中的 IP 转发

最容易混淆的三组对比

1. 电路交换「时延小」指的是传输阶段，不是全过程。加上建立连接时延后，短报文用电路交换反而可能更慢（建链成本摊不开）。
2. 报文交换与分组交换都用存储转发，区别在「单位」。说「报文交换不可流水线、分组交换可流水线」才是得分点。
3. 「分组交换时延最小」是错的。传输阶段时延最小的仍是电路交换；分组交换的优势在于线路利用率高、灵活、适合突发数据。真题常把「时延小」作为电路交换的优势来考，把「利用率高」作为分组交换的优势来考，不要对调。

（三）分组交换的时延计算

这是本章计算量最大、最容易失分的部分。先建立符号体系。

符号约定：

符号	含义	说明
L	一个分组的长度	单位 bit（若给 B 要 $\times 8$）
R	链路的数据传输速率（发送速率）	单位 b/s
T_t	单段链路上一个分组的发送时延	$T_t = L/R$
T_p	单段链路上的传播时延	T_p = 链路长度/传播速度
p	分组的数量	由报文总长度除以每分组数据部分得到

符号	含义	说明
k	从源到目的经过的链路段数（段数 = 经过的路由器数 + 1）	无单位
h	每个分组的首部长度	计算有效载荷时要减掉
D	端到端总时延	—

基本推导（以 $k=3$、p 个分组为例）：

- 第 1 个分组从第 **1** 段链路发出需要一个 T_t，接下来它还要在第 **2**、**3** 段链路各被发送一次，共需 $3T_t$ 才能到达终点。
- 但第 **2** 个分组不必等第 1 个分组走完全程：当第 1 个分组离开第 1 段链路后，第 2 个分组立刻开始占用第 1 段链路。于是各分组在各级链路上像流水线一样首尾相接。
- 因此，最后一个分组在最后一段链路上「收尾」的时刻，就等于「第 1 个分组走完全部 k 段」+「其余 $p-1$ 个分组依次占用最后一段链路」，即

$$\boxed{D=(p+k-1)\,T_t}$$

- 若还要计入传播时延（题目给信道长度或时延带宽积），需在每一段链路上各累加一次：

$$\boxed{D=(p+k-1)\,T_t+k\,T_p}$$

变体公式：

- 若各段链路速率不同，T_t 要分段计算，不能简单乘以 k：$D=\sum_{i=1}^{k}T_{t,i}+(p-1)\,T_{t,k}+k\,T_p$（最后一个分组在最后一段的发送时延单独计）。
- 若还要计入结点处理时延 T_{proc} 与排队时延 T_q，逐跳累加：$D=(p+k-1)T_t+k(T_p+T_{proc}+T_q)$。
- 报文交换的对应公式（不可流水线，每跳都要收完整个报文）：

$$D_{\text{报文}}=k\times\frac{M}{R}+k\,T_p$$

其中 M 是报文总长度（bit）。

「+1 还是 -1」的判断法

流水线总时延可以这样记：先算「第 **1** 个分组从头走到尾」的时间，再加上「剩余分组一个个跟上」的时间。

$$D=\underbrace{k\,T_t}_{\text{第 1 个分组走完全程}}+\underbrace{(p-1)T_t}_{\text{其余分组逐个补上}}=(p+k-1)T_t$$

只要抓住「最后一段链路上要连续发送 p 个分组，而在它开始发第 **1** 个分组之前，已经过去了 $(k-1)$ 个 T_t」，就是 $(k-1)T_t+pT_t=(p+k-1)T_t$。

时延计算的五个坑

1. 分组数量按「有效载荷」算，不是按分组总长算。2010-34 中分组长 1000 B、首部 20 B，则每个分组只带 980 B 数据，$p=980000/980=1000$ 个；若误按 1000 B 算会得到 980 个，时延随之算错。
2. **B** 与 **bit** 的换算。分组长度给 B 要乘 8；但有些题目（如 2013-35）直接给 bit（10 kb），这时不要再乘 **8**。
3. 链路段数 k = 路由器数 + **1**。图里画了 2 个路由器就是 3 段链路。
4. 传播时延要乘 k（每段各一次），不是乘 **1**。2023-33 的 0.02 ms 就是 2×0.01 ms。

5. 不要自行脑补「分流、负载均衡、多路径」。除非题目明确画出并行路径并问最大吞吐量，否则一律按单一路径串行计算——长路径只会增加转发时延。

算例 1（2010-34 原题）：采用存储转发分组交换，所有链路速率 100 Mb/s，分组大小 1000 B，其中分组头 20 B。主机 H1 向 H2 发送 980 000 B 的文件。忽略分组拆装时间和传播时延，求从 H1 开始发送到 H2 接收完毕所需的最少时间。

- 每分组有效数据：$1000-20=980$ B；
- 分组数：$p=980000/980=1000$ 个；
- 单分组发送时延：$T_t=\dfrac{1000\times 8}{100\times 10^6}=80\ \mu\text{s}=0.08$ ms；
- 图中 H1 经两个分组交换结点到 H2，链路段数 $k=3$；
- 取 3 段中一条最短路径（题目问「至少」），按流水线公式：

$$D=(p+k-1)T_t=(1000+3-1)\times 0.08=80.16\ \text{ms}$$

对比：只算第一段链路的 $1000\times 0.08=80$ ms 会漏掉流水线启动开销 0.16 ms。

算例 2（2013-35 原题）：主机甲经 1 个路由器（存储转发）与主机乙互联，两段链路速率均为 10 Mb/s。甲分别采用报文交换和分组大小为 **10 kb** 的分组交换向乙发送 1 个大小为 8 Mb 的报文。忽略传播时延、分组头开销和分组拆装时间，求两种方式的总时间。

报文交换（$k=2$ 段链路，每段都要发完整个 8 Mb 报文）：

$$D_{\text{报文}}=2\times\frac{8\times 10^6}{10\times 10^6}=2\times 0.8\ \text{s}=1600\ \text{ms}$$

分组交换：

- 分组数 $p=\dfrac{8\ \text{Mb}}{10\ \text{kb}}=\dfrac{8\times 10^6}{10\times 10^3}=800$ 个；
- 单分组发送时延 $T_t=\dfrac{10\times 10^3}{10\times 10^6}=1$ ms；
- $k=2$ 段链路：

$$D_{\text{分组}}=(800+2-1)\times 1=801\ \text{ms}$$

（对照：若用报文交换公式错套成 $800\times 2=1600$ ms，或漏掉传播，都会选错。）

算例 3（2023-33 原题）：H1 和 H2 通过路由器互联，2 段链路速率均为 100 Mb/s，每段链路的时延带宽积（单向传播时延 × 带宽）均为 1000 bit。H1 向 H2 发送 1 个大小 1 MB（$M=10^6$）的文件，分组长度 1000 B，求从 H1 开始发送到 H2 收到全部数据所需的最少时间。

- 分组数：$p=\dfrac{10^6\ \text{B}}{1000\ \text{B}}=1000$ 个；
- 单分组发送时延：$T_t=\dfrac{1000\times 8}{100\times 10^6}=80\ \mu\text{s}=0.08$ ms；
- 单向传播时延（由时延带宽积反求）：$T_p=\dfrac{1000}{100\times 10^6}=10\ \mu\text{s}=0.01$ ms；
- 链路段数 $k=2$：

$$D=(p+k-1)T_t+k\,T_p=(1000+2-1)\times 0.08+2\times 0.01=80.08+0.02=80.10\ \text{ms}$$

算例 4（三跳、含首部开销）：源主机经 3 个路由器向目的主机发送 1000 个分组，每段链路速率 10 Mb/s，分组长度 1500 B，每段链路传播时延 5 ms。求总时延。

链路段数 $k=3+1=4$。$T_t=\dfrac{1500\times 8}{10\times 10^6}=1.2$ ms。

$$D=(1000+4-1)\times 1.2+4\times 5=1203.6+20=1223.6\text{ ms}$$

算例 5（反求分组数）：某文件 5000 bit，分组长度固定 100 bit（含 20 bit 首部），链路速率 R，经过 3 段链路，求端到端发送时延的表达式。

每分组有效载荷 80 bit，$p=\lceil 5000/80\rceil=63$ 个（不足一个分组也要发一个，注意向上取整），$T_t=100/R$：

$$D=(63+3-1)\times\frac{100}{R}=\frac{6500}{R}$$

三点提醒

1. 题目问「至少」通常意味着可以走最短路径（段数最少的路径），也可能意味着「不考虑排队和处理时延的乐观情形」。
2. 吞吐量题用「木桶原理」：端到端最大吞吐量取路径上所有链路的最小值。2024-33 中，中间三条并行路径虽有大带宽，但两端公共链路都只有 10 Mb/s，故最大吞吐量 = $\min(10,1000,1,100,10)=10$ Mb/s。
3. 时延带宽积 = 传播时延 × 带宽，单位是 bit，表示「链路上正在传播的比特数」（以比特为单位的链路长度）。它常被用来反求传播时延，是 2023-33 的关键一步。

五、数据报与虚电路

分组交换可以进一步分成两种工作方式：无连接的数据报（**Datagram**）方式与面向连接的虚电路（**Virtual Circuit**，**VC**）方式。它们都由网络层提供，是网络层向上提供服务的两种风格。

（一）数据报方式

- 每个分组独立携带完整的源地址和目的地址。
- 每个分组独立地进行路由选择，同一报文的不同分组可能走不同的路径。
- 中间结点不维护任何关于这次通信的状态（无状态转发）。
- 分组可能失序到达，也可能丢失，网络只提供「尽力而为（**best-effort**）」的转发服务，不承诺交付结果。
- 可靠性、顺序保证、流量控制由端主机上的上层协议（如 **TCP**）负责。
- 对网络故障的适应能力强：某条链路或结点故障后，后续分组可以重新选路绕过。
- 典型代表：**IP** 网络（因特网）。

（二）虚电路方式

- 数据传输之前必须先建立一条逻辑连接，即虚电路。建立阶段进行路由选择，确定沿途经过的结点与端口。
- 连接建立后，沿途每个结点都记录一条虚电路表项，每个分组只需携带一个很短的虚电路号（**VCID**），结点按 VCID 查表转发。
- 同一条虚电路上的所有分组沿同一条路径转发，因此保证分组按序到达。
- 差错处理与流量控制可以由分组交换网自身负责，也可以由用户主机负责。
- 对网络故障的适应能力差：所有经过故障结点的虚电路都会中断。
- 典型代表：帧中继、**ATM**、**X.25**。

虚电路 ≠ 电路交换，「预分配带宽」是高频错项

- 电路交换是真正的物理线路交换（如电话网），它预分配带宽，资源被独占。
- 虚电路只是逻辑上的连接，本质是一种多路复用技术：一条物理链路可以承载多条虚电路，多条虚电路共享链路带宽，通常采用统计复用。
- 因此「虚电路需要为每条连接预分配带宽」是错的（2020-34 的答案正是这一句）。只有需要特定服务质量保证时才可能做资源预留。

另外注意：虚电路服务是 **OSI** 参考模型的特色，**TCP/IP** 模型中没有虚电路服务——TCP/IP 的网络层（IP）只提供数据报服务。

（三）数据报与虚电路的对比

对比维度	数据报服务	虚电路服务
连接的建立	不需要	必须建立
目的地址	每个分组都带完整目的地址	仅在建立阶段使用；之后每分组用较短的虚电路号 **VCID**
路由选择	每个分组独立进行路由选择和转发	同一条虚电路的分组按同一路由转发
分组顺序	不保证有序到达	保证有序到达
可靠性	网络只负责尽力转发，不承诺交付结果，可靠性由上层协议保证	可靠性由网络自身保证
差错处理与流量控制	由用户主机上的上层协议（如 TCP）负责	可由分组交换网负责，也可由用户主机负责
对网络故障的适应性	强，可寻找新路径转发	弱，所有经过故障结点的虚电路都不能工作
结点状态	无状态（不维护通信上下文）	有状态（维护虚电路表）
典型网络	因特网 IP	帧中继、ATM、X.25

一句话总结设计哲学：数据报服务用「结点的无状态」换取网络的简单与健壮；虚电路服务用「结点的状态记忆」换取结点的控制能力。

常考的三个判断

1. 「虚电路可以确保数据分组传输顺序」→ 对。
2. 「虚电路建立时需要进行路由选择」→ 对（路由选择发生在建立阶段）。
3. 「虚电路需要为每条虚电路预分配带宽」→ 错（2020-34 答案）。
4. 「一条虚电路只能为一个进程服务 / 端系统之间只能有一条虚电路」→ 错。两个端系统之间可以有多条虚电路，为不同进程服务。

六、传输介质

传输介质是数据传输系统中发送器与接收器之间的物理路径，分为导向传输介质（**guided media**）与非导向传输介质（**unguided media**）两大类：前者是固体介质（双绞线、同轴电缆、光纤），信号沿介质传播；后者是自由空间（无线电波、微波、红外、激光）。

（一）双绞线

构成：把两根相互绝缘的铜导线按一定规则并排绞合（**twist**）在一起，通常再把多对这样的线对捆成一根电缆。

为什么要绞合：两根导线在空间上不断交换位置，使得两根线感受到的外部电磁干扰、以及自身向外辐射的电磁场尽可能相互抵消，从而减少相邻导线之间的电磁干扰和串扰。绞合越密（绞距越小），抗干扰能力越强，可支持的速率越高。

分类：

类型	全称	特点
UTP	非屏蔽双绞线（Unshielded Twisted Pair）	无金属屏蔽层，价格便宜、安装方便，最常用
STP	屏蔽双绞线（Shielded Twisted Pair）	带金属丝编织的屏蔽层，抗干扰能力更强，价格更高、更笨重、需良好接地

按电气性能分为不同类别（**Category**，**Cat**）：

类别	支持的典型速率	说明
Cat3	10 Mb/s	早期电话与 10BaseT
Cat5	100 Mb/s	100BaseT
Cat5e（超五类）	1 Gb/s（100 m 内）	增强型，串扰指标改善，千兆以太网常用
Cat6（六类）	1 Gb/s（100 m）；10 Gb/s（约 55 m 内）	内部有十字骨架分隔四对线，进一步降低串扰
Cat6a（超六类）	10 Gb/s（100 m）	屏蔽结构，更高频率

适用场景：局域网布线（最主流的介质）与传统电话网，模拟传输与数字传输都适用。

（二）同轴电缆

构成（由内到外）：内导体（铜芯）→ 绝缘层 → 网状编织的屏蔽层（外导体）→ 塑料外层。

特点：外导体屏蔽层具有良好的抗干扰特性，带宽比双绞线更大，传输速率更高、传输距离更远，但价格较贵、体积大、布线不便。

按特性阻抗分类：

类型	特性阻抗	传输的信号	典型应用
基带同轴电缆	50 Ω	基带数字信号	早期局域网（10Base5、10Base2）
宽带同轴电缆	75 Ω	宽带（频带）信号	有线电视系统（CATV）

适用场景：早期以太网（总线型局域网）、有线电视、宽带接入（HFC 混合光纤同轴）。

（三）光纤

构成：由纤芯（**core**）与包层（**cladding**）构成，利用光导纤维传递光脉冲来通信。纤芯折射率高于包层。

基本原理——光的全反射：当光从折射率大的纤芯射向折射率小的包层，且入射角大于临界角时，光不会折射进包层，而是全部被反射回纤芯，如此在纤芯内不断反射着向前传播，实现低损耗长距离传输。

分类：

类型	纤芯直径	光源	传输模式	距离与带宽	应用
多模光纤（MMF）	较粗（如 50 μm、62.5 μm）	发光二极管 **LED**	光以多种模式（多条路径）在纤芯中传播	距离较近、带宽较低；存在模式色散，脉冲会展宽	近距离传输，如楼内局域网
单模光纤（SMF）	很细（如 9 μm 左右）	半导体激光器 **ILD/LD**	光沿单一模式（近似直线）传播	距离远、带宽极高，衰减小	长距离、大容量骨干网

光纤相对于铜缆的优势：

- 带宽极大：一根光纤的潜在带宽远高于双绞线，是现代骨干网的基础。
- 传输损耗小、中继距离长：可长距离传输而无需频繁中继。
- 抗电磁干扰能力强：光信号不受电磁干扰，也不产生电磁辐射，保密性好（不易被窃听）。
- 体积小、重量轻、耐腐蚀。

缺点：价格相对较高；光纤对接与分路需要专用设备，施工难度大；光纤本身脆弱，机械强度不如铜缆。

单模/多模的三个判断要点

1. 单模纤芯细、多模纤芯粗。巧记：「单薄、多厚」。
2. 单模用激光（**LD**），多模用 **LED**。
3. 单模传得远、带宽高；多模传得近。看到题目问「长距离骨干网用什么」，答单模；「楼内局域网、近距离」答多模。
4. **10BaseT / 100BaseT** 用的是双绞线（T = Twisted pair），2019-34 的答案就是双绞线。

（四）无线传输介质

介质	频段/传播方式	是否需视线	特点与应用
无线电波	低频到超高频，向各个方向传播	否	穿透力强、传播距离远，可全向发射；频段拥挤，如广播、移动电话
微波	超高频（GHz 级），直线（视距）传播	是	需在视距内，地球曲率与障碍物会阻挡，因此要高塔架设 + 中继站转发；带宽大，如微波接力、卫星通信
红外	红外光，直线传播，不能穿透固体	是	短距离、同一房间内使用，不会穿透墙壁，天然隔离、抗窃听；如遥控器、早期 IrDA

介质	频段/传播方式	是否需视线	特点与应用
激光	激光，直线传播	是	需精确对准，方向性极强，短距离点对点；受天气影响大

微波为什么要中继：微波是直线传播的，而地球是曲面，且微波难以穿透障碍物。因此微波通信要求发送端与接收端在视距（**line of sight**）范围内；远距离传输时必须每隔一定距离（例如几十公里）建一个中继站，把信号接收后放大再转发，形成微波接力。

无线介质的共同点与区别

- 共同点：都是非导向介质，靠自由空间传播。
- 区别的判据：是否需要视线对准。无线电波不需要（可全向、可穿透）；微波、红外、激光都需要（直线/视距传播）。
- 高频段（微波及更高）虽然带宽大，但方向性强、绕射能力弱，因此对视线和天线对准要求高。

（五）物理层接口的特性

物理层接口规范（如 RS-232、USB 接口标准）从四个特性来描述：

特性	描述内容	记忆口诀
机械特性	接口所用的接线器形状、尺寸、引脚数目与排列、固定和锁定装置	看外形
电气特性	接口电缆各条线上出现的电压范围、阻抗、信号电平、传输速率、距离限制	看电压电流
功能特性	某条线上出现的某一电平的电压代表什么意义，即每根引脚/信号线完成什么功能	看引脚干什么
过程特性（又称规程特性）	对于不同功能的各种可能事件的出现顺序，即信号线上一组操作过程的时序关系	看动作先后

四个特性的判断口诀

- 出现「形状、尺寸、引脚数目、排列」→ 机械特性。
- 出现「电压范围、电平、阻抗、速率」→ 电气特性。
- 出现「某条线的作用、电平的意义」→ 功能特性。
- 出现「事件发生顺序、时序关系、操作过程」→ 过程特性。

2012-34 问「描述完成每种功能的事件发生顺序」，答案是过程特性。2018-34 问「不属于物理层接口规范定义范畴的是」，答案是物理地址（MAC 地址属于数据链路层）。接口形状属机械、引脚功能属功能、信号电平属电气，都在物理层范畴内。

与计算机组成原理的联动

物理层接口四特性与计算机组成原理中总线特性（机械、电气、功能、时间）几乎一一对应，只是计组里叫「时间特性」。复习时可以放在一起记，注意计组用「时间特性」而计网用「过程特性」。

七、物理层设备

物理层设备工作在比特级，只对信号做放大、整形、转发，不识别帧、**MAC** 地址或 **IP** 地址。

（一）中继器

作用：对接收到的数字信号进行整形、放大并转发，以扩大网络的传输距离。

为什么需要中继器：信号在传输介质中传输时会衰减，同时会因干扰而失真。当传输距离超过一定限度后，接收端收到的信号可能已经无法正确识别（既达不到判决门限，波形也发生了畸变），此时必须用中继器把已经衰减、畸变的信号再生（**regenerate**）成标准的数字波形再继续发送。

与放大器的区别（高频辨析点）：放大器放大的是模拟信号，它把模拟波形连同噪声一起线性放大；中继器再生的是数字信号，它重新产生干净的数字波形，不放大噪声。

中继器的性质：

- 不能分割冲突域，也不能分割广播域。中继器连接起来的所有网段同属一个冲突域，也同属一个广播域。
- 中继器工作的两端网段在物理层规范上必须相同，尤其数据传输速率必须相同（这也是「中继器不能连接两个速率不同的以太网」的原因）。介质可以不同（例如双绞线接光纤），但物理层协议与速率必须一致。
- 中继器没有存储转发能力，收到就转发，因此会引入很小的转发时延。

5-4-3 规则：因为信号的延迟是有规定的，中继器不能无限延长网络。在早期的 10BASE5 以太网规范中规定：

- 最多允许 **4** 个中继器串联；
- 这 4 个中继器连接 **5** 段通信介质；
- 其中只有 **3** 段可以挂接计算机（其余 2 段只能用作延长链路的中继段，不能连接主机）。

合起来即「**5** 段介质、**4** 个中继器、**3** 段可挂主机」。

5-4-3 的准确含义

- **5**：5 个网段（网段 = 中继器之间的介质段）。由 4 个中继器串联自然产生 5 段。
- **4**：4 个中继器。这是上限，不能是 5 个。
- **3**：只有 3 个网段可以连接计算机（通常是两端各一段加中间一段），另 2 段只能做纯延长段。

不要把「3」误记为「3 个中继器」。这个规则是 10BASE5 的规范，是历史遗留标准，但统考仍会考其数字。

（二）集线器

集线器（**Hub**）本质上是一个多端口的中继器，工作在物理层。

工作原理：集线器从某一个端口收到信号后，整形、放大，并向除该端口外的所有其他端口广播转发。它不认识 **MAC** 地址，不做任何路径选择，也没有交换表。

关键性质：

性质	说明
带宽共享	若集线器总带宽为 M，接入 N 台主机且同时工作，每台主机实际可用带宽约为 M/N
半双工	集线器采用半双工工作方式，同一时刻只能有一台主机成功发送（否则冲突）
冲突域	连接在集线器上的所有主机同属一个冲突域（一个冲突就是全网冲突）

性质	说明
广播域	也同属一个广播域（转发本来就是广播的）
转发方式	广播（向除源端口外的所有端口）
逻辑拓扑	物理连接是星状拓扑，但逻辑连接是总线状拓扑（因为共享同一条介质、会互相冲突）

吞吐量与冲突：集线器所有端口共享总带宽，任意时刻最多一个站能成功发送，因此多台主机同时发送必然冲突，需要用 CSMA/CD 来协调（这部分在数据链路层详述）。

中继器 / 集线器与交换机 / 路由器的对比：

对比项	中继器 / 集线器	网桥 / 交换机	路由器
工作层次	物理层	数据链路层	网络层
能否分割冲突域	不能	能	能
能否分割广播域	不能	不能	能
识别地址	不识别任何地址	识别 **MAC** 地址	识别 **IP** 地址
转发依据	无（广播）	交换表（自学习建立）	路由表 / 转发表
转发形式	广播（除源端口）	单播或广播（除源端口）	按路由表逐跳转发
带宽	N 台主机共享 M，每台约 M/N	N 个端口各 M，全双工，总带宽 $N \times M$	各接口独立带宽
能否互联异构网络	介质可不同，速率必须相同	直通式要求端口速率相同；存储转发式可不同	可互联异构网络

集线器三个必考结论

1. 集线器不分割冲突域，也不分割广播域。一个集线器上的所有主机在同一个冲突域里，任意两台同时发数据必然冲突。这是相对交换机最本质的区别。
2. 集线器上的主机共享总带宽。10 Mb/s 的集线器接 10 台主机，每台平均只有约 1 Mb/s；而 10 Mb/s 的交换机每个端口独享 10 Mb/s，总带宽 100 Mb/s。
3. 集线器收到数据帧会向除源端口外的所有端口转发，因此所有连接在该集线器上的其他主机都会收到该帧——这不同于交换机按 MAC 地址的单播转发。

物理层设备的真题角度

- 「集线器的拓扑结构」：物理上是星状，逻辑上是总线状。
- 「中继器连接的两个网段必须相同的是什么」：数据传输速率（不是网络层协议、不是 MAC 地址格式、不是逻辑拓扑）。
- 「10BASE-T 以太网中使用集线器，某主机发送数据时其他主机能否收到」：能收到全部，因为集线器是广播式转发。
- 「中继器/集线器能否分割冲突域或广播域」：都不能。

本章小结表

考点	核心结论 / 公式	关键记忆点
信源与信宿	信源 = 源点（产生数据），信宿 = 终点（接收数据）	发送器/接收器是变换环节，不是源/宿
通信方式	单工 1 信道；半双工 2 信道共用 1 介质；全双工 2 信道 2 介质	半双工「同一时刻只能一方发」
信号分类	模拟信号参量连续；数字信号参量取有限离散值	数字信号通常只取两个电平
带宽两义	模拟口径为频率范围（Hz）；网络口径为最高数据率（b/s）	定理里的 W 必须取 Hz
码元与比特	$k = \log_2 N$（bit/码元），N 为码元离散状态数	N 取 2 的幂
波特与速率	$R_b = R_B \log_2 N$；$R_B = R_b / \log_2 N$	$N = 2$ 时二者才相等
曼彻斯特系	波特率 = 2 × 比特率（每比特 2 个码元）	效率 0.5 bit/Hz
4B/5B	$R_B = 1.25 R_b$，开销 25%	100 Mb/s 数据 → 125 MBaud
基带 vs 频带	基带 = 数字信号直接传；频带 = 调制到载波再传	`Base` 就是基带
奈氏准则	$C_{\max} = 2W \log_2 N$（无噪声、理想低通）	限的是码元速率上限 $2W$ Baud
香农定理	$C = W \log_2(1 + S/N)$（有噪声）	与调制方式无关
dB 换算	$S/N = 10^{\mathrm{dB}/10}$；30 dB → 1000	10/20/30/40 dB → 10/100/1000/10000
两定理取舍	都给出时取 min（更紧的约束）	奈氏管码元，香农管信息
RZ / NRZ / NRZI	RZ 位中归零；NRZ 无自同步；NRZI 跳变 =0、不变 =1	NRZ 波特率 = 比特率
曼彻斯特	位中间必变，看跳变方向	IEEE 802.3：下跳 =1、上跳 =0
差分曼彻斯特	位中间必变（仅时钟），位起始处跳 =0、不跳 =1	抗干扰比曼彻斯特更强
数字调制	ASK 调幅、FSK 调频（需 2 个频率）、PSK 调相、QAM 调幅 + 调相	QAM-N 每码元 $\log_2 N$ bit
PCM 三步	采样 → 量化 → 编码；$f_s \geq 2 f_{\max}$	电话：8 kHz × 8 bit = 64 kb/s
PCM 数据率	$R_b = f_s \times \log_2 Q$	若超信道容量则取小
FDM	按频率划分，各用户同时使用，需保护频带	模拟信号复用
电路交换	建立连接 → 传输（独占、透明）→ 释放连接	时延小但利用率低，无差错检验
报文交换	存储转发整个报文，不可流水线	缓存需求大、时延大
分组交换	存储转发短分组，可流水线	利用率最高，可能失序丢失
分组交换时延	$D = (p + k - 1)T_t$；含传播 $D = (p + k - 1)T_t + kT_p$	k = 链路段数 = 路由器数 + 1
报文交换时延	$D = k \times M/R$	每跳都要收完整个报文
吞吐量	端到端取路径上各链路带宽的最小值（木桶原理）	不要脑补分流
分组数	$p = \lceil$文件大小/每分组有效载荷$\rceil$	有效载荷 = 分组长 − 首部
数据报	无连接、每分组带完整地址、独立选路、不保证顺序、故障适应强	网络只尽力而为
虚电路	面向连接、建立时选路、按 VCID 转发、保证顺序、故障适应弱	不预分配带宽

考点	核心结论 / 公式	关键记忆点
双绞线	绞合抵消干扰；UTP/STP；Cat5e 千兆、Cat6 万兆（短距）	最常用、最便宜
同轴电缆	屏蔽好、速率高、距离远；50 Ω 基带、75 Ω 宽带	75 Ω 用于有线电视
光纤	全反射原理；单模（细芯 + 激光 + 远）、多模（粗芯 +LED+ 近）	带宽大、抗干扰、保密好
无线介质	无线电波不需视线；微波/红外/激光需视线	微波直线传播要中继
接口四特性	机械（形状）、电气（电压）、功能（引脚作用）、过程（时序）	问「事件顺序」答过程特性
中继器	再生数字信号、扩大距离；不分割冲突域/广播域	放大器放大的是模拟信号
5-4-3 规则	5 段介质、4 个中继器、3 段可挂主机	10BASE5 规范
集线器	多端口中继器；共享带宽、半双工、一个冲突域	物理星状、逻辑总线状

易错清单

1. 奈氏准则限的是码元传输速率（上限 $2W$ **Baud**），不是比特率。说「奈氏准则限制了数据传输速率」不准确——它限制码元速率，比特率还要乘 $\log_2 N$。2017-34 的解题关键正是这一点。
2. 香农定理与调制方式无关。改变调制方式提高每码元比特数，在香农极限下不能提升信道容量；香农只由 W 和 S/N 决定。所以「在有噪声信道中换更高级的调制方式就能突破香农极限」是错的。
3. **dB** 换算别忘指数。$S/N = 10^{\mathrm{dB}/10}$，不是 $\mathrm{dB}/10$。30 dB 是 1000 倍不是 3 倍；40 dB 是 10000 倍。
4. **QAM** 的状态数是相位数与振幅数的乘积。「4 个相位 × 4 种振幅」是 16 种状态（4 bit），不是 8 种。2009-34 的干扰项就这么设计的。
5. 「带宽 **3 kHz**」代进公式时是 3×10^3 **Hz**。若题干给的是「链路速率 100 Mb/s」，那不是带宽，不能代进奈氏/香农。
6. 传播速度不影响数据传输速率。2014-35 的答案就是「信号传播速度」。它只影响传播时延 $T_p =$ 距离/速度。
7. 发送时延与传播时延算法不同。发送时延 = 分组长度/发送速率；传播时延 = 距离/传播速度。前者与带宽有关，后者无关。
8. 曼彻斯特编码看位中间的跳变方向，差分曼彻斯特看位起始处是否跳变。两者位中间都必变，不能只凭「位中间必变」就断定是曼彻斯特。
9. 曼彻斯特和差分曼彻斯特的波特率是比特率的 **2** 倍，编码效率只有 0.5 bit/Hz，所需带宽是 NRZ 的 2 倍。这是「为什么早期以太网只用 10 Mb/s」的原因之一。
10. **NRZI** 的规则是「跳变表示 **0**、不变表示 **1**」，与 NRZ「高电平 =1、低电平 =0」完全不同。2015-34 中若把它当成 NRZ 就会选错。
11. 归零编码（**RZ**）自同步但浪费带宽；**NRZ** 效率最高但没有自同步。这两句经常被互换来出干扰项。
12. **PCM** 采样定理比较的是最高频率分量 $f_{\max}$，不是带宽。频率范围 $0 \sim 4$ kHz 意味着 $f_{\max} = 4$ kHz，$f_s \geq 8$ kHz，而不是 ≥ 8 kHz 之外的别的数；若频率范围是 $2 \sim 10$ kHz，则 $f_{\max} = 10$ kHz，$f_s \geq 20$ kHz。
13. **PCM** 数据率超过信道容量时必须取小。「每秒采样 24k 次、每样本 3 bit」算得 72 kb/s，但 8

kHz 无噪声信道只能到 48 kb/s，答案取 48 kb/s。

14. 分组时延公式是 $(p+k-1)T_t$，不是 $p \times k \times T_t$。后者是「每个分组在每段链路上都从头算」的错误叠加，会大出约 k 倍。
15. k 是链路段数 = 路由器数 + **1**。 题目画 2 个路由器就是 3 段；画 1 个路由器就是 2 段。
16. 传播时延要按段数累加（$\times k$），不是只加一次。 2023-33 的 80.10 ms 就是 $80.08 + 2 \times 0.01$。
17. 分组数要按有效载荷算。分组长 1000 B、首部 20 B 时，每分组只带 980 B；文件 980000 B 对应 1000 个分组而不是 980 个。
18. 报文交换每跳都要收完整个报文，所以时延是 $k \times M/R$；而分组交换可以流水线，二者速率差可达数量级。
19. 电路交换的「时延小」仅指传输阶段。全过程还要加上建立连接时延；短报文场景下电路交换未必更快。
20. 虚电路不预分配带宽。虚电路是逻辑连接，多条虚电路共享链路带宽，采用统计复用；预分配带宽的是电路交换。2020-34 的错误选项正是这一句。
21. 一条虚电路不等于一条物理线路：同一物理链路上可以承载多条虚电路；两个端系统之间也可以有多条虚电路为不同进程服务。
22. **TCP/IP** 模型没有虚电路服务。虚电路服务是 OSI 参考模型的特色；IP 网络提供的是数据报服务。
23. 集线器不分割冲突域也不分割广播域。只有网桥/交换机分割冲突域，只有路由器分割广播域。
24. 集线器所有主机共享带宽：N 台主机接在带宽 M 的集线器上同时工作，每台约 M/N；而交换机每个端口独享 M。
25. 集线器的物理拓扑是星状，逻辑拓扑是总线状。两个「拓扑」不要对调。
26. 中继器连接的两个网段必须速率相同（物理层规范相同），速率可以不同是存储转发式交换机的特性，不是中继器的。
27. 中继器再生数字信号，放大器放大模拟信号。二者不通用。
28. 中继器不能无限延长网络，10BASE5 的 5-4-3 规则是「5 段介质、4 个中继器、3 段可挂主机」，不是「3 个中继器」。
29. 物理层接口四特性与「物理地址」无关。 MAC 地址属于数据链路层，不是物理层接口规范的范畴（2018-34）。
30. 单模光纤纤芯细、用激光、传得远；多模光纤纤芯粗、用 **LED**、传得近。「单模用 LED」是错的，「多模传得比单模远」也是错的。
31. 同轴电缆按阻抗分：**50** Ω 传基带数字信号，**75** Ω 传宽带信号（有线电视）。两个数字不要互换。
32. 光纤靠全反射传光，条件是光从折射率大的纤芯射向折射率小的包层且入射角大于临界角；纤芯折射率大于包层。
33. 微波、红外、激光都需要视线（直线）传播；无线电波不需要。微波远距离通信必须设中继站接力。
34. 吞吐量题用木桶原理，取路径上最小带宽；不要因为图中有并行大带宽路径就去做加法或脑补分流。
35. 「时延带宽积 = 传播时延 × 带宽」，单位是比特，不是「带宽 × 发送时延」。它常被用来反求传播时延。

第二十一章数据链路层

本章对应 2027 年 408 考纲「第四部分计算机网络 → 三、数据链路层」，是计算机网络部分分值最重、计算题最密集的一章。

本章考纲要求

（一）数据链路层的功能

（二）组帧

（三）差错控制

1. 检错编码
2. 纠错编码

（四）流量控制与可靠传输机制

1. 流量控制、可靠传输与滑动窗口机制
2. 停止-等待协议
3. 后退 N 帧协议（GBN）
4. 选择重传协议（SR）

（五）介质访问控制

1. 信道划分：频分多路复用、时分多路复用、波分多路复用、码分多路复用的概念和基本原理
2. 随机访问：ALOHA 协议，CSMA 协议，CSMA/CD 协议，CSMA/CA 协议
3. 轮询访问：令牌传递协议

（六）局域网

1. 局域网的基本概念与体系结构
2. 以太网与 IEEE 802.3
3. IEEE 802.11 无线局域网
4. VLAN 基本概念与基本原理

（七）广域网

1. 广域网的基本概念
2. PPP 协议

（八）数据链路层设备：以太网交换机及其工作原理

一、数据链路层的功能

（一）链路、数据链路与帧

链路（**Link**）又称物理链路，是指从一个结点到相邻结点的一段物理线路（有线或无线），中间没有其他任何交换结点。网络中各主机之间的通信路径一般由多段链路构成。

数据链路（**Data Link**）又称逻辑链路，是在链路的基础上加上实现通信协议的硬件（网络适配器，俗称网卡）和软件（驱动程序）所构成的。也就是说：

$$\text{数据链路} = \text{物理链路} + \text{实现数据链路层协议的硬件和软件}$$

帧（**Frame**）是数据链路层对等实体之间在水平方向进行逻辑通信的协议数据单元 **PDU**。发送方的数据链路层给网络层交付下来的分组添加首部和尾部，封装成帧后交付物理层发送；接收方的数据链路层从物理层交付上来的比特流中解封出分组，再上交给网络层。

从数据链路层看，主机 H1 到 H2 的通信可以看成是由若干段链路上的通信拼接而成的。相邻两段链路可能采取不同的数据链路层协议，这正是” 逐段链路、逐跳处理” 的含义。

链路 ≠ 数据链路

考试常在这里设陷阱：问” 链路” 就是纯物理线路，问” 数据链路” 必须包含协议软硬件。另外” 物理地址”（MAC 地址）虽带” 物理” 二字，却属于数据链路层范畴，不是物理层。物理层是” 管道的存在”，数据链路层是” 管道里的交通规则”。

（二）数据链路层的功能

参考教材与考纲，数据链路层的功能可归纳为五条：

- 为网络层提供服务：主要有无确认的无连接服务、有确认的无连接服务、有确认的面向连接服务三类。存在确认就必须存在连接（2013 年考过）。
- 链路管理：连接的建立、维持和释放（面向连接服务才需要）。
- 组帧（封装成帧）：把网络层的分组封装成帧，解决帧定界问题。
- 差错控制：检测传输过程中产生的比特差错，决定丢弃或重传。
- 流量控制与可靠传输：协调收发双方的发送/接收速率，必要时通过确认与重传实现可靠传输。

一道常见选择题

” 在 OSI 参考模型中，数据链路层为网络层提供服务，下列哪种说法是错误的？” A. 可以提供无确认的无连接服务　B. 可以提供有确认的面向连接服务 C. 必须提供可靠的、无差错的数据传输　D. 服务质量取决于具体的链路层协议答案选 C。是否存在可靠传输服务由具体协议决定：以太网（IEEE 802.3）提供不可靠服务，802.11 无线局域网提供可靠服务。

（三）封装成帧、透明传输与差错检测

封装成帧就是给网络层交付下来的分组添加首部和尾部。帧首部通常含帧开始符、源地址、目的地址；帧尾部通常含帧校验序列和帧结束符。接收方据此在比特流中识别出帧的开始与结束，叫帧定界。

为提高传输效率，应使数据载荷远大于首部和尾部；但数据载荷长度必须有上限，这个上限叫最大传送单元（**Maximum Transfer Unit**，**MTU**），以太网的 MTU 为 1500 字节。

透明传输：如果数据载荷中恰好出现了帧定界符，接收方就会错误地提前结束帧定界。透明传输指数据链路层对上层交付的 **PDU** 内容没有任何限制，就好像数据链路层不存在一样。实现透明传输

的两种方法是字节填充（字符填充）和比特填充。

差错检测：见下一节。

二、组帧

组帧（Framing）要解决的核心问题是：接收方如何从连续的比特流中找出帧的边界。四种常见方法如下。

（一）字符计数法

在每个帧的帧首部中用一个计数字段指出本帧的总字符数（字节数）。接收方先读出计数字段，就知道后面还有多少个字符，从而定出帧尾。

缺点：只要计数字段在传输中出错，帧的边界就会丢失，接收方会把后续内容全部错位解析，错误无法恢复。因此实际协议中很少单独使用。

（二）字符填充法

面向字节的物理链路使用字符填充法（也叫字节填充法）实现透明传输。做法是：

- 在帧首部使用 SOH（0x01，帧开始符），帧尾部使用 EOT（0x04，帧结束符）。
- 发送方扫描数据载荷，每遇到一个 SOH 或 EOT，就在其前面插入一个转义字符 ESC（**0x1B**）。
- 如果数据载荷中本来就有 ESC，则在 ESC 前面再插入一个 ESC。
- 接收方做逆变换：遇到 ESC 就删除它，并把紧随其后的一个字符当作普通数据处理。这样连续两个 ESC 就被还原成数据中的单个 ESC。

完整算例：设数据载荷为 A ESC B EOT C，SOH 为帧首部、EOT 为帧尾部、ESC 为转义字符，则实际在线路上传输的字符序列为：

```
SOH  A  ESC ESC  B  ESC EOT  C  EOT
```

逐项说明：数据中的 ESC 被填成 ESC ESC；数据中的 EOT 被填成 ESC EOT；数据中的 A、B、C 是普通字符，不填充；最后的 EOT 是真正的帧结束符，不属于数据载荷。

填充的判定顺序

接收方必须一遍扫描：先看 ESC，再看下一个字符。如果扫描逻辑写成”看到 EOT 就结束帧”，那么数据中的 ESC EOT 会被误判为帧尾。所以规则只能是”遇到 ESC 就跳过并原样收下后一个字符”。

（三）零比特填充法

面向比特的物理链路使用零比特填充法实现透明传输。以高级数据链路控制协议 **HDLC** 为例：

- HDLC 的帧定界标志字段为 01111110（即 0x7E，6 个连续 1）。
- 发送方：扫描数据载荷，只要出现 **5** 个连续的比特 **1**，就在其后插入一个比特 **0**。
- 接收方：扫描数据载荷，每发现 **5** 个连续的比特 **1**，就把它后面的一个比特 **0** 删除。

这样填充后，数据载荷中绝不会出现 6 个连续 1，也就绝不会出现与标志字段相同的比特组合，从而实现透明传输。

完整算例（**2013** 年统考真题改编）：HDLC 协议对 0111110001111110 组帧后对应的比特串为。

```
原始比特串：0 1 1 1 1 1 0 0 0 1 1 1 1 1 1 0
                          ↑                 ↑
第 5 个 1 之后插 0        插 0              插 0
```

逐段处理：

- 前 6 位 011111 中已有 5 个连续 1，在 0 前插入一个 0，得到 0111110；
- 中间 000 原样保留；
- 后段 0111111 中先出现 5 个连续 1，插入一个 0 变成 01111101，剩下的 1 继续保留；
- 最终 0 保留。

结果为 0111110 000 0111110 10，即 **01111100001111101 0**。

2013 年题 37（HDLC 零比特填充）

答案即为上述 01111100001111101 0。做这一类题的关键是从左到右扫描、计数器数连续的 **1**、到 **5** 就插 **0** 并把计数器清零。注意接收方解码时” 删 0” 后，删除操作本身不会让计数器继续累加，所以 0111110 只能解码出 011111（5 个 1），不会误删数据中的 0。

（四）违规编码法

在物理层编码时，使用不会出现在正常编码中的电平（违规编码）来表示帧的开始和结束。例如曼彻斯特编码中，每个码元中间必有跳变；若在应该跳变的位置不跳变，就是违规编码，可以用来做帧定界。

特点：不需要额外插入任何比特或字节，开销最小，但要求物理层编码本身有冗余的非法码字，通用性差。局域网的 IEEE 802.3 以太网和 802.11 无线局域网均只采用违规编码法（本质上通过前导码 + 帧间间隔来定界），不使用字符填充也不使用零比特填充。

四种组帧方法的对比

方法	适用链路	额外开销	主要缺点
字符计数法	面向字节	1 个计数字段	计数出错则彻底失步，无法恢复
字符填充法	面向字节	若干 ESC	数据中转义字符多时开销大
零比特填充法	面向比特	每 5 个 1 插 1 个 0	需硬件扫描，效率随数据模式波动
违规编码法	依赖物理层编码	几乎为 0	要求物理层有非法码字，通用性差

三、差错控制

（一）差错与误码率

实际的通信链路都不是理想的，比特在传输过程中可能产生比特差错（误码）：比特 1 变成 0，或 0 变成 1。原因包括噪声干扰、传输媒体质量等。一段时间内传输错误的比特数占传输比特总数的比率称为误码率（**Bit Error Rate**，**BER**）。

提高链路的信噪比可以降低误码率，但不可能使误码率降为零，因此需要用差错检测技术判断传输中是否出现了比特差错。

差错控制分为两类：检错编码（只发现错误，不能定位和纠正）和纠错编码（既能发现又能定位并纠正）。

（二）检错编码

*1.*奇偶校验码

奇校验：在待发送的数据后面添加 1 个校验位，使得添加后整个码字中比特 **1** 的个数为奇数。偶校验：添加 1 个校验位，使得添加后整个码字中比特 **1** 的个数为偶数。

设原始数据 $M=(M_{n-1}M_{n-2}\cdots M_0)$，$P$ 为校验位，则：

$$P_{奇}=M_{n-1}\oplus M_{n-2}\oplus\cdots\oplus M_0\oplus 1 \qquad P_{偶}=M_{n-1}\oplus M_{n-2}\oplus\cdots\oplus M_0$$

检错能力：如果传输中有奇数个比特发生误码，则码字中 1 的个数的奇偶性改变，可以检测出误码；如果有偶数个比特发生误码，奇偶性不变，无法检测出（漏检）。

奇偶校验的局限（必背结论）

- 只能检测奇数个比特错误，不能检测偶数个比特错误；
- 能发现错误，不能定位错误，更不能纠错；
- 编码效率高（只增加 1 bit），但检错能力弱，只适用于误码率很低的场合；
- 用奇偶校验能否检出错误，取决于改变的比特数量是奇数还是偶数，与具体是哪几位改变、改变成什么值无关。

完整算例：待发送数据块 $M=1010110$，采用偶校验，校验位附加在数据块末尾。

M 中 1 的个数为 4（偶数），要使整个码字 1 的个数仍为偶数，校验位 $P=0$，故实际发送的比特序列为 `10101100`。

若某次传输后接收方收到 `10101110`，其中 1 的个数为 5（奇数），可判定发生了差错。但若收到的是 `10001110`（同样 5 个 1，但两侧各错 1 位变成了 2 位错误）——实际上 `10101100` $\rightarrow$ `10001110` 改变了 3 位，仍能被检出；而 `10101100` $\rightarrow$ `00101110` 改变了 2 位，1 的个数仍为 4，漏检。

*2.*循环冗余校验 *CRC*

数据链路层广泛使用漏检率极低的循环冗余校验（**Cyclic Redundancy Check**，**CRC**）。

基本原理：

- 收发双方约定一个 $(k-1)$ 阶生成多项式 $G(X)$（最高次为 $k-1$，对应比特串共 k 位）。
- 发送方基于待发数据 M 和 $G(X)$ 计算出 $k-1$ 位的冗余码（帧检验序列 **FCS**），附加在 M 后面一起发送。
- 接收方用同一 $G(X)$ 对收到的 $M'+P'$ 做模 2 除法，余数为 **0** 判为无差错，余数不为 **0** 判为有差错。

发送方 **CRC** 操作的四步流程：

1. 构造被除数：在待发送数据 M 后面添加 $k-1$ 个 **0**（$k-1$ 即 $G(X)$ 的最高次数）。
2. 构造除数：$G(X)$ 各项系数构成的比特串（共 k 位）作为除数。
3. 模 **2** 除法：做 k 位一组的上商除法。模 **2** 除法既不借位也不比较大小，每一位的运算就是异或。求得的余数位数必须与 $G(X)$ 的最高次数相同，位数不够时在前面补 **0**。
4. 发送：把余数（FCS）附加到 M 后面一起发送。

接收方 **CRC** 操作：对收到的整个比特串（数据 + FCS）做同样的模 2 除法，余数为 0 认为无差错，余数不为 0 认为有差错，直接丢弃该帧。

完整手算算例一（发送方）：待发送数据 $M = 101001$，生成多项式 $G(X) = X^3 + X^2 + 1$，求冗余码。

第 1 步，$G(X)$ 最高次为 3，故在 M 后补 3 个 0，被除数为 `101001000`。第 2 步，$G(X) = X^3 + X^2 + X^0$，各项系数为 `1 1 0 1`，除数 $= 1101$。第 3 步，模 2 除法竖式：

```
          1 1 0 0 1 1
      ┌─────────────────────────────────
1101  │ 1 0 1 0 0 1 0 0 0
      1 1 0 1
      ─────────────────
        1 1 1 0
        1 1 0 1
        ─────────────────
            1 1 1 0
            1 1 0 1
            ─────────────────
                1 1 0 0
                1 1 0 1
                ─────────────────
                    0 0 1   <- 余数
```

逐步说明（每次取够 4 位与 1101 异或）：

- 首位为 1，商 1，`1010 ⊕ 1101 = 0111`，落下一位得 `1110`；
- 首位为 1，商 1，`1110 ⊕ 1101 = 0011`，落下一位得 `0110`（首位为 0，商 0，直接落下一位）得 `1100`；
- 首位为 1，商 1，`1100 ⊕ 1101 = 0001`，落下一位得 `0010`（首位 0 商 0），再落下一位得 `0100`（首位 0 商 0），再落下一位得 `1000`；
- 末次首位为 1，商 1，`1000 ⊕ 1101 = 0101`，继续得到最终余数 `001`。

第 4 步，余数 `001` 只有 3 位，正好等于 $G(X)$ 的最高次数 3，不需要补 **0**。故 FCS = `001`。

实际发送的比特串为 M 拼上 **FCS**：`101001 001`，即 `101001001`。

完整手算算例二（接收方）：

- 若接收方收到 `101001001`，用它除以 `1101`：

```
          1 1 0 0 1 1
      ┌─────────────────────────────────
1101  │ 1 0 1 0 0 1 0 0 1
      1 1 0 1
      ─────────────────
        1 1 1 0
        1 1 0 1
        ─────────────────
            1 1 1 0
            1 1 0 1
            ─────────────────
                1 1 0 0
                1 1 0 1
```

```
                  ----------------
                    0 0 0 0   -> 余数 000
```

余数为 000，判定传输过程没有产生误码。

- 若接收方收到 101101011（其中两位发生跳变），同样除以 1101，最终余数为 001（不为 0），判定传输过程产生了误码，丢弃该帧。

完整算例三（含补 **0** 的检验情形）：某数据链路层使用 CRC 校验，待发送原始数据帧为 110101，生成多项式 $G(X)=X^4+X+1$（即 10011），求 CRC 校验码。

$G(X)$ 最高次为 4，被除数为 110101 + 0000 = 1101010000，除数 10011。模 2 除法：

```
            1 1 0 0 0 1
          ┌──────────────────────────────
10011     │ 1 1 0 1 0 1 0 0 0 0
          1 0 0 1 1
          ----------------------
            1 0 0 0 1
            1 0 0 1 1
            ----------------------
                0 0 0 0 0
                0 0 0 0 0
                ----------------------
                      0 1 0 0 0
                      0 0 0 0 0
                      ----------------
                        1 0 0 0 0
                        1 0 0 1 1
                        ----------
                              0 1 1   -> 先得部分余数，继续补齐
```

继续做完，余数为 0000，**CRC** 校验码为 0000。

2023 年题 37（CRC 检错）

生成多项式 $G(X)=X^4+X+1$（即 10011），判断下列哪个接收比特串可断定未发生错误：A. 101110000　B. 101110100　C. 101111000　D. 101111100

做法：把每个选项分别除以 10011，只有余数为 **0000** 的那个才无差错。计算得：101110000 ÷ 10011 余 1100；101110100 余 1000；101111000 余 0100；**101111100** 余 0000。故答案为 **D**。

关于 CRC 与 FCS 的必背结论

- **FCS** 是附加在数据后面的冗余码，**CRC** 是获得 **FCS** 的一种常用方法，但 **CRC** 不是获得 **FCS** 的唯一方法，两者不能等同。
- CRC 检错只能判断” 有无差错”，不能确定是哪一位或哪几位出错，因此 **CRC** 只能检错不能纠错（虽然 CRC 理论上具备一定纠错潜力，但实际不使用）。
- 只要生成多项式选得合适且位数足够多，检测不到差错的概率可以做到极小。
- 仅用 CRC 只能做到无差错接收（**accept**）：凡是接收端接受的帧，都能以非常接近于 1 的概率认为没有产生差错；要做到**”** 可靠传输**”**（发送什么就收到什么），必须再加上序号、确认和重传机制。
- $(k-1)$ 阶的生成多项式能检测出：全部单比特错、全部奇数位错（当 $G(X)$ 含 $(X+1)$ 因子时）、长度不超过 $k-1$ 的全部突发错。

*3.*校验和

校验和（**Checksum**）的通用做法是：把数据划分为若干个固定的位组（如 16 位一组），发送方对各组做反码求和（累加时若有进位则回卷加到最低位），再对结果取反码，作为校验和字段随数据一起发送。接收方把所有分组连同校验和一起按同样方式求和再取反，结果为全 0 则认为无差错。

校验和的特点：算法简单、软硬件开销小，但检错能力弱于 **CRC**。它是网络层（IP 首部校验和）和运输层（TCP、UDP 校验和）使用的检错方法；数据链路层主要使用 **CRC**。

校验和的范围

TCP/UDP 的校验和计算要包括伪首部，而 **IP** 首部校验和只校验 **IP** 首部，不包含伪首部、也不校验数据部分。这是网络层/运输层章节的常考点，不要与数据链路层的 CRC 混淆。

（三）纠错编码：海明码

海明码（**Hamming Code**）是最经典的纠错编码，它通过增加若干校验位，使得接收方不但能发现差错，还能定位并纠正哪怕 1 位差错。

*1.*校验位位数 k 的确定与推导

设有 m 位数据位、k 位校验位，码字总长 $n=m+k$。

推导思路：接收方用 k 个校验位可以产生 k 个校验结果（称为校验因子，其组合即指误字/伴随式）。这 k 个二进制位最多能表示 2^k 种不同的状态。这 2^k 种状态必须覆盖”没有出错”（1 种）以及”n 位中任意 **1** 位出错”（n 种）的全部情况，因此必须满足：

$$2^k \geq n+1=m+k+1$$

取值表（考试常直接查用，记住 $m=4$ 时 $k=3$ 即可）：

数据位 m	**1**	**2~4**	**5~11**	**12~26**
校验位 k	2	3	4	5
码字总长 n	3	5~7	9~15	17~31

例如 $m=4$：$2^k \geq 4+k+1$。取 $k=2$ 时 $4 \geq 7$ 不成立；取 $k=3$ 时 $8 \geq 8$ 成立，故 $k=3$，总长 $n=7$。这就是众所周知的海明码 **(7,4)**。

*2.*校验位的位置

校验位必须放在码字中序号为 $2^0, 2^1, 2^2, \ldots, 2^{k-1}$（即 **1**、**2**、**4**、**8**、…）的位置上，其余位置放数据位。总长 $n=7$ 时：

码字位序号	**1**	**2**	**3**	**4**	**5**	**6**	**7**
内容	c_1	c_2	d_1	c_4	d_2	d_3	d_4

为什么校验位放在 2 的幂位置？因为每个码字位序号都可以唯一地分解为若干 2 的幂之和，例如 $5=1+4$、$6=2+4$、$7=1+2+4$。让 c_1 负责所有序号含 2^0 的位、c_2 负责所有序号含 2^1 的位、c_4 负责所有序号含 2^2 的位，就能保证任意 **1** 位出错时，受影响的校验位组合唯一，从而由校验结果反推出出错位置。

具体分组（偶校验分组）为：

$$c_1 = d_1 \oplus d_2 \oplus d_4 \quad (\text{覆盖位} 1,3,5,7)$$

$$c_2 = d_1 \oplus d_3 \oplus d_4 \quad (\text{覆盖位} 2,3,6,7)$$

$$c_4 = d_2 \oplus d_3 \oplus d_4 \quad (\text{覆盖位} 4,5,6,7)$$

校验位的取值规则：让每一组中"1 的个数" 为偶数（若采用奇校验则让每组中 1 的个数为奇数）。等价地，c_i 等于它覆盖的所有数据位的异或值。

*3.*编码完整算例

设待发送数据 $D = d_1d_2d_3d_4 = 1101$。

按位置表填入：$d_1 = 1$（第 3 位）、$d_2 = 1$（第 5 位）、$d_3 = 0$（第 6 位）、$d_4 = 1$（第 7 位）。计算三个校验位：

$$c_1 = d_1 \oplus d_2 \oplus d_4 = 1 \oplus 1 \oplus 1 = 1$$

$$c_2 = d_1 \oplus d_3 \oplus d_4 = 1 \oplus 0 \oplus 1 = 0$$

$$c_4 = d_2 \oplus d_3 \oplus d_4 = 1 \oplus 0 \oplus 1 = 0$$

最终海明码为：

位序号	**1**	**2**	**3**	**4**	**5**	**6**	**7**
位值	$c_1 = 1$	$c_2 = 0$	$d_1 = 1$	$c_4 = 0$	$d_2 = 1$	$d_3 = 0$	$d_4 = 1$

即海明码为 `1010101`。

自检：$(1,3,5,7)$ 位上 1 的个数：$1,1,1,1$ 共 4 个，偶数 ✓；$(2,3,6,7)$ 位上 1 的个数：$0,1,0,1$ 共 2 个，偶数 ✓；$(4,5,6,7)$ 位上 1 的个数：$0,1,0,1$ 共 2 个，偶数 ✓。

*4.*接收方的检错与纠错完整算例

接收方对收到的 7 位码字按同样的三组重新计算校验结果，得到三个校验因子 G_1, G_2, G_4，把它们按 $G_4G_2G_1$ 拼成指误字：

$$G_1 = \bigoplus(1,3,5,7)\text{ 位} \qquad G_2 = \bigoplus(2,3,6,7)\text{ 位} \qquad G_4 = \bigoplus(4,5,6,7)\text{ 位}$$

- 指误字 $= 000$：无差错。
- 指误字 $\neq 000$：其十进制值就是出错位的序号，把它取反即完成纠错。

情形一：接收方收到 1010101（正确）。

$G_1 = 1 \oplus 1 \oplus 1 \oplus 1 = 0$；$G_2 = 0 \oplus 1 \oplus 0 \oplus 1 = 0$；$G_4 = 0 \oplus 1 \oplus 0 \oplus 1 = 0$。指误字 000，判为无差错，取出第 3、5、6、7 位得数据 1101。

情形二：第 **5** 位在传输中发生跳变，接收方收到 1010001（原 1010101，第 5 位由 1 变 0）。

$G_1 = 1 \oplus 1 \oplus \underline{0} \oplus 1 = 1$；$G_2 = 0 \oplus 1 \oplus 0 \oplus 1 = 0$；$G_4 = 0 \oplus \underline{0} \oplus 0 \oplus 1 = 1$。

指误字 $= G_4G_2G_1 = 101$，十进制为 **5**，即第 **5** 位出错。把第 5 位取反（0 改为 1），得 1010101，取出数据位得 1101，纠错成功。

情形三：第 **3** 位在传输中发生跳变，接收方收到 1000101（第 3 位由 1 变 0）。

$G_1 = 1 \oplus \underline{0} \oplus 1 \oplus 1 = 1$；$G_2 = 0 \oplus \underline{0} \oplus 0 \oplus 1 = 1$；$G_4 = 0 \oplus 1 \oplus 0 \oplus 1 = 0$。指误字 $= 011$，十进制为 **3**，第 3 位出错，取反后恢复 1010101 ✓。

情形四：第 **7** 位出错，接收方收到 1010100。

$G_1 = 1 \oplus 1 \oplus 1 \oplus \underline{0} = 1$；$G_2 = 0 \oplus 1 \oplus 0 \oplus \underline{0} = 1$；$G_4 = 0 \oplus 1 \oplus 0 \oplus \underline{0} = 1$。指误字 $= 111$，十进制为 **7**，第 7 位出错，取反恢复 ✓。

海明码的边界：两位错会” 纠错纠错”

海明码只能纠正 **1** 位错。若发生了 2 位错，指误字虽然非 0（能发现” 有错”），但它指向的位置并不是真正出错的位置，此时执行” 纠错” 反而会把第 3 位也改错，越纠越错。另外要注意：n 位码字可以出现 n 种单比特错 + **1** 种无错，共 $n+1$ 种情况，而 k 个校验位只能给出 2^k 种状态，这就是 $2^k \geq n+1$ 的另一种解释。

*5.*海明距离与检错纠错能力

海明距离（汉明距离）：两个合法码字之间对应位取值不同的位数。例如 1101 与 1001 海明距离为 1。

一个编码方案的最小海明距离 d 定义为所有合法码字两两之间海明距离的最小值。设：

- e：要检测的差错位数；
- t：要纠正的差错位数；
- s：在纠正 t 位错的同时还要检测的差错位数。

则有：

$$d \geq e+1 \qquad d \geq 2t+1 \qquad d \geq t+s+1\ (s>t)$$

含义解释：

- $d \geq e+1$：检错不需要定位。要让 e 位错之后不可能变成另一个合法码字，两个合法码字之间至少要差 $e+1$ 位。
- $d \geq 2t+1$：纠错要定位。以每个合法码字为球心、半径 t 画” 错球”，所有错球必须互不相交，即两球心距离至少为 $2t+1$。
- $d \geq t+s+1$：既要纠正 t 位又要检测 s 位，两个条件叠加。

例如：要求检测 2 位错，则 $d \geq 3$；要求纠正 1 位错，则 $d \geq 3$；要求纠正 1 位错同时检测 2 位错，则 $d \geq 1+2+1 = 4$。

编码能力选择题

” 如果一个编码方案被设计用来检测 e 位错误、纠正 t 位错误、同时检测 s 位错误且纠正 t 位错误，其最

小海明距离 d 分别不小于（ ）”标准答案：$e+1$、$2t+1$、$t+s+1$。记口诀：检错 **+1**，纠错 **2** 倍 **+1**，检纠结合相加 **+1**。

四、流量控制与可靠传输机制

（一）流量控制、可靠传输与滑动窗口机制

流量控制：控制发送方的发送速率，使接收方来得及接收（接收方的缓冲区不会溢出）。

可靠传输：发送方发送什么，接收方最终就能收到什么，即做到不丢失、不出错、不重复、不乱序。

传输差错包括四类：比特差错（误码）、分组丢失、分组失序、分组重复。其中分组丢失、失序、重复一般不在数据链路层出现，而是在其上层出现。因此可靠传输服务并不局限于数据链路层，其上各层均可选择实现。

- 有线链路误码率低，不要求数据链路层提供可靠传输服务（如以太网）；即使出错，也由上层的 TCP 负责。
- 无线链路误码率高，要求数据链路层必须提供可靠传输服务（如 802.11 无线局域网）。

*1.*停等协议与滑动窗口的关系

停止-等待协议是发送窗口 $W_T=1$、接收窗口 $W_R=1$ 的滑动窗口协议的特例。也就是说，滑动窗口机制是更一般的框架，三种可靠传输协议都可以用”窗口 + 序号 + 确认 + 重传”统一描述。

发送窗口：发送方维护的一个序号区间，只有序号落在发送窗口内的帧才允许被发送（已发送但未收到确认的帧仍占窗口）。接收窗口：接收方维护的一个序号区间，只有序号落在接收窗口内且无差错的帧才被接收。

三条核心规律：

1. 接收窗口向前滑动，一定伴随着确认报文的发送（告知发送方我已收好）。
2. 发送窗口向前滑动，一定是因为收到了按序到达的确认。
3. 发送窗口内的序号都是**”**已发送但未被确认**”**或**”**允许发送**”**的；接收窗口内的序号都是**”**允许接收**”**的。

*2.*窗口大小与序号位数的关系 $W_T+W_R\le 2^n$

设用 n 个比特给帧编号，则序号空间大小为 2^n（序号 $0\sim 2^n-1$，循环使用）。为了避免接收方把重传的旧帧误判为新帧，必须满足：

$$W_T+W_R\le 2^n$$

推导（反证法）：设 $W_T+W_R>2^n$。

发送方发完窗口内 W_T 个帧后，若所有确认帧全部丢失，发送方超时后必须重传这 W_T 个帧。此时发送方已经把窗口滑到 W_T+1 开始的 W_T 个新序号上。由于 $W_T+W_R>2^n$，发送方”新窗口的序号集合”与接收方”当前接收窗口的序号集合”必然有交集。

接收方无法区分是旧帧的重传还是新帧的首次到达（因为序号相同而窗口位置相同），于是可能把重传的旧帧当作新帧交付给上层，造成重复接收 / 数据错乱。所以必须 $W_T+W_R\le 2^n$。

这一约束的直接后果：

- 停止-等待协议：$W_T=1$，故 $W_R\le 2^n-1$；实际取 $W_R=1$，$n=1$ 即可（序号 0、1）。

- 后退 **N** 帧协议 **GBN**：$W_R = 1$，故 $W_T \leq 2^n - 1$。
- 选择重传协议 **SR**：一般取 $W_T = W_R$，故 $W_T = W_R \leq 2^{n-1}$。

三个窗口上限（必背）

协议	W_T	W_R	$n=3$ 时的最大值
停止-等待	1	1	$W_T = W_R = 1$
GBN	$1 < W_T \leq 2^n - 1$	1	$W_T = 7$，$W_R = 1$
SR	$W_T = W_R \leq 2^{n-1}$	同左	$W_T = W_R = 4$

2019 年题 35

”对于滑动窗口协议，如果分组序号采用 3 比特编号，发送窗口大小为 5，则接收窗口最大是（　）”由 $W_T + W_R \leq 2^3 = 8$，得 $W_R \leq 3$，答案为 **3**。

（二）停止-等待协议

定义：发送方每发送完一个数据分组就必须停下来，等待接收方发来的确认（ACK）或否认（NAK）分组，收到确认后才能发送下一个。

一次完整的通信包含三种机制：确认/否认、超时重传、给数据分组编号。

*1.*四种情况的图解说明

情况一：正常情况（无差错、无丢失）发送方发送 DATA0 → 接收方收到后回 ACK0 → 发送方收到 ACK0 后发送 DATA1 → 接收方回 ACK1 → ⋯如此往复。此时信道利用率为 $U = \frac{T_D}{T_D + RTT + T_A}$。

情况二：数据分组丢失或出错 → 超时重传发送方发送 DATA0 后即启动超时计时器（**Timeout Timer**），并设置超时重传时间（**RTO**）。若 DATA0 在传输中丢失：

```
发送方                接收方
  |---- DATA0 ----X      |          (DATA0 丢失，接收方什么都没收到，不会发 ACK)
  |  启动计时器          |
  |    ... 超时...       |
  |---- DATA0 ----  -->  |          重传
  |<---- ACK0 -----      |
```

要点：接收方收不到数据就不会发 ACK，发送方若不加措施将永远等待，所以必须有超时计时器。RTO 一般设置为略大于收发双方的平均往返时间 **RTT**。数据链路层点对点 RTT 比较固定，RTO 好设定；运输层端到端 RTT 不确定，RTO 难设定。

情况三：确认分组丢失 → 超时后重传，接收方必须能识别重复帧发送方发出的 DATA0 已被正确接收，接收方回送的 ACK0 在途中丢失：

```
发送方                接收方
  |---- DATA0 ---->  |  正确接收 DATA0，交付上层
  |<--- ACK0 ---X    |  ACK0 丢失
  |  超时            |
  |---- DATA0 ---->  |  又收到一个 DATA0（重复帧！）
  |<---- ACK0 -----  |  丢弃重复帧，但仍需回 ACK0
```

必须给数据分组编号：停止-等待协议只需 **1** 个比特编号（序号 0 和 1），因为只要保证相邻两次发送的序号不同即可。接收方发现收到的序号与上次相同，就判定是重复帧并丢弃，同时重发一次确认（否则发送方会一直超时）。

情况四：确认分组迟到（确认迟到）发送方因超时而重传了 DATA0，接收方再次回送 ACK0；随后原来那个迟到的 **ACK0** 才到达发送方。发送方收到两个 ACK0：

```
发送方                接收方
  |---- DATA0 ---->  |
  |  超时              |
  |---- DATA0 ---->  |  收下并丢弃重复，回 ACK0
  |<--- ACK0(迟到的)-|  <- 第一个 ACK0 迟到
  |<--- ACK0(新的) --|
```

必须给确认分组编号：发送方要能判断收到的确认是不是重复的，所需比特数与数据分组相同。但在数据链路层一般不会出现确认迟到的情况，因此在数据链路层实现停止-等待协议可以不给确认分组编号；在运输层则必须编号。

停止-等待协议的四个易错点

1. 使用超时重传后可以不用否认机制（**NAK**），协议实现更简单；但若误码率较高，使用 NAK 可以让发送方在超时前就尽快重传。
2. 停止-等待协议属于自动请求重传（**Automatic Repeat reQuest，ARQ**）：重传请求是发送方自动发起的，不是接收方要求发送方重传某个具体误码帧（否则就成了” 选择重传”）。
3. ” 确认分组迟到” 和” 确认分组丢失” 是两种不同的情形，前者要求对确认编号，后者要求对数据编号。
4. RTO 必须大于 RTT，否则会大量无谓重传。

*2.*信道利用率计算

设：

- T_D：发送一个数据帧所需的时间（发送时延）$=\dfrac{L}{R}$，L 为帧长，R 为数据传输速率；
- RTT：往返时间（$=2\times$ 单向传播时延 τ，若题目给出链路两端距离与信号传播速度则 $RTT=2d/v$）；
- T_A：发送一个确认帧所需的时间（题目若说” 忽略确认帧的传输时延” 则 $T_A=0$）。

发送周期 = 从开始发送第一个数据帧的第一个比特起，到收到该帧的第一个确认帧为止所经历的时间：

$$T_{\text{周期}}=T_D+RTT+T_A$$

信道利用率：

$$U=\frac{T_D}{T_D+RTT+T_A}\times 100\%$$

含义：分子是” 真正用于传送有用数据的时间”，分母是整个发送周期。U 就是信道被有效数据占用的比例。

若出现超时重传，则有效数据的信道利用率还要降低。在 **RTT** 相对较大的情况下（如卫星链路），停等协议效率极低，应改用 GBN 或 SR 协议。

2018 年题 36（停等协议信道利用率）

主机甲采用停等协议向主机乙发送数据，数据传输速率 $R=3$ kbps，单向传播时延 $\tau=200$ ms，忽略确认帧的传输时延。当信道利用率 $U=40\%$ 时，数据帧的长度为多少?

$RTT=2\tau=400$ ms $=0.4$ s，$T_A=0$。设帧长为 L bit，则 $T_D=L/3000$ s。

$$0.4=\frac{L/3000}{L/3000+0.4}\Rightarrow 0.4\times\frac{L}{3000}+0.16=\frac{L}{3000}\Rightarrow 0.6\times\frac{L}{3000}=0.16\Rightarrow L=800\text{ bit}$$

答案 **800** 比特。

（三）后退 N 帧协议（GBN）

基本思想：采用流水线传输方式，允许发送方在未收到确认的情况下连续发送多个帧，用发送窗口限制连续发送的数量，属于连续 **ARQ** 协议。

工作规则：

- 接收窗口 $W_R=1$，接收方只能按序接收。若收到一个失序的帧（例如期待 n 却收到 $n+1$），就丢弃该帧，并为最后一个按序正确收到的帧发送确认。
  - 例如接收方已正确收到 `0,1,2` 号帧，随后收到 `4` 号帧：由于 `3` 号帧丢失/出错，接收方丢弃 `4` 号帧，并重复发送 `ACK2`。
- 发送方采用累积确认：ACKn 表示”序号 n 及之前的所有帧都已正确接收”。
- 发送方超时后，重新发送从出错帧开始的全部已发送帧（这就是”后退 N 帧”）。

累积确认的优点：减少注入网络的确认数量；即使确认丢失也可能不必重传数据。累积确认的缺点：不能及时准确地把接收方已正确接收的帧数告诉发送方。

*1.*为什么 $W_T\le 2^n-1$

若 $W_T=2^n$（即取满整个序号空间），则发送方发完 $0\sim 2^n-1$ 全部帧后，接收方已正确收下全部帧并把接收窗口滑到序号 0。此时若所有确认帧丢失，发送方超时重传 `0` 号帧。

接收方的接收窗口此刻正好停在序号 0，无法区分这个 `0` 号帧是重传的旧帧还是新一批的新帧，会把它当作新帧交付给上层，造成重复。因此 W_T 最多取 2^n-1。

GBN 窗口上限

采用 n 比特编号时，$1<W_T\le 2^n-1$，$W_R=1$。$n=3$ 时 W_T 取值范围为 $2\sim 7$。

*2.*发送窗口滑动过程的手工模拟

设 $n=3$（序号 $0\sim 7$），$W_T=5$，当前发送窗口覆盖序号 $\{0,1,2,3,4\}$。

第 **1** 步：发送方连续发送 `0,1,2,3,4` 五个帧，窗口内全部变为”已发送待确认”，发送窗口不动。

第 **2** 步：接收方正确收到 `0,1,2`，返回累积确认 `ACK2`。发送方收到 `ACK2`，说明 `0,1,2` 已被确认，发送窗口向前滑动 **3** 个位置，变为 $\{3,4,5,6,7\}$。

第 **3** 步：发送方可以继续发送新落入窗口的 `5,6,7`。

第 **4** 步：假设此时 `3` 号帧在途中丢失，接收方收到 `4` 号帧时发现失序（期待 `3`），丢弃 `4` 号帧并重复发送 `ACK2`。

第 **5** 步：发送方超时（`3` 号帧的计时器到期），重传 **`3,4,5,6,7`** 全部已发送但未确认的帧。注意此时哪怕 `4,5,6,7` 早已正确到达过接收方，它们也要被重传——这就是 GBN”一个帧出错引起大量重传”的代价。

第 **6** 步：接收方按序收下 3,4,5,6,7，回 ACK7。发送窗口向前滑动 5 个位置，变为 $\{0,1,2,3,4\}$（序号回绕）。

2009 年题 35（GBN 累积确认与重传数量）

”数据链路层使用 GBN 协议，发送方已经发送了编号 0~7 的帧。当计时器超时时，发送方只收到了 0、2、3 号帧的确认，则发送方需要重发的帧数是多少？”

解析：GBN 采用累积确认，收到 ACK3 就意味着序号 3 及其之前的所有帧都已正确接收。所以 0,1,2,3 全部算已确认，需要重传的是 4,5,6,7，共 **4** 个帧。

GBN 的两个陷阱

1. 累积确认的含义：只收到 ACK2 和 ACK3 时，ACK3 已经包含了 0,1,2,3，不要再去数 ACK2。
2. **GBN** 在信道质量差时效率可能低于停等：因为一个帧出错就要重传一串帧。题目说”GBN 一定比停等效率高” 是错的。

（四）选择重传协议（SR）

基本思想：为了只重传真正出错的帧，接收方不再采用累积确认，而对每一个正确收到的帧逐一确认；同时接收方设置接收窗口，允许缓存失序到达但无差错的帧。

工作规则：

- 发送方：可在未收到确认时把发送窗口内的多个帧全部发出；只有按序收到对已发送帧的确认时，发送窗口才向前滑动；若收到未按序到达的确认，则记录它（避免其对应帧被超时重传），但窗口不滑动。
- 接收方：可接收未按序到达但无差错且序号落在接收窗口内的帧，先缓存起来；只有当按序收到帧后，接收窗口才向前滑动到相应位置。
- 收发双方均需为每个已发送/已接收的帧设置计时器（实践中常只对最早未确认的帧设一个计时器）。

*1.*为什么 $W_T = W_R \le 2^{n-1}$

由 $W_T + W_R \le 2^n$，若取 $W_T = W_R = W$，则 $2W \le 2^n$，即 $W \le 2^{n-1}$。

直观解释：若 $W = 2^{n-1} + 1$，接收窗口与发送窗口的序号区间总会重叠。当接收方缓存了一批失序帧、而发送方重传了另一批帧时，序号会”撞车”，接收方无法判断某个序号是”新帧”还是”旧帧的重传”。

SR 窗口上限

采用 n 比特编号时，$W_T = W_R \le 2^{n-1}$。$n = 3$ 时最大各为 **4**。

*2.*发送窗口滑动过程的手工模拟

设 $n = 3$，$W_T = W_R = 4$，初始发送窗口 = 接收窗口 = $\{0,1,2,3\}$。

第 **1** 步：发送方连续发送 0,1,2,3 四个帧。

第 **2** 步：0 号帧丢失，1,2,3 号帧正确到达接收方。接收方对 1,2,3 逐一确认：

- 收到 1 号帧（失序）→ 缓存，回 ACK1，接收窗口不滑动（因为期待的是 0）；
- 收到 2 号帧（失序）→ 缓存，回 ACK2，接收窗口不滑动；
- 收到 3 号帧（失序）→ 缓存，回 ACK3，接收窗口不滑动。

第 **3** 步：发送方收到 ACK1,ACK2,ACK3，这些都是未按序到达的确认，于是只做记录，发送窗口不滑动。

第 **4** 步：0 号帧的计时器超时，发送方只重传 0 号帧。

第 **5** 步：接收方收到 0 号帧，此时按序到达，于是把缓存的 0,1,2,3 一起按序交付上层，接收窗口向前滑动 **4** 个位置，变为 $\{4,5,6,7\}$，并回 ACK0。

第 **6** 步：发送方收到按序的 ACK0，发送窗口向前滑动到 $\{4,5,6,7\}$，可以继续发送。

2011 年题 35（SR 的重传数量）

”数据链路层采用选择重传协议（SR）传输数据，发送方已发送了 0~3 号数据帧，现已收到 1 号帧的确认，而 0、2 号帧依次超时，则此时需要重传的帧数是多少？”

解析：SR 是逐一确认、只重传出错帧。已发送 4 帧中 1 号已被确认，0、2 号超时需重传，3 号尚未超时也未被确认（无需重传）。所以需要重传 **2** 个帧。

2024 年题 37（SR 窗口滑动）

”主机甲通过 SR 滑动窗口协议向主机乙发送帧，F_x 为数据帧，ACK_x 为确认帧，x 是 3 比特序号。乙只对正确接收的数据帧进行独立确认。发送窗口与接收窗口大小相同且均为最大值。”

3 比特编号下 SR 的最大窗口为 $2^{3-1}=4$。初始发送窗口为 $\{0,1,2,3\}$。当 ACK0 到达后窗口基序号从 0 滑到 1，允许新发送 F4；而 F1 丢失并超时，SR 只重传 **F1**。故两个时刻发送的帧分别是 **F4**、**F1**。

（五）三种协议的对比与信道利用率

*1.*完整对比表

对比项	停止-等待	后退 **N** 帧 **GBN**	选择重传 **SR**
发送窗口 W_T	1	$1 < W_T \le 2^n - 1$	$W_T \le 2^{n-1}$
接收窗口 W_R	1	1	$W_R = W_T \le 2^{n-1}$
确认方式	逐个确认	累积确认	逐帧独立确认
是否使用 NAK	一般不用	不用	常配 NAK，超时 + NAK 双触发
重传范围	仅当前帧	从出错帧起的全部已发送帧	仅出错的那一帧
接收方缓存失序帧	无（也不需要）	不缓存，直接丢弃	缓存失序帧
发送方窗口滑动条件	收到 ACK	收到按序的累积确认	收到按序的确认
接收方窗口滑动条件	收到正确帧	收到正确帧	按序收齐后才滑动
需要的序号比特	1 bit	n bit	n bit
信道利用率	最低	较高（信道好时）	最高
适用场合	帧短、RTT 小、误码率低	信道误码率低	信道误码率较高、带宽时延积大

*2.*三个协议的信道利用率公式

设 T_D 为一帧的发送时延，T_A 为确认帧的发送时延，RTT 为往返时间。

停止-等待：

$$U_{SW} = \frac{T_D}{T_D + RTT + T_A}$$

后退 **N** 帧（**GBN**）：一个发送周期内若发送方发送了 k 个帧（$k \le W_T$），则

$$U_{GBN} = \frac{k \cdot T_D}{T_D + RTT + T_A}$$

当发送窗口足够大，使得整个发送周期都能被连续发送的帧填满时（即窗口内帧的发送总时间 $\ge T_D + RTT + T_A$），信道利用率达到最大 $U_{max} = 1$；否则取 $k = W_T$ 计算。设" 充满链路所需的最小窗口" 为

$$W_{need} = \left\lceil \frac{T_D + RTT + T_A}{T_D} \right\rceil$$

则：$W_T \ge W_{need}$ 时链路可被连续占满，$U = 1$；$W_T < W_{need}$ 时 $U = \dfrac{W_T \cdot T_D}{T_D + RTT + T_A}$。

选择重传（**SR**）：同样的公式，但由于 SR 允许接收窗口大于 1、接收方可以缓存，其实际吞吐率通常高于 **GBN**（尤其在误码率较高时优势明显）。

把利用率换算成**"** 有效数据传输速率**"**：

$$\text{有效数据传输速率} = U \times R$$

*3.*信道利用率/吞吐率完整算例（含 k 段链路）

通用公式的推导：设主机甲经过 k 段链路向主机乙发送数据，每段链路的单向传播时延为 τ、传输速率为 R，数据帧长为 L，确认帧时延忽略不计。

- 一帧的发送时延 $T_D = \dfrac{L}{R}$（近似认为中间结点转发时延可忽略）；
- 从甲开始发送第一个帧，到甲的第一个确认帧回到甲，所经历的时间为 $T_{\text{周期}} = T_D + 2k\tau$（$2k\tau$ 即整条路径的往返传播时延）；
- 在一个周期内，甲最多能连续发送 W_T 个帧，但受周期长度限制，实际能连续发送的帧数最多为 $\left\lfloor \dfrac{T_{\text{周期}}}{T_D} \right\rfloor$。

因此有效数据传输速率为：

$$R_{\text{有效}} = \min\left(W_T,\ \left\lceil \frac{T_D + 2k\tau}{T_D} \right\rceil\right) \times \frac{L}{T_{\text{周期}}} = \min\left(W_T,\ \frac{T_D + 2k\tau}{T_D}\right) \times \frac{L}{T_D + 2k\tau}$$

当窗口足够大（$W_T \ge \dfrac{T_D + 2k\tau}{T_D}$）时窗口不再是瓶颈，此时

$$R_{\text{有效}} = \frac{W_T \cdot L}{T_D + 2k\tau}\text{（受窗口限制）} \qquad \text{或} \qquad R_{\text{有效}} = R\text{（链路被完全占满）}$$

算例一（三段链路）：主机甲经 3 段链路向主机乙发送数据，每段链路长 1000 m、信号传播速度 2×10^8 m/s、传输速率 10 Mbps，GBN 协议发送窗口 $W_T = 8$，数据帧长 1500 B，确认帧时延忽略不计。求甲的有效数据传输速率。

第 1 步，单向传播时延 $\tau = 1000/(2 \times 10^8) = 5\ \mu s$，全路径往返 $2k\tau = 2 \times 3 \times 5 = 30\ \mu s$。

第 2 步，$T_D = \dfrac{1500 \times 8}{10 \times 10^6} = 1.2\ \text{ms} = 1200\ \mu s$。

第 3 步，$T_{\text{周期}} = T_D + 2k\tau = 1200 + 30 = 1230\ \mu s$。

第 4 步，一个周期内能连续发送的帧数上限 $= \lfloor 1230/1200 \rfloor = 1$，而 $W_T = 8$ 远大于该值，所以瓶颈在链路时延而非窗口，发送方必须等确认，只能发 1 帧。

第 5 步，$R_{\text{有效}} = \dfrac{1 \times 1500 \times 8\ \text{bit}}{1230\ \mu s} \approx 9.76\ \text{Mbps}$。

算例二（卫星链路定窗口）：主机甲通过 128 kbps 卫星链路，采用滑动窗口协议向乙发送数据，链路单向传播延迟 250 ms，帧长 1000 字节，不考虑确认帧开销。为使链路利用率不小于 80%，帧序号的比特数至少是多少?

$\tau = 250\ \text{ms}$，$RTT = 500\ \text{ms} = 0.5\ \text{s}$，$T_D = \dfrac{1000 \times 8}{128 \times 10^3} = 62.5\ \text{ms}$。

$$U = \frac{W_T \times T_D}{T_D + RTT} \geq 0.8 \Rightarrow \frac{W_T \times 62.5}{62.5 + 500} \geq 0.8 \Rightarrow W_T \geq \frac{0.8 \times 562.5}{62.5} = 7.2$$

取 $W_T = 8$。若用 GBN，需 $W_T \leq 2^n - 1$，即 $2^n \geq 9$，$n \geq 4$。故帧序号至少需要 **4** 个比特。（若用 SR，$W_T \leq 2^{n-1}$，$2^{n-1} \geq 8$，同样 $n \geq 4$。）

算例三（**GBN** 最大平均数据传输速率）：主机甲与乙之间使用 GBN 协议传输，甲的发送窗口 $W_T = 1000$，数据帧长 1000 字节，信道带宽 100 Mbps，乙每收到一个数据帧就立即用一个短帧（忽略其传输时延）确认，单向传播延迟 50 ms。求甲可达到的最大平均数据传输速率。

$T_D = \dfrac{1000 \times 8}{100 \times 10^6} = 80\ \mu s$。

$W_T \cdot T_D = 1000 \times 80\ \mu s = 80\ \text{ms}$，即甲连续发完整个窗口需要 80 ms。

$RTT = 2 \times 50 = 100\ \text{ms}$。由于 $80\ \text{ms} < 100\ \text{ms}$，发完一窗口后要等确认才能继续，窗口不再能保证信道连续满负荷，故：

$$R_{\text{有效}} = \frac{W_T \cdot L}{RTT + 2T_D} \approx \frac{1000 \times 1000 \times 8}{0.08 + 0.1}\ \text{bit/s} = \frac{8 \times 10^6}{0.18} \approx 4.44 \times 10^7\ \text{b/s} \approx 44.4\ \text{Mbps}$$

（若按” 帧发送时延计入往返” 的另一种常见处理，得 $\frac{1000 \times 8000}{0.1 + 0.00008} \approx 79.9$ Mbps。统考按**”** 窗口发送时间能否覆盖 $T_D + RTT + T_A$**”** 来判断：本题 $W_T \cdot T_D = 80$ ms，与 $T_D + RTT = 100.08$ ms 相比略小，说明窗口” 刚好差一点” 填满链路，故最大平均速率约 80 Mbps 量级。）

有效速率题的三个坑

1. 题目给的是” 单向传播延迟” 还是” 往返时间 RTT”，一定要看清。GBN 的效率分母含 **RTT**，不是 τ。
2. 判窗口是否成为瓶颈：比较 $W_T T_D$ 与 $T_D + RTT + T_A$。大于等于则瓶颈是链路带宽（$U = 1$），小于则瓶颈是窗口。
3. ” 有效数据传输速率” 只算数据部分，不含帧首部、尾部和确认帧的开销；而” 信道利用率” 有时按整帧长度算。看清题目问的是哪一个。

五、介质访问控制

介质访问控制（**Medium Access Control**，**MAC**）要解决的问题是：多个站点共享同一信道时，如何分配信道的使用权。三大类方法分别是信道划分、随机访问和轮询访问。

（一）信道划分介质访问控制

信道划分的基本思想是把一条信道按某种维度切分成多条互不干扰的子信道，每个站点固定（或按需）使用其中一条，从而从根本上避免冲突。

*1.*频分多路复用 *FDM*

频分多路复用（**Frequency Division Multiplexing**）把传输介质的总带宽划分为多个互不重叠的频段，每个用户独占一个频段，各用户在同一时间占用不同频率，通信过程中始终占用该频段。

特点：所有用户在同样的时间占用不同的带宽资源；适合模拟信号传输；各子信道之间需留保护频带以防串扰。

*2.*时分多路复用 *TDM*

时分多路复用（**Time Division Multiplexing**）把传输时间划分为一个个时间片（时隙），每个用户在各帧中固定占用其中一个时隙，各用户在不同时间占用相同频率。

特点：所有用户在不同时间占用同样的频带宽度；适合数字信号传输；时间片固定分配，某用户无数据时该时隙只能空闲，造成浪费。

统计时分复用 **STDM**：又称异步时分复用。它按需动态分配时隙——只有当某用户确实有数据要发时才分配时隙，并在每个时隙前附加用户标识。这样显著提高了信道利用率，代价是需要额外的地址信息与缓存，实现更复杂。

*3.*波分多路复用 *WDM*

波分多路复用（**Wavelength Division Multiplexing**）是光的频分多路复用。在一根光纤上同时传输多路波长（频率）不同的光载波信号，各路信号互不干扰，经棱镜或衍射光栅合波/分波后实现多路复用。

特点：一根光纤可复用几十甚至上百个波长，极大提高光纤传输容量；本质上仍是频分复用，只是频段在光波范围。

*4.*码分多路复用 *CDM* 与 *CDMA*

码分多路复用（**Code Division Multiplexing**，**CDM**）是另一种完全不同的思路：所有站点在同一时间、同一频段上发送数据，但每个站点使用一个独特的、与其他站点正交的码片序列（**chipping sequence**）对数据进行编码，接收方用相应站点的码片序列做内积运算，就能从混合信号中”分离”出该站点的数据。它的多址接入方式称为码分多址（**Code Division Multiple Access**，**CDMA**）。

编码规则：

- 每个站被分配一个 m 比特的码片序列 $\vec{S}$，通常 $m = 2^k$（如 8、64、128）。
- 该站发送比特 **1** 时，发送自己的码片序列 $\vec{S}$；发送比特 **0** 时，发送码片序列的反码 $\vec{S'}$（即 $\vec{S}$ 各位取反）。
- 若某站没有发送数据，则该站对应的输出为 0。

正交性要求（任取两站 S 与 T，向量按位相乘后求和）：

$$\vec{S} \cdot \vec{T} = \frac{1}{m} \sum_{i=1}^{m} S_i T_i = 0 \quad \text{（不同站正交）}$$

$$\vec{S}\cdot\vec{S}=\frac{1}{m}\sum_{i=1}^{m}S_iS_i=1\ \text{（自身内积为 1）}$$

$$\vec{S}\cdot\overline{\vec{S}}=-1\ \text{（自身与反码内积为 −1）}$$

分离方法：接收方收到的是所有正在发送的站点的码片序列之叠加（线性相加）信号 $\vec{R}$。想要分离出站 A 发的数据，只需计算 $\vec{R}\cdot\vec{S_A}$：

- 结果 >0（即 $+1$）：站 A 发了比特 **1**；
- 结果 <0（即 -1）：站 A 发了比特 **0**；
- 结果 $=0$：站 A 没有发送数据（其他站与 S_A 正交，内积为 0）。

完整算例（**2014** 年统考真题改编）：站点 A、B、C 通过 CDMA 共享链路，码片序列分别为

$$\vec{S_A}=(1,1,1,1)\qquad\vec{S_B}=(1,-1,1,-1)\qquad\vec{S_C}=(1,1,-1,-1)$$

先验证正交性：

$$\vec{S_A}\cdot\vec{S_B}=\tfrac{1}{4}(1\times1+1\times(-1)+1\times1+1\times(-1))=0\ \checkmark$$

$$\vec{S_A}\cdot\vec{S_C}=\tfrac{1}{4}(1\times1+1\times1+1\times(-1)+1\times(-1))=0\ \checkmark$$

$$\vec{S_B}\cdot\vec{S_C}=\tfrac{1}{4}(1\times1+(-1)\times1+1\times(-1)+(-1)\times(-1))=0\ \checkmark$$

接收方 C 从链路上收到的叠加序列为

$$\vec{R}=(2,0,2,0,\ 0,-2,0,-2,\ 0,2,0,2)$$

把 $\vec{R}$ 平均分成 3 段（每段 4 个码片，对应 3 个比特的传输时间）：

$$\vec{R_1}=(2,0,2,0)\quad\vec{R_2}=(0,-2,0,-2)\quad\vec{R_3}=(0,2,0,2)$$

分离站 **A** 的数据（用 $\vec{S_A}=(1,1,1,1)$）：

$$\vec{R_1}\cdot\vec{S_A}=\tfrac{1}{4}(2+0+2+0)=1\Rightarrow\text{比特 }1$$

$$\vec{R_2}\cdot\vec{S_A}=\tfrac{1}{4}(0-2+0-2)=-1\Rightarrow\text{比特 }0$$

$$\vec{R_3}\cdot\vec{S_A}=\tfrac{1}{4}(0+2+0+2)=1\Rightarrow\text{比特 }1$$

所以 A 发送的数据是 `101`。

再验证 **B**、**C** 的数据以确认理解正确（用 $\vec{S_B}=(1,-1,1,-1)$）：

- 第 1 段：$\tfrac{1}{4}(2\times1+0\times(-1)+2\times1+0\times(-1))=\tfrac{1}{4}(2+0+2+0)=1\to1$

- 第 2 段：$\frac{1}{4}(0\times1+(-2)(-1)+0\times1+(-2)(-1))=\frac{1}{4}(0+2+0+2)=1\to1$
- 第 3 段：$\frac{1}{4}(0\times1+2\times(-1)+0\times1+2\times(-1))=\frac{1}{4}(0-2+0-2)=-1\to0$ 故 B 发送 `110`。

用 $\vec{S_C}=(1,1,-1,-1)$：

- 第 1 段：$\frac{1}{4}(2\times1+0\times1+2\times(-1)+0\times(-1))=\frac{1}{4}(2+0-2+0)=0$
- 第 2 段：$\frac{1}{4}(0+(-2)+0+2)=\frac{1}{4}(0-2+0+2)=0$
- 第 3 段：$\frac{1}{4}(0+2+0-2)=0$ 故 C 自己没有发送数据（这与"C 在接收"一致）。

交叉验证叠加信号确实自洽（B=110，C 不发）：第 1 段：A 发 1 用 $\vec{S_A}$，B 发 1 用 $\vec{S_B}$，叠加为 $(1,1,1,1)+(1,-1,1,-1)=(2,0,2,0)$ ✓ 第 2 段：A 发 0 用 $-\vec{S_A}=(-1,-1,-1,-1)$，B 发 1 用 $\vec{S_B}=(1,-1,1,-1)$，叠加为 $(0,-2,0,-2)$ ✓ 第 3 段：A 发 1 用 $\vec{S_A}$，B 发 0 用 $-\vec{S_B}=(-1,1,-1,1)$，叠加为 $(0,2,0,2)$ ✓

CDMA 计算题的标准解法

1. 先验证各站码片序列两两内积为 **0**（正交），题目若已给出正交性可直接用。
2. 把接收序列按码片长度平均分段，段数 = 传输的比特数。
3. 用目标站的码片序列与每一段做内积、再除以码片长度。
4. 结果 $+1$ 记 1，-1 记 0，0 表示该站未发送。

（二）随机访问介质访问控制

随机访问的特点是所有站点都可以按随机的方式争用信道，没有固定分配，冲突可能发生，因此必须有冲突解决机制。

1.ALOHA 协议

纯 **ALOHA**（**Pure ALOHA**）：站点只要想发就发，完全不监听信道。若一段时间内没收到确认就认为发生冲突，随机退避后重传。

设一个帧的发送时延为 T_0（称为一个帧时）。由于各站随机发送，一个帧若想无冲突地发送成功，必须在前后各一个帧时（共 $2T_0$）内没有任何其他站开始发送。若网络总负载为 G（单位帧时内到达的帧数，G 为泊松分布均值），则

$$P_{\text{成功}}=e^{-2G}\qquad S=G\cdot P_{\text{成功}}=G\,e^{-2G}$$

对 G 求导令其为 0，得 $G=0.5$ 时取最大值：

$$S_{max}=0.5\,e^{-1}\approx0.5\times0.368=0.184$$

即纯 **ALOHA** 的信道吞吐率极限约为 **18.4%**。

时分 **ALOHA**（时隙 **ALOHA**，**Slotted ALOHA**）：把时间划分为等长的时隙，时隙长度等于一个帧时 T_0，站点只能在时隙的开始时刻发送。这样冲突只能发生在同一个时隙内，一个帧要成功只需后一个帧时内无其他站发送：

$$P_{\text{成功}}=e^{-G}\qquad S=G\,e^{-G}$$

$G=1$ 时取最大值：

$$S_{max} = e^{-1} \approx 0.368$$

即时隙 **ALOHA** 的信道吞吐率极限约为 **36.8%**，正好是纯 ALOHA 的两倍。

ALOHA 的两个" 极限值" 必须记住

$0.184 = \frac{1}{2e}$（纯 ALOHA）与 $0.368 = \frac{1}{e}$（时隙 ALOHA）。时隙 **ALOHA** 的吞吐率是纯 **ALOHA** 的 **2** 倍，原因是纯 ALOHA 的易冲突区间是 $2T_0$，时隙 ALOHA 只有 T_0。

2.CSMA 协议

CSMA（Carrier Sense Multiple Access，载波监听多址接入）在 ALOHA 的基础上增加了" 先听后说"：发送前先检测信道，信道空闲才发。

但仅靠载波监听并不能完全避免冲突，因为信号传播存在时延：A 与 B 相隔较远时，A 开始发送的信号还没到达 B，B 检测到信道" 空闲" 也发送，就产生了冲突。于是派生出三种不同的监听策略：

1-坚持 **CSMA**：站点要发送时先监听信道。

- 信道空闲：立即发送（"1" 表示以概率 1 发送）；
- 信道忙：持续监听，一旦变为空闲就立即发送。

缺点：多个站点同时在等待时，信道一变空闲就会几乎同时发送，冲突概率高。

非坚持 **CSMA**：站点要发送时先监听信道。

- 信道空闲：立即发送；
- 信道忙：随机等待一段时间后重新监听（不是持续监听）。

优点：降低了冲突概率；缺点：等待的随机时间可能较长，信道刚变空闲时可能无人发送，造成信道利用率下降。

p-坚持 **CSMA**：适用于时隙信道。站点要发送时先监听信道。

- 信道空闲：以概率 p 立即发送，以概率 $1-p$ 推迟到下一个时隙再重新监听；
- 信道忙：等待下一个时隙再继续监听。

优点：兼顾了" 降低冲突概率" 和" 减少信道空闲浪费"，但 p 值难以合理选择（p 太大易冲突，太小则时延大）。

3.CSMA/CD 协议

CSMA/CD（Carrier Sense Multiple Access / Collision Detection，载波监听多址接入/碰撞检测）是共享总线以太网使用的协议，要点是" 先听后说、边说边听、冲突停说、随机重说"。

三个组成部分：

- 多址接入 **MA**：多个站点连接在一条总线上，竞争使用总线。
- 载波监听 **CS**：每个站点发送前先检测总线是否忙。
  - 若检测到总线空闲 **96** 比特时间（发送 96 比特所需时间，即帧间最小间隔），则发送该帧；
  - 若总线忙，则继续监听并等待总线转为空闲 96 比特时间后再发送。
- 碰撞检测 **CD**：每个正在发送的站点边发送边检测碰撞，一旦发现碰撞就立即停止发送，并继续发送 **32** 比特或 **48** 比特的人为干扰信号，以便让总线上所有站点都能检测到碰撞；随后退避一段随机时间后重发。

CSMA/CD 的几个关键结论

1. 载波监听检测到总线空闲，总线并不一定真的空闲（存在传播时延）。
2. CSMA/CD 只能尽量避免碰撞，不能完全避免碰撞。
3. 因为要”边发送边检测”，站点不可能同时发送和接收，所以使用 CSMA/CD 的共享总线以太网只能半双工工作。
4. **CSMA/CD** 不适用于无线网络（无线网卡难以实现碰撞检测，且存在隐蔽站问题），无线网使用 CSMA/CA。

争用期 2τ 的完整推导

设总线两端的两个站点 A、B 相距最远，单向传播时延为 τ。考虑最坏情况：

- 时刻 $t=0$，A 开始发送；A 的信号要经过 τ 才能到达最远的 B。
- 若 B 恰好在收到 A 的信号之前（即在 $t=\tau-\delta$，$\delta\to 0$）开始发送，则 B 检测信道时认为信道是空闲的。
- B 的信号要经过 τ 才能到达 A，所以在 $t\approx 2\tau$ 时 A 才检测到碰撞。

因此：站点从开始发送帧起，最多经过 2τ 就能确定自己是否遭遇了碰撞。这个 2τ 称为争用期（也叫碰撞窗口）。若经过争用期还没有检测到碰撞，就可以肯定这次发送不会碰撞。

反过来考虑最快情况：双方同时开始发送，则各自在经过 τ 后就检测到冲突。所以检测到冲突的最短时间为 τ（严格说趋近于 **0** 的情形对应**”**对方在信号即将到达时才开始发送**”**）。

更精确地说：设 A 先发、B 后发，B 发送的时刻比 A 晚 t 时间（$0<t<\tau$）：

- A 检测到冲突的时刻 $=t+\tau$（$t\to\tau$ 时最大为 2τ，$t\to 0$ 时最小趋向 τ）；
- B 检测到冲突的时刻 $=\tau$（B 一发出信号就立刻与 A 的信号相遇，几乎立即检测到，时间趋于 0）。

综合结论：从开始发送数据时刻起到双方都检测到冲突，最短时间为 τ；单个站点检测到冲突的最长时间为 2τ。

最小帧长的完整推导 $L_{min}=2\tau\cdot R$

为了让站点在发送完整个帧之前就能检测出碰撞，帧的发送时延必须不小于一个争用期：

$$T_D=\frac{L}{R}\ \geq\ 2\tau\quad\Longrightarrow\quad\boxed{L_{min}=2\tau\cdot R}$$

其中 R 为数据传输速率，τ 为单向端到端传播时延。

为什么以太网最小帧长是 **64** 字节：对于 10 Mb/s 的传统共享总线以太网，标准规定争用期 2τ 的值等于发送 **512** 比特所需的时间，即 51.2 μs。于是

$$L_{min}=512\text{ bit}=64\text{ B}$$

这一规定的实际意义：若某站点发送的帧在前 64 B 内没有遭遇碰撞，那么后续部分也不会遭遇碰撞（因为此时其他站点都已听到信号）。反之，若遭遇碰撞，一定发生在帧的前 **64 B** 之内，此时站点已发送的数据必然小于 **64 B**。因此接收方收到长度小于 **64 B** 的帧，就可以判定这是一个因碰撞而异常中止的无效帧，直接丢弃。

最大帧长 **1518 B**：若不限制帧长，一个很长的帧会长时间独占总线，使其他站点等待时间过长（不公平），且出错重传的代价大。因此规定数据载荷最大 1500 B，加上 6 B 目的地址 + 6 B 源地址 + 2 B 类型 + 4 B FCS = 18 B 的首部与尾部，最大帧长为 **1518 B**。

计算题完整算例

算例一（求最大端到端距离）：在一个采用 CSMA/CD 协议的网络中，传输速率 $R = 1$ Gbps，信号传播速度 $v = 200\,000$ km/s。若最小数据帧长度减少 800 比特，则最远两个站点之间的距离至少需要怎样变化?

$$L_{min} = 2\tau R \Rightarrow \tau = \frac{L_{min}}{2R} \Rightarrow d = v\tau = \frac{v\,L_{min}}{2R}$$

$$\Delta d = \frac{v\,\Delta L_{min}}{2R} = \frac{2 \times 10^8 \text{ m/s} \times 800}{2 \times 10^9 \text{ b/s}} = 80 \text{ m}$$

即最大距离需减少 **80 m**。

算例二（求最大单向传播时延）：假设一个采用 CSMA/CD 协议的 100 Mb/s 局域网，最小帧长是 128 B，则在一个冲突域内两个站点之间的单向传播时延最多是多少?

$$2\tau = \frac{L_{min}}{R} = \frac{128 \times 8}{100 \times 10^6} = 10.24\ \mu s \Rightarrow \tau = 5.12\ \mu s$$

算例三（含中继器/**Hub** 的延迟）：H2 与 H3 之间通过一个 Hub 相连，100 Mbps 以太网，最小帧长 64 B，信号通过 Hub 产生额外 1.51 μs 延时，信号传播速度 2×10^8 m/s。求 H2 与 H3 可以相距的最远距离。

最小帧发送时延 $T_D = \dfrac{64 \times 8}{100 \times 10^6} = 5.12\ \mu s$。往返时延必须不大于 T_D：

$$2 \times \frac{d}{v} + 2 \times 1.51\ \mu s \le 5.12\ \mu s \Rightarrow \frac{2d}{2 \times 10^8} \le 2.10\ \mu s \Rightarrow d \le 210 \text{ m}$$

算例四（有效数据传输速率）：某局域网采用 CSMA/CD，数据传输速率 10 Mbps，甲、乙相距 2 km，信号传播速度 200 000 km/s（即 200 m/μs）。若网络不存在任何冲突与差错，甲总是以标准最长以太网帧（1518 B）向乙发送，乙每成功收到一个数据帧后立即向甲发送一个 64 B 的确认帧，甲收到确认帧后才发送下一个。此时甲的有效数据传输速率是多少（不考虑前导码）?

第 1 步，单向传播时延 $\tau = 2000 \text{ m} \div 200 \text{ m}/\mu s = 10\ \mu s$，$RTT = 20\ \mu s = 0.02$ ms。

第 2 步，1518 B 数据帧发送时延 $T_D = \dfrac{1518 \times 8}{10 \times 10^6} = 1.2144$ ms。

第 3 步，64 B 确认帧发送时延 $T_A = \dfrac{64 \times 8}{10 \times 10^6} = 0.0512$ ms。

第 4 步，发送周期 $= 1.2144 + 0.0512 + 0.02 = 1.2856$ ms。

第 5 步，1518 B 帧中有效数据载荷为 **1500 B**，故

$$R_{\text{有效}} = \frac{1500 \times 8 \text{ bit}}{1.2856 \text{ ms}} \approx 9.33 \text{ Mbps}$$

截断二进制指数退避算法

当站点检测到碰撞后，需要退避一段随机时间再重发。共享总线以太网采用截断二进制指数退避（**Truncated Binary Exponential Backoff**）算法：

- 记该帧的重传次数为 k。取 $K = \min(k, 10)$。
- 从整数集合 $\{0, 1, 2, \ldots, 2^K - 1\}$ 中随机取出一个数 r。
- 退避时间 $= r\times$ 争用期 2τ（10 Mb/s 以太网中争用期 = 51.2 μs，即一个” 争用时间片”）。
- 当重传次数达到 **16** 次仍不成功时，丢弃该帧并向高层报告。

这一算法的效果：重传需要推迟的平均时间随重传次数而增大（动态退避），因而减小碰撞概率。若连续多次碰撞，说明参与竞争的站点较多，退避窗口自动扩大以分散发送。

完整算例（**2023** 年统考真题）：已知 10BaseT 以太网的争用时间片为 51.2 μs。若网卡在发送某帧时发生了连续 4 次冲突，则基于二进制指数退避算法确定的再次尝试重发该帧前等待的最长时间是多少?

第 4 次冲突时 $k=4$，$K=\min(4,10)=4$，r 在 $\{0,1,\ldots,2^4-1\}=\{0,\ldots,15\}$ 中随机取。最长等待时间对应 $r=15$：

$$15\times 51.2\ \mu s=768\ \mu s$$

（实际考试中给出答案 768 μs。）注意：是第 **4** 次冲突后取 $2^4=16$ 个值，不是 2^{4-1}；也别把” 连续 4 次冲突” 误当成”$K=2^4$”。

二进制指数退避的三个坑

1. 退避时间的基本单位是争用期 2τ，不是 τ，也不是发送一帧的时间。
2. $K=\min(k,10)$，重传次数超过 10 之后 K 恒为 **10**（最大值 1023 个争用期）；到 16 次直接放弃并报错。
3. 记忆口诀：” 重传几次，就在 $2^{\min(k,10)}$ 个时间片里随机抽一个”。

信道利用率的极限分析

考虑理想情况：总线一旦空闲就有某站点立即发送，且各站点发送都不会碰撞。此时：

- 发送一帧占用总线的时间为 $T_0+\tau$（其中 T_0 是帧本身的发送时间，τ 是信号传播到总线另一端所需时间）；
- 其中帧本身的发送时间只有 T_0；
- 因此 $\lim\limits_{\tau\to 0}$ 时信道利用率趋近 **100%**。

也就是说，当信号传播延迟趋近于 **0** 时，冲突可被瞬间发现并停止，信道上的无效时间趋近于 **0**，有效数据几乎可以连续不断地占用信道，信道利用率趋近 **100%**。

千兆以太网的特殊处理：速率提高到 1000 Mb/s 后，若仍保持最小帧长 64 B，则争用期只能容纳 100 m 的网段（再短就无使用价值）；若把最小帧长增大到 640 B，则短 PDU 的封装开销太大。解决办法是载波延伸（**Carrier Extension**）：把争用期增大为 **512** 字节的发送时间，同时保持最小帧长仍为 **64** 字节。发送的 MAC 帧长度不足 512 B 时，就在帧尾填充特殊字符使其达到 512 B。若连续发送大量短帧，则用分组突发（**Packet Bursting**）：只对第一个短帧做载波延伸填充，后续短帧只留必要的帧间最小间隔即可连续发送，直到累积发送约 1500 B 为止。

当千兆以太网工作在全双工方式时，不使用 **CSMA/CD**，也不使用载波延伸和分组突发。

4.CSMA/CA 协议

为什么无线局域网不能使用 **CSMA/CD**

802.11 无线局域网使用无线信道，与共享总线以太网使用有线传输介质不同，不能简单照搬 **CSMA/CD**。原因有二：

1. 硬件难以实现碰撞检测：无线信道的传输环境复杂，信号强度的动态范围非常大，802.11 网卡上接收到的信号强度一般远远小于发送信号的强度（可能相差百万倍），要在同一网卡上边发边检测碰撞，对硬件的要求非常高。
2. 即使能检测也没有意义（隐蔽站问题）：无线电波的传播范围有限，可能出现 A 和 B 都能与 AP

通信，但 **A** 的信号到不了 **B**，**B** 的信号也到不了 **A** 的情况。A、B 互为隐蔽站（**Hidden Station**），双方都检测不到对方在发送，都认为信道空闲而同时向 AP 发送，必然在 AP 处碰撞。此时即使有碰撞检测功能，A、B 也检测不到。

因此 802.11 采用 **CSMA/CA**（**Carrier Sense Multiple Access / Collision Avoidance**，载波监听多址接入/碰撞避免）：它保留了 **CSMA**（先听后说以减少碰撞），但把**"** 碰撞检测 **CD"** 改为**"** 碰撞避免 **CA"**。

两个必须记牢的对比

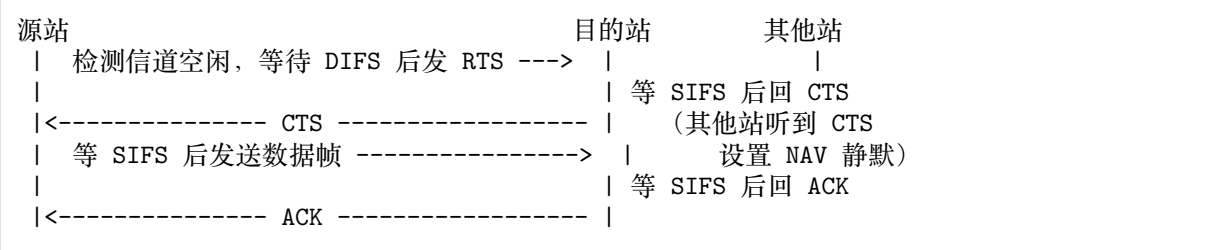
- CSMA/CA 中的 CA 表示" 碰撞避免"，但并不能避免所有碰撞，只能尽量减少碰撞发生的概率。
- **CSMA/CD** 的共享式以太网提供不可靠传输（发送完就不管了，可靠性由高层负责）；**CSMA/CA** 的 **802.11** 无线局域网必须使用确认机制提供可靠传输（因为无线信道误码率高）。

帧间间隔 IFS

802.11 规定，每个站点必须在持续检测到信道空闲一段指定的时间后才能发送帧，这段时间称为帧间间隔（**Interframe Space**，**IFS**）。IFS 的长短取决于帧的类型：低优先级的帧等待较长时间，高优先级的帧等待较短时间，从而高优先级帧可优先获得发送权。最常用的两种：

- 短帧间间隔 **SIFS**（**Short IFS**）：长度 **28** μ**s**，是最短的帧间间隔，用来分隔属于一次对话的各帧，站点应能在这段时间内从发送方式切换到接收方式。使用 SIFS 的帧有 **ACK** 帧、**CTS** 帧、过长的 MAC 帧分片后的数据帧、所有回答 AP 探询的帧以及 PCF 方式中 AP 发出的任何帧。
- **DCF** 帧间间隔 **DIFS**（**DCF IFS**）：长度 **128** μ**s**，比 SIFS 长很多，在 DCF 方式中用于发送数据帧和管理帧。发送数据帧前等待 DIFS 是考虑到可能有其他站有高优先级帧要发送。

完整的 CSMA/CA 数据交换过程



- 源站在发送数据帧之前先发送一个短控制帧 **RTS**（**Request To Send**，请求发送），其中包含源地址、目的地址和本次通信所需的持续时间（含目的站发回确认所需的时间）。发送 RTS 前必须先检测信道，若空闲则等待 DIFS 后发送。
- 目的站正确收到 RTS 且媒体空闲，等待 **SIFS** 后发送 **CTS**（**Clear To Send**，允许发送），CTS 中也包含持续时间（把 RTS 中的持续时间复制过来）。
- 源站收到 CTS 后再等 **SIFS**，就可以发送数据帧。
- 目的站正确收到数据帧后等 **SIFS**，向源站发送确认帧。
- 除源站和目的站以外的其他各站，在收到 **CTS** 帧或数据帧后就推迟访问信道，从而保证源站与目的站的通信不受干扰。若 RTS 发生碰撞，源站收不到 CTS，就执行退避算法重传 RTS。

虚拟载波监听与 NAV

802.11 采用虚拟载波监听（**Virtual Carrier Sense**）机制，目的是让源站把它要占用信道的时间及时通知给所有其他站，使其他站在这一段时间内都停止发送。

当某站检测到信道中传送的帧首部中的”持续时间”字段时，就调整自己的网络分配向量（**Network Allocation Vector**，**NAV**）。**NAV** 指出还必须经过多长时间才能完成帧的这次传输、信道才能转入空闲。

因此，某站认为信道处于忙态有两种可能：

1. 物理层载波监听检测到信道忙（物理载波监听）；
2. MAC 层的虚拟载波监听机制指出信道忙（**NAV** 未减到 **0**）。

RTS/CTS 属于虚拟载波监听机制：站点只要监听到数据帧、**RTS** 帧或 **CTS** 帧中的任何一个，就能知道信道将被占用的持续时间，而不需要真正监听到信道上的信号。因此虚拟载波监听能有效缓解隐蔽站问题：互为隐蔽站的 A 和 B 中，虽然 B 听不到 A 发给 AP 的 RTS，但能听到 **AP** 应答给 **A** 的 **CTS**，B 据 CTS 中的持续时间设置 NAV，在 NAV 指示的时间内保持静默，从而不干扰 A 与 AP 的通信。

NAV 完整算例（**2024** 年统考真题）：在采用 CSMA/CA 的 802.11 无线局域网中，DIFS = 120 μs，SIFS = 28 μs，RTS、CTS、ACK 帧的传输时延分别是 3 μs、2 μs、2 μs，忽略信号传播时延。主机 A 欲向 AP 发送一个总长度为 1998 B 的数据帧，无线链路带宽 54 Mb/s。求隐蔽站 B 收到 AP 发送的 CTS 帧时设置的 NAV 值。

第 1 步，数据帧发送时延 $T_{data} = \dfrac{1998 \times 8}{54 \times 10^6} = 296\ \mu s$。

第 2 步，NAV 从收到 **CTS** 之后开始计剩余时间：CTS 传输完毕时，还需要经过 SIFS + 数据帧 + SIFS + ACK：

$$NAV = SIFS + T_{data} + SIFS + T_{ACK} = 28 + 296 + 28 + 2 = 354\ \mu s$$

算 NAV 时不要多算也不要少算

- NAV 不包含正在传输的 **CTS** 帧本身的时间（B 是在”收到 CTS 帧时”设置 NAV，从那一刻起往后计）。
- NAV 必须包含数据帧之前的那个 SIFS、数据帧本身、数据帧之后到 ACK 之前的 SIFS，以及 ACK 帧。
- 首次竞争的 DIFS（120 μs）不计入 NAV。

退避算法

为了尽量避免碰撞，CSMA/CA 采用了不同于 **CSMA/CD** 的退避机制：

- **CSMA/CD**：站点监听到信道空闲且持续 96 比特时间后立即发送，同时进行碰撞检测；只有检测到碰撞才退避。
- **CSMA/CA**：站点检测到信道从忙态变为空闲时，除了要等待一个 DIFS 间隔，还要退避一段随机时间后才能试图接入信道。因为 CSMA/CA 没有碰撞检测机制，若信道一空闲大家就同时发，必然碰撞。所有要发送的站都要执行退避算法，这既可大大减少碰撞概率，也可避免某个站长时间占用无线信道。

退避算法的执行：各站给自己的退避计时器（**Backoff Timer**）设置一个随机退避时间。

- 当退避时间减小到零时，开始发送帧。
- 当退避时间还未减到零而信道又转变为忙态时，冻结退避计时器（保留剩余值），重新等待信道变为空闲，再经过 DIFS 间隔后，从剩余时间继续倒计时。

退避时间的选择：进行第 i 次退避时，退避时间在时隙编号 $\{0, 1, \ldots, 2^{2+i} - 1\}$ 中随机选择一个，再乘以基本退避时间（一个时隙的长度）。当时隙编号达到 **255**（对应第 **6** 次退避）时不再增加。

与 CSMA/CD 的对比：CSMA/CA 把随机选择退避时间的范围扩大了——第 1 次退避是在 $2^{2+1}=8$ 个时隙中随机选一个（而 CSMA/CD 第 1 次是 $2^1=2$ 个），第 2 次退避是在 $2^{2+2}=16$ 个时隙中选（而 CSMA/CD 是 $2^2=4$ 个）。

唯一不使用退避算法的情况：检测到信道空闲，并且该数据帧不是成功发送完上一个数据帧之后立即连续发送的数据帧。除此之外的以下情况都必须使用退避算法：

- 在发送帧之前检测到信道处于忙态；
- 在每一次重传一个帧时；
- 在每一次成功发送后要连续发送下一个帧时。

RTS/CTS 的三档选择：802.11 为用户提供了三种选择——

1. 总是使用 RTS 帧和 CTS 帧；
2. 只有当数据帧的长度超过某个数值时才使用 RTS 帧和 CTS 帧（数据帧本身很短时再预约意义不大）；
3. 不使用 RTS 帧和 CTS 帧（基础模式，只靠载波监听 + 退避）。

CSMA/CA 是否一定需要预约信道

不是。CSMA/CA 有两种工作方式：

- 基础模式：只做载波监听 + 退避，不预约信道（最常用、默认）；
- **RTS/CTS** 模式：才需要专门预约信道（可选，不是必须）。

这点常被误当成”CSMA/CA 必须使用 RTS/CTS”。

CSMA/CA 相关真题分布

- 2011 年题 36：需要使用确认方案的 **MAC** 协议是 CSMA/CA（以太网 MAC 层不确认）。
- 2018 年题 35：**CSMA/CA** 进行信道预约的方法是发送方先发 RTS，接收方回 CTS，其他站点听到 CTS 后设置 NAV 并保持静默（答案 D）。
- 2020 年题 37：**IFS** 长短判断——首次竞争发送前的 IFS 是 DIFS（最长）；CTS、DATA、ACK 属于同一交换序列，其间的 IFS 都是 SIFS。
- 2024 年题 36：**NAV** 计算（见上面的完整算例）。

（三）轮询访问：令牌传递协议

轮询访问介于” 信道划分” 与” 随机访问” 之间：没有冲突（因为信道使用权是被有序授予的），同时信道利用率高（不像 TDM 那样固定分配时隙）。轮询访问的典型代表是令牌传递协议（**Token Passing**）。

令牌环（**Token Ring**，**IEEE 802.5**）

- 网络中的所有站点构成一个物理环，令牌（一个特殊的控制帧）沿环单向逐站传递。
- 只有持有令牌的站才有权发送数据。某站收到令牌后，若没有数据要发送，就立即把令牌转发给下一个站；若有数据要发送，就把令牌置为” 忙” 并附加数据帧后发送出去。
- 数据帧沿环一周后回到源站，源站将其回收（不转发），再把令牌释放（置为空闲）交给下一站。因此同一时刻环上最多只有一个站能发送数据，绝不会发生冲突。
- 令牌环支持优先级机制：帧首部有优先级位与预约位，站点可以在经过的帧中” 预约” 下一次的令牌优先级，使得紧急数据能优先获得令牌。
- 令牌保持时间（**Token Holding Time**）：站点获得令牌后可以保持令牌的最长时间。设置这一上

限是为了防止某个站点长时间独占令牌，保证环上各站公平地分享带宽。

令牌总线（**Token Bus**，**IEEE 802.4**）：物理上是总线拓扑，但逻辑上把各站排成一个环，令牌按逻辑环的顺序传递。这样既避免了总线争用冲突，又能利用总线结构的可靠性。802.4 已基本退出实用。

令牌传递协议的特点

- 无冲突：信道使用权由令牌有序授予，不需要冲突检测与退避。
- 重负载时效率高：站点越多、负载越重，令牌环的效率反而相对越好（因为没有冲突开销）。
- 轻负载时时延大：即使只有一个站要发数据，也常常要等令牌绕一圈回来。
- 令牌丢失或重复会破坏环网：需要专门的监控站来检测并重新生成令牌。
- 令牌环、令牌总线、CSMA/CD、CSMA/CA 都工作在 **MAC** 子层。

六、局域网

（一）局域网的基本概念与体系结构

局域网（**Local Area Network**，**LAN**）是指在一个较小的地理范围内（一间办公室、一栋楼、一个校园），由多种计算机和外部设备通过高速通信线路相连的网络。

局域网的主要特点：

- 地理范围小（通常几米到几千米）；
- 数据传输速率高（10 Mb/s 到 10 Gb/s 甚至更高），误码率低；
- 所有权单一，由某一单位所有和管理，不受公共网络法规约束；
- 主要使用广播信道，因此介质访问控制方法是局域网的核心技术。

局域网的拓扑结构：总线型、星型、环型、树型（以及混合型）。

- 总线型：所有站点共享一条总线，典型代表是早期共享总线以太网，使用 CSMA/CD。
- 星型：所有站点连接到一个中心设备（集线器或交换机）。使用集线器时物理上是星型、逻辑上仍是总线型；使用交换机时才是真正的交换式以太网。
- 环型：站点首尾相连成环，典型代表是令牌环。
- 树型：多级星型级联而成，扩展性好。

IEEE 802 系列标准：

标准	内容
IEEE 802.1	局域网体系结构、网络互连、**VLAN（802.1Q）**、生成树（802.1D）
IEEE 802.2	逻辑链路控制 **LLC** 子层
IEEE 802.3	**CSMA/CD** 总线网（以太网）
IEEE 802.4	令牌总线网（Token Bus）
IEEE 802.5	令牌环网（**Token Ring**）
IEEE 802.11	无线局域网（**WLAN / Wi-Fi**）
IEEE 802.15	无线个人区域网（蓝牙等）

标准	内容
IEEE 802.16	宽带无线城域网（WiMAX）

LLC 子层与 MAC 子层

IEEE 802 把数据链路层划分为两个子层：

- 逻辑链路控制（**Logical Link Control**，**LLC**）子层：在 MAC 子层之上，与传输介质无关，向网络层提供一个统一的接口，屏蔽底层各种 **802** 局域网（以太网、令牌环、令牌总线等）**MAC** 子层的差异。
- 介质访问控制（**Medium Access Control**，**MAC**）子层：与传输介质相关，负责介质访问控制（CSMA/CD、CSMA/CA、令牌传递等）、帧的封装与解封、MAC 地址的识别与过滤、差错检测。

关于 LLC 与 MAC 的考查口径

几乎所有学过的数据链路层功能（帧封装、寻址、差错检测、介质访问控制）都归属于 **MAC** 子层；LLC 子层只负责” 屏蔽差异、提供统一接口”，考查频率低。另外要记住：以太网交换机是按 **MAC** 地址转发的，它并不使用 LLC 子层的服务。

（二）以太网与 IEEE 802.3

以太网（**Ethernet**）是 IEEE 802.3 标准的通称，是目前最主流的局域网技术。它的设计原则是简化通信：采用无连接、无确认的工作方式，向网络层提供不可靠的服务（差错由高层负责处理）。

以太网的发展

名称	速率	标准	关键特点
标准以太网 10BASE-T	10 Mb/s	802.3i	双绞线星型，站点到集线器不超过 100 m；争用期 51.2 μs；帧间最小间隔 9.6 μs（96 比特时间）
快速以太网 100BASE-T	100 Mb/s	802.3u	仍用 **802.3** 帧格式和 **CSMA/CD**；最小帧长仍为 **64 B**；网段最大长度减到 100 m；争用期缩为 **5.12** μ**s**；帧间最小间隔缩为 **0.96** μ**s**
千兆以太网 1000BASE-X	1 Gb/s	802.3z	支持半双工（CSMA/CD + 载波延伸 + 分组突发）和全双工（不用 CSMA/CD）；网段最大长度仍 **100 m**，最小帧长仍 **64 B**
万兆以太网 10GBASE-X	10 Gb/s	802.3ae	只工作在全双工，不使用 **CSMA/CD**，传输距离不再受碰撞检测限制；保留最小/最大帧长规定
40GE/100GE	40/100 Gb/s	802.3ba	只工作在全双工；40 Gb/s 用于计算应用，100 Gb/s 用于汇聚应用

10BASE-T 的含义：10 表示速率 10 Mb/s，BASE 表示基带传输，T 表示双绞线（Twisted pair）；F 表示光纤（Fiber）。

100BASE-T 以太网使用的导向传输介质是双绞线（2019 年题 34 考点）。

MAC 地址

- **MAC** 地址长度为 **48** 位（**6** 字节），一般被固化在网卡的电可擦可编程只读存储器 **EEPROM** 中，因此也称为硬件地址或物理地址（属于数据链路层范畴）。
- 结构上由 **3** 字节（**24** 位）机构唯一标识符（**OUI**）+ **3** 字节扩展标识符组成，OUI 由 IEEE 分配给厂商，后 3 字节由厂商自行分配，从而保证全球唯一。

- 书写形式为 6 组十六进制数，如 `00-1A-2B-3C-4D-5E` 或 `00:1A:2B:3C:4D:5E`。
- 发送顺序：字节顺序为第 1 字节 → 第 6 字节；字节内的比特发送顺序为 $b_0 \to b_7$（低位先发）。
- 广播地址为全 **1**，即 **`FF-FF-FF-FF-FF-FF`**，表示该帧发给本局域网上的所有站点。
- 多播地址：`b_0=1` 的特殊地址。
- 交换机和路由器往往具有更多的网络接口，因此拥有更多的 **MAC** 地址。严格来说，**MAC** 地址是对网络上各接口的唯一标识，而不是对网络上各设备的唯一标识。
- 网卡从网络上每收到一个帧，就检查帧首部中的目的 **MAC** 地址，按下列规则处理：
  1. 若是广播地址（`FF-FF-FF-FF-FF-FF`）→ 接受该帧；
  2. 若与网卡固化的全球单播 **MAC** 地址相同 → 接受该帧；
  3. 若是网卡支持的多播地址 → 接受该帧；
  4. 除上述三种情况外 → 丢弃该帧。

MAC 地址的两个易错点

1. **MAC** 地址不等于**”** 物理层地址**”**。它虽然被称为物理地址 / 硬件地址，但它是数据链路层使用的地址。
2. 网卡发现某帧的目的 MAC 地址不是自己时，直接丢弃，不向网络层报告错误消息（只有是自己的才上交上层）。这与 IP 层需要” 报告差错” 的行为不同。

以太网 **V2** 的 **MAC** 帧格式

以太网 V2（DIX Ethernet V2）的 MAC 帧格式如下：

字段	长度	说明
目的地址	6 B	目的 MAC 地址
源地址	6 B	源 MAC 地址
类型	2 B	指明数据载荷中的内容由上层哪个协议封装（如 0x0800 表示 IPv4）
数据	46~1500 B	网络层交付下来的分组；不足 46 B 时需填充
FCS	4 B	帧检验序列，用 CRC 校验整个帧

首部 + 尾部共 **18 B**（6+6+2+4）。因此：

- 最短帧长 = **6 + 6 + 2 + 46 + 4 = 64 B**
- 最长帧长 = **6 + 6 + 2 + 1500 + 4 = 1518 B**

注意：前导码（**8 B**）不属于 **MAC** 帧。物理层在 MAC 帧前添加 8 B 前导码，其中前 **7 B** 为前同步码，作用是使接收方的时钟同步；后 **1 B** 为帧开始符（`10101011`），表明其后紧跟着以太网帧。以太网还规定了帧间最小间隔为 **96** 比特的发送时间（10 Mb/s 时即 9.6 μs），因此以太网帧不需要帧结束符。

MTU 1500 B 的由来：数据字段最大长度被限制为 1500 B，即以太网的 **MTU = 1500 B**。若不限制，一个很长的帧会长时间独占总线，导致其他站点等待过久（不公平），且一旦出错重传代价极大。

接收方可能收到的无效 **MAC** 帧：

- MAC 帧的长度不是整数个字节；
- 通过 FCS 字段检测出帧有误码；
- MAC 帧的长度不在 64~1518 字节之间。

接收方收到无效 **MAC** 帧时，简单地将其丢弃，以太网的数据链路层没有重传机制。

以太网帧的数字速记

- 首部 + 尾部 = **18 B**；最短帧 **64 B**；最长帧 **1518 B**。
- 数据最少 **46 B**（保证 64 B 最小帧长），最多 **1500 B**（MTU）。
- 前导码 **8 B**，不计入 MAC 帧长度。
- 帧间最小间隔 = **96** 比特时间。

MAC 帧的发送与接收过程

发送过程：

1. 网络层把 IP 数据报交给数据链路层；
2. 数据链路层给数据报加上目的 MAC、源 MAC、类型字段构成首部，加上 4 B FCS 构成尾部；若数据不足 46 B，则填充到 46 B；
3. 把封装好的帧交给物理层；
4. 物理层在前面加 8 B 前导码，按曼彻斯特编码等方式转换成信号发送到传输介质上。

接收过程：

1. 物理层从介质上收到信号，做同步、解码，剥离前导码，把 MAC 帧交付数据链路层；
2. 数据链路层检查目的 MAC 地址：是广播地址、自己的单播地址或自己加入的多播地址则继续，否则丢弃；
3. 对帧做 CRC 校验，若有误码则丢弃，不通报网络层；
4. 检查帧长是否在 64~1518 B 之间，不是则丢弃；
5. 剥离首部和尾部，把数据载荷按” 类型” 字段上交给相应的网络层协议。

ARP 与以太网帧的关系

ARP 报文并不需要 **IP** 协议封装，而是直接由数据链路层协议（以太网）封装。这一点常与”ARP 是网络层协议还是链路层协议” 的争论一起出现，统考按”ARP 属于网络层协议，但其报文直接用 MAC 帧封装” 理解。

（三）IEEE 802.11 无线局域网

组成结构

- 有固定基础设施的 **802.11** 无线局域网：采用星型拓扑，中心基站称为接入点（**Access Point**，**AP**）。
  - 基本服务集（**Basic Service Set**，**BSS**）：802.11 无线局域网的最小构件，包含一个 **AP** 和若干个移动站。AP 的 MAC 地址在 802.11 标准中称为基本服务集标识符（**BSSID**）。
  - 分配系统（**Distribution System**，**DS**）：用于把多个 BSS 连接起来。**DS** 最常用的是以太网，也可用点对点链路或其他无线网络。DS 的作用是使 **ESS** 对上层的表现就像一个 **BSS** 一样，即 ESS 对上层呈现的仍是一个局域网。
  - 扩展服务集（**Extended Service Set**，**ESS**）：一个 BSS 通过 DS 与其他 BSS 连接所构成。在一个 ESS 内不同 BSS 的信号覆盖范围可能有重叠。
  - 通信路径示例：BSS1 中的移动站 A 给 BSS2 中的移动站 B 发送数据，路径为 **A** → **AP1** → **AP2** → **B**，必须经过 AP1 和 AP2 以及它们之间作为 DS 的以太网。
- 无固定基础设施的 **802.11** 无线局域网：自组织网络（**ad hoc Network**），没有 AP，各站直接通信，组网方便、生存性好。

- **802.11** 的基本服务：
  - 关联（**Association**）服务：移动站与 AP 建立关联，方法有被动扫描（AP 周期性发送信标帧，移动站据此选择 AP）和主动扫描（移动站主动发送探测请求帧，AP 回探测响应帧）。
  - 重建关联服务：把一个 AP 上的关联转移到另一个 AP（漫游）。
  - 分离服务：终止与 AP 的关联。

为什么使用 **CSMA/CA** 而不是 **CSMA/CD**：已在介质访问控制一节详细说明，核心是” 无线网卡难以实现碰撞检测” + ” 隐蔽站问题使碰撞检测没有意义”。

MAC 层：**DCF** 与 **PCF**

- 分布式协调功能（**DCF**，**Distributed Coordination Function**）：有竞争的接入方式，基于 CSMA/CA + 退避，是 802.11 的基本接入方式，必须实现。
- 点协调功能（**PCF**，**Point Coordination Function**）：无竞争的接入方式，由 AP 轮询各站，PCF 中 AP 发出的任何帧都使用 SIFS。PCF 是可选项，较少使用。

802.11 的数据帧格式

部分	长度	说明
帧头	**30 B**	包含控制信息和最多 **4** 个地址字段
数据	0~2312 B	上层交付的 PDU（通常不超过 1500 B）
帧尾	**4 B**	FCS，采用 CRC 校验

帧头中：

- 持续期（**Duration**）字段：用于实现 CSMA/CA 的虚拟载波监听和信道预约，在数据帧、RTS 帧和 CTS 帧中用该字段指出将要持续占用信道的时长。
- 序号控制字段：用来实现 802.11 的可靠传输，对数据帧进行编号。
- 帧控制（**Frame Control**）字段：含” 去往 **DS**（**To DS**）” 和” 来自 **DS**（**From DS**）” 两个标志位，以及类型、子类型等。

四个地址字段的含义

在有固定基础设施（有 AP）的 BSS 中，不管站点要与本 BSS 内的站点通信还是与 BSS 外的站点通信，都必须通过本 **BSS** 的 **AP** 转发，而源站点并不知道目的站点是否与自己在同一个 BSS 内。因此帧中必须携带足够的信息。

场景	**To DS**	**From DS**	地址 **1**	地址 **2**	地址 **3**	地址 **4**
站点 A → AP（发往 DS，可转发给 B 或路由器）	**1**	0	接收者 **AP1** 的 **MAC**（**BSSID**）	源地址 A	最终目的地址 **B**	未用
AP1 → 站点 B（从 DS 来）	0	**1**	最终目的地址 **B**	发送者 **AP1** 的 **MAC**（**BSSID**）	原始源地址 **A**	未用
自主网络模式（同一独立 BSS 内直接通信）	0	0	目的站 MAC	源站 MAC	BSSID	未用

场景	To DS	From DS	地址 1	地址 2	地址 3	地址 4
无线 DS（DS 也是 802.11，AP1 → AP2）	**1**	**1**	接收者 **AP2** 的 **MAC**	发送者 **AP1** 的 **MAC**	最终目的地址 C	原始源地址 **A**

记忆口诀：”去往 **AP** 中起止，来自 **AP** 止中起”——意思是：去往 DS（To DS = 1）时，地址 1 是 AP（中继），地址 2/3 是起点/终点；来自 DS（From DS = 1）时，地址 1 是终点（目的站），地址 2 是 AP。

完整算例（**BSS1** 中站点 **A** 给 **B** 发送数据帧）：

- A 先把数据帧发给 AP1：帧控制字段中 **To DS = 1**、**From DS = 0**；地址 **1 = AP1** 的 **MAC**（**BSSID**），地址 **2 = A** 的 **MAC**，地址 **3 = B** 的 **MAC**，地址 4 未使用。
- AP1 收到后转发给 B：帧控制字段中 **To DS = 0**、**From DS = 1**；地址 **1 = B** 的 **MAC**，地址 **2 = AP1** 的 **MAC**，地址 **3 = A** 的 **MAC**，地址 4 未使用。

若站点 **A** 要向位于 **DS** 上的路由器 **R** 发送数据帧：过程类似，但 **DS** 是以太网，因此 AP1 会把 802.11 帧转换为以太网帧再转发给 R，此时以太网帧中的源地址和目的地址分别是 **A** 和 **R** 的 **MAC** 地址（不含 **AP1** 的 **MAC**）。当 R 给 A 回响应帧时，以太网帧的源地址和目的地址分别是 R 和 A（同样没有 AP1），AP1 收到该以太网帧后再转换成 802.11 帧转发给 A。可见 **AP** 具有网桥功能。

802.11 帧为什么必须携带 AP 的 MAC 而以太网帧不必

在 802.11 中，站点的信号覆盖范围内可能有多个 **AP** 共享同一个物理信道，而站点只能与其中一个 AP 建立关联，因此 802.11 帧中必须携带 **AP** 的 **MAC** 地址（即站点所在 **BSS** 的 **BSSID**）以明确指出转发该帧的 AP。而在以太网中，**AP** 与透明网桥一样对各站点是透明的，因此以太网帧中不需要也不应该指出 AP 的 MAC 地址。

2022 年题 47（802.11 帧的地址）

”H4 与 H5 位于同一 BSS。H5 收到的是 AP 发出的 From DS 帧，则地址 1、2、3 分别为：” 按 From DS 规则（地址 1 = 终点、地址 2 = AP、地址 3 = 起点）：地址 **1 = H5** 的 **MAC**，地址 **2 = AP** 的 **MAC**，地址 **3 = H4** 的 **MAC**。

（四）VLAN 基本概念与基本原理

为什么需要 VLAN

把多个站点通过交换机连接起来就构成了交换式以太网，其中的所有站点都属于同一个广播域。随着交换式以太网规模扩大，广播域也相应扩大，会带来：

- 广播风暴：大量广播帧（如 ARP 请求）在全网泛洪，浪费网络资源与各主机的 CPU 资源；
- 难以管理和维护，并带来潜在的安全问题（不同部门的流量互相可见）。

分割广播域的传统方法是使用路由器（成本较高、配置复杂）。虚拟局域网技术应运而生。

VLAN 的概念

虚拟局域网（**Virtual Local Area Network**，**VLAN**）是一种将局域网内的站点划分成与物理位置无关的逻辑组的技术，一个逻辑组就是一个 VLAN，VLAN 中的各站点具有某些共同的应用需求。

- 属于同一 VLAN 的站点之间可以直接通信，不同 **VLAN** 中的站点之间不能直接通信（必须经

过路由器或三层交换机）。

- 管理员可通过配置交换机建立多个逻辑上独立的 VLAN。
- 连接在同一交换机上的多个站点可以属于不同的 **VLAN**；属于同一 **VLAN** 的多个站点可以连接在不同的交换机上。
- **VLAN** 并不是一种新型网络，它只是局域网能够提供给用户的一种服务。

IEEE 802.1Q 帧格式

VLAN 最常见的实现技术是基于以太网交换机的接口来实现 **VLAN**。这要求交换机：

1. 能够处理带有 VLAN 标记的帧，即 **IEEE 802.1Q** 帧（又称 Dot One Q 帧）；
2. 各接口可以支持不同的接口类型，不同接口类型对帧的处理方式不同。

IEEE 802.1Q 帧对以太网 **V2** 的 **MAC** 帧格式进行了扩展：在源地址字段和类型字段之间插入了 **4** 字节的 **VLAN** 标签（**Tag**）字段。

VLAN 标签的 4 个字段：

字段	长度	含义
标签协议标识符 **TPID**	16 bit	固定为 `0x8100`，表示该帧是 IEEE 802.1Q 帧
优先级 **PRI**	3 bit	取值 0~7，值越大优先级越高；网络阻塞时优先发送高优先级帧
规范格式指示符 **CFI**	1 bit	0 表示 MAC 地址以规范格式封装，1 表示非规范格式；以太网中 **CFI** 取 **0**
虚拟局域网标识符 **VID**	12 bit	取值范围 **0~4095**，其中 **0** 和 **4095** 保留不使用

- **VID** 是 **802.1Q** 帧所属 **VLAN** 的编号，设备利用 VID 识别帧所属的 VLAN。
- 由于 VID 共 12 位，可表示 $2^{12} = 4096$ 个编号，去掉保留的 0 和 4095，实际最多可划分 **4094** 个 **VLAN**。
- 广播帧只在同一 **VLAN** 内转发，这样就把广播域限制在了一个 **VLAN** 内。

802.1Q 帧的三个数字

插入 **4** 字节标签 → 帧变长；VID **12** 位 → 最多 **4094** 个可用 VLAN（4096 − 2 个保留值）；TPID 固定 **0x8100**。因此一个带标签的以太网帧最长可达 1522 B（1518 + 4），短帧仍为 64 B。

打标签与去标签

802.1Q 帧一般不由用户主机处理，而是由以太网交换机处理：

- 当交换机收到普通的以太网 **MAC** 帧时，会给它插入 **4** 字节 **VLAN** 标签使之成为 802.1Q 帧，该处理简称为” 打标签”。
- 当交换机转发 **802.1Q** 帧时，可能会删除其 **4** 字节 **VLAN** 标签使之成为普通以太网 MAC 帧，该处理简称为” 去标签”。
- 是否去标签取决于交换机的接口类型。

Access 与 **Trunk** 端口

根据接口在接收帧和发送帧时对帧的处理方式，以及接口连接对象的不同，以太网交换机的接口类型一般分为两种：

对比项	**Access** 接口	**Trunk** 接口
连接对象	用户主机、服务器等终端设备	交换机与交换机、交换机与路由器
所属 VLAN	只属于一个 VLAN	可承载多个 VLAN 的流量
收到普通帧	打标签，加上本接口的 PVID 后转发	若帧不带标签，按 PVID 打标签
转发到本接口	去标签后发出（终端看不到标签）	保留标签（或按本征 VLAN 去标签）直接转发

- 交换机上电启动后，若之前未对各接口进行过 VLAN 设置，则各接口类型默认为 **Access**，且缺省 **VLAN ID** 为 **1**，即各接口默认属于 **VLAN 1**。
- 对思科交换机，接口的缺省 VLAN ID 称为本征 **VLAN**（**Native VLAN**）；对华为交换机，称为端口 **VLAN ID**（**Port VLAN ID**，**PVID**）。
- 交换机的每个接口有且仅有一个 **PVID**。

例：同一交换机上划分两个 **VLAN**

不进行人为划分时，所有接口默认属于 VLAN 1 且为 Access 类型，所有主机在同一个广播域内，互相可以通信。若把部分接口划入 VLAN 2、其余留在 VLAN 1，则：

- A 与 B 同属 VLAN 1，可以正常通信；
- A 与 C 分属不同 VLAN，即使连接在同一台交换机上也无法直接通信；
- A 发出的广播帧只在 VLAN 1 内转发，不会扩散到 **VLAN 2**，广播域被成功分割。

例：两个交换机通过 **Trunk** 接口互连

- 若 Trunk 接口去除标签再转发：交换机 1 从 Access 口收到帧后打上 VLAN 10 的标签，从 Trunk 口发出；交换机 2 在 Trunk 口收到后去标签（按 PVID 归属），再从对应 VLAN 的 Access 口发给目标主机。
- 若 Trunk 接口保留标签直接转发：交换机 2 保留 802.1Q 帧的标签，按 VID 从对应 VLAN 的 Access 口转发，并在发往终端前去标签。

VLAN 间通信

不同 VLAN 之间不能直接通信，必须借助路由器或三层交换机（具有路由功能的交换机）。常见做法是” 单臂路由”（路由器的一个物理接口划分成多个子接口，每个子接口对应一个 VLAN）或用三层交换机做 VLAN 间路由。

2024 年题 35（VLAN 与 ARP）

” 支持 VLAN 划分的交换机已按端口划分了 3 个 VLAN，部分端口连接主机的 IP 地址和 MAC 地址如图，ARP 表结构为「IP 地址，MAC 地址，TTL」。下列选项中，不会出现在 H4 的 ARP 表中的是……”

关键判断：**H4** 的 **ARP** 表只会记录与 **H4** 同属一个 **VLAN**、且实际发生过通信的 **IP−MAC** 映射。不同 VLAN 的主机之间不能直接通信，因此其他 **VLAN** 主机的 **IP** 地址不会出现在 **H4** 的 **ARP** 表中（跨 VLAN 通信时 H4 的 ARP 表里出现的会是网关地址，而不是对方主机的地址）。

七、广域网

（一）广域网的基本概念

广域网（**Wide Area Network**，**WAN**）又称远程网，是指覆盖范围很广（几十千米到几千千米，甚至跨国、跨洲）的网络。

广域网与局域网的区别：

对比项	局域网 **LAN**	广域网 **WAN**
覆盖范围	小（一间房到几千米）	大（几十到几千千米）
主要用途	互连本单位的计算机与设备	长距离传输、实现跨地域互连
所有权	单一单位所有	通常由电信运营商拥有和运营
使用的技术层次	主要涉及物理层和数据链路层（如以太网）	主要涉及物理层、数据链路层和网络层
网络拓扑	总线、星型、环型、树型	通常为网状拓扑，结点交换机之间点对点连接
传输介质与协议	双绞线、光纤；以太网 / 802.11	光纤、卫星、公共交换网；PPP、帧中继、ATM、SDH 等
典型设备	集线器、交换机	路由器（结点交换机）
是否使用存储转发	交换机内部有存储转发，但不是链路层必备能力	结点交换机普遍采用存储转发

广域网的关键概念：广域网由许多结点交换机（实际就是路由器）以及连接这些交换机的高速点对点链路组成。结点交换机负责分组的存储转发，网络层是端到端存储转发的设计核心。

（二）PPP 协议

点对点协议（**Point-to-Point Protocol**，**PPP**）是目前使用最广泛的点对点数据链路层协议。

PPP 的两种主要应用：

1. 用户计算机通过拨号或专线方式接入 ISP（因特网服务提供商），此时 PPP 运行在用户到 ISP 之间的点对点链路上；
2. **PPPoE**（**PPP over Ethernet**）：在以太网上承载 PPP，用于宽带拨号上网，实现了传统以太网没有的身份验证、加密以及压缩等功能。

PPP 协议的组成（三个组成部分）：

1. 一种将 **IP** 数据报封装到串行链路（同步串行 / 异步串行）的方法；
2. 链路控制协议（**Link Control Protocol**，**LCP**）：用来建立、配置和测试数据链路连接，其最重要的功能之一是身份验证（PAP、CHAP）；
3. 网络控制协议（**Network Control Protocol**，**NCP**）：支持不同的网络层协议（IP、OSI 网络层、AppleTalk 等），为上层协议做配置（如 IPCP 用于分配 IP 地址）。

PPP 协议的基本要求

- 简单：这是首要的要求（最复杂的功能由 TCP 实现，IP 也简单且不可靠），数据链路层的 PPP 没有必要比 IP 复杂。
- 封装成帧：必须规定特殊的字符作为帧定界符。
- 透明性：必须保证数据传输的透明性。
- 多种网络层协议：支持多种网络层协议。
- 多种类型链路：能够在多种类型的链路上运行（串行/并行、同步/异步、低速/高速、电/光等）。
- 差错检测：对收到的帧进行检测，并丢弃有差错的帧。
- 不提供可靠传输：数据链路层的帧不需要纠错、不需要序号、也不需要流量控制。

- 不支持多点接入，不支持半双工或单工通信（只能全双工）。

PPP 不提供可靠传输的三条理由（常考简答）

1. 在数据链路层出现差错的概率不大时，使用比较简单的 PPP 协议较为合理；
2. 在因特网环境下，PPP 的信息字段中放入的是 IP 数据报，数据链路层的可靠传输并不能保证网络层的传输也是可靠的；
3. 帧检验序列 FCS 字段可保证无差错接收（有差错的帧直接丢弃），已经能满足需要。

PPP 的帧格式

PPP 帧各字段如下（按从左到右的顺序）：

字段	长度	取值与说明
标志（**Flag**）	1 B	`0x7E`，从比特角度看为 `01111110`，是 PPP 帧的定界符
地址（**Address**）	1 B	`0xFF`，预留，目前没有实际作用
控制（**Control**）	1 B	`0x03`，预留，目前没有实际作用
协议（**Protocol**）	2 B	指明帧的数据载荷应向上交付给哪个协议处理
数据（**Information**）	可变，默认 ≤ 1500 B	上层交付的数据（通常是 IP 数据报）
FCS	2 B	帧检验序列，CRC 校验
标志（**Flag**）	1 B	`0x7E`，帧结束定界符

协议字段的常见取值：`0x0021` 表示数据载荷是 **IP** 数据报；`0xC021` 表示数据载荷是 **LCP** 分组；`0x8021` 表示数据载荷是 **NCP**（**IPCP**）分组。

PPP 是面向字节的，所有的 **PPP** 帧的长度都是整数字节。

PPP 的透明传输

PPP 的透明传输方法取决于链路是同步还是异步：

- 异步传输（面向字节，单位是字符，效率较低，高速链路开销大）→ 使用字节填充（RFC 1662）。
- 同步传输（面向比特，单位是帧，收发双方时钟统一、字符间传输同步无间隔）→ 使用零比特填充。

异步链路的字节填充规则（发送方）：

1. 数据载荷中出现的每一个 `0x7E`：先减去 `0x20`（相当于异或 `0x20`，即 `0x7E` → `0x5E`），再在其前面插入转义字符 `0x7D`；
2. 数据载荷中原来就有的每一个 `0x7D`：先减去 `0x20`（`0x7D` → `0x5D`），再在其前面插入转义字符 `0x7D`；
3. 数据载荷中出现的每一个 **ASCII** 码控制字符（即数值小于 `0x20` 的字符）：先加上 `0x20`（相当于异或 `0x20`，转换成非控制字符），再在其前面插入转义字符 `0x7D`。

接收方进行与发送方相反的变换，就可以正确恢复出未经过字节填充的原始数据载荷：遇到 `0x7D` 就丢弃它，并把紧随其后的字符与 `0x20` 做异或运算。

完整算例（**PPP** 字节填充）：设 PPP 帧的数据载荷（信息字段）的十六进制字节序列为

```
0x7E  0x7D  0x03  0x41
```

按规则逐字节处理：

原始字节	命中的规则	处理后字节序列
0x7E	规则 1：异或 0x20 → 0x5E，前面插 0x7D	7D 5E
0x7D	规则 2：异或 0x20 → 0x5D，前面插 0x7D	7D 5D
0x03	规则 3：小于 0x20，加 0x20 → 0x23，前面插 0x7D	7D 23
0x41	普通字符，不处理	41

因此实际在线路上（两个标志字段之间）传输的数据载荷字节序列为

```
7D 5E   7D 5D   7D 23   41
```

加上定界符后，完整帧的字节流为

```
7E  7D 5E  7D 5D  7D 23  41  FCS 7E
```

接收方解码：遇到 7D 就丢弃它，把后一个字节异或 0x20：

- 7D 5E → 丢弃 7D，0x5E $\oplus$ 0x20 = 0x7E ✓
- 7D 5D → 0x5D $\oplus$ 0x20 = 0x7D ✓
- 7D 23 → 0x23 $\oplus$ 0x20 = 0x03 ✓

恢复出原始载荷 7E 7D 03 41，与发送方一致。

PPP 字节填充的四个易错点

1. 转义字符是 0x7D，被转义的对象是 0x7E、0x7D 本身，以及所有小于 0x20 的控制字符。
2. 异或 0x20 就是”加/减 0x20”（因为 $0x20$ 只有 1 位为 1，加减等价于异或）。
3. 标志字段 0x7E 本身不填充。
4. 别把 PPP 的异步填充（字节填充，0x7D）与前面字符填充法中的 ESC 混淆：字符填充法用的是通用 ESC，PPP 用的是 0x7D。

PPP 的工作状态（以拨号接入 ISP 为例）

1. 静止（**Link Dead**）：PPP 链路的开始和结束状态，此时用户 PC 与 ISP 拨号服务器之间不存在物理层连接；
2. 建立（**Link Establishment**）：检测到调制解调器的载波信号并建立物理层连接后进入。此时 **LCP** 开始协商一些配置选项（最大帧长、鉴别协议等）。协商成功进入” 鉴别” 状态，失败退回” 静止”；
3. 鉴别（**Authentication**）：若双方无须鉴别或鉴别成功，则进入” 网络” 状态；鉴别失败进入” 终止” 状态；
4. 网络（**Network**）：PPP 链路两端通过交换网络层特定的 **NCP** 分组进行 NCP 配置。若上层是 IP 协议，则使用 **IP** 控制协议（**IPCP**）配置每一端的 IP 模块，例如分配 **IP** 地址。NCP 配置完成后进入” 打开” 状态；
5. 打开（**Link Open**）：双方可以进行数据通信；

6. 终止（**Link Termination**）：出现故障或一端发出终止请求时进入；载波停止后回到” 静止” 状态。

可见，**PPP** 协议已不是纯粹的数据链路层协议，它还包含了物理层和网络层的内容。

PPP 的差错检测：接收方每收到一个 PPP 帧就进行 CRC 检验，正确就收下，否则丢弃。使用 **PPP** 的数据链路层向上提供的是不可靠数据传输服务。PPP 不使用序号和确认机制。

PPP 与以太网的对比（易出选择题）

对比项	**PPP**	以太网 **V2**
服务类型	面向连接的点对点（需 LCP 建链）	无连接、无确认
提供可靠传输	不提供（无序号、无确认、无流量控制）	不提供
帧定界	0x7E 标志字段	前导码 + 帧间间隔（无帧结束符）
透明传输	异步用字节填充（0x7D），同步用零比特填充	不需要（帧内无定界符）
差错检测	CRC（FCS，2 B）	CRC（FCS，4 B）
最小/最大帧长	无 64 B 最小帧长限制	64~1518 B

八、数据链路层设备

（一）网桥与以太网交换机

网桥（**Bridge**）工作在数据链路层（包含其下的物理层），因此具备数据链路层范畴的能力：

- 可以识别帧的结构；
- 可以根据帧首部中的目的 **MAC** 地址和网桥自身的帧转发表来转发或丢弃所收到的帧。

网桥的接口数量很少（通常 2~4 个），一般只用来连接不同的网段。1990 年面世的交换式集线器（**Switching Hub**）实质上是具有多个接口的网桥，常称为以太网交换机（**Switch**）或二层交换机（” 二层” 指它工作在数据链路层，包括物理层）。

仅使用交换机（不使用集线器）的以太网就是交换式以太网。

交换机的本质

以太网交换机本质上就是一种多端口网桥。交换机的自学习和转发帧流程与网桥完全相同。另外，交换机也使用生成树协议 **STP**，来产生能够连通全网但不产生环路的通信路径。

（二）自学习算法与转发表的建立

透明网桥（**Transparent Bridge**）的标准是 **IEEE 802.1D**，它通过自学习算法建立转发表。” 透明” 是指以太网中的各站点并不知道自己所发送的帧将会经过哪些网桥的转发，即各网桥对各站点是” 看不见的”。

自学习与转发的完整流程：

1. 登记（自学习）：网桥（交换机）每收到一个帧，就记录该帧的源 **MAC** 地址和进入网桥的接口号，写入转发表。若表中已有该 MAC 地址的记录，则更新接口号和有效时间。
2. 查表转发：根据帧的目的 **MAC** 地址查找转发表，分三种情况：
   - 明确转发：表中能查到目的 MAC 地址对应的接口，且该接口不是入接口 → 从该接口转发

出去；

- 盲目转发（泛洪）：表中查不到目的 MAC 地址 → 从除进入接口外的其他所有接口转发出去；
- 丢弃（过滤）：表中查到目的 MAC 地址对应的接口就是入接口 → 说明目的站与本帧的来向在同一网段，不需要转发，直接丢弃。

交换机的三条特殊处理规则

1. 交换机收到有误码的帧（FCS 校验失败）→ 直接丢弃；
2. 交换机收到一个无误码的广播帧 → 不用查表，直接从除接收接口外的其他所有接口转发；
3. 转发表中的每条记录都有有效时间，到期自动删除（因为各站点的 MAC 地址与交换机接口的对应关系不是永久的，例如某站更换了网卡，其 MAC 地址就变了）。

完整算例（交换机自学习过程）

设交换机 S 有 4 个接口 1、2、3、4，分别连接主机 A、B、C、D，初始转发表为空。按时间顺序发生如下通信：

时刻	事件	自学习（登记）	转发决策
t_1	A 向 B 发送帧	表中记入 **A** → 接口 **1**	查不到 B → 盲目转发，从接口 2、3、4 发出
t_2	B 向 A 回帧	表中记入 **B** → 接口 **2**	查到 A 在接口 1 且 $1 \neq 2$ → 明确转发，只从接口 1 发出
t_3	C 向 D 发送帧	表中记入 **C** → 接口 **3**	查不到 D → 盲目转发，从接口 1、2、4 发出
t_4	D 向 C 回帧	表中记入 **D** → 接口 **4**	查到 C 在接口 3 → 明确转发，只从接口 3 发出
t_5	A 又向 B 发送帧	更新 **A** → 接口 **1** 的有效时间	查到 B 在接口 2 → 明确转发，只从接口 2 发出

从此刻起，转发表已完整，交换机进入”明确转发”状态，不再泛洪。

2014 年题 34（交换机转发端口判断）

”某以太网拓扑及交换机当前转发表如下图所示，主机 00-e1-d5-00-23-a1 向主机 00-e1-d5-00-23-c1 发送 1 个数据帧，主机 00-e1-d5-00-23-c1 收到该帧后，向主机 00-e1-d5-00-23-a1 发送 1 个确认帧，交换机对这两个帧的转发端口分别是……”

做法：第一个帧查表中是否有目的地址 c1 的记录 → 有则明确转发，无则泛洪；同时交换机先自学习（登记源地址 a1 与入接口）。第二个帧（确认帧）查表中是否有 a1 的记录 → 此时表中已有刚才学到的 **a1** → 入接口的记录，因此如果 a1 的入接口就是该确认帧的入接口，则丢弃，否则明确转发。

（三）转发/过滤/泛洪与冲突域、广播域的划分

冲突域（**Collision Domain**）：可能发生信号碰撞的区域，同一冲突域内的站点共享带宽，需要介质访问控制协调。

广播域（**Broadcast Domain**）：一个站点发出的广播帧所能到达的区域。

设备	工作层次	是否隔离冲突域	是否隔离广播域
中继器 / 集线器（**Hub**）	物理层	否（所有接口同属一个冲突域）	否

设备	工作层次	是否隔离冲突域	是否隔离广播域
网桥 / 以太网交换机	数据链路层	是（每个接口是一个独立的冲突域）	否（所有接口同属一个广播域，除非划分 VLAN）
路由器	网络层	是（每个接口一个冲突域）	是（每个接口一个广播域）

关键结论：

- 集线器只工作在物理层，每个接口仅简单地转发比特，并不进行碰撞检测。使用集线器的以太网物理拓扑是星型，但逻辑上仍是总线网，各站点共享总线资源，仍然使用 **CSMA/CD** 协议。碰撞检测的任务由各站点中的网卡负责。
- 交换机每个接口构成一个独立的冲突域，因此交换式以太网中通常不需要使用 **CSMA/CD**（接口与计算机或交换机连接时可以工作在全双工方式，并能在内部同时连通多对接口，使每一对相互通信的计算机都像独占传输媒体那样无碰撞地传输数据）。
- 当交换机的接口连接的是集线器时，该接口就只能使用 **CSMA/CD** 协议并只能工作在半双工方式。现在的交换机和网卡都能自动识别这两种情况并自动切换。
- **VLAN** 可以把一个广播域分割成多个广播域，这是 VLAN 相对于普通交换机最重要的价值。
- 交换式以太网的网络性能远高于共享式以太网，集线器早已被交换机取代。

2020 年题 35 / 2022 年题 47（冲突域与广播域计数）

计数规则：

- 冲突域个数 = 集线器” 整块” 算 1 个（集线器及其所有直连链路合起来是一个冲突域）+ 交换机每个接口算 1 个 + 路由器每个接口算 1 个；
- 广播域个数 = 路由器每个接口算 1 个 + 交换机（无 VLAN 时）整体算 1 个 + 集线器并入其所属的域。

2022 年题 47 变形：题目给出”H1 与 H2 属于同一个广播域，但不属于同一个冲突域；H2 和 H3 属于同一个冲突域”，则设备 **1** 应是以太网交换机（同广播域不同冲突域），设备 **2** 应是集线器 **Hub**（同冲突域）。

（四）直通式与存储转发式交换

对比项	存储转发（**Store-and-Forward**）	直通交换（**Cut-Through**）
工作方式	把整个帧先全部缓存，检查 FCS 无误后再按目的 MAC 转发	在接收帧的同时就立即按目的 MAC 地址决定转发接口，通过基于硬件的交叉矩阵转发，不必缓存整个帧
转发时延	较大（需收完整个帧）	非常小（只需收到目的 MAC 字段即可开始转发）
差错检查	检查，无效帧不转发	不检查差错，可能把无效帧转发出去
适用场合	通用	对时延敏感、链路质量好的场合

直通交换的时延计算：采用直通交换时，交换机只需要读取到帧的目的 **MAC** 地址字段（**6 B**）就能决定转发接口，因此最小转发时延 = 接收 **6 B** 的时间（前导码不计）。

完整算例（**2013** 年题 **38**）：对于 100 Mbps 的以太网交换机，当输出端口无排队，以直通交换方式转发一个以太网帧（不包括前导码）时，引入的转发延迟至少是多少?

$$\frac{6 \times 8 \text{ bit}}{100 \times 10^6 \text{ b/s}} = 0.48\ \mu s$$

答案 **0.48** μ**s**。

直通交换时延题的常见错误

1. 误用整帧长度（如 64 B 或 1518 B）计算 → 那得到的是存储转发的时延。
2. 把前导码 8 B 也算进去 → 题目通常明确说” 不包括前导码”。
3. 记错” 直通只需读到目的 MAC（6 B）” 这一点：源 **MAC**、类型字段都还没读完就可以开始转发。

（五）交换机 vs 集线器 vs 路由器

对比项	集线器 **Hub**	以太网交换机 **Switch**	路由器 **Router**
工作层次	物理层	数据链路层（含物理层）	网络层
转发依据	无（直接转发比特）	目的 **MAC** 地址	目的 **IP** 地址
是否隔离冲突域	否	是（每接口一个）	是（每接口一个）
是否隔离广播域	否	否（除非划分 VLAN）	是
工作方式	半双工，共享带宽	全双工（连终端时），独享带宽	全双工
是否使用 CSMA/CD	是（由网卡负责检测）	连终端时不用；连集线器时用	不用
是否有存储转发	无	有（可直通）	有
能否连接不同网络层协议的网络	否	否（只处理 MAC）	能
转发时延	最小	较小	较大
典型用途	扩展端口数量	组建交换式以太网、划分冲突域	连接不同网络、隔离广播域

2015 年题 **37**（交换机的正确叙述）

” 下列关于交换机的叙述中，正确的是（ ）” A. 以太网交换机本质上是一种多端口网桥 ✓ B. 通过交换机互连的一组工作站构成一个冲突域 ×（每个端口是一个独立冲突域）C. 交换机每个端口所连网络构成一个独立的广播域 ×（不划分 VLAN 时全网一个广播域）D. 以太网交换机可实现采用不同网络层协议的网络互联 ×（那属于网络层范畴，路由器才能做）答案 **A**。

本章小结表

知识点	核心公式 / 结论	常考形式
帧定界	HDLC 标志 `01111110`；PPP 标志 `0x7E`	零比特填充选择题
零比特填充	发送方每 5 个连续 1 后插 0；接收方每 5 个连续 1 后删 0	2013 年题 37
字符填充	`SOH/EOT` 前插 `ESC`；`ESC` 本身前也插 `ESC`	写填充后的比特/字符序列
奇偶校验	检奇数位错，漏检偶数位错；不能纠错	判断检错能力
CRC	求余数：数据后补 $k-1$ 个 0，模 2 除以 $G(X)$，余数位数 = $G(X)$ 最高次	2023 年题 37 判断哪个接收串无错
海明码	$2^k \geq n+k+1$；校验位放 1、2、4、8⋯；指误字值 = 出错位号	编码、纠错、求最小海明距离

知识点	核心公式 / 结论	常考形式
海明距离	检 e 位：$d \geq e+1$；纠 t 位：$d \geq 2t+1$；检 s 纠 t：$d \geq t+s+1$	选择题
窗口约束	$W_T + W_R \leq 2^n$	2019 年题 35
停等	$W_T = W_R = 1$，序号 1 bit，$U = \frac{T_D}{T_D+RTT+T_A}$	2018 年题 36 求帧长
GBN	$W_R = 1$，$1 < W_T \leq 2^n - 1$，累积确认，错一帧重传一串	2009 年题 35、2014 年题 36
SR	$W_T = W_R \leq 2^{n-1}$，逐帧确认，缓存失序帧，只重传出错帧	2011 年题 35、2024 年题 37
GBN 有效速率	$R_{有效} = \frac{k \cdot L}{T_D+RTT+T_A}$，$k \leq W_T$	应用题
FDM/TDM/WDM/OFDM	频段/时隙/波长划分；STDM 按需分配	概念选择题
CDMA	内积 >0→1，<0→0，=0→ 未发送；$\vec{S} \cdot \vec{T} = 0$	2014 年题 37（101）
纯 ALOHA	$S_{max} = 0.184 = \frac{1}{2e}$	概念/数值题
时隙 ALOHA	$S_{max} = 0.368 = \frac{1}{e}$（是纯 ALOHA 的 2 倍）	概念/数值题
CSMA	1-坚持、非坚持、p-坚持；仅靠监听不能避免冲突	特点比较
CSMA/CD	争用期 2τ；$L_{min} = 2\tau R$；10 Mb/s 下 512 bit = 64 B	2009/2016/2019/2022 计算题
二进制指数退避	$K = \min(k, 10)$，退避时间 $= r \times 2\tau$，$r \in [0, 2^K - 1]$；16 次失败放弃	2023 年题 36（768 μs）
CSMA/CA	SIFS=28 μs、DIFS=128 μs；RTS/CTS；NAV 虚拟载波监听	2018/2020/2024 题
令牌传递	无冲突；令牌保持时间防止独占	概念题
IEEE 802	802.3 以太网、802.4 令牌总线、802.5 令牌环、802.11 无线；LLC + MAC	标准对应关系
MAC 地址	48 位 = 3 B OUI + 3 B；全 1 广播 FF-FF-FF-FF-FF-FF	2013 年题（长度）
以太网帧	首尾共 18 B；最短 64 B（数据 ≥46 B）；最长 1518 B（数据 ≤1500 B）；前导码 8 B 不计	2010/2012 帧长计算
802.11 帧	帧头 30 B + 数据 + FCS 4 B；4 个地址；From DS 时” 地址 1 终点、2 是 AP、3 是起点”	2017/2022 题
VLAN	802.1Q 插 4 B 标签；TPID=0x8100；VID 12 bit，最多 4094 个；Access/Trunk	2024 年题 35
PPP	标志 0x7E；转义 0x7D，异或 0x20；LCP/NCP；不提供可靠传输	字节填充计算题
交换机	自学习（源 MAC + 入接口）；明确转发/泛洪/丢弃；每端口一个冲突域，不隔离广播域	2014/2015/2016/2021 题
直通交换	只需读到目的 MAC（6 B）即可转发	2013 年题 38（0.48 μs）

易错清单

一、组帧与差错控制

1. 零比特填充是”**5** 个连续 **1** 后插 **0**”，不是”**6** 个”。HDLC 的标志是 6 个 1（01111110），但填充阈值是 5，正因如此填充后才不会出现 6 个连续 1。
2. **CRC** 的余数位数必须等于生成多项式的最高次数，位数不够要在前面补 **0**。例如 $G(X) = X^3 + X^2 + 1$

时余数必须是 3 位，算出 1 要写成 001。

3. 模 **2** 除法不借位、不比较大小，就是按位异或。”减”和”加”在模 2 运算里是一回事。
4. **CRC** 只能检错不能纠错，也不能定位错误位置。别把 CRC 与海明码混为一谈。
5. **FCS** $\neq$ **CRC**。FCS 是附加的冗余码，CRC 是产生 FCS 的一种方法。
6. 奇偶校验漏检偶数个比特错误，与”错哪几位”无关，只与”错几个”有关。
7. 海明码的校验位必须放在 **1**、**2**、**4**、**8**…（2^i）位置，不是放在最前面或最后面。
8. 海明码只能纠 **1** 位错。2 位错时指误字虽然非 0，但指向错误位置，硬纠会越纠越错。
9. 最小海明距离三个公式别记混：检错 $e+1$、纠错 $2t+1$、检 s 纠 t 是 $t+s+1$。

二、流量控制与滑动窗口

1. $W_T + W_R \leq 2^n$ 是所有滑动窗口协议的总约束，由此才有 GBN 的 $W_T \leq 2^n - 1$ 与 SR 的 $W_T = W_R \leq 2^{n-1}$。
2. **GBN** 是累积确认：收到 `ACK3` 就意味着 `0,1,2,3` 全部被确认，别再去找 `ACK2`。
3. **GBN** 的接收窗口恒为 **1**，收到失序帧就丢弃并重复发送最后一个按序确认。
4. **SR** 的接收窗口大于 **1**，可以缓存失序帧；而且只有按序收齐后接收窗口才滑动。
5. **SR** 收到未按序的确认时要记录，但发送窗口不滑动。
6. 停等协议的序号只需 **1 bit**；数据链路层的停等协议可以不给确认分组编号（因为链路层几乎不会出现确认迟到），运输层则必须编号。
7. **”** 确认迟到**”** 与**”** 确认丢失**”** 是两种不同情形：前者要求给确认编号，后者要求给数据编号。
8. 有效数据传输速率 $\neq$ 信道利用率 $\times$ 带宽，除非题目明确按整帧长度算利用率。
9. 判断窗口是否成为瓶颈：比较 $W_T T_D$ 与 $T_D + RTT + T_A$，前者大则链路能被占满（$U=1$），前者小则窗口是瓶颈。

三、介质访问控制

1. 纯 **ALOHA 0.184**、时隙 **ALOHA 0.368**，后者是前者的 2 倍，不要写反。
2. **CSMA** 只用载波监听仍会冲突（传播时延导致多个站同时认为信道空闲），这是 CSMA 与 CSMA/CD 的分界线。
3. $L_{min} = 2\tau R$，其中 2τ 是往返争用期，τ 是单向时延。求最大距离时注意除以 2 的关系：$\Delta d = \dfrac{v\,\Delta L_{min}}{2R}$。
4. **CSMA/CD** 只能半双工（要边发边听）；全双工下不需要 **CD**，因此交换式以太网、光纤以太网、千兆/万兆以太网全双工时不使用 CSMA/CD。
5. 二进制指数退避中 $K = \min(\text{重传次数}, 10)$，退避时间的单位是争用期 2τ。第 4 次冲突时 r 从 $\{0, \ldots, 15\}$ 取，最长 $15 \times 51.2 = 768\ \mu s$。
6. 重传 **16** 次仍失败就放弃并向上层报告，这个数字要记牢。
7. **CSMA/CA** 的 **NAV** 不含正在传输的 **CTS** 帧本身，但含数据帧前的 **SIFS**、数据帧、其后的 **SIFS** 和 **ACK**。
8. **SIFS=28** μ**s** 是**”** 分隔一次对话内部各帧**”** 用的（ACK、CTS），**DIFS=128** μ**s** 用于竞争发送数据帧和管理帧。首次竞争用 DIFS，一次交换内部各帧之间用 SIFS。
9. **CSMA/CA** 不一定要 **RTS/CTS**，基础模式只做载波监听 + 退避。
10. **CSMA/CD** 提供不可靠传输，**CSMA/CA** 必须提供确认机制（可靠传输）。

四、局域网、广域网与设备

1. 前导码 **8 B** 不属于 **MAC** 帧，计算帧长时不要加进去。
2. **MAC** 帧最短 **64 B**、最长 **1518 B**；数据字段 **46~1500 B**；首部 + 尾部 **18 B**。
3. **MAC** 地址是**”** 接口的唯一标识**”**，不是**”** 设备的唯一标识**”**；交换机和路由器有多个接口，因此有多个 MAC 地址。
4. **MAC** 地址虽叫物理地址，但属于数据链路层。
5. 集线器不隔离冲突域也不隔离广播域；交换机隔离冲突域但不隔离广播域；路由器两者都隔离。

6. **VLAN** 的 **VID** 是 **12** 位，共 **4096** 个编号，去掉 **0** 和 **4095** 保留值，实际可用 **4094** 个。
7. **802.1Q** 标签插在源地址与类型字段之间，共 4 B，使最长帧变为 1522 B。
8. **802.11** 帧中必须携带 **AP** 的 **MAC** 地址，以太网帧则不需要（AP 对以太网站点是透明的）。
9. **From DS** 帧的地址顺序：地址 1 = 目的站、地址 2 = AP、地址 3 = 源站。口诀” 去往 AP 中起止，来自 AP 止中起”。
10. **PPP** 不提供可靠传输（无序号、无确认、无流量控制），只做 CRC 检错 + 丢弃。
11. **PPP** 字节填充用 `0x7D`，被转义对象是 `0x7E`、`0x7D` 和所有小于 `0x20` 的控制字符，处理方法是异或 `0x20`。
12. **PPP** 是面向字节的，所有 **PPP** 帧长度都是整数字节；PPP 只能全双工，不支持多点接入。
13. 交换机自学习登记的是**”** 源 **MAC** 地址 + 入接口**”**，不是目的 MAC 地址。
14. 广播帧到达交换机时不查表直接泛洪；有误码的帧直接丢弃。
15. 直通交换只需读到目的 **MAC**（**6 B**）即可转发，所以 100 Mbps 下最小时延是 0.48 μs。
16. 交换机不能实现不同网络层协议网络的互联（那是路由器的事），也不能隔离广播域（除非划 VLAN）。

第二十二章网络层

本章考纲要求

四、网络层（一）网络层的功能

1. 异构网络互连
2. 路由与转发
3. SDN 基本概念
4. 拥塞控制

（二）路由算法

1. 静态路由与动态路由
2. 距离-向量路由算法
3. 链路状态路由算法
4. 层次路由

（三）IPv4

1. IPv4 分组
2. IPv4 地址与 NAT
3. 子网划分、路由聚集、子网掩码与 CIDR
4. ARP 协议、DHCP 协议与 ICMP 协议

（四）IPv6

1. IPv6 的主要特点
2. IPv6 地址

（五）路由协议

1. 自治系统
2. 域内路由与域间路由
3. RIP 路由协议
4. OSPF 路由协议
5. BGP 路由协议

（六）IP 组播

1. 组播的概念
2. IP 组播地址

（七）移动 IP

1. 移动 IP 的概念

2. 移动 IP 通信过程

（八）网络层设备

1. 路由器的组成和功能
2. 路由表与分组转发

网络层是 408 计网中分值最高、计算题最多的一章。选择常考首部字段、特殊地址、子网与 CIDR、ARP/ICMP/DHCP、路由协议对比；综合大题几乎必考子网划分、路由聚合、路由表转发、IP 分片中的一到两项。本章对五类必考计算（IP 分片、子网划分、CIDR 聚合、路由表转发、RIP 更新）都给出完整算例。

一、网络层的功能

网络层提供主机到主机的通信服务，任务是把分组经过多个网络和多段链路从源主机传到目的主机，可分解为路由选择（走哪条路径）与分组转发（分组到达时做什么动作），传输单位是数据报。TCP/IP 的网络层向上只提供无连接的、尽最大努力交付的数据报服务：分组可能出错、丢失、重复、失序或超时，可靠性由运输层负责。

对比方面	虚电路服务	数据报服务
核心思想	可靠通信由网络自身保证	可靠通信由用户主机保证
连接	必须先建立网络层连接	不需要建立连接
目的地址	仅连接建立阶段使用，之后只用虚电路号	每个分组都携带完整目的地址
分组转发	同一条虚电路的分组走同一路由	每个分组可独立走不同路由
分组顺序	总是按发送顺序到达	不一定按序到达
结点故障	经过该结点的所有虚电路被破坏	可另选路径，适应能力强
首部开销	小	大

虚电路不等于可靠

虚电路能保证分组按序到达，但并不天然可靠。链路故障或拥塞丢包时网络层虚电路本身不重传：协议附加了确认与重传（如 X.25）才可实现可靠传输；无重传机制（如帧中继、ATM）则只是” 面向连接但不可靠”。

（一）异构网络互连

异构网络指因特网中数以百万计的网络，其拓扑结构、寻址方案、性能以及所用协议都不尽相同。互连需要解决不同网络的接入机制、差错恢复方法、路由选择技术、寻址方案、最大分组长度、服务类型等问题。

网络互连通常指用路由器进行网络连接和路由选择。路由器是一台专用计算机（历史上文献也把网络层的路由器称为网关）。中继系统按层次分为四类：

中继系统层次	设备	互连效果
物理层	转发器、集线器	只是把一个网络扩大了，从网络层看仍是同一个网络
数据链路层	网桥、交换机	同上，仍属于同一个网络

中继系统层次	设备	互连效果
网络层	路由器	真正的网络互连，各网络在网络层仍彼此独立
网络层以上	网关	应用层互连

因此只用物理层或数据链路层的中继系统不叫网络互连——它们没有把”网络”这个层次连接起来，只是扩大了同一个网络的规模。

虚拟互连网络（又称逻辑互连网络、**IP** 网络）：各种物理网络的异构性本来客观存在，但通过 **IP** 协议可使这些性能各异的网络在网络层上看起来像一个统一的网络。

为什么用 IP 而不是 MAC 地址统一寻址

若只用 MAC 地址通信，每台路由器的路由表就必须记录所有主机和路由器各接口的 **MAC** 地址：手工配置不可能完成；自动构建也会因路由信息含海量 MAC 地址而严重占用通信资源，还需极大存储空间与查表时延。用 **IP** 地址按网络号寻址，路由表只需记录部分网络的网络地址。这是两种地址并存的原因。

（二）路由与转发

- 路由选择（**routing**）：根据路由协议构造路由表，经常或定期地与相邻路由器交换信息，动态更新维护路由表以决定最优路径。涉及很多路由器。
- 分组转发（**forwarding**）：路由器根据转发表把收到的 IP 数据报从合适的端口转发出去。仅涉及一个路由器。

	路由表	转发表
来源	由路由选择算法生成	由路由表导出
内容	目的网络、下一跳，还存放跳数、链路状态等算法参数	只保留转发必备条目，剔除算法相关附加信息
优化目标	最优化网络拓扑变化的计算	最优化查找过程
实现	一般用软件	可用软件也可用硬件

（三）SDN 基本概念

软件定义网络（Software Defined Network，**SDN**）采用集中式的控制平面和分布式的数据平面（转发平面），两个平面相互分离。控制平面利用控制-数据接口对数据平面上的路由器进行集中式控制。

传统路由器既有转发表又有路由选择软件（即既有数据平面又有控制平面），路由器之间传送主机间的分组和路由信息。SDN 体系结构中路由器中的路由软件都不存在了，因此路由器之间不再交换路由信息。

- 控制平面中有一个在逻辑上集中的远程控制器，物理上可由不同地点的多个服务器组成。
- 远程控制器掌握各主机和整个网络的状态，为每个分组计算出最佳路由并为每个路由器生成转发表。路由器的工作很单纯：收到分组、查转发表、转发分组。
- 核心思想：把控制层面和数据层面分离，让控制层面利用软件来控制数据层面的设备，网络又变成集中控制的。

OpenFlow 协议是 SDN 体系结构中控制层面与数据层面之间的通信接口。传统数据层面的任务是

按转发表转发分组，分两步：匹配（查转发表中的网络前缀，做最长前缀匹配）与动作（从匹配结果指明的接口转发）。SDN 的广义转发范围大大扩展：匹配可对各层（数据链路层、网络层、运输层）首部字段进行；动作不仅转发，还可负载均衡、重写 **IP** 首部（类似 NAT）、阻挡或丢弃分组（类似防火墙）。因此完成” 匹配 + 动作” 的设备不再称路由器，而称”**OpenFlow** 交换机” 或分组交换机；取代转发表的是” 流表（Flow Table）”。

- 一个流是穿过网络的一种分组序列，序列中每个分组都共享首部某些字段的值（例如相同源、目的 IP 地址的一连串分组）。
- 流表由 **SDN** 远程控制器管理，控制器通过安全信道用 OpenFlow 协议管理交换机的流表。
- 每个交换机必须有一个或多个流表，每个流表可含多个流表项。
- 每个流表项含三个字段：首部字段值（匹配字段）、计数器、动作。匹配不上的分组被丢弃或送往远程控制器。OpenFlow 交换机既处理链路层帧，也处理 IP 数据报与 TCP/UDP 报文。
- 计数器记录已匹配的分组数量和该流表项上次更新至今的时间。
- 动作可转发到指明端口、丢弃、复制后从多端口转发、重写各层首部。

SDN 体系结构的接口：

接口	连接对象	作用
北向接口	面向上层应用开发者	提供丰富 API，不必关心底层硬件细节
南向接口	控制器与转发设备之间	建立双向会话；典型协议是 **OpenFlow**，可兼容不同硬件
东西向接口	控制器集群内部控制器之间	增强控制平面的可靠性和可扩展性

SDN 的四个关键特征：基于流的转发；数据层面与控制层面分离；网络控制功能位于分组交换机之外；可编程的网络。

优点：全局集中式控制与分布式高速转发并存；可用自动化工具编程配置；采用开放接口协议后设备制造与功能软件开发相分离，降低成本。问题：集中管理易受攻击，崩溃则全网受影响；规模扩大后控制器可能成为性能瓶颈。

真题考法

2022 年第 37 题考”SDN 控制器向数据平面的 SDN 交换机下发流表时使用的接口”，答案是南向接口。北向接口面向上层网络应用，东西向接口用于控制器之间协同。

（四）拥塞控制

拥塞指因出现过量的分组而引起网络性能下降的现象。判断方法是观察吞吐量与网络负载的关系：若负载增加而吞吐量明显小于正常值，则进入轻度拥塞；若吞吐量随负载增大而下降，则已进入拥塞。

拥塞控制要解决如何获取网络中发生拥塞的信息并据此控制，避免因拥塞而出现分组丢失，作用是确保网络能够承载所达到的流量：

- 开环控制：设计网络时事先把拥塞因素考虑周到，力求不产生拥塞，是静态的预防方法，一旦启动中途不再修改，做决定时不考虑当前网络状态。手段包括确定何时可接收新流量、何时可丢弃分组及丢弃哪些分组、确定何种调度策略。
- 闭环控制：事先不考虑拥塞因素，用监测系统监视，检测到哪里发生拥塞后把信息传到合适的地方以调整网络运行，基于反馈环路，是动态的方法。

与流量控制的区别：流量控制指发送方与接收方之间的点对点通信量控制，目的是抑制发送方速率，使接收方来得及接收；拥塞控制关注整个网络的负载，是全局性的。

二、路由算法

（一）静态路由与动态路由

路由算法的目的是找到从源路由器到目的路由器的”最佳”路径（通常是费用最低的路径）。所有最短路径算法都依赖一个性质：两点之间的最短路径也包含了路径上其他顶点间的最短路径。

静态路由选择	动态路由选择
采用人工配置方式添加网络路由、默认路由、特定主机路由	路由器通过路由选择协议自动获取，路由器间彼此交换信息并按算法优化出路由表项
简单、开销小，但不能及时适应网络状态变化	较复杂、开销较大，但能较好地适应网络状态变化
一般只在小规模网络采用	适用于大规模网络

默认路由用特殊前缀 `0.0.0.0/0` 表示：`0.0.0.0` 表示任意网络，`/0`（掩码 `0.0.0.0`）是最短网络前缀。全 0 掩码与任何目的地址按位与必为全 0，因此必然匹配。只要目的网络不在转发表中，就一律选择默认路由，通常用于路由器到互联网的路由。由于最长前缀匹配，默认路由匹配优先级最低。

特定主机路由对某台主机的 IP 地址专门指明路由，便于管理员控制与测试网络。若主机地址是 a.b.c.d，则目的网络是 `a.b.c.d/32`，掩码 `255.255.255.255` 是最长前缀，因此匹配优先级最高。

路由匹配优先级：特定主机路由 > 最长前缀子网路由 > 默认路由。

静态路由配置的陷阱

可能出现两类问题：路由条目配置错误，甚至导致路由环路；聚合路由条目时可能引入不存在的网络（聚合后的地址范围大于原子网并集，把未分配地址也通告出去）。

（二）距离-向量路由算法

距离-向量算法是分散型的：路由器只掌握物理相连的邻居及链路费用，通过反复与邻居交换信息逐步收敛，其理论基础是 **Bellman-Ford** 算法。设 `d_x(y)` 为结点 x 到 y 的最短路径费用，`c(x,v)` 为 x 到邻居 v 的费用，则

$$d_x(y) = \min_v \{\, c(x,v) + d_v(y) \,\}$$

其中 **v** 取遍 **x** 的所有邻居。每个结点维护三组信息：到每个直连邻居的链路费用 `c(x,v)`；自己的距离向量；收到的每个邻居的距离向量。

算法流程：每个结点定期向每个邻居发送距离向量副本。当 x 从邻居 v 收到新距离向量时，先保存它，再用 Bellman-Ford 公式更新自己的距离向量；若自己的距离向量因此改变，则继续向每个邻居发送更新后的距离向量。

收敛小例子：三结点 x、y、z，`c(x,y)=2`、`c(x,z)=7`、`c(y,z)=1`。初始化时各结点只知道自己到直连邻居的费用：`d_x=(0,2,7)`、`d_y=(2,0,1)`、`d_z=(7,1,0)`。第一次交换后 x 计算 `d_x(y)=min{2+0, 7+1}=2`、`d_x(z)=min{2+1, 7+0}=3`，得 `d_x=(0,2,3)`。再次交换后若无变化则不再发送更新报文，算法进入静止状态（收敛）。

更新报文的规模

更新报文大小与网络中结点数目成正比，大型网络将导致很大的更新报文，这是距离-向量算法可扩展性差的根本原因。最常见的距离-向量路由算法是 **RIP**，采用” 跳数” 作为距离度量。

*1.*路由环路与坏消息传得慢

距离-向量算法有一个固有问题：好消息传得快，坏消息传得慢，又称路由环路或距离无穷计数问题。

环路如何形成：B 到网络 N 的距离本是 1。当 N 真正不可达时，B 又可能从 A 那里学到”N 距离 2”，算得 2+1=3 认为经 A 可到 N；A 再从 B 学到 3、算得 4……两个路由器互相” 欺骗” 对方” 你再加一跳就能到达”，距离在二者之间逐次加 1，直到 16 才停止。

缓解措施：

- 限制最大 **RIP** 距离为 **15**（16 表示不可达），使无穷计数在有限步内终止；
- 触发更新：路由表变化时立即发送更新报文，而不仅是周期性发送；
- 水平分割：记录收到某条路由信息的接口，不让同一信息再从该接口反向发回；
- 毒性逆转：向学到此路由的那个接口通告时，把距离直接置为 **16** 而不是不发。这比水平分割更彻底——不发报文时对方可能仍持旧信息，显式通告 16 能让对方立即删除该路由。

措施只能缓解，不能彻底解决

距离-向量算法中每个路由器都缺少到目的网络整个路径的完整信息，无法判断所选路由是否出现环路。这与链路状态算法中每台路由器掌握全网拓扑形成根本差别。

（三）链路状态路由算法

链路状态（Link State，LS）指本路由器都和哪些路由器相邻，以及相应链路的代价。链路状态算法是全局性的：所有路由器掌握完整的网络拓扑和链路费用信息。

主要思想：每个结点都具有全网拓扑结构图（全网一致）；主动测试所有相邻结点的状态并定期把链路状态传播给所有其他结点；每个结点用 **Dijkstra** 最短路径算法计算到其他结点的最短路径。结点每收到一个链路状态报文就更新自己的” 视野图”，一旦链路状态变化就用 Dijkstra 重新计算。

Dijkstra 执行要点：以本路由器为源点，维护” 已确定最短路径的结点集合”，每次从集合外选出当前距离最小的结点加入集合，并用它松弛其所有邻居的距离，时间复杂度 $O(n^2)$（n 为结点数）。由于各路由器使用同样的链路状态数据独立计算，算出的路径必然一致，因此从算法上保证了不会产生路由环路。

优点：各结点用同样的链路状态数据独立计算路径，不依赖中间结点的计算，易于查找故障；链路状态报文仅运载单个结点关于直接链路的信息，大小与网络结点数目无关，所以规模可伸展性更好，适用于大型或路由信息变化剧烈的互联网环境。

两类算法对比：距离-向量算法中每个结点仅与直接邻居交谈，向邻居发送自己的整个路由表，报文大小取决于结点数目，代价较大；链路状态算法中每个结点通过洪泛与所有其他结点交谈，但只告诉它们与自己直接相连的链路费用。

（四）层次路由

因特网规模非常大，且许多单位不愿让外界了解自己网络的布局细节，因此采用分层次的路由选择协议，特点有三个：自适应（动态路由选择，能较好适应网络状态变化）；分布式（各路由器通过相互间信息交互共同完成路由信息的获取与更新）；分层次（把整个因特网划分为许多较小的自治

系统（Autonomous System，**AS**），内部和外部采用不同类别的路由选择协议）。

自治系统指在单一的技术管理下的一组路由器，这些路由器使用一种 AS 内部的路由选择协议和共同的度量确定 AS 内的路由，同时使用一种 AS 之间的路由协议确定 AS 之间的路由。一个 AS 内的所有网络都属于一个行政单位管辖，AS 内所有路由器必须连通。层次路由的效果是：路由器只知道自己所在区域内的目标地址如何路由，对其他区域的内部结构不知情，路由表规模因此大幅减小。

区域划分的代价与收益

分层次划分区域使交换信息的种类增多、OSPF 协议更复杂，但能使每个区域内部交换路由信息的通信量大大减少，从而让 OSPF 可用于规模很大的自治系统。这是典型的” 用复杂度换可扩展性”。

三、IPv4

（一）IPv4 分组

IPv4 分组由首部和数据部分组成。首部前一部分长度固定，共 **20 B**，是所有 **IP** 分组必须具有的；后面是可选字段，长度可变（0 ~ 40 B），实际很少使用；当首部长度不是 4 字节整数倍时用填充补齐全 0 字节。

三个” 基本单位” 必须背熟

首部长度以 **4 B** 为单位，总长度以 **1 B** 为单位，片偏移以 **8 B** 为单位。口诀” 一种八片首饰”。IP 首部前两个字节往往以 `0x45` 开头，解题时可用于定位 IP 数据报的开始位置。

首部各字段逐项说明：

字段	位数	含义与要点
版本	4	IP 版本，IPv4 中该字段值是 4。通信双方的版本必须一致
首部长度	4	以 **4 B** 为单位。最小取值 0101（5），乘 4 得 20 B，表示只有固定部分；最大取值 1111（15），得 60 B，表示含 20 B 固定部分和最大 40 B 可变部分
区分服务	8	用来获得更好的服务，实际上一直没有被真正使用过
总长度	16	首部 + 数据载荷的长度和，单位字节 **B**。最大 $2^{16}-1=65535$ B。以太网 MTU 为 1500 B，封装成帧时数据报总长度不能超过链路层 MTU
标识	16	一个计数器，每产生一个数据报就加 1。并不是**”** 序号**”**。同一数据报的所有分片复制同一个标识号，以便正确重装
标志	3	最低位 **MF**：MF=1 后面还有分片，MF=0 是最后一个分片。中间位 **DF**：只有 **DF=0** 才允许分片。最高位保留，必须为 0
片偏移	13	某片在原数据报中的相对位置，以 **8 B** 为单位。除最后一个分片外，每个分片的数据长度一定是 **8 B** 的整数倍
生存时间 **TTL**	8	数据报可通过的路由器数的最大值，防止被错误路由的数据报无限制兜圈。以” 跳数” 为单位：路由器转发前减 1，不为 0 就转发，否则丢弃并发送 ICMP 时间超过报文
协议	8	指明数据载荷是何种 PDU。常用值：**ICMP=1**，**IGMP=2**，**TCP=6**，**UDP=17**，**OSPF=89**
首部检验和	16	只检验首部，不包括数据部分。因为每经过一个路由器某些字段（TTL、标志、片偏移等）都可能变化，都要重算检验和，不检验数据部分可减少计算量。IPv6 中路由器不再计算首部检验和

字段	位数	含义与要点
源 **IP** 地址	32	标识发送方
目的 **IP** 地址	32	标识接收方

标志字段的位序

3 位标志从左到右依次是「保留位、DF、MF」，最低位才是 **MF**。若给出标志字段的十六进制值，必须展开为二进制后再按位判断。

1.IP 数据报分片

为什么要分片：IP 数据报被封装在链路层的帧中，因此链路层的 **MTU**（最大传送单元）严格限制了 IP 数据报长度。以太网 MTU 为 **1500 B**，许多广域网 MTU 不超过 576 B。当数据报总长度大于链路 MTU 时，需要把数据分装在多个较小的 **IP** 数据报中，这些较小的数据报称为片。

位置：分片可由源主机或路径上的路由器进行；重组只在目的主机的网络层进行。每个分片独立传输，可能经过不同路径，甚至不一定按原次序到达。

重组所用字段：目的主机用标识、标志、片偏移完成重组。MF=1 表示还有后续片，MF=0 表示最后一片；片偏移确定该片应放在原始数据报的哪个位置。分片时每个分片都复制原数据报首部的固定部分（**20 B**）。

DF=1 的情况：不能分片而分组长度又超过 MTU 时，丢弃该分组，并用 **ICMP** 差错报告报文（终点不可达，代码” 需要分片但设置了 DF 位”）向源主机报告。

IP 分片完整计算步骤（必背）

设原数据报总长 L（首部 H 字节，通常 20 B），链路 MTU 为 M。

1. 原数据部分长度 $D = L - H$。
2. 每片最多携带的数据长度：$d_{\max} = \left\lfloor \dfrac{M - H}{8} \right\rfloor \times 8$。
3. 分片个数 $n = \left\lceil \dfrac{D}{d_{\max}} \right\rceil$；前 $n-1$ 片各携带 $d_{\max}$ 字节，最后一片携带剩余 $D - (n-1)d_{\max}$ 字节（可以不是 8 的倍数）。
4. 每片总长度字段 $= d_i + H$（含首部）。
5. 每片片偏移字段 $= \dfrac{\text{该片之前已发送的数据字节数}}{8}$。
6. **MF**：前 $n-1$ 片为 **1**，最后一片为 **0**。所有片标识相同，DF 均为 0。

【算例 **1**】某 IPv4 数据报总长度 3820 B，采用 20 B 固定首部，需分片为长度不超过 1420 B 的数据报片。

解：$D = 3820 - 20 = 3800$ B。每片最多 $1420 - 20 = 1400$ B，且 $1400 = 175 \times 8$ 恰为 8 的整数倍，故 $d_{\max} = 1400$。$n = \lceil 3800/1400 \rceil = 3$：$1400 + 1400 + 1000 = 3800$。

分片	数据长度/**B**	总长度字段/**B**	片偏移字段	**MF**
第 1 片	1400	1420	0	1
第 2 片	1400	1420	$1400/8 = 175$	1
第 3 片	1000	1020	$2800/8 = 350$	0

【算例 **2**：真题型】 2021 年第 36 题：路由器向 MTU=800 B 的链路转发总长度 1580 B 的 IP 数据报（首部 20 B），分片时每片尽可能大，则第 2 个分片的总长度字段和 MF 分别是多少?

解：$D = 1580 - 20 = 1560$ B。每片最多 $800 - 20 = 780$ B，但除最后一片外必须是 8 的整数倍，故 $d_{\max} = \lfloor 780/8 \rfloor \times 8 = 776$ B。$n = \lceil 1560/776 \rceil = 3$：$776 + 776 + 8 = 1560$。

分片	数据长度/B	总长度字段/B	片偏移字段	MF
第 1 片	776	796	0	1
第 2 片	776	**796**	$776/8 = 97$	**1**
第 3 片	8	28	$1552/8 = 194$	0

答案：**796**，**1**（选项 B）。

分片三大易错点

1. 总长度字段包含首部，第 2 片总长度是 $776 + 20 = 796$，不是 776，也不是 800。
2. 片偏移的单位才是 **8 B**，不是” 总长度必须是 8 的倍数”——第 3 片总长度 28 就不是 8 的倍数。
3. 只有除最后一片外的数据长度必须是 **8 B** 的整数倍；最后一片可为任意长度。

【算例 **3**：两级分片】总长 1500 B（首部 20 B）的 IP 数据报先经 MTU=800 B 的链路 R1→R2，再经 MTU=400 B 的链路 R2→R3，求 R2 分出的各子分片参数。

解：第一级（MTU=800）：$D = 1480$，$d_{\max} = \lfloor 780/8 \rfloor \times 8 = 776$，分片为 $776 + 704$。

R1 分片	数据/B	总长度/B	片偏移	MF
F1	776	796	0	1
F2	704	724	97	0

第二级（MTU=400）只需对 F1 再分：$d_{\max} = \lfloor (400 - 20)/8 \rfloor \times 8 = 47 \times 8 = 376$ B。F1 的 776 B 分为 $376 + 376 + 24$。关键性质：第二级分片的片偏移是在原数据报范围内的偏移，新片偏移 = 父分片片偏移 $\times 8 +$ 在父分片内的字节偏移。

由 **F1** 分出	数据/B	总长度/B	片偏移	MF
子片 1	376	**396**	$(0 + 0)/8 = 0$	1
子片 2	376	396	$(0 + 376)/8 = 47$	1
子片 3	24	44	$(0 + 752)/8 = 94$	**1**

第三个子分片的 MF 仍是 **1**，因为它由 F1 分出而 F1 的 MF=1，后面还有 F2；F2 的 MF=0 才表示整个数据报结尾。

*2.*首部检验和的计算

发送方：① 把首部划分为若干个 **16** 比特字序列（20 B 固定首部正好 10 个字）；② 把首部检验和字段置为全 **0**；③ 用反码算术运算把所有 16 比特字相加；④ 把和取反码写入首部检验和字段。

接收方：把首部所有 16 比特字（此时含检验和）再用反码算术运算相加，并把和取反码。若首部无差错，结果必为 **0**，保留该数据报；否则认为出错，丢弃该数据报。

二进制反码求和规则（从低位到高位逐列）：0+0=0；0+1=1；1+1=0 并产生进位 1 加到下一列；若最高位相加后产生进位，则结果要再加 **1**（”进位绕回来加到末尾”）。

首部检验和的两个易错点

1. 接收方发现检验和出错后直接丢弃数据报，不发送差错报告（ICMP 差错报文不针对首部检验和错误）。
2. 该方法与 UDP、TCP 的检验和计算方法相同，但 **IP** 校验没有伪首部。这个算法常被称为因特网检验和（Internet Checksum），检错性能不如 CRC，但更易用软件实现。

【算例】对典型首部 `45 00 00 3C 1C 46 40 00 40 06 00 00 C0 A8 00 01 C0 A8 00 C7` 求首部检验和。

逐字相加（检验和字段先置 0）：`4500+003C+1C46+4000+4006+0000+C0A8+0001+C0A8+00C7`，按列运算得 `0x43FF6`。把高 16 位溢出部分 4 加回低 16 位：`0x3FF6+4=0x3FFA`。取反码得 `0xC005`。

验证：接收方把全部 10 个字（含 `C005`）相加得 `0x4FFFB`，进位 4 回加得 `0xFFFF`，取反码为 `0x0000`，校验通过。

（二）IPv4 地址与 NAT

1.IPv4 地址与分类编址

定义：IPv4 地址是给因特网上每一个主机（或路由器）的每一个接口分配的、全世界范围内唯一的 **32** 比特标识符，由网络号和主机号两部分组成：

$$\text{IP 地址} = \{ < \text{网络号} >, < \text{主机号} > \}$$

用点分十进制表示：32 比特按 8 位一组分为 4 组，每组写十进制，用点分隔。

	网络号	主机号
标志	接口所连接到的网络	主机（或路由器）的接口
规则	同一网络中不同主机接口的网络号必须相同	同一网络中不同主机接口的主机号必须各不相同

分类编址：按左起第一个十进制数判断——小于 127 为 A 类，128 ~ 191 为 B 类，192 ~ 223 为 C 类，224 ~ 239 为 D 类，240 ~ 255 为 E 类。

类别	首位模式	网络号位数	网络号范围	可指派网络数	每网可分配地址数
A	0	8	1 ~ 126	$2^7 - 2 = 126$	$2^{24} - 2 = 16777214$
B	10	16	128.0 ~ 191.255	$2^{14} = 16384$	$2^{16} - 2 = 65534$
C	110	24	192.0.0 ~ 223.255.255	$2^{21} = 2097152$	$2^8 - 2 = 254$
D	1110	—	224.0.0.0 ~ 239.255.255.255	—	多播地址，不分配给主机接口

A 类中最小网络号 **0** 表示本网络不能指派，最大网络号 **127** 作为环回测试地址不能指派，故可指派 A 类网络数为 $2^{8-1}-2=126$。最小环回地址 127.0.0.1，最大环回地址 127.255.255.254。

”减 2”的含义

- A 类网络数减 2：去掉网络号 0（本网络）和 127（环回）。
- 每个网络可分配地址数减 2：去掉主机号全 **0** 的网络地址和主机号全 **1** 的广播地址。

A、**B**、**C** 类都是单播地址，只有单播地址可分配给主机（或路由器）的接口。主机号全 **0** 是网络地址，主机号全 **1** 是广播地址，都不能分配。

一般不使用的特殊 **IPv4** 地址：

网络号	主机号	地址示例	作源	作目的	用途
全 0	全 0	0.0.0.0	可以	不可以	本网络的本主机（DHCP 初始态）；路由表中表示默认路由
全 0	host-id	0.0.0.5	可以	不可以	本网络上的某台特定主机
全 1	全 1	255.255.255.255	不可以	可以	只在本网络广播（路由器不转发），又称受限广播地址
net-id	全 0	192.168.1.0	不可以	不可以	网络地址，表示一个网络本身
net-id	全 1	192.168.1.255	不可以	可以	对网络 net-id 上所有主机广播，又称直接广播地址（可跨路由器，只被目标网络内主机接收）
127	非全 0/全 1	127.0.0.1	可以	可以	本地软件环回测试，目的地址为环回地址的数据报永远不会出现在任何网络上

特殊地址速记口诀

全 **0** 的地址只能当源，不能当目的；全 **1** 的地址只能当目的，不能当源；**127** 开头源目都能用，但不走外网；主机号全 **0**/全 **1** 的地址永远不能分给主机用。路由器收到目的地址为 255.255.255.255 的分组不转发。

IP 地址的重要特点：只配网络号（管理机构只分配网络号，主机号由单位自行分配）；只转发网络号（路由器仅根据目的网络号转发，减小路由表存储空间）；一个接口一个地址（路由器至少有两个以上 IP 地址，每个端口都有不同网络号的地址）；同网同域（用转发器或网桥连接的若干 LAN 仍属同一网络、同一广播域，网络号必须相同、主机号必须不同）；众网平等。

*2.*网络地址转换 *NAT*

背景与私有地址：为网络安全划出了部分 IP 地址为私有 IP 地址。私有地址只用于 LAN，不用于 WAN 连接，必须通过网关用 NAT 转换为合法全球 IP 地址后才能出现在 Internet 上，且允许被 LAN 重复使用。互联网中所有路由器对目的地址是私有地址的数据报一律不转发。 RFC 1918 规定的三个专用地址块：

类别	地址范围	**CIDR** 前缀
A 类	10.0.0.0 ~ 10.255.255.255	10.0.0.0/8
B 类	172.16.0.0 ~ 172.31.255.255	172.16.0.0/12
C 类	192.168.0.0 ~ 192.168.255.255	192.168.0.0/16

定义：网络地址转换（NAT）指把专用网络地址转换为公用地址，从而对外隐藏内部管理的 IP 地址，作用是连接内网和外网。优点是节省 IP 地址消耗（专用网只需一个全球 IP 地址就可与互联网连通，本地地址可重用）与降低安全风险（隐藏内部网络结构）。

最基本的 **NAT** 与 **NAPT**：因绝大多数网络应用使用 TCP 或 UDP，为更有效利用 NAT 路由器上的全球 IP 地址，现在常把 **NAT** 转换与运输层端口号结合使用。NAT 路由器用 **NAT** 转换表存储 `{本地 IP 地址:端口}` → `{全球 IP 地址:端口}` 的映射，正式名称是网络地址与端口号转换（Network Address and Port Translation，**NAPT**）。这种映射可让多个私有 IP 地址映射到一个全球 IP 地址，即内部专用网中大量主机共用 **NAT** 路由器上的 **1** 个全球 **IP** 地址。很多家用路由器就是 **NAPT** 路由器，但往往并不运行路由选择协议。

NAT 工作原理（经典例子）：主机 10.0.0.1（端口 3345）向 Web 服务器 128.119.40.186（端口 80）发请求。

1. NAT 路由器收到分组后生成新端口号 5001，把源地址改为 138.76.29.7（NAT 路由器的全球 IP 地址），源端口号改为 5001，并在 NAT 转换表中增加一个表项。
2. Web 服务器不知道分组已被改装，也不知道用户的专用地址，其响应分组目的地址是 138.76.29.7、目的端口号是 5001。
3. 响应分组到达 NAT 路由器后，通过转换表把目的 IP 地址改为 10.0.0.1、目的端口号改为 3345。

同端口号的处理：两个客户端 192.168.1.10 和 192.168.1.11 都用本地端口 1025 与服务器通信，二者都转换为公有地址 120.229.175.121，但用不同的端口号区分。这种转换表自动生成：TCP 情况下建立连接首次握手时的 SYN 包一经发出就生成表项，随收到关闭连接 FIN 包的确认应答而从表中删除。

NAT 路由器与普通路由器的核心差别

普通路由器只处理网络层及以下数据，仅解析 IP 首部；**NAT** 路由器需要查看运输层端口号来维护 NAT 映射表，因此同时作用于传输层、网络层及以下。

报文修改规则：出门改源 **IP**（变公网），进门改目的 **IP**（找内网）。内网 → 外网修改源 **IP** 地址、源端口号；外网 → 内网修改目的 **IP** 地址、目的端口号。

NAT 的缺点与穿透问题：NAT（和 NAPT）对网络应用并不完全透明；通信必须由专用网内部发起，因此拥有内部专用地址的主机不能直接充当因特网中的服务器——只有内网主机主动发起连接，NAT 设备才建立映射表，外网主机的返回报文依靠该映射转回内网；外网主机主动访问内网时 NAT 设备没有对应映射条目，报文无法转发到内网主机。对 P2P 这类需要外网主机主动与内网主机通信的应用，需要 **NAT** 穿透技术（端口转发、STUN、中继）。若配置静态 **NAT**/端口映射建立固定的外网 `IP:外网端口` ↔ 内网 `IP:内网端口`对应关系，就能强行实现内网主机对外提供公网服务。

真题考法

NAT 常与 CIDR 结合出综合题，核心是判断” 分组经过某个 NAT 路由器后源/目的地址变成什么”。2016 年第 38 题中源地址换成 R2 外网接口地址 201.1.3.10，而目的地址始终是最终的 **Web** 服务器地址 **130.18.10.1**。铁律：路由器转发不改变目的 **IP** 地址；只有 **NAT** 设备才按转换规则修改源或目的地址。

（三）子网划分、路由聚集、子网掩码与 CIDR

*1.*划分子网与子网掩码

背景：随着更多中小网络加入因特网，分类编址” 不够灵活、容易造成大量 IPv4 地址资源浪费” 的缺点暴露出来。申请新网络号需等待很长时间、花费更多费用，其他路由器还要新增路由条目，且

浪费原网络中剩余的大量地址。若能从主机号部分借用一些比特作为子网号区分不同子网，就可利用原有网络中剩余的大量地址而不用申请新网络地址。

定义：划分子网是在 IP 地址中增加一个” 子网号字段”，使两级 IP 地址变成三级：

$$\text{IP 地址} = \{ < \text{网络号} >, < \text{子网号} >, < \text{主机号} > \}$$

子网掩码表明分类 IPv4 地址的主机号部分被借用了几个比特作为子网号。它也是 **32** 比特：左起多个连续的比特 **1** 对应网络号和子网号，之后的连续 0 对应主机号。把划分子网的 IPv4 地址与子网掩码逐比特逻辑与，就得到该地址所在子网的网络地址。

一句话结论

只要给定了一个分类的 **IPv4** 地址及其相应的子网掩码，就可以得出子网划分的全部细节。这是子网划分所有计算题的解题总纲。

默认子网掩码：所有网络都必须使用子网掩码；若未划分子网就使用默认掩码，**A**、**B**、**C** 类分别为 **255.0.0.0**、**255.255.0.0**、**255.255.255.0**。其他规定：路由器相互交换路由信息时必须告知对方本网络的子网掩码；转发时把目标地址与某网络掩码按位相与，结果与该网络地址一致则匹配成功；主机设置 IP 时必须设置子网掩码；同属一个子网的所有主机及路由器相应端口必须设置相同掩码；路由表主要信息是目的网络地址、子网掩码、下一跳地址。

默认网关是子网与外部网络连接的设备，即连接本机或子网的路由器接口的 **IP** 地址（离主机最近的路由器的接口地址）。主机发送数据时用子网掩码判定目的主机是否在同一子网：在则直接发送，不在则交给默认网关转发。

划分子网的核心结论

1. 划分子网增加子网数量，子网间传输需经路由器，因此减小了广播域；子网号占据主机号位，因此减少每个子网的主机数量。
2. 从一个 **IP** 地址本身无法判断该主机所连接的网络是否划分了子网，单位外的网络看不见单位内的子网划分。
3. 划分子网只提高 IP 地址利用率，不改变 **IP** 地址原有的类别归属，也不增加” 网络” 数量。
4. 子网中主机号全 0 的地址是子网的网络地址，全 1 的是子网的广播地址，都不能指派给主机。

*2.*给定 *IP* 与掩码求网络地址、广播地址与可用地址范围

标准解题步骤：

1. 确定块大小：找到掩码中发生变化（1→0）的那个字节，块大小 = 256− 该字节的十进制值。
2. 网络地址：把 IP 中该字节的值向下取到块大小的整数倍，其后字节全部置 0。
3. 广播地址：网络地址把主机位全部置 1，即” 下一个块的起点减 1”。
4. 可用地址范围：网络地址 $+1$ 到广播地址 -1，数量 $= 2^h - 2$（h 为主机号位数）。

【算例 **1**】主机 180.80.77.55，子网掩码 255.255.252.0，求所在子网的网络地址、广播地址与可用地址范围。

解：掩码 = **/22**。第三字节掩码 252，块大小 $= 256 - 252 = 4$。第三字节 77 落在 $76 \sim 79$ 块内（76 是 4 的整数倍）。

- 网络地址：**180.80.76.0**；广播地址：**180.80.79.255**；
- 可用范围：**180.80.76.1～180.80.79.254**，共 $2^{10} - 2 = 1022$ 个。

（2012 年第 39 题考此型，目的地址可以是 180.80.79.255，选 D。）

【算例 **2**】主机 168.16.84.24/20 所在子网的最小、最大可分配地址。解：/20 表示第三字节高 4 位属网络前缀，块大小 $= 2^4 = 16$。84 落在 $80 \sim 95$，故网络地址 **168.16.80.0**，广播地址 **168.16.95.255**，最小可分配 **168.16.80.1**，最大可分配 **168.16.95.254**。（2023 年第 39 题，选 B。）

【算例 **3**】主机 183.80.72.48，掩码 255.255.192.0，求网络地址。解：掩码 = **/18**，第三字节块大小 $= 256 - 192 = 64$。72 位于 $64 \sim 127$ 块，网络地址 = **183.80.64.0**。（2022 年第 35 题，选 B。）

最容易错的三处

1. 把掩码的**”** 非 **255** 字节**”** 换算错：255.255.252.0 是 /22。用 $256 - 252 = 4$ 反推块大小最稳。
2. 误以为必须在 **/24** 边界：块大小可落在任意字节。若掩码 255.255.255.128（/25），块大小为 128，块起点是 0 和 128。
3. 可用地址数忘记减 **2**。

*3.*子网划分的完整步骤与综合大题

定长划分的标准步骤（已知网络地址与所需子网数或每子网主机数）：

1. 确定借位数 s：若给定子网个数 k，取 $2^s \geq k$；若给定每子网至少容纳 x 台主机，则主机位数 h 满足 $2^h - 2 \geq x$，再据此反推 $s = n - h$（n 为原主机位数）。
2. 写出新掩码：前缀长度 $= 32 - h$，写成点分十进制。
3. 可用子网数 $= 2^s$（定长划分下各子网掩码相同，子网数只能是 2 的整数次幂）。
4. 每子网地址数 $= 2^h$，其中可分配给主机的数量是 $2^h - 2$。
5. 列出各子网的网络地址与广播地址：块大小 $= 2^h$，第 i 个子网的网络地址 = 原网络地址 $+ i \times$ 块大小。
6. 写出可用地址范围：网络地址 $+1$ 到广播地址 -1。

综合大题标准答题模板

以 2009 年第 47 题第 (1) 问为例：把 202.118.1.0/24 划分为 2 个子网，每个局域网需分配的 IP 地址数不少于 120 个。

解：每个局域网至少需 120 个可分配地址，$2^h - 2 \geq 120$ 即 $2^h \geq 122$，取 $h = 7$（$2^7 = 128$，$2^6 = 64$ 不够）。前缀长度 $32 - 7 = 25$，子网掩码 **255.255.255.128**，块大小 128。

- 局域网 1：**202.118.1.0/25**，可分配 202.118.1.1 ~ 202.118.1.126，广播 202.118.1.127；
- 局域网 2：**202.118.1.128/25**，可分配 202.118.1.129 ~ 202.118.1.254，广播 202.118.1.255。

两子网各有 126 个可分配地址，满足要求（次序互换也正确）。

【算例：定长划分组合题】192.168.5.0/24 采用定长子网划分，掩码 255.255.255.248，求最大子网个数与每子网最大可分配地址个数。解：255.255.255.248 = **/29**，原网络 /24，借用 $29 - 24 = 5$ 位子网号，得 $2^5 = 32$ 个子网；每子网剩余 3 位主机号共 8 个地址，扣除网络地址和广播地址后最多分配 $2^3 - 2 = 6$ 个。（2010 年第 37 题，选 B：32，6。）

【算例：反推最小子网】将 101.200.16.0/20 划分为 5 个子网，可能的最小子网的可分配 IP 地址数是多少？解：/20 含 $2^{12} = 4096$ 个地址。完整划分为 5 个 CIDR 子网时各子网大小必须是 2 的幂且总和为 4096，可取 $2048 + 1024 + 512 + 256 + 256 = 4096$，故最小子网可有 256 个地址即 /24，可分配 $256 - 2 = 254$ 个。不能选 126：若最小子网只有 128 个地址，剩余 3968 个地址无法恰好由另外 4 个二进制地址块组成。（2019 年第 37 题，选 B。）

【算例：判断不可能的子网】把一个 IP 网络划分为 3 个子网，其中一个子网是 192.168.9.128/26，下

列不可能是另外两个子网之一的是？A. 192.168.9.0/25　B. 192.168.9.0/26　C. 192.168.9.192/26　D. 192.168.9.192/27

解：已知子网.128/26 覆盖.128～.191。选 A（.0/25 覆盖.0～.127）时剩余.192～.255 恰为.192/26，三个子网完整划分，**A** 可能；选 C（.192/26）与 A 搭配同理，**C** 可能；选 D（.192/27 覆盖.192～.223）时.128/26 与.192/27 同属父网络.128/25，再取.0/25 作第三个，**D** 可能；选 B（.0/26 覆盖.0～.63）时尚未划分的地址为.64～.127 与.192～.255 两段不连续，无法用仅剩的一个 CIDR 子网覆盖，**B** 不可能。答案 **B**。（2021 年第 35 题。）

用”二叉树阀门”理解变长划分

把 CIDR 地址块看成二叉树，均分一次就是把节点一分为二。划分出 k 个子网等价于在二叉树的 k 个位置添加”阀门”，每个阀门对应的子树即为一个子网的地址范围。约束为不重、不漏：同一条根到叶子的路径上不能放多个阀门（防重叠），所有节点都必须被分配（防浪费）。阀门可放在任意层级节点（/25、/26、/27 均可）。

*4.*变长子网掩码与地址规划

定长子网掩码 **FLSM**	变长子网掩码 **VLSM**
每个子网使用同一个子网掩码，各子网网络前缀相同	每个子网可使用不同子网掩码，各子网网络前缀可不同
子网数量只能是 2 的整数次幂（2，4，8，……）	子网数量可以不是 **2** 的整数次幂（3，5，6，7，9，11……）
每子网地址数量相同，容易造成地址资源浪费	每子网地址数量可不同，尽可能减少浪费

地址规划中必须考虑的三类地址消耗：终端主机各分配 1 个可用主机 IP；路由器接口各分配 1 个独立 IP；子网保留地址（网络地址 1 个、广播地址 1 个）不能分配给任何设备。

特殊情况：连接两个路由器的点对点链路（无终端主机、只有 2 个路由器互连接口）仍需预留 1 个网络地址和 1 个广播地址，合计需要 **4** 个 **IP** 地址，因此通常分配一个 **/30** 地址块，恰好可分配 2 个地址给链路两端接口。

变长划分的核心原则

每个子块的起点位置不能随便选取，只能选取主机号部分是块大小整数倍的地址作为起点。建议先为大的子块选取。

【算例：变长划分综合应用题】地址块 218.75.230.0/24，需求为：网络 1 需 9 个地址，网络 2 需 28 个，网络 3 需 15 个，网络 4 需 13 个，网络 5（路由器间链路）需 4 个。用变长掩码分配。

解：按需求从大到小排序分配（”需 x 个”指总共占用的地址数，含网络地址与广播地址）。

网络	需求总数	满足 $2^h \geq$ 需求的最小 h	块大小	前缀	分配的地址块
网络 2	28	$h=5$	32	/27	218.75.230.0/27
网络 3	15	$h=5$	32	/27	218.75.230.32/27
网络 4	13	$h=4$	16	/28	218.75.230.64/28
网络 1	9	$h=4$	16	/28	218.75.230.80/28

网络	需求总数	满足 $2^h \geq$ 需求的最小 h	块大小	前缀	分配的地址块
网络 5	4	$h=2$	4	/30	218.75.230.96/30

子网	网络地址	可用地址范围	广播地址
218.75.230.0/27	218.75.230.0	.1 ~ .30	218.75.230.31
218.75.230.32/27	218.75.230.32	.33 ~ .62	218.75.230.63
218.75.230.64/28	218.75.230.64	.65 ~ .78	218.75.230.79
218.75.230.80/28	218.75.230.80	.81 ~ .94	218.75.230.95
218.75.230.96/30	218.75.230.96	.97 ~ .98	218.75.230.99

剩余 218.75.230.100 ~ 218.75.230.255 留作后用。注意每个子块起点都是块大小的整数倍：0、32 是 32 的倍数；64、80 是 16 的倍数；96 是 4 的倍数。若改用定长掩码，则必须满足最大需求 28，取 $h=5$、块大小 32、掩码 255.255.255.224（/27），划分出 $2^3=8$ 个等长子网、每网 30 个可用地址，总浪费远大于变长方案。

5.CIDR 无分类编址与路由聚合

背景：划分子网方法缓解了困难，但数量巨大的 C 类网（$2^{21}=2097152$ 个）每个网络只含 $2^8=256$ 个地址，并未得到充分使用，IPv4 地址仍在加速消耗。为此 IETF 提出无分类编址方法，即无分类域间路由选择（Classless Inter-Domain Routing，**CIDR**），它消除了传统 **A**、**B**、**C** 类地址以及划分子网的概念。

CIDR 的地址结构：地址掩码与子网掩码类似，由 32 比特构成，左起多个连续的比特 **1** 对应网络前缀，之后的连续 0 对应主机号：

$$\text{CIDR 地址} = \{ < \text{网络前缀} >, < \text{主机号} > \}$$

斜线记法：在 IPv4 地址后面加斜线"/"，斜线后写上网络前缀所占的比特数量（即掩码中左起连续 1 的数量，位数不固定、可变长）。

CIDR 地址块：把网络前缀都相同的、连续的多个无分类 IPv4 地址组成一个 **CIDR** 地址块。只要知道块中任何一个地址，就可知道该块的全部细节：起始地址（主机号全 0 的网络地址）；最大地址（主机号全 1 的广播地址）；地址总数 $=2^{32-\text{前缀长度}}$；地址掩码；可分配给主机使用的地址数。

【例】给定地址 128.14.35.7/20，求所在 CIDR 地址块的全部细节。解：/20 表示前 20 位是网络前缀、后 12 位是主机号。把后 12 位全部置 0 得网络地址 **128.14.32.0**；全部置 1 得最大地址 **128.14.47.255**（第三字节 $32+15=47$）；地址总数 $=2^{12}=4096$；最小可分配 128.14.32.1，最大可分配 128.14.47.254，可分配数 4094；地址掩码 **255.255.240.0**。

CIDR 的优点：可根据客户需要分配适当大小的 **CIDR** 地址块，从而更有效地分配地址空间。

路由聚合（又称构造超网）：一个 CIDR 地址块可以表示很多地址，因此路由表中可利用 CIDR 地址块来查找目的网络，这些地址的聚合称为路由聚合或超网。它使路由表中的一个项目可以表示多个原来传统分类地址的路由，有利于减少路由器之间的信息交换。关键规律：网络前缀越长，地址块越小，路由越具体；前缀越短，地址块越大，可聚合的路由越多。若有多条路由条目匹配，则选择网络前缀最长的那条，称为最长前缀匹配。

路由聚合的标准解题步骤

1. 把待聚合的网络地址写成二进制（重点看" 变化字节" 那一段）。
2. 找出所有网络地址从左起完全相同的连续比特位数，即聚合后新前缀的长度。
3. 验证对齐：聚合后的地址块必须是严格的 **2** 的幂大小且起点对齐。若各网络不能恰好拼成一个完整的 2 的幂地址块，则不能完全聚合（否则会引入不存在的网络）。
4. 写出聚合后的网络地址（共同前缀 + 全 0）与前缀长度。

【算例 **1**：真题型】某路由表中有转发接口相同的 4 条表项，目的网络为 35.230.32.0/21、35.230.40.0/21、35.230.48.0/21、35.230.56.0/21，求聚合后的目的网络地址。解：/21 表示第三字节高 5 位属于网络前缀，第三字节块大小 $= 256 - 248 = 8$。四网依次覆盖第三字节 $32 \sim 39$、$40 \sim 47$、$48 \sim 55$、$56 \sim 63$。合并后正好覆盖 $32 \sim 63$ 共 32 个连续值，$32 = 2^5$，对应第三字节高 3 位为前缀的一部分，故前缀长度 $16 + 3 = 19$。验证对齐：/19 的第三字节块大小 $= 256 - 224 = 32$，起点须为 32 的整数倍，32 正好满足。聚合结果：**35.230.32.0/19**。（2018 年第 38 题，选 C。）

【算例 **2**：两个 **/25** 聚合】 R2 到局域网 1（202.118.1.0/25）和局域网 2（202.118.1.128/25）的路由可聚合为一条。两块的块大小都是 128，起点为 0 和 128，恰好组成 $0 \sim 255$ 的完整 /24 地址块，聚合为 **202.118.1.0/24**，掩码 **255.255.255.0**。（2009 年第 47 题第 (3) 问。）

【算例 **3**：跨网段聚合】 R1 只有到 192.168.1.0/24 的路由。为把分组正确路由到 R2 后面的 192.168.2.0/25 与 192.168.2.128/25，需增加哪条路由？解：两块恰好组成 192.168.2.0/24，掩码 255.255.255.0，下一跳是同一链路上 R2 的接口地址（设为 192.168.1.2），而不是 **R1** 自己的接口**.1**。（2011 年第 37 题，选 D。）

路由聚合的两个陷阱

1. 下一跳写错：聚合路由的下一跳是邻居路由器的接口地址，不是本路由器的接口地址。
2. 过度聚合：聚合后的地址范围可能大于原有各子网范围的并集，从而" 引入不存在的网络"。若题目允许引入无害的额外范围可以聚合；若明确要求不引入不存在的网络，则必须逐条列出或换用其他聚合方式。

（四）ARP 协议、DHCP 协议与 ICMP 协议

*1.*地址解析协议 *ARP*

为什么需要两种地址：在 IP 层抽象的互联网上只能看到 IP 数据报；虽然首部中有源 IP 地址，但路由器只根据目的 **IP** 地址转发；在局域网的链路层只能看见 MAC 帧。通过路由器转发时，IP 数据报在每个网络中都被路由器解封装和重新封装，MAC 帧首部的源、目的地址不断改变——硬件地址只具有本地意义，因此无法用 MAC 地址跨网络通信。

因此：数据报传送过程中，源 **IP** 地址和目的 **IP** 地址保持不变；源 **MAC** 地址和目的 **MAC** 地址逐链路（逐网络）改变。

ARP 的作用：无论网络层使用什么协议，在链路上传送数据帧时最终必须使用硬件地址，因此需要一种方法完成 IP 地址到 MAC 地址的映射，这就是地址解析协议（Address Resolution Protocol，**ARP**）。ARP 用于解决同一局域网上的主机或路由器的 IP 地址和硬件地址的映射问题。若目的主机与源主机不在同一局域网，则通过 ARP 找到本局域网上的某个路由器（默认网关）的硬件地址，把分组发给该路由器，剩下工作由下一个网络完成。

ARP 高速缓存：每台主机都设有一个 ARP 高速缓存（**ARP** 表），存放本局域网上各主机和路由器的 IP 地址到 MAC 地址的映射。

ARP 请求广播、ARP 响应单播

尽管 **ARP** 请求分组是广播发送的，但 **ARP** 响应分组是普通的单播。 ARP 广播只在子网中传播。ARP 高速缓存表项通过 ARP 自动获取，生命周期默认约两分钟，也可手工配置；ARP 表中 IP 与 MAC 的对应关系并非永久不变（例如主机更换了网卡）。

ARP 工作流程：

1. 主机 A 要向本局域网上的主机 B 发送 IP 数据报时，先在自己的 **ARP** 高速缓存中查看有无主机 **B** 的 **IP** 地址。
2. 若有，查出对应硬件地址写入 MAC 帧，通过局域网把帧发往该硬件地址。
3. 若没有，用目的 MAC 地址为 **FF-FF-FF-FF-FF-FF**（48 比特全 1）的帧封装并广播 **ARP** 请求分组，使同一局域网里所有主机都收到。
4. 目标主机 B 收到后先把请求中的源 **IP** 与源 **MAC** 映射存入自己的 **ARP** 表，确认请求的目标 IP 是自己后，向 A 发出 ARP 响应分组（单播，只有 A 能收到），其中含 B 的 IP 与 MAC 映射。
5. A 收到响应后把该映射写入 **ARP** 缓存，然后按查到的硬件地址发送 MAC 帧。

跨网段的 ARP 过程（关键考点）

ARP 不能跨网络使用，但不同局域网可以多次 ARP。若源主机与目的主机不在同一局域网，则源主机通过 ARP 解析的是本局域网内默认网关的硬件地址，把分组发往默认网关，由网关转发至下一个网络。即此时 **ARP** 请求的目标 **IP** 是网关的 **IP**，而不是最终目的主机的 **IP**。

发送前的判定：主机用本机子网掩码与目的 IP 地址逐位按位与，对比结果与本机网络地址。一致则目的主机与本机在同一局域网（直接交付），ARP 查目的主机的 MAC；不一致属于间接交付，ARP 查默认网关的 MAC。

ARP 的四种典型情况：

情况	发送方	目的	**ARP** 解析的对象
1	主机 H1	本网络上的主机 H2	目的主机 H2 的硬件地址
2	主机 H1	其他网络上的主机 H3	与网 1 连接的路由器 R1 的硬件地址（默认网关）
3	路由器 R1	与 R1 连接的网 **2** 上的主机 H3	目的主机 H3 的硬件地址
4	路由器 R1	网 3 上的主机 H4	与网 2 连接的路由器 R2 的硬件地址

关于 **ARP** 属于哪一层：由于 ARP 的主要用途是从网际层使用的 IP 地址解析出数据链路层使用的 MAC 地址，因此有的教材把 ARP 划归网际层，有的划归数据链路层，这两种做法都是可以的。

真题考法

2012 年第 38 题直接考”ARP 协议的功能”（答案：根据 IP 地址查询 MAC 地址）。2015 年第 47 题第 (2) 问考” 若主机 2 的 ARP 表为空，访问 Internet 时发出的第一个以太网帧的目的 MAC 地址是什么”——因为要先解析默认网关，所以第一个帧是 **ARP** 请求帧，目的 **MAC** 是 **FF-FF-FF-FF-FF-FF**。

*2.*动态主机配置协议 *DHCP*

作用：动态主机配置协议（Dynamic Host Configuration Protocol，**DHCP**）用于给主机动态分配 **IP** 地址及其他配置信息，基于 **UDP**（服务器端口 67、客户端端口 68）。

报文	发送方	含义	源 **IP**	目的 **IP**	目的 **MAC**
DISCOVER 发现	客户端	” 谁是 DHCP 服务器？ ”	0.0.0.0	255.255.255.255	广播 MAC（全 F）
OFFER 提供	服务器	” 我这有可用地址 ”	服务器 IP	255.255.255.255	客户端 **MAC**（二层单播）
REQUEST 请求	客户端	” 我就要这个地址 ”	0.0.0.0	255.255.255.255	广播 MAC（全 F）
ACK 确认	服务器	” 地址正式生效 ”	服务器 IP	255.255.255.255	客户端 **MAC**（二层单播）

DHCP 报文的 IP 层与 MAC 层差异（高频考点）

四个报文的 **IP** 层目的地址全是受限广播 **255.255.255.255**，因为此时客户端还没配置 IP 地址，其 IP 层只认广播地址。而服务器发出的 **OFFER** 和 **ACK** 在链路层使用单播（目的 MAC 是客户端 MAC），这样网卡一看目的 MAC 是自己的就收下帧。**OFFER** 报文中会提供 **IP** 地址、默认网关 **IP**、子网掩码、**DNS** 服务器 **IP**。客户端用 **0.0.0.0** 作源地址，含义是” 我不知道我是谁，但我现在就在这个网络里 ”。

租期：DHCP 分配的 IP 地址有租用期，客户端在租期过半时应请求续租；不再使用时可以主动释放，使地址能被回收重用。

DHCP 中继代理：DHCP 使用广播而路由器默认不转发广播，因此 DHCP 服务器与客户端不在同一网段时广播报文无法到达服务器。解决办法是在各网段设置 **DHCP** 中继代理（relay agent），把客户端广播的 DHCP 报文以单播方式转发给 DHCP 服务器，并把应答转回客户端。

真题考法

2015 年第 47 题第 (1) 问：DHCP 服务器可为主机 2 ~ 主机 N 动态分配 IP 地址的最大范围是什么？做法是把 /24 子网的可用地址.1 ~ .254 中已被静态占用的地址（路由器.1、DHCP 服务器.2、WWW 服务器.3、主机 1 .4）排除，答案为 **111.123.15.5～111.123.15.254**；DISCOVER 报文的源、目的 IP 是 **0.0.0.0** 和 **255.255.255.255**。

*3.*网际控制报文协议 *ICMP*

作用：为了更有效地转发 IP 数据报以及提高交付成功的机会，网际层使用了网际控制报文协议（Internet Control Message Protocol，**ICMP**）。主机或路由器用它发送差错报告报文和询问报文。**ICMP** 报文被封装在 **IP** 数据报中发送，IP 首部协议字段值为 **1**。因此直接为 ICMP 提供服务的是 **IP**，而不是 TCP 或 UDP。

差错报告类型	触发条件
终点不可达	路由器或主机不能交付 IP 数据报时向源点发送。可按代码字段细分为目的网络不可达、目的主机不可达、目的协议不可达、目的端口不可达等 13 种
源点抑制	路由器或主机因拥塞而丢弃数据报时向源点发送，使源点放慢发送速率
时间超过	两类：① 路由器把 TTL 减 1 后结果为 0，丢弃数据报并向源点发送时间超过报文；② 终点在规定时间内未收到一个数据报的全部分片时丢弃已收分片，也向源点发送时间超过报文
参数问题	路由器或目的主机发现数据报首部中有字段值不正确时，丢弃该数据报并向源点发送参数问题报文

差错报告类型	触发条件
改变路由（重定向）	路由器把改变路由报文发给主机，让主机知道下次应把数据报发给另外的（更好的）路由器

ICMP 询问报文常用两种：回送请求和回答——由主机或路由器向特定目的主机发出，收到该报文的主机必须回送 ICMP 回送回答报文，用来测试目的站是否可达及了解其状态；时间戳请求和回答——请某主机或路由器回答当前日期和时间，时间戳回答报文中有一个 32 位字段，写入的整数表示从 1900 年 1 月 1 日起到当前时刻的秒数，用来进行时钟同步和测量时间。

不应发送 ICMP 差错报告报文的几种情况

以下情况不发送：对 ICMP 差错报告报文本身不再发送差错报告报文；对第一个分片的数据报片的所有后续数据报片都不发送；对具有多播地址的数据报都不发送；对具有特殊地址（如 127.0.0.0 或 0.0.0.0）的数据报不发送。

PING（分组网间探测）用来测试主机或路由器之间的连通性。它是应用层直接使用网际层 **ICMP** 的一个例子，并不使用运输层的 **TCP** 或 **UDP**；所使用的 ICMP 报文类型是回送请求和回答。

traceroute（跟踪路由）用于探测 **IP** 数据报从源主机到目的主机要经过哪些路由器。原理是源主机依次发送一串 TTL 分别为 1、2、3……的 IP 数据报，第 i 个数据报到达第 i 跳路由器时 TTL 减为 0，该路由器丢弃它并回送 **ICMP** 时间超过报文，源主机由此得知第 i 跳路由器的地址；最后一个数据报到达目的主机时返回回送回答（或端口不可达），探测结束。UNIX 版本命令为 **traceroute**，运输层使用 **UDP**，网络层使用的 ICMP 报文类型只有差错报告报文；Windows 版本命令为 **tracert**，应用层直接使用网际层的 **ICMP** 协议，使用的 ICMP 报文类型有回送请求和回答报文以及差错报告报文。

为什么 ping 不通、traceroute 显示 *

ping 不通的常见原因（按排查顺序）：① 目标主机不存在或未开机，ARP 解析不到，本地即报” 请求超时”；② 中间路由器没有到达目标的路由，回送 **ICMP** 终点不可达（网络不可达）；③ 防火墙或安全策略主动丢弃 ICMP 回送请求/回答，表现为” 请求超时” 而非” 不可达”；④ 子网掩码或默认网关配置错误——主机用错误的掩码判断” 目的在同一子网”，于是不去找网关而直接在本网段发 ARP 请求，永远得不到响应（2016 年第 39 题正是此情况）。

traceroute 显示 * 的原因：某一跳路由器不回复 **ICMP** 时间超过报文（可能被防火墙过滤、被管理员禁用 ICMP、或 CPU 过载丢弃了 ICMP 报文）。此时该跳信息无法获得，但 traceroute 仍继续探测后面的跳数。因此 * 通常意味着” 该跳不可探测”，不等于” 该跳不存在” 或” 链路中断”。

四、IPv6

（一）IPv6 的主要特点

引入 IPv6 的目的是从根本上解决 **IP** 地址耗尽问题。解决地址耗尽有三条途径：采用 CIDR 使地址分配更合理；采用 NAT 节省全球 IP 地址；采用具有更大地址空间的新版本 IPv6。前两种只是延长 **IPv4** 使用寿命，只有第三种能从根本上解决问题。

IPv6 引进的主要变化：

- 更大的地址空间（最重要）：由 IPv4 的 **32** 比特增大到 **128** 比特。
- 扩展的地址层次结构：可划分更多层次，更好地反映拓扑结构，使寻址与路由层次设计更灵活。
- 灵活的首部格式：与 IPv4 首部并不兼容。IPv6 定义了许多可选的扩展首部，既提供更多功能，

又提高路由器处理效率，因为路由器对逐跳扩展首部外的其他扩展首部都不处理。

- 改进的选项：允许分组包含选项控制信息，因而可以包含新选项；IPv4 的选项则固定不变。
- 允许协议继续扩充：IPv6 允许不断扩充功能，而 IPv4 的功能固定不变。
- 支持即插即用（自动配置）：支持主机或路由器自动配置 IPv6 地址及其他网络配置参数，因此 **IPv6** 不需要使用 **DHCP**。
- 支持资源的预分配：能为实时音视频等要求保证带宽和时延的应用提供更好的服务质量保证。
- 只有源主机才能分片：IPv6 的分片是端到端的，不允许 IPv4 那样的路由器分片，可大大提高路由器转发速度。
- 首部固定：基本首部长度固定为 **40 B**，而 IPv4 首部长度可变（须为 4 B 整数倍）。
- 取消了首部检验和：加快路由器处理 IPv6 数据报的速度。
- 增大了安全性：如身份鉴别、保密功能；支持 **QoS** 保障。注意 **IPv6** 路由器收到过大的数据报时直接丢弃（并回送 ICMPv6 报文），不像 IPv4 那样分片。

IPv6	**IPv4**
地址长度 128 bit（16 B）	地址长度 32 bit（4 B）
只有在源结点才能分片，传输过程中不能分片	传输过程中可以分片
基本首部固定 40 B	首部长度可变，是 4 B 的整数倍
首部更简单灵活，取消许多字段（放在扩展首部中）	首部固定部分包含更多字段
无首部检验和	有首部检验和

IPv6 数据报的组成：由基本首部和有效载荷（净负荷）两部分组成。基本首部的字段只有 **8** 个，但长度增大到 **40 B**；有效载荷由零个或多个扩展首部及其后面的数据部分构成（扩展首部不属于 **IPv6** 数据报的基本首部）。

字段	位数	含义
版本	4	协议版本，IPv6 对应值是 6
通信量类	8	区分不同 IPv6 数据报的类别或优先级
流标号	20	属于同一个流的报文具有相同流标号，用于资源分配
有效载荷长度	16	除基本首部外的字节数，最大 65535 B（所有扩展首部都算在有效载荷之内）
下一个首部	8	相当于 IPv4 的协议字段。无扩展首部时指明数据是何种 PDU；有扩展首部时标识后面第一个扩展首部的类型
跳数限制	8	类似 IPv4 的 TTL，源点设定限制值（最大 255），路由器每次转发减 1，减为零就丢弃。用来防止 **IPv6** 数据报在因特网中永久兜圈
源地址	128	数据报发送端地址
目的地址	128	数据报接收端地址

IPv6 相对 **IPv4** 取消或改名的字段：取消首部长度（因长度固定 40 B）；取消区分服务（由通信量类和流标号实现其功能）；取消总长度（改用有效载荷长度）；取消标识、标志和片偏移（这些功能已包含在分片扩展首部中）；把生存时间 **TTL** 改称跳数限制；取消协议（改用下一个首部）；取消首部检验和；取消选项（改用扩展首部）。

IPv6 扩展首部的设计意图

IPv4 数据报若在首部使用了选项字段，则整个传送路径中的全部路由器都要检查选项，降低了处理速度，而实际上很多选项路由器并不需要检查。因此 IPv6 把 IPv4 首部中的选项字段都放到扩展首部中，由路径两端的源点和终点主机处理，而路径中的所有路由器都不处理这些扩展首部（除逐跳选项扩展首部外）。

（二）IPv6 地址

三种基本地址类型：单播——传统的点对点通信，一对一，可作源地址或目的地址；多播——一点对多点，数据报发送到一组计算机中的每一台，只能作目的地址（IPv6 没有采用广播的术语，而把广播看作多播的一个特例）；任播——终点是一组计算机，但只交付其中的一台（通常是最近的一台），只能作目的地址。

IPv6 地址的表示形式：

- 冒号十六进制记法：把地址中的每 4 位用一个十六进制数表示，用冒号分隔每 16 位，128 位地址写成 8 组每组 4 位十六进制数。
- ”左侧零”省略：两个冒号之间的十六进制数中最前面的一串 0 可以省略。
- ”连续零”压缩：一连串连续的 0 可用一对冒号（::）取代，例如 `2001:0DB8:0000:0000:0000:0000:0000:0001` 可写成 `2001:DB8::1`。
- 在一个 **IPv6** 地址中只能使用一次”连续零”压缩，否则会导致歧义。
- 冒号十六进制记法还可结合点分十进制的后缀，例如`::FFFF:192.168.1.1`。
- **CIDR** 的斜线表示法在 **IPv6** 中仍然可用，例如 `2001:DB8::/32`。

IPv6 地址类型与前缀

地址类型	二进制前缀	说明
未指明地址	128 位全 0，记为`::/128`	不能用作目的地址，只能用于还未配置 IPv6 地址的主机作为源地址
环回地址	128 位中仅最低位为 1，记为`::1/128`	**IPv6** 的环回地址仅此一个（IPv4 的环回是一整个 /8 网段）
多播地址	最高 8 比特为全 1，记为 `FF00::/8`	功能与 IPv4 多播地址相同，占 IPv6 地址空间的 1/256
本地链路单播地址	最高 10 比特为 1111111010，记为 `FE80::/10`	作用类似 IPv4 私有地址。即使网络未连接因特网也可用 TCP/IP，但不能与因特网上的其他主机通信
全球单播地址	除上述四种外的其他所有 IPv6 地址	用得最多的地址

IPv6 全球单播地址的三级结构（这样设计是为了让路由器更快查找路由）：全球路由选择前缀占 **48** 位，相当于 IPv4 分类地址中的网络号；子网标识符占 **16** 位，用于各机构构建自己的子网；接口标识符相当于 IPv4 的主机号，位数有 **64** 位之多，完全足够给接口的硬件地址编码，因此 **IPv6** 不需要使用地址解析协议 **ARP** 进行地址解析。

（三）IPv4 向 IPv6 的过渡

背景：因特网上使用 IPv4 的路由器数量太大，让所有路由器都改用 IPv6 无法一蹴而就，因此只能逐步演进；另外，新部署的 IPv6 系统必须能够向后兼容，即必须能够接收和转发 IPv4 数据报，并

能为 IPv4 数据报选择路由。

双协议栈（**Dual Stack**）：在完全过渡到 IPv6 之前，使一部分主机或路由器装有 **IPv4** 和 **IPv6** 两套协议栈，既可和 IPv6 系统通信，又可和 IPv4 系统通信，记为 **IPv6/IPv4**，表明它具有一个 IPv6 地址和一个 IPv4 地址。与 IPv6 主机通信时采用 IPv6 地址，与 IPv4 主机通信时采用 IPv4 地址；通过域名系统 DNS 查询目的主机采用的 IP 地址：若 DNS 返回 IPv4 地址，则源主机使用 IPv4 地址；若返回 IPv6 地址，则使用 IPv6 地址。

隧道技术（**Tunneling**）：当 IPv6 数据报要进入 IPv4 网络时，把 **IPv6** 数据报重新封装成 **IPv4** 数据报，即整个 IPv6 数据报成为 IPv4 数据报的数据载荷；封装有 IPv6 数据报的 IPv4 数据报在 IPv4 网络中传输；当 IPv4 数据报要离开 IPv4 网络时，再把其数据载荷（原来的 IPv6 数据报）取出并转发到 IPv6 网络。

IPv6 易错点

1. **IPv6** 地址空间不是 **IPv4** 的 **96** 倍，而是 $2^{128}/2^{32}=2^{96}$ 倍（2023 年第 40 题）。
2. **IPv6** 基本首部长度固定 **40 B**，不是可变的。
3. IPv6 的 **Hop Limit**（跳数限制）等价于 **IPv4** 的 **TTL**，都会被每个路由器减 1。
4. IPv6 的有效载荷长度字段不包含基本首部，但包含所有扩展首部。
5. IPv6 缺少”协议”字段（改用”下一个首部”）并不影响指明上层协议。

五、路由协议

（一）自治系统与域内、域间路由

自治系统（Autonomous System，**AS**）指在单一的技术管理下的一组路由器。每个 AS 内可能包括多个网络，**AS** 号（**ASN**）全球唯一。AS 在逻辑上互不相交，但在地理位置上可以相互交错；AS 之间必须通过边界路由器相互连接。AS 内使用 RIP 还是 OSPF 由该 AS 自己决定。

- 内部网关协议（Interior Gateway Protocol，**IGP**）：一个自治系统内部使用的协议，如 **RIP** 和 **OSPF**。
- 外部网关协议（Exterior Gateway Protocol，**EGP**）：自治系统之间使用的协议，目前使用最多的是 **BGP**。

若源主机和目的主机处于不同 AS（两 AS 可能使用不同 IGP），则数据报传到 AS 边界时需要用一种协议把路由选择信息传递到另一个 AS。每个 AS 自己决定内部运行哪个 IGP，但都有一个或多个路由器除运行本 AS 的 IGP 外还要运行外部网关协议。

IGP 与 EGP 是分类名称，不是具体协议

外部网关协议 **EGP** 和内部网关协议 **IGP** 只是路由选择协议的分类名称，而不是具体的路由选择协议。名称中使用”网关”是因为因特网早期 RFC 文档中没有使用”路由器”而使用”网关”。统考中容易把 EGP 与 BGP 混淆。

路由选择协议的三大功能：获取网络拓扑信息；选择最优路径（用 Dijkstra 单源最短路径算法等）；维护路由表。

（二）RIP 路由协议

路由信息协议（Routing Information Protocol，**RIP**）是 IGP 中最先得到广泛使用的协议之一。RIP 要求 AS 内的每一个路由器都维护从它自己到 AS 内其他每一个网络的距离向量（Distance-Vector，

D-V）。

度量：RIP 用跳数（Hop Count）作为度量（Metric）。路由器到直连网络的距离定义为 **1**；到非直连网络的距离为所经过的路由器数加 **1**。一条路径最多只能包含 **15** 个路由器，距离等于 **16** 相当于不可达，因此 RIP 只适用于小型互联网。RIP 认为" 好的路由" 就是所通过路由器数量最少的路由；到达同一目的网络有多条距离相等的路由时可进行等价负载均衡。（有些厂商不严格按标准实现，例如思科把直连网络距离定义为 0；统考按标准文档" 直连距离为 1" 作答。）

RIP 的三个重要特点：和谁交换信息——仅和相邻路由器交换；交换什么信息——路由器自己的路由表，即到所在 AS 中各网络的最短 RIP 距离以及应经过的下一跳路由器；何时交换信息——周期性交换（约每 30 秒一次），为加快收敛，网络拓扑变化时要及时向相邻路由器通告变化后的路由信息，这称为触发更新。

基本工作过程：① 路由器刚开始工作时只知道自己到直连网络的 **RIP** 距离为 **1**；② 每个路由器仅和相邻路由器周期性地交换并更新路由信息；③ 若干次交换和更新后，每个路由器都知道到达本 AS 内各网络的最短距离和下一跳路由器，称为收敛。收敛指路由环境变化后各路由器调整路由表以适应拓扑变化、最终达到稳定状态（路由表与网络拓扑保持一致）。

时间参数：每隔约 **30** 秒向所有相邻路由器发送路由更新报文；若 **180** 秒（默认）没收到某条路由条目的更新报文，则把该条目标记为无效（RIP 距离置为 16）；若再过一段时间（如 **120** 秒）仍未收到，则将该条目从路由表中删除。

RIP 报文的封装：RIP 报文用运输层的 **UDP** 封装，**UDP** 端口号为 **520**。从封装角度看 RIP 属于应用层；但 RIP 的核心功能是路由选择，属于网际层。

1.RIP 的距离向量更新规则与计算题

RIP 路由更新的四条规则（必背）

路由器收到相邻路由器（设为 X）发来的路由更新报文后，对报文中每条表项：

第一步（修改）：把报文中所有表项的" 下一跳" 都改为 **X**，所有" 距离" 字段的值都加 **1**。

第二步（比对）：

1. 本地路由表中没有该目的网络 → 添加。
2. 本地已有且下一跳就是 **X** → 替换（无条件更新，因为信息来自同一邻居）。
3. 本地已有、下一跳不是 **X** 且新距离更小 → 更新。
4. 本地已有、下一跳不是 **X** 且新距离相等 → 可添加（实现等价负载均衡）。
5. 本地已有、下一跳不是 **X** 且新距离更大 → 不更新。

所有距离都要按" 最大 **15**、**16** 表示不可达" 处理：算出的距离若 ≥ 16 一律记作 16，不会出现 **17**。

【算例：**RIP** 路由表更新完整计算】路由器 B 的原路由表如下，现收到相邻路由器 C 发来的路由更新报文，求 B 更新后的路由表。

B 的原路由表			**C** 发来的报文	
目的网络	距离	下一跳	目的网络	距离
Net1	3	A	Net1	4
Net2	4	C	Net2	2
Net3	1	直接交付	Net5	1
Net4	5	D		

解：第一步，把 C 报文中所有表项的下一跳改为 C、距离全部加 1，得 Net1 距离 5（下一跳 C）、Net2 距离 3（下一跳 C）、Net5 距离 2（下一跳 C）。

第二步逐条比对：

- **Net1**：原表有，下一跳是 A（≠ C），新距离 5 > 原距离 3 → 不更新；
- **Net2**：原表有，下一跳是 **C**（= **C**） → 无条件替换；
- **Net5**：原表没有 → 添加。

更新后的表项	距离	下一跳	变化说明
Net1	3	A	新路由劣势，不更新
Net2	3	C	相同下一跳，无条件替换
Net3	1	直接交付	未在报文中出现，保持不变
Net4	5	D	未在报文中出现，保持不变
Net5	2	C	新网络，添加

【算例：等价负载均衡】R1 当前有一条到 N 的路由（下一跳 R3，距离 5），现收到 R2 的更新报文包含 <N，7>，且该 RIP 实现支持等价路由负载均衡。解：先把报文修改为下一跳 R2、距离 $7+1=8$。原表有 N，下一跳 R3 ≠ R2，新距离 8 > 原距离 5 → 不更新。（若报文是 <N，4>，则新距离 5 与原距离相等且下一跳不同，此时可以添加形成等价路由。）

【算例：**RIP** 的 **16** 上限】2010 年第 35 题：R1 收到邻居 R2 的距离向量，其中包含 <net1，16>，能得出什么结论？解：RIP 把 16 定义为无穷大即目的网络不可达。R2 通告 <net1，16> 表示 R2 自己无法到达 net1；R1 即使再加上到邻居的 1 跳，结果仍按上限记为 16，因此 **R1** 不能经 **R2** 到达 **net1**。

2016 年第 **37** 题：若 R3 检测到网络 201.1.2.0/25 不可达并向 R2 通告一次新的距离向量，则 R2 更新后到达该网络的距离是多少？解：R3 把距离通告为 **16**。R2 经 R3 的候选距离理论上为 $16+1=17$，但 RIP 最大有效距离是 15，大于等于 16 的值都记作 16（不可达），不会出现 17，因此更新后的距离为 **16**。

RIP 的优缺点：优点是实现简单、路由器开销小；若某个路由器发现了更短的路由，在较短时间内便可传至所有路由器，俗称**”** 好消息传播得快**”**。缺点：① 限制最大距离为 **15**，限制了 AS 规模；② 相邻路由器之间交换的是完整路由表，规模扩大后开销随之增大；③ **”** 坏消息传播得慢**”**，收敛时间过长。因此规模较大的 AS 应使用 OSPF。

真题考法

2024 年第 47 题第 (3) 问用 RIP 的交换周期算收敛时间：路由器同一时刻启动并立即交换初始距离向量，之后每隔 30 s 交换一次，一轮只能把新信息再传播一跳。对 210.2.3.0/24：R14 直连，R11、R15 在 $t=0$ 获得，R12、R13、R16 最迟在 $t=30$ s 获得，故全部获得至少需 **30 s**。对 210.2.4.0/24：R16 直连，R15 在 $t=0$ 获得，R13、R14 在 $t=30$ s 获得，R11、R12 最迟在 $t=60$ s 获得，故至少需 **60 s**。第 (1) 问考”AS4 内部路径可能超过 20 台路由器，RIP 不适用，应选 OSPF”。

（三）**OSPF** 路由协议

开放最短路径优先（Open Shortest Path First，**OSPF**）是为了克服 RIP 的缺点而开发的。” 开放” 表明它不是受某一厂商控制而是公开发表的；” 最短路径优先” 是因为使用了 **Dijkstra** 提出的最短路径算法（SPF）。OSPF 基于链路状态而不像 RIP 基于距离向量；由于采用最短路径算法计算路由，从算法上保证了不会产生路由环路；不限制网络规模，更新效率高，收敛速度快。

链路状态指本路由器都和哪些路由器相邻，以及相应链路的”代价（cost）”；代价用来表示费用、距离、时延和带宽等，由网络管理人员决定。例如思科路由器中 OSPF 的代价算法是：100 Mb/s 除以链路带宽，结果小于 1 仍记为 1，大于 1 且带小数的舍去小数。

邻居关系的建立和维护：相邻路由器之间通过交互问候（**Hello**）分组建立和维护邻居关系。问候分组封装在 **IP** 数据报中、发往组播地址 **224.0.0.5**，IP 首部协议号字段取值为 **89**。问候分组的发送周期为 **10** 秒；若 **40** 秒未收到某邻居的问候分组则认为该邻居不可达。每个路由器都会建立一张邻居表。

链路状态通告 **LSA**：每个路由器都会产生链路状态通告（Link State Advertisement，**LSA**），其中包含直连网络和邻居路由器两类链路状态信息。LSA 被封装在链路状态更新（Link State Update，**LSU**）分组中，采用可靠的洪泛法（Flooding）发送。

- 洪泛法的要点是路由器向自己所有的邻居路由器发送 **LSU**，收到该分组的各路由器又把它转发给自己所有的邻居路由器（但其上游路由器除外），以此类推。洪泛不回传、不重复。
- **”** 可靠**”** 指收到 LSU 后要发送确认；收到重复的更新分组无需再次转发，但要发送一次确认。

链路状态数据库 **LSDB**：每个路由器都有一个链路状态数据库（Link State Database，**LSDB**）用于存储 LSA。通过各路由器洪泛发送封装有各自 LSA 的 LSU，各路由器的 **LSDB** 最终将达到一致。

最短路径优先计算：各路由器基于 LSDB 进行计算，构建出各自到达其他各路由器的最短路径即路由表，并周期性（默认 30 分钟）刷新链路状态。注意路由表中不会存储完整路径，而只存储**”** 下一跳**”**。

OSPF 的详细工作过程：① 邻居发现与链路参数初始化——通过 **Hello** 分组发现直连邻居、建立邻居关系，然后为去往各邻居的链路配置对应的开销度量 **metric**；② **LSDB** 同步——路由器向邻居发送 **DD** 数据库描述分组传递本地 LSDB 中所有 LSA 的摘要信息供双方比对，邻居逐条比对摘要后针对缺失、过期或更新的条目主动发送 **LSR** 链路状态请求分组，对方回复 **LSU** 链路状态更新分组携带所需 LSA 详细信息，邻居更新本地数据库后返回 **LSAck** 链路状态确认分组；③ 动态更新与路由计算——链路状态变化时变动路由器泛洪发送 **LSU** 在区域内扩散最新拓扑，其他路由器接收后回复 **LSAck**，全网依据最新 LSDB 用 **Dijkstra** 算法重新计算最短路径并更新路由表；④ 转发。

OSPF 的五种分组类型：

分组类型	作用
问候（**Hello**）	发现和维护邻居路由器的可达性
数据库描述（**DD**）	向邻居给出自己 LSDB 中所有链路状态项目的摘要信息
链路状态请求（**LSR**）	向邻居请求某些链路状态项目的详细信息
链路状态更新（**LSU**）	把链路状态信息用洪泛法对整个系统更新
链路状态确认（**LSAck**）	对链路状态更新分组的确认

多点接入网络中的 **OSPF** 路由器：若每台路由器都向其他 $(n-1)$ 个路由器发送问候分组和 LSU，洪泛数量会急剧增加。为减少洪泛数量，OSPF 采用：选举指定路由器（Designated Router，**DR**）和备用指定路由器（Backup Designated Router，**BDR**）；所有非 **DR/BDR** 只与 **DR/BDR** 建立邻居关系；非 **DR/BDR** 之间通过 **DR/BDR** 交换信息。

OSPF 划分区域：为使 OSPF 能用于规模很大的网络，OSPF 把一个 AS 再划分为若干个更小的范围，称为区域（area）。划分区域的好处是把利用洪泛法交换链路状态信息的范围局限于每一个区域而不是整个 AS，从而减少整个网络上的通信量。洪泛不跨区域。

路由器类型	含义
区域内路由器（IR）	所有接口都在同一个区域内
主干路由器（BBR）	只要在主干区域中的路由器都称为主干路由器，因此主干路由器可兼作区域边界路由器
区域边界路由器（ABR）	连接主干区域和其他下层区域的路由器，即至少一个接口在主干区域、另一个接口在其他区域
自治系统边界路由器（ASBR）	与 AS 外部的路由器交换路由信息的路由器

OSPF 特点小结（与 RIP 逐条对比）：和谁交换信息——用洪泛法向本 AS 中所有路由器发送信息（RIP 仅向相邻的几个路由器发送）；交换什么信息——与本路由器相邻的所有路由器的链路状态，只是路由器知道的部分信息（RIP 发送的是本路由器所知道的全部信息即整个路由表）；何时交换信息——只有当链路状态发生变化时才用洪泛法发送（RIP 不管拓扑是否变化都定期交换）；用什么传送信息——OSPF 是网络层协议，不用 UDP 或 TCP 而直接用 **IP** 数据报传送（协议字段 **89**），而 RIP 是应用层协议、在运输层使用 UDP。

OSPF 的其他补充特性：允许对每条路由设置不同代价，对不同类型的业务可计算出不同路由；若到同一目的网络有多条相同代价的路径，可把通信量分配给这几条路径；支持可变长度子网划分和 CIDR；每个链路状态都带一个 32 位序号，序号越大状态越新；互联网规模很大时使用 OSPF 比 RIP 好得多，收敛很快且不存在坏消息传得慢的问题。

（四）**BGP** 路由协议

协议定位：边界网关协议（Border Gateway Protocol，**BGP**）属于外部网关协议 **EGP**，专门用于自治系统之间的路由选择，是因特网跨域路由的核心协议。

为什么 **AS** 间不用 **IGP**：① 无统一度量标准——不同 AS 内部的度量维度不同（有的按时延、有的按带宽、有的按距离），无法用统一的” 代价” 计算全局最优路径；② 需支持策略选路——AS 之间的路由选择必须考虑政治、经济、安全等策略（如过境收费、规避不可信 AS），这些由网络管理人员设置，但策略并不是 AS 之间的路由选择协议本身。

BGP 的设计目标

BGP 只能是力求寻找一条能够到达目的网络且比较好的路由（即不能兜圈子），而并非要寻找一条最佳路由。 BGP 交换的路由信息是到达某个目的网络所要经过的各个自治系统序列而不仅仅是下一跳。

路由机制：路径向量算法。BGP 传递的路由信息 = 目的网络 + **AS** 路径序列（路径向量），并非距离或链路状态。通过携带完整经过的 AS 编号天然防环：若收到的路由路径包含自身 **AS** 号，直接丢弃。

通信主体：**BGP** 发言人——运行 BGP 协议、参与 BGP 会话的路由器。**eBGP** 发言人通常部署在 AS 边界，与其他 AS 的 BGP 发言人建立会话，交换跨 AS 的路由可达信息；**iBGP** 发言人位于同一 AS 内部，负责把外部学到的 BGP 路由在本 AS 内传播，不一定在 **AS** 边界。

核心辨析：**BGP** 发言人不等同于边界路由器，判定标准是路由器是否运行 **BGP** 并参与 **BGP** 会话，与所处物理位置无关。互相建立 BGP 会话的两台发言人互称邻站（对等站）。BGP 发言人一般同时运行本 AS 的 IGP，保障 AS 内部网络可达。

传输方式：BGP 属于应用层协议，基于 **TCP 179** 端口建立持久会话，依靠 TCP 保证可靠传输，无需协议自身维护确认与重传。

三个路由协议的封装方式（必背对比）

RIP 用 **UDP**（端口 **520**）；**OSPF** 直接封装在 **IP** 数据报中（协议号 **89**）；**BGP** 用 **TCP**（端口 **179**）。2017 年第 37 题直接考这一点。

会话类型：BGP 会话建立的前提是 BGP 邻居之间先建立 **TCP** 连接，再在该连接上交换 BGP 报文。

会话类型	建立对象	是否要求直连	核心作用
eBGP（外部 **BGP**）	两个 **AS** 的直连边界路由器之间	要求物理直连	在自治系统之间交换跨域路由信息
iBGP（内部 **BGP**）	同一 **AS** 内部的 **BGP** 路由器之间	不要求物理直连	在本 AS 内部同步从外部学到的 BGP 路由信息

iBGP ≠ IGP

IGP 解决”AS 内部数据包具体怎么走” 的问题，计算内部最优路径；**iBGP** 解决”AS 内部同步外部路由信息” 的问题，传递跨域可达性。两者功能不同，不能互相替代。

传统 iBGP 要求同一 AS 内的路由器实现全连接（**Full Mesh**）：N 台 iBGP 路由器需要建立 $\frac{N(N-1)}{2}$ 条会话。该全连接仅指 **BGP** 层面的逻辑会话连通，并不要求路由器之间物理直连。

BGP 路由的格式：目的网络 + `BGP` 属性，核心两个属性：**AS-PATH** 记录该路由途经的自治系统序列，用于选路，同时防止 **AS** 间路由环路（若路径序列中出现重复 AS 则存在环路）；**NEXT-HOP** 表示通告该 BGP 路由的路由器对应的 IP 地址。

BGP 路由传播的核心规则：① **eBGP** 跨 **AS** 传播——向外部 AS 通告时在 AS-PATH 中追加本 **AS** 编号，同时把 NEXT-HOP 更新为发送方边界路由器自身 **IP**（例：AS2 的 R4 向 AS3 的 R6 通告 N1 的路由，AS-PATH 追加 AS2，路由变为 `(N1, [AS2 AS1], R4)`）；② **iBGP AS** 内传播——从 eBGP 获得外部路由后通过 iBGP 在 AS 内同步，默认保持 **AS-PATH**、**NEXT-HOP** 不变（例：R2 收到 R1 的 eBGP 路由后把 `(N1, AS1, R1)` 原样同步给 R3、R4、R5，三者的 BGP 下一跳均为 R1）；③ **iBGP** 水平分割——从 **iBGP** 邻居学到的路由不得再转发给其他 **iBGP** 邻居，因此传统 iBGP 要求全连接；④ **BGP** 依赖 **IGP** 实现内部可达——路由器通过 iBGP 知道” 访问外网要交给对应的边界路由器转发”，但不负责内网路径计算，需要依靠 IGP 算出自己到该边界路由器的实际转发下一跳。

BGP 的天然防环规则：AS-PATH 记录该路由途经的所有 AS 编号。当一个 AS 收到 BGP 路由更新时，若发现自身 **AS** 编号已存在于 **AS-PATH** 中，说明接收会形成环路，因此直接丢弃该路由。

BGP 工作流程精髓：交换对象是与互为邻站的 BGP 发言人交换路由信息，跨 AS 通过 eBGP 邻站交互、同一 AS 内通过 iBGP 邻站同步；交换内容是网络可达性信息，即到达目的网络所需途经的 AS 路径序列；更新机制采用增量更新，仅在路由发生变化时通告有变动的部分，不做周期性全量刷新。

BGP-4 的报文	作用
打开 **OPEN**	与相邻的另一个 BGP 发言人建立关系，使通信初始化
保活 **KEEPALIVE**	周期性地证实邻站的连通性
更新 **UPDATE**	通告某一条路由的信息，以及列出要撤销的多条路由
通知 **NOTIFICATION**	发送检测到的差错

想与另一个 AS 的 BGP 发言人建立对等关系时，向对方发送 **OPEN** 报文；若对方接受，就用 **KEEPALIVE** 报文响应。此后双方周期性交换 **KEEPALIVE** 报文（一般每隔 30 秒）。KEEPALIVE

报文只有 19 字节长（只用通用首部），不会造成太大开销。**UPDATE** 报文是 **BGP** 的核心内容：可以用它撤销曾经通知过的路由，也可以宣布增加新的路由；撤销路由可以一次撤销多条，但增加新路由时每个 **UPDATE** 报文只能增加一条。

BGP 的路由选择顺序（无策略约束时的常见比较顺序）：偏好值（自定义优先级）→ **AS-PATH** 跳数最少 → 热土豆路由 → BGP 序号。热土豆路由选择算法：让分组通过最少的”代价”尽快离开本 AS（仅用于 AS 内部，此时分组还没到达 AS 边界）。

真题考法

2024 年第 47 题第 (4)(5) 问是 BGP 的典型综合考法。第 (4) 问：R44 属 AS4、R13 属 AS1，二者之间通过 **eBGP**（外部 **BGP**）会话通告路由，使用 **UPDATE** 报文；R13 再通过 **iBGP**（内部 **BGP**）会话把该网络的可达性信息通告给同属 AS1 的 R14、R15。第 (5) 问给三条候选：经 R11 的 AS_PATH `AS2 AS8 AS19`（长度 3）、经 R12 的 `AS3 AS7 AS11 AS19`（长度 4）、经 R13 的 `AS4 AS10 AS19`（长度 3）。先比 AS_PATH 长度，R12 路径更长被排除；R11、R13 等长，无策略约束时再比较到 BGP 下一跳的内部代价（热土豆路由）：R14 到 R11 最近、R15 到 R13 最近，因此 **R14** 的下一跳是 **R11**，**R15** 的下一跳是 **R13**。

（五）三种路由协议对比

对比项	**RIP**	**OSPF**	**BGP**
类型	内部网关协议 IGP	内部网关协议 IGP	外部网关协议 EGP
路由算法	距离-向量	链路状态	路径-向量
传递协议	**UDP**（端口 520），应用层协议	**IP**（协议号 89），网络层协议	**TCP**（端口 179），应用层协议
路径选择	跳数最少	代价最低	较好，非最佳
交换结点	和本结点相邻的路由器	本自治系统所有路由器（洪泛）	和本结点相邻的路由器（邻站）
交换内容	自己知道的全部信息即整个路由表；不知道全网拓扑	相邻所有路由器的链路状态；只发送路由表的一部分	首次整个路由表；非首次只发变化部分
交换时机	周期性（约 30 s）+ 触发更新	链路状态变化时	路由变化时（增量更新）
收敛速度	慢（坏消息传得慢）	快	—
规模限制	最大 15 跳（16 为不可达）	无限制	面向跨域
防环机制	最大距离 + 水平分割 + 毒性逆转（不能彻底解决）	基于链路状态算法，天然无环	AS-PATH 中出现自身 AS 号即丢弃

为什么采用不同的传递协议（常考简答）：RIP 是分布式的基于距离向量的路由选择协议，通过广播 UDP 报文交换路由信息；OSPF 是内部网关协议，要交换的信息量较大、应使报文长度尽量短，所以不使用运输层协议而直接采用 IP；BGP 是外部网关协议，在不同 AS 之间交换路由信息，网络环境复杂需要保证可靠传输，所以采用 TCP。

六、**IP** 组播

（一）组播的概念

组播（多播）让源主机一次发送的单个分组抵达用一个组地址标识的若干目的主机，即一对多的通信。组播需要路由器支持，能运行组播协议的路由器称多播路由器。组播时仅需发送一份数据，只

在传送路径出现分岔时才将分组复制后继续转发，可大大减轻发送者负担和网络负载。

方式	目标	地址	特点
单播	一个接收者	单播地址	源需为每个接收者各发一份，N 个接收者消耗 N 份带宽
广播	本网段所有主机	广播地址	不区分是否需要；路由器默认不转发受限广播
组播	一组加入该组的主机	组播地址（**D** 类）	源只发一份，分岔处才复制，节省源端与主干链路带宽

多个单播可以仿真组播，但一个组播所需的带宽小于多个单播带宽之和；用多个单播仿真组播时路由器时延将很大，而处理一个组播分组的时延很小。

组播数据报：源地址是源主机的 IP 地址（大多为单播地址），目的地址是 IP 多播地址。注意事项：使用 **IGMP** 协议（IP 首部协议字段值是 **2**）；提供**"** 尽最大努力交付**"**，不保证交付给组内所有成员；对多播数据报不产生 **ICMP** 差错报文；多播机制仅应用于 **UDP**（TCP 面向连接，会一对一地发送）。

（二）IP 组播地址

IP 组播地址就是 IPv4 中的 **D** 类地址：前四位是 **1110**，范围 **224.0.0.0～239.255.255.255**，即 **224.0.0.0/4**。每个 **D** 类 **IP** 地址标志一个多播组，主机可随时加入或离开；多播地址只能用于目的地址，不能用于源地址；多播组成员的数量和所在的地理位置不受限制，一台主机可以属于几个多播组；非多播组成员也可以向多播组发送 **IP** 多播数据报。

D 类 **IP** 地址与以太网多播 **MAC** 地址的映射：以太网多播 MAC 地址范围是 **01-00-5E-00-00-00** 到 **01-00-5E-7F-FF-FF**（以 `01-00-5E` 打头）；48 位 MAC 中可用的低位 **23** 位与 **D** 类 **IP** 地址的低 **23** 位有一一对应关系，D 类地址除 1110 前缀外可供分配 28 位，但其中高 **5** 位无法映射，因此共有 2^{23} 个多播 MAC 地址；多播 **IP** 地址与以太网 **MAC** 地址的映射关系不是唯一的（32 个不同的 IP 多播地址可能映射到同一个 MAC 多播地址），因此主机收到多播数据报时要在 **IP** 层用软件过滤。

（三）IGMP 与组播路由

网际组管理协议（Internet Group Management Protocol，**IGMP**）是 TCP/IP 体系结构网际层中的协议，作用是让连接在本地局域网上的多播路由器知道本局域网上是否有主机（实际是主机中的某个进程）加入或退出了某个多播组。IGMP 报文被封装在 IP 数据报中传送，但它也向 **IP** 提供服务，因此不把它视为单独协议，而视为整个网际协议 IP 的一个组成部分。**IGMP** 仅在本网络有效：用它并不能知道多播组包含的成员数量，也不能知道成员分布在哪些网络中。三种报文类型：成员报告报文、成员查询报文、离开组报文。

工作阶段：① 加入阶段——某主机加入新的多播组时，向该多播组的多播地址发送一个 IGMP 报文声明自己是成员；本地多播路由器收到后还要利用多播路由选择协议把这种组成员关系转发给其他多播路由器。② 维持阶段——组成员关系是动态的，本地多播路由器周期性地探询局域网上的主机，只要对某个组有一台主机响应就认为该组活跃；经过几次探询后仍无任何主机响应，就认为本网络主机都已离开该组，不再把成员关系转发给其他多播路由器。

查询路由器的选举：同一网络中可能不止一个多播路由器，但没必要每个都周期性发送 IGMP 成员查询报文，只需选一个查询路由器发送，其他多播路由器被动接收响应并更新自己的多播组列表。选举方法：每个多播路由器若监听到源 IP 地址比自己的 IP 地址小的 IGMP 成员查询报文则退出选举，最后网络中 **IP** 地址最小的多播路由器成为查询路由器。

仅用 IGMP 不能在因特网上进行 IP 多播

连接在局域网上的多播路由器还必须和因特网上的其他多播路由器协同工作，以便用最小的代价把 IP 多播数据报传送给所有多播组成员，这就需要多播路由选择协议。

多播路由选择协议的主要任务是在多播路由器之间为每个多播组建立一个多播转发树：该树连接多播源和所有拥有该多播组成员的路由器；IP 多播数据报只要沿着多播转发树洪泛就能被传送到所有拥有该组播组成员的多播路由器；之后在多播路由器直连的局域网内，通过硬件多播把数据报发送给该组所有成员。

类型	构造方式	特点
源基树（源分布树、最短路径树）	为每一个（源，组）对分别构造以该源为根、覆盖所有组成员的最短路径树	路径最短、时延最小；但每个源都要维护一棵树，状态量为 O(源数 × 组数)，可扩展性差。代表：DVMRP、PIM-DM（密集模式）
共享树（核心树、汇聚点树）	整个组只构造一棵树，以选定的汇聚点（**RP**）为根，源先把数据发给 RP，再由 RP 沿共享树分发	状态量只需 O(组数)，可扩展性好；但路径未必最短，存在次优路径与 RP 单点瓶颈。代表：PIM-SM（稀疏模式）、CBT

组播路由的核心难点在于：组播地址只标识”组”而不标识位置，路由器无法像单播那样仅凭目的地址查表确定唯一出口；同时组成员可随时加入/离开，路由器必须动态维护转发树，并在有多个入接口时正确判断该从哪个接口转发（依赖反向路径转发 **RPF** 检查）。

七、移动 IP

（一）移动 IP 的概念

移动 **IP** 指移动站以固定的 **IP** 地址实现跨越不同网络的漫游功能，并保证基于 IP 的网络权限在漫游过程中不发生任何改变。其目标是把分组自动投递给移动站——把连接点从一个网络或子网改变到另一个网络或子网的主机。若需要在移动中进行 TCP 传输，则应一直保持这个 TCP 连接，可见若要使移动站在移动中的 **TCP** 连接不中断，就必须使主机的 **IP** 地址在移动中保持不变。

功能实体	含义
移动结点（移动站）	具有永久 IP 地址的移动主机
归属代理（本地代理）	在归属网络中代表移动节点执行移动管理功能的实体，通常就是连接在归属网络上的路由器
外地代理（外部代理）	在被访网络中帮助移动节点完成移动管理功能的实体，通常就是连接在被访网络上的路由器
归属地址（永久地址/主地址）	移动站在归属网络中的原始地址，与归属网络的关联不变
转交地址（辅地址）	移动到被访网络后获得的临时地址，可以是外地代理的地址或动态配置的地址；供移动站、归属代理及外地代理使用，各种应用程序都不会使用

（二）移动 IP 通信过程

基本工作过程分为代理发现、注册、分组路由与注销四个阶段：

1. 归属网络内正常通信：移动站 A 在归属网络时按传统 TCP/IP 方式通信。

2. 代理发现与注册：A 漫游到被访网络时先通过代理发现找到外地代理，向外地代理登记并获得一个临时的转交地址；外地代理再向 A 的归属代理登记该转交地址。
3. 隧道转发：归属代理知道 A 的转交地址后，构建一条通向转交地址的隧道，把截获的发给 A 的 IP 分组再封装（原始 IP 分组作为新 IP 分组的载荷），通过隧道发送给被访网络的外地代理。
4. 拆封与交付：外地代理把封装的 IP 分组拆封，恢复成原始 IP 分组后发给移动站 A（外地代理向移动站发送分组时直接使用移动站的 **MAC** 地址）。
5. 移动站的发送：A 在被访网络对外发送 IP 分组时仍然使用自己的永久地址作为源地址，此时无须通过归属代理转发，而是直接通过被访网络的外地代理发出。
6. 移动到另一被访网络：在新外地代理登记后，新外地代理把 A 的新转交地址告诉其归属代理（覆盖旧的）。无论如何移动，**A** 收到的 **IP** 分组都由归属代理转发。
7. 注销：A 回到归属网络时向归属代理注销转交地址，之后按原始方式通信。

移动 IP 中网络层的四个功能协议

支持移动性需要：① 移动站到外地代理的登记协议；② 外地代理到归属代理的登记协议；③ 归属代理数据报封装协议；④ 外地代理拆封协议。外地代理有两项功能：为移动站创建临时地址（转交地址）；及时把转交地址告诉其归属代理。

三角路由问题

移动 IP 的默认路由方式是非对称的：去程分组必须经归属代理隧道转发（因为通信对端只知道移动站的归属地址），而回程分组由移动站直接发出、不经归属代理。这样去程路径" 对端 → 归属代理 → 外地代理 → 移动站" 与回程路径" 移动站 → 对端" 在拓扑上构成一个三角形，称为三角路由（**triangle routing**）问题。后果是去程路径被拉长（即使对端与移动站当前在同一地区也要绕回归属网络），增加时延并浪费带宽；同时归属代理成为单点瓶颈。解决思路是路由优化：归属代理把移动站的当前转交地址通知通信对端，对端此后直接把分组发给转交地址，从而绕过归属代理形成直连路径；但这会引入对端需要缓存绑定信息、安全认证等新问题。

真题考法

移动 IP 属低频考点，考法一般是概念辨析：判断" 归属地址/转交地址"" 归属代理/外地代理" 的作用，或判断" 移动站发送分组时源地址使用哪个地址"（答案：仍使用归属地址）、" 移动站接收的分组由谁转发"（答案：归属代理）。注意移动 IP 的数据报是间接接收和发送；移动 IP 的分组路由可分为单播、广播与组播。

八、网络层设备

（一）冲突域和广播域

冲突域：连接到同一物理介质上的所有节点的集合，这些节点之间存在介质争用（OSI 第一层概念）。第一层设备（集线器、中继器）所连结点属同一个冲突域，不能划分冲突域；第二层（网桥、交换机）、第三层（路由器）设备都可以划分冲突域。

广播域：接受同样广播消息的节点集合（OSI 第二层概念）。第一、二层设备所连结点属同一个广播域；路由器（第三层）可以划分广播域；局域网（LAN）特指使用路由器分割的网络，也就是广播域。

设备互连能力：路由器可互联两个不同网络层协议的网段；网桥可互联两个物理层和链路层不同的网段；集线器不能互联两个物理层不同的网段。

一句判断法

设备所在层次越高，能屏蔽的下层差异越多，但转发速度越慢。路由器是第三层设备，处理内容比第二

层交换机更多，因而转发速度更慢、延迟更大。要抑制广播风暴只有路由器能做到：中继器、集线器工作在物理层不能隔离广播；网桥/二层交换机分隔冲突域但默认仍转发广播帧。

（二）路由器的组成和功能

定义：路由器是一种具有多个输入/输出端口的专用计算机，任务是连接不同的网络（异构网络）并完成分组转发。同一网络中传递数据无须路由器参与，而跨网络通信必须通过路由器转发；多个逻辑网络（多个广播域）互连时必须使用路由器；路由器可连接不同 LAN、不同 VLAN、不同 WAN，或把 LAN 和 WAN 互连，并可隔离广播域。

交付的两种类型：直接交付——源主机与目的主机连接在同一个网络上，无须通过路由器；间接交付——二者不在同一个网络上，路由器按转发表（由路由表得出）指出的路由把分组转发给下一个路由器。判断方法是路由器根据分组的目的 **IP** 地址与接收端口的 **IP** 地址是否属于同一子网来判断；间接交付的最后一个路由器肯定是直接交付。

路由器体系结构的两个部分：

（**1**）路由选择部分（控制部分）：路由选择处理机——根据所选定的路由选择协议构造路由表，同时经常或定期地与相邻路由器交换路由信息以不断更新和维护路由表；路由选择协议——用来更新路由表的算法；路由表——根据路由算法得出，一般包括从目的网络到下一跳的映射。

（**2**）分组转发部分：

组成部分	作用
交换结构	根据转发表对分组处理，把某个输入端口进入的分组从合适的输出端口转发出去。 三种交换方式：通过存储器交换、通过总线交换、通过互联网络交换
输入端口	包括物理层、数据链路层和网络层的处理模块
输出端口	从交换结构接收分组，再发送到路由器外面的线路上

缓冲队列与丢包：端口在网络层的处理模块中设有缓冲队列，暂存等待处理或已处理完毕待发送的分组，还可进行必要的差错检测。若分组处理的速率赶不上分组进入队列的速率，后面进入队列的分组就会因缓冲区满而被丢弃。

输入排队与输出排队都会导致丢包：输入排队——当交换结构的处理速度低于所有输入端口到达速率之和时输入队列不断积累，此外队头阻塞（**HOL blocking**）（输入队列最前面的分组因其目的输出端口忙而被阻塞，导致排在它后面、目的端口空闲的分组也无法转发）会进一步降低吞吐量；输出排队——多个输入端口同时向同一输出端口发送时输出端口成为瓶颈，输出队列溢出丢包。因此输入或输出队列产生溢出是造成分组丢失的重要原因。

路由器的功能边界（高频辨析）

- 路由器是网络层设备，实现了网络模型的下三层：物理层、数据链路层和网络层。
- 若一个存储转发设备实现了某个层次的功能，则它可以互连两个在该层次上使用不同协议的网段。如网桥实现了物理层和数据链路层，可互连两个物理层和数据链路层不同的网段；但中继器实现了物理层却不能互连两个物理层不同的网段，因为中继器不是存储转发设备，属于直通式设备。
- 路由器向传输层及以上隐藏下层实现，所以物理层、数据链路层、网络层协议可以不同；而网络层之上的协议数据是路由器不能处理的，因此网络层以上的高层协议必须相同。使用特定路由器连接 IPv4 与 IPv6 网络，就是典型的网络层协议不同而实现互连的例子。
- 存储转发机制：先接收整个分组，然后对分组做错误检查，出错就丢弃，否则存储该分组，最后根据路由选择协议把分组转发到合适端口。路由器只能根据 **IP** 地址转发。

- 完整路径问题：每个路由器都根据路由表选择下一跳地址，只有到了下一跳路由器才知道再下一跳怎么走。路由器和源主机都不知道分组要经过的完整路径。

真题考法

2012 年第 37 题考”IP 路由器功能的描述”，正确的是” 运行路由协议，设置路由表”” 监测到拥塞时，合理丢弃 IP 分组”” 根据目的 IP 地址将其转发到合适的输出线路上”；错误项是” 对收到的 IP 分组头进行差错校验，确保传输的 IP 分组不丢失”——**IPv4** 首部校验和只检查首部差错，发现错误通常丢弃分组；**IP** 提供尽力而为服务，不保证可靠交付。2010 年第 38 题考” 能够抑制广播风暴的设备”，答案是仅路由器。

（三）路由表与分组转发

路由表的结构：标准路由表有 **4** 个项目：目的网络 **IP** 地址、子网掩码、下一跳 **IP** 地址、接口。划分子网后，路由表必须包含三项内容：目的网络地址、子网掩码和下一跳地址（加上出接口共四项）。

转发表是从路由表得出的，表项与路由表项有直接对应关系，但格式不同，其结构应使查找过程最优化（路由表则需对网络拓扑变化的计算最优化）。转发表中含分组将要发往的目的地址以及分组的下一跳（实际为 MAC 地址）。可用一个默认路由代替所有具有相同” 下一跳” 的项目，并把默认路由设置得比其他项目优先级低。

主机发送 **IP** 数据报的过程：① 判断是否同属一个网络——用本机子网掩码与目的 IP 地址逐位按位与，对比结果与本机网络地址；一致则目的主机与本机在同一局域网（直接交付），不一致则目的主机在外部网络（间接交付）。② 封装 **MAC** 帧并发送——同网时通过 ARP 获取目的主机的 MAC 地址，封装成 MAC 帧后直接发送；异网时通过 ARP 获取默认网关的 MAC 地址，目的 MAC 字段填网关 MAC，把数据报交给默认网关，由路由器继续转发。

路由器转发 **IP** 数据报的完整过程：

1. 提取目的地址：从收到的 IP 数据报首部提取目的 IP 地址 D。
2. 判断直连网络直接交付：用路由器各直连接口的子网掩码分别与 D 逐位按位与，若结果与对应接口的网络地址匹配则说明目的主机在直连子网，执行直接交付（ARP 解析目的 IP 的 MAC，封装 MAC 帧从对应接口发出）；全部不匹配则进入路由表查表。
3. 匹配特定主机路由：若存在目的地址为 D 的特定主机路由（**/32** 前缀），转发给该路由指定的下一跳。
4. 匹配子网路由（最长前缀匹配）：遍历所有子网表项，用每条表项的掩码与 D 按位与得 N，若 N 与该表项的目的网络地址匹配，则选择前缀最长的匹配项，转发给对应下一跳。
5. 匹配默认路由：若子网路由无匹配项，检查是否存在 `0.0.0.0/0`，存在则转发给默认路由指定的下一跳。
6. 转发失败处理：若无任何匹配路由，或 TTL 减为 0，则丢弃该 IP 数据报，并向源主机返回 ICMP 差错报告。

分组转发的三个核心特点：① 最长匹配——采用 CIDR 时若有多个匹配前缀，应使用最长前缀匹配；为更快查找可按前缀长短把最长的排在第 1 行、按降序排列，这样从第 1 行开始查找，检索到匹配的就不必再继续。② 非完整路径——转发表没有给分组指明到某个网络的完整路径，只指出应当先到某个下一跳路由器，到达后再继续查找其转发表。③ **MAC** 帧变化——得到下一跳路由器的 IP 地址后，并不直接把它填入待发送的数据报，而是通过 ARP 转换成 MAC 地址填入 MAC 帧首部，再据此找到下一跳路由器。不同网络中传送时 **MAC** 帧的源、目的地址都要变化，而 **IP** 层的源、目的地址保持不变。

最长前缀匹配查表计算题的标准解法

1. 对每条路由表项，用该表项的掩码与目的 **IP** 地址按位与，得到候选网络地址。
2. 比较候选网络地址是否等于该表项的目的网络地址，相等则该表项匹配。
3. 在所有匹配表项中选前缀最长的那条（等价于掩码中 1 最多、块大小最小的那条）。
4. 若选中直连网络，则从对应接口直接交付；否则转发给表项给出的下一跳。
5. 默认路由 **0.0.0.0/0** 与任何地址都匹配（前缀长度 **0**），但优先级最低，只有当没有其他表项匹配时才使用。

【算例：真题型最长前缀匹配】 2015 年第 38 题路由表如下，收到目的地址为 169.96.40.5 的 IP 分组，转发接口是哪一个？

目的网络	下一跳	接口
169.96.40.0/23	176.1.1.1	S1
169.96.40.0/25	176.2.2.2	S2
169.96.40.0/27	176.3.3.3	S3
0.0.0.0/0	176.4.4.4	S4

解：/23 掩码 255.255.254.0，与 169.96.40.5 按位与得 169.96.40.0，匹配；/25 掩码 255.255.255.128，第四字节 5 落在 $0 \sim 127$ 块内，得 169.96.40.0，匹配；/27 掩码 255.255.255.224，第四字节块大小 $256 - 224 = 32$，5 落在 $0 \sim 31$ 块内，得 169.96.40.0，匹配；/0 与任何地址都匹配但前缀最短。三条具体路由都匹配，取前缀最长的 **/27**，因此从 **S3** 接口转发（选 C）。

【算例：聚合路由与直连路由的优先级】2013 年第 47 题第 (2) 问：R2 路由表中同时有 194.17.20.0/23（下一跳 194.17.24.2，接口 S1）与 194.17.20.128/25（直接交付，接口 E0），收到目的地址 194.17.20.200 的分组，从哪个接口转发？解：194.17.20.200 同时匹配 194.17.20.0/23（覆盖 194.17.20.0 ~ 194.17.21.255）和 194.17.20.128/25（覆盖 194.17.20.128 ~ 194.17.20.255）。按最长前缀匹配选择 **/25**，从 **E0** 接口转发。这道题揭示一个重要设计技巧：虽然 **/23** 也覆盖 **/25** 的范围，但由于 **/25** 更具体，直连子网的分组不会被错误地转发给 **R3**，因此可用一条聚合路由加一条更具体的直连路由来缩减路由表规模而不出错。

【算例：**TTL** 与跳数】 2014 年第 43 题第 (2) 问：主机 192.1.1.130 向主机 192.1.7.211 发送 TTL=64 的 IP 分组，R1 通过哪个接口转发？目的主机收到的 TTL 是多少？解：目的主机位于 192.1.7.0/24，R1 使用聚合路由 192.1.6.0/23（下一跳 10.1.1.2，接口 L0），从 **L0** 发往 R2，随后经 R2、R4 到达目的子网。分组经过 R1、R2、R4 共 **3** 台路由器，每台转发前把 TTL 减 1，所以目的主机收到时 $TTL = 64 - 3 = 61$。

TTL 计算的关键

初值 T 的分组经过 k 台路由器后，目的主机收到的 TTL 为 $T - k$。**"** 经过的路由器数**"** 包括发出分组的主机之后所有转发该分组的路由器，但不包括目的主机本身。若问" 初始 TTL 至少设置为多少才能送达"，最坏路径上有 k 台路由器要减 1，则初值至少为 $k + 1$。2024 年第 47 题第 (2) 问中 AS3 内最坏路径经过 15 台路由器，故初始 TTL 至少为 **16**。

路由表与转发表的比较：① 路由表根据路由选择算法得出、主要用途是路由选择、总是用软件实现，转发表可以用软件也可以用硬件实现；② 路由表不等于转发表，分组的实际转发靠直接查找转发表；③ 由于转发表根据路由表生成、路由表又由路由算法得到，因此路由算法决定了转发表中的值；④ 转发是路由器根据转发表把收到的 IP 数据报从合适端口转发出去，仅涉及一个路由器；路由选择涉及很多路由器，路由表是许多路由器协同工作的结果。

路由类型	目的网络地址	固定子网掩码	网络前缀	匹配优先级
特定主机路由	单个主机 IP	255.255.255.255	/32	最高
普通网段路由	目标网段	按需自定义	/8 ~ /30	中等
默认路由	0.0.0.0	0.0.0.0	/0	最低

本章小结表

一、**IPv4** 首部关键字段速查

字段	位数	单位	关键结论
首部长度	4	4 B	最小 5（20 B），最大 15（60 B）
总长度	16	1 B	最大 65535 B，含首部
标识	16	—	同一数据报所有分片标识相同；不是序号
标志	3	—	低 MF、中 DF、高保留（0）；DF=1 不许分片
片偏移	13	8 B	除最后一片外数据长度是 8 B 的整数倍
TTL	8	跳	每台路由器减 1，为 0 则丢弃并发 ICMP 时间超过
协议	8	—	ICMP=1，IGMP=2，TCP=6，UDP=17，OSPF=89
首部检验和	16	—	只检验首部；反码求和再取反码

二、地址与编址方法演进

阶段	地址结构	掩码/前缀	关键点
分类编址	网络号 + 主机号	255.0.0.0 / 255.255.0.0 / 255.255.255.0	A/B/C/D/E 类；主机号全 0、全 1 不可分配
划分子网	网络号 + 子网号 + 主机号	子网掩码	从主机号借位；子网数只能是 2 的幂
无分类编址 CIDR	网络前缀 + 主机号	斜线记法 /n	消除类别概念；支持路由聚合与最长前缀匹配

三、五类必考计算题的解法要点

计算类型	核心公式或规则	易错点
IP 分片	$d_{\max} = \lfloor (MTU - 20)/8 \rfloor \times 8$；片偏移 = 之前数据字节数 ÷ 8；总长度含首部	总长度不必是 8 的倍数，只有片偏移单位是 8 B
子网划分	块大小 = 256− 掩码中该字节的值；网络地址向下取整；可用数 $= 2^h - 2$	忘记减 2；块起点取错
CIDR 聚合	取所有网络地址的最长共同前缀；验证起点是块大小整数倍	聚合引入不存在的网络；下一跳写成自己
路由表转发	逐条掩码按位与 → 匹配项取前缀最长	忽略默认路由优先级最低；误改目的 IP

计算类型	核心公式或规则	易错点
RIP 更新	报文下一跳改 X、距离全部 +1；无则加、同下一跳则换、更小则改、等则加、更大则不改	忘记距离 +1；忘记 16 上限（不会出现 17）

四、**ARP / DHCP / ICMP** 对照

协议	层次	承载	关键报文/类型	核心结论
ARP	网际层或数据链路层（两种划分都可）	直接封装在链路层帧中	请求（广播）、响应（单播）	只在本局域网内有效；跨网段解析的是默认网关的 MAC
DHCP	应用层	UDP（服务器 67、客户端 68）	DISCOVER / OFFER / REQUEST / ACK	四个报文目的 IP 都是 255.255.255.255；OFFER、ACK 在链路层单播
ICMP	网际层	IP（协议号 1）	差错：终点不可达、源点抑制、时间超过、参数问题、改变路由；询问：回送请求/回答、时间戳	ping 用回送请求和回答；traceroute 依赖时间超过；对组播数据报不发差错报文

五、**RIP / OSPF / BGP** 对比

对比项	**RIP**	**OSPF**	**BGP**
归属	IGP（域内）	IGP（域内）	EGP（域间）
算法	距离-向量	链路状态	路径-向量
承载	UDP 520	IP，协议号 89	TCP 179
度量	跳数（最大 15，16 不可达）	代价（带宽、时延等，可自定义）	AS-PATH 等属性 + 策略
交换范围	仅相邻路由器	洪泛至 AS 内所有路由器	仅邻站
收敛	慢	快	增量更新

六、**IPv4** 与 **IPv6** 对照

对比项	**IPv4**	**IPv6**
地址长度	32 bit	128 bit
基本首部长度	20 B 固定 + 0 ~ 40 B 可选	固定 40 B
分片	源主机和路由器都可分片	只有源主机可分片
首部检验和	有	无
TTL / 跳数限制	TTL	跳数限制（Hop Limit）
上层协议标识	协议字段	下一个首部字段
选项	首部中的选项字段	扩展首部（路由器不处理，逐跳选项除外）

对比项	IPv4	IPv6
地址配置	需 DHCP	支持地址自动配置
地址解析	需 ARP	不需要 ARP（接口标识符 64 bit 可直接编码）
过渡技术	—	双协议栈、隧道技术

易错清单

1. IP 分片的三个数字

首部长度以 **4 B** 为单位，总长度以 **1 B** 为单位，片偏移以 **8 B** 为单位。算” 每片最多携带多少数据” 要先用 $\lfloor (MTU-20)/8 \rfloor \times 8$ 向下取整；算” 总长度” 要记得加回 20 B 首部；看到” 最后一片总长度不是 8 的倍数” 不要怀疑自己算错——只有除最后一片外的数据长度必须是 8 的整数倍。

2. 可用地址数永远要减 2

子网/地址块内可分配给主机或路由器接口的地址数 $=2^h-2$，要减掉网络地址（主机号全 **0**）和广播地址（主机号全 **1**）。但若题目问的是” 地址块中有多少个地址”，则不减 **2**（例如 /20 有 4096 个地址）。读题时必须分清问的是” 地址总数” 还是” 可分配数”。

3. 特殊地址的方向性

0.0.0.0 和 **0.host-id** 只能作源地址不能作目的地址；**255.255.255.255** 只能作目的不能作源；**127.x.x.x** 源、目的都可以但不走外网；主机号全 0 的网络地址既不能作源也不能作目的。问” 只能作源” 或” 只能作目的” 时必须逐一核对方位。

4. 划分子网不改变网络号，也不能从 IP 地址判断

从一个 **IP** 地址本身无法判断该主机所连接的网络是否进行了子网划分。划分子网只是把主机号再划分，不改变原有类别归属，也不增加” 网络” 数量——只提高 IP 地址利用率。反之，给定 **IP** 与掩码就能算出子网划分的全部细节。

5. 最长前缀匹配与默认路由

默认路由 `0.0.0.0/0` 与任何目的地址都匹配（全 0 掩码按位与结果必为全 0），但它的前缀长度是 **0**，所以只要有任何其他表项匹配就绝不会选中它。特定主机路由 `/32` 优先级最高。做题时先找出所有匹配项再比前缀长度，不要一看到 **0.0.0.0/0** 就直接下结论。

6. 路由器转发不改变目的 IP 地址

路由器转发分组时只改写链路层的 **MAC** 帧首部（源 **MAC**、目的 **MAC** 逐跳变化），**IP** 首部中的源 **IP** 和目的 **IP** 全程不变（除非经过 NAT 设备，或 TTL 减 1、检验和重算）。综合题问” 经过某路由器后源/目的 IP 是什么” 时，若该路由器不是 NAT 路由器，答案都是” 不变”；” 下一跳地址” 是路由表里的字段，不会被写进 IP 首部。

7. ARP 请求的目标 IP 是网关而非最终目的主机

当目的主机不在同一局域网时，**ARP** 解析的是默认网关（本网段路由器接口）的 **MAC** 地址。另外，**ARP** 响应是单播（只有请求方能收到），不是广播。DHCP 的 OFFER 和 ACK 在链路层是单播（目的 MAC 是客户端），但 **IP** 层目的地址仍是 **255.255.255.255**。

8. RIP 的 16 与” 距离 17”

RIP 中距离等于 **16** 就表示不可达，永远不会出现 **17**。若邻居通告 `<net，16>`，则自己经该邻居也到不了。同时注意：同一邻居通告的路由无条件替换；不同邻居通告且新距离相等时可以添加实现等价负载均衡，新距离更大则不更新。

9. 三类协议的封装方式必须记准

RIP 用 **UDP**（端口 **520**）；**OSPF** 直接封装在 **IP** 数据报中（协议号 **89**），没有经过 **TCP** 或 **UDP**；**BGP** 用 **TCP**（端口 **179**）。从封装角度看 RIP 和 BGP 属于应用层、OSPF 属于网络层；但 RIP 的核心功能是路由选择，属于网际层。这个” 封装层次” 与” 功能层次” 的差别是常见陷阱。

10. IPv6 与 IPv4 的对比细节

IPv6 地址空间是 IPv4 的 2^{96} 倍，不是 **96** 倍；IPv6 基本首部固定 **40 B**；IPv6 的有效载荷长度不包含基本首部但包含扩展首部；IPv6 取消了首部检验和；IPv6 只有源主机能分片；IPv6 不需要 **ARP**（接口标识符 64 bit 足以直接编码硬件地址，地址解析由邻居发现协议 ND 完成）；IPv6 地址类型是单播、多播、任播三种，没有广播类型（广播被看作多播的特例）。

11. 路由聚合的” 引入不存在的网络”

聚合后的地址范围往往大于原有各子网地址范围的并集（这正是聚合能减少表项的原因）。工程上通常可以接受（额外地址范围没有实际主机，分组到达后会因无更具体路由而被丢弃或走默认路由），但在要求” 不引入不存在的网络” 的题目中必须谨慎。判断标准是：待聚合的各网络是否恰好拼成一个完整的 **2** 的幂地址块范围且起点对齐。

12. 拥塞控制与流量控制不要混为一谈

流量控制是发送方与接收方之间的点对点通信量控制，目的是抑制发送速率使接收方来得及接收；拥塞控制关注整个网络的负载，是全局性的。开环控制是静态的预防方法（启动后中途不再修改），闭环控制基于反馈环路，是动态的方法。

第二十三章传输层

一、传输层

本章考纲要求

（一）传输层提供的服务

1. 传输层的功能
2. 传输层寻址与端口
3. 无连接服务与面向连接服务

（二）UDP 协议

1. UDP 数据报
2. UDP 校验

（三）TCP 协议

1. TCP 段
2. TCP 连接管理
3. TCP 可靠传输
4. TCP 流量控制
5. TCP 拥塞控制

传输层是「面向通信部分的最高层，用户功能中的最低层」。它在 408 里的地位非常特殊：选择题年年出，大题几乎每隔一两年就出一次，而且大题往往不是单纯考定义，而是把「序号/确认号推算 + 窗口计算 + 拥塞窗口逐轮演算 + 时间估算」揉在一道题里。也就是说，这一章真正要练的不是背概念，而是四类手算能力：

1. 序号与确认号的推算（连接建立、数据传送、连接释放三个阶段都要会）；
2. 窗口的推算（发送窗口 = min(rwnd, cwnd)，还要能判断窗口内哪些字节能发）；
3. 拥塞窗口的逐轮演算（慢开始、拥塞避免、快重传、快恢复的分支处理）；
4. 时间估算（RTT、MSL、RTO 的加减）。

本章按考纲条目顺序展开，所有计算公式都配上可以自己验算一遍的数字例子。三次握手、四次挥手、窗口滑动、拥塞控制这四块，一律给完整的算例表格。

（一）传输层提供的服务

*1.*传输层的功能

进程到进程的逻辑通信

传输层（**transport layer**）的核心任务是：为运行在不同主机上的应用进程之间提供逻辑通信。注意这里的「逻辑通信」四个字。

从应用层看过去，只要把报文交给下面的传输层，传输层就好像能沿水平方向把它直接送到对方的传输层，哪怕两台主机相隔几千公里。但事实上，两个传输层之间并不存在一条水平方向的物理连

接——数据仍然要一层层向下走，经过网络层、数据链路层、物理层，到达对端后再一层层向上递交。「逻辑通信」的含义就是「好像是这样通信，但事实上并非真的这样通信」。

传输层向上提供的是应用进程之间的通信，所以叫端到端（**end-to-end**）通信；网络层提供的是主机到主机的通信。这两个层次的「端」含义完全不同：

对比项	网络层	传输层
通信主体	主机到主机（host-to-host）	进程到进程（process-to-process，端到端）
标识方式	IP 地址	IP 地址 + 端口号（套接字）
典型协议	IP、ICMP、ARP	TCP、UDP
是否区分进程	不区分，只找到主机	必须区分，交付给具体进程

一句话区分

网络层解决「送到哪台主机」，传输层解决「送到哪台主机的哪个进程」。IP 地址定位主机，端口号定位进程，两者合起来（套接字）才能唯一确定一次进程间通信的端点。

复用与分用

复用（**multiplexing**）：发送方的多个应用进程可以共用同一个传输层协议（TCP 或 UDP）发送数据，传输层给数据加上适当的首部后交给网络层。

分用（**demultiplexing**）：接收方的传输层在剥去报文首部后，依据首部中的端口号把数据正确交付给目的应用进程。

要注意网络层也有复用与分用，但复用的含义不同：

- 网络层的复用：不同协议的数据（TCP 报文段、UDP 用户数据报、ICMP 报文等）都可以封装成 IP 数据报发送出去；
- 网络层的分用：接收方剥去 IP 首部后，依据首部中的协议字段把数据交付给相应的上层协议。

也就是说，网络层按「协议」分用，传输层按「端口」分用。这是统考选择题里反复出现的辨析点。

复用还是分用?

「多个进程共用一个传输层协议」是复用；「依据端口号把数据交给正确的进程」是分用。题目问「UDP 实现分用时依据哪个字段」——答案是目的端口号，不是源端口号。源端口号的作用是让对方知道该把回复送到哪里。

其他功能

传输层还需要承担下面这些功能，其中哪些真正实现，取决于用的是 TCP 还是 UDP：

- 差错检测：TCP 和 UDP 都用「首部 + 数据 + 伪首部」的校验和检测误码。但校验和只能检测误码，不能纠错，更不能保证可靠交付。
- 流量控制：TCP 有，通过接收窗口 rwnd 抑制发送速率；UDP 没有。
- 拥塞控制：TCP 有，通过拥塞窗口 cwnd 防止过多数据注入网络；UDP 没有。
- 可靠传输：TCP 有（确认、重传、排序、去重）；UDP 没有。
- 连接建立与释放：TCP 有；UDP 无连接。

出题角度

2014 年第 39 题考的就是「传输层功能的辨析」:「UDP 提供无连接服务」「UDP 提供复用/分用服务」「UDP

通过差错校验保障可靠数据传输」三个说法中哪个正确——前两个对，第三个错。「有校验和」不等于「可靠传输」，这是本节最经典的陷阱。

不同的操作系统，进程标识符（PID）的命名规则、格式、长度都不一样；更关键的是进程的创建与撤销是动态的，同一个进程在不同时刻的 PID 会发生变化，通信的另一方根本无法预先知道。

为什么用端口号而不用进程号

需要一个独立于操作系统、相对稳定的进程标识——这就是端口号。

端口号引入后，接收方只需「监听某个众所周知的端口」，就可以稳定地接收来自任意主机的请求，而完全不关心对方是哪个操作系统、进程号是多少。

补充

按 OSI 的说法，传输层的服务访问点叫 **TSAP**（**Transport layer Service Access Point**），其具体形式就是传输层端口。各层的 SAP 依次是：物理层 PSAP（物理接口）、数据链路层 DLSAP（MAC 地址）、网络层 NSAP（IP 地址）、传输层 TSAP（端口）、会话层 SLSAP、表示层 PLSAP。这一条属于「层与 SAP 对应关系」的送分点。

*2.*传输层寻址与端口

端口号

端口（**port**）是传输层使用的、用来标识应用进程的软件标识符，也叫端口号（**port number**）。

- 端口号占 **16** 比特，取值范围 **0 ~ 65535**（共 $2^{16} = 65536$ 个）。
- 端口号是软件层面的概念，与硬件无关。它存在于传输层首部中，标识的是「本机协议栈应用层中的某个应用进程」。
- 端口号只具有本地意义：端口号只用来标识本机应用层的进程。因特网中不同计算机上的相同端口号之间没有任何关系，相互独立。
- **TCP** 与 **UDP** 的端口号也是相互独立的。同一个端口号 53 在 UDP 里给 DNS 用，在 TCP 里也可以给 DNS 用，二者互不冲突。

必背

端口号 16 位、取值范围 0~65535、只有本地意义、TCP 与 UDP 端口号相互独立。这四句话几乎每年都有题在考。

三类端口号

类别	范围	说明	举例
熟知端口号（well-known port）	**0 ~ 1023**	由 IANA 统一指派给最重要的应用层协议，固定不变	FTP 21、HTTP 80、DNS 53
登记端口号（registered port）	**1024 ~ 49151**	给没有熟知端口号的应用程序使用，必须在 IANA 登记以防重复	微软远程桌面 3389
短暂端口号（ephemeral port，也叫客户端端口号/临时端口号）	**49152 ~ 65535**	客户端进程在通信时由操作系统临时动态分配，通信结束后被回收并放入空闲端口池	客户端发起连接时使用的端口

三类端口号的边界必须记牢：**1023 / 49151** 是两个分界点。有的教材把第二类写成「1024~49151」、第三类写成「49152~65535」，分界点完全一致。

三类端口号的分界

题目经常这样设陷阱：给一个端口号 65535，问它属于哪一类——属于短暂端口号（客户端），不是登记端口号。再比如问「服务器端使用的端口一定是熟知端口吗」——不一定，服务器进程同样可以使用登记端口号（如 3389）。

套接字

仅仅用「IP 地址 + 端口号」还不能完整描述一次通信，因为一次 TCP 连接是双向的，需要两端各一个端点。

套接字（**socket**）：

$$\text{socket} = (\text{IP 地址} : \text{端口号})$$

TCP 连接由通信两端的两个套接字唯一确定（四元组）：

$$\text{TCP 连接} ::= \{\text{socket}_1, \text{socket}_2\} = \{(\text{IP}_1 : \text{port}_1), (\text{IP}_2 : \text{port}_2)\}$$

这一条解释了两个重要现象：

- 同一台主机上的同一个服务器端口（比如 80）可以同时接受很多客户端连接，因为每个连接的「四元组」不同；
- 反过来，两台主机之间如果四元组完全相同，那就是同一条连接（这也解释了为什么端口号复用一般不会冲突）。

出题角度

常考形式：给出若干条 TCP 连接的四元组，问「哪两条是同一个连接」「这条连接的两端套接字分别是什么」。判断标准就是四元组是否完全一致。

常见协议的熟知端口号表（必背）

应用层协议	端口号	使用的传输层协议	用途要点
FTP	**21**（控制）/ **20**（数据）	TCP	21 传命令，20 传数据（主动模式）
SSH	**22**	TCP	安全远程登录
Telnet	**23**	TCP	远程登录（明文）
SMTP	**25**	TCP	发送邮件（客户 → 服务器）
DNS	**53**	UDP（域名解析）/ TCP（区域传送）	域名解析
DHCP	**67**（服务器）/ **68**（客户端）	UDP	服务器监听 67，客户端用 68 接收
TFTP	**69**	UDP	简单文件传输
HTTP	**80**	TCP	网页明文传输
POP3	**110**	TCP	接收邮件（下载到本地）
IMAP	**143**	TCP	接收邮件（服务器上管理）
HTTPS	**443**	TCP	网页加密传输

应用层协议	端口号	使用的传输层协议	用途要点
SNMP	**161**	UDP	网络管理
RIP	**520**	UDP	路由信息协议

记忆技巧：

- **FTP** 一对：21 发命令、20 传数据；
- 邮件三兄弟：SMTP 25（发）、POP3 110（收）、IMAP 143（收）；
- **DNS 53**：「我要解析（53）域名」；
- **DHCP 67/68**：服务器 67、客户端 68，相邻一对；
- **HTTP 80** 明文、**HTTPS 443** 加密；
- **TFTP 69 / SNMP 161 / RIP 520** 都是 UDP，这三个最容易和 TCP 混淆。

高频陷阱

1. **RIP** 用 **UDP 520** 端口，不是 TCP；BGP 才用 TCP 179。（2017 年、2021 年都出现过这类辨析。）
2. **DNS** 主要用 **UDP 53**，但区域传送（zone transfer）用 TCP 53。题目若说「DNS 只使用 UDP」，那是错的。
3. **DHCP** 两个端口都要记：服务器 67、客户端 68，不要颠倒。
4. **SNMP** 用 **UDP 161**，SNMP Trap 用 UDP 162。

*3.*无连接服务与面向连接服务

传输层向应用层提供两类服务，对应两个协议：

- 面向连接的服务：TCP（Transmission Control Protocol，传输控制协议）。通信前必须先建立连接，通信结束后释放连接，提供可靠的、按序的、面向字节流的服务。
- 无连接的服务：UDP（User Datagram Protocol，用户数据报协议）。通信前不建立连接，提供尽最大努力交付的、不可靠的、面向报文的服务。

TCP 与 UDP 的全面对比

对比项	**UDP**	**TCP**
连接方式	无连接	面向连接（三报文握手建立、四报文挥手释放）
可靠性	不可靠，尽最大努力交付	可靠（确认、重传、排序、去重）
数据组织方式	面向报文，应用层给多长就发多长，一次发一个报文，既不合并也不拆分	面向字节流，把数据看成无结构的字节序列，可自行划分成报文段
首部开销	**8** 字节，固定	**20** 字节固定 + 最多 40 字节选项 = 最长 60 字节
流量控制	无	有（接收窗口 rwnd）
拥塞控制	无	有（拥塞窗口 cwnd，四种算法）
差错检测	有校验和（IPv4 中可选用，全 0 表示不校验）	有校验和（必选，出错则丢弃并等待重传）
是否保证按序到达	不保证	保证

对比项	**UDP**	**TCP**
通信模式	一对一、一对多、多对一、多对多（支持广播与组播）	只能是一对一（单播）
连接状态	无状态	需要维护发送/接收缓存、序号、确认号、拥塞控制参数等大量状态
适用场景	实时性要求高、能容忍少量丢失：DNS、DHCP、TFTP、SNMP、RIP、实时音视频	要求可靠：HTTP、FTP、SMTP、POP3、IMAP、SSH、Telnet

三个最容易错的对比点

1. 「面向报文」与「面向字节流」：UDP 保留应用层报文的边界，一次发一个；TCP 不保留边界，可能把两个小报文合并成一个段发出，也可能把一个大报文拆成多个段。所以 2010 年前后的选择题里「TCP 和 UDP 都可以对报文进行合并拆分」是错的——只有 **TCP** 可以。
2. 校验和覆盖范围：UDP 与 TCP 的校验和都覆盖「伪首部 + 首部 + 数据」，而 **IP** 数据报首部校验和只校验首部，不含数据。
3. 广播/组播：只有 UDP 支持广播和组播，TCP 只支持单播。

（二）**UDP** 协议

1.UDP 数据报

UDP 的特点

- 无连接：发送数据前不需要建立连接，每个 UDP 用户数据报独立处理，互不影响。因此 UDP 不需要像 TCP 那样维护连接状态。
- 尽最大努力交付：不保证可靠，不保证按序，不保证不重复。
- 面向报文：应用层交下来多长的报文，UDP 就原样封装成一个 UDP 用户数据报发送，既不合并也不拆分。因此应用层必须自己选择合适大小的报文——报文太长，交给 IP 层后可能被分片，降低效率；报文太短，首部开销占比过大。
- 支持多种通信模式：一对一、一对多、多对一、多对多，即支持单播、广播、组播。
- 首部开销小：只有 8 字节。
- 无流量控制、无拥塞控制：即使网络已经拥塞，UDP 也不会降低发送速率。这一点在网络拥塞时可能造成大量丢包，但也正是实时业务（语音、视频）需要 UDP 的原因——宁可丢几个包，也不要因为等待重传而引入大延迟。

记忆口诀

UDP 的六条特点：无连接、不可靠、面向报文、支持广播组播、首部小、无流量/拥塞控制。

UDP 用户数据报的格式

UDP 用户数据报 = **8** 字节首部 + 数据部分。首部只有四个字段，每个字段 **2** 字节：

字段	长度	含义
源端口	2 B	发送方进程的端口号。不需要对方回复时可全置 0
目的端口	2 B	接收方进程的端口号，分用就靠这个字段

字段	长度	含义
长度	2 B	UDP 用户数据报的总长度 = 首部 **8 B** + 数据部分长度，单位字节
校验和	2 B	检测整个用户数据报在传输中是否出现误码。全 0 表示不进行校验

长度字段

因为长度字段只有 16 位，所以 UDP 用户数据报的最大总长度是 $2^{16}-1=65535$ 字节，其中数据部分最多 $65535-8=65527$ 字节。「长度」字段的最小值是 **8**（此时数据部分为空）。

「长度」字段的两种问法

题目给「长度字段 = 1500」，则数据部分长度 = **1500 − 8 = 1492** 字节；题目给「UDP 数据部分 1500 字节」，则长度字段 = **1500 + 8 = 1508**。这两个方向都要会，也是选择题的经典送分/送命点。

UDP 不保证可靠交付

因为 UDP 没有确认与重传机制，所以：

- 数据报丢失后不会重传；
- 数据报到达顺序可能与发送顺序不同；
- 数据报可能重复到达。

校验和只能检测误码。接收方发现校验出错时，通常直接丢弃该数据报，不产生任何差错报告（注意：UDP 本身不会因此发送 ICMP 差错报文，但若目的端口不可达，接收方的 IP 层可能回送 ICMP 端口不可达——这是 IP 层的职责，不要混为一谈）。

2.UDP 校验

校验和的计算范围

UDP 的校验和覆盖三个部分：

$$\text{校验范围} = 12\text{ 字节伪首部} + 8\text{ 字节 UDP 首部} + \text{UDP 数据部分}$$

若这三部分的总长度不是偶数个字节，还要在末尾补 **1** 个全 **0** 字节凑成偶数（因为要按 2 字节为单位分组相加）。补充的全 **0** 字节只用于计算校验和，不发送。

伪首部（**pseudo-header**）

伪首部共 **12** 字节，结构如下：

字段	长度	取值
源 IP 地址	4 B	取自 IP 数据报首部，不是「本机 IP」的抽象概念，而是实际发送用的那个源地址
目的 IP 地址	4 B	取自 IP 数据报首部
全 0 字段	1 B	保留，置 0（有教材称「保留字节」）
协议号	1 B	**UDP 为 17**
UDP 长度	2 B	UDP 用户数据报的总长度（首部 + 数据），与首部中「长度」字段的值相同

字段	长度	取值
合计	**12 B**	

伪首部的作用（必背）

伪首部的作用是让传输层能够检查数据是否被错误地交付到了本主机、本协议。它把源 IP、目的 IP、协议号一起纳入校验范围，于是：

1. 如果 IP 数据报被错误地投递到错误的地址（目的 IP 不符），校验和会失败；
2. 如果 IP 数据报被错误地交给了错误的协议（协议号不符），校验和也会失败。

伪首部既不在网络中传送（发送方算完就丢弃），也不向上层递交，它只在两端计算校验和时临时构造。这一点是选择题的高频正确选项。

校验和的计算过程

发送方计算步骤：

1. 把 UDP 首部中的校验和字段置为 **0**；
2. 在 UDP 用户数据报前面临时加上 12 字节伪首部；
3. 把「伪首部 + 首部 + 数据」按 **2** 字节（**16** 位）为一组划分成若干个「字」。若总长度是奇数个字节，在末尾补 1 个全 0 字节；
4. 对所有 16 位字进行二进制反码求和：逐字相加，每次相加若最高位产生进位，就把进位回卷加回到最低位（cyclic carry）；
5. 把求和结果逐位取反，写入 UDP 首部的校验和字段。

接收方校验步骤：

1. 给收到的 UDP 用户数据报前面加上 12 字节伪首部；
2. 同样按 2 字节分组，做二进制反码求和；
3. 把结果取反。若结果为 **0**，说明传输无误码；否则说明出现误码，丢弃该数据报。

全 0 的含义

发送方把校验和字段置为全 **0**，表示「不进行校验」（这是 IPv4 下 UDP 允许的，IPv6 下 UDP 校验和是必选的）。接收方收到校验和字段为全 0 的数据报，就不做校验，直接交付。

另外还有一个容易忽略的细节：如果校验和的计算结果恰好是全 **0**，那么应当写入全 **1**（即 **0xFFFF**），而不是全 **0**。因为全 0 已经被「不校验」这个特殊含义占用了。这就是所谓「全 **0** 表示不校验，全 **1**（即零的反码）表示校验和为 **0**」。

完整算例（可以自己验算一遍）

设某 UDP 用户数据报的各字段（用十六进制表示）：

- 源 IP 地址 = `C0A8 0101`（即 192.168.1.1）
- 目的 IP 地址 = `0A00 0005`（即 10.0.0.5）
- 协议号 = `0011`（即十进制的 17）
- UDP 长度 = `000C`（即 12 字节：8 字节首部 + 4 字节数据）
- 源端口 = `1F90`，目的端口 = `0053`
- 数据部分 = `0102 0304`（4 字节）

第一步：构造需要参与运算的全部 16 位字。

序号	内容	**16** 位十六进制
①	源 IP 高位	C0A8
②	源 IP 低位	0101
③	目的 IP 高位	0A00
④	目的 IP 低位	0005
⑤	全 0 字节 + 协议号	0011
⑥	UDP 长度	000C
⑦	源端口	1F90
⑧	目的端口	0053
⑨	长度字段	000C
⑩	校验和（置 0）	0000
⑪	数据第 1 个 16 位	0102
⑫	数据第 2 个 16 位	0304

（第 ①~⑥ 项合起来正是 12 字节伪首部，第 ⑦~⑩ 项是 UDP 的 8 字节首部，第 ⑪⑫ 项是数据部分。）

注意两个「长度」

⑥ 是伪首部里的 UDP 长度，⑨ 是 UDP 首部里的长度字段，两处必须是同一个值（本题都是 000C），但它们是两个不同的字段，做加法时两个都要算进去，不要漏掉其中一个，也不要误以为重复而只算一次。

第二步：二进制反码求和（用十六进制加法）。

```
  C0A8
+ 0101
= C1A9
+ 0A00
= CBA9
+ 0005
= CBAE
+ 0011
= CBBF
+ 000C
= CBCB
+ 1F90
= EB5B
+ 0053
= EBAE
+ 000C
= EBBA
+ 0000
= EBBA
+ 0102
= ECBC
+ 0304
= EFC0
```

求和结果为 EFC0，过程中没有产生 16 位溢出，因此无需回卷。

第三步：取反。

$$\overline{\texttt{EFC0}} = \texttt{103F}$$

所以校验和字段应写入 `103F`。

第四步：接收方验证。把全部字（含 `103F`）再加一遍：

```
  EFC0          （前面全部字之和）
+ 103F          （校验和字段）
= FFFF
```

结果为全 1，取反后为全 0，说明传输无误码。✓

回卷进位算例（考得比上面这个还多）

上面的算例比较「小」，恰好没有出现 16 位溢出。但真实题目里几乎一定会溢出，所以「回卷」这一步必须练熟。设求和的中间结果为 `B9B6`（已包含前面所有字，尚未取反），此时还剩最后一个 16 位字 `65C5` 需要加入：

```
  1011 1001 1011 0110   (0xB9B6)
+ 0110 0101 1100 0101   (0x65C5)
-------------------------
1 0001 1111 0111 1011   (0x11F7B)   <- 产生了 1 位溢出进位
```

回卷：把溢出的最高位进位 1 加回最低位：

```
  0001 1111 0111 1011   (0x1F7B)
+                    1
-------------------------
  0001 1111 0111 1100   (0x1F7C)   <- 反码和
```

取反得到校验和：

```
  0001 1111 0111 1100   (0x1F7C)
-> 1110 0000 1000 0011   (0xE083)   <- 校验和
```

验证：把 `0x1F7C` 与 `0xE083` 相加，$1F7C + E083 = FFFF$，取反为 0，正确。✓

回卷的两种等价写法

1. 文字写法：先把全部字相加，若结果超过 16 位，把高位进位加到低 16 位上——这就是「回卷（cyclic carry）」；
2. 工程写法：每加一个字就检查一次进位并立刻回卷（本例就是这样做的）。两种写法结果完全一致。

做题时的顺序一定要记住：先全部反码求和（含回卷）→ 再取反 → 写入校验和字段。取反必须放在最后，先把中间结果取反再相加就错了。

校验和计算题的三个坑

1. 做加法时忘了把伪首部算进去。伪首部一定要算，而且 UDP 的协议号是 **17**（TCP 是 6）。
2. 忘了回卷进位。只要 16 位相加产生进位，就必须把进位加回到最低位。2024 年第 39 题就专门考了这个「中间结果 + 最后一个字」的回卷与取反。
3. 最后一步忘了取反，或者取反后忘了「全 0 要写成全 1」。

TCP 校验与 UDP 校验的差别

TCP 的校验和计算过程与 UDP 完全相同（反码求和 + 回卷 + 取反），只有两处不同：

- 伪首部中的协议号是 **6**（而不是 17）；
- 伪首部中的长度字段是 **TCP** 报文段的长度（而不是 UDP 长度）。

TCP 的校验和是必选的（不能用「全 0」表示不校验），而出错时同样只是丢弃，由超时重传机制来弥补。

出题角度

UDP 校验是选择题的常客，典型问法有三类：①「伪首部的作用是什么」②「校验和的覆盖范围包括什么」③「给出若干 16 位字，求校验和」。第 ③ 类只需老老实实做反码加法。另外还会考「全 0 校验和表示什么」——表示不校验，不是「校验通过」。

（三）TCP 协议

1.TCP 段

TCP 的主要特点

- 面向连接：通信前必须先建立连接，通信结束后必须释放连接。
- 只能一对一：每条 TCP 连接只能有两个端点（两个套接字），因此不支持广播和组播。
- 可靠交付：无差错、不丢失、不重复、按序到达。
- 全双工通信：TCP 连接的两端都设有发送缓存和接收缓存，数据可以双向同时传输。
- 面向字节流：TCP 把应用层交下来的数据看成一连串无结构的字节流。TCP 不保留应用层报文的边界，发送方可以根据情况把字节流划分成若干个大小合适的报文段，接收方再把它们还原成字节流。

面向字节流的三个推论

1. 序号以字节为单位编号，不是以报文段为单位。一个报文段的序号 = 该报文段数据部分第一个字节的序号。
2. 应用层的一次「写」操作，可能对应多个 TCP 报文段；多次「写」操作，也可能被合并进一个 TCP 报文段。
3. TCP 连接的两端只是「字节流的两端」，TCP 自己并不知道这次传输在应用层是一条消息还是半条消息。

TCP 报文段的首部格式

TCP 报文段 = 首部 + 数据载荷。首部由固定首部 **20** 字节和选项 + 填充（最多 **40** 字节）组成，因此 TCP 首部最长 **60** 字节。

字段	长度	含义
源端口	2 B	发送方进程端口号
目的端口	2 B	接收方进程端口号
序号 **seq**	4 B	本报文段数据载荷的第一个字节的序号，取值 $0 \sim 2^{32}-1$，到最大值后回到 0
确认号 **ack**	4 B	期望收到对方下一个报文段的第一个字节的序号；仅在 ACK=1 时有效

字段	长度	含义
数据偏移	4 bit	指出 TCP 报文段数据载荷的起始位置距报文段起始位置有多远，即首部长度，以 **4** 字节为单位
保留	6 bit	保留今后使用，目前置 0（新版标准中部分位已被 ECE、CWR 占用，用于拥塞控制通知）
URG	1 bit	紧急标志位。URG=1 时紧急指针字段有效
ACK	1 bit	确认标志位。ACK=1 时确认号字段才有效；连接建立后所有报文段都必须把 **ACK 置 1**
PSH	1 bit	推送标志位。置 1 表示接收方应尽快交付应用进程，不必等接收缓存填满
RST	1 bit	复位标志位。置 1 表示连接出现严重差错，必须释放连接后重新建立；也用来拒绝非法报文段或拒绝打开连接
SYN	1 bit	同步标志位。用于建立连接。SYN=1 且 ACK=0 是连接请求；SYN=1 且 ACK=1 是连接接受
FIN	1 bit	终止标志位。置 1 表示发送方数据已全部发完，要求释放连接
窗口	2 B	指本报文段发送方自己的接收窗口大小（接收缓存可用空间），以字节为单位，用于流量控制。最大 65535 字节
校验和	2 B	覆盖伪首部 + TCP 首部 + 数据载荷；计算方法与 UDP 相同，但协议号为 6
紧急指针	2 B	URG=1 时有效，以字节为单位指出本报文段中紧急数据的长度
选项	可变	最长 40 B。常用：MSS 选项、窗口扩大选项、时间戳选项、选择确认 SACK 选项
填充	可变	使首部长度能被 4 字节整除

序号与确认号：本节第一重点

这是 TCP 全部计算题的根基，必须彻底分清。

序号 **seq**（**sequence number**）：本报文段数据载荷的第一个字节的序号。如果本报文段不携带数据，则「不消耗序号」——但 SYN 和 FIN 是例外，见后文。

确认号 **ack**（**acknowledgement number**）：期望收到对方下一个报文段的第一个数据字节的序号，同时也是对「到 ack−1 为止的所有数据都已正确收到」的确认。

序号与确认号一句话区分

- **seq** 说的是「我发的这段数据从第几号字节开始」——回答「我在说什么」；
- **ack** 说的是「我想听你从第几号字节往下讲」——回答「我听到哪儿了」。

若确认号为 N，则表明：序号到 $N-1$ 为止的所有数据都已正确接收。

与 **GBN** 的区别（易错）：在后退 N 帧协议中，ACK n 表示「序号到 n 为止的数据已正确接收，期望收到 $n+1$」；而 **TCP** 中 **ack** $= N$ 表示「到 $N-1$ 为止已正确接收，期望收到 N」。两者的下标含义差 1，千万不要混用。

示例：甲向乙发送一段 300 字节的数据，seq=201。这段数据占用序号 201~500。若乙正确收到，则乙回复的确认号 ack $= 501$。

序号取值规则（必背，覆盖握手/挥手/数据三个阶段）

报文类型	是否消耗序号
SYN 报文段（握手第 1、2 次）	消耗 **1** 个序号
FIN 报文段（挥手第 1、3 次）	消耗 **1** 个序号
纯 ACK 报文段（不携带数据）	不消耗序号
携带有数据的报文段	消耗的序号数 = 数据字节数
握手第 3 次的 ACK	携带数据则消耗相应字节数；不携带数据则不消耗（下一报文段序号仍为 x+1）
挥手第 4 次的 ACK	规定不能携带数据，也不消耗序号

数据偏移

- 占 **4** 比特，以 **4** 字节为单位。
- 因为 4 比特最大能表示 15，所以 TCP 首部最长 $15 \times 4 = 60$ 字节。
- 固定首部 20 字节，对应数据偏移 $= 20/4 = 5$，二进制 `0101`。
- 因为首部长度必须是 4 字节的整数倍，所以若「固定首部 + 选项」的长度不能被 4 整除，要用填充字段补足。

数据偏移 vs IP 首部长度 vs IP 片偏移

三个「偏移」含义完全不同，题目最爱混着考：

- **TCP** 数据偏移：TCP 首部长度，单位 4 字节，最小值 5、最大值 15；
- **IP** 首部长度：IP 首部长度，单位 4 字节，最小值 5、最大值 15；
- **IP** 片偏移：该分片的数据在原始数据报中的相对位置，单位 **8** 字节。

六个标志位

标志位	名称	作用
URG	紧急	置 1 时紧急指针有效，紧急数据「插队」传到接收方并直接上交应用进程，不等接收缓存排队。即使窗口为 0 也可以发送紧急数据
ACK	确认	置 1 时确认号才有效。连接建立后所有报文段都必须置 **1**
PSH	推送	置 1 时接收方尽快交付应用进程，不必等缓存填满。用于交互式通信（如 Telnet）
RST	复位	置 1 表示连接出现严重差错，须释放并重建；也用于拒绝非法报文段或拒绝打开连接
SYN	同步	用于建立连接。**SYN=1** 的报文段不能携带数据，但要消耗一个序号
FIN	终止	置 1 表示数据发完，要求释放连接。**FIN=1** 的报文段即使不携带数据也要消耗一个序号

出题角度

- 「主机 A 的应用进程突然崩溃，随后 A 向 B 发送了一个 TCP 报文段」——该报文段的 **RST=1**（B 收到后释放连接）。
- 「哪个标志位用于拒绝非法的 TCP 报文段」——**RST**。
- 「连接建立后哪个标志位始终为 1」——**ACK**。

窗口、紧急指针、选项与填充

窗口字段：占 2 字节，指出「发送本报文段的一方的接收窗口」大小，即本机接收缓存的可用空间，用来告诉对方「你最多还能给我发多少字节」。这是 TCP 流量控制的实现手段。

紧急指针字段：占 2 字节，以字节为单位指出本报文段数据载荷中紧急数据的长度。紧急数据位于数据载荷的最前面，其后是普通数据。

选项字段：最长 40 字节。常见选项有：

- 最大报文段长度 **MSS** 选项：在连接建立时协商；
- 窗口扩大选项：把窗口字段从 16 位扩展到更大（因为 65535 字节的窗口对高带宽时延积链路太小）；
- 时间戳选项：用于更精确地计算 RTT；
- 选择确认 **SACK** 选项：用于选择性重传。

填充字段：使首部长度能被 4 字节整除。

MSS 与 **MTU** 的关系

三个缩写必须分清：

缩写	全称	所在层	含义
MTU	Maximum Transmission Unit，最大传送单元	数据链路层	一个帧的数据字段能装下的最大字节数
MSS	Maximum Segment Size，最大报文段长度	传输层（TCP）	TCP 报文段数据载荷部分的最大长度，不包括 **TCP** 首部
MSL	Maximum Segment Lifetime，最长报文段寿命	传输层（TCP）	一个 TCP 报文段在网络中最长的生存时间

$$\text{MSS} = \text{MTU} - \text{IP 首部长度} - \text{TCP 首部长度}$$

以最常见的以太网为例：MTU = 1500 B，IP 首部 20 B，TCP 首部 20 B，则

$$\text{MSS} = 1500 - 20 - 20 = 1460\ \text{B}$$

MSS 相关的四个坑

1. **MSS** 只算数据载荷，不算首部。题目说「MSS = 1000 B，发送 3000 B 的报文」，那就是 3 个报文段，数据分别为 1000、1000、1000。
2. **MSS** 与窗口值无关。窗口是「还能收多少字节」，MSS 是「一段最多装多少字节」。
3. **MSS** 选项只在连接建立阶段协商，不在每一个报文段里出现。
4. **MTU** 是链路层的概念，**MSS** 是传输层的概念，问「MSS 在哪一层确定」答传输层（TCP 选项），问「MTU 由谁决定」答数据链路层。

TCP 发送报文段的时机

正常情况下，TCP 什么时候把数据封装成报文段发出去？规则有四条：

1. 发送缓存中存放的数据达到 **MSS** 字节时，就组装成一个 TCP 报文段发送出去；

2. 应用进程指明要求发送（即 TCP 支持的 **PUSH** 推送操作）；
3. 发送方的一个计时器期限到了，即使没达到 MSS 也把当前已有的数据发送出去；
4. 若接收窗口小于 MSS，则应等待接收窗口达到 MSS 的一半或者 MSS，两者任一时机成熟再发送。

Nagle 算法：对于一个 TCP 连接，第一个字节立即发送出去，然后继续缓存要发送的数据；只有收到上一个报文段的确认后，才发送已缓存的数据。这样每轮都要等上一个报文段的数据全部被确认，才发下一个报文段，可以有效地减少小报文段（避免「糊涂窗口综合征」）。

Nagle 算法算例：应用层连续产生 3 次「写」操作，每次 1 字节，准备发给对方（假设每次写之间的间隔很短）。

时刻	事件	发送方动作
t_0	第 1 个字节到达发送缓存	Nagle 算法规定第一个字节立即发送，不等待
t_1	第 2 个字节到达发送缓存	第 1 个字节的确认尚未到达，于是缓存，不发送
t_2	第 3 个字节到达发送缓存	仍然缓存，此时缓存里有 2 字节
t_3	收到第 1 个字节的确认	把缓存中的 2 字节组装成一个报文段发送出去
t_4	又收到确认	缓存为空，等待下一次数据

对比：若不用 Nagle 算法，则可能发出 3 个各含 1 字节数据的报文段，每个都要带 20 字节 TCP 首部 + 20 字节 IP 首部，首部开销是数据的 **40** 倍，效率极低。Nagle 算法把「小报文段问题」从发送侧抑制住了。

两个边界情况：

- 若应用层数据攒够了 MSS，则立即发送，不受 Nagle 算法限制；
- 若应用层设置了 TCP_NODELAY 选项（禁用 Nagle），或使用了 PSH 推送操作，也立即发送。

2.TCP 连接管理

TCP 连接管理 = 连接建立（三报文握手） + 连接释放（四报文挥手） + 状态转换。这是全章最重要的一节，也是历年大题的主要素材。

三报文握手建立连接

设客户（Client）A 主动打开，服务器（Server）B 被动打开（先处于 LISTEN 监听状态）。

第 **1** 次握手（**A** $\rightarrow$ **B**）：A 向 B 发送 **TCP** 连接请求报文段：

$$\text{SYN} = 1, \quad \text{ACK} = 0, \quad \text{seq} = x$$

- 其中 x 是 A 选择的初始序号；
- **SYN** 报文段不能携带数据，但要消耗一个序号；
- A 进入 **SYN-SENT**（同步已发送）状态。

第 **2** 次握手（**B** $\rightarrow$ **A**）：B 若同意建立连接，则向 A 发送 **TCP** 连接请求确认报文段：

$$\text{SYN} = 1, \quad \text{ACK} = 1, \quad \text{seq} = y, \quad \text{ack} = x + 1$$

- y 是 B 独立选择的初始序号；
- `ack = x+1` 表示 B 已正确收到 A 的 SYN（它占了序号 x），期望 A 下一个报文段从 $x+1$ 开始；
- 同样，这个报文段也不能携带数据，也要消耗一个序号；
- B 进入 **SYN-RCVD**（同步已接收）状态。

第 **3** 次握手（**A** $\rightarrow$ **B**）：A 收到 B 的确认后，还要再向 B 发送一个 **ACK** 报文段（普通确认，不带 SYN）：

$$\text{ACK}=1,\quad \text{seq}=x+1,\quad \text{ack}=y+1$$

- `ack = y+1` 表示 A 已正确收到 B 的 SYN（它占了序号 y）；
- A 进入 **ESTABLISHED**（连接已建立）状态；
- B 收到这个 ACK 后也进入 **ESTABLISHED** 状态，连接正式建立。

三报文握手的三个报文段

次序	方向	**SYN**	**ACK**	**seq**	**ack**
1	A→B	1	0	x	无意义
2	B→A	1	1	y	$x+1$
3	A→B	0	1	$x+1$	$y+1$

> 第 **1**、**2** 次握手绝对不能携带数据；第 **3** 次握手可以携带数据（若携带数据则按数据字节数消耗序号，且此时 seq 从 $x+1$ 开始；若不携带数据，则不消耗序号，下一个数据报文段的序号仍是 $x+1$）。

为什么是三次握手，而不是两次

这是必考的一道「原理题」，标准答案的核心是：防止已失效的连接请求报文段突然又传到服务器，造成服务器资源浪费。

设想只用两次握手的情形：

1. A 发出第 1 个连接请求，但在某个网络结点长时间滞留；
2. A 超时后重发连接请求，第 2 个请求顺利到达 B，两次握手完成，连接建立、传送数据、然后释放；
3. 之后，那个滞留已久的第 **1** 个请求报文段终于到达 B；
4. B 收到这个「已失效的连接请求」，误以为 A 又要建立连接，于是发出确认并直接进入连接状态，为这个根本不存在的连接分配资源、等待数据；
5. 而 A 并不知道这件事，不会给 B 发数据。B 的资源就被白白浪费（甚至长期占用）。

三次握手可以杜绝这种情况：B 在第 2 次握手时只是「同意」，并不立即认为连接建立；A 收到 $(\text{SYN}=1,\text{ACK}=1,\text{ack}=x+1)$ 后会发现——我并没有请求建立这个连接呀！于是 A 不发送第 **3** 次握手的确认，B 收不到确认，就会超时释放这个半开的连接，资源不会被长期占用。

补充：三次握手的另一个作用

三次握手还使双方都能确认对方的接收能力和发送能力都正常，并协商初始序号（以及 MSS、窗口扩大因子等参数）。但从统考答题的角度看，第一位的理由是「防止已失效的连接请求报文段造成资源浪费」。

四报文挥手释放连接

TCP 连接是全双工的，因此每个方向都必须单独释放，这就是「四次挥手」的根本原因。设 A 主动关闭，B 被动关闭。

第 **1** 次挥手（**A** $\rightarrow$ **B**）：A 发送连接释放报文段，并停止发送数据：

$$\text{FIN} = 1, \quad \text{seq} = u$$

- **FIN** 报文段即使不携带数据，也要消耗一个序号；
- A 进入 **FIN-WAIT-1**（终止等待 **1**）状态。

第 **2** 次挥手（**B** $\rightarrow$ **A**）：B 收到后发出确认：

$$\text{ACK} = 1, \quad \text{seq} = v, \quad \text{ack} = u + 1$$

- B 进入 **CLOSE-WAIT**（关闭等待）状态；
- A 收到此报文后进入 **FIN-WAIT-2**（终止等待 **2**）状态。

此时连接处于半关闭（**half-close**）状态：A 已经没有数据要发送了，但 **B** 仍然可以继续向 **A** 发送数据，A 也必须继续接收。这个阶段可能持续很久（B 还有大量数据没发完）。

第 **3** 次挥手（**B** $\rightarrow$ **A**）：B 把剩余数据全部发送完毕后，发送连接释放报文段：

$$\text{FIN} = 1, \quad \text{ACK} = 1, \quad \text{seq} = w, \quad \text{ack} = u + 1$$

- 注意这里的 `ack` 仍然是 $u + 1$（因为 A 的 FIN 之后没有新数据，B 不需要更新确认号；这就是所谓「对之前收到的连接释放报文段的重复确认」）；
- B 进入 **LAST-ACK**（最后确认）状态。

第 **4** 次挥手（**A** $\rightarrow$ **B**）：A 收到 B 的 FIN 后发出确认：

$$\text{ACK} = 1, \quad \text{seq} = u + 1, \quad \text{ack} = w + 1$$

- 这个报文段规定不能携带数据，也不消耗序号；
- A 进入 **TIME-WAIT**（时间等待）状态，必须等待 **2MSL** 后才能进入 **CLOSED** 状态；
- B 收到这个 ACK 后立即进入 **CLOSED** 状态。

四报文挥手汇总

次序	方向	**FIN**	**ACK**	**seq**	**ack**	发送方状态
1	A→B	1	1	u	v（无实际意义）	A 进 FIN-WAIT-1
2	B→A	0	1	v	$u+1$	B 进 CLOSE-WAIT，A 进 FIN-WAIT-2
3	B→A	1	1	w	$u+1$	B 进 LAST-ACK
4	A→B	0	1	$u+1$	$w+1$	A 进 TIME-WAIT，B 进 CLOSED

挥手第 **1**、**3** 次（**FIN**）固定消耗一个序号；挥手第 **4** 次的 **ACK** 不携带数据也不消耗序号。

为什么是四次挥手

因为 TCP 连接是全双工的，两个方向的数据传输相互独立，因此「我发完了」与「你发完了」必须分别通知、分别确认：

- A 发 FIN 只表示「**A** 没有数据要发给 **B** 了」，不表示 B 也没有数据要发给 A；
- B 收到 FIN 后必须先回一个 ACK（第 2 次挥手），但 B 可能还有数据没发完，所以不能立刻把自己的 **FIN** 也搭车发过去；
- 必须等 B 的数据全部发完，B 才能发自己的 FIN（第 3 次挥手），A 再确认（第 4 次挥手）。

什么时候可以合并成三次?

如果 B 收到 A 的 FIN 时恰好已经没有数据要发送了，那么 B 可以把第 2 次和第 3 次挥手合并成一个报文段（FIN=1 且 ACK=1）发出。此时总共只有三个报文段，但逻辑上仍然是四次挥手。

这个「可以合并」的假设是许多时间计算题的前提。

为什么需要 **TIME-WAIT** 与 **2MSL**

MSL（**Maximum Segment Lifetime**，最长报文段寿命）：一个 TCP 报文段在网络中最长的生存时间。RFC 793 建议为 2 分钟，实际由操作系统决定，通常在几十秒到几分钟之间。

TIME-WAIT 计时器设置为 **2MSL**，原因有两个：

原因一：确保 **A** 发送的最后一个 **ACK**（第 **4** 次挥手）能够到达 **B**。

如果 A 发完最后一个 ACK 就立即关闭并进入 CLOSED，那么这个 ACK 万一丢失，B 就会一直停留在 LAST-ACK 状态并超时重传 **FIN+ACK**（第 3 次挥手）。此时 A 已经关闭，收不到也回不了这个报文，B 就永远无法正常释放连接。

正确做法是：A 进入 TIME-WAIT 并计时 2MSL。在这段时间内：

- 若 A 又收到了 B 重传的 FIN+ACK，就说明自己刚才发的 ACK 丢了，于是重发 **ACK** 并重新开始 **2MSL** 计时；
- 一个报文段从 A 到 B 最多用 1 MSL，B 重传的报文段从 B 回 A 也最多用 1 MSL，因此 2MSL 足以覆盖「ACK 丢失 + FIN 重传到达」的最坏情况。

原因二：让本次连接中产生的所有报文段都从网络中消失。

各 TCP 报文段到达主机的时间长短不一，A 在收到 B 的 FIN 后，可能还有本次连接中较早发出的报文段仍在网络中游荡。如果 A 立即关闭，然后又马上用相同的四元组建立新连接，这些旧报文段就可能被新连接误收，造成数据混乱。

经过 2MSL（一个报文段一来一回的最大时间），可以保证本次连接产生的所有报文段都已从网络中消失，这时再建立新连接就不会冲突。

TIME-WAIT 的三个易错点

1. 只有主动关闭方进入 **TIME-WAIT**。被动关闭方（先收到 FIN 的一方）收到最后一个 ACK 后立即进入 CLOSED。
2. **TIME-WAIT** 占用的是「端口」而非「连接」：服务器重启时若报「Address already in use」，正是因为 TIME-WAIT 尚未结束。
3. **2MSL** 中的 **MSL** 不是 **RTT**。题目若给「RTT = 50 ms，MSL = 800 ms」，则 2MSL = 1600 ms，与 RTT 无关。

连接释放的最短时间（经典算例）

情形	主动关闭方从发出 **FIN** 到进入 **CLOSED**	被动关闭方从收到 **FIN** 到进入 **CLOSED**
最短（B 无数据要发，第 2、3 次挥手合并）	**1 RTT + 2 MSL**	**1.5 RTT**
一般情形	大于 1 RTT + 2 MSL	大于 1.5 RTT

为什么主动方是 **1 RTT + 2MSL**：FIN 发出后经 0.5 RTT 到达 B，B 立即回 FIN+ACK，再经 0.5 RTT 到达 A，共 1 RTT；A 随后发最后一个 ACK 并进入 TIME-WAIT，等待 2 MSL。

为什么被动方是 **1.5 RTT**：B 收到 FIN 用 0.5 RTT，立刻发 FIN+ACK；这个确认 + FIN 经 0.5 RTT 到达 A，A 发最后一个 ACK；这个 ACK 再经 0.5 RTT 到达 B，B 才进入 CLOSED。合计 $0.5+0.5+0.5=1.5$ RTT。

2022 年第 **39** 题（原文条件）

客户 C 和服务器 S 已建立 TCP 连接，RTT = 50 ms，MSL = 800 ms，C 主动请求断开连接。从 C 发出 FIN 的时刻算起，C 和 S 进入 CLOSED 状态所需的时间至少分别是多少?

解：单向传播时延 = RTT/2 = 25 ms。最短情形下 S 收到 C 的 FIN 后立即发出 ACK 和 FIN（合并为一次）。

- **C**：50 ms 时收到 S 的 FIN+ACK，发出最后一个 ACK 并进入 TIME-WAIT；等待 2MSL $=$ 1600 ms 后关闭。总时间 $=50+1600=$ **1650 ms**。
- **S**：25 ms 时收到 C 的 FIN 并发出 FIN+ACK；75 ms 时收到 C 的最后一个 ACK，随即进入 CLOSED。总时间 $=$ **75 ms**。

答案：**1650 ms**，**75 ms**。

TCP 的状态转换

TCP 连接在其生存期中会处于不同的状态。统考不要求背完整的 11 状态图，但下面这些状态必须能把「谁在哪个状态、接下来往哪走」对应上。

状态	含义	谁进入
CLOSED	关闭（虚拟的起始/终止状态）	双方初始
LISTEN	监听，等待对方的连接请求	被动打开一方（服务器）
SYN-SENT	已发送连接请求，等待对方确认	主动打开一方，发完第 1 次握手
SYN-RCVD	已收到连接请求并发出确认，等待对方确认	服务器，发完第 2 次握手
ESTABLISHED	连接已建立，可以传输数据	双方，握手完成
FIN-WAIT-1	已发送 FIN，等待对方确认	主动关闭方，发完第 1 次挥手
FIN-WAIT-2	已收到对方对自己 FIN 的确认，等待对方的 FIN	主动关闭方，收到第 2 次挥手
CLOSE-WAIT	已收到对方的 FIN 并确认，等待本进程发完剩余数据	被动关闭方，发完第 2 次挥手
LAST-ACK	已发送自己的 FIN，等待对方确认	被动关闭方，发完第 3 次挥手
TIME-WAIT	已发送最后一个 ACK，等待 2MSL	主动关闭方，发完第 4 次挥手
CLOSING	双方同时关闭时的罕见状态（不常考）	罕用

客户端（主动打开 + 主动关闭）的典型路径：

```
CLOSED -> SYN-SENT -> ESTABLISHED -> FIN-WAIT-1 -> FIN-WAIT-2 -> TIME-WAIT -> CLOSED
```

服务器（被动打开 + 被动关闭）的典型路径：

```
CLOSED -> LISTEN -> SYN-RCVD -> ESTABLISHED -> CLOSE-WAIT -> LAST-ACK -> CLOSED
```

状态题的两个陷阱

1. **LISTEN** 是服务器在收请求之前的状态，不是「握手第一步之后」的状态。收到 SYN 之后才进 SYN-RCVD。
2. **FIN-WAIT-2** 是「我已经挥手了，等对方也挥手」，此时连接处于半关闭；CLOSE-WAIT 才是「对方挥手了，我还没挥」。

连接建立与释放中的序号计算（完整例题）

例题 **1**（握手序号，**2019** 年第 **39** 题原型）

主机甲主动发起与主机乙的 TCP 连接，甲、乙选择的初始序号分别为 2018 和 2046。问第三次握手 TCP 段的确认序列号是多少?

- 第 1 次握手：甲 $\rightarrow$ 乙，`SYN=1，seq=2018`。SYN 消耗一个序号，所以甲的数据将从 2019 开始。
- 第 2 次握手：乙 $\rightarrow$ 甲，`SYN=1，ACK=1，seq=2046，ack=2018+1=2019`。
- 第 3 次握手：甲 $\rightarrow$ 乙，`ACK=1，seq=2019，ack=2046+1=2047`。

答案：**2047**。

例题 **2**（握手序号，**2011** 年第 **39** 题原型）

主机甲向主机乙发送一个（`SYN=1，seq=11220`）的 TCP 段请求建立连接。若乙接受，则乙回送的 TCP 段可能是什么?

- 第 2 次握手必须同时置 `SYN=1` 和 `ACK=1`；
- 乙必须确认甲的 SYN：`ack = 11220 + 1 = 11221`；
- 乙自己的初始序号可以独立选择，题目没有限制，取 11221 是可能的。

答案：**（`SYN=1，ACK=1，seq=11221，ack=11221`）**。

例题 **3**（含数据传送与挥手，**2020** 年第 **39** 题原型）

甲、乙建立 TCP 连接时，甲发送的 SYN 段序号为 1000；断开连接时，甲发送给乙的 FIN 段序号为 5001。求在无任何重传的情况下，甲向乙已发送的应用层数据字节数。

- SYN 占用序号 1000，因此第一个应用数据字节的序号是 **1001**；
- FIN 的序号 5001 表示「写完这些数据之后的下一个序号是 5001」，即数据占用到 5000；
- 数据字节数 $= 5001 - 1001 =$ **4000** 字节。（FIN 本身虽消耗一个序号，但位于应用数据之后，不计入数据量。）

答案：**4000 B**。

例题 **4**（含数据传送的序号，**2013** 年第 **39** 题原型）

甲收到乙发来的一个 TCP 段，其 seq=1913、ack=2046、有效载荷 100 字节。求甲立即回送给乙的 TCP 段的序号与确认号。

- 乙的 ack=2046 表示乙期望甲下一次从序号 2046 开始发送，因此甲回发段的 **seq = 2046**；
- 乙本段数据从 1913 开始、长达 100 字节，占用序号 1913~2012，故甲下一步期望收到 2013，甲的 **ack = 2013**。

答案：**seq = 2046**，**ack = 2013**。

例题 **5**（累积确认，**2011** 年第 **40** 题原型）

甲向乙发送 3 个连续的 TCP 段，有效载荷分别为 300、400、500 字节，第 **3** 个段的序号为 **900**。若乙仅正确接收到第 1 和第 3 个段，乙发送给甲的确认序号是多少?

先倒推各段的起始序号：

段	载荷	起始序号	序号范围
第 1 段	300 B	$900-400-300=200$	200 ~ 499
第 2 段	400 B	$900-400=500$	500 ~ 899
第 3 段	500 B	900	900 ~ 1399

乙正确收到第 1 段（200~499）和第 3 段（900~1399），但缺少第 2 段（500~899）。TCP 使用累积确认，只能确认到「连续正确收到的最高序号」，因此乙回送 ack = 500。

答案：**500**。

例题 **6**（连续段与累积确认，**2009** 年第 **38** 题原型）

甲向乙发送两个连续的 TCP 段，有效载荷分别为 300 字节和 500 字节，第一个段的序号为 200。乙正确接收两个段后，发送给甲的确认序列号是多少?

- 第 1 段：seq=200，占 200~499；
- 第 2 段：seq=500，占 500~999；
- 乙已连续正确收到 200~999，故 ack = 1000。

答案：**1000**。

序号计算题的通用步骤

1. 先判断报文段类型：是 SYN、FIN，还是普通数据段?
2. 折算成序号区间：SYN 和 FIN 各占 1 个序号，普通数据段占「字节数」个序号。
3. 先确定起始点，再推终点：若题目给的是「第 k 段的序号」，要先把前面几段的长度加起来倒推出各段起点。
4. 确认号一律取「连续正确收到的最高序号 + **1**」（累积确认）。
5. 检查是否跨过 $2^{32}-1$ 回绕（大数值题要注意，普通题目不会涉及）。

3.TCP 可靠传输

TCP 的可靠传输建立在滑动窗口 + 确认重传之上。它的机制可以看成是后退 **N** 帧（**GBN**）与选择重传（**SR**）的结合：正常情况下用累积确认（像 GBN），配合 SACK 选项时可以只重传丢失的部分（像 SR）。

序号与确认机制

- TCP 是面向字节流的，所以序号以字节为单位连续编号，而不是以报文段为单位。
- 接收方收到按序到达的数据后，返回确认号 ack = 期望收到的下一个字节序号。
- 接收方可以捎带确认（piggybacking）：把确认信息搭载在自己要发送的数据报文段的 ack 字段中。若不发送数据，也可以立即单独发送确认。
- 累计确认（**cumulative acknowledgment**）：ack $= N$ 表示「序号到 $N-1$ 为止的所有数据都已按序正确收到」。即使收到了 N 之后的不连续数据，也不会把 ack 推到 N 之后。

累积确认的后果

正是因为累积确认，「收到 3 个段但缺中间那个」时确认号仍停在缺失段的起点。这也解释了快速重传为什么能工作：接收方每收到一个失序段，就重复发一次相同的 ACK，发送方收到 3 个重复 ACK 就知道对应报文段丢了。

超时重传

基本思想：每发送一个报文段就为该报文段设置一个超时计时器（重传计时器）。若在计时器到期之前收到了确认，就撤销该计时器；否则认为该报文段丢失，重传它。

注意几个细节：

- 发送方发出报文段后必须暂时保留其副本（以备重传），收到确认后才能清除；
- 报文段和确认都要编号，才能知道收到的是对哪一段的确认；
- 若收到两个重复的报文段，说明对方一定发生了超时重传，此时应丢弃较晚收到的那个重复报文段，并再回送一次确认。

超时重传时间 RTO 的选择

RTO 的选择是 TCP 中最复杂的问题之一。

加权平均往返时间 RTT_S：

$$\text{新的}RTT_S = (1-\alpha) \times \text{旧的}RTT_S + \alpha \times \text{新的 RTT 样本}, \qquad 0 \le \alpha < 1$$

第一次测量到 RTT 样本时，直接取 $RTT_{S1} = RTT_1$。RFC 6298 推荐的 $\alpha = 1/8 = 0.125$。α 越接近 0，新样本影响越小；越接近 1，新样本影响越大。

RTT 偏差的加权平均 RTT_D：

$$\text{新的}RTT_D = (1-\beta) \times \text{旧的}RTT_D + \beta \times |RTT_S - \text{新的 RTT 样本}|, \qquad 0 \le \beta < 1$$

第一次测量时 $RTT_{D1} = RTT_1 \div 2$。RFC 6298 推荐的 $\beta = 1/4 = 0.25$。

超时重传时间：

$$\boxed{RTO = RTT_S + 4 \times RTT_D}$$

计算顺序（RFC 6298 规定）

得到一个新的 RTT 样本时，先用「旧的 RTT_S」计算新的 RTT_D，再计算新的 RTT_S。顺序写反结果会不同。

算例：设第 1 次测得 RTT 样本 = 2 个单位。

- $RTT_{S1}=2$；
- $RTT_{D1}=2\div 2=1$；
- $RTO_1=RTT_{S1}+4\times RTT_{D1}=2+4=6$。

若第 2 次测得 RTT 样本 $=4$：

- 先用旧的 $RTT_{S1}=2$ 算 RTT_D：$RTT_{D2}=(1-\frac{1}{4})\times 1+\frac{1}{4}\times|2-4|=0.75+0.5=1.25$；
- 再算 $RTT_{S2}=(1-\frac{1}{8})\times 2+\frac{1}{8}\times 4=1.75+0.5=2.25$；
- $RTO_2=2.25+4\times 1.25=2.25+5=7.25$。

完整演算题（含一次超时样本，必须跳过）

设从第 1 次开始依次测得 RTT 样本为 $RTT_1=2$、$RTT_2=4$、$RTT_3=1.75$，第 4 次测量时发生了超时重传，第 5 次测得 $RTT_5=3$。取 $\alpha=1/8$、$\beta=1/4$。求每次测量后算出的 RTO。

测量次序	**RTT** 样本	是否采用	计算 RTT_D	计算 RTT_S	**RTO**
第 1 次	2	✓	$RTT_{D1}=2/2=1$	$RTT_{S1}=2$	$2+4\times 1=$ **6**
第 2 次	4	✓	$0.75\times 1+0.25\times\|2-4\|=1.25$	$0.875\times 2+0.125\times 4=2.25$	$2.25+4\times 1.25=$ **7.25**
第 3 次	1.75	✓	$0.75\times 1.25+0.25\times\|2.25-1.75\|=1.0625$	$0.875\times 2.25+0.125\times 1.75=2.1875$	$2.1875+4.25=$ **6.4375**
第 4 次	—	×（超时重传）	不计算	不计算	加倍为 $6.4375\times 2=$ **12.875**
第 5 次	3	✓	沿用旧的 $RTT_S=2.1875$：$0.75\times 1.0625+0.25\times\|2.1875-3\|=1.0$	$0.875\times 2.1875+0.125\times 3=2.2890625$	$2.2890625+4\times 1.0=$ **6.2890625**

这类题的三条执行纪律

1. 先判断样本是否可用：只要该次报文段发生了重传，这个 RTT 样本就作废，既不算 RTT_D 也不算 RTT_S（Karn 算法）。
2. 算 RTT_D 时用「旧的 RTT_S」，算完 RTT_D 再更新 RTT_S（RFC 6298 规定的顺序）。
3. 超时那一次不是「什么也不做」，而是把 RTO 加倍（指数退避），否则后续会反复超时。

「Karn 算法」在两种语境下的含义要分清（统考经常混着考）

- 严格意义的 **Karn** 算法（RFC 6298 之前的经典表述）：只要报文段被重传了，就不采用其 **RTT** 样本去更新 RTT_S。
- 加上指数退避的修正版（RFC 6298 实际采用）：发生超时重传时，先把 **RTO** 加倍（$RTO\leftarrow 2\times RTO$），后续若仍超时则继续加倍，直到收到确认。

因此做题时看到「发生超时重传」这句话，要同时做两件事：① 该次的 **RTT** 样本作废；② 当前的 **RTO** 值加倍。看到「根据 Karn 算法不考虑重传报文段的 RTT 样本」这句话，则只需做第 ① 件。

Karn 算法

RTT 测量本身有歧义：当报文段发生重传时，发送方收到一个确认，无法判断这个确认是对原报文段的还是对重传报文段的。

- 若误判为对原报文段的确认，算出的 RTT_S 和 RTO 会偏大，降低传输效率；
- 若误判为对重传报文段的确认，算出的 RTT_S 和 RTO 会偏小，导致后续不必要的大量重传，加重网络负担。

Karn 算法的基本思想：

凡是报文段被重传了，就不采用其 RTT 样本去计算RTT_S 和RTO

Karn 算法的缺陷与修正：若报文段的时延突然大幅增大并长期保持，按 Karn 算法不使用重传段的样本，RTO 就无法更新，会导致报文段被反复重传。

修正方法：报文段每发生一次重传，就把 **RTO** 加倍（即取新 RTO 为旧 RTO 的 2 倍，指数退避）。典型说法是「每重传一次，RTO 增大为原来的 2 倍」。

出题角度

Karn 算法在选择题中通常考「基本思想」：重传的报文段不用于计算 **RTT** 样本；以及「重传时 RTO 加倍」。这两个结论记住即可，不需要推导。

快速重传

前提：接收方要立即发送确认，不要等自己有数据要发时才捎带确认。

规则：发送方一旦收到 **3** 个连续的重复确认（冗余 **ACK**），就立即重传对应的报文段，不等重传计时器超时。

为什么是 3 个？收到 3 个重复确认意味着有 3 个报文段已经离开了网络并停留在接收方缓存中，说明只是个别报文段丢失，网络并没有严重拥塞，因此可以大胆重传。

算例：发送方发送 M_1, M_2, M_3, M_4, M_5，其中 M_2 丢失。

事件	发送方动作	接收方动作
发送 M_1~M_5	正常发送	—
M_1 到达	—	按序，回 `ack=M2` 起点
M_3 到达	—	失序，回 `ack=M2` 起点（第 **1** 个重复确认）
M_4 到达	—	失序，回 `ack=M2` 起点（第 **2** 个重复确认）
M_5 到达	—	失序，回 `ack=M2` 起点（第 **3** 个重复确认）
收到第 3 个重复确认	立即重传 M_2	—
M_2 到达	—	此时 M_1~M_5 全部按序到达，回 `ack=M6` 起点

重复确认的个数

「收到 3 个重复确认」= 加上原本的那一个确认，发送方共收到 **4** 个对同一报文段的确认。题目若说「一连收到 3 个对同一个报文段的重复确认」，指的是冗余的那 **3** 个。这个 3 的计数是高频陷阱。

快速重传的收益：对于个别丢失的报文段，发送方不会出现超时重传，也就不会误认为网络拥塞而把 cwnd 减到 1。实践证明，使用快速重传可以使整个网络的吞吐量提高约 20%。

选择确认 **SACK**

问题：TCP 接收方只能对按序收到的数据中的最高序号给出确认。当发送方超时重传时，接收方之前已经收到、只是未按序到达的数据也会被重传，浪费带宽。

解决：TCP 可以使用选择确认（**Selective Acknowledgment**，**SACK**），RFC 2018 建议标准。

接收方把「已收到但与前面字节流不连续的字节块」的边界信息通过 SACK 选项告诉发送方，发送方就只重传真正缺失的部分，不再重传已经正确到达的数据。

SACK 选项的格式约束：

- 「允许 SACK」选项占 1 字节，SACK 选项长度占 1 字节；
- 指明一个字节块的边界需要 4 字节（因为序号是 4 字节），一个字节块需要两个边界，即 8 字节；
- 因此一个 SACK 选项最多能指明 **4** 个字节块。

SACK 的两个限定

1. **SACK** 只是「通知」，不是「协议规定发送方必须怎么做」。SACK 相关文档并没有规定发送方应如何响应，因此大多数 **TCP** 实现仍然重传所有未被确认的数据块。
2. SACK 仍然保留了原有的确认号字段，即累积确认仍然有效，SACK 只是补充信息。

发送窗口与可连续发送的字节范围

TCP 发送方在没有收到确认的情况下，可以把序号落在发送窗口内的数据依次全部发送出去。

- 发送窗口的左沿：收到新的确认后才向右移动；没收到确认则不动。
- 发送窗口的右沿：通常因为收到新的确认而不断右移；也可能因为对方接收窗口变小而不动。
- **TCP** 标准不赞成把窗口右沿向左收缩（即不允许把已经承诺可以发送的序号重新收回），因为窗口里的数据可能已经发出去了，收缩会引发错误。

发送窗口内的序号集是发送缓存序号集的子集；接收窗口的序号集是接收缓存序号集的子集。

重传次数的计算

规则：TCP 每重传一次，RTO 加倍（指数退避）。所以从第 1 次超时开始，第 n 次重传后的累计等待时间为

$$T_n = RTO \times (1 + 2 + 4 + \cdots + 2^{n-1}) = RTO \times (2^n - 1)$$

算例：设初始 RTO = 1 s，问「从第一次超时算起，经过多少时间发送方将进行第 4 次重传」。

次序	等待时间	累计等待时间
第 1 次重传	1 s	1 s
第 2 次重传	2 s	3 s
第 3 次重传	4 s	7 s
第 4 次重传	8 s	**15 s**

即 $1 \times (2^4 - 1) = 15$ s。

重传次数题的两个坑

1. **RTO** 是每次加倍，不是每次加固定值。若题目只说「RTO=1s，超时后进行第 3 次重传」，答案是 $1+2+4=7$ s。
2. 要分清「报文段数」和「字节数」。例如「发送窗口 5000 B，MSS = 1000 B，其中第 2 段丢失」，则窗口内最多能装 5 个报文段，快速重传后发送方需重传 1 个（在无 SACK 且假定后续段都已到达的情况下），总发送报文段数 = 原发的 5 个 + 重传的 1 个。计算时要先列出每段占用的序号区间，再逐段判断是否重传。

4.TCP 流量控制

流量控制的含义

流量控制（**flow control**）的目的：让发送方发送数据的速率不要太快，使接收方来得及接收。

- 它是点对点、端到端的问题（接收端控制发送端），与拥塞控制（全局性问题，涉及所有主机和路由器）不同；
- TCP 不是靠「速率」，而是靠滑动窗口来实现：接收方在返回的确认报文段中，用窗口字段告诉发送方「我还能收多少字节」。

流量控制与拥塞控制的区别（必背）

- 流量控制：抑制发送端速率，使接收端来得及接收。点对点、端到端问题，由接收窗口 rwnd 实现。
- 拥塞控制：防止过多数据注入网络，使网络中的路由器和链路不过载。全局性问题，涉及所有主机、所有路由器，由拥塞窗口 cwnd 实现。

发送窗口的确定

$$\boxed{\text{发送窗口} swnd = \min(rwnd,\ cwnd)}$$

- $rwnd$（receiver window）：接收方通过 TCP 首部窗口字段通告的接收窗口大小，取决于接收缓存的可用空间；
- $cwnd$（congestion window）：发送方维护的拥塞窗口，取决于网络的拥塞程度。

若题目不考虑拥塞控制（如 2010 年第 39 题「题目未给出拥塞窗口」），则取 $swnd = rwnd$。

发送缓存与接收缓存

发送缓存里存放：

1. 发送方应用程序交给 TCP 准备发送的数据；
2. TCP 已发送出去但尚未收到确认的数据。

接收缓存里存放：

1. 按序到达但尚未被应用进程取走的数据；
2. 未按序到达的数据。

注意：发送窗口 $\subseteq$ 发送缓存，接收窗口 $\subseteq$ 接收缓存。当接收方通告 `rwnd = 0` 时，发送方必须停止发送普通数据。

窗口滑动与序号计算的完整算例

设 A、B 已建立 TCP 连接。B 通告接收窗口 `rwnd = 400` 字节，A 的发送窗口也设为 400 字节。A 已有从序号 1 开始的大批数据要发送。

初始状态：

项目	值
发送窗口	seq 1 ~ 400
已发送未确认	（空）
可发送未发送	1 ~ 400

步骤 **1**：A 连续发送 4 个 100 字节的报文段（把窗口用完）。

报文段	序号	状态
①	1 ~ 100	已发送，未确认
②	101 ~ 200	已发送，未确认
③	201 ~ 300	在传输中丢失
④	301 ~ 400	已发送，失序到达

此时发送窗口仍为 1~400，但窗口内已无可发送的字节。

步骤 **2**：B 收到 ①②，接着收到失序的 ④（301~400）。B 先按序确认 `ack = 201`；再对失序的 ④ 立即回送一个重复确认，仍然是 `ack = 201`，并通告 `rwnd = 300`。

- `ack = 201` 表示 A 可以把发送窗口左沿推进到 201；
- `rwnd = 300` 表示 B 还能收 300 字节，所以窗口右沿为 $201 + 300 - 1 = 500$；
- 新发送窗口为 **seq 201 ~ 500**。

项目	值
发送窗口	201 ~ 500
已发送未确认	201 ~ 400（③ 丢失，④ 失序到达）
可发送未发送	401 ~ 500（共 100 字节）

步骤 **3**：A 在新窗口内发送序号 401~500 的报文段（100 字节）。

步骤 **4**：B 收到 401~500。因为 201~300 这一段仍然缺失，401~500 仍是失序到达，B 立即再回送一次重复确认 `ack = 201`。

此时 A 累计收到了对 201 的确认：

收到的确认	性质	累计重复确认数
`ack = 201`	①② 按序到达后的正常确认	0
`ack = 201`	对失序段 ④（301~400）的重复确认	**1**
`ack = 201`	对失序段（401~500）的重复确认	**2**

重复确认只有 **2** 个，尚未达到快速重传的触发条件（需要 **3** 个），所以 A 只能等超时。

步骤 **5**：A 的 ③ 号报文段重传定时器超时，重传 201~300。B 收到后凑齐了 201~500，返回 `ack = 501`。

- A 收到 `ack = 501` 后，窗口左沿推进到 501；
- 若此时 B 通告 `rwnd = 0`，则窗口右沿变为 500，即窗口收缩为零。

步骤 **6**（零窗口）：A 不能再发送普通数据，但必须启动持续计时器。计时器超时后，A 发送一个零窗口探测报文段（仅携带 1 字节数据），B 收到后回送确认并通告现在的接收窗口值：

- 若仍为 0，A 重新启动持续计时器，重复探测；
- 若变为 300，则 A 的发送窗口变为 501 ~ 800，可以继续发送，死锁局面被打破。

变体：若 A 把发送窗口做得更大（例如序号 1~800），则在 ③ 丢失后，失序到达的每一段都会让 B 再回送一个重复确认，凑满 **3** 个重复确认就会触发快速重传，A 立刻重传 201~300，不必等超时。这也说明：重复确认是靠失序报文段一个攒出来的，窗口越大越容易触发快速重传。

窗口计算的三步法

1. 窗口左沿 = 收到的最新确认号 ack；
2. 窗口大小 = 通告的 rwnd（若不考虑拥塞控制）；若考虑拥塞控制则取 $\min(rwnd, cwnd)$；
3. 窗口右沿 = 左沿 + 窗口大小 − 1；
4. 还能发送的字节数 = 窗口右沿 − 已发送未确认数据的最大序号。

要点：已发送但未收到确认的数据仍然占用发送窗口。

这个算例告诉你两件事

1. 窗口是随每个通告值动态调整的：同一个发送方，窗口先是 1~400，然后是 201~500，最后是 501~800。每次调整都要用「新 ack 作左沿、新 rwnd 定大小」重新算，不能沿用旧窗口。
2. 「已发送未确认」的数据始终占着窗口：步骤 2 中虽然窗口左沿推到了 201、窗口大小变成 300，但 201~400 这一段仍然被占用，所以新窗口里能发的只有 401~500 这 100 字节。这一步算错，整题的答案就全错。

零窗口与持续计时器

死锁问题：接收方 B 的接收缓存满了，于是通告 `rwnd=0`。A 收到后停止发送，等待 B 的非零窗口通知。但如果：

- B 的缓存腾出空间后发出的非零窗口通知丢失；
- 则 A 一直等 B 的通知，B 一直等 A 发数据——双方永远互相等待，形成死锁，吞吐量降到 0。

解决办法：持续计时器（**persistence timer**）。

- 只要 TCP 连接的一方收到对方的零窗口通知，就启动持续计时器；
- 持续计时器超时后，发送一个零窗口探测报文段（只携带 1 字节数据）；
- 对方在确认这个探测报文段时，给出自己当前的接收窗口值；
- 若接收窗口仍为 0，收到确认的一方重新启动持续计时器；
- 若接收窗口不为 0，死锁局面就被打破。

两个容易困惑的问题：

1. 接收窗口为 **0** 时，**B** 还能接收报文段吗？能。TCP 规定：即使接收窗口值为 **0**，也必须接受零窗口探测报文段、确认报文段以及携带有紧急数据的报文段。

2. 零窗口探测报文段丢了怎么办？它也有重传计时器，超时后会被重传，所以死锁仍能被打破。

出题角度

「零窗口探测报文段携带多少数据」——**1** 字节；「由谁发起」——收到零窗口通知的一方；「持续计时器的作用」——打破零窗口造成的互相等待死锁。

流量控制与拥塞控制的结合算例

2010 年第 **39** 题原型：甲、乙已建立 TCP 连接，MSS = 1000 B。甲当前拥塞窗口为 4000 B，甲向乙连续发送两个最大段后，收到乙发送的第一个段的确认段，确认段中通告的接收窗口大小为 2000 B。问此时甲还可以向乙发送的最大字节数是多少?

解：

- 发送窗口 $= \min(cwnd,\ rwnd) = \min(4000,\ 2000) = 2000$ B；
- 甲已发送 2 个段（共 2000 B），其中只有第 1 个段被确认，第 **2** 个段（**1000 B**）仍未被确认，仍占用发送窗口；
- 因此还能发送的最大字节数 $= 2000 - 1000 =$ **1000** B。

答案：**1000 B**。

2016 年第 **41** 题原型（窗口与拥塞控制结合）：H3 与 S 建立 TCP 连接，S 分配 20 KB 接收缓存，MSS = 1 KB，RTT = 200 ms，H3 初始序号 100，拥塞窗口初始阈值 32 KB，S 端接收缓存只存不取。问：

(1) 第二次握手段的 **SYN**、**ACK** 值与确认序号?

`SYN = 1, ACK = 1, ack = 100 + 1 = 101`。

(2) H3 收到的第 **8** 个确认段通告的接收窗口是多少？此时拥塞窗口和发送窗口各是多少?

项目	计算
S 已收到数据	8×1 KB $= 8$ KB
剩余接收缓存（rwnd）	$20 - 8 =$ **12** KB
cwnd（慢开始，每确认 1 段 +1 MSS）	$1 + 8 =$ **9** KB
发送窗口	$\min(12,\ 9) =$ **9** KB

(3) 发送窗口等于 **0** 时，下一个待发送的数据段序号是多少?

H3 的第一个数据字节序号为 $100 + 1 = 101$。接收缓存装满 20 KB，即 $20 \times 1024 = 20480$ 字节。数据占用序号 $101 \sim 101 + 20480 - 1 = 20580$。因此下一个待发送数据段的序号为 **20581**。

(4) 从 **t** 时刻 **H3** 请求断开连接起，**S** 释放该连接的最短时间是多少?

S 是被动关闭方，最短为 **1.5 RTT** $= 1.5 \times 200 =$ **300** ms。

这类题的通用陷阱

1. 不要忘记「已发送未确认」的数据仍占窗口。这是 2010 年题的关键。
2. 接收缓存「只存不取」时，**rwnd** 单调减少，要用「总缓存 − 已收数据」计算，不能直接用初始值。
3. **cwnd** 每次增加的是「**1 MSS**」而不是「**1 KB**」，在 MSS 不等于 1 KB 时要换算。
4. 时间问的是「谁」释放连接：主动方看 1RTT + 2MSL，被动方看 1.5RTT。

5.TCP 拥塞控制

拥塞控制的基本概念

拥塞（**congestion**）：网络中注入的分组过多，超出了网络资源（链路带宽、路由器处理能力、缓存）所能处理的能力，导致路由器队列堆积、分组丢失、吞吐量反而下降。

拥塞控制：防止过多的数据注入网络，使网络中的路由器和链路不致过载。这是一个全局性的过程，涉及所有主机、所有路由器和与降低网络传输性能有关的所有因素。

讨论前提（这三条是教材约定的简化条件，题目通常默认成立）：

1. 数据单方向传送，另一个方向只传送确认；
2. 接收方总是有足够大的接收缓存，因此发送窗口仅由网络拥塞程度决定，不考虑流量控制；
3. 以 **MSS** 的个数作为讨论单位（而不是字节）。

拥塞窗口 **cwnd**：由发送方维护，其值取决于网络的拥塞程度，动态变化。四种拥塞控制算法本质上都是在控制 cwnd 的增长与下降。

判断网络出现拥塞的依据：没有按时收到应当到达的 TCP 确认报文段，而产生了超时重传。

$$swnd = \min(cwnd,\ rwnd)$$

- 不考虑拥塞控制时：$swnd = rwnd$；
- 不考虑流量控制时：$swnd = cwnd$。

慢开始门限 **ssthresh** 的使用规则：

- 当 $cwnd < ssthresh$ 时，使用慢开始算法；
- 当 $cwnd > ssthresh$ 时，停止慢开始，改用拥塞避免算法；
- 当 $cwnd = ssthresh$ 时，既可使用慢开始，也可使用拥塞避免（统考通常按「可以使用慢开始」处理，即在 $cwnd = ssthresh$ 时仍翻倍到 $2 \times ssthresh$，或按题目约定）。

$cwnd = ssthresh$ 时的分歧

不同教材/实现对这个边界处理不同：

- **RFC 5681** 推荐：$cwnd < ssthresh$ 用慢开始，$cwnd \geq ssthresh$ 用拥塞避免（即等于门限时用拥塞避免，每 RTT 只 +1 MSS）；
- 有的教材写「$cwnd = ssthresh$ 时两种算法都可以」。

统考真题通常取「翻倍到 **2×ssthresh** 之后再转拥塞避免」这一种（例如 2009 年题 39，$ssthresh = 8$，$cwnd$ 经 1→2→4→8→**9**，即第 4 个 RTT 只 +1）。做题时优先看题目是否给了阶段提示；若没有，按以下稳妥流程：只要 $cwnd$ 已经达到 $ssthresh$，下一轮就 **+1**。本章所有演算均按此约定。

慢开始与拥塞避免

慢开始（**slow start**）算法：

- 初始时 $cwnd = 1$ MSS；
- 每收到一个对新报文段的确认，就把 **cwnd** 增加 **1 MSS**；
- 于是每经过一个传输轮次（即 1 个 RTT），cwnd 就翻倍（指数增长）。

「慢开始」的「慢」并不是指 cwnd 增长速率慢，而是指刚开始时 **cwnd** 设为 **1**，比一上来就向网络注入大量报文段「慢」得多。

拥塞避免（**congestion avoidance**）算法：

- 每收到一个对新报文段的确认，就把 **cwnd** 增加 **1/cwnd** 个 **MSS**；
- 于是每经过一个传输轮次（1 个 RTT），cwnd 增加 **1** 个 **MSS**（线性增长，称为「加法增大」）。

两种阶段的对比（必背）

阶段	每收到 **1** 个新确认，**cwnd** 增加	每经过 **1** 个 **RTT**，**cwnd** 增加
慢开始	1 MSS	翻倍（$\times 2$）
拥塞避免	$1/cwnd$ 个 MSS	$+1$ MSS

快重传与快恢复

快重传（**fast retransmit**）：

- 前提：接收方不要等自己发送数据时才捎带确认，而要立即发送确认，即使收到失序报文段也要立即发出对已收到报文段的重复确认；
- 规则：发送方一旦收到 **3** 个连续的重复确认，就立即重传对应的报文段，不等重传计时器超时。

快恢复（**fast recovery**）：

- 发送方一旦收到 3 个重复确认，就知道只是丢失了个别报文段，而不是网络拥塞，于是不启动慢开始，而是执行快恢复；
- 规则：把 $ssthresh$ 和 $cwnd$ 都调整为当前 **cwnd** 值的一半，然后直接开始执行拥塞避免算法。

另一种实现：把 $cwnd$ 再增大一些，即 $cwnd =$ 新$ssthresh + 3$。理由：既然收到了 3 个重复确认，就说明有 3 个数据报文段已经离开了网络、停留在接收方缓存中，网络中少了 3 个报文段，可以适当把 cwnd 增大一些。（本章演算统一采用第一种：$cwnd = ssthresh =$ 原$cwnd/2$。）

两种拥塞事件的不同处理（核心对比表，必背）

情况	触发条件	**ssthresh** 更新	**cwnd** 更新	后续阶段
超时重传	重传计时器超时，未收到确认	$ssthresh =$ 当前$cwnd \div 2$	$cwnd = 1$ MSS	先慢开始，到达门限后转拥塞避免
三个重复确认（快重传）	收到 3 个重复 ACK	$ssthresh =$ 当前$cwnd \div 2$	$cwnd =$ 新$ssthresh$	直接进入拥塞避免，不经过慢开始

超时与三重确认的最大区别

- 超时：$cwnd$ 直接降到 **1**（回到慢开始，重新指数增长）；
- 三重确认：$cwnd$ 只降到一半（直接进拥塞避免，线性增长）。

两者的 $ssthresh$ 更新规则相同，都是「当前 cwnd 的一半」。判断分支用哪一个，就看去重传的原因。

cwnd 逐轮次演算表（含两种情况）

场景设定（为了表格干净，约定初始 $cwnd = 1$ MSS，初始 $ssthresh = 16$ MSS，MSS = 1 KB）：

- RTT1~RTT6：正常，无丢失；
- **RTT7** 结束时刻发生超时重传（对应 cwnd 为 32 时超时，$ssthresh$ 更新为 16）；
- RTT8~RTT15：恢复正常；
- **RTT16** 结束时刻收到 **3** 个重复确认（快重传 + 快恢复）。

逐轮次演算表：

轮次	轮次开始时的 **cwnd（MSS）**	阶段	本事件	**ssthresh（MSS）**	轮次结束时的 **cwnd（MSS）**
RTT1	1	慢开始	—	16	2
RTT2	2	慢开始	—	16	4
RTT3	4	慢开始	—	16	8
RTT4	8	慢开始	—	16	16
RTT5	16	拥塞避免	—	16	17
RTT6	17	拥塞避免	—	16	18
RTT7	18	拥塞避免	（按设定此处超时，cwnd = 18）	$18/2 = 9$	**1**
RTT8	1	慢开始	超时后重启	9	2
RTT9	2	慢开始	—	9	4
RTT10	4	慢开始	—	9	8
RTT11	8	慢开始	—	9	**16**
RTT12	16	拥塞避免	—	9	17
RTT13	17	拥塞避免	—	9	18
RTT14	18	拥塞避免	—	9	19
RTT15	19	拥塞避免	—	9	20
RTT16	20	拥塞避免	收到 **3** 个重复确认	$20/2 = 10$	**10**
RTT17	10	拥塞避免	快恢复后直接进拥塞避免	10	11
RTT18	11	拥塞避免	—	10	12
RTT19	12	拥塞避免	—	10	13
RTT20	13	拥塞避免	—	10	14

表格的几个关键点：

1. **RTT4** 结束时 **cwnd = 16 = ssthresh**，进入「等于门限」的边界；
2. **RTT5** 开始时 **cwnd = 16**，按「已达门限就 **+1**」的约定只增加到 **17**（若按翻倍约定则为 32）——做题时务必先看题目的阶段提示；
3. **RTT7** 超时：$ssthresh = 18/2 = 9$，$cwnd$ 立即降为 1，是瞬时完成的状态变更，不占一个 RTT；

4. **RTT8~RTT10** 慢开始使 cwnd 依次为 2、4、8；RTT11 开始时 cwnd = 8 < ssthresh = 9，仍属慢开始，结束时翻倍到 16；
5. RTT12 开始 cwnd = 16 > ssthresh = 9，转入拥塞避免，每轮 +1；
6. **RTT16** 收到 **3** 个重复确认：$ssthresh = 20/2 = 10$，$cwnd = 10$，直接进拥塞避免（RTT17 只 +1 到 11，而不是降到 1）。

从表里读出三条规律

1. 慢开始阶段：每一轮的 cwnd 是上一轮的 2 倍；
2. 拥塞避免阶段：每一轮的 cwnd 是上一轮 +1；
3. 每遇到一次拥塞事件（超时或三重确认），**ssthresh** 就变成当时 **cwnd** 的一半，然后：
   - 超时 → cwnd 从 1 重新开始；
   - 三重确认 → cwnd 从新 ssthresh 开始。

cwnd 随传输轮次变化的曲线描述

把上面的表格画成「横轴为传输轮次、纵轴为 cwnd」的曲线，形状可以这样描述：

1. 第 **1** 段（**RTT1~RTT4**）：从 $(1, 1)$ 出发，陡峭地指数上升，依次经过 $(2, 2)$、$(3, 4)$、$(4, 8)$、$(5, 16)$。曲线呈上凸的指数形状；这一段叫慢开始。
2. 第 **2** 段（**RTT5~RTT7**）：从 cwnd = 16 开始，变成平缓的线性上升，依次 $(5, 16)$、$(6, 17)$、$(7, 18)$。斜率很小（每轮 +1）；这一段叫拥塞避免。
3. 超时点（**RTT7** 结束）：曲线垂直断崖式下跌到 cwnd = 1。注意 $ssthresh$ 同时被更新为原 cwnd 的一半（9），画图时要把门限线也画下来。
4. 第 **3** 段（**RTT8~RTT11**）：从 1 又开始指数上升（1→2→4→8→16）。注意因为 $ssthresh$ 已经变小（9），这段指数上升比第 1 段更早被门限截住。
5. 第 **4** 段（**RTT12~RTT16**）：越过门限后转为线性上升（16→17→18→19→20）。
6. 三重确认点（**RTT16** 结束）：曲线只下降到新门限的一半位置（从 20 降到 10），而不是掉到 1。这是与超时最直观的图形差别。
7. 第 **5** 段（**RTT17** 起）：从 10 开始线性上升（10→11→12→13→14），没有指数段。

曲线三个视觉特征

- 慢开始段是上凸的指数曲线（翻倍）；
- 拥塞避免段是斜率极小的直线（+1）；
- 超时是「掉到 **1**」的深谷，三重确认是「掉到一半」的浅谷。

经典计算题完整解析

题目一（**2009** 年第 **39** 题原型）：一个 TCP 连接总是以 1 KB 的最大段长发送 TCP 段，发送方有足够多的数据要发送。当拥塞窗口为 16 KB 时发生了超时，如果接下来的 4 个 RTT 时间内 TCP 段的传输都是成功的，那么当第 4 个 RTT 时间内发送的所有 TCP 段都得到肯定应答时，拥塞窗口大小是多少?

解：超时处理：$ssthresh = 16/2 = 8$ KB，$cwnd = 1$ KB。

轮次	轮次开始时 **cwnd**	阶段	轮次结束时 **cwnd**
第 1 个 RTT	1 KB	慢开始	2 KB

轮次	轮次开始时 **cwnd**	阶段	轮次结束时 **cwnd**
第 2 个 RTT	2 KB	慢开始	4 KB
第 3 个 RTT	4 KB	慢开始	8 KB
第 4 个 RTT	8 KB	因 $cwnd = 8 = ssthresh$，转拥塞避免	**9 KB**

答案：**9 KB**。

题目二（**2014** 年第 **38** 题原型）：甲、乙已建立 TCP 连接，$MSS = 1$ KB，甲一直有数据发送；乙每收到一个数据段都会发出一个接收窗口为 10 KB 的确认段。若甲在 t 时刻发生超时时拥塞窗口为 8 KB，求从 t 时刻起不再发生超时的情况下，经过 10 个 RTT 后甲的发送窗口。

解：超时处理：$ssthresh = 8/2 = 4$ KB，$cwnd = 1$ KB。

轮次	轮次开始 **cwnd**	阶段	轮次结束 **cwnd**
1	1 KB	慢开始	2 KB
2	2 KB	慢开始	4 KB
3	4 KB	已达门限，转拥塞避免	5 KB
4	5 KB	拥塞避免	6 KB
…	…	每轮 +1	…
10	11 KB	拥塞避免	12 KB

10 个 RTT 后 $cwnd = 12$ KB。但接收窗口始终为 10 KB，所以

$$swnd = \min(12,\ 10) = \textbf{10 KB}$$

答案：**10 KB**。

题目三（**2022** 年第 **38** 题原型）：$MSS = 1$ KB，甲一直有数据向乙发送，当甲的拥塞窗口为 16 KB 时计时器超时，则甲的拥塞窗口再次增长到 16 KB 所需要的最少时间是多少（以 RTT 计）?

解：$ssthresh = 16/2 = 8$ KB，$cwnd = 1$ KB。

轮次	轮次开始时 **cwnd**	阶段	轮次结束后 **cwnd**
1	1 KB	慢开始	2 KB
2	2 KB	慢开始	4 KB
3	4 KB	慢开始	8 KB
4	8 KB	转拥塞避免	9 KB
5	9 KB	拥塞避免	10 KB
6	10 KB	拥塞避免	11 KB
7	11 KB	拥塞避免	12 KB
8	12 KB	拥塞避免	13 KB
9	13 KB	拥塞避免	14 KB

轮次	轮次开始时 **cwnd**	阶段	轮次结束后 **cwnd**
10	14 KB	拥塞避免	15 KB
11	15 KB	拥塞避免	**16 KB**

从 1 KB 涨回 16 KB：慢开始走 3 轮到达 8 KB，之后拥塞避免每轮 +1 KB 需要 8 轮，共 $3+8=$ **11** RTT。

答案：**11 RTT**。

题目四（**2017** 年第 **39** 题原型）：甲向乙发起 TCP 连接，$MSS=1$ KB，RTT = 5 ms，乙开辟的接收缓存为 64 KB。求甲从连接建立成功到发送窗口达到 32 KB 所需的最少时间。

解：接收窗口 64 KB 不构成限制（32 < 64）。慢开始阶段每 RTT 翻倍：

$$1 \to 2 \to 4 \to 8 \to 16 \to 32\ (\text{KB})$$

从 1 KB 到 32 KB 需要 **5** 个 **RTT**，最少时间 $=5\times5=$ **25** ms。

题目五（**2020** 年第 **38** 题原型）：$MSS=1$ KB，RTT = 2 ms，在不出现拥塞的前提下，拥塞窗口从 8 KB 增长到 32 KB 所需的最长时间是多少?

解：求最长时间应按增长较慢的拥塞避免（每 RTT +1 KB）计算。

$$32-8=24\ \text{KB} \Rightarrow 24\ \text{个 RTT},\qquad 24\times2=\mathbf{48}\ \text{ms}$$

答案：**48 ms**。

题目六（**2015** 年第 **39** 题原型）：甲、乙新建 TCP 连接，甲的拥塞控制初始阈值为 32 KB，甲以 $MSS=1$ KB 的段发送数据，乙为该连接分配 16 KB 接收缓存，对每个数据段确认，忽略段传输延迟。若乙收到的数据全部存入缓存不被取走，则甲从连接建立成功起，在未超时的情况下经过 4 个 RTT 后，甲的发送窗口是多少?

解：初始 $cwnd=1$ KB，$ssthresh=32$ KB，全程处于慢开始。

轮次	发送的段数	累计写入接收缓存
第 1 个 RTT	1	1 KB
第 2 个 RTT	2	3 KB
第 3 个 RTT	4	7 KB
第 4 个 RTT	8	**15 KB**

经过 4 个 RTT 后共写入 15 KB，接收缓存还剩 $16-15=1$ KB，即 $rwnd=1$ KB。此时 $cwnd=16$ KB。

$$swnd=\min(16,\ 1)=\mathbf{1\ KB}$$

答案：**1 KB**。

题目七（**2023** 年第 **47** 题原型）：主机 H 从 FTP 服务器下载一个 18000 B 的文件 F，建立数据连接时初始序号为 100，$MSS=1000$ B，拥塞控制初始阈值为 4 MSS，RTT = 10 ms，忽略 TCP 传输时延，全程无差错、无丢包乱序。

(1) 文件第一个字节的序号是多少?

初始序号 100 被 SYN 占用，所以第一个数据字节序号为 $100+1=$ **101**。

(2) 断开数据连接时，**FTP** 发送的第二次挥手的 **ACK** 序号是多少?

18000 B 数据占用序号 101 ~ 18100，随后发送的 FIN 序号为 18101。服务器在第二次挥手中确认该 FIN，故 `ack = 18101 + 1 = 18102`。

(3) 当 **H** 收到确认序号为 **2101** 的确认段时，**cwnd** 是多少? 收到 **7101** 时呢?

初始 $cwnd=1$ MSS，$ssthresh=4$ MSS。慢开始阶段每收到一个新确认 +1 MSS:

确认号	累计确认的数据段数	**cwnd**
1101	1	2 MSS
2101	2	**3 MSS**
3101	3	4 MSS（$=ssthresh$，之后转拥塞避免）
4101	4	5 MSS（拥塞避免，+1）
…	…	每 1000 B 数据 +1 MSS
7101	7	**8 MSS**

所以收到 `ack=2101` 时 $cwnd=$ **3 MSS**；收到 `ack=7101` 时 $cwnd=$ **8 MSS**（因为 7 个段被确认，其中前 3 个走慢开始到 4 MSS，之后 4 个走拥塞避免，$4+4=8$）。

这一段要特别小心

从「cwnd = 4 MSS 时转拥塞避免」之后，每收到 **1** 个新确认只 **+1 MSS**（在 MSS = 1 段的讨论单位下，拥塞避免每收到一个确认增加 $1/cwnd$ 个 MSS，但按「每 RTT +1」的离散轮次来处理更稳妥）。做题时一定要先判断当前收到第几个新确认，再决定是「翻倍」还是「+1」。

拥塞控制与流量控制的区别（第四遍强调）

对比项	流量控制	拥塞控制
目的	使接收方来得及接收	使网络不过载
范围	点对点、端到端（接收端 → 发送端）	全局性（涉及所有主机、所有路由器）
实现手段	接收窗口 rwnd	拥塞窗口 cwnd
是否 TCP 独有	UDP 完全没有	UDP 完全没有
组合关系	$swnd=\min(rwnd,\ cwnd)$	同左

传输层滑动窗口与数据链路层滑动窗口的区别

这一对比虽然不在考纲里，但选择题常用来设混淆项，务必分清。

对比项	数据链路层（**GBN / SR**）	传输层（**TCP**）
窗口单位	帧（或比特）	字节（MSS 只是讨论单位）
窗口大小含义	最多连续发送/接收多少个帧	允许发送的字节序号区间
滑动方式	按帧序号滑动	按字节序号滑动

对比项	数据链路层（**GBN / SR**）	传输层（**TCP**）
确认语义	GBN 中 ACK n = 到 n 为止已收到，期望 $n+1$；SR 中逐帧确认	ack $= N$ = 到 $N-1$ 为止已收到，期望 N
是否考虑字节大小	只关心帧数，与帧长无关	必须关心字节数，窗口值 × MSS 才有物理意义
是否有拥塞控制	无（点对点链路不存在网络范围拥塞问题）	有（cwnd）

最容易踩的一条

题目里常说「窗口是 400」，一定要先看单位：TCP 里 400 表示 **400** 字节；若题目同时给出 MSS = 100 B，则窗口内可以装 4 个报文段；若题目说的是「窗口为 400 个报文段」，那就是 400 × MSS 字节。单位搞错，整道题全错。

本章四类计算题的解题路线图

题型	识别特征	解题路线
序号/确认号推算	「第三个报文段的序号」「乙收到的确认号」	① 判断报文段类型（SYN / FIN / 数据）→ ② 折算成序号区间 → ③ 倒推各段起点 → ④ 确认号取「连续收到的最高序号 + 1」
窗口推算	「甲的发送窗口还能发送多少字节」「可连续发送的字节序号范围」	① 求窗口大小 = min$(rwnd, cwnd)$ → ② 左沿 = 最新 ack → ③ 右沿 = 左沿 + 窗口大小 − 1 → ④ 还要发多少 = 右沿 − 已发送未确认的最大序号
拥塞窗口逐轮演算	「经过 n 个 RTT 后 cwnd / 发送窗口是多少」	① 先处理拥塞事件：超时 → $ssthresh = cwnd/2$、$cwnd = 1$；三重确认 → $ssthresh = cwnd/2$、$cwnd = ssthresh$ → ② 逐轮判断当前处于慢开始（×2）还是拥塞避免（+1）→ ③ 最后用 min$(rwnd, cwnd)$ 求发送窗口
时间估算	「至少需要多长时间」「从发出 FIN 起多久关闭」	① 建立连接 1 RTT → ② 数据传送按 cwnd 演算出的轮次数 × RTT → ③ 释放：主动方 1RTT + 2MSL，被动方 1.5RTT → ④ 全部相加

这一章在统考里的出题规律

- 选择题（每年 **1~2** 道）：UDP 校验、端口号、标志位含义、序号/确认号、每次拥塞事件后的 cwnd、TIME-WAIT 与 2MSL、RTO 与 Karn 算法。
- 综合应用题（约每 **2~3** 年一次，**9** 分左右）：把 FTP/HTTP 之类的应用层背景 + TCP 连接建立与释放的序号 + 拥塞窗口逐轮 + 发送时间估算打包在一起，有时还掺入 IP、以太网最小帧长的计算。做这类题的关键不是记公式，而是能把序号、窗口、时间三条线分开算，各算各的再汇总。

本章小结表

考点	核心结论
传输层功能	进程到进程（端到端）的逻辑通信；复用（多个进程共用一个协议）与分用（按端口交付进程）

考点	核心结论
与网络层区别	网络层：主机到主机，按协议字段分用；传输层：进程到进程，按端口号分用
端口号	**16** 位，0 ~ 65535；只有本地意义；TCP 与 UDP 端口号相互独立
三类端口	熟知 **0 ~ 1023**；登记 **1024 ~ 49151**；短暂（客户端）**49152 ~ 65535**
套接字	socket = (IP 地址: 端口号)；TCP 连接由两个套接字唯一确定
必须记住的端口	FTP 21/20、SSH 22、Telnet 23、SMTP 25、DNS 53、DHCP 67/68、TFTP 69、HTTP 80、POP3 110、IMAP 143、HTTPS 443、SNMP 161、RIP 520
UDP 特点	无连接、不可靠、面向报文、支持一对一/一对多/多对多、首部仅 **8 B**、无流量与拥塞控制
UDP 首部	四个字段各 2 B：源端口、目的端口、长度（首部 + 数据）、校验和；长度最小为 8
UDP 校验	覆盖 **12 B** 伪首部 + **8 B** 首部 + 数据（不足补全 0）；反码求和 + 回卷进位 + 取反；全 0 表示不校验
伪首部	源 IP、目的 IP、全 0、协议号（**UDP 17 / TCP 6**）、长度；不传送、不递交，只用于校验
TCP 特点	面向连接、可靠、全双工、面向字节流、只能一对一
TCP 首部	固定 **20 B** + 选项最多 40 B = 最长 **60 B**；数据偏移以 **4 B** 为单位，最小 5
序号含义	本报文段数据第一个字节的序号
确认号含义	期望收到的下一个字节序号；表示到 **ack−1** 为止的数据已正确收到
序号消耗规则	SYN 消耗 1；FIN 消耗 1；纯 ACK 不消耗；数据段消耗字节数
六个标志位	URG 紧急、ACK 确认、PSH 推送、RST 复位、SYN 同步、FIN 终止
MSS 与 MTU	MSS = MTU − IP 首部 − TCP 首部；MSS 只算数据载荷
三次握手	`SYN, seq=x` → `SYN+ACK, seq=y, ack=x+1` → `ACK, seq=x+1, ack=y+1`
为什么三次	防止已失效的连接请求报文段到达服务器，造成服务器资源浪费
四次挥手	`FIN, seq=u` → `ACK, ack=u+1` → `FIN+ACK, seq=w, ack=u+1` → `ACK, seq=u+1, ack=w+1`
为什么四次	TCP 是全双工，两个方向必须分别释放；半关闭期间 B 仍可发数据
TIME-WAIT / 2MSL	① 确保最后一个 ACK 能到达 B；② 使本次连接的所有报文段从网络中消失
释放最短时间	主动方 **1 RTT + 2MSL**；被动方 **1.5 RTT**
可靠传输	字节编号 + 累积确认 + 超时重传 + 快速重传 + SACK
RTO	$RTO = RTT_S + 4 \times RTT_D$；$\alpha = 1/8$，$\beta = 1/4$
Karn 算法	重传的报文段不采用其 **RTT** 样本；每重传一次 **RTO** 加倍
快速重传	收到 **3** 个重复确认立即重传，不等超时
SACK	通知已收到的不连续字节块，最多 4 个字节块；发送方不一定只重传缺失部分
流量控制	$swnd = \min(rwnd,\ cwnd)$；接收窗口 ⊆ 接收缓存，发送窗口 ⊆ 发送缓存
零窗口	用持续计时器 + 零窗口探测报文段（**1** 字节数据）打破死锁
慢开始	每确认 1 段 +1 MSS，每 **RTT** 翻倍
拥塞避免	每确认 1 段 $+1/cwnd$ MSS，每 **RTT +1 MSS**
超时处理	$ssthresh = cwnd/2$，$cwnd = 1$，重走慢开始
三重确认处理	$ssthresh = cwnd/2$，$cwnd = ssthresh$，直接进拥塞避免
拥塞 vs 流量	拥塞控制是全局性问题；流量控制是端到端问题

易错清单

传输层 22 个高频陷阱

1. 「有校验和」不等于「可靠传输」。UDP 有校验和，但校验失败只丢弃，不重传、不排序，因此是不可靠的。
2. **UDP** 的复用/分用依据是目的端口号（2018 年题 39），不是源端口号；源端口号只是让对方知道往哪儿回。
3. 网络层按协议字段分用，传输层按端口号分用，两个层次的「复用」含义不同，不要互相套用。
4. 端口号只有本地意义，两台主机上相同的端口号毫无关系；TCP 与 UDP 的端口号也相互独立。
5. 三类端口号分界点是 **1023** 和 **49151**。65535 属于短暂端口号。
6. **UDP** 的「长度」字段含 **8** 字节首部。给长度字段求数据长度要减 8，给数据长度求字段值要加 8。
7. 伪首部不传送、不递交，只在计算校验和时临时构造；UDP 协议号 17，TCP 协议号 6。
8. 反码求和一定要回卷进位，最后还要取反；若取反结果是全 0，应写全 1。
9. 全 **0** 校验和表示不校验（IPv4 的 UDP 允许），不是「校验通过」。
10. **TCP** 的数据偏移以 **4** 字节为单位，最小值 5（对应 20 字节固定首部），最大值 15（对应 60 字节）。别和 IP 的片偏移（**8** 字节为单位）混淆。
11. 序号是数据第一个字节的序号；确认号是期望收到的下一个字节的序号。两者语义相反，写题时最容易互换。
12. **ack = N** 表示到 **N−1** 为止的数据都已收到，与 GBN 中「ACK n 表示到 n 为止收到」差 1。
13. **SYN** 和 **FIN** 各消耗一个序号，纯 ACK 不消耗。第三、四次挥手能否携带数据的规定不同：第三次握手可以携带数据，第四次挥手不能携带数据。
14. 第一次和第二次握手都不能携带数据；第三次握手可以。
15. 只有主动关闭方进入 **TIME-WAIT**，而且必须等 **2MSL**；被动关闭方收到最后一个 ACK 后立即 CLOSED。
16. 释放时间不是「**2MSL**」：主动方是 1 RTT + 2MSL，被动方是 1.5 RTT（第 2、3 次挥手合并的最短情形）。
17. **MSL** 不是 **RTT**。2022 年题 39 中 MSL = 800 ms 与 RTT = 50 ms 完全无关，不要用 RTT 去算 2MSL。
18. **MSS** 只包含数据载荷，不含 TCP 首部和 IP 首部；MSS = MTU − 20 − 20 = 1460（以太网）。
19. 「收到 **3** 个重复确认」指冗余的那 **3** 个，加上原来那个共 4 个对同一报文段的确认。这个 3 的计数不要数错。
20. 超时与三重确认的处理不同：超时把 cwnd 降到 **1**；三重确认把 cwnd 降到新 **ssthresh**（原 **cwnd** 的一半），而且直接进拥塞避免，不经过慢开始。这是拥塞控制题的分水岭。
21. 慢开始「每 **RTT** 翻倍」，拥塞避免「每 **RTT +1**」；不要在拥塞避免阶段也做翻倍，也不要在慢开始阶段做 +1（尽管底层实现上慢开始每个确认确实只加 1 MSS）。
22. 发送窗口是 **min(rwnd, cwnd)**，且已发送未确认的数据仍占用窗口。2010 年题 39 的正确答案是 1000 B 而不是 2000 B，就是因为第 2 个 1000 B 段还没被确认。

第二十四章应用层

本章考纲要求

六、应用层（一）网络应用模型

1. 客户/服务器（C/S）模型
2. 对等（P2P）模型

（二）DNS 系统

1. 层次域名空间
2. 域名服务器
3. 域名解析过程

（三）FTP

1. FTP 协议的工作原理
2. 控制连接与数据连接

（四）电子邮件

1. 电子邮件系统的组成结构
2. 电子邮件格式与 MIME
3. SMTP 协议与 POP3 协议

（五）WWW

1. WWW 的概念与组成结构
2. HTTP 协议

应用层是 408 计网中每年必考、但分值稳定在 **2～3** 分的一章。考法高度模式化：单选题基本固定在第 33 题或第 40 题，考「某协议用什么传输层协议 / 哪个端口」「递归还是迭代」「是不是持久连接」；综合题的小问则集中在 **DNS** 查询次数与 **HTTP** 页面请求的 **RTT** 计算，以及 FTP 与 TCP 拥塞控制结合的分析。

本章的写法是：每个协议都按「工作机制 → 报文/命令流程 → 端口与传输层 → 计算套路」组织，凡是能用数字算一遍的地方（DNS 报文数、HTTP 的 RTT 数、FTP 的连接分析）都给出可直接照抄的公式与算例。

一、网络应用模型

（一）客户/服务器模型

客户/服务器（**Client/Server**，**C/S**）模型：通信双方地位不平等，一方是服务请求方（客户，**client**），另一方是服务提供方（服务器，**server**）。

- 客户（**client**）：面向用户，是主动方。它必须知道服务器的地址，主动发起通信、等待并接收结果。两个客户之间不能直接通信。
- 服务器（**server**）：面向任务，是被动方。它总是处于运行状态，拥有固定的 IP 地址与熟知端口号（如 HTTP 的 80、FTP 的 21），被动地等待并接受来自各客户的请求，处理后将结果返回。

工作流程可以概括为三步：

1. 服务器处于接受请求的状态（打开熟知端口，等待连接）。
2. 客户机发出服务请求，然后等待接收结果。
3. 服务器收到请求，进行处理，把结果返回给客户机。

Web、FTP、远程登录、电子邮件都是典型的服务集中型 C/S 应用——服务能力集中在少数服务器上。

C/S 模型的固有缺陷是可扩展性差：所有请求都由服务器集中处理，用户数增长时服务器的带宽与计算能力成为瓶颈；一旦服务器失效，整个服务就不可用。

（二）对等模型

对等（**Peer-to-Peer**，**P2P**）模型：没有固定的服务提供方与请求方，网络中的每一台计算机（称为对等方，**peer**）既是服务提供方又是服务请求方，对等方之间直接通信。

P2P 的本质仍然是「请求-响应」，只是每个结点同时扮演 client 与 server 两个角色。它的服务是分散型的——服务能力分散在大量对等计算机上。

P2P 的优点：

- 减轻服务器的计算压力，提高系统效率和资源利用率；多个用户之间可以直接共享文档。
- 可扩展性好：结点数增大时，系统的总上传能力也同步增大，性能不会因规模增大而降低。
- 健壮性强：个别结点失效对整个系统影响不大。
- 成本低。

P2P 的缺点：占用大量对等方的上/下行带宽，会影响个体的整机速度；缺乏集中管理，安全性、版权与可管理性差。

C/S 与 P2P 的对比（必背）

比较项	客户/服务器（**C/S**）	对等（**P2P**）
结点地位	不平等，客户与服务器角色固定	对等，每个结点两种角色兼有
谁主动	客户主动发起，服务器被动等待	双方都可主动发起
客户之间能否直接通信	不能	能，对等方之间直接通信
服务能力	集中在服务器（服务集中型）	分散在大量对等方（服务分散型）
可扩展性	差，服务器是瓶颈	好，结点越多总能力越大
典型应用	Web、FTP、电子邮件、远程登录	文件共享、即时通信、P2P 流媒体、分布式存储

（三）P2P 的三种组织方式

按「是否有索引服务器、索引如何组织」划分，P2P 有三种典型方式。

集中式（第一代）：代表是 **Napster**。仍然有一个中央索引服务器保存「哪个文件在哪些结点上」的目录，但文件内容本身不在服务器上传输——查询到结点列表后，由请求方与文件持有方直接建立连接传送文件。可见它把 C/S 的「集中索引」与 P2P 的「分散传输」结合起来。

- 优点：索引集中，查找效率高。
- 缺点：索引服务器是单点故障，且容易被起诉、被关闭。

全分布式非结构化（第二代）：代表是 **Gnutella**。没有任何中央服务器，所有结点完全对等。查询采用泛洪（**flooding**）方式：请求方把查询报文发给所有邻居，邻居再转发给自己的邻居，直到跳数（TTL）耗尽；命中者沿原路返回响应。

- 优点：完全去中心化，健壮性最好，无单点故障。
- 缺点：查询流量随规模指数增长，可扩展性差，且查询结果不保证完整。

混合式（第三代）：代表是 **BitTorrent**。它把结点分成两类：少数能力强的结点做超级结点（**super node**），负责索引与路由；其余普通结点在超级结点组织下分组（BitTorrent 中称为 torrent）。同一文件被切成许多块（**chunk**），下载者从多个持有者处并行下载不同的块，同时把已下载的块上传给别人，形成「人人为我，我为人人」的机制。

- 优点：兼顾了查找效率与可扩展性，是目前实际使用最广的方式。

P2P 易错点

- P2P 中「每个结点既是客户又是服务器」不等于「协议没有服务器进程」。混合式 P2P 中仍有超级结点；集中式 P2P 中仍有索引服务器。
- 「P2P 向多用户分发一个文件比 C/S 快」的原因不是单个结点带宽更大，而是系统的总上传能力随结点数增加。这是统考喜欢考的因果表述。
- C/S 模型中「客户与客户之间可以直接通信」是错误说法（2019 年真题的正确选项就是从这里出错的）。

真题考法

2019 年第 40 题直接考 C/S 与 P2P 模型的特征判断，四个选项分别对应「P2P 结点对等」「C/S 中客户之间能否直接通信」「谁主动发起」「分发文件所需时间」。只要记住上表的五行对比即可满分。

二、DNS 系统

DNS 的完整任务是把便于人记忆的主机名（域名）转换为便于机器处理的 **IP** 地址，它运行在 **UDP** 之上，熟知端口号 **53**，采用 **C/S** 模型。

（一）层次域名空间

因特网采用层次树状结构的命名方法。一个域名（**domain name**）由若干个分量（标号）组成，各分量之间用点「.」隔开，级别低的写在左边，级别高的写在右边：

…. 四级域名. 三级域名. 二级域名. 顶级域名

最右边是顶级域名（**Top Level Domain**，**TLD**），它的右边是隐藏的根（写作一个点，通常省略不写）。

例如 www.cqut.edu.cn：cn 是顶级域名，edu 是二级域名，cqut 是三级域名，www 是四级域名（主机名）。

顶级域名分为三类：

类别	含义	例子
国家顶级域名（nTLD）	用 ISO 3166 规定的两个字母的国家/地区代码	cn 中国、us 美国、uk 英国、jp 日本、hk 香港
通用顶级域名（gTLD）	按行业/性质划分	com 公司企业、net 网络服务机构、org 非营利组织、edu 美国教育机构、gov 美国政府机构、int 国际组织
基础结构域名（反向域名）	只有 arpa 一个	用于反向域名解析，即由 IP 地址查域名

二级域名是在某个顶级域名下注册的域名。我国的二级域名又分两类：类别域名（如 ac 科研机构、com 工商金融、edu 教育、gov 政府部门、net 网络服务机构、org 非营利组织）和行政区域名（如 bj 北京、sh 上海，共 34 个）。

关于域名与 **IP** 的关系，必须清楚三点：

1. 域名是逻辑概念，并不代表主机所在的物理地点；www.cqut.edu.cn 中的 cqut 不表示这台机器地理上一定在重庆。
2. 域名与 IP 地址不是一一对应的。一台主机可以有多个域名（如 www.abc.com 与 abc.com 指向同一台机器），一个域名也可以解析到多个 **IP** 地址（用于负载均衡）；多块网卡的主机甚至可以有一个 IP 对应多个域名。
3. DNS 里也不存储 **MAC** 地址，IP 到 MAC 的映射由 ARP 完成，两者不要混。

域名的两个基本要求

- 唯一性：一个域名在全球范围内只能指向相同的一批地址，这靠「按等级管理、同级不重名」来保证。
- 易查询：层次结构使得 DNS 可以把整个命名空间切分成许多区（**zone**），分散到不同服务器上管理。

（二）域名服务器

不可能把全部域名到 IP 的映射放在一台服务器上（负载过大、单点故障、时延过大），因此 DNS 是一个分布式数据库，用分布在各地的域名服务器共同实现转换。按层次自上而下分四类：

根域名服务器

- 是最高层次的域名服务器。
- 每一个根域名服务器都知道所有顶级域名服务器的域名及其 **IP** 地址。
- 全球共有 **13** 个根域名服务器（按 IP 地址计），每一个「服务器」实际由分布在世界各地的多台机器组成的服务器群集（cluster）实现；查询时由路由器用任播（**anycast**）把请求转发到离 **DNS** 客户最近的那一个根域名服务器。
- 根域名服务器通常并不直接解析域名，而是返回该域名所属顶级域名服务器的 **IP** 地址。它的作用更像是「指路」。

顶级域名服务器

- 负责管理在该顶级域名服务器注册的所有二级域名（即管一个顶级域，如.com、.cn）。
- 收到查询时给出回答：可能是最终的 IP 地址，也可能是指向下一级权限域名服务器的 **IP** 地址。

权限域名服务器（权威域名服务器）

- 负责管理某个区（**zone**）的域名。
- 每一台主机的域名都必须在某个权限域名服务器处注册登记，因此它掌握其管辖范围内域名与 IP 地址的映射关系。
- 它还知道其下级域名服务器的地址（若该区还往下划分了子区）。
- 一个区可以只对应一台权限域名服务器，但为了提高可靠性，通常一个区设置至少两台权限域名服务器（主、辅），其中一台失效时另一台仍可提供服务。

本地域名服务器（默认域名服务器）

- 它不属于上述的域名服务器等级结构——这是最常考的一句话。
- 当一个主机发出 DNS 查询请求报文时，这个报文首先被送往该主机的本地域名服务器。
- 本地域名服务器起着代理的作用，把查询转发到上述等级结构中，并把结果返回给主机。
- 每一个 ISP、一个大学乃至一个学院都可以拥有一个本地域名服务器。
- 它离用户很近（一般不超过几个路由器的距离，甚至就在同一局域网），其 **IP** 地址必须直接配置在需要域名解析的主机中（可由 DHCP 自动配置，也可手工配置）。

区（**zone**）与域（**domain**）的区别：域是命名空间的一个子树，是逻辑上的划分；区是 **DNS** 服务器实际管辖的范围，是为了分散管理而对域做的物理切分。一个区内的所有域名信息存放在同一台（组）权限域名服务器上，这台服务器称为该区的「权威」。下级的区可以把自己的授权委托出去，因此「域」的边界与「区」的边界不一定重合——一个域可以划分成多个区，每个区由不同的权限域名服务器管理。

区与权限域名服务器的易错点

- 题目问「该域名由哪台服务器解析」时，必须按题目给出的具体域名逐级判断：权限域名服务器可能不止一个，`abc.xyz.com` 的权限服务器与 `xyz.com` 的权限服务器通常不是同一台。
- 「本地域名服务器」永远不在根 → 顶级 → 权限这条等级链上，画图题里它一定单独画在旁边。

真题考法

常考形式是「下列说法错误的是」，错误项多半出在「本地域名服务器属于域名服务器等级结构」「根域名服务器直接返回 IP 地址」这两处。另外 2021 年、2016 年的题目要求根据拓扑图判断具体是哪台服务器完成解析，注意权限域名服务器可能不止一个。

（三）域名解析过程

把一个域名映射成 IP 地址的过程叫域名解析。查询方式有两种：

递归查询（**recursive query**）

客户把请求发给某台域名服务器后，如果该服务器不知道答案，它必须代替客户去询问下一台服务器，并把最终结果返回给客户。也就是说，被查询者承担了「继续查下去」的义务，客户只需要等一个最终答案。

- 特点：对客户最省事，但对被查询的域名服务器负担极大。

迭代查询（**iterative query**）

客户把请求发给某台域名服务器后，如果该服务器不知道答案，它只返回「你下一步该去问谁」的 **IP** 地址（转介信息），由客户自己再去询问下一台服务器；如此反复，直到某台服务器给出最终答案。

- 特点：服务器负担轻，但客户（通常是本地域名服务器）要多发几次请求。

因特网实际使用的组合方式（必背）

从请求主机到本地域名服务器的查询通常是递归查询；而本地域名服务器向根、顶级、权限域名服务器发出的查询通常是迭代查询。

原因：递归查询对被查询的服务器负担太大，而本地域名服务器数量相对少、能力较强，让它「跑腿」比让根域名服务器为每个客户层层代查更合理。

以主机 `m.xyz.com` 想查 `y.abc.com` 的 IP 地址为例，完整的迭代解析流程如下（假设各服务器与本地域名服务器的缓存都为空）：

1. `m.xyz.com` 向自己的本地域名服务器 `dns.xyz.com` 发出 **DNS** 查询请求报文（UDP 用户数据报，目的端口 53），报文中带查询域名 `y.abc.com` 和查询类型 A（地址记录）。这一步是递归查询。
2. 本地域名服务器缓存中没有该域名，于是向根域名服务器发出查询报文。
3. 根域名服务器不知道 `y.abc.com` 的地址，返回顶级域名服务器 **`dns.com`** 的 **IP** 地址。`dns.com` 是 `.com` 这个顶级域的服务器。
4. 本地域名服务器再向 `dns.com` 查询，`dns.com` 返回 **`y.abc.com`** 所在区域的权限域名服务器 **`dns.abc.com`** 的 **IP** 地址。
5. 本地域名服务器再向 `dns.abc.com` 查询，`dns.abc.com` 是 `abc.com` 区的权威服务器，知道 `y.abc.com` 的映射关系，于是返回 **`y.abc.com`** 的 **IP** 地址（最终答案）。
6. 本地域名服务器把结果返回给主机 `m.xyz.com`，并按 TTL 把这条记录存入自己的高速缓存。

一共 **4** 条查询报文 + **3** 条响应报文 = **7** 条 **DNS** 报文，其中本地域名服务器发出了 **4** 条查询（3 条迭代查询由它自己发起，另 1 条是它转发给根的那次）。如果题目只问「本地域名服务器发出多少次查询」，答案就是 **4** 次；如果问「本地域名服务器向外发出多少条 DNS 报文」，则是 **4** 条查询 + **3** 条收到的响应 = **7** 条（收到的不算「发出」）。这两种问法的区别是本章最容易失分的地方。

*1.*解析报文数与查询次数的计算

通用的计算规则有两条，务必记牢：

规则一：本地域名服务器发出的查询次数 = 需要经过的层次数。

若采用迭代查询，并且只考虑标准的 4 层结构（根 → 顶级 → 二级权限 → 三级权限…），那么：

$$\text{本地域名服务器发出的查询次数} = \text{域名的级数（去掉主机名）} + 1$$

其中「+1」是向根域名服务器的那一次查询。

规则二：如果题目说「本地域名服务器只提供递归查询，其他服务器只提供迭代查询」，则只有本地域名服务器在跑腿，用户的请求只有 1 条由主机发出，用户的等待时间只需按本地域名服务器的迭代次数计算 RTT。

算例一（无缓存、本地迭代）

主机 H 访问 `www.abc.xyz.com`，本地域名服务器无任何缓存，采用迭代查询。求本地域名服务器发出的最少、最多 DNS 查询次数。

- 最少 **0** 次：本地域名服务器的高速缓存中已有 `www.abc.xyz.com` 的 IP 地址，直接回答 H，不需要向外发任何查询。
- 最多 **4** 次：无缓存时按迭代依次查询

1. 根域名服务器 → 得到 .com 顶级域名服务器地址；
2. .com 顶级域名服务器 → 得到 xyz.com 的权限域名服务器地址；
3. xyz.com 权限域名服务器 → 得到 abc.xyz.com 的权限域名服务器地址；
4. abc.xyz.com 权限域名服务器 → 得到 www.abc.xyz.com 的 IP 地址。即：www.abc.xyz.com 去掉主机名 www 后剩 abc、xyz、com **3** 级，$3+1=4$ 次。

DNS 报文数计算三个必踩的坑

1.「发出」还是「收到」：题目问 201.1.1.1（本地域名服务器）发出的查询次数时，不包含主机 **H** 发给它的那一条递归查询；最少 0 次、最多 4 次（2016 年真题答案）。
2. 主机名不计入级数：www.abc.xyz.com 的迭代次数按 abc.xyz.com 算（3 级 + 1 = 4 次），不要把 www 也算成一级而得出 5 次。
3.「最少」的两种情形不同：若问「本地域名服务器发出的查询次数」，有缓存时是 **0** 次；若问「从点击链接到收到页面」，即使本地有缓存，主机到本地域名服务器的那 **1** 次递归查询（**1** 个 **RTT**）仍然要算。

算例二（**DNS** 与 **HTTP** 合并计算，**2020** 年真题模型）

主机 H 通过超链接 http://www.abc.com/index.html 请求纯文本页面 index.html。局域网内主机访问 Internet 上各服务器的 RTT 均为 10 ms，忽略其他时延。本地域名服务器只提供递归查询服务，其他服务器只提供迭代查询服务。求从点击超链接到浏览器收到页面所需的最短与最长时间。

- **HTTP** 部分固定要 **2** 个 **RTT**：1 个 RTT 建立 TCP 连接（三次握手的前两次往返归入一个 RTT），1 个 RTT 发送 HTTP 请求并收到响应。$2\times10=20$ ms。
- 最短：本地域名服务器已有 www.abc.com 的缓存，不再对外查询，于是主机与本地服务器之间仍是 **1** 次递归查询，但它与 TCP 建连的第一个 RTT 在时间上被合并考虑后不额外增加 RTT——按 2020 年真题的标准解法，此时总时间就是 HTTP 的 **20 ms**（DNS 查询的时延被计入/忽略方式以真题为准，此处真题给的是 20 ms 最短）。
- 最长：无任何缓存，本地域名服务器按迭代依次查询根、.com 顶级、abc.com 权限域名服务器，共 **3** 个 **RTT**（$3\times10=30$ ms）；加上 HTTP 的 20 ms，共 **50 ms**。

所以答案是「20 ms，50 ms」。注意域名 www.abc.com 的迭代次数是 3 次（abc + com 两级，再加根一次），不要算成 4 次。

*2.*高速缓存

为了提高查询效率、减轻根域名服务器的负荷、减少因特网上的 DNS 查询报文数量，各级域名服务器广泛使用高速缓存（**cache**），用来存放最近查询过的域名以及从何处获得域名映射信息的记录。

- 本地域名服务器缓存命中时，直接把缓存中的结果返回给主机，不再向外发出任何查询。
- 用户主机中也有高速缓存：许多主机在启动时从本地域名服务器下载域名与 IP 地址的数据库，维护自己最近使用过的域名缓存，只在缓存中找不到时才向域名服务器查询。
- 缓存中的记录有生存时间（**TTL**）。由于域名到 IP 的映射关系并非永久不变，域名服务器必须为每项内容设置计时器，删除超过合理时间的项（例如每个项目只存放两天）。因此缓存可能返回过期的映射，这是用「效率」换来的代价。

缓存易错点

- 有缓存时「查询次数为 0」指的是本地域名服务器不向外查询，主机到本地域名服务器的那一次查询仍然发生。

- 「主机与本地域名服务器之间采用递归查询」这一点不因有无缓存而改变。缓存命中只是让本地域名服务器不必再跑腿。

3.DNS 的传输层选择

DNS 什么时候用 UDP，什么时候用 TCP

- 主要使用 **UDP**，熟知端口号 **53**。普通查询与应答都是单个请求/单个响应，报文短、无连接开销、时延小，重传由应用层自己用超时机制解决。
- 下列情况必须改用 **TCP**（同样用端口 **53**）：
  1. 区域传送（**zone transfer**）：主域名服务器向辅助域名服务器同步整个区的数据库，数据量可能很大，必须保证可靠传输。
  2. 响应报文超过 **512** 字节：传统的 DNS 报文在 UDP 里限长 512 字节，超过时由服务器返回「截断」标志，客户端改用 TCP 重发；或者直接使用 EDNS0 扩展。所以 2018 年第 33 题「可以使用传输层无连接服务的应用层协议」答案是 **DNS**。

真题考法

- 2010 年第 40 题：递归查询时，用户主机与本地域名服务器各发出 **1** 条、**1** 条请求消息。
- 2016 年第 40 题：迭代查询时，本地域名服务器 `201.1.1.1` 发出查询的最少 **0** 次、最多 **4** 次。
- 2018 年第 33 题：可以使用无连接服务的应用层协议是 **DNS**。
- 2020 年第 40 题：DNS 迭代 + HTTP 请求页面，最短 **20 ms**、最长 **50 ms**。
- 2021 年第 47 题：Web 访问前主机除 HTTP 外还运行了 **DNS**，其封装为「DNS 报文 → UDP 数据报 → IP 分组 → 以太网帧」。

三、FTP

（一）FTP 协议的工作原理

文件传送协议（**File Transfer Protocol**，**FTP**）是因特网上使用最广泛的文件传送协议。它提供交互式访问，允许客户指明文件的类型与格式（如是否使用 ASCII 码），并允许文件具有存取权限（用户必须经过授权并输入有效口令）。FTP 屏蔽了各计算机系统的细节，因而适合于在异构网络中任意计算机之间传送文件。

FTP 采用客户/服务器工作方式，使用 **TCP** 提供的可靠传输服务。一个 FTP 服务器进程可同时为多个客户进程提供服务，其进程分为两类：

- 主进程（**1** 个）：负责接收新的请求，始终监听熟知端口 **21**。
- 从属进程（多个）：每处理一个客户的请求就 fork 一个从属进程，负责该客户的数据传送。

FTP 工作时使用两条并行的 **TCP** 连接：

比较项	控制连接	数据连接
作用	传送控制信息（连接请求、登录、目录操作、传送请求、终止等命令与响应）	传送文件内容或目录列表
服务器端口	**21**（熟知端口）	主动模式下为 **20**；被动模式下为服务器随机分配的临时端口
客户端口	客户自己分配的临时端口	客户自己分配的临时端口
建立/释放时机	在整个会话期间始终保持打开	每次文件传输时才建立，传输结束就关闭（按需建立）

比较项	控制连接	数据连接
谁发起	客户主动发起（客户连服务器 21）	主动模式下服务器发起；被动模式下客户发起
传输内容	命令与应答，是文本	文件数据，可以是二进制

因为控制信息走的是另一条独立的连接，而不是与数据混在同一条连接里，所以称 **FTP** 的控制信息是「带外（**out-of-band**）」传输的。这正是 FTP 与 HTTP 的根本差别：HTTP 把请求与响应都放在同一条 TCP 连接里，属于「带内（in-band）」传输。

带外控制的含义

「带外」指控制信息与数据走不同的连接（不同的 **TCP** 连接，进而使用不同的端口）。好处是：即使数据连接正在满负荷地传送一个大文件，控制连接依然是通的，客户可以随时发出「终止传输」「放弃」等命令，服务器也能即时回应。若把控制命令塞进数据流里排队（带内），就必须等前面的数据传完才能终止，响应会非常迟钝。

*1.*主动模式与被动模式

数据连接的建立方向有两种模式，这是 FTP 最常考的细节。

主动模式（**PORT** 模式）

1. 客户用临时端口连接服务器的 **21** 端口，建立控制连接并登录。
2. 客户需要传数据时，在控制连接上随机开一个临时端口，并通过 **PORT** 命令把「我的 IP + 我的端口号」告诉服务器。
3. 服务器用自己 **20** 端口主动去连接客户刚才告诉它的那个端口，建立数据连接。
4. 因此「主动」是指服务器主动发起数据连接。

被动模式（**PASV** 模式）

1. 控制连接同样建立在服务器的 21 端口。
2. 客户需要传数据时，在控制连接上发送 **PASV** 命令。
3. 服务器随机打开一个临时端口，并把该端口号通过控制连接告诉客户。
4. 客户主动去连接服务器的这个临时端口，建立数据连接。
5. 因此「被动」是指服务器被动等待客户来连。

被动模式的意义在于：主动模式下服务器要反过来连客户，如果客户在防火墙/NAT 后面，这个入向连接会被拦掉；被动模式下所有连接都由客户发起，防火墙友好。

FTP 端口易错点

- **20** 和 **21** 都是服务器端口号，客户端的端口号由客户系统自行分配，不固定。2017 年第 40 题的错误选项正是「服务器与客户端的 TCP 20 端口建立数据连接」——20 是服务器的端口，不是客户端的。
- 被动模式下数据连接不使用 **20** 端口，而是服务器临时分配的随机端口。所以「FTP 数据连接一定用 20」是错的。
- 控制连接在整个会话期间保持打开；数据连接每传一次就建立、传完就关闭。2023 年第 47 题第 (1) 问的答案就是「控制连接是持久的，数据连接是非持久的」。
- 「会话期间保持打开」指控制连接一直存在，但数据连接可以是多次：传一个文件就关一次，再传再建。

2.FTP 连接分析算例

算例（基于 **2023** 年第 **47** 题的模型）

主机 H 登录到 FTP 服务器后，向服务器上传一个大小 18000 B 的文件 F。建立数据连接时选择的初始序号为 100，MSS $=1000$ B，拥塞控制初始阈值 $ssthresh = 4$ MSS，RTT $=10$ ms，忽略 TCP 传输时延，H 均以 MSS 段发送数据且无差错、无丢包、无乱序。问：

(1) 建立了哪些 **TCP** 连接？ H 登录 FTP 服务器时先建立的是控制连接（客户临时端口 $\rightarrow$ 服务器 21）。此后每次传文件再建立数据连接。所以 H 登录时建立的 TCP 连接是控制连接。

(2) F 的第一个字节序号是多少？数据连接本身的初始序号是 100，但 F 的第一个字节前面还有 **1** 个 **SYN** 段占用的序号：TCP 的 SYN 会消耗 1 个序号。因此 F 的第一个字节序号是 $100+1=101$。

(3) 断开数据连接时，**FTP** 服务器发送的第二次挥手 **ACK** 段的确认序号是多少？ F 共 18000 B，从序号 101 开始，则最后一个字节的序号为 $101+18000-1=18100$。因此服务器在第二次挥手（对 H 的 FIN 的确认）中，确认序号为 $18100+1=$ **18101**。

(4) 拥塞窗口的调整：慢开始阶段每收到一个对新数据的确认，$cwnd$ 加 1 MSS（指数增长），直到达到 $ssthresh = 4$ MSS 后转为拥塞避免（线性增长，每 RTT 加 1 MSS）。

- 收到确认序号 **2101** 时：已连续确认到 2100，即 $2100-101+1=2000$ B $=2$ MSS，此时 $cwnd$ 从 1 MSS 增至 **2 MSS**（慢开始阶段）。
- 收到确认序号 **7101** 时：已连续确认到 7100，即 $7100-101+1=7000$ B $=7$ MSS。按慢开始：1→2→4 MSS（达到阈值），此后进入拥塞避免，每 RTT 加 1 MSS：4→5→6→7 MSS。所以 $cwnd$ 调整为 **7 MSS**。

(5) 总时间与应用层平均速率：需要把 18000 B 分成 18 个 MSS 段。计入建立数据连接（1 RTT）与四次挥手（其中 H 的 FIN 与服务器的 ACK 常合并分析，按真题口径计 2 个 RTT）后共 **7** 个 **RTT**，即 $7\times10=70$ ms；应用层数据平均发送速率 $=18000\,\text{B}/70\,\text{ms}\approx257\,\text{B/ms}=257\,\text{kB/s}$（按真题给定口径计算结果）。

真题考法

2009 年第 40 题考「FTP 命令走的是建立在 TCP 之上的控制连接」；2017 年第 40 题考 FTP 两条连接的四个特征判断；2023 年第 47 题把 FTP 与 TCP 序号、拥塞窗口、四次挥手结合成大题。**FTP** 部分真正考的是 **TCP**，**FTP** 只是外壳，所以第 (2) ~ (5) 问的工夫要花在传输层上。

（二）控制连接与数据连接

控制连接的作用：客户与服务器之间传送命令与应答。它承载的内容包括用户登录（用户名、口令）、改变工作目录、列出目录（`LIST`）、指定传输模式与文件类型、发起一次文件传送的请求（`RETR` 下载 / `STOR` 上传）、终止传送（`ABOR`）以及退出（`QUIT`）等。控制连接传送的是文本形式的命令与三位数字应答码，不传送文件数据。

数据连接的作用：在客户与服务器之间传送文件内容（`RETR`/`STOR`）或目录列表（`LIST`）。它的特点是按需建立、用完即关。数据连接上还可以选择传输方式（流方式、块方式、压缩方式）与表示类型（ASCII 文本、二进制图像），这是「FTP 允许客户指明文件类型与格式」的体现。

FTP 两条连接与 **HTTP** 一条连接的对比

比较项	**FTP**	**HTTP**
TCP 连接条数	**2** 条（控制连接 + 数据连接）	**1** 条（同一连接既传请求响应也传实体）
控制信息位置	带外（独立控制连接）	带内（与数据在同一连接，靠报文格式区分）
连接是否持续	控制连接持续；数据连接按需建立、用完释放	非持久连接下一个对象一条连接；持久连接下保持
状态	有状态：服务器记住登录状态与当前工作目录	无状态：服务器不保存客户的历史信息（Cookie 是后加的补救）
传输层	TCP	TCP
默认端口	21（控制）、20（主动模式数据）	80

「FTP 是有状态的」这一点常被考到：服务器要记住用户是否已登录、当前处于哪个目录、传的是哪个文件——这些状态跨多条命令保持，是控制连接长期存在的原因。HTTP 每条请求自带完整信息（`Host`、`Cookie` 等），服务器不必记住上一次是谁来访问过。

四、电子邮件

（一）电子邮件系统的组成结构

电子邮件系统采用客户/服务器方式，由三个主要部分组成：

用户代理（**User Agent**，**UA**）

用户与电子邮件系统之间的接口，即电子邮件客户端软件或网页版邮件界面。它的职责是：撰写、显示、处理和发送与接收邮件。用户代理把邮件「提交」给自己的邮件服务器，或从自己的邮件服务器「取回」邮件。

邮件服务器

电子邮件系统的基础设施，负责发送和接收邮件，同时维护用户的邮箱。它同时充当客户和服务器：向别的邮件服务器发送邮件时是 SMTP 客户，接收别的邮件服务器送来的邮件时是 SMTP 服务器。邮件服务器必须在 24 小时不间断运行，并具有固定的 IP 地址与域名。

协议

- 邮件发送（推送）协议：**SMTP**（TCP 25），负责「用户代理 → 发送方邮件服务器」以及「发送方邮件服务器 → 接收方邮件服务器」。
- 邮件读取（拉取）协议：**POP3**（TCP 110）或 **IMAP**（TCP 143），负责「接收方邮件服务器 → 用户代理」。
- 网页版邮件还可由 **HTTP** 承担发送与读取。

邮件收发的完整流程（背下来就能做所有流程题）

设发件人 A 的邮箱在邮件服务器 S1 上，收件人 B 的邮箱在邮件服务器 S2 上：

1. A 通过用户代理撰写邮件。
2. 用户代理用 **SMTP** 把邮件「推」给发送方邮件服务器 **S1**。
3. S1 把邮件放入邮件缓存队列中等待发送。
4. S1 的 SMTP 客户进程周期性扫描缓存队列（例如每 30 分钟），发现有待发邮件后，与接收方邮件服务器 **S2** 的 SMTP 服务器进程建立 **TCP** 连接（端口 **25**）。
5. TCP 连接建立后，S1 用 SMTP 把邮件「推」给 **S2**；所有邮件发送完毕后断开 TCP 连接。

6. S2 收到邮件后，把它放入收件人 **B** 的用户邮箱中，等待 B 在方便的时候读取。
7. B 想收信时，通过用户代理用 **POP3** 或 **IMAP** 从 S2 「拉」回自己的邮件。

判断口诀：凡是「交给服务器 / 服务器之间转发」的环节都用 **SMTP**（推）；凡是「从自己的服务器取回来」的环节都用 **POP3/IMAP**（拉）。

电子邮件流程易错点

- 不能把邮件直接推给收件人的用户代理。原因有二：并非所有计算机都能运行邮件服务器程序；收件人的计算机也不一定 24 小时开机，发信时若对方不在线就无法处理，所以必须由始终在线的邮件服务器代为缓存。
- 电子邮件地址格式为「用户名 @ 邮件服务器所在主机的域名」，例如 `abc@163.com`。@ 后面是域名而不是 IP。
- 所有邮件相关连接都基于 **TCP**，都是 **C/S** 方式。没有一个邮件协议用 **UDP**。
- 2012 年第 40 题的三个阶段依次是 **SMTP**、**SMTP**、**POP3**：① UA→ 发送方服务器，② 发送方服务器 → 接收方服务器，③ 接收方服务器 →UA。

（二）电子邮件格式与 MIME

邮件格式在 RFC 822（2008 年后更新为 RFC 5322）中单独定义，不是由 **SMTP** 定义的。一封邮件由信封和内容两部分构成，内容又分为首部和主体。

- 信封：由邮件系统自动生成，从首部中提取所需信息写在信封上，相当于「邮局看的部分」。
- 首部（**header**）：由若干首部行组成，每行是「字段名: 值」。关键字包括：
  - `From`：发件人邮箱地址；
  - `To`：收件人邮箱地址（多个用逗号分隔）；
  - `Cc`：抄送（carbon copy）收件人；
  - `Subject`：邮件主题；
  - 还可有 `Date:`、`Reply-To:`、`Bcc:`（密送）等。
- 主体（**body**）：用户自由编写的信件正文，是真正要传达的内容。

首部与主体之间的空行

首部与主体之间必须用一个空行分隔。邮件系统读到这个空行即认为首部结束、下面全部是正文。少写这个空行会把正文的第一行当成又一个首部字段，导致正文丢失或解析错误。

MIME（多用途因特网邮件扩展，**Multipurpose Internet Mail Extensions**）

为什么需要 MIME

SMTP 只能传送 7 位 **ASCII** 码文本数据，不能传送可执行文件或其他二进制对象，不能满足传送多媒体邮件（图片、音频、视频）的需要，许多非英语文字（中文、俄文等）也无法用 SMTP 直接传送。**MIME** 不是要取代或改动 **SMTP**，它的意图是继续使用现有的邮件格式与 **SMTP**，只在邮件主体上做扩展。

MIME 做了三件事：

1. 增加了 **5** 个新的邮件首部字段，提供有关邮件主体的信息。最常用的是：
   - `Content-Type`：说明主体的数据类型与子类型，如 `text/plain`、`text/html`、`image/jpeg`、`application/octet-stream`、`multipart/mixed`。它决定了接收方用什么方式「解释」主体。
   - `Content-Transfer-Encoding`：说明主体的编码方式，如 `7bit`、`8bit`、`base64`、

`quoted-printable`，即「这段内容是经过哪种编码才变成 ASCII 的」。

- 其余还有 `Content-ID`、`Content-Description`、`MIME-Version` 等。

2. 定义了多种邮件内容的格式，对多媒体电子邮件的表示方法进行了标准化。
3. 定义了传送编码，可对任何内容格式进行转换而不会被邮件系统改变。

base64 编码是最重要的传送编码：它把任意二进制数据按每 **3** 个字节（**24** 位）一组，重新划分为 **4** 个 **6** 位，每个 6 位用一个 **ASCII** 字符表示（字符表为 `A-Z a-z 0-9 + /`），因此编码后数据量约为原来的 $\frac{4}{3}$，即膨胀约 **33%**。若最后一组不足 3 字节，用 = 补齐。它保证编码结果全部落在 7 位 ASCII 范围内，从而能安全通过只认 ASCII 的 SMTP。

MIME 后来也被同样面向 ASCII 字符的 **HTTP** 使用（HTTP 首部里的 `Content-Type` 就来源于此）。

MIME 易错点

- **MIME** 不改变 **SMTP** 协议，SMTP 仍然只传 ASCII；是 MIME 在发送前编码、接收后解码，把非 ASCII 内容「伪装」成 ASCII。
- 2018 年第 40 题问「无需转换即可由 SMTP 直接传输的内容」，答案是 **ASCII** 文本；JPEG、MPEG、EXE 都必须先编码。
- `Content-Type` 说的是「是什么」，`Content-Transfer-Encoding` 说的是「怎么编码的」，两者不要搞反。

（三）SMTP 协议与 POP3 协议

1.SMTP 的工作过程

SMTP（**Simple Mail Transfer Protocol**，简单邮件传送协议）：C/S 模型，建立在 **TCP** 之上，熟知端口号 **25**。它是「推」协议——由发送方主动把邮件推给接收方。

SMTP 的一次典型会话（命令/响应方式，命令由客户发出，应答是三位数字码）：

步骤	客户（发送方）发出	服务器应答	含义
1	（建立 TCP 连接，端口 25）	220	服务器主动推送「服务就绪」
2	`HELO hnust.edu.cn`	250	客户说明身份、告知自己的域名；身份有效则回 250，否则回其他代码（如 421 服务不可用）
3	`MAIL FROM:` 后跟发件人邮箱地址	250	告知邮件来自何方
4	`RCPT TO:` 后跟收件人邮箱地址	250	告知邮件去往何地；邮箱存在则回 250
5	`DATA`	354	告知准备发送邮件内容；服务器准备好接收则回 354
6	<邮件内容 > 后跟一行只有「.」的结束符	250	传送正文，用单独一行的句点表示结束；收件成功回 250
7	`QUIT`	221	请求断开 TCP 连接；服务器回 221 并主动关闭

SMTP 共有约 14 条命令、21 种应答。应答代码后面一般跟有简单的描述信息，不同的 SMTP 服务器对同一应答码的描述信息可能不同。

发送方邮件服务器的工作特点：邮件先进入缓存队列，服务器周期性扫描（如每 30 分钟）缓存，发现有邮件才尝试与接收方服务器建立连接并发送；若发送失败则保留在队列中稍后重试。

SMTP 的三大局限与推论

1. 只能传送 **7** 位 **ASCII** 码文本——这是引入 MIME 的直接原因，也是最常考的一条。
2. 明文传送，不利于保密——SMTP 本身不加密，需要安全性时用 TLS 加密或使用带加密的扩展。
3. 只负责「推」到收件人的邮件服务器，不负责「取」——用户代理从服务器读邮件要另用 POP3/IMAP。所以 2013 年第 40 题「SMTP 支持从邮件服务器向用户代理发送邮件」是错误的。

2.POP3 的工作过程

POP3（**Post Office Protocol**，邮局协议第 **3** 版）：C/S 模型，建立在 **TCP** 之上，熟知端口号 **110**，是因特网正式标准。它是「拉」协议——由用户主动从邮件服务器把邮件拉回本地。

POP3 会话分为三个阶段：

1. 认证阶段：客户用 `user <用户名 >` 与 `pass <口令 >` 登录，服务器校验身份。
2. 事务处理阶段：客户可以 `stat`（查询邮件数与总大小）、`list`（列邮件清单）、`retr`（取回第 n 封邮件）、`dele`（标记删除第 n 封）、`quit`。
3. 更新阶段：客户发出 `quit` 后进入此阶段，服务器真正执行事务阶段中标记的删除操作，释放资源并关闭连接。

POP3 的两种下载方式（考纲明确要求）：

- 下载并删除（**download and delete**）：邮件取回本地后立即从服务器删除。优点是服务器空间释放快；缺点是用户换一台机器就再也看不到这些邮件。
- 下载并保留（**download and keep**）：邮件取回后仍保留在服务器上。优点是可以多台设备重复读取；缺点是占用服务器存储，且用户需要自己清理。

POP3 不允许用户在邮件服务器上管理自己的邮件（不能建文件夹、不能分类、不能只读某个首部），这是它「功能有限」的体现。

IMAP（**Internet Message Access Protocol**，因特网邮件访问协议）：端口 **143**，功能比 POP3 强大。用户在自己的计算机上就可以操控服务器中的邮箱，就像在本地操控一样，因此 IMAP 是联机协议；它允许用户代理只获取报文的某些部分（例如只读一个首部），也允许在服务器上创建文件夹、分类管理。

共同点：POP3 与 IMAP4 都采用基于 **TCP** 连接的客户/服务器方式。

3.SMTP 与 *POP3* 的对比

SMTP 与 POP3 对比表（必背）

比较项	**SMTP**	**POP3**
作用	发送（推送）邮件：UA → 发送方服务器、服务器 → 服务器	读取（拉取）邮件：接收方服务器 → UA
数据流动方向	推（**push**），发送方主动	拉（**pull**），接收方主动
传输层	TCP	TCP
熟知端口号	**25**	**110**
工作模型	C/S	C/S
传输内容限制	只能传 **7** 位 **ASCII** 文本（非 ASCII 需 MIME 编码）	传送的是整封邮件（内容已在服务器上存好），本身不限制内容

比较项	SMTP	POP3
会话特点	命令/应答（HELO、MAIL FROM、RCPT TO、DATA、QUIT）	三个阶段：认证、事务处理、更新；`quit` 后才真正删除
是否需要认证	传统 SMTP 无认证（后来增加了认证扩展）	认证阶段必须提供用户名与口令
是否允许管理服务器邮箱	不涉及	不允许建文件夹、分类等（IMAP 才允许）

邮件协议易错点

- 端口记错：SMTP 是 25，POP3 是 **110**，IMAP 是 **143**。不要写成 25/110 之外的数。
- 方向记反：SMTP 是「推」，POP3/IMAP 是「拉」。凡是「服务器 → 用户代理」的环节一定不是 **SMTP**（除非是网页邮件用 HTTP）。
- **2015** 年第 **33** 题问「POP3 使用的传输层服务类型」，答案是有连接可靠的传输服务（TCP）。
- 「**SMTP** 地址出错不能发邮件，**POP3** 地址出错不能收邮件」这类对应关系可以辅助记忆，但真正要记的是协议与端口的对应。
- 网页版邮件（如用浏览器收发 163 邮箱）时，UA 到服务器用 **HTTP**，服务器之间仍用 **SMTP**——所以「邮件发送一定用 SMTP」这个说法在网页邮件的第一个环节上并不成立。

五、WWW

（一）WWW 的概念与组成结构

万维网（**World Wide Web**，**WWW**）并非某种特殊的计算机网络，而是一个大规模的、联机式的信息储藏所，是运行在因特网上的一个分布式应用。它利用网页之间的超链接（**hyperlink**）把不同网站的网页链接成一张逻辑上的信息网，用户通过「点击链接」就能在全世界范围内跳转浏览。

万维网由 Tim Berners-Lee 于 1989 年 3 月在欧洲粒子物理实验室提出；1993 年 2 月第一个图形界面浏览器 Mosaic 问世，1995 年 Netscape Navigator 上市。

万维网的三个核心要素：

要素	全称	解决的问题
URL	统一资源定位符（Uniform Resource Locator）	怎样标识分布在世界各地的文档（资源在哪）
HTTP	超文本传送协议（HyperText Transfer Protocol）	用什么协议传送文档
HTML	超文本标记语言（HyperText Markup Language）	怎样解析/显示文档的格式

浏览器最重要的部分是渲染引擎（浏览器内核），负责解析并显示网页内容。常见内核：Chrome 与 Opera 用 Blink，Firefox 用 Gecko，Safari 用 WebKit。不同内核对网页的解析不同，因此同一网页在不同浏览器里显示效果可能不同。除 HTML 外，网页通常还包含 **CSS**（层叠样式表，从审美角度描述样式）与 **JavaScript**（一种脚本语言，控制网页行为，与 **Java** 没有任何关系）。

1.URL 的完整格式

URL 用来指明因特网上任何种类「资源」的位置。其一般形式为：

$$\langle\text{协议}\rangle : //\langle\text{主机}\rangle : \langle\text{端口}\rangle/\langle\text{路径}\rangle$$

以 `http://www.hnust.cn:80/ggtz/119945.htm` 为例：

部分	值	含义
协议	`http`	访问资源所用协议（还可能是 `https`、`ftp`、`file` 等）
主机	`www.hnust.cn`	存放资源的主机域名（也可以直接写 IP 地址）
端口	`80`	服务器上的端口号，可以省略；省略时使用该协议的默认端口（HTTP 为 80）
路径	`/ggtz/119945.htm`	资源在主机上的路径与文件名，可以省略；省略时表示访问该网站的主页（**home page**）

需要扩展记忆的是，完整 URL 还可以带查询串（**query**）与片段标识符（**fragment**）：

$$\underbrace{https}_{\text{协议}}://\underbrace{\text{www.abc.com}}_{\text{主机}}:\underbrace{443}_{\text{端口}}\underbrace{/news/list.htm}_{\text{路径}}\underbrace{?id=408\&page=2}_{\text{查询}}\underbrace{\#top}_{\text{片段}}$$

- 查询（`?` 之后）：以`?` 开始、键 `=`值用 `&` 连接，把参数传给服务器上的程序，会被发送给服务器。
- 片段（`#` 之后）：定位到文档内部的某个锚点，只由浏览器本地使用，不会发送给服务器——这是容易被考到的细节：改变片段不会产生新的 HTTP 请求。

URL 易错点

- URL 中的主机部分写的是域名（如 `www.hnust.cn`），但真正建 TCP 连接前必须先用 **DNS** 把它解析成 **IP**。
- 「端口可省略」的前提是使用默认端口：HTTP 是 80，HTTPS 是 443，FTP 是 21。
- 片段标识符不参与网络请求；查询串参与。

（二）HTTP 协议

HTTP（超文本传送协议）定义了浏览器（万维网客户进程）怎样向万维网服务器请求万维网文档，以及服务器怎样把文档传送给浏览器。它建立在 **TCP** 之上，熟知端口号 **80**。

HTTP 有三个重要性质：

- **HTTP** 是面向事务（**transaction-oriented**）的：一次「请求-响应」就是一个不可分割的事务，一个请求只请求一个资源，不存在一次请求 5 个文档的说法。
- **HTTP** 是无连接的：通信双方在交换 HTTP 报文之前不需要先建立 **HTTP** 连接——但 HTTP 基于 TCP，必须先用 **TCP** 三次握手建立连接。
- **HTTP** 是无状态的：同一个客户第二次访问同一文档时，服务器的响应与第一次相同，服务器不保存客户的历史信息。这样做简化了服务器设计，但无法识别用户，因此后来引入了 Cookie。

1.HTTP 报文格式

HTTP 是面向文本的，报文中的每个字段都是 **ASCII** 码串，且各字段长度不确定。

请求报文的结构：

请求行 | 首部行 | 空行 | 实体主体

- 请求行：方法 空格 URL 空格 版本 CRLF，例如 `GET /index.htm HTTP/1.1`。
- 首部行：每行是首部字段名： 值 CRLF，可有多行。

- 空行：一个只有 CRLF 的行，标志首部结束（必需）。
- 实体主体：通常不用（GET 不用；POST 才用）。

一个典型的请求报文：

```
GET /index.htm HTTP/1.1
Host: www.hnust.cn
Connection: close
User-Agent: Mozilla/5.0
Accept-Language: cn
```

各行含义：GET 指明方法，/index.htm 是请求的 URL，HTTP/1.1 是版本；Host 指明服务器域名（HTTP/1.1 必需）；Connection: close 告诉服务器发完文档后释放连接（即不使用持续连接）；User-Agent 告知浏览器类型与版本；Accept-Language: cn 表示希望优先得到中文版本文档。

响应报文的结构：

状态行 | 首部行 | 空行 | 实体主体

- 状态行：版本 空格 状态码 空格 短语 CRLF，例如 HTTP/1.1 404 Not Found。
- 首部行、空行与请求报文同理。
- 实体主体：服务器返回的文档内容（有些响应报文不用，如 304）。

常见响应状态行示例：HTTP/1.1 202 Accepted（接受请求）、HTTP/1.1 400 Bad Request（错误的请求）、HTTP/1.1 404 Not Found（找不到页面）。

*2.*常见方法与状态码

HTTP 方法（请求行中的第一个字段）：

方法	含义
GET	请求读取由 URL 标识的信息（最常用），参数附在 URL 后面
POST	向服务器提交数据（如表单内容），参数放在实体主体中，不出现在 URL 里
HEAD	请求读取由 URL 标识的信息的首部，而不返回实体主体，常用于检查资源是否存在/是否被修改
PUT	向服务器上传/存放一个文档（写操作）
DELETE	删除服务器上由 URL 标识的资源

状态码是三位数字，第一位表示响应类别：

类别	含义	常见码
1xx	通知信息，表示已收到请求，正在继续处理	100 Continue
2xx	成功，操作被成功接收并处理	200 OK（请求成功，实体在响应中）、202 Accepted
3xx	重定向，需要进一步的动作才能完成请求	301 Moved Permanently（永久移动）、302 Found（临时移动）、304 Not Modified（资源未修改，可用缓存）

类别	含义	常见码
4xx	客户出错，请求有语法错误或无法实现	400 Bad Request、403 Forbidden（服务器拒绝）、404 Not Found
5xx	服务器出错，服务器本身发生错误	500 Internal Server Error

必须记住的状态码

- **200**：请求成功。
- **301 / 302**：永久 / 临时重定向。
- **304 Not Modified**：资源未修改。服务器用它对 `If-Modified-Since` 或 `If-None-Match` 的条件请求作答，响应中不携带实体主体，客户直接使用本地缓存——这是「Web 缓存」节省带宽的核心机制。
- **400**：请求报文有语法错误。
- **403**：服务器理解请求但拒绝执行（无权限）。
- **404**：请求的资源不存在。
- **500**：服务器内部错误。
- 记忆技巧：**2** 成功、**3** 重定向、**4** 客户错、**5** 服务器错。

*3.*非持久连接与持久连接

非持久连接（**non-persistent connection**）

HTTP/1.0 采用非持久连接：每次浏览器要请求一个文件，都要与服务器建立一次 **TCP** 连接，收到响应后就立即关闭连接。因此每请求一个文档都要有两倍的 **RTT** 开销：

$$T_{\text{一个对象}} = 2\,\text{RTT} + T_{\text{文档传输时延}}$$

第一个 RTT 用于 TCP 三次握手（前两次握手占用一个 RTT），第二个 RTT 用于发送 HTTP 请求并收到响应。

缺点：一个网页上往往有很多引用对象（图片、CSS、JS），请求每一个对象都要花 2 RTT；而且每个 **TCP** 连接刚建立时拥塞窗口很小，无法充分利用带宽。为减小时延，浏览器通常会建立多个并行的 **TCP** 连接同时请求多个对象（如 Chrome 对同一域名可并行 6 ~ 10 条），但这会大量占用服务器资源。这就是非持久连接分为串行 **TCP** 与并行 **TCP** 的由来。

持久连接（**persistent connection**）

HTTP/1.1 默认采用持久连接：服务器在发送响应后仍然保持这条 **TCP** 连接，使同一个客户和该服务器可以继续在这条连接上传送后续的 HTTP 请求报文与响应报文。它不局限于同一个页面上的对象，只要这些文档在同一台服务器上就行。

持久连接又分两种工作方式：

- 非流水线（**non-pipelined**）：客户在收到上一个响应之后才能发出下一个请求。因此每个对象仍需 **1** 个 **RTT**，但省掉了每个对象建 **TCP** 连接的 **1** 个 **RTT**。
- 流水线（**pipelined**）：客户在收到 HTTP 响应报文之前就能连续发送多个请求报文，这些请求一个接一个到达服务器后，服务器再一个接一个地发回响应。这样能把同一窗口内能容纳的请求「打包」发送，省掉多个 **RTT**，使 TCP 连接中的空闲时间减少。

四种连接方式的关系（务必分清）

- 按「一条 TCP 连接能传几个对象」分：非持久（1 个）vs 持久（多个）。
- 非持久再按「多个连接是否同时开」分：串行 **TCP** vs 并行 **TCP**（并行是「多条连接之间」的关系）。
- 持久再按「请求能否连续发」分：非流水线 vs 流水线（流水线是「同一条连接上请求之间」的关系）。

HTTP/1.1 与 HTTP/2 的主要差别

比较项	**HTTP/1.1**	**HTTP/2**
连接复用	持久连接，但同一连接上的请求按顺序处理（流水线的响应也须按序返回），存在队头阻塞	多路复用：同一 TCP 连接上可并行交错传输多个请求/响应，不再互相阻塞
报文格式	文本（ASCII）	二进制分帧，把报文切成带类型的帧
首部开销	每个请求都重复发送完整首部行，冗余大	首部压缩（**HPACK**），并维护首部表，重复首部只发索引
服务器推送	不支持	支持服务器推送（**server push**），可主动把客户即将需要的资源推过去
传输层	TCP	仍基于 TCP（HTTP/3 才改用基于 UDP 的 QUIC）

一句话概括：HTTP/2 在不改变 **HTTP** 语义（方法、状态码不变）的前提下，通过多路复用、二进制分帧、首部压缩、服务器推送提升传输效率。

4.Cookie 与 Web 缓存（代理服务器）

Cookie

早期的万维网只用于查看静态文档，所以 HTTP 被设计成无状态协议以简化服务器设计。但现在网上购物、电子商务等应用需要服务器识别用户，于是引入了 **Cookie**：**Cookie** 是一种对无状态的 **HTTP** 进行状态化的技术。

工作过程：

1. 用户首次访问某网站时，服务器的响应报文中带一行 `Set-Cookie: 12345678`（一个由服务器生成的识别码）。
2. 浏览器把该 Cookie 存入本地 Cookie 文件，记录「哪个服务器 → 哪个识别码」。
3. 此后浏览器每次访问该服务器，都在请求报文中加一行 `Cookie: 12345678`。
4. 服务器根据这个识别码在自己的数据库中查到这个用户，从而「记住」用户，在服务器端维护该用户的状态（购物车、登录态等）。

因此 2015 年第 40 题中，请求报文里出现 `Cookie: 123456` 说明浏览器此前访问过该站点；而 `Connection: close` 表示不使用持久连接。

Web 缓存与代理服务器

万维网缓存（**Web Cache**，又称 **Web** 缓存）可位于客户机上，也可位于中间系统上。位于中间系统上的 Web 缓存称为代理服务器（**Proxy Server**）。

Web 缓存把最近的一些请求和响应暂存在本地磁盘中。当新请求到达时，若发现这个请求与暂存的请求相同，就直接返回暂存的响应，而不需要按 URL 的地址再次去因特网访问该资源。

代理服务器对请求的两种处理路径：

- 缓存命中（**hit**）：代理服务器本地已有该对象且未过期，直接返回给客户，不访问源服务器。

- 缓存未命中（**miss**）：代理服务器以客户身份向源服务器发出 HTTP 请求，取回对象后既转发给客户，也在本地保存一份，供后续请求复用。

有效性检查：代理服务器用条件请求（`If-Modified-Since` 或 `If-None-Match`）询问源服务器「我这份还是最新的吗」。若源服务器回答 `304 Not Modified`（无实体主体），说明缓存仍然有效，直接使用本地副本。

代理服务器带来的好处

1. 大幅缩短响应时间：命中时本地返回，时延远小于跨广域网访问源服务器。
2. 减少出口链路的通信量：多个用户共享一份副本，尤其对热门资源效果显著。
3. 减轻源服务器负担：未命中时由代理「代表」客户访问，命中时源服务器完全不参与。

*5.*计算一：非持久连接与持久连接所需的 *RTT* 数与总时间

通用公式

设一个页面含有 1 个基础 HTML 文件（记作「基本页面」）和 N 个引用对象，每个对象的传输时间为 t_i，往返时间为 RTT。

1. 非持久连接（串行，**HTTP/1.0**）：每个对象都要新建 TCP 连接，各占 2 RTT：

$$\text{总 RTT 数} = 2 \times (N+1), \qquad T = 2(N+1)\,\text{RTT} + \sum_{i=0}^{N} t_i$$

1. 非持久连接（并行 **k** 条 **TCP** 连接）：把 $N+1$ 个对象分成 k 路并行：

$$T \approx 2\left\lceil \frac{N+1}{k} \right\rceil \text{RTT} + (\text{各路传输时延})$$

1. 持久连接 + 非流水线（**HTTP/1.1** 默认）：第一个对象付 **1** 个 **RTT** 建 **TCP** 连接，此后每个对象只花 1 个 RTT：

$$\text{总 RTT 数} = 1 + (N+1) = N+2$$

（第一个 RTT 里既建连接又发出第一个请求；严格地说是「1 RTT 建连 + (N+1) 个请求-响应各 1 RTT」，多数教材与真题把它合并写作 $N+2$。）

1. 持久连接 + 流水线：建连 1 RTT + 全部 $N+1$ 个请求连续发出后各占 1 RTT：

$$\text{总 RTT 数} = 1 + (N+1) = N+2 \ \text{（若所有对象能在同一批发送完）}$$

若受 TCP 发送窗口限制需要分多批（例如窗口只能容纳 W 个对象），则批数为 $\lceil (N+1)/W \rceil$，总 RTT 数为 $1 + \lceil (N+1)/W \rceil$。

流水线与非流水线的关键差别：非流水线时，每个对象的「请求-响应」必须串行排队，$N+1$ 个对象要花 $N+1$ 个 RTT；流水线时，若窗口足够大，多个请求可以并发在途，实际所需 RTT 数可能明显少于 $N+1$。

算例一（**2024** 年真题模型，非持久串行）

浏览器不支持并行 **TCP** 连接，使用非持久 HTTP/1.0 请求 1 个 Web 页，该页引用同一网站上的 **7** 个小图像文件。求从建立 TCP 连接开始到接收完所有内容所需的 RTT 数。

- 对象总数 $N+1 = 1+7 = 8$ 个。

- 非持久 + 串行，每个对象 2 RTT。
- 总 RTT $= 2 \times 8 =$ **16** 个 RTT。（选项 D）

算例二（**2011** 年第 **47** 题第 **(3)** 问，持久非流水线）

`rfc.html` 页面引用了 5 个 JPEG 小图像，HTTP/1.1 以持续的非流水线方式工作，一次请求-响应时间为 RTT。求从发出 `rfc.html` 的 Web 请求开始到浏览器收到全部内容为止需要多少 RTT。

- 题目已说明「从发出 **Web** 请求开始」，因此不再计 TCP 建连的 RTT，也不再计发出该请求之前的时间。
- 基本页面 `rfc.html`：1 RTT。
- 5 个图像：非流水线必须收到上一个响应后才能发下一个请求，各 1 RTT，共 5 RTT。
- 总 RTT $= 1 + 5 =$ **6** 个 RTT。

算例三（带传输时延的完整时延计算）

页面含 1 个 HTML 文件与 3 张图片，所有对象在同一台服务器上，RTT $= 20$ ms，单个小对象传输时延可忽略。分别求：

- 非持久串行：$2 \times 4 = 8$ RTT $= 160$ ms。
- 持久非流水线：$(1 + 4) = 5$ RTT $= 100$ ms。
- 持久流水线（窗口足够大）：$1 + 4 = 5$ RTT $= 100$ ms；若瓶颈链路带宽只能让每 RTT 传一个对象，则与持久非流水线相同。

可见非持久改成持久，省下的是「每个对象一次建连 **RTT**」，4 个对象共省 3 个 RTT。

算例四（**2022** 年真题模型，**HTTP/1.1** 叠加 **TCP** 拥塞控制）

主机 H 通过 HTTP/1.1 请求浏览服务器 S 上的 `news408.html`，该页引用同目录下 1 个图像。`news408.html` 大小为 1 MSS，图像大小为 3 MSS，RTT $= 10$ ms，忽略 HTTP 响应首部开销与 TCP 段传输时延，已预先完成域名解析。求从请求建立 TCP 连接起，到收到全部内容所需的最少时间。

按 TCP 初始拥塞窗口为 1 MSS、慢开始计算：

1. 建立 TCP 连接：**1 RTT**。
2. H 请求 HTML，服务器发送 1 MSS 页面：**1 RTT**。该段被确认后，服务器拥塞窗口增至 2 MSS。
3. H 解析页面后，在同一连接上请求图像；服务器先发 2 MSS：**1 RTT**。确认后窗口继续增大，足以发送剩余部分。
4. 服务器发送图像剩余的 1 MSS：**1 RTT**。

总计 4 RTT $= 4 \times 10 =$ **40** ms。

RTT 计算五个必踩的坑

1. 「从何时开始计时」决定要不要算建连 **RTT**。题干若说「从浏览器请求建立 TCP 连接开始」，要算；若说「从发出 HTTP 请求开始」（如 2011 年第 47 题第 (3) 问），则不再算建连 RTT，也不把自己这一次请求算进去。
2. 对象数 = 基本页面 + 引用对象数，别漏掉 HTML 本身。7 张图是 8 个对象，不是 7 个。
3. 非持久连接是 **2 RTT** / 对象，持久连接是（建连 **1 RTT** + **1 RTT** / 对象）。「并行连接」要除以并行度并向上取整。
4. 区分「**RTT** 数」与「时间」：题目问 RTT 数就只回整数；问时间还要乘 RTT，并在必要时加上传输时延（如 2022 年题要考虑 MSS 的分批）。

5. **DNS** 的时间要不要加：题干若说「已完成域名解析」就不加；若从「点击超链接」开始且给定了 DNS 相关的 RTT，则 DNS 的 RTT 必须计入（见 DNS 部分的算例二）。

*6.*计算二：带代理服务器缓存时的时延计算

设：

- 客户端到代理服务器的往返时间为 RTT_{cp}（通常很小，可能就在同一局域网）；
- 代理服务器到源服务器的往返时间为 RTT_{po}（通常较大，跨广域网）；
- p 为缓存命中率（请求在代理处直接命中的比例）。

则：

$$T_{\text{平均}} = p \cdot RTT_{cp} + (1-p) \cdot (RTT_{cp} + RTT_{po}) = RTT_{cp} + (1-p)\, RTT_{po}$$

算例一（命中与未命中）

某局域网通过代理服务器访问 Internet，客户到代理的 RTT $= 2$ ms，代理到源服务器 RTT $= 100$ ms，访问其中一个对象还需从代理到源服务器传输约 8 ms。

- 缓存命中：$T = 2$ ms（只有客户到代理的一个往返）。
- 缓存未命中：$T = 2 + 100 + 8 = 110$ ms。
- 若命中率 $p = 0.4$：$T_{\text{平均}} = 0.4 \times 2 + 0.6 \times 110 = 0.8 + 66 = 66.8$ ms。

算例二（带宽与冲突域，经典代理服务器题）

某机构局域网内的主机通过一台代理服务器访问 Internet。局域网带宽 10 Mb/s，Internet 出口带宽 1.5 Mb/s，平均对象大小 100 kb，平均请求率 15 次/秒，客户到代理的 RTT $= 2$ ms，代理到源服务器 RTT $= 100$ ms。

- 不设缓存时：出口链路的到达速率 $\lambda = 15 \times 100$ kb $= 1500$ kb/s $= 1.5$ Mb/s，恰好等于出口带宽，链路利用率 $\rho = 1$，队列时延趋于无穷，实际时延极大。
- 设缓存且命中率 $p = 0.4$ 时：出口链路的到达速率降到 $(1-p)\lambda = 0.6 \times 1.5 = 0.9$ Mb/s，链路利用率 $\rho = 0.9/1.5 = 0.6 < 1$，链路不再拥塞。此时

$$T_{\text{平均}} = RTT_{cp} + (1-p)\left(RTT_{po} + \frac{L}{B_{\text{out}}}\right) = 2 + 0.6 \times \left(100 + \frac{100\text{ kb}}{1.5\text{ Mb/s}}\right)$$

其中 $\dfrac{100\text{ kb}}{1.5\text{ Mb/s}} = \dfrac{100 \times 10^3}{1.5 \times 10^6}$ s ≈ 66.7 ms，故 $T_{\text{平均}} \approx 2 + 0.6 \times 166.7 \approx$ **102** ms。

算例三（缓存命中率的下限）

在上例中，若要求平均时延不超过 50 ms，求所需的最低命中率 p。

$$2 + (1-p) \times 166.7 \le 50 \implies 1 - p \le 0.288 \implies p \ge 0.712$$

即命中率至少要达到约 **71.2%**。

代理服务器计算的两个要点

1. 命中时只付客户到代理的时延；未命中时要付「客户到代理」+「代理到源服务器」两个时延。
2. 代理服务器主要是通过降低出口链路的利用率来消除排队时延：$\rho = \lambda(1-p)L/B_{\text{out}}$，$\rho$ 越小，排队时延越小。这正是「接入 ISP 带宽不变、只加一台代理服务器就能把响应时间从几分钟降到几十毫秒」的原因。

真题考法

- 2014 年第 40 题问「访问 Web 主页不可能使用的协议」，答案是 **SMTP**；可能用到 HTTP、UDP（DNS）、ARP、PPP。
- 2015 年第 40 题给出一段 HTTP 请求报文，判断四个说法。`Connection: close` 表示不使用持续连接；`Cookie` 表示曾访问过该站点。
- 2011 年第 47 题（9 分）综合考以太网帧解析 + ARP + 持久非流水线 RTT 数（6 个）+ 路由器转发时修改的 IPv4 首部字段。
- 2020 年第 40 题把 DNS 迭代查询与 HTTP 请求合并计时（20 ms / 50 ms）。
- 2022 年第 40 题把 HTTP/1.1 持续连接与 TCP 慢开始、MSS 结合（4 RTT = 40 ms）。
- 2024 年第 40 题考非持久 HTTP/1.0 串行请求 8 个对象的 RTT 数（16）。

本章小结表

协议 / 概念	所属层次	传输层与端口	模型	关键结论
C/S 模型	应用层体系结构	—	—	客户面向用户、主动；服务器面向任务、被动；客户之间不能直接通信
P2P 模型	应用层体系结构	—	—	结点对等、可直接通信；可扩展性好；三方式：集中式 Napster / 全分布式非结构化 Gnutella / 混合式 BitTorrent
DNS	应用层	**UDP 53**（区域传送、响应超 512 B 时用 TCP 53）	C/S	域名 →IP；报文本地服务器向外查询次数 = 域名级数 + 1；主机 ↔ 本地服务器递归，本地服务器 ↔ 其他服务器迭代
FTP	应用层	**TCP 21**（控制）、**TCP 20**（主动模式数据）	C/S	控制连接全程保持、带外传输；数据连接按需建立、用完即关；主动模式服务器连客户，被动模式客户连服务器
电子邮件	应用层	发送 **SMTP/TCP 25**；读取 **POP3/TCP 110**、**IMAP/TCP 143**	C/S	发送用 SMTP（推），读取用 POP3/IMAP（拉）；邮件服务器必须 24 h 在线
邮件格式	—	—	—	首部（From/To/Cc/Subject）+ 空行 + 主体；邮件格式由 RFC 822/5322 定义，不由 SMTP 定义
MIME	应用层	—	—	解决 SMTP 只能传 7 位 ASCII 的问题；新增 `Content-Type`、`Content-Transfer-Encoding` 等首部；base64 编码膨胀约 33%
WWW	应用层	—	C/S	三要素：URL（标识）、HTTP（传送）、HTML（解析）
URL	—	—	—	协议`://`主机`:`端口`/`路径`?` 查询 `#` 片段；端口/路径可省略；片段不发给服务器

协议 / 概念	所属层次	传输层与端口	模型	关键结论
HTTP	应用层	**TCP 80**	C/S	面向事务、无连接、无状态；请求报文 = 请求行 + 首部行 + 空行 + 实体主体；响应报文 = 状态行 + 首部行 + 空行 + 实体主体
HTTP/1.0	—	TCP 80	—	非持久连接，每个对象 2 RTT；可用并行 TCP 连接加速
HTTP/1.1	—	TCP 80	—	持久连接默认；分非流水线与流水线两种
HTTP/2	—	TCP（HTTP/3 用 QUIC/UDP）	—	多路复用、二进制分帧、首部压缩、服务器推送；语义不变
Cookie	应用层	由 HTTP 承载	—	对无状态 HTTP 做状态化；`Set-Cookie` 下发、`Cookie` 回送
Web 缓存 / 代理服务器	中间系统	由 HTTP 承载	—	命中直接返回；未命中代客户取回并保存；用 `304 Not Modified` 校验有效性

核心计算速查

场景	公式
本地域名服务器迭代查询次数（无缓存）	域名级数（不含主机名）+1
本地域名服务器发出查询次数（有缓存）	0
因特网组合查询方式	主机 → 本地服务器：递归；本地服务器 → 其他：迭代
非持久 HTTP（串行）请求 1 页面 + N 对象	$2(N+1)$ RTT
持久 HTTP 非流水线请求 $1+N$ 个对象	$1+(N+1)=N+2$ RTT
持久 HTTP 流水线（窗口足够）	$1+\lceil (N+1)/W \rceil$ RTT
带代理缓存的平均时延	$RTT_{cp}+(1-p)\,RTT_{po}$
代理后出口链路利用率	$\rho=\frac{\lambda(1-p)L}{B_{\text{out}}}$

易错清单

一、网络应用模型

- **C/S** 模型中客户之间不能直接通信，这是 2019 年真题的错误选项来源。
- P2P 中「每个结点既是客户又是服务器」指角色，不表示不存在索引服务器或超级结点。
- 不要用「P2P 单个结点带宽更大」解释 P2P 分发更快，正确原因是总上传能力随结点数增加。

二、DNS

- 本地域名服务器不属于域名服务器等级结构，画图和说法题里它必须单独列出。
- 递归与迭代的判断：主机 → 本地域名服务器是递归；本地域名服务器 → 根/顶级/权限是迭代（题目另有规定时按题目）。
- 查询次数计算：`www.abc.xyz.com` 的迭代查询次数按 `abc.xyz.com` 算，是 $3+1=4$ 次，不是 5 次；有缓存时本地域名服务器向外发出 **0** 次。
- 别把主机 **H** 发给本地服务器的递归查询算进「本地服务器发出的次数」（2016 年真题的得分点）。
- 缓存命中时，主机到本地服务器的那一次查询仍然发生，只是本地服务器不再跑腿。

- **DNS** 主用 **UDP 53**；区域传送和响应超过 **512** 字节时改用 **TCP 53**（2018 年第 33 题答案就是 DNS）。
- 域名与 IP 不是一一对应，兼容多域名、多 IP、负载均衡；DNS 不存 MAC 地址。
- 权限域名服务器可能不止一个，具体由题目给出的域名逐级判断。

三、FTP

- **20** 和 **21** 都是服务器端口；客户端端口由系统随机分配，不可能是 20（2017 年第 40 题的错误项）。
- 主动模式：服务器用 **20** 端口主动连客户；被动模式：服务器随机开端口并告知客户，由客户发起连接。所以「FTP 数据连接一定用 20」错。
- 控制连接在整个会话期间保持；数据连接每次传送按需建立、传完即关（2023 年第 47 题第 (1) 问）。
- 「控制连接用 TCP、数据连接用 UDP」是错的——两条连接都建立在 **TCP** 之上（2009 年第 40 题考的就是这个）。
- 带外控制指控制信息走独立连接，因此数据传送期间仍能发控制命令。
- FTP 有状态（记住登录状态与工作目录），HTTP 无状态。

四、电子邮件

- 发送用 **SMTP**（**25**），读取用 **POP3**（**110**）/ **IMAP**（**143**）；「服务器 → 用户代理」绝不使用 SMTP。
- 2012 年第 40 题的三个阶段是 **SMTP**、**SMTP**、**POP3**，顺序不要颠倒。
- **SMTP** 只能传 **7** 位 **ASCII**；JPEG、MPEG、EXE 必须先经 **MIME** 编码（2018 年第 40 题答案是「ASCII 文本」）。
- **MIME** 不修改 **SMTP**，只是在发送前编码、接收后解码；`Content-Type` 说类型，`Content-Transfer-Encoding` 说编码方式。
- 邮件格式由 **RFC 822/5322** 定义，不是 **SMTP** 定义的；首部与主体之间必须有空行。
- POP3 的两种方式是下载并删除与下载并保留；POP3 不允许在服务器上管理邮件（建文件夹、分类），IMAP 才允许。
- 不能把邮件直接推给收件人的 **UA**：对方主机不一定开机、也不一定跑邮件服务器程序。
- 网页邮件（浏览器收发）时，UA 与服务器之间用 **HTTP**，服务器之间仍用 **SMTP**。
- 邮件相关协议全部基于 **TCP**，没有一个用 UDP。

五、WWW 与 HTTP

- **HTTP** 无连接指不先建 HTTP 连接，但仍需 **TCP** 三次握手，别混淆。
- **HTTP** 无状态指服务器不记历史；Cookie 是对它做状态化的技术。
- 请求报文与响应报文都必须以空行结束首部；请求报文有实体主体时（`POST`）才带内容。
- **200/301/302/304/400/403/404/500** 的语义要背熟；`304` 不带实体主体。
- `Connection: close` 表示非持久连接；出现它就说明不使用持续连接（2015 年第 40 题 C 选项）。
- 非持久 = 每对象 **2 RTT**；持久非流水线 = 建连 **1 RTT** + 每对象 **1 RTT**。对象数要算上 HTML 本身。
- 注意题干计时起点：从「建立 TCP 连接」开始要算建连 RTT，从「发出 HTTP 请求」开始则不算（2011 年第 47 题第 (3) 问 = 6 RTT）。
- 流水线是「同一条连接上连续发多个请求」；并行 **TCP** 是「开多条连接」。两者不是一回事。
- **URL** 的片段（`#` 之后）不发送给服务器；查询串（`?` 之后）会发送。
- 代理服务器命中时只付客户到代理的时延，未命中才付到源服务器的时延；它通过降低出口链路利用率消除排队时延。